Principios y práctica del aprendizaje profundo de representaciones

o Una teoría matemática de la memoria

Sam Buchanan*, UC Berkeley & TTIC

Druv Pai*, UC Berkeley

Peng Wang, Universidad de Macau & Universidad de Michigan

Yi Ma, Universidad de Hong Kong & UC Berkeley

Este manuscrito se ha preparado para un curso sobre los principios del aprendizaje profundo y la inteligencia en general, a entregarse por los autores en sus respectivas instituciones. Esta versión previa a su publicación es de libre acceso para ver y descargar solo para uso personal y no se permiten su redistribución, su reventa ni su uso en obras derivadas.
Copyright © 2026 Sam Buchanan, Druv Pai, Peng Wang, Yi Ma

1 de marzo de 2026

* Estos autores contribuyeron por igual.

Prefacio

«*Todos los caminos conducen a Roma.*»

Objetivo. Este libro revela y estudia un problema común y fundamental que subyace a casi todas las prácticas modernas de inteligencia artificial. Se trata de *¿cómo aprender de manera eficaz y eficiente una distribución de datos de baja dimensión en un espacio de alta dimensión y luego transformar esta distribución en una representación compacta y estructurada?* Para cualquier sistema inteligente, natural o artificial, una representación de este tipo puede considerarse en general como *memoria* o *conocimiento* (empírico) aprendido a partir de datos percibidos del mundo externo. En los últimos años, a menudo se le denomina informalmente un «modelo del mundo.»

Público objetivo. Este libro de texto tiene como objetivo proporcionar una introducción sistemática a los principios matemáticos y computacionales del aprendizaje de representaciones (profundas) de tales distribuciones de datos, como forma computable de memoria, para *estudiantes de último año de licenciatura y de posgrado principiantes.* Los principales prerequisitos para este libro son el álgebra lineal a nivel universitario, probabilidad y estadística, así como optimización. Cierta familiaridad con los conceptos básicos del procesamiento de señales (representaciones esparsas y sensores comprimidos en particular), de teoría de la información, así como de control por retroalimentación mejoraría la comprensión del lector.

Motivación. La principal motivación para escribir este libro es que ha habido avances extraordinarios en estos últimos años, tanto por los autores como por numerosos colegas, que buscan establecer un enfoque riguroso y basado en principios para entender las redes neuronales profundas y, más en general, la inteligencia como tal. La metodología deductiva defendida por este nuevo enfoque, que es altamente complementaria, está en contraste directo con la metodología dominante que subyace a las prácticas contemporáneas en inteligencia artificial, que son inductivas y basadas en ensayo y error. La falta de comprensión de esos poderosos modelos y sistemas de IA ha generado

un creciente entusiasmo y temor en el seno de la sociedad. Creemos que es necesario, más que nunca, hacer un esfuerzo serio por establecer un enfoque basado en principios para comprender lo que es la inteligencia. Un objetivo general de este libro es proporcionar pruebas teóricas y experimentales sólidas que demuestren que es ahora posible estudiar la inteligencia como una disciplina científica y matemática. Argumentaremos que la inteligencia es la capacidad fundamental de desarrollar nuevas memorias (o conocimientos) o de corregir las existentes. Por lo tanto, podemos considerar este libro como un primer intento de desarrollar *una teoría matemática de la inteligencia*, al nivel de aprender conocimientos empíricos en forma de memoria, tal como sugiere el subtítulo del libro.

A nivel técnico, el marco teórico presentado en este libro ayuda a reconciliar una brecha antigua entre el enfoque clásico del modelado de estructuras de datos principalmente basado en modelos analíticos geométricos, algebráicos y probabilísticos (p.ej. subespacios, distribuciones gaussianas y ecuaciones) y el enfoque «moderno» basado en modelos no paramétricos diseñados empíricamente a partir de datos (p.ej. redes profundas). Como resultado, la unificación de estas dos metodologías aparentemente separadas es posible e incluso natural si se entiende que todas tratan de aprender y representar estructuras *de baja dimensión* en la distribución de datos de interés. Son simplemente maneras diferentes de descubrir, representar y explotar estas estructuras de baja dimensión. Desde esta perspectiva, incluso muchas técnicas computacionales aparentemente no relacionadas, desarrolladas independientemente en campos separados y en distintos momentos, pueden ser comprendidas mejor ahora en un marco teórico y computacional común y es probable que puedan estudiarse en su conjunto a partir de ahora. Como veremos en este libro, estas técnicas, entre otras, incluyen:

1. La codificación y decodificación compresiva con pérdidas desarrollada en la teoría de la información clásica y la teoría de la codificación frente al aprendizaje de representaciones moderno;

2. La eliminación de ruido y la reducción de dimensionalidad en el procesamiento clásico de señales frente a los modelos de difusión y eliminación de ruido de los métodos generativos modernos;

3. La inferencia bayesiana por vía del máximo a posteriori o del muestreo condicionado frente a la optimización con restricciones por la vía de técnicas de continuación [1].

Contenido principal. Creemos que el marco conceptual y computacional unificado que se presenta en este libro será de gran utilidad para aquellos lectores que verdaderamente quieran aclarar los misterios y malentendidos de las redes neuronales profundas y la inteligencia (artificial). Más allá de lo anterior, este marco pretende equipar a los lectores con principios rectores para

[1] como el método del lagrangiano aumentado

desarrollar sistemas significativamente mejores y verdaderamente inteligentes en el futuro. Más específicamente, aparte de una introducción (capítulo) informal, el contenido técnico principal del libro se organizará en forma de seis temas (capítulos) íntimamente relacionados:

1. Comenzaremos con los modelos clásicos y más básicos del análisis de componentes principales (PCA), del análisis de componentes independientes (ICA) y del aprendizaje de diccionarios (DL), las que suponen que las distribuciones de baja dimensión de interés poseen estructuras lineales e independientes. A partir de estos modelos simples idealizados que han sido ampliamente estudiados y comprendidos en procesamiento de señales y detección por compresión, introduciremos las ideas más fundamentales[2] sobre cómo aprender y cómo representar distribuciones de baja dimensión.

2. Para generalizar estos modelos clásicos y extender sus soluciones a distribuciones de baja dimensión que no puedan ser representadas fácilmente por un modelo analítico sencillo, introducimos un principio computacional universal para el aprendizaje de tales distribuciones: *la compresión*. Como veremos, la compresión de datos proporciona una visión unificadora de todos los enfoques clásicos y modernos aparentemente diferentes para el aprendizaje de distribuciones o representaciones, incluyendo la minimización de entropía, la eliminación de ruido mediante score-matching, compresión con pérdidas y distorsión de tasa, así como el aprendizaje discriminativo de representaciones mediante maximización de la ganancia de información.

3. En este marco unificador, las redes neuronales profundas (DNNs) modernas, como ResNet, CNN y Transformer, pueden todas interpretarse matemáticamente (desenrollarse) como algoritmos de optimización que iterativamente logran mejor compresión y mejor representación al reducir la longitud o la tasa de codificación o al ganar información. Este marco no solo ayuda a explicar las arquitecturas de redes profundas diseñadas empíricamente hasta ahora, sino que también conduce a nuevos diseños arquitectónicos que pueden ser significativamente más simples y eficientes.

4. Más allá de lo anterior, para que la representación aprendida para una distribución de datos sea correcta y consistente, se hacen necesarias las arquitecturas de *autocodificadores* que incluyen tanto codificación como decodificación. Para que un sistema de aprendizaje sea plenamente automático y de operación contínua, introduciremos un poderoso *esquema de transcripción en circuito cerrado* que permite que un sistema autocodificador se autocorrija y, por tanto, se autoperfeccione, por medio de un juego minimax entre codificador y decodificador.

5. Para conectar la teoría con la práctica, entonces estudiaremos cómo la distribución de datos y su representación aprendidas pueden utilizarse como un potente a priori para llevar a cabo inferencias bayesianas u optimizaciones

[2] tales como a eliminación de ruidos y la reducción de dimensionalidad

con restricciones que se manifiestan en casi todos los tipos de tareas y configuraciones más ampliamente utilizadas en la práctica de la inteligencia artificial moderna, incluyendo la estimación, el completado y la generación condicionados de datos reales de alta dimensión, tales como imágenes y textos.

6. Por último, pero no menos importante, mostraremos paso a paso cómo aprender de manera efectiva y eficiente las representaciones profundas de distribuciones de baja dimensión a partir de conjuntos de datos del mundo real a gran escala, incluyendo datos visuales, movimientos corporales y datos textuales. Estos se usarán en numerosas aplicaciones prácticas, tales como clasificación, completado, segmentación, generación de imágenes, estimación de movimiento y tareas similares para datos de texto.

Resumen. Para resumir, el contenido técnico presentado en este libro establece fuertes conexiones conceptuales y técnicas entre el enfoque analítico clásico y el enfoque moderno impulsado por datos, entre modelos paramétricos simples y los modelos no paramétricos profundos, entre las diversas prácticas inductivas y un marco deductivo unificado basado en principios fundamentales. Revelaremos que numerosos enfoques aparentemente no relacionados o incluso contradictorios entre sí, aunque se hayan desarrollado en campos distintos, con terminologías diversas y en momentos históricos diferentes, sin embargo todos aspiran a lograr un objetivo común:

perseguir y explotar las estructuras intrínsecas de baja dimensión de distribuciones de datos incrustadas en espacios de alta dimensión.

A tal fin, el libro nos guiará a través de un recorrido completo desde la formulación teórica a la deducción matemática, seguida de la implementación computacional y las aplicaciones prácticas.

Prefacio a la versión 2.0

Como libro de código abierto, la primera versión, titulada «*Aprendizaje de representaciones profundas de distribuciones de datos*», lanzada el 18 de agosto de 2025, se ha actualizado de manera frecuente y constante en los últimos seis meses. Al mismo tiempo, basados en nuestra experiencia usando el libro en un nuevo curso impartido en la Universidad de Hong Kong en el semestre de otoño de 2025, así como en los comentarios que hemos recibido de colegas, tutores y estudiantes, hemos identificado numerosos puntos a lo largo del libro que merecen una mayor revisión y ampliación.

Por ello hemos decidido introducir algunos cambios y actualizaciones sustanciales en el contenido y la organización del libro para la nueva *Versión 2.0* y hemos cambiado su título a:

Principios y práctica del aprendizaje profundo de representaciones
o Una teoría matemática de la memoria

Esta nueva versión también nos permite poner de relieve las fuertes conexiones conceptuales y técnicas entre los contenidos de los diferentes capítulos y secciones del libro, de manera que creemos que el valor pedagógico global del libro ha mejorado significativamente en comparación con la primera versión.

En nuestra opinión, la nueva versión logra una presentación mucho más unificada, completa y fluida del tema. La nueva versión también incorpora nuevos aportes teóricos recientemente desarrollados, así como un número creciente de aplicaciones prácticas para datos del mundo real que no se han documentado de manera apropiada ni explicado en forma sistemática en otros lugares. Esta Versión 2.0 servirá como solución oportuna para esta situación.

Principales cambios en la Version 2.0

- Hemos dividido en dos el Capítulo 3 de la primera versión, que ahora son los Capítulos 3 y 4. El nuevo Capítulo 3 se centra en el proceso de eliminación de ruido para el aprendizaje de distribuciones de baja dimensión. Hemos agregado una nueva sección en la que se caracterizan las condiciones bajo las que el proceso conduce a una generalización o memorización. El nuevo Capítulo 4 se centra en el aprendizaje de representaciones basado en un enfoque de codificación con pérdidas y en el principio de maximización de la ganancia de información. Como la

versión anterior solo ilustraba cómo aplicar el principio en el aprendizaje de representaciones de imágenes en una configuración supervisada, en la nueva versión se agrega un estudio de casos correspondientes a una configuración no supervisada. También proporciona una justificación teórica de algunos métodos populares de aprendizaje no supervisado tales como el aprendizaje contrastivo y DINO en particular.

- En el Capítulo 5 actualizado sobre el diseño de representaciones por vía de optimización desenrollada, hemos agregado una derivación basada en principios para una versión causal de la arquitectura de transformador transparente (white-box) CRATE, que resulta importante para el procesamiento de datos secuenciales tales como textos, como vemos frecuentemente en aplicaciones.

- En el Capítulo 6 actualizado sobre el aprendizaje de representaciones consistentes, hemos agregado una nueva sección que aumenta la claridad sobre las relaciones entre el aprendizaje de distribuciones y el aprendizaje de representaciones. Esto da una clarificación y justificación teóricas para los métodos populares y prácticos basados en autocodificación, como la autocodificación variacional o los autocodificadores de representaciones.

- En el Capítulo 7 actualizado hemos añadido una sección que ofrece una explicación basada en principios del aprendizaje de representaciones con datos pareados y generación condicionada a través de la perspectiva de la información mutua. Proporciona una justificación teórica para métodos populares y prácticos tales como CLIP y formas de atención cruzada.

- El Capítulo 8 ampliado sobre aplicaciones ahora incluye numerosas implementaciones detalladas del aprendizaje de distribuciones y de representaciones de datos reales, incluyendo imágenes bidimensionales, objetos naturales tridimensionales, movimientos corporales, así como lenguajes naturales, en casi todos las configuraciones populares y prácticas, supervisadas, débilmente supervisadas, no supervisadas y condicionadas.

- El Capítulo 9 final, sobre líneas de investigación abiertas, también se ha reescrito significativamente. Buscamos ofrecer una taxonomía más clara de los diferentes niveles de la inteligencia, de modo de poder clarificar qué hemos hecho y qué entendemos acerca de la inteligencia (con este libro) y qué no hemos hecho. Creemos que los problemas abiertos asociados con las formas superiores de inteligencia pueden formularse adecuadamente de manera de que puedan estudiarse cualitativa y cuantitativamente con medios científicos y matemáticos, en lugar de permanecer como un asunto misterioso o meramente filosófico.

Contribuidores a la Versión 2.0

Además de los autores, numerosos estudiantes y colegas han colaborado a este proyecto y han aportado contenido valioso a diferentes partes del libro durante la

preparación de la *Versión 2.0*. Se presenta a continuación una lista incompleta de las personas y sus contribuciones específicas, en orden alfabético:

- Tianzhe Chu: Experimentos en la Sección 8.4, así como herramientas de IA para el libro.
- Prof. Shenghua Gao: Secciones 8.8 y 8.9.
- Bingbing Huang: Secciones 8.8 y 8.9.
- Kerui Min: Traducción al chino.
- Prof. Qing Qu: Sección 3.3.
- Shengbang Peter Tong: Secciones 6.3 y 8.6.2.
- Chengyu Wang: Secciones 8.8 y 8.9.
- Ziyang Robin Wu: Secciones 8.3 y desarrollo del sitio web.
- Chun-Hsiao Daniel Yeh: Secciones 8.6 y 8.7.
- Brent Yi: Sección 8.10.
- Jingfeng Yang: Sección 8.3.
- Dr. Yaodong Yu: El capítulo 5 se basa en su tesis doctoral.
- Dr. Zibo Zhao: Sección 8.8.

También queremos agradecer al Dr. Kevin Murphy y al Dr. Bill Mark por sus extensos comentarios técnicos sobre el manuscrito; a Jan Cavel por haber contribuido una traducción no oficial al rumano, así como a Stephen Butterfill y Jeroen Van Goey por haber contribuido correcciones y ajustes al manuscrito.

Declaratoria de acceso abierto

> «*A menudo comparo el código abierto con la ciencia. La ciencia ha adoptado toda esta noción del desarrollo abierto de ideas y de mejoras en base a las ideas de otros. Esto hizo de la ciencia lo que es hoy en día y permitió los increíbles avances que hemos logrado. Y eso lo comparo con la brujería y la alquimia, donde la abertura era algo que no se practicaba.*»
>
> – Linus Torvalds

Este libro está destinado a ser un proyecto de «código abierto» que se pondrá a disposición del público en el sitio web

https://ma-lab-berkeley.github.io/deep-representation-learning-book/

para todos los interesados en los principios científicos, matemáticos y computacionales de la inteligencia. El libro se actualizará de manera continua y periódica por todos los que quieran contribuir a este proyecto, incluyendo sus versiones traducidas, los materiales educativos, y diversas herramientas de IA para apoyar el estudio, la enseñanza y la investigación de este tema.

A través de este libro esperamos iniciar un «programa de conocimiento abierto» que promueva que todo el conocimiento científico sea de acceso abierto, para su organización, pedagogía, difusión y avance.

Agradecimientos

Apoyo financiero. Este libro se basa principalmente en resultados de la investigación desarrollada a partir de 2018. Gracias a los generosos fondos de arranque de la Universidad de California, Berkeley (2018) y la Universidad de Hong Kong (2023), Yi Ma pudo iniciar y centrarse en esta nueva y apasionante línea de investigación durante los últimos ocho años. Durante estos años, para la investigación en esta dirección, Yi Ma y su equipo en Berkeley han sido apoyados por las las siguientes subvenciones de investigación:

- El proyecto de multi-universidad *THEORINET* (Transferable, Hierarchical, Expressive, Optimal, Robust, and Interpretable NETworks) para las Fundaciones del Aprendizaje Profundo, conjuntamente financiado por la Fundación Simons y la National Science Foundation (DMS grant #2031899);
- El proyecto *Closed-Loop Data Transcription via Minimaxing Rate Reduction* financiado por Office of Naval Research (grant N00014-22-1-2102);
- El proyecto *Principled Approaches to Deep Learning for Low-dimensional Structures* financiado por la National Science Foundation (CISE grant #2402951).

Este libro no habría sido posible sin el apoyo financiero para estos proyectos de investigación. Los autores han obtenido una inspiración tremenda de los resultados de investigación desarrollados por colegas y estudiantes que han participado en estos proyectos.

Notación

En este libro utilizamos la notación de una variedad de disciplinas matemáticas, resumida a continuación. Se recomienda al lector que consulte esta lista como referencia a medida de que vaya leyendo los diversos capítulos del libro. Hemos intentado mantener su uso lo más coherente posible entre capítulos.

Como un primer ejemplo, para un número natural n el conjunto $\{1, 2, \ldots, n\}$ se denota por $[n]$.

Notación de álgebra lineal

- Los escalares (ya sean deterministas o aleatorios) no están en negrilla, por ejemplo, x, X.

- Los vectores (ya sean deterministas o aleatorios) están en negrilla minúscula, por ejemplo, $\boldsymbol{x}$, $\boldsymbol{\pi}$. Los componentes de vectores se denotan x_i o π_i, alternativamente $(\boldsymbol{x})_i$. Los términos genéricos (que pueden ser escalares, vectores, matrices o tensores de orden superior) también están en negrilla.

- Las matrices (ya sean deterministas o aleatorias) están en negrilla mayúscula, por ejemplo, $\boldsymbol{X}, \boldsymbol{\Pi}$. Los elementos de matrices se denotan X_{ij} o Π_{ij}, alternativamente $(\boldsymbol{X})_{ij}$. Las columnas de matrices son vectores en negrilla minúscula, por ejemplo, $\boldsymbol{x}_i$ es la i-ésima columna de $\boldsymbol{X}$, a no ser que se defina de otra manera.

- La transpuesta de una matriz se denota con $\top$, por ejemplo, $\boldsymbol{A}^\top$. La adjunta con *, p.ej., $\boldsymbol{A}^*$. La pseudo-inversa con $\dagger$, p.ej., $\boldsymbol{A}^\dagger$.

- El rango de una matriz es $\operatorname{rank}(\boldsymbol{A})$, la traza es $\operatorname{tr}(\boldsymbol{A})$, el determinante es $\det(\boldsymbol{A})$ y el determinante logarítmico es $\operatorname{logdet}(\boldsymbol{A})$.

- La multiplicación elemento por elemento de matrices es $\boldsymbol{A} \odot \boldsymbol{B}$. La cuadratura elemento por elemento es $\boldsymbol{A}^{\odot 2}$, etc. De manera similar, el producto Kronecker de matrices es $\boldsymbol{A} \otimes \boldsymbol{B}$. El producto Kronecker iterado es $\boldsymbol{A}^{\otimes 2}$, etc.

- Para una función $f\colon \mathbb{R} \to \mathbb{R}$, su aplicación elemento por elemento a una matriz $\boldsymbol{X}$ es $f[\boldsymbol{X}]$, esto es, con corchetes. Para una función $f\colon \mathbb{R}^d \to \mathbb{R}^k$ y una matriz $\boldsymbol{X} \in \mathbb{R}^{d\times n}$, podemos difundir f para aplicarla a cada columna sin utilizar una notación especial, esto es, $f(\boldsymbol{X}) = [f(\boldsymbol{x}_1), \ldots, f(\boldsymbol{x}_n)] \in \mathbb{R}^{k\times n}$.
- La esfera unidad (euclidiana) es $\mathbb{S}^{n-1} \subseteq \mathbb{R}^n$. la bola unidad (euclidiana) es $\mathbb{B}^n \subseteq \mathbb{R}^n$.
- El conjunto de matrices ortogonales es $\mathsf{O}(m,n) \subseteq \mathbb{R}^{m\times n}$ o $\mathsf{O}(n) = \mathsf{O}(n,n)$.
- El conjunto de las matrices simétricas es $\mathsf{Sym}(n)$, el de las matrices PSD simétricas es $\mathsf{PSD}(n)$, el de las PD simétricas es $\mathsf{PD}(n)$.
- Los valores propios de una matriz simétrica $\boldsymbol{S} \in \mathsf{Sym}(n)$ son todos reales por el teorema espectral; se denotan $\lambda_i(\boldsymbol{S})$ y se ordenan de modo que $\lambda_1(\boldsymbol{S}) \geq \cdots \geq \lambda_n(\boldsymbol{S})$.
- Los valores singulares de $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ con $\operatorname{rank}(\boldsymbol{A}) = r$ se denotan $\sigma_i(\boldsymbol{A})$ y se ordenan de modo que $\sigma_1(\boldsymbol{A}) \geq \cdots \geq \sigma_r(\boldsymbol{A}) > 0$. Por convención, se define $\sigma_{r+1}(\boldsymbol{A}) = \sigma_{r+2}(\boldsymbol{A}) = \cdots = 0$.
- La proyección sobre un conjunto $\mathcal{K}$ se denota $\operatorname{proj}_{\mathcal{K}}$.
- Las normas ℓ^p de vectores se denotan como $\|\boldsymbol{x}\|_p$, $p \in [0,\infty]$.
- La norma operador euclidiana en matrices es $\|\boldsymbol{A}\| = \sigma_1(\boldsymbol{A})$.
- La norma Frobenius de matrices es $\|\boldsymbol{A}\|_F = \operatorname{tr}(\boldsymbol{A}^\top \boldsymbol{A})$.
- El producto escalar euclidiano de vectores es $\langle \boldsymbol{x}, \boldsymbol{y}\rangle = \boldsymbol{y}^\top \boldsymbol{x}$.
- El producto escalar Frobenius de matrices es $\langle \boldsymbol{X}, \boldsymbol{Y}\rangle = \operatorname{tr}(\boldsymbol{Y}^\top \boldsymbol{X})$.
- Un vector o matriz con todos los elementos iguales a uno se denota $\mathbf{1}$ (la dimensión debería ser obvia por contexto). De manera similar, un vector o matriz con todos los elementos iguales a cero se denota $\mathbf{0}$. La matriz identidad se denota como $\boldsymbol{I}$.

Notación de probabilidades

- La medida de probabilidad base es $\mathbb{P}$, es decir, la probabilidad de que ocurra un evento A es $\mathbb{P}[A]$. Podemos especificar la distribución de una variable aleatoria mediante subíndices, es decir, $\mathbb{P}_{\boldsymbol{x}\sim\mu}[\boldsymbol{x} \in S]$, describe las probabilidades asociadas con una variable aleatoria $\boldsymbol{x}$ con distribución (medida) μ. Si dos variables aleatorias $\boldsymbol{x}$ y $\boldsymbol{x}'$ tienen la misma distribución, escribimos $\boldsymbol{x} \overset{\mathrm{d}}{=} \boldsymbol{x}'$.
- El operador esperanza es $\mathbb{E}$, con la misma advertencia sobre el uso de subíndices.

- El operador covarianza es Cov, con la misma advertencia sobre el uso de subíndices.
- El operador correlación es Corr, con la misma advertencia sobre el uso de subíndices.
- En ciertos contextos de modelado probabilístico donde se supone una densidad (especialmente en Capítulos 3 y 7), también se usa la notación $p_{\boldsymbol{x}}$ para la densidad de una variable aleatoria $\boldsymbol{x}$. Esto se extiende a las densidades condicionales, p.ej. $p_{\boldsymbol{x}|\boldsymbol{y}}$ para la densidad de $\boldsymbol{x}$ condicionada por $\boldsymbol{y}$.
- En general se usan letras griegas para denotar las realizaciones de variables aleatorias (en contraste con las variables aleatorias propiamente tales). Por ejemplo, $p_{\boldsymbol{x}}(\boldsymbol{\xi})$, $p_{\boldsymbol{x}|\boldsymbol{y}}(\boldsymbol{\xi} \mid \boldsymbol{\nu})$, $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y} = \boldsymbol{\nu}]$, etc.
- El conjunto de distribuciones de probabilidades sobre un conjunto finito $\mathcal{X}$ se denota $\Delta(\mathcal{X})$. (Para $\mathcal{X} = [n]$, esto es el simplex probabilístico que se puede incrustar en $\mathbb{R}^n$).
- La distribución gaussiana con media $\boldsymbol{\mu}$ y covarianza $\boldsymbol{\Sigma}$ se denota $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.
- La distribución uniforme sobre un conjunto compacto $\mathcal{X}$ se denota $\mathcal{U}(\mathcal{X})$.

Notación de aprendizaje automático

- Los codificadores y reductores de ruido normalmente se denotan por f y suelen tener parámetros $\theta \in \Theta$. Normalmente escribiremos las características como $\boldsymbol{z} = f_\theta(\boldsymbol{x})$.
- Los decodificadores normalmente se denotan por g y suelen tener parámetros η. Normalmente escribiremos $\hat{\boldsymbol{x}} = g_\eta(\boldsymbol{z})$ para denotar la autocodificación correspondiente a $\boldsymbol{x}$.
- Cuando f y g se implementan como redes neurales, tendrán el mismo número de capas sin pérdida de generalidad; escribiremos $f = f^L \circ f^{L-1} \circ \cdots \circ f^2 \circ f^1 \circ f^{\text{pre}}$ y $g = g^{\text{post}} \circ g^L \circ g^{L-1} \circ \cdots \circ g^2 \circ g^1$.
- Escribiremos las características a la entrada a la capa ℓ como $\boldsymbol{z}^\ell$ de modo que $\boldsymbol{z}^{\ell+1} = f^\ell(\boldsymbol{z}^\ell)$ con $\boldsymbol{z}^1 = f^{\text{pre}}(\boldsymbol{x})$ y $\boldsymbol{z} = \boldsymbol{z}^{L+1}$. Del mismo modo, las características de autocodificación serán $\hat{\boldsymbol{x}}^\ell$ con $\hat{\boldsymbol{x}}^{\ell+1} = g^\ell(\hat{\boldsymbol{x}}^\ell)$ con $\hat{\boldsymbol{x}}^1 = \boldsymbol{z}$ y $\hat{\boldsymbol{x}} = g^{\text{post}}(\hat{\boldsymbol{x}}^{L+1})$.
- Para la reducción de ruido, la optimización y para procesos con un índice de tiempo continuo, el tiempo casi siempre es un subíndice, p.ej., $\boldsymbol{x}_t$. En el caso de las secuencias discretas el índice puede ser superíndice o subíndice.

Notación para modelación y optimización

- Tal como en las secciones anteriores de notación de probabilidades y de aprendizaje automático, cuando tenemos un modelo, digamos, para las realizaciones de una distribución de datos definida en $\mathbb{R}^D$, siempre lo denotamos con variables «ordinarias», sin marca diacrítica, p.ej. $\boldsymbol{x}$.

- En los casos en que suponemos que los datos $\boldsymbol{x}$ son generados por un modelo de una clase restringida de *modelos paramétricos*, digamos, con un parámetro $\boldsymbol{\mu}$, también denotamos el *parámetro verdadero* sin marca diacrítica.[3]

- Las *variables de decisión* (o parámetros aprendibles) del ajuste de un modelo a datos observados se denotan con una marca diacrítica de «tilde», p.ej. $\tilde{\boldsymbol{x}}$, en casos donde existe un modelo subyacente $\boldsymbol{x}$, o con tipografía simple en casos donde no haya confusión (p.ej., si un modelo empíricamente ajustado para datos *no paramétricos* $\boldsymbol{x}$ involucra parámetros «medios» $\boldsymbol{\mu}$).

- Las *soluciones óptimas* de problemas de optimización se denotan con una marca diacrítica de «estrella», ya sea como superíndice o subíndice (p.ej. $\boldsymbol{x}_\star$ o $\boldsymbol{x}^\star$).

- Los *estimadores* para datos que corresponden a un modelo estadístico específico, especialmente para la media de ese modelo estadístico, se denotan con una marca diacrítica de «barra», p.ej. $\bar{\boldsymbol{x}}$ (especialmente para la reducción de ruido por error cuadrático medio en Capítulo 3).

- Las *aproximaciones* asociadas a procedimientos computacionales del modelado de datos se denotan con una marca diacrítica de «sombrero», p.ej. $\hat{\boldsymbol{x}}$ para la salida de un autocodificador o un modelo generativo que aproxime datos $\boldsymbol{x}$.[4]

[3] Esto podría surgir ya sea por haber hecho una suposición sobre de los datos, o bien debido a que los datos hayan sido generados sintéticamente. Esta convención nos permite adoptar una notación concisa tanto en contextos 'probabilísticos' como 'analíticos' de modelación. Aquellos lectores más familiares con estos últimos tipos de esquemas puede que estén acostumbrados a usar notaciones tales como $\boldsymbol{\mu}_\sharp$ o $\boldsymbol{\mu}_o$ para este caso; en contraste con nuestra notación con variables de decisión acentuadas.

[4] En general distinguimos estos casos de las soluciones óptimas $\boldsymbol{x}_\star$, aunque en algunos casos especiales sean lo mismo (p.ej., en varios ejemplos de Capítulo 2). Piénsese en la distinción entre el parámetro $\boldsymbol{\mu}$ de un modelo, que puede haberse ajustado con optimización, y los resultados del muestreo con ese parámetro aprendido.

Índice

Prefacio ii

Prefacio a la Versión 2.0 vi

Declaración de código abierto ix

Agradecimientos xi

Notación xiii

1 Una introducción informal a la inteligencia 1
1.1 Inteligencia, cibernética e inteligencia artificial 1
1.2 ¿Qué aprender? 10
1.2.1 Predictibilidad 10
1.2.2 Baja dimensión 12
1.3 ¿Cómo aprender? 18
1.3.1 Enfoques analíticos basados en modelos 18
1.3.2 Enfoques empíricos basados en datos 29
1.4 Un enfoque unificador 41
1.4.1 Aprendizaje de representaciones parsimoniosas 41
1.4.2 Aprendizaje de representaciones informativas 46
1.4.3 Aprendizaje de representaciones consistentes 47
1.4.4 Aprendizaje de representaciones autoconsistentes 50
1.5 Tender un puente entre teoría y práctica en inteligencia artificial 51

2 El aprendizaje de estructuras lineales e independientes 55
2.1 Subespacio de baja dimensión 57
2.1.1 Análisis de componentes principales (PCA) 57
2.1.2 Búsqueda de estructuras de rango bajo por el método de las potencias 63
2.1.3 Aprendizaje local e incremental mediante PCA en línea . 64
2.1.4 La perspectiva estadística: PCA probabilístico 65
2.1.5 Completamiento enmascarado de matrices de bajo rango . 68
2.2 Una mezcla de subespacios completos de baja dimensión 69

2.2.1 Mezclas de subespacios y diccionarios con codificación esparsa . . . 71
2.2.2 Aprendizaje de diccionarios completos . . . 73
2.2.3 Relación con ICA y la curtosis . . . 76
2.3 Una mezcla de subespacios sobrecompletos de baja dimensión . . . 79
2.3.1 Codificación esparsa con diccionario sobrecompleto . . . 81
2.3.2 Aprendizaje de diccionarios sobrecompletos . . . 83
2.3.3 Codificación esparsa profunda aprendida . . . 85
2.4 Resumen y notas . . . 89
2.5 Ejercicios y ampliaciones . . . 91

3 Búsqueda de distribuciones de baja dimensión mediante eliminación de ruido 99
3.1 Minimización de la entropía y compresión . . . 101
3.1.1 Entropía y tasas de codificación . . . 101
3.1.2 Entropía diferencial . . . 101
3.1.3 Minimización de la tasa de codificación . . . 102
3.2 Compresión mediante eliminación de ruido . . . 104
3.2.1 Procesos de difusión y de eliminación de ruido . . . 104
3.2.2 Muestreo de una distribución mediante eliminación iterativa de ruido . . . 119
3.3 Memorización, generalización y tasas de codificación . . . 129
3.4 Resumen y notas . . . 135
3.5 Ejercicios y ampliaciones . . . 136

4 Aprendizaje de representaciones mediante compresión con pérdidas 139
4.1 Compresión mediante codificación con pérdidas . . . 141
4.1.1 La necesidad de la codificación con pérdida . . . 142
4.1.2 La tasa-distorsión y la geometría de los datos . . . 144
4.1.3 Tasa de codificación con pérdida para una distribución gaussiana de baja dimensión . . . 150
4.1.4 Agrupamiento de una mezcla de gaussianas de baja dimensión . . . 153
4.2 Transformación mediante maximización de la ganancia de información . . . 161
4.2.1 Aprendizaje de representaciones a partir de datos etiquetados . . . 164
4.2.2 Representaciones discriminantes lineales . . . 167
4.2.3 El principio de reducción máxima de la tasa de codificación 171
4.2.4 Propiedades de optimización en la reducción de la tasa de codificación . . . 176
4.3 Aprendizaje de representaciones para datos de imagen . . . 179
4.3.1 Aprendizaje supervisado de representaciones . . . 179
4.3.2 Aprendizaje no supervisado de representaciones . . . 181
4.4 Resumen y notas . . . 186

4.5 Ejercicios y ampliaciones . . . 186

5 Las representaciones profundas como optimización desenrollada 189
5.1 Redes profundas transparentes *white-box* mediante optimización desenrollada . . . 192
5.1.1 Redes profundas a partir del descenso por gradiente desenrollado . . . 192
5.1.2 Redes convolucionales a partir de la reducción de tasa en invariantes . . . 200
5.2 Transformadores transparentes (white-box) mediante optimización desenrollada . . . 210
5.2.1 Optimización desenrollada para la reducción esparsa de la tasa (de codificación) . . . 211
5.2.2 Arquitectura general del Transformer *white-box*: CRATE 219
5.3 Variantes por diseño de arquitecturas profundas . . . 222
5.3.1 Arquitectura Transformer de atención exclusiva . . . 222
5.3.2 Atención de tiempo lineal: Transformer de estadísticas de token . . . 226
5.3.3 CRATE causal . . . 232
5.4 Resumen y Notas . . . 237
5.5 Ejercicios y extensiones . . . 238

6 Representaciones consistentes y autoconsistentes 243
6.1 Aprendizaje de representaciones consistentes . . . 246
6.1.1 Autocodificación lineal mediante PCA . . . 247
6.1.2 PCA no lineal y autocodificación . . . 248
6.1.3 Autocodificación esparsa . . . 251
6.1.4 Autocodificación variacional . . . 252
6.1.5 Aprendizaje conjunto de representación y distribución . . . 255
6.2 Aprendizaje de representaciones autoconsistentes . . . 258
6.2.1 Transcripción en bucle cerrado mediante juegos de Stackelberg . . . 261
6.2.2 Una mezcla de gaussianas de baja dimensión . . . 267
6.3 Aprendizaje continuo de representaciones autoconsistentes . . . 269
6.3.1 Aprendizaje incremental por clase . . . 269
6.3.2 Aprendizaje continuo no supervisado muestra por muestra 276
6.4 Resumen y Notas . . . 281
6.5 Ejercicios y extensiones . . . 282

7 Inferencia con distribuciones de baja dimensión 285
7.1 Inferencia bayesiana y optimización con restricciones . . . 286
7.1.1 Inferencia bayesiana con distribuciones de baja dimensión 286
7.1.2 Optimización con restricciones en subvariedades . . . 289
7.1.3 Entornos prácticos representativos para la inferencia . . . 291
7.2 Inferencia condicional con una distribución de datos conocida . . . 293
7.3 Inferencia condicional con una representación de datos aprendida 296

7.3.1 Completado de imágenes mediante autocodificación enmascarada . . . 296
7.3.2 Muestreo condicional con ajuste de mediciones . . . 299
7.4 Inferencia condicional con datos y mediciones emparejados . . . 313
7.4.1 Generación de imágenes condicionada por clase . . . 314
7.4.2 Generación de imágenes condicionada por pies de imagen 326
7.5 Inferencia condicional con autoconsistencia de la medición . . . 332
7.5.1 Modelos lineales de medición . . . 333
7.5.2 Aprendizaje de un modelo del mundo en 3D a partir de imágenes 2D . . . 334
7.6 Resumen y Notas . . . 337
7.7 Ejercicios y extensiones . . . 339

8 Aprendizaje de representaciones para datos y tareas del mundo real 345
8.1 Introducción . . . 345
8.1.1 Estructura del capítulo . . . 346
8.1.2 Configuración técnica general . . . 347
8.2 Aprendizaje no supervisado de representaciones de imágenes . . . 350
8.2.1 Datos . . . 350
8.2.2 La tarea y el objetivo simplificados de DINO . . . 351
8.2.3 Arquitectura: *Vision Transformer* . . . 355
8.2.4 Estrategia de optimización . . . 359
8.2.5 Metodología de evaluación . . . 363
8.2.6 Configuración experimental y resultados . . . 366
8.3 Representación de imágenes con supervisión débil mediante la vinculación de texto . . . 369
8.3.1 Datos . . . 370
8.3.2 La tarea y el objetivo del CLIP . . . 370
8.3.3 Arquitectura: *Vision Tower* . . . 371
8.3.4 Arquitectura: *Text Tower* . . . 371
8.3.5 Estrategia de optimización . . . 373
8.3.6 Metodología de evaluación . . . 373
8.3.7 Configuración experimental y resultados . . . 375
8.3.8 Extensión simplificada: LIFT . . . 377
8.4 Representación supervisada de imágenes mediante clasificación . 380
8.4.1 Tarea y objetivo . . . 380
8.4.2 La arquitectura de CRATE . . . 381
8.4.3 Optimización . . . 381
8.4.4 Metodología de evaluación . . . 382
8.4.5 Diseño experimental y resultados . . . 382
8.5 Representación de imágenes por autocodificación enmascarada . 385
8.5.1 La tarea MAE y su objetivo . . . 385
8.5.2 Arquitectura MAE basada en CRATE . . . 386
8.5.3 Optimización . . . 388
8.5.4 Evaluación . . . 388

8.5.5 Experimentos . . . 389
8.6 Generación de imágenes mediante autocodificación y muestreo . 390
8.6.1 Autocodificación variacional de datos de imágenes 391
8.6.2 Autocodificación de representaciones de datos de imágenes 394
8.6.3 Muestreo a partir de representaciones aprendidas mediante eliminación de ruido . . . 397
8.6.4 Implementación de la eliminación de ruido mediante una red U-Net . . . 398
8.6.5 Implementación de la eliminación de ruido mediante un Transformer de difusión . . . 402
8.7 Representación y generación condicionadas de imágenes 406
8.7.1 Formulación del problema . . . 406
8.7.2 La implementación básica: generación de imágenes a partir de texto . . . 407
8.7.3 Mecanismos avanzados de condicionamiento . . . 412
8.7.4 Extensión a la generación de vídeos . . . 417
8.8 Generación de objetos 3D condicionada por imágenes y texto . . 419
8.8.1 Representaciones de formas en 3D, conjuntos de datos y tokenización . . . 420
8.8.2 Tarea . . . 425
8.8.3 Arquitectura y función objetivo . . . 426
8.8.4 Experimentos y análisis . . . 430
8.9 Reconstrucción de forma y pose en 3D basada en la generación . 436
8.9.1 Tarea y objetivo . . . 438
8.9.2 Representación y tokenización de formas 3D . . . 440
8.9.3 Arquitectura e implementación . . . 441
8.9.4 Modelado de flujo en cascada . . . 443
8.9.5 Experimentos . . . 445
8.9.6 Conclusión . . . 452
8.10 Generación condicional de movimientos del cuerpo humano . . . 452
8.10.1 Representación del cuerpo humano . . . 453
8.10.2 Tarea y configuración . . . 455
8.10.3 Diseño de la representación condicional . . . 456
8.10.4 La distribución a priori condicional . . . 458
8.10.5 Incorporación de medidas mediante guiado . . . 459
8.10.6 Algoritmo de muestreo condicional . . . 460
8.10.7 Conclusiones de los experimentos . . . 462
8.10.8 Discusión . . . 466
8.11 Representación y generación del lenguaje natural . . . 467
8.11.1 Datos . . . 467
8.11.2 Tarea y objetivo . . . 468
8.11.3 Arquitectura: CRATE causal . . . 472
8.11.4 Estrategia de optimización . . . 474
8.11.5 Metodología de evaluación . . . 475
8.11.6 Configuración experimental y resultados . . . 476
8.12 Escalado y mejora de los *Transformers white-box* . . . 477

8.12.1 Aumento del ancho de la red: CRATE-α 477
8.12.2 Transformadores de complejidad temporal lineal 481
8.12.3 Transformers de solo atención 483
8.13 Resumen y notas 485
8.14 Ejercicios y extensiones 486

9 Problemas abiertos y direcciones sobre inteligencia 487
9.1 ¿Hacia la inteligencia autónoma: cerrar el círculo? 488
9.2 Hacia la inteligencia natural: ¿Más allá de la retropropagación? . 490
9.3 Hacia la inteligencia científica: ¿más allá del test de Turing? . . . 492
9.3.1 La evolución de la inteligencia en la naturaleza 492
9.3.2 De la inteligencia inductiva a la deductiva 494
9.3.3 Pruebas científicas de inteligencia 496

Apéndices 501

A Métodos de optimización 501
A.1 Descenso por gradiente más pronunciado 501
A.1.1 Descenso por gradiente clásico para problemas suaves . . 502
A.1.2 Descenso por gradiente precondicionado para problemas mal condicionados 506
A.1.3 Descenso por gradiente proximal para problemas no suaves 509
A.1.4 Descenso de gradiente estocástico para problemas a gran escala 511
A.1.5 Uniéndolo todo: Adam 513
A.2 Cálculo de gradientes mediante diferenciación automática 515
A.2.1 Diferenciales 516
A.2.2 Diferenciación automática 519
A.2.3 Retropropagación 521
A.3 Teoría de juegos y optimización minimax 525
A.3.1 El aprendizaje de los equilibrios de Stackelberg 528
A.3.2 Consideraciones prácticas para el aprendizaje de los equilibrios de Stackelberg 532
A.4 Ejercicios 533

B Entropía, difusión, eliminación de ruido y codificación con pérdida 535
B.1 Entropía diferencial de distribuciones de baja dimensión 536
B.2 Procesos de difusión y eliminación de ruido 537
B.2.1 El proceso de difusión aumenta la entropía con el paso del tiempo 538
B.2.2 El proceso de eliminación de ruido reduce la entropía con el paso del tiempo 540
B.2.3 Lemas técnicos y resultados intermedios 544
B.3 Codificación con pérdida y empaquetamiento esférico 555

B.3.1 Demostración de la relación entre la distorsión de la tasa y la cobertura 556
B.3.2 Demostración del Lema B.6 559

Bibliography **563**

Capítulo 1

Una introducción informal a la inteligencia

> "*Así como el aumento constante de la entropía es la ley fundamental del universo, así también lo es, para la vida, el ser cada vez más altamente estructurada y luchar contra la entropía.*"
>
> – Václav Havel

1.1 Inteligencia, cibernética e inteligencia artificial

El mundo que habitamos no es plenamente aleatorio ni completamente impredecible.[1] En cambio sigue cierto orden, ciertos patrones y leyes que lo hacen predecible en gran medida.[2] El propio surgimiento de la vida y su persistencia dependen de esta predictibilidad. Solo aprendiendo y memorizando lo que es predecible en el entorno, la vida puede sobrevivir y prosperar, ya que las decisiones y acciones acertadas dependen de predicciones confiables. Dado que el mundo ofrece una variedad aparentemente ilimitada de fenómenos predecibles, los seres inteligentes—animales y humanos—han desarrollado sentidos cada vez más agudos: la vista, el oído, el tacto, el gusto y el olfato. Estos sentidos recogen datos sensoriales a gran velocidad para percibir las regularidades del entorno. Por tanto, una tarea fundamental de todos los seres inteligentes es

aprender y memorizar la información predecible a partir de volúmenes masivos de datos sensoriales.

[1] Si el mundo fuese plenamente aleatorio, un ser inteligente no tendría necesidad de aprender o memorizar nada.

[2] Algunos deterministas, otros probabilísticos.

Antes de poder entender cómo esto se logra es necesario abordar tres interrogantes:

- ¿Cómo puede modelarse y representarse matemáticamente la información predecible?
- ¿Cómo se puede aprender dicha información computacionalmente, de manera efectiva y eficiente, a partir de datos?
- ¿Cómo se debería organizar y estructurar de la mejor manera esta información como memoria o conocimiento (empírico), ya sea en nuestra mente o en una máquina, para apoyar las futuras predicciones e inferencias?

Este libro tiene como objetivo proporcionar algunas respuestas a estas preguntas. Estas respuestas nos ayudarán a mejorar nuestra comprensión de la inteligencia, especialmente los principios y mecanismos computacionales que la habilitan. La evidencia sugiere que todas las formas de inteligencia—desde la inteligencia de bajo nivel observable en las formas de vida tempranas y primitivas hasta las formas más altas de inteligencia, la práctica de la ciencia moderna—comparten un conjunto común de principios y mecanismos. A continuación lo explicamos con más detalle.

Surgimiento y la evolución de la inteligencia. Una condición necesaria para el surgimiento de la vida en la Tierra hace aproximadamente cuatro mil millones de años es que el ambiente sea en gran medida predecible. La vida ha desarrollado mecanismos que le permiten aprender lo que sea predecible sobre el medio ambiente, codificar esa información y usarla para la supervivencia. Hablando en general, a esta habilidad de adquirir conocimientos sobre el mundo la llamamos *inteligencia*. En gran medida, la evolución de la vida es obra de los mecanismos de la inteligencia [Ben23], al igual que la evolución del universo es obra de las leyes de la física. En las etapas tempranas de la vida, la inteligencia se desarrolla principalmente a través de dos tipos de mecanismos de aprendizaje: los *filogenéticos* y los *ontogenéticos* [Wie61].

La *inteligencia filogenética* se refiere al aprendizaje a través de la evolución de las especies. Las especies heredan y sobreviven principalmente basándose en la memoria y en el conocimiento codificado en el ADN o los genes de sus progenitores. En gran medida, podríamos decir que el ADN puede considerarse como los grandes modelos preentrenados de la naturaleza, porque estos desempeñan un papel muy parecido. La principal característica de la inteligencia filogenética es que los individuos tienen una capacidad de aprendizaje limitada. El aprendizaje se lleva a cabo mediante un mecanismo de «prueba-y-error» basado en mutaciones aleatorias de los genes y las especies evolucionan por medio de la selección natural —la supervivencia del más apto— tal como se muestra en la Figura 1.1. Esto puede verse como la implementación, por parte de la naturaleza, de lo que ahora se conoce como «aprendizaje por refuerzo», [SB18] o potencialmente de una «búsqueda de

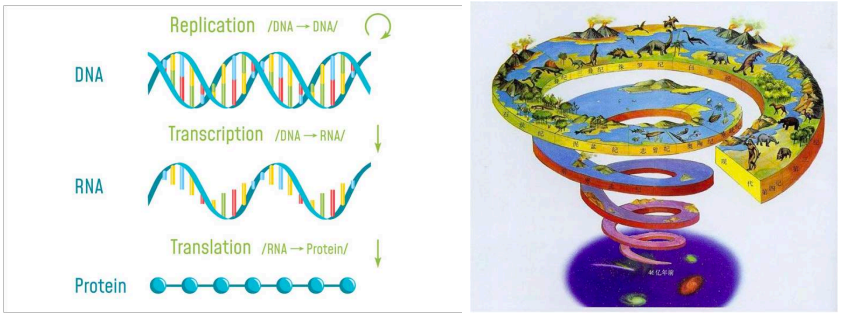


Figura 1.1: La evolución de la inteligencia filogenética: La memoria o el conocimiento del mundo exterior se codifica y transmite por vía del ADN (a la izquierda) y se decodifica de ADN a ARN y a proteínas. En etapas tempranas de la evolución de la vida (a la derecha), la inteligencia desarrolla memoria o conocimiento al nivel de la especie por vía de mutaciones (aleatorias) de genes y la selección natural —«que sobreviva el más apto»— lo que puede verse como una forma primitiva de aprendizaje por refuerzo.

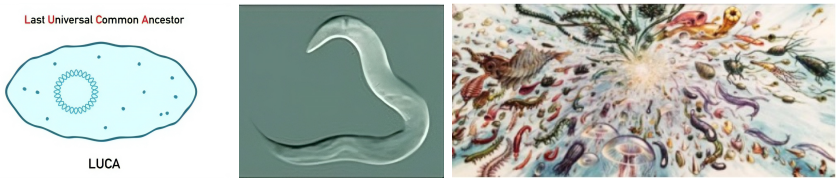


Figura 1.2: La evolución de la vida, desde el antepasado de toda la vida actual (LUCA—last universal common ancestor), un organismo de tipo unicelular que vivió hace 3500–4300 millones de años [MÁM+24], hasta el surgimiento del primer sistema nervioso en especies similares a gusanos (al medio), hace unos 550 millones de años [WVM+25] y hasta la explosión de las formas de vida en el Período Cámbrico (a la derecha), hace unos 530 millones de años.

arquitecturas neuronales»[ZL17]. Sin embargo, tal proceso de «prueba-y-error» puede ser extremadamente lento, exigente en recursos e impredecible. Desde el surgimiento de las primeras formas de vida, aproximadamente hace 4400 a 3800 millones de años[BBH+15], la vida se ha basado en esta forma de evolución.[3]

La *inteligencia ontogenética* se refiere a aquellos mecanismos de aprendizaje que permiten que un individuo aprenda a través de sus propios sentidos, memorias y predicciones dentro de su entorno específico, mejorando y adaptando sus conductas. El aprendizaje ontogenético empezó a ser posible después del surgimiento del sistema nervioso hace alrededor de 550–600 millones de años (en organismos similares a gusanos) [WVM+25], como se muestra en la

[3]Los lectores más perspicaces quizá noten una sorprendente similitud entre la forma en que se ha desarrollado la vida primitiva y la forma en que evolucionan hoy en día los grandes modelos de lenguaje (LLMs).

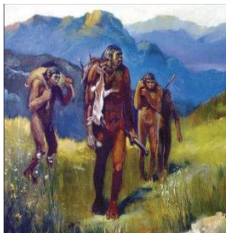
Figura 1.2 en el centro. Con su sistema sensorial y nervioso, un individuo puede formar y perfeccionar de manera continua su propia memoria del mundo, adicionalmente a lo que haya heredado mediante ADN o genes. Esta capacidad mejoró considerablemente las posibilidades de supervivencia del individuo contribuyendo a la explosión de las formas de vida en el Cámbrico, hace alrededor de 530 millones de años [Par04]. En comparación con el aprendizaje filogenético, el aprendizaje ontogenético es más eficiente y predecible, pudiendo llevarse a cabo dentro de los límites de los recursos de un individuo.

Ambos tipos de aprendizaje se basan en la retroalimentación del entorno externo —penalizaciones (la muerte) o recompensas (alimento)— que se aplican a las acciones de una especie o de un individuo.[4] Este hallazgo inspiró a Norbert Wiener a concluir en su programa de cibernética [Wie48] que todos los seres inteligentes, ya sean especies o individuos, se basan en mecanismos de retroalimentación en circuito cerrado para aprender y mejorar su memoria y conocimiento del mundo. Además, desde las plantas pasando por peces, aves y mamíferos, las especies más avanzadas se basan progresivamente en el aprendizaje ontogenético: permanecen con sus padres por períodos más largos después de su nacimiento y aprenden de ellos, porque los individuos de una misma especie deben sobrevivir en ambientes muy diversos. En otras palabras, a medida que las especies se vuelven cada vez más «inteligentes», dependen cada vez menos de su ADN heredado («modelos base preentrenados») y cada vez más de su capacidad para aprender por sí mismas después de nacer.

La evolución de la inteligencia humana. Desde la aparición del *Homo sapiens* hace alrededor de 2.5 millones de años [Har15], ha surgido una nueva forma superior de inteligencia, que evoluciona de manera más eficiente y económica. Las sociedades humanas desarrollaron lenguajes —primero orales, después escritos— como se muestra en la Figura 1.3. El lenguaje hace posible que los individuos se comuniquen y compartan información útil, permitiendo que la comunidad humana se comporte como un único organismo inteligente que aprende más rápido y retiene más conocimientos que cualquier individuo. Los textos escritos desempeñan, por tanto, un papel análogo al del ADN y los genes, ya que permiten que las sociedades acumulen y transmitan conocimientos entre territorios y generaciones. Podemos referirnos a este tipo de inteligencia como *inteligencia colectiva de la sociedad*, o *inteligencia societal*, diferenciándola de la inteligencia filogenética de las especies y de la inteligencia ontogenética de los individuos. Esta acumulación de conocimientos sostiene a todas las civilizaciones (antiguas).

Hace unos dos o tres mil años, la inteligencia humana dio otro gran salto cualitativo, permitiendo a filósofos y matemáticos desarrollar un conocimiento que va mucho más allá de la organización de observaciones empíricas. El desarrollo de conceptos y símbolos abstractos, como números, tiempo, espacio, lógica y geometría, dio origen a un lenguaje totalmente novedoso y riguroso de la *matemática*. Además, el desarrollo de la capacidad de formular hipótesis

[4] Las mutaciones génicas en el caso de las especies o las acciones ejecutadas por el individuo.



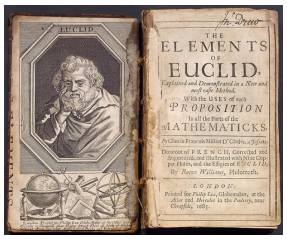


Figura 1.3: El desarrollo de la comunicación verbal y de los lenguajes orales (hace 70,000–30,000 años), de los lenguajes escritos (alrededor de 3000 AC) [Sch14] y de las matemáticas abstractas (alrededor de 500–300 AC) [Hea+56] constituyen tres hitos clave en la evolución de la inteligencia humana.

y comprobar su veracidad mediante la deducción lógica o la experimentación sentó las bases de la *ciencia* moderna. Por primera vez, los humanos pudieron descubrir y desarrollar nuevos conocimientos de manera proactiva y sistemática, con una precisión y un rigor sin precedentes. Esta forma de inteligencia ha mejorado significativamente la eficiencia en la adquisición de conocimientos sobre lo desconocido. A esta forma avanzada de inteligencia la llamaremos «inteligencia científica», debido a que es indispensable para el descubrimiento deductivo y científico.

Por lo tanto, según lo que podemos aprender de la naturaleza, cada vez que usamos la palabra «inteligencia», debemos especificar a qué nivel o forma nos referimos:

$$\textbf{inteligencia filogenética} \implies \textbf{ontogenética} \implies \textbf{social} \implies \textbf{científica}. \tag{1.1.1}$$

Se requieren caracterizaciones y distinciones claras porque queremos estudiar la inteligencia como disciplina de carácter científico-matemático. Aunque todas las formas puedan compartir el objetivo común de aprender y mejorar la memoria y el conocimiento del mundo, los mecanismos computacionales y las implementaciones físicas pueden diferir entre sí. Creemos que el lector entenderá y apreciará mejor sus rasgos comunes y sus diferencias después de estudiar este libro. Es por eso que dejamos las discusiones científicas o filosóficas más profundas sobre la relación entre estas diferentes formas o niveles de la inteligencia para el final del libro, en el Capítulo 9. De momento, basta con señalar que, en cuanto al tipo de conocimiento adquirido, las tres primeras etapas de la inteligencia comparten más rasgos en común: todas aprenden conocimiento (o memoria) de forma mayoritariamente empírica y luego lo utilizan de manera predominantemente inductiva. Este libro se centrará en estudiar y poner de manifiesto cómo este tipo de conocimiento puede aprenderse a partir de datos captados del mundo exterior. En este punto, los mecanismos que subyacen a la creación de conocimiento matemático y científico avanzado siguen siendo en gran parte un enigma, aunque creemos que deben construirse sobre la capacidad de desarrollar conocimiento empírico. Dejamos esta cuestión

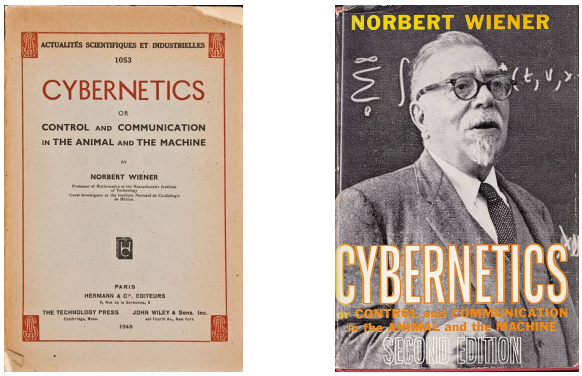


Figura 1.4: El libro «Cybernetics» (1948) de Norbert Wiener [Wie48] (a la izquierda) y su segunda edición (1961) [Wie61] (a la derecha).

como problema abierto para su discusión en el Capítulo 9.

El origen de la inteligencia artificial — la cibernética. En la década de 1940, impulsados por el esfuerzo bélico, los científicos, inspirados por la inteligencia natural, buscaron emular la inteligencia animal mediante máquinas, dando origen al movimiento de la «Cibernética», encabezado por Norbert Wiener [Kli11]. Wiener estudió zoología en Harvard, durante sus estudios de licenciatura, antes de convertirse en matemático y especialista en teoría de control. Consagró su vida a la comprensión y construcción de sistemas autónomos que pudieran reproducir una inteligencia similar a la de los animales. En la actualidad, el programa de la Cibernética muchas veces se interpreta estrechamente como algo centrado sobre todo en los sistemas de control por retroalimentación, área en la que Wiener hizo sus contribuciones técnicas más significativas. Sin embargo, el programa era mucho más amplio y profundo: buscaba comprender la inteligencia en su totalidad —al menos en el nivel animal— e influyó en la obra de toda una generación de científicos de renombre, entre los que se cuentan Warren McCulloch, Walter Pitts, Claude Shannon, John von Neumann y Alan Turing.

Se considera que Wiener fue el primero en estudiar la inteligencia *como sistema*, en lugar de centrarse en sus componentes o aspectos aislados. Su visión integral se publicó en el célebre libro de 1948 *Cibernética: o el control y la comunicación en los animales y las máquinas* [Wie48]. En ese libro y en su segunda edición publicada en 1961 [Wie61] (véase la Figura 1.4), intentó identificar diversas características y mecanismos necesarios de los sistemas inteligentes, incluyendo (pero sin limitarse a):

- Cómo *medir y almacenar* información (en el cerebro) y cómo comunicarse con los demás.[5] Esta idea llevó a Claude Shannon a formular la *teoría de*

[5] Wiener fue el primero en señalar que la «información» no es ni materia ni energía, sino una cantidad independiente digna de estudiarse.

la información en 1948 [Sha48].

- Cómo *corregir errores* en predicciones y estimaciones basándose en la información existente. El propio Wiener ayudó a formalizar la teoría del control por retroalimentación (en circuito cerrado) en la década de 1940.

- Cómo aprender a *tomar mejores decisiones* al interactuar con un entorno poco cooperativo o incluso hostil. John von Neumann formalizó este tema como *teoría de juegos* en 1944 [NMR44].

En 1943, inspirados por el programa de Cibernética de Wiener, el científico de la cognición Warren McCulloch y el lógico Walter Pitts formalizaron conjuntamente el primer modelo computacional de una neurona [MP43], llamado *neurona artificial* y más tarde lo ilustró en la Figura 1.15. Sobre la base de este modelo, Frank Rosenblatt construyó el Perceptrón Mark I en la década de 1950—una máquina física que contenía cientos de neuronas artificiales de ese tipo [Ros57]. El Perceptrón fue la primera red neuronal artificial materializada físicamente; véase la Figura 1.17. Cabe destacar que la arquitectura universal de computadoras de John von Neumann, propuesta en 1945, también fue diseñada para facilitar el objetivo de construir *máquinas computacionales* que pudieran materializar físicamente los mecanismos sugeridos por el programa de cibernética [Neu58].

Los lectores más perspicaces se darán cuenta de que los años cuarenta fueron realmente mágicos: se inventaron numerosas ideas fundamentales y se formalizaron diversas teorías influyentes, entre las que se cuentan el modelo matemático de las neuronas, las redes neuronales artificiales, las teorías de la información, de control, de juegos y las computadoras. En la Figura 1.5 se muestra a algunos de los pioneros. Cada una de estas contribuciones ha evolucionado, transformándose en el fundamento de un campo científico o de ingeniería que sigue ejerciendo un tremendo impacto en nuestras vidas. Todos ellos se inspiraron en el objetivo de desarrollar máquinas que emulen la inteligencia en la naturaleza. Los registros históricos muestran que el movimiento de la cibernética encabezado por Wiener ejerció influencia sobre casi todos estos pioneros y su obra. En gran medida, el programa de Wiener puede considerarse el verdadero precursor del actual programa de «inteligencia encarnada»; de hecho, el propio Wiener articuló una visión de dicho programa con notable claridad y concreción [Wie61].

Aunque Wiener identificó numerosas características y mecanismos esenciales de la inteligencia (encarnada), no ofreció una receta clara para la integración de estas en un sistema autónomo inteligente completo. Desde la perspectiva actual, algunos de sus puntos de vista fueron incompletos o inexactos. En particular, en el último capítulo de la segunda edición de *Cybernetics* [Wie61], insistió en la necesidad de *abordar la no linealidad* si se quiere que las máquinas emulen los mecanismos de aprendizaje típicos de la naturaleza, pero no ofreció ninguna solución concreta ni eficaz para este difícil problema. Para ser justos, incluso la teoría de los sistemas lineales se aún estaba en su etapa embrionaria en aquella

Figura 1.5: Pioneros de las fundaciones teóricas y computacionales de la inteligencia: Norbert Wiener (cibernética y teoría de control), Claude Shannon (teoría de la información), John von Neumann (teoría de juegos) y Alan Turing (teoría de la computación).

época,[6] y los sistemas no lineales parecían mucho menos accesibles.

No obstante, no podemos evitar de maravillarnos ante la clarividencia de Wiener respecto a la importancia de la no linealidad. Como se mostrará en este libro, la respuesta llegó sólo recientemente: la no linealidad puede ser tratada de manera efectiva a través de la linearización progresiva y la transformación que llevan a cabo las redes neuronales y representaciones profundas (véase el Capítulo 5). Más aún, demostraremos en este libro cómo todos los mecanismos listados con anterioridad pueden integrarse de manera natural en un sistema completo que exhibe ciertas características de un sistema autónomo inteligente (véase el Capítulo 6).

El origen de la inteligencia artificial. El subtítulo del libro *Cybernetics* de Wiener —*Control y comunicación en animales y máquinas*— revela que en la década de 1940 la investigación estaba primordialmente orientada a emular la inteligencia a nivel animal. Como se ha señalado anteriormente, las agendas de aquella época estaban dominadas por el movimiento cibernético de Wiener.

Alan Turing estuvo entre los primeros en reconocer esta limitación. En su célebre artículo de 1950 titulado «*Computing Machinery and Intelligence*» [Tur50], Turing planteó formalmente la cuestión de «¿las máquinas pueden pensar?» En otras palabras, si las máquinas podrían imitar inteligencia de nivel humano hasta el punto en que la inteligencia de las máquinas fuera indistinguible de la inteligencia humana—lo que hoy se conoce como la *prueba de Turing*.

Alrededor de 1955, un grupo de científicos jóvenes y ambiciosos intentó desmarcarse del programa de cibernética, que por entonces era dominante, y forjar su propio legado. Aceptaron el desafío de Turing de imitar la inteligencia humana y propusieron organizar un taller en el Dartmouth College para el verano de 1956. En su propuesta declaraban [MMR+06]:

«*El estudio se basará en la conjetura de que todos los aspectos del*

[6] El famoso logro de Wiener, el filtro de Wiener [Wie42; Wie49], que presentaremos más adelante, solo se aplicaba a sistemas lineales.

> *aprendizaje o cualquier otra característica de la inteligencia pueden, en principio, describirse con tal precisión que se pueda construir una máquina capaz de simularlos. Se intentará descubrir cómo hacer que las máquinas utilicen el lenguaje, formar abstracciones y conceptos, resuelvan tipos de problemas que ahora están reservados a los humanos y se perfeccionen a sí mismas.*»

Su objetivo era formalizar y estudiar la inteligencia superior que distingue a los seres humanos de los animales. Su programa incluía la abstracción, los métodos simbólicos, el lenguaje natural y el razonamiento deductivo (inferencia causal, deducción lógica, etc.), es decir, la inteligencia científica. El organizador del taller, John McCarthy—entonces un joven profesor adjunto de matemáticas en el Dartmouth College—acuñó el ahora famoso término «inteligencia artificial» (IA) refiriéndose a las máquinas que muestran inteligencia científica, al describir formalmente los problemas y objetivos del taller.

¿Renacimiento de la «inteligencia artificial» o «cibernética»? A lo largo de la última década, la inteligencia de las máquinas ha experimentado un desarrollo explosivo, impulsado en su mayor parte por las redes neuronales artificiales profundas, a raíz del trabajo de Geoffrey Hinton y sus estudiantes, en 2012 [KSH12]. Este período es aclamado como el «Renacimiento de la IA». Sin embargo, en lo que respecta a las tareas que realmente se abordan (reconocimiento, generación, predicción) y a las técnicas desarrolladas (aprendizaje por refuerzo, aprendizaje por imitación, codificación, decodificación, eliminación de ruido y compresión), estamos en gran medida emulando mecanismos comunes de la inteligencia de las primeras formas de vida y de los animales. Incluso en este aspecto, como aclararemos en este libro, los modelos y sistemas actuales de «IA» no han implementado de manera completa ni correcta todos los mecanismos necesarios para la inteligencia a nivel animal que ya se conocían en el movimiento de la Cibernética de la década de 1940.

En sentido estricto, los recientes avances en inteligencia maquinal de la última década se ajustan mal al programa de IA de Dartmouth en 1956. En cambio, lo que se ha logrado es más cercano a los objetivos del clásico programa de cibernética de Wiener de la década de 1940. Probablemente sea más apropiado denominar a la era actual el «Renacimiento de la Cibernética». El reciente auge de la denominada «IA encarnada» para robots y agentes autónomos se alinea aún más con los objetivos del programa de cibernética. Solo cuando comprendamos plenamente, desde una perspectiva científico-matemática, lo que realmente hemos logrado, podremos determinar qué problemas siguen sin resolverse y qué caminos conducen a la verdadera naturaleza de la inteligencia.[7] Ese es uno de los principales objetivos de este libro.

[7] Por lo tanto, retomaremos este tema al final de este libro, en el último capítulo, el 9.

1.2 ¿Qué aprender?

1.2.1 Predictibilidad

Los datos que contienen información útil se manifiestan de muchas formas. En su forma más natural, se pueden modelar como secuencias que son predecibles y computables. El concepto de una secuencia predecible y computable fue fundamental para la teoría de la información [Sha48] y la informática, lo que condujo en gran medida a la invención de las computadoras [Tur36]. El papel de las secuencias predecibles en la inferencia (inductiva) fue estudiado por Ray Solomonoff, Andrey Kolmogorov y otros en la década de 1960 [Kol98] como generalización de la clásica teoría de la información de Claude Shannon [Sha48]. Para ayudar a los lectores a comprender el concepto de secuencias predecibles, comenzaremos con algunos ejemplos concretos.

El caso escalar. Podría decirse que la secuencia discreta predecible más simple es la secuencia de los números naturales:

$$S = 1, 2, 3, 4, 5, 6, \ldots, n, n+1, \ldots \tag{1.2.1}$$

en la que el siguiente número x_{n+1} se define como el número anterior x_n más 1:

$$x_{n+1} = x_n + 1. \tag{1.2.2}$$

Se puede generalizar el concepto de predictibilidad a cualquier secuencia $\{x_n\}_{n=1}^{\infty}$ con $x_n \in \mathbb{R}$ si el número siguiente x_{n+1} puede computarse siempre a partir de su predecesor x_n:

$$x_{n+1} = f(x_n), \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.3}$$

donde $f(\cdot)$ es una función *computable* (escalar).[8] El trabajo pionero de Alan Turing de 1936 [Tur36] proporciona una definición rigurosa de computabilidad. En la práctica, a menudo suponemos además que f es computable de manera eficiente y tiene propiedades deseables, como la continuidad y la diferenciabilidad. La necesidad de estas propiedades quedará clara más adelante, una vez que comprendamos conceptos más sofisticados de computabilidad y su papel en el aprendizaje automático y la inteligencia artificial.

El caso multivariable. El valor siguiente también puede depender de dos predecesores. Por ejemplo, la famosa *secuencia de Fibonacci*

$$S = 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots \tag{1.2.4}$$

satisface

$$x_{n+2} = x_{n+1} + x_n, \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.5}$$

[8] Aquí destacamos que la función $f(\cdot)$ misma es computable, lo que significa que puede implementarse como un programa en una computadora.

De manera más general, podemos escribir

$$x_{n+2} = f(x_{n+1}, x_n), \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.6}$$

para cualquier función computable f con dos parámetros de entrada. Extendiendo aún más, el siguiente valor puede depender de los d valores precedentes:

$$x_{n+d} = f(x_{n+d-1}, \ldots, x_n), \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.7}$$

El número entero d se denomina el *grado* de la recursión. La expresión anterior (1.2.7) se denomina *autoregresión* y la secuencia resultante es *autoregresiva*. Cuando f es lineal, decimos que es una *autoregresión lineal*.

El caso vectorial. Para simplificar la notación, definimos un vector $\boldsymbol{x} \in \mathbb{R}^d$ que recoge d valores consecutivos de la secuencia:

$$\boldsymbol{x}_n \doteq [x_{n+d-1}, \ldots, x_n]^\top, \quad \boldsymbol{x}_n \in \mathbb{R}^d,\ n = 1, 2, 3, \ldots \tag{1.2.8}$$

Con esta notación, la relación recursiva (1.2.7) se convierte en

$$\boldsymbol{x}_{n+1} = g(\boldsymbol{x}_n) \in \mathbb{R}^d, \quad n = 1, 2, 3, \ldots \tag{1.2.9}$$

donde $g(\cdot)$ está unívocamente determinada por la función f en (1.2.7) y mapea un vector d-dimensional a un vector de dimensión d. En diferentes contextos, un vector de este tipo a veces es llamado «estado» o «token». Nótese que (1.2.7) define un mapeo $\mathbb{R}^d \to \mathbb{R}$, mientras que aquí tenemos $g\colon \mathbb{R}^d \to \mathbb{R}^d$.

Predicción controlada. También podemos definir una secuencia predecible que depende de otra secuencia predecible como entrada:

$$\boldsymbol{x}_{n+1} = f(\boldsymbol{x}_n, \boldsymbol{u}_n) \in \mathbb{R}^d, \quad n = 1, 2, 3, \ldots, \tag{1.2.10}$$

donde $\{\boldsymbol{u}_n\}$ con $\boldsymbol{u}_n \in \mathbb{R}^k$ es una secuencia predecible (computable). En otras palabras, el vector siguiente $\boldsymbol{x}_{n+1} \in \mathbb{R}^d$ depende tanto de $\boldsymbol{x}_n \in \mathbb{R}^d$ como de $\boldsymbol{u}_n \in \mathbb{R}^k$. En teoría de control, la secuencia $\{\boldsymbol{u}_n\}$ generalmente se denomina «entrada de control» y la secuencia $\boldsymbol{x}_n$ el «estado» o la «salida» del sistema (1.2.10). Un ejemplo clásico es un sistema dinámico lineal:

$$\boldsymbol{x}_{n+1} = \boldsymbol{A}\boldsymbol{x}_n + \boldsymbol{B}\boldsymbol{u}_n, \quad \boldsymbol{A} \in \mathbb{R}^{d\times d}, \boldsymbol{B} \in \mathbb{R}^{d\times k}, \tag{1.2.11}$$

que se estudia ampliamente en teoría de control [CD91].

A menudo la entrada de control viene dada por una función computable del propio estado $\boldsymbol{x}_n$:

$$\boldsymbol{u}_n = h(\boldsymbol{x}_n), \quad n = 1, 2, 3, \ldots \tag{1.2.12}$$

Como resultado, la secuencia $\{\boldsymbol{x}_n\}$ se obtiene por la composición de dos funciones computables f y h:

$$\boldsymbol{x}_{n+1} = f(\boldsymbol{x}_n, h(\boldsymbol{x}_n)), \quad n = 1, 2, 3, \ldots \tag{1.2.13}$$

De esta manera, la secuencia $\{\boldsymbol{x}_n\}$ nuevamente se vuelve una secuencia autoregresiva predecible. Cuando la entrada $\boldsymbol{u}_n$ depende de la salida $\boldsymbol{x}_n$, decimos que la secuencia resultante es generada por un sistema «en bucle cerrado» (1.2.13). Dado que el sistema en bucle cerrado ya no depende de ninguna entrada externa, decimos que tal sistema se ha hecho *autónomo*. Esto puede verse como un caso especial de autoregresión. Por ejemplo, si elegimos $\boldsymbol{u}_n = \boldsymbol{F}\boldsymbol{x}_n$ en el sistema lineal anterior (1.2.11), el sistema en bucle cerrado se transforma en

$$\boldsymbol{x}_{n+1} = \boldsymbol{A}\boldsymbol{x}_n + \boldsymbol{B}\boldsymbol{u}_n = \boldsymbol{A}\boldsymbol{x}_n + \boldsymbol{B}\boldsymbol{F}\boldsymbol{x}_n = (\boldsymbol{A} + \boldsymbol{B}\boldsymbol{F})\boldsymbol{x}_n, \tag{1.2.14}$$

lo que es una autoregresión lineal.

Procesos continuos. Las secuencias predecibles tienen contrapartes continuas naturales, que denominamos procesos predecibles. El proceso más simple de este tipo es el propio tiempo, $x(t) = t$.

Más generalmente, un proceso $\boldsymbol{x}(t)$ es *predecible* si, en cada instante t, su valor en el instante $t + \delta t$ se determina por su valor en t, donde δt es un incremento infinitesimal. Por lo general, $\boldsymbol{x}(t)$ es continuo y suave, por lo que la variación $\delta\boldsymbol{x}(t) = \boldsymbol{x}(t+\delta t) - \boldsymbol{x}(t)$ es infinitesimalmente pequeña. Estos procesos típicamente se describen mediante ecuaciones diferenciales (multivariables)

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t)), \quad \boldsymbol{x} \in \mathbb{R}^d. \tag{1.2.15}$$

En teoría de sistemas [CD91; Sas99], la ecuación (1.2.15) se conoce como modelo de espacio de estados. Un proceso controlado se modela por

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t), \boldsymbol{u}(t)), \quad \boldsymbol{x} \in \mathbb{R}^d, \boldsymbol{u} \in \mathbb{R}^k, \tag{1.2.16}$$

donde $\boldsymbol{u}(t)$ es una señal de entrada computable.

Ejemplo 1.1. La segunda ley de Newton determina la trayectoria $\boldsymbol{x}(t) \in \mathbb{R}^3$ de un cuerpo en movimiento sometido a una fuerza $\boldsymbol{F}(t) \in \mathbb{R}^3$:

$$m\ddot{\boldsymbol{x}}(t) = \boldsymbol{F}(t). \tag{1.2.17}$$

Cuando no hay fuerza, es decir, $\boldsymbol{F}(t) \equiv \boldsymbol{0}$, esto se reduce a la primera ley de Newton: el objeto se mueve a velocidad constante $\boldsymbol{v} \in \mathbb{R}^3$:

$$\ddot{\boldsymbol{x}}(t) = \boldsymbol{0} \iff \text{existe } \boldsymbol{v} \in \mathbb{R}^3 \text{ tal que } \dot{\boldsymbol{x}}(t) = \boldsymbol{v}. \tag{1.2.18}$$

■

1.2.2 Baja dimensión

Aprender a predecir. Ahora supongamos que se han observado o proporcionado muchos segmentos de la secuencia:

$$\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_i, \ldots, \boldsymbol{x}_N\} \tag{1.2.19}$$

extraídos de una secuencia predecible $\{x_n\}_{n=1}^{\infty}$. Sin pérdida de generalidad, supongamos que cada segmento tiene una longitud $D \gg d$, de manera que

$$\boldsymbol{x}_i = [x_{j(i)}, x_{j(i)+1}, \ldots, x_{j(i)+D-1}]^\top \in \mathbb{R}^D \tag{1.2.20}$$

para algún $j \in \mathbb{N}$. Luego se nos da un nuevo segmento $\boldsymbol{x}_t$ y se nos pide predecir sus valores futuros.

La dificultad es que la función generadora f y su orden d no se conocen:

$$x_{n+d} = f(x_{n+d-1}, \ldots, x_n). \tag{1.2.21}$$

Por lo tanto, el objetivo es aprender f y d a partir de los segmentos de muestra $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N$. La tarea principal del aprendizaje de la predicción es:

Dados numerosos segmentos de muestra de una secuencia predecible, ¿cómo podemos identificar de manera eficaz y eficiente la función f?

Predecibilidad y baja dimensión. Para identificar la función predictiva f, podemos observar una característica común de los segmentos de cualquier secuencia predecible dada por (1.2.21). Si tomamos un segmento largo, digamos de longitud $D \gg d$, y lo consideramos como un vector

$$\boldsymbol{x}_i = [x_i, x_{i+1}, \ldots, x_{i+D-1}]^\top \in \mathbb{R}^D, \tag{1.2.22}$$

entonces el conjunto de todos esos vectores $\{\boldsymbol{x}_i\}$ dista mucho de ser aleatorio y no puede ocupar todo el espacio $\mathbb{R}^D$. En cambio, tiene como máximo d grados de libertad: una vez que se conocen las primeras d entradas de cualquier $\boldsymbol{x}_i$, las entradas restantes quedan determinadas de manera unívoca. En otras palabras, todos los vectores $\{\boldsymbol{x}_i\}$ se encuentran en una superficie de dimensión d. En matemáticas, a una superficie de este tipo se le denomina subvariedad y se denota como $\mathcal{M} \subset \mathbb{R}^D$.

En la práctica, si elegimos una longitud de segmento D suficientemente grande, todos los segmentos muestreados a partir de la misma función predictiva se encuentran en una superficie con una dimensión intrínseca d, significativamente menor que la del espacio ambiente D. Por ejemplo, si la secuencia viene dada por la autoregresión lineal

$$x_{n+2} = ax_{n+1} + bx_n, \tag{1.2.23}$$

para constantes $a, b \in \mathbb{R}$ y si muestreamos segmentos de longitud $D = 10$, entonces todas las muestras se ubican en un plano de dimensión dos en $\mathbb{R}^{10}$, tal como se ilustra en la Figura 1.6. Al identificar este subespacio bidimensional se determinan completamente las constantes a y b en (1.2.23).

En términos más generales, cuando la función predictiva f es lineal, como en los sistemas que se muestran en (1.2.11) y (1.2.14), los segmentos largos siempre se encuentran en un subespacio lineal de baja dimensión. La identificación de la función predictiva equivale, en gran medida, a la identificación de este

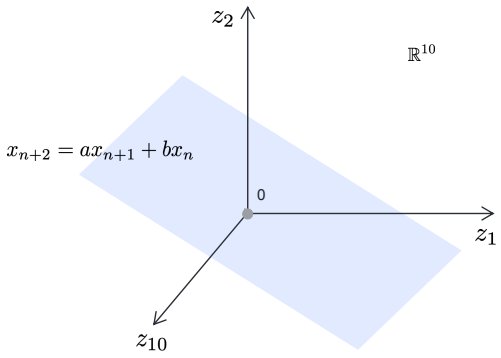


Figura 1.6: Un subespacio de dos dimensiones en un espacio ambiente de dimensión diez.

subespacio, un problema conocido como análisis de componentes principales. Discutiremos tales modelos y métodos clásicos en el Capítulo 2.

Esta observación se extiende a secuencias predecibles en general: si podemos identificar la superficie de baja dimensión en la que se encuentran las muestras del segmento, podemos identificar la función predictiva f.[9] No podemos dejar de subrayar la importancia de esta propiedad: *Todas las muestras de segmentos largos de una secuencia predecible se encuentran en una subvariedad de baja dimensión.* En gran medida, la ciencia y las matemáticas modernas tratan precisamente de identificar y describir esa estructura de baja dimensión de los fenómenos observados. Esto explica por qué los descubrimientos científicos más importantes y famosos acerca de nuestro mundo físico se describen a menudo mediante ecuaciones, tal y como ilustra la figura 1.7, ya que las ecuaciones restringen las cantidades observadas —que de otro modo serían independientes— a variedades de menor dimensión.

Según la teoría de cuerdas, que unifica casi todas las leyes físicas que conocemos sobre el mundo, todo el universo evoluciona en un espacio de 10 dimensiones, desde el Big Bang; véase la Figura 1.8. Como veremos en este libro, todos los métodos de aprendizaje modernos también explotan la propiedad de baja dimensión de los datos, ya sea de manera implícita o explícita.

Nota 1.1 (La entropía en física versus en inteligencia). En la figura 1.7, se observa que la única ley física que *no* se describe en forma de ecuación es la segunda ley de la termodinámica, la cual establece que la entropía de cualquier sistema físico cerrado, como el universo entero, siempre *aumenta*. En este libro se verá que el objetivo de la inteligencia (y la ciencia) consiste en identificar estructuras en los fenómenos observados que puedan *reducir* la entropía en nuestra predicción. Esto se estudiará con bastante detalle en el capítulo 3 y en el Apéndice B.

[9] Bajo condiciones moderadas, existe una correspondencia biunívoca entre la superficie de baja dimensión y la función f. Este hecho se ha aprovechado en problemas como la identificación de sistemas, que analizaremos más adelante.

17 Equations That Changed the World
by Ian Stewart

1.	**Pythagoras's Theorem**	$a^2 + b^2 = c^2$	Pythagoras,530 BC
2.	**Logarithms**	$\log xy = \log x + \log y$	John Napier, 1610
3.	**Calculus**	$\frac{\mathrm{d}f}{\mathrm{d}t} = \lim_{h \to 0} \frac{f(t+h) - f(t)}{h}$	Newton, 1668
4.	**Law of Gravity**	$F = G\frac{m_1 m_2}{r^2}$	Newton, 1687
5.	**The Square Root of Minus One**	$i^2 = -1$	Euler, 1750
6.	**Euler's Formula for Polyhedra**	$V - E + F = 2$	Euler, 1751
7.	**Normal Distribution**	$\Phi(x) = \frac{1}{\sqrt{2\pi\rho}} e^{\frac{(x-\mu)^2}{2\rho^2}}$	C.F. Gauss, 1810
8.	**Wave Equation**	$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$	J. d'Almbert, 1746
9.	**Fourier Transform**	$f(\omega) = \int_{\infty}^{\infty} f(x) e^{-2\pi i x \omega} \mathrm{d}x$	J. Fourier, 1822
10.	**Navier-Stokes Equation**	$\rho\left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v}\right) = -\nabla p + \nabla \cdot \mathbf{T} + \mathbf{f}$	C. Navier, G. Stokes, 1845
11.	**Maxwell's Equations**	$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}$ $\quad \nabla \cdot \mathbf{H} = 0$ $\nabla \times \mathbf{E} = -\frac{1}{c}\frac{\partial \mathbf{H}}{\partial t}$ $\quad \nabla \times \mathbf{H} = \frac{1}{c}\frac{\partial E}{\partial t}$	J.C. Maxwell, 1865
12.	**Second Law of Thermodynamics**	$\mathrm{d}S \geq 0$	L. Boltzmann, 1874
13.	**Relativity**	$E = mc^2$	Einstein, 1905
14.	**Schrodinger's Equation**	$ih\frac{\partial}{\partial t}\Psi = H\Psi$	E. Schrodinger, 1927
15.	**Information Theory**	$H = -\sum p(x) \log p(x)$	C. Shannon, 1949
16.	**Chaos Theory**	$x_{t+1} = kx_t(1 - x_t)$	Robert May, 1975
17.	**Black-Scholes Equation**	$\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} + \frac{\partial V}{\partial t} - rV = 0$	F. Black, M. Scholes, 1990

Figura 1.7: 17 ecuaciones famosas que cambiaron el mundo. La mayoría de estas ecuaciones describe leyes físicas del universo. En cuanto a la inteligencia, la ecuación 15 de entropía jugará un papel muy especial y explicaremos por qué en este libro, especialmente en el capítulo 3.

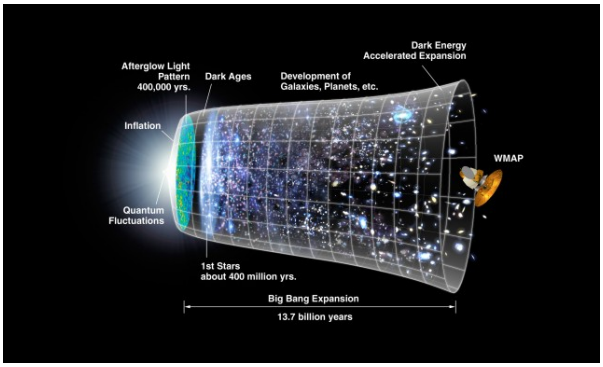


Figura 1.8: La evolución del Universo, según la mejor teoría física conocida hasta la fecha, se cree que tiene lugar en un espacio de 10 dimensiones.

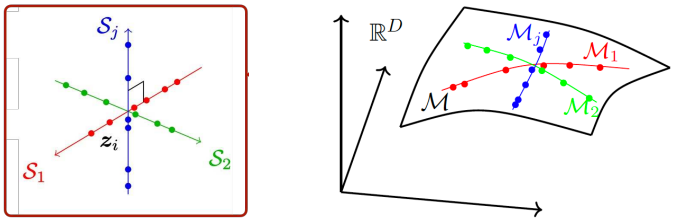


Figura 1.9: Datos distribuidos sobre una mezcla de subespacios (ortogonales) $\{\mathcal{S}_j\}$ (a la izquierda) o subvariedades $\{\mathcal{M}_j\}$ (a la derecha).

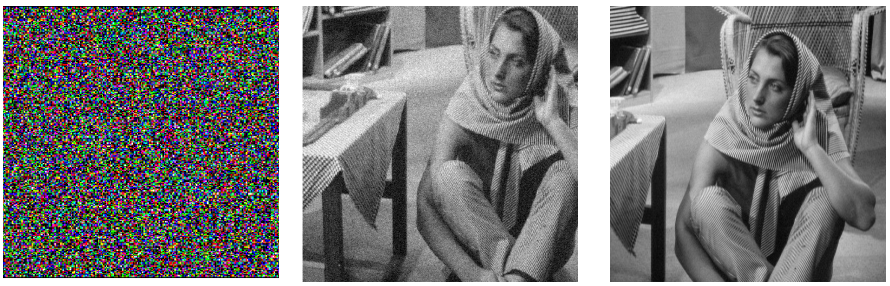


Figura 1.10: Una imagen de ruido aleatorio (a la izquierda) frente a una imagen con ruido (centro) y la imagen original sin ruido (a la derecha).

En situaciones reales, los datos observados suelen proceder de múltiples secuencias predecibles. Por ejemplo, una secuencia de video puede contener varios objetos en movimiento. En tales casos, los datos yacen sobre una mezcla de subespacios lineales de baja dimensión o subvariedades no lineales, tal como se ilustra en la Figura 1.9.

Propiedades de baja dimensión. La correlación temporal en secuencias predecibles no es la única razón de que los datos sean de baja dimensión. Por ejemplo, el espacio de todas las imágenes es enorme, pero en su mayor parte se compone de imágenes aleatorias sin estructura, como se muestra en la Figura 1.10 (a la izquierda). Las imágenes y los vídeos naturales, sin embargo, presentan un alto grado de redundancia debido a las fuertes correlaciones espaciales y temporales entre los valores de los píxeles. Esta redundancia nos permite identificar fácilmente si una imagen tiene ruido o está limpia, como se muestra en la Figura 1.10 (en la parte central y a derecha). En consecuencia, la distribución de las imágenes naturales tiene una dimensión intrínseca muy baja en relación con el número total de píxeles.

Dado que el aprendizaje de estructuras de baja dimensión es tan importante como ubicuo, el libro *Análisis de datos de alta dimensión con modelos de baja dimensión: principios, cálculo y aplicaciones* [WM22] comienza con la siguiente afirmación: «*El problema de identificar la estructura de baja dimensión de las*

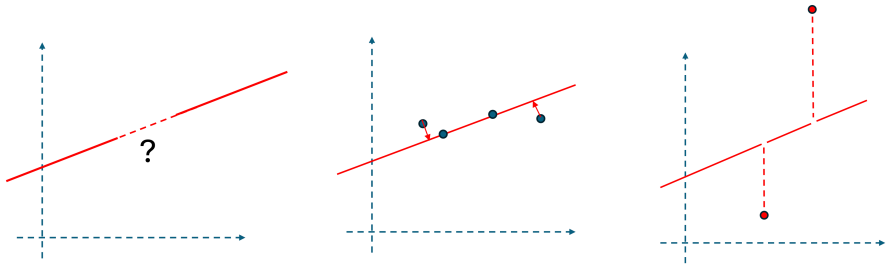


Figura 1.11: Ilustración de las propiedades de una estructura (lineal) de baja dimensión: permite el autocompletado (a la izquierda), la eliminación de ruido (al centro) y la corrección de errores (a la derecha).

señales o los datos en espacios de alta dimensión es uno de los problemas más fundamentales que, a lo largo de su larga historia, ha entrelazado numerosos campos de la ingeniería y las matemáticas, tales como la teoría de sistemas, el procesamiento de señales, el reconocimiento de patrones, el aprendizaje automático y la estadística.»

Al restringir el punto de datos observado $\boldsymbol{x}$ a una superficie o subvariedad de baja dimensión, hacemos que sus componentes sean muy dependientes entre sí o estén correlacionadas y, en cierto sentido, sean «predecibles» a partir de los valores de las demás componentes. Por ejemplo, si sabemos que los datos están restringidos a una superficie de dimensión d en $\mathbb{R}^D$, podemos realizar varias tareas útiles más allá de la predicción:

- **autocompletado**: si se dan más de d entradas de una muestra típica $\boldsymbol{x}$, las entradas restantes suelen determinarse en forma unívoca;[10]

- **eliminación de ruido**: si los valores de una muestra $\boldsymbol{x}$ son perturbados por un ruido leve, este ruido puede eliminarse eficazmente proyectando $\boldsymbol{x}$ de vuelta sobre la superficie;

- **corrección de errores**: si un pequeño número de entradas desconocidas de $\boldsymbol{x}$ se corrompen en forma arbitraria, pueden corregirse de manera eficiente.

La Figura 1.11 ilustra estas propiedades usando una estructura lineal de baja dimensión—una línea de dimensión uno en un plano bidimensional.

En condiciones suaves, estas propiedades se generalizan a muchas otras estructuras de baja dimensión en espacios de alta dimensión [WM22]. Veremos que estas propiedades útiles— por ejemplo, el autocompletado y la eliminación de ruido— inspiran el desarrollo de métodos eficaces en el aprendizaje de tales estructuras.

[10] La predicción se convierte en un caso especial de esta propiedad.

Para simplificar, hasta ahora hemos utilizado el caso determinista para introducir los conceptos de predictibilidad y baja dimensión, en el cual los datos se sitúan exactamente sobre estructuras geométricas como subespacios o superficies. Sin embargo, en la práctica, los datos siempre contienen cierta incertidumbre o aleatoriedad. En este caso, podemos suponer que los datos siguen una distribución probabilística con densidad $p(\boldsymbol{x})$. Una distribución se considera «de baja dimensión» si su densidad se concentra en torno a una estructura geométrica de baja dimensión—un subespacio, una superficie o una combinación de ambos— tal y como se muestra en la Figura 1.9. Esta estructura también se conoce como el «soporte» de la distribución. Una vez aprendida, esta densidad $p(\boldsymbol{x})$ sirve como una potente distribución a priori para estimar $\boldsymbol{x}$ a partir de observaciones parciales, ruidosas o corruptas:

$$\boldsymbol{y} = f(\boldsymbol{x}) + \boldsymbol{n}, \tag{1.2.24}$$

calculando la estimación condicional $\hat{\boldsymbol{x}}(\boldsymbol{y}) = \mathbb{E}(\boldsymbol{x} \mid \boldsymbol{y})$ o mediante el muestreo de la distribución condicional $\hat{\boldsymbol{x}}(\boldsymbol{y}) \sim p(\boldsymbol{x} \mid \boldsymbol{y})$.[11]

En numerosos trabajos anteriores se han estudiado las propiedades de conjuntos de datos cuyas distribuciones se apoyan en una superficie o subvariedad de baja dimensión, por ejemplo, [ACM12; LMR17]. Nos gustaría destacar aquí su importancia fundamental y la planteamos como el único supuesto en el que este libro basa su enfoque deductivo para comprender las redes profundas y la inteligencia en general:

> **El supuesto principal:** *Cualquier sistema inteligente o método de aprendizaje debería (y puede) basarse en la predictibilidad del mundo; por lo tanto, la distribución de las muestras de datos de alta dimensión observadas tiene un soporte de baja dimensión.*

La pregunta que queda por resolver es cómo aprender estas estructuras de baja dimensión de manera correcta y computacionalmente eficiente a partir de datos de alta dimensión. Como veremos, los modelos paramétricos estudiados en los enfoques analíticos clásicos y los modelos no paramétricos, como las redes profundas, tan populares en la práctica actual, no son más que diferentes medios para alcanzar un mismo fin.

1.3 ¿Cómo aprender?

1.3.1 Enfoques analíticos basados en modelos

Tengamos en cuenta que, aunque el cálculo de una función predictiva sea factible, eso no implica que sea factible o escalable aprender dicha función a partir de un número finito de segmentos muestreados. Un enfoque clásico para garantizar la factibilidad consiste en formular supuestos explícitos sobre

[11] Las tecnologías modernas de IA generativa, como la generación condicionada de imágenes, se basan en gran medida en este principio, tal y como veremos en el Capítulo 7

la familia de estructuras de baja dimensión con la que estamos trabajando. Históricamente, debido a las limitaciones en materia de cálculo y datos, los primeros modelos analíticos que se estudiaron fueron modelos simples e idealizados, ya que a menudo ofrecen soluciones eficientes en forma cerrada o numéricas. Además, ofrecen nuevas luces sobre problemas más generales y ya aportan soluciones útiles para casos importantes, aunque limitados. En el pasado, cuando los recursos computacionales eran escasos, los modelos analíticos que permitían soluciones eficientes en forma cerrada o soluciones numéricas eran los únicos que se podían implementar. Las *estructuras lineales* fueron la primera clase de modelos que se estudió a fondo.

Por ejemplo, podría decirse que el caso más sencillo es suponer que los datos se distribuyen en torno a un único subespacio de baja dimensión en un espacio de alta dimensión. De manera más o menos equivalente, se puede suponer que los datos se distribuyen según una distribución gaussiana de baja dimensión casi degenerada. Identificar dicho subespacio o gaussiana a partir de un número finito de muestras (con ruido) constituye el problema clásico del análisis de componentes principales (PCA) y se han desarrollado algoritmos eficaces para esta clase de modelos [Jol02]. La familia de modelos puede hacerse cada vez más compleja y expresiva. Asimismo se puede suponer que los datos se distribuyen en torno a una mezcla de componentes de baja dimensión (subespacios o gaussianas de baja dimensión), como en el análisis de componentes independientes (ICA) [BJC85], el aprendizaje de diccionarios (DL), el análisis de componentes principales generalizado (GPCA) [VMS05] o la clase más general de modelos esparsos de baja dimensión que se han estudiado extensivamente en los últimos años en campos como la detección compresiva [WM22].

En todas estas familias de modelos analíticos, el problema central consiste en identificar el modelo más compacto dentro de cada familia que mejor se ajuste a los datos proporcionados. A continuación, ofrecemos una breve descripción de estos modelos analíticos clásicos, pero dejamos un estudio más sistemático para el Capítulo 2. En teoría, estos modelos analíticos han ofrecido profundas perspectivas sobre las propiedades geométricas y estadísticas de las estructuras de baja dimensión. A menudo proporcionan soluciones en forma cerrada o algoritmos eficientes y escalables, que resultan muy útiles para datos cuya distribución puede aproximarse bien mediante dichos modelos. Y lo que es más importante, en el caso de problemas más generales, dan una idea de lo fácil o difícil que puede resultar identificar estructuras de baja dimensión y cuáles son las ideas básicas para abordar ese tipo de problemas.

Sistemas dinámicos lineales

El filtro de Wiener. Como se explicó en la Sección 1.2.1, una tarea central de la inteligencia consiste en aprender qué es lo predecible en las secuencias de observaciones. La clase más simple de secuencias predecibles —o señales— es

la generada por un proceso *lineal e invariante en el tiempo* (LTI):

$$x[n] = h[n] * z[n] + \epsilon[n], \tag{1.3.1}$$

donde $*$ es la operación de convolución, z es la entrada y h es la función de respuesta al impulso.[12] Aquí, $\epsilon[n]$ denota el ruido aditivo de la observación. Dado el proceso de entrada $\{z[n]\}$ y las observaciones del proceso de salida $\{x[n]\}$, el objetivo es encontrar el $h[n]$ óptimo tal que $\hat{x}[n] = h[n] * z[n]$ prediga $x[n]$ de manera óptima. La calidad de la predicción (es decir, su bondad) se mide mediante el error cuadrático medio (MSE):

$$\min_h \mathbb{E}[\epsilon[n]^2] = \mathbb{E}[\|x[n] - h[n] * z[n]\|_2^2]. \tag{1.3.2}$$

La solución óptima $h[n]$ se denomina filtro (de eliminación de ruido). Norbert Wiener —quien también fue quien inició el movimiento de Cibernética— estudió este problema en la década de 1940 y derivó una elegante solución en forma cerrada conocida como el *filtro de Wiener* [Wie42; Wie49]. Este resultado, también conocido como estimación y filtrado por mínima varianza, se convirtió en uno de los pilares del procesamiento de señales. Los lectores interesados pueden consultar [MKS+04, Apéndice B] para obtener una derivación detallada de este tipo de estimador.

El filtro de Kalman. La idea de atenuar el ruido en un sistema dinámico o filttrarlo fue posteriormente ampliada por Rudolph Kalman en la década de 1960 al caso de un sistema lineal invariante en el tiempo descrito por un modelo de espacio de estados de dimensión finita:

$$\boldsymbol{z}[n] = \boldsymbol{A}\boldsymbol{z}[n-1] + \boldsymbol{B}\boldsymbol{u}[n] + \boldsymbol{\epsilon}[n]. \tag{1.3.3}$$

El problema consiste en estimar el estado del sistema $\boldsymbol{z}[n]$ a partir de observaciones con ruido de la forma

$$\boldsymbol{x}[n] = \boldsymbol{C}\boldsymbol{z}[n] + \boldsymbol{w}[n], \tag{1.3.4}$$

donde $\boldsymbol{w}$ es ruido (blanco). El estimador de estado causal[13] óptimo que minimiza el error medio cuadrático mínimo (EMCM) de predicción

$$\min \mathbb{E}[\|\boldsymbol{x}[n] - \boldsymbol{C}\boldsymbol{z}[n]\|_2^2] \tag{1.3.5}$$

se expresa en forma cerrada mediante el denominado *filtro de Kalman* [Kal60]. Este resultado constituye una piedra angular de la teoría moderna de control, ya que permite estimar el estado de un sistema dinámico a partir de observaciones con ruido. Con ello, se puede introducir una retroalimentación lineal de estado,

[12] Por lo general, se supone que h tiene ciertas propiedades deseables, como una longitud finita o un espectro de banda limitada.

[13] Esto significa que el estimador solo puede utilizar observaciones hasta el paso temporal actual n. El filtro de Kalman es siempre causal, mientras que el filtro de Wiener no tiene por qué serlo.

por ejemplo, $\boldsymbol{u}[n] = \boldsymbol{F}\hat{\boldsymbol{z}}[n]$, y hacer que el sistema en bucle cerrado sea totalmente autónomo, tal y como se muestra en la Ecuación (1.2.13). Los lectores interesados pueden consultar [MKS+04, Apéndice B] para ver una derivación detallada del filtro de Kalman.

La identificación de sistemas dinámicos lineales. Para derivar el filtro de Kalman, se supone que los parámetros del sistema $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ son conocidos. Si no se proporcionan de antemano, el problema se vuelve más complejo y se conoce como *identificación de sistemas*: cómo *aprender* los parámetros $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ a partir de numerosas muestras de la secuencia de entrada $\{\boldsymbol{u}[n]\}$ y de la secuencia de observación $\{\boldsymbol{x}[n]\}$. Este es un problema clásico de la teoría de sistemas. Si el sistema es lineal, las secuencias de entrada y salida $\{\boldsymbol{u}[n], \boldsymbol{x}[n]\}$ se encuentran conjuntamente en un determinado subespacio de baja dimensión[14]. Por lo tanto, el problema de identificación equivale esencialmente a identificar este subespacio de baja dimensión [LV09; LV10; VM96].

Cabe señalar que los problemas anteriores tienen dos cosas en común: en primer lugar, se supone que las secuencias o señales sin ruido son generadas por una familia explícita de modelos paramétricos; en segundo lugar, estos modelos son esencialmente lineales. Conceptualmente, sea $\boldsymbol{x}_o$ una variable aleatoria cuya distribución «verdadera» está soportada sobre un subespacio lineal de baja dimensión, digamos S. En gran medida, tanto el filtro de Wiener como el filtro de Kalman tratan de estimar dicho $\boldsymbol{x}_o$ a partir de sus observaciones ruidosas:

$$\boldsymbol{x} = \boldsymbol{x}_o + \boldsymbol{\epsilon}, \quad \boldsymbol{x}_o \sim S, \tag{1.3.6}$$

donde $\boldsymbol{\epsilon}$ suele ser un ruido (o proceso) aleatorio gaussiano. Por lo tanto, sus soluciones se basan en identificar un subespacio lineal de baja dimensión que se ajuste mejor a los datos ruidosos observados. Al proyectar los datos sobre este subespacio, se obtienen las operaciones óptimas de eliminación de ruido, todas en forma cerrada.

Modelos lineales y lineales mixtos

Análisis de componentes principales. A partir de los problemas expuestos en el procesamiento clásico de señales y la identificación de sistemas, vemos que la tarea principal que subyace a todos estos problemas consiste en aprender un *único* subespacio lineal de baja dimensión a partir de observaciones con ruido. Matemáticamente, podemos modelar dicha estructura como

$$\boldsymbol{x} = \boldsymbol{u}_1 z_1 + \boldsymbol{u}_2 z_2 + \cdots + \boldsymbol{u}_d z_d + \boldsymbol{\epsilon} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\epsilon}, \quad \boldsymbol{U} \in \mathbb{R}^{D \times d}, \tag{1.3.7}$$

donde $\boldsymbol{\epsilon} \in \mathbb{R}^D$ es un ruido aleatorio de pequeña magnitud. La Figura 1.12 ilustra una distribución de este tipo con dos componentes principales. El problema consiste en encontrar la base del subespacio $\boldsymbol{U}$ a partir de numerosas muestras

[14] que tiene la misma dimensión que el orden del modelo de espacio de estados (1.3.3).

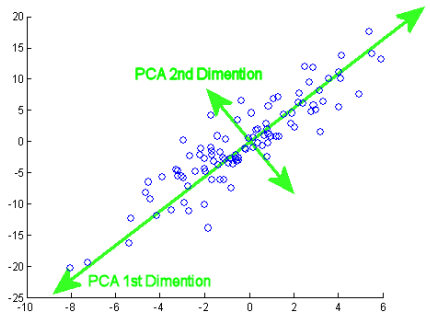


Figura 1.12: Una distribución con dos componentes principales.

de $\boldsymbol{x}$. Un enfoque habitual consiste en minimizar el error de la proyección sobre el subespacio:

$$\min_{\boldsymbol{U}} \mathbb{E}[\|\boldsymbol{x} - \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{z}\|_2^2]. \tag{1.3.8}$$

En esencia, se trata de una tarea de eliminación de ruido: una vez que se ha encontrado correctamente la base $\boldsymbol{U}$, podemos eliminar el ruido de la muestra ruidosa $\boldsymbol{x}$ proyectándola sobre el subespacio de baja dimensión generado por $\boldsymbol{U}$:

$$\boldsymbol{x} \to \hat{\boldsymbol{x}} = \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{x}. \tag{1.3.9}$$

Si el ruido es pequeño y el subespacio de baja dimensión $\boldsymbol{U}$ se aprende correctamente, esperamos que $\boldsymbol{x} \approx \hat{\boldsymbol{x}}$. Por lo tanto, PCA es un caso especial de lo que se conoce como esquema de «autocodificador», que codifica los datos proyectándolos sobre un espacio de menor dimensión, y decodifica un código de menor dimensión para volver al espacio original:

$$\boldsymbol{x} \xrightarrow{\boldsymbol{U}^\top} \boldsymbol{z} \xrightarrow{\boldsymbol{U}} \hat{\boldsymbol{x}}. \tag{1.3.10}$$

Debido a la sencillez de la estructura de datos, tanto el codificador $\mathcal{E}$ como el decodificador $\mathcal{D}$ se convierten en operadores lineales simples (de proyección y elevación).

Este problema clásico de la estadística se conoce como análisis de componentes principales (PCA). Fue estudiado por primera vez por Pearson en 1901 [Pea01] y, posteriormente, de manera independiente por Hotelling en 1933 [Hot33]. El tema se resume en forma sistemática en el libro clásico [Jol02; Jol86]. También se puede suponer explícitamente que los datos $\boldsymbol{x}$ se distribuyen según una única gaussiana de baja dimensión:

$$\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}\boldsymbol{U}^\top + \sigma \boldsymbol{I}), \quad \boldsymbol{U} \in \mathbb{R}^{D \times d}, \tag{1.3.11}$$

lo cual equivale a suponer que $\boldsymbol{z}$ en el modelo PCA (1.3.7) sigue una distribución normal estándar. Este problema se conoce como PCA probabilístico [TB99] y tiene la misma solución computacional que el PCA.

En este libro reexaminamos el PCA en el Capítulo 2 desde la perspectiva del aprendizaje de una distribución de baja dimensión. Nuestro objetivo es utilizar este modelo sencillo e idealizado para transmitir algunas de las ideas más fundamentales para el aprendizaje de una representación compacta de una distribución de baja dimensión, incluyendo conceptos importantes como la compresión mediante la eliminación de ruido y la autocodificación para obtener una representación coherente.

Análisis de componentes independientes. El análisis de componentes independientes (ICA) fue propuesto originalmente por [BJC85] como un modelo clásico para *aprender una buena representación*. De hecho, en un principio se propuso como un simple modelo matemático de nuestra memoria. El modelo ICA adopta una forma aparentemente similar a la del modelo PCA mencionado anteriormente (1.3.7), al suponer que la variable aleatoria observada $\boldsymbol{x}$ es una superposición lineal de múltiples componentes independientes z_i:[15]

$$\boldsymbol{x} = \boldsymbol{u}_1 z_1 + \boldsymbol{u}_2 z_2 + \cdots + \boldsymbol{u}_d z_d + \boldsymbol{\epsilon} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\epsilon}. \tag{1.3.12}$$

Sin embargo, en este caso se supone que las componentes z_i son variables independientes *no gaussianas*. Por ejemplo, una opción muy utilizada es

$$z_i = \sigma_i \cdot w_i, \quad \sigma_i \sim \mathrm{Ber}(p), \tag{1.3.13}$$

donde σ_i es una variable aleatoria de Bernoulli y w_i podría ser un valor constante u otra variable aleatoria, por ejemplo, una gaussiana.[16] El problema ICA tiene como objetivo identificar tanto $\boldsymbol{U}$ como $\boldsymbol{z}$ a partir de muestras observadas de $\boldsymbol{x}$. La Figura 1.13 ilustra la diferencia entre el ICA y el PCA.

Aunque el mapeo de $\boldsymbol{z}$ a $\boldsymbol{x}$ (la decodificación) parece lineal y sencilla una vez que se han aprendido $\boldsymbol{U}$ y $\boldsymbol{z}$, el mapeo de $\boldsymbol{x}$ a $\boldsymbol{z}$ (la codificación) puede ser complicado y es posible que no se pueda representar mediante un simple mapeo lineal. Por lo tanto, el ICA generalmente proporciona una autocodificación de la forma:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\boldsymbol{U}} \hat{\boldsymbol{x}}. \tag{1.3.14}$$

Por lo tanto, a diferencia del PCA, el ICA es algo más difícil de analizar y resolver. En la década de 1990, investigadores como Erkki Oja y Aapo Hyvärinen [HO97; HO00b] realizaron importantes contribuciones teóricas y algorítmicas al ICA. En el Capítulo 2, estudiaremos y presentaremos una solución para el ICA a partir de la cual quedará claro el mapeo de codificación $\mathcal{E}$. De hecho, Aapo Hyvärinen propuso el método general de *score matching* en 2005 [Hyv05], en parte para aprender distribuciones tales como las mezclas gaussianas, como en el ICA. Abordaremos este tema con mayor detalle más adelante y de nuevo en el 3.

[15] En el PCA, las «componentes» son los vectores aprendidos, mientras que en el ICA son escalares. Se trata simplemente de una diferencia de nomenclatura y de convención.

[16] ¡Aunque w sea gaussiana, σw ya no es una variable gaussiana!

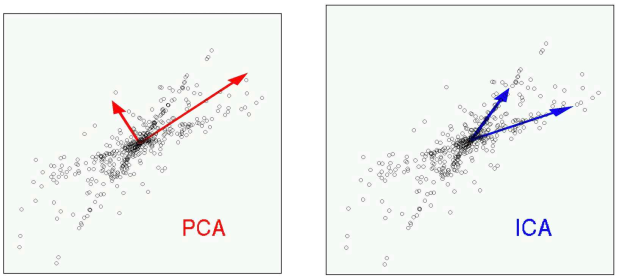


Figura 1.13: PCA (a la izquierda) versus ICA (a la derecha). Obsérvese que el PCA encuentra los vectores *ortogonales* que abarcan aproximadamente los datos, mientras que el ICA encuentra vectores (no necesariamente ortogonales) que se combinan *de manera independiente* para formar aproximadamente los datos, así como los coeficientes de esta combinación.

Estructuras esparsas y detección compresiva. Si p en (1.3.13) es muy pequeño, la probabilidad de que cualquier componente sea distinto de cero es baja. En este caso, decimos que $\boldsymbol{x}$ es un vector generado de manera esparsa y que se concentra en una unión de subespacios lineales cuya dimensión es $k = p \cdot d$. Por lo tanto, podemos ampliar el modelo ICA a una familia más general de estructuras de baja dimensión conocidas como modelos esparsos.

Un modelo k-esparso es el conjunto de todos los *vectores k-esparsos*:

$$\mathcal{Z} = \{\boldsymbol{z} \in \mathbb{R}^n \mid \|\boldsymbol{z}\|_0 \leq k\}, \tag{1.3.15}$$

donde $\|\cdot\|_0$ cuenta el número de entradas distintas de cero. Por lo tanto, $\mathcal{Z}$ es la unión de todos los subespacios de dimensión k alineados con los ejes de coordenadas, tal como se ilustra en la Figura 1.9 (a la izquierda). Un problema fundamental en el procesamiento de señales y la estadística consiste en recuperar un vector esparso $\boldsymbol{z}$ a partir de sus observaciones lineales

$$\boldsymbol{x} = \boldsymbol{A}\boldsymbol{z} + \boldsymbol{\epsilon}, \quad \boldsymbol{A} \in \mathbb{R}^{m \times n}, \tag{1.3.16}$$

donde $\boldsymbol{A}$ es dado, normalmente $m < n$, y $\boldsymbol{\epsilon}$ es un ruido pequeño. Este problema, que a primera vista parece sencillo, es NP-difícil de resolver e incluso difícil de aproximar (véase [WM22] para más detalles).

A pesar de su rica historia, que se remonta al siglo XVIII [Bos50], no existía ningún algoritmo de eficacia probada para esta clase de problemas, aunque entre los años de 1960 y 1990 se propusieron muchos algoritmos heurísticos. Algunos resultaron eficaces en la práctica, pero carecían de una justificación rigurosa. Se produjo un gran avance a principios de la década de 2000, cuando matemáticos como David Donoho, Emmanuel Candés y Terence Tao [CT05a; CT05b; Don05] establecieron un marco teórico riguroso que caracteriza las condiciones precisas bajo las cuales el problema de la recuperación esparsa puede resolverse de

manera efectiva y eficiente mediante la minimización convexa de ℓ^1:

$$\min \|\boldsymbol{z}\|_1 \quad \text{subject to} \quad \|\boldsymbol{x} - \boldsymbol{A}\boldsymbol{z}\|_2 \leq \epsilon, \tag{1.3.17}$$

donde $\|\cdot\|_1$ es la norma ℓ^1 de promoción de esparcidad de un vector y ϵ es una constante pequeña. Cualquier solución da como resultado una (auto) codificación esparsa

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\boldsymbol{A}} \hat{\boldsymbol{x}}. \tag{1.3.18}$$

Describiremos dicho algoritmo (y, por lo tanto, dicha correspondencia) en el Capítulo 2, revelando conexiones fundamentales entre la codificación esparsa y el aprendizaje profundo.[17]

Las condiciones para que la minimización de ℓ^1 tenga éxito son sorprendentemente generales: el número mínimo de mediciones m necesarias para una recuperación exitosa solo es proporcional a la dimensión intrínseca k. Este es el fenómeno de la *detección compresiva* [Can06]. Se extiende a amplias familias de estructuras de baja dimensión, incluidas las matrices de bajo rango. Estos resultados cambiaron radicalmente nuestra comprensión de la recuperación de estructuras de baja dimensión en espacios de alta dimensión. David Donoho, entre otros, celebró este cambio de tendencia como la «bendición de la dimensionalidad» [D D00], en contraste con la habitual creencia pesimista en la «maldición de la dimensionalidad». La teoría completa y el conjunto de resultados se presentan en [WM22].

No se puede subestimar la importancia computacional de este marco. Transformó problemas que antes se consideraban intratables en otros que no solo son abordables, sino que pueden resolverse de manera escalable mediante algoritmos extremadamente eficientes:

$$\text{intratable} \implies \text{tratable} \implies \text{escalable}. \tag{1.3.19}$$

Los algoritmos cuentan con rigurosas garantías teóricas de corrección, siempre que se cumplan unos requisitos precisos en cuanto a datos y cálculo. La naturaleza deductiva de este enfoque contrasta marcadamente con la práctica empírica e inductiva de las redes neuronales profundas. Sin embargo, ahora sabemos que ambos enfoques comparten un objetivo común: la búsqueda de estructuras de baja dimensión en espacios de alta dimensión.

Aprendizaje de diccionarios. Desde el punto de vista conceptual, surge un problema aún más complejo que el de la codificación esparsa (1.3.16) cuando la matriz de observación $\boldsymbol{A}$ es desconocida y debe aprenderse a partir de un conjunto de observaciones (posiblemente ruidosas) $\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n]$:

$$\boldsymbol{X} = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{E}, \quad \boldsymbol{A} \in \mathbb{R}^{m \times n}. \tag{1.3.20}$$

Aquí solo se nos da $\boldsymbol{X}$, no el correspondiente $\boldsymbol{Z} = [\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_n]$ ni el término de ruido $\boldsymbol{E} = [\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2, \ldots, \boldsymbol{\epsilon}_n]$, salvo que se sabe que los $\boldsymbol{z}_i$ son esparsos. Este es

[17] Ya en 2010 se señalaron similitudes entre los algoritmos de codificación esparsa y las redes profundas [GL10].

el *problema del aprendizaje de diccionarios*, que generaliza el problema del ICA (1.3.12) comentado anteriormente. En otras palabras, dado que la distribución de los datos $\boldsymbol{X}$ es la imagen de una distribución esparsa estándar $\boldsymbol{Z}$ bajo una transformación lineal $\boldsymbol{A}$, queremos aprender tanto $\boldsymbol{A}$ como su mapeo «inverso» $\mathcal{E}$ para obtener un autocodificador:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z} \xrightarrow{\boldsymbol{A}} \hat{\boldsymbol{X}}. \tag{1.3.21}$$

Tanto el PCA como el ICA y el aprendizaje de diccionarios parten del supuesto de que la distribución de los datos se apoya en estructuras lineales o lineales por tramos de baja dimensión, o se aproxima a ellas. Cada método requiere aprender una base (global o local) para estas estructuras a partir de muestras con ruido. En el Capítulo 2 estudiamos cómo identificar este tipo de estructuras de baja dimensión mediante estos modelos clásicos. En particular, veremos que todos estos modelos de baja dimensión, lineales o lineales por tramos, pueden aprenderse de manera eficiente mediante la misma familia de algoritmos rápidos conocida como *iteración de potencias* [ZMZ+20]. Aunque estos modelos lineales o lineales por tramos son algo idealizados para la mayoría de los datos del mundo real, comprenderlos es un primer paso esencial para comprender distribuciones de baja dimensión más generales.

Distribuciones generales

Las distribuciones de los datos del mundo real, como imágenes, vídeos y audio, son demasiado complejas como para ser modeladas mediante los modelos lineales o los procesos gaussianos citados anteriormente, que están, en cierta medida, idealizados. Normalmente no sabemos *a priori* de qué familia de modelos paramétricos se derivan. A lo largo de la historia, se han realizado numerosos intentos por desarrollar modelos analíticos para estos datos. En concreto, el ganador de la Medalla Fields, David Mumford, en la década de 1990 dedicó un esfuerzo considerable a intentar comprender y modelar las estadísticas de las imágenes naturales [Mum96]. Él y sus alumnos, entre ellos Song-Chun Zhu, se inspiraron en la física estadística y adoptaron sus técnicas, proponiendo numerosos modelos estadísticos y estocásticos para la distribución de imágenes naturales, tales como campos aleatorios o procesos estocásticos [HM99; LPM03; MG99; ZM97a; ZWM97; ZM97b]. Sin embargo, estos modelos analíticos solo lograron un éxito limitado a la hora de generar muestras que se asemejaran mucho a las imágenes naturales. Es evidente que, para datos del mundo real como las imágenes, necesitamos desarrollar métodos más potentes y unificadores que permitan identificar sus estructuras de baja dimensión más generales y obtener representaciones eficientes, por ejemplo, con fines de codificación y decodificación [ACM12; LMR17].

En la práctica, en el caso de una distribución general de datos del mundo real, por lo general solo disponemos de muchas muestras de dicha distribución: la llamada *distribución empírica*. En tales casos, normalmente no podemos esperar contar con una forma analítica clara para sus estructuras de baja

dimensión, ni para los operadores de eliminación de ruido resultantes.[18] Por lo tanto, necesitamos desarrollar una solución más general para estas distribuciones empíricas, no necesariamente en forma cerrada, pero al menos que se pueda calcular de manera eficiente. Si lo hacemos correctamente, las soluciones de los modelos lineales antes mencionados deberían convertirse en sus casos particulares.

Eliminación de ruido. En la década de 1950, los estadísticos comenzaron a interesarse por el problema de reducir el ruido en los datos muestreados de una distribución arbitraria. Sea $\boldsymbol{x}_o$ una variable aleatoria con función de densidad de probabilidad $p_o(\cdot)$. Supongamos que observamos una versión ruidosa de vx_o:

$$\boldsymbol{x} = \boldsymbol{x}_o + \sigma \boldsymbol{g}, \tag{1.3.22}$$

donde $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ es un ruido gaussiano estándar y σ es el nivel de ruido de la observación. Sea $p(\cdot)$ la función de densidad de probabilidad de $\boldsymbol{x}$.[19] Cabe destacar que la esperanza a posteriori de $\boldsymbol{x}_o$ dada $\boldsymbol{x}$ se puede calcular mediante una elegante fórmula, conocida como fórmula de Tweedie [Rob56]:[20]

$$\hat{\boldsymbol{x}}_o = \mathbb{E}[\boldsymbol{x}_o \mid \boldsymbol{x}] = \boldsymbol{x} + \sigma^2 \nabla \log p(\boldsymbol{x}). \tag{1.3.23}$$

Como se puede observar en la fórmula, la función $\nabla \log p(\boldsymbol{x})$ desempeña un papel muy especial en la reducción del ruido de la observación $\boldsymbol{x}$ en este caso. El ruido $\boldsymbol{g}$ se puede estimar explícitamente como

$$\hat{\boldsymbol{g}} = \frac{\boldsymbol{x} - \hat{\boldsymbol{x}}_o}{\sigma} = -\sigma \nabla \log p(\boldsymbol{x}), \tag{1.3.24}$$

para lo cual solo necesitamos conocer la distribución $p(\cdot)$ de $\boldsymbol{x}$, y no la distribución real $p_o(\cdot)$ para $\boldsymbol{x}_o$. Una consecuencia importante de este resultado es que, si añadimos ruido gaussiano a cualquier distribución, el proceso de eliminación de ruido puede llevarse a cabo fácilmente si logramos obtener de alguna manera la función $\nabla \log p(\boldsymbol{x})$.

Dado que se trata de un resultado tan importante y útil, ha sido redescubierto y utilizado en muchos contextos y ámbitos diferentes. Por ejemplo, tras la fórmula de Tweedie [Rob56], fue redescubierta unos años más tarde por [Miy61], donde se denominó «eliminación de ruido bayesiana empírica». A principios de la década de 2000, la función $\nabla \log p(\boldsymbol{x})$ fue redescubierta en el contexto del aprendizaje de una distribución general y Aapo Hyvärinen [Hyv05] la denominó «función de *score*». Su relación con la eliminación de ruido (bayesiana empírica) fue reconocida poco después por [Vin11]. El grupo de Eero Simoncelli ha realizado generalizaciones a otras distribuciones de medición (más

[18] Esto difiere de los casos del PCA, el filtro de Wiener y el filtro de Kalman.

[19] Es decir, $p(\boldsymbol{x}) = \int_{-\infty}^{\infty} \varphi_\sigma(\boldsymbol{x} - \boldsymbol{z}) p_o(\boldsymbol{z}) \mathrm{d}\boldsymbol{z}$, donde φ_σ es la función de densidad de la distribución gaussiana $\mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$.

[20] Herbert Robbins atribuyó el mérito de esta fórmula a Maurice Kenneth Tweedie, según se desprende de su correspondencia personal.

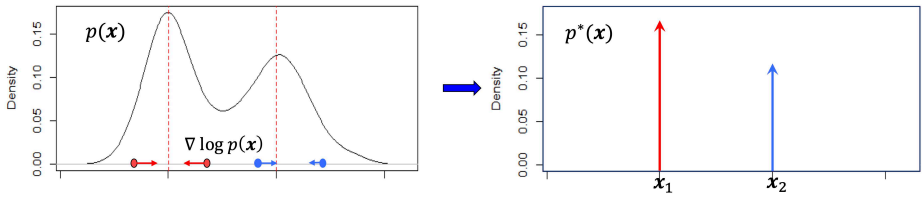


Figura 1.14: Interpretación geométrica de una función de *score* $\nabla \log p(\boldsymbol{x})$ para una distribución con densidad $p(\boldsymbol{x})$ a la izquierda. La operación generada por la función de *score* empuja la distribución hacia las zonas de mayor densidad. El objetivo es que, según una determinada medida de compacidad (por ejemplo, la entropía o la longitud de codificación), la distribución resultante esté más «comprimida». En el límite, la distribución converge hacia una que tiene un soporte de menor dimensión, como la $p^*(\boldsymbol{x})$ mostrada a la derecha.

allá del ruido gaussiano) [RS11]. Hoy en día, la aplicación más directa de la fórmula de Tweedie y la eliminación de ruido es la generación de imágenes mediante la eliminación iterativa de ruido [HJA20; KS21].

Minimización de entropía. El score tiene una interpretación intuitiva desde el punto de vista de la teoría de la información y la geometría. En teoría de la información, $-\log p(\boldsymbol{x})$ corresponde al número de bits que se necesitan para codificar $\boldsymbol{x}$[21]. El gradiente $\nabla \log p(\boldsymbol{x})$ apunta hacia regiones de mayor densidad de probabilidad, como se muestra en la Figura 1.14 (a la izquierda). Moverse en esta dirección reduce el número de bits necesarios para codificar $\boldsymbol{x}$. Por lo tanto, el operador $\nabla \log p(\boldsymbol{x})$ empuja a la distribución a «encogerse» hacia regiones de alta densidad. Formalmente, se puede demostrar que la entropía (diferencial)

$$H(\boldsymbol{x}) = -\int p(\boldsymbol{w}) \log p(\boldsymbol{w}) \mathrm{d}\boldsymbol{w} \tag{1.3.25}$$

disminuye bajo esta operación (véanse el Capítulo 3 y el Apéndice B). Con un libro de códigos óptimo, la distribución resultante alcanza una tasa de codificación más baja y, por lo tanto, está más comprimida. Si se repite este proceso de eliminación de ruido indefinidamente, se obtiene una distribución cuya masa de probabilidad se concentra en un soporte de menor dimensión. Por ejemplo, la distribución $p(\boldsymbol{x})$ en la Figura 1.14 (a la izquierda) converge a

[21] al menos para variables discretas, como se detalla en el Capítulo 3.

$p^*(\boldsymbol{x})$ (a la derecha):[22][23]

$$H(\boldsymbol{x}) = -\int p(\boldsymbol{w}) \log p(\boldsymbol{w}) \mathrm{d}\boldsymbol{w} \quad \xrightarrow{\text{disminuyendo}} \quad H^*(\boldsymbol{x}) = -\int p^*(\boldsymbol{w}) \log p^*(\boldsymbol{w}) \mathrm{d}\boldsymbol{w}. \tag{1.3.26}$$

A medida que la distribución converge hacia $p^*(\boldsymbol{x})$, su entropía diferencial se aproxima al infinito negativo debido a una diferencia técnica entre las definiciones de entropía continua y discreta. El Capítulo 3 resuelve esto utilizando una medida unificada de *tasa-distorsión*.

Más adelante en este capítulo y en los Capítulos 3 y 4, exploramos cómo este sencillo marco de eliminación de ruido y compresión unifica potentes métodos para el aprendizaje de distribuciones de baja dimensión en espacios de alta dimensión, incluidas las distribuciones de imágenes naturales.

1.3.2 Enfoques empíricos basados en datos

En la práctica, resulta difícil modelar datos importantes del mundo real —como imágenes, sonidos y texto— con los modelos lineales idealizados, lineales por tramos u otros modelos analíticos que se han comentado en la sección anterior. Por ello, a lo largo de la historia se han propuesto muchos modelos y métodos no paramétricos para modelar o capturar la distribución empírica de los datos y, posteriormente, utilizar dichos «modelos» para la inferencia. Estos modelos, especialmente las redes neuronales profundas, a menudo se han inspirado en el sistema nervioso biológico, ya que el cerebro de un animal o de un ser humano procesa esos datos con una eficiencia y eficacia notables. En comparación con los modelos analíticos presentados en la sección anterior, estos modelos basados en datos son los menos comprendidos, a pesar de que han dominado en gran medida la práctica de la tecnología de IA durante aproximadamente la última década. Por lo tanto, uno de los principales objetivos de este libro es ofrecer un marco conceptual sólido y unificado (en gran parte a través de los capítulos 5 a 8) para comprender tanto estos modelos empíricos «modernos» como los modelos analíticos clásicos y poner de manifiesto las conexiones fundamentales entre ellos.

Redes neuronales artificiales clásicas

La neurona artificial. Inspirándose en el sistema nervioso del cerebro, el primer modelo matemático de una neurona artificial[24] fue propuesto por Warren McCulloch[25] y Walter Pitts en 1943 [MP43]. Describe la relación entre la

[22] En sentido estricto, $p^*(\boldsymbol{x})$ es una función generalizada: $p^*(\boldsymbol{x}) = p^*(\boldsymbol{x}_1)\delta(\boldsymbol{x} - \boldsymbol{x}_1) + p^*(\boldsymbol{x}_2)\delta(\boldsymbol{x} - \boldsymbol{x}_2)$ con $p^*(\boldsymbol{x}_1) + p^*(\boldsymbol{x}_2) = 1$.

[23] Obsérvese que en esta sección analizamos el proceso de reducir el ruido y comprimir en forma iterativa una distribución con ruido y de alta entropía hasta que converja a la distribución de datos de baja entropía. Como veremos en el Capítulo 3, este problema es dual al problema de aprender una codificación óptima para la distribución de datos, lo cual también es una aplicación útil [RAG+24].

[24] conocida como «unidad de umbral lineal» o «perceptrón»

[25] un profesor de psiquiatría de la Universidad de Chicago en aquella época

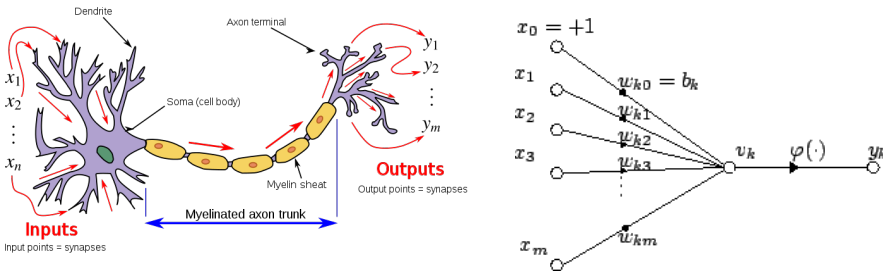


Figura 1.15: El primer modelo matemático de una neurona artificial (a la derecha) que emula la manera en que una neurona (a la izquierda) procesa señales.

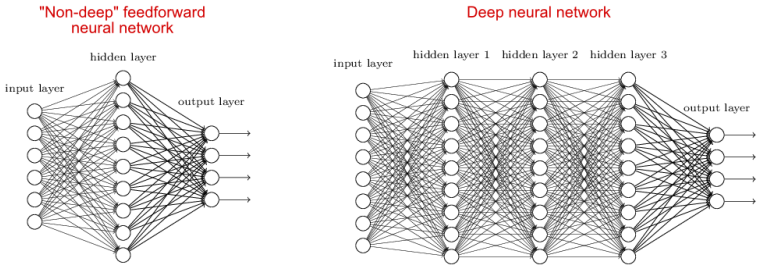


Figura 1.16: Una red con una sola capa oculta (a la izquierda) versus una red profunda (a la derecha).

entrada x_i y la salida o_j de la siguiente manera:

$$o_j = \varphi\Big(\sum_i w_{ji} x_i\Big), \tag{1.3.27}$$

donde $\varphi(\cdot)$ es alguna función de activación no lineal, que suele representarse mediante una función umbral. Este modelo se ilustra en la Figura 1.15. Como podemos ver, esta estructura ya comparte las principales características de una unidad básica en las redes neuronales profundas modernas. El modelo se basa en observaciones sobre el funcionamiento de una neurona individual en nuestro sistema nervioso. Sin embargo, los investigadores no sabían exactamente qué funciones podría realizar y llevar a cabo un conjunto de neuronas de este tipo. En un plano más técnico, tampoco tenían claro qué función de activación no lineal $\varphi(\cdot)$ debían utilizar. Por lo tanto, a lo largo de la historia se han propuesto muchas variantes.[26]

[26] Función escalonada, umbral duro o blando, unidad lineal rectificada (ReLU), sigmoide, etc. [DSC22].

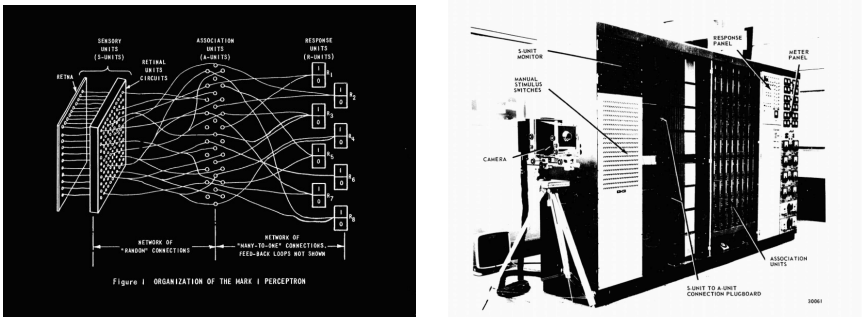


Figura 1.17: La máquina Perceptrón Mark I desarrollada por Frank Rosenblatt a finales de la década de 1950.

La red neuronal artificial. En la década de 1950, Frank Rosenblatt fue el primero en construir una máquina con una *red* de este tipo de neuronas artificiales, tal y como se muestra en la Figura 1.17. La máquina, denominada Mark I Perceptron, consta de una capa de entrada, una capa de salida y una única capa oculta de 512 neuronas artificiales, tal y como se muestra en la Figura 1.17 (a la izquierda), que es similar a lo que se ilustra en Figura 1.16 (a la izquierda). Se diseñó para clasificar imágenes ópticas de letras. Sin embargo, la capacidad de una red de una sola capa es limitada y solo puede aprender patrones linealmente separables. En el libro de 1969 *Perceptrons: An Introduction to Computational Geometry* de Marvin Minsky y Seymour Papert [MP69], se demostró que la arquitectura de una sola capa del Perceptrón Mark I no puede aprender una función XOR. Este resultado frenó considerablemente el interés por las redes neuronales artificiales, aunque más tarde se demostró que una red multicapa puede aprender una función XOR [RHW86a]. De hecho, una red multicapa lo suficientemente grande, como se muestra en la Figura 1.16 (a la derecha), compuesta por neuronas simples de este tipo, puede simular cualquier máquina de estados finitos, incluso la máquina universal de Turing.[27] Sin embargo, el estudio de las redes neuronales artificiales entró posteriormente en su primer invierno en la década de 1970.

Redes neuronales convolucionales. Los primeros experimentos con redes neuronales artificiales, como el Perceptrón Mark I, durante las décadas de 1950 y 1960 resultaron algo decepcionantes. Sugirieron que el simple acto de conectar neuronas de manera general, como en los perceptrones multicapa (MLP), podría no ser suficiente. Para construir redes eficaces y eficientes, resulta de gran ayuda comprender el propósito o la función colectiva que deben cumplir las neuronas de la red, de modo que se puedan organizar y aprender de manera especializada. Así pues, en aquel momento decisivo, el estudio de la inteligencia artificial volvió

[27] No hay que confundir lo que las redes neuronales son capaces de hacer en teoría con el hecho de que sea factible o fácil entrenar una red neuronal que realice determinadas funciones deseadas.

a buscar inspiración en el sistema nervioso animal.

Se sabe que la mayor parte de nuestro cerebro se dedica a procesar información visual [Pla99]. En las décadas de 1950 y 1960, David Hubel y Torsten Wiesel estudiaron sistemáticamente las cortezas visuales de los gatos. Descubrieron que la corteza visual contiene diferentes tipos de células —células simples y células complejas— que resultan sensibles a estímulos visuales de diferentes orientaciones y ubicaciones [HW59]. Hubel y Wiesel ganaron el Premio Nobel de Fisiología o Medicina de 1981 por este descubrimiento revolucionario.

En el ámbito de las redes neuronales artificiales, el trabajo de Hubel y Wiesel inspiró a Kunihiko Fukushima a diseñar la red «neocognitron» en 1980, la cual consiste en neuronas artificiales que emulan a las neuronas biológicas de la corteza visual [Fuk80]. Esta se conoce como la primera *red neuronal convolucional* (CNN), y su arquitectura se ilustra en la Figura 1.18. A diferencia del perceptrón, el neocognitrón tenía más de una capa oculta y podía considerarse una red profunda, tal y como se muestra en la Figura 1.16 (a la derecha).

Inspirándose también en el funcionamiento de las neuronas de la corteza visual del gato, Fukushima fue el primero en introducir la *unidad lineal rectificada* (ReLU):

$$\varphi(x) = \max\{0, x\} = \begin{cases} x, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0, \end{cases} \tag{1.3.28}$$

como función de activación $\varphi(\cdot)$ en 1969 [Fuk69]. Sin embargo, no fue hasta los últimos años que ReLU comenzó a utilizarse ampliamente en las redes neuronales profundas (convolucionales) modernas. En este libro se explicará por qué ReLU es una buena opción una vez que analizemos las principales operaciones que llevan a cabo las redes profundas: la compresión.

Las redes de tipo CNN siguieron evolucionando en la década de 1980 y se introdujeron y estudiaron muchas variantes diferentes. Sin embargo, a pesar de las notables capacidades de las redes profundas y de las arquitecturas mejoradas inspiradas en la neurociencia, seguía siendo extremadamente difícil entrenar esas redes profundas para tareas reales, como la clasificación de imágenes. Para que una red funcionara, era necesario recurrir a muchas heurísticas y trucos inexplicables, lo que limitaba el atractivo y la aplicabilidad de las redes neuronales. Un avance significativo se produjo alrededor de 1989, cuando Yann LeCun utilizó con éxito la *retropropagación* (BP) para entrenar una red neuronal convolucional profunda destinada al reconocimiento de dígitos escritos a mano [LBD+89], conocida posteriormente como LeNet (véase la Figura 1.19). Tras varios años de desarrollo constante, su perseverancia dio sus frutos: el rendimiento de LeNet acabó siendo lo suficientemente bueno como para su uso práctico a finales de la década de 1990 [LBB+98a]. El Servicio Postal de los Estados Unidos lo utilizaba para reconocer dígitos escritos a mano (en los códigos postales). LeNet se consideraba la red «prototipo» de todas las redes neuronales profundas modernas, tales como AlexNet [KSH12] y ResNet

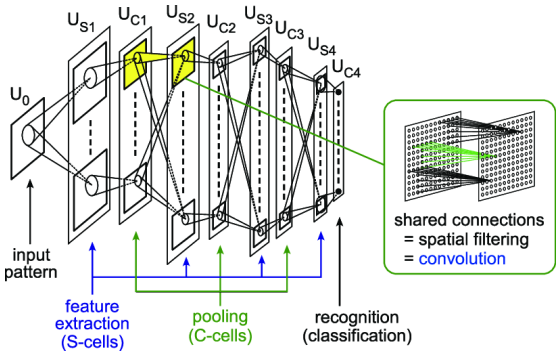


Figura 1.18: El origen de las redes neuronales convolucionales: el Neocognitron, creado por Kunihiko Fukushima en 1980. Cabe destacar que la alternancia entre capas de convoluciones y de agrupación emula las funciones de las células simples y las células complejas descubiertas en la corteza visual de los gatos.

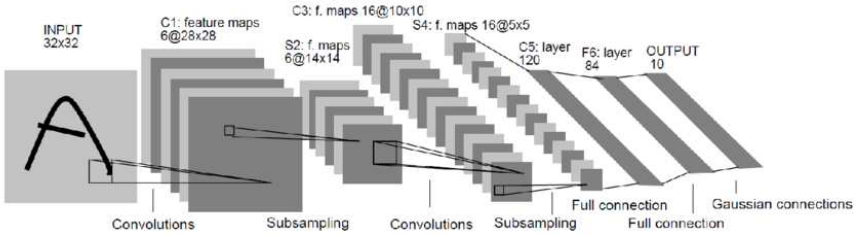


Figura 1.19: La red neuronal convolucional LeNet-5, diseñada por Yann LeCun en 1989.

[HZR+16b], de las que hablaremos más adelante. Por este trabajo, Yann LeCun recibió el Premio Turing de 2018.[28]

Retropropagación. A lo largo de la historia, el futuro de las redes neuronales profundas ha estado ligado a la facilidad y la eficiencia con que pueden entrenarse. La retropropagación (BP) se introdujo con este fin. Un perceptrón multicapa puede expresarse como una composición de transformaciones lineales y activaciones no lineales:

$$h(\boldsymbol{W}_1, \ldots, \boldsymbol{W}_L) = f^L(\boldsymbol{W}_L f^{L-1}(\boldsymbol{W}_{L-1} \cdots f^2(\boldsymbol{W}_2 f^1(\boldsymbol{W}_1 \boldsymbol{x})))). \tag{1.3.29}$$

Para entrenar los pesos de la red $\{\boldsymbol{W}_l\}_{l=1}^L$ mediante el descenso por gradiente, debemos evaluar el gradiente $\partial h / \partial \boldsymbol{W}_l$. La regla de la cadena en cálculo muestra que los gradientes pueden calcularse de manera eficiente para este tipo de funciones, una técnica que más tarde se denominó retropropagación o BP; véase el Apéndice A para más detalles. La BP ya era conocida en el

[28] Junto con otros dos pioneros de las redes profundas, Yoshua Bengio y Geoffrey Hinton.

ámbito del control óptimo y la programación dinámica durante las décadas de 1960 y 1970 y apareció en la tesis doctoral de Paul Werbos de 1974 [Wer74; Wer94]. En 1986, David Rumelhart y otros fueron los primeros en aplicarla para entrenar un perceptrón multicapa (MLP) [RHW86a]. Desde entonces, la BP se ha convertido en la técnica predominante para entrenar redes profundas, ya que es escalable y puede implementarse de manera eficiente en plataformas informáticas paralelas y distribuidas. Sin embargo, es probable que la naturaleza no utilice la BP,[29] ya que el mecanismo resulta demasiado costoso para su implementación física.[30] Esto deja un amplio margen para mejoras futuras.

A pesar de los avances algorítmicos mencionados, el entrenamiento de redes profundas seguía siendo complicado y computacionalmente costoso en las décadas de 1980 y 1990. A finales de la década de 1990, las máquinas de vectores de soporte (SVM) [CV95] habían ganado popularidad como una alternativa superior para las tareas de clasificación.[31] Las SVM ofrecían dos ventajas: un marco de aprendizaje estadístico riguroso conocido como la teoría de Vapnik–Chervonenkis (VC) y algoritmos eficientes de optimización convexa [BV04]. El auge de las máquinas de vectores de soporte marcó el inicio de un segundo invierno para las redes neuronales a principios de la década de 2000.

Autocodificación compresiva. A finales de la década de 1980 y a lo largo de la de 1990, las redes neuronales artificiales ya se usaban para aprender representaciones de baja dimensión para datos de alta dimensión como imágenes. Se había mostrado que las redes neuronales podían aprender PCA directamente a partir de los datos [BH89; Oja82], en lugar de usar los métodos clásicos expuestos en la Sección 1.3.1. Durante ese período también se argumentó que, gracias a su capacidad para modelar transformaciones no lineales, las redes neuronales podían aprender representaciones de baja dimensión para datos con distribuciones no lineales. De manera similar al caso del PCA lineal, es posible aprender simultáneamente un codificador no lineal de reducción de dimensiones f y un decodificador g, ambos modelados mediante una red neuronal profunda [Kra91; RHW86a]:

$$\boldsymbol{X} \xrightarrow{f} \boldsymbol{Z} \xrightarrow{g} \hat{\boldsymbol{X}}. \tag{1.3.30}$$

[29] Como hemos comentado anteriormente, la naturaleza aprende casi en forma generalizada a corregir los errores mediante la retroalimentación en bucle cerrado.

[30] La BP de extremo a extremo resulta computacionalmente inmanejable para estructuras neuronales tan complejas como el cerebro, especialmente si las actualizaciones deben realizarse en tiempo real. En cambio, las actualizaciones localizadas en pequeñas secciones del circuito neuronal son mucho más factibles desde el punto de vista biológico. Existen relativamente pocos trabajos sobre la aplicación de estas reglas de «aprendizaje local» al entrenamiento de redes profundas, eludiendo la BP [BS16; LTP25; MSS+22]. Creemos que esta es una oportunidad emocionante para mejorar la escalabilidad del entrenamiento de redes profundas.

[31] Ideas similares para problemas de clasificación se remontan a la tesis doctoral que Thomas Cover defendió en 1964, que se resumió y publicó en un artículo ese mismo año [Cov64].

Al garantizar la consistencia entre los datos decodificados $\hat{\boldsymbol{X}}$ y los originales $\boldsymbol{X}$—por ejemplo, minimizando un error de reconstrucción del tipo MSE[32]:

$$\min_{f,g} \|\boldsymbol{X} - \hat{\boldsymbol{X}}\|_2^2, \qquad \text{donde} \quad \hat{\boldsymbol{X}} = g(f(\boldsymbol{X})), \tag{1.3.31}$$

un autocodificador puede aprenderse directamente a partir de los datos $\boldsymbol{X}$.

¿Pero cómo podemos garantizar que una autocodificación de este tipo realmente capture las verdaderas estructuras de baja dimensión en $\boldsymbol{X}$ en lugar de producir uona representación trivial redundante? Por ejemplo, uno podría simplemente elegir que f y g sean mapeos de identidad y definir $\boldsymbol{Z} = \boldsymbol{X}$. Para asegurar que la autocodificación valga la pena, la representación resultante debería ser compresiva, según alguna medida calculable de complejidad. En 1993, Geoffrey Hinton y sus colegas propusieron usar la longitud de código como tal medida, transformando la meta de autocodificación en la búsqueda de representación que minimice la longitud de código [HZ93]. Este trabajo estableció una conexión fundamental entre el principio de longitud mínima de descripción[Ris78] y la minimización de la energía libre (Helmholtz). Los trabajos posteriores del grupo de Hinton [HS06] demostraron empíricamente que esa autocodificación es capaz de aprender representaciones significativas de baja dimensión para imágenes del mundo real. Pierre Baldi presentó un estudio exhaustivo sobre los autocodificadores en 2011 [Bal11], justo antes de que las redes profundas alcanzaran una gran popularidad. Analizaremos las medidas de complejidad y la autocodificación con más detalle en la Sección 1.4 y presentaremos un estudio sistemático de la autocodificación compresiva en el Capítulo 6 desde una perspectiva más unificada.

Las redes neuronales profundas modernas

Durante casi 30 años —desde la década de 1980 hasta la de 2010— las redes neuronales no fueron tomadas en serio por la mayor parte de la comunidad del aprendizaje automático. Las primeras redes profundas, como LeNet, mostraron un rendimiento prometedor en problemas de clasificación a pequeña escala, como el reconocimiento de dígitos, pero su diseño era en gran medida empírico, los conjuntos de datos disponibles eran ínfimos y la retropropagación resultaba computacionalmente prohibitiva para el hardware de la época. Estos factores provocaron una disminución del interés y un estancamiento en los avances y solo un puñado de investigadores siguió adelante.

Clasificación y reconocimiento. El enorme potencial de las redes profundas solo pudo ponerse de manifiesto una vez que se dispuso de datos y potencia computacional suficientes. En la década de 2010 habían surgido grandes conjuntos de datos como ImageNet y las GPU se habían hecho suficientemente

[32]Se sabe que los errores de tipo MSE son problemáticos para datos de imágenes con estructuras no lineales complejas. Como veremos en breve, gran parte de la investigación reciente sobre métodos generativos, incluidas las GAN, se ha centrado en encontrar mejores funciones de distancia sustitutivas entre los datos originales $\boldsymbol{X}$ y los datos regenerados $\hat{\boldsymbol{X}}$.

Figura 1.20: Las arquitecturas de LeNet [LBD+89] versus AlexNet [KSH12].

poderosas como para que la retropropagación resultara viable incluso para redes mucho más grandes que LeNet. En 2012, la red convolucional profunda AlexNet —que debe su nombre a Alex Krizhevsky, uno de los autores [KSH12]— atrajo amplia atención al superar a los métodos de clasificación existentes operando sobre ImageNet por un margen significativo. [33] En la Figura 1.20 se compara a AlexNet con LeNet. AlexNet retiene muchas características de LeNet — simplemente es más grande y reemplaza las activaciones por sigmoide de LeNet por ReLUs. Este trabajo contribuyó a que se le otorgara el Premio Turing de 2018 a Geoffrey Hinton.

Este éxito temprano inspiró a la comunidad de la inteligencia artificial a explorar nuevas arquitecturas para redes neuronales, así como variaciones y mejoras. Empíricamente, se descubrió que las redes más grandes y más profundas arrojan un mejor desempeño en tareas como la clasificación de imágenes. Entre las notables arquitecturas que surgieron se cuentan VGG[SZ15], GoogLeNet [SLJ+14], ResNet [HZR+16a] y, más recientemente, los transformers[VSP+17b]. Pese al rápido mejoramiento empírico del desempeño, las explicaciones teóricas de esas arquitecturas —y, de existir, las relaciones entre ellas— siguieron siendo escasas. Una meta de este libro consiste en desvelar el objetivo común que estas redes persiguen optimizar y en explicar sus características compartidas, tales como múltiples capas de operadores lineales intercalados con activaciones no lineales (véase el Capítulo 5).

[33] Las redes profundas ya habían logrado resultados de vanguardia en tareas de reconocimiento de voz, pero estos éxitos recibieron mucha menos atención que el salto cualitativo logrado en la clasificación de imágenes.

Aprendizaje por refuerzo. Los primeros éxitos de las redes profundas se dieron principalmente en tareas de clasificación supervisada, como el reconocimiento del habla y de imágenes. Más tarde, se empezaron a utilizar las redes profundas para aprender políticas de toma de decisiones o de control para los juegos. En este contexto, las redes profundas modelan la política óptima de decisión/control (es decir, una distribución de probabilidad sobre las acciones que se deben realizar para maximizar la recompensa esperada) o la función de valor óptimo asociada (una estimación de la recompensa esperada partiendo de un estado dado), tal y como se muestra en la Figura 1.21. Los parámetros de la red se optimizan de manera incremental[34] en función de las recompensas obtenidas por el éxito o el fracaso al jugar la partida con la política actual. Este paradigma de aprendizaje se denomina *aprendizaje por refuerzo* [SB18]; tiene su origen en la práctica de los sistemas de control de finales de la década de 1960 [MM70; WF65] y se remonta, a través de una larga y rica historia, a la *programación dinámica* de Richard Bellman [Bel57] y el *aprendizaje por ensayo y error* [Min54] de Marvin Minsky en la década de 1950.

Desde el punto de vista de la implementación, la combinación de redes profundas y aprendizaje por refuerzo resultó ser potente: las redes profundas pueden aproximar políticas de control y funciones de valor para entornos del mundo real que resultan difíciles de modelar analíticamente. Esto culminó en el sistema AlphaGo de DeepMind, que dejó atónito al mundo en 2016 al derrotar al destacado jugador de Go Lee Sedol y, en 2017, al campeón mundial Jie Ke.[35]

El éxito de AlphaGo sorprendió a la comunidad informática, la que durante largo tiempo había considerado que el espacio de estados de ese juego era demasiado vasto para cualquier solución eficiente en términos de cálculo y tamaño de muestra. La única explicación plausible es que el valor óptimo y la función de política del Go poseen una estructura favorable significativa: cualitativamente, sus dimensiones intrínsecas son suficientemente bajas como para que puedan ser bien aproximadas por redes neuronales que se pueden entrenar a partir de un número manejable de muestras.

Generación y predicción. Desde cierta perspectiva, las primeras aplicaciones de las redes profundas en la década de 2010 se centraban en extraer información relevante de los datos $\boldsymbol{X}$ y codificarla en una representación específica para cada tarea $\boldsymbol{Z}$ (supongamos que $\boldsymbol{Z}$ denota las etiquetas de clase en tareas de clasificación [36]):

$$\boldsymbol{X} \xrightarrow{f} \boldsymbol{Z}. \tag{1.3.32}$$

[34] Normalmente mediante retropropagación (BP)

[35] En 1996, Deep Blue, de IBM, hizo historia al derrotar al gran maestro ruso Garry Kasparov en una partida de ajedrez utilizando técnicas tradicionales de aprendizaje automático como la búsqueda y poda de árboles, métodos que han demostrado ser menos escalables y poco eficaces para juegos más complejos como el Go.

[36] Esta es una notación muy atípica para las etiquetas — la notación habitual es $\boldsymbol{y}$ — pero resulta útil para nuestros propósitos considerar las etiquetas como representaciones muy comprimidas y esparsas de los datos. Véase el Ejemplo 1.2 para más detalles.

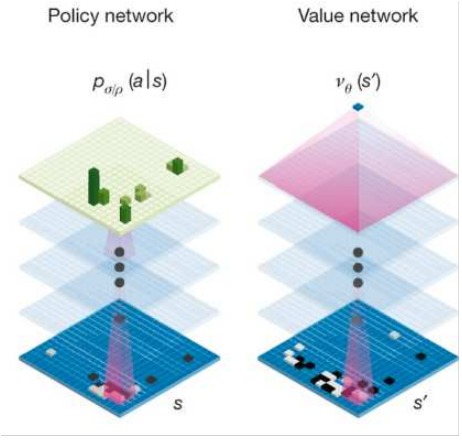


Figura 1.21: AlphaGo: el uso de redes neuronales profundas para modelar la función óptima de política o de valor para el juego Go.

En este caso, la función de mapeo aprendida f no necesita conservar la mayor parte de la información distributiva sobre $\boldsymbol{X}$; basta con retener solo las estadísticas suficientes para la tarea. Por ejemplo, una muestra $\boldsymbol{x} \in \boldsymbol{X}$ podría ser la imagen de una manzana, a la que f asigna la etiqueta de clase $\boldsymbol{z} =$ "manzana".

En muchos entornos modernos—como los denominados grandes modelos de propósito general («fundamentales»)—es posible que también tengamos que decodificar $\boldsymbol{Z}$ para recuperar el $\boldsymbol{X}$ correspondiente con una precisión preestablecida:

$$\boldsymbol{Z} \xrightarrow{g} \hat{\boldsymbol{X}}. \tag{1.3.33}$$

Como $\boldsymbol{X}$ típicamente representa datos observados en el mundo exterior, un buen decodificador nos permite simular o predecir lo que sucede en el mundo. En una tarea de «texto a imagen» o «texto a video», por ejemplo, $\boldsymbol{z}$ es el texto que describe la imagen deseada $\boldsymbol{x}$, y el decodificador debe generar un $\hat{\boldsymbol{x}}$ cuyo contenido concuerde con $\boldsymbol{x}$. Dada una clase de objetos $\boldsymbol{z} =$ "manzana", el decodificador g debería producir una imagen $\hat{\boldsymbol{x}}$ que se vea como una manzana, aunque no necesariamente idéntica al original $\boldsymbol{x}$.

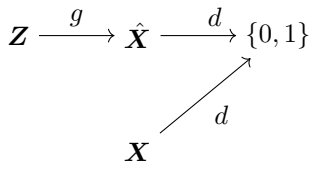
Generación por la vía de enfoques discriminativos. Para que las imágenes generadas $\hat{\boldsymbol{X}}$ parezcan imágenes naturales verdaderas $\boldsymbol{X}$, debemos ser capaces de evaluar y minimizar alguna distancia:

$$\min \operatorname{dist}(\boldsymbol{X}, \hat{\boldsymbol{X}}). \tag{1.3.34}$$

Resulta que la mayoría de las distancias motivadas por la teoría son extremadamente difíciles —si no imposibles— de calcular y optimizar para

distribuciones en espacios de alta dimensión con baja dimensión intrínseca.[37]

En 2007, Zhuowen Tu, decepcionado por los primeros intentos analíticos de modelar y generar imágenes naturales, decidió probar un enfoque radicalmente diferente. En un artículo publicado en CVPR 2007 [Tu07], sugirió por primera vez el aprendizaje de un modelo generativo para imágenes mediante un enfoque discriminativo. La idea es simple: si la evaluación de la distancia $\operatorname{dist}(\boldsymbol{X}, \hat{\boldsymbol{X}})$ resulta difícil, se puede, en su lugar, entrenar un discriminador d para separar $\hat{\boldsymbol{X}}$ de $\boldsymbol{X}$:



donde los rótulos 0 y 1 indican si una imagen es generada o real. Intuitivamente, mientras más difícil se hace separar $\hat{\boldsymbol{X}}$ de $\boldsymbol{X}$, más probablemente similares son.

El trabajo de Tu [Tu07] demostró por primera vez la viabilidad de aprender un modelo generativo a partir de un enfoque discriminativo. Sin embargo, el trabajo utilizaba métodos tradicionales para la generación de imágenes y la clasificación de la distribución (como el boosting), que resultaron ser lentos y difíciles de implementar. A partir de 2012, las redes neuronales profundas ganaron popularidad para la clasificación de imágenes. En 2014, Ian Goodfellow y sus colegas volvieron a proponer la generación de imágenes naturales mediante un enfoque discriminativo [GPM+14a]. Sugirieron modelar tanto el generador g como el discriminador d mediante redes neuronales profundas. Además, propusieron aprender g y d a través de un juego minimax:

$$\min_{g} \max_{d} \ell(\boldsymbol{X}, \hat{\boldsymbol{X}}), \tag{1.3.35}$$

donde $\ell(\cdot)$ es alguna función de pérdida natural asociada a la tarea de clasificación. En esta formulación, el discriminador d maximiza su éxito al separar $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$, mientras el generador g hace lo contrario. Este enfoque se denomina *redes generativas adversativas* (GANs). Se demostró que, una vez entrenadas con un gran conjunto de datos, las GAN pueden, de hecho, generar imágenes fotorrealistas. En parte debido a la influencia de este trabajo, Yoshua Bengio recibió el Premio Turing de 2018.

El enfoque discriminativo parece sortear hábilmente una dificultad fundamental en el aprendizaje de distribuciones. Sin embargo, hablando rigurosamente, este enfoque no resuelve plenamente la dificultad fundamental. En [GPM+14a] se demuestra que, si se elige adecuadamente la función

[37] Esto sigue siendo cierto incluso cuando se especifica una familia paramétrica de distribuciones para $\boldsymbol{X}$. La distancia suele volverse mal condicionada o mal definida para distribuciones con soportes de baja dimensión. Peor aún, la familia elegida podría aproximar en forma deficiente la verdadera distribución de interés.

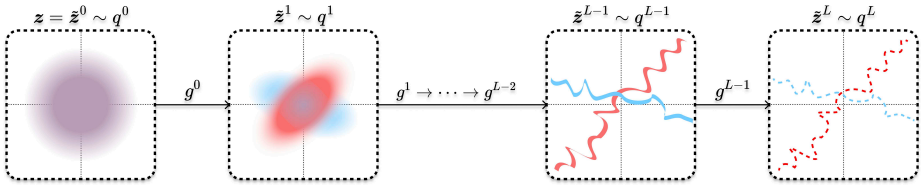


Figura 1.22: Ilustración de un proceso iterativo de eliminación de ruido y compresión que, a partir de una distribución gaussiana $q^0 = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, converge a una distribución arbitraria de datos de baja dimensión $q^L = p$.

de pérdida, la formulación minimax resulta matemáticamente equivalente a minimizar la distancia de Jensen-Shannon (véase [CT91]) entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$. Esto sigue siendo un problema difícil para dos distribuciones de baja dimensión en un espacio de alta dimensión. Por lo tanto, las GAN suelen basarse en numerosas heurísticas y trucos de ingeniería, y a menudo adolecen de problemas de inestabilidad, como el colapso de modos.[38]

Generación por vía de la eliminación de ruido y la difusión. En 2015, poco después de que se introdujeran las redes GAN y ganaran popularidad, Surya Ganguli y sus estudiantes propusieron que un proceso iterativo de eliminación de ruido modelado por una red profunda podría utilizarse para aprender una distribución general, como la de las imágenes naturales [SWM+15]. Su método se inspiró en las propiedades de unos procesos gaussianos y binomiales especiales estudiados por William Feller en 1949 [Fel49].[39]

Poco después, se demostró que los operadores de eliminación de ruido basados en la función de *score* [Hyv05], presentados brevemente en Sección 1.3.1, eran más generales y unificaban los procesos y algoritmos de eliminación de ruido y difusión [HJA20; SE19; SSK+21]. La Figura 1.22 ilustra el proceso que transforma una distribución gaussiana genérica $q^0 = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ en una distribución empírica (arbitraria) $p(\boldsymbol{x})$ ejecutando una secuencia de operaciones iterativas de eliminación de ruido (o compresión):

$$\boldsymbol{z}^0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}) \xrightarrow{g^0} \boldsymbol{z}^1 \xrightarrow{g^1} \cdots \xrightarrow{g^{L-1}} \boldsymbol{z}^L \sim p(\boldsymbol{x}). \tag{1.3.36}$$

A estas alturas, la eliminación de ruido (y la difusión) ha sustituido a las GAN y se ha convertido en el método predominante para el aprendizaje de distribuciones de imágenes y vídeos, lo que ha dado lugar a populares motores comerciales de generación de imágenes como Midjourney y Stability.ai. En el Capítulo 3 presentaremos y estudiaremos de manera sistemática el método de

[38] No obstante, tal formulación minimax ofrece una aproximación práctica de la distancia. Simplifica la implementación y evita ciertas dificultades que surgen al calcular directamente la distancia.

[39] ¡De nuevo, en la mágica época de los 40!

eliminación de ruido y difusión para aprender una distribución general de baja dimensión.

1.4 Un enfoque unificador

Hasta ahora, hemos ofrecido una breve exposición del objetivo principal y la historia de la inteligencia artificial, junto con muchas ideas y enfoques importantes relacionados con ella. En los últimos años, tras el éxito empírico de las redes neuronales profundas, se han realizado considerables esfuerzos para desarrollar marcos teóricos que nos ayuden a comprender las redes profundas diseñadas empíricamente, ya sea en lo que se refiere a componentes específicas que parecen necesarios (por ejemplo, el dropout [SHK+14], la normalización [BKH16; IS15], la atención [VSP+17b]) o en cuanto a sus comportamientos globales (por ejemplo, el doble descenso [BHM+19], el colapso neuronal [PHD20]).

Motivado en parte por esta tendencia, este libro persigue varios objetivos importantes y ambiciosos:

- Desarrollar un marco teórico que proporcione interpretaciones matemáticas rigurosas de las redes neuronales profundas.
- Asegurar la corrección de la distribución de los datos aprendida y su coherencia con la representación aprendida.
- Demostrar que el marco permite crear arquitecturas de alto rendimiento y sirve de guía para futuras mejoras prácticas.

En los últimos años, cada vez hay más indicios de que estos objetivos pueden en efecto alcanzarse aprovechando la teoría y las soluciones de los modelos analíticos clásicos de baja dimensión examinados con anterioridad (que se tratan con mayor detalle en el Capítulo 2) e integrando ideas fundamentales de campos relacionados, a saber, la teoría de la información y la codificación, la teoría de control y de juegos, así como la optimización. El objetivo de este libro es ofrecer una introducción sistemática a este nuevo enfoque.

1.4.1 Aprendizaje de representaciones parsimoniosas

Una condición necesaria para que cualquier tarea de aprendizaje resulte factible es que las secuencias de interés sean *calculables*, al menos en el sentido de Alan Turing [Tur36]. Es decir, una secuencia puede calcularse mediante un programa en una computadora común.[40] Además de ser computable, exigimos que el cálculo sea *manejable*.[41] Es decir, el costo computacional (espacial y temporal) de aprender y calcular la secuencia no debería crecer exponencialmente con la

[40] De hecho, hay secuencias bien definidas que no son computables.

[41] No es necesario que nos planteemos predecir cosas cuya complejidad computacional sea inmanejable, por ejemplo, que crezca exponencialmente con la longitud o la dimensión de la secuencia.

longitud. Además, tal y como vemos en la naturaleza (y en la práctica moderna de la inteligencia artificial), para la mayoría de las tareas prácticas, un sistema inteligente necesita aprender qué es lo predecible a partir de grandes volúmenes de datos en un espacio de muy alta dimensión, como los datos de visión, sonido y tacto. Por lo tanto, para hablar de inteligencia no es necesario considerar todas las secuencias y estructuras computables y manejables; conviene centrarnos únicamente en las secuencias y estructuras predecibles que admiten realizaciones *escalables* de sus algoritmos de aprendizaje y computación:

$$\textbf{computable} \implies \textbf{manejable} \implies \textbf{escalable}. \tag{1.4.1}$$

Esto es porque cualquier algoritmo que los seres inteligentes usen para aprender información útil éste tiene que ser *escalable*. Más concretamente, la complejidad computacional de los algoritmos debería escalar en forma adecuada —por lo general, de manera lineal o incluso sublineal— en función del tamaño y la dimensión de los datos. A nivel técnico, esto requiere que las operaciones en las que se basan los algoritmos para aprender sólo pueden utilizar información de oráculo que pueda calcularse de manera eficiente a partir de los datos. Más específicamente, cuando la dimensión es alta y la escala es grande,el único oráculo que se puede permitir calcular es o bien la información geométrica de primer orden sobre los datos[42] o bien la información estadística de segundo orden[43]. El objetivo principal de este libro es desarrollar un marco teórico y computacional en el que podamos desarrollar sistemáticamente soluciones o algoritmos eficientes y eficaces con tales oráculos y operaciones escalables para *aprender* estructuras de baja dimensión a partir de los datos muestreados y, posteriormente, la función predictiva.

Búsqueda de baja dimensiionalidad mediante compresión. A partir de los ejemplos de secuencias que presentamos en la Sección 1.2.1, queda claro que algunas secuencias son fáciles de modelar y calcular, mientras que otras son más difíciles. El costo computacional de una secuencia depende de la complejidad de la función predictora f. Cuanto mayor sea el grado de regresión d, más costoso resulta calcularlo. La función f podría ser una simple función lineal, o bien una función no lineal cuya especificación y cálculo resulten arbitrariamente difíciles.

Es razonable pensar que si una secuencia —según cualquier criterio que elijamos— es más difícil de especificar y calcular, entonces también será más difícil de aprender a partir de sus segmentos muestreados. Sin embargo, para cualquier secuencia predecible, existen de hecho muchas formas, a menudo infinitas, de especificarla. Por ejemplo, para la secuencia simple $x_{n+1} = ax_n$, también podríamos definir la misma secuencia como $x_{n+1} = ax_n + bx_{n-1} - bx_{n-1}$. Por lo tanto, sería muy útil desarrollar un concepto objetivo y riguroso de «complejidad» para cualquier secuencia computable dada.

[42] como aproximar una estructura no lineal localmente con subespacios lineales y calcular el gradiente de una función objetivo.

[43] como la covarianza o la correlación de los datos o sus características. Como veremos, las arquitecturas profundas más populares, como los Transformers, se basan en este tipo de información.

Andrey Kolmogorov, un matemático ruso, fue uno de los primeros en definir la complejidad para cualquier secuencia computable.[44] Propuso que, entre todos los programas que calculan la misma secuencia, la longitud del programa más corto mide su complejidad. Esto concuerda con la navaja de Occam: *elegir la teoría más simple que explique la misma observación.* Formalmente, sea p un programa que genera una secuencia $\boldsymbol{x}$ en la máquina universal $\mathcal{U}$. La complejidad de Kolmogorov de $\boldsymbol{x}$ es:

$$K(\boldsymbol{x}) = \min_{p\,:\,\mathcal{U}(p)=\boldsymbol{x}} L(p). \tag{1.4.2}$$

Por lo tanto, la complejidad mide el grado de parsimonia con el que podemos especificar o calcular la secuencia. Esta definición es conceptualmente importante e históricamente inspiró profundos estudios en complejidad computacional e informática teórica.

La longitud del programa más corto representa la compresión extrema de la secuencia, lo que cuantifica el beneficio que obtenemos al aprender su mecanismo generativo. Sin embargo, la complejidad de Kolmogorov es, por lo general, *incomputable* [CT91] y su aproximación resulta prácticamente imposible. Por lo tanto, tiene poca utilidad práctica: no permite predecir las dificultades del aprendizaje ni evaluar lo bien que hemos aprendido.

Una medida computable de parsimonia. Para fines prácticos, necesitamos una medida eficientemente computable de complejidad para secuencias generadas por la misma función predictiva.[45] Cabe señalar que parte de la razón por la que la complejidad de Kolmogorov no es computable se debe en parte a que su definición no es constructiva.

Para introducir una medida computable de la complejidad, podemos adoptar un enfoque más constructivo, tal y como proponía Claude Shannon en el marco de la teoría de la información [CT91; Sha48].[46] Suponiendo que la secuencia $\boldsymbol{x}$ se extrae de una distribución probabilística $p(\boldsymbol{x})$, la entropía de la distribución:[47]

$$h(\boldsymbol{x}) \doteq -\int p(s)\log p(s)\mathrm{d}s \tag{1.4.3}$$

proporciona una medida natural de su complejidad. Esta medida también tiene una interpretación natural como número medio de bits binarios que se requieren para codificar tal secuencia, como veremos en el Capítulo 3.

[44] Muchos contribuyeron a esta noción de complejidad de secuencias, entre los que destacan Ray Solomonoff y Greg Chaitin. Los tres desarrollaron la teoría algorítmica de la información por separado: Solomonoff en 1960, Kolmogorov en 1965 [Kol98] y Chaitin alrededor de 1966 [Cha66].

[45] En la práctica, lo que típicamente nos interesa es aprender la función predictiva f, en lugar de cualquier secuencia particular generada por f.

[46] Este marco ha guiado con éxito la práctica ingenieril en la industria de las comunicaciones durante más de 80 años.

[47] Aquí consideramos la entropía diferencial, ya que suponemos que la secuencia está formada por variables continuas. Para variables discretas, utilizaríamos $H(\boldsymbol{x}) = -\sum_i p(s_i)\log p(s_i)$.

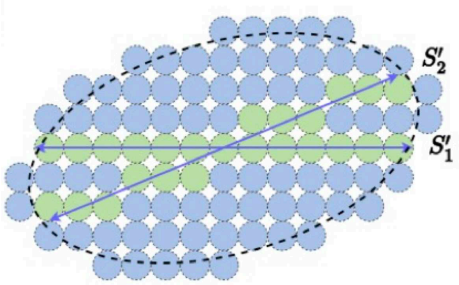


Figura 1.23: Comparación de dos esquemas de codificación. Imaginemos que la distribución verdadera de los datos se ubica a lo largo de las dos líneas con flechas S_1' and S_2'. Las muestras de estas líneas pueden codificarse con un libro de códigos de todas las bolas azules, o bien con un libro de códigos formado únicamente por las bolas verdes. El segundo esquema arroja una longitud/tasa de codificación mucho menor para la misma precisión.

Para ilustrar las principales ideas de este enfoque, tomemos un gran número de segmentos largos de secuencia:

$$\{x_1, x_2, \ldots, x_i, \ldots, x_N\} \subset \mathbb{R}^D, \tag{1.4.4}$$

generado por una función predictiva f. Sin pérdida de generalidad, suponemos que todas las secuencias tienen la misma longitud D, de modo que cada secuencia puede ser vista como vector en $\mathbb{R}^D$. Introducimos un esquema de codificación (con un libro de códigos), denotado $\mathcal{E}$, que mapea cada segmento x_i a una secuencia única de bits binarios $\mathcal{E}(x_i)$ y un esquema de decodificación correspondiente $\mathcal{D}$ que mapea las cadenas de bits binarios de vuelta a segmentos de secuencias (digamos que sujetos a un error máximo de $\epsilon \geq 0$ ya que tal esquema de codificación puede tener pérdidas). El esquema más sencillo consiste en llenar el espacio abarcado por todos los segmentos con bolas de radio ϵ, tal como se muestra en Figura 1.23. A continuación, numeramos las bolas, y cada secuencia se codifica como el índice binario de la bola más cercana, mientras que cada índice binario se decodifica de nuevo para obtener el centro de la bola correspondiente. Por lo tanto, cada segmento puede recuperarse con una precisión de ϵ a partir de su secuencia de bits.

Luego, para una codificación $\mathcal{E}$, la complejidad de la función predictiva f puede evaluarse como la longitud de codificación media $L(\mathcal{E}(x))$ de todas las secuencias x que genera, conocida como la tasa de codificación:[48]

$$R(x \mid \mathcal{E}) = \mathbb{E}[L(\mathcal{E}(x))] \approx \frac{1}{N} \sum_{i=1}^{N} L(\mathcal{E}(x_i)). \tag{1.4.5}$$

[48]Se puede hacer esto más preciso tomando $R(\boldsymbol{x} \mid \mathcal{E})$ como la longitud de codificación esperada para todos los segmentos $\boldsymbol{x}$ de longitud D.

La medida de la tasa de codificación cambiará si utilizamos un esquema de codificación (o libro de códigos) diferente. En la práctica, cuanto mejor conozcamos la estructura de baja dimensión en torno a la cual se distribuyen los segmentos, más eficiente será el libro de códigos que podamos diseñar, tal como se ilustra en la Figura 1.23. Los lectores más perspicaces se darán cuenta de que, desde el punto de vista conceptual, el proceso de eliminación de ruido ilustrado en la Figura 1.22 se asemeja mucho al paso del esquema de codificación con las bolas azules al de las bolas verdes.

Dados dos esquemas de codificación $\mathcal{E}_1$ y $\mathcal{E}_2$ para los segmentos, si la diferencia entre las tasas de codificación es positiva:

$$R(x \mid \mathcal{E}_1) - R(x \mid \mathcal{E}_2) > 0, \tag{1.4.6}$$

podemos decir que el esquema de codificación $\mathcal{E}_2$ es mejor. Esta diferencia puede verse como una medida de cuánta más información tiene $\mathcal{E}_2$ en comparación con $\mathcal{E}_1$ acerca de la distribución de los datos. En gran medida, el objetivo de aprender la distribución de los datos es equivalente al de hallar el esquema de codificación y decodificación más eficiente que minimiza la tasa de codificación con una precisión deseada:

$$\min_{\mathcal{E},\mathcal{D}} R(x \mid \mathcal{E}) \quad \text{sujeto a} \quad \operatorname{dist}(x, \mathcal{D}(\mathcal{E}(x))) \leq \epsilon. \tag{1.4.7}$$

Como veremos en el Capítulo 4, la tasa mínima que se puede lograr está estrechamente relacionada con la noción de entropía $H(x)$ (1.4.3).

Nota 1.2. La idea de medir la complejidad de los datos mediante esquemas de codificación explícitos ha motivado varios criterios de aprendizaje propuestos para revisar la complejidad de Kolmogorov y mejorar su computabilidad [WD99], entre ellos la longitud mínima del mensaje (MML) propuesta en 1968 [WB68] y la longitud mínima de descripción (MDL) en 1978 [HY01; Ris78]. Estos objetivos incluyen la longitud del esquema de codificación $\mathcal{E}$ en sí mismo (incluido su libro de códigos) además de los datos vx de interés: $L(\mathcal{E}(x)) + L(\mathcal{E})$. Sin embargo, si el objetivo es aprender un libro de códigos de tamaño finito y aplicarlo a un gran número de secuencias, el costo amortizado del libro de códigos puede ignorarse, ya que

$$\frac{1}{N}\left(L(\mathcal{E}) + \sum_{i=1}^{N} L(\mathcal{E}(x_i))\right) \approx \frac{1}{N}\sum_{i=1}^{N} L(\mathcal{E}(vx_i)) \tag{1.4.8}$$

a medida que N se hace grande.

Una vez más, se puede considerar que el esquema de codificación óptimo resultante es aquel que logra la mejor compresión de los datos observados. En general, en comparación con la complejidad de Kolmogorov, la longitud de codificación dada por cualquier esquema de codificación siempre será mayor:

$$K(\boldsymbol{x}) < L(\mathcal{E}(\boldsymbol{x})). \tag{1.4.9}$$

Por lo tanto, minimizar la tasa o longitud de codificación equivale, en esencia, a minimizar una cota superior de la complejidad de Kolmogorov, que de otro modo sería no calculable.

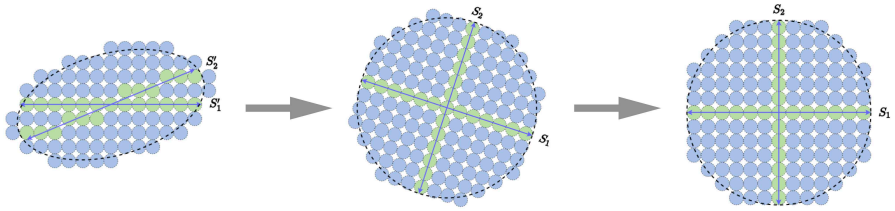


Figura 1.24: Transformación de la distribución de baja dimensión de los datos identificada —aquí sobre dos subespacios lineales— en una representación más informativa y estructurada, en la que los dos subespacios se vuelven más incoherentes.

1.4.2 Aprendizaje de representaciones informativas

Si el objetivo simplemente fuese comprimir los datos proporcionados por el solo hecho de hacerlo, los códigos óptimos que se acercan a la complejidad de Kolmogorov se convertirían, en teoría, en secuencias casi aleatorias o carentes de estructura [Cha66].[49] Sin embargo, nuestro verdadero objetivo al aprender la distribución de datos de las secuencias S es poder utilizarla repetidamente y con facilidad en predicciones futuras. Por lo tanto, aunque la compresión nos permite identificar la distribución de baja dimensión en los datos, nos gustaría codificar esta distribución de una manera *estructurada y organizada* para que la representación resultante sea informativa y eficiente de utilizar[50], a modo de «memoria». La Figura 1.24 ilustra de manera intuitiva por qué es deseable dicha transformación.

Como veremos en el Capítulo 4, estas estructuras deseadas en la representación final pueden promoverse de manera precisa al elegir una medida natural de *ganancia de información* basada en las tasas de codificación de los esquemas de codificación elegidos.[51] A lo largo de este libro, un enfoque de codificación tan explícito y constructivo ofrece un potente marco computacional para aprender buenas representaciones de estructuras de baja dimensión en datos del mundo real. En muchos casos de importancia práctica, la función de longitud de codificación puede calcularse de manera eficiente o aproximarse con precisión. En algunos casos bien comportados, incluso podemos obtener fórmulas en forma cerrada, por ejemplo, modelos de subespacio y gaussianos.

Además, este marco computacional da lugar a un enfoque basado en principios que revela en forma natural el papel que desempeñan las redes profundas en este proceso de aprendizaje. Como derivaremos sistemáticamente

[49] Cualquier código con estructura puede comprimirse aún más.

[50] Por ejemplo, para obtener muestras de la distribución bajo diferentes condiciones.

[51] La ganancia de información es un concepto muy extendido en la teoría algorítmica de la información y se ha incorporado en varios enfoques de este tema, siendo la más reciente la teoría de la información $\mathcal{V}$ [XZS+20]. Esta instanciación concreta, al igual que muchas otras medidas de complejidad que hemos comentado, sufre de un **condicionamiento cada vez peor** a medida que el soporte de los datos se vuelve más compacto y de baja dimensión, por lo que no resulta adecuada para nuestras necesidades.

en el Capítulo 5, las capas de una red profunda realizan operaciones que optimizan en forma incremental la función objetivo de interés. Desde esta perspectiva, el papel de las redes profundas puede interpretarse precisamente como la emulación de un determinado algoritmo de optimización iterativa —por ejemplo, el descenso por gradiente—para optimizar el objetivo de ganancia de información. Las capas de las arquitecturas profundas resultantes pueden dotarse de interpretaciones estadísticas y geométricas precisas, es decir, realizar operaciones incrementales de codificación y decodificación compresivas. En consecuencia, las redes profundas derivadas se convierten en «cajas blancas» transparentes que son totalmente explicables desde el punto de vista matemático.

1.4.3 Aprendizaje de representaciones consistentes

Para resumir lo que hemos comentado hasta ahora, denotemos los datos como

$$\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_i, \ldots, \boldsymbol{x}_N\} \subset \mathbb{R}^D, \tag{1.4.10}$$

y sea $\boldsymbol{Z} = \mathcal{E}(\boldsymbol{X})$ el código de $\boldsymbol{X}$ mediante algún codificador $\mathcal{E}$:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z}. \tag{1.4.11}$$

En el contexto del aprendizaje automático, a $\boldsymbol{Z}$ se le suele llamar las «características» de $\boldsymbol{X}$. Nótese que sin conocer la distribución subyacente de $\boldsymbol{X}$, no sabemos qué codificador $\mathcal{E}$ se debe utilizar para conservar la información más útil sobre la distribución de $\boldsymbol{X}$. En la práctica, la gente suele empezar probando un esquema de codificación compacto que se adapte bien a una tarea concreta. En concreto, se intenta entrenar un codificador que optimice una determinada medida (empírica) de parsimonia para la representación aprendida:

$$\min \rho(\boldsymbol{Z}). \tag{1.4.12}$$

Ejemplo 1.2. Por ejemplo, la clasificación de imágenes es un caso de este tipo: asignamos a todas las imágenes de una misma clase un único código y a las imágenes de diferentes clases códigos distintos, es decir, vectores «one-hot»:

$$\boldsymbol{x} \mapsto \boldsymbol{z} \in \{[1,0,0,\ldots,0,0],\ [0,1,0\ldots,0,0],\ \ldots,\ [0,0,0,\ldots,0,1].\} \tag{1.4.13}$$

Ahora bien, un clasificador $f(\cdot)$ puede modelarse como una función que predice la probabilidad de pertenencia de un $\boldsymbol{x}$ dado a cada una de las k clases: $\hat{\boldsymbol{z}} = f(\boldsymbol{x}) \in \mathbb{R}^K$. Entonces, la «calidad» de un clasificador se puede medir mediante la denominada *entropía cruzada*:[52]

$$\ell(\hat{\boldsymbol{z}}, \boldsymbol{z}) = \sum_{k=1}^{K} -z_k \log \hat{z}_k, \tag{1.4.14}$$

[52]La entropía cruzada puede considerarse como una medida de distancia entre la distribución real de $\boldsymbol{z}$ y la de la predicción $\hat{\boldsymbol{z}}$. También puede considerarse como la longitud de codificación esperada de $\boldsymbol{z}$ si utilizamos el libro de códigos óptimo para $\hat{\boldsymbol{z}}$ a fin de codificar $\boldsymbol{z}$. La entropía cruzada alcanza su mínimo cuando $\boldsymbol{z}$ y $\hat{\boldsymbol{z}}$ tienen la misma distribución.

donde z_k indica la k-ésima entrada del vector $\boldsymbol{z}$. Como indicaron los primeros trabajos sobre redes profundas [KSH12], si se dispone de suficientes datos, un esquema de codificación de este tipo a menudo puede representarse mediante una red profunda y aprenderse en forma integral mediante la optimización de la entropía cruzada. ■

La pérdida de entropía cruzada $\ell(\hat{\boldsymbol{z}}, \boldsymbol{z})$ puede considerarse como una medida especial de parsimonia $\rho(\boldsymbol{z})$ asociada con una familia particular de esquemas de codificación que son apropiados para la clasificación. Sin embargo, una codificación de este tipo obviamente tiene *grandes pérdidas*. El $\boldsymbol{z}$ aprendido no contiene ninguna otra información acerca de $\boldsymbol{x}$ excepto su tipo de clase. Por ejemplo, al asignarle a una imagen (un código que representa) la clase «manzana», ya no sabemos qué tipo específico de manzana se representa en la imagen original con base sólo en esa etiqueta.

Por supuesto, el otro extremo es exigir que el esquema de codificación sea *sin pérdida*. Es decir, que exista una correspondencia biunívoca entre $\boldsymbol{x}$ y su código $\boldsymbol{z}$. Sin embargo, como veremos en Capítulo 4, la codificación sin pérdida (o compresión) resulta sre impráctica, a menos que $\boldsymbol{x}$ sea discreta. En el caso de una variable aleatoria continua, solo podemos considerar esquemas de codificación con pérdida, de modo que la longitud de codificación de los datos sea finita. Es decir, solo codificamos los datos hasta una precisión predeterminada. Como desarrollaremos con más detalle en Capítulo 4, la codificación con pérdida no es solo una opción práctica, sino que desempeña un papel fundamental a la hora de hacer posible el aprendizaje de la distribución continua subyacente a partir de muestras finitas de dicha distribución.

Para muchos fines de aprendizaje, queremos que la característica $\boldsymbol{z}$, aunque *con pérdidas*, conserve más información sobre $\boldsymbol{x}$ que simplemente su tipo de clase. En este libro, presentaremos una medida de parsimonia más general basada en la longitud o tasa de codificación asociada a una familia más general de esquemas de codificación: la codificación con una mezcla de subespacios o gaussianas. Esta familia tiene la capacidad de aproximar con gran exactitud distribuciones arbitrarias del mundo real hasta un cierto grado de precisión. Como veremos en el Capítulo 4 y Capítulo 5, dicha medida no solo conservará la mayor parte de la información sobre la distribución de $\boldsymbol{X}$, sino que también favorecerá ciertas estructuras geométricas y estadísticas deseables para la representación aprendida $\boldsymbol{Z}$.

Autocodificación bidireccional para la consistencia. En un contexto de aprendizaje más amplio, el objetivo principal de un esquema de codificación compresiva $\mathcal{E}$ es identificar las estructuras de baja dimensión en los datos $\boldsymbol{X}$ para utilizarlas en la realización de predicciones en el espacio de datos original. Esto requiere que el esquema de codificación aprendido $\mathcal{E}$ admita un esquema de decodificación eficiente, denotado por $\mathcal{D}$, que mapea la representación codificada $\boldsymbol{Z}$ de vuelta al espacio de datos:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z} \xrightarrow{\mathcal{D}} \hat{\boldsymbol{X}}. \tag{1.4.15}$$

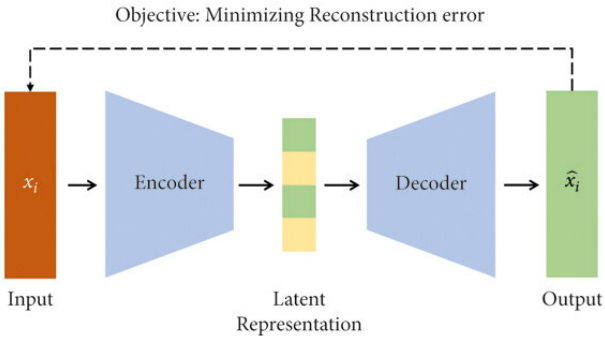


Figura 1.25: Ilustración de la arquitectura de un autocodificador.

A este par de codificación-decodificación $(\mathcal{E}, \mathcal{D})$ lo denominamos *autocodificación*. La Figura 1.25 ilustra este proceso.

Idealmente, la decodificación es aproximadamente una «inversa» de la codificación, de modo que los datos (la distribución) $\hat{\boldsymbol{X}}$ decodificados a partir de $\boldsymbol{Z}$ se asemejan a los datos originales (la distribución) $\boldsymbol{X}$ en cierta medida.[53] Si esto se cumple, podemos recuperar o predecir a partir de $\boldsymbol{Z}$ lo que ocurre en el espacio de datos original. En este caso, decimos que el par $(\mathcal{E}, \mathcal{D})$ proporciona una autocodificación *consistente*. Para la mayoría de los efectos prácticos, no solo necesitamos que exista tal decodificación, sino que también sea posible llevarla a cabo de manera eficiente y implementarla físicamente. Por ejemplo, si $\boldsymbol{x}$ es una variable de valor real, se requiere la cuantización para que cualquier esquema de decodificación sea realizable en una máquina de estados finitos, tal como explicaremos en el Capítulo 4. Por lo tanto, en general, los esquemas de codificación y decodificación viables conllevan necesariamente pérdidas. Una cuestión fundamental es cómo aprender una representación compacta (con pérdidas) $\boldsymbol{Z}$ que, aun así, pueda predecir bien $\boldsymbol{X}$.

En términos generales, tanto el codificador como el decodificador pueden modelarse y implementarse mediante redes profundas y aprenderse resolviendo un problema de optimización de la siguiente forma:

$$\min_{f,g}[\ell(\boldsymbol{X}, \hat{\boldsymbol{X}}) + \rho(\boldsymbol{Z})], \qquad \text{where} \quad \boldsymbol{Z} = f(\boldsymbol{X}), \quad \hat{\boldsymbol{X}} = g(\boldsymbol{Z}) \tag{1.4.16}$$

donde $\ell(\cdot, \cdot)$ es una función de distancia que promueve la similitud entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$[54] y $\rho(\boldsymbol{Z})$ es una medida que favorece la parsimonia y la riqueza de información de $\boldsymbol{Z}$. El análisis de componentes principales clásico (PCA) [Jol02] es un ejemplo típico de autocodificación consistente, que estudiaremos con gran detalle en el Capítulo 2, como precursor de estructuras de baja dimensión más generales. En Capítulo 6, estudiaremos cómo aprender una autocodificación consistente para distribuciones generales (por ejemplo, no lineales) de baja dimensión.

[53] Más adelante aclararemos qué entendemos por «similar».

[54] Ya sea a nivel de muestras o a nivel de distribución, dependiendo de la elección de ℓ.

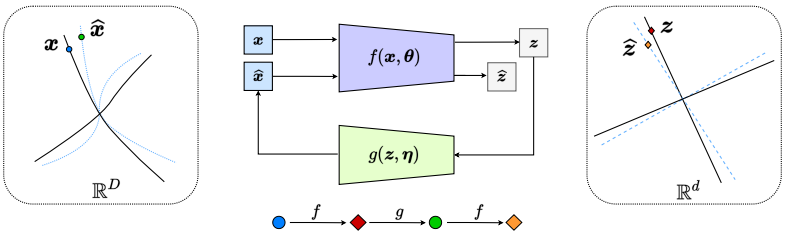


Figura 1.26: Ilustración de una transcripción en bucle cerrado. Aquí usamos un mapeo f para representar al codificador $\mathcal{E}$ y g para representar al decodificador $\mathcal{D}$.

1.4.4 Aprendizaje de representaciones autoconsistentes

En el objetivo de autocodificación anterior, hay que evaluar cuán cerca están los datos decodificados $\hat{\boldsymbol{X}}$ de los originales $\boldsymbol{X}$. Esto suele requerir una supervisión externa o el conocimiento de una medida de similitud adecuada. El cálculo de la similitud entre $\hat{\boldsymbol{X}}$ y $\boldsymbol{X}$ puede resultar prohibitivamente costoso, o incluso imposible. [55] En la naturaleza, los animales aprenden de manera autónoma sin comparar su estimación $\hat{\boldsymbol{X}}$ con los datos reales $\boldsymbol{X}$ en el espacio de datos; por lo general, carecen de esa opción.[56]

¿Cómo, entonces, puede un sistema aprender sin supervisión externa ni comparación? ¿Cómo puede saber que $\hat{\boldsymbol{X}}$ es consistente con $\boldsymbol{X}$ sin una comparación directa? Esto nos lleva a la idea de «cerrar el bucle». Bajo condiciones suaves (especificadas con precisión en el Capítulo 6), garantizar la consistencia entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ se reduce a codificar $\hat{\boldsymbol{X}}$ como $\hat{\boldsymbol{Z}}$ y verificar la consistencia entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$. A este concepto lo denominamos *autoconsistencia*, como se ilustra a continuación:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z} \xrightarrow{\mathcal{D}} \hat{\boldsymbol{X}} \xrightarrow{\mathcal{E}} \hat{\boldsymbol{Z}}, \tag{1.4.17}$$

A este proceso lo denominamos *transcripción en bucle cerrado*,[57] tal y como se muestra en la Figura 1.26.

Podría decirse que es cierto que cualquier ser inteligente autónomo solo necesita aprender una representación autoconsistente $\boldsymbol{Z}$ de los datos observados $\boldsymbol{X}$, ya que verificar la consistencia en el espacio de datos original —lo que a menudo significa en el mundo externo—resulta demasiado costoso o incluso físicamente inviable. La formulación en bucle cerrado permite aprender una codificación óptima $f(\cdot, \boldsymbol{\theta})$ y una decodificación $g(\cdot, \boldsymbol{\eta})$ mediante un juego

[55] Por ejemplo, minimizar la distancia entre distribuciones que existe entre las dos variables aleatorias resulta difícil tanto desde el punto de vista estadístico como computacional.

[56] Tanto en los animales como en los seres humanos, los ojos procesan la información visual mucho antes de que llegue al cerebro; el cerebro no cuenta con un mecanismo para procesar la información visual en bruto y debe basarse en lo que extraen los ojos para realizar algún aprendizaje.

[57] Inspirado en la transcripción entre el ADN y el ARN u otras proteínas.

min-max que depende únicamente de la característica interna (aprendida) $\boldsymbol{Z}$:

$$\max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} [\ell(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) + \rho(\boldsymbol{Z})], \qquad \text{where} \quad \boldsymbol{Z} = f(\boldsymbol{X}), \quad \hat{\boldsymbol{X}} = g(\boldsymbol{Z}), \quad \hat{\boldsymbol{Z}} = f(\hat{\boldsymbol{X}}) \tag{1.4.18}$$

donde $\ell(\boldsymbol{Z}, \hat{\boldsymbol{Z}})$ es una función de pérdida basada en las tasas de codificación de las características $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$, lo cual, como veremos, puede resultar mucho más fácil de calcular. Una vez más, $\rho(\boldsymbol{Z})$ es alguna medida que favorece la parsimonia y la riqueza de información de $\boldsymbol{Z}$. De manera algo sorprendente, como veremos en el Capítulo 6, bajo condiciones bastante moderadas, como que $\boldsymbol{X}$ sea de dimensión suficientemente baja, ¡la autoconsistencia entre $(\boldsymbol{Z}, \hat{\boldsymbol{Z}})$ implica consistencia en $(\boldsymbol{X}, \hat{\boldsymbol{X}})$! Además, también veremos que un sistema en bucle cerrado nos permitirá aprender la distribución de manera *continua e incremental*,[58] sin sufrir problemas como el olvido catastrófico asociado a los modelos de bucle abierto.

1.5 Tender un puente entre teoría y práctica en inteligencia artificial

Hasta ahora, hemos presentado varios marcos relacionados para aprender una representación compacta y estructurada $\boldsymbol{Z}$ de una distribución de datos dada $\boldsymbol{X}$:

- la codificación en bucle abierto (1.4.11);
- la autocodificación bidireccional (1.4.15);
- la transcripción en bucle cerrado (1.4.17).

En este libro, estudiaremos de manera sistemática los tres marcos, uno tras otro:

$$\textbf{circuito abierto} \implies \textbf{bidireccional} \implies \textbf{en bucle cerrado}, \tag{1.5.1}$$

en el Capítulo 5, la Sección 6.1 y la Sección 6.2, respectivamente.

Como veremos en este libro, estos marcos, incluidos muchos de los ya discutidos en este capítulo, pueden considerarse ahora como intentos de abordar *parcialmente* el problema de aprender una representación *completa y computable* de la distribución de los datos $\boldsymbol{X}$:

$$\boldsymbol{X} \longleftrightarrow \boldsymbol{Z} \longleftrightarrow \boldsymbol{G}, \tag{1.5.2}$$

en los que se incluyen dos conjuntos de mapeos:

1. una autocodificación entre los datos $\boldsymbol{X}$ y la representación $\boldsymbol{Z}$ de sus características informativas,
2. un proceso de difusión y eliminación de ruido entre la representación aprendida $\boldsymbol{Z}$ y una distribución simple $\boldsymbol{G}$, por ejemplo, gaussiana.

[58] Es decir, aprender una clase a la vez o incluso una muestra a la vez.

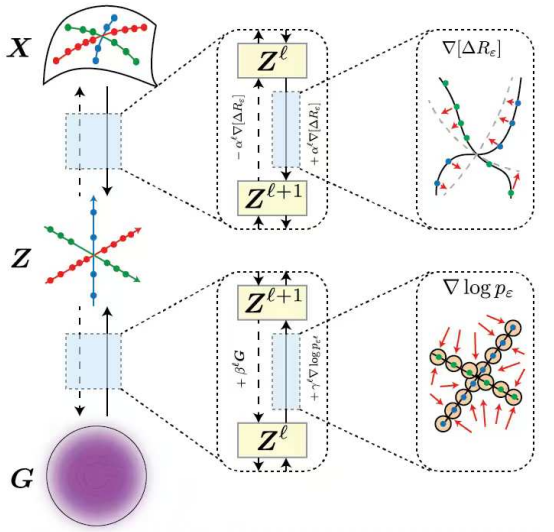


Figura 1.27: Ilustración de una representación aprendida, informativa y estructurada de características $\boldsymbol{Z}$ para los datos $\boldsymbol{X}$ a la que se puede acceder de manera eficiente por medio de un proceso de eliminación de ruido a partir de $\boldsymbol{G}$.

Este último proceso nos permite acceder de las características estructuradas de baja dimensión aprendidas $\boldsymbol{Z}$ y muestrearlas para tareas como la regeneración de los datos originales $\hat{\boldsymbol{X}}$.[59] El panorama geométrico general asociado a estos dos procesos (de transformación) se ilustra en la Figura 1.27.

Como veremos en el Capítulo 5, la función de las redes profundas modernas puede interpretarse precisamente como la realización de los operadores[60] necesarios para llevar a cabo las transformaciones anteriores entre distribuciones. Las arquitecturas populares de redes neuronales profundas (como las CNN, ResNet o Transformers) que se desarrollaron de manera empírica se interpretarán matemáticamente, se justificarán y se mejorarán mediante este enfoque.

En los últimos años, se han propuesto muchos marcos teóricos para ayudar a comprender las redes profundas. Sin embargo, la mayoría no ha conseguido ofrecer soluciones escalables que igualen el rendimiento de los métodos empíricos aplicados a datos y tareas del mundo real. Muchas teorías ofrecen poca orientación práctica para seguir mejorando empíricamente los resultados. El Capítulo 7 y el Capítulo 8 demostrarán cómo el marco presentado en este libro

[59] Nótese que, incluso si las características aprendidas se encuentran en un único subespacio lineal de baja dimensión, la forma más eficaz de identificar dicho subespacio es, en esencia, mediante un proceso de eliminación de ruido: el método de potencias que desarrollaremos en el 2.

[60] mediante optimización desenrollada, como veremos

puede salvar la brecha entre la teoría y la práctica. El Capítulo 7 mostrará cómo utilizar la distribución aprendida y su representación para realizar inferencias (bayesianas) en tareas prácticas relacionadas con la generación (condicional), la estimación y la predicción. El Capítulo 8 aportará pruebas experimentales convincentes de que las redes y los sistemas diseñados a partir de principios fundamentales pueden alcanzar un rendimiento competitivo —y potencialmente superior— en tareas como el aprendizaje de representaciones visuales, la clasificación de imágenes, su completado, su segmentación y la generación de texto.

De vuelta a la inteligencia. Como se mencionó al principio, una tarea fundamental de cualquier ser inteligente es adquirir información predecible a partir de los datos percibidos. Ahora comprendemos la naturaleza computacional de esta tarea y reconocemos que se trata de un proceso interminable por dos razones:

- Es poco probable que la memoria o el conocimiento adquirido a partir de datos —por ejemplo, mediante esquemas de codificación y decodificación— sea correcto u óptimo. La inteligencia debe ser capaz de perfeccionarse cuando las predicciones de nuevas observaciones contengan errores.
- Los datos observados aún no abarcan toda la información predecible. La inteligencia debe reconocer cuándo la memoria o el conocimiento actuales son inadecuados y adquirir nueva información cada vez que esté disponible.

Por lo tanto, la inteligencia *no* consiste en recopilar todos los datos informativos por adelantado y entrenar un modelo para que memorice su distribución (de baja dimensión). En cambio, requiere mecanismos computacionales capaces de mejorar continuamente el conocimiento actual de la distribución y de adquirir nueva información cuando sea necesario. Una característica fundamental de cualquier ser o sistema inteligente —ya sea un animal, un ser humano, una civilización o toda la comunidad científica— es *la capacidad de mejorar continuamente o de adquirir nueva información o conocimientos por sí mismo* a través de un bucle cerrado de retroalimentación sobre la eficacia con la que su memoria o sus conocimientos actuales logran explicar o predecir sus observaciones. En consecuencia, en cierta medida, podemos expresar simbólicamente la relación entre la inteligencia y el conocimiento[61] de la siguiente manera:

$$\begin{aligned} \text{Inteligencia}(t) &= \frac{\mathrm{d}}{\mathrm{d}t}\,\text{Conocimiento}(t), \\ \text{Conocimiento}(t) &= \int_0^t \text{Inteligencia}(s)\mathrm{d}s. \end{aligned}$$

[61] Aquí entendemos «conocimiento» en el sentido más amplio posible, incluyendo tanto la memoria empírica como el conocimiento científico.

Un marco de retroalimentación en bucle cerrado es un mecanismo universal para cualquier sistema capaz de superarse a sí mismo y de aprender por sí mismo —siendo el refuerzo una forma primitiva, como en la selección natural— o de competir, siendo la investigación científica su forma más avanzada a través de la formulación de hipótesis o conjeturas y la falsificación o verificación. Todos los seres y sistemas inteligentes de la naturaleza utilizan este tipo de mecanismos de bucle cerrado para aprender en todos los niveles y a todas las escalas. Esta omnipresencia inspiró los primeros intentos de modelar la inteligencia mediante máquinas autónomas u ordenadores semiautónomos, en particular el movimiento cibernético iniciado y promovido por Norbert Wiener en la década de 1940.

Esperamos que este libro ayude a los lectores a comprender mejor los objetivos científicos, los principios matemáticos y los mecanismos computacionales que subyacen a la inteligencia, al menos en lo que respecta a la formación de la memoria útil o el conocimiento empírico. Creemos que este trabajo no solo tiende un puente entre la teoría y la práctica, sino que también conecta el presente con el futuro: aclara lo que hemos hecho hasta ahora y, al mismo tiempo, revela lo que no hemos hecho. Por lo tanto, sienta las bases para el estudio futuro de posibles nuevos mecanismos a la base de la inteligencia de alto nivel propia del ser humano: la verdadera «inteligencia artificial» a la que se refería el programa de Dartmouth de 1956. Dichos mecanismos pueden crear conceptos abstractos y realizar inferencias deductivas que, en nuestra opinión, van más allá de la compresión de datos empíricos para la memoria, así como realizar inferencias inductivas que se estudian y explican en este libro y que se están aprovechando en la tecnología actual de la «IA». Analizaremos los problemas pendientes para comprender y, posiblemente, hacer realidad una inteligencia más avanzada en el Capítulo 9 final.

Capítulo 2

El aprendizaje de estructuras lineales e independientes

> «*El arte de hacer matemáticas consiste en encontrar ese caso particular que contiene todos los gérmenes de la generalidad.*»
>
> – David Hilbert

Los datos reales tienen una estructura de baja dimensión. Para entender por qué esto es así, consideremos el humilde caso del ruido estático en un televisor cuando el satélite no funciona. En cada fotograma (aproximadamente cada $\frac{1}{30}$ segundos), el ruido estático RGB en una pantalla de tamaño $H \times W$ se muestrea, a grandes rasgos, en forma independiente a partir de una distribución uniforme en $[0,1]^{3\times H\times W}$. En teoría, la imagen estática *podría* convertirse en una imagen natural en cualquier fotograma, pero aunque alguien pase mil años mirando la pantalla del televisor, eso no sucederá. Esta discrepancia se explica por el hecho de que el conjunto de $H \times W$ imágenes naturales ocupa una fracción infinitesimal del hipercubo $[0,1]^{3\times H\times W}$. En particular, es de dimensión extremadamente baja, en comparación con la dimensión del espacio que lo rodea. Se producen fenómenos similares con todos los demás tipos de datos naturales, como texto, audio o vídeo. Por lo tanto, cuando diseñamos sistemas y métodos para procesar datos naturales y aprender su estructura o distribución, esta es una propiedad fundamental de los datos naturales que debemos tener en cuenta.

Por lo tanto, nuestra tarea principal consiste en aprender una distribución que tenga baja dimensión intrínseca en un espacio de alta dimensión. En el resto de este capítulo, analizaremos varios métodos *clásicos* para llevar a cabo esta tarea en varios modelos algo *idealizados* de la distribución a saber, modelos geométricamente lineales o estadísticamente independientes. Aunque estos modelos y métodos son importantes y útiles por sí mismos, los analizamos

aquí porque motivan, inspiran y sirven como precursores o análogos de métodos más modernos para distribuciones más generales que recurren al aprendizaje profundo (de representaciones).

Nuestro enfoque principal (y la formulación general del problema) se puede resumir así:

Problema: *Dadas una o varias observaciones (con ruido o incompletas) de una verdad de referencia (la muestra verdadera) de la distribución de los datos reales, obtenga una estimación de dicha muestra.*

Este enfoque sustenta varios métodos clásicos de procesamiento de datos, que abordamos en este capítulo.

- Sección 2.1 — Análisis de componentes principales (PCA): Dadas unas muestras ruidosas procedentes de una distribución con soporte en *un subespacio de baja dimensión*, obtener una estimación de la muestra verdadera que se sitúa en este subespacio.
- Sección 2.2 — Aprendizaje de diccionario completo y análisis de componentes independientes (ICA): Dadas unas muestras ruidosas procedentes de una distribución definida en *una unión (que no es el espacio generado) de unos cuantos subespacios de baja dimensión*, obtener una estimación de las muestras verdaderas.
- Sección 2.3 — Codificación esparsa y aprendizaje de diccionarios: Dadas unas muestras ruidosas procedentes de *una distribución definida sobre combinaciones de unos pocos vectores incoherentes*, como por ejemplo los ejes de coordenadas, obtener una estimación de la muestra verdadera, que también tenga esa propiedad.

En este capítulo, tal y como se ha descrito anteriormente, formulamos supuestos simplificadores de modelado que, en esencia, suponen que (el soporte de) la distribución de los datos presenta estructuras geométricamente (casi, por tramos) lineales y componentes estadísticamente independientes, tal y como se ilustra en la Figura 2.1. En el Capítulo 1 nos hemos referido a este tipo de modelos de datos como «modelos analíticos». Estas hipótesis de modelado nos permiten derivar algoritmos eficientes con garantías demostrables de eficiencia, tanto en términos de datos como de complejidad computacional, para el procesamiento de datos a gran escala. Sin embargo, se trata de modelos imperfectos para las distribuciones de datos del mundo real, que suelen ser complejas, por lo que sus supuestos subyacentes solo se cumplen de manera aproximada. Esto significa que las garantías que ofrece el análisis detallado de estos algoritmos también solo se cumplen de manera aproximada en el caso de datos reales.

No obstante, las técnicas que se analizan en este capítulo son útiles por sí mismas y, además de eso, sirven como «caso especial con gérmenes de

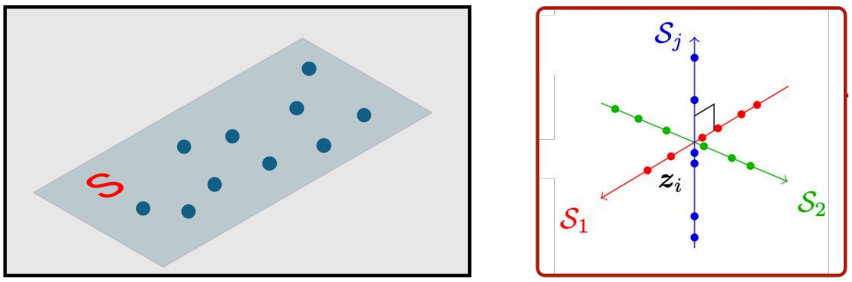


Figura 2.1: Datos sobre un único subespacio de baja dimensión (izquierda), por ejemplo, $\mathcal{S} = \text{col}(\boldsymbol{U})$ o una mezcla de subespacios de baja dimensión (derecha), por ejemplo, $\mathcal{S}_j = \text{col}(\boldsymbol{U}_j)$.

generalidad», por así decirlo, en el sentido de que ofrecen una motivación e intuición orientativas para los paradigmas más generales en el ámbito del aprendizaje (profundo) de distribuciones más generales que abordaremos más adelante. Como veremos en capítulos posteriores, en la era del aprendizaje profundo, los enfoques modernos «basados en datos» o «no paramétricos» para aprender las distribuciones de datos (de baja dimensión) adoptan, en esencia, los mismos conceptos e ideas para el aprendizaje.

Una nota sobre la notación. Recomendamos al lector que consulte el capítulo Notación, en particular la sección «Notación de modelado y optimización», al comenzar a leer este capítulo. A lo largo del libro empleamos las mismas convenciones de notación para el modelado de datos y la optimización de los parámetros de estos modelos. Los modelos sencillos y explícitos que estudiamos en este capítulo ayudan a aclarar el contenido «semántico» de estas convenciones.

2.1 Subespacio de baja dimensión

2.1.1 Análisis de componentes principales (PCA)

Quizás la hipótesis de modelado más simple posible para una estructura de baja dimensión sea la denominada hipótesis de «bajo rango». Sea D la dimensión de nuestro espacio de datos; supongamos que nuestros datos pertenecen a un subespacio de baja dimensión $d \ll D$, posiblemente con algunas pequeñas perturbaciones.[1] Esta suposición resulta ser prácticamente válida para algunos datos sorprendentemente complejos, como las imágenes de dígitos escritos a

[1] Además, posiblemente, un desplazamiento no trivial con respecto a $\mathbf{0}$; en la práctica, tenemos en cuenta este desplazamiento restando la media empírica de las muestras de datos (calculada sobre las muestras de datos) a cada muestra y en el siguiente capítulo ya no le prestamos más atención.

Figura 2.2: Imágenes de rostros humanos alineados y números escritos a mano. A pesar de la aparente gran variedad de sus formas, cada conjunto de estos datos genera (aproximadamente) un subespacio (casi) lineal de muy baja dimensión. Esto se puede expresar con precisión técnica en la notación de (2.1.1): para una dimensión de subespacio relativamente pequeña d (digamos, ≈ 20, en comparación con la dimensión $D = 784$ de los dígitos de MNIST, los vectores de error $\boldsymbol{\varepsilon}_i$ son pequeños para cada $i \in [N]$ (donde $N = 6 \cdot 10^4$ para MNIST es el tamaño del conjunto de datos). Podemos *aprender* el subespacio óptimo de este tipo a partir de los datos.

mano y los datos faciales [RD03], tal como se muestra en la Figura 2.2, pero, como veremos, se prestará exepcionalmente bien a un análisis exhaustivo.

Formulación del problema. Para expresarlo en notación matemática, representamos un subespacio $\mathcal{S} \subseteq \mathbb{R}^D$ de dimensión d mediante una matriz ortonormal $\boldsymbol{U} \in \mathsf{O}(D, d) \subseteq \mathbb{R}^{D \times d}$ tal que las columnas de $\boldsymbol{U}$ generan $\mathcal{S}$. Luego, decimos que nuestros datos $\{\boldsymbol{x}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ tienen una estructura (aproximada) de bajo rango si existen una matriz ortonormal $\boldsymbol{U} \in \mathsf{O}(D, d)$, vectores $\{\boldsymbol{z}_i\}_{i=1}^N \subseteq \mathbb{R}^d$ y *pequeños* vectores $\{\boldsymbol{\varepsilon}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ tales que

$$\boldsymbol{x}_i = \boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i, \quad \forall i \in [N]. \tag{2.1.1}$$

Aquí, $\boldsymbol{\varepsilon}_i$ son perturbaciones que impiden que los datos sean perfectamente de bajo rango; su presencia en nuestro modelo nos permite cuantificar hasta qué punto nuestro análisis sigue siendo relevante cuando se producen desviaciones respecto a nuestro modelo. El verdadero subespacio de soporte es $\mathcal{S} \doteq \operatorname{col}(\boldsymbol{U})$, el espacio generado por las columnas de $\boldsymbol{U}$. Para procesar toda la información posible a partir de estos datos, necesitamos recuperar $\mathcal{S}$; para ello basta con recuperar $\boldsymbol{U}$, también denominadas *componentes principales*. Afortunadamente, se trata de una tarea computacionalmente viable denominada **Análisis de Componentes Principales** (PCA por su sigla en inglés) y ahora veremos cómo resolverla.

Dados los datos $\{\boldsymbol{x}_i\}_{i=1}^N$ distribuidos como en (2.1.1), nuestro objetivo es recuperar el modelo $\boldsymbol{U}$. Un enfoque natural consiste en encontrar el subespacio

$\boldsymbol{U}^\star$ y los vectores latentes $\{\boldsymbol{z}_i^\star\}_{i=1}^N$ que proporcionen la mejor aproximación $\boldsymbol{x}_i \approx \boldsymbol{U}^\star \boldsymbol{z}_i^\star$. Es decir, nuestro objetivo es resolver el problema

$$\min_{\tilde{\boldsymbol{U}}, \{\tilde{\boldsymbol{z}}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2, \tag{2.1.2}$$

donde $\tilde{\boldsymbol{U}}$ está restringida a ser una matriz ortonormal, tal como se ha indicado anteriormente. Omitiremos esta restricción en las exposiciones similares que siguen, en aras de la brevedad.

Codificación y decodificación en subespacios mediante eliminación de ruido. Para simplificar este problema, para un valor fijo de $\tilde{\boldsymbol{U}}$, tenemos (la demostración como ejercicio):

$$\min_{\{\tilde{\boldsymbol{z}}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2 = \frac{1}{N} \sum_{i=1}^N \min_{\tilde{\boldsymbol{z}}_i} \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2 \tag{2.1.3}$$

$$= \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{U}}^\top \boldsymbol{x}_i\|_2^2. \tag{2.1.4}$$

Es decir, la solución óptima $(\boldsymbol{U}^\star, \{\boldsymbol{z}_i^\star\}_{i=1}^N)$ del problema de optimización anterior cumple que $\boldsymbol{z}_i^\star = (\boldsymbol{U}^\star)^\top \boldsymbol{x}_i$.

Ahora, podemos reformular el problema de optimización original en términos de $\tilde{\boldsymbol{U}}$ y $\{\tilde{\boldsymbol{z}}_i\}_{i=1}^N$ como un problema de optimización que se define únicamente sobre $\tilde{\boldsymbol{U}}$, es decir, para obtener la base $\boldsymbol{U}^\star$ y los códigos compactos $\{\boldsymbol{z}_i^\star\}_{i=1}^N$ basta con resolver cualquiera de los siguientes dos problemas equivalentes:

$$\min_{\tilde{\boldsymbol{U}}, \{\tilde{\boldsymbol{z}}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2 = \min_{\tilde{\boldsymbol{U}}} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{U}}^\top \boldsymbol{x}_i\|_2^2. \tag{2.1.5}$$

Téngase en cuenta que el problema que aparece en el miembro derecho de (2.1.5) es un problema de *eliminación de ruido*: dadas observaciones ruidosas $\boldsymbol{x}_i$ de datos de bajo rango, nuestro objetivo es encontrar la copia *sin ruido* de $\boldsymbol{x}_i$, que bajo el modelo (2.1.1) es $\boldsymbol{U}\boldsymbol{z}_i$. Es decir, la entrada sin ruido $\hat{\boldsymbol{x}}_i = \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{x}_i$. Nótese que este es el punto del subespacio que se encuentra más cerca de $\boldsymbol{x}_i$, tal como se muestra en la Figura 2.3. Aquí, al resolver el problema equivalente de encontrar el mejor subespacio, parametrizado por la base aprendida $\boldsymbol{U}^\star$, aprendemos una aproximación al *denoiser*, es decir, la matriz de proyección $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top \approx \boldsymbol{U}\boldsymbol{U}^\top$ que proyecta el punto de datos ruidoso $\boldsymbol{x}_i$ sobre el subespacio $\mathcal{S}$.

Al reunir el proceso anterior, obtenemos básicamente un sencillo esquema de codificación-decodificación que mapea un punto de datos $\boldsymbol{x}$ en $\mathbb{R}^D$ a un espacio (latente) de menor dimensión $\mathbb{R}^d$ y, a continuación, de vuelta a $\mathbb{R}^D$:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}=(\boldsymbol{U}^\star)^\top} \boldsymbol{z} \xrightarrow{\mathcal{D}=\boldsymbol{U}^\star} \hat{\boldsymbol{x}}. \tag{2.1.6}$$

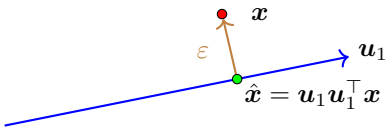


Figura 2.3: **La geometría de PCA.** Un punto de datos $\boldsymbol{x}$ (en rojo) se proyecta sobre el subespacio unidimensional aprendido, generado por el vector base unitario $\boldsymbol{u}_1$ (flecha azul). La proyección $\boldsymbol{U}\boldsymbol{U}^\top\boldsymbol{x} = \boldsymbol{u}_1\boldsymbol{u}_1^\top\boldsymbol{x}$ (en verde) es la versión sin ruido de $\boldsymbol{x}$ utilizando la estructura de baja dimensión dada por $\boldsymbol{u}_1$ y $\boldsymbol{\varepsilon}$ (flecha marrón) representa el residuo de la proyección, es decir, el ruido.

En este contexto, $\boldsymbol{z} \in \mathbb{R}^d$ se puede considerar como el código compacto de baja dimensión (o una representación latente) de un punto de datos $\boldsymbol{x} \in \mathbb{R}^D$, mientras que la base del subespacio aprendido $\boldsymbol{U}^\star$ se puede entender como el libro de códigos asociado, cuyas columnas son las palabras de código óptimas (aprendidas). El proceso logra eliminar el ruido de $\boldsymbol{x}$ mediante su proyección sobre el subespacio generado por $\boldsymbol{U}^\star$.

Cálculo de la base del subespacio. Por ahora, continuamos con nuestro cálculo. Sea $\boldsymbol{X} = \left[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\right] \in \mathbb{R}^{D\times N}$ la matriz cuyas columnas son las observaciones $\boldsymbol{x}_i$. Tenemos (demostración como ejercicio)

$$\arg\min_{\tilde{\boldsymbol{U}}} \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top\boldsymbol{x}_i\|_2^2 = \arg\max_{\tilde{\boldsymbol{U}}} \frac{1}{N}\sum_{i=1}^{N} \|\tilde{\boldsymbol{U}}^\top\boldsymbol{x}_i\|_2^2 \tag{2.1.7}$$

$$= \arg\max_{\tilde{\boldsymbol{U}}} \operatorname{tr}\left\{\tilde{\boldsymbol{U}}^\top\left(\frac{\boldsymbol{X}\boldsymbol{X}^\top}{N}\right)\tilde{\boldsymbol{U}}\right\}. \tag{2.1.8}$$

Por lo tanto, para calcular las componentes principales, se busca la matriz ortonormal $\tilde{\boldsymbol{U}}$ que maximice el término $\operatorname{tr}(\tilde{\boldsymbol{U}}^\top(\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{U}})$. Podemos demostrar por inducción que esta matriz $\boldsymbol{U}^\star$ tiene columnas que son los *primeros d vectores propios unitarios* de $\boldsymbol{X}\boldsymbol{X}^\top/N$. Dejamos toda la demostración al lector en el Ejercicio 2.1, pero aquí tratamos el caso base de la inducción. Supongamos que $d = 1$. Entonces solo tenemos que recuperar un único vector unitario $\boldsymbol{u}$, por lo que el problema anterior se reduce a

$$\max_{\tilde{\boldsymbol{u}}:\ \|\tilde{\boldsymbol{u}}\|_2=1} \tilde{\boldsymbol{u}}^\top(\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{u}}. \tag{2.1.9}$$

Este es el llamado *cociente de Rayleigh* de $\boldsymbol{X}\boldsymbol{X}^\top/N$. Aplicando el teorema espectral, diagonalizamos $\boldsymbol{X}\boldsymbol{X}^\top/N = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top$, donde $\boldsymbol{Q}$ es ortogonal y $\boldsymbol{\Lambda}$ es diagonal con entradas no negativas. Por lo tanto

$$\tilde{\boldsymbol{u}}^\top(\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{u}} = \tilde{\boldsymbol{u}}^\top\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top\tilde{\boldsymbol{u}} = (\boldsymbol{Q}^\top\tilde{\boldsymbol{u}})^\top\boldsymbol{\Lambda}(\boldsymbol{Q}^\top\tilde{\boldsymbol{u}}). \tag{2.1.10}$$

Dado que $\boldsymbol{Q}$ es una transformación ortogonal invertible, $\boldsymbol{Q}^\top\tilde{\boldsymbol{u}}$ es un vector unitario y optimizar con respecto a $\tilde{\boldsymbol{u}}$ equivale a optimizar con respecto a $\tilde{\boldsymbol{w}} \doteq$

$\boldsymbol{Q}^\top \tilde{\boldsymbol{u}}$. Por lo tanto, podemos escribir

$$\tilde{\boldsymbol{u}}^\top (\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{u}} = \tilde{\boldsymbol{w}}^\top \boldsymbol{\Lambda}\tilde{\boldsymbol{w}}, \tag{2.1.11}$$

cuyas soluciones óptimas $\boldsymbol{w}^\star$ entre los vectores unitarios son vectores «one-hot» cuya única entrada distinta de cero (y, por lo tanto, unitaria) se encuentra en uno de los índices correspondientes al mayor valor propio de $\boldsymbol{X}\boldsymbol{X}^\top/N$. Esto significa que $\boldsymbol{u}^\star = \boldsymbol{Q}\boldsymbol{w}^\star$, la solución óptima del problema original, corresponde a un vector propio unitario de $\boldsymbol{X}\boldsymbol{X}^\top/N$ (es decir, una columna de $\boldsymbol{Q}$) que corresponde al mayor valor propio. Generalizando adecuadamente esto al caso $d > 1$ y resumiendo todo lo expuesto anteriormente, obtenemos el siguiente teorema informal.

Teorema 2.1. *Supongamos que nuestro conjunto de datos $\{\boldsymbol{x}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ se puede escribir como*

$$\boldsymbol{x}_i = \boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i, \qquad \forall i \in [N], \tag{2.1.12}$$

donde $\boldsymbol{U} \in \mathsf{O}(D,d)$ representa la estructura de bajo rango, $\{\boldsymbol{z}_i\}_{i=1}^N \subseteq \mathbb{R}^d$ son los códigos compactos de los datos y $\{\boldsymbol{\varepsilon}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ son pequeños vectores que indican la desviación de nuestros datos respecto al modelo de bajo rango. Entonces, las componentes principales $\boldsymbol{U}^\star \in \mathsf{O}(D,d)$ de nuestro conjunto de datos vienen dadas por los d primeros vectores propios de $\boldsymbol{X}\boldsymbol{X}^\top/N$, donde $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}$ y corresponden aproximadamente al denoiser lineal óptimo: $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top \approx \boldsymbol{U}\boldsymbol{U}^\top$.

Aquí no indicamos tasas explícitas de aproximación, ya que pueden resultar bastante técnicas. En el caso especial en que $\boldsymbol{\varepsilon}_i = \boldsymbol{0}$ para todo i, el $\boldsymbol{U}^\star$ aprendido abarca el soporte de las muestras $\{\boldsymbol{x}_i\}_{i=1}^N$. Si, además, los $\boldsymbol{z}_i$ son lo suficientemente diversos (por ejemplo, abarcan todo el espacio $\mathbb{R}^d$), entonces tendríamos una recuperación perfecta: $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top = \boldsymbol{U}\boldsymbol{U}^\top$.

Nota 2.1. En algunas tareas de análisis de datos, la matriz de datos $\boldsymbol{X}$ se organiza de tal manera que cada punto de datos es una *fila* en lugar de una *columna*, como se muestra aquí. En este caso, las componentes principales son los d primeros vectores propios de $\boldsymbol{X}^\top\boldsymbol{X}/N$.

Nota 2.2 (Selección de base mediante valores propios de la eliminación de ruido). En muchos casos, nuestros datos no estarán distribuidos realmente según un modelo de subespacio más ruido, o bien no conoceremos la verdadera dimensión subyacente d del subespacio. En este caso, tenemos que elegir d; este problema se denomina *selección de modelos*. En el caso restringido del PCA, una forma de realizar la selección de modelos consiste en calcular $\boldsymbol{X}\boldsymbol{X}^\top/N$ y buscar casos en los que los valores propios adyacentes disminuyan bruscamente; esto es un indicador de que el índice del valor propio más grande es la «dimensión verdadera d» el resto de los valores propios de $\boldsymbol{X}\boldsymbol{X}^\top/N$ se deben al ruido o a las perturbaciones $\boldsymbol{\varepsilon}_i$. La selección de modelos es un problema difícil y, hoy en día, en la era del aprendizaje profundo, donde se conoce como «optimización de hiperparámetros», suele realizarse mediante el método de fuerza bruta o la optimización bayesiana, a menudo utilizando la pérdida en un conjunto de datos de «validación» como objetivo.

Nota 2.3 (Eliminación de ruido en muestras). La expresión del miembro derecho de (2.1.5), es decir,

$$\min_{\tilde{\boldsymbol{U}}} \frac{1}{N} \sum_{i=1}^{N} \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top \boldsymbol{x}_i\|_2^2, \tag{2.1.13}$$

es lo que se conoce como un *problema de eliminación de ruido*, llamado así porque se trata de un problema de optimización cuya solución *elimina el ruido de las muestras para que estas se ajusten al subespacio*. La eliminación de ruido —el aprendizaje de un mapa que elimina el ruido de muestras ruidosas para que se ajuste a la estructura de datos (como en (2.1.1), aunque quizá en forma más compleja)— es un método habitual para el aprendizaje de distribuciones que se analizará en lo que sigue y a lo largo de todo el manuscrito. Nótese que ya hemos tratado este concepto, pero vale la pena repetirlo debido a su gran importancia en los capítulos siguientes.

Nota 2.4 (Interpretación como red neuronal). Si realizamos un análisis de componentes principales (PCA), recuperamos aproximadamente el soporte de la distribución codificada por el parámetro $\boldsymbol{U}^\star$. El mapa de eliminación de ruido aprendido adopta entonces la forma $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top$. Además de ser un *denoiser*, esto puede considerarse como una *red neuronal lineal simple de dos capas con pesos vinculados* RN: la primera capa multiplica por $(\boldsymbol{U}^\star)^\top$ y la segunda multiplica por $\boldsymbol{U}^\star$, es decir

$$\text{reduce}_{\text{r}}\text{uido}(\boldsymbol{x}) = \underbrace{\boldsymbol{U}^\star \circ \underbrace{\text{id} \circ \underbrace{\underbrace{(\boldsymbol{U}^\star)^\top \boldsymbol{x}}_{\text{primera “capa”}}}_{\text{post-activación de la primera “capa”}}}_{\text{salida de la “RN”}}}. \tag{2.1.14}$$

Si lo comparamos con una red neuronal estándar de dos capas RN, observamos una similitud estructural:

$$\text{RN}(\boldsymbol{x}) = \underbrace{\boldsymbol{W}^\star \circ \underbrace{\text{ReLU} \circ \underbrace{\underbrace{(\boldsymbol{U}^\star)^\top \boldsymbol{x}}_{\text{primera capa}}}_{\text{post-activación de la primera capa}}}_{\text{salida de la RN}}}. \tag{2.1.15}$$

En concreto, el PCA puede interpretarse como *el aprendizaje de un autocodificador simple de dos capas de eliminación de ruido*,[2] uno de los ejemplos más simples de una red neuronal no trivial. En este marco, las **representaciones aprendidas** serían simplemente $(\boldsymbol{U}^\star)^\top \boldsymbol{x} \approx \boldsymbol{z}$. De este modo, el PCA sirve como problema modelo para el aprendizaje de representaciones (profundas), al que recurriremos más adelante en la monografía. Obsérvese que, en esta analogía, las representaciones reflejan —o son proyecciones de— los datos de entrada hacia una estructura aprendida de baja dimensión. Esta propiedad será especialmente relevante en el futuro.

[2] De hecho, como hemos mencionado en el capítulo anterior, el PCA fue uno de los primeros problemas en el que se utilizaron redes neuronales para resolverlo [BH89; Oja82].

2.1.2 Búsqueda de estructuras de rango bajo por el método de las potencias

Existe una forma computacionalmente eficiente de estimar los vectores propios principales (asociados a los autovalores más grandes) de $\boldsymbol{X}\boldsymbol{X}^\top/N$ o de cualquier matriz simétrica semidefinida positiva $\boldsymbol{M}$, denominada *método de las potencias*. Este método constituye la base de varios enfoques algorítmicos para el análisis de datos de alta dimensión que trataremos más adelante en este capítulo, por lo que lo abordamos aquí.

Sea $\boldsymbol{M}$ una matriz simétrica semidefinida positiva. Existe una base ortonormal para $\mathbb{R}^D$ formada por los vectores propios $(\boldsymbol{w}_i)_{i=1}^D$ de $\boldsymbol{M}$, con los valores propios correspondientes $\lambda_1 \geq \cdots \geq \lambda_D \geq 0$. Por definición, cualquier vector propio $\boldsymbol{w}_i$ cumple que $\lambda_i \boldsymbol{w}_i = \boldsymbol{M}\boldsymbol{w}_i$. Por lo tanto, para cualquier $\lambda_i > 0$, $\boldsymbol{w}_i$ es un «punto fijo» de la siguiente ecuación:

$$\boldsymbol{w} = \frac{\boldsymbol{M}\boldsymbol{w}}{\|\boldsymbol{M}\boldsymbol{w}\|_2}. \tag{2.1.16}$$

Teorema 2.2 (Método de las potencias)**.** *Supongamos que $\lambda_1 > \lambda_i$ para todo $i > 1$. Si calculamos el punto fijo de (2.1.16) utilizando la siguiente iteración:*

$$\boldsymbol{v}_0 \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{1}), \qquad \boldsymbol{v}_{t+1} \leftarrow \frac{\boldsymbol{M}\boldsymbol{v}_t}{\|\boldsymbol{M}\boldsymbol{v}_t\|_2}, \tag{2.1.17}$$

entonces, en el límite, $\boldsymbol{v}_t$ convergerá al vector propio unitario de $\boldsymbol{M}$ asociado al mayor de los valores propios.

Demostración. En primer lugar, nótese que, para todo t, tenemos

$$\boldsymbol{v}_t = \frac{\boldsymbol{M}\boldsymbol{v}_{t-1}}{\|\boldsymbol{M}\boldsymbol{v}_{t-1}\|_2} = \frac{\boldsymbol{M}^2\boldsymbol{v}_{t-2}}{\|\boldsymbol{M}\boldsymbol{v}_{t-1}\|_2\|\boldsymbol{M}\boldsymbol{v}_{t-2}\|_2} = \cdots = \frac{\boldsymbol{M}^t\boldsymbol{v}_0}{\prod_{s=1}^t \|\boldsymbol{M}\boldsymbol{v}_s\|_2}. \tag{2.1.18}$$

Por lo tanto, $\boldsymbol{v}_t$ tiene la misma dirección que $\boldsymbol{M}^t\boldsymbol{v}_0$ y es de norma unitaria, por lo que podemos escribir

$$\boldsymbol{v}_t = \frac{\boldsymbol{M}^t\boldsymbol{v}_0}{\|\boldsymbol{M}^t\boldsymbol{v}_0\|_2}. \tag{2.1.19}$$

Dado que todos los vectores propios $\boldsymbol{w}_i$ de $\boldsymbol{M}$ forman una base ortonormal para $\mathbb{R}^D$, podemos escribir

$$\boldsymbol{v}_0 = \sum_{i=1}^D \alpha_i \boldsymbol{w}_i, \tag{2.1.20}$$

donde, dado que $\boldsymbol{v}_0$ es gaussiano, los α_i son todos distintos de cero con probabilidad 1. Por lo tanto, podemos utilizar nuestra expresión anterior para $\boldsymbol{v}_t$ y escribir

$$\boldsymbol{v}_t = \frac{\boldsymbol{M}^t\boldsymbol{v}_0}{\|\boldsymbol{M}^t\boldsymbol{v}_0\|_2} = \frac{\sum_{i=1}^D \lambda_i^t \alpha_i \boldsymbol{w}_i}{\|\sum_{i=1}^D \lambda_i^t \alpha_i \boldsymbol{w}_i\|_2} = \frac{\sum_{i=1}^D \lambda_i^t \alpha_i \boldsymbol{w}_i}{\sqrt{\sum_{i=1}^D \lambda_i^{2t} \alpha_i^2}}. \tag{2.1.21}$$

Ahora, consideremos el caso en el que $\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_D \geq 0$. (El caso en el que se repiten los valores propios principales se resuelve de manera similar.) Entonces podemos escribir

$$\boldsymbol{v}_t = \frac{\alpha_1 \boldsymbol{w}_1 + \sum_{i=2}^{D} (\lambda_i/\lambda_1)^t \alpha_i \boldsymbol{w}_i}{\sqrt{\alpha_1^2 + \sum_{i=2}^{D} (\lambda_i/\lambda_1)^{2t} \alpha_i^2}}. \tag{2.1.22}$$

Dado que $\lambda_1 > \lambda_i$ para todo $i > 1$, los términos dentro de la suma tienden exponencialmente a 0 y el resultado es el límite

$$\lim_{t \to \infty} \boldsymbol{v}_t = \frac{\alpha_1}{|\alpha_1|} \boldsymbol{w}_1 = \text{sign}(\alpha_1) \boldsymbol{w}_1, \tag{2.1.23}$$

que es el vector propio unitario asociado al valor propio principal de $\boldsymbol{M}$. El valor propio principal λ_1 de $\boldsymbol{M}$ puede estimarse mediante $\boldsymbol{v}_t^\top \boldsymbol{M} \boldsymbol{v}_t$, que converge a λ_1 a una velocidad comparable. □

Nota 2.5 (El método de las potencias como proceso de eliminación de ruido). Nótese que el cálculo de la componente principal mediante el método de las potencias puede interpretarse como un proceso de «eliminación de ruido»: partiendo de un ruido aleatorio, converge iterativamente hacia un subespacio unidimensional. De hecho, como mostraremos más adelante en el Capítulo 3, si suponemos que *el soporte de la distribución de los datos es un único subespacio*, el operador de eliminación de ruido asociado (o la función de *score*) es exactamente el mismo que la iteración de potencias anterior. En el Capítulo 3, mostraremos cómo generalizar ese proceso de eliminación de ruido a distribuciones de estructuras de baja dimensión más generales. Se trata, en esencia, de un enfoque universal y muy eficaz para aprender cualquier distribución de baja dimensión.

Para hallar el segundo vector propio principal, aplicamos el algoritmo de las potencias a $\boldsymbol{M} - \lambda_1 \boldsymbol{v}_1 \boldsymbol{v}_1^\top$, que tiene vectores propios $(\boldsymbol{w}_i)_{i=2}^D$ y los valores propios correspondientes $(\lambda_i)_{i=2}^D$. Repitiendo este procedimiento d veces seguidas, podemos estimar de manera muy eficiente los d vectores propios principales de $\boldsymbol{M}$ para cualquier matriz simétrica semidefinida positiva $\boldsymbol{M}$. Por lo tanto, también podemos aplicarlo a $\boldsymbol{X}\boldsymbol{X}^\top/N$ para recuperar las primeras d componentes principales, que es lo que buscábamos desde un principio. Obsérvese que este enfoque recupera una componente principal a la vez; en secciones posteriores lo compararemos con otros enfoques algorítmicos, como el descenso por gradiente sobre todos los vectores propios.

2.1.3 Aprendizaje local e incremental mediante PCA en línea

Nótese que, en la construcción anterior, la transformación lineal $\boldsymbol{U}$ (es decir, la matriz de componentes principales) utilizada para la codificación y la decodificación se calcula con antelación «fuera de línea» a partir de todos los

datos de entrada. Una pregunta es si esta transformación se puede aprender «en línea» a medida que llegan los datos de entrada. Esta pregunta fue respondida por el trabajo de Oja en 1982 [Oja82].

Ejemplo 2.1 (Esquema de aprendizaje hebbiano normalizado para PCA). Consideremos una secuencia de vectores aleatorios independientes e idénticamente distribuidos $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_i, \ldots \in \mathbb{R}^n$ con covarianza $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$. Sea $\boldsymbol{u}_0 \in \mathbb{R}^n$ y definamos la respuesta de un vector de entrada $\boldsymbol{x}_i$ ante un vector de pesos $\boldsymbol{u}_i$ como su producto interno:

$$\eta_i = \boldsymbol{u}_i^T \boldsymbol{x}_i \tag{2.1.24}$$

y actualicemos el vector de pesos según el siguiente esquema:

$$\boldsymbol{u}_{i+1} = \frac{\boldsymbol{u}_i + \gamma \eta_i \boldsymbol{x}_i}{\|\boldsymbol{u}_i + \gamma \eta_i \boldsymbol{x}_i\|_2} \tag{2.1.25}$$

para alguna ganancia pequeña $\gamma > 0$. Este esquema de actualización puede considerarse como un esquema normalizado de aprendizaje hebbiano, en el que los pesos de las conexiones entre neuronas se fortalecen si tanto la entrada $\boldsymbol{x}$ como la salida η (o sus productos) son fuertes. Se puede considerar que el vector de pesos $\boldsymbol{u}$ se «aprende» a partir de una forma de retroalimentación procedente de la salida η. Entonces, bajo supuestos razonables, Oja [Oja82] ha demostrado que $\boldsymbol{u}_i$ converge al vector propio asociado al valor propio más grande de $\boldsymbol{\Sigma}$. ■

Para interpretar el esquema hebbiano normalizado (2.1.25), consideremos el objetivo de PCA para la muestra de datos (2.1.13) como una aproximación a la versión *poblacional* del objetivo, definida mediante una esperanza sobre la distribución de las muestras $\boldsymbol{x}$:

$$\min_{\boldsymbol{U}^\top \boldsymbol{U} = \boldsymbol{I}} \mathbb{E}_{\boldsymbol{x}} \left[\left\| \boldsymbol{x} - \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{x} \right\|_2^2 \right] \tag{2.1.26}$$

Profundizaremos en este objetivo en la siguiente sección. Entonces, el esquema hebbiano normalizado puede interpretarse como una aproximación de primer orden a un esquema de descenso por gradiente proyectado *estocástico* sobre la Ecuación (2.1.26) (con un tamaño de lote 1, y con el número de columnas de $\boldsymbol{U}$ igual a 1) siempre que $\|\boldsymbol{u}\|_2 = 1$, lo cual se mantiene mediante la operación de proyección en (2.1.25). Un *algoritmo en línea* de este tipo es un elemento básico útil para algoritmos más complejos que que procesan datos con estructura secuencial y causal, como veremos con más detalle en la Sección 5.3.3 y la Sección 8.11. Curiosamente y en términos más generales, el sencillo algoritmo de PCA en línea sugiere que *algoritmos más sencillos que la retropropagación (de extremo a extremo) pueden lograr aprender pares de codificador y decodificador*, lo cual revisaremos brevemente en el Capítulo 9 como un objetivo importante para el estudio futuro de la inteligencia.

2.1.4 La perspectiva estadística: PCA probabilístico

Nótese que la formulación anterior no establece ningún supuesto estadístico sobre el proceso generador de datos. Sin embargo, es habitual incluir elementos

estadísticos en un modelo de datos dado, ya que pueden aportar interpretaciones adicionales más esclarecedoras sobre el resultado del análisis. Por lo tanto, nos surge la pregunta natural: *¿Cuál es el análogo estadístico de una estructura de baja dimensión?* Nuestra respuesta es que una *distribución* de baja dimensión es aquella cuyo soporte se concentra en torno a una estructura geométrica de baja dimensión.

Para ilustrar este punto, abordamos el *análisis probabilístico de componentes principales (PPCA)* [Row97; TB99]. Esta formulación puede considerarse una variante estadística del PCA convencional. Desde el punto de vista matemático, consideramos ahora nuestros datos como muestras de una variable aleatoria $\boldsymbol{x}$ que toma valores en $\mathbb{R}^D$ (lo que a veces también se denomina un *vector aleatorio*). Decimos que $\boldsymbol{x}$ tiene una estructura estadística (aproximada) de bajo rango si y solo si existen una matriz ortonormal $\boldsymbol{U} \in \mathsf{O}(D, d)$, una variable aleatoria $\boldsymbol{z}$ que toma valores en $\mathbb{R}^d$, así como una *pequeña* variable aleatoria $\boldsymbol{\varepsilon}$ que toma valores en $\mathbb{R}^D$ tal que $\boldsymbol{z}$ y $\boldsymbol{\varepsilon}$ son independientes y

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}. \tag{2.1.27}$$

Nuestro objetivo es, una vez más, recuperar $\boldsymbol{U}$. Con este fin, planteamos el problema análogo al de la Sección 2.1.1, es decir, la optimización sobre los subespacios de soporte $\tilde{\boldsymbol{U}}$ y las variables aleatorias $\boldsymbol{z}$ para resolver el siguiente problema:

$$\min_{\tilde{\boldsymbol{U}}, \tilde{\boldsymbol{z}}} \mathbb{E} \, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{z}}\|_2^2. \tag{2.1.28}$$

Dado que estamos buscando la mejor variable aleatoria $\boldsymbol{z}$ de este tipo, podemos calcular su valor concreto $\boldsymbol{z}(\boldsymbol{x})$ por separado para cada valor de $\boldsymbol{x}$. Llevando a cabo los mismos cálculos que en Sección 2.1.1, obtenemos

$$\min_{\tilde{\boldsymbol{U}}, \tilde{\boldsymbol{z}}} \mathbb{E} \, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{z}}\|_2^2 = \min_{\tilde{\boldsymbol{U}}} \mathbb{E} \min_{\tilde{\boldsymbol{z}}(\boldsymbol{x})} \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{z}}(\boldsymbol{x})\|_2^2 = \min_{\tilde{\boldsymbol{U}}} \mathbb{E} \, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top \boldsymbol{x}\|_2^2, \tag{2.1.29}$$

reafirmando una vez más el hecho de que el subespacio estimado con componentes principales $\boldsymbol{U}^\star$ corresponde a un *denoiser* $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top$ que se proyecta sobre ese subespacio. Como antes, obtenemos

$$\arg\min_{\tilde{\boldsymbol{U}}} \mathbb{E} \, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top \boldsymbol{x}\|_2^2 = \arg\max_{\tilde{\boldsymbol{U}}} \mathbb{E} \, \|\tilde{\boldsymbol{U}}^\top \boldsymbol{x}\|_2^2 \tag{2.1.30}$$

$$= \arg\max_{\tilde{\boldsymbol{U}}} \operatorname{tr}(\tilde{\boldsymbol{U}}^\top \, \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]\tilde{\boldsymbol{U}}), \tag{2.1.31}$$

y la solución a este último problema consiste simplemente en las d primeras componentes principales de la matriz de momentos de segundo orden $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$. En realidad, los problemas anteriores son visualmente muy similares a las ecuaciones para calcular las componentes principales de la subsección anterior, salvo que $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ sustituye a $\boldsymbol{X}\boldsymbol{X}^\top/N$. De hecho, esta última cantidad es una estimación de la primera. Ambas formulaciones hacen efectivamente lo mismo y tienen la misma solución práctica: calcular los vectores singulares izquierdos de la matriz de datos $\boldsymbol{X}$, o, lo que es lo mismo, los vectores propios superiores

de la matriz de covarianza estimada $\boldsymbol{X}\boldsymbol{X}^\top/N$. Sin embargo, la formulación estadística tiene una interpretación adicional. Supongamos que $\mathbb{E}[\boldsymbol{z}] = \mathbf{0}$ y $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$. Tenemos que

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{U}\,\mathbb{E}[\boldsymbol{z}] + \mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}, \tag{2.1.32}$$

de modo que $\operatorname{Cov}[\boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$. Ahora, desarrollando $\operatorname{Cov}[\boldsymbol{x}]$ tenemos

$$\operatorname{Cov}[\boldsymbol{x}] = \boldsymbol{U}\operatorname{Cov}[\boldsymbol{z}]\boldsymbol{U}^\top + \operatorname{Cov}[\boldsymbol{\varepsilon}] = \boldsymbol{U}\,\mathbb{E}[\boldsymbol{z}\boldsymbol{z}^\top]\boldsymbol{U}^\top + \operatorname{Cov}[\boldsymbol{\varepsilon}]. \tag{2.1.33}$$

En particular, si $\operatorname{Cov}[\boldsymbol{\varepsilon}]$ es pequeño, se cumple que $\operatorname{Cov}[\boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ es aproximadamente una matriz de bajo rango, concretamente de rango d. Por lo tanto, los d vectores propios principales de $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ resumen esencialmente toda la matriz de covarianza. Pero también son las componentes principales, por lo que podemos interpretar el análisis de componentes principales como una descomposición de bajo rango de $\operatorname{Cov}[\boldsymbol{x}]$.

Nota 2.6. Al aplicar el enfoque probabilístico del PCA, logramos una comprensión más clara y cuantitativa de cómo se relaciona con la eliminación de ruido. En primer lugar, consideremos el problema de eliminación de ruido en (2.1.29), es decir

$$\min_{\tilde{\boldsymbol{U}}} \mathbb{E}\,\|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top\boldsymbol{x}\|_2^2. \tag{2.1.34}$$

No es muy difícil demostrar que si $\tilde{\boldsymbol{U}}$ tiene d columnas y si $\boldsymbol{\varepsilon}$ es una variable aleatoria gaussiana isotrópica, es decir, con distribución $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\boldsymbol{I})$,[3], entonces para *cualquier* solución óptima $\boldsymbol{U}^\star$ a este problema, tenemos

$$\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top = \boldsymbol{U}\boldsymbol{U}^\top \tag{2.1.35}$$

y así, el verdadero subespacio de soporte, sea $\mathcal{S} \doteq \operatorname{col}(\boldsymbol{U})$, se obtiene como el espacio generado por las columnas de $\boldsymbol{U}^\star$, ya que

$$\mathcal{S} = \operatorname{col}(\boldsymbol{U}) = \operatorname{col}(\boldsymbol{U}\boldsymbol{U}^\top) = \operatorname{col}(\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top) = \operatorname{col}(\boldsymbol{U}^\star). \tag{2.1.36}$$

En concreto, el *mapa de eliminación de ruido* estimado $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top$ es una proyección ortogonal sobre $\mathcal{S}$, que empuja los puntos con ruido hacia el subespacio de soporte de la verdad de referencia (ground truth). Podemos obtener un resultado técnico similar en el caso de que solo dispongamos de un número finito de muestras como en el Teorema 2.1, pero esto requiere más esfuerzo y complejidad técnica. En resumen, tenemos el siguiente teorema informal.

Teorema 2.3. *Supongamos que la variable aleatoria $\boldsymbol{x}$ se puede escribir como*

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon} \tag{2.1.37}$$

donde $\boldsymbol{U} \in \mathsf{O}(D, d)$ captura la estructura de bajo rango, $\boldsymbol{z}$ es un vector aleatorio que toma valores en $\mathbb{R}^d$ y $\boldsymbol{\varepsilon}$ es un vector aleatorio que toma valores en $\mathbb{R}^D$, de tal

[3]Otras distribuciones también funcionan siempre y cuando abarquen todo $\mathbb{R}^D$, pero la gaussiana es la más fácil de manejar en este caso.

manera que $\boldsymbol{z}$ y $\boldsymbol{\varepsilon}$ son independientes y $\boldsymbol{\varepsilon}$ es pequeño. Entonces, las componentes principales $\boldsymbol{U}^\star \in \mathsf{O}(D,d)$ de nuestro conjunto de datos vienen dadas por los d primeros vectores propios de $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ y corresponden aproximadamente al denoiser lineal óptimo: $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top \approx \boldsymbol{U}\boldsymbol{U}^\top$.

2.1.5 Completamiento enmascarado de matrices de bajo rango

En las subsecciones anteriores, examinamos el problema de *aprender una distribución geométrica o estadística de bajo rango*, en la que los datos se obtuvieron mediante muestreo de un subespacio con ruido aditivo. Pero este no es el único tipo de perturbación derivada de una distribución de baja dimensión que vale la pena estudiar. En esta sección, presentamos otra clase de errores no aditivos que están cobrando cada vez más importancia en el aprendizaje profundo. Consideremos el caso en el que tenemos unos datos $\{\boldsymbol{x}_i\}_{i=1}^N$ generados según (2.1.1). Ahora los organizamos en una matriz $\boldsymbol{X} = \big[\boldsymbol{x}_1, \dots, \boldsymbol{x}_N\big] \in \mathbb{R}^{D\times N}$. A diferencia del caso anterior, no observamos $\boldsymbol{X}$ directamente; en su lugar, imaginamos que nuestra observación se vio corrupta en el camino y que obtuvimos

$$\boldsymbol{Y} = \boldsymbol{M} \odot \boldsymbol{X}, \tag{2.1.38}$$

donde $\boldsymbol{M} \in \{0,1\}^{D\times N}$ es una *máscara* que conocemos y $\odot$ es la multiplicación elemento por elemento. En este caso, nuestro objetivo es recuperar $\boldsymbol{X}$ (a partir de ahí podemos utilizar el PCA para recuperar $\boldsymbol{U}$, etc.), partiendo únicamente de la observación corrupta $\boldsymbol{Y}$, la máscara $\boldsymbol{M}$ y el conocimiento de que $\boldsymbol{X}$ es (aproximadamente) de bajo rango. Esto se denomina *completado de matrices de bajo rango*.

Este y otros problemas similares generalizados de recuperación de estructuras de bajo rango se denominan problemas de «PCA robusta». Existen muchas fuentes excelentes que analizan algoritmos y enfoques para resolver estos problemas [WM22]. Por lo tanto, no entraremos en detalle sobre el método de solución en este contexto. En su lugar, analizaremos bajo qué condiciones es *plausible* resolver este problema. Por un lado, en el caso más absurdo, supongamos que cada entrada de la matriz $\boldsymbol{X}$ se eligiera independientemente de todas las demás. Entonces no habría ninguna posibilidad de recuperar $\boldsymbol{X}$ con exactitud, incluso si solo faltara una entrada y tuviéramos $DN-1$ entradas. Por otro lado, supongamos que $\boldsymbol{X}$ tuviera exactamente rango 1, lo cual es una condición extremadamente fuerte sobre la estructura de baja dimensión de los datos y que estuviéramos trabajando con la máscara

$$\boldsymbol{M} = \begin{bmatrix} \mathbf{1}_{(D-1)\times 1} & \mathbf{0}_{(D-1)\times(N-1)} \\ 1 & \mathbf{1}_{1\times(N-1)} \end{bmatrix}. \tag{2.1.39}$$

Entonces sabemos que los datos se distribuyen en una recta y conocemos un vector en esa recta: no es más que la primera columna de la matriz $\boldsymbol{Y} = \boldsymbol{M}\odot\boldsymbol{X}$. A partir de la última coordenada de cada columna, que también nos revela la máscara, podemos resolver cada columna, ya que para cada coordenada final

solo hay un vector en la recta que pasa por esa coordenada. De este modo, podemos reconstruir $\boldsymbol{X}$ con total precisión, para lo que solo necesitamos un número lineal de observaciones $D + N - 1$.

En el mundo real, los problemas concretor se sitúan en algún punto intermedio entre los dos casos límite mencionados anteriormente. Sin embargo, las diferencias entre estos dos extremos, así como la discusión anterior sobre el PCA, revelan un núcleo general de verdad:

> *Cuanto menor sea la dimensión y más estructurada sea la distribución de los datos, tanto más fácil resulta procesarlos y menos observaciones se necesitan, siempre y cuando el algoritmo aproveche eficazmente esta estructura de baja dimensión.*

Como cabía esperar, nos encontraremos con este motivo una y otra vez en el resto del manuscrito, empezando ya en la siguiente sección.[4]

2.2 Una mezcla de subespacios completos de baja dimensión

Como hemos visto, los modelos de señales de bajo rango son lo suficientemente ricos en estructura como para ofrecer una visión global de la interacción entre la dimensionalidad baja de los datos y los algoritmos computacionales eficientes y escalables para la representación y recuperación en presencia de errores. Estos modelos implican un proceso de aprendizaje de representaciones *lineal* y simétrico (2.1.6):

$$\boldsymbol{z} = \mathcal{E}(\boldsymbol{x}) = \boldsymbol{U}^\top \boldsymbol{x}, \quad \hat{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}) = \boldsymbol{U}\boldsymbol{z},$$

de la que se puede demostrar que se aprende a partir de un número finito de muestras de $\boldsymbol{x}$ mediante el análisis de componentes principales (resuelto de manera eficiente, por ejemplo, con el método de potencias) siempre que la distribución de $\boldsymbol{x}$ sea realmente lineal. Se trata de una hipótesis restrictiva, pues, tal y como objetó Harold Hotelling, el distinguido estadístico del siglo XX,[5] cuando George Dantzig presentó por primera vez su teoría de la programación lineal[Dan02],

> *"...todos sabemos que el mundo es no lineal."*

Aun teniendo en cuenta su elegancia y simplicidad, la suposición de bajo rango es demasiado restrictiva como para ser ampliamente aplicable al modelado

[4] En capítulos posteriores, veremos que es posible completar datos del mundo real, como imágenes, con solo una fracción de observaciones sin enmascarar; por ejemplo, la autocodificación enmascarada de imágenes en la Sección 8.5. Esto se debe también a que la distribución de las imágenes naturales presenta estructuras de dimensiones bastante bajas (aunque no lineales) en el espacio de todas las imágenes.

[5] Por casualidad, también famoso por sus contribuciones al desarrollo y la denominación del Análisis de Componentes Principales [Hot33].

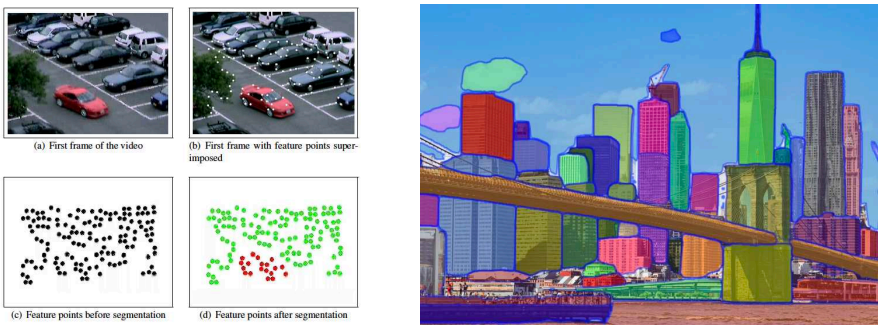


Figura 2.4: **Izquierda:** características rastreadas en objetos que se mueven en forma independiente dentro de una escena. **Derecha:** fragmentos de imagen asociados a diferentes regiones de una imagen.

de datos del mundo real. Una limitación clave es la suposición de un *único* subespacio lineal responsable de generar las observaciones estructuradas. En muchas aplicaciones prácticas, la estructura generada por una *mezcla* de subespacios distintos de baja dimensión ofrece un modelo más realista. Por ejemplo, consideremos una secuencia de vídeo que captura el movimiento de varios objetos distintos, cada uno de los cuales con su propio desplazamiento independiente (Figura 2.4 a la izquierda). Bajo supuestos adecuados sobre los movimientos individuales, cada objeto se convierte en responsable de un subespacio independiente de baja dimensión en la secuencia concatenada de fotogramas de vídeo [VM04]. Como otro ejemplo, pensemos en la modelación de imágenes naturales mediante el aprendizaje de un modelo para la distribución de *parches*, conjuntos de píxeles espacialmente contiguos dentro de una imagen (Figura 2.4 a la derecha). A diferencia del entorno simplificado de la Figura 2.2, donde las imágenes con poses correspondientes pueden aproximarse bien mediante un único subespacio de baja dimensión, el fragmento situado en una ubicación específica de una imagen natural puede corresponder a objetos con propiedades muy diferentes, por ejemplo, colores o formas distintos debido a los bordes de la oclusión. Por lo tanto, modelar la distribución de los parches con un único subespacio resulta estéril, pero una *mezcla* de subespacios, uno para cada región, ofrece unos resultados que funcionan sorprendentemente bien en la práctica, por ejemplo, para fines de segmentación o compresión [MRY+11].[6] Veremos un ejemplo concreto en un capítulo posterior (Ejemplo 4.10).

En esta sección, comenzaremos examinando los fundamentos conceptuales y algorítmicos del aprendizaje de representaciones estructuradas cuando la distribución de los datos puede modelarse mediante *una mezcla de subespacios de baja dimensión*, tal y como se ilustra en la Figura 2.1. En este contexto, el mapeo del decodificador será casi tan simple como en el caso de un único

[6] Volveremos sobre esta observación en el Capítulo 5, donde demostraremos que puede generalizarse de manera significativa para aprender representaciones de conjuntos de datos modernos a gran escala.

subespacio: simplemente lo representamos mediante

$$\hat{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}) = \left(\sum_{k=1}^{K} \pi_k(\boldsymbol{z}) \boldsymbol{U}_k \right) \boldsymbol{z}, \tag{2.2.1}$$

donde $\pi_k : \mathbb{R}^d \to \{0, 1\}$ es un conjunto de variables aleatorias de ponderación *esparsas*, de tal manera que en la suma solo se selecciona un único subespacio $\mathcal{S}_k = \text{col}(\boldsymbol{U}_k)$. Sin embargo, la tarea de codificar dichos datos $\boldsymbol{x}$ en representaciones adecuadas $\boldsymbol{z}$ y de aprender dicho par codificador-decodificador a partir de los datos, resultará más compleja.

Veremos cómo las ideas que provienen de la amplia bibliografía sobre *representación esparsa* y *análisis de componentes independientes* conducen a una reformulación natural de la arquitectura del decodificador anterior desde la perspectiva de la esparcidad, la arquitectura correspondiente del codificador (obtenida mediante un algoritmo similar al método de potencias, análogo al del análisis de componentes principales) y garantías sólidas de corrección y eficiencia para el aprendizaje de dichos pares codificador-decodificador a partir de los datos. En este sentido, el caso de la estructura lineal mixta de baja dimensión ya nos lleva a muchas de las componentes conceptuales clave del aprendizaje de representaciones estructuradas que desarrollaremos con una generalidad mucho mayor en este libro.

2.2.1 Mezclas de subespacios y diccionarios con codificación esparsa

Sea $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K$, cada uno de tamaño $D \times d$, una colección de bases ortonormales para K subespacios de dimensión d en $\mathbb{R}^D$. Decir que $\boldsymbol{x}$ sigue una distribución de mezcla de subespacios parametrizada por $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K$ significa, en términos geométricos, que

$$\boldsymbol{x} = \boldsymbol{U}_k \boldsymbol{z} \quad \text{for some } \; k \in [K], \;\; \boldsymbol{z} \in \mathbb{R}^d. \tag{2.2.2}$$

El análogo estadístico de este modelo geométrico, tal como vimos en el caso del PCA y la estructura lineal, es que $\boldsymbol{x}$ sigue una distribución de *mezcla de gaussianas*: es decir,

$$\boldsymbol{x} \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top), \quad \text{for some } \; \pi_k \geq 0, \;\; \sum_{k=1}^{K} \pi_k = 1. \tag{2.2.3}$$

En otras palabras, para cada $k \in [K]$, $\boldsymbol{x}$ es gaussiano en el subespacio de baja dimensión $\text{col}(\boldsymbol{U}_k)$ con probabilidad π_k.

Nota 2.7 (Mezcla de gaussianas vs superposición de gaussianas). Hay que tener en cuenta que el modelo anterior (2.2.3) es una *mezcla* de distribuciones gaussianas, que no debe confundirse con una mezcla de variables gaussianas por superposición, por ejemplo

$$\boldsymbol{x} = \sum_{i=1}^{n} w_i \boldsymbol{x}_i, \quad \boldsymbol{x}_i \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_i \boldsymbol{U}_i^\top), \tag{2.2.4}$$

donde $\boldsymbol{x}_i$ son vectores aleatorios gaussianos independientes y w_i son un conjunto de pesos fijos. Como sabemos por las propiedades de los vectores gaussianos, dicha superposición $\boldsymbol{x}$ seguirá siendo una distribución gaussiana.

Por ahora, nos centramos en la perspectiva geométrica que ofrece (2.2.2). Existe una alternativa algebraicamente conveniente a esta representación condicional. Consideremos un vector de representación *elevado* $\boldsymbol{z} = [\boldsymbol{z}_1^\top, \dots, \boldsymbol{z}_K^\top]^\top \in \mathbb{R}^{dK}$, tal que $\boldsymbol{z}$ es *d-esparso* y tiene su soporte en uno de los K bloques consecutivos no superpuestos de d coordenadas de entre las dK. Entonces, (2.2.2) puede escribirse de forma equivalente como

$$\boldsymbol{x} = \underbrace{\begin{bmatrix} | & \dots & | \\ \boldsymbol{U}_1 & \dots & \boldsymbol{U}_K \\ | & \dots & | \end{bmatrix}}_{\boldsymbol{U}} \underbrace{\begin{bmatrix} \boldsymbol{z}_1 \\ \vdots \\ \boldsymbol{z}_K \end{bmatrix}}_{\boldsymbol{z}}, \quad \left\| \begin{bmatrix} \|\boldsymbol{z}_1\|_2 \\ \vdots \\ \|\boldsymbol{z}_K\|_2 \end{bmatrix} \right\|_0 = 1. \tag{2.2.5}$$

Aquí, la «norma» ℓ^0 $\|\cdot\|_0$ mide la esparcidad contando el número de entradas distintas de cero:

$$\|\boldsymbol{z}\|_0 = |\{i \mid z_i \neq 0\}|, \tag{2.2.6}$$

y la matriz $\boldsymbol{U} \in \mathbb{R}^{D \times Kd}$ se denomina *diccionario*, cuyos bloques de columnas $\{\boldsymbol{U}_i\}_{i=1}^K$ se llaman palabras de código. En general, si el número de subespacios en la mezcla K es lo suficientemente grande, no hay ningún límite en cuanto al número de columnas que contiene el diccionario $\boldsymbol{U}$. En el caso de que $Kd < D$, $\boldsymbol{U}$ se denomina *subcompleta*, cuando $Kd = D$, se denomina *completa*; y cuando $Kd > D$, se denomina *sobrecompleta*.

Ahora bien, (2.2.5) sugiere una relajación conveniente para la manejabilidad del análisis: en lugar de modelar $\boldsymbol{x}$ como procedente de una mezcla de K subespacios específicos $\boldsymbol{U}_1, \dots, \boldsymbol{U}_K$, podríamos, en su lugar, comenzar con un diccionario $\boldsymbol{U} \in \mathbb{R}^{D \times m}$, donde m puede ser menor o mayor que D, y simplemente buscar representar $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z}$ con la esparcidad $\|\boldsymbol{z}\|_0$ suficientemente pequeña. Esto nos lleva al *modelo de diccionario esparso* para $\boldsymbol{x}$:

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}, \quad \|\boldsymbol{z}\|_0 \ll d, \tag{2.2.7}$$

donde $\boldsymbol{\varepsilon}$ representa un vector de ruido desconocido. Desde el punto de vista geométrico, esto implica que $\boldsymbol{x}$ se encuentra cerca del espacio generado por un subconjunto de $\|\boldsymbol{z}\|_0$ columnas de $\boldsymbol{U}$, lo que convierte esto en una instancia del modelo de mezcla de subespacios (2.2.2) con un valor muy elevado de K y correlaciones específicas entre los subespacios $\boldsymbol{U}_k$.

Diccionario ortogonal para una codificación esparsa. Ahora podemos formular el problema del aprendizaje de representaciones estructuradas para mezclas de subespacios de baja dimensión que estudiaremos en esta sección. Suponemos que disponemos de muestras $\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_N]$ procedentes de un modelo de diccionario esparso desconocido (2.2.7), posiblemente con ruido añadido $\boldsymbol{\varepsilon}_i$. Partamos de la suposición de que el diccionario $\boldsymbol{U}$ en el modelo

de diccionario esparso (2.2.7) es completo y ortogonal,[7] y que cada vector de coeficientes $\boldsymbol{z}$ es d-esparso, con $d \ll D$. En este contexto, el aprendizaje de representaciones equivale a aprender correctamente el diccionario ortogonal $\boldsymbol{U}$ mediante optimización: entonces podemos tomar $\mathcal{E}(\boldsymbol{x}) = \boldsymbol{U}^\top \boldsymbol{x}$ como codificador y $\mathcal{D}(\boldsymbol{z}) = \boldsymbol{U}\boldsymbol{z}$ como decodificador y $\mathcal{D} = \mathcal{E}^{-1}$. En forma de diagrama:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}=\boldsymbol{U}^\top} \boldsymbol{z} \xrightarrow{\mathcal{D}=\boldsymbol{U}} \hat{\boldsymbol{x}}. \tag{2.2.8}$$

Observamos que el par $(\mathcal{E}, \mathcal{D})$ de autocodificadores para el aprendizaje completo de un diccionario es simétrico, al igual que en el caso de un único subespacio lineal, lo que hace que la tarea computacional de codificación y decodificación no sea más difícil que en el caso lineal. Por otro lado, la tarea de aprender el diccionario $\boldsymbol{U}$ es estrictamente más difícil que aprender un único subespacio lineal mediante el PCA. Para comprender por qué no podemos limitarnos a utilizar el PCA para aprender correctamente el diccionario ortogonal $\boldsymbol{U}$, observemos que la función de pérdida que dio origen al PCA, es decir, (2.1.5), es completamente invariante ante las rotaciones de las filas de la matriz $\boldsymbol{U}$: es decir, si $\boldsymbol{Q}$ es cualquier matriz ortogonal $d \times d$, entonces tanto $\boldsymbol{U}$ como $\boldsymbol{U}\boldsymbol{Q}$ son viables y tienen una pérdida idéntica para (2.1.5). El modelo de diccionario esparso no es, en absoluto, invariante ante tales transformaciones: si reemplazáramos $\boldsymbol{U}$ por $\boldsymbol{U}\boldsymbol{Q}$ y realizáramos la rotación correspondiente $\boldsymbol{Q}^\top \boldsymbol{z}$ de los coeficientes de representación $\boldsymbol{z}$, en general destruiríamos la estructura de esparcidad de $\boldsymbol{z}$, lo que violaría la suposición del modelo. Por lo tanto, necesitamos nuevos algoritmos para aprender diccionarios ortogonales.

2.2.2 Aprendizaje de diccionarios completos

En esta sección, derivaremos algoritmos para resolver el problema del aprendizaje de diccionarios ortogonales. Para ser más precisos, suponemos que el vector observado $\boldsymbol{x} \in \mathbb{R}^D$ sigue un modelo estadístico

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}, \tag{2.2.9}$$

donde $\boldsymbol{U} \in \mathbb{R}^{D \times D}$ es un diccionario ortogonal desconocido, $\boldsymbol{z}$ es un vector de ruido aleatorio, cuyas componentes z_i son independientes entre sí, gaussianas y de pequeña magnitud, mientras $\boldsymbol{\varepsilon} \in \mathbb{R}^D$ es un vector aleatorio independiente de pequeños ruidos (gaussianos). El objetivo es recuperar $\boldsymbol{U}$ (y, por lo tanto, $\boldsymbol{z}$) a partir de muestras de $\boldsymbol{x}$.

Aquí suponemos que cada componente independiente z_i sigue una distribución de la forma

$$z_i \sim \mathrm{Ber}(\theta) \cdot \mathcal{N}(0, 1/\theta).$$

Es decir, es el producto de una variable aleatoria de Bernoulli con probabilidad θ de tomar el valor 1 y probabilidad $1 - \theta$ de tomar el valor 0 y una variable

[7] Se puede demostrar que, para el caso completo, no perdemos generalidad al hacer la suposición de ortogonalidad (Ejercicio 2.4).

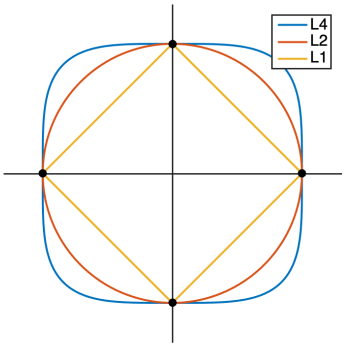


Figura 2.5: Maximizar la norma ℓ^4 o minimizar la norma ℓ^1 promueve la esparcidad (para vectores sobre la esfera).

aleatoria gaussiana independiente con varianza $1/\theta$. Esta distribución se conoce formalmente como distribución *de Bernoulli-Gauss*. La normalización se elige de tal manera que $\mathrm{Var}(z_i) = 1$ y, por lo tanto, $\mathbb{E}[\|\boldsymbol{z}\|_2^2] = d$. Este supuesto de modelado implica que el vector de componentes independientes $\boldsymbol{z}$ suele ser muy esparso: calculamos $\mathbb{E}\left[\|\boldsymbol{z}\|_0\right] = d\theta$, que es pequeño cuando θ es inversamente proporcional a d.

Nota 2.8 (La suposición de ortogonalidad). A primera vista, la suposición de que el diccionario $\boldsymbol{U}$ es ortogonal podría parecer algo restrictiva. Pero, en realidad, no hay ninguna pérdida de generalidad. Se puede considerar como un diccionario completo a cualquier matriz cuadrada invertible $\boldsymbol{U}$. Con muestras generadas a partir de este diccionario: $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{Z} \in \mathbb{R}^{D\times N}$, es fácil demostrar que, con cierto precondicionamiento de la matriz de datos $\boldsymbol{X}$:

$$\bar{\boldsymbol{X}} = \left(\frac{1}{N\theta}\boldsymbol{X}\boldsymbol{X}^\top\right)^{-\frac{1}{2}}\boldsymbol{X}, \tag{2.2.10}$$

entonces existe una matriz ortogonal $\boldsymbol{U}_o \in \mathsf{O}(D)$ tal que

$$\bar{\boldsymbol{X}} = \boldsymbol{U}_o\boldsymbol{Z}. \tag{2.2.11}$$

Véase el Ejercicio 2.4 o [SQW17b] para más detalles.

Aprendizaje de diccionarios mediante el algoritmo MSP Supongamos ahora que se nos da un conjunto de observaciones:

$$\boldsymbol{x}_i = \boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i,\ \forall i \in [N]. \tag{2.2.12}$$

Sea $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ y $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$. El objetivo es recuperar $\boldsymbol{U}$ a partir de los datos $\boldsymbol{X}$. Por lo tanto, para toda matriz ortogonal $\boldsymbol{A} \in \mathsf{O}(D)$,

$$\boldsymbol{A}\boldsymbol{x}_i = \boldsymbol{A}\boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{A}\boldsymbol{\varepsilon}_i \tag{2.2.13}$$

sería casi esparsa si $\boldsymbol{A} = \boldsymbol{U}^\top$ (puesto que, según la hipótesis, el ruido $\boldsymbol{\varepsilon}_i$ es de magnitud pequeña).

Además, dado que $\boldsymbol{U}$ es ortogonal y que $\boldsymbol{\varepsilon}$ es pequeño, el vector $\boldsymbol{x}$ tiene una norma esperada predecible, es decir, $\mathbb{E}[\|\boldsymbol{x}\|_2^2] \approx \mathbb{E}[\|\boldsymbol{z}\|_2^2] = d$. Es un hecho conocido que, para vectores en una esfera, maximizar la norma ℓ^4 es equivalente a minimizar la norma ℓ^0 (para favorecer la esparcidad),

$$\arg\max_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_4 \quad \Leftrightarrow \quad \arg\min_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_0. \tag{2.2.14}$$

Esto se ilustra en la Figura 2.5.

Una matriz ortogonal $\boldsymbol{A}$ conserva la norma euclidiana (ℓ^2): $\|\boldsymbol{A}\boldsymbol{x}\|_2^2 = \|\boldsymbol{x}\|_2^2$. Por lo tanto, para hallar el diccionario ortogonal correcto $\boldsymbol{U}$ a partir de $\boldsymbol{X}$, podemos intentar resolver el siguiente programa de optimización:

$$\max_{\tilde{\boldsymbol{A}}\in\mathsf{O}(D)} \frac{1}{4}\left\|\tilde{\boldsymbol{A}}\boldsymbol{X}\right\|_4^4 = \frac{1}{4}\sum_{i=1}^{N}\left\|\tilde{\boldsymbol{A}}\boldsymbol{x}_i\right\|_4^4 \tag{2.2.15}$$

Esto se conoce como el problema de maximización de la norma ℓ^4 [ZMZ+20]. Una vez que encontramos la solución $\boldsymbol{A}^\star$, podemos calcular la transpuesta $\boldsymbol{U}^\star = (\boldsymbol{A}^\star)^\top$.

Nota 2.9. También se sabe que, para vectores en una esfera, minimizar la norma ℓ^1 equivale a minimizar la norma ℓ^0 (para favorecer la esparcidad),

$$\arg\min_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_1 \quad \Leftrightarrow \quad \arg\min_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_0,$$

lo cual también se ilustra en la Figura 2.5. Este hecho también puede aprovecharse para aprender el diccionario $\boldsymbol{A}$ de manera eficaz y eficiente. En realidad, este enfoque ya se había explorado con anterioridad a la norma ℓ^4 empleada en este trabajo. Los lectores interesados pueden consultar el trabajo de [QZL+20b].

Obsérvese que el problema anterior equivale al siguiente problema de optimización con restricciones:

$$\min -\frac{1}{4}\left\|\tilde{\boldsymbol{A}}\boldsymbol{X}\right\|_4^4 \quad \text{subject to} \quad \tilde{\boldsymbol{A}}^\top\tilde{\boldsymbol{A}} = \boldsymbol{I}. \tag{2.2.16}$$

Como se muestra en [WM22], utilizando el método de los multiplicadores de Lagrange, se puede deducir que la solución óptima al problema debe satisfacer la siguiente condición de «punto fijo»:

$$\boldsymbol{A}^\star = \mathcal{P}_{\mathrm{O}(D)}[(\boldsymbol{A}^\star\boldsymbol{X})^{\odot 3}\boldsymbol{X}^\top], \tag{2.2.17}$$

donde $\mathcal{P}_{\mathrm{O}(D)}[\,\cdot\,]$ es la proyección sobre el espacio de matrices ortogonales $\mathrm{O}(D)$.[8]

[8] Para cualquier matriz $\boldsymbol{M}$ con SVD $\boldsymbol{M} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$, se tiene $\mathcal{P}_{\mathrm{O}(D)}[\boldsymbol{M}] = \boldsymbol{U}\boldsymbol{V}^\top$. Se deja esto como ejercicio para el lector.

Para calcular el punto fijo de la ecuación anterior, en forma similar a a la utilizada para calcular los vectores propios en el PCA (2.1.16), podemos utilizar la siguiente iteración de potencias:

$$\tilde{\boldsymbol{A}}_{t+1} = \mathcal{P}_{\mathrm{O}(D)}[(\tilde{\boldsymbol{A}}_t \boldsymbol{X})^{\odot 3} \boldsymbol{X}^\top]. \tag{2.2.18}$$

Esto se conoce como el algoritmo de *coincidencia, estiramiento y proyección* (MSP) propuesto por [ZMZ+20]. Se demostró que, en condiciones generales, un algoritmo voraz de este tipo converge efectivamente hacia la solución correcta a un ritmo superlineal.

Nota 2.10 (Optimalidad global de la maximización de ℓ^4). El problema de maximización con restricciones de ℓ^4 es un programa no convexo. En general, *no* se debe esperar que algún algoritmo voraz (por ejemplo, del tipo de descenso por gradiente) converja hacia la solución óptima global. Sorprendentemente, se puede demostrar que, a diferencia de los programas no convexos generales, el paisaje de la maximización de ℓ^4 sobre una esfera

$$\min -\frac{1}{4} \left\| \tilde{\boldsymbol{q}}^\top \boldsymbol{X} \right\|_4^4 \quad \text{subject to} \quad \tilde{\boldsymbol{q}}^\top \tilde{\boldsymbol{q}} = 1. \tag{2.2.19}$$

es muy benigno: todos los mínimos locales están cerca de los óptimos globales y todos los puntos críticos son puntos de silla con una dirección de curvatura negativa. Por lo tanto, cualquier método de descenso capaz de evitar los puntos de silla estrictos encuentra, de manera demostrable, soluciones óptimas globales. Para una formulación más rigurosa, se remite al lector interesado a [QZL+20a].

Nota 2.11 (Red profunda lineal estable). El proceso iterativo anterior de cálculo del diccionario tiene una interpretación natural e incremental de «aprendizaje profundo». Definamos $\delta \boldsymbol{A}_{t+1} = \tilde{\boldsymbol{A}}_{t+1} \tilde{\boldsymbol{A}}_t^\top$ y $\boldsymbol{Z}_t = \tilde{\boldsymbol{A}}_t \boldsymbol{X}$, entonces es fácil demostrar que

$$\delta \boldsymbol{A}_{t+1} = \mathcal{P}_{\mathrm{O}(D)}[(\boldsymbol{Z}_t)^{\odot 3} \boldsymbol{Z}_t^\top].$$

Si $\tilde{\boldsymbol{A}}_t$ converge al diccionario correcto $\boldsymbol{U}$, entonces dicho proceso iterativo de codificación es esencialmente equivalente a una «red lineal profunda»:

$$\boldsymbol{Z} \longleftarrow \boldsymbol{Z}_{t+1} = \underbrace{\delta \boldsymbol{A}_{t+1} \delta \boldsymbol{A}_t \dots \delta \boldsymbol{A}_1}_{\text{capas construidas en el paso directo}} \boldsymbol{X}.$$

Téngase en cuenta que el cálculo de las transformaciones de incremento $\delta \boldsymbol{A}_{t+1}$ en cada capa depende únicamente de la salida (de características) de la capa anterior $\boldsymbol{Z}_t$. La red es estable por naturaleza, ya que cada capa es una transformación ortogonal que conserva la norma. A pesar de su semejanza con una red profunda lineal, la retropropagación no es necesaria para el aprendizaje de cada capa. ¡Todas las capas se entrenan en una sola pasada hacia adelante!

2.2.3 Relación con ICA y la curtosis

En el modelo de Bernoulli-Gauss, las variables z_i son independientes y no gaussianas. Además, existe una clara correspondencia entre el aprendizaje de

diccionarios y el análisis clásico de componentes independientes (ICA), hasta el punto de que los algoritmos diseñados para resolver un problema pueden utilizarse para resolver el otro.[9]

Con el fin de derivar un algoritmo basado en ICA, nos centramos en una función objetivo conocida como *curtosis*, que se utiliza en el ICA como consecuencia directa de la naturaleza no gaussiana de las componentes. La *curtosis*, o cumulante de cuarto orden, de una variable aleatoria X de media cero se define como

$$\operatorname{kurt}(X) = \mathbb{E}\, X^4 - 3(\mathbb{E}\, X^2)^2. \tag{2.2.20}$$

Si solo disponemos de un número finito de muestras de la variable aleatoria X organizadas en un vector $\boldsymbol{x} = [x_1, \ldots, x_N]$, definimos la curtosis a través de su media empírica, lo que da como resultado

$$\operatorname{kurt}(\boldsymbol{x}) = \frac{1}{N}\|\boldsymbol{x}\|_4^4 - \frac{3}{N^2}\|\boldsymbol{x}\|_2^4. \tag{2.2.21}$$

Por último, en el caso de los vectores aleatorios, definimos su curtosis como la suma de la curtosis escalar de cada componente. La curtosis es una función de pérdida natural para ICA, ya que, para una variable X gaussiana, la curtosis es cero; el lector puede comprobar además que la distribución de Bernoulli-Gauss tiene una curtosis positiva. Por lo tanto, un procedimiento natural para buscar componentes independientes no gaussianas consiste en buscar un conjunto de direcciones mutuamente ortogonales $\tilde{\boldsymbol{U}} \in \mathbb{R}^{d\times k}$, de tal manera que $\tilde{\boldsymbol{U}}^\top \boldsymbol{X}$ tenga la máxima curtosis, donde $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{Z} \in \mathbb{R}^{D\times N}$ es la matriz Bernoulli-gaussiana de datos ICA (de tal manera que $\tilde{\boldsymbol{U}}$ aproxima $\boldsymbol{U}$ salvo permutaciones de las columnas). Formalmente, nuestro objetivo es resolver el problema

$$\max_{\tilde{\boldsymbol{U}}^\top \tilde{\boldsymbol{U}} = \boldsymbol{I}} \operatorname{kurt}(\tilde{\boldsymbol{U}}^\top \boldsymbol{X}). \tag{2.2.22}$$

En un extremo, podemos establecer $k = D$ e intentar recuperar todo el diccionario $\boldsymbol{U}$ de una sola vez. Se puede demostrar que este problema se puede resolver con el algoritmo MSP que hemos visto anteriormente. En el otro extremo, podemos fijar $k = 1$ y tratar de recuperar una sola dirección (columna de $\boldsymbol{U}$) a la vez, realizando una *deflación*, es decir, sustituyendo la matriz de datos $\boldsymbol{X}$ por $(\boldsymbol{I} - \boldsymbol{u}\boldsymbol{u}^\top)\boldsymbol{X}$, después de cada paso antes de buscar otra dirección. Existe una disyuntiva natural entre la escalabilidad del enfoque incremental $k = 1$ y la eficiencia y solidez del enfoque $k = D$.

ICA incremental: corrección y algoritmo FastICA. El algoritmo FastICA, propuesto por Hyvärinen y Oja [HO97], es un algoritmo rápido de

[9] Analizamos este tema con mayor profundidad en el Ejercicio 2.3, donde se establece una relación entre la naturaleza no gaussiana de las componentes independientes y el concepto puramente geométrico de simetría. Este problema está relacionado con nuestra observación anterior de que el PCA no sirve para recuperar diccionarios ortogonales de uso esparso: en el contexto estadístico, puede relacionarse con la invariancia rotacional de la distribución gaussiana (Ejercicio 2.2).

punto fijo para resolver el esquema de maximización de la curtosis con $k = 1$ para ICA. El problema en cuestión es

$$\max_{\|\tilde{\boldsymbol{u}}\|_2^2=1} \operatorname{kurt}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}}). \tag{2.2.23}$$

En primer lugar, realizaremos un análisis muy básico de este objetivo para verificar su corrección. Obsérvese que, mediante el cambio de variables $\tilde{\boldsymbol{w}} = \boldsymbol{U}^\top \tilde{\boldsymbol{u}}$, este problema es equivalente a

$$\max_{\|\tilde{\boldsymbol{w}}\|_2^2=1} \operatorname{kurt}(\boldsymbol{Z}^\top \tilde{\boldsymbol{w}}).$$

Este objetivo es tan sencillo que podemos hacer afirmaciones contundentes sobre su corrección como formulación para recuperar el diccionario $\boldsymbol{U}$. Por ejemplo, en el contexto poblacional donde $N \to \infty$, podemos utilizar las propiedades de aditividad de la curtosis (Ejercicio 2.5) y nuestra normalización supuesta de las componentes independientes para expresar el problema anterior en forma equivalente como

$$\max_{\|\tilde{\boldsymbol{w}}\|_2^2=1} \sum_{i=1}^{d} \operatorname{kurt}(z_i)\tilde{w}_i^4. \tag{2.2.24}$$

Se puede mostrar que, bajo el supuesto Bernoulli-gaussiano, el paisaje de optimización de este problema es «benigno» (Ejercicio 2.7) — lo que significa que todos los máximos locales de la función objetivo corresponden a la recuperación de una de las componentes independientes.

En la práctica, no podemos calcular directamente el cambio de variables que lleva de $\tilde{\boldsymbol{u}}$ a $\tilde{\boldsymbol{w}}$, ya que requiere el diccionario de referencia $\boldsymbol{U}$, por lo que desarrollamos directamente algoritmos para la parametrización de $\tilde{\boldsymbol{u}}$ y la matriz de datos $\boldsymbol{X}$. Entonces, una forma eficaz y escalable de calcular uno de estos máximos es mediante algoritmos de optimización de primer orden, que siguen en forma iterativa el gradiente de la función objetivo y se proyectan sobre el conjunto de restricciones $\{\tilde{\boldsymbol{u}} \mid \|\tilde{\boldsymbol{u}}\|_2^2 = 1\}$. Dado que hemos supuesto que cada z_i cumple que $\operatorname{Var}(z_i) = 1$, tenemos que, para cualquier $\tilde{\boldsymbol{u}}$ y para un N suficientemente grande

$$\operatorname{kurt}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}}) \approx \tfrac{1}{N}\|\boldsymbol{X}^\top \tilde{\boldsymbol{u}}\|_4^4 - 3\|\tilde{\boldsymbol{u}}\|_2^4. \tag{2.2.25}$$

De este modo, podemos obtener una aproximación correspondiente al gradiente:

$$\nabla_{\tilde{\boldsymbol{u}}} \operatorname{kurt}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}}) \approx \tfrac{4}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 12\|\tilde{\boldsymbol{u}}\|_2^2 \tilde{\boldsymbol{u}}.$$

El algoritmo FastICA utiliza un método de punto fijo para calcular las direcciones de máxima curtosis. Se parte de las condiciones de optimalidad de primer orden para el problema de maximización de la curtosis, dadas la aproximación del gradiente anterior y el conjunto de restricciones, que son las siguientes:

$$\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} = \underbrace{\left\langle \tilde{\boldsymbol{u}}, \boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} \right\rangle}_{\lambda} \tilde{\boldsymbol{u}}, \tag{2.2.26}$$

donde el valor concreto de λ se determina utilizando la restricción de norma unitaria sobre $\tilde{\boldsymbol{u}}$. En el Ejercicio 2.6 se describen los fundamentos matemáticos necesarios para derivar estas condiciones de optimalidad a partir de los principios básicos. La Ecuación (2.2.26) es satisfecha por *cualquier* punto crítico del problema de maximización de la curtosis; queremos derivar una ecuación que sea satisfecha únicamente por los maximizadores. Tras observar que $\lambda = \|\boldsymbol{X}^\top \tilde{\boldsymbol{u}}\|_4^4$, reexpresamos en forma equivalente la ecuación (2.2.26) como la ecuación modificada

$$\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}} = \left(\frac{\lambda}{N} - 3\right)\tilde{\boldsymbol{u}}, \tag{2.2.27}$$

y constatamos que cualquier maximizador de (2.2.23) debe satisfacer $\lambda/N - 3 > 0$, suponiendo que N sea suficientemente grande. Por lo tanto, podemos *normalizar* ambos lados de (2.2.27), lo que nos da la siguiente ecuación de punto fijo que cumple todo maximizador de (2.2.23):

$$\frac{\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}}}{\left\|\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}}\right\|_2} = \tilde{\boldsymbol{u}}. \tag{2.2.28}$$

Al iterar el mapeo definido por el lado izquierdo de esta expresión de punto fijo, se obtiene el algoritmo FastICA de Hyvärinen y Oja [HO97]:

$$\begin{aligned} \tilde{\boldsymbol{v}}^+ &= \tfrac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}}, \\ \tilde{\boldsymbol{u}}^+ &= \tilde{\boldsymbol{v}}^+ / \left\|\tilde{\boldsymbol{v}}^+\right\|_2 . \end{aligned} \tag{2.2.29}$$

Resulta que el algoritmo FastICA converge con extrema rapidez (de hecho, a una velocidad *cúbica*) hacia las columnas del diccionario $\boldsymbol{U}$; los lectores interesados pueden consultar [HO97] para obtener más detalles. Al comparar el algoritmo FastICA (2.2.29) con el método de potencias estudiado en la Sección 2.1.1 para el problema del PCA y el algoritmo MSP (2.2.18), observamos una sorprendente similitud. De hecho, FastICA es, en esencia, un método de potencias modificado, que utiliza el gradiente de la curtosis empírica en lugar de un simple gradiente lineal del objetivo del PCA.

2.3 Una mezcla de subespacios sobrecompletos de baja dimensión

Como hemos visto, el aprendizaje de diccionarios completos cuenta con una elegante teoría computacional en la que se mantiene una estructura de autocodificación simétrica $\mathcal{E}(\boldsymbol{x}) = \boldsymbol{U}^\top \boldsymbol{x}$, $\mathcal{D}(\boldsymbol{z}) = \boldsymbol{U}\boldsymbol{z}$, junto con un algoritmo escalable similar al método de potencias (el algoritmo MSP) para aprender un diccionario/libro de códigos ortogonal $\boldsymbol{U}$ a partir de los datos. No obstante, para aprender representaciones de distribuciones generales de datos de alta dimensión, es necesario ampliar el tamaño del libro de códigos más allá del requisito de ortogonalidad, lo que significa que debemos tener $\boldsymbol{A} \in \mathbb{R}^{D\times m}$,

con $m \gg D$, lo que corresponde al caso de un diccionario/libro de códigos *sobrecompleto*,[10] y el modelo de señal

$$\boldsymbol{x} = \boldsymbol{A}\boldsymbol{z} + \boldsymbol{\varepsilon}, \quad \|\boldsymbol{z}\|_0 = d \ll m. \tag{2.3.1}$$

Existen motivos tanto geométricos como físicos y de modelización para pasar al caso sobrecompleto. Desde el punto de vista geométrico, recordemos que en nuestra reducción original del modelo de datos de mezcla de subespacios al modelo de diccionario esparso, una mezcla de K subespacios en $\mathbb{R}^D$, cada uno de dimensión d, dio lugar a un diccionario de forma $\boldsymbol{A} \in \mathbb{R}^{D \times Kd}$. En otras palabras, los diccionarios sobrecompletos corresponden a mezclas *más ricas* de subespacios, con más modos distintos de variabilidad para modelar la distribución de datos de alta dimensión. En lo que respecta al modelado, podemos realizar un experimento computacional con datos del mundo real que ponga de manifiesto la capacidad de modelado adicional que aporta una representación sobrecompleta.

Ejemplo 2.2. A partir de una muestra de imágenes de dígitos escritos a mano, la Figura 2.6(a) muestra el resultado de ajustar un diccionario ortogonal al conjunto de datos. Por el contrario, la Figura 2.6(b) muestra el resultado de aplicar un algoritmo de optimización para el aprendizaje de diccionarios sobrecompletos (que presentaremos en detalle más adelante en este capítulo) a estas muestras. Nótese que las representaciones se vuelven mucho más esparsas y los libros de códigos mucho más interpretables: consisten en primitivas fundamentales para los trazos que componen los dígitos, es decir, aristas orientadas. ■

De hecho, el aprendizaje de diccionarios sobrecompletos se propuso originalmente como un algoritmo biológicamente plausible para la representación de imágenes, basado en evidencia empírica sobre cómo las primeras capas de la corteza visual representan los estímulos visuales [OF97; OF96].

En el resto de esta sección, expondremos los fundamentos conceptuales y computacionales del aprendizaje de diccionarios sobrecompletos. Suponiendo que el modelo (2.3.1) se cumple con códigos esparsos $\boldsymbol{z}$, un diccionario sobrecompleto $\boldsymbol{A}$ y un nivel de esparcidad d, y dadas las muestras $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ de $\boldsymbol{x}$, queremos aprender un codificador $\mathcal{E} : \mathbb{R}^D \to \mathbb{R}^m$ que asigne a cada $\boldsymbol{x}$ su código esparso $\boldsymbol{z}$, y un decodificador $\mathcal{D}(\boldsymbol{z}) = \boldsymbol{A}\boldsymbol{z}$ que reconstruya cada $\boldsymbol{x}$ a partir de su código esparso. En forma de diagrama:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\mathcal{D}=\boldsymbol{A}} \hat{\boldsymbol{x}}. \tag{2.3.2}$$

Comenzaremos por la construcción del codificador $\mathcal{E}$. Trabajaremos por etapas: en primer lugar, *dado el diccionario verdadero* $\boldsymbol{A}$, mostraremos cómo el problema de la *codificación esparsa* ofrece un algoritmo elegante, escalable y cuya corrección es demostrable para recuperar el código esparso $\boldsymbol{z}$ de $\boldsymbol{x}$. Aunque

[10] Cambiamos la notación aquí de $\boldsymbol{U}$ a $\boldsymbol{A}$ para enfatizar la no ortogonalidad y la forma no cuadrada del diccionario sobrecompleto $\boldsymbol{A}$.

este problema es NP-difícil en el peor de los casos, puede resolverse de manera eficiente y escalable para diccionarios $\boldsymbol{A}$ que sean *incoherentes*, es decir, que tengan columnas que no estén demasiado correlacionadas. La arquitectura del codificador que resulta de esta solución ya no será simétrica: veremos que tiene la forma de una red profunda primitiva, que depende del diccionario $\boldsymbol{A}$.

A continuación, pasaremos a la tarea de aprender el decodificador $\mathcal{D}$, o lo que es lo mismo, el diccionario sobrecompleto $\boldsymbol{A}$. Derivaremos un algoritmo para el aprendizaje de diccionarios sobrecompletos que nos permita aprender simultáneamente el libro de códigos $\boldsymbol{A}$ y los códigos esparsos $\boldsymbol{z}$, utilizando ideas de la codificación esparsa. Por último, analizaremos una perspectiva más moderna sobre la codificación esparsa aprendible que nos lleva a una estructura de codificador-decodificador totalmente asimétrica, como alternativa a (2.3.2). En este caso, el decodificador corresponderá a una solución incremental al problema del aprendizaje de diccionarios esparsos y dará como resultado un par codificador-decodificador similar a una red profunda para el aprendizaje de diccionarios esparsos. Esta estructura anticipará muchos de los avances que se tratarán en el resto de la monografía, a medida que pasemos de los modelos analíticos a las redes neuronales modernas.

2.3.1 Codificación esparsa con diccionario sobrecompleto

En esta sección, analizaremos el modelo de datos (2.3.1), que da cabida a combinaciones lineales esparsas de numerosos motivos, o *átomos*. Dados los datos $\{\boldsymbol{x}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ que satisfacen este modelo, es decir, que pueden expresarse como

$$\boldsymbol{x}_i = \boldsymbol{A}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i, \qquad \forall i \in [N] \tag{2.3.3}$$

para algún diccionario $\boldsymbol{A} \in \mathbb{R}^{D\times m}$ con m átomos, códigos esparsos $\boldsymbol{z}_i$ tales que $\|\boldsymbol{z}_i\|_0 \leq d$ y pequeños errores $\boldsymbol{\varepsilon}_i$, el problema de la codificación esparsa consiste en recuperar los códigos $\boldsymbol{z}_i$ con la mayor precisión posible a partir de los datos $\boldsymbol{x}_i$, dado el diccionario $\boldsymbol{A}$. Los algoritmos eficientes para resolver este problema tienen éxito cuando el diccionario $\boldsymbol{A}$ es *incoherente* en el sentido de que los productos internos $\boldsymbol{a}_i^\top \boldsymbol{a}_j$ son uniformemente pequeños, por lo que los átomos son casi ortogonales.[11]

Nótese que podemos agrupar los $\boldsymbol{x}_i$ en $\boldsymbol{X} = \big[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\big] \in \mathbb{R}^{D\times N}$, agrupar los $\boldsymbol{z}_i$ en $\boldsymbol{Z} = \big[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N\big] \in \mathbb{R}^{d\times N}$ y agrupar los $\boldsymbol{\varepsilon}_i$ en $\boldsymbol{E} = \big[\boldsymbol{\varepsilon}_1, \ldots, \boldsymbol{\varepsilon}_N\big] \in \mathbb{R}^{D\times N}$, para reescribir (2.3.3) como

$$\boldsymbol{X} = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{E}. \tag{2.3.4}$$

Un enfoque natural para resolver el problema de la codificación esparsa consiste en buscar las señales más esparsas que sean consistentes con nuestras observaciones, lo que nos lleva naturalmente al siguiente problema de optimización:

$$\min_{\tilde{\boldsymbol{Z}}\in\mathbb{R}^{d\times N}} \Big\{ \|\boldsymbol{X} - \boldsymbol{A}\tilde{\boldsymbol{Z}}\|_F^2 + \lambda \|\tilde{\boldsymbol{Z}}\|_1 \Big\}, \tag{2.3.5}$$

[11] Resulta que, en un espacio de alta dimensión, es bastante fácil empaquetar un número de vectores casi ortogonales que es mucho mayor que la dimensión del espacio [WM22].

donde se sabe que la norma ℓ^1 $\|\tilde{\boldsymbol{Z}}\|_1$, tomada elemento por elemento, fomenta la esparcidad de la solución [WM22]. Este problema de optimización se conoce como el problema LASSO.

Sin embargo, a diferencia del PCA o del problema del aprendizaje de diccionarios completos, no existe un algoritmo claro del tipo de iteración de potencias para calcular el $\boldsymbol{Z}^\star$ óptimo. Una alternativa natural es plantearse resolver el problema de optimización anterior con algoritmos del tipo de descenso por gradiente. Sea $\mathcal{L}(\tilde{\boldsymbol{Z}}) = \|\boldsymbol{X} - \boldsymbol{A}\tilde{\boldsymbol{Z}}\|_2^2 + \lambda\|\tilde{\boldsymbol{Z}}\|_1$. En principio, podríamos intentar calcular $\boldsymbol{Z}^\star$ mediante las siguientes iteraciones:

$$\tilde{\boldsymbol{Z}}_{t+1} \leftarrow \tilde{\boldsymbol{Z}}_t + \eta\nabla\mathcal{L}(\tilde{\boldsymbol{Z}}_t). \tag{2.3.6}$$

Sin embargo, dado que el término asociado a la norma ℓ^1 $\|\tilde{\boldsymbol{Z}}\|_1$ no es suave, no podemos limitarnos a aplicar el método del descenso por gradiente. Para este tipo de función, necesitamos sustituir el gradiente $\nabla\mathcal{L}(\tilde{\boldsymbol{Z}})$ por algo que generalice el concepto de gradiente, conocido como subgradiente $\partial\mathcal{L}(\tilde{\boldsymbol{Z}})$:

$$\tilde{\boldsymbol{Z}}_{t+1} \leftarrow \tilde{\boldsymbol{Z}}_t + \eta\partial\mathcal{L}(\tilde{\boldsymbol{Z}}_t). \tag{2.3.7}$$

Sin embargo, se sabe que la convergencia del descenso por subgradiente suele ser muy lenta. Por lo tanto, para este tipo de problemas de optimización, lo habitual es utilizar un algoritmo denominado «descenso de gradiente proximal». Ofrecemos un resumen técnico de este método en la Apéndice A.1.3.

La aplicación del gradiente proximal a la función objetivo de LASSO (2.3.5) da lugar al clásico *algoritmo iterativo de contracción y umbralización* (ISTA), que implementa la iteración

$$\tilde{\boldsymbol{Z}}_1 \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{2.3.8}$$

$$\tilde{\boldsymbol{Z}}_{t+1} = S_{\eta\lambda}\Big(\tilde{\boldsymbol{Z}}_t - 2\eta\boldsymbol{A}^\top(\boldsymbol{A}\tilde{\boldsymbol{Z}}_t - \boldsymbol{X})\Big), \quad \forall t \geq 1, \tag{2.3.9}$$

con un paso $\eta \geq 0$ y el operador de umbralización suave S_α definido para escalares mediante

$$S_\alpha(x) \doteq \begin{cases} x - \alpha, & x \geq \alpha, \\ 0, & -\alpha < x < \alpha, \\ x + \alpha, & x \leq -\alpha \end{cases} \tag{2.3.10}$$

$$= \operatorname{sign}(x)\max\{|x| - \alpha, 0\}, \tag{2.3.11}$$

y se aplica elemento por elemento a la matriz de entrada. Al tratarse de un algoritmo de descenso por gradiente proximal aplicado a un problema convexo, la convergencia hacia un óptimo global está garantizada y es posible derivar directamente una tasa de convergencia precisa [WM22].

Ahora nos detendremos un momento para comentar la forma funcional del operador de actualización en (2.3.9). Adopta la forma de

$$\tilde{\boldsymbol{Z}}_{t+1} = \texttt{nolinealidad}(\tilde{\boldsymbol{Z}}_t + \texttt{lineal}^\top(\texttt{lineal}(\tilde{\boldsymbol{Z}}_t) + \texttt{sesgo})). \tag{2.3.12}$$

Esta forma funcional es muy similar a la de una capa de red residual, es decir,

$$\tilde{\boldsymbol{Z}}_{t+1} = \tilde{\boldsymbol{Z}}_t + \texttt{lineal}_1^\top(\texttt{nolinealidad}(\texttt{lineal}_2(\tilde{\boldsymbol{Z}}_t) + \texttt{sesgo}_1) + \texttt{bias}_2, \quad (2.3.13)$$

simplemente desacoplando las transformaciones lineales (es decir, las multiplicaciones matriciales), añadiendo un sesgo y desplazando la no linealidad. La no linealidad en (2.3.9) es notablemente similar a la función de activación ReLU de uso común $x \mapsto \max\{x, 0\}$ en el aprendizaje profundo —como se ilustra en la Figura 2.7, el operador de umbral suave es como una activación ReLU «con signo», que además se desplaza mediante un sesgo. Por lo tanto, la ISTA puede considerarse como una pasada hacia adelante de una red neuronal primitiva (recurrente de una sola capa). En el Capítulo 5 defendemos que este tipo de operaciones son esenciales para el aprendizaje de representaciones profundas.

2.3.2 Aprendizaje de diccionarios sobrecompletos

Recordemos que tenemos el modelo de datos

$$\boldsymbol{X} = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{E}, \quad (2.3.14)$$

donde $\boldsymbol{Z}$ es un vector esparso y nuestro objetivo anterior era estimar $\boldsymbol{Z}$ a partir de los datos $\boldsymbol{X}$ y los átomos del diccionario $\boldsymbol{A}$. Ahora pasamos al caso más práctico y más difícil, en el que no conocemos ni $\boldsymbol{A}$ ni $\boldsymbol{Z}$ y tratamos de determinarlos a partir de un gran conjunto de datos.

Una generalización directa de la Ecuación (2.3.5) sugiere resolver el problema

$$\min_{\tilde{\boldsymbol{A}}, \tilde{\boldsymbol{Z}}} \left\{ \|\boldsymbol{X} - \tilde{\boldsymbol{A}}\tilde{\boldsymbol{Z}}\|_F^2 + \lambda \|\tilde{\boldsymbol{Z}}\|_1 \right\}. \quad (2.3.15)$$

Sin embargo, el término bilineal en la Ecuación (2.3.15) introduce una ambigüedad de escala que dificulta la convergencia: dado cualquier punto $(\tilde{\boldsymbol{A}}, \tilde{\boldsymbol{Z}})$ y cualquier constante $c > 0$, la sustitución $(c^{-1}\tilde{\boldsymbol{A}}, c\tilde{\boldsymbol{Z}})$ da lugar a un valor de pérdida

$$\|\boldsymbol{X} - \tilde{\boldsymbol{A}}\tilde{\boldsymbol{Z}}\|_F^2 + c\lambda \|\tilde{\boldsymbol{Z}}\|_1. \quad (2.3.16)$$

Es evidente que esta pérdida se minimiza respecto de c al tomar $c \to 0$, lo que equivale a hacer tender el diccionario reescalado $c^{-1}\tilde{\boldsymbol{A}}$ «al infinito». En particular, los iterados de cualquier algoritmo de optimización que resuelva (2.3.15) no convergerán.

Este problema con (2.3.15) se trata añadiendo una restricción a la escala de las columnas del diccionario $\tilde{\boldsymbol{A}}$. Por ejemplo, es habitual suponer que cada columna $\boldsymbol{A}_j$ del diccionario $\boldsymbol{A}$ en (2.3.14) tiene una norma ℓ^2 acotada — digamos, sin pérdida de generalidad, por 1. A continuación, lo imponemos como una restricción, estableciendo el objetivo

$$\min_{\tilde{\boldsymbol{Z}}, \tilde{\boldsymbol{A}} : \|\tilde{\boldsymbol{A}}_j\|_2 \le 1} \left\{ \|\boldsymbol{X} - \tilde{\boldsymbol{A}}\tilde{\boldsymbol{Z}}\|_F^2 + \lambda \|\tilde{\boldsymbol{Z}}\|_1 \right\}. \quad (2.3.17)$$

Los problemas de optimización con restricciones, como el de la ecuación (2.3.17), pueden resolverse mediante una gran variedad de algoritmos sofisticados

[NW06]. Sin embargo, una solución simple y escalable es, de hecho, la que ofrece el mismo algoritmo de descenso por gradiente proximal que utilizamos para resolver el problema de la codificación esparsa en la sección anterior. Podemos codificar cada restricción como un término de regularización adicional, mediante la función característica del conjunto de restricciones—los detalles se exponen en el Ejemplo A.1. La aplicación del descenso por gradiente proximal al problema regularizado resultante es equivalente al *descenso por gradiente proyectado*, en el que, en cada iteración, los iterados, tras dar un paso de descenso por gradiente, se proyectan sobre el conjunto de restricciones.

Nota 2.12 (Maximización de ℓ^4 vs minimización de ℓ^1). Cabe señalar que la formulación del problema anterior se deriva naturalmente de la formulación LASSO (2.3.5) para la codificación esparsa. Promovemos la esparcidad de la solución mediante la norma ℓ^1. No obstante, si solo nos interesa recuperar el diccionario sobrecompleto $\boldsymbol{A}$, el esquema de maximización de ℓ^4 introducido en la Sección 2.2.2 también se generaliza al caso sobrecompleto, sin ninguna modificación significativa. Los lectores interesados pueden consultar el trabajo de [QZL+20a].

El problema anterior (2.3.17), al que denominamos *aprendizaje de diccionarios sobrecompletos*, es no convexo, ya que en este caso tanto $\boldsymbol{A}$ como $\boldsymbol{Z}$ son desconocidos. No se puede resolver fácilmente con las herramientas estándar de optimización convexa. No obstante, dado que es interesante, fácil de formular y de gran relevancia práctica, se han dedicado numerosos trabajos importantes a los distintos algoritmos y análisis teóricos relacionados con este problema. En el marco de este trabajo, presentamos un enfoque idiomático para resolver este problema que se ajusta más al espíritu del aprendizaje profundo.

A partir de nuestra experiencia con el problema LASSO anterior, es fácil ver que, para las dos incógnitas $\boldsymbol{A}$ y $\boldsymbol{Z}$, si fijamos una y optimizamos la otra, cada subproblema es, de hecho, convexo y fácil de resolver. Esto sugiere, naturalmente, que podríamos intentar resolver el programa anterior (2.3.17) minimizando alternativamente con respecto a $\boldsymbol{Z}$ o $\boldsymbol{A}$, por ejemplo, utilizando el descenso por gradiente. Si a esto le sumamos una elección natural de inicialización, se obtiene el siguiente esquema iterativo:

$$\tilde{\boldsymbol{Z}}^{\ell+1} = S_{\eta\lambda}\Big(\tilde{\boldsymbol{Z}}^{\ell} - 2\eta\boldsymbol{A}_+^\top(\boldsymbol{A}_+\tilde{\boldsymbol{Z}}^{\ell} - \boldsymbol{X})\Big), \quad \tilde{\boldsymbol{Z}}^1 = \boldsymbol{0}, \quad \forall \ell \in [L] \tag{2.3.18}$$

$$\boldsymbol{Z}^+ = \tilde{\boldsymbol{Z}}^L, \tag{2.3.19}$$

$$\tilde{\boldsymbol{A}}_{t+1} = \mathrm{proj}_{\|(\,\cdot\,)_j\|_2 \le 1,\, \forall j}\left(\tilde{\boldsymbol{A}}_t - 2\nu(\tilde{\boldsymbol{A}}_t\boldsymbol{Z}^+ - \boldsymbol{X})(\boldsymbol{Z}^+)^\top\right),$$
$$\text{where} \quad (\tilde{\boldsymbol{A}}_1)_j \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \tfrac{1}{D}\boldsymbol{I}), \ \ \forall j \in [m], \qquad \forall t \in [T], \tag{2.3.20}$$

$$\boldsymbol{A}_+ = \tilde{\boldsymbol{A}}_T, \tag{2.3.21}$$

donde la operación de proyección en la actualización de $\boldsymbol{A}$ garantiza que cada columna tenga una norma ℓ^2 máxima no mayor que la unidad, mediante $\boldsymbol{A}_j \mapsto \boldsymbol{A}_j / \max\{\|\boldsymbol{A}_j\|_2, 1\}$ y donde $\boldsymbol{A}_+$ se inicializa con cada columna independiente e idénticamente distribuida $\mathcal{N}(\boldsymbol{0}, \frac{1}{D}\boldsymbol{I})$. Lo anterior consiste en un

«bloque» de minimización alternada y se repiten dichos bloques —cada uno con inicializaciones independientes— hasta alcanzar la convergencia. Arriba, hemos utilizado dos índices distintos, $\{t\}$ y $\{\ell\}$ para indicar las iteraciones. Como veremos más adelante, esto nos permite interpretar las dos actualizaciones por separado en el contexto del aprendizaje profundo.

A pesar de que el problema del aprendizaje de diccionarios es un problema no convexo, se ha demostrado que los algoritmos de minimización alternada convergen efectivamente hacia la solución correcta, al menos localmente. Véase, por ejemplo, [AAJ+16]. A modo de demostración práctica, se utilizó el algoritmo anterior (con $L = T = 1$) para generar los resultados del aprendizaje de diccionarios sobrecompletos que se muestran en la Figura 2.6.

2.3.3 Codificación esparsa profunda aprendida

La idea principal del algoritmo de minimización alternada para el aprendizaje de diccionarios sobrecompletos en la sección anterior (Ecuaciones (2.3.18) y (2.3.20)) es observar que *cuando fijamos* $\boldsymbol{A}$*, la actualización ISTA para* $\boldsymbol{Z}^\ell$ (2.3.18) *se asemeja al paso hacia adelante de una red neuronal profunda con pesos dados por* $\boldsymbol{A}$ *(y* $\boldsymbol{A}^\top$*).* Pero, en general, no conocemos el verdadero $\boldsymbol{A}$, y la estimación actual $\boldsymbol{A}_+$ podría ser errónea. Por lo tanto, es necesario actualizarla aún más utilizando (2.3.20) a partir del residuo obtenido al utilizar la estimación actual de los códigos esparsos $\boldsymbol{Z}^+$ para reconstruir $\boldsymbol{X}$. El algoritmo de minimización alternada repite estos dos procedimientos hasta alcanzar la convergencia. Pero, en su lugar, podemos extrapolar y diseñar otros procedimientos de aprendizaje combinando estas ideas con técnicas del aprendizaje profundo. Esto da lugar a arquitecturas de red más interpretables, lo cual será un tema recurrente a lo largo de este manuscrito.

ISTA aprendido. La interpretación anterior, en términos de redes profundas, de la minimización alternada es más conceptual que práctica, ya que el proceso podría resultar bastante ineficiente y requerir muchas capas o iteraciones para converger. Pero esto se debe principalmente a que intentamos deducir tanto $\boldsymbol{Z}$ como $\boldsymbol{A}$ a partir de $\boldsymbol{X}$. El problema puede simplificarse considerablemente y las iteraciones anteriores pueden hacerse mucho más eficientes en el contexto *supervisado*, donde disponemos de un conjunto de datos de pares de entrada y salida $(\boldsymbol{X}, \boldsymbol{Z})$ distribuidos según (2.3.14) y solo buscamos aprender $\boldsymbol{A}^\ell$ para las iteraciones de codificación esparsa aprendible por capas (2.3.29):[12]

$$\boldsymbol{Z}^{\ell+1} = S_{\eta\lambda}\big(\boldsymbol{Z}^\ell - 2\eta(\boldsymbol{A}^\ell)^\top(\boldsymbol{A}^\ell\boldsymbol{Z}^\ell - \boldsymbol{X})\big), \quad \forall \ell \in [L]. \tag{2.3.22}$$

Si denotamos el operador para cada iteración como $\boldsymbol{Z}^{\ell+1} = f(\boldsymbol{A}^\ell, \boldsymbol{Z}^\ell)$, la iteración anterior puede ilustrarse mediante un diagrama:

$$\boldsymbol{X}, \boldsymbol{Z}^1 \xrightarrow{f(\boldsymbol{A}^1, \cdot)} \boldsymbol{Z}^2 \xrightarrow{f(\boldsymbol{A}^2, \cdot)} \boldsymbol{Z}^3 \xrightarrow{f(\boldsymbol{A}^3, \cdot)} \cdots \boldsymbol{Z}^L \xrightarrow{f(\boldsymbol{A}^L, \cdot)} \boldsymbol{Z}^{L+1} \approx \boldsymbol{Z}.$$

[12] Omitimos los acentos de «tilde» aquí por motivos de concisión, ya que la indexación deja clara la dependencia de la optimización.

Por lo tanto, dada la arquitectura secuencial, para aprender el operador $\boldsymbol{A}^\ell$ en cada capa, resulta totalmente natural hacerlo, por ejemplo, mediante la retropropagación (BP),[13] minimizando el error entre el código final $\boldsymbol{Z}^L$ y la verdad de referencia (ground truth) $\boldsymbol{Z}$:

$$\min_{\{\boldsymbol{A}^\ell\}} \left\| \boldsymbol{Z}^L(\boldsymbol{A}^1, \ldots, \boldsymbol{A}^L) - \boldsymbol{Z} \right\|_2^2. \tag{2.3.23}$$

Esta es la base del algoritmo Learned ISTA (LISTA) [GL10], que puede considerarse como el algoritmo de aprendizaje de una red neuronal profunda, que intenta emular el proceso de codificación esparsa de $\boldsymbol{X}$ a $\boldsymbol{Z}$. En concreto, se puede considerar como un *algoritmo simple de aprendizaje de representaciones*. De hecho, esta misma metodología puede servir de base para comprender las representaciones calculadas en arquitecturas de red más potentes, como los transformers. Desarrollamos estas implicaciones en detalle en el Capítulo 5.

Autocodificadores esparsos. La motivación original del aprendizaje de diccionarios sobrecompletos era proporcionar un modelo generativo sencillo para datos de alta dimensión. Hemos visto con LISTA que, además, los algoritmos iterativos para el aprendizaje de diccionarios sobrecompletos utilizados de manera esparsa ofrecen una interpretación de las redes profundas de tipo ReLU, que generalizaremos en capítulos posteriores a distribuciones de datos más complejas que (2.3.14). Pero también cabe señalar que, incluso en la era moderna de los grandes modelos, el modelo de generación de datos (2.3.14) ofrece una base práctica y útil para *interpretar características en redes profundas preentrenadas a gran escala*, tales como los transformers, siguiendo la hipótesis de que las características (no interpretables, *a priori*) en estas redes consisten en «superposiciones» esparsas de características subyacentes, las cuales son a su vez interpretables [EHO+22b]. Estos paradigmas de aprendizaje *sin supervisión* suelen ser también más eficientes en cuanto al uso de datos que LISTA, que requiere grandes cantidades de pares etiquetados $(\boldsymbol{X}, \boldsymbol{Z})$ para el entrenamiento supervisado.

Podemos utilizar el desarrollo del algoritmo LISTA que acabamos de ver para comprender las prácticas habituales en este campo de investigación. En su aplicación más sencilla (véase [GTT+25; HCS+24]), se recopila un gran número de características de una red profunda preentrenada h a partir de diferentes entradas $\boldsymbol{x}_i$, las cuales se seleccionan en función de la tarea de interpretación deseada.[14] Para simplificar, utilizaremos h para denotar el mapa de características preseleccionado en cuestión, con características de dimensión

[13] Véase la Apéndice A.2 para una breve descripción de la retropropagación.

[14] Por ejemplo, las entradas $\boldsymbol{x}_i$ podrían corresponder a textos que contengan fragmentos de código informático en diferentes lenguajes de programación, siendo nuestra tarea tratar de identificar características interpretables en un mapa de características del transformer h que correspondan a diferentes aspectos destacados de la entrada, tales como el lenguaje de programación específico (distinto entre las «clases» de la entrada) o la necesidad de insertar un paréntesis de cierre en la posición actual (común entre las «clases» de la entrada). Analizamos el uso de redes profundas, y en particular de los transformers, para el aprendizaje de representaciones de texto con mayor detalle en el Capítulo 8.

D; dadas N entradas de muestra, sea $\boldsymbol{H} \in \mathbb{R}^{D\times N}$ la matriz completa de características de h. A continuación, se entrena un autocodificador esparso $f : \mathbb{R}^D \to \mathbb{R}^d$, con un decodificador $g : \mathbb{R}^d \to \mathbb{R}^D$, mediante la función de pérdida LASSO (2.3.5):

$$\min_{f,g} \|\boldsymbol{H} - g(f(\boldsymbol{H}))\|_F^2 + \lambda \|f(\boldsymbol{H})\|_1, \tag{2.3.24}$$

donde el autocodificador esparso f adopta la forma de una red neuronal de una sola capa, es decir, $f(\boldsymbol{h}_i) = \sigma(\boldsymbol{W}_{\text{enc}}(\boldsymbol{h}_i - \boldsymbol{b}) + \boldsymbol{b}_{\text{enc}})$, donde $\sigma(x) = \max\{x, 0\}$ es la función de activación ReLU y el decodificador g es lineal, de modo que $g(\boldsymbol{z}_i) = \boldsymbol{W}_{\text{dec}}\boldsymbol{z} + \boldsymbol{b}$.[15]

El procedimiento de parametrización y entrenamiento (2.3.24) puede parecer inicialmente una aplicación arbitraria del aprendizaje profundo al problema de la codificación esparsa, pero en realidad está muy en consonancia con los algoritmos que hemos estudiado anteriormente para la codificación esparsa por capas con un diccionario aprendido. En particular, recordemos la arquitectura LISTA $\boldsymbol{Z}^L = f(\boldsymbol{A}^L, f(\boldsymbol{A}^{L-1}, \cdots, f(\boldsymbol{A}^1, \boldsymbol{X})\cdots)$. En el caso especial de $L = 2$, tenemos

$$\boldsymbol{Z}^2 = f(\boldsymbol{A}^1, \boldsymbol{X}) = S_{\eta\lambda}\big(\boldsymbol{Z}^1 - 2\eta(\boldsymbol{A}^1)^\top(\boldsymbol{A}^1\boldsymbol{Z}^1 - \boldsymbol{X})\big). \tag{2.3.25}$$

Supongamos que los códigos esparsos $\boldsymbol{Z}$ en cuestión son no negativos, es decir, que $\boldsymbol{Z} \geq \boldsymbol{0}$.[16] Entonces (véase el Ejemplo A.3), podemos considerar una arquitectura LISTA equivalente obtenida a partir del objetivo de codificación esparsa con una restricción adicional de no negatividad sobre $\boldsymbol{Z}$ como

$$\boldsymbol{Z}^2 = f(\boldsymbol{A}^1, \boldsymbol{X}) = \max\left\{\boldsymbol{Z}^1 - 2\eta(\boldsymbol{A}^1)^\top(\boldsymbol{A}^1\boldsymbol{Z}^1 - \boldsymbol{X}) - \lambda\eta\boldsymbol{1}, 0\right\}, \tag{2.3.26}$$

y, tras algunas operaciones algebraicas, expresarlo como

$$\boldsymbol{Z}^2 = f(\boldsymbol{A}^1, \boldsymbol{X}) = \max\left\{2\eta(\boldsymbol{A}^1)^\top + \left(\boldsymbol{Z}^1 - 2\eta(\boldsymbol{A}^1)^\top\boldsymbol{A}^1\boldsymbol{Z}^1 - \lambda\eta\boldsymbol{1}\right), 0\right\}. \tag{2.3.27}$$

Dada la posibilidad de modificar la inicialización del código esparso $\boldsymbol{Z}^1$ como un parámetro aprendible (que, en el marco actual, debe tener todas las columnas iguales al mismo vector aprendible), esto adopta la forma de una red neuronal ReLU con sesgo aprendible, ¡idéntica al autocodificador esparso f! Además, para *decodificar* los códigos esparsos aprendidos $\boldsymbol{Z}^2$, lo más natural es aplicar el diccionario aprendido $\boldsymbol{Z}^2 \mapsto \boldsymbol{A}^1\boldsymbol{Z}^2$. Entonces, la única diferencia entre esto y el decodificador SAE g es el sesgo adicional $\boldsymbol{b}$, que técnicamente puede absorberse en $\boldsymbol{H}$ y f en el objetivo de entrenamiento (2.3.24).

[15] Dado que f es un operador de esparcidad, lo denotamos de manera análoga al f utilizado anteriormente, por ejemplo, en LISTA, donde f denota un algoritmo de optimización para codificar en forma esparsa la entrada. A lo largo del libro, se utilizará f para referirse a esas transformaciones esparsas y estructuradas de otro modo, incluidas las de los *codificadores de redes profundas* (Capítulos 4 y 5).

[16] En el modelo de generación de datos (2.3.14), un par arbitrario de un diccionario y un código esparsos $(\boldsymbol{A}, \boldsymbol{Z})$ puede sustituirse por uno en el que $\boldsymbol{Z} \geq \boldsymbol{0}$ simplemente duplicando el número de columnas en $\boldsymbol{A}$, por lo que, desde el punto de vista del modelado, se trata de una suposición muy moderada.

Por lo tanto, el procedimiento para parametrizar y entrenar SAE coincide con el entrenamiento de LISTA con $L = 1$ y un objetivo de entrenamiento modificado — utilizando el objetivo LASSO (2.3.5), que sigue siendo *no supervisado*, en lugar de la pérdida de reconstrucción supervisada (2.3.23) utilizada en el método LISTA estándar. En concreto, podemos entender la arquitectura SAE a partir de nuestra interpretación de la arquitectura LISTA en términos de codificación esparsa por capa, tal y como se describe en (2.3.29). Esta conexión sugiere una gran variedad de nuevas estrategias de diseño para mejorar la metodología de la interpretabilidad práctica, muchas de las cuales siguen sin haberse explorado, lo cual resulta tentador. Comenzamos a establecer algunas conexiones con la metodología más amplia de la autocodificación en el Capítulo 6.

¿Codificación esparsa aprendida por capa? En el contexto supervisado, LISTA ofrece un análogo de red neuronal profunda de la iteración de codificación esparsa, con diccionarios aprendidos por capas, inspirado en la minimización alternada; incluso en el contexto no supervisado, se puede aplicar la misma metodología al aprendizaje, al igual que con los autocodificadores esparsos. Pero la relación entre los algoritmos de optimización que buscan estructuras de baja dimensión y las arquitecturas de redes profundas es mucho más profunda que eso y sugiere un abanico de arquitecturas de aprendizaje neuronal escalables y naturales que podrían incluso utilizarse sin retropropagación.

A modo de ejemplo sencillo, retomamos las iteraciones de minimización alternadas (2.3.18) y (2.3.20). Este esquema reinicializa aleatoriamente el diccionario $\boldsymbol{A}_1$ en cada una de esas actualizaciones. Una mejora utiliza en su lugar el «inicio en caliente», en el que el residuo se genera utilizando la estimación anterior $\boldsymbol{A}_+$ para el diccionario. Si consideramos entonces cada actualización ISTA (2.3.18) como una capa y permitimos que el diccionario asociado, ahora acoplado a las actualizaciones del código esparso como $\boldsymbol{A}_\ell$, se actualice con el tiempo, esto da lugar a un esquema de codificación esparsa «aprendible por capas»:

$$\boldsymbol{Z}^1 = \boldsymbol{0}, \quad (\boldsymbol{A}_1)_j \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \tfrac{1}{D}\boldsymbol{I}), \ \ \forall j \in [m], \tag{2.3.28}$$

$$\boldsymbol{Z}^{\ell+1} = S_{\eta\lambda}\big(\boldsymbol{Z}^\ell - 2\eta(\boldsymbol{A}_\ell)^\top(\boldsymbol{A}_\ell\boldsymbol{Z}^\ell - \boldsymbol{X})\big), \tag{2.3.29}$$

$$\boldsymbol{A}_{\ell+1} = \boldsymbol{A}_\ell - 2\nu(\boldsymbol{A}_\ell\boldsymbol{Z}^{\ell+1} - \boldsymbol{X})(\boldsymbol{Z}^{\ell+1})^\top. \tag{2.3.30}$$

Téngase en cuenta que esta iteración corresponde a un reetiquetamiento de (2.3.18) y (2.3.20) para $T = L = 1$, sobre un número infinito de bloques. Cada uno de los pasos «internos» que actualizan $\boldsymbol{Z}$ puede considerarse como una pasada hacia adelante de una sola capa, mientras que cada uno de los pasos «externos» que actualizan $\boldsymbol{A}$ puede considerarse como una pasada hacia atrás de una sola capa, de una red neuronal profunda primitiva. En particular, este algoritmo constituye el caso más sencillo en el que se manifiesta una clara distinción entre la optimización hacia adelante y el aprendizaje hacia atrás. Esta distinción entre las actualizaciones hacia adelante de las «características» efímeras $\boldsymbol{Z}$ de los datos $\boldsymbol{X}$ y las actualizaciones hacia atrás de los parámetros

persistentes $\boldsymbol{A}$ sigue observándose en las redes neuronales y los autocodificadores actuales; hablaremos mucho más sobre ello en el Capítulo 5 y el Capítulo 6.

Obsérvese que el esquema por capas anterior también sugiere una alternativa plausible a la actual estrategia de optimización de extremo a extremo, que se basa principalmente en la retropropagación (BP), tal como se detalla en el Apéndice A.2.3. Liberar el entrenamiento de redes grandes de la dependencia de BP sería uno de los mayores retos y oportunidades del futuro, tal y como analizaremos con más detalle al final del libro en el Capítulo 9.

2.4 Resumen y notas

Mensajes clave. Los modelos idealizados que hemos presentado en este capítulo —el PCA, el ICA y el aprendizaje de diccionarios— se desarrollaron a lo largo del Siglo XX. Muchos libros se han dedicado a estos métodos, por lo que aquí solo hemos intentado ofrecer una visión general de los conceptos y resultados clave. Como veremos en los próximos capítulos, aunque podría decirse que se trata de modelos bastante idealizados y sencillos para distribuciones de datos de baja dimensión, las ideas básicas e incluso los algoritmos resultantes para identificar tales distribuciones pueden generalizarse o ampliarse de manera natural a cualquier distribución de baja dimensión: esa es la tesis de este libro. Además, como veremos en los próximos capítulos, estos modelos básicos (lineales) también pueden servir, en gran medida, como prototipos locales para aproximar de manera eficaz distribuciones generales con estructuras no lineales de baja dimensión.

Más antecedentes y referencias. Jolliffe [Jol86] atribuye el análisis de componentes principales a Pearson [Pea01] y, en forma independiente, a Hotelling [Hot33]. En matemáticas, el resultado principal sobre el problema relacionado de la aproximación de bajo rango en normas unitariamente invariantes se atribuye a Eckart y Young [EY36], así como a Mirsky en su forma más general [Mir60]. El PCA sigue desempeñando un papel importante en la investigación, siendo quizás el problema modelo más simple para el aprendizaje no supervisado de representaciones: ya en la década de 1980, trabajos como Oja [Oja82] y Baldi and Hornik [BH89] utilizaron el problema para comprender el aprendizaje en redes neuronales primitivas y, más recientemente, ha servido como herramienta para comprender marcos de aprendizaje de representaciones más complejos, como los modelos de difusión [WZZ+24].

El análisis de componentes independientes fue propuesto por B. Ans et al. [BJC85] y impulsado de manera pionera por Aapo Hyvärinen en la década de 1990 y principios de la década del 2000 en una serie de trabajos de gran influencia: véase Hyvärinen and Oja [HO00b] para un resumen. Como modelo sencillo para la estructura que surge en los datos reales, inicialmente se utilizó ampliamente en aplicaciones como la separación ciega de fuentes, donde cada componente independiente z_i representa una fuente independiente (como el

Tabla 2.1: Resumen de los métodos (generalizados) de potencias presentados en este capítulo.

Problema	Algoritmo	Iteración	Tipo de estructura impuesta
PCA	Método de la potencia	$\boldsymbol{u}_t = \frac{\boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{u}_t}{\|\boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{u}_t\|_2}$	1-dim. subespacio (vector unitario)
ICA	FastICA	$\boldsymbol{u}_{t+1} = \frac{\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{u}_t)^{\odot 3} - 3\boldsymbol{u}_t}{\left\|\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{u}_t)^{\odot 3} - 3\boldsymbol{u}_t\right\|_2}$	1-dim. subespacio (vector unitario)
DL completo	Algoritmo MSP	$\boldsymbol{U}_{t+1} = \mathcal{P}_{\mathrm{O}(D)}[(\boldsymbol{U}_t\boldsymbol{X})^{\odot 3}\boldsymbol{X}^\top]$	D-dim. subespacio (matriz orto.)

sonido asociado a un instrumento distinto en una grabación musical) que se superpone para producir la observación $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z}$.

El problema del aprendizaje de diccionarios puede considerarse, en el caso completo u ortogonal, como uno de los problemas fundamentales del procesamiento de señales del siglo XX, especialmente en la teoría de los sistemas lineales, donde la base de Fourier desempeña un papel clave; a partir de la década de 1980, el campo del análisis armónico computacional se consolidó en torno al estudio de dichos diccionarios alternativos para clases de señales en las que la aproximación óptima solo podía realizarse en una base distinta de la de Fourier (por ejemplo, las ondículas) [DVD+98]. Sin embargo, la importancia del caso de las bases redundantes, o los diccionarios sobrecompletos, solo se puso de relieve tras el trabajo pionero de Olshausen y Field [OF97; OF96]. Los primeros trabajos en este campo establecieron las bases conceptuales y algorítmicas para el aprendizaje de diccionarios sobrecompletos de uso esparso, a menudo destinados a representar imágenes naturales [AEB06; DM03; Don01; EA06; GJB15; MBP14; MK07]. Más tarde, el gran interés teórico que suscitó el problema, como un problema modelo importante y no trivial para el aprendizaje de representaciones sin supervisión, llevó a que fuera estudiado por las comunidades de procesamiento de señales, aprendizaje automático teórico y informática teórica, centrándose en particular en las condiciones bajo las cuales el problema podía resolverse de manera demostrable y eficiente. Una lista no exhaustiva de trabajos destacados en esta línea incluye los de Spielman et al. [SWW12] y Sun et al. [SQW17a] en el caso completo; Arora et al. [AGM+15]; Barak et al. [BKS15]; y Qu et al. [QZL+20a]. Muchas cuestiones teóricas profundas sobre este problema, que es fácil de formular, siguen sin resolverse, quizá en parte debido a una tensión con la dificultad NP del problema en el peor caso (véase, por ejemplo, Tillmann [Til15]).

Un aspecto que deseamos destacar del estudio de estos modelos analíticos clásicos para estructuras de baja dimensión es el papel común que desempeñan los diversos *métodos de potencia generalizados*: algoritmos que convergen muy rápidamente, al menos localmente, hacia diversos tipos de estructuras de baja dimensión. La terminología utilizada para esta clase de algoritmos se basa en el trabajo de M. Journée et al. [MYP+10]. A grandes rasgos, siguiendo el modelo de la iteración clásica de potencias para el cálculo del vector propio principal de

una matriz semidefinida $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, es decir

$$\boldsymbol{u}_{t+1} = \frac{\boldsymbol{A}\boldsymbol{u}_t}{\|\boldsymbol{A}\boldsymbol{u}_t\|_2}, \tag{2.4.1}$$

esta clase de algoritmos consiste en una operación de «potenciación» que involucra una matriz $\boldsymbol{A}$ asociada a los datos, junto con una operación de «proyección» que impone el tipo de estructura deseado. La Tabla 2.1 presenta un resumen de los algoritmos que hemos estudiado en este capítulo y que siguen esta estructura. El lector podrá apreciar la aplicabilidad de esta metodología a diferentes tipos de estructuras de baja dimensión y a diferentes funciones de pérdida (es decir, tanto la pérdida cuadrática del PCA como las pérdidas de tipo curtosis del ICA), así como la ausencia de un algoritmo de este tipo para el aprendizaje de diccionarios sobrecompletos, a pesar de la amplitud de la bibliografía existente sobre estos algoritmos. Consideramos que el desarrollo de métodos de potencias para otras familias de estructuras de baja dimensión, en particular aquellas relevantes para aplicaciones en las que predomina el aprendizaje profundo, constituye una de las cuestiones de investigación más importantes (y aún sin resolver) que se plantean en este capítulo.

La relación que establecemos en la Sección 2.2.1 entre la hipótesis de distribución de la mezcla geométrica de subespacios y la hipótesis del diccionario esparso, más conveniente desde el punto de vista analítico, ya ha sido mencionada en trabajos anteriores, especialmente en aquellos centrados en el análisis de componentes principales generalizado y en aplicaciones como la agrupación de subespacios; véase, por ejemplo, el trabajo de Vidal et al. [VMS16]. La hipótesis de la mezcla de subespacios seguirá desempeñando un papel importante a lo largo de este trabajo, tanto como caso de prueba analítico para diferentes paradigmas algorítmicos y como base para derivar diferentes arquitecturas de redes profundas, tal como ocurre con LISTA en la Sección 2.3.3, pero que puede adaptarse a distribuciones de datos más complejas.

2.5 Ejercicios y ampliaciones

Ejercicio 2.1. Demuestre que, para cualquier matriz simétrica $\boldsymbol{A}$, la solución al problema $\max_{\boldsymbol{U} \in \mathsf{O}(D,d)} \operatorname{tr}\left(\boldsymbol{U}^\top \boldsymbol{A}\boldsymbol{U}\right)$ es la matriz $\boldsymbol{U}^\star$, cuyas columnas son los d primeros vectores propios unitarios de $\boldsymbol{A}$.

Ejercicio 2.2. Sea $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$ una variable aleatoria gaussiana con componentes independientes, cada uno con varianza σ^2. Demuestre que, para cualquier matriz ortogonal $\boldsymbol{Q}$ (es decir, $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$), la variable aleatoria $\boldsymbol{Q}\boldsymbol{z}$ tiene la misma distribución que $\boldsymbol{z}$. *(Pista: recuerde la fórmula de la función de densidad de probabilidad gaussiana y la fórmula de la densidad de una función lineal de una variable aleatoria.)*

Ejercicio 2.3. El concepto —o noción— de identificabilidad estadística analizado anteriormente puede relacionarse con las *simetrías* de la clase de modelos, lo que permite entender la estimación de una manera puramente determinista sin ningún supuesto estadístico.

Consideremos el modelo $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{Z}$ para matrices $\boldsymbol{X}, \boldsymbol{U}, \boldsymbol{Z}$ de tamaños compatibles.

1. Muestre que si $\boldsymbol{A}$ es cualquier matriz cuadrada invertible de tamaño compatible, entonces el par $(\boldsymbol{U}\boldsymbol{A}, \boldsymbol{A}^{-1}\boldsymbol{Z})$ también produce $\boldsymbol{X}$ según el mismo modelo. A esto lo llamamos una *simetría de* $\mathsf{GL}(d)$.

2. Supongamos que cada columna de $\boldsymbol{Z}$ es una observación independiente e identicamente distribuida procedente de un modelo estadístico común $\boldsymbol{z}$, que además tiene media cero y componentes independientes z_i con varianza positiva. Muestre que, para cualquier matriz cuadrada invertible $\boldsymbol{A}$, si $\boldsymbol{A}\boldsymbol{z}$ tiene componentes no correlacionadas, entonces $\boldsymbol{A}$ puede escribirse como $\boldsymbol{D}_1\boldsymbol{Q}\boldsymbol{D}_2$, donde $\boldsymbol{Q}$ es una matriz ortogonal y $\boldsymbol{D}_1$ y $\boldsymbol{D}_2$ son matrices diagonales. *Esto vincula la suposición de «independencia» en el ICA con un efecto de «ruptura de simetría», que solo permite simetrías de escala y rotacionales.*

Ejercicio 2.4. Consideremos el modelo $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z}$, donde $\boldsymbol{U} \in \mathbb{R}^{D\times d}$ con $D \geq d$ es fijo y tiene rango d, mientras que $\boldsymbol{z}$ es una variable aleatoria de media cero. Sean $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ observaciones i.i.d. de este modelo.

1. Muestre que la matriz $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ tiene un rango no mayor que d y que por lo tanto existe una matriz ortonormal $\boldsymbol{V} \in \mathbb{R}^{D\times d}$ tal que $\boldsymbol{X} = \boldsymbol{V}\boldsymbol{Y}$, donde $\boldsymbol{Y} \in \mathbb{R}^{d\times N}$. (*Pista: usar el PCA.*)

2. Demuestre que la *matriz blanqueada* $(\boldsymbol{Y}\boldsymbol{Y}^\top)^{-1/2}\boldsymbol{Y}$ existe en media siempre que $\mathrm{Cov}(\boldsymbol{z})$ sea no singular y que tiene una covarianza empírica igual a la matriz identidad.[17]

3. Muestre, utilizando la descomposición en valores singulares de $\boldsymbol{U}$, que la matriz $\boldsymbol{V}$ puede elegirse de tal manera que la matriz blanqueada satisfaga $(\boldsymbol{Y}\boldsymbol{Y}^\top)^{-1/2}\boldsymbol{Y} = \boldsymbol{W}[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$, donde $\boldsymbol{W}$ es una matriz ortonormal.

Ejercicio 2.5. Sean X e Y variables aleatorias independientes de media cero.

1. Demuestre que $\mathrm{kurt}(X + Y) = \mathrm{kurt}(X) + \mathrm{kurt}(Y)$.

2. Para cualquier $\alpha \in \mathbb{R}$, demuestre que $\mathrm{kurt}(\alpha X) = \alpha^4 \,\mathrm{kurt}(X)$.

Ejercicio 2.6. Sea $f : \mathbb{R}^d \to \mathbb{R}$ una función objetivo dada, dos veces continuamente diferenciable. Consideremos el problema de optimización con restricciones esféricas

$$\max_{\|\boldsymbol{u}\|_2^2=1} f(\boldsymbol{u}). \tag{2.5.1}$$

En este ejercicio, derivaremos las expresiones que proporcionamos en la derivación de FastICA para maximizar la curtosis sobre la esfera mediante un

[17] En particular, se puede demostrar matemáticamente que esto basta para garantizar que la matriz blanqueada exista con alta probabilidad siempre que $\boldsymbol{z}$ satisfaga una desigualdad de concentración adecuada y N sea suficientemente grande.

algoritmo de ascenso por gradiente. Estas expresiones son casos especiales de una amplia teoría del cálculo y la optimización en variedades, de las cuales la esfera es un ejemplo concreto. Un estudio técnico en profundidad de este campo queda fuera del alcance de nuestro trabajo, por lo que solo mencionamos dos referencias clave para el lector interesado: el libro de texto pionero de Absil, Mahony y Sepulchre [AMS09] y un tratado introductorio más reciente de Boumal [Bou23].

1. Para cualquier conjunto de restricciones $\mathcal{M}$ que sea una subvariedad diferenciable de $\mathbb{R}^d$, el *espacio tangente* en un punto $\boldsymbol{u} \in \mathcal{M}$ es, en términos informales, la mejor aproximación lineal local a la variedad $\mathcal{M}$ en el punto $\boldsymbol{u}$. En el importante caso especial en el que $\mathcal{M}$ se define localmente en $\boldsymbol{u}$ como un conjunto de nivel de una función $F : \mathbb{R}^d \to \mathbb{R}$, es decir

$$U \cap \mathcal{M} = F^{-1}(\{0\})$$

para un conjunto abierto $U \subset \mathcal{M}$ tal que $\boldsymbol{u} \in U$, el espacio tangente a $\mathcal{M}$ en $\boldsymbol{u}$ se puede calcular mediante la derivación:

$$T_{\boldsymbol{u}}\mathcal{M} = \operatorname{Ker}(DF_{\boldsymbol{u}}).$$

Es fácil ver, que la esfera tiene la ecuación definitoria $F(\boldsymbol{u}) = \|\boldsymbol{u}\|_2^2 - 1$. Demuestre, utilizando estos datos, que el espacio tangente a la esfera en $\boldsymbol{u}$ viene dado por

$$T_{\boldsymbol{u}}\mathbb{S}^{d-1} = \{\boldsymbol{v} \in \mathbb{R}^d \mid \langle \boldsymbol{v}, \boldsymbol{u} \rangle = 0\},$$

y que la proyección ortogonal sobre este subespacio es $\boldsymbol{P}_{\boldsymbol{u}}^{\perp} = \boldsymbol{I} - \boldsymbol{u}\boldsymbol{u}^{\top}$.

2. El campo vectorial

$$\operatorname{grad} f(\boldsymbol{u}) = \boldsymbol{P}_{\boldsymbol{u}}^{\perp} \nabla f \tag{2.5.2}$$

se conoce como el *gradiente riemanniano* de la función f restringida a la esfera. Las *condiciones de optimalidad de primer orden* para el problema de optimización (2.5.1) pueden expresarse en términos del gradiente riemanniano:

$$\operatorname{grad} f(\boldsymbol{u}) = \boldsymbol{0}.$$

Desde el punto de vista geométrico, esto significa que el gradiente euclidiano de f en $\boldsymbol{u}$ debe ser ortogonal al espacio tangente a la esfera en $\boldsymbol{u}$. Supongamos ahora que $\boldsymbol{v} \in \mathbb{R}^d$ es distinto de cero. Demuestre que

$$\operatorname{proj}_{\mathbb{S}^{d-1}}(\boldsymbol{v}) \doteq \min_{\|\boldsymbol{u}\|_2^2 = 1} \|\boldsymbol{u} - \boldsymbol{v}\|_2 = \frac{\boldsymbol{v}}{\|\boldsymbol{v}\|_2},$$

utilizando las condiciones de optimalidad de primer orden.

3. Al optimizar en $\mathbb{R}^d$, se comprueban las condiciones de optimalidad de segundo orden (para determinar si un punto crítico es un punto máximo, un punto mínimo o un punto de silla) utilizando la *matriz hessiana* $\nabla^2 f(\boldsymbol{u})$. Muestre, derivando el gradiente riemanniano $\operatorname{grad} f(\boldsymbol{u})$ para la esfera con

respecto a $\boldsymbol{u}$, tal como se hizo en la primera parte de este ejercicio, que el objeto correspondiente para determinar las condiciones de optimalidad de segundo orden en la optimización con restricciones esféricas es el *hessiano riemanniano*, definido como

$$\operatorname{Hess} f(\boldsymbol{u}) = \boldsymbol{P}_{\boldsymbol{u}}^{\perp} \left(\nabla^2 f(\boldsymbol{u}) - \langle \nabla f(\boldsymbol{u}), \boldsymbol{u} \rangle \boldsymbol{I} \right) \boldsymbol{P}_{\boldsymbol{u}}^{\perp}. \tag{2.5.3}$$

Ejercicio 2.7. En este ejercicio, esbozamos un argumento conocido en la literatura como *análisis de paisaje* para el problema de maximización de la curtosis de una población con restricción esférica (2.2.24). Demostraremos que, cuando hay al menos un componente independiente con curtosis positiva, sus maximizadores globales conducen efectivamente a la recuperación de una columna del diccionario $\boldsymbol{U}$. Para simplificar, supondremos que $\operatorname{kurt}(z_i) \neq 0$ para cada $i = 1, \ldots, d$.

1. Utilizando los resultados de la Parte 1 del Ejercicio 2.6, demuestre que la condición de optimalidad de primer orden para (2.2.24) es

$$\left(\sum_{i=1}^{d} \operatorname{kurt}(z_i) w_i^4 \right) \boldsymbol{w} = \operatorname{kurt}(\boldsymbol{z}) \odot \boldsymbol{w}^{\odot 3}, \tag{2.5.4}$$

   donde la curtosis se calcula elemento por elemento, $\odot$ denota la multiplicación elemento por elemento de vectores y $\boldsymbol{w}^{\odot 3}$ denota el cubo elemento por elemento de su argumento.

2. Demuestra que los vectores $\boldsymbol{w}$ de norma unitaria que además satisfacen (2.5.4) adoptan todos la siguiente forma. Sea $S^+ = \{i \in [d] \mid \operatorname{kurt}(z_i) > 0\}$ y $S^- = \{i \in [d] \mid \operatorname{kurt}(z_i) < 0\}$. Sea S un subconjunto de S^+ o de S^-. Entonces

$$\boldsymbol{w}_S = \sum_{i \in S} \pm \sqrt{\frac{1}{\operatorname{kurt}(z_i) \sum_{j \in S} \frac{1}{\operatorname{kurt}(z_j)}}} \boldsymbol{e}_i \tag{2.5.5}$$

   satisface (2.5.4), donde $\boldsymbol{e}_i$ es el vector que contiene un 1 en la posición i y 0 en todas las demás y el signo $\pm$ indica la elección de un signo positivo o negativo.

3. Supongamos que existe al menos un i tal que $\operatorname{kurt}(z_i) > 0$. Utilizando los resultados de la Parte 2 del Ejercicio 2.6, demuestre que los únicos máximos locales de la función objetivo de (2.2.24) son los vectores con un solo elemento distinto de cero y signo $\pm \boldsymbol{e}_i$, donde $i \in S^+$. Se concluye que los maximizadores globales de (2.2.24) son los vectores con un solo elemento distinto de cero y signo, correspondientes a las componentes con máxima curtosis. (*Pista: cuente el número de valores propios positivos y negativos de la matriz hessiana riemanniana* (2.5.3) *en cada punto crítico.*)

4. Supongamos ahora que $\operatorname{kurt}(z_j) < 0$ para todo $j = 1, \ldots, d$. Esto corresponde a una instanciación «sobredeflacionada» del problema de maximización de la

curtosis. Utilizando de nuevo los resultados de la Parte 2 del Ejercicio 2.6, muestre que los únicos máximos locales de la función objetivo de (2.2.24) son los vectores densos con signo $\sum_{i=1}^{d} \pm \boldsymbol{e}_i$. Esto demuestra que la formulación de optimización (2.2.24) no puede aplicarse sin más en forma ingenua.

Ejercicio 2.8. Este ejercicio sigue la estructura y el formalismo presentados en el Ejercicio 2.6, pero los aplica en este caso al grupo ortogonal $\mathsf{O}(d) = \{\boldsymbol{U} \in \mathbb{R}^{d\times d} \mid \boldsymbol{U}^\top \boldsymbol{U} = \boldsymbol{I}\}$. Consulte la descripción del Ejercicio 2.6 para obtener los fundamentos conceptuales necesarios; el formalismo se aplica de manera idéntica al caso en el que el espacio envolvente es el conjunto de matrices $d \times d$, siempre y cuando se tenga en cuenta que el producto interno pertinente para matrices es $\langle \boldsymbol{X}, \boldsymbol{Y} \rangle = \operatorname{tr}(\boldsymbol{X}^\top \boldsymbol{Y})$. Una excelente referencia general sobre la optimización en el grupo ortogonal es la obra de Edelman, Arias y Smith [EAS98].

Sea $f : \mathbb{R}^{d\times d} \to \mathbb{R}$ una función objetivo dada que sea dos veces continuamente diferenciable. Consideremos el problema de optimización con restricciones ortogonales

$$\max_{\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}} f(\boldsymbol{Q}). \tag{2.5.6}$$

1. Es fácil ver que el grupo ortogonal tiene la ecuación definitoria $F(\boldsymbol{Q}) = \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$. Demuestre, a partir de este hecho, que el espacio tangente al grupo ortogonal en $\boldsymbol{Q}$ viene dado por

$$T_{\boldsymbol{Q}}\mathsf{O}(d) = \{\boldsymbol{Q}\boldsymbol{\Omega} \in \mathbb{R}^{d\times d} \mid \boldsymbol{\Omega}^\top = -\boldsymbol{\Omega}\},$$

y que la proyección ortogonal sobre este subespacio es

$$\mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)}(\boldsymbol{\Delta}) = \boldsymbol{Q}\operatorname{skew}(\boldsymbol{Q}^\top \boldsymbol{\Delta}),$$

donde $\operatorname{skew}(\boldsymbol{\Delta}) = \frac{1}{2}(\boldsymbol{\Delta} - \boldsymbol{\Delta}^\top)$ es la proyección ortogonal sobre el conjunto de matrices antisimétricas. El campo vectorial

$$\operatorname{grad} f(\boldsymbol{Q}) = \mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)} \left(\nabla f(\boldsymbol{Q})\right) \tag{2.5.7}$$

se conoce como el *gradiente riemanniano* de la función f restringida al grupo ortogonal. Las *condiciones de optimalidad de primer orden* para el problema de optimización (2.5.6) pueden expresarse en términos del gradiente riemanniano:

$$\operatorname{grad} f(\boldsymbol{Q}) = \boldsymbol{0}.$$

2. Demuestre, derivando el gradiente riemanniano $\operatorname{grad} f(\boldsymbol{Q})$ para el grupo ortogonal con respecto a $\boldsymbol{Q}$, tal como se hizo en la primera parte de este ejercicio, que la *Hessiana riemanniana* viene dada por

$$\operatorname{Hess} f(\boldsymbol{Q}) = \mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)} \left(\nabla^2 f(\boldsymbol{Q}) - \operatorname{sym}(\boldsymbol{Q}^\top \nabla f(\boldsymbol{Q})) \otimes \boldsymbol{I}\right) \mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)}, \tag{2.5.8}$$

donde $\operatorname{sym}(\boldsymbol{\Delta}) = \frac{1}{2}(\boldsymbol{\Delta} + \boldsymbol{\Delta}^\top)$ denota la proyección ortogonal sobre el conjunto de matrices simétricas y $\otimes$ denota el producto de Kronecker de matrices. Tenga cuidado de interpretar los operadores que aparecen en la expresión

anterior como *transformaciones lineales sobre matrices* $d \times d$, **y no** como matrices $d \times d$ en sí mismas. Las *condiciones de optimalidad de segundo orden* para el problema de optimización (2.5.6) pueden expresarse en términos de la hessiana riemanniana:

$$\operatorname{Hess} f(\boldsymbol{Q}) \preceq \mathbf{0}.$$

En un problema de minimización, el signo se invierte.

(*Pista: La clave está en manipular los cálculos para obtener la forma* (2.5.8)*, que es la más compacta posible. Para ello, utiliza el siguiente isomorfismo del producto de Kronecker: si* $\boldsymbol{A}$*,* $\boldsymbol{X}$ *y* $\boldsymbol{B}$ *son matrices de tamaños compatibles, entonces se tiene que*

$$(\boldsymbol{B}^\top \otimes \boldsymbol{A}) \operatorname{vec}(\boldsymbol{X}) = \operatorname{vec}(\boldsymbol{A}\boldsymbol{X}\boldsymbol{B}),$$

donde $\operatorname{vec}(\boldsymbol{X})$ *denota el «apilamiento de izquierda a derecha» de las columnas del argumento matricial en un vector. Utilizamos este isomorfismo en* (2.5.8) *con el fin de definir el producto de Kronecker de dos matrices como un operador sobre matrices en forma canónica.*)

3. Supongamos ahora que $\boldsymbol{X} \in \mathbb{R}^{d \times d}$ es de rango completo. En esta y la siguiente parte del ejercicio, consideramos la proyección de $\boldsymbol{X}$ sobre el grupo ortogonal:

$$\operatorname{proj}_{\mathsf{O}(d)}(\boldsymbol{X}) \doteq \min_{\boldsymbol{Q} \in \mathsf{O}(d)} \|\boldsymbol{Q} - \boldsymbol{X}\|_F^2. \tag{2.5.9}$$

Demostraremos que la solución a este problema está dada por

$$\operatorname{proj}_{\mathsf{O}(d)}(\boldsymbol{X}) = \boldsymbol{U}\boldsymbol{V}^\top,$$

donde $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top$ es una descomposición en valores singulares de $\boldsymbol{X}$.

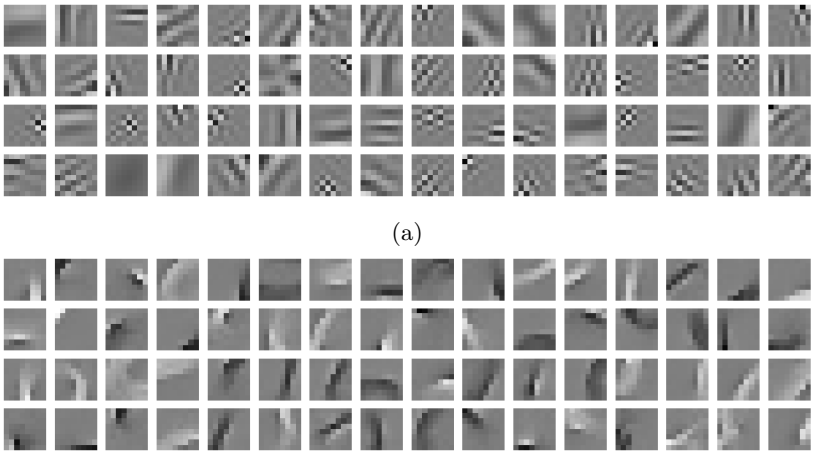
1. Utilizando las condiciones de optimalidad de primer y segundo orden, demuestre que todo minimizador local $\boldsymbol{Q}$ de (2.5.9) satisface

$$\begin{aligned} \left(\boldsymbol{Q}^\top \boldsymbol{X}\right)^\top &= \boldsymbol{Q}^\top \boldsymbol{X}, \\ \boldsymbol{Q}^\top \boldsymbol{X} &\succeq \mathbf{0}. \end{aligned}$$

(*Pista: utilizar la linealidad del producto de Kronecker en cualquiera de sus dos argumentos cuando el otro es fijo.*)

2. A partir de estas condiciones, muestre que para todo mínimo local $\boldsymbol{Q}$ de (2.5.9), se cumple que $\boldsymbol{Q}^\top \boldsymbol{X} = (\boldsymbol{X}^\top \boldsymbol{X})^{1/2}$. (*Pista: Utilice el siguiente resultado clásico de álgebra lineal: si* $\boldsymbol{S} \succeq \mathbf{0}$ *es una matriz simétrica semidefinida positiva, entonces* $(\boldsymbol{S}^\top \boldsymbol{S})^{1/2} = \boldsymbol{S}$.)
3. Utilizando la descomposición en valores singulares $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top$, concluya que

$$\boldsymbol{U}\boldsymbol{V}^\top = \operatorname{proj}_{\mathsf{O}(d)}(\boldsymbol{X}).$$



(a)

(b)

Figura 2.6: Comparación de los átomos de diccionario aprendidos para diccionarios completos (ortogonales) y sobrecompletos, entrenados para reconstruir fragmentos de 8×8 tomados de los dígitos del MNIST. Ambos diccionarios se entrenan durante 6000 épocas en 10 000 fragmentos aleatorios con contenido no trivial y los códigos esparsos se calculan con el objetivo LASSO y $\lambda = 0.1$ (véase (2.3.5)). Las escalas de colores utilizan el negro para los valores negativos y el blanco para los valores positivos. **En la parte superior:** Un diccionario ortogonal aprendido con el algoritmo MSP (2.2.18) está limitado a tener un máximo de 64 átomos; Los átomos aprendidos se corresponden aproximadamente con un diccionario de tipo «pico y placa», y alcanzan niveles relativamente bajos de esparcidad en la reconstrucción en los datos de prueba no utilizados (los códigos son aproximadamente 17-esparsos en promedio, con respecto a un umbral de 10^{-1}). **En la parte inferior:** Por el contrario, un diccionario sobrecompleto (en este caso, con 8^3 átomos; visualizamos un subconjunto aleatorio de 64) aprende átomos de diccionario semánticamente significativos correspondientes a aristas orientadas con signo, que pueden ensamblarse para crear fragmentos de dígitos y lograr niveles superiores de reconstrucción y esparcidad. Los códigos son aproximadamente 20-esparsos en promedio, aunque son 8 veces más grandes que los del diccionario ortogonal. Para calcular el diccionario, utilizamos un optimizador basado en la minimización linealizada alternante proximal sobre una versión adecuadamente regularizada de (2.3.17).

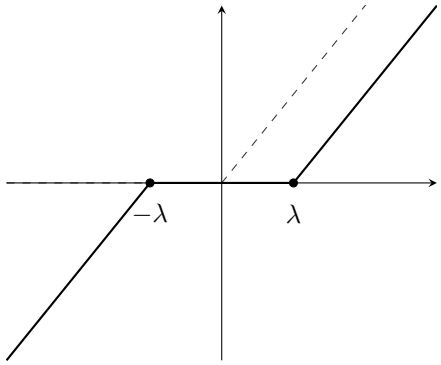


Figura 2.7: Gráfico de la función de umbralización suave S_λ (línea continua) comparada con la función de activación ReLU $x \mapsto \max\{x, 0\}$ (línea de trazos). Entre $-\lambda$ y $+\lambda$, el valor de S_λ se recorta a 0, mientras que el valor (absoluto) de las entradas grandes simplemente se reduce en λ. A diferencia de ReLU, S_λ es simétrica y actúa tanto sobre las entradas positivas como sobre las negativas.

Capítulo 3

Búsqueda de distribuciones de baja dimensión mediante eliminación de ruido

"*La información es la resolución de la incertidumbre.*"

—Claude Shannon, 1948

En el Capítulo 2, mostramos cómo se aprenden clases simples de distribuciones cuyos soportes se supone que son un único subespacio de baja dimensión, una mezcla de tales subespacios o gaussianas de bajo rango. Para simplificar, se considera que los diferentes modos lineales o gaussianos (ocultos) son ortogonales o independientes,[1] tal y como se ilustra en la Figura 2.1. Para estas distribuciones especiales, se pueden derivar algoritmos de aprendizaje sencillos y eficaces con garantías de corrección y eficiencia, siendo claras las interpretaciones geométricas y estadísticas de las operaciones algorítmicas asociadas.

En la práctica, tanto la linealidad como la independencia son supuestos idealizados que las distribuciones de datos reales de alta dimensión *rara vez* cumplen. Lo único que podemos suponer es que la dimensión intrínseca de la distribución es muy baja en comparación con la dimensión del espacio ambiente en el que los datos están incrustados. Por lo tanto, en este capítulo mostramos cómo aprender una clase más general de distribuciones de baja dimensión en un espacio de alta dimensión que *no* es necesariamente (ni por tramos) lineal.

Las distribuciones del mundo real suelen contener múltiples componentes o modos, por ejemplo, correspondientes a diferentes clases de objetos en las imágenes. Es posible que estos modos no sean estadísticamente independientes e incluso puedan tener dimensiones intrínsecas diferentes. Por lo tanto,

[1] O pueden reducirse a tales casos idealizados en condiciones favorables.

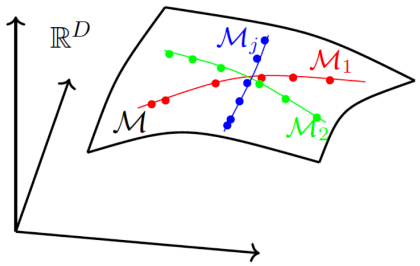


Figura 3.1: Los datos $\boldsymbol{X}$ se distribuyen sobre una mezcla de subvariedades de baja dimensión $\cup_j \mathcal{M}_j$ en un espacio envolvente de muy alta dimensión, por ejemplo, $\mathbb{R}^D$. Este es un caso mucho más general que los casos especiales estudiados en el Capítulo 2, donde se suponía que las subvariedades eran lineales por tramos y, por lo tanto, admitían formas paramétricas y soluciones analíticas.

generalmente suponemos que los datos se distribuyen sobre una mezcla de subvariedades no lineales de baja dimensión incrustadas en un espacio de alta dimensión; la Figura 3.1 ilustra este tipo de distribución. Otra limitación es que, en situaciones del mundo real, normalmente solo tenemos acceso a un número finito de muestras de la distribución de datos, en lugar de disponer de un conocimiento previo de la distribución exacta en sí.

Para aprender dicha distribución bajo estas condiciones, debemos abordar tres preguntas fundamentales:

- ¿Cuál es un enfoque general para aprender y representar una distribución de baja dimensión en un espacio de alta dimensión?
- ¿Cómo medimos la complejidad de la distribución para poder aprovechar eficazmente su baja dimensionalidad en el aprendizaje?
- ¿Cómo podemos hacer que el proceso de aprendizaje sea computacionalmente manejable y escalable, especialmente cuando la dimensión del entorno es elevada y el tamaño de la muestra es grande?

Como veremos, la idea fundamental de la *compresión*, o *reducción de dimensiones*, cuya eficacia ya hemos demostrado en el caso lineal/independiente, sigue siendo un principio rector para el desarrollo de modelos y métodos computacionales para distribuciones no lineales generales de baja dimensión.

Dada su importancia teórica y práctica, analizamos con mayor detalle cómo se materializa este marco basado en la compresión cuando la distribución de interés puede modelarse o aproximarse adecuadamente mediante una mezcla de subespacios de baja dimensión o gaussianas de bajo rango. Como se muestra en los capítulos siguientes, estas clases de modelos, aparentemente restrictivas, sirven como bloques básicos para la construcción de algoritmos que realizan la compresión en distribuciones generales.

3.1 Minimización de la entropía y compresión

3.1.1 Entropía y tasas de codificación

En el Capítulo 1 mencionamos que el objetivo del aprendizaje es encontrar la forma más sencilla de generar un conjunto de datos dado. En términos conceptuales, la complejidad de Kolmogorov fue concebida para proporcionar una medida de complejidad de este tipo, pero no es computable y no está asociada a ningún esquema implementable que pueda reproducir realmente los datos. Por lo tanto, necesitamos una medida de complejidad alternativa, computable y aplicable. Esto nos lleva al concepto de *entropía*, introducido por Shannon en 1948 [Sha48].

Para ilustrar el carácter constructivo de la entropía, comencemos con el caso más sencillo. Supongamos que tenemos una variable aleatoria discreta que toma N valores distintos, o *tokens*, $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ con una probabilidad uniforme de $1/N$. Podemos codificar cada token $\boldsymbol{x}_i$ utilizando la representación binaria de $\log_2 N$ bits del número i. Este esquema de codificación puede generalizarse para codificar distribuciones discretas arbitrarias [CT91]: dada una distribución p tal que $\sum_{i=1}^{N} p(\boldsymbol{x}_i) = 1$, se puede asignar a cada token $\boldsymbol{x}_i$ con probabilidad de $p(\boldsymbol{x}_i)$ un código binario de longitud $\log_2[1/p(\boldsymbol{x}_i)] = -\log_2 p(\boldsymbol{x}_i)$ bits. Por lo tanto, el número promedio de bits, o *la tasa de codificación*, necesario para codificar cualquier muestra de la distribución $p(\cdot)$ viene dado por la expresión[2]:

$$H(p) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})] = -\sum_{i=1}^{N} p(\boldsymbol{x}_i) \log p(\boldsymbol{x}_i). \tag{3.1.1}$$

Esto se conoce como la *entropía* de la distribución (discreta) $p(\cdot)$. Obsérvese que esta entropía es siempre no negativa y es cero si y solo si $p(\boldsymbol{x}_i) = 1$ para algún $\boldsymbol{x}_i$ con $i \in [N]$.[3] Para una variable aleatoria $\boldsymbol{x}$, solemos escribir $H(\boldsymbol{x})$ para denotar la entropía de la distribución marginal de $\boldsymbol{x}$.

3.1.2 Entropía diferencial

Cuando la variable aleatoria $\boldsymbol{x} \in \mathbb{R}^D$ es continua y tiene una densidad de probabilidad p, se puede interpretar dicho límite (3.1.1) como relacionado con una integral:

$$h(p) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})] = -\int_{\mathbb{R}^D} p(\boldsymbol{\xi}) \log p(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi}. \tag{3.1.2}$$

Más precisamente, dada una distribución de probabilidad continua p en $\mathbb{R}^D$, podemos discretizar $\mathbb{R}^D$ en una unión de cubos disjuntos C_i^ϵ de longitud de arista $\epsilon^{1/D}$, cada uno (digamos) centrado en $\boldsymbol{x}_i$, y definir la distribución discretizada

[2] Según la convención de la teoría de la información [CT91], el log aquí tiene base 2. Por lo tanto, la entropía se mide en bits (binarios).

[3] Aquí utilizamos el hecho de que $\lim_{p\to 0} p \log p = 0$.

p^ϵ mediante $p^\epsilon(\boldsymbol{x}_i) = \mathbb{P}_{\boldsymbol{x}\sim p}[\boldsymbol{x} \in C_i^\epsilon]$. Entonces se puede demostrar que $H(p^\epsilon) + \log \epsilon \approx h(p)$. Tomando ϵ suficientemente pequeño, observamos que la entropía diferencial $h(p)$ puede ser negativa. Los lectores interesados pueden consultar [CT91] para obtener una explicación más detallada. Al igual que con la entropía discreta, a menudo escribiremos $h(\boldsymbol{x})$ para referirnos a la entropía de la distribución (marginal) de $\boldsymbol{x}$.

Ejemplo 3.1 (Entropía de las distribuciones gaussianas). Mediante un cálculo directo, se puede mostrar que la entropía de una variable aleatoria gaussiana $x \sim \mathcal{N}(\mu, \sigma^2)$ es

$$h(x) = \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2}. \tag{3.1.3}$$

La entropía de una variable aleatoria gaussiana multivariante $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ en $\mathbb{R}^D$ es

$$h(\boldsymbol{x}) = \frac{D}{2}(1 + \log(2\pi)) + \frac{1}{2}\operatorname{logdet}(\boldsymbol{\Sigma}). \tag{3.1.4}$$

■

Al igual que ocurre con la entropía de una distribución discreta, nos gustaría que la entropía diferencial estuviera relacionada con la tasa de codificación de algún esquema de codificación viable. Por ejemplo, como se mencionó anteriormente, si discretizamos el dominio de la distribución con una malla de tamaño $\epsilon > 0$ con centros $\boldsymbol{x}_i$, un posible esquema de codificación consiste en asignar cada $\boldsymbol{x}$ a su centro más cercano $\boldsymbol{x}_i$ y la entropía de la distribución discreta resultante se aproxima a la entropía diferencial [CT91].

Existen ciertas salvedades asociadas a la definición de la entropía diferencial. En el caso de una distribución en un espacio de alta dimensión, cuando su soporte se vuelve degenerado (de baja dimensión), su entropía diferencial diverge hacia $-\infty$. Este hecho se demuestra en el Teorema B.1, pero incluso en el caso explícito y sencillo de las distribuciones gaussianas (3.1.4), cuando la covarianza $\boldsymbol{\Sigma}$ es singular, podemos ver que $\operatorname{logdet}(\boldsymbol{\Sigma}) = -\infty$, por lo que tenemos $h(\boldsymbol{x}) = -\infty$ (Nota: Según el Teorema B.1, las distribuciones gaussianas tienen la mayor entropía entre todas las distribuciones con la misma media y covarianza). En una situación así, no resulta evidente cómo cuantificar o codificar adecuadamente dicha distribución. Sin embargo, las distribuciones (gaussianas) degeneradas son precisamente los casos más sencillos posibles, y podría decirse que los más importantes, de distribuciones de baja dimensión en un espacio de alta dimensión. En el próximo capítulo, abordaremos una solución completa a esta aparente dificultad relacionada con la degeneración.

3.1.3 Minimización de la tasa de codificación

El problema de aprendizaje consiste en recuperar una distribución (potencialmente continua) $p(\boldsymbol{x})$ a partir de muestras $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ extraídas de ella. Para facilitar la exposición, escribimos $\boldsymbol{X} = \big[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\big] \in \mathbb{R}^{D\times N}$. Dado que las distribuciones de interés son (casi) de baja dimensión, esperamos que su entropía (diferencial) sea muy pequeña. Sin embargo, a diferencia de

las situaciones estudiadas en el capítulo anterior, desconocemos la familia de modelos (analíticos) de baja dimensión a la que pertenece $p(\boldsymbol{x})$. Por lo tanto, comprobar si la entropía es baja parece ser la única pauta en la que podemos basarnos para identificar y modelar la distribución.

Ahora bien, si solo se tienen las muestras sin saber cuáles $p(\boldsymbol{x})$, en teoría podrían interpretarse como muestras de cualquier distribución genérica. En particular, podrían interpretarse como cualquiera de las siguientes:

1. como muestras de la propia distribución empírica $p^{\boldsymbol{X}}$, que asigna una probabilidad de $1/N$ a cada una de las N muestras $\boldsymbol{x}_i, i \in [N]$;
2. como muestras de una distribución normal estándar $\boldsymbol{x}^n \sim p^n \doteq \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$ con una varianza σ^2 lo suficientemente grande (digamos, mayor que los cuadrados de las normas de las muestras);
3. como muestras de una distribución normal $\boldsymbol{x}^e \sim p^e \doteq \mathcal{N}(\boldsymbol{0}, \hat{\boldsymbol{\Sigma}})$, siendo la covarianza $\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top$ la covarianza empírica de las muestras;
4. como muestras de una distribución $\hat{\boldsymbol{x}} \sim \hat{q}(\boldsymbol{x})$ que se aproxima mucho a la distribución verdadera p.

La pregunta es: ¿qué interpretación es mejor y en qué sentido? Supongamos que creemos que estos datos $\boldsymbol{X}$ provienen de una distribución concreta $q(\boldsymbol{x})$, que podría ser una de las anteriores. Entonces podríamos codificar los puntos de datos con el libro de códigos óptimo para la distribución $q(\boldsymbol{x})$. Entonces, a medida que el número de muestras N aumenta, la longitud media de codificación requerida (o tasa de codificación) viene dada por la denominada *entropía cruzada*:

$$\mathrm{CE}(p \parallel q) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[-\log q(\boldsymbol{x})]. \tag{3.1.5}$$

Si hemos identificado la distribución correcta $p(\boldsymbol{x})$, la tasa de codificación viene dada por la entropía $h(p) = \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})]$. Resulta que la longitud de codificación anterior $\mathrm{CE}(p \parallel q)$ es siempre mayor o igual que la entropía $h(\boldsymbol{x})$, a menos que $p = q$.

Por lo tanto, dado un conjunto de datos muestreados $\boldsymbol{X}$, para determinar qué distribución caracteriza mejor los datos entre p^n, p^e y $\hat{q}$, podemos comparar sus tasas de codificación para $\boldsymbol{X}$ y ver cuál ofrece la tasa más baja. Sabemos por lo anterior que la tasa de codificación (teóricamente alcanzable) de una distribución está estrechamente relacionada con su entropía. En general,

$$h(p^n) > h(p^e) > h(\hat{q}). \tag{3.1.6}$$

Si los datos $\boldsymbol{X}$ se codificaran mediante el libro de códigos asociado a cada una de estas distribuciones, la tasa de codificación de $\boldsymbol{X}$ disminuiría, por lo tanto, en el mismo orden:

$$\mathrm{CE}(p \parallel p^n) > \mathrm{CE}(p \parallel p^e) > \mathrm{CE}(p \parallel \hat{q}). \tag{3.1.7}$$

Esta observación ofrece una pauta general para buscar una distribución $p(\boldsymbol{x})$ que tenga una estructura de baja dimensión. Sugiere dos posibles enfoques:

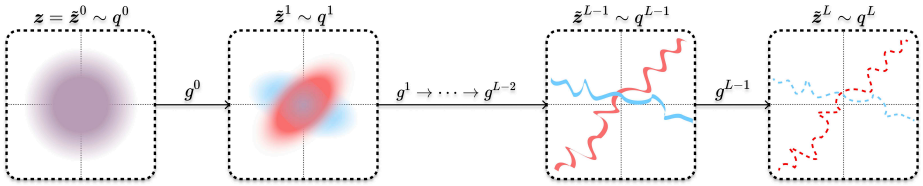


Figura 3.2: Ilustración de un proceso iterativo de eliminación de ruido que, partiendo de una distribución gaussiana isotrópica, converge hacia una distribución de datos arbitraria.

A1. Partiendo de una distribución general (por ejemplo, una distribución normal) con alta entropía, transformar gradualmente la distribución hacia la distribución (empírica) de los datos mediante la reducción de la entropía;

A2. Dentro de una amplia familia de distribuciones (paramétricas o no paramétricas) con esquemas de codificación explícitos que codifican los datos en cuestión, buscar progresivamente mejores esquemas de codificación que ofrezcan tasas de codificación más bajas.

A nivel conceptual, tanto el enfoque A1 como el A2 pretenden lograr lo mismo. Para A1, debemos asegurarnos de que exista tal ruta de transformación y de que sea computable. Para A2, es necesario que la familia elegida sea lo suficientemente amplia y pueda aproximarse de cerca (o abarcar) a la distribución verdadera. En cualquiera de los dos enfoques, debemos asegurarnos de que las soluciones con menor entropía o mejores tasas de codificación puedan calcularse de manera eficiente y converjan rápidamente hacia la distribución deseada.[4] Exploraremos ambos enfoques en las secciones restantes de este capítulo.

3.2 Compresión mediante eliminación de ruido

En esta sección describimos una forma *natural* y *computacionalmente tratable* de aprender una distribución $p_{\boldsymbol{x}}$ mediante el aprendizaje de una codificación paramétrica que minimice la entropía o la tasa de codificación de la representación y luego utilizando esta codificación para transformar muestras de alta entropía de una distribución gaussiana estándar en muestras de baja entropía de la distribución objetivo, tal como se ilustra en la Figura 3.2. Esta metodología combina los enfoques A1 y A2 mencionados anteriormente para aprender y obtener muestras de la distribución.

3.2.1 Procesos de difusión y de eliminación de ruido

En primer lugar, queremos un procedimiento que reduzca la entropía de una variable aleatoria muy ruidosa, generando una muestra de menor entropía a

[4] Por ejemplo, la distribución de datos del mundo real, como imágenes y textos.

partir de la distribución de los datos. Aquí describimos un posible enfoque, quizá el más natural. Primero, diseñamos un método para *aumentar gradualmente* la entropía de las muestras ya extraídas de la distribución de datos. A continuación, buscamos una *inversa aproximada* de este proceso. En general, el aumento de la entropía no tiene inverso, ya que parte de la información de la distribución original puede destruirse. Por lo tanto, nos limitamos a un caso especial en el que (1) la operación que aumenta la entropía es simple, computable y reversible, así como (2) contamos con una codificación (paramétrica) de la distribución de los datos, tal como se ha insinuado anteriormente. Como veremos, estas dos condiciones hacen que nuestro enfoque sea viable.

Procesos de difusión

Aumentaremos la entropía de lo que podría considerarse la forma más sencilla posible, es decir, *añadiendo ruido gaussiano isotrópico*. Más precisamente, dada la variable aleatoria $\boldsymbol{x}$, podemos considerar el *proceso estocástico* $(\boldsymbol{x}_t)_{t\in[0,T]}$ que le añade ruido en forma gradual, es decir,

$$\boldsymbol{x}_t \doteq \boldsymbol{x} + t\boldsymbol{g}, \qquad \forall t \in [0,T], \tag{3.2.1}$$

donde $T \in [0,\infty)$ es un horizonte temporal y $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ se extrae en forma independiente de $\boldsymbol{x}$. Este proceso es un ejemplo de *proceso de difusión*, llamado así porque distribuye (es decir, *difunde*) la masa de probabilidad por todo $\mathbb{R}^D$ a medida que pasa el tiempo, aumentando la entropía. Esta intuición queda confirmada gráficamente en la Figura 3.3, y de manera rigurosa, mediante el siguiente teorema.

Teorema 3.1 (Versión simplificada del Teorema B.2)**.** *Supongamos que* $(\boldsymbol{x}_t)_{t\in[0,T]}$ *sigue el modelo* (3.2.1)*. Para cualquier* $t \in (0,T]$*, la variable aleatoria* $\boldsymbol{x}_t$ *tiene una densidad de probabilidad* p_t *y una entropía diferencial* $h(\boldsymbol{x}_t) > -\infty$*. Además, bajo ciertas condiciones técnicas sobre* $\boldsymbol{x}$*,*

$$\frac{\mathrm{d}}{\mathrm{d}t} h(\boldsymbol{x}_t) > 0, \qquad \forall t \in (0,T], \tag{3.2.2}$$

es decir, la entropía de la distribución de $\boldsymbol{x}_t$ *aumenta con el paso del tiempo* t*.*

La demostración es elemental, pero bastante extensa, por lo que la posponemos hasta la Apéndice B.2.1. La principal consecuencia, aún no expresada, de este resultado es que $h(\boldsymbol{x}_t) > h(\boldsymbol{x}_s)$ para todo $t > s \geq 0$. Para comprobarlo, nótese que si $s = 0$ y $h(\boldsymbol{x}) = h(\boldsymbol{x}_0) = -\infty$, entonces $h(\boldsymbol{x}_t) > -\infty$ para todo $t > 0$, por lo que la conclusión es verdadera; de lo contrario, $h(\boldsymbol{x}_t) = h(\boldsymbol{x}_s) + \int_s^t \left[\frac{\mathrm{d}}{\mathrm{d}u} h(\boldsymbol{x}_u)\right] \mathrm{d}u > h(\boldsymbol{x}_s)$ por el teorema fundamental del cálculo, por lo que en ambos casos $h(\boldsymbol{x}_t) > h(\boldsymbol{x}_s)$ para todo $t > s \geq 0$.

Ejemplo 3.2 (Añadir ruido a una mezcla de distribuciones gaussianas). En este ejemplo, consideramos añadir ruido a una mezcla de K distribuciones gaussianas. Elegimos K vectores de medias $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^D$, K matrices de covarianza $\boldsymbol{\Sigma}_1, \ldots, \boldsymbol{\Sigma}_K \in \mathsf{PSD}(D)$ y probabilidades $\pi_1, \ldots, \pi_K \in [0,1]$ tales que $\sum_{k=1}^K \pi_k = 1$. Entonces, $\boldsymbol{x}$ se muestrea de la siguiente manera.

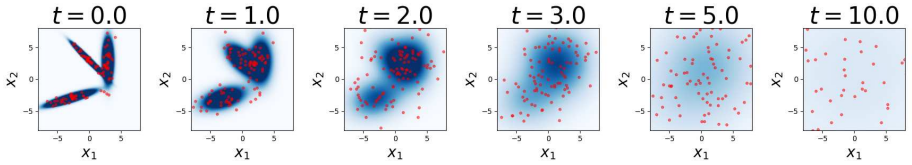


Figura 3.3: **Difusión de una mezcla de distribuciones gaussianas $\boldsymbol{x}$.** De izquierda a derecha, observamos la evolución de la densidad p_t de $\boldsymbol{x}_t$ a medida que t crece de 0 a 10, junto con algunas muestras representativas (en rojo). El plano se colorea según la densidad de probabilidad p_t; las regiones de alta densidad aparecen en un azul más oscuro. Observamos que la masa de probabilidad se vuelve menos concentrada a medida que aumenta t, lo que indica que la entropía aumenta.

- En primer lugar, se elige un índice (o *etiqueta*) $y \in [K]$ de tal manera que $y = k$ con probabilidad π_k.
- En segundo lugar, $\boldsymbol{x}$ se obtiene mediante muestreo de la distribución gaussiana $\mathcal{N}(\boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y)$; es decir, si $y = k$, entonces $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

Por lo tanto, $\boldsymbol{x}$ sigue una distribución de mezcla gaussiana

$$\boldsymbol{x} \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \tag{3.2.3}$$

Los modelos de mezcla gaussiana son relevantes porque pueden aproximar muchas distribuciones: dada cualquier distribución para $\boldsymbol{x}$, existe una mezcla gaussiana que la aproxima con precisión arbitraria. Por este motivo, los modelos de mezcla gaussiana son una alternativa popular y analíticamente manejable para las distribuciones de datos multimodales y complejas. En este libro los utilizamos ampliamente como «problema modelo» para comprender y desarrollar casi todos los conceptos. Por ahora, representamos los datos $\boldsymbol{x}$ mediante una mezcla gaussiana. Tras añadir ruido, la distribución de $\boldsymbol{x}_t$ vuelve a ser una mezcla gaussiana:

$$\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g} \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k + t^2\boldsymbol{I}). \tag{3.2.4}$$

Para ver con un ejemplo cómo evoluciona la entropía, supongamos que $K = 1$, de modo que $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Según (3.1.4), la entropía de $\boldsymbol{x}_t$ es

$$h(\boldsymbol{x}_t) = \frac{D}{2}(1 + \log(2\pi)) + \frac{1}{2}\operatorname{logdet}(\boldsymbol{\Sigma} + t^2\boldsymbol{I}). \tag{3.2.5}$$

Al derivar con respecto al tiempo (véase el Apéndice A) se obtiene

$$\frac{\mathrm{d}}{\mathrm{d}t}h(\boldsymbol{x}_t) = t\operatorname{tr}((\boldsymbol{\Sigma} + t^2\boldsymbol{I})^{-1}) = \sum_{i=1}^{D} \frac{t}{\lambda_i(\boldsymbol{\Sigma}) + t^2} > 0, \tag{3.2.6}$$

donde $\lambda_i(\boldsymbol{\Sigma})$ son los valores propios (no negativos) de $\boldsymbol{\Sigma}$. Dado que esta derivada es positiva, la entropía $h(\boldsymbol{x}_t)$ de la distribución ruidosa aumenta con el tiempo. ■

Procesos de eliminación de ruido

La operación inversa a la adición de ruido se conoce como *eliminación o eliminación de ruido*. Se trata de un tema clásico y ampliamente estudiado en el procesamiento de señales y la teoría de sistemas, que se materializa (por ejemplo) en el filtro de Wiener y el filtro de Kalman (véase la Sección 1.3.1). Varios problemas tratados en el Capítulo 2, como el PCA, el ICA y el aprendizaje de diccionarios, son casos concretos del problema de la eliminación de ruido. Para un t fijo y el modelo de ruido gaussiano aditivo (3.2.1), el problema de eliminación de ruido puede formularse como el intento de aprender una función $\bar{\boldsymbol{x}}^\star(t,\cdot)$ que constituya la mejor aproximación posible (en esperanza) de la variable aleatoria verdadera $\boldsymbol{x}$, dados tanto t como $\boldsymbol{x}_t$:

$$\bar{\boldsymbol{x}}^\star(t,\cdot) \in \arg\min_{\bar{\boldsymbol{x}}(t,\cdot)} \mathbb{E}\,\|\boldsymbol{x} - \bar{\boldsymbol{x}}(t,\boldsymbol{x}_t)\|_2^2. \tag{3.2.7}$$

La solución a este problema, al optimizar $\bar{\boldsymbol{x}}(t,\cdot)$ sobre todas las funciones posibles (cuadráticamente integrables), es el denominado *filtro óptimo de eliminación de ruido de Bayes* y resulta ser la esperanza condicional:

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{\xi}) \doteq \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]. \tag{3.2.8}$$

Esta expresión justifica la notación $\bar{\boldsymbol{x}}$, que calcula una esperanza condicional (es decir, la media condicional o el promedio condicional). En resumen, intenta eliminar el ruido de la entrada ruidosa y genera la mejor estimación posible (en términos de esperanza y con respecto a la distancia ℓ^2) de la variable aleatoria original sin ruido.

Ejemplo 3.3 (Eliminación del ruido gaussiano en una mezcla de distribuciones gaussianas). En este ejemplo calculamos el filtro óptimo de eliminación de ruido de Bayes para una clase importante de distribuciones: el modelo de mezcla gaussiana del Ejemplo 3.2. Definamos $\varphi(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma})$ como la densidad de probabilidad de $\mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$ evaluada en $\boldsymbol{x}$. Con esta notación, la densidad de $\boldsymbol{x}_t$ es

$$p_t(\boldsymbol{x}_t) = \sum_{k=1}^{K} \pi_k \varphi(\boldsymbol{x}_t;\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k + t^2\boldsymbol{I}). \tag{3.2.9}$$

Dado y, las variables siguen una distribución gaussiana multivariada: si escribimos $\boldsymbol{x} = \boldsymbol{\mu}_y + \boldsymbol{\Sigma}_y^{1/2}\boldsymbol{u}$, donde $(\cdot)^{1/2}$ es la raíz cuadrada matricial y $\boldsymbol{u} \sim \mathcal{N}(\boldsymbol{0},\boldsymbol{I})$ independientemente de y (y de $\boldsymbol{g}$), entonces tenemos

$$\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{x}_t\end{bmatrix} = \begin{bmatrix}\boldsymbol{\mu}_y\\ \boldsymbol{\mu}_y\end{bmatrix} + \begin{bmatrix}\boldsymbol{\Sigma}_y^{1/2} & \boldsymbol{0}\\ \boldsymbol{\Sigma}_y^{1/2} & t\boldsymbol{I}\end{bmatrix}\begin{bmatrix}\boldsymbol{u}\\ \boldsymbol{g}\end{bmatrix}. \tag{3.2.10}$$

Esto muestra que $\boldsymbol{x}$ y $\boldsymbol{x}_t$ siguen una distribución gaussiana multivariada (condicionada en y), tal como se afirmaba. Por lo tanto, podemos escribir

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{x}_t \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_y \\ \boldsymbol{\mu}_y \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_y & \boldsymbol{\Sigma}_y \\ \boldsymbol{\Sigma}_y & \boldsymbol{\Sigma}_y + t^2 \boldsymbol{I} \end{bmatrix} \right). \tag{3.2.11}$$

Por lo tanto, la esperanza condicional de $\boldsymbol{x}$ dada $\boldsymbol{x}_t$ (es decir, el filtro óptimo de eliminación de ruido de Bayes dado y) es, como es bien sabido, (véase también el Ejercicio 3.2)

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y] = \boldsymbol{\mu}_y + \boldsymbol{\Sigma}_y (\boldsymbol{\Sigma}_y + t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \boldsymbol{\mu}_y). \tag{3.2.12}$$

Para hallar el *denoiser* óptimo de Bayes global, utilizamos la ley de las esperanzas iteradas, obteniendo

$$\begin{aligned} \bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) &= \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] && (3.2.13) \\ &= \mathbb{E}[\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y] \mid \boldsymbol{x}_t] && (3.2.14) \\ &= \sum_{k=1}^{K} \mathbb{P}[y = k \mid \boldsymbol{x}_t] \, \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y = k]. && (3.2.15) \end{aligned}$$

La probabilidad puede abordarse de la siguiente manera. Sea $p_{t|y}$ la densidad de probabilidad de $\boldsymbol{x}_t$ condicionada a y. Entonces

$$\begin{aligned} \mathbb{P}[y = k \mid \boldsymbol{x}_t] &= \frac{p_{t|y}(\boldsymbol{x}_t \mid k) \pi_k}{p_t(\boldsymbol{x}_t)} && (3.2.16) \\ &= \frac{\pi_k \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})}{\sum_{i=1}^{K} \pi_i \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i + t^2 \boldsymbol{I})}. && (3.2.17) \end{aligned}$$

Por otro lado, la esperanza condicional es la ya descrita:

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y = k] = \boldsymbol{\mu}_k + \boldsymbol{\Sigma}_k (\boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \boldsymbol{\mu}_k). \tag{3.2.18}$$

Para resumir todo lo anterior, el verdadero filtro de eliminación de ruido óptimo de Bayes es

$$\begin{aligned} \bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) = \sum_{k=1}^{K} \Bigg\{ & \frac{\pi_k \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})}{\sum_{i=1}^{K} \pi_i \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i + t^2 \boldsymbol{I})} && (3.2.19) \\ & \cdot \left(\boldsymbol{\mu}_k + \boldsymbol{\Sigma}_k (\boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \boldsymbol{\mu}_k) \right) \Bigg\}. && (3.2.20) \end{aligned}$$

Este ejemplo es especialmente importante, y varios casos particulares nos proporcionarán una gran perspectiva conceptual más adelante. Por ahora, intentemos extraer alguna intuición geométrica a partir de la forma funcional del filtro óptimo de eliminación de ruido (3.2.19).

Para comprender (3.2.19) de manera intuitiva, fijemos primero $K = 1$ (es decir, una sola gaussiana) de tal manera que $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Diagonalicemos

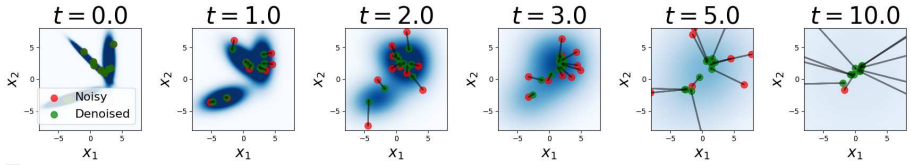


Figura 3.4: ***Denoiser* óptimo de Bayes y *score* de un modelo de mezcla gaussiana.** En el mismo contexto que en la Figura 3.3, mostramos el efecto del {textitdenoiser óptimo de Bayes $\bar{\boldsymbol{x}}^\star$ graficando $\boldsymbol{x}_t$ (rojo) y $\bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t)$ (verde) para un valor arbitrario de t y $\boldsymbol{x}_t$. Según la fórmula de Tweedie, Teorema 3.2, el residuo entre ambos es proporcional a l *score* (Hyvärinen) $\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)$. Podemos observar que el *score* apunta hacia los modos de la distribución de $\boldsymbol{x}_t$.

entonces $\boldsymbol{\Sigma} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^\top$. Entonces, el filtro óptimo de eliminación de ruido de Bayes es

$$\bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) = \boldsymbol{\mu} + \boldsymbol{\Sigma}(\boldsymbol{\Sigma} + t^2\boldsymbol{I})^{-1}(\boldsymbol{x}_t - \boldsymbol{\mu}) \tag{3.2.21}$$

$$= \boldsymbol{\mu} + \boldsymbol{V} \begin{bmatrix} \lambda_1/(\lambda_1 + t^2) & & \\ & \ddots & \\ & & \lambda_D/(\lambda_D + t^2) \end{bmatrix} \boldsymbol{V}^\top(\boldsymbol{x}_t - \boldsymbol{\mu}), \tag{3.2.22}$$

donde $\lambda_1, \ldots, \lambda_D$ son los valores propios de $\boldsymbol{\Sigma}$. Se observa que este filtro de eliminación de ruido consta de tres pasos:

- Trasladar la entrada $\boldsymbol{x}_t$ por $\boldsymbol{\mu}$.
- Contraer la entrada (trasladada) $\boldsymbol{x}_t - \boldsymbol{\mu}$ en cada dirección de vector propio por un factor de $\lambda_i/(\lambda_i + t^2)$. Si la entrada trasladada es de bajo rango y algunos valores propios de $\boldsymbol{\Sigma}$ son cero, el filtro de eliminación de ruido contrae inmediatamente estas direcciones hasta 0, lo que garantiza que la salida de la contracción sea igualmente de bajo rango.
- Trasladar el resultado hacia atrás por $\boldsymbol{\mu}$.

Esta es la interpretación geométrica del filtro de eliminación de ruido de una *única* gaussiana. El *denoiser* global del modelo de mezcla gaussiana (3.2.19) utiliza K *denoisers* de este tipo, ponderando su salida mediante las probabilidades a posteriori $\mathbb{P}[y = k \mid \boldsymbol{x}_t]$. Si las medias de las distribuciones gaussianas están bien separadas, estas probabilidades a posteriori son muy cercanas a 0 o 1 cerca de cada media o grupo. En este régimen, el *denoiser* global (3.2.19) tiene la misma interpretación geométrica que el *denoiser* gaussiano único mencionado anteriormente. ■

Intuitivamente y como podemos ver en el Ejemplo 3.3, el *denoiser* óptimo de Bayes $\bar{\boldsymbol{x}}^\star(t, \cdot)$ debería desplazar su entrada $\boldsymbol{x}_t$ hacia los modos de la distribución de $\boldsymbol{x}$. Resulta que podemos cuantificar esto mostrando que el *denoiser* óptimo de Bayes *da un paso de ascenso por gradiente* en la log-densidad de $\boldsymbol{x}_t$, que (recordemos) hemos denotado como p_t. Es decir, aplicar el filtro de eliminación

de ruido implica pasar de la iteración de entrada a una región de mayor probabilidad dentro de esta distribución (perturbada). Para valores pequeños de t, la perturbación es pequeña, por lo que nuestra intuición inicial es, por lo tanto, casi totalmente correcta. La imagen se muestra en la Figura 3.4 y se formula rigurosamente como la fórmula de Tweedie [Rob56].

Teorema 3.2 (La fórmula de Tweedie)**.** *Supongamos que* $(\boldsymbol{x}_t)_{t\in[0,T]}$ *obedece a* (3.2.1)*. Sea* p_t *la densidad de* $\boldsymbol{x}_t$ *(tal como se ha definido anteriormente). Entonces*

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] = \boldsymbol{x}_t + t^2 \nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t). \tag{3.2.23}$$

Demostración. Para demostrarlo, supongamos que $\boldsymbol{x}$ tiene una densidad,[5] y llamemos a esta densidad p. Sean $p_{0|t}$ y $p_{t|0}$ las densidades condicionales de $\boldsymbol{x} = \boldsymbol{x}_0$ dado $\boldsymbol{x}_t$ y de $\boldsymbol{x}_t$ dado $\boldsymbol{x}$, respectivamente. Sea $\varphi(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ la densidad de $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ evaluada en $\boldsymbol{x}$, de modo que $p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}) = \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I})$. Entonces, un cálculo sencillo nos da

$$\begin{aligned}
\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t) &= \frac{\nabla_{\boldsymbol{x}_t} p_t(\boldsymbol{x}_t)}{p_t(\boldsymbol{x}_t)} && (3.2.24)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \nabla_{\boldsymbol{x}_t} \int_{\mathbb{R}^D} p(\boldsymbol{x}) p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}) \mathrm{d}\boldsymbol{x} && (3.2.25)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \nabla_{\boldsymbol{x}_t} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \mathrm{d}\boldsymbol{x} && (3.2.26)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) [\nabla_{\boldsymbol{x}_t} \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I})] \mathrm{d}\boldsymbol{x} && (3.2.27)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \left[-\frac{\boldsymbol{x}_t - \boldsymbol{x}}{t^2} \right] \mathrm{d}\boldsymbol{x} && (3.2.28)\\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) [\boldsymbol{x} - \boldsymbol{x}_t] \mathrm{d}\boldsymbol{x} && (3.2.29)\\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \boldsymbol{x} \mathrm{d}\boldsymbol{x} && (3.2.30)\\
&\quad - \frac{\boldsymbol{x}_t}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \mathrm{d}\boldsymbol{x} \\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}) \boldsymbol{x} \mathrm{d}\boldsymbol{x} - \frac{\boldsymbol{x}_t}{t^2 p_t(\boldsymbol{x}_t)} p_t(\boldsymbol{x}_t) && (3.2.31)\\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p_t(\boldsymbol{x}_t) p_{0|t}(\boldsymbol{x} \mid \boldsymbol{x}_t) \boldsymbol{x} \mathrm{d}\boldsymbol{x} - \frac{\boldsymbol{x}_t}{t^2 p_t(\boldsymbol{x}_t)} p_t(\boldsymbol{x}_t) && (3.2.32)\\
&= \frac{1}{t^2} \int_{\mathbb{R}^D} p_{0|t}(\boldsymbol{x} \mid \boldsymbol{x}_t) \boldsymbol{x} \mathrm{d}\boldsymbol{x} - \frac{\boldsymbol{x}_t}{t^2} && (3.2.33)\\
&= \frac{1}{t^2} \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] - \frac{\boldsymbol{x}_t}{t^2} && (3.2.34)
\end{aligned}$$

[5] Obsérvese que el teorema es cierto sin esta suposición y que puede demostrarse mediante una modificación mínima de esta demostración para utilizar medidas de probabilidad en lugar de densidades.

$$= \frac{\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] - \boldsymbol{x}_t}{t^2}. \tag{3.2.35}$$

Una simple reorganización de la igualdad anterior demuestra el teorema. □

Este resultado establece una conexión entre la eliminación de ruido y la optimización: el *denoiser* óptimo de Bayes da un único paso de ascenso por gradiente en la densidad de datos perturbada p_t, y el tamaño del paso se reduce en forma adaptativa (es decir, da pasos más precisos) a medida que la perturbación de la distribución de datos se va reduciendo. La cantidad $\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)$ se denomina *puntuación (de Hyvärinen)* y aparece con frecuencia en discusiones sobre la eliminación de ruido, etc.; apareció por primera vez en un artículo de Aapo Hyvärinen en el contexto del ICA [Hyv05].

De manera similar a cómo un solo paso de descenso por gradiente casi nunca es suficiente para minimizar un objetivo en la práctica cuando se inicia lejos del óptimo, la salida del *denoiser* óptimo de Bayes $\bar{\boldsymbol{x}}^\star(t, \cdot)$ casi nunca se encuentra en una región de alta probabilidad de la distribución de datos cuando t es grande, *especialmente* cuando los datos tienen una estructura de baja dimensión. Ilustramos este punto de manera explícita en el siguiente ejemplo.

Ejemplo 3.4 (Eliminación de ruido en una mezcla de dos puntos). Sea x una variable aleatoria uniforme en el conjunto de dos puntos $\{-1, +1\}$ y sea $(x_t)_{t\in[0,T]}$ una variable aleatoria que sigue la ecuación (3.2.1). Se trata precisamente de un modelo de mezcla gaussiana degenerada con distribuciones a priori iguales a $\frac{1}{2}$, medias $\{-1, +1\}$ y covarianzas ambas iguales a 0. Para un valor fijo de $t > 0$, podemos utilizar el cálculo del *denoiser* óptimo de Bayes en (3.2.19) para obtener (la demostración se deja como ejercicio)

$$\bar{x}^\star(t, x_t) = \frac{\varphi(x_t; +1, t^2) - \varphi(x_t; -1, t^2)}{\varphi(x_t; 1, t^2) + \varphi(x_t; -1, t^2)} = \tanh\Big(\frac{x_t}{t^2}\Big). \tag{3.2.36}$$

Para valores de t cercanos a 0, esta cantidad se aproxima a $\{-1, +1\}$ para casi todas las entradas $\bar{x}^\star(t, x_t)$. Sin embargo, para t grande, esta cantidad no se encuentra necesariamente ni siquiera aproximadamente en el soporte original de x, que, como recordaremos, es $\{-1, +1\}$. En particular, para $x_t \approx 0$ se cumple que $\bar{x}^\star(t, x_t) \approx 0$, lo cual se encuentra completamente entre los dos puntos posibles. Por lo tanto, $\bar{x}^\star$ *no dará como resultado un* x *«realista»*. Más rigurosamente, la distribución de $\bar{x}^\star(t, x_t)$ es muy diferente de la distribución de x. ■

Eliminación de ruido incremental

Por lo tanto, para eliminar el ruido de la muestra muy ruidosa $\boldsymbol{x}_T$ (donde, recordemos, T es el tiempo máximo), no podemos utilizar el filtro de eliminación de ruido solo una vez. En cambio, debemos aplicarlo muchas veces, en forma análoga al descenso por gradiente con tamaños de paso decrecientes, para converger hacia un punto estacionario $\hat{\boldsymbol{x}}$. Es decir, utilizamos el filtro de eliminación de ruido para pasar de $\boldsymbol{x}_T$ a $\hat{\boldsymbol{x}}_{T-\delta}$, que se aproxima a $\boldsymbol{x}_{T-\delta}$, luego

de $\hat{\boldsymbol{x}}_{T-\delta}$ a $\hat{\boldsymbol{x}}_{T-2\delta}$, y así sucesivamente, desde $\hat{\boldsymbol{x}}_\delta$ hasta $\hat{\boldsymbol{x}} = \hat{\boldsymbol{x}}_0$. Cada paso de eliminación de ruido hace que la acción del filtro de eliminación de ruido se parezca más a un paso de gradiente sobre la densidad (logarítmica) original.

De manera más formal, discretizamos uniformemente el intervalo $[0, T]$ en $L+1$ pasos temporales $0 = t_0 < t_1 < \cdots < t_L = T$, es decir,

$$t_\ell = \frac{\ell}{L} T, \qquad \ell \in \{0, 1, \ldots, L\}. \tag{3.2.37}$$

A continuación, para cada $\ell \in [L] = \{1, 2, \ldots, L\}$, procediendo de $\ell = L$ a $\ell = 1$, ejecutamos la iteración

$$\begin{aligned}
\hat{\boldsymbol{x}}_{t_{\ell-1}} &= \mathbb{E}[\boldsymbol{x}_{t_{\ell-1}} \mid \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}] && (3.2.38)\\
&= \mathbb{E}[\boldsymbol{x} + t_{\ell-1}\boldsymbol{g} \mid \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}] && (3.2.39)\\
&= \mathbb{E}\left[\boldsymbol{x} + t_{\ell-1} \cdot \frac{\boldsymbol{x}_{t_\ell} - \boldsymbol{x}}{t_\ell} \;\middle|\; \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}\right] && (3.2.40)\\
&= \frac{t_{\ell-1}}{t_\ell}\hat{\boldsymbol{x}}_{t_\ell} + \left(1 - \frac{t_{\ell-1}}{t_\ell}\right)\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}] && (3.2.41)\\
&= \left(1 - \frac{1}{\ell}\right)\hat{\boldsymbol{x}}_{t_\ell} + \frac{1}{\ell}\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}). && (3.2.42)
\end{aligned}$$

Al inicio de la iteración, cuando ℓ es grande, apenas confiamos en la salida del *denoiser* y, en su mayor parte, conservamos el iterado actual. Esto tiene sentido, ya que el filtro de eliminación de ruido puede presentar una enorme varianza (véase el Ejemplo 3.4). Cuando ℓ es pequeño, el *denoiser* se «fija» en los modos de la distribución de los datos, ya que un paso de la eliminación de ruido es, en esencia, un paso de gradiente sobre la log-densidad de la distribución verdadera y podemos confiar en que no generará muestras poco verosímiles; por lo tanto, la etapa de eliminación de ruido está determinada por la salida del *denoiser*. Cuando $\ell = 1$, incluso descartamos el iterado actual y conservamos únicamente la salida del *denoiser*.

Visualizamos el proceso de convergencia en $\mathbb{R}^3$ en la Figura 3.5. Aunque más adelante desarrollaremos algunos resultados rigurosos sobre la convergencia, ahora ofrecemos un ejemplo en forma cerrada del proceso iterativo de eliminación de ruido aplicado a la eliminación de ruido de una distribución gaussiana de datos.

Ejemplo 3.5 (Eliminación de ruido iterativa para una sola gaussiana). Para comprender de manera intuitiva el procedimiento iterativo de reducción de ruido descrito anteriormente, la aplicamos para eliminar el ruido de $\boldsymbol{x}_T$, definido en (3.2.1), donde $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Utilizando (3.2.42), calculamos

$$\begin{aligned}
\hat{\boldsymbol{x}}_{t_{\ell-1}} - \boldsymbol{\mu} &= \left(1 - \frac{1}{\ell}\right)\hat{\boldsymbol{x}}_{t_\ell} + \frac{1}{\ell}\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}) - \boldsymbol{\mu} && (3.2.43)\\
&= \left(1 - \frac{1}{\ell}\right)(\hat{\boldsymbol{x}}_{t_\ell} - \boldsymbol{\mu}) + \frac{1}{\ell}(\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}) - \boldsymbol{\mu}). && (3.2.44)
\end{aligned}$$

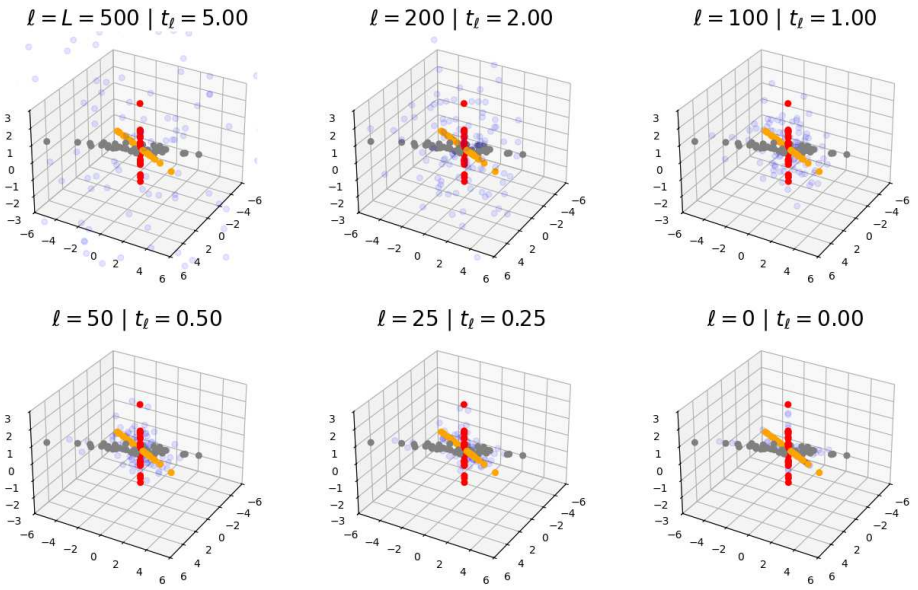


Figura 3.5: **Eliminación del ruido en una mezcla de gaussianas de rango bajo.** Cada figura muestra muestras de la distribución real de los datos (gris, naranja, rojo) y muestras sometidas al proceso de eliminación de ruido (3.2.82) (celeste). En la parte superior izquierda, el proceso acaba de comenzar y el ruido es muy intenso. A medida que avanza el proceso, el ruido se desplaza cada vez más hacia el soporte de la distribución de datos de rango bajo. Por último, en la parte inferior derecha, las muestras generadas se alinean perfectamente con el soporte de los datos y se asemejan mucho a las muestras extraídas del modelo de mezcla gaussiana de rango bajo.

A continuación, al sustituir la expresión de $\bar{\boldsymbol{x}}^\star$ que obtenemos de (3.2.22) y realizar algunas operaciones tediosas de álgebra matricial, obtenemos

$$\boldsymbol{x}_{t_{\ell-1}} - \boldsymbol{\mu} = \boldsymbol{V} \begin{bmatrix} 1 - \frac{\ell T^2}{\lambda_1 L^2 + \ell^2 T^2} & & \\ & \ddots & \\ & & 1 - \frac{\ell T^2}{\lambda_D L^2 + \ell^2 T^2} \end{bmatrix} \boldsymbol{V}^\top (\boldsymbol{x}_{t_\ell} - \boldsymbol{\mu}). \quad (3.2.45)$$

Obsérvese que todas las entradas de esta matriz diagonal están en el intervalo $(0, 1)$. Si tomamos la norma ℓ^2 y utilizamos una cota sencilla pero ajustada, se cumple que

$$\|\boldsymbol{x}_{t_{\ell-1}} - \boldsymbol{\mu}\|_2 \leq \underbrace{\max_{i \in [D]} \left\{ 1 - \frac{\ell T^2}{\lambda_i L^2 + \ell^2 T^2} \right\}}_{c_{\ell, L}} \cdot \|\boldsymbol{x}_{t_\ell} - \boldsymbol{\mu}\|_2. \quad (3.2.46)$$

A primera vista, esta desigualdad parece implicar que la iteración de eliminación de ruido es, conceptualmente, muy similar a un mapeo de contracción, como la

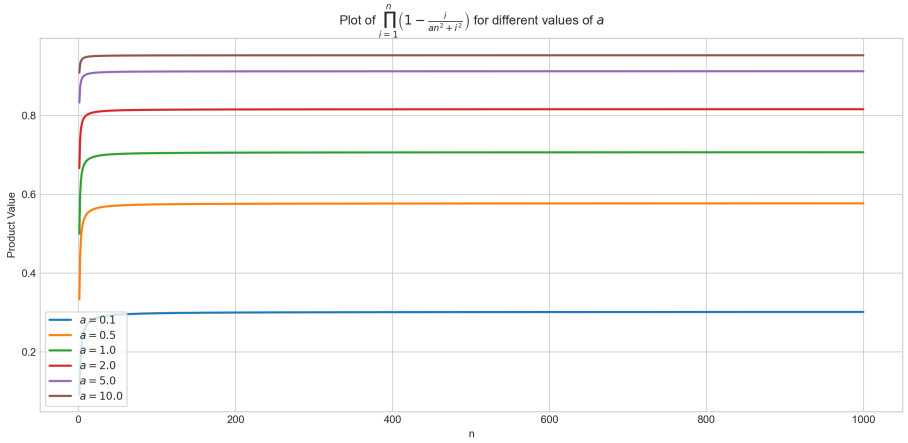


Figura 3.6: **La iteración de eliminación de ruido en el Ejemplo 3.5 no es un mapeo de contracción.** Gráfico de los productos de los coeficientes de contracción $\prod_{\ell=1}^{L} c_{\ell,L}$ en el caso de que los datos se encuentran en $D = 1$, el horizonte temporal $T = 1$ y la varianza de los datos $\lambda_1 = a$, para distintos valores de a. Si la iteración de eliminación de ruido fuera (conceptualmente similar a) una aplicación de contracción, estos productos tenderían a 0, pero, en cambio, convergen hacia valores distintos de cero.

iteración de potencias (2.1.17), que lleva el iterado $\boldsymbol{x}_t$ a la media $\boldsymbol{\mu}$, ya que todos los valores de $c_{\ell,L}$ pertenecen a $(0, 1)$ para cualquier ℓ y L. Sin embargo, esto no es así. Para verlo, obsérvese que, al iterar (3.2.46), se cumple que

$$\|\boldsymbol{x}_0 - \boldsymbol{\mu}\|_2 \leq \prod_{\ell=1}^{L} c_{\ell,L} \cdot \|\boldsymbol{x}_T - \boldsymbol{\mu}\|_2. \tag{3.2.47}$$

Si cada actualización de eliminación de ruido tuviera propiedades similares a las de un mapeo de contracción, entonces $\prod_{\ell=1}^{L} c_{\ell,L}$ tendería a 0 cuando $L \to \infty$ y la iteración convergería a $\boldsymbol{\mu}$ extremadamente rápido, lo que haría que el muestreador fuera completamente inútil o vacuo. Pero, en realidad, la iteración de eliminación de ruido *no es, en realidad, un mapeo de contracción*, ya que $c_{\ell,L}$ varía con ambos índices ℓ y L, de modo que $\prod_{\ell=1}^{L} c_{\ell,L}$ no tiende a 0 cuando $L \to \infty$, como se demuestra en la Figura 3.6 (también es posible calcular el límite analíticamente). En realidad, el muestreador garantiza que las iteraciones finales se dispersen a lo largo del soporte de la distribución de datos, lo que permite un muestreo real a partir de la distribución de datos, algo muy distinto del objetivo de la iteración de potencias (por ejemplo). ■

Recordemos que queríamos construir el proceso iterativo de eliminación de ruido para reducir la entropía de los iterados. Aunque lo hicimos por un rodeo, invirtiendo un proceso que añade entropía, ahora es el momento de pagar las consecuencias y confirmar que nuestro proceso iterativo de eliminación de ruido reduce la entropía.

Teorema 3.3 (Versión simplifiada del Teorema B.3)**.** *Supongamos que* $(\boldsymbol{x}_t)_{t\in[0,T]}$ *obedece a* (3.2.1)*. Entonces, bajo ciertas condiciones técnicas sobre* $\boldsymbol{x}$*, para todo* $s<t$ *con* $s,t\in(0,T]$*,*

$$h(\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]) < h(\boldsymbol{x}_t). \tag{3.2.48}$$

El enunciado completo del teorema y la propia demostración requieren cierto grado de tecnicismo, por lo que se posponen hasta la Apéndice B.2.2.

Cálculo del *denoiser*

Lo último que comentamos aquí es que, en muchas ocasiones, *no podremos calcular* $\bar{\boldsymbol{x}}^\star(t,\cdot)$ para ningún t, ya que no disponemos de la distribución p_t. Pero podemos intentar *aprender una a partir de los datos*. Recordemos que el *denoiser* $\bar{\boldsymbol{x}}^\star$ se define en (3.2.7) como la minimización del error cuadrático medio $\mathbb{E}\,\|\bar{\boldsymbol{x}}(t,\boldsymbol{x}_t)-\boldsymbol{x}\|_2^2$. Podemos utilizar este error cuadrático medio como función de pérdida o función objetivo para entrenar el filtro de eliminación de ruido. Por ejemplo, podemos parametrizar $\bar{\boldsymbol{x}}(t,\cdot)$ mediante una red neuronal, escribiéndola como $\bar{\boldsymbol{x}}_\theta(t,\cdot)$ y optimizar la pérdida en el espacio de parámetros Θ:

$$\min_{\theta\in\Theta} \mathbb{E}\,\|\bar{\boldsymbol{x}}_\theta(t,\boldsymbol{x}_t)-\boldsymbol{x}\|_2^2. \tag{3.2.49}$$

La solución a este problema de optimización, implementada mediante descenso por gradiente o un algoritmo similar, nos dará un $\bar{\boldsymbol{x}}_{\theta^\star}(t,\cdot)$ que es una buena aproximación a $\bar{\boldsymbol{x}}^\star(t,\cdot)$ (al menos si el entrenamiento funciona) y que utilizaremos como nuestro *denoiser*.

¿Cuál sería una buena arquitectura para esta red neuronal $\bar{\boldsymbol{x}}_{\theta^\star}(t,\cdot)$? Para responder a esta pregunta, intentaremos primero comprender el caso en que los datos $\boldsymbol{x}$ son de *bajo rango*, es decir, que tienen un soporte de bajo rango. La forma más canónica de representar datos de rango bajo es mediante una *distribución gaussiana de rango bajo* , por lo que comenzamos nuestro análisis suponiendo que $\boldsymbol{x}$ se extrae (aproximadamente) de una distribución gaussiana de rango bajo:

$$\boldsymbol{x}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{U}\boldsymbol{U}^\top), \tag{3.2.50}$$

donde $\boldsymbol{U}\in\mathsf{O}(D,P)$ es una matriz ortogonal alta con $P\le D$ (utilizando una media nula esencialmente por conveniencia notacional). El *denoiser* óptimo bajo (3.2.50) es (según el Ejemplo 3.3):

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{x}_t)=\boldsymbol{U}\boldsymbol{U}^\top(\boldsymbol{U}\boldsymbol{U}^\top+t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t. \tag{3.2.51}$$

La matriz $\boldsymbol{U}\boldsymbol{U}^\top+t^2\boldsymbol{I}$ es una perturbación de bajo rango de la matriz de rango completo $t^2\boldsymbol{I}$, por lo que se presta a una simplificación mediante la *identidad de Sherman-Morrison-Woodbury*, es decir, para matrices $\boldsymbol{A},\boldsymbol{C},\boldsymbol{U},\boldsymbol{V}$ tales que $\boldsymbol{A}$ y $\boldsymbol{C}$ son invertibles,

$$(\boldsymbol{A}+\boldsymbol{U}\boldsymbol{C}\boldsymbol{V})^{-1}=\boldsymbol{A}^{-1}-\boldsymbol{A}^{-1}\boldsymbol{U}(\boldsymbol{C}^{-1}+\boldsymbol{V}\boldsymbol{A}^{-1}\boldsymbol{U})^{-1}\boldsymbol{V}\boldsymbol{A}^{-1}. \tag{3.2.52}$$

Demostramos esta identidad en el Ejercicio 3.3. Por ahora, aplicamos esta identidad con $\boldsymbol{A} = t^2\boldsymbol{I}$, $\boldsymbol{U} = \boldsymbol{U}$, $\boldsymbol{V} = \boldsymbol{U}^\top$ y $\boldsymbol{C} = \boldsymbol{I}$, obteniendo

$$(\boldsymbol{U}\boldsymbol{U}^\top + t^2\boldsymbol{I})^{-1} = \frac{1}{t^2}\boldsymbol{I} - \frac{1}{t^4}\boldsymbol{U}\left(\boldsymbol{I} + \frac{1}{t^2}\boldsymbol{U}^\top\boldsymbol{U}\right)^{-1}\boldsymbol{U}^\top \tag{3.2.53}$$

$$= \frac{1}{t^2}\boldsymbol{I} - \frac{1}{t^4\left(1+\frac{1}{t^2}\right)}\boldsymbol{U}\boldsymbol{U}^\top \tag{3.2.54}$$

$$= \frac{1}{t^2}\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}\boldsymbol{U}^\top\right). \tag{3.2.55}$$

Esto implica

$$\boldsymbol{U}\boldsymbol{U}^\top(\boldsymbol{U}\boldsymbol{U}^\top + t^2\boldsymbol{I})^{-1} = \frac{1}{t^2}\boldsymbol{U}\boldsymbol{U}^\top\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}\boldsymbol{U}^\top\right) \tag{3.2.56}$$

$$= \frac{1}{t^2}\left(1 - \frac{1}{1+t^2}\right)\boldsymbol{U}\boldsymbol{U}^\top \tag{3.2.57}$$

$$= \frac{1}{1+t^2}\boldsymbol{U}\boldsymbol{U}^\top. \tag{3.2.58}$$

Nuestro filtro de eliminación de ruido óptimo es una proyección lineal (reescalada) sobre el subespacio de rango bajo que sustenta la distribución de nuestros datos. Esto es básicamente lo mismo que el PCA probabilístico del Capítulo 2 (véase Sección 2.1.4). Podemos formalizar esta intuición considerando una clase de *denoisers* parametrizados por subespacios:

$$\bar{\boldsymbol{x}}_{\boldsymbol{V}}(t, \boldsymbol{x}_t) = \frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{x}_t, \tag{3.2.59}$$

y mostrando que el problema de eliminación de ruido $\min_{\boldsymbol{V}\in\mathsf{O}(D,P)} \mathbb{E}\,\|\bar{\boldsymbol{x}}_{\boldsymbol{V}}(t,\boldsymbol{x}_t) - \boldsymbol{x}\|_2^2$ obtiene la misma solución que el PCA probabilístico (Sección 2.1.4). De hecho, al sustituir nuestro *denoiser* por subespacios en la función de pérdida de entrenamiento (3.2.7), obtenemos

$$\min_{\boldsymbol{V}\in\mathsf{O}(D,P)} \mathbb{E}\,\|\bar{\boldsymbol{x}}_{\boldsymbol{V}}(t,\boldsymbol{x}_t) - \boldsymbol{x}\|_2^2 = \mathbb{E}\left\|\frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top(\boldsymbol{x}+t\boldsymbol{g}) - \boldsymbol{x}\right\|_2^2, \tag{3.2.60}$$

donde la igualdad se debe a (3.2.1). Condicionando respecto a $\boldsymbol{x}$ (integrando así sobre $\boldsymbol{g}$), calculamos

$$\mathbb{E}_{\boldsymbol{g}}\left\|\frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top(\boldsymbol{x}+t\boldsymbol{g}) - \boldsymbol{x}\right\|_2^2 \tag{3.2.61}$$

$$= \left\|\frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{x} - \boldsymbol{x}\right\|_2^2 - \frac{t}{1+t^2}\,\mathbb{E}_{\boldsymbol{g}}\left\langle \boldsymbol{x} - \frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{x}, \boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{g}\right\rangle \tag{3.2.62}$$

$$+ \frac{t^2}{(1+t^2)^2}\,\mathbb{E}_{\boldsymbol{g}}\left\|\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{g}\right\|_2^2 \tag{3.2.63}$$

$$= \left\| \frac{1}{1+t^2} \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{x} - \boldsymbol{x} \right\|_2^2 + \frac{t^2 P}{(1+t^2)^2} \tag{3.2.64}$$

donde la segunda igualdad se deduce de $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ y del hecho de que $\boldsymbol{V} \in \mathsf{O}(D, P)$, lo que implica que $\mathbb{E}_{\boldsymbol{g}} \|\boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{g}\|_2^2 = \mathbb{E}_{\boldsymbol{g}} \|\boldsymbol{V}^\top \boldsymbol{g}\|_2^2 = P$. Por lo tanto, el problema de eliminación de ruido es equivalente a

$$\min_{\boldsymbol{V} \in \mathsf{O}(D,P)} \left\{ \mathbb{E} \left\| \frac{1}{1+t^2} \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{x} - \boldsymbol{x} \right\|_2^2 \right. \tag{3.2.65}$$

$$\left. = \mathbb{E} \|\boldsymbol{x}\|_2^2 + \left(\frac{1}{(1+t^2)^2} - \frac{2}{1+t^2} \right) \mathbb{E} \|\boldsymbol{V}^\top \boldsymbol{x}\|_2^2 \right\}, \tag{3.2.66}$$

lo cual, a su vez, equivale del problema

$$\max_{\boldsymbol{V} \in \mathsf{O}(D,P)} \mathbb{E} \|\boldsymbol{V}^\top \boldsymbol{x}\|_2^2 \tag{3.2.67}$$

que, como se demostró en la Sección 2.1.1, es equivalente al problema

$$\min_{\boldsymbol{V} \in \mathsf{O}(D,P)} \mathbb{E} \|\boldsymbol{x} - \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{x}\|_2^2, \tag{3.2.68}$$

es decir, la formulación probabilística del PCA (2.1.30).

Por lo tanto, si supusiéramos que nuestros datos procedieran de una distribución gaussiana de rango bajo, nuestro algoritmo de eliminación de ruido consistiría básicamente en aplicar sucesivamente muchas iteraciones de PCA a datos cada vez menos ruidosos. Aunque este método parece muy sencillo, en realidad puede generar comportamientos de eliminación de ruido complejos y relativamente potentes cuando se aplica a datos reales, tal y como ha demostrado PCANet [CJG+15]. No obstante, para obtener un operador de eliminación de ruido más general que pueda ser aplicable a con distribuciones de datos complejas y multimodales, consideraremos una clase más amplia de distribuciones de datos: la omnipresente clase de *modelos de mezcla gaussiana de bajo rango*, cuyo *denoiser* calculamos en el Ejemplo 3.2 y el Ejemplo 3.3. En el contexto de la eliminación de ruido, dado que los modelos de mezcla gaussiana pueden aproximar una clase muy amplia de distribuciones con una precisión arbitraria, en la práctica, la optimización dentro de la clase de *denoisers* para modelos de mezcla gaussiana puede, potencialmente, proporcionarnos algo cercano al *denoiser* óptimo para la distribución real de los datos.

Formalmente, supongamos ahora que $\boldsymbol{x}$ se extrae de un modelo de mezcla gaussiana de bajo rango $\frac{1}{K} \sum_{k=1}^K \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top)$, donde $\boldsymbol{U}_k \in \mathsf{O}(D, P)$ son matrices ortogonales altas cuyas columnas abarcan los subespacios de dimensión P que soportan las K componentes gaussianas de rango bajo. Escribimos

$$\boldsymbol{x} \sim \frac{1}{K} \sum_{k=1}^K \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top). \tag{3.2.69}$$

Entonces, el *denoiser* óptimo según (3.2.1) es (de acuerdo al Ejemplo 3.3)

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{x}_t) = \sum_{k=1}^{K}\left\{\frac{\varphi(\boldsymbol{x}_t;\boldsymbol{0},\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})}{\sum_{i=1}^{K}\varphi(\boldsymbol{x}_t;\boldsymbol{0},\boldsymbol{U}_i\boldsymbol{U}_i^\top+t^2\boldsymbol{I})}\right. \tag{3.2.70}$$

$$\left.\cdot\left(\boldsymbol{U}_k\boldsymbol{U}_k^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t\right)\right\}. \tag{3.2.71}$$

Al aplicar la misma identidad de Sherman-Morrison-Woodbury que en (3.2.55), obtenemos

$$(\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})^{-1} = \frac{1}{t^2}\left(\boldsymbol{I}-\frac{1}{1+t^2}\boldsymbol{U}_k\boldsymbol{U}_k^\top\right). \tag{3.2.72}$$

Podemos entonces calcular las probabilidades a posteriori de la siguiente manera. Dado que los $\boldsymbol{U}_k$ son ortogonales, $\det(\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})$ es igual para todo k. Por lo tanto

$$\frac{\varphi(\boldsymbol{x}_t;\boldsymbol{0},\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})}{\sum_{i=1}^{K}\varphi(\boldsymbol{x}_t;\boldsymbol{0},\boldsymbol{U}_i\boldsymbol{U}_i^\top+t^2\boldsymbol{I})} \tag{3.2.73}$$

$$= \frac{\exp\left(-\frac{1}{2}\boldsymbol{x}_t^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t\right)}{\sum_{i=1}^{K}\exp\left(-\frac{1}{2}\boldsymbol{x}_t^\top(\boldsymbol{U}_i\boldsymbol{U}_i^\top+t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t\right)} \tag{3.2.74}$$

$$= \frac{\exp\left(-\frac{1}{2t^2}\boldsymbol{x}_t^\top\left(\boldsymbol{I}-\frac{1}{1+t^2}\boldsymbol{U}_k\boldsymbol{U}_k^\top\right)\boldsymbol{x}_t\right)}{\sum_{i=1}^{K}\exp\left(-\frac{1}{2t^2}\boldsymbol{x}_t^\top\left(\boldsymbol{I}-\frac{1}{1+t^2}\boldsymbol{U}_i\boldsymbol{U}_i^\top\right)\boldsymbol{x}_t\right)} \tag{3.2.75}$$

$$= \frac{\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2+\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2+\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)} \tag{3.2.76}$$

$$= \frac{\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2\right)\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2\right)\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)} \tag{3.2.77}$$

$$= \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)}. \tag{3.2.78}$$

Se trata de una operación softmax ponderada por la proyección de $\boldsymbol{x}_t$ sobre cada subespacio, cuantificada por $\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2$ (con temperatura $2t^2(1+t^2)$). Por su parte, los *denoisers* por componente pueden expresarse en forma análoga a (3.2.58) para obtener:

$$\boldsymbol{U}_k\boldsymbol{U}_k^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top+t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t = \frac{1}{1+t^2}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t. \tag{3.2.79}$$

Combinando todo esto, obtenemos

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{x}_t) = \frac{1}{1+t^2}\sum_{k=1}^{K}\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t, \tag{3.2.80}$$

es decir, una proyección de $\boldsymbol{x}_t$ sobre cada uno de los K subespacios, ponderada por una función softmax de una función cuadrática de $\boldsymbol{x}_t$. ¡Esta forma funcional es similar a un *mecanismo de atención* en una arquitectura de transformer! Como veremos en el Capítulo 5, esto no es en absoluto una coincidencia; el profundo vínculo entre la eliminación de ruido y la compresión con pérdida (que se tratará en la Sección 4.1) hace que los *denoisers* basados en transformers sean tan eficaces en la práctica. Por lo tanto, nuestra teoría del modelo de mezcla gaussiana justifica el uso de redes neuronales tipo transformador para la eliminación de ruido.

En general, la arquitectura del *denoiser* aprendido corresponde a un esquema de codificación (paramétrico implícito) de los datos dados, ya que puede utilizarse para eliminar el ruido o proyectar sobre la distribución de los datos. Entrenar un *denoiser* equivale a encontrar un mejor esquema de codificación, lo que implementa parcialmente el enfoque A2 (es decir, buscar una distribución mediante la exploración de esquemas con mejores tasas de codificación) al final de la Sección 3.1.3. A continuación, discutimos cómo implementar el otro enfoque.

3.2.2 Muestreo de una distribución mediante eliminación iterativa de ruido

Recordemos que, al final de la Sección 3.1.3, analizamos dos enfoques para obtener una distribución con estructura de baja dimensión. El primer enfoque (A1) comienza con una distribución normal de alta entropía y reduce gradualmente su entropía hasta alcanzar la distribución de los datos. A este procedimiento lo llamamos *muestreo*, ya que generamos nuevas muestras. Ha llegado el momento de analizar cómo poner esto en práctica con el conjunto de herramientas que hemos creado.

Sabemos cómo eliminar el ruido de muestras muy ruidosas $\boldsymbol{x}_T$ para obtener aproximaciones $\hat{\boldsymbol{x}}$ cuyas distribuciones se asemejan a la de la variable aleatoria objetivo $\boldsymbol{x}$. Sin embargo, el procedimiento de muestreo nos obliga a partir de una distribución modelo que *no* se vea influida por la distribución de $\boldsymbol{x}$ y a utilizar el *denoiser* para guiar las iteraciones hacia la distribución de $\boldsymbol{x}$. ¿Cómo podemos hacerlo? Una forma de hacerlo se justifica a continuación:

$$\frac{\boldsymbol{x}_T}{T} = \frac{\boldsymbol{x} + T\boldsymbol{g}}{T} = \frac{\boldsymbol{x}}{T} + \boldsymbol{g} \xrightarrow{T \to +\infty} \boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}). \tag{3.2.81}$$

Por lo tanto, $\boldsymbol{x}_T \approx \mathcal{N}(\boldsymbol{0}, T^2\boldsymbol{I})$. Esta aproximación es bastante buena para casi todas las distribuciones prácticas, y se muestra en la Figura 3.7.

Así pues, al discretizar $[0, T]$ en $0 = t_0 < t_1 < \cdots < t_L = T$ de manera uniforme utilizando $t_\ell = T\ell/L$ (como en la sección anterior), una forma posible de obtener muestras a partir de ruido puro es:

- Muestrear $\hat{\boldsymbol{x}}_T \sim \mathcal{N}(\boldsymbol{0}, T^2\boldsymbol{I})$ (independientemente de todo lo demás).

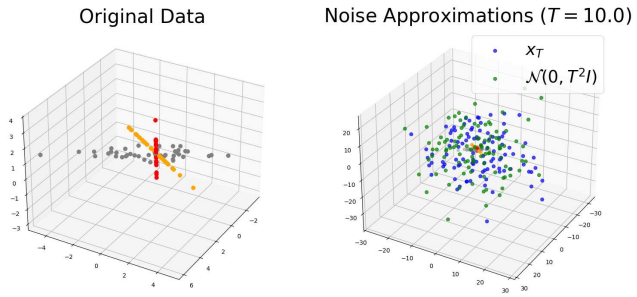


Figura 3.7: **Visualización de $\boldsymbol{x}_T$ vs $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$.** *Izquierda:* Gráfico de los datos del modelo de mezcla gaussiana $\boldsymbol{x}$. *Derecha:* Gráfico de $\boldsymbol{x}$, así como de $\boldsymbol{x}_T$ y una muestra independiente de $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$, para $T = 10$. En el gráfico de la derecha, $\boldsymbol{x}$ se representa con los mismos colores que en el de la izquierda; sin embargo, las muestras de $\boldsymbol{x}_T$ y $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$ son, en promedio, mucho más grandes que las muestras de $\boldsymbol{x}$, por lo que este último parece mucho más pequeño debido a la escala. A pesar de esta gran ampliación, observamos claramente las similitudes en las distribuciones de $\boldsymbol{x}_T$ y $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$.

- Ejecutar la iteración de eliminación de ruido tal y como se indica en la Sección 3.2.1, es decir,

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} = \left(1 - \frac{1}{\ell}\right)\hat{\boldsymbol{x}}_{t_\ell} + \frac{1}{\ell}\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}). \tag{3.2.82}$$

- Output $\hat{\boldsymbol{x}} = \hat{\boldsymbol{x}}_0$.

Conceptualmente, esto es todo lo que hay detrás de los *modelos de difusión*, que transforman el ruido en muestras de datos de acuerdo con la primera propiedad deseada. Sin embargo, aún quedan algunos pasos por dar antes de obtener modelos que puedan realmente tomar muestras de distribuciones de datos reales, como imágenes, bajo restricciones prácticas de recursos. En lo que sigue, presentamos y justificamos varios de esos pasos.

Paso 1: Diferentes discretizaciones. El primer paso se basa en la observación de que *no necesitamos realizar tantas iteraciones de eliminación de ruido* para valores grandes de t. La Figura 3.5 muestra que las primeras 200 o 300 iteraciones de las 500 iteraciones del proceso de muestreo se dedican a reducir el ruido hacia la distribución de los datos en su conjunto, antes de que las iteraciones restantes empujen las muestras hacia un subespacio. Dado un número fijo de iteraciones L, esto sugiere que deberíamos asignar más pasos temporales t_ℓ cerca de $t = 0$ que cerca de $t = T$. Durante el muestreo (y el entrenamiento), podemos, por lo tanto, utilizar una discretización alternativa de $[0, T]$ en $0 \leq t_0 < t_1 < \cdots < t_L \leq T$, como por ejemplo una *discretización exponencial*:

$$t_\ell = C_1(e^{C_2\ell} - 1), \qquad \forall \ell \in \{0, 1, \ldots, L\} \tag{3.2.83}$$

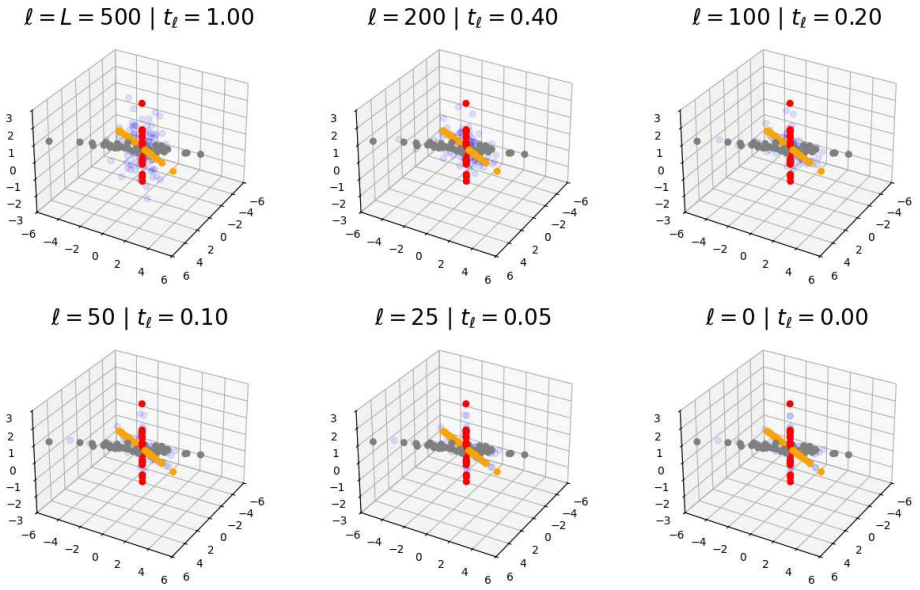


Figura 3.8: **Eliminación del ruido en una mezcla de distribuciones gaussianas mediante el proceso de difusión VP.** Utilizamos la misma configuración de la figura y la misma distribución de datos que en la Figura 3.5. Nótese que, en comparación con la Figura 3.5, la distribución del ruido está mucho más concentrada alrededor del origen.

donde $C_1, C_2 > 0$ son constantes que pueden ajustarse para obtener un rendimiento óptimo en la práctica; los análisis teóricos suelen especificar también dichas constantes óptimas. La iteración de eliminación de ruido/muestreo se convierte entonces en

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} \doteq \frac{t_{\ell-1}}{t_\ell}\hat{\boldsymbol{x}}_{t_\ell} + \left(1 - \frac{t_{\ell-1}}{t_\ell}\right)\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}), \tag{3.2.84}$$

con $\hat{\boldsymbol{x}}_{t_L} \sim \mathcal{N}(\mathbf{0}, t_L^2\boldsymbol{I})$.

Paso 2: Modelos de ruido diferentes. El segundo paso consiste en considerar modelos ligeramente distintos a los de (3.2.1). La motivación fundamental para esto es la siguiente. En la práctica, la distribución del ruido $\mathcal{N}(\mathbf{0}, t_L^2\boldsymbol{I})$ se convierte en una estimación cada vez menos precisa de la covarianza real en dimensiones elevadas; es decir, (3.2.81) se convierte en una aproximación cada vez menos precisa, especialmente con datos anisotrópicos de alta dimensión. El aumento de la distancia entre $\mathcal{N}(\mathbf{0}, t_L^2\boldsymbol{I})$ y la distribución real de $\boldsymbol{x}_{t_L}$ puede hacer que el *denoiser* funcione peor en tales circunstancias. En teoría, $\boldsymbol{x}_{t_L}$ nunca converge a ninguna distribución a medida que t_L aumenta, por lo que esta configuración es difícil de analizar de extremo a extremo. En

este caso, nuestra solución consiste en *añadir ruido y reducir la contribución de* $\boldsymbol{x}$ *al mismo tiempo, de modo que* $\boldsymbol{x}_T$ *converja cuando* $T \to \infty$. La tasa de ruido añadido se denota por $\sigma\colon [0,T] \to \mathbb{R}_{\geq 0}$ y la tasa de contracción se denota por $\alpha\colon [0,T] \to \mathbb{R}_{\geq 0}$, de modo que σ es *creciente* y α es (no estrictamente) *decreciente*, y

$$\boldsymbol{x}_t \doteq \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{g}, \qquad \forall t \in [0,T]. \tag{3.2.85}$$

En la configuración anterior, $\alpha_t = 1$ y $\sigma_t = t$ y esto se conoce como el *proceso de varianza explosiva (VE)*. Una opción habitual que reduce la contribución de $\boldsymbol{x}$, como lo describimos inicialmente, es aquella en la que $T = 1$ (de modo que $t \in [0,1]$), $\alpha_t = \sqrt{1-t^2}$ y $\sigma_t = t$; este es el *proceso de preservación de la varianza (VP)*. Nótese que, en el proceso VP, $\boldsymbol{x}_1 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ exactamente, por lo que basta con tomar muestras de esta distribución estándar y aplicar el proceso de eliminación de ruido en forma iterativa. Como resultado, el proceso VP es mucho más fácil de analizar teóricamente y más estable empíricamente.[6]

Con este planteamiento más general, la fórmula de Tweedie (3.2.23) se convierte en

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] = \frac{1}{\alpha_t}\left(\boldsymbol{x}_t + \sigma_t^2 \nabla \log p_t(\boldsymbol{x}_t)\right). \tag{3.2.86}$$

La iteración de eliminación de ruido (3.2.84) se transforma en

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} = \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \hat{\boldsymbol{x}}_{t_\ell} + \left(\alpha_{t_{\ell-1}} - \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \alpha_{t_\ell}\right) \bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}). \tag{3.2.87}$$

Ejemplo 3.6 (El modelo de mezcla gaussiana). Para un modelo de mezcla gaussiana (véase el Ejemplo 3.2), su filtro de eliminación de ruido (3.2.19) se convierte en

$$\bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) = \sum_{k=1}^{K} \left\{ \frac{\pi_k \varphi(\boldsymbol{x}_t; \alpha_t \boldsymbol{\mu}_k, \alpha_t^2 \boldsymbol{\Sigma}_k + \sigma_t^2 \boldsymbol{I})}{\sum_{i=1}^{K} \pi_i \varphi(\boldsymbol{x}_t; \alpha_t \boldsymbol{\mu}_i, \alpha_t^2 \boldsymbol{\Sigma}_i + \sigma_t^2 \boldsymbol{I})} \right. \tag{3.2.88}$$
$$\left. \cdot \left(\boldsymbol{\mu}_k + \alpha_t \boldsymbol{\Sigma}_k (\alpha_t^2 \boldsymbol{\Sigma}_k + \sigma_t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \alpha_t \boldsymbol{\mu}_k)\right) \right\}.$$

La Figura 3.8 muestra iteraciones del procedimiento de muestreo. ■

Cabe señalar que la iteración de eliminación de ruido (3.2.87) da lugar a un algoritmo de muestreo denominado DDIM («Denoising Diffusion Implicit Model») [SME20], y es uno de los algoritmos de muestreo más populares utilizados actualmente en los modelos de difusión. Lo resumimos en el Algoritmo 3.1.

[6] ¿Por qué utilizar toda la configuración de α, σ? Como veremos en el Ejercicio 3.5, esta encapsula y unifica muchos de los procesos propuestos, incluido el recientemente popular proceso denominado *flow matching*. A pesar de ello, los procesos VE y VP siguen siendo los más populares tanto desde el punto de vista empírico como teórico (hasta ahora), por lo que los abordaremos en esta sección.

Algoritmo 3.1: Muestreo mediante *denoiser*.

Input: Una lista ordenada de pasos de tiempo $0 \leq t_0 < \cdots < t_L \leq T$ que se utilizarán para el muestreo.
Input: Un filtro de eliminación de ruido $\bar{\boldsymbol{x}}\colon \{t_\ell\}_{\ell=1}^L \times \mathbb{R}^D \to \mathbb{R}^D$.
Input: Funciones de escala y de nivel de ruido $\alpha, \sigma\colon \{t_\ell\}_{\ell=0}^L \to \mathbb{R}_{\geq 0}$.
Output: Una muestra $\hat{\boldsymbol{x}}$, aproximadamente de la distribución de $\boldsymbol{x}$.

function DDIMSampler($\bar{\boldsymbol{x}}, (t_\ell)_{\ell=0}^L$)
  Inicializar $\hat{\boldsymbol{x}}_{t_L} \sim$ la distribución aproximada de $\boldsymbol{x}_{t_L}$
  `# VP` $\implies \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, `VE` $\implies \mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$.
  for $\ell = L, L-1, \ldots, 1$ **do**
    Calcular

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} \doteq \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \hat{\boldsymbol{x}}_{t_\ell} + \left(\alpha_{t_{\ell-1}} - \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \alpha_{t_\ell} \right) \bar{\boldsymbol{x}}(t_\ell, \hat{\boldsymbol{x}}_{t_\ell})$$

  end for
  return $\hat{\boldsymbol{x}}_{t_0}$
end function

Paso 3: Optimización de los procesos de entrenamiento. Si usáramos el procedimiento prescrito en la Sección 3.2.1 para entrenar un *denoiser* independiente $\bar{\boldsymbol{x}}(t, \cdot)$ para cada instante t que se utilizará en el algoritmo de muestreo, *¡tendríamos que entrenar L denoisers independientes!* Esto es muy ineficiente: lo habitual es que tengamos que entrenar L redes neuronales distintas, lo que requiere L veces más tiempo de entrenamiento y memoria de almacenamiento y luego quedarnos atados a utilizar esos intervalos de tiempo para el muestreo de por siempre. En cambio, podemos *entrenar una sola red neuronal* para eliminar el ruido en todos los instantes t, tomando como entrada las variables continuas $\boldsymbol{x}_t$ y t (en lugar de solo $\boldsymbol{x}_t$ como antes). Mecánicamente, nuestra pérdida de entrenamiento se promedia a lo largo de t, es decir, resuelve el siguiente problema:

$$\min_\theta \mathbb{E} \, \|\bar{\boldsymbol{x}}_\theta(\tau, \boldsymbol{x}_\tau) - \boldsymbol{x}\|_2^2, \tag{3.2.89}$$

donde $\tau \sim \mathcal{U}([0, T])$, es decir, se extrae uniformemente del intervalo $[0, T]$. Al igual que en el Paso 1, donde utilizamos más intervalos de tiempo más cercanos a $t = 0$ para garantizar un mejor proceso de muestreo, es posible que queramos asegurarnos de que el *denoiser* sea de mayor calidad cuanto más cerca de $t = 0$ y por lo tanto *ponderar la pérdida* de modo que t cerca de 0 tenga mayor ponderación.[7] Si w_t es el peso en el momento t, la pérdida ponderada tendría

[7] En la práctica, la distribución de la t muestreada también puede ser no uniforme, con el fin de garantizar (junto con el esquema de ponderación) que la estimación de Monte Carlo de la pérdida tenga una varianza baja, lo que mejora considerablemente la estabilidad del entrenamiento.

la forma

$$\min_{\theta} \mathbb{E}[w_\tau \|\bar{\boldsymbol{x}}_\theta(\tau, \boldsymbol{x}_\tau) - \boldsymbol{x}\|_2^2]. \tag{3.2.90}$$

Una opción razonable para el peso en la práctica es $w_t = \alpha_t^2/\sigma_t^2$. La razón exacta se explicará en el siguiente párrafo, pero, en general, sirve para dar mayor peso a las pérdidas correspondientes a t cerca de 0, sin dejar de mantener una estabilidad numérica razonable. Además, por supuesto, en la práctica no podemos calcular la esperanza, por lo que utilizamos la media de Monte Carlo, que es la más sencilla, para estimarla. La serie de cambios que se han realizado aquí ofrece varias ventajas conceptuales y computacionales: no necesitamos entrenar múltiples *denoisers*, podemos entrenar con un solo conjunto de pasos de tiempo y realizar el muestreo utilizando un subconjunto (o incluso otros distintos), etc. El proceso completo se analiza en el Algoritmo 3.2. Sin duda, este proceso funciona en la práctica; comentamos algunos resultados en las Secciones 8.6 a 8.10.

Paso 4: Cambio del objetivo de estimación. Es habitual, en su lugar, reorientar todo el proceso de eliminación de ruido en torno a los *predictores de ruido*, es decir, las estimaciones de $\mathbb{E}[\boldsymbol{g} \mid \boldsymbol{x}_t]$. En la práctica, los predictores de ruido son ligeramente más fáciles de entrenar porque su salida es (casi) siempre de un tamaño comparable al de una variable aleatoria gaussiana, por lo que el entrenamiento es más estable numéricamente, pero pueden requerir modelos más grandes, ya que predicen un objetivo de dimensión completa en lugar de datos potencialmente de baja dimensión [LH25]. Por (3.2.85) tenemos

$$\boldsymbol{x}_t = \alpha_t \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] + \sigma_t \mathbb{E}[\boldsymbol{g} \mid \boldsymbol{x}_t] \implies \mathbb{E}[\boldsymbol{g} \mid \boldsymbol{x}_t] = \frac{1}{\sigma_t}\left(\boldsymbol{x}_t - \alpha_t \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t]\right), \tag{3.2.91}$$

por lo que cualquier predictor de $\boldsymbol{x}$ puede transformarse en un predictor de $\boldsymbol{g}$ utilizando la relación anterior, es decir,

$$\bar{\boldsymbol{g}}(t, \boldsymbol{x}_t) = \frac{1}{\sigma_t}\boldsymbol{x}_t - \frac{\alpha_t}{\sigma_t}\bar{\boldsymbol{x}}(t, \boldsymbol{x}_t), \tag{3.2.92}$$

y viceversa. Por lo tanto, una buena red para estimar $\bar{\boldsymbol{g}}$ es lo mismo que una buena red para estimar $\bar{\boldsymbol{x}}$ *más una conexión residual* (como se ve, por ejemplo, en los transformers). Sus pérdidas son también las mismas que las del *denoiser*, salvo por el factor α_t/σ_t, es decir,

$$\mathbb{E}[w_\tau \|\boldsymbol{g} - \bar{\boldsymbol{g}}(\tau, \boldsymbol{x}_\tau)\|_2^2] = \mathbb{E}\left[w_\tau \frac{\alpha_\tau^2}{\sigma_\tau^2}\|\boldsymbol{x} - \bar{\boldsymbol{x}}(\tau, \boldsymbol{x}_\tau)\|_2^2\right], \tag{3.2.93}$$

recordando que $\tau \sim \mathcal{U}([0, T])$. Se han propuesto otros objetivos para diferentes tareas, por ejemplo, $\mathbb{E}[\boldsymbol{v}_t \mid \boldsymbol{x}_t]$ donde $\boldsymbol{v}_t = \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_t$ (denominada predicción v o predicción de velocidad), etc. Esto es equivalente a la eliminación de ruido y la predicción de ruido, pero también resulta fundamentalmente útil para generalizaciones de la difusión, como el flow matching, que analizaremos en la Nota 3.3.

Algoritmo 3.2: Aprender un *denoiser* a partir de datos.

Input: Conjunto de datos $\mathcal{D} \subseteq \mathbb{R}^D$.
Input: Una lista ordenada de pasos de tiempo $0 \leq t_0 < \cdots < t_L \leq T$ que se utilizarán para el muestreo.
Input: Una función de ponderación $w\colon \{t_\ell\}_{\ell=1}^L \to \mathbb{R}_{\geq 0}$.
Input: Funciones de escala y nivel de ruido $\alpha, \sigma\colon \{t_\ell\}_{\ell=0}^L \to \mathbb{R}_{\geq 0}$.
Input: Un espacio de parámetros Θ y una arquitectura de *denoiser* $\bar{\boldsymbol{x}}_\theta$.
Input: Un algoritmo de optimización para los parámetros.
Input: El número de iteraciones de optimización M.
Input: El número de extracciones de Monte Carlo N por paso de entrenamiento.
Output: Un *denoiser* entrenado $\bar{\boldsymbol{x}}_{\theta^\star}$.

function TrainDenoiser($\mathcal{D}, \Theta$)
  Inicializar $\theta^{(1)} \in \Theta$
  for $m \in [M]$ **do**
    for $i \in [N]$ **do**
      `# Extraer una muestra del conjunto de datos.`
      $\boldsymbol{x}^{(m,i)} \sim \mathcal{D}$
      `# Muestrear un intervalo de tiempo.`
      $t^{(m,i)} \overset{\text{i.i.d.}}{\sim} \mathcal{U}(\{t_\ell\}_{\ell=1}^L)$
      `# Calcular ponderaciones y escalas.`
      $(\alpha^{(m,i)}, \sigma^{(m,i)}, w^{(m,i)}) = (\alpha_{t^{(m,i)}}, \sigma_{t^{(m,i)}}, w_{t^{(m,i)}})$
      `# Muestrear un vector de ruido.`
      $\boldsymbol{g}^{(m,i)} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$
      `# Calcular la muestra ruidosa.`
      $\boldsymbol{x}_t^{(m,i)} \doteq \alpha^{(m,i)} \boldsymbol{x}^{(m,i)} + \sigma^{(m,i)} \boldsymbol{g}^{(m,i)}$
    end for
    `# Calcular la pérdida de entrenamiento.`
    $\hat{\mathcal{L}}^{(m)} \doteq \dfrac{1}{N} \displaystyle\sum_{i=1}^{N} w^{(m,i)} \| \boldsymbol{x}^{(m,i)} - \bar{\boldsymbol{x}}_{\theta^{(m)}}(t^{(m,i)}, \boldsymbol{x}_t^{(m,i)}) \|_2^2$
    `# Actualizar los parámetros del modelo.`
    $\theta^{(m+1)} \doteq \texttt{OptimUpdate}^{(m)}(\theta^{(m)}, \nabla_{\theta^{(m)}} \hat{\mathcal{L}}^{(m)})$
  end for
  `# Devolver el denoiser entrenado.`
  return $\bar{\boldsymbol{x}}_{\theta^{(M+1)}}$
end function

Tasa de error de muestreo del proceso de eliminación de ruido

Hemos introducido muchos cambios en nuestro proceso platónico original de generación y eliminación de ruido. Para asegurarnos de que el nuevo proceso sigue funcionando en la práctica, podemos calcular ejemplos numéricos (como en la Figura 3.8). Para asegurarnos de que es teóricamente sólido, podemos demostrar una *cota de la tasa de error* para el algoritmo de muestreo, lo que demuestra que la tasa de error es pequeña. A continuación, presentamos una cota de este tipo tomada de la literatura, que muestra que la distribución de los resultados del muestreador converge, en la denominada *distancia de variación total (TV)*, hacia la distribución verdadera. La distancia TV entre dos variables aleatorias $\boldsymbol{x}$ y $\boldsymbol{y}$ se define como:[8]

$$\mathsf{TV}(\boldsymbol{x}, \boldsymbol{y}) \doteq \sup_{A \subseteq \mathbb{R}^d} \left| \mathbb{P}[\boldsymbol{x} \in A] - \mathbb{P}[\boldsymbol{y} \in A] \right|. \tag{3.2.94}$$

Si $\boldsymbol{x}$ y $\boldsymbol{y}$ están muy cerca (uniformemente), entonces el supremo será pequeño. Así pues, la distancia TV mide la proximidad entre variables aleatorias. (Se trata, efectivamente, de una métrica, como sugiere su nombre; la demostración es un ejercicio.)

Teorema 3.4 ([LY24] Teorema 1, simplificado)**.** *Supongamos que* $\mathbb{E}\,\|\boldsymbol{x}\|_2 < \infty$*. Si se elimina el ruido de* $\boldsymbol{x}$ *según el proceso VP con una discretización exponencial*[9] *como en* (3.2.83)*, la salida* $\hat{\boldsymbol{x}}$ *del Algoritmo 3.1 satisface la cota de variación total*

$$\mathsf{TV}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \tilde{\mathcal{O}}\Bigg(\underbrace{\frac{D}{L}}_{\textit{error de discretización}} + \underbrace{\sqrt{\frac{1}{L}\sum_{\ell=1}^{L} \frac{\alpha_{t_\ell}^2}{\sigma_{t_\ell}^4}\, \mathbb{E}\,\|\bar{\boldsymbol{x}}^\star(t_\ell, \boldsymbol{x}_{t_\ell}) - \bar{\boldsymbol{x}}(t_\ell, \boldsymbol{x}_{t_\ell})\|_2^2}}_{\textit{error medio excesivo del denoiser}} \Bigg) \tag{3.2.95}$$

donde $\bar{\boldsymbol{x}}^\star$ *es el denoiser óptimo de Bayes para* $\boldsymbol{x}$ *y* $\tilde{\mathcal{O}}$ *es una versión de la notación «big-*$\mathcal{O}$*» que ignora los factores logarítmicos en* L*.*

La técnica de demostración de muy alto nivel consiste, como se ha comentado anteriormente, en limitar el error en cada paso, distinguir las fuentes de error (entre el error de discretización y el del *denoiser*) y asegurarse cuidadosamente de que los errores no se acumulen en exceso (o incluso se anulen entre sí).

Téngase en cuenta que si $L \to \infty$ y aprendemos correctamente el *denoiser* óptimo de Bayes $\bar{\boldsymbol{x}} = \bar{\boldsymbol{x}}^\star$ (de modo que el error excedente es 0), entonces

[8] El supremo aquí se toma únicamente sobre conjuntos A *medibles*, no a todos los conjuntos sin excepción. Al igual que con todas las demás reservas relacionadas con la teoría de la medida que aparecen en este libro, no dudes en ignorarlo a efectos prácticos.

[9] La definición exacta es bastante extensa en nuestra notación y solo se define hasta diversas constantes absolutas, por lo que la omitimos aquí por brevedad. Se encuentra en el artículo original [LY24].

el proceso de muestreo en el Algoritmo 3.1 produce una *inversa perfecta (en distribución)* del proceso de adición de ruido, ya que la tasa de error en el Teorema 3.4 tiende a 0,[10] como se ha argumentado heurísticamente más arriba.

Nota 3.1. ¿Qué pasa si los datos son de baja dimensión, es decir, si se encuentran en un *subespacio de rango bajo* del espacio de alta dimensión $\mathbb{R}^D$? Si la distribución de los datos tiene soporte compacto —por ejemplo, si los datos se normalizan al hipercubo unitario, lo cual suele garantizarse como paso previo al procesamiento de datos reales, como las imágenes— es posible obtener mejores resultados. Concretamente, los autores de [LY24] también definen una medida de la *dimensión intrínseca aproximada* κ utilizando la asintótica del denominado número de cobertura, que es extremadamente similar en su intuición (si no en su implementación) a la función de tasa-distorsión que se presenta en el siguiente capítulo. A continuación, muestran que, al aplicar una pequeña modificación concreta al muestreador DDIM del Algoritmo 3.1 (es decir, perturbando ligeramente los coeficientes de actualización), el error de discretización pasa a ser $\tilde{\mathcal{O}}(\kappa/L)$ en lugar de $\tilde{\mathcal{O}}(D/L)$ como en el Teorema 3.4. Por lo tanto, al utilizar este algoritmo modificado, L no tiene por qué ser demasiado grande, incluso cuando D alcanza valores de miles o millones, ya que los datos reales tienen una estructura de baja dimensión. Sin embargo, en la práctica utilizamos el muestreador DDIM, por lo que L debería tener una dependencia moderada de D para lograr tasas de error consistentes. La elección concreta de L supone un equilibrio entre la complejidad computacional (por ejemplo, el tiempo de ejecución o el consumo de memoria) del muestreo y la complejidad estadística de aprender un *denoiser* para estructuras de baja dimensión. El valor de L suele ser diferente en la fase de entrenamiento (donde un L mayor permite una mejor cobertura del intervalo $[0, T]$, lo que ayuda a la red a aprender una relación que se generaliza sobre t) y en la fase de muestreo (donde un L menor implica un muestreo más eficiente). Incluso se pueden seleccionar los intervalos de tiempo en forma adaptativa en el momento del muestreo para optimizar este equilibrio [BLZ+22].

Nota 3.2. En otros trabajos, el proceso inverso se define como un desplazamiento hacia atrás en el índice temporal t mediante una ecuación de diferencias explícita, o una ecuación diferencial en el límite $L \to \infty$, o hacia adelante en el tiempo utilizando la transformación $\boldsymbol{y}_t = \boldsymbol{x}_{T-t}$, de tal manera que si t aumenta, entonces $\boldsymbol{y}_t$ se acerca a $\boldsymbol{x}_0$. Cada artículo utiliza su propia notación y a veces puede resultar confuso comprender las relaciones entre los resultados de los distintos artículos. En este trabajo nos esforzamos por mantener la coherencia: avanzamos en el tiempo para introducir ruido y retrocedemos en el tiempo para eliminarlo. Al leer otro trabajo en el que no queda claro el índice temporal, o al intentar implementar un algoritmo que presenta una ambigüedad similar, hay una forma de resolverla en todos los casos: el proceso de muestreo siempre debe tener un *coeficiente positivo* en el término del *denoiser* (o, equivalentemente, en la función de *score*) al dar un paso.

[10] Se obtienen resultados similares para los procesos VE; las herramientas teóricas utilizadas para las demostraciones difieren debido a la estructura de los procesos estocásticos subyacentes.

Nota 3.3. Los procesos de difusión y eliminación de ruido, junto con los modelos asociados, ofrecen una forma de transportar la distribución de los datos hacia y desde una distribución de alta entropía (gaussiana o aproximadamente gaussiana). Por otro lado, puede resultar útil transportar la distribución de los datos hacia y desde una distribución objetivo arbitraria, que podría ser gaussiana, una mezcla gaussiana o incluso otro tipo de distribución de datos. Si tenemos este requisito, podemos utilizar el marco de *flow matching* para lograrlo. Supongamos que la variable aleatoria objetivo es $\boldsymbol{y}$ y que nuestro flujo es

$$\boldsymbol{x}_t = \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{y}, \qquad \forall t \in [0,1]. \tag{3.2.96}$$

de tal manera que $\alpha_0 = \sigma_1 = 1$ y $\alpha_1 = \sigma_0 = 0$, de modo que $\boldsymbol{x}_0 = \boldsymbol{x}$ y $\boldsymbol{x}_1 = \boldsymbol{y}$. Nuestra estrategia no cambia en absoluto: podemos entrenar un *denoiser* $\bar{\boldsymbol{x}}_\theta$ para eliminar el ruido de $\boldsymbol{x}_t$ y volver a obtener $\boldsymbol{x}$ (o, lo que es lo mismo, un predictor de «ruido» para predecir $\boldsymbol{y}$) y luego aplicar nuestros algoritmos iterativos de eliminación de ruido para partir de una realización de $\boldsymbol{y}$ y obtener $\boldsymbol{x}$. Lo único que falta es la fórmula de Tweedie que relaciona el *score* $\nabla \log p_t$ del *denoiser* con el *denoiser* propiamente tal; dicha fórmula no existe en general.

En la práctica, inspirándose en el concepto de aprender un llamado *mapa de transporte* entre pares de datos $\boldsymbol{x}$ y $\boldsymbol{y}$, los profesionales suelen preferir entrenar un *predictor de velocidad* $\bar{\boldsymbol{v}}_\theta$ para predecir $\boldsymbol{v}_t = \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_t = \alpha_t' \boldsymbol{x} + \sigma_t' \boldsymbol{y}$ en lugar de un *denoiser* $\bar{\boldsymbol{x}}_\theta$, o más bien entrenar $\bar{\boldsymbol{v}}_\theta$ para predecir $\mathbb{E}[\boldsymbol{v}_t \mid \boldsymbol{x}_t]$. En este caso, lo más sencillo es elegir los coeficientes $\alpha_t = 1 - t$ y $\sigma_t = t$ (para $t \in [0,1]$), de modo que el objetivo de predicción sea $\boldsymbol{v}_t = \boldsymbol{y} - \boldsymbol{x}$ para cada instante t. Dado el estimador $\bar{\boldsymbol{v}}_\theta$, el algoritmo de muestreo estándar recorre el campo hacia atrás en el tiempo; es decir, partiendo de una realización de $\boldsymbol{y}$, podemos actualizar iterativamente el iterado de la siguiente manera:

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} = \hat{\boldsymbol{x}}_{t_\ell} - \bar{\boldsymbol{v}}_\theta(t_\ell, \hat{\boldsymbol{x}}_{t_\ell})\Delta t, \tag{3.2.97}$$

donde $\Delta t = t_\ell - t_{\ell-1}$ es el intervalo de tiempo. Téngase en cuenta que la predicción de velocidad es algebraicamente equivalente a la eliminación de ruido (véase el Ejercicio 3.6). Las Secciones 8.6 y 8.9 contienen aplicaciones prácticas del flow matching.

Nota 3.4. En la práctica, la difusión suele requerir muchos pasos y muchas evaluaciones de la función del *denoiser*. Por lo tanto, es lógico preguntarse: «¿Podemos acelerar el proceso de muestreo utilizando un número menor de pasos y evaluaciones de la función?». La respuesta es afirmativa y a esos modelos se les suele llamar *modelos de consistencia* o, de manera más descriptiva, *modelos de pocos pasos*. La idea central consiste en utilizar un *denoiser* que tome dos veces $\bar{\boldsymbol{x}}_\theta(s,t,\boldsymbol{x}_t)$ y estime $\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]$, lo cual, en cierto sentido, es una tarea más sencilla, o bien utilizar un predictor de velocidad $\bar{\boldsymbol{v}}_\theta(s,t,\boldsymbol{x}_t)$ y estimar la velocidad media $\mathbb{E}[(\boldsymbol{x}_t - \boldsymbol{x}_s)/(t-s) \mid \boldsymbol{x}_t]$. Estos filtros de eliminación de ruido tienen condiciones de consistencia adicionales, tales como $\bar{\boldsymbol{x}}_\theta(u,t,\bar{\boldsymbol{x}}_\theta(s,u,\boldsymbol{\xi})) = \bar{\boldsymbol{x}}_\theta(s,t,\boldsymbol{\xi})$, lo que motiva el uso de pérdidas de entrenamiento auxiliares. Una vez que se aprende dicho predictor, el proceso de muestreo se vuelve muy simple

y eficiente, ya que solo requiere unos pocos pasos de la forma

$$\hat{\boldsymbol{x}}_s = \bar{\boldsymbol{x}}_\theta(s, t, \hat{\boldsymbol{x}}_t) = \hat{\boldsymbol{x}}_t - \bar{\boldsymbol{v}}_\theta(s, t, \hat{\boldsymbol{x}}_t) \cdot (t - s). \tag{3.2.98}$$

Entre las formulaciones relevantes se incluyen los mapas de flujo [BAV25] y los flujos medios [GDB+25].

3.3 Memorización, generalización y tasas de codificación

El cálculo que se presenta al final de la Sección 3.2.1 parece indicar (en términos generales) que, en la práctica, utilizar una red de tipo transformer es una buena opción para entrenar o aproximar un *denoiser*. Esto es razonable, pero ¿qué problema hay en utilizar cualquier red neuronal corriente (como un perceptrón multicapa (MLP)) y simplemente escalarla hasta el infinito? Para responder a esta pregunta, consideremos una restricción que hasta ahora hemos ignorado o pasado por alto en este capítulo: solo disponemos de datos finitos con los que entrenar el *denoiser*. De hecho, consideremos el problema de entrenamiento (3.2.90), salvo que esta vez suponemos que nuestros datos $\boldsymbol{x}$ se extraen de la distribución empírica $p^{\boldsymbol{X}}$ sobre N muestras de entrenamiento $\boldsymbol{X} = [\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)}]$. Dado que $p^{\boldsymbol{X}}$ está soportado en un subconjunto de dimensión 0 del espacio envolvente $\mathbb{R}^D$, tenemos que $h(p^{\boldsymbol{X}}) = -\infty$, por lo que, en cierto sentido, $p^{\boldsymbol{X}}$ es la distribución más simple (es decir, la de menor entropía) que podría generar nuestras observaciones $\boldsymbol{X}$. En resumen, ¡podría decirse que esta hipótesis *tiene el menor sesgo inductivo* respecto a la distribución de los datos! Muchas narrativas populares sobre el aprendizaje profundo y los modelos generativos quieren hacernos creer que, debido a esta propiedad, la mejor manera de resolver el problema del modelado generativo es escalar un *denoiser* genérico al máximo y aprovechar las ventajas correspondientes. Sin embargo, este consejo es (¡demostrablemente!) erróneo en este contexto. Para entender por qué, notemos que la distribución empírica $p^{\boldsymbol{X}}$ es, en realidad, una mezcla de distribuciones gaussianas con covarianzas nulas:

$$p^{[\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)}]} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{N}(\boldsymbol{x}^{(i)}, \boldsymbol{0}). \tag{3.3.1}$$

En consecuencia, sabemos que la solución óptima (a nivel global) al problema de eliminación de ruido (3.2.90) con $\boldsymbol{x}$ extraído de la distribución empírica se expresa de la siguiente manera:

$$\min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}[w_\tau \|\boldsymbol{x}^{(i)} - \bar{\boldsymbol{x}}_\theta(\tau, \boldsymbol{x}_\tau^{(i)})\|_2^2] \tag{3.3.2}$$

donde $\boldsymbol{x}_t^{(i)}$ es la versión con ruido de la muestra de entrenamiento $\boldsymbol{x}^{(i)}$. El optimizador de esta pérdida sobre todos los eliminadores de ruido

(cuadráticamente integrables) $\bar{\boldsymbol{x}}$ es el *denoiser* gaussiano óptimo asociado del Ejemplo 3.3, es decir,

$$\bar{\boldsymbol{x}}^{\star}(t, \boldsymbol{x}_t) = \sum_{i=1}^{N} \frac{\exp(-\|\boldsymbol{x}_t - \alpha_t \boldsymbol{x}^{(i)}\|_2^2/(2\sigma_t^2))}{\sum_{j=1}^{N} \exp(-\|\boldsymbol{x}_t - \alpha_t \boldsymbol{x}^{(j)}\|_2^2/(2\sigma_t^2))} \boldsymbol{x}^{(i)}. \tag{3.3.3}$$

Esta es una combinación convexa de los datos $\boldsymbol{x}^{(i)}$ y los coeficientes se vuelven más «agudos» (es decir, más cercanos a 0 o 1) a medida que $t \to 0$. En particular, para valores pequeños de t, el *denoiser* es casi exactamente igual a la muestra más cercana del conjunto de entrenamiento y la actualización acerca la iteración al punto más cercano, de modo que al final del muestreo, la iteración $\hat{\boldsymbol{x}}$ es (casi) igual a una muestra de entrenamiento $\boldsymbol{x}^{(i)}$. Reafirmemos una vez más este punto: *si utilizamos el denoiser* (3.3.3) *y un buen muestreador, nuestras muestras regenerarán los datos de entrenamiento.* Esto queda confirmado por nuestra garantía teórica sobre el muestreo Teorema 3.4, que, cuando se aplica en este contexto, establece que si llevamos el número de pasos de muestreo a ∞ y utilizamos el programa de pasos de tiempo prescrito, el modelo de difusión asociado al *denoiser* (3.3.3) siempre tomará muestras de la distribución empírica y, por lo tanto, siempre reproducirá una muestra de entrenamiento. Este fenómeno constituye una instanciación especialmente estricta de la *memorización* en el modelado generativo. [11]

Dado que, en la práctica, solo disponemos de un conjunto de datos fijo y finito con el que podemos entrenar nuestro *denoiser*, el problema de aprendizaje que pretendemos resolver en este caso es la pérdida de eliminación de ruido con muestra finita (3.3.2) (en comparación con la pérdida de eliminación de ruido con muestra infinita (3.2.90), por ejemplo). Por lo tanto, *si tenemos un modelo arbitrariamente grande y lo optimizamos lo mejor posible, aprenderá a memorizar y a regenerar el conjunto de entrenamiento.* En otras palabras, este modelo a gran escala no es mucho más interesante que una simple base de datos con los datos de entrenamiento.

El argumento anterior es válido para modelos de arbitrariamente potentes que puedan ajustar el (verdadero) *denoiser* memorizador. Por lo tanto, es pertinente preguntarse si esto se confirma realmente en la práctica: ¿los modelos muy grandes (en relación con el conjunto de datos) aprenden a memorizar los datos de entrenamiento? Resulta que esto es cierto desde el punto de vista empírico. Uno de los primeros estudios al respecto fue [YCK+23]. Para comprender sus resultados, definimos la razón de memorización como la probabilidad de memorización:

$$\text{TasaMem}(\bar{\boldsymbol{x}}) = \mathbb{P}_{\hat{\boldsymbol{x}} \sim \text{DM}(\bar{\boldsymbol{x}})}[\hat{\boldsymbol{x}} \text{ ha sido memorizado}], \tag{3.3.4}$$

donde $\text{DM}(\bar{\boldsymbol{x}})$ es la distribución inducida por el modelo de difusión que utiliza

[11] La memorización es un fenómeno mucho más amplio cuya caracterización exacta depende del dominio en el que se entrena el modelo (por ejemplo, texto, imágenes, vídeos, audio, etc.). Más adelante en esta sección analizaremos medidas más heurísticas de la memorización.

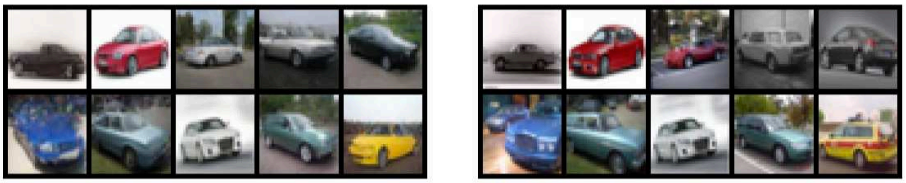


Figura 3.9: **Muestras generadas (izquierda) a partir de un modelo de difusión entrenado con** 6.000 **muestras y (derecha) sus puntos más cercanos en el conjunto de datos de entrenamiento (en la norma euclidiana).** Observamos que las muestras generadas son, en esencia, equivalentes a nivel de pixel a los datos de entrenamiento más similares. Ambos conjuntos de imágenes provienen de [YCK+23].

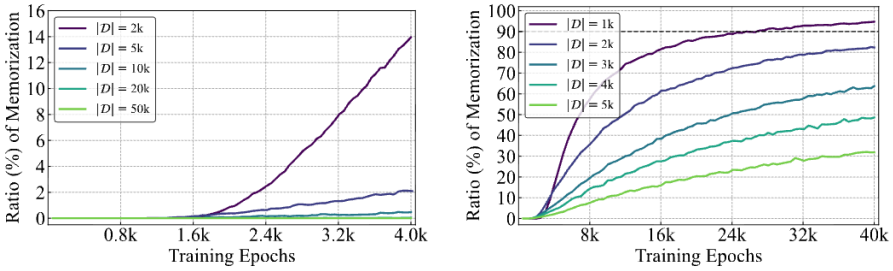


Figura 3.10: **Memorización frente al tamaño del conjunto de datos para un modelo fijo, con (izquierda) 4 000 épocas de entrenamiento y (derecha) 40 000 épocas de entrenamiento con conjuntos de datos más pequeños.** Observamos en el gráfico de la izquierda que los conjuntos de datos lo suficientemente grandes nunca se memorizarán (por ejemplo, de 10 000 a 50 000 muestras), mientras que los conjuntos de datos pequeños (de 1 000 a 5 000 muestras) se memorizarán tras un entrenamiento suficiente. Esto significa que, a medida que el conjunto de datos se vuelve más grande en relación con el modelo, la proporción de memorización al final del entrenamiento pasa de cerca de 1 a cerca de 0 con relativa rapidez, lo que indica la posible existencia de una transición de fase de la memorización a la generalización a medida que aumenta el tamaño del conjunto de datos. Tomada de [GDP+23].

el *denoiser* $\bar{\boldsymbol{x}}$, y la muestra $\hat{\boldsymbol{x}}$ se memoriza utilizando el siguiente criterio:

$$\hat{\boldsymbol{x}} \text{ está memorizado} \iff \frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}_{(1)}(\hat{\boldsymbol{x}})\|_2}{\|\hat{\boldsymbol{x}} - \boldsymbol{x}_{(2)}(\hat{\boldsymbol{x}})\|_2} < \frac{1}{3}, \tag{3.3.5}$$

donde $\boldsymbol{x}_{(i)}(\hat{\boldsymbol{x}})$ es el i-ésimo punto más cercano a $\hat{\boldsymbol{x}}$ en el conjunto de datos de entrenamiento (en la norma euclidiana) y $1/3$ es una constante que, según se ha demostrado empíricamente, concuerda bien con la percepción humana de la memorización. Nótese que esta definición se corresponde exactamente con nuestra definición intuitiva anterior de «memorización», según la cual cada uno de los puntos muestreados está mucho más cerca de una muestra de entrenamiento concreta que de todas las demás muestras de entrenamiento. Demostraron que este tipo de memorización se produce en la práctica cuando el modelo es mucho más grande (en términos del número de parámetros) en comparación con los datos; la Figura 3.9 muestra que, en este caso, las muestras generadas son, en esencia, copias perfectas, píxel a píxel, de las muestras de entrenamiento. Poco después, [GDP+23] llevó a cabo un estudio minucioso para caracterizar cómo diversos factores, tales como las arquitecturas y el tiempo de entrenamiento, influyen en la memorización, demostrando, entre otras cosas, que a medida que el tamaño del conjunto de datos aumenta en relación con un modelo fijo, el índice de memorización cambia rápidamente de un valor cercano a 1 a uno cercano a 0, es decir, el modelo deja de memorizar rápidamente. Esto indica la posibilidad de una transición de fase entre la memorización y la generalización a medida que el modelo se vuelve más pequeño en relación con los datos. [BPM+25] investiga la existencia y el mecanismo teórico de esta transición de fase en datos sintéticos extraídos de un modelo de mezcla gaussiana, demostrando que existe y proporcionando una caracterización parcial del tamaño del modelo a partir del cual los modelos comienzan o dejan de memorizar. Otros trabajos teóricos, como [GVM25; KG24; NZM+24; ZLL+25] también intentan ofrecer una caracterización teórica de la transición de fase en función del tamaño del modelo y del conjunto de datos en diferentes contextos.

Con el fin de demostrar empíricamente la existencia de una transición de fase entre la memorización y la generalización en datos reales, [ZZL+24] replanteó la definición de memorización mediante el siguiente razonamiento. En el espacio de dimensiones muy elevadas de las imágenes naturales, las distancias euclidianas entre puntos se vuelven mucho más estrictas [WM22], en el sentido de que, si dos imágenes no coinciden a la perfección a nivel de píxeles, sus distancias euclidianas se concentrarán en forma marcada en torno a un valor fijo (que escala con la dimensión de las imágenes). Esto no tiene en cuenta los casos en los que ciertos *conceptos* (como los personajes de dibujos animados protegidos por derechos de autor) se generan en posiciones novedosas, lo que dificulta enormemente establecer criterios de memorización eficaces y prácticos basados en la distancia euclidiana. Para solucionar esto, [ZZL+24] utiliza una función específica f_{SSCD} (una red neuronal preentrenada denominada «detector de copias autosupervisado» [PRR+22]) para mapear imágenes a un espacio euclidiano y luego toma el producto interno normalizado para obtener

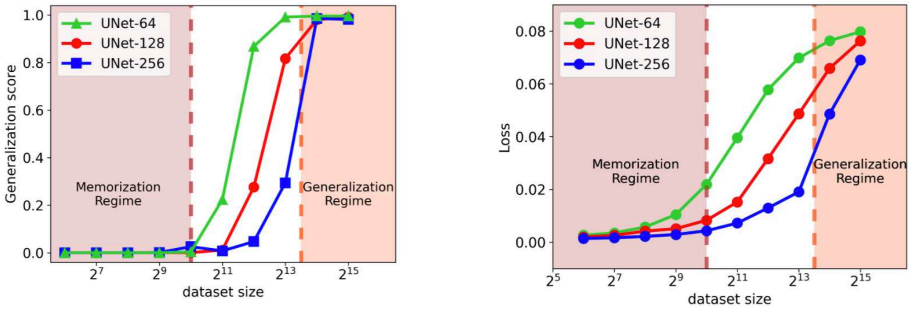


Figura 3.11: **La puntuación de generalización (izquierda) y la pérdida de entrenamiento (derecha) para un modelo fijo y un volumen de datos creciente**, extraído de [ZZL+24]. La puntuación de generalización es 1− la fracción de muestras que se memorizan, y las diferentes líneas corresponden a modelos de eliminación de ruido de distintos tamaños (UNet-64 es más pequeño que UNet-128, que a su vez es más pequeño que UNet-256). Las figuras muestran que, a medida que el tamaño del conjunto de datos aumenta en relación con el tamaño del modelo, la pérdida aumenta significativamente y deja de producirse la memorización, lo que muestra que nuestra teoría e intuición se aplican a modelos reales.

su similitud (coseno), de tal manera que se afirma que las imágenes que son similares en virtud de esta similitud coseno lo son también conceptualmente en el sentido necesario para la memorización. Entonces, [ZZL+24] afirma que una muestra $\hat{\boldsymbol{x}}$ ha sido memorizada si su similitud con cualquier punto de entrenamiento es $> 3/5$, es decir,

$$\hat{\boldsymbol{x}} \text{ está memorizada} \iff \max_{i\in[N]} \frac{f_{\text{SSCD}}(\hat{\boldsymbol{x}})^\top f_{\text{SSCD}}(\boldsymbol{x}^{(i)})}{\|f_{\text{SSCD}}(\hat{\boldsymbol{x}})\|_2 \|f_{\text{SSCD}}(\boldsymbol{x}^{(i)})\|_2} > \frac{3}{5} \tag{3.3.6}$$

La Figura 3.11 muestra el resultado de tomar numerosas muestras (alrededor de 10 000) de modelos de difusión entrenados con conjuntos de datos de entrenamiento de distintos tamaños, en cuanto a la estimación de la memorización y el cálculo de la pérdida de entrenamiento. El resultado concuerda con lo expuesto anteriormente: cuando el modelo es demasiado grande en relación con los datos de entrenamiento y la pérdida de entrenamiento es demasiado pequeña, se produce memorización y las muestras reproducen los datos de entrenamiento. Así pues, nuestra intuición se confirma en la práctica: *¡en el caso de los modelos generativos de difusión, escalar el modelo no es lo único que se necesita!*

Entonces, la pregunta es: ¿qué tipo de *denoiser* deberíamos usar? La clave es elegir una arquitectura de red que pueda aproximar el verdadero *denoiser* (por ejemplo, el que corresponde a una distribución de rango bajo, como en (3.2.80)) sin aproximar el *denoiser* memorizador (3.3.3). Algunos trabajos han analizado esta delgada línea y por qué los modelos de difusión modernos, que utilizan arquitecturas basadas en transformers y convoluciones, pueden memorizar y generalizar en diferentes regímenes [KG24; NZM+24; ZLL+25].

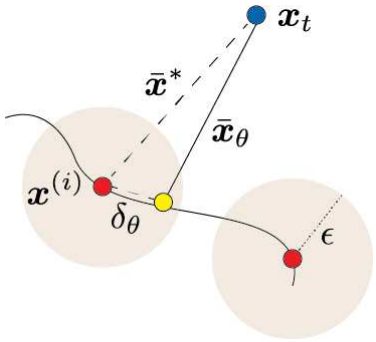


Figura 3.12: **Visualización del funcionamiento de un *denoiser* bien entrenado** como una perturbación del filtro empírico para *valores pequeños de* t, cuando el filtro memorizador actúa como una proyección sobre el punto más cercano del conjunto de entrenamiento.

Volvamos por fin a la premisa inicial de este capítulo, descrita en la Sección 3.1.3: encontrar la distribución de entropía mínima que modela los datos es la única forma de aprender la distribución. Esperamos que el análisis anterior haya demostrado que esta afirmación es falsa. Sin embargo, una afirmación relacionada, también defendida en la Sección 3.1.3 debido a su equivalencia en las situaciones que habíamos considerado entonces, podría seguir siendo cierta: encontrar la distribución con la *tasa de codificación* mínima es una forma de aprender la distribución. Supongamos por un momento que $\boldsymbol{x}$ tiene una densidad p; la tasa de codificación (es decir, la entropía cruzada) de la distribución empírica $p^{\boldsymbol{X}}$ con respecto a p es $\mathrm{CE}(p \mid p^{\boldsymbol{X}}) = \infty$. Es evidente que esta tasa de codificación no es mínima en ningún sentido; si utilizamos las mismas aproximaciones sucesivamente mejores p^n, p^e y $\hat{q}$ analizadas en la Sección 3.1.3, se cumple que

$$\infty = \mathrm{CE}(p \parallel p^{\boldsymbol{X}}) > \mathrm{CE}(p \parallel p^n) > \mathrm{CE}(p \parallel p^e) > \mathrm{CE}(p \parallel \hat{q}) \approx h(p). \tag{3.3.7}$$

Por lo tanto, la minimización de la tasa de codificación parece una forma sensata de aprender la distribución y, al mismo tiempo, evitar este problema. Sin embargo, ahora surge un problema de manejabilidad: ¿cómo podemos diseñar algoritmos que aprendan la distribución minimizando la tasa de codificación y evitar los inconvenientes asociados a una minimización excesiva de la entropía?

Una pista consiste en analizar qué ocurre con los modelos de difusión entrenados con éxito que no memorizan. Al expresar el *denoiser* entrenado $\bar{\boldsymbol{x}}_\theta$ como una perturbación del *denoiser* $\bar{\boldsymbol{x}}^\star$, es decir,

$$\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{x}_t) = \bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) + \boldsymbol{\delta}_\theta(t, \boldsymbol{x}_t) \tag{3.3.8}$$

podemos hacernos una idea gráfica del funcionamiento de un filtro de eliminación de ruido bien entrenado, como se muestra en la Figura 3.12. A grandes rasgos, el *denoiser* entrenado calcula el filtro de memorización más una perturbación.

Para un t pequeño (quizás incluso en el último paso de la eliminación de ruido), el *denoiser* memorizador apunta completamente en la dirección de una sola muestra de entrenamiento (la muestra de entrenamiento que se va a regenerar en la iteración actual). La distribución que se está aprendiendo, o respecto a la cual se elimina el ruido, es simplemente la distribución empírica con unas bolas de radio ϵ alrededor de cada punto,[12] donde ϵ es una cota superior de la magnitud de la perturbación, visualizado en la Figura 3.12 por las bolas de color naranja pálido.[13] En el caso de modelos de difusión cada vez más grandes y bien entrenados, el radio de perturbación ϵ se reduce a 0 y el modelo aprende la distribución empírica. Por otro lado, un valor de ϵ lo suficientemente grande como para que las bolas correspondientes a puntos diferentes se superpongan o estén muy cerca puede implicar que el modelo de difusión generalice bien.

Por lo tanto, nuestro modelo de difusión eficaz puede considerarse como una minimización de una *tasa de codificación con pérdida*, es decir, una tasa de codificación con precisión de ϵ (de modo que se puedan ignorar las bolas de radio ϵ alrededor de cada punto). Esta noción de la tasa de codificación con pérdida resulta ser lo que conviene minimizar, en principio, y será *extremadamente* útil. Exploraremos sus aplicaciones en el Capítulo 4 y a lo largo del resto de este libro.

3.4 Resumen y notas

Mensajes clave. En este capítulo, hemos mostrado que, para lograr una distribución general de baja dimensión, un enfoque computacional universal consiste en partir de una distribución genérica y comprimirla hasta alcanzar dicha distribución mediante la reducción (iterativa) del ruido o de la entropía. Como vimos en el capítulo anterior, esto ocurre incluso en el caso de las que podrían considerarse las distribuciones más simples de un único subespacio, como el cálculo del PCA mediante el algoritmo de iteración de potencias. Como veremos en los próximos capítulos, las funciones fundamentales de las redes profundas modernas no es ajustar distribuciones de datos o funciones arbitrarias (con ruido), sino más bien realizar o aproximar esos operadores de compresión o de eliminación de ruido (con pérdida). En capítulos posteriores derivaremos las formas precisas de esos operadores y las arquitecturas profundas asociadas.

Más extensiones y referencias. El uso de la eliminación de ruido y la difusión para el muestreo tiene una larga trayectoria. Probablemente, el primer trabajo que trata específicamente de un modelo de difusión sea [SWM+15], pero antes de este ya existían numerosos trabajos sobre la eliminación del

[12] Más formalmente, esta distribución perturbada tiene su soporte en (un subconjunto de) la unión de bolas de radio ϵ centradas en cada muestra de entrenamiento. Esta visión está respaldada por trabajos anteriores, p. ej., [SKC+25].

[13] En dimensiones elevadas, se plantea la hipótesis de que un mejor modelo para estas perturbaciones no se describe mediante esferas, sino más bien mediante formas mucho más anisotrópicas y adaptadas a la geometría [FPH+25], aunque la intuición sigue siendo similar.

ruido como problema computacional y estadístico. El más relevante de ellos es probablemente [Hyv05], que utiliza explícitamente la función de *score* para eliminar el ruido y realizar un análisis de componentes independientes. Los trabajos posteriores más populares surgieron, en esencia, de manera prácticamente simultánea: [HJA20; SE19]. Desde entonces, miles de artículos se han apoyado en los modelos de difusión; retomamos este tema en el Capítulo 6.

Muchos de estos trabajos utilizan un proceso estocástico distinto al de la simple combinación lineal (3.2.85). De hecho, todos los trabajos mencionados anteriormente hacen hincapié en la necesidad de añadir ruido gaussiano *independiente* al inicio de cada paso del proceso hacia adelante. Los trabajos de carácter teórico utilizan el movimiento browniano o ecuaciones diferenciales estocásticas para formular el proceso hacia adelante [SSK+21]. Sin embargo, dado que las combinaciones lineales de distribuciones gaussianas siguen dando lugar a distribuciones gaussianas, las *distribuciones marginales* de dichos procesos siguen adoptando la forma de (3.2.85). La mayor parte de nuestro análisis solo requiere que las distribuciones marginales sean lo que son, por lo que nuestro modelo, aunque sea demasiado simplista, resulta suficiente para casi todo. El único caso en el que las distribuciones marginales no son suficientes es cuando derivamos una expresión para $\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]$ en términos de $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t]$. Los distintos procesos (de ruido) dan lugar a diferentes expresiones de este tipo, que pueden utilizarse para el muestreo (y, por supuesto, existen otras formas de obtener muestreadores eficientes, como el siempre popular muestreador DDPM). El proceso descrito en (3.2.85) es un proceso estocástico genuino, cuya iteración de eliminación de ruido «natural» adopta la forma del popular algoritmo DDIM [SME20]. (Ni siquiera esta equivalencia es trivial; citamos [DGG+25] como justificación.)

Además del trabajo teórico [LY24] tratado en la Sección 3.2.2, y la línea de trabajo en la que se basa, que estudia la eficiencia de *muestreo* de los modelos de difusión cuando los datos tienen una estructura de baja dimensión, existe un amplio conjunto de trabajos que estudia la eficiencia de *entrenamiento* de los modelos de difusión cuando los datos tienen una estructura de baja dimensión. En concreto, Chen et al. [CHZ+23] y Wang et al. [WZZ+24] caracterizaron el error de aproximación y estimación de los *denoisers* cuando los datos pertenecen a una mezcla de gaussianas de bajo rango, demostrando que el número de muestras de entrenamiento necesarias para aprender con precisión la distribución escala con la dimensión intrínseca de los datos, en lugar de la dimensión ambiente. Existe una considerable *labor metodológica* que intenta aprovechar la estructura de baja dimensión de los datos para realizar diversas tareas con modelos de difusión. Destacamos tres: la edición de imágenes [CZG+24], el marcado de agua [LZQ24] y el «desaprendizaje» [CZL+25], aunque esta lista no es exhaustiva.

3.5 Ejercicios y ampliaciones

Ejercicio 3.1. Mostrar que (3.2.8) es la solución óptima del problema (3.2.7).

Ejercicio 3.2. Consideremos vectores aleatorios $\boldsymbol{x} \in \mathbb{R}^D$ y $\boldsymbol{y} \in \mathbb{R}^d$, de tal manera que el par $(\boldsymbol{x}, \boldsymbol{y}) \in \mathbb{R}^{D+d}$ sea gaussiano en conjunto. Esto significa que

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_{\boldsymbol{x}} \\ \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} \right),$$

donde los parámetros de media y covarianza se dan por

$$\boldsymbol{\mu}_{\boldsymbol{x}} = \mathbb{E}[\boldsymbol{x}], \quad \boldsymbol{\mu}_{\boldsymbol{y}} = \mathbb{E}[\boldsymbol{y}], \quad \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} = \mathbb{E}\left[\begin{bmatrix} \boldsymbol{x} - \mathbb{E}[\boldsymbol{x}] \\ \boldsymbol{y} - \mathbb{E}[\boldsymbol{y}] \end{bmatrix} \begin{bmatrix} \boldsymbol{x} - \mathbb{E}[\boldsymbol{x}] \\ \boldsymbol{y} - \mathbb{E}[\boldsymbol{y}] \end{bmatrix}^\top \right]$$

Supongamos que $\boldsymbol{\Sigma}_{\boldsymbol{y}}$ es positiva definida (y, por lo tanto, invertible); entonces, que la matriz de covarianza sea semidefinida positiva es equivalente a la condición del complemento de Schur $\boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top \succeq \mathbf{0}$.

En este ejercicio, demostraremos que la distribución condicional $p_{\boldsymbol{x}|\boldsymbol{y}}$ es gaussiana: es decir,

$$p_{\boldsymbol{x}|\boldsymbol{y}} \sim \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{x}} + \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} (\boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}}), \boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top\right). \tag{3.5.1}$$

Una vía directa para demostrar este resultado consiste en manipular el cociente definitoria de las densidades $p_{\boldsymbol{x},\boldsymbol{y}}/p_{\boldsymbol{y}}$. A continuación esbozamos un argumento algebraicamente conciso de este tipo.

1. Compruebe la siguiente identidad matricial para la covarianza:

$$\begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{I}_D & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \\ \mathbf{0} & \boldsymbol{I}_d \end{bmatrix} \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} \begin{bmatrix} \boldsymbol{I}_D & \mathbf{0} \\ \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{I}_d \end{bmatrix}. \tag{3.5.2}$$

   Esta identidad se obtiene aplicando dos rondas de eliminación gaussiana en bloques a la matriz de covarianza.

2. Partiendo de la identidad anterior, demuestre que

$$\begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix}^{-1} = \boldsymbol{P} \begin{bmatrix} \left(\boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top\right)^{-1} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \end{bmatrix} \boldsymbol{P}^\top, \tag{3.5.3}$$

   donde

$$\boldsymbol{P} = \begin{bmatrix} \boldsymbol{I}_D & \mathbf{0} \\ -\boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{I}_d \end{bmatrix}, \tag{3.5.4}$$

   y siempre que las inversas pertinentes estén definidas.[14] Concluya que

$$\begin{bmatrix} \boldsymbol{x} - \boldsymbol{\mu}_{\boldsymbol{x}} \\ \boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix}^{-1} \begin{bmatrix} \boldsymbol{x} - \boldsymbol{\mu}_{\boldsymbol{x}} \\ \boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix} \tag{3.5.5}$$

$$= \begin{bmatrix} \boldsymbol{x} - \boldsymbol{m} \\ \boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}^\top \begin{bmatrix} \left(\boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top\right)^{-1} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \end{bmatrix} \begin{bmatrix} \boldsymbol{x} - \boldsymbol{m} \\ \boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}, \tag{3.5.6}$$

   donde $\boldsymbol{m} = \boldsymbol{\mu}_{\boldsymbol{x}} + \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} (\boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}})$.

[14] Cuando el término del complemento de Schur no sea invertible, se mantiene el mismo resultado sustituyendo su inversa por la pseudoinversa de Moore-Penrose. En particular, la distribución condicional (3.5.1) se reduce a una distribución gaussiana degenerada.

3. Dividiendo $p_{\boldsymbol{x},\boldsymbol{y}}/p_{\boldsymbol{y}}$, demuestre la Ecuación (3.5.1). (*Pista: Utilizando las identidades anteriores, solo debería ser necesario un mínimo de álgebra. Para la constante de normalización, utilice la Ecuación* (3.5.3) *para descomponer el determinante de manera similar.*)

Ejercicio 3.3. Demuestre la identidad de Sherman-Morrison-Woodbury, es decir, para matrices $\boldsymbol{A}$, $\boldsymbol{C}$, $\boldsymbol{U}$, $\boldsymbol{V}$ tales que $\boldsymbol{A}$, $\boldsymbol{C}$ y $\boldsymbol{A}+\boldsymbol{U}\boldsymbol{C}\boldsymbol{V}$ sean invertibles,

$$(\boldsymbol{A}+\boldsymbol{U}\boldsymbol{C}\boldsymbol{V})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{U}(\boldsymbol{C}^{-1}+\boldsymbol{V}\boldsymbol{A}^{-1}\boldsymbol{U})^{-1}\boldsymbol{V}\boldsymbol{A}^{-1}. \tag{3.5.7}$$

Ejercicio 3.4. Vuelva a derivar lo siguiente, suponiendo que $\boldsymbol{x}_t$ sigue el modelo de ruido generalizado (3.2.85).

- La fórmula de Tweedie: (3.2.86).
- La iteración del DDIM: (3.2.87).
- El *denoiser* óptimo de Bayes para un modelo de mezcla gaussiana: (3.2.88).

Ejercicio 3.5.

1. Implemente las fórmulas derivadas en el Ejercicio 3.4, creando un muestreador para mezclas gaussianas.
2. Reproduzca la Figura 3.4 y la Figura 3.8.
3. A continuación, presentamos *Flow Matching (FM)* (véase la Nota 3.3):

$$\alpha_t = 1-t, \qquad \sigma_t = t. \tag{3.5.8}$$

   Implemente este proceso utilizando el mismo marco y pruébelo para el muestreo en espacios de alta dimensión. ¿Qué proceso ofrece resultados mejores o más estables?

Ejercicio 3.6. Considere la interpolación, algo familiar

$$\boldsymbol{x}_t = \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{y}, \qquad \forall t \in [0,T]. \tag{3.5.9}$$

donde $\boldsymbol{y}$ es una variable aleatoria (no necesariamente gaussiana). Definimos la *velocidad* como

$$\boldsymbol{v}_t = \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_t = \alpha_t'\boldsymbol{x} + \sigma_t'\boldsymbol{y}, \qquad \forall t \in [0,T]. \tag{3.5.10}$$

Calcule una relación algebraica que exprese $\mathbb{E}[\boldsymbol{v}_t \mid \boldsymbol{x}_t]$ en términos de $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t]$ y viceversa, demostrando que la predicción de la velocidad es algebraicamente equivalente a la eliminación de ruido y, por tanto, también a la predicción de ruido (véase la Nota 3.3).

Capítulo 4

Aprendizaje de representaciones mediante compresión con pérdidas

> "*Cuando puedes medir aquello de lo que hablas y expresarlo en números, sabes algo al respecto; pero cuando no puedes medirlo, cuando no puedes expresarlo en números, tu conocimiento es de un tipo exiguo e insatisfactorio: puede ser el comienzo del conocimiento, pero apenas has avanzado, en tus pensamientos, a la etapa de la ciencia, sea cual sea el tema de que se trate.*"
>
> – Lord Kelvin, 1883

Recapitulemos lo que hemos visto hasta ahora. Hemos mostrado que es posible identificar y acceder a una distribución con un soporte de dimensión arbitrariamente bajas, como la que se ilustra en la Figura 4.1 (izquierda), transformando gradualmente la distribución de ruido puro (alta entropía) en la distribución que se desea aprender mediante *eliminación iterativa de ruido*. Hemos mostrado que el *denoiser* en cada iteración está directamente relacionado con el gradiente de la log-densidad de la distribución mediante la fórmula de Tweedie (3.2.23). Demostramos que, en la práctica, ese filtro de eliminación de ruido $\bar{\boldsymbol{x}}_\theta$ puede estimarse utilizando un número finito de muestras. Por lo tanto, desarrollamos un método general para aprender o aproximarse a una distribución de datos.

No obstante, en esta metodología, la codificación de la distribución está implícita en la forma funcional y los parámetros del *denoiser*, si los tuviera. De hecho, los lectores más atentos habrán notado que, para una distribución general, nunca hemos especificado explícitamente cuál debería ser la forma funcional del *denoiser*. En la práctica, la gente suele modelarlo con alguna red neuronal profunda que tiene una arquitectura empíricamente diseñada.

Además, aunque sabemos que el proceso de eliminación de ruido anterior reduce la entropía, no sabemos en qué medida, ni conocemos la entropía de las distribuciones intermedias y finales. Por lo tanto, sería muy útil conocer la forma (óptima) de la operación de eliminación de ruido y cómo medir el cambio en la entropía. Por último, al final del capítulo anterior demostramos que si el proceso de eliminación de ruido reduce la entropía *en exceso*, esto conduce a que solo se aprendan los datos de entrenamiento, sin posibilidad de interpolación o extrapolación. La innovación metodológica que resuelve todas estas cuestiones se desarrollará en el próximo capítulo.

Medida de información. Recordemos que nuestro objetivo general es modelar datos procedentes de una distribución continua con un soporte de baja dimensión. Si queremos identificar el modelo más simple que genera los datos, podríamos considerar tres medidas típicas de parsimonia: la dimensión, el volumen o la entropía diferencial. Si usamos la dimensión, es obvio que el mejor modelo para un conjunto de datos dado es la propia distribución empírica, que es de dimensión cero.[1] Si, en cambio, utilizamos la entropía de los datos, entonces para todas las distribuciones con soporte de baja dimensión, la entropía diferencial es siempre negativa infinita. O bien, si utilizamos el volumen del espacio que abarcan los datos, el volumen del soporte de cualquier modelo de baja dimensión es siempre cero. Entonces, entre todas las distribuciones con un soporte de baja dimensión que podrían haber generado las mismas muestras de datos, ¿cómo podemos decidir cuál es la mejor basándonos en estas medidas de parsimonia que no permiten distinguirlas entre sí en absoluto?

En este capítulo, analizamos un marco que mitiga la dificultad técnica mencionada anteriormente al asociar la distribución aprendida con un esquema explícito y computable de codificación y decodificación, siguiendo y haciendo más explícito el segundo enfoque sugerido al final de la Sección 3.1.3. Como veremos, este enfoque nos permite, en esencia, aproximar con precisión la entropía de las distribuciones aprendidas en términos de una *tasa de codificación con pérdida* asociada al esquema de codificación, tal como se describe y justifica (aunque no se formaliza) al final de la Sección 3.3. A grandes rasgos, esta medida puede considerarse como una noción generalizada del volumen de la distribución de los datos empíricos. Con una medida de este tipo, podemos cuantificar con precisión en qué medida se reduce la entropía mediante una transformación de la distribución, por ejemplo, mediante un *denoiser*. Además, con una medida de este tipo, podemos obtener, al menos en principio, el operador óptimo de eliminación de ruido que reduce la entropía más rápidamente.

El aprendizaje de representaciones. Aunque el proceso de difusión y eliminación de ruido presentado en el capítulo anterior nos permite, en

[1] En la Introducción (Capítulo 1), sostuvimos que un objetivo importante de la inteligencia es buscar estructuras de baja dimensión en los datos observados. Sin embargo, desde el punto de vista técnico, no se debe interpretar el término «de baja dimensión» de manera literal. El objetivo de este capítulo es aclarar con precisión qué estructuras de baja dimensión debemos buscar y cómo hacerlo.

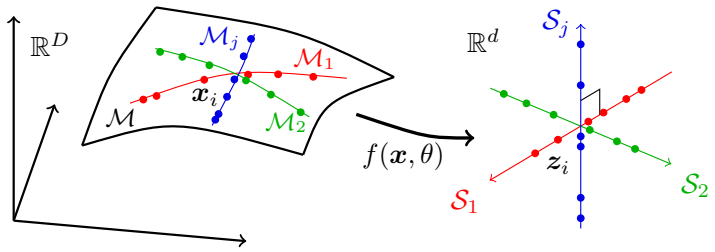


Figura 4.1: Aprender una representación estructurada (derecha) de la distribución de los datos (izquierda). Obsérvese que, además de aprender (y muestrear) la distribución de los datos originales como hicimos en el Capítulo 3 anterior, aquí queremos aprender una transformación $f(\boldsymbol{x}, \theta)$ para la distribución de los datos, de modo que se obtenga una representación con mejores propiedades geométricas o estadísticas deseadas, por ejemplo, lineal e incoherente.

principio, aprender una distribución arbitraria de baja dimensión, no ofrece una representación explícita de dicha distribución. Como veremos, se accederá a la distribución aprendida en repetidas ocasiones y se utilizará para tareas posteriores (por ejemplo, como distribución a priori para la inferencia bayesiana en el Capítulo 7). Por lo tanto, es conveniente que podamos transformar la distribución de los datos, por ejemplo mediante un mapeo $f(\boldsymbol{x}, \theta)$, a alguna forma estructurada estándar que facilite un acceso eficiente, como una mezcla de subespacios lineales (incoherentes) o gaussianas de bajo rango, tal y como se ilustra en la figura de la derecha de la Figura 4.1. En este capítulo formalizaremos este proceso como «aprendizaje de representaciones».

Como también veremos, para una distribución arbitraria, la medida de información anterior (tasa de codificación) no se puede calcular (fácilmente). Sin embargo, en el caso de una mezcla de subespacios o gaussianas de bajo rango, podemos obtener una fórmula cerrada precisa para la medida. Al optimizar esta medida, podremos encontrar la representación óptima. Como veremos en el siguiente Capítulo 5, la optimización de la medida también nos permite derivar formas explícitas del operador que realiza la transformación $f(\boldsymbol{x}, \theta)$ de la manera más eficiente. Esto dará lugar a una explicación basada en principios sobre la arquitectura de las redes profundas, así como a diseños de arquitectura profunda más eficientes.

4.1 Compresión mediante codificación con pérdidas

En esta sección, desarrollamos la *tasa de codificación con pérdida* como una medida computable de complejidad para distribuciones con un soporte de baja

dimensión, un contexto en el que las medidas clásicas como la entropía, la dimensión y el volumen no logran distinguir entre los modelos candidatos. En primer lugar, justificamos la necesidad de la codificación con pérdidas en relación con estas patologías (Sección 4.1.1); a continuación, demostramos que la función de tasa-distorsión relaciona esta medida de la teoría de la codificación con la geometría del soporte de datos por medio del recubrimiento esférico (Teorema 4.1). Esta conexión constituye el puente entre los enfoques basados en la teoría de la información y los enfoques geométricos para el aprendizaje de distribuciones de baja dimensión. Por último, desarrollamos una aproximación estrecha en forma cerrada de la tasa de codificación con pérdida para las distribuciones gaussianas (Sección 4.1.3), así como un algoritmo eficaz para agrupar mezclas de distribuciones gaussianas de baja dimensión (Sección 4.1.4), que se generalizará a distribuciones no lineales en la siguiente sección.

4.1.1 La necesidad de la codificación con pérdida

Ya hemos abordado en varias ocasiones una dificultad: si al final aprendemos la distribución a partir de un número finito de muestras y nuestra clase de funciones de *denoiser* contiene suficientes funciones, ¿cómo nos aseguramos de que estamos tomando muestras de la distribución *verdadera* (con soportes de baja dimensión), en lugar de cualquier otra distribución que pueda generar ese número finito de muestras con alta probabilidad? Expongamos algunas de las dificultades conceptuales y técnicas con ejemplos concretos.

Ejemplo 4.1 (Volumen, dimensión y entropía). Para el ejemplo que se muestra en la parte superior de la Figura 4.2, supongamos que hemos tomado algunas muestras de una distribución uniforme en una recta (por ejemplo, en un plano bidimensional). El volumen de la recta o de los conjuntos de muestras es cero. Desde el punto de vista geométrico, la distribución empírica sobre el conjunto finito de muestras obtenido es aquella de *dimensión mínima* capaz de generar dicho conjunto finito de muestras.[2] Pero esto parece estar en contraste con otra medida más de la complejidad: la entropía. La entropía (diferencial) de la recta es el infinito negativo, pero la entropía (discreta) de este conjunto de muestras es finita y positiva. Así pues, parece que nos encontramos ante una situación paradójica según estos criterios habituales de parsimonia o complejidad: no son capaces de diferenciar adecuadamente entre (los modelos de) distribuciones con soportes de baja dimensión y algunos parecen diferenciarlos incluso de maneras totalmente opuestas.[3] ■

Ejemplo 4.2 (Densidad). Consideremos los dos conjuntos de puntos de datos muestreados que se muestran en la Figura 4.2. Geométricamente, son esencialmente iguales: cada conjunto consta de ocho puntos y cada punto ha aparecido con una frecuencia igual a 1/8. La única diferencia es que, en el

[2] Un conjunto de muestras discretas tiene dimensión cero, mientras que la recta de soporte tiene dimensión uno.

[3] Por supuesto, en sentido estricto, la entropía diferencial y la entropía discreta no son directamente comparables.

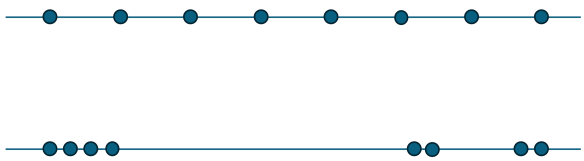


Figura 4.2: Ocho puntos situados en una línea con diferentes configuraciones geométricas; distribuidos uniformemente (arriba) o muy agrupados (abajo).

segundo conjunto de datos, algunos puntos están lo suficientemente «cerca» como para que se considere que tienen una mayor densidad alrededor de su respectivo «grupo». ¿Cuál de las dos es más relevante para la distribución real que podría haber generado las muestras? ¿Cómo podemos resolver esta ambigüedad a la hora de interpretar este tipo de distribuciones (empíricas)? ■

Existe una dificultad técnica adicional relacionada con la construcción de un esquema explícito de codificación y decodificación para un conjunto de datos. Dado un conjunto de datos muestreados en $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, ¿cómo se puede diseñar un esquema de codificación que sea implementable en máquinas con recursos computacionales y de memoria finitos? Nótese que incluso para representar un número real cualquiera se necesita un número infinito de dígitos o bits. Por lo tanto, cabe preguntarse si la entropía de una distribución es una medida directa de la complejidad de su esquema de codificación (óptimo). Analizamos este tema con otro ejemplo sencillo.

Ejemplo 4.3 (Precisión). Consideremos una distribución discreta $\boldsymbol{X} = [e, \pi]$ en la que hay una probabilidad igual de 1/2 de que que toma los valores del número de Euler $e \approx 2.71828$ y de $\pi \approx 3.14159$ **con probabilidad 1/2 cada uno. La entropía de esta distribución es $H = 1$, lo que sugiere que se pueden codificar los dos números mediante un único bit, 0 o 1, respectivamente. ¿Pero se puede implementar un esquema de decodificación para este código en una máquina de estados finitos? La respuesta es, de hecho, no, ya que se necesita un número infinito de bits para describir con precisión cualquiera de los dos números. ■

Por lo tanto, por lo general es imposible disponer de un esquema de codificación y decodificación capaz de reproducir con precisión muestras procedentes de una distribución arbitraria de valores reales.[4] Pero no tendría mucho valor práctico codificar una distribución sin poder decodificarla para muestras extraídas de la misma distribución.

Por lo tanto, para garantizar que cualquier esquema de codificación/decodificación sea computable y se pueda implementar con memoria y recursos computacionales finitos, debemos cuantificar la muestra $\boldsymbol{x}$ y codificarla solo hasta una cierta precisión, digamos $\epsilon > 0$. *Al hacerlo, en*

[4] Es decir, si se desea codificar dichas muestras con precisión, la única forma es memorizar cada una de las muestras.

esencia, consideramos que dos puntos de datos cualesquiera son equivalentes si su distancia es menor que ϵ. Más precisamente, nos gustaría considerar esquemas de codificación

$$\boldsymbol{x} \mapsto \hat{\boldsymbol{x}} \tag{4.1.1}$$

de tal manera que el error esperado causado por la cuantización esté acotado por ϵ. Es matemáticamente más conveniente, y conceptualmente prácticamente idéntico, acotar el error esperado *al cuadrado* por ϵ^2, es decir,

$$\mathbb{E}[d(\boldsymbol{x}, \hat{\boldsymbol{x}})^2] \leq \epsilon^2. \tag{4.1.2}$$

Por lo general, se elige que la distancia d sea la distancia euclidiana, o la norma L^2.[5] Adoptaremos esta opción en lo sucesivo.

4.1.2 La tasa-distorsión y la geometría de los datos

Por supuesto, de entre todos los esquemas de codificación que satisfacen la restricción anterior, nos gustaría elegir aquel que minimice la tasa de codificación resultante. Para una variable aleatoria dada $\boldsymbol{x}$ y una precisión ϵ, esta tasa se conoce como la *tasa-distorsión*, denotada como $\mathcal{R}_\epsilon(\boldsymbol{x})$.

Para comprender mejor esta tasa-distorsión, recurrimos a un teorema fundamental de la teoría de la información, demostrado originalmente en Shannon [Sha59], que relaciona esta tasa-distorsión con otras magnitudes de la teoría de la información. Para comprender este teorema, primero debemos definir una medida de *diferencia* entre dos distribuciones conocida como la divergencia de *Kullback-Leibler* (KL), o *entropía relativa*, que se denota por $\mathsf{KL}(\cdot \parallel \cdot)$ y se define en relación con la entropía y la entropía cruzada (véase la Sección 3.1.3) como

$$\begin{aligned} \mathsf{KL}(p \parallel q) &= \mathrm{CE}(p \parallel q) - h(p) = \mathbb{E}_{\boldsymbol{x}\sim p}[-\log q(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})] && (4.1.3)\\ &= \mathbb{E}_{\boldsymbol{x}\sim p}[\log(p(\boldsymbol{x})/q(\boldsymbol{x}))]. && (4.1.4) \end{aligned}$$

La divergencia de KL siempre es no negativa y es igual a cero si y solo si $p = q$;[6] en el caso de que p y q tengan el mismo soporte y ambas sean densidades de probabilidad, esto se puede demostrar mediante el siguiente cálculo, basado en la desigualdad de Jensen y en el hecho de que la función $\log(\cdot)$ es estrictamente cóncava:

$$\begin{aligned} -\mathsf{KL}(p \parallel q) &= -\mathbb{E}_{\boldsymbol{x}\sim p}[\log(p(\boldsymbol{x})/q(\boldsymbol{x}))] = \mathbb{E}_{\boldsymbol{x}\sim p}[\log(q(\boldsymbol{x})/p(\boldsymbol{x}))] && (4.1.5)\\ &\leq \log \mathbb{E}_{\boldsymbol{x}\sim p}[q(\boldsymbol{x})/p(\boldsymbol{x})] = \log \int_{\mathbb{R}^D} \frac{q(\boldsymbol{\xi})}{p(\boldsymbol{\xi})} p(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} && (4.1.6)\\ &= \log \int_{\mathbb{R}^D} q(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} = \log \mathbb{E}_{\boldsymbol{x}\sim q}[1] = \log 1 = 0. && (4.1.7) \end{aligned}$$

[5] De manera más general, podemos sustituir d^2 por cualquiera de las llamadas *divergencias*.

[6] Técnicamente, esta igualdad debe interpretarse en el sentido de «casi en todas partes», es decir, salvo, quizás, en un conjunto de medida (volumen) nula, ya que este conjunto no afectaría al valor de ninguna integral.

A continuación, definimos la *información mutua* $I(\cdot;\cdot)$, que es una medida de la *dependencia* entre dos variables aleatorias, como

$$I(\boldsymbol{x};\hat{\boldsymbol{x}}) = \mathsf{KL}(p(\boldsymbol{x},\hat{\boldsymbol{x}}) \parallel p(\boldsymbol{x})p(\hat{\boldsymbol{x}})), \tag{4.1.8}$$

Una vez establecidas las definiciones anteriores, podemos expresar la tasa-distorsión $\mathcal{R}_\epsilon$ en términos puramente probabilísticos de la siguiente forma:

$$\mathcal{R}_\epsilon(\boldsymbol{x}) = \min_{p(\hat{\boldsymbol{x}}|\boldsymbol{x}):\mathbb{E}[\|\boldsymbol{x}-\hat{\boldsymbol{x}}\|_2^2]\leq\epsilon^2} I(\boldsymbol{x};\hat{\boldsymbol{x}}). \tag{4.1.9}$$

Nótese que la minimización en (4.1.9) se realiza sobre todas las distribuciones condicionales $p(\hat{\boldsymbol{x}} \mid \boldsymbol{x})$ que satisfacen la restricción de distorsión $\mathbb{E}[\|\boldsymbol{x}-\hat{\boldsymbol{x}}\|_2^2] \leq \epsilon^2$. Cada una de estas distribuciones condicionales da lugar a una distribución conjunta $p(\boldsymbol{x},\hat{\boldsymbol{x}}) = p(\hat{\boldsymbol{x}} \mid \boldsymbol{x})p(\boldsymbol{x})$, que determina la información mutua (4.1.8). Muchas propiedades convenientes de la información mutua (y, por lo tanto, de la tasa-distorsión) se derivan de las propiedades correspondientes de la divergencia de KL. Por definición, sabemos que $\mathcal{R}_\epsilon(\boldsymbol{x})$ es una función *no creciente* en ϵ.

Ejemplo 4.4 (Tasa-distorsión de una fuente gaussiana (Teorema 13.3.3, [CT91])). Consideremos una variable aleatoria gaussiana $\boldsymbol{x} \sim \mathcal{N}(0,\sigma^2)$. En este caso, es bien sabido que la función de tasa-distorsión se puede calcular en forma cerrada como:

$$\mathcal{R}_\epsilon(x) = \begin{cases} \frac{1}{2}\log\left(\sigma^2/\epsilon^2\right), & 0 \leq \epsilon \leq \sigma, \\ 0, & \epsilon > \sigma \end{cases}. \tag{4.1.10}$$

Esto se extiende de manera particularmente interesante al caso de D dimensiones. Supongamos que $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0},\boldsymbol{\Sigma})$, donde $\boldsymbol{\Sigma} \in \mathsf{PSD}(D)$ tiene valores propios $\lambda_1 \geq \cdots \geq \lambda_D \geq 0$. Entonces, la tasa-distorsión puede obtenerse mediante el «relleno de agua» como

$$\mathcal{R}_\epsilon(\boldsymbol{x}) = \frac{1}{2}\sum_{i=1}^{D} \log\left(\frac{\lambda_i}{\min\{\kappa,\lambda_i\}}\right), \tag{4.1.11}$$

donde el «nivel de agua» κ cumple que $\sum_{i=1}^{D}\min\{\kappa,\lambda_i\} = \epsilon^2$. Intuitivamente, las direcciones con una varianza λ_i mayor que κ se codifican de manera efectiva, mientras que las direcciones con una menor varianza se descartan, lo que garantiza que la restricción de distorsión se cumpla con una información mutua mínima. Consulte el Ejercicio 4.4 para ver una derivación detallada de esta fórmula cerrada. ■

Nota 4.1. Resulta que la tasa-distorsión es una aproximación viable a la entropía de $\boldsymbol{x}$ en el siguiente sentido. Supongamos que $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$ son vectores aleatorios continuos. Entonces, la información mutua se puede escribir como

$$I(\boldsymbol{x};\hat{\boldsymbol{x}}) = h(\boldsymbol{x}) - h(\boldsymbol{x} \mid \hat{\boldsymbol{x}}), \tag{4.1.12}$$

donde $h(\boldsymbol{x} \mid \hat{\boldsymbol{x}}) = -\mathbb{E}[\log_2 p(\boldsymbol{x} \mid \hat{\boldsymbol{x}})]$ es la *entropía condicional* de $\boldsymbol{x}$ dado $\hat{\boldsymbol{x}}$. Por lo tanto, la tasa de codificación mínima se alcanza cuando la diferencia entre la

entropía de $\boldsymbol{x}$ y la entropía condicional de $\boldsymbol{x}$ dado $\hat{\boldsymbol{x}}$ se minimiza entre todas las distribuciones que satisfacen la restricción: $\mathbb{E}[\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2] \leq \epsilon^2$.

De hecho, no es necesario suponer que $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$ sean continuos para llegar a una conclusión como la anterior. Por ejemplo, si ambos vectores aleatorios son discretos, tras una interpretación adecuada de la divergencia de KL para vectores aleatorios de valores discretos, tenemos que

$$I(\boldsymbol{x}; \hat{\boldsymbol{x}}) = H(\boldsymbol{x}) - H(\boldsymbol{x} \mid \hat{\boldsymbol{x}}), \tag{4.1.13}$$

donde $H(\boldsymbol{x})$ se define en la ecuación (3.1.1) de la Sección 3.1.3. En términos más generales, se pueden emplear conceptos matemáticos avanzados de la teoría de la medida para definir la información mutua (y, por lo tanto, la tasa-distorsión) para cualquier variable aleatoria $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$, incluidas aquellas con distribuciones de baja dimensión bastante exóticas; véase Cover and Thomas [CT91, §8.5].

Nota 4.2. Dado un conjunto de puntos de datos en $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, siempre se pueden interpretar como muestras de una distribución discreta uniforme con una probabilidad igual a $1/N$ sobre estos N vectores. La entropía de dicha distribución es $H(\boldsymbol{X}) = \frac{1}{N} \log_2 N$.[7] Sin embargo, aunque $\boldsymbol{X}$ sea una distribución uniforme sobre sus muestras, la tasa de codificación $\mathcal{R}_\epsilon(\boldsymbol{X})$ alcanzable con un esquema de codificación con pérdida podría ser significativamente inferior a $H(\boldsymbol{X})$ si estas muestras no están tan uniformemente distribuidas y muchas se agrupan muy cerca unas de otras. Por lo tanto, para la segunda distribución que se muestra en la Figura 4.2, si se elige adecuadamente el error de cuantización ϵ, la tasa de codificación con pérdida que se puede alcanzar puede ser significativamente menor que si se codificara como una distribución uniforme.[8] Obsérvese también que, con el concepto de tasa-distorsión, la dificultad mencionada en el Ejemplo 4.3 también desaparece: podemos elegir dos números racionales que estén lo suficientemente cerca de cada uno de los dos números irracionales. El esquema de codificación resultante tendrá una complejidad finita.

Ejemplo 4.5. A veces, puede darse una situación opuesta, en la que queremos fijar primero la tasa de codificación e intentar encontrar un esquema de codificación que minimice la distorsión. Por ejemplo, supongamos que solo queremos utilizar un número fijo de códigos para los puntos muestreados de una distribución, y queremos saber cómo diseñar los códigos de tal manera que se minimice la distorsión promedio o máxima durante el esquema de codificación/decodificación. En el caso de una distribución uniforme en un cuadrado unitario, nos preguntamos con qué precisión podemos codificar los puntos extraídos de esta distribución, digamos, con n bits. Este problema equivale a preguntarse cuál es el radio mínimo (es decir, la distorsión) tal que podamos cubrir el cuadrado unitario con 2^n discos de ese radio. La Figura 4.3 muestra cubrimientos aproximadamente óptimos de un cuadrado con $n = 4, 6, 8,$

[7] Nótese, una vez más, que aunque podamos codificar estos vectores con esta tasa de codificación, no podemos decodificarlos con una precisión arbitraria.

[8] No obstante, para esta distribución uniforme discreta, cuando ϵ es lo suficientemente pequeño, siempre se cumple que $H(\boldsymbol{X}) \approx \mathcal{R}_\epsilon(\boldsymbol{X})$.

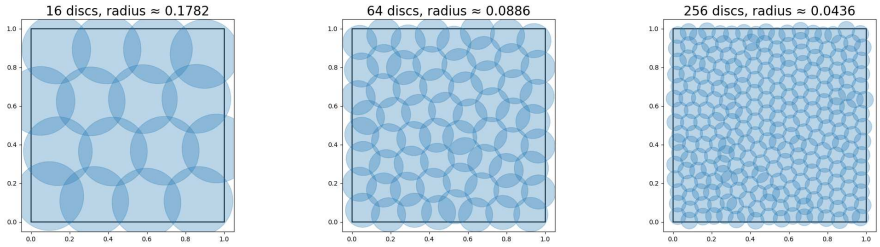


Figura 4.3: Aproximaciones a las soluciones óptimas para 2^4, 2^6 y 2^8 discos que cubren un cuadrado, junto con los radios correspondientes, calculados mediante un algoritmo heurístico de optimización.

de modo que $2^n = 16, 64, 256$ discos, respectivamente. Obsérvese que los radios óptimos de los discos disminuyen a medida que aumenta el número de discos 2^n. ■

Resulta ser un problema notoriamente difícil obtener expresiones en forma cerrada para la función de tasa-distorsión (4.1.9) para distribuciones generales $p(\boldsymbol{x})$. Sin embargo, tal y como sugiere el Ejemplo 4.5, existen casos especiales importantes en los que la *geometría* del soporte de la distribución $p(\boldsymbol{x})$ puede vincularse con la función de tasa-distorsión y, por lo tanto, con la tasa de codificación óptima para un nivel de distorsión ϵ. De hecho, este ejemplo puede generalizarse a cualquier contexto en el que el soporte de $p(\boldsymbol{x})$ sea un conjunto compacto suficientemente regular —incluidas las distribuciones de baja dimensión— y $p(\boldsymbol{x})$ esté uniformemente distribuída en su soporte. Esto abarca una gran cantidad de casos de interés práctico. Formalizamos esta noción en el siguiente resultado, que establece esta propiedad para un caso especial.

Teorema 4.1. *Supongamos que $\boldsymbol{x}$ es una variable aleatoria tal que su soporte $K \doteq \operatorname{Supp}(\boldsymbol{x})$ es un conjunto compacto. Definamos el número de cobertura $\mathcal{N}_\epsilon(K)$ como el número mínimo de bolas de radio ϵ que pueden cubrir K, es decir,*

$$\mathcal{N}_\epsilon(K) \doteq \min\left\{ n \in \mathbb{N} \colon \exists \boldsymbol{p}_1, \ldots, \boldsymbol{p}_n \in K \ \textit{s.t.}\ K \subseteq \bigcup_{i=1}^{n} B_\epsilon(\boldsymbol{p}_i) \right\}, \tag{4.1.14}$$

donde $B_\epsilon(\boldsymbol{p}) = \{\boldsymbol{\xi} \in \mathbb{R}^D \mid \|\boldsymbol{\xi} - \boldsymbol{p}\|_2 \leq \epsilon\}$ es la bola euclidiana de radio ϵ centrada en $\boldsymbol{p}$. Entonces se cumple que

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \leq \log_2 \mathcal{N}_\epsilon(K). \tag{4.1.15}$$

Si, además, $\boldsymbol{x}$ sigue una distribución uniforme en K y K es una unión de hiperesferas de baja dimensión mutuamente ortogonales,[9] *entonces se cumple una cota inferior correspondiente:*

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq \log_2 \mathcal{N}_\epsilon(K) - O(D). \tag{4.1.16}$$

[9] De hecho, es posible tratar casos de K altamente irregulares, como variedades compactas o

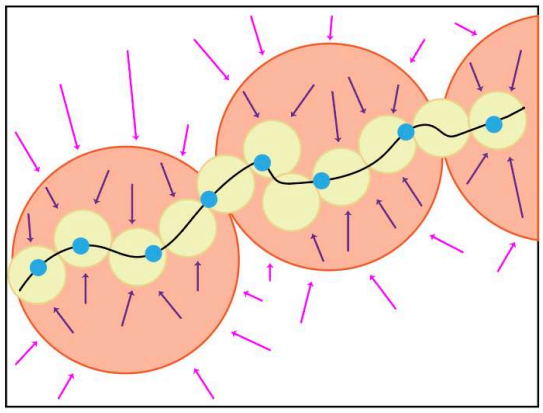


Figura 4.4: **La aproximación de una distribución de baja dimensión mediante bolas de radio ϵ.** Podemos observar que, a medida que el parámetro ϵ se reduce, la unión de las bolas de radio ϵ se aproxima cada vez mejor al soporte de la distribución real (negra). Además, los filtros de eliminación de ruido asociados (cuyo mapeo entrada-salida viene dada por las flechas proporcionadas) obtenidos al aproximar la distribución real mediante una mezcla de distribuciones gaussianas, cada una con covarianza $(\epsilon^2/D)\boldsymbol{I}$, se acercan cada vez más a los filtros de eliminación de ruido verdaderos. Para valores grandes de ϵ, estos eliminadores de ruido no se acercan en absoluto a la distribución real, mientras que para valores pequeños de ϵ se aproximan mucho a los eliminadores de ruido reales. El Teorema 4.1 establece que esta aproximación caracteriza la función de tasa-distorsión para distorsiones pequeñas ϵ, unificando los enfoques paralelos de minimización de la tasa de codificación y eliminación de ruido para el aprendizaje de distribuciones de baja dimensión sin patologías.

Demostración. La demostración de este teorema excede el alcance de este libro, por lo que la remitimos al Apéndice B.3. □

La cota superior del Teorema 4.1 solo requiere que Supp($\boldsymbol{x}$) sea un conjunto compacto, mientras que nuestra demostración de la cota inferior requiere supuestos de regularidad más fuertes: concretamente, que $\boldsymbol{x}$ esté uniformemente distribuido sobre Supp($\boldsymbol{x}$) y que Supp($\boldsymbol{x}$) sea una mezcla de gaussianas de rango bajo (con la normalización adecuada).[10] Bajo estos supuestos, la implicación

incluso fractales, con un resultado paralelo, pero su formulación se vuelve mucho más técnica: véase Riegler et al. [RBK18; RKB23]. En el Apéndice B.3 ofrecemos una demostración sencilla que muestra el resultado para uniones de hiperesferas ortogonales, una suposición que equivale a una mezcla de distribuciones gaussianas de rango bajo con soportes ortogonales cuando la dimensión de cada subespacio es suficientemente grande. Véase el Apéndice B.3 para un análisis más detallado de esta hipótesis.

[10] Más adelante en este capítulo, en la Sección 4.2.3, veremos una función objetivo cuyos máximos son mezclas normalizadas de gaussianas de bajo rango que corresponden a los datos de entrada de cierta manera. El Capítulo 5 desarrollará aún más este marco para el aprendizaje profundo, demostrando que la suposición estructural que hacemos para demostrar la cota inferior no es excesivamente restrictiva.

del Teorema 4.1 puede resumirse de la siguiente manera: para una codificación suficientemente precisa de la distribución de $\boldsymbol{x}$ (es decir, a medida que $\epsilon \to 0$), el marco de codificación de mínima tasa-distorsión es equivalente al problema del empaquetamiento de bolas en el soporte de $\boldsymbol{x}$. Así pues, la tasa-distorsión puede considerarse como una forma «sensible a la probabilidad» de aproximar el soporte de la distribución de $\boldsymbol{x}$ mediante una mezcla de muchas bolas pequeñas (Figura 4.4).

Ahora analizaremos otra relación entre esto y la jerarquía de complejidad «eliminación de ruido-difusión-entropía» que hemos comentado anteriormente en este capítulo.

Nota 4.3. El ingrediente clave en la demostración de la cota inferior en el Teorema 4.1 es un resultado importante de la teoría de la información conocido como el *límite inferior de Shannon* para la tasa-distorsión, que lleva el nombre de Claude Shannon, quien lo derivó por primera vez en un caso especial [Sha59]. En él se establece la siguiente estimación para la función de tasa-distorsión, para cualquier variable aleatoria $\boldsymbol{x}$ con una densidad $p(\boldsymbol{x})$ y una norma al cuadrado con esperanza finita [LZ94]:

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq h(\boldsymbol{x}) - \log_2 \operatorname{vol}(B_\epsilon) - C_D, \tag{4.1.17}$$

donde $C_D > 0$ es una constante que depende únicamente de D. Además, esta cota inferior es, de hecho, óptima cuando $\epsilon \to 0$: es decir,

$$\lim_{\epsilon \to 0} \mathcal{R}_\epsilon(\boldsymbol{x}) - [h(\boldsymbol{x}) - \log_2 \operatorname{vol}(B_\epsilon) - C_D] = 0. \tag{4.1.18}$$

Así pues, cuando la distorsión ϵ es pequeña, podemos pensar únicamente en términos de la cota inferior de Shannon, en lugar del problema de optimización (generalmente inmanejable) que define la relación tasa-distorsión (4.1.9).

La cota inferior de Shannon constituye el nexo entre la tasa de codificación, la minimización de la entropía / la eliminación de ruido y los enfoques de empaquetamiento geométrico de esferas para el aprendizaje de distribuciones de baja dimensión. Obsérvese que, en el caso especial de una densidad uniforme $p(\boldsymbol{x})$, (4.1.17) se convierte en

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq -\int_K \frac{1}{\operatorname{vol}(K)} \log_2 \frac{1}{\operatorname{vol}(K)} \mathrm{d}\boldsymbol{\xi} - \log_2 \operatorname{vol}(B_\epsilon) - C_D \tag{4.1.19}$$

$$= \log_2 \operatorname{vol}(K)/\operatorname{vol}(B_\epsilon) - C_D. \tag{4.1.20}$$

La relación $\operatorname{vol}(K)/\operatorname{vol}(B_\epsilon)$ aproxima el número de bolas de radio ϵ necesarias para cubrir K mediante un argumento del peor caso, lo cual es preciso para conjuntos K suficientemente regulares cuando ϵ es pequeño (véase el Apéndice B.3 para más detalles). Mientras tanto, recordemos el modelo gaussiano de eliminación de ruido $\boldsymbol{x}_\epsilon = \boldsymbol{x} + \epsilon \boldsymbol{g}$ mencionado anteriormente en este capítulo, donde $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ es independiente de $\boldsymbol{x}$. Curiosamente, la entropía diferencial de la distribución conjunta $(\boldsymbol{x}, \epsilon \boldsymbol{g})$ se puede calcular como

$$h(\boldsymbol{x}, \epsilon \boldsymbol{g}) = -\int p(\boldsymbol{\xi}) p(\boldsymbol{\gamma}) \log_2 p(\boldsymbol{\xi}) p(\boldsymbol{\gamma}) \mathrm{d}\boldsymbol{\xi} \mathrm{d}\boldsymbol{\gamma} \tag{4.1.21}$$

$$= h(\boldsymbol{x}) + h(\epsilon \boldsymbol{g}). \tag{4.1.22}$$

Hemos visto la entropía gaussiana calculada en la Ecuación (3.1.4): cuando ϵ es pequeño, es igual, salvo por constantes aditivas, a la cantidad $-\log_2 \mathrm{vol}(B_\epsilon)$ que hemos visto en la cota inferior de Shannon. En ciertos casos especiales (por ejemplo, datos apoyados en subespacios incoherentes de bajo rango), cuando ϵ es pequeño y el soporte de $\boldsymbol{x}$ es suficientemente regular, la distribución de $\boldsymbol{x}_\epsilon$ admite incluso una buena aproximación local mediante el producto de las distribuciones $p(\boldsymbol{x})$ y $p(\boldsymbol{g})$, lo que justifica el cálculo anterior. Por lo tanto, el proceso de eliminación de ruido gaussiano ofrece otra interpretación más de la cota inferior de Shannon, ya que se deriva de la entropía de una versión con ruido de $\boldsymbol{x}$, con un nivel de ruido proporcional al nivel de distorsión ϵ.

Por lo tanto, este enfoque de tasa-distorsión finita mediante el recubrimiento de esferas revalida o generaliza todas las medidas previas de complejidad de la distribución, lo que nos permite diferenciar y clasificar las distintas distribuciones de manera unificada. Estas perspectivas interrelacionadas se visualizan en la Figura 4.4.

En el caso de una distribución general con niveles de distorsión finitos, suele ser imposible hallar su función de tasa-distorsión en forma analítica. A menudo hay que recurrir al cálculo numérico[11]. Sin embargo, como veremos, en nuestro contexto a menudo necesitamos conocer la tasa-distorsión como una función explícita de un conjunto de puntos de datos o de sus representaciones. Esto se debe a que queremos utilizar la tasa de codificación como medida de la calidad de las representaciones. Una forma analítica explícita facilita determinar cómo transformar la distribución de los datos para mejorar la representación. Por lo tanto, deberíamos trabajar con distribuciones cuyas funciones de tasa-distorsión adopten formas analíticas explícitas. Con este fin, comenzaremos con la familia de distribuciones más simple y, a la vez, la más importante.

4.1.3 Tasa de codificación con pérdida para una distribución gaussiana de baja dimensión

Supongamos ahora que disponemos de un conjunto de muestras de datos en $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ procedente de cualquier distribución.[12] Nos gustaría elaborar un esquema constructivo que pueda codificar los datos con cierta precisión, digamos

$$\boldsymbol{x}_i \mapsto \hat{\boldsymbol{x}}_i, \quad \text{sujeto a} \quad \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|_2 \leq \epsilon. \tag{4.1.23}$$

Nótese que se trata de una condición suficiente, explícita e interpretable que garantiza que los datos se codifiquen de tal manera que $\frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|_2^2 \leq \epsilon^2$. Esta última desigualdad es precisamente la restricción de tasa-distorsión para la distribución empírica dada y su codificación. Por ejemplo, en el Ejemplo 4.5,

[11] Los lectores interesados pueden consultar [Bla72] para conocer un algoritmo clásico que calcula numéricamente la función de tasa-distorsión para una distribución discreta.

[12] O bien, estos puntos de datos podrían considerarse ellos mismos como una distribución (empírica).

utilizamos este criterio simplificado para determinar la distorsión mínima y el esquema de codificación explícitos para una tasa de codificación dada.

Sin pérdida de generalidad, supongamos que la media de $\boldsymbol{X}$ es cero, es decir, $\frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i = \boldsymbol{0}$. Sin ningún conocimiento previo sobre la naturaleza de la distribución subyacente a $\boldsymbol{X}$, podemos considerar que $\boldsymbol{X}$ se ha obtenido a partir de una distribución gaussiana $\mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma})$ con la covarianza[13]

$$\boldsymbol{\Sigma} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\top}. \tag{4.1.24}$$

Obsérvese que, desde el punto de vista geométrico, $\boldsymbol{\Sigma}$ caracteriza una región elipsoidal en la que se encuentran la mayoría de las muestras $\boldsymbol{x}_i$.

Podemos considerar $\hat{\boldsymbol{X}} = [\hat{\boldsymbol{x}}_1, \ldots, \hat{\boldsymbol{x}}_N]$ como una versión con ruido de $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$:

$$\hat{\boldsymbol{x}}_i = \boldsymbol{x}_i + \boldsymbol{w}_i, \tag{4.1.25}$$

donde $\boldsymbol{w}_i \sim \mathcal{N}(\boldsymbol{0}, \epsilon^2\boldsymbol{I}/D)$ es un ruido gaussiano independiente de $\boldsymbol{x}_i$. Entonces, la covarianza de $\hat{\boldsymbol{x}}_i$ viene dada por

$$\hat{\boldsymbol{\Sigma}} = \mathbb{E}\left[\hat{\boldsymbol{x}}_i\hat{\boldsymbol{x}}_i^{\top}\right] = \frac{\epsilon^2}{D}\boldsymbol{I} + \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\top}. \tag{4.1.26}$$

Nótese que el volumen de la región generada por los vectores $\boldsymbol{x}_i$ es proporcional a la raíz cuadrada del determinante de la matriz de covarianza

$$\text{volumen}(\hat{\boldsymbol{x}}_i) \propto \sqrt{\det\left(\hat{\boldsymbol{\Sigma}}\right)} = \sqrt{\det\left(\frac{\epsilon^2}{D}\boldsymbol{I} + \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\top}\right)}. \tag{4.1.27}$$

El volumen engendrado por cada vector aleatorio $\boldsymbol{w}_i$ es proporcional a

$$\text{volumen}(\boldsymbol{w}_i) \propto \sqrt{\det\left(\frac{\epsilon^2}{D}\boldsymbol{I}\right)}. \tag{4.1.28}$$

Para codificar los vectores que se encuentran dentro de la región abarcada por $\hat{\boldsymbol{x}}_i$, podemos cubrir dicha región con bolas no superpuestas de radio ϵ, como se ilustra en la Figura 4.5. Cuando el volumen de la región abarcada por $\hat{\boldsymbol{x}}_i$ es significativamente mayor que el volumen de la bola de radio ϵ, el número total de bolas que necesitamos para cubrir la región es aproximadamente igual a el cociente entre ambos volúmenes:

$$\#\,\epsilon\text{-bolas} \approx \frac{\text{volumen}(\hat{\boldsymbol{x}}_i)}{\text{volumen}(\boldsymbol{w}_i)} = \sqrt{\det\left(\boldsymbol{I} + \frac{D}{N\epsilon^2}\boldsymbol{X}\boldsymbol{X}^{\top}\right)}. \tag{4.1.29}$$

Si utilizamos números binarios para etiquetar todas las bolas de radio ϵ en la región de interés, el número total de bits necesarios es, por lo tanto,

$$\mathcal{R}_{\epsilon}(\boldsymbol{X}) \approx \log_2(\#\,\epsilon\text{-bolas}) \approx R_{\epsilon}(\boldsymbol{X}) \doteq \frac{1}{2}\log\det\left(\boldsymbol{I} + \frac{D}{N\epsilon^2}\boldsymbol{X}\boldsymbol{X}^{\top}\right). \tag{4.1.30}$$

[13] Se sabe que, dada una varianza fija, la distribución gaussiana alcanza la máxima entropía. Es decir, establece un límite superior para lo que podría ser el peor de los casos en cuanto a la tasa de codificación posible.

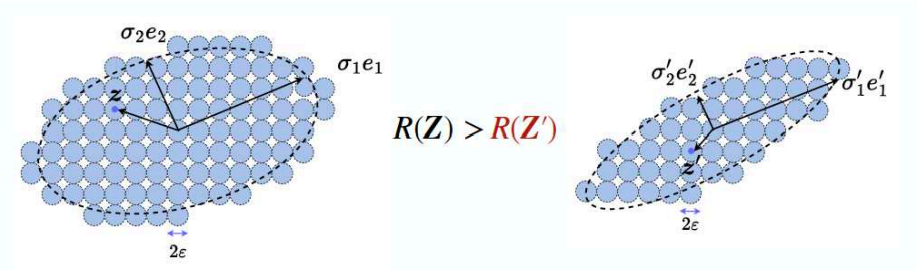


Figura 4.5: Cubrir la región abarcada por los vectores de datos utilizando bolas de radio ϵ. Cuanto mayor sea el volumen del espacio, más bolas se necesitarán y, por lo tanto, más bits se necesitarán para codificar y enumerar las bolas. Cada vector de valor real $\boldsymbol{x}$ puede codificarse como el índice de la bola en la que se encuentra ese vector.

Nótese que, a partir del Ejemplo 4.4, se puede mostrar que el lado derecho de esta fórmula coincide con la tasa-distorsión (hasta una precisión de ϵ) de una fuente gaussiana con matriz de covarianza $\frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top + (\epsilon^2/D)\boldsymbol{I}$ (por ejemplo, la matriz de covarianza empírica de los datos más un término de ruido). De este modo, un argumento geométrico sobre el volumen de los vectores encerrados y el empaquetamiento de esferas se traduce en un argumento basado en la teoría de la información sobre la tasa-distorsión de una fuente gaussiana perturbada.

Ejemplo 4.6. La Figura 4.5 muestra un ejemplo de una distribución bidimensional con un soporte elipsoidal, que se aproxima al soporte de una distribución gaussiana bidimensional. La región está cubierta por pequeñas bolas de tamaño ϵ. Todas las bolas están numeradas del 1 a, digamos, n. Entonces, dado cualquier vector $\boldsymbol{x}$ en esta región, solo necesitamos determinar cuál es la bola de radio ϵ cuyo centro está más cerca, lo que se denota como $\text{bola}_\epsilon(\boldsymbol{x})$. Para recordar $\boldsymbol{x}$, solo necesitamos recordar el número de esta bola, cuyo almacenamiento requiere $\log(n)$ bits. Si necesitamos decodificar $\boldsymbol{x}$ a partir de este número, simplemente tomamos $\hat{\boldsymbol{x}}$ como el centro de la bola. Esto da lugar a un esquema explícito de codificación y decodificación:

$$\boldsymbol{x} \longrightarrow \text{bola}_\epsilon(\boldsymbol{x}) \longrightarrow \hat{\boldsymbol{x}} = \text{centro de bola}_\epsilon(\boldsymbol{x}). \tag{4.1.31}$$

Estos centros de las bolas pueden denominarse «códigos» del libro de códigos (o diccionario) del esquema de codificación. Es fácil ver que la precisión de este esquema de codificación-decodificación (con pérdida) es aproximadamente el radio ϵ de las bolas. Es evidente que $\mathcal{R}_\epsilon(\boldsymbol{Z})$ es el número promedio de bits necesarios para codificar el número de bola de cada vector $\boldsymbol{z}$ con este esquema de codificación y, por lo tanto, puede denominarse la *tasa de codificación* asociada a este esquema. ■

A partir de la derivación anterior, sabemos que la tasa de codificación $\mathcal{R}_\epsilon(\boldsymbol{X})$ es (aproximadamente) alcanzable mediante un esquema explícito de codificación (y decodificación). Presenta dos propiedades interesantes:

- En primer lugar, se puede observar que $R_\epsilon(\boldsymbol{X})$ se asemeja mucho a la función de tasa-distorsión de una fuente gaussiana [CT91] (véase el Ejemplo 4.4). De hecho, cuando ϵ es pequeño, la expresión anterior es una aproximación muy cercana a la tasa-distorsión de una fuente gaussiana, tal como se señala en [MDH+07a].

- En segundo lugar, la misma tasa de codificación en forma cerrada $R_\epsilon(\boldsymbol{X})$ puede derivarse como una aproximación de $\mathcal{R}_\epsilon(\boldsymbol{X})$ si se supone que los datos $\boldsymbol{X}$ proceden de un subespacio lineal. Esto se puede demostrar cuantificando adecuadamente la descomposición en valores singulares (SVD) de $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ y construyendo un esquema de codificación con pérdida para vectores en el subespacio generado por $\boldsymbol{U}$ [MDH+07a].

En nuestro contexto, la expresión en forma cerrada $R_\epsilon(\boldsymbol{X})$ es bastante fundamental: se trata de la tasa de codificación asociada a un esquema explícito y natural de codificación con pérdidas para datos extraídos de una distribución gaussiana o de un subespacio lineal. Como veremos en el próximo capítulo, esta fórmula desempeña un papel importante en la comprensión de la arquitectura de las redes neuronales profundas.

4.1.4 Agrupamiento de una mezcla de gaussianas de baja dimensión

Como ya hemos comentado anteriormente, el conjunto de datos dado $\boldsymbol{X}$ suele tener estructuras intrínsecas de baja dimensión. Por lo tanto, codificarlo como una distribución gaussiana general resultaría muy redundante. Si podemos identificar esas estructuras intrínsecas en $\boldsymbol{X}$, podríamos diseñar esquemas de codificación mucho mejores que ofrezcan tasas de codificación mucho más bajas. O, dicho de otra forma, los códigos utilizados para codificar el $\boldsymbol{X}$ en cuestión pueden comprimirse. Veremos que la compresión ofrece una forma computable y unificadora de identificar tales estructuras. En esta sección, ilustramos esta importante idea con la familia más básica de estructuras de baja dimensión: una mezcla de gaussianas (de baja dimensión) o subespacios.

Ejemplo 4.7. La Figura 4.6 muestra un ejemplo en el que los datos $\boldsymbol{X}$ se distribuyen alrededor de dos subespacios (o gaussianas de baja dimensión). Si se consideran y se codifican conjuntamente como una sola gaussiana, el libro de códigos discreto (con pérdida) asociado, representado por todas las bolas azules, es obviamente muy redundante. Podemos intentar identificar las ubicaciones de los dos subespacios, denotados por S_1 y S_2 y diseñar un libro de códigos que abarque únicamente esos dos subespacios, es decir, las bolas verdes. Si logramos particionar correctamente las muestras de los datos $\boldsymbol{X}$ en los dos subespacios: $\boldsymbol{X} = [\boldsymbol{X}_1, \boldsymbol{X}_2]\boldsymbol{\Pi}$ con $\boldsymbol{X}_1 \in S_1$ y $\boldsymbol{X}_2 \in S_2$, donde $\boldsymbol{\Pi}$ denota una matriz de permutación, entonces la tasa de codificación resultante para los datos será mucho menor. Esto proporciona una representación más parsimoniosa y, por lo tanto, más deseable de los datos. ■

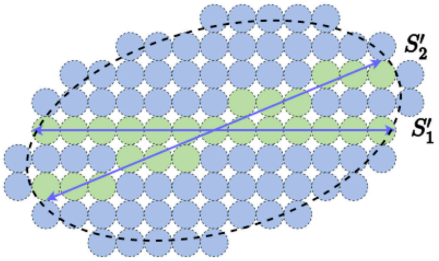


Figura 4.6: Comparación de dos esquemas de codificación con pérdidas para datos que se distribuyen alrededor de dos subespacios. El primero Uno consiste en empacar (azul) bolas de radio ϵ para todo el espacio generado por los dos subespacios; el segundo consiste en empacar bolas solo en una vecindad tabular alrededor de los dos subespacios. Este último, obviamente, tiene un libro de códigos mucho más pequeño y da como resultado una tasa de codificación mucho menor para las muestras en los subespacios.

Por lo tanto, en términos más generales, si los datos proceden de cualquier mezcla de subespacios o gaussianas de baja dimensión, sería deseable identificar esas componentes y codificar los datos basándose en las dimensiones intrínsecas de dichas componentes. Resulta que no perdemos mucha generalidad al suponer que los datos provienen de una mezcla de distribuciones gaussianas de baja dimensión. Esto se debe a que una mezcla de distribuciones gaussianas puede aproximar con gran precisión a la mayoría de las distribuciones generales [BDS16].

El problema de agrupamiento. Ahora bien, en el caso de esta familia concreta de distribuciones, ¿cómo podemos identificar de manera eficaz y eficiente esas componentes de baja dimensión a partir de un conjunto de muestras

$$\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N]\,, \tag{4.1.32}$$

extraído de ellos? En otras palabras, dado el conjunto de datos completo $\boldsymbol{X}$, queremos dividirlo, o agruparlo, en varios subconjuntos, digamos K subconjuntos:

$$\boldsymbol{X\Pi} = [\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_K], \tag{4.1.33}$$

donde cada subconjunto está formado por muestras extraídas de una sola gaussiana o subespacio de baja dimensión y $\boldsymbol{\Pi}$ es una matriz de permutación que indica la pertenencia a la partición. Cabe señalar que, dependiendo de la situación, la partición podría ser determinista o probabilística. Como se muestra en [MDH+07b], en el caso de una mezcla de distribuciones gaussianas, la partición probabilística no da lugar a una menor tasa de codificación. Por lo tanto, en aras de la simplicidad, aquí consideramos únicamente una partición determinista.

Agrupamiento mediante compresión con pérdidas. La principal dificultad para resolver el problema de agrupamiento mencionado anteriormente radica en que normalmente no conocemos el número de agrupamientos K, ni tampoco la dimensión de cada componente. El estudio de este problema de agrupamiento cuenta con una larga trayectoria. El libro de texto [VMS16] ofrece una exposición sistemática y exhaustiva de los diferentes enfoques de este problema. Para encontrar un enfoque eficaz para este problema, primero debemos comprender y aclarar por qué queremos agrupar los datos. En otras palabras, ¿qué ganamos exactamente al agrupar los datos, en comparación con no hacerlo? ¿Cómo medimos esa ganancia? Desde el punto de vista de la compresión de datos, una agrupación correcta debería dar lugar a un esquema de codificación (y decodificación) más eficiente.

Para cualquier conjunto de datos dado $\boldsymbol{X}$, ya existen dos esquemas de codificación obvios que sirven de referencia. Estos representan dos formas extremas de codificar los datos:

- Simplemente considerar todas las muestras juntas como si procedieran de una única distribución gaussiana. La tasa de codificación asociada, tal y como se ha deducido anteriormente, viene dada por:

$$\mathcal{R}_\epsilon(\boldsymbol{X}) \approx R_\epsilon(\boldsymbol{X}) = \frac{1}{2}\log\det\left(\boldsymbol{I} + \frac{D}{N\epsilon^2}\boldsymbol{X}\boldsymbol{X}^\top\right). \tag{4.1.34}$$

- Simplemente memorizar todas las muestras por separado asignando un número diferente a cada una. La tasa de codificación sería:

$$\mathcal{R}_0(\boldsymbol{X}) = \log(N). \tag{4.1.35}$$

Tengamos en cuenta que cualquiera de los dos esquemas de codificación puede convertirse en la solución «óptima» para una determinada elección (extrema) del error de cuantización ϵ:

1. *Régimen perezoso*: Si elegimos un valor de ϵ extremadamente grande, todas las muestras de $\boldsymbol{X}$ pueden quedar cubiertas por una sola bola. La tasa es $\lim_{\epsilon\to\infty}\mathcal{R}_\epsilon \to \frac{1}{2}\log\det(\boldsymbol{I}) = 0$.

2. *Régimen de memorización*: Si ϵ es extremadamente pequeño, cada muestra de $\boldsymbol{X}$ queda cubierta por una bola de ϵ diferente, por lo que el total es N. La tasa es $\lim_{\epsilon\to 0}\mathcal{R}_\epsilon \to \log(N)$.

Nótese que el primer esquema corresponde al caso en el que no nos interesa en absoluto ningún aspecto relevante de la distribución. No se desea reservar ningún bit para información relevante. A esto lo llamamos el «régimen perezoso». El segundo esquema corresponde al caso en el que se desea decodificar cada muestra con una precisión extremadamente alta. Por lo tanto, sería mejor «memorizar» cada muestra. A esto lo llamamos el «régimen de memorización».

Nótese que estas asintóticas en términos de ϵ son exactamente las mismas que se analizaron al presentar la tasa de codificación con pérdida como una

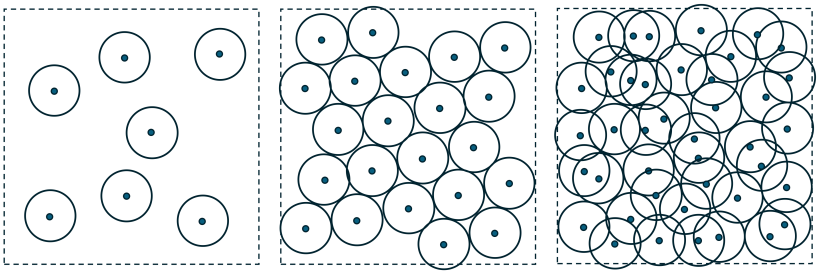


Figura 4.7: Varias de muestras aleatorias en un plano bidimensional. Consideremos un disco de radio ϵ asociado a cada muestra, con la muestra como centro. La densidad de las muestras aumenta de izquierda a derecha.

magnitud que los modelos de difusión entrenados de manera realista aproximan, tal como se muestra al final de la Sección 3.3. En la situación actual, codificamos *explícitamente* la distribución con una precisión de ϵ; en el caso de los modelos de difusión, codificamos *implícitamente* la distribución. Ciertas propiedades del esquema, como la memorización y la generalización, se manifiestan de manera similar en ambos casos y, de hecho, se puede estudiar un caso para comprender el otro.

Ejemplo 4.8. Para determinar cuándo es preferible o no el régimen de memorización, consideremos un número, digamos N, de muestras distribuidas aleatoriamente en un área unitaria sobre un plano bidimensional.[14] Imaginemos que intentamos diseñar un esquema de codificación con pérdida con un error de cuantización fijo ϵ. Esto equivale a colocar un disco de ϵ alrededor de cada muestra, como se muestra en la Figura 4.7. Cuando N es pequeño, la probabilidad de que todos los discos se superpongan entre sí es nula. En este caso, es necesario y óptimo un libro de códigos de tamaño N. Cuando N o la densidad alcanza un determinado valor crítico N_c, con alta probabilidad todos los discos comienzan a superponerse y a conectarse en un único grupo que cubre todo el plano; este fenómeno se conoce como «percolación» en el continuo [Gil61; MM12]. Cuando N supera este valor, los discos se superponen en gran medida. El número N de discos resulta muy redundante, ya que solo queremos codificar puntos en el plano con la precisión dada ϵ. El número de discos necesarios para cubrir todas las muestras es muy inferior a N.[15] ■

Tanto el régimen «perezoso» como el de memorización son en cierta medida triviales y quizás tengan poco interés teórico o práctico. Cualquiera de ambos esquemas estaría lejos de ser óptimo si se utilizara para codificar un gran número de muestras extraídas de una distribución que tenga un *soporte compacto y de baja dimensión*. El régimen interesante se encuentra entre estos dos.

[14] Supongamos que los puntos se generan mediante un proceso de Poisson con una densidad de N puntos por área unitaria.

[15] De hecho, existen algoritmos eficientes para encontrar dicha cobertura [BBF+01].

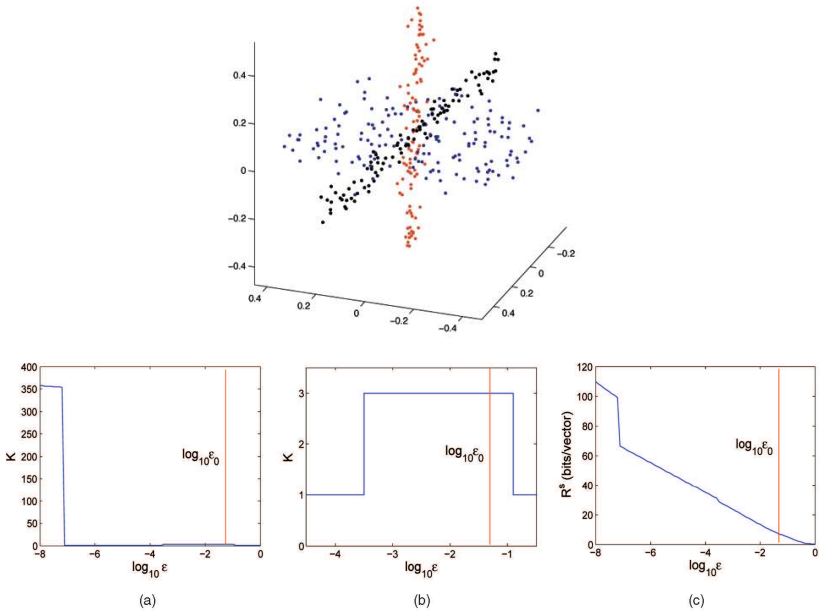


Figura 4.8: Parte superior: 358 muestras ruidosas extraídas de dos rectas y un plano en $\mathbb{R}^3$. Parte inferior: el efecto de variar ϵ sobre el resultado de la agrupación y la tasa de codificación. La línea roja marca la varianza ϵ_0 del ruido gaussiano añadido a las muestras. Obsérvese que los gráficos de la parte inferior izquierda y del centro muestran la misma magnitud con diferentes posiciones y escalas de los ejes.

Ejemplo 4.9. La Figura 4.8 muestra un ejemplo con muestras ruidosas extraídas de dos rectas y un plano en $\mathbb{R}^3$. Como se observa en la figura (c) de la derecha, la tasa de codificación óptima disminuye en forma monótona a medida que aumentamos ϵ, tal y como se esperaba a partir de la propiedad de la función de tasa-distorsión. Las figuras (a) y (b) muestran, al variar ϵ desde valores muy pequeños (cercanos a cero) hasta valores muy grandes (cercanos al infinito), el número óptimo de grupos cuando la tasa de codificación es mínima. Se pueden observar claramente el régimen «perezoso» y el régimen de memorización en los dos extremos de los gráficos. Pero también se puede observar en el figura (b) que, cuando el error de cuantización ϵ se elige en torno al nivel de la varianza real del ruido ϵ_0, el número óptimo de grupos es el número «correcto» tres, que representa dos líneas y un plano. Nos referimos informalmente a este régimen intermedio como el «régimen de generalización». Obsérvese que se produce una transición de fase brusca entre estos regímenes.[16] ■

A partir de la discusión y los ejemplos anteriores, vemos que, cuando el

[16]Hasta donde sabemos, no existe una justificación teórica rigurosa para estos comportamientos de transición de fase.

error de cuantización en relación con la densidad de la muestra[17] se encuentra dentro de un rango adecuado, minimizar la tasa de codificación con pérdida nos permitiría descubrir la distribución subyacente (de baja dimensión) de los datos muestreados. *Por lo tanto, la cuantización, que comenzó como una elección de carácter práctico, parece estar convirtiéndose en algo necesario para aprender una distribución continua a partir de su distribución empírica con un número finito de muestras.* Aunque sigue siendo esquiva una teoría rigurosa que explique este fenómeno, aquí, con fines de aprendizaje, nos interesa cómo aprovechar este fenómeno para diseñar algoritmos capaces de encontrar la distribución correcta.

Utilicemos el sencillo ejemplo que se muestra en la Figura 4.6 para ilustrar las ideas básicas. Si se pueden particionar todas las muestras en $\boldsymbol{X}$ en dos grupos en $\boldsymbol{X}_1$ y $\boldsymbol{X}_2$, con N_1 y N_2 muestras respectivamente, entonces la tasa de codificación asociada sería[18]

$$R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) = \frac{N_1}{N} R_\epsilon(\boldsymbol{X}_1) + \frac{N_2}{N} R_\epsilon(\boldsymbol{X}_2), \tag{4.1.36}$$

donde utilizamos $\boldsymbol{\Pi}$ para indicar la pertenencia a la partición. Si la partición respeta las estructuras de baja dimensión de la distribución — en este caso, la pertenencia de $\boldsymbol{X}_1$ y $\boldsymbol{X}_2$ a los dos subespacios, respectivamente, entonces la tasa de codificación resultante debería ser significativamente menor que la de los dos esquemas básicos anteriores:

$$R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) \ll R_\epsilon(\boldsymbol{X}), \quad R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) \ll R_0(\boldsymbol{X}). \tag{4.1.37}$$

En general, podemos reformular el problema de agrupamiento como un problema de optimización que minimiza la tasa de codificación:

$$\min_{\boldsymbol{\Pi}} \left\{ R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) \doteq \sum_{k=1}^{K} \frac{N_k}{N} R_\epsilon(\boldsymbol{X}_k) \right\}. \tag{4.1.38}$$

Estrategias de optimización para el agrupamiento. La pregunta que queda por resolver es cómo optimizar el objetivo de tasa de codificación anterior para encontrar los grupos óptimos. Existen tres enfoques naturales para abordar este objetivo:

1. Podemos empezar con el conjunto completo $\boldsymbol{X}$ como un único grupo (es decir, el régimen «perezoso») y luego buscar (por ejemplo, al azar) la forma de dividirlo de modo que se obtenga una tasa de codificación menor.

2. A la inversa, podemos partir de cada muestra $\boldsymbol{x}_i$ como un grupo independiente (es decir, el régimen de memorización) y buscar fusionar grupos que den lugar a una tasa de codificación menor.

[17] o la densidad de la muestra en relación con el error de cuantización

[18] Aquí ignoramos algunos bits de sobrecarga necesarios para codificar la pertenencia de cada muestra, por ejemplo, mediante la codificación de Huffman.

3. Por otra parte, si pudiéramos representar (o aproximar) la función de pertenencia $\mathbf{\Pi}$ mediante unos parámetros continuos, podríamos utilizar métodos de optimización como el descenso por gradiente (GD).

El primer enfoque no resulta tan atractivo desde el punto de vista computacional, ya que el número de particiones posibles que hay que probar es exponencial con respecto al número de muestras. Por ejemplo, el número de particiones de $\boldsymbol{X}$ en dos subconjuntos de igual tamaño es $\binom{N}{N/2}$, lo cual se dispara a medida que N aumenta. Analizaremos el tercer enfoque en el próximo Capítulo 5. Allí veremos cómo el papel de las redes neuronales profundas, en particular los transformers, está relacionado con el objetivo de la tasa de codificación.

El segundo enfoque fue propuesto originalmente en el trabajo de [MDH+07b]. Este trabajo demuestra las ventajas de poder evaluar la tasa de codificación de manera eficiente (por ejemplo, mediante una fórmula analítica). Gracias a ello, los grupos (de baja dimensión) de datos pueden identificarse en forma bastante eficiente y eficaz mediante el principio de minimización de la longitud de codificación (MCL). Nótese que, para un grupo $\boldsymbol{X}_k$ con N_k muestras, el número de bits binarios necesarios para codificar todas las muestras de $\boldsymbol{X}_k$ viene dada por:[19]

$$L(\boldsymbol{X}_k) = N_k R_\epsilon(\boldsymbol{X}_k). \tag{4.1.39}$$

Si tenemos dos grupos $\boldsymbol{X}_k$ y $\boldsymbol{X}_l$ y queremos codificar las muestras como dos grupos separados, el número de bits binarios necesarios es

$$L^c(\boldsymbol{X}_k, \boldsymbol{X}_l) = N_k R_\epsilon(\boldsymbol{X}_k) + N_l R_\epsilon(\boldsymbol{X}_l) - N_k \log \frac{N_k}{N_k + N_l} - N_l \log \frac{N_l}{N_k + N_l}.$$

Los dos últimos términos corresponden al número de bits necesarios para codificar la pertenencia de las muestras según el código de Huffman.

Entonces, dados cualesquiera dos grupos distintos $\boldsymbol{X}_1$ y $\boldsymbol{X}_2$, podemos decidir si fusionarlos o no basándonos en la diferencia entre las dos longitudes de codificación:

$$L(\boldsymbol{X}_k \cup \boldsymbol{X}_l) - L^c(\boldsymbol{X}_k, \boldsymbol{X}_l) \tag{4.1.40}$$

es positivo o negativo y $\boldsymbol{X}_k \cup \boldsymbol{X}_l$ denota la unión de los conjuntos de muestras en $\boldsymbol{X}_k$ y $\boldsymbol{X}_l$. Si es positivo, significa que la longitud de codificación se reduciría si fusionamos los dos grupos en uno solo. Este simple hecho da lugar al siguiente algoritmo de agrupamiento propuesto por [MDH+07b]:

Cabe señalar que este algoritmo es manejable, ya que el número total de comparaciones (por pares) y fusiones es de aproximadamente $O(N^2 \log N)$. Sin embargo, debido a su naturaleza voraz, no existe garantía teórica de que el proceso converja hacia la solución de agrupamiento óptima a nivel global. No obstante, según se señala en [MDH+07b], en la práctica, este algoritmo, aparentemente sencillo, funciona extremadamente bien. Los resultados de la

[19] De hecho, una estimación más precisa de la longitud de codificación es $L(\boldsymbol{X}_k) = (N_k + D)R_\epsilon(\boldsymbol{X}_k)$ donde los bits adicionales se utilizan para codificar la base del subespacio [MDH+07b]. Aquí omitimos esta sobrecarga por simplicidad.

Algoritmo 4.1: Descenso más pronunciado por pares de longitud de codificación

Input: N puntos de datos $\{\boldsymbol{x}_i\}_{i=1}^N$
Output: Un conjunto $\mathcal{C}$ de grupos

procedure PairwiseSteepestDescentOfCodingLength($\{\boldsymbol{x}_i\}_{i=1}^N$)
  `# Inicializar` N `grupos` $\boldsymbol{X}_k$ `con un elemento cada uno`
  $\mathcal{C} \leftarrow \{\{\boldsymbol{x}_i\}\}_{i=1}^N$
  while $|\mathcal{C}| > 1$ **do**
    `# Si ninguna fusión ahorra bits`
    if $\min_{\boldsymbol{X}_k, \boldsymbol{X}_l \in \mathcal{C}} [L(\boldsymbol{X}_k \cup \boldsymbol{X}_l) - L^c(\boldsymbol{X}_k, \boldsymbol{X}_l)] \geq 0$ **then**
      `# Retorno anticipado: devuelve` $\mathcal{C}$ `y finaliza`
      return $\mathcal{C}$
    else
      `# Fusiona los grupos que ahorren más bits`
      $\boldsymbol{X}_{k^*}, \boldsymbol{X}_{l^*} \leftarrow \arg\min_{\boldsymbol{X}_k, \boldsymbol{X}_l \in \mathcal{C}} [L(\boldsymbol{X}_k \cup \boldsymbol{X}_l) - L^c(\boldsymbol{X}_k, \boldsymbol{X}_l)]$
      `# Eliminar grupos no fusionados y volver a agregar el grupo fusionado`
      $\mathcal{C} \leftarrow [\mathcal{C} \setminus \{\boldsymbol{X}_{k^*}, \boldsymbol{X}_{l^*}\}] \cup \{\boldsymbol{X}_{k^*} \cup \boldsymbol{X}_{l^*}\}$
    end if
  end while
  `# Si todas las fusiones generan ahorros, devolver un grupo`
  return $\mathcal{C}$
end procedure

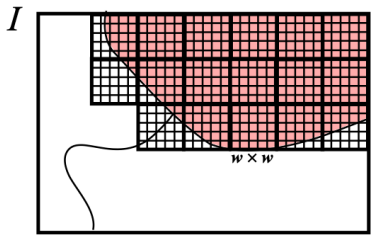


Figura 4.9: Fragmentos de imagen con un tamaño de $w \times w$ pixels.

agrupación representados en la Figura 4.8 se calcularon, de hecho, mediante este algoritmo.

Ejemplo 4.10 (Segmentación de imágenes). La medida de longitud de codificación mencionada anteriormente y el algoritmo de agrupamiento asociado suponen que la distribución de los datos es una mezcla de distribuciones gaussianas (de baja dimensión). Aunque esto pueda parecer un tanto idealizado, la medida y el algoritmo pueden resultar muy útiles e incluso potentes en situaciones en las que el modelo sea (aproximadamente) válido.

Por ejemplo, una imagen natural suele estar compuesta por múltiples regiones con texturas casi homogéneas. Si tomamos muchas ventanas pequeñas de cada región, estas deberían parecerse a muestras extraídas de una distribución gaussiana (de baja dimensión), tal como se ilustra en la Figura 4.9. La Figura 4.10 muestra los resultados de la segmentación de imágenes obtenidos al aplicar el algoritmo de agrupamiento mencionado anteriormente directamente a los fragmentos de imagen. Se pueden encontrar más detalles técnicos sobre la adaptación del algoritmo al problema de la segmentación de imágenes en [MRY+11]. ■

4.2 Transformación mediante maximización de la ganancia de información

Hasta ahora, en este capítulo hemos analizado cómo identificar una distribución con estructuras de baja dimensión mediante el principio de compresión. Como hemos visto en las dos secciones anteriores, la compresión computacional puede llevarse a cabo mediante la operación de eliminación de ruido o por vía de la agrupación en clústeres. La Figura 4.11 ilustra este concepto con nuestro ejemplo favorito.

Es importante señalar que el algoritmo de agrupamiento y las fórmulas de la tasa de codificación desarrollados en la sección anterior partían del supuesto de que la distribución de los datos es *ya* (aproximadamente) una mezcla de gaussianas de baja dimensión. En la práctica, los datos brutos —imágenes, audio, texto— rara vez cumplen esta hipótesis de manera directa: sus estructuras intrínsecas suelen ser *no lineales*. El objetivo de esta sección

Figura 4.10: Resultados de la segmentación basados en el algoritmo de agrupamiento aplicado a los fragmentos de imagen.

es desarrollar un marco para *transformar* los datos de tal manera que sus representaciones adopten la forma de una mezcla de subespacios incoherentes de baja dimensión, momento en el que las herramientas de la sección anterior se vuelven directamente aplicables. Es decir, la sección anterior responde a la pregunta «cómo comprimir y empacar datos que ya son gaussianos», mientras que esta sección responde a la pregunta «cómo transformar los datos para que se conviertan en gaussianos». Como veremos en el Capítulo 5, las redes profundas son precisamente el mecanismo computacional que permite llevar a cabo tales transformaciones y la arquitectura de la red se deriva de la optimización del objetivo de la tasa de codificación desarrollado aquí.

De hecho, el objetivo final de identificar una distribución de datos es utilizarla para facilitar ciertas tareas posteriores, como la segmentación, la clasificación o la generación (de imágenes). Por lo tanto, la forma en que se «representa» la distribución resultante es de vital importancia para determinar cómo se puede recuperar y utilizar de manera eficiente y eficaz la información relacionada con estas tareas posteriores. Esto plantea, naturalmente, una pregunta fundamental: *¿qué hace que una representación sea verdaderamente «buena» para su uso posterior?* A continuación, exploraremos las propiedades esenciales que debe poseer una representación significativa y útil y cómo estas propiedades pueden caracterizarse y perseguirse explícitamente mediante la maximización de la ganancia de información.

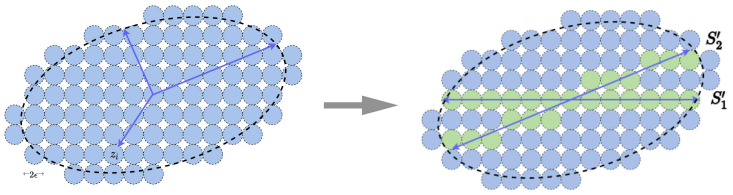


Figura 4.11: Identificación de una distribución de baja dimensión con dos subespacios (izquierda) mediante la eliminación de ruido o la agrupación, partiendo de una distribución gaussiana aleatoria genérica (derecha).

Cómo evaluar la calidad de las representaciones. Se puede considerar un conjunto de datos dado como un conjunto de muestras de un vector aleatorio $\boldsymbol{x}$ con una distribución determinada en un espacio de alta dimensión, por ejemplo, $\mathbb{R}^D$. Por lo general, la distribución de $\boldsymbol{x}$ tiene una dimensión intrínseca mucho menor que el espacio circundante. En términos generales, *aprender una representación* se refiere a aprender una aplicación continua, digamos $f(\cdot)$, que transforma $\boldsymbol{x}$ en lo que se denomina *vector de características* $\boldsymbol{z}$ en otro espacio (normalmente de menor dimensión), digamos $\mathbb{R}^d$, donde $d < D$. Se espera que a través de tal mapeo

$$\boldsymbol{x} \in \mathbb{R}^D \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z} \in \mathbb{R}^d, \tag{4.2.1}$$

las estructuras intrínsecas de baja dimensión de $\boldsymbol{x}$ se identifiquen y se representen mediante $\boldsymbol{z}$ de una manera más compacta y estructurada, con el fin de facilitar tareas posteriores como la clasificación o la generación. La característica $\boldsymbol{z}$ puede considerarse como un código compacto (aprendido) de los datos originales $\boldsymbol{x}$, por lo que la aplicación f también se denomina *codificador*. La pregunta fundamental del aprendizaje de representaciones es

¿Cuál sería una medida rigurosa y eficaz para evaluar la calidad de las representaciones?

Conceptualmente, la calidad de una representación $\boldsymbol{z}$ depende de qué tan bien identifique la información más relevante y suficiente de $\boldsymbol{x}$ para las tareas posteriores y qué tan eficientemente represente dicha información. Durante mucho tiempo, se creía y se defendía que la «suficiencia» o «calidad» de una representación de características aprendida debía definirse en función de una tarea específica. Por ejemplo, $\boldsymbol{z}$ solo tiene que ser suficiente para predecir la etiqueta de clase $\boldsymbol{y}$ en un problema de clasificación. A continuación, comencemos con el problema clásico de la clasificación de imágenes y analizaremos por qué ese concepto de «representación» específica para cada tarea es limitado y debe generalizarse.

4.2.1 Aprendizaje de representaciones a partir de datos etiquetados

Supongamos que $\boldsymbol{x} \in \mathbb{R}^D$ es un vector aleatorio extraído de una mezcla de K distribuciones (componentes) $\mathcal{D} = \{\mathcal{D}_k\}_{k=1}^K$. Dado un conjunto finito de muestras independientes e idénticamente distribuidas $\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}$ del vector aleatorio $\boldsymbol{x}$, *buscamos una buena representación* mediante una aplicación continua $f(\boldsymbol{x}) : \mathbb{R}^D \to \mathbb{R}^d$ que capte las estructuras intrínsecas de $\boldsymbol{x}$ y facilite al máximo la tarea posterior de clasificación.[20] Para facilitar la tarea de aprender la distribución $\mathcal{D}$, en el popular contexto de la clasificación supervisada, se asigna a cada muestra $\boldsymbol{x}_i$ una etiqueta de clase verdadera (o una palabra de código para cada clase), representada normalmente por un vector one-hot $\boldsymbol{y}_i \in \mathbb{R}^K$.

Codificación de la información de clases mediante la entropía cruzada. Amplios estudios han mostrado que, para muchos conjuntos de datos prácticos (por ejemplo, imágenes, audio y lenguajes naturales), el mapeo (de codificación) entre los datos $\boldsymbol{x}$ y su etiqueta de clase $\boldsymbol{y}$ puede modelarse de manera eficaz mediante el entrenamiento de una red profunda,[21] aquí denotada

$$f(\boldsymbol{x}, \theta) : \boldsymbol{x} \mapsto \boldsymbol{y}$$

con parámetros de red $\theta \in \Theta$, donde Θ es el espacio de parámetros. Para que la salida $f(\boldsymbol{x}, \theta)$ coincida bien con la etiqueta $\boldsymbol{y}$, nos interesa minimizar la *pérdida de entropía cruzada* sobre un conjunto de entrenamiento $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^N$:[22]

$$\min_{\theta\in\Theta} \mathbb{E}[\mathrm{CE}(\boldsymbol{y} \parallel f(\boldsymbol{x}, \theta))] \approx -\frac{1}{N}\sum_{i=1}^{N} \langle \boldsymbol{y}_i, \log\left(f(\boldsymbol{x}_i, \theta)\right)\rangle. \tag{4.2.2}$$

Los parámetros óptimos θ de la red suelen obtenerse optimizando el objetivo anterior mediante un método eficiente de descenso por gradientes, en el que los gradientes se calculan por retropropagación, tal y como se describe en la Sección A.2.3 del Apéndice A.

A pesar de su eficacia y su enorme popularidad, este enfoque presenta dos limitaciones importantes: (1) Su único objetivo es predecir las etiquetas $\boldsymbol{y}$, incluso si estas pudieran estar mal etiquetadas. Los estudios empíricos muestran que las redes profundas, utilizadas como una «caja negra», pueden

[20] La clasificación es el ámbito en el que el aprendizaje profundo mostró sus primeros éxitos, lo que desató un interés explosivo por las redes profundas. Aunque nuestro estudio se centra en la clasificación, creemos que las ideas y los principios pueden generalizarse en forma natural a otros contextos, como la regresión.

[21] Por ahora, no nos preocupemos por qué red debemos usar aquí y por qué. El objetivo aquí es considerar cualquier red profunda probada empíricamente. Dejaremos la justificación de las arquitecturas de red para el próximo capítulo.

[22] Aquí interpretamos $\boldsymbol{y}$ como una distribución de probabilidad *one-hot* concreta sobre $[K] = \{1, 2, \ldots, K\}$, y $f(\boldsymbol{x}, \theta) \in \Delta([K]) \subseteq \mathbb{R}^K$ como una distribución de probabilidad sobre el mismo conjunto, que por lo tanto puede compararse mediante la entropía cruzada a pesar de estar representada computacionalmente como vectores en $\mathbb{R}^K$.

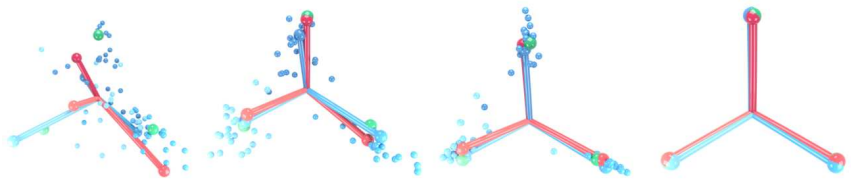


Figura 4.12: Evolución de las salidas de la penúltima capa de una red neuronal VGG13 cuando se entrena con el conjunto de datos CIFAR-10 utilizando 3 clases seleccionadas al azar. Figura tomada de [PHD20].

incluso adaptarse a etiquetas aleatorias [ZBH+17]. (2) Con este tipo de ajuste de datos de extremo a extremo, a pesar de los numerosos esfuerzos empíricos por interpretar las características así aprendidas, no queda claro hasta qué punto las características intermedias aprendidas por la red captan las estructuras intrínsecas de los datos que hacen posible, en primer lugar, una clasificación significativa. Las propiedades geométricas y estadísticas precisas de las características aprendidas también suelen quedar ocultas, lo que da lugar a una falta de interpretabilidad y a la consiguiente ausencia de garantías de rendimiento (por ejemplo, generalización, transferibilidad y robustez, etc.) en el aprendizaje profundo. Por lo tanto, *uno de los objetivos de esta sección es abordar dichas limitaciones reformulando el objetivo para aprender representaciones explícitamente significativas y útiles para los datos* $\boldsymbol{x}$*, sin limitarse a la clasificación.*

Características discriminativas mínimas mediante cuello de botella de información. Un enfoque habitual para interpretar el papel de las redes profundas consiste en considerar que las salidas de las capas intermedias de la red seleccionan ciertas características latentes $\boldsymbol{z} = f(\boldsymbol{x}, \theta) \in \mathbb{R}^d$ de los datos que permiten distinguir entre varias clases. Las representaciones aprendidas $\boldsymbol{z}$ facilitan entonces la posterior tarea de clasificación para predecir la etiqueta de clase $\boldsymbol{y}$ mediante la optimización de un clasificador $g(\boldsymbol{z})$:

$$\boldsymbol{x} \xrightarrow{f(\boldsymbol{x},\theta)} \boldsymbol{z} \xrightarrow{g(\boldsymbol{z})} \boldsymbol{y}. \tag{4.2.3}$$

Sabemos por (4.1.13) que *la información mutua* entre dos variables aleatorias, digamos $\boldsymbol{x}, \boldsymbol{z}$, se define como

$$I(\boldsymbol{x}; \boldsymbol{z}) = H(\boldsymbol{x}) - H(\boldsymbol{x} \mid \boldsymbol{z}), \tag{4.2.4}$$

donde $H(\boldsymbol{x}|\boldsymbol{z})$ es la entropía condicional de $\boldsymbol{x}$ dado $\boldsymbol{z}$. La información mutua también se conoce como *la ganancia de información*: Mide en qué medida se puede reducir la entropía de la variable aleatoria $\boldsymbol{x}$ una vez que se conoce $\boldsymbol{z}$. O, dicho de otro modo, mide cuánta información contiene $\boldsymbol{z}$ sobre $\boldsymbol{x}$. La formulación del *cuello de botella de la información* (IB) [TZ15] postula además

que la función de la red es aprender $\boldsymbol{z}$ como la estadística mínima suficiente para predecir $\boldsymbol{y}$. Formalmente, busca maximizar la información mutua $I(\boldsymbol{z}, \boldsymbol{y})$ entre $\boldsymbol{z}$ y $\boldsymbol{y}$, al tiempo que minimiza la información mutua $I(\boldsymbol{x}, \boldsymbol{z})$ entre $\boldsymbol{x}$ y $\boldsymbol{z}$:

$$\max_{\theta \in \Theta} \text{IB}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}) \doteq I(\boldsymbol{z}; \boldsymbol{y}) - \beta I(\boldsymbol{x}; \boldsymbol{z}) \quad \text{s.t. } \boldsymbol{z} = f(\boldsymbol{x}, \theta), \tag{4.2.5}$$

donde $\beta > 0$.

Si se logran superar algunas limitaciones asociadas a este marco [KTV18], como por ejemplo cómo evaluar con precisión la información mutua con un número finito de muestras provenientes de distribuciones degeneradas, este marco puede resultar útil para explicar ciertos comportamientos de las redes profundas. Por ejemplo, un trabajo reciente [PHD20] demuestra, de hecho, que las representaciones aprendidas mediante la pérdida de entropía cruzada (4.2.2) presentan un fenómeno de *colapso neuronal*. Es decir, las características de cada clase se mapean a un vector unidimensional, mientras que el resto de la información de la clase se suprime, tal y como se ilustra en la Figura 4.12.[23]

Nota 4.4 (Colapso Neuronal). El colapso neuronal se refiere a un fenómeno observado en redes neuronales profundas entrenadas para la clasificación, en el que las representaciones de características aprendidas y los pesos del clasificador muestran un comportamiento altamente simétrico y estructurado durante la fase final del entrenamiento [PHD20; YWZ+22; ZDZ+21]. Concretamente, dentro de cada clase, las características colapsan hacia la media de su clase y entre las clases, estas medias se separan al máximo, formando una configuración equiangular en forma de simplex. El clasificador lineal se alinea con la media de la clase, salvo por un cambio de escala. Además, el clasificador de la última capa converge en elegir aquella clase cuya media de entrenamiento sea la más cercana. El colapso neuronal revela conexiones profundas entre la dinámica de la optimización, la generalización y las estructuras geométricas que surgen en el aprendizaje supervisado.

A partir del ejemplo de clasificación anterior, vemos que la representación así aprendida $\boldsymbol{z}$ de $\boldsymbol{x}$ da lugar a un codificador muy sencillo que, en esencia, asigna a cada clase de datos una sola palabra de código: el vector «one-hot» que representa cada clase. Desde el punto de vista de la compresión con pérdidas, un codificador de este tipo presenta demasiadas pérdidas como para conservar la información en la distribución de los datos. Otra información, como la que resulta útil para tareas tales como la segmentación de imágenes, el completado de imágenes o la generación de imágenes, se pierde en gran medida en un proceso de aprendizaje supervisado de este tipo.

Para remediar esta situación, queremos aprender un esquema de codificación diferente de tal manera que la representación de características resultante $\boldsymbol{z}$ pueda capturar tanta información como sea posible sobre la distribución de los datos, sin limitarse únicamente a la información útil para la clasificación. Más concretamente, queremos que la representación $\boldsymbol{z} = f(\boldsymbol{x}, \theta)$ sea tal que exista

[23] Observaremos este fenómeno en experimentos con datos de imágenes reales más adelante en el Sección 4.3.1.

un decodificador $g(\boldsymbol{z}, \eta)$ de modo que:

$$\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2 \leq \epsilon \quad \text{s.t.} \quad \hat{\boldsymbol{x}} = g(f(\boldsymbol{x})). \tag{4.2.6}$$

Introduciremos formalmente dicho esquema de codificación-decodificación como un «autocodificador» y dicha representación $\boldsymbol{z}$ como una representación «consistente» en el Capítulo 6. Por ahora, el lector puede considerarlo simplemente como una generalización del código de cuantización $\hat{\boldsymbol{x}}$ introducido en la sección anterior para la tasa-distorsión (en el dominio de datos nativos $\boldsymbol{x}$).

En la formulación del cuello de botella de información (4.2.5), no se requiere que la representación de características aprendida $\boldsymbol{z}$ sea consistente. Por lo tanto, como mínimo debería fijarse un objetivo de aprendizaje (consistente) de representaciones (RL):

$$\begin{aligned} &\max_{\theta,\eta} \ \mathrm{RL}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}) \doteq I(\boldsymbol{z}; \boldsymbol{y}) - \beta I(\boldsymbol{x}; \boldsymbol{z}) \\ &\text{s.t.} \quad \boldsymbol{z} = f(\boldsymbol{x}, \theta), \quad \|\boldsymbol{x} - g(\boldsymbol{z}, \eta)\|_2 \leq \epsilon. \end{aligned} \tag{4.2.7}$$

Resolver el problema anterior en el contexto más general es una tarea muy desalentadora. En general, ni siquiera sabemos qué clases de codificadores o decodificadores podrían garantizar la consistencia[24] Además, dado que desconocemos la distribución de la representación $\boldsymbol{z}$, calcular la información mutua $I(\boldsymbol{z}; \boldsymbol{y})$ puede resultar muy complicado, si no imposible.

Antes de estudiar cómo abordar este problema en contextos generales (en los capítulos siguientes), en el resto de este capítulo mostraremos cómo resolverlo, de manera limitada, en el caso particular en el que la distribución de los datos se compone de clases bien diferenciadas (de baja dimensión).

4.2.2 Representaciones discriminantes lineales

Que los datos proporcionados $\boldsymbol{X}$ de una distribución mixta $\mathcal{D}$ puedan clasificarse o agruparse de manera efectiva depende de cuán separables (o discriminantes) sean (o puedan hacerse) las distribuciones componentes $\mathcal{D}_k$. Una hipótesis de trabajo muy extendida es que la distribución de cada clase presenta estructuras intrínsecas de *dimensión* relativamente *baja*. Por lo tanto, podemos suponer que la distribución $\mathcal{D}_k$ de cada clase tiene su soporte en una subvariedad de baja dimensión, digamos $\mathcal{M}_k$ con dimensión $d_k \ll D$, y que la distribución $\mathcal{D}$ de $\boldsymbol{x}$ está definida sobre la mezcla de esas subvariedades, $\mathcal{M} = \cup_{k=1}^{K} \mathcal{M}_k$, en el espacio ambiental de alta dimensión $\mathbb{R}^D$, como se ilustró anteriormente en la Figura 4.1.

No solo necesitamos identificar la distribución de baja dimensionalidad, sino que también queremos representar la distribución de una forma que facilite al máximo las tareas posteriores, como la clasificación, la agrupamiento en clústeres y la generación condicionada (como veremos más adelante). Para ello, necesitamos que nuestras representaciones de características aprendidas tengan las siguientes propiedades:

[24] En la práctica, las personas tienden a aprenderlas, al margen del objetivo mencionado anteriormente, mediante ensayo y error, como veremos con más detalle en el Capítulo 6.

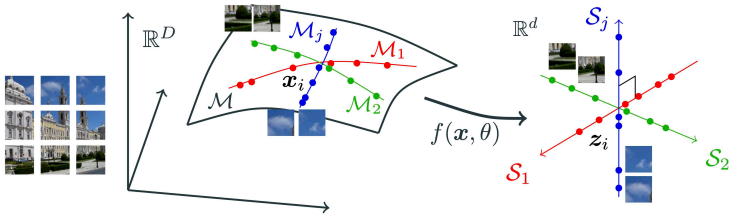


Figura 4.13: La distribución $\mathcal{D}$ de datos de alta dimensión $\boldsymbol{x} \in \mathbb{R}^D$ tiene su soporte en una variedad $\mathcal{M}$ y sus clases se encuentran en subvariedades de baja dimensión $\mathcal{M}_k$. Nuestro objetivo es aprender una aplicación $f(\boldsymbol{x}, \theta)$ parametrizada por θ tal que $\boldsymbol{z}_i = f(\boldsymbol{x}_i, \theta)$ se encuentre en una unión de subespacios con máxima descorrelación $\{\mathcal{S}_k\}$.

1. *Comprimibilidad intraclase:* Las características de las muestras de la misma clase deben estar fuertemente *correlacionadas*, en el sentido de que pertenezcan a un subespacio lineal de baja dimensión.

2. *Discriminación entre clases:* Las características de las muestras de diferentes clases deben estar muy *descorrelacionadas* y pertenecer a subespacios lineales de baja dimensión diferentes e incoherentes.

3. *Representación con máxima diversidad:* La dimensión (o varianza) de las características de cada clase debe ser *lo más grande posible*, siempre y cuando sean incoherentes con respecto a las de las demás clases.

Nos referimos a dicha representación como la *representación discriminativa lineal* (LDR).

Obsérvese que la primera propiedad se ajusta bien al el objetivo del clásico *análisis de componentes principales* (PCA) que hemos expuesto en la Sección 2.1.1. En esencia, esto significa que las características de las muestras que pertenecen a la misma clase[25] deben volverse o hacerse altamente correlacionadas. En otras palabras, la representación de características debe ser *contractiva* para las muestras que son similares.

La segunda propiedad exige que, en el caso de las muestras que pertenecen a clases diferentes o que se presumen que son distintas entre sí, sus características deben hacerse (geométricamente) incoherentes o (estadísticamente) independientes, tal y como se ilustra anteriormente en la Figura 4.1. La Figura 4.13 ilustra cuándo la distribución de los datos es, en realidad, una mezcla de subvariedades de baja dimensión. Mediante la compresión (eliminación de ruido o agrupación), identificamos en primer lugar la distribución real de los datos (izquierda). A continuación, queremos transformar la distribución de tal manera que las subvariedades acaben convirtiéndose en subespacios lineales mutuamente incoherentes o independientes (derecha).

[25] incluidas las instancias augmentadas de las mismas muestras

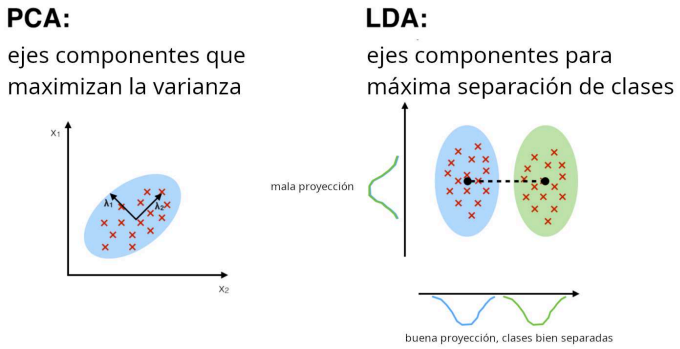


Figura 4.14: Comparación entre el PCA y el LDA. Figuras tomadas de https://sebastianraschka.com/Articles/2014_python_lda.html.

Nota 4.5 (Análisis discriminante lineal). Se puede observar que el segundo criterio se asemeja al del clásico *análisis discriminante lineal* (LDA) [HTF09]. El análisis discriminante lineal (LDA) [HTF09] es una técnica supervisada de reducción de dimensionalidad cuyo objetivo es encontrar una proyección lineal de los datos que maximice la separabilidad de las clases. En concreto, a partir de datos etiquetados, el LDA busca una transformación lineal que proyecte las entradas de alta dimensión en un espacio de menor dimensión en el que las clases estén separadas al máximo. Tengamos en cuenta que el PCA es un método no supervisado que proyecta los datos en direcciones de máxima varianza sin tener en cuenta las etiquetas de clase. Mientras que el PCA se centra exclusivamente en preservar la estructura global de la varianza, el LDA aprovecha explícitamente la información de las etiquetas para mejorar el poder discriminativo; véase la comparación en la Figura 4.14. Pero el LDA solo realiza una proyección lineal de los datos originales. Como veremos, el LDR puede ser más general: busca una transformación, probablemente no lineal, para que la representación resultante sea lineal e incoherente.

Nota 4.6 (Aprendizaje contrastivo). Cabe señalar que el *aprendizaje contrastivo* convencional [HCL06; HFW+19; OLV18] pretende alcanzar objetivos similares en cuanto a la representación de características, tal y como establecen las dos primeras propiedades. En el caso de datos que pertenecen a k clases diferentes, un par elegido al azar $(\boldsymbol{x}_i, \boldsymbol{x}_j)$ tiene una alta probabilidad de pertenecer a clases diferentes si k es grande.[26] Por lo tanto, es deseable que la representación $\boldsymbol{z}_i = f(\boldsymbol{x}_i, \boldsymbol{\theta})$ de una muestra $\boldsymbol{x}_i$ sea altamente incoherente con respecto a las $\boldsymbol{z}_j$ de otras muestras $\boldsymbol{x}_j$, mientras que coherente con la característica de su versión transformada $\tau(\boldsymbol{x}_i)$, denotada como $\boldsymbol{z}(\tau(\boldsymbol{x}_i))$ para τ en cierto conjunto dado de aumentos $\mathcal{T}$. Por lo tanto, se propuso de manera heurística que, para fomentar el poder discriminativo de la representación aprendida, se podría

[26] Por ejemplo, cuando $k \geq 100$, un par aleatorio tiene un 99% de probabilidad de pertenecer a clases diferentes.

intentar minimizar la denominada *pérdida contrastiva*:

$$\min_{\boldsymbol{\theta}} -\log \frac{\exp(\langle \boldsymbol{z}_i, \boldsymbol{z}(\tau(\boldsymbol{x}_i))\rangle)}{\sum_{j\neq i}\exp(\langle \boldsymbol{z}_i, \boldsymbol{z}_j\rangle)}, \tag{4.2.8}$$

que es pequeña siempre que el producto interno $\langle \boldsymbol{z}_i, \boldsymbol{z}(\tau(\boldsymbol{x}_i))\rangle$ sea grande y $\langle \boldsymbol{z}_i, \boldsymbol{z}_j\rangle$ sea pequeño para $i \neq j$. Esta pérdida contrastiva y muchas de sus variantes se han utilizado ampliamente en métodos populares de aprendizaje de características (autosupervisado), como las características de tipo DINO [CMM+21; ODM+24] para datos visuales. Como veremos, basándonos en el marco de codificación con pérdidas estudiado en la sección anterior, podremos llegar a una función objetivo fundamentada que promueva ambas propiedades y, al mismo tiempo, simplifique significativamente prácticas como el aprendizaje de las características de tipo DINO (tal y como lo expondremos con gran detalle en el Capítulo 8).

La tercera propiedad también es importante porque queremos que las características aprendidas revelen todas las causas posibles por las que una clase es diferente de todas las demás. Por ejemplo, para distinguir una «manzana» de una «naranja», no solo nos fijamos en el color, sino también en la forma y en las hojas. Lo ideal sería que la dimensión de cada subespacio $\{\mathcal{S}_k\}$ fuera igual a la de la subvariedad correspondiente $\mathcal{M}_k$. Esta propiedad será importante si queremos que la función $f(\boldsymbol{x}, \theta)$ sea *invertible* para tareas como la generación de imágenes. Por ejemplo, si tomamos diferentes puntos de muestra del subespacio de características de «manzana», deberíamos poder decodificarlos, mediante una aplicación inversa $g(\boldsymbol{z}, \eta)$, para generar diversas imágenes de manzanas. Este par de aplicaciones mutuamente inversas f y g constituye un autocodificador que estudiaremos con gran detalle en el Capítulo 6. Obsérvese que las representaciones de características aprendidas a partir de la minimización de la entropía cruzada (4.2.2) normalmente no presentan esta propiedad, ya que adolecen del denominado fenómeno del *colapso neuronal* comentado anteriormente (véase la Nota 4.4).

En general, aunque las estructuras intrínsecas de cada clase o grupo puedan ser de baja dimensión, no son en absoluto simplemente lineales (o gaussianas) en su representación original $\boldsymbol{x}$, y es necesario convertirlas primero en lineales mediante alguna transformación no lineal.[27] Por lo tanto, en general, utilizamos la transformación no lineal $f(\boldsymbol{x}, \theta)$ para buscar una representación de los datos tal que los subespacios que representan todas las clases sean subespacios lineales máximamente incoherentes. Para ser más precisos, queremos aprender una función $\boldsymbol{z} = f(\boldsymbol{x}, \theta)$ que mapee cada una de las subvariedades $\mathcal{M}_k \subset \mathbb{R}^D$ (Figura 4.13 izquierda) a un subespacio *lineal* $\mathcal{S}_k \subset \mathbb{R}^d$ (Figura 4.13 derecha). En cierta medida, los subespacios múltiples resultantes $\{\mathcal{S}_k\}$ pueden considerarse como *componentes principales generalizadas* discriminantes [VMS16] o, si son ortogonales, *componentes independientes* [HO00a] de las características resultantes $\boldsymbol{z}$ para los datos originales $\boldsymbol{x}$. Como veremos en el siguiente

[27] Analizaremos cómo se puede hacer esto de manera explícita en el Capítulo 6.

Capítulo 5, las redes profundas desempeñan precisamente la función de modelar y llevar a cabo esta transformación no lineal de la distribución de los datos a representaciones discriminantes lineales.

4.2.3 El principio de reducción máxima de la tasa de codificación

Aunque las tres propiedades —*discriminación entre clases*, *compresibilidad intraclase* y *representación de máxima diversidad* — de las representaciones discriminativas lineales (LDR) son todas propiedades muy deseables de la representación aprendida $\boldsymbol{z}$, no son en absoluto fáciles de obtener: ¿Son estas propiedades compatibles entre sí, de modo que podamos esperar alcanzarlas todas a la vez? Si es así, ¿existe algún objetivo *sencillo pero basado en principios* que permita evaluar la calidad de las representaciones resultantes en función de todas estas propiedades? La clave para responder a estas preguntas es encontrar una «medida de compacidad» o «ganancia de información» fundamentada para la distribución de una variable aleatoria $\boldsymbol{z}$ o a partir de un número finito de muestras $\{\boldsymbol{z}_i\}_{i=1}^N$ de la misma. Dicha medida debería caracterizar de manera directa y precisa las propiedades geométricas o estadísticas intrínsecas de la distribución, en términos de su dimensión intrínseca o volumen. A diferencia de la entropía cruzada (4.2.2) o el cuello de botella de información (4.2.5), dicha medida no debe depender exclusivamente de las etiquetas de clase, de modo que pueda funcionar en entornos más generales, como los supervisados, autosupervisados, semisupervisados y no supervisados (como veremos en secciones posteriores).

Sin pérdida de generalidad, supongamos que la distribución $\mathcal{D}$ del vector aleatorio $\boldsymbol{x}$ se define sobre una mezcla de distribuciones, es decir, $\mathcal{D} = \cup_{k=1}^K \mathcal{D}_k$, donde cada $\mathcal{D}_k \subset \mathbb{R}^D$ tiene una dimensión intrínseca baja en el espacio ambiente de alta dimensión $\mathbb{R}^D$. Sea $\boldsymbol{X}_k \in \mathbb{R}^{D\times N_k}$ la matriz de datos cuyas columnas son muestras extraídas de la distribución $\mathcal{D}_k$, donde N_k indica el número de muestras para cada $k = 1, \ldots, K$. Entonces, utilizamos $\boldsymbol{X} = [\boldsymbol{X}_1, \ldots, \boldsymbol{X}_K] \in \mathbb{R}^{D\times N}$ para denotar todas las muestras, donde $N = \sum_{k=1}^K N_k$. Recordemos que también utilizamos $\boldsymbol{x}_i$ para denotar la i-ésima muestra de $\boldsymbol{X}$, es decir, $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$. Bajo una aplicación de codificación:

$$\boldsymbol{x} \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z}, \tag{4.2.9}$$

las muestras de entrada se mapean a $\boldsymbol{z}_i = f(\boldsymbol{x}_i)$ para cada $i = 1, \ldots, N$. Abusando un poco de la notación, también escribimos $\boldsymbol{Z}_k = f(\boldsymbol{X}_k)$ y $\boldsymbol{Z} = f(\boldsymbol{X})$. Por lo tanto, tenemos $\boldsymbol{Z} = [\boldsymbol{Z}_1, \ldots, \boldsymbol{Z}_K]$ y $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots \boldsymbol{z}_N]$.

Por un lado, para que las características aprendidas sean discriminativas, es preferible que las características de las diferentes clases o clúster sean *lo más incoherentes posible* entre sí. Por lo tanto, en conjunto deberían abarcar un espacio del mayor volumen (o dimensión) posible y la tasa de codificación de todo el conjunto $\boldsymbol{Z}$ debería ser lo más alta posible. Por otro lado, las características aprendidas de una misma clase o clúster deberían estar muy correlacionadas

y ser coherentes. Por lo tanto, cada clase o grupo solo debería abarcar un espacio (o subespacio) de un volumen muy reducido y la tasa de codificación debe ser lo más baja posible. A continuación, explicaremos cómo medir la tasa de codificación de las características aprendidas.

La tasa de codificación de las características. Cabe destacar que una dificultad práctica a la hora de evaluar la tasa de codificación es que la distribución subyacente de las representaciones de las características $\boldsymbol{Z}$ suele ser desconocida. Para solucionar esto, podemos aproximar las características $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$ como muestras extraídas de una distribución gaussiana multivariante. Bajo esta hipótesis, tal y como se analiza en la Sección 4.1.3, la compacidad de las características $\boldsymbol{Z}$ *en su conjunto* puede medirse en términos de la longitud media de codificación por muestra, denominada *tasa de codificación*, sujeta a un nivel de precisión $\epsilon > 0$ (véase (4.1.30)) definida de la siguiente manera:

$$R_\epsilon(\boldsymbol{Z}) = \frac{1}{2}\operatorname{logdet}\left(\boldsymbol{I} + \frac{d}{N\epsilon^2}\boldsymbol{Z}\boldsymbol{Z}^\top\right). \tag{4.2.10}$$

Por otro lado, esperamos que una transformación no lineal $f(\boldsymbol{x})$ asigne a cada subvariedad específica de clase $\mathcal{M}_k \subset \mathbb{R}^D$ un subespacio lineal maximamente incoherente $\mathcal{S}_k \subset \mathbb{R}^D$, de manera que las características aprendidas $\boldsymbol{Z}$ se encuentren en una unión de subespacios de baja dimensión. Esta estructura permite una evaluación más precisa de la tasa de codificación mediante el análisis separado de cada subespacio. Recordemos que las columnas de $\boldsymbol{Z}_k$ representan las características de las muestras en $\boldsymbol{X}_k$ para cada $k = 1, \ldots, K$. La tasa de codificación de las características en $\boldsymbol{Z}_k$ se puede calcular de la siguiente manera:

$$R_\epsilon(\boldsymbol{Z}_k) = \frac{N_k}{2N}\log\det\left(\boldsymbol{I} + \frac{d}{N_k\epsilon^2}\boldsymbol{Z}_k\boldsymbol{Z}_k^\top\right) \tag{4.2.11}$$

Entonces, la suma de las tasas de codificación promedio de las características de cada clase es

$$R_\epsilon^c(\boldsymbol{Z}) \doteq \sum_{k=1}^{K} R_\epsilon(\boldsymbol{Z}_k), \tag{4.2.12}$$

Por lo tanto, una buena representación $\boldsymbol{Z}$ de $\boldsymbol{X}$ es aquella que logra una gran diferencia entre la tasa de codificación del conjunto y la de todas las clases:

$$\Delta R_\epsilon(\boldsymbol{Z}) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}). \tag{4.2.13}$$

Obsérvese que, tal y como hemos comentado anteriormente en este capítulo, esta diferencia puede interpretarse como la cantidad de «información ganada» al identificar los clústeres de baja dimensión correctos $\boldsymbol{Z}_k$ dentro del conjunto global $\boldsymbol{Z}$.

Si elegimos que nuestra función de mapeo de características $f(\cdot)$ sea una red neuronal profunda $f(\cdot, \theta)$ con parámetros de red θ, el proceso general de

representación de características y la consiguiente reducción de la tasa se pueden ilustrar mediante el siguiente diagrama:

$$\boldsymbol{X} \xrightarrow{f(\boldsymbol{x},\theta)} \boldsymbol{Z} \xrightarrow{\epsilon} \Delta R_\epsilon(\boldsymbol{Z}). \tag{4.2.14}$$

Recordemos que ΔR_ϵ es *monótono* en la escala de las características $\boldsymbol{Z}$. Para garantizar una comparación justa entre las diferentes representaciones, es esencial *normalizar la escala* de las características aprendidas. Esto se puede lograr ya sea imponiendo que la norma de Frobenius de cada clase $\boldsymbol{Z}_k$ sea proporcional al número de características en $\boldsymbol{Z}_k \in \mathbb{R}^{d\times N_k}$, es decir, $\|\boldsymbol{Z}_k\|_F^2 = N_k$, o bien normalizando cada característica para que se encuentre en la esfera unitaria, es decir, $\boldsymbol{z}_i \in \mathbb{S}^{d-1}$, donde N_k denota el número de muestras en la clase k-ésima. Esta formulación ofrece una justificación natural de la necesidad de la «normalización por lotes» en la práctica del entrenamiento de redes neuronales profundas [IS15].

Una vez que las representaciones son comparables, el objetivo pasa a ser el de aprender un conjunto de características $\boldsymbol{Z} = f(\boldsymbol{X},\theta)$ tal que maximicen la reducción entre la tasa de codificación de todas las características y la de la suma de características con respecto a sus clases:

$$\begin{aligned} &\max_\theta \Delta R_\epsilon\big(\boldsymbol{Z}\big) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}), \\ &\text{s.t.} \quad \boldsymbol{Z} = f(\boldsymbol{X},\theta),\ \|\boldsymbol{Z}_k\|_F^2 = N_k,\ k = 1,\dots,K. \end{aligned} \tag{4.2.15}$$

Denominamos esto el principio de la *reducción máxima de la tasa de codificación* (MCR^2), como una encarnación de la famosa cita de Aristóteles:

"*El todo es más que la suma de sus partes.*"

Para aprender la mejor representación, se requiere que *la totalidad sea superior en la máxima medida a la suma de sus partes*. En gran medida, la cantidad ΔR puede considerarse como una realización computable del término de información mutua $I(\boldsymbol{z};\boldsymbol{y})$ introducido en el objetivo (4.2.7).

Ejemplo 4.11. Para aclarar mejor estas ideas, analicemos el ejemplo que se muestra en la Figura 4.15. Desde el punto de vista de la compresión, la representación de la derecha es *la más compacta* en el sentido de que la diferencia entre la tasa de codificación cuando todas las características se codifican como una sola gaussiana (azul) y la que se obtiene cuando las características se agrupan adecuadamente y se codifican como dos subespacios separados (verde) es máxima.[28] ■

Tengamos presente que el principio MCR^2 mencionado anteriormente está diseñado para problemas de aprendizaje supervisado, en los que se conocen las pertenencias a grupos (o etiquetas de clase). Existen diversas formas de aplicar este principio, o generalizaciones adecuadas, de manera que se eluda este requisito de las etiquetas de clase; analizaremos estas opciones más adelante en

[28] Intuitivamente, el cociente entre el «volumen» de todo el espacio abarcado por todas las características y el que realmente ocupan las características es máximo.

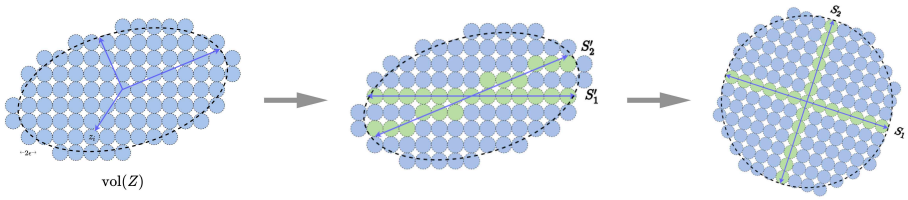


Figura 4.15: Tras identificar la distribución de los datos en baja dimensión, deseamos dar un paso más en la transformación de la distribución de los datos en una representación estructurada más informativa: R es el número de bolas de radio ϵ que cubren todo el espacio y R^c es la suma de los números correspondientes a todos los subespacios (las bolas verdes). ΔR es la diferencia entre ambas (el número de bolas azules).

este capítulo y en el siguiente Capítulo 5. Para tener un primer acercamiento, extendemos el principio a problemas de aprendizaje no supervisado mediante la introducción de una matriz de pertenencia, que codifica la asignación (potencialmente suave) de cada punto de datos a grupos o clústeres latentes. Concretamente, sea $\boldsymbol{\Pi} = \{\boldsymbol{\Pi}_k\}_{k=1}^K \subset \mathbb{R}^{N\times N}$ un conjunto de matrices diagonales cuyos elementos diagonales codifican la pertenencia de las N muestras a K clases. Es decir, $\boldsymbol{\Pi}$ se encuentra en un simplex $\Omega \doteq \{\boldsymbol{\Pi} : \boldsymbol{\Pi}_k \geq \mathbf{0} : \sum_{k=1}^K \boldsymbol{\Pi}_k = \boldsymbol{I}_N\}$. Entonces, podemos definir la tasa de codificación promedio con respecto a la partición $\boldsymbol{\Pi}$ como

$$R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \doteq \sum_{k=1}^K \frac{\operatorname{tr}(\boldsymbol{\Pi}_k)}{2N} \log\det\left(\boldsymbol{I} + \frac{d}{\operatorname{tr}(\boldsymbol{\Pi}_k)\epsilon^2}\boldsymbol{Z}\boldsymbol{\Pi}_k\boldsymbol{Z}^\top\right). \tag{4.2.16}$$

Dado $\boldsymbol{Z}$, $R_\epsilon^c(\boldsymbol{Z}|\boldsymbol{\Pi})$ es una función cóncava de $\boldsymbol{\Pi}$. En consecuencia, el principio MCR^2 para problemas de aprendizaje no supervisado se formula de la siguiente manera:

$$\begin{aligned} &\max_{\boldsymbol{\Pi},\theta} \Delta R_\epsilon\big(\boldsymbol{Z} \mid \boldsymbol{\Pi}\big) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \\ &\text{s.t.} \quad \boldsymbol{Z} = f(\boldsymbol{X},\theta),\ \|\boldsymbol{Z}\boldsymbol{\Pi}_k\|_F^2 = N_k,\ k = 1,\ldots,K,\ \boldsymbol{\Pi} \in \Omega. \end{aligned} \tag{4.2.17}$$

En comparación con (4.2.15), esta formulación permite la optimización conjunta tanto de las pertenencias a grupos como de los parámetros de la red. En particular, cuando $\boldsymbol{\Pi}$ se establece como una matriz de pertenencia a grupos que asigna N puntos de datos a K grupos, el problema (4.2.17) puede recuperar el problema (4.2.15).

Nota 4.7 (Cálculo de las tasas de codificación). Es un resultado clásico que calcular el logaritmo del determinante de una matriz $n \times n$ requiere un tiempo de orden $\mathcal{O}(n^3)$. Por lo tanto, calcular las tasas de codificación ΔR_ϵ o R_ϵ (etc.) parece una tarea inmanejable a la escala del análisis de datos moderno. Hay varias formas de evadirlo, que utilizaremos sin mayor explicación en los experimentos que sirven de base al resto del libro.

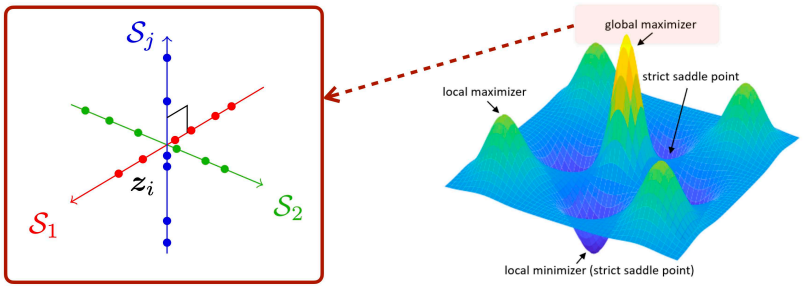


Figura 4.16: **Paisaje de optimización local:** Según el Teorema 4.2, el máximo global del objetivo de reducción de la tasa corresponde a una solución con subespacios mutuamente incoherentes.

- *Utilizar la estructura de la tasa de codificación.* Supongamos que $\boldsymbol{Z}$ es un elemento de $\mathbb{R}^{d\times n}$. Entonces se cumple que

$$\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top)=\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}^\top\boldsymbol{Z}) \tag{4.2.18}$$

  por lo que calcular el producto matricial y las tasas de codificación requiere un tiempo del orden de $\min\{d^2n+d^3, dn^2+n^3\}$.

- *Utilizar la simetría de la matriz de Gram.* Obsérvese que $\boldsymbol{Z}\boldsymbol{Z}^\top$ (y, de manera similar, $\boldsymbol{Z}^\top\boldsymbol{Z}$) es una matriz simétrica. Por lo tanto, resulta relativamente sencillo (aunque sigue siendo de tiempo cúbico) calcular sus valores propios $\lambda_i(\boldsymbol{Z}\boldsymbol{Z}^\top)$. Entonces, el logaritmo del determinante se puede expresar de la siguiente manera:

$$\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top)=\sum_{i=1}^{d}\log(1+\alpha\lambda_i(\boldsymbol{Z}\boldsymbol{Z}^\top)), \tag{4.2.19}$$

  (y análogamente para $\boldsymbol{Z}^\top\boldsymbol{Z}$). Sin embargo, este método se vuelve innecesario ante el siguiente, que aprovecha mejor la estructura de los datos de entrada y ofrece mejores resultados.

- *Utilizar la definitud positiva de los términos que intervienen en el determinante logarítmico.* Dado que $\boldsymbol{Z}\boldsymbol{Z}^\top$ (y, de manera análoga, $\boldsymbol{Z}^\top\boldsymbol{Z}$) es simétrico y semidefinido positivo, la cantidad $\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top$ es simétrica y definida positiva. Esto significa que podemos calcular la descomposición de Cholesky $\boldsymbol{L}\boldsymbol{L}^\top=\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top$, donde $\boldsymbol{L}\in\mathbb{R}^{d\times d}$ es una matriz triangular inferior con entradas diagonales positivas. Entonces

$$\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top)=2\sum_{i=1}^{d}\log(L_{ii}). \tag{4.2.20}$$

  En la práctica, este método suele ser el más económico, especialmente cuando también se decide estratégicamente si calcular $\boldsymbol{Z}\boldsymbol{Z}^\top$ o $\boldsymbol{Z}^\top\boldsymbol{Z}$, aunque su complejidad temporal sigue siendo cúbica.

4.2.4 Propiedades de optimización en la reducción de la tasa de codificación

En este apartado, estudiamos las propiedades de optimización de la función MCR^2 mediante el análisis de sus soluciones óptimas y la estructura de su paisaje de optimización. Para sortear la dificultad técnica que plantean las redes neuronales, consideramos una versión simplificada del problema (4.2.15) de la siguiente manera:

$$\max_{\boldsymbol{Z}} \; R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}) \qquad \text{s.t.} \quad \|\boldsymbol{Z}_k\|_F^2 = N_k, \; k = 1, \ldots, K. \tag{4.2.21}$$

En teoría, el principio MCR^2 (4.2.21) se caracteriza por una gran generalizabilidad y puede aplicarse a representaciones $\boldsymbol{Z}$ de *cualquier* distribución, siempre y cuando las tasas R_ϵ y R_ϵ^c de dichas distribuciones puedan evaluarse con precisión y eficiencia. La representación óptima $\boldsymbol{Z}^*$ debería presentar algunas propiedades geométricas y estadísticas interesantes. Aquí revelamos algunas propiedades útiles de la representación óptima en el caso especial de los subespacios, que tienen muchas aplicaciones importantes en el aprendizaje automático. Cuando la representación deseada para $\boldsymbol{Z}$ consiste en múltiples subespacios, las tasas R_ϵ y R_ϵ^c en (4.2.21) vienen dadas por (4.2.10) y (4.2.12), respectivamente. En el máximo de reducción de tasa, MCR^2 alcanza sus representaciones óptimas, denotadas $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*, \ldots, \boldsymbol{Z}_K^*]$ con $\operatorname{rank}(\boldsymbol{Z}_k^*) \leq d_k$. Se puede demostrar que $\boldsymbol{Z}^*$ tiene las siguientes propiedades deseadas (véase [YCY+20] para un enunciado formal y demostraciones detalladas).

Teorema 4.2 (Caracterización de las soluciones globalmente óptimas). *Supongamos que $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*, \ldots, \boldsymbol{Z}_K^*]$ es una solución óptima global del Problema (4.2.21). Se cumplen las siguientes afirmaciones:*

- Discriminación entre clases*: Siempre que el espacio envolvente sea suficientemente grande ($d \geq \sum_{k=1}^K d_k$), los subespacios son todos ortogonales entre sí,* es decir, *$(\boldsymbol{Z}_k^*)^\top \boldsymbol{Z}_l^* = \boldsymbol{0}$ para $k \neq l$.*
- Representación de máxima diversidad*: Siempre que la precisión de codificación sea suficientemente alta, es decir, $\epsilon^4 < c \cdot \min_k \left\{ \frac{N_k}{N} \frac{d^2}{d_k^2} \right\}$, donde $c > 0$ es una constante. Cada subespacio alcanza su dimensión máxima, es decir, $\operatorname{rank}(\boldsymbol{Z}_k^*) = d_k$. Además, los $d_k - 1$ valores singulares más grandes de $\boldsymbol{Z}_k^*$ son iguales.*

Este teorema indica que el principio MCR^2 favorece el *embedding* de los datos en múltiples subespacios independientes (como se ilustra en Figura 4.16), con características distribuidas *isotrópicamente* en cada subespacio (salvo, quizás, en una dimensión). Cabe destacar que este teorema también confirma que las características aprendidas por el principio MCR^2 presentan las propiedades discriminativas de baja dimensión deseadas que se analizaron en Sección 4.2.2. Además, de entre todas esas representaciones discriminativas, prefiere aquella de mayor dimensión en el espacio ambiente. Esto difiere sustancialmente del objetivo del cuello de botella de información (4.2.5).

MCR^2 regularizado. El teorema anterior caracteriza las propiedades de los óptimos globales de los objetivos de reducción de tasa. ¿Qué ocurre con otros óptimos, como los locales? Debido a las restricciones de la norma de Frobenius, resulta difícil analizar el problema (4.2.21) desde una perspectiva de la teoría de la optimización. Es por eso que consideramos la formulación lagrangiana de (4.2.21). Esto puede considerarse como una relajación estricta o incluso un problema equivalente al de (4.2.21), cuyas soluciones óptimas coinciden bajo ajustes específicos del parámetro de regularización; véase [WLP+24, Proposition 3.2]. En concreto, la formulación que estudiamos, a la que en adelante nos referiremos como el *problema MCR^2 regularizado*, es la siguiente:

$$\max_{\boldsymbol{Z}} \; R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}) - \frac{\lambda}{2}\|\boldsymbol{Z}\|_F^2, \tag{4.2.22}$$

donde $\lambda > 0$ es el parámetro de regularización. Aunque el programa (4.2.22) es extremadamente no cóncavo e implica inversas de matrices en el cálculo de su gradiente, no obstante podemos caracterizar explícitamente sus óptimos locales y globales de la siguiente manera (véase [WLP+24, Theorem 3.1] para una demostración detallada).

Teorema 4.3 (Óptimos locales y globales). *Sea N_k el número de muestras de entrenamiento en la k-ésima clase para cada $k \in \{1,\dots,K\}$, $N_{\max} \doteq \max\{N_1,\dots,N_K\}$, $\alpha = d/(N\epsilon^2)$, y $\alpha_k = d/(N_k\epsilon^2)$ con $k \in \{1,\dots,K\}$. Dada una precisión de codificación $\epsilon > 0$, si el parámetro de regularización satisface*

$$\lambda \in \left(0, \frac{d(\sqrt{N/N_{\max}} - 1)}{N(\sqrt{N/N_{\max}} + 1)\epsilon^2}\right], \tag{4.2.23}$$

en consecuencia se cumplen las siguientes afirmaciones:
*(i) (***Maximizadores locales***) $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*,\dots,\boldsymbol{Z}_K^*]$ es un maximizador local del problema (4.2.22) si y solo si el k-ésimo bloque admite la siguiente descomposición*

$$\boldsymbol{Z}_k^* = \left(\frac{\eta_k + \sqrt{\eta_k^2 - 4\lambda^2 N/N_k}}{2\lambda\alpha_k}\right)^{1/2} \boldsymbol{U}_k \boldsymbol{V}_k^\top, \tag{4.2.24}$$

donde (a) $r_k = \operatorname{rank}(\boldsymbol{Z}_k^*)$ satisface $r_k \in [0, \min\{N_k, d\})$ y $\sum_{k=1}^K r_k \le \min\{N, d\}$, (b) $\boldsymbol{U}_k \in \mathcal{O}^{d\times r_k}$ cumple que $\boldsymbol{U}_k^\top \boldsymbol{U}_l = \boldsymbol{0}$ para todo $k \neq l$, $\boldsymbol{V}_k \in \mathcal{O}^{N_k \times r_k}$, y (c) $\eta_k := (\alpha_k - \alpha) - \lambda(N/N_k + 1)$ para cada $k \in \{1,\dots,K\}$.
*(ii) (***Maximizadores globales***) $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*,\dots,\boldsymbol{Z}_K^*]$ es un maximizador global del problema (4.2.22) si y solo si (a) cumple todas las condiciones anteriores y $\sum_{k=1}^K r_k = \min\{N, d\}$, y (b) para todo $k \neq l \in [K]$ que satisfaga $N_k < N_l$ y $r_l > 0$, tenemos que $r_k = \min\{N_k, d\}$.*

Este teorema caracteriza explícitamente los óptimos locales y globales del problema (4.2.22). Intuitivamente, esto demuestra que las características representadas por cada maximizador local del problema (4.2.22) son de baja

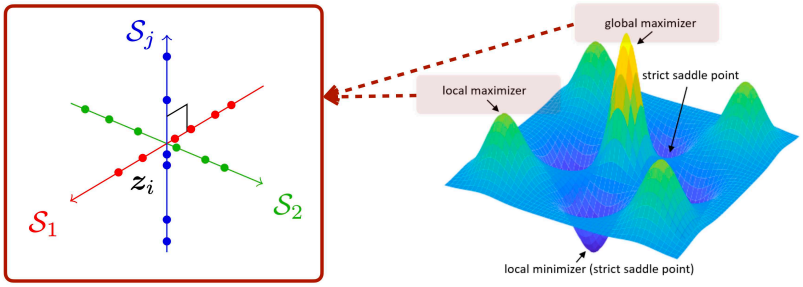


Figura 4.17: **Paisaje de optimización global:** Según [LSJ+16; SQW15], los Teoremas 4.3 y 4.4, tanto los máximos globales como los locales del objetivo de reducción de la tasa (regularizado) corresponden a una solución con subespacios mutuamente incoherentes. Todos los demás puntos críticos son puntos de silla estrictos.

dimensión y discriminativas. Aunque hemos caracterizado las soluciones óptimas locales y globales en Teorema 4.3, sigue sin saberse si estas soluciones pueden calcularse de manera eficiente utilizando el algoritmo de descenso por gradiente (GD) para resolver el problema (4.2.22), ya que el GD puede quedarse atascado en otros puntos críticos, como un punto de silla. Afortunadamente, [LSJ+16; SQW15] demostraron que si una función es dos veces continuamente diferenciable y satisface la *propiedad de silla estricta*, es decir, cada punto crítico ya es, o un maximizador local o bien un punto de silla estricto[29], GD converge casi seguro a un maximizador local con inicialización aleatoria. Investigamos el paisaje de optimización global del problema (4.2.22) caracterizando todos sus puntos críticos de la siguiente manera (véase [WLP+24, Teorema 3.3] para obtener demostraciones detalladas).

Teorema 4.4 (**Paisaje benigno de optimización global**). *Dada una precisión de codificación $\epsilon > 0$, si el parámetro de regularización satisface (4.2.23), se cumple que cualquier punto crítico $\boldsymbol{Z}$ del problema (4.2.22) es o bien un maximizador local o bien un punto de silla estricto.*

En conjunto, los dos teoremas anteriores demuestran que las características aprendidas asociadas a cada maximizador local del objetivo de reducción de tasa —y no solo a los maximizadores globales— se estructuran como subespacios incoherentes de baja dimensión. Además, el objetivo de reducción de la tasa (regularizado) (4.2.15) presenta un paisaje muy benigno, con solo maximizadores locales y puntos de silla estrictos como puntos críticos, tal y como se ilustra en Figura 4.17. Según [LSJ+16; SQW15], los Teoremas 4.3 y 4.4 implican que las representaciones de baja dimensión y discriminativas (LDR) pueden encontrarse

[29] Decimos que un punto crítico es un punto de silla estricto del problema (4.2.22) si tiene una dirección con curvatura estrictamente positiva [SQW15]. Esto incluye tanto los puntos de silla clásicos con curvatura estrictamente positiva como los minimizadores locales estrictos.

de manera eficiente aplicando el descenso por gradiente (estocástico) al objetivo de reducción de tasa (4.2.15) a partir de una inicialización aleatoria. Estos resultados también explican indirectamente por qué en la Sección 4.3.1, si la red elegida es lo suficientemente expresiva y está bien entrenada, la representación resultante suele ofrecer una representación lineal incoherente que probablemente corresponda a la solución óptima global. Se remite a los lectores interesados a [WLP+24] para consultar las demostraciones.

4.3 Aprendizaje de representaciones para datos de imagen

4.3.1 Aprendizaje supervisado de representaciones

A continuación, mostramos cómo el objetivo MCR^2 (4.2.15) ayuda a aprender mejores representaciones que la entropía cruzada (CE) (4.2.2) para la clasificación de imágenes. Aquí, para simplificar,[30] adoptamos una popular arquitectura de redes neuronales, la ResNet-18 [HZR+16b], para modelar el mapeo de características $\boldsymbol{z} = f(\boldsymbol{x}, \theta)$. Optimizamos los parámetros de la red neuronal θ mediante el algoritmo de retropropagación (detallado en la Apéndice A.2.3) para maximizar la reducción de la tasa de codificación. Se pueden encontrar más detalles sobre la implementación de este experimento en [CYY+22].

La Figura 4.18 ilustra cómo evolucionan las dos tasas y su diferencia (tanto para los datos de entrenamiento como para los de prueba) a lo largo de las épocas de entrenamiento: tras una fase inicial, R_ϵ aumenta gradualmente mientras que R_ϵ^c disminuye, lo que indica que las características $\boldsymbol{Z}$ se están expandiendo en su conjunto, mientras que cada clase $\boldsymbol{Z}_k$ se está comprimiendo.

En primer lugar, evaluamos el rendimiento con el conjunto de datos de clasificación de imágenes CIFAR-10 [KH+09] y los resultados se muestran en la tabla de la Figura 4.19. Como podemos ver, el objetivo MCR^2 alcanza un rendimiento comparable al de la pérdida de entropía cruzada (CE). Además, el objetivo MCR^2 es más robusto ante la corrupción en las etiquetas de clase, ya que aprende una representación conjunta (un subespacio) para cada clase, en lugar de intentar ajustarse a las etiquetas de las muestras individuales.

La Figura 4.20 muestra la distribución de los valores singulares de las características aprendidas $\boldsymbol{Z}_k$ por clase. A partir de la distribución de los valores singulares que se muestra en la Figura 4.20a, vemos que las características $\boldsymbol{z}$ aprendidas con MCR^2 para cada clase se distribuyen en un subespacio de dimensión alrededor de 10-12, mientras que las características de entropía cruzada (véase la Figura 4.20b) son de una dimensión mucho menor.[31]

[30] Estudiaremos y justificaremos rigurosamente cómo diseñar o derivar una arquitectura profunda adecuada a partir del objetivo MCR^2 y para este en el Capítulo 5.

[31] Si se continúa entrenando con más iteraciones, las características de CE podrían incluso colapsar en una sola dimensión, lo que se conoce como el fenómeno del colapso neuronal , ya comentado en la Nota 4.4.

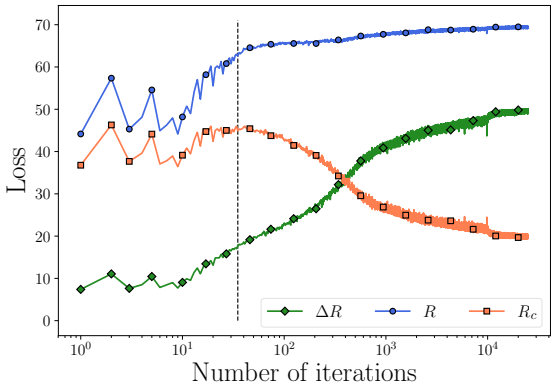


Figura 4.18: Evolución de R_ϵ, R_ϵ^c y ΔR_ϵ durante el proceso de entrenamiento.

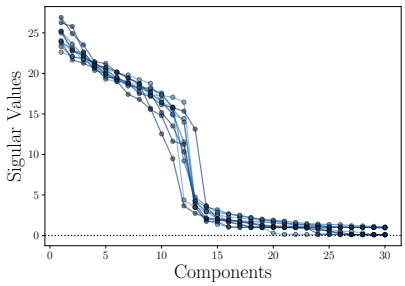
	Ratio=0.0	Ratio=0.1	Ratio=0.2	Ratio=0.3	Ratio=0.4	Ratio=0.5
CE Training	0.939	0.909	0.861	0.791	0.724	0.603
MCR^2 Training	**0.940**	**0.911**	**0.897**	**0.881**	**0.866**	**0.843**

Figura 4.19: Comparación de la exactitud de clasificación entre el entrenamiento con la pérdida de entropía cruzada (CE) o el objetivo MCR^2, con diferentes proporciones de etiquetas de clase corrompidas aleatoriamente.

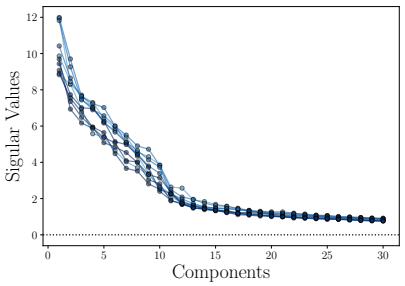
La Figura 4.21 muestra las similitudes coseno entre las características aprendidas, ordenadas por clase. También comparamos las similitudes de las características aprendidas utilizando la entropía cruzada (CE) (4.2.2) y el objetivo MCR^2 (4.2.15). En los gráficos se puede ver claramente que las características aprendidas por el objetivo MCR^2 para las diferentes clases son mucho más incoherentes (o independientes) que las características aprendidas utilizando la pérdida de entropía cruzada. Dentro de cada clase, las características también son menos coherentes y, por lo tanto, más diversas, tal y como indica la distribución de sus valores singulares que se muestra en la Figura 4.20a.

Visualizamos las imágenes que se proyectan a lo largo de diferentes vectores singulares en el subespacio de características de cada clase, a las que denominamos imágenes «principales». La Figura 4.22 muestra estas imágenes principales a lo largo de cada uno de los 10 vectores singulares principales. Como podemos ver, las imágenes correspondientes a cada vector singular son muy similares en cuanto a su aspecto y estructura, mientras que las imágenes correspondientes a diferentes vectores singulares son muy distintas entre sí.

A pesar de que el objetivo MCR^2 parece permitirnos aprender mejores representaciones, hay una aparente falta de justificación de las arquitecturas de red utilizadas en los experimentos anteriores. Aún no está claro por qué la red adoptada aquí (la ResNet-18) es adecuada para representar la función $f(\boldsymbol{x}, \theta)$ y mucho menos para interpretar los operadores de capa y los parámetros

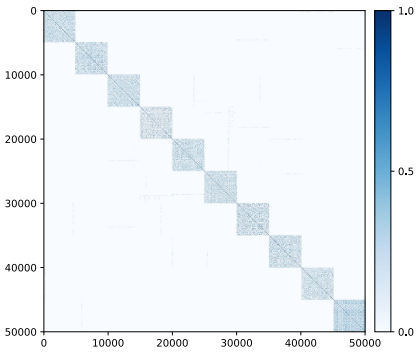


(a) Vectores de componentes principales (PCA) de las características de clases individuales, aprendidas por MCR^2.



(b) PCA de las características de las clases individuales aprendidas mediante la entropía cruzada.

Figura 4.20: Comparación de las componentes principales de las características aprendidas con MCR^2 frente a las obtenidas mediante entropía cruzada.



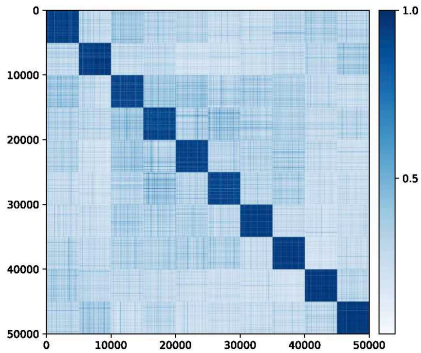


Figura 4.21: Similitud de coseno entre las características aprendidas usando el objetivo MCR^2 (**izquierda**) y la pérdida CE (**derecha**).

θ aprendidos en su interior. En el Capítulo 5, *mostraremos cómo derivar arquitecturas y componentes de redes en forma totalmente "transparente" o de "caja blanca" a partir del objetivo deseado (por ejemplo, la reducción de la tasa).*

4.3.2 Aprendizaje no supervisado de representaciones

Como hemos visto en la sección anterior, la información sobre las clases facilita en cierta medida el aprendizaje de una representación estructurada de la distribución de datos (mixta). Sin embargo, a menudo no se dispone de dicha información, al menos no a gran escala.

A continuación, presentamos cómo extender el principio MCR^2 al entorno no supervisado. A diferencia del aprendizaje supervisado, en el que se conocen

(a) Bird (b) Ship

Figura 4.22: Las 10 mejores imágenes «principales» para las clases «Pájaro» y «Barco» en el conjunto de datos de imágenes CIFAR-10.

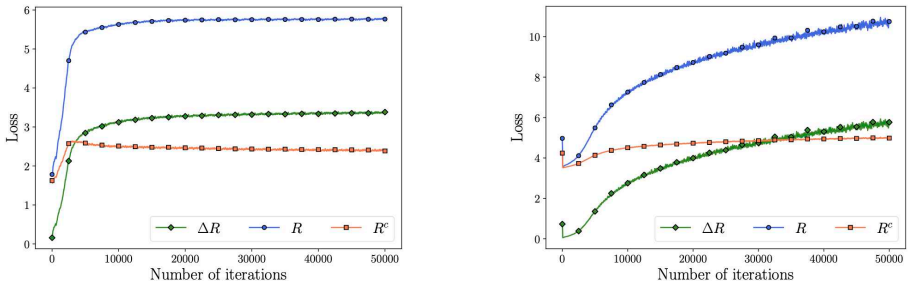
las etiquetas de clase, en el aprendizaje no supervisado se desconoce a qué grupo pertenecen las muestras de datos. En este caso, podemos aplicar cierta clase de aumentos o transformaciones, digamos τ con una distribución P_τ, a cada muestra. Por ejemplo, en el caso de las imágenes, los aumentos suelen incluir traslaciones, rotaciones o recortes de cada muestra de imagen. Podemos considerar que las muestras augmentadas de cada punto de datos pertenecen al mismo grupo o clúster.

Más concretamente, dado un conjunto de N muestras de datos $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, generamos m vistas augmentadas $\{\tau_i\}_{i=1}^m$ para cada punto de datos $\boldsymbol{x}_k$, denotadas como $\tilde{\boldsymbol{X}}_k = [\tau_1(\boldsymbol{x}_k), \tau_2(\boldsymbol{x}_k), \ldots, \tau_m(\boldsymbol{x}_k)]$. Tratamos las muestras augmentadas $\tilde{\boldsymbol{X}}_i$ como la clase i-ésima con un total $K = N$ grupos/clústeres. Ahora podemos aplicar el principio MCR^2 tal como lo hicimos anteriormente en el caso del aprendizaje supervisado. Concretamente, aprendemos la función de mapeo de características $f(\boldsymbol{x}, \theta)$ maximizando la reducción de la tasa de codificación de la siguiente manera:

$$\begin{aligned} &\max_\theta \Delta R_\epsilon\big(\boldsymbol{Z}\big) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}), \\ &\text{s.t.} \quad \boldsymbol{Z} = f(\tilde{\boldsymbol{X}}, \theta),\ \|\boldsymbol{Z}_k\|_F^2 = m,\ k = 1, \ldots, N. \end{aligned} \tag{4.3.1}$$

Aquí, $\tilde{\boldsymbol{X}} = [\tilde{\boldsymbol{X}}_1, \ldots, \tilde{\boldsymbol{X}}_N]$ y $\boldsymbol{Z}_k = f(\tilde{\boldsymbol{X}}_k, \theta)$ denotan las características de las muestras augmentadas de $\boldsymbol{x}_k$ para cada $k = 1, \ldots, N$.

Adoptemos de nuevo ResNet-18 como la función de mapeo de características $f(\boldsymbol{x}, \theta)$ y optimicemos los parámetros de la red θ para maximizar el objetivo no supervisado (4.3.1). Tras el entrenamiento con el conjunto de datos CIFAR-10, visualizamos la evolución de las dos tasas y su diferencia (tanto para los datos de entrenamiento como para los de prueba) a lo largo de las épocas de



(a) MCR^2 (b) MCR^2-ctrl

Figura 4.23: Evolución de las tasas de MCR^2 durante el proceso de entrenamiento para el aprendizaje no supervisado en CIFAR-10.

entrenamiento en la Figura 4.23a. En forma similar al caso del aprendizaje supervisado, tras una fase inicial, R_ϵ aumenta gradualmente mientras que R_ϵ^c disminuye, lo que indica que las características $\boldsymbol{Z}$ se están expandiendo en su conjunto, mientras que cada grupo $\boldsymbol{Z}_k$ se está comprimiendo. En comparación con el caso supervisado que se muestra en la Figura 4.18, podemos observar que, en el caso no supervisado, la tasa de codificación total R_ϵ aumenta rápidamente y el objetivo global MCR^2 se satura en un óptimo local. Para mitigar esta situación, podemos modificar la estrategia de optimización controlando la dinámica de R_ϵ y R_ϵ^c. En concreto, podemos reescalar las tasas sustituyendo la tasa de codificación $R_\epsilon(\boldsymbol{Z})$ en (4.3.1) por

$$\tilde{R}_\epsilon(\boldsymbol{Z}) \doteq \frac{1}{2\gamma_1}\log\det\left(\boldsymbol{I} + \frac{\gamma_2 d}{Nm\epsilon^2}\boldsymbol{Z}\boldsymbol{Z}^\top\right).$$

Al establecer $\gamma_1 = \gamma_2 = N$, llegamos a un objetivo de aprendizaje modificado, denominado MCR^2-CTRL, cuya dinámica de entrenamiento se muestra en la Figura 4.23b. Como podemos ver, con este objetivo MCR^2 controlado, las características se comprimen primero y luego se expanden gradualmente, lo que da lugar a un valor más alto del objetivo global $\Delta R_\epsilon(\boldsymbol{Z})$.

Para evaluar la calidad de las representaciones aprendidas y verificar si presentan buenas estructuras de subespacios, adoptamos un algoritmo de agrupamiento de subespacios denominado EnSC [YLR+16]. Como métricas de evaluación, tenemos en cuenta la información mutua normalizada (NMI), la precisión de agrupamiento (ACC) y el índice de Rand ajustado (ARI) [32]. A modo de comparación, analizamos métodos como JULE [YPB16], RTM [NMM19], DEC [XGF16], DAC [CWM+17] y DCCM [WLW+19]. Los resultados de la agrupación se resumen en la Figura 4.24. Como podemos ver, las características aprendidas al maximizar el objetivo MCR^2-CTRL alcanzan una mayor precisión de agrupación que las aprendidas por otros algoritmos.

[32] Consulte el apéndice en [YCY+20] para ver sus definiciones específicas.

Table 2: Clustering results on CIFAR10, CIFAR100, and STL10 datasets.

DATASET	METRIC	K-MEANS	JULE	RTM	DEC	DAC	DCCM	MCR^2-CTRL
CIFAR10	NMI	0.087	0.192	0.197	0.257	0.395	0.496	**0.630**
	ACC	0.229	0.272	0.309	0.301	0.521	0.623	**0.684**
	ARI	0.049	0.138	0.115	0.161	0.305	0.408	**0.508**
CIFAR100	NMI	0.084	0.103	-	0.136	0.185	0.285	**0.387**
	ACC	0.130	0.137	-	0.185	0.237	0.327	**0.375**
	ARI	0.028	0.033	-	0.050	0.087	0.173	**0.178**
STL10	NMI	0.124	0.182	-	0.276	0.365	0.376	**0.446**
	ACC	0.192	0.182	-	0.359	0.470	0.482	**0.491**
	ARI	0.061	0.164	-	0.186	0.256	0.262	**0.290**

Figura 4.24: Comparación de la precisión de la agrupación entre el entrenamiento con el objetivo MCR^2-CTRL y otros algoritmos de aprendizaje no supervisado en los conjuntos de datos CIFAR-10, CIFAR-100 y STL-10.

Sin embargo, hay un inconveniente principal: el método de aprendizaje no supervisado basado en MCR^2 requiere una costosa operación $\log\det$ de complejidad $\mathsf{O}(d^3)$ *por muestra* al calcular R^c_ϵ. Esto se vuelve rápidamente prohibitivo en lo computacional a medida que intentamos escalar a conjuntos de datos más grandes. En lugar de minimizar la tasa de codificación exacta para cada grupo de muestras augmentadas $\tilde{\boldsymbol{X}}_i$, podemos aproximar el efecto de la compresión de características en cada grupo maximizando la *similitud coseno* entre las características de las muestras augmentadas:[33]

$$\begin{aligned} &\max_{\theta} R_\epsilon(\boldsymbol{Z}) + \frac{\gamma}{N}\sum_{k=1}^{N}\sum_{i\neq j}\boldsymbol{z}_{k,j}^\top \boldsymbol{z}_{k,i}, \\ &\text{s.t.} \quad \boldsymbol{z}_{i,j} = f(\tau_j(\boldsymbol{x}_i),\theta),\ \ \|\boldsymbol{z}_{i,j}\|_2^2 = 1, \end{aligned} \tag{4.3.2}$$

donde $\gamma > 0$ es un hiperparámetro que controla la intensidad del término de similitud coseno, ya que difiere de la tasa de codificación en magnitud. Este objetivo evita la costosa operación $\log\det$ y es mucho más escalable. De hecho, este objetivo está estrechamente relacionado con métodos populares (sin supervisión) de *aprendizaje contrastivo*, tales como SimCLR [CKN+20] y DINO [CTM+21], que han sido diseñados para aprender representaciones de alta calidad a partir de una gran cantidad de imágenes.

Como mostraremos más adelante en la Sección 8.2.2 del Capítulo 8, este objetivo (4.3.2) puede implementarse de manera muy eficiente para aprender una representación altamente estructurada $\boldsymbol{Z}$ que es muy lineal y discriminativa. La representación de características aprendida ofrece un rendimiento de vanguardia en grandes conjuntos de datos del mundo real, comparable o incluso superior al de las características aprendidas por DINO, pero sin los considerables esfuerzos

[33] Dejamos como ejercicio para los lectores que verifiquen bajo qué condiciones la minimización de la tasa de codificación por clase $R^c(\boldsymbol{Z})$ es (aproximadamente) equivalente a maximizar la similitud coseno entre las características $\boldsymbol{Z}$.

Método	Modelo	Épocas	20-NN	Sondeo Lineal
DINO	ViT-B	100	72.9	76.3
SimDINO	ViT-B	100	**74.9**	**77.3**
DINO	ViT-L	100	–	–
SimDINO	ViT-L	100	**75.6**	**77.4**

Tabla 4.1: **Rendimiento de la clasificación** en el conjunto de prueba de ImageNet-1k para DINO y SimDINO, mediante tanto la precisión del método de los k vecinos más cercanos ($k = 20$) como el sondeo lineal. SimDINO supera a DINO tanto en precisión como en estabilidad del entrenamiento. El entrenamiento de DINO en ViT-L es inestable y diverge rápidamente, mientras que SimDINO se entrena establemente.

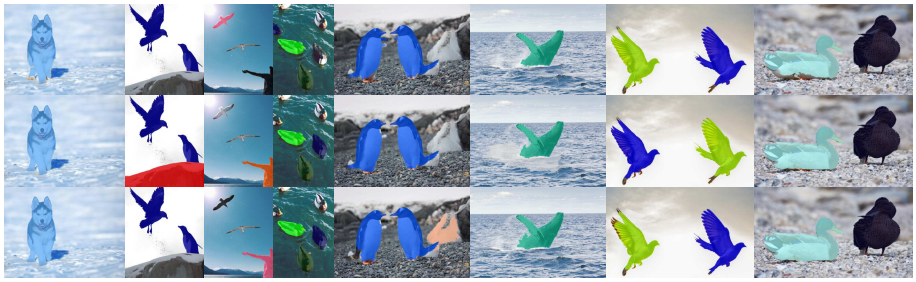


Figura 4.25: **Visualización de los resultados de la segmentación no supervisada** obtenidas con DINO ViT-B/16 (fila 1), SimDINO ViT-B/16 (fila 2) y SimDINO ViT-L/16 (fila 3).

de ingeniería que requiere DINO. Por ello, denominamos a este objetivo *DINO simplificado* (SimDINO). A modo de avance, la Tabla 4.1 muestra que SimDINO supera a DINO en la clasificación de ImageNet-1k utilizando tanto la precisión de los k vecinos más cercanos como el sondeo lineal (es decir, el entrenamiento de un clasificador lineal sobre modelos preentrenados). Esto demuestra que el objetivo da lugar a representaciones muy estructuradas y linearizadas. También podemos utilizar estas representaciones para tareas posteriores, como la segmentación no supervisada[34], tal y como se ilustra en la Figura 4.25. En el Capítulo 8 se presentarán más detalles sobre la implementación y los resultados experimentales de SimDINO.

[34]En concreto, adoptamos MaskCut [WGY+23], un enfoque no supervisado eficaz para extraer características de una red de visión con pesos congelados, a fin de realizar la detección de objetos y la segmentación de instancias.

4.4 Resumen y notas

Mensajes clave. En este capítulo, hemos estudiado los conceptos e ideas básicos subyacentes a la forma de aprender una distribución a partir de un número finito de muestras y obtener un esquema de codificación y decodificación *computable* para dicha distribución. En el caso de las distribuciones continuas y, posiblemente, de baja dimensión, hemos argumentado que dicho esquema es necesariamente *con pérdidas*; de ahí la importancia del concepto de tasa-distorsión. El ruido introducido por la distorsión, debido a la cuantización, desempeña un papel relacionado pero distinto del ruido que hemos estudiado en el capítulo anterior. Se podría considerar que el ruido considerado en el Capítulo 3 se encuentra «fuera» de la subvariedad del soporte de la distribución, mientras que el ruido analizado en este capítulo se encuentra «dentro» del soporte. De manera informal, podemos referirnos al proceso de difusión/eliminación de ruido fuera del soporte como «continuación» y al proceso de difusión/eliminación de ruido dentro del soporte como «percolación». Un análisis más unificado y una comprensión más profunda de su efecto combinado son objeto de futuros estudios.

En este capítulo, hemos introducido por primera vez el concepto de aprendizaje de una *representación* para una distribución. Hemos considerado el caso posiblemente más sencillo: el de la clasificación. Si se considera la etiqueta de clasificación $\boldsymbol{y}$ como un código fijo simple (con pérdidas enormes) para los datos $\boldsymbol{x}$, podríamos buscar una representación consistente[35] $\boldsymbol{z} = f(\boldsymbol{x})$ de los datos $\boldsymbol{x}$ de tal manera que tienda a maximizar la información mutua entre la representación y este código simple:

$$\max I(\boldsymbol{z}, \boldsymbol{y}) = H(\boldsymbol{z}) - H(\boldsymbol{z} \mid \boldsymbol{y}). \tag{4.4.1}$$

Además, exigimos que dicha representación $\boldsymbol{z}$ no solo sea consistente con $\boldsymbol{x}$, sino también mucho más *estructurada* que la distribución nativa de los datos de $\boldsymbol{x}$: las clases son linealmente discriminativas. Esto haría que la representación aprendida fuera mucho más eficiente en su uso que la distribución nativa de los datos para tareas posteriores como el reconocimiento, la segmentación y el completado. El objetivo de reducción máxima de la tasa de codificación introducido es una materialización natural y computable de estos requisitos. En el Capítulo 7, veremos cómo generalizar este objetivo a un contexto mucho más amplio, en el que $\boldsymbol{y}$ podría ser un código con pérdida más general (por ejemplo, un pie de imagen) o una variable aleatoria relacionada con $\boldsymbol{x}$.

4.5 Ejercicios y ampliaciones

Ejercicio 4.1. Demuestre las siguientes propiedades de la función $\log\det(\cdot)$.

[35] Es decir, podemos recuperar fielmente $\boldsymbol{x}$ a partir de $\boldsymbol{z}$, ya que aclararemos el concepto de representaciones «consistentes» más adelante en el Capítulo 6.

1. Demuestre que
$$f(\boldsymbol{X}) = \log\det(\boldsymbol{X})$$
es una función cóncava. (**Pista:** La función $f(\boldsymbol{x})$ es cóncava si y solo si la función unidimensional $f(\boldsymbol{x}+t\boldsymbol{h})$ es cóncava para todos los valores de $\boldsymbol{x}$ y $\boldsymbol{h}$.)

2. Demuestre que
$$\log\det(\boldsymbol{I}+\boldsymbol{X}^\top\boldsymbol{X}) = \log\det(\boldsymbol{I}+\boldsymbol{X}\boldsymbol{X}^\top)$$

3. Sea $\boldsymbol{A}\in\mathbb{R}^{n\times n}$ una matriz definida positiva. Demuestre que:
$$\log\det(\boldsymbol{A}) = \sum_{i=1}^{n}\log(\lambda_i), \tag{4.5.1}$$
donde $\lambda_1,\lambda_2,\ldots,\lambda_n$ son los valores propios de $\boldsymbol{A}$.

4. Demuestre que el gradiente de la función $\log\det(\cdot)$ viene dado por
$$\nabla\log\det(\boldsymbol{A}) = \boldsymbol{A}^{-1},$$
para una matriz definida positiva $\boldsymbol{A}$.

Ejercicio 4.2. Sea $\boldsymbol{Z}=[\boldsymbol{Z}_1,\ldots,\boldsymbol{Z}_K]\in\mathbb{R}^{d\times N}$, donde cada $\boldsymbol{Z}_k\in\mathbb{R}^{d\times N_k}$ y $N=\sum_{k=1}^{K}N_k$.

1. Demuestre que
$$\log\det\left(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top\right)\le\sum_{k=1}^{K}\log\det\left(\boldsymbol{I}+\alpha\boldsymbol{Z}_k\boldsymbol{Z}_k^\top\right),$$
donde $\alpha>0$.

2. Derive la condición necesaria y suficiente bajo la cual se cumple la igualdad.

Ejercicio 4.3. Sean $\boldsymbol{z}_1$ y $\boldsymbol{z}_2$ dos vectores de longitud unitaria. Sea $\boldsymbol{Z}=[\boldsymbol{z}_1,\boldsymbol{z}_2]$. Demuestre que los tres objetivos siguientes son equivalentes:

- $\max \boldsymbol{z}_1^\top\boldsymbol{z}_2$.
- $\min\|\boldsymbol{z}_1-\boldsymbol{z}_2\|_2^2$.
- $\max\log\det(\boldsymbol{I}+\boldsymbol{Z}\boldsymbol{Z}^\top)$.

Sin embargo, es posible que ya no sean necesariamente equivalentes cuando hay más de dos vectores en un espacio de alta dimensión. El log det es más general, ya que mide el volumen total que abarcan todos los vectores en el espacio de alta dimensión.

Ejercicio 4.4. Derive la función de tasa-distorsión para el vector aleatorio gaussiano presentado en el Ejemplo 4.4.

Capítulo 5

Las representaciones profundas como optimización desenrollada

"*Lo que no puedo crear, no lo entiendo.*"
— Richard Feynman

En capítulos anteriores, hemos mostrado cómo identificar estructuras de baja dimensión en espacios de alta dimensión, *centrándonos principalmente en estructuras lineales*. Por ejemplo, introdujimos el análisis de componentes principales (PCA) para aprender el *denoiser* lineal $\hat{\boldsymbol{U}}^\top$ cuando los datos observados $\boldsymbol{x}$ siguen el modelo estadístico $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}$. En este contexto, las representaciones aprendidas son los datos de entrada linealmente transformados $\hat{\boldsymbol{U}}^\top \boldsymbol{x}$. Bajo el supuesto de un modelo lineal, es posible aprender la estructura lineal de baja dimensión mediante algoritmos de optimización eficientes, con sólidas garantías teóricas. Más aún, el supuesto del modelo lineal abarca una amplia gama de aplicaciones y problemas, entre los que se incluyen el reconocimiento facial, la recuperación de imágenes por resonancia magnética y la recuperación de texturas estructurales [WM22].

Por otro lado, el modelo lineal puede resultar limitado cuando se trata de aplicaciones del mundo real, especialmente cuando los datos de entrada $\boldsymbol{x}$ son complejos, como el habla y los lenguajes naturales, las imágenes y los videos, así como los movimientos robóticos. Las distribuciones de baja dimensión de dichos datos suelen ser *no lineales*. El tratamiento de la no linealidad cuenta con una larga trayectoria en diversas disciplinas, como la teoría de control, el procesamiento de señales y el reconocimiento de patrones. Se han realizado esfuerzos considerables para tratar de extender los métodos y soluciones de los modelos lineales con el fin de abordar la no linealidad, incluidos los primeros intentos de ampliar el PCA al PCA no lineal (como veremos con más detalle en

el Capítulo 6). En la mayoría de los casos, los métodos se diseñan partiendo de ciertas suposiciones sobre la distribución de los datos y se adaptan a problemas específicos.

Más recientemente, las redes neuronales profundas han logrado un éxito notable en una amplia gama de datos y aplicaciones. Una red neuronal

$$f(\cdot, \boldsymbol{\theta}) \colon \boldsymbol{x} \xrightarrow{f^{\text{pre}}} \boldsymbol{z}^1 \to \cdots \to \boldsymbol{z}^\ell \xrightarrow{f^\ell} \boldsymbol{z}^{\ell+1} \to \cdots \to \boldsymbol{z}^{L+1} = \boldsymbol{z}. \tag{5.0.1}$$

puede aprender características o representaciones efectivas para aplicaciones aguas abajo. Por ejemplo, una red neuronal profunda entrenada $f(\cdot, \boldsymbol{\theta})$ puede aplicarse para mapear imágenes a vectores de características, es decir, $\boldsymbol{z}_i = f(\boldsymbol{x}_i, \boldsymbol{\theta})$, mientras que se puede entrenar un clasificador lineal a partir de dichas representaciones $\{\boldsymbol{z}_i\}$. Un avance notable es AlexNet [KSH12], una red neuronal convolucional profunda entrenada con más de un millón de imágenes naturales, que superó (en tareas de predicción) a todos los enfoques anteriores basados en características definidas manualmente. Una de las diferencias clave entre AlexNet y los enfoques anteriores es que el primero *aprende los parámetros de la transformación no lineal a partir de enormes cantidades de datos* entrenando mediante retropropagación (BP) [RHW86b], tal y como se detalla en la Sección A.2.3 del Apéndice A.

En la práctica habitual posterior, se modela el mapeo f con otras redes neuronales artificiales profundas diseñadas empíricamente y se aprenden los parámetros $\boldsymbol{\theta}$ a partir de una inicialización aleatoria mediante el algoritmo de retropropagación. Desde la aparición de AlexNet [KSH12], las arquitecturas de las redes profundas modernas siguen siendo revisadas y mejoradas en forma empírica. Arquitecturas de red como VGG [SZ14], ResNet [HZR+16b], DenseNet [HLV+17], CNN, RNN o LSTM [HS97], Transformer [VSP+17b] y mezclas de expertos (MoE) [FZS22; SMM+17], etc., han seguido ampliando los límites del rendimiento. Como parte del esfuerzo por mejorar el rendimiento de las redes profundas, se ha examinado empíricamente casi todas las componentes de las redes y se han propuesto diversas revisiones y mejoras. No se limitan a funciones de activación no lineales [KUM+17; MHN13; NIG+18; XWC+15], conexiones de salto [HZR+16b; RFB15b], normalizaciones [BKH16; IS15; MKK+18; UVL16; WH20], muestreo ascendente/descendente o agrupación [SMB10], convoluciones [KSH12; LBB+98b], etc. Sin embargo, casi todas estas modificaciones se han desarrollado a lo largo de años de estudios empíricos de prueba y error o de ablación. Algunas prácticas recientes llegaron incluso al extremo de buscar estructuras de red y estrategias de entrenamiento efectivas mediante técnicas de búsqueda aleatoria a gran escala, como "Neural Architecture Search" (Búsqueda de Arquitectura Neuronal) [BGN+17; ZL17], AutoML [HKV19] y "Learning to Learn" (Aprender a Aprender)[ADG+16].

A pesar de la amplia aplicación de las redes neuronales profundas, no está claro cuáles son los principios de diseño subyacentes de una red construida de este modo. En particular, no está claro qué función matemática desempeña cada capa de la red. En este capítulo, basándonos en los resultados de los capítulos anteriores, desarrollamos un marco teórico que proporcionará una interpretación

matemática totalmente rigurosa del papel de una red profunda, incluyendo sus capas individuales y la red en su conjunto.

Para comprender las redes profundas y entender cómo deberían diseñarse mejor, debemos partir del objetivo del aprendizaje de representaciones. En los capítulos anteriores, hemos argumentado que el objetivo consiste en identificar la distribución de los datos, que es intrínsecamente de baja dimensión, y luego transformarla en una representación compacta y estructurada (por ejemplo, lineal por tramos). Como hemos visto en el capítulo anterior, el enfoque general para identificar una distribución de datos de baja dimensión consiste en un proceso de compresión que minimiza progresivamente la entropía o la tasa de codificación de la distribución. Sin embargo, hasta ahora hemos estado utilizando redes profundas diseñadas en forma empírica para modelar o aproximar las operaciones destinadas a optimizar estos objetivos, como la función de *score* para la eliminación de ruido (en la Sección 1.3.1) o la transformación que maximiza la reducción de la tasa (en la Sección 4.2.4).

Como hemos argumentado en el capítulo anterior, en la Sección 4.2 en particular, se puede evaluar la calidad de la representación resultante mediante la información que la representación refinada por sobre una representación «perezosa» que modela todos los datos como una gran distribución gaussiana.[1] En particular, si *utilizamos una mezcla de gaussianas (subespacios)*[2] *como distribuciones prototípicas para aproximar la distribución no lineal que nos interesa*, entonces podemos medir de manera eficiente la tasa de codificación de dicha representación utilizando la (suma de las) funciones de tasa-distorsión de las gaussianas asociadas. Entonces, la cantidad de *información ganada* o la entropía (relativa) reducida con dicho modelado puede medirse mediante la diferencia entre la tasa de codificación de la representación «perezosa» simplificada y la de la representación más refinada. Por lo tanto, el objetivo del aprendizaje de representaciones es maximizar esta ganancia de información, también conocida como el objetivo de reducción de tasa.

Como veremos en este capítulo, una vez que el objetivo del aprendizaje de representaciones queda claro, la función de una red neuronal profunda consiste precisamente en ayudar a optimizar ese objetivo de modo iterativo. Cada capa de una red neuronal profunda puede derivarse en forma natural como un paso de optimización iterativo destinado a maximizar de manera incremental la ganancia de información, incluyendo las populares arquitecturas de ResNet, CNN y Transformadores, así como otras variantes más avanzadas. En particular, este capítulo tiene como objetivo responder a las siguientes preguntas sobre las redes profundas:

- Sección 5.1 — dada una medida de calidad para una representación aprendida, ¿cómo construir el mapeo no lineal de los datos a la representación óptima mediante la optimización desenrollada del objetivo?
- Sección 5.2 — ¿cómo podría el mencionado enfoque de «desenrollado»

[1] que hemos visto en el capítulo anterior como una opción concreta de interpretación del conjunto de datos muestreado.

[2] que hemos estudiado a fondo en el capítulo anterior

ofrecer una interpretación fundamentada de las populares arquitecturas Transformers? Si es así, ¿cuáles son los mecanismos de optimización y los objetivos asociados?

- Sección 5.3 — ¿cómo nos ayudaría este marco a diseñar arquitecturas profundas más eficientes o más parsimoniosas?

5.1 Redes profundas transparentes *white-box* mediante optimización desenrollada

Ahora bien, si estamos de acuerdo en que maximizar la reducción de tasa o la ganancia de información conduce a la representación deseada, tal como se analiza en la Sección 4.2, la pregunta que queda por resolver es cómo construir y aprender un mapeo (no lineal) desde los datos $\boldsymbol{X}$ hacia la representación óptima $\boldsymbol{Z}^*$. Esto implica diseñar una arquitectura de red y un algoritmo de aprendizaje que puedan captar de manera eficaz las estructuras subyacentes de los datos y materializar fielmente la representación óptima.

5.1.1 Redes profundas a partir del descenso por gradiente desenrollado

En el capítulo anterior, presentamos el objetivo de reducción de tasa (4.2.15) como un objetivo fundamentado para aprender representaciones discriminativas lineales de los datos. Sin embargo, no hemos especificado la arquitectura de la función de mapeo de características $\boldsymbol{z} = f(\boldsymbol{x}, \boldsymbol{\theta})$ para extraer dichas representaciones de los datos de entrada $\boldsymbol{x}$. Una opción sencilla es utilizar una red profunda convencional, como ResNet, para implementar $f(\boldsymbol{x}, \boldsymbol{\theta})$. Como hemos visto en el Sección 4.3.1, esta opción suele dar lugar a un rendimiento razonable en la práctica. No obstante, siguen existiendo varios problemas sin resolver al adoptar una red profunda arbitraria. Aunque la representación de características aprendida es ahora más interpretable, la red en sí misma sigue *sin* serlo. No está claro por qué cualquier red de «black-box» elegida es capaz siquiera de optimizar el objetivo MCR^2 deseado. Los buenos resultados empíricos (por ejemplo, con una ResNet) no justifican necesariamente la elección concreta de arquitecturas y operadores de la red: ¿Por qué es necesario un modelo de gran profundidad en capas? ¿Qué es lo que las capas adicionales pretenden mejorar o simplificar? ¿Qué anchura y profundidad son adecuadas? ¿O existe alguna justificación rigurosa para las convoluciones (en su popular forma multicanal) y los operadores no lineales (por ejemplo, ReLU o softmax) que se utilizan?

En este capítulo, demostramos que el uso del ascenso por gradiente para maximizar la reducción de tasa $\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{\Pi})$, tal como se define en (4.2.15), conduce de manera natural a una red profunda transparente tipo *white-box* que realiza el mapeo deseado. Todas las capas de la red, los operadores lineales y no lineales, así como los parámetros se *construyen explícitamente siguiendo un*

enfoque de propagación puramente hacia adelante. Además, estas arquitecturas de red se asemejan a las redes profundas existentes diseñadas empíricamente, lo que proporciona justificaciones fundamentadas para su diseño.

Ascenso por gradiente para la reducción de la tasa de codificación. En el capítulo anterior se vio que, para buscar una representación discriminativa lineal (LDR), matemáticamente estamos buscando, en esencia, una función continua $f(\cdot) : \boldsymbol{x} \mapsto \boldsymbol{z}$ que mapee los datos $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}$ (o características iniciales extraídas de los datos[3]) a una representación óptima $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N] \in \mathbb{R}^{d\times N}$ que maximice el siguiente objetivo de reducción de la tasa de codificación (véase también (4.2.17)):

$$\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \doteq \underbrace{\frac{1}{2}\log\det\Big(\boldsymbol{I} + \alpha \boldsymbol{Z}\boldsymbol{Z}^\top\Big)}_{R_\epsilon(\boldsymbol{Z})} - \underbrace{\sum_{k=1}^{K}\frac{\gamma_k}{2}\log\det\Big(\boldsymbol{I} + \alpha_k \boldsymbol{Z}\boldsymbol{\Pi}_k\boldsymbol{Z}^\top\Big)}_{R_\epsilon^c(\boldsymbol{Z}\mid\boldsymbol{\Pi})}, \tag{5.1.1}$$

donde $\epsilon > 0$ es un error de cuantización prescrito y, para simplificar, denotamos[4]

$$\alpha \doteq \frac{d}{N\epsilon^2}, \quad \alpha_k \doteq \frac{d}{\operatorname{tr}(\boldsymbol{\Pi}_k)\epsilon^2}, \quad \gamma_k \doteq \frac{\operatorname{tr}(\boldsymbol{\Pi}_k)}{N}, \quad \text{for } k = 1, \ldots, K. \tag{5.1.2}$$

La pregunta se reduce, en realidad, a si existe una forma *constructiva* de encontrar una aplicación continua de este tipo $f(\cdot, \boldsymbol{\theta})$ de $\boldsymbol{x}$ a $\boldsymbol{z}$. Para ello, consideremos la maximización incremental del objetivo ΔR_ϵ como una función de $\boldsymbol{Z} \subseteq \mathbb{S}^{d-1}$. Aunque puede haber muchos esquemas de optimización entre los que elegir, por simplicidad consideraremos primero el esquema de «ascenso por gradiente proyectado» (PGA), posiblemente el más sencillo:[5]

$$\boldsymbol{Z}^{\ell+1} \propto \boldsymbol{Z}^\ell + \eta \cdot \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) \quad \text{s.t.} \quad \boldsymbol{Z}^{\ell+1} \subseteq \mathbb{S}^{d-1}, \quad \ell = 1, 2, \ldots, \tag{5.1.3}$$

para un tamaño de paso $\eta > 0$ y el iterado inicial dado por $\boldsymbol{Z}^0 = \boldsymbol{X}$.[6] Este esquema puede interpretarse como la forma en que se deben ajustar gradualmente las ubicaciones de las características actuales $\boldsymbol{Z}^\ell$, inicializadas a partir de los datos de entrada $\boldsymbol{X}$, para que el conjunto resultante $\boldsymbol{Z}^{\ell+1}$ mejore la reducción de la tasa ΔR_ϵ, tal como se ilustra en la Figura 5.1.

[3] Como veremos, la necesidad de dicha extracción de características se abordará en la siguiente sección.

[4] Obsérvese que utilizamos una notación ligeramente simplificada en comparación con el Capítulo 3.

[5] Obsérvese que utilizamos el subíndice j en $\boldsymbol{Z}_j$ para indicar las características de la j-ésima clase y el superíndice ℓ en $\boldsymbol{Z}^\ell$ para indicar todas las características en la iteración o capa ℓ-ésima.

[6] Una vez más, para simplificar, aquí suponemos en primer lugar que las características iniciales $\boldsymbol{Z}^1$ son los propios datos. Obsérvese que aquí ℓ denota el número de iteraciones. Por lo tanto, los datos y las características tienen la misma dimensión d. Sin embargo, esto no tiene por qué ser así. Como veremos en la siguiente sección, las características iniciales pueden ser algunas características (elevadas) de los datos para empezar y, en principio, podrían tener una dimensión diferente (mucho mayor). Todas las iterados posteriores tienen la misma dimensión.

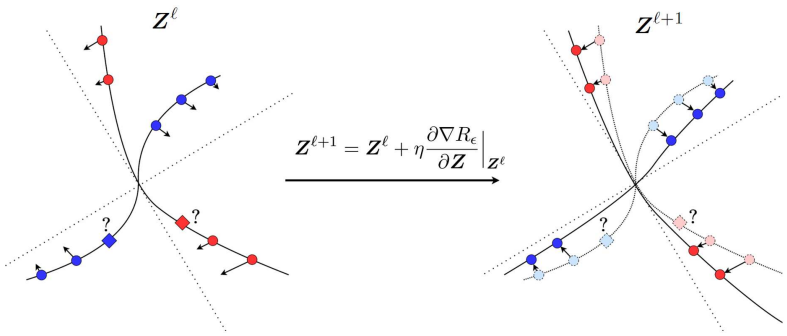


Figura 5.1: Deformación incremental mediante flujo de gradiente para, por un lado, aplanar los datos de cada clase en un subespacio y, por otro, separar las diferentes clases.

Un cálculo sencillo muestra que el gradiente $\partial \Delta R_\epsilon / \partial \boldsymbol{Z}$ implica evaluar las siguientes derivadas de los dos términos de ΔR_ϵ:

$$\frac{1}{2}\frac{\partial \log\det(\boldsymbol{I}+\alpha \boldsymbol{Z}\boldsymbol{Z}^\top)}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) = \underbrace{\alpha(\boldsymbol{I}+\alpha \boldsymbol{Z}^\ell(\boldsymbol{Z}^\ell)^\top)^{-1}}_{\boldsymbol{E}^\ell \in \mathbb{R}^{d\times d}} \boldsymbol{Z}^\ell, \tag{5.1.4}$$

$$\frac{1}{2}\frac{\partial \left(\gamma_k \log\det(\boldsymbol{I}+\alpha_k \boldsymbol{Z}\boldsymbol{\Pi}_k\boldsymbol{Z}^\top)\right)}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) = \gamma_k \underbrace{\alpha_k(\boldsymbol{I}+\alpha_k \boldsymbol{Z}^\ell\boldsymbol{\Pi}_k(\boldsymbol{Z}^\ell)^\top)^{-1}}_{\boldsymbol{C}_k^\ell \in \mathbb{R}^{d\times d}} \boldsymbol{Z}^\ell\boldsymbol{\Pi}_k. \tag{5.1.5}$$

Nótese que, en lo anterior, la matriz $\boldsymbol{E}^\ell$ solo depende de $\boldsymbol{Z}^\ell$ y tiene como objetivo *expandir* todas las características para aumentar la tasa de codificación global; la matriz $\boldsymbol{C}_k^\ell$, a su vez, depende de las características de la clase k y tiene como objetivo *comprimirlas* para reducir la tasa de codificación de cada clase. Entonces, el gradiente completo $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) \in \mathbb{R}^{d\times N}$ tiene la forma:

$$\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) = \underbrace{\boldsymbol{E}^\ell}_{\text{Expansion}} \boldsymbol{Z}^\ell - \sum_{k=1}^{K} \gamma_k \underbrace{\boldsymbol{C}_k^\ell}_{\text{Compression}} \boldsymbol{Z}^\ell\boldsymbol{\Pi}_k. \tag{5.1.6}$$

Nota 5.1 (Interpretación de $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_j^\ell$ como operadores lineales). Para cualquier $\boldsymbol{z}^\ell \in \mathbb{R}^d$,

$$\boldsymbol{E}^\ell\boldsymbol{z}^\ell = \alpha(\boldsymbol{z}^\ell - \boldsymbol{Z}^\ell\boldsymbol{q}_\star^\ell), \quad \text{donde} \quad \boldsymbol{q}_\star^\ell \doteq \arg\min_{\boldsymbol{q}^\ell} \left\{\alpha\|\boldsymbol{z}^\ell - \boldsymbol{Z}^\ell\boldsymbol{q}^\ell\|_2^2 + \|\boldsymbol{q}^\ell\|_2^2\right\}. \tag{5.1.7}$$

Obsérvese que $\boldsymbol{q}_\star^\ell$ es exactamente la solución de la regresión por crestas para todos los puntos de datos $\boldsymbol{Z}^\ell$ en cuestión. Por lo tanto, $\boldsymbol{E}^\ell$ (y de manera similar para $\boldsymbol{C}_k^\ell$) es aproximadamente (es decir, cuando N es lo suficientemente grande) la proyección sobre el complemento ortogonal del subespacio generado

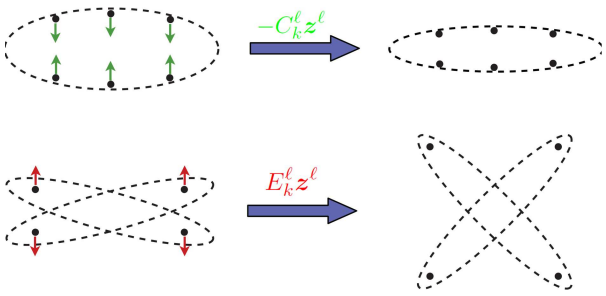


Figura 5.2: Interpretación de $\boldsymbol{C}_k^\ell$ y $\boldsymbol{E}^\ell$: $\boldsymbol{C}_k^\ell$ comprime cada clase contrayendo las características en un subespacio de baja dimensión; $\boldsymbol{E}^\ell$ expande todas las características contrastando y repeliendo características entre diferentes clases.

por las columnas de $\boldsymbol{Z}^\ell$. Otra forma de interpretar la matriz $\boldsymbol{E}^\ell$ es mediante la descomposición en valores propios de la matriz de covarianza $\boldsymbol{Z}^\ell(\boldsymbol{Z}^\ell)^\top$. Suponiendo que $\boldsymbol{Z}^\ell(\boldsymbol{Z}^\ell)^\top \doteq \boldsymbol{U}^\ell\boldsymbol{\Lambda}^\ell(\boldsymbol{U}^\ell)^\top$, donde $\boldsymbol{\Lambda}^\ell \doteq \operatorname{diag}\left(\lambda_1^\ell, \ldots, \lambda_d^\ell\right)$, tenemos

$$\boldsymbol{E}^\ell = \alpha \boldsymbol{U}^\ell \operatorname{diag}\left(\frac{1}{1+\alpha\lambda_1^\ell}, \ldots, \frac{1}{1+\alpha\lambda_d^\ell}\right)\left(\boldsymbol{U}^\ell\right)^\top. \tag{5.1.8}$$

Por lo tanto, la matriz $\boldsymbol{E}^\ell$ actúa sobre un vector $\boldsymbol{z}^\ell$ estirándolo de tal manera que las direcciones de gran varianza se reducen, mientras que se mantienen las direcciones de varianza cercana a cero. Estas son precisamente las direcciones (5.1.4) en las que desplazamos las características para que el volumen total se expanda y la tasa de codificación aumente, de ahí el signo positivo. Por el contrario, las direcciones asociadas a (5.1.5) son «residuos» de las características de cada clase que se desvían del subespacio al que se supone que pertenecen. Estas son exactamente las direcciones en las que las características deben comprimirse de nuevo hacia su respectivo subespacio, de ahí el signo negativo (véase la Figura 5.2).

En esencia, las operaciones lineales $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ en el método de ascenso por gradiente para la reducción de tasa se determinan mediante los datos de entrenamiento que realizan «autoregresiones». Los recientes avances en la comprensión de la regresión de cresta en una configuración sobreparametrizada [WX20; YYY+20] indican que el uso de datos aparentemente muestreados en forma redundante (de cada subespacio) como regresores no conduce a sobreajuste.

Incremento del mapa de características guiado por gradientes. Nótese que en lo anterior, el ascenso por gradiente considera todas las características $\boldsymbol{Z}^\ell = [\boldsymbol{z}_1^\ell, \ldots, \boldsymbol{z}_N^\ell]$ como variables libres. El incremento $\boldsymbol{Z}^{\ell+1} - \boldsymbol{Z}^\ell = \eta \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell)$ aún no proporciona una transformación en todo el dominio de características $\boldsymbol{z}^\ell \in \mathbb{R}^d$. Según la ecuación (5.1.6), el gradiente no puede evaluarse en un punto cuya pertenencia se desconoce, tal como se ilustra en la Figura 5.1. Por lo

tanto, para hallar la $f(\boldsymbol{x}, \boldsymbol{\theta})$ óptima en forma explícita, podemos considerar la construcción de una pequeña transformación de incremento $g(\cdot, \theta^\ell)$ sobre la característica de la ℓ-ésima capa $\boldsymbol{z}^\ell$ para emular el esquema de gradiente (proyectado) anterior:

$$\boldsymbol{z}^{\ell+1} \propto \boldsymbol{z}^\ell + \eta \cdot g(\boldsymbol{z}^\ell, \theta^\ell) \quad \text{sujeto a} \quad \boldsymbol{z}^{\ell+1} \in \mathbb{S}^{d-1} \tag{5.1.9}$$

de tal manera que $\left[g(\boldsymbol{z}_1^\ell, \boldsymbol{\theta}^\ell), \ldots, g(\boldsymbol{z}_N^\ell, \boldsymbol{\theta}^\ell)\right] \approx \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell)$. Es decir, necesitamos aproximar el flujo del gradiente $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}$ que deforma localmente todas las características (de entrenamiento) $\{\boldsymbol{z}_i^\ell\}_{i=1}^N$ con un mapeo continuo $g(\boldsymbol{z}, \boldsymbol{\theta})$ definido en todo el espacio de características $\boldsymbol{z}^\ell \in \mathbb{R}^d$. Obsérvese que el incremento (5.1.9) puede interpretarse como una versión discretizada de una ecuación diferencial continua:

$$\dot{\boldsymbol{z}} = g(\boldsymbol{z}, \theta). \tag{5.1.10}$$

Por lo tanto, la red (profunda) así construida puede interpretarse como una determinada ecuación diferencial neuronal ordinaria [CRB+18]. Sin embargo, a diferencia de la ODE neuronal, donde el flujo g se elige como estructura genérica, aquí nuestro $g(\boldsymbol{z}, \boldsymbol{\theta})$ tiene el propósito de emular el flujo de gradiente de la reducción de tasa en el conjunto de características (como se muestra en la Figura 5.1):

$$\dot{\boldsymbol{Z}} = \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}},$$

y su estructura se deriva y se determina íntegramente a partir de este objetivo, sin ningún otro supuesto previo ni heurística.

Al examinar la estructura del gradiente (5.1.6), se sugiere que una opción natural para la transformación del incremento $g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell)$ es de la forma:

$$g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \doteq \boldsymbol{E}^\ell \boldsymbol{z}^\ell - \sum_{k=1}^K \gamma_k \pi_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell \in \mathbb{R}^d, \tag{5.1.11}$$

donde $\pi_k(\boldsymbol{z}^\ell) \in [0, 1]$ indica la probabilidad de que $\boldsymbol{z}^\ell$ pertenezca a la clase k-ésima. Los parámetros del mapa de incremento $\boldsymbol{\theta}^\ell$ dependen: En primer lugar, de un conjunto de mapas lineales representados por $\boldsymbol{E}^\ell$ y $\{\boldsymbol{C}_k^\ell\}_{k=1}^K$ que dependen únicamente de las estadísticas de las características del entrenamiento $\boldsymbol{Z}^\ell$. En segundo lugar, de la pertenencia $\{\pi_k(\boldsymbol{z}^\ell)\}_{k=1}^K$ de cualquier característica $\boldsymbol{z}^\ell$. Nótese que, en las muestras de entrenamiento $\boldsymbol{Z}^\ell$, para las que se conocen las pertenencias $\boldsymbol{\Pi}_k$, la función $g(\boldsymbol{z}^\ell, \boldsymbol{\theta})$ así definida proporciona exactamente los valores del gradiente $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell)$.

Dado que solo disponemos de la función de pertenencia para los datos de entrenamiento, la función $g(\cdot)$ definida en (5.1.11) solo puede evaluarse sobre los datos de entrenamiento. Para extrapolar $g(\cdot)$ a todo el espacio de características, necesitamos estimar $\pi_k(\boldsymbol{z}^\ell)$ en su segundo término. En el aprendizaje profundo convencional, este mapa suele modelarse como una red profunda y aprenderse a partir de los datos de entrenamiento, por ejemplo, mediante *retropropagación*.

Sin embargo, nuestro objetivo aquí no es (todavía) aprender un clasificador preciso $\pi_k(\boldsymbol{z}^\ell)$. En cambio, solo necesitamos una estimación lo suficientemente buena de la información de clase para que $g(\cdot)$ aproxime bien el gradiente $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}$.

A partir de la interpretación geométrica de las aplicaciones lineales $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ dada por la Nota 5.1, el término $\boldsymbol{p}_k^\ell \doteq \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell$ puede considerarse como (aproximadamente) la proyección de $\boldsymbol{z}^\ell$ sobre el complemento ortogonal de cada clase j. Por lo tanto, $\|\boldsymbol{p}_j^\ell\|_2$ es pequeño si $\boldsymbol{z}^\ell$ está en la clase j y grande en caso contrario. Esto nos motiva a estimar su pertenencia basándonos en la siguiente función softmax:

$$\widehat{\boldsymbol{\pi}}(\boldsymbol{z}^\ell) \doteq \operatorname{softmax}\left(-\lambda \begin{bmatrix} \|\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell\|_2 \\ \vdots \\ \|\boldsymbol{C}_K^\ell \boldsymbol{z}^\ell\|_2 \end{bmatrix}\right) \tag{5.1.12}$$

$$= \frac{1}{\sum_{k=1}^K \exp(-\lambda \|\boldsymbol{C}_k^\ell \boldsymbol{z}^\ell\|_2)} \begin{bmatrix} \exp(-\lambda \|\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell\|_2) \\ \vdots \\ \exp(-\lambda \|\boldsymbol{C}_K^\ell \boldsymbol{z}^\ell\|_2) \end{bmatrix} \in [0,1]^K. \tag{5.1.13}$$

Por lo tanto, el segundo término de (5.1.11) puede aproximarse mediante esta pertenencia estimada:

$$\sum_{k=1}^K \gamma_k \pi_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell \;\approx\; \sum_{k=1}^K \gamma_k \widehat{\pi}_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell \;\doteq\; \boldsymbol{\sigma}\Big([\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell, \ldots, \boldsymbol{C}_K^\ell \boldsymbol{z}^\ell]\Big), \tag{5.1.14}$$

que se denota como un operador no lineal $\boldsymbol{\sigma}(\cdot)$ sobre las salidas de la característica $\boldsymbol{z}^\ell$ a través de K grupos de filtros: $[\boldsymbol{C}_1^\ell, \ldots, \boldsymbol{C}_K^\ell]$. Obsérvese que la no linealidad surge debido a una asignación «suave» de la pertenencia a una clase basada en las respuestas de las características que producen esos filtros.

En general, al combinar (5.1.9), (5.1.11) y (5.1.14), la transformación incremental de las características de $\boldsymbol{z}^\ell$ a $\boldsymbol{z}^{\ell+1}$ queda ahora así:

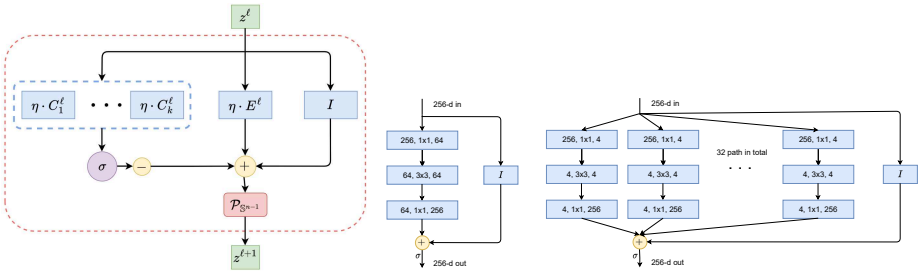
$$\boldsymbol{z}^{\ell+1} \propto \boldsymbol{z}^\ell + \eta \cdot \boldsymbol{E}^\ell \boldsymbol{z}^\ell - \eta \cdot \boldsymbol{\sigma}\big([\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell, \ldots, \boldsymbol{C}_K^\ell \boldsymbol{z}^\ell]\big) \tag{5.1.15}$$

$$= \boldsymbol{z}^\ell + \eta \cdot g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \qquad \text{s.t.} \quad \boldsymbol{z}^{\ell+1} \in \mathbb{S}^{d-1}, \tag{5.1.16}$$

con la función no lineal $\boldsymbol{\sigma}(\cdot)$ definida anteriormente, donde $\boldsymbol{\theta}^\ell$ que recoge todos los parámetros por capa. Es decir, $\boldsymbol{\theta}^\ell = \big\{\boldsymbol{E}^\ell, \boldsymbol{C}_1^\ell, \ldots, \boldsymbol{C}_K^\ell, \gamma_k, \lambda\big\}$. Obsérvese que las características de cada capa siempre se «normalizan» sistemáticamente mediante la proyección sobre la esfera unitaria $\mathbb{S}^{d-1}$, denotada como $\mathcal{P}_{\mathbb{S}^{d-1}}$, es decir, dividiendo cada característica por su norma.[7] La forma del incremento en (5.1.15) puede ilustrarse mediante un diagrama en la Figura 5.3(a).

Red profunda para optimizar la reducción de tasa Obsérvese que el incremento está construido para emular el ascenso por gradiente para la

[7] Esto refleja el bloque LayerNorm habitual en las redes neuronales, que calcula la característica proyectada y, a continuación, la multiplica y suma mediante parámetros entrenables [BKH16]; véase la Sección 8.2.3 para obtener más detalles.



(a) **ReduNet** (b) **ResNet** y **ResNeXt**.

Figura 5.3: Arquitecturas de la red ReduNet y comparación con otras. **(a)**: Estructura de capas de **ReduNet** derivada de una iteración de ascenso por gradiente para optimizar la reducción de tasa. **(b)** (izquierda): Una capa de ResNet [HZR+16b]; y **(b)** (derecha): Una capa de ResNeXt [XGD+17]. Como veremos en la Sección 5.1.2, los operadores lineales $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ de ReduNet se convierten naturalmente en convoluciones multicanal cuando se impone la invariancia al desplazamiento.

reducción de tasa ΔR_ϵ. Por lo tanto, al transformar las características en forma iterativa mediante el proceso anterior, esperamos que la reducción de tasa aumente, como veremos en la sección experimental. Este proceso iterativo, una vez que converge, por ejemplo tras L iteraciones, proporciona el mapa de características deseado $f(\boldsymbol{x}, \boldsymbol{\theta})$ sobre la entrada $\boldsymbol{x} = \boldsymbol{z}^0$, precisamente en forma de una *red profunda*, en la que cada capa tiene la estructura que se muestra en la Figura 5.3 a la izquierda:

$$f(\boldsymbol{x}, \boldsymbol{\theta}) \;=\; f^L \circ f^{L-1} \circ \cdots \circ f^1 \circ f^0(\boldsymbol{z}^0), \tag{5.1.17}$$

$$f^\ell(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \;\doteq\; \boldsymbol{z}^{\ell+1} = \mathcal{P}_{\mathbb{S}^{n-1}}[\boldsymbol{z}^\ell + \eta \cdot g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell)], \tag{5.1.18}$$

$$g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \;=\; \boldsymbol{E}^\ell \boldsymbol{z}^\ell - \boldsymbol{\sigma}\big([\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell, \ldots, \boldsymbol{C}_K^\ell \boldsymbol{z}^\ell]\big). \tag{5.1.19}$$

Dado que esta red profunda se deriva de la maximización de la tasa **de reducción**, la denominamos **ReduNet**. Al comparar la arquitectura de ReduNet con las de redes populares diseñadas empíricamente, **ResNet** y **ResNeXt**, tal como se muestra en la Figura 5.3, la similitud resulta casi inquietante. Desde un punto de vista conceptual, ReduNet también podría utilizarse para justificar la popular arquitectura de mezcla de expertos (**MoE**) [SMM+17], ya que cada canal paralelo, $\boldsymbol{C}_k^\ell$, puede considerarse un «experto» entrenado para cada clase de objetos.

Resumimos el entrenamiento y la evaluación de ReduNet en Algoritmo 5.1 y Algoritmo 5.2, respectivamente. Nótese que todos los parámetros de la red se construyen explícitamente capa por capa mediante una *propagación hacia adelante*. ¡Esta construcción no requiere retropropagación! Las características así aprendidas pueden utilizarse directamente para la clasificación, por ejemplo, mediante un clasificador de subespacio más cercano.

Ejemplo 5.1. Para dar cierta intuición de cómo ReduNet transforma las características, presentamos un ejemplo sencillo con gaussianas 3D mezcladas

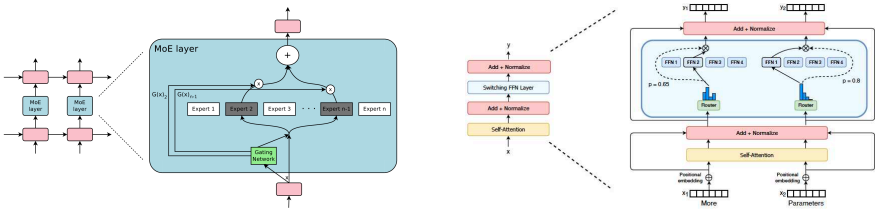


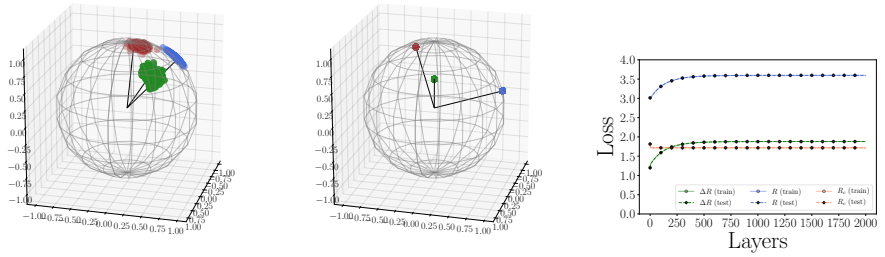
Figura 5.4: Izquierda: una red profunda de mezcla de expertos (MoE) [SMM+17]. Derecha: un Switch Transformer que fomenta la esparcidad [FZS22], utilizado para implementar MoE con 1,7 billones de parámetros.

y visualizamos cómo se transforman las características en la Figura 5.5. Consideremos una mezcla de tres distribuciones gaussianas en $\mathbb{R}^3$ que se proyecta sobre $\mathbb{S}^2$. Primero generamos puntos de datos para 3 clases: para $k = 1, 2, 3$, $\boldsymbol{X}_k = [\boldsymbol{x}_{k,1}, \ldots, \boldsymbol{x}_{k,m}] \in \mathbb{R}^{3\times m}$, $\boldsymbol{x}_{k,i} \sim \mathcal{N}(\boldsymbol{\mu}_k, \sigma_k^2 \boldsymbol{I})$ y $\pi(\boldsymbol{x}_{k,i}) = k$. Fijamos $m = 500, \sigma_1 = \sigma_2 = \sigma_3 = 0.1$ y $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\mu}_3 \in \mathbb{S}^2$. A continuación, proyectamos todos los puntos de datos sobre $\mathbb{S}^2$, es decir, $\boldsymbol{x}_{k,i}/\|\boldsymbol{x}_{k,i}\|_2$. Para construir la red (calculando $\boldsymbol{E}^\ell, \boldsymbol{C}_k^\ell$ para la capa ℓ-ésima), establecemos el número de iteraciones/capas $L = 2,000$, el tamaño del paso $\eta = 0.5$ y la precisión $\epsilon = 0.1$. Hacemos esto únicamente para demostrar que nuestro marco conduce a redes profundas estables incluso con miles de capas. En la práctica, puede que no sean necesarias miles de capas y se puede detener el proceso cuando la incorporación de nuevas capas empiece a dar resultados cada vez menores. Para este ejemplo, bastan unas doscientas capas. Por lo tanto, el claro objetivo de optimización proporciona un criterio natural para determinar la profundidad necesaria de la red.

Como se muestra en la Figura 5.5, podemos observar que, tras la aplicación del mapeo $f(\cdot, \theta)$, las muestras de la misma clase se comprimen en gran medida y convergen en un único grupo o clúster, y que el ángulo entre dos grupos diferentes es aproximadamente $\pi/2$, lo cual concuerda adecuadamente con la solución óptima $\boldsymbol{Z}^\star$ de la pérdida MCR^2 en $\mathbb{S}^2$. La pérdida MCR^2 de las características en las diferentes capas se puede observar en la Figura 5.5(**c**). Empíricamente, hemos observado que la ReduNet construida es capaz de maximizar la pérdida MCR^2 y converge de manera estable, mientras que las muestras de la misma clase convergen en un solo grupo y los puntos de diferentes grupos son ortogonales entre sí.[8] Además, al tomar nuevas muestras de las mismos distribuciones, observamos que las nuevas muestras de la misma clase convergen sistemáticamente hacia el mismo centro de clúster que las muestras de entrenamiento.

■

[8] Debido a la baja dimensión, cada grupo se reduce a un subespacio de dimensión 1 en la esfera (por ejemplo, un solo punto y su antípoda); este es el caso extremo de mantener la dimensión dentro del grupo al mínimo. En dimensiones superiores, por supuesto, los subespacios estarán menos comprimidos.



(a) $\boldsymbol{X}_{\text{entrenamiento}}$ (b) $\boldsymbol{Z}_{\text{entrenamiento}}$ (c) Pérdida (entrenamiento/evaluación)

Figura 5.5: Muestras originales y representaciones aprendidas para la mezcla de gaussianas en 3D. Visualizamos los puntos de datos $\boldsymbol{X}$ (antes de la aplicación de $f(\cdot, \theta)$) en (**a**) y las características aprendidas $\boldsymbol{Z}$ (después de la aplicación de $f(\cdot, \theta)$) en (**b**) mediante un diagrama de dispersión. En cada diagrama de dispersión, cada color representa una clase de muestras. En (**c**), también mostramos los diagramas que representan la evolución de los valores de las funciones objetivo.

5.1.2 Redes convolucionales a partir de la reducción de tasa en invariantes

En la sección anterior, derivamos la arquitectura por capas de una red profunda, la ReduNet, utilizando la optimización desenrollada para el objetivo de reducción de tasa. Concretamente, el término de compresión $R^c_\epsilon(\boldsymbol{Z} \mid \boldsymbol{\Pi})$ en (5.1.1) está diseñado para comprimir representaciones de la misma clase. Sin embargo, esta formulación no tiene en cuenta posibles transformaciones o deformaciones del dominio de los datos de entrada. Por ejemplo, desplazar ligeramente un objeto hacia la derecha no cambia la etiqueta semántica de una imagen. En esta sección, demostraremos cómo se pueden derivar capas convolucionales maximizando un objetivo de reducción de tasa que sea invariante ante ciertas deformaciones del dominio, como las rotaciones y traslaciones de imágenes.

En muchas tareas de agrupamiento o clasificación (como la detección de objetos en imágenes), consideramos que dos muestras son *equivalentes* si difieren por ciertas clases de deformaciones o augmentos del dominio $\mathcal{T} = \{\tau\}$. Por lo tanto, solo nos interesan las estructuras de baja dimensión que sean *invariantes* ante tales deformaciones (es decir, $\boldsymbol{x} \in \mathcal{M}$ si y solo si $\tau(\boldsymbol{x}) \in \mathcal{M}$ para todo $\tau \in \mathcal{T}$), que se sabe que tienen estructuras geométricas y topológicas sofisticadas y que pueden ser difíciles de aprender con precisión en la práctica, incluso con CNNs rigurosamente diseñadas [CW16a]. En este marco, esto se puede formular de una manera muy natural: todas las instancias equivalentes deben incrustarse en el mismo subespacio, de modo que el propio subespacio sea invariante respecto a las transformaciones que se están considerando.

En muchas aplicaciones, como los datos en serie de tiempo o los datos de imágenes, el significado semántico (las etiquetas) de los datos es *invariante*

ante ciertas transformaciones $\mathfrak{g} \in \mathbb{G}$ (para algún grupo $\mathbb{G}$) [CW16c; ZKR+17]. Por ejemplo, el significado de una señal de audio es invariante ante un desplazamiento en el tiempo; y la identidad de un objeto en una imagen es invariante ante una traslación en el plano de la imagen. Por lo tanto, preferimos que el mapeo de características $f(\boldsymbol{x}, \boldsymbol{\theta})$ sea rigurosamente invariante ante tales transformaciones:

$$\textit{Invariancia de grupo: } f(\boldsymbol{x} \circ \mathfrak{g}, \boldsymbol{\theta}) \sim f(\boldsymbol{x}, \boldsymbol{\theta}), \ \forall \mathfrak{g} \in \mathbb{G}, \tag{5.1.20}$$

donde «$\sim$» indica que dos características pertenecen a la misma clase de equivalencia. Aunque para garantizar la invariancia o la equivariancia, los operadores convolucionales han sido una práctica común en las redes profundas [CW16c], sigue siendo un reto en la práctica entrenar una red convolucional (diseñada empíricamente) desde cero que pueda *garantizar* la invariancia incluso ante transformaciones simples como la traslación y la rotación [AW18; ETT+17]. Un enfoque alternativo consiste en diseñar cuidadosamente los filtros de convolución de cada capa para garantizar la invariancia traslacional en una amplia gama de señales, por ejemplo, utilizando ondículas como en ScatteringNet [BM13] y trabajos posteriores [WB18]. Sin embargo, para garantizar la invariancia frente a señales genéricas, el número de convoluciones necesarias suele crecer exponencialmente con la profundidad de la red. Esa es la razón por la que este tipo de red no puede construirse con tanta profundidad, sino que suele tener solo unas pocas capas.

Ahora mostraremos que el principio MCR^2 es compatible con la invariancia de una manera natural y precisa: solo necesitamos asignar todas las versiones transformadas $\{\boldsymbol{x} \circ \mathfrak{g} \mid \mathfrak{g} \in \mathbb{G}\}$ a la misma clase que los datos $\boldsymbol{x}$ y mapear cada característica $\boldsymbol{z}$ al mismo subespacio $\mathcal{S}$. Por lo tanto, toda la información equivariante respecto al grupo se codifica únicamente dentro del subespacio, y cualquier clasificador definido sobre el conjunto resultante de subespacios será automáticamente invariante respecto a dichas transformaciones de grupo. Véase la Figura 5.6 para una ilustración de los ejemplos de rotación en 1D y traslación en 2D. A continuación, demostraremos rigurosamente que, cuando el grupo $\mathbb{G}$ realiza desplazamientos circulares en una dimensión, la red profunda resultante se convierte naturalmente en una *red convolucional multicanal*. Dado que la red así construida solo necesita garantizar la invariancia para los datos de entrada $\boldsymbol{X}$ o sus características $\boldsymbol{Z}$, el número de convoluciones necesarias se mantiene, de hecho, constante a lo largo de una red muy profunda, a diferencia de lo que ocurre con ScatteringNet.

Datos en serie unidimensionales e invariancia de traslación Para clasificar datos unidimensionales $\boldsymbol{x} = [x(0), x(1), \ldots, x(D-1)] \in \mathbb{R}^D$ invariantes ante traslación, tomamos $\mathbb{G}$ como el grupo de todas las traslaciones circulares. Cada observación $\boldsymbol{x}_i$ genera una familia $\{\boldsymbol{x}_i \circ \mathfrak{g} \,|\, \mathfrak{g} \in \mathbb{G}\}$ de copias desplazadas,

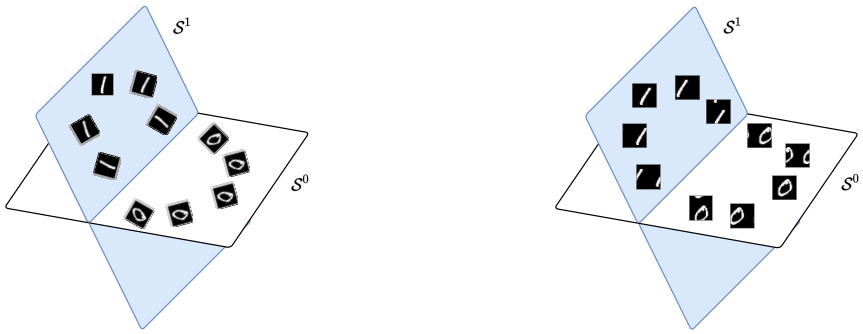


Figura 5.6: Ilustración de la representación buscada que es equivariante/invariante ante la rotación de la imagen (izquierda) o la traslación (derecha): todas las imágenes transformadas de cada clase se mapean al mismo subespacio, que es incoherente con respecto a otros subespacios. Las características incrustadas en cada subespacio son equivariantes ante las transformaciones del grupo, mientras que cada subespacio es invariante a dichas transformaciones.

que son las columnas de la matriz circulante $\mathsf{circ}(\boldsymbol{x}_i) \in \mathbb{R}^{D\times D}$ dada por

$$\mathsf{circ}(\boldsymbol{x}) \doteq \begin{bmatrix} x(0) & x(D-1) & \dots & x(2) & x(1) \\ x(1) & x(0) & x(D-1) & \cdots & x(2) \\ \vdots & x(1) & x(0) & \ddots & \vdots \\ x(D-2) & \vdots & \ddots & \ddots & x(D-1) \\ x(D-1) & x(D-2) & \dots & x(1) & x(0) \end{bmatrix} \in \mathbb{R}^{D\times D}. \tag{5.1.21}$$

Remitimos al lector a [KS12] para conocer las propiedades de las matrices circulares. Para simplificar, sea $\boldsymbol{Z}^1 \doteq [\boldsymbol{z}_1^1, \dots, \boldsymbol{z}_N^1] = \boldsymbol{X} \in \mathbb{R}^{d\times N}$.[9] Entonces, ¿qué ocurre si construimos la ReduNet a partir de sus familias circulantes $\mathsf{circ}(\boldsymbol{Z}^1) = \big[\mathsf{circ}(\boldsymbol{z}_1^1), \dots, \mathsf{circ}(\boldsymbol{z}_N^1)\big] \in \mathbb{R}^{d\times(dN)}$? Es decir, nuestro objetivo es comprimir y proyectar todas estas matrices sobre un mismo subespacio mediante la ReduNet.

Obsérvese que ahora la matriz de covarianza de los datos:

$$\begin{aligned} \mathsf{circ}(\boldsymbol{Z}^1)\mathsf{circ}(\boldsymbol{Z}^1)^\top &= \big[\mathsf{circ}(\boldsymbol{z}_1^1), \dots, \mathsf{circ}(\boldsymbol{z}_N^1)\big]\big[\mathsf{circ}(\boldsymbol{z}_1^1), \dots, \mathsf{circ}(\boldsymbol{z}_N^1)\big]^\top \\ &= \sum_{i=1}^{N} \mathsf{circ}(\boldsymbol{z}_i^1)\mathsf{circ}(\boldsymbol{z}_i^1)^\top \in \mathbb{R}^{d\times d} \end{aligned} \tag{5.1.22}$$

asociada a esta familia de muestras es *de manera automática* una matriz circulante (simétrica). Además, dado que la propiedad circulante se

[9] Una vez más, para simplificar el análisis, supongamos por ahora que las características iniciales $\boldsymbol{Z}^1$ coinciden con el propio $\boldsymbol{X}$, de modo que comparten la misma dimensión d, es decir, $D = d$. No obstante eso no tiene por qué ser así, ya que pronto veremos que es necesario elevar $\boldsymbol{X}$ a una dimensión superior.

conserva bajo sumas, inversas y productos, las matrices $\boldsymbol{E}^1$ y $\boldsymbol{C}_k^1$ son también automáticamente matrices circulantes, cuya aplicación a un vector de características $\boldsymbol{z} \in \mathbb{R}^d$ puede implementarse utilizando la convolución circular "$\circledast$". Específicamente, tenemos la siguiente proposición.

Proposición 5.1 (Estructuras convolucionales de $\boldsymbol{E}^1$ y $\boldsymbol{C}_k^1$)**.** *La matriz*

$$\boldsymbol{E}^1 = \alpha\big(\boldsymbol{I} + \alpha \mathsf{circ}(\boldsymbol{Z}^1)\mathsf{circ}(\boldsymbol{Z}^1)^\top\big)^{-1} \tag{5.1.23}$$

es una matriz circulante y representa una convolución circular:

$$\boldsymbol{E}^1\boldsymbol{z} = \boldsymbol{e}_1 \circledast \boldsymbol{z},$$

donde $\boldsymbol{e}_1 \in \mathbb{R}^d$ es el primer vector columna de $\boldsymbol{E}^1$ y «$\circledast$» es la convolución circular, definida como

$$(\boldsymbol{e}_1 \circledast \boldsymbol{z})_i \doteq \sum_{j=0}^{d-1} e_1(j)x(i+d-j \ \mathit{mod}\ d).$$

Del mismo modo, las matrices $\boldsymbol{C}_k^1$ asociadas a cualquier subconjunto de $\boldsymbol{Z}^1$ también son convoluciones circulares.

No solo los parámetros de la primera capa $\boldsymbol{E}^1$ y $\boldsymbol{C}_k^1$ de la ReduNet adquieren la forma de convoluciones circulantes, sino que también las características de la siguiente capa siguen siendo matrices circulantes. Es decir, la transformación incremental de características en (5.1.15) aplicada a todas las versiones desplazadas de un $\boldsymbol{z}^1 \in \mathbb{R}^d$, dada por

$$\mathsf{circ}(\boldsymbol{z}^1) + \eta \cdot \boldsymbol{E}^1\mathsf{circ}(\boldsymbol{z}^1) - \eta \cdot \boldsymbol{\sigma}\Big([\boldsymbol{C}_1^1\mathsf{circ}(\boldsymbol{z}^1), \ldots, \boldsymbol{C}_K^1\mathsf{circ}(\boldsymbol{z}^1)]\Big), \tag{5.1.24}$$

es una matriz circulante. Esto implica que no es necesario construir familias circulantes a partir de las características de la segunda capa, como lo hicimos para la primera capa. Al denotar

$$\boldsymbol{z}^2 \propto \boldsymbol{z}^1 + \eta \cdot g(\boldsymbol{z}^1, \boldsymbol{\theta}^1) = \boldsymbol{z}^1 + \eta \cdot \boldsymbol{e}_1 \circledast \boldsymbol{z}^1 - \eta \cdot \boldsymbol{\sigma}\Big([\boldsymbol{c}_1^1 \circledast \boldsymbol{z}^1, \ldots, \boldsymbol{c}_K^1 \circledast \boldsymbol{z}^1]\Big), \tag{5.1.25}$$

las características del siguiente nivel se pueden escribir como

$$\mathsf{circ}(\boldsymbol{Z}^2) = \big[\mathsf{circ}(\boldsymbol{z}_1^1 + \eta g(\boldsymbol{z}_1^1, \boldsymbol{\theta}^1)), \ldots, \mathsf{circ}(\boldsymbol{z}_N^1 + \eta g(\boldsymbol{z}_N^1, \boldsymbol{\theta}^1))\big].$$

Siguiendo con el razonamiento inductivo, vemos que todas las matrices $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ basadas en dicho $\mathsf{circ}(\boldsymbol{Z}^\ell)$ son circulantes, y lo mismo ocurre con todas las características. Gracias a las propiedades de los datos, ReduNet ha adoptado la forma de una red convolucional, *¡sin necesidad de elegir explícitamente esta estructura!*

Un compromiso fundamental entre invariancia y esparcidad. Sin embargo, hay un problema: en general, el conjunto de todas las permutaciones circulares de un vector $\boldsymbol{z}$ da lugar a una matriz de rango completo. Es decir, las d características «augmentadas» asociadas a cada muestra (y, por lo tanto, a cada clase) suelen abarcar ya todo el espacio $\mathbb{R}^d$. Por ejemplo, todas las versiones desplazadas de una función delta $\delta(d)$ pueden generar cualquier otra señal como su superposición ponderada (densa). El objetivo MCR^2 (4.2.15) no podrá distinguir las clases como subespacios diferentes.

Un remedio natural consiste en mejorar la separabilidad de los datos «elevando» la señal original a un espacio de mayor dimensión, por ejemplo, tomando sus respuestas a múltiples filtros $\boldsymbol{k}_1, \ldots, \boldsymbol{k}_C \in \mathbb{R}^d$:

$$\boldsymbol{z}[c] = \boldsymbol{k}_c \circledast \boldsymbol{x} = \mathsf{circ}(\boldsymbol{k}_c)\boldsymbol{x} \in \mathbb{R}^d, \quad c = 1, \ldots, C. \tag{5.1.26}$$

Los filtros pueden ser filtros prediseñados que promueven la invariancia[10] o aprendidos en forma adaptativa a partir de los datos[11] o seleccionados al azar, tal como lo hacemos en nuestros experimentos. Esta operación eleva cada señal original $\boldsymbol{x} \in \mathbb{R}^d$ a una característica de C canales, denotada como $\bar{\boldsymbol{z}} \doteq [\boldsymbol{z}[1], \ldots, \boldsymbol{z}[C]]^\top \in \mathbb{R}^{C \times d}$. Entonces, podemos construir la ReduNet sobre representaciones vectoriales de $\bar{\boldsymbol{z}}$, denotadas como $\vec{(\bar{\boldsymbol{z}})} \doteq [\boldsymbol{z}[1]^\top, \ldots, \boldsymbol{z}[C]^\top] \in \mathbb{R}^{dC}$. La versión circular asociada $\mathsf{circ}(\bar{\boldsymbol{z}})$ y su matriz de covarianza de los datos, denotada como $\bar{\boldsymbol{\Sigma}}(\bar{\boldsymbol{z}})$, para todas sus versiones desplazadas se definen como:

$$\mathsf{circ}(\bar{\boldsymbol{z}}) \doteq \begin{bmatrix} \mathsf{circ}(\boldsymbol{z}[1]) \\ \vdots \\ \mathsf{circ}(\boldsymbol{z}[C]) \end{bmatrix} \in \mathbb{R}^{dC \times d}, \tag{5.1.27}$$

$$\bar{\boldsymbol{\Sigma}}(\bar{\boldsymbol{z}}) \doteq \begin{bmatrix} \mathsf{circ}(\boldsymbol{z}[1]) \\ \vdots \\ \mathsf{circ}(\boldsymbol{z}[C]) \end{bmatrix} \begin{bmatrix} \mathsf{circ}(\boldsymbol{z}[1])^\top, \ldots, \mathsf{circ}(\boldsymbol{z}[C])^\top \end{bmatrix} \in \mathbb{R}^{dC \times dC}, \tag{5.1.28}$$

donde $\mathsf{circ}(\boldsymbol{z}[c]) \in \mathbb{R}^{d \times d}$ con $c \in [C]$ es la versión circular del canal c-ésimo de la característica $\bar{\boldsymbol{z}}$. Entonces, las columnas de $\mathsf{circ}(\bar{\boldsymbol{z}})$ solo generarán, como máximo, un subespacio propio de dimensión d en $\mathbb{R}^{dC}$. Sin embargo, esta simple operación de elevación (si es lineal) no basta aún para que las clases sean separables: las características asociadas a otras clases generarán el *mismo* subespacio de dimensión d. Esto refleja un conflicto fundamental entre la invariancia y el modelado lineal (de subespacios): *no se puede esperar que señales desplazadas y superpuestas arbitrariamente pertenezcan a la misma clase.*

[10] Para señales unidimensionales, como el audio, se puede recurrir a la transformada de Fourier de corto plazo (STFT) convencional; para imágenes bidimensionales, se puede recurrir a las ondas bidimensionales, como en ScatteringNet [BM13].

[11] En el caso de los filtros aprendidos, estos pueden aprenderse como componentes principales de las muestras, tal como en PCANet [CJG+15], o a partir del aprendizaje de diccionarios de convolución [LB19; QLZ19].

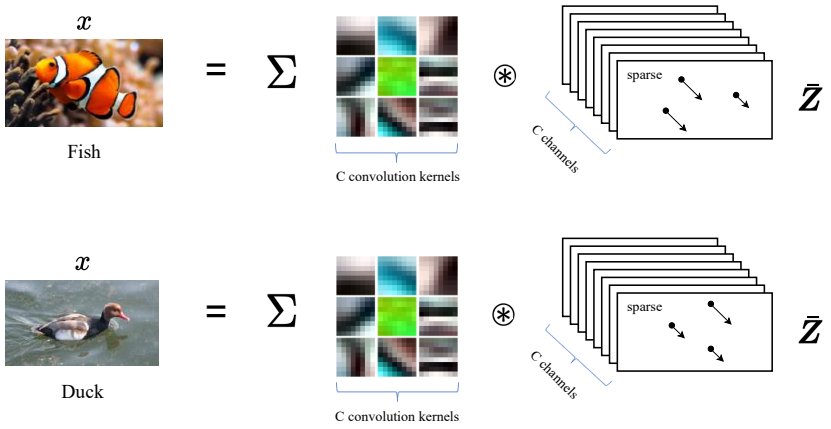


Figura 5.7: Cada señal de entrada $\boldsymbol{x}$ (en este caso, una imagen) puede representarse como una superposición de convoluciones esparsas con múltiples núcleos $\boldsymbol{d}_c$ en un diccionario $\boldsymbol{D}$.

Una manera de resolver este conflicto consiste en aprovechar una estructura adicional dentro de cada clase, en forma de *esparcidad*: las señales dentro de cada clase no se generan como una superposición lineal arbitraria de ciertos átomos (o motivos) básicos, sino únicamente como combinaciones *esparsas* de estos y sus versiones desplazadas, tal como se muestra en la Figura 5.7. Más concretamente, sea $\boldsymbol{D}_k = [\boldsymbol{d}_{k,1}, \ldots, \boldsymbol{d}_{k,c}]$ una matriz con un conjunto de átomos asociados a la clase k, también conocido como diccionario; entonces, cada señal $\boldsymbol{x}$ de esta clase se genera en forma esparsas como:

$$\boldsymbol{x} = \boldsymbol{d}_{k,1} \circledast z_1 + \ldots + \boldsymbol{d}_{k,c} \circledast z_c = \mathsf{circ}(\boldsymbol{D}_k)\boldsymbol{z}, \tag{5.1.29}$$

para algún vector esparso $\boldsymbol{z}$. Las señales de las diferentes clases se generan entonces mediante diferentes diccionarios cuyos átomos (o motivos) son incoherentes entre sí. Debido a esta incoherencia, es poco probable que las señales de una clase sean representadas en forma esparsa por átomos de cualquier otra clase. Por lo tanto, todas las señales pueden representarse como

$$\boldsymbol{x} = \big[\mathsf{circ}(\boldsymbol{D}_1), \mathsf{circ}(\boldsymbol{D}_2), \ldots, \mathsf{circ}(\boldsymbol{D}_K)\big]\bar{\boldsymbol{z}}, \tag{5.1.30}$$

donde $\bar{\boldsymbol{z}}$ es un vector esparso.[12] Existe una amplia bibliografía sobre cómo aprender los diccionarios de esparcidad más compactos y óptimos a partir de datos de muestra, por ejemplo, [LB19; QLZ19], y posteriormente resolver el problema inverso y calcular el código esparso asociado $\boldsymbol{z}$ o $\bar{\boldsymbol{z}}$. Estudios recientes

[12] Cabe señalar que desde hace tiempo se han propuesto y utilizado modelos similares de representación esparsa con fines de clasificación en aplicaciones como el reconocimiento facial, demostrando una eficacia excelente [WWG+12; WYG+09]. Recientemente, el modelo de codificación esparsa convolucional ha sido propuesto por [PRE17] como un marco para interpretar las estructuras de los de las redes convolucionales profundas.

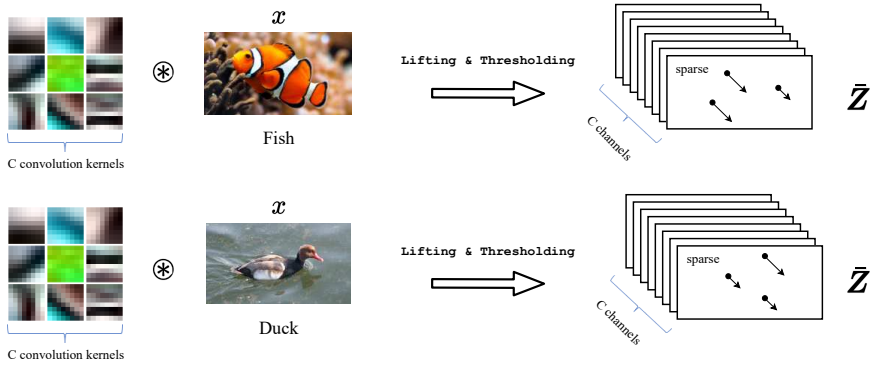


Figura 5.8: Estimación del código esparso $\bar{\boldsymbol{z}}$ de una señal de entrada $\boldsymbol{x}$ (en este caso, una imagen) mediante convoluciones con varios núcleos $\boldsymbol{k}_c$ y, a continuación, la esparcificación.

de [QLZ20; QZL+20a] incluso demuestran que, en condiciones generales, el problema del aprendizaje de diccionarios de convolución puede resolverse de manera eficaz y eficiente.

No obstante, para tareas como la clasificación, no nos interesa necesariamente el diccionario óptimo exacto ni el código esparso exacto para cada señal individual. Lo que nos interesa principalmente es que, en conjunto, el conjunto de códigos esparsos de cada clase sea suficientemente separable del de las demás clases. Bajo el supuesto del modelo generativo esparso, si los núcleos de convolución $\{\boldsymbol{k}_c\}_{c=1}^C$ se ajustan bien a la «transpuesta» o «inversa» de los diccionarios de esparcidad anteriores $\boldsymbol{D} = [\boldsymbol{D}_1, \dots, \boldsymbol{D}_K]$, también conocidos como los *filtros de análisis* [NDE+13; RE14], las señales de una clase solo tendrán respuestas altas a un pequeño subconjunto de esos filtros y respuestas bajas a los demás (debido a la suposición de incoherencia). No obstante, en la práctica, a menudo basta con un número suficientemente grande de, digamos, C filtros aleatorios $\{\boldsymbol{k}_c\}_{c=1}^C$ para garantizar que las características extraídas de los C canales

$$\big[\boldsymbol{k}_1 \circledast \boldsymbol{x}, \boldsymbol{k}_2 \circledast \boldsymbol{x}, \dots, \boldsymbol{k}_C \circledast \boldsymbol{x}\big]^\top = \big[\mathsf{circ}(\boldsymbol{k}_1)\boldsymbol{x}, \dots, \mathsf{circ}(\boldsymbol{k}_C)\boldsymbol{x}\big]^\top \in \mathbb{R}^{C\times d} \quad (5.1.31)$$

para las diferentes clases presenten distintos patrones de respuesta ante distintos filtros, lo que permite separarlas [CJG+15].

Por lo tanto, en nuestro marco, en gran medida el número de canales (o el ancho de la red) desempeña realmente el papel de *recurso estadístico*, mientras que el número de capas (la profundidad de la red) desempeña el papel de *recurso computacional*. La teoría de la detección compresiva caracteriza con precisión cuántas mediciones se necesitan para preservar las estructuras intrínsecas de baja dimensión (incluida la separabilidad) de los datos [WM22].

Las respuestas multicanal $\bar{\boldsymbol{z}}$ deben ser esparsas. Por lo tanto, para aproximar el código esparso $\bar{\boldsymbol{z}}$, podemos aplicar un *umbral no lineal elemento por elemento que promueva la esparcidad*, por ejemplo, $\boldsymbol{\tau}(\cdot)$, a las salidas del filtro anterior,

estableciendo que las respuestas bajas (por ejemplo, con valor absoluto inferior a ϵ) o negativas sean cero:

$$\bar{\boldsymbol{z}} \doteq \boldsymbol{\tau}\left(\left[\mathsf{circ}(\boldsymbol{k}_1)\boldsymbol{x}, \ldots, \mathsf{circ}(\boldsymbol{k}_C)\boldsymbol{x}\right]^\top\right) \in \mathbb{R}^{C\times d}. \tag{5.1.32}$$

La Figura 5.8 ilustra las ideas básicas. Se puede consultar [RE14] para un estudio más sistemático sobre el diseño del operador de umbralización esparcificante. Sin embargo, aquí no nos interesa tanto obtener los mejores códigos esparsos, siempre y cuando los códigos sean suficientemente separables. Por lo tanto, el operador no lineal $\boldsymbol{\tau}$ puede elegirse simplemente como un umbral blando o una función ReLU. Se puede suponer que estas características presumiblemente esparsas $\bar{\boldsymbol{z}}$ se encuentran en una subvariedad (no lineal) de dimensión inferior de $\mathbb{R}^{dC}$, la cual puede linealizarse y separarse de las otras clases mediante capas ReduNet posteriores, como se ilustra más adelante en la Figura 5.9.

La red ReduNet construida a partir de la versión circulante de estas características multicanal $\bar{\boldsymbol{Z}} \doteq [\bar{\boldsymbol{z}}_1, \ldots, \bar{\boldsymbol{z}}_N] \in \mathbb{R}^{C\times d\times N}$, es decir, $\mathsf{circ}(\bar{\boldsymbol{Z}}) \doteq [\mathsf{circ}(\bar{\boldsymbol{z}}_1), \ldots, \mathsf{circ}(\bar{\boldsymbol{z}}_N)] \in \mathbb{R}^{dC\times dN}$, conserva las buenas propiedades de invariancia descritas anteriormente: los operadores lineales, ahora denominados $\bar{\boldsymbol{E}}$ y $\bar{\boldsymbol{C}}_k$, siguen siendo circulantes por bloques, y representan convoluciones circulares unidimensionales multicanal. Concretamente, obtenemos el siguiente resultado.

Proposición 5.2 (Estructuras de convolución multicanal de $\bar{\boldsymbol{E}}$ y $\bar{\boldsymbol{C}}_k$)**.** *La matriz*

$$\bar{\boldsymbol{E}} \doteq \alpha\left(\boldsymbol{I} + \alpha\,\mathsf{circ}(\bar{\boldsymbol{Z}})\mathsf{circ}(\bar{\boldsymbol{Z}})^\top\right)^{-1} \tag{5.1.33}$$

es circulante por bloques, es decir,

$$\bar{\boldsymbol{E}} = \begin{bmatrix} \bar{\boldsymbol{E}}_{1,1} & \cdots & \bar{\boldsymbol{E}}_{1,C} \\ \vdots & \ddots & \vdots \\ \bar{\boldsymbol{E}}_{C,1} & \cdots & \bar{\boldsymbol{E}}_{C,C} \end{bmatrix} \in \mathbb{R}^{dC\times dC}, \tag{5.1.34}$$

donde cada $\bar{\boldsymbol{E}}_{c,c'} \in \mathbb{R}^{d\times d}$ es una matriz circulante. Además, $\bar{\boldsymbol{E}}$ representa una convolución circular multicanal, es decir, para cualquier señal multicanal $\bar{\boldsymbol{z}} \in \mathbb{R}^{C\times n}$ tenemos

$$\bar{\boldsymbol{E}} \cdot \mathsf{vec}(\bar{\boldsymbol{z}}) = \mathsf{vec}(\bar{\boldsymbol{e}} \circledast \bar{\boldsymbol{z}}). \tag{5.1.35}$$

En la expresión anterior, $\bar{\boldsymbol{e}} \in \mathbb{R}^{C\times C\times d}$ es un núcleo convolucional multicanal donde $\bar{\boldsymbol{e}}[c, c'] \in \mathbb{R}^d$ es el primer vector columna de $\bar{\boldsymbol{E}}_{c,c'}$ y $\bar{\boldsymbol{e}} \circledast \bar{\boldsymbol{z}} \in \mathbb{R}^{C\times d}$ es la convolución circular multicanal definida como

$$(\bar{\boldsymbol{e}} \circledast \bar{\boldsymbol{z}})[c] \doteq \sum_{c'=1}^{C} \bar{\boldsymbol{e}}[c, c'] \circledast \bar{\boldsymbol{z}}[c'], \quad \forall c = 1, \ldots, C. \tag{5.1.36}$$

Del mismo modo, las matrices $\bar{\boldsymbol{C}}_k$ asociadas a cualquier subconjunto de $\bar{\boldsymbol{Z}}$ son también convoluciones circulares multicanal.

Según la Proposición 5.2, la ReduNet invariante ante traslaciones es, por su propia construcción, una red convolucional profunda para señales unidimensionales multicanal. Obsérvese que, incluso si los kernels de elevación iniciales están separados (5.1.32), la inversa de la matriz en (5.1.33) para calcular $\bar{\boldsymbol{E}}$ (de manera similar para $\bar{\boldsymbol{C}}_k$) introduce «interferencia cruzada» entre todos los C canales. Por lo tanto, estas convoluciones multicanal, en general, *no* son separables en profundidad, a diferencia de las redes Xception [Cho17] que se propusieron en su momento para simplificar las redes neuronales convolucionales multicanal.[13]

Nota 5.2 (Reducción de la complejidad computacional en el dominio de frecuencias). El cálculo de $\bar{\boldsymbol{E}}$ en (5.1.33) requiere invertir una matriz de tamaño $dC \times dC$, lo que en general tiene una complejidad de $O(d^3C^3)$. Sin embargo, al aprovechar el hecho de que una matriz circulante puede diagonalizarse mediante la matriz de la Transformada de Fourier Discreta (DFT), la complejidad puede reducirse significativamente. Como se muestra en [CYY+22], para calcular $\bar{\boldsymbol{E}}$ y $\bar{\boldsymbol{C}}_k \in \mathbb{R}^{dC \times dC}$, solo necesitamos calcular en el dominio de la frecuencia la inversa de d bloques de tamaño $C \times C$, por lo que la complejidad total es de $O(dC^3)$.

Arquitectura general de la red y comparación. A partir de la derivación anterior, observamos que, para encontrar una representación discriminativa lineal (LDR) para múltiples clases de señales o imágenes que sea invariante ante traslaciones, la codificación esparsa, una arquitectura multicapa con convoluciones multicanal, diferentes funciones de activación no lineales y el cálculo del espectro se convierten en componentes *necesarias* para alcanzar el objetivo de manera eficaz y eficiente. La Figura 5.9 ilustra el proceso general de aprendizaje de dicha representación mediante la reducción invariante de tasas sobre los códigos esparsos de entrada.

Ejemplo 5.2 (Clasificación invariante de dígitos). A continuación, presentamos los resultados empíricos de ReduNet en el aprendizaje de características invariantes a la *rotación* en el conjunto de datos real MNIST de 10 clases. Imponemos una cuadrícula polar sobre la imagen $\boldsymbol{x} \in \mathbb{R}^{H \times W}$, cuyo centro geométrico coincide con el centro de la cuadrícula polar 2D (tal como se ilustra en la Figura 5.10). Para cada radio r_i, $i \in [C]$, podemos muestrear Γ píxeles con respecto a cada ángulo $\gamma_l = l \cdot (2\pi/\Gamma)$ con $l \in [\Gamma]$. Entonces, dada una imagen de muestra $\boldsymbol{x}$ del conjunto de datos, representamos la imagen en coordenadas polares (muestreadas) como una señal multicanal $\boldsymbol{x}_p \in \mathbb{R}^{\Gamma \times C}$. El objetivo aquí es aprender una representación invariante ante rotaciones, es decir, esperamos aprender $f(\cdot, \theta)$ tal que $\{f(\boldsymbol{x}_p \circ \mathfrak{g}, \theta)\}_{\mathfrak{g} \in \mathbb{G}}$ se encuentren en el mismo subespacio, donde $\mathfrak{g}$ es el desplazamiento cíclico en el ángulo polar. Utilizamos $N = 100$ muestras de entrenamiento (10 de cada clase) y establecemos $\Gamma = 200$, $C = 15$ para el muestreo polar. Al realizar el muestreo anterior en coordenadas polares, podemos obtener la matriz de datos $\boldsymbol{X}_p \in \mathbb{R}^{(\Gamma \cdot C) \times N}$. Para ReduNet, establecemos el número de capas/iteraciones $L = 40$, la precisión $\epsilon = 0.1$ y el

[13] Queda por determinar qué estructuras adicionales en los datos darían lugar a convoluciones separables en profundidad.

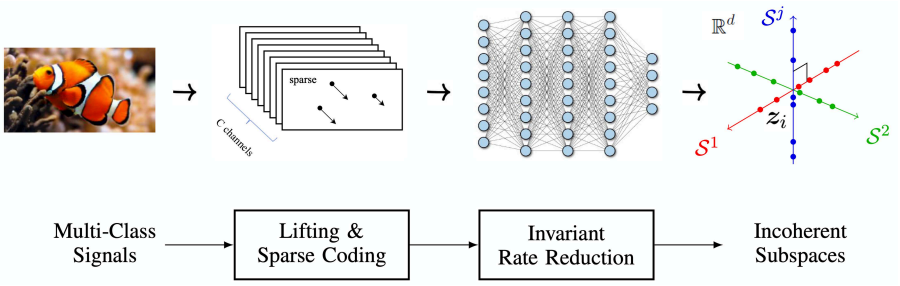


Figura 5.9: El proceso general para clasificar señales multiclase con invariancia al desplazamiento: elevación multicanal, codificación esparsa, seguida de una ReduNet de convolución multicanal para la reducción de tasa invariante. Estas componentes son *necesarios* para mapear señales multiclase invariantes al desplazamiento a subespacios incoherentes (lineales) como una LDR. Cabe señalar que las arquitecturas de la mayoría de las redes neuronales modernas de gran profundidad se asemejan a este proceso. El LDR aprendido de este modo facilita tareas posteriores, como la clasificación.

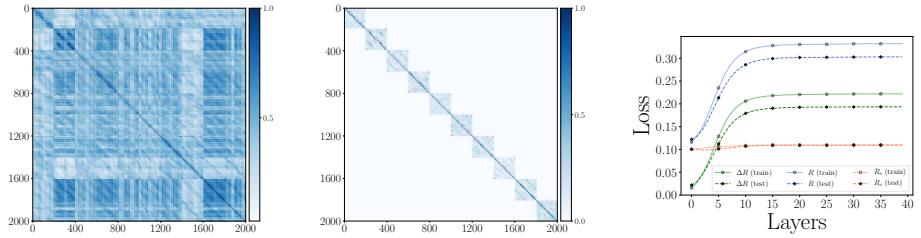
tamaño de paso $\eta = 0.5$. Antes de la primera capa, realizamos una elevación de la entrada mediante una convolución circulante unidimensional con 20 núcleos gaussianos aleatorios de tamaño 5.

Para evaluar la representación aprendida, cada muestra de entrenamiento se aumenta con 20 versiones rotadas de sí misma, cada una desplazada con un paso de 10. Calculamos las similitudes coseno entre $m \times 20$ entradas de entrenamiento augmentadas $\boldsymbol{X}_{\text{rotation}}$ y los resultados se muestran en la Figura 5.11 (**a**). Comparamos las similitudes coseno entre las características aprendidas de todas las versiones aumentadas, es decir, $\bar{\boldsymbol{Z}}_{\text{rotation}}$ y resumimos los resultados en la Figura 5.11 (**b**). Como vemos, la ReduNet invariante a la rotación así construida es capaz de mapear los datos de entrenamiento (así como todas sus versiones rotadas) de las 10 clases diferentes en 10 subespacios casi ortogonales. Es decir, los subespacios aprendidos son verdaderamente invariantes ante la transformación de desplazamiento en ángulo polar. A continuación, extraemos al azar otras 100 muestras de prueba y aplicamos el mismo procedimiento de augmentación. En la Figura 5.11 (**c**), visualizamos la pérdida MCR^2 en la representación de la capa ℓ de la ReduNet sobre los conjuntos de datos de entrenamiento y de prueba. A partir de estos resultados, podemos observar que la ReduNet construida es efectivamente capaz de maximizar la pérdida MCR^2, así como de generalizar a los datos de prueba.

■

(a) $\boldsymbol{X}_{\text{rotación}}$ (b) $\boldsymbol{Z}_{\text{rotación}}$ (c) Pérdida

Figura 5.10: Ejemplos de imágenes de los dígitos MNIST, giradas en 18° cada una. (**Izquierda**) Diagrama de la representación en coordenadas polares; (**Derecha**) Imágenes giradas de los dígitos «0» y «1».



(a) $\boldsymbol{X}_{\text{rotación}}$ (b) $\boldsymbol{Z}_{\text{rotación}}$ (c) Pérdida

Figura 5.11: (a) y (b) son mapas de calor de la similitud coseno entre los datos de entrenamiento rotados $\boldsymbol{X}_{\text{rotation}}$ y las características aprendidas $\bar{\boldsymbol{Z}}_{\text{rotation}}$ para la invariancia a la rotación. (d) muestra las pérdidas MCR^2 de entrenamiento y validación por capas.

5.2 Transformadores transparentes (white-box) mediante optimización desenrollada

Como hemos visto en la sección anterior, utilizamos el problema de la clasificación para ofrecer una interpretación rigurosa de las principales características arquitectónicas de redes profundas populares como ResNet y las CNN: cada capa de estas redes puede considerarse como una imitación de un paso de gradiente que aumenta el objetivo de reducción de tasa (o ganancia de información). Esta perspectiva también lleva a un hecho algo sorprendente: los parámetros y operadores de las capas de una red tan profunda, la ReduNet, pueden calcularse en forma puramente directa.

A pesar de la importancia teórica y conceptual de ReduNet, hay varios factores que limitan su aplicabilidad práctica. En primer lugar, como hemos comentado anteriormente, el costo computacional de calcular los operadores matriciales en cada capa en forma directa puede ser muy elevado. En segundo lugar, es posible que los operadores así calculados sean menos efectivos a la

hora de optimizar el objetivo y que se requieran miles de iteraciones (y, por lo tanto, de capas). Como hemos visto en la Sección 2.3.3 para LISTA, estos dos problemas pueden resolverse permitiendo optimizar esos operadores y haciendo que sean aprendibles mediante retropropagación.[14]

El contexto de clasificación supervisada en el que se derivó ReduNet también presenta ciertas limitaciones. En la práctica, es posible que una imagen no pertenezca a una sola clase, ya que puede contener varios objetos. Por lo tanto, sería más general suponer que las diferentes regiones de la imagen pertenecen a diferentes modelos de baja dimensión (por ejemplo, una distribución gaussiana o un subespacio). Como veremos, tal generalización daría lugar a una arquitectura sencilla y general que unifica las operaciones de reducción de tasa y de eliminación de ruido que hemos visto en el capítulo anterior. Además, la arquitectura resultante se asemeja a la popular arquitectura de transformer.

5.2.1 Optimización desenrollada para la reducción esparsa de la tasa (de codificación)

En los últimos años, a medida que la cantidad y la calidad de los datos disponibles para entrenar redes profundas han aumentado de manera espectacular, el campo ha pasado de considerar las entradas como «átomos» a considerarlas como «secuencias de tokens», en las que cada token forma parte de la entrada global. Este cambio de perspectiva ofrece ventajas tanto filosóficas como prácticas: desde el punto de vista filosófico, nos permite medir y modelar cada parte de la entrada y sus interacciones; desde el punto de vista práctico, nos permite utilizar redes profundas similares, *modelos secuencia-a-secuencia* como los omnipresentes transformers, para aprender todo tipo de distribuciones de datos. Ahora nos corresponde comprender el aprendizaje de representaciones en el modelo «centrado en tokens» de datos como secuencias de tokens, y hacer frente a los retos mencionados anteriormente (como, por ejemplo, en el contexto del marco de reducción de tasa, la falta de etiquetas para cada token).

Formalmente, sea $\boldsymbol{X} = \left[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\right] \in \mathbb{R}^{D \times N}$ un conjunto de variables aleatorias que representan nuestra fuente de datos. En tareas de visión, cada $\boldsymbol{x}_i \in \mathbb{R}^D$ se interpreta como un *token*, que suele corresponder a un fragmento de imagen. En tareas de lenguaje, cada $\boldsymbol{x}_i \in \mathbb{R}^D$ se interpreta como una *incrustación de token*, es decir, una representación vectorial continua de un token discreto, como una palabra o una subpalabra.[15] Los $\boldsymbol{x}_i$ pueden tener estructuras de correlación arbitrarias. Utilizamos $\boldsymbol{Z} = \left[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N\right] \in \mathbb{R}^{d \times N}$ para denotar las variables aleatorias que definen nuestras representaciones, donde $\boldsymbol{z}_i \in \mathbb{R}^d$ es la representación del token correspondiente $\boldsymbol{x}_i \in \mathbb{R}^D$.

Nota 5.3. En los transformers, cada muestra de entrada se convierte habitualmente en una secuencia de *tokens*. Un token es una unidad básica

[14] O, tal vez, mediante una combinación de optimización hacia adelante y hacia atrás.

[15] Con un ligero desvío terminológico, a lo largo de este capítulo nos referiremos tanto a los tokens discretos como a sus *embeddings* asociados simplemente como «tokens» por razones de conveniencia.

de información derivada de la entrada bruta: en el procesamiento del lenguaje natural, los tokens suelen ser palabras o subpalabras; en la visión artificial, corresponden a fragmentos de imagen; y en otras modalidades, pueden representar pasos de tiempo, ubicaciones espaciales u otras unidades específicas del dominio. Un *embedding de token* es una representación vectorial continua de un token que sirve como entrada para un transformer. Asigna a cada token un punto en un espacio de alta dimensión, lo que permite al modelo procesar entradas simbólicas mediante cálculos numéricos. Una *representación de token* es un vector que codifica la información semántica o estructural de un token, y suele ser generada por las capas intermedias o finales de un transformer. Estas representaciones están diseñadas para capturar características informativas de la entrada que resultan útiles para tareas posteriores, tales como la clasificación, la generación o la regresión. Véase la Sección 8.2 para obtener más detalles sobre este vocabulario y cómo se relaciona con las implementaciones.

Objetivo para el aprendizaje de una representación estructurada y compacta. Inspirándonos en el análisis previo sobre la reducción de tasa (Sección 5.1), proponemos que el objetivo del aprendizaje de representaciones en este modelo centrado en tokens para nuestros datos sea encontrar un mapeo de características $f\colon \boldsymbol{X} \in \mathbb{R}^{D\times N} \to \boldsymbol{Z} \in \mathbb{R}^{d\times N}$ que transforme los tokens de entrada $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^D$ con una distribución potencialmente no lineal y multimodal a representaciones de tokens *linealizadas (por tramos) y compactas* $\{\boldsymbol{z}_i\}_{i=1}^N \subset \mathbb{R}^d$. Si bien la distribución *conjunta* de las representaciones de los tokens $\{\boldsymbol{z}_i\}_{i=1}^N$ puede ser sofisticada (y específica para cada tarea), sostenemos además que es razonable y práctico exigir que la distribución *marginal* objetivo de las representaciones individuales de los tokens sea altamente comprimida y estructurada, lo que la hace apta para una codificación compacta. En particular, requerimos que la distribución sea *una mezcla de distribuciones gaussianas de baja dimensión (digamos K)* , de tal manera que la k-ésima gaussiana tenga media $\mathbf{0} \in \mathbb{R}^d$, covarianza $\boldsymbol{\Sigma}_k \succeq \mathbf{0} \in \mathbb{R}^{d\times d}$, y soporte generado por la base ortonormal $\boldsymbol{U}_k \in \mathbb{R}^{d\times p}$. Denotamos por $\boldsymbol{U}_{[K]} = \{\boldsymbol{U}_k\}_{k=1}^K$ el conjunto de bases de todas las distribuciones gaussianas.

Como es habitual, para aprender buenas representaciones, queremos maximizar la *ganancia de información* de las representaciones finales de los tokens, o más concretamente, maximizar una noción adecuada de la reducción de tasa (véase Sección 4.2.3), es decir,

$$\max_{\boldsymbol{Z}\in\mathbb{R}^{d\times N}} \Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}). \tag{5.2.1}$$

Aquí, el primer término R_ϵ es una estimación de la tasa de codificación con pérdida para el conjunto completo de representaciones de tokens. Más concretamente, si consideramos las representaciones de tokens $\{\boldsymbol{z}_i\}_{i=1}^N$ como muestras independientes e idénticamente distribuidas de una única distribución gaussiana de media cero, su tasa de codificación con pérdida, sujeta a una

precisión de cuantización $\epsilon > 0$, viene dada por

$$R_\epsilon(\boldsymbol{Z}) \doteq \frac{1}{2}\text{logdet}\left(\boldsymbol{I} + \frac{d}{N\epsilon^2}\boldsymbol{Z}^\top\boldsymbol{Z}\right) = \frac{1}{2}\text{logdet}\left(\boldsymbol{I} + \frac{d}{N\epsilon^2}\boldsymbol{Z}\boldsymbol{Z}^\top\right). \tag{5.2.2}$$

El segundo término R_ϵ^c es una estimación de la tasa de codificación con pérdida bajo el libro de códigos $\boldsymbol{U}_{[K]}$, que se expresa como

$$R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq \sum_{k=1}^{K} R_\epsilon(\boldsymbol{U}_k^\top\boldsymbol{Z}) = \frac{1}{2}\sum_{k=1}^{K}\log\det\left(\boldsymbol{I} + \frac{p}{N\epsilon^2}(\boldsymbol{U}_k^\top\boldsymbol{Z})^\top(\boldsymbol{U}_k^\top\boldsymbol{Z})\right). \tag{5.2.3}$$

Nota 5.4. La expresión (5.2.3) para la tasa de codificación puede considerarse como una generalización de la tasa de codificación R_ϵ^c utilizada en el objetivo de reducción de tasa original (4.2.16). En particular, el objetivo original se define con respecto a un conjunto de etiquetas de pertenencia conocidas $\{\boldsymbol{\Pi}_k\}$ específicas de la realización de datos particular de $\boldsymbol{X}$. Por el contrario, el objetivo actual se define con respecto a los subespacios $\boldsymbol{U}_{[K]}$, que son independientes de cualquier realización concreta, pero se supone que admiten la distribución de las representaciones de tokens. Supongamos que una representación de token $\boldsymbol{z}_i$ pertenece a un subespacio $\boldsymbol{U}_k$ y que estos subespacios son aproximadamente ortogonales entre sí, es decir, $\boldsymbol{U}_k^\top\boldsymbol{U}_l \approx \boldsymbol{0}$ para todo $k \neq l$. Entonces, se puede verificar que las proyecciones $\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{z}_i = \boldsymbol{z}_i$ y $\boldsymbol{U}_l\boldsymbol{U}_l^\top\boldsymbol{z}_i \approx \boldsymbol{0}$ para todo $l \neq k$. Estas proyecciones ortogonales sirven efectivamente como etiquetas de pertenencia implícitas, identificando el subespacio al que pertenece cada representación de token.

Nota 5.5. Cabe destacar que este conjunto de propiedades deseables para el aprendizaje de representaciones concuerda bien con dos hipótesis empíricas bien establecidas sobre la estructura de las representaciones de tokens en los modelos entrenados que utilizan la omnipresente arquitectura de redes neuronales de transformers: la «hipótesis de la representación lineal» [JRR+24; PCV24] y la «hipótesis de la superposición» [EHO+22a; YCO+21]. La hipótesis de la representación lineal postula que las representaciones de tokens en los modelos de transformer se encuentran en subespacios lineales de baja dimensión que codifican características semánticas. De manera similar, la hipótesis de la superposición sugiere que estas representaciones pueden expresarse, de manera aproximada, como una combinación lineal esparsa de estos vectores de características. Como veremos en la Figura 5.17 más adelante en este capítulo, la base del subespacio $\boldsymbol{U}_k$ puede interpretarse como un conjunto de características semánticas, donde cada característica corresponde a un aspecto específico del significado del token. Las representaciones de los tokens se expresan entonces, aproximadamente, como combinaciones lineales esparsas de estas bases del subespacio, capturando lss componentes semánticas esenciales del token e ignorando las dimensiones irrelevantes.

Reducción esparsa de la tasa de codificación Tenga en cuenta que el objetivo de reducción de tasa (5.2.1) es invariante ante rotaciones conjuntas

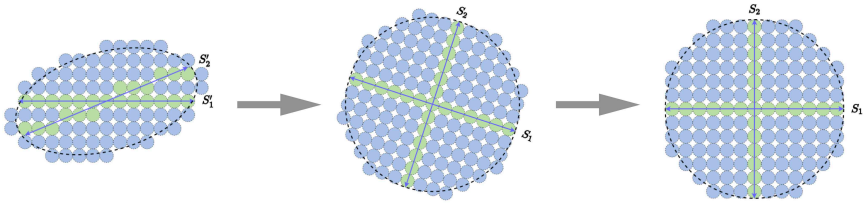


Figura 5.12: **Comparación de tres conjuntos de representaciones mediante la reducción de tasa y la esparcidad.** Cada S_i representa un subespacio lineal y el número de bolas azules representa la diferencia entre las tasas de codificación $\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) = R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]})$.

arbitrarias de las representaciones y los subespacios. En particular, la optimización del objetivo de reducción de tasa puede no conducir en forma natural a representaciones alineadas con los ejes (es decir, *esparsas*). Por ejemplo, considere los tres conjuntos de representaciones aprendidas en la Figura 5.12. La reducción de la tasa de codificación aumenta de (a) a (b), pero como es invariante ante rotaciones, se mantiene igual de (b) a (c). Por lo tanto, nos gustaría transformar las representaciones (y sus subespacios de soporte) de tal manera que las representaciones $\boldsymbol{Z}$ acaben siendo esparsas[16] con respecto a las coordenadas estándar del espacio de representaciones resultante como en la Figura 5.12(c). Desde el punto de vista computacional, podemos combinar los dos objetivos anteriores en un único objetivo de optimización:

$$\max_{f \in \mathcal{F}} \left[\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) - \lambda \|\boldsymbol{Z}\|_0\right] \qquad \text{s.t. } \boldsymbol{Z} = f(\boldsymbol{X}), \tag{5.2.4}$$

donde $\mathcal{F}$ denota una clase general de funciones y la norma ℓ_0 $\|\boldsymbol{Z}\|_0$ promueve la esparcidad de las representaciones finales de tokens $\boldsymbol{Z} = f(\boldsymbol{X})$.

En la práctica, la norma ℓ_0 suele sustituirse por la norma ℓ_1 para mejorar la manejabilidad computacional y permitir el uso de técnicas de optimización convexa [WM22]. Motivados por esto, relajamos el Problema (5.2.4) en consecuencia, lo que da lugar a una formulación que se mantiene fiel al objetivo de esparcidad original, al tiempo que resulta más apta para algoritmos eficientes, tal como se muestra a continuación:

$$\max_{f \in \mathcal{F}} \left[\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) - \lambda \|\boldsymbol{Z}\|_1\right] \qquad \text{s.t. } \boldsymbol{Z} = f(\boldsymbol{X}), \tag{5.2.5}$$

Aunque se trate de un ligero abuso de la terminología, a menudo también nos referimos a esta función objetivo como la *reducción esparsa de tasa* .

Arquitectura de red transparente *white-box* mediante optimización desenrollada Aunque resulte fácil de formular, la optimización de cada término del objetivo anterior plantea un desafío computacional [WM22]. Por lo tanto, es natural adoptar un enfoque de aproximación que realice la

[16] Concretamente, con pocas entradas distintas de cero.

transformación global f para optimizar (5.2.4) mediante una concatenación de múltiples, digamos L, operaciones simples *incrementales y locales* f^ℓ que empujan la distribución de la representación hacia la distribución deseada del modelo parsimonioso:

$$f\colon \boldsymbol{X} \xrightarrow{f^{\text{pre}}} \boldsymbol{Z}^1 \to \cdots \to \boldsymbol{Z}^\ell \xrightarrow{f^\ell} \boldsymbol{Z}^{\ell+1} \to \cdots \xrightarrow{f^L} \boldsymbol{Z}^{L+1} = \boldsymbol{Z}, \tag{5.2.6}$$

donde $f^{\text{pre}} : \mathbb{R}^D \to \mathbb{R}^d$ es el mapeo de preprocesamiento que transforma cada token de entrada $\boldsymbol{x}_i \in \mathbb{R}^D$ en las representaciones iniciales de los tokens $\boldsymbol{z}_i^1 \in \mathbb{R}^d$. Cada *mapeo incremental hacia adelante* $\boldsymbol{Z}^{\ell+1} = f^\ell(\boldsymbol{Z}^\ell)$, o una «capa», transforma la distribución de tokens para *optimizar* el objetivo de reducción esparsa de tasa mencionado anteriormente (5.2.4), condicionado a la distribución de su entrada $\boldsymbol{Z}^\ell$.

Nota 5.6. A diferencia de otros enfoques de optimización desenrollada, como el ReduNet (véase la Sección 5.1), aquí *modelamos explícitamente* la distribución de $\boldsymbol{Z}^\ell$ en cada capa, por ejemplo, como una mezcla de subespacios lineales o generada en forma esparsa a partir de un diccionario. Los parámetros del modelo se aprenden a partir de los datos (por ejemplo, mediante la *retropropagación* con entrenamiento de extremo a extremo). Esta separación entre la «optimización» hacia adelante y el «aprendizaje» hacia atrás aclara la función matemática de cada capa como un operador que transforma la distribución de su entrada, mientras que la distribución de la entrada, a su vez, es modelada (y posteriormente aprendida) por los parámetros de la capa.

Ahora mostramos cómo derivar estas operaciones incrementales y locales desde una perspectiva de optimización desenrollada para resolver el problema (5.2.5). Una vez que decidimos utilizar un enfoque incremental para optimizar el problema (5.2.5), existen diversas opciones posibles para lograr la optimización. Dado un modelo para $\boldsymbol{Z}^\ell$, por ejemplo, una mezcla de subespacios $\boldsymbol{U}_{[K]}$, optamos por un método de *minimización alternada* en dos pasos con una sólida base teórica. Primero, *comprimimos* los tokens $\boldsymbol{Z}^\ell$ mediante un descenso de gradiente para minimizar el término de tasa de codificación $R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}^\ell)$. Específicamente, tomamos un paso de gradiente en R_ϵ^c con una tasa de aprendizaje κ de la siguiente manera:

$$\boldsymbol{Z}^{\ell+1/2} = \boldsymbol{Z}^\ell - \kappa \nabla_{\boldsymbol{Z}} R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}^\ell). \tag{5.2.7}$$

A continuación, *esparsificamos* los tokens comprimidos, generando $\boldsymbol{Z}^{\ell+1}$ mediante un paso de gradiente proximal adecuadamente relajado para minimizar el término restante $\lambda\|\boldsymbol{Z}\|_1 - R_\epsilon(\boldsymbol{Z})$. Como argumentaremos en detalle más adelante, podemos encontrar dicho $\boldsymbol{Z}^{\ell+1}$ resolviendo un problema de representación esparsa con respecto a un diccionario ortogonal completo $\boldsymbol{D}^\ell$:

$$\boldsymbol{Z}^{\ell+1} = \arg\min_{\boldsymbol{Z}} \left\{ \lambda\|\boldsymbol{Z}\|_1 + \frac{1}{2}\|\boldsymbol{Z}^{\ell+1/2} - \boldsymbol{D}^\ell \boldsymbol{Z}\|_F^2 \right\}. \tag{5.2.8}$$

A continuación, presentamos los detalles técnicos de cada uno de los dos pasos anteriores y derivamos actualizaciones eficientes para su implementación.

La autoatención como descenso por gradiente en la tasa de codificación de las representaciones de tokens. En el primer paso (5.2.7), el gradiente de la tasa de codificación $\nabla_{\boldsymbol{Z}} R_\epsilon^c$ resulta costoso de calcular, ya que implica K inversas de matrices distintas, una para cada uno de los K subespacios con base $\boldsymbol{U}_k^\ell$:

$$\nabla_{\boldsymbol{Z}} R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) = \frac{p}{N\epsilon^2} \sum_{k=1}^{K} \boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{Z} \Big(\boldsymbol{I} + \frac{p}{N\epsilon^2} (\boldsymbol{U}_k^\top \boldsymbol{Z})^\top (\boldsymbol{U}_k^\top \boldsymbol{Z}) \Big)^{-1}. \quad (5.2.9)$$

Ahora demostramos que este gradiente puede aproximarse en forma natural utilizando el denominado operador de autoatención de subespacios de múltiples cabezas (MSSA), que tiene una forma funcional similar a la del operador de autoatención de múltiples cabezas [VSP+17b] con K cabezas (es decir, una para cada subespacio, procedente de cada inversa de matriz). Aquí, aproximamos el gradiente (5.2.9) utilizando la serie de Neumann de primer orden (véase Ejercicio 5.2):[17]

$$\nabla_{\boldsymbol{Z}} R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \quad (5.2.10)$$

$$\approx \frac{p}{N\epsilon^2} \sum_{k=1}^{K} \boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{Z} \Big(\boldsymbol{I} - \frac{p}{N\epsilon^2} (\boldsymbol{U}_k^\top \boldsymbol{Z})^\top (\boldsymbol{U}_k^\top \boldsymbol{Z}) \Big) \quad (5.2.11)$$

$$= \frac{p}{N\epsilon^2} \Big(\sum_{k=1}^{K} \boldsymbol{U}_k \boldsymbol{U}_k^\top \Big) \boldsymbol{Z} - \Big(\frac{p}{N\epsilon^2} \Big)^2 \sum_{k=1}^{K} \boldsymbol{U}_k (\boldsymbol{U}_k^\top \boldsymbol{Z}) (\boldsymbol{U}_k^\top \boldsymbol{Z})^\top (\boldsymbol{U}_k^\top \boldsymbol{Z}). \quad (5.2.12)$$

En la aproximación anterior, calculamos la similitud entre las representaciones proyectadas de los tokens $\{\boldsymbol{U}_k^\top \boldsymbol{z}_i\}_{i=1}^n$ mediante una autocorrelación entre las características proyectadas, expresada como $(\boldsymbol{U}_k^\top \boldsymbol{Z})^\top (\boldsymbol{U}_k^\top \boldsymbol{Z})$. Aunque se normalicen todas las características de los tokens dentro de $\boldsymbol{Z}$, la norma de esta matriz puede crecer arbitrariamente[18] a medida que aumenta la longitud de la secuencia, lo que hace que nuestra aproximación de Neumann sea cada vez menos precisa (ya que el gradiente verdadero no crece arbitrariamente en tamaño con la longitud de la secuencia). Para resolver este problema práctico y generalizar nuestra formulación, podemos utilizar en su lugar un núcleo arbitrario para medir la similitud. Un núcleo especialmente adecuado que se ajusta a nuestra intención de realizar la autoregresión únicamente con respecto a los tokens que pertenecen al mismo subespacio es el núcleo local (Nadaraya-Watson), que implica la estimación de pertenencia mediante softmax [CMP+21], es decir, el cálculo de la similitud (relativa) como $\text{softmax}((\boldsymbol{U}_k^\top \boldsymbol{Z})^\top (\boldsymbol{U}_k^\top \boldsymbol{Z}))$. Supongamos ahora que los subespacios se expanden «tanto como sea posible», es decir, que

[17] Esta aproximación solo tiene sentido cuando $\boldsymbol{Z}$ tiene una norma pequeña, lo cual, en la práctica, puede lograrse mediante estrategias de normalización (como la denominada «normalización por capas»). No la incluimos aquí por razones de conveniencia notacional, aunque el operador de normalización por capas correspondiente sí se incluye en la arquitectura final.

[18] Dada la elección práctica de $\epsilon^2 = p/N$, es decir, se utiliza un código más preciso para incorporar más información de más muestras.

todas las bases de los subespacios $\boldsymbol{U}_{[K]}$ juntas generan todo el espacio. Entonces, tenemos que $\sum_{k=1}^{K} \boldsymbol{U}_k \boldsymbol{U}_k^\top = \boldsymbol{I}$. Por lo tanto, (5.2.10) se convierte en

$$\nabla_{\boldsymbol{Z}} R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \approx \frac{p}{N\epsilon^2}\boldsymbol{Z} - \left(\frac{p}{N\epsilon^2}\right)^2 \text{MSSA}(\boldsymbol{Z}^\ell \mid \boldsymbol{U}_{[K]}^\ell), \tag{5.2.13}$$

donde MSSA se define mediante un operador SSA de la siguiente manera:

$$\text{SSA}(\boldsymbol{Z} \mid \boldsymbol{U}_k) \doteq (\boldsymbol{U}_k^\top \boldsymbol{Z})\,\text{softmax}((\boldsymbol{U}_k^\top \boldsymbol{Z})^\top (\boldsymbol{U}_k^\top \boldsymbol{Z})),\ \forall k \in [K], \tag{5.2.14}$$

$$\text{MSSA}(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq \frac{p}{N\epsilon^2}\left[\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K\right] \begin{bmatrix} \text{SSA}(\boldsymbol{Z} \mid \boldsymbol{U}_1) \\ \vdots \\ \text{SSA}(\boldsymbol{Z} \mid \boldsymbol{U}_K) \end{bmatrix}. \tag{5.2.15}$$

Al sustituir (5.2.13) en (5.2.7), se obtiene una expresión que puede aproximarse naturalmente mediante

$$\boldsymbol{Z}^{\ell+1/2} = \left(1 - \frac{\kappa p}{N\epsilon^2}\right)\boldsymbol{Z}^\ell + \frac{\kappa p}{N\epsilon^2}\text{MSSA}(\boldsymbol{Z}^\ell \mid \boldsymbol{U}_{[K]}^\ell). \tag{5.2.16}$$

Nota 5.7. El operador SSA de (5.2.14) se asemeja al *operador de atención* de un transformer típico [VSP+17b], excepto que aquí los operadores lineales de valor, clave y consulta se configuran todos como la base del subespacio (idénticos entre sí), es decir, $\boldsymbol{V}_k = \boldsymbol{K}_k = \boldsymbol{Q}_k = \boldsymbol{U}_k^*$. Por lo tanto, denominamos $\text{SSA}(\cdot \mid \boldsymbol{U}_k) : \mathbb{R}^{d\times n} \to \mathbb{R}^{p\times n}$ un operador de **S**ubspace **S**elf-**A**ttention (SSA). A continuación, el operador MSSA completo de la ecuación (5.2.15), definido formalmente como $\text{MSSA}(\cdot \mid \boldsymbol{U}_{[K]}) \colon \mathbb{R}^{d\times n} \to \mathbb{R}^{d\times n}$ y denominado operador **M**ulti-Head **S**ubspace **S**elf-**A**ttention (MSSA), agrega las salidas de los cabezales de atención promediándolas mediante pesos dependientes del modelo, un concepto similar al popular operador de autoatención multi-cabezal en las redes Transformer existentes. El paso de gradiente global (5.2.16) se asemeja a la autoatención múlti-cabezal implementada con una conexión de salto en los transformers.

MLP como descenso por gradiente proximal para la codificación esparsa de representaciones de tokens. Para el segundo paso de la minimización alternante, necesitamos minimizar $\lambda\|\boldsymbol{Z}\|_1 - R_\epsilon(\boldsymbol{Z})$. Obsérvese que el gradiente $\nabla R_\epsilon(\boldsymbol{Z})$ requiere una inversa de matriz y, por lo tanto, el gradiente proximal ingenuo (véase Apéndice A.1.3) para optimizar este problema se vuelve inmanejable en problemas a gran escala. Por lo tanto, adoptamos un enfoque diferente para equilibrar la diversidad representativa y el esparzamiento: postulamos un diccionario (completo) incoherente u ortogonal $\boldsymbol{D}^\ell \in \mathbb{R}^{d\times d}$ y buscamos ralear los iterados intermedios $\boldsymbol{Z}^{\ell+1/2}$ con respecto a $\boldsymbol{D}^\ell$. Es decir, $\boldsymbol{Z}^{\ell+1/2} \approx \boldsymbol{D}^\ell \boldsymbol{Z}^{\ell+1}$ donde $\boldsymbol{Z}^{\ell+1}$ es más esparso; es decir, es una *codificación esparsa* de $\boldsymbol{Z}^{\ell+1/2}$. El diccionario $\boldsymbol{D}^\ell$ se utiliza para ralear todos los tokens simultáneamente. Por la suposición de incoherencia, tenemos que $(\boldsymbol{D}^\ell)^\top(\boldsymbol{D}^\ell) \approx \boldsymbol{I}$. Por lo tanto, a partir de (5.2.2) tenemos que

$$R_\epsilon(\boldsymbol{Z}^{\ell+1/2}) \approx R_\epsilon(\boldsymbol{D}^\ell \boldsymbol{Z}^{\ell+1}) \approx R_\epsilon(\boldsymbol{Z}^{\ell+1}). \tag{5.2.17}$$

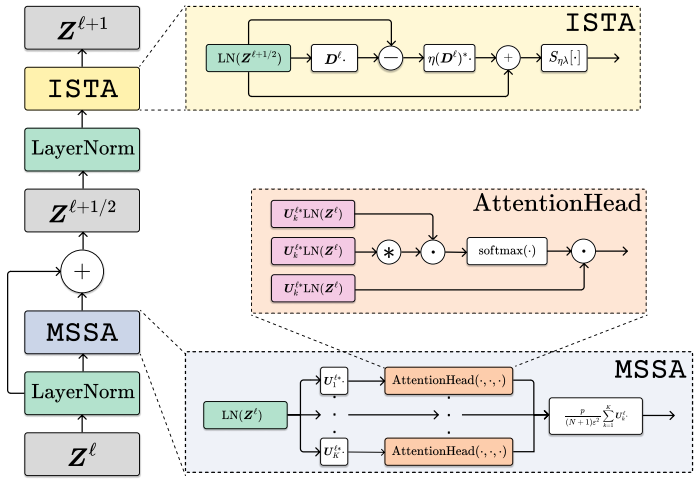


Figura 5.13: **Una capa de la arquitectura del codificador CRATE.** La arquitectura completa no es más que una concatenación de dichas capas, con un tokenizador inicial, un cabezal de preprocesamiento y un cabezal final específico para la tarea (es decir, un cabezal de clasificación).

Para resolver el $\lambda\|\boldsymbol{Z}\|_1 - R_\epsilon(\boldsymbol{Z})$, planteamos el siguiente problema de optimización:

$$\boldsymbol{Z}^{\ell+1} \approx \arg\min_{\boldsymbol{Z}} \|\boldsymbol{Z}\|_1 \quad \text{sujeto a} \quad \boldsymbol{Z}^{\ell+1/2} = \boldsymbol{D}^\ell \boldsymbol{Z}.$$

El programa anterior de representación esparsa suele resolverse relajándolo a un programa convexo sin restricciones, conocido como LASSO [WM22]:

$$\boldsymbol{Z}^{\ell+1} \approx \arg\min_{\boldsymbol{Z}} \left[\lambda\|\boldsymbol{Z}\|_1 + \frac{1}{2}\|\boldsymbol{Z}^{\ell+1/2} - \boldsymbol{D}^\ell \boldsymbol{Z}\|_F^2 \right]. \tag{5.2.18}$$

En nuestra implementación, también añadimos una restricción de no negatividad a $\boldsymbol{Z}^{\ell+1}$ y resolvemos el LASSO no negativo correspondiente:

$$\boldsymbol{Z}^{\ell+1} \approx \arg\min_{\boldsymbol{Z}\geq \boldsymbol{0}} \left[\lambda\|\boldsymbol{Z}\|_1 + \frac{1}{2}\|\boldsymbol{Z}^{\ell+1/2} - \boldsymbol{D}^\ell \boldsymbol{Z}\|_F^2 \right]. \tag{5.2.19}$$

A continuación, optimizamos en forma incremental la Ecuación (5.2.19) realizando un paso desenrollado de *descenso de gradiente proximal*, conocido como el paso ISTA [BT09], para obtener la actualización:

$$\boldsymbol{Z}^{\ell+1} = \mathrm{ISTA}(\boldsymbol{Z}^{\ell+1/2} \mid \boldsymbol{D}^\ell), \tag{5.2.20}$$

$$\text{where} \quad \mathrm{ISTA}(\boldsymbol{Z} \mid \boldsymbol{D}) \doteq \mathrm{ReLU}(\boldsymbol{Z} - \eta \boldsymbol{D}^\top(\boldsymbol{D}\boldsymbol{Z} - \boldsymbol{Z}) - \eta\lambda\boldsymbol{1}). \tag{5.2.21}$$

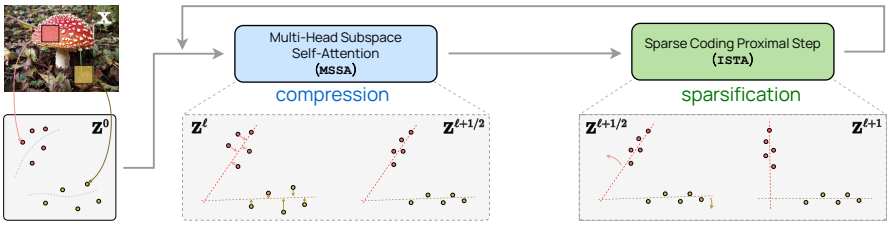


Figura 5.14: **El «bucle principal» del diseño de redes profundas transparentes tipo *white-box* de CRATE.** Tras codificar los datos de entrada como una secuencia de tokens $\boldsymbol{Z}^0$, CRATE construye una red profunda que transforma los datos en una configuración canónica de subespacios de baja dimensión mediante sucesivas ***compresiones*** frente a un modelo local de la distribución, generando $\boldsymbol{Z}^{\ell+1/2}$ y ***esparcificación*** frente a un diccionario global, generando $\boldsymbol{Z}^{\ell+1}$. Al apilar repetidamente estos bloques y entrenar los parámetros del modelo mediante retropropagación, se obtiene una representación potente e interpretable de los datos.

5.2.2 Arquitectura general del Transformer *white-box*: CRATE

Ahora diseñamos una arquitectura de transformer white-box, denominada Coding RATE Transformer (CRATE), mediante el desenrollado de las actualizaciones anteriores. Al combinar los dos pasos anteriores, (5.2.16) y (5.2.20):

1. Compresión local de tokens dentro de una muestra hacia una estructura de mezcla de subespacios, generando el bloque de autoatención de subespacios multicabezal – MSSA;

2. Esparcificación global de conjuntos de tokens en todas las muestras mediante codificación esparsa, lo que da lugar al bloque de esparcidad – ISTA;

podemos obtener la siguiente capa Transformer basada en la reducción de tasa de compresión, que se ilustra en la Figura 5.13,

$$\boldsymbol{Z}^{\ell+1/2} \doteq \boldsymbol{Z}^\ell + \mathrm{MSSA}(\boldsymbol{Z}^\ell \mid \boldsymbol{U}^\ell_{[K]}), \qquad \boldsymbol{Z}^{\ell+1} \doteq \mathrm{ISTA}(\boldsymbol{Z}^{\ell+1/2} \mid \boldsymbol{D}^\ell). \quad (5.2.22)$$

Al componer varias capas de este tipo siguiendo la construcción incremental de nuestra representación en (5.2.6), obtenemos una arquitectura transparente *white-box* de Transformer que proyecta los tokens de datos hacia una unión compacta y esparsa de subespacios incoherentes, donde $f^{\mathrm{pre}} : \mathbb{R}^{D\times N} \to \mathbb{R}^{d\times N}$ es el mapeo de preprocesamiento que transforma los tokens de entrada $\boldsymbol{X} \in \mathbb{R}^{D\times N}$ en representaciones de la primera capa $\boldsymbol{Z}^1 \in \mathbb{R}^{d\times N}$. En la Figura 5.14 se muestra un esquema general de esta arquitectura.

Nota 5.8 (**El papel del paso hacia adelante y de la retropropagación**). A diferencia de otros enfoques de optimización desenrollada, como el ReduNet

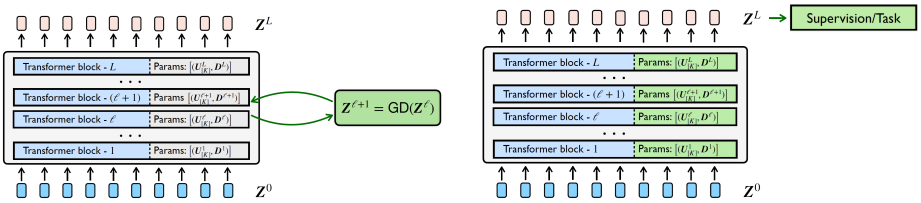
[CYY+22], *modelamos explícitamente* la distribución de cada $\boldsymbol{Z}^\ell$ y $\boldsymbol{Z}^{\ell+1/2}$ en cada capa, ya sea mediante una mezcla de subespacios lineales o generada en forma esparsa a partir de un diccionario. Hemos presentado la siguiente interpretación: en cada capa ℓ, las bases aprendidas para los subespacios $\boldsymbol{U}_{[K]}^\ell$ y los diccionarios aprendidos $\boldsymbol{D}^\ell$ sirven conjuntamente como un *libro de códigos* o *filtro de análisis* que codifica y transforma las representaciones intermedias en cada capa ℓ. Como la distribución de entrada a la capa ℓ se modela primero mediante $\boldsymbol{U}_{[K]}^\ell$ y luego se transforma mediante $\boldsymbol{D}^\ell$, la distribución de entrada varía entre capas, por lo que necesitamos un libro de códigos propio para cada capa a fin de obtener la codificación más parsimoniosa. Los parámetros de estos libros de códigos (es decir, las bases del subespacio y los diccionarios), que hasta ahora se asumía como perfectamente conocidos, en realidad se aprenden a partir de los datos (por ejemplo, mediante *retropropagación* dentro del entrenamiento de extremo a extremo).

Por lo tanto, nuestra metodología presenta una clara separación conceptual entre la «optimización» hacia adelante y el «aprendizaje» hacia atrás para la red neuronal profunda *white-box* así derivada. Es decir, en su paso hacia adelante, interpretamos cada capa como un operador que, condicionado en un modelo aprendido (es decir, un libro de códigos) para la distribución de su entrada, transforma esta distribución hacia una representación más parsimoniosas. En su retropropagación, el libro de códigos de este modelo, para la distribución de la entrada en cada capa, se actualiza para ajustarse mejor a una determinada relación (supervisada) entre entrada y salida, tal y como se ilustra en la Figura 5.15. Esta interpretación conceptual implica un cierto agnosticismo de las representaciones del modelo respecto a la tarea y la función de pérdida concretas; en particular, muchos tipos de tareas y funciones de pérdida garantizarán que los modelos de cada capa estén bien ajustados, lo que asegura que el modelo produzca representaciones parsimoniosas.

Hay tres preguntas que surgen de manera natural sobre CRATE:

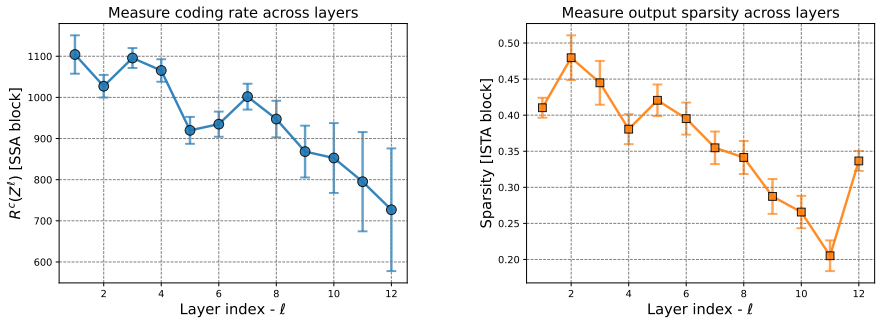
- Estos modelos CRATE construidos, ¿aprenden características útiles mediante la optimización de la reducción esparsa de la tasa?
- Estos modelos CRATE construidos, ¿usan sus características de alta calidad para obtener buenos resultados en tareas de predicción?
- ¿Muestran estos modelos CRATE construidos fenómenos interesantes, por ejemplo, relacionados con la interpretabilidad, como resultado de su diseño simplificado?

Sostenemos que la respuesta a cada pregunta es «sí». En primer lugar, en la Figura 5.16 se analiza cómo la red optimiza cada término de la reducción de la tasa de esparcidad; cada capa disminuye de manera consistente la medida de compresión y las medidas de esparcidad de las características durante la propagación hacia adelante. En segundo lugar, en la Tabla 5.1 mostramos que CRATE presenta una precisión «top-1», comparable, en configuraciones de aprendizaje por transferencia, a la arquitectura más comúnmente utilizada



(a) **Pasada directa** (b) **Retropropagación**

Figura 5.15: **Los roles de la pasada directa y de la retropropagación en CRATE**. (a) Dados subespacios y diccionarios fijos $\{(\boldsymbol{U}_{[K]}^{\ell}, \boldsymbol{D}^{\ell})\}_{\ell=1}^{L}$, cada capa optimiza de manera incremental la reducción de la tasa de esparcidad de las representaciones en la pasada hacia adelante; (b) La retropropagación aprende subespacios y diccionarios $\{(\boldsymbol{U}_{[K]}^{\ell}, \boldsymbol{D}^{\ell})\}_{\ell=1}^{L}$ a partir de los datos de entrenamiento.



(a) (b)

Figura 5.16: **Optimización de las medidas de compresión y esparcidad mediante la pasada directa de CRATE.** Medimos la compresión $R_\epsilon^c(\boldsymbol{Z}^{\ell+1/2})$ (a) y la esparcidad $\|\boldsymbol{Z}^{\ell+1}\|_1$ (b) en la salida de cada capa. Observamos que ambas medidas disminuyen a lo largo del paso hacia adelante, como predice la teoría, y que en casi todas las capas se observa una disminución monótona de cada medida. (La excepción es la esparcidad de la última capa, ya que las características densas son importantes para la predicción.)

de red neuronal Vision Transformer (ViT) [DBK+21] en un número similar de parámetros, con un comportamiento de escalabilidad prometedor: obtenemos mejoras sostenids y coherentes al aumentar el tamaño del modelo. En general, CRATE logra un rendimiento prometedor en conjuntos de datos a gran escala del mundo real mediante la implementación directa de nuestra arquitectura basada en principios. Ofrecemos más detalles sobre la implementación y el análisis de los resultados experimentales en la Sección 8.4.

Tabla 5.1: **Precisión de clasificación «top-1» de CRATE en diversos conjuntos de datos con diferentes escalas de modelo cuando se preentrenan en ImageNet-1K.** Para ImageNet-1K/ImageNet-1K ReaL, evaluamos directamente la precisión «top-1». Para otros conjuntos de datos, utilizamos modelos entrenados previamente en ImageNet como inicialización y evaluamos el rendimiento del aprendizaje por transferencia mediante el ajuste fino.

Modelo	CRATE-T	CRATE-S	CRATE-B	CRATE-L	ViT-T	ViT-S
# parámetros	6.09M	13.12M	22.80M	77.64M	5.72M	22.05M
ImageNet-1K	66.7	69.2	70.8	71.3	71.5	72.4
ImageNet-1K ReaL	74.0	76.0	76.5	77.4	78.3	78.4
CIFAR10	95.5	96.0	96.8	97.2	96.6	97.2
CIFAR100	78.9	81.0	82.7	83.6	81.8	83.2
Oxford Flowers-102	84.6	87.1	88.7	88.3	85.1	88.5
Oxford-IIIT-Pets	81.4	84.9	85.3	87.4	88.5	88.6

5.3 Variantes por diseño de arquitecturas profundas

Hasta ahora, esperamos haber aportado pruebas convincentes de que el papel de las redes profundas (populares) consiste en aplicar ciertos algoritmos de optimización para minimizar la tasa de codificación (o maximizar la ganancia de información) de las representaciones aprendidas. Sin embargo, los lectores familiarizados con los métodos de optimización habrán notado que las arquitecturas mencionadas (ReduNet o CRATE) corresponden a técnicas de optimización bastante básicas. Es posible que tengan mucho margen de mejora en cuanto a eficiencia o eficacia. Además, si creemos que el marco teórico propuesto para interpretar las redes profundas es correcto, entonces no solo debería ayudar a explicar las arquitecturas existentes, sino también debería guiarnos en el desarrollo de arquitecturas más eficientes y efectivas. En esta sección, mostramos que esto es cierto: las nuevas arquitecturas resultantes no solo son totalmente interpretables, sino que también presentan corrección garantizada y una eficiencia mejorada.

5.3.1 Arquitectura Transformer de atención exclusiva

En esta subsección, planteamos y respondemos la pregunta: «¿Cuál es la red neuronal más simple posible que escale razonablemente bien y comprima la representación en forma demostrable?». Según nuestra teoría previa, la respuesta debería ser una pila de capas MSSA, concebidas como operadores de compresión con pérdida. Esta arquitectura de red es lo suficientemente simple como para que sea posible analizar el rendimiento de compresión o de eliminación de ruido en forma rigurosa y sistemática. Con este fin, formalicemos nuestro modelo de subespacios: las representaciones iniciales de los tokens $\boldsymbol{Z}^1$ se obtienen mediante muestreo de una mezcla ruidosa de distribuciones gaussianas de bajo rango, con soporte en subespacios generados por $\boldsymbol{U}_k \in \mathbb{R}^{d\times p}$, de la siguiente manera: particionamos los N índices de tokens en subconjuntos

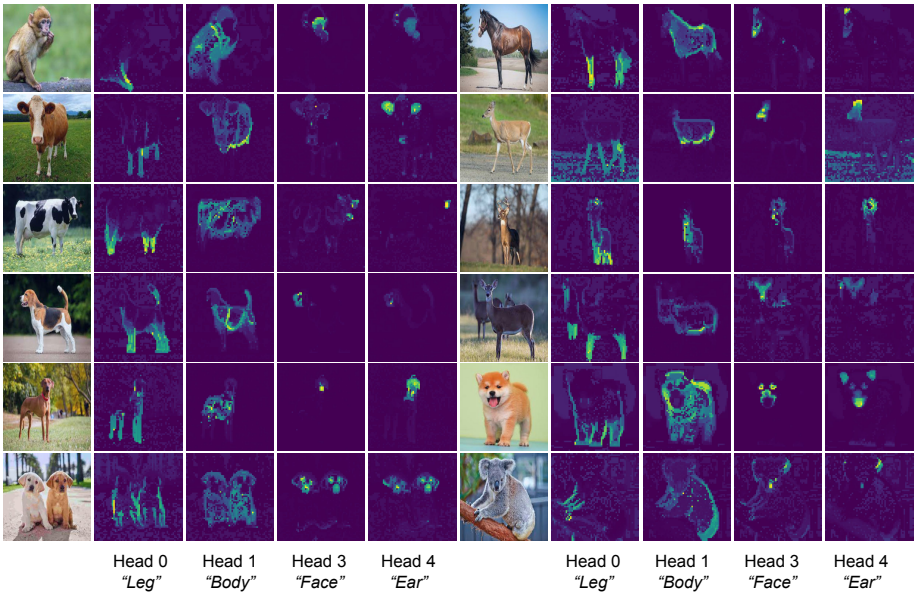


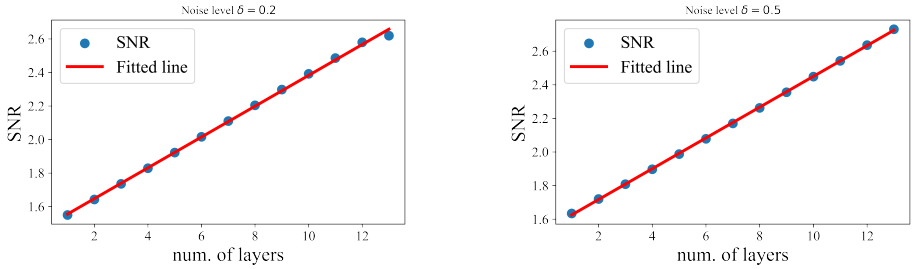
Figura 5.17: **Visualización de los mapas de atención de CRATE: ¿en qué se fija el modelo para realizar una clasificación?** Visualizamos los mapas de atención de la última capa, que constituyen la última fuente de interacción entre tokens en la red; por lo tanto representan en cierto sentido, la descripción más fina de qué regiones de la imagen de entrada utiliza el modelo para realizar una predicción. Observamos un comportamiento *extremadamente interesante*: cuando se entrenan exclusivamente en tareas de clasificación, los mapas de atención identifican los objetos en primer plano de la imagen; además, ¡se *especializan en forma natural en* diferentes tipos de objetos en primer plano! Este resultado empírico nos ofrece una interpretación de los subespacios de las capas posteriores como si cada uno correspondiera a un conjunto casi disjunto de *conceptos*.

conocidos $C_1, \ldots, C_K$, de tal manera que:

$$\boldsymbol{z}_i = \underbrace{\boldsymbol{U}_k \boldsymbol{a}_i}_{signal} + \underbrace{\sum_{j \neq k}^{K} \boldsymbol{U}_j \boldsymbol{e}_{i,j}}_{noise}, \ \forall i \in C_k, \tag{5.3.1}$$

donde $\boldsymbol{a}_i \overset{i.i.d.}{\sim} \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_{p_k})$ y $\boldsymbol{e}_{i,j} \overset{i.i.d.}{\sim} \mathcal{N}(\boldsymbol{0}, \delta^2 \boldsymbol{I}_{p_j})$ para todo $i \in C_k$ y $k \in [K]$, $\{\boldsymbol{a}_i\}$ y $\{\boldsymbol{e}_{i,j}\}$ son mutuamente independientes.

Un operador de eliminación de ruido para representaciones de tokens. Ahora, mostramos que el operador MSSA (véase (5.2.15)) puede eliminar en forma incremental el ruido de las representaciones de tokens generadas a



(a) Nivel de ruido $\delta = 0.2$ (b) Nivel de ruido $\delta = 0.5$

Figura 5.18: **Desempeño en la eliminación de ruido del Transformer basado únicamente en atención.** Aquí, muestreamos las representaciones iniciales de los tokens a partir de una mezcla de gaussianas de bajo rango en la Ecuación (5.3.1). A continuación, aplicamos (5.3.2) para actualizar las representaciones de los tokens y reportamos la relación señal-ruido (SNR) en cada capa.

partir del modelo anterior. Concretamente, consideramos un operador MSSA generalizado: para cada $\ell \in [L]$,

$$\boldsymbol{Z}^{\ell+1} = \boldsymbol{Z}^{\ell} + \eta \sum_{k=1}^{K} \boldsymbol{U}_k \boldsymbol{U}_k^T \boldsymbol{Z}^{\ell} \varphi(\boldsymbol{Z}^{\ell^T} \boldsymbol{U}_k \boldsymbol{U}_k^T \boldsymbol{Z}^{\ell}), \tag{5.3.2}$$

donde $\eta > 0$ es el tamaño de paso y φ es una función que actúa sobre cada columna de la matriz de entrada, como softmax, ReLU elemento a elemento, etc. Para simplificar nuestra teoría, suponemos que los subespacios de la Ecuación (5.3.1) son ortogonales entre sí, es decir, $\boldsymbol{U}_k^T \boldsymbol{U}_j = \boldsymbol{0}$ para todo $k \neq j$. Cabe señalar que esta suposición no es restrictiva, ya que en espacios de alta dimensión, los subespacios de baja dimensión elegidos aleatoriamente son incoherentes entre sí con alta probabilidad, es decir, $\boldsymbol{U}_k^T \boldsymbol{U}_j \approx \boldsymbol{0}$ [WM22].[19]

Ahora, sean las columnas de $\boldsymbol{Z}_k^{\ell}$ las representaciones de tokens del subespacio k-ésimo en la capa ℓ-ésima. Para cuantificar la capacidad de eliminación de ruido, definimos la relación señal-ruido (SNR) para cada bloque de las representaciones de tokens en la capa ℓ de la siguiente manera:

$$\mathrm{SNR}(\boldsymbol{Z}_k^{\ell}) \doteq \frac{\|\boldsymbol{U}_k \boldsymbol{U}_k^T \boldsymbol{Z}_k^{\ell}\|_F}{\|(\boldsymbol{I} - \boldsymbol{U}_k \boldsymbol{U}_k^T) \boldsymbol{Z}_k^{\ell}\|_F}, \quad \forall k \in [K]. \tag{5.3.3}$$

Para simplificar nuestro análisis, suponemos que $|C_1| = \cdots = |C_K| = N/K$ y

$$\begin{bmatrix} \boldsymbol{U}_1 & \ldots & \boldsymbol{U}_K \end{bmatrix} \in \mathcal{O}^{d \times Kp}. \tag{5.3.4}$$

Con la configuración anterior, ahora podemos caracterizar el desempeño en la eliminación de ruido de un operador MSSA ligeramente modificado, que aplica un umbral a los términos que son demasiado pequeños en la función softmax.

[19] Se pueden generalizar directamente nuestros resultados a subespacios no ortogonales, con un análisis ligeramente más sofisticado.

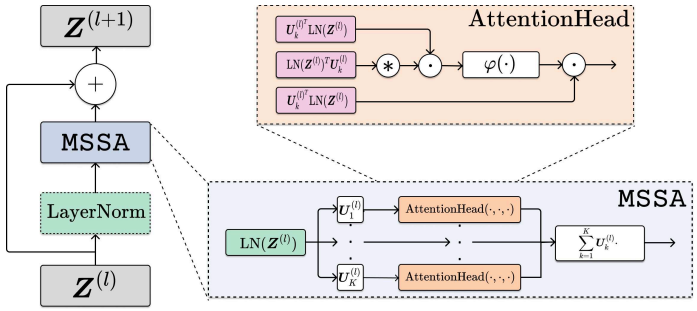


Figura 5.19: **Detalles de la arquitectura Transformer de «solo atención».** Cada capa consta del operador MSSA y una conexión residual (*skip connection*). Además, se incluye LayerNorm solo para tareas de lenguaje. En la práctica, se aplica la retropropagación para entrenar los parámetros del modelo utilizando muestras de entrenamiento.

Teorema 5.1. *Sea* $\boldsymbol{Z}^1$ *tal como se define en la Ecuación* (5.3.1) *y sea* $\varphi(\cdot)$ *en* (5.3.2) $\varphi(\boldsymbol{x}) = h(\sigma(\boldsymbol{x}))$*, donde* $\sigma : \mathbb{R}^N \to \mathbb{R}^N$ *es la función softmax y* $h : \mathbb{R}^N \to \mathbb{R}^N$ *es una función de umbralización elemento a elemento con* $h(x) = \tau$ *si* $x > \tau$ *y* $h(x) = 0$ *si* $x \le \tau$ *para cada* $i \in [N]$*. Supongamos que* $p \gtrsim \log N$*,* $\delta \lesssim \sqrt{\log N}/\sqrt{p}$ *y*

$$\tau \in \left(\frac{1}{2}, \frac{1}{1 + N\exp(-9p/32)}\right].$$

Para N *suficientemente grande, se cumple con probabilidad de al menos* $1 - KLN^{-\Omega(1)}$ *que, para cada* $\ell \in [L]$*,*

$$\mathrm{SNR}(\boldsymbol{Z}_k^{\ell+1}) = (1 + \eta\tau)\mathrm{SNR}(\boldsymbol{Z}_k^{\ell}), \ \forall k \in [K]. \tag{5.3.5}$$

Este teorema demuestra que, cuando las representaciones iniciales de los tokens se muestrean a partir de una mezcla de distribuciones gaussianas de bajo rango con un nivel de ruido de $O(\sqrt{\log N}/\sqrt{p})$, cada aplicación del operador MSSA modificado elimina el ruido de las representaciones de los tokens a un ritmo lineal. Cabe destacar que nuestros resultados teóricos están bien respaldados por las observaciones experimentales realizadas con el operador MSSA *real* en la Figura 5.18. Este teorema proporciona una base teórica para la capacidad práctica de eliminación de ruido de la arquitectura del transformer derivada mediante el desenrollado de (5.3.2).

Transformadores de solo atención. Para verificar que esta capacidad de eliminación de ruido se traduce en un rendimiento razonable con datos y tareas reales, proponemos formalmente una arquitectura de transformer basada exclusivamente en la atención, a la que abreviamos como AoT. Concretamente, al desenrollar los pasos iterativos de optimización (5.3.2) como capas de una red

Tabla 5.2: **Evaluación del AoT en tareas de visión**: precisión top-1 en ImageNet.

Modelos	Precisión	Nº de parámetros
AoT	71.7%	22M
CRATE	79.5%	39M
ViT	72.4 %	22M

profunda, construimos una arquitectura de transformer tal y como se muestra en la Figura 5.19. Cada capa de la arquitectura propuesta consta únicamente del operador MSSA y una conexión residual. Para las tareas lingüísticas, incorporamos además LayerNorm antes del operador MSSA para mejorar el rendimiento. La arquitectura completa se construye apilando dichas capas, junto con pasos esenciales de preprocesamiento y posprocesamiento específicos de la tarea, tales como la codificación posicional, el *embedding* de tokens y una cabeza final específica de la tarea para adaptarse a diferentes aplicaciones. La Tabla 5.2 muestra el rendimiento de este transformer minimalista en la clasificación de imágenes; el rendimiento es razonable, a pesar de su diseño minimalista. En la siguiente sección, discutiremos una mejora conceptual a la formulación CRATE que *también* incrementará el rendimiento y la eficiencia.

5.3.2 Atención de tiempo lineal: Transformer de estadísticas de token

En esta subsección, proponemos un nuevo operador de atención de transformer cuya complejidad computacional escala linealmente con el número de tokens, basándonos en el objetivo de reducción de la tasa de codificación. En concreto, derivamos una nueva forma variacional del objetivo MCR^2 y demostramos que la arquitectura resultante del descenso de gradiente desenrollado de este objetivo variacional da lugar a un nuevo módulo de atención denominado Token Statistics Self-Attention (TSSA). TSSA tiene *una complejidad computacional y de memoria lineales* y se aparta radicalmente de la arquitectura de atención típica que calcula las similitudes por pares entre tokens. Al sustituir el operador de atención tradicional de un transformer con TSSA se obtiene una nueva arquitectura de red neuronal denominada Token Statistics Transformer (ToST), que es muy eficiente y presenta un rendimiento empírico prometedor. Como preludio, recordemos de la ecuación (4.2.17) que $\mathbf{\Pi} = [\boldsymbol{\pi}_1, \ldots, \boldsymbol{\pi}_K] \in \mathbb{R}^{N \times K}$ denota una matriz estocástica de «asignación de grupos» (es decir, $\mathbf{\Pi}\mathbf{1} = \mathbf{1}$ y $\Pi_{ik} \geq 0,\ \forall (i,k) \in [N] \times [K]$), donde Π_{ik} denota la probabilidad de asignar el i-ésimo token al k-ésimo grupo.

Una nueva forma variacional para las tasas de codificación. Para empezar, consideramos una forma general de objetivos del tipo MCR^2 basados en funciones cóncavas del espectro de una matriz. Concretamente, para una

matriz PSD dada $\boldsymbol{M} \in \mathsf{PSD}(d)$ y cualquier escalar $c \geq 0$, tenemos que $\log\det(\boldsymbol{I} + c\boldsymbol{M}) = \sum_{i=1}^{d} \log(1 + c\lambda_i(\boldsymbol{M}))$, donde $\lambda_i(\boldsymbol{M})$ es el i-ésimo mayor valor propio de $\boldsymbol{M}$. Además, cabe señalar que $\log(1+c\sigma)$ es una función cóncava y no decreciente de σ. Por lo tanto, describimos nuestros resultados en términos de una forma más general de MCR^2 basada en funciones espectrales generales de matrices PSD de la forma $F(\boldsymbol{M}) = \sum_{i=1}^{d} f(\lambda_i(\boldsymbol{M}))$, donde f es cóncava y no decreciente. En particular, como ya se vio más arriba en este capítulo, el mecanismo de atención surge del desenrollado de la componente de compresión de MCR^2, por lo que consideramos una función de compresión más general al estilo de MCR^2:

$$R_{c,f}(\boldsymbol{Z}, \boldsymbol{\Pi}) \doteq \frac{1}{2} \sum_{k=1}^{K} \frac{N_k}{N} F\left(\frac{1}{N_k} \boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top\right). \tag{5.3.6}$$

Con respecto al objetivo anterior, observamos ahora el siguiente resultado:

Teorema 5.2. *Sea $f\colon [0,\infty) \to \mathbb{R}$ una función no decreciente y cóncava, que cumple $f(0) = 0$ y sea $F\colon \mathsf{PSD}(d) \to \mathbb{R}$ una función de la forma $F(\boldsymbol{M}) = \sum_{i=1}^{d} f(\lambda_i(\boldsymbol{M}))$. Entonces, para cada $\boldsymbol{M} \in \mathsf{PSD}(d)$ y $\boldsymbol{Q} \in \mathsf{O}(d)$, tenemos*

$$F(\boldsymbol{M}) \leq \sum_{i=1}^{d} f\left((\boldsymbol{Q}^\top \boldsymbol{M} \boldsymbol{Q})_{ii}\right). \tag{5.3.7}$$

Además, la desigualdad de (5.3.7) se cumple con igualdad para cualquier $\boldsymbol{Q}$ que diagonalice $\boldsymbol{M}$, y si f es estrictamente cóncava, entonces la desigualdad de (5.3.7) se cumple con igualdad si y solo si $\boldsymbol{Q}$ diagonaliza $\boldsymbol{M}$.

A partir del resultado anterior, podemos sustituir (5.3.6) por un objetivo variacional equivalente de la forma

$$R^{\mathrm{var}}_{c,f}(\boldsymbol{Z}, \boldsymbol{\Pi} \mid \boldsymbol{U}_{[K]}) \doteq \frac{1}{2} \sum_{k=1}^{K} \frac{N_k}{N} \sum_{i=1}^{d} f\left(\frac{1}{N_k} (\boldsymbol{U}_k^\top \boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top \boldsymbol{U}_k)_{ii}\right), \tag{5.3.8}$$

donde la equivalencia se entiende en el sentido de que, para una elección óptima de $\{\boldsymbol{U}_k \in \mathsf{O}(d)\}_{k=1}^{K}$ matrices, tal como se describe en el Teorema 5.2 (es decir, matrices ortogonales que diagonalizan cada $\boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top$) lograremos una ajustada coincidente con $R^{\mathrm{var}}_{c,f}(\boldsymbol{Z}, \boldsymbol{\Pi} \mid \boldsymbol{U}_{[K]}) = R_{c,f}(\boldsymbol{Z}, \boldsymbol{\Pi})$. Tenga en cuenta que, en general, alcanzar esta cota requeriría seleccionar, para cada instancia muestreada de $\boldsymbol{Z}$, un nuevo conjunto óptimo de matrices de parámetros $\boldsymbol{U}_k$ que diagonalicen cada $\boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top$, lo cual es claramente impracticable como arquitectura de la red. En cambio, desde un punto de vista alternativo, en lugar de considerar los datos ($\boldsymbol{Z}$) como fijos e intentar optimizar los parámetros $\boldsymbol{U}_k$ para alcanzar la cota variacional más ajustada, podemos adoptar el principio de diseño de «desenrollado» algorítmico descrito anteriormente y diseñar un operador que perturbe $\boldsymbol{Z}$ para minimizar en forma incremental $R^{\mathrm{var}}_{c,f}(\cdot \mid \boldsymbol{U}_{[K]})$. Para explicitar este punto, cada cota variacional se vuelve ajustada cuando los

espacios propios de $\boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top$ se alinean con las columnas de $\boldsymbol{U}_k$, por lo que al rotar las columnas apropiadas de $\boldsymbol{Z}$ (es decir, aquellas que corresponden a entradas grandes en $\boldsymbol{\pi}_k$) para alinearlas con $\boldsymbol{U}_k$ podemos acercarnos a una cota variacional ajustada. Es decir, en lugar de rotar $\boldsymbol{U}_k$ para alinearlo con los datos de cada instancia de $\boldsymbol{Z}$, podemos girar las características de los tokens de cada $\boldsymbol{Z}$ para alinearlas con $\boldsymbol{U}_k$.

Siguiendo este enfoque, calculamos un paso de descenso por gradiente en $R^{\mathrm{var}}_{c,f}$ con respecto a $\boldsymbol{Z}$. Para iniciar este cálculo, sea $\boldsymbol{\pi} \in \mathbb{R}^N$ un vector arbitrario no negativo elemento a elemento. Entonces tenemos

$$\nabla_{\boldsymbol{Z}} \frac{1}{2}\sum_{i=1}^{d} f((\boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi})\boldsymbol{Z}^\top)_{ii}) = \mathrm{Diag}(\nabla f[\boldsymbol{Z}^{\circ 2}\boldsymbol{\pi}])\boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}), \tag{5.3.9}$$

donde ∇f es el gradiente de f y (recordemos que) $\nabla f[\cdot]$ aplica ∇f a cada elemento del vector entre corchetes. En particular, para $f(x) = \log(1+(d/\epsilon^2)x)$, $\nabla f(x) = (d/\epsilon^2)(1+(d/\epsilon^2)x)^{-1}$ es simplemente una activación no lineal. Además, (recordemos que) $N_k = \langle \boldsymbol{\pi}_k, \mathbf{1}\rangle$. Por lo tanto, el gradiente de $R^{\mathrm{var}}_{c,f}$ respecto a $\boldsymbol{Z}$ es:

$$\nabla_{\boldsymbol{Z}} R^{\mathrm{var}}_{c,f}(\boldsymbol{Z}, \boldsymbol{\Pi} \mid \boldsymbol{U}_{[K]}) \tag{5.3.10}$$

$$= \frac{1}{n}\sum_{k=1}^{K} \boldsymbol{U}_k \underbrace{\mathrm{Diag}\left(\nabla f\left[(\boldsymbol{U}_k^\top \boldsymbol{Z})^{\circ 2}\frac{\boldsymbol{\pi}_k}{\langle \boldsymbol{\pi}_k, \mathbf{1}\rangle}\right]\right)}_{\doteq \boldsymbol{D}(\boldsymbol{Z}, \boldsymbol{\pi}_k \mid \boldsymbol{U}_k)} \boldsymbol{U}_k^\top \boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k). \tag{5.3.11}$$

(Obsérvese que la constante $1/N$ se deriva de una constante $(N_k/N)\cdot(1/N_k) = 1/N$ presente en cada término de la suma.) Si ahora consideramos un paso de gradiente con respecto al token j-ésimo $\boldsymbol{z}_j$, llegamos a nuestro operador de compresión incremental propuesto, es decir, nuestro sustituto de un operador de *autoatención* + residual:

$$\boldsymbol{z}_j^+ = \boldsymbol{z}_j - \tau \nabla_{\boldsymbol{z}_j} R^{\mathrm{var}}_{c,f}(\boldsymbol{Z}, \boldsymbol{\Pi} \mid \boldsymbol{U}_{[K]}) \tag{5.3.12}$$

$$= \boldsymbol{z}_j - \frac{\tau}{N}\sum_{k=1}^{K} \Pi_{jk}\boldsymbol{U}_k \boldsymbol{D}(\boldsymbol{Z}, \boldsymbol{\pi}_k \mid \boldsymbol{U}_k)\boldsymbol{U}_k^\top \boldsymbol{z}_j \tag{5.3.13}$$

para cada $j \in [n]$, donde $\tau > 0$ es un parámetro de tamaño de paso para la optimización incremental.

Interpretación del modelo. Dado el operador de atención propuesto en (5.3.12), recordemos primero que las filas de $\boldsymbol{\Pi}$ son no negativas y suman 1, por lo que nuestro operador toma un promedio ponderado de K operadores similares a «cabezas de atención» y luego añade una conexión residual. Utilizando que $\sum_{k=1}^{K} \Pi_{jk} = 1$, podemos reescribir (5.3.12) como:

$$\boldsymbol{z}_j^+ = \sum_{k=1}^{K} \Pi_{jk}\Big[\boldsymbol{z}_j \underbrace{-\frac{\tau}{n}\boldsymbol{U}_k \boldsymbol{D}(\boldsymbol{Z}, \boldsymbol{\pi}_k \mid \boldsymbol{U}_k)\boldsymbol{U}_k^\top}_{\text{acción de un cabezal de atención}} \boldsymbol{z}_j\Big]. \tag{5.3.14}$$

Es decir, podemos considerar que cada cabezal de atención primero proyecta las características de los tokens sobre la base $\boldsymbol{U}_k$ multiplicándolas por $\boldsymbol{U}_k^\top$, las multiplica por la matriz diagonal $\boldsymbol{D}(\boldsymbol{Z}, \boldsymbol{\pi}_k \mid \boldsymbol{U}_k)$ (abreviada como $\boldsymbol{D}_k$), proyectándolas de nuevo en la base estándar mediante la multiplicación por $\boldsymbol{U}_k$ y restando esto de las características originales del token a través de la conexión residual. El aspecto central de nuestra capa de atención es el cálculo de $\boldsymbol{D}_k$. En particular, dado que $\Pi_{jk} \geq 0$, entonces $\boldsymbol{\pi}_k / \langle \boldsymbol{\pi}_k, \mathbf{1} \rangle \in \mathbb{R}^N$ forma una distribución de probabilidad que indica qué tokens pertenecen al k-ésimo grupo. Como resultado, $(\boldsymbol{U}_k^\top \boldsymbol{Z})^{\circ 2} \boldsymbol{\pi}_k / \langle \boldsymbol{\pi}_k, \mathbf{1} \rangle$ estima el segundo momento de $\boldsymbol{U}_k^\top \boldsymbol{Z}$ bajo la distribución dada por $\boldsymbol{\pi}_k / \langle \boldsymbol{\pi}_k, \mathbf{1} \rangle$. Además, dado que f es una función cóncava no decreciente, $\nabla f(x)$ disminuye monótonamente hacia 0 a medida que x aumenta, por lo que las entradas de $\boldsymbol{D}_k$ (que tienen la forma $\nabla f(x)$) alcanzan su máximo en $x = 0$ y decaen monótonamente a 0 a medida que x aumenta.

A partir de esto, llegamos a la interpretación fundamental de nuestros operadores de cabeza de atención + residual $[\boldsymbol{I} - (\tau/n)\boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{U}_k^\top]$. Es decir, este operador realiza una proyección aproximada de rango bajo dependiente de los datos, en la que se conservan las direcciones que tienen una gran «potencia» tras la proyección $\boldsymbol{U}_k^\top \boldsymbol{Z}$ (es decir, las direcciones que tienen un segundo momento elevado $(\boldsymbol{U}_k^\top \boldsymbol{Z})^{\circ 2} \boldsymbol{\pi}_k / \langle \boldsymbol{\pi}_k, \mathbf{1} \rangle$), mientras que las direcciones que no la tienen se suprimen. Para comprenderlo, recordemos que los elementos de $\boldsymbol{D}_k$ disminuyen monótonamente hasta 0 a medida que aumenta el segundo momento, por lo que, para las direcciones con momentos de segundo orden grandes, el operador de atención + residual actúa en gran medida como el operador de identidad. Por el contrario, para las direcciones con un momento de segundo orden pequeño, nuestro operador resta una proyección de los tokens a lo largo de esas direcciones, lo que da como resultado la eliminación de dichas direcciones. En comparación con el operador de autoatención estándar, nuestro método claramente no calcula ninguna similitud por pares entre tokens. Más bien, las interacciones entre los elementos de $\boldsymbol{Z}$ influyen en el operador únicamente a través de su contribución a la estadística del segundo momento utilizada para construir los $\boldsymbol{D}_k$. No obstante, al igual que el operador de autoatención estándar, nuestro método sigue teniendo una interpretación clara como una forma de compresión hacia una estructura de bajo rango dependiente de los datos, en el sentido de que realiza una proyección aproximada de bajo rango, en la que las direcciones específicas que se eliminan son aquellas que no están fuertemente alineadas con otros tokens del grupo.

Consideraciones computacionales. Tras haber presentado nuestro operador de atención propuesto, analizaremos ahora cómo se puede calcular en la práctica. En primer lugar, hasta este momento de la presentación, hemos evitado abordar cómo se «agrupan» los tokens en los distintos cabezales de atención mediante la matriz $\boldsymbol{\Pi}$, pero es evidente que se necesita una forma de construir $\boldsymbol{\Pi}$ para implementar nuestro método. Además, nuestra forma variacional en el Teorema 5.2 requiere que las matrices $\boldsymbol{U}$ sean cuadradas y

ortogonales, pero lo ideal sería utilizar matrices más pequeñas (es decir, reducir el número de columnas en $\boldsymbol{U}$) para mejorar la eficiencia, así como eliminar las restricciones de ortogonalidad.

En la práctica, no imponemos las restricciones de ortogonalidad. Para reducir el número de columnas en las matrices $\boldsymbol{U}$, observamos que, al igual que en CRATE [YBP+23], si suponemos que las características $\boldsymbol{Z}$ dentro del grupo k se agrupan (aproximadamente) en torno a un subespacio de baja dimensión —digamos, de dimensión p — entonces la covarianza dentro del grupo k $\boldsymbol{Z}\mathrm{Diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top$ es de rango bajo, donde recordemos que [YCY+20] demuestra que la geometría óptima de $\boldsymbol{Z}$ se da cuando cada grupo es un subespacio de rango bajo, ortogonal a los demás grupos. Por lo tanto, podemos encontrar explícitamente una base ortonormal de baja dimensión para la imagen de esta covarianza, es decir, el espacio lineal generado por los datos del grupo k. Si la dimensión es $p \leq d$, la base puede representarse mediante una matriz ortogonal $d \times p$ $\boldsymbol{U}_k \in \mathsf{O}(d,p)$. En este caso, podemos establecer una cota superior más eficiente para $R_{c,f}$ utilizando estas matrices de base ortogonal de rango bajo. Para demostrarlo, utilizamos una versión más general del Teorema 5.2 para obtener el siguiente corolario.

Corolario 5.1. *Sea* $f\colon [0,\infty) \to \mathbb{R}$ *una función no decreciente y cóncava, que cumple* $f(0) = 0$ *y sea* $F\colon \mathsf{PSD}(p) \to \mathbb{R}$ *una función de forma* $F(\boldsymbol{M}) = \sum_{i=1}^{p} f(\lambda_i(\boldsymbol{M}))$. *Sean* $\boldsymbol{Z}$, $\boldsymbol{\Pi}$ *fijos. Entonces, para todo* $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K \in \mathsf{O}(d,p)$ *tal que* $\mathrm{image}(\boldsymbol{Z}\,\mathrm{diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top) \subset \mathrm{image}(\boldsymbol{U}_k)$ *para todo* $k \in [K]$, *tenemos*

$$R_{c,f}(\boldsymbol{Z},\boldsymbol{\Pi}) \leq R^{\mathrm{var}}_{c,f}(\boldsymbol{Z},\boldsymbol{\Pi} \mid \boldsymbol{U}_{[K]}), \tag{5.3.15}$$

donde $R^{\mathrm{var}}_{c,f}$ *se define formalmente en* (5.3.8). *La igualdad se cumple si* $\boldsymbol{U}_k$ *diagonaliza* $\boldsymbol{Z}\,\mathrm{diag}(\boldsymbol{\pi}_k)\boldsymbol{Z}^\top$ *para cada* $k \in [K]$ *y si* f *es fuertemente cóncava, entonces esta condición de igualdad se convierte en una condición de tipo "si y solo si."*

El último paso para definir nuestro operador de atención consiste en estimar la pertenencia al grupo $\boldsymbol{\Pi}$. Para ello, planteamos un modelo sencillo de cómo cada característica $\boldsymbol{z}_j$ se desvía de su subespacio de soporte y luego hallamos la asignación óptima de subespacios. En [YBP+23] se muestra que si modelamos independientemente cada $\boldsymbol{z}_j$ como perteneciente a un modelo de mezcla gaussiana de baja dimensión, donde cada gaussiana tiene una matriz de covarianza con espectro idéntico y está apoyada en un subespacio con base ortonormal $\boldsymbol{U}_k$, más ruido gaussiano independiente con covarianza $\eta\boldsymbol{I}$, entonces la probabilidad a posteriori de que cada token $\boldsymbol{z}_j$ pertenezca a cada subespacio viene dada por la matriz de asignación $\boldsymbol{\Pi} = \boldsymbol{\Pi}(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]})$ de la siguiente manera:

$$\boldsymbol{\Pi} = \begin{bmatrix} \boldsymbol{\nu}(\boldsymbol{z}_1 \mid \boldsymbol{U}_{[K]})^\top \\ \vdots \\ \boldsymbol{\nu}(\boldsymbol{z}_n \mid \boldsymbol{U}_{[K]})^\top \end{bmatrix}, \quad \text{where} \tag{5.3.16}$$

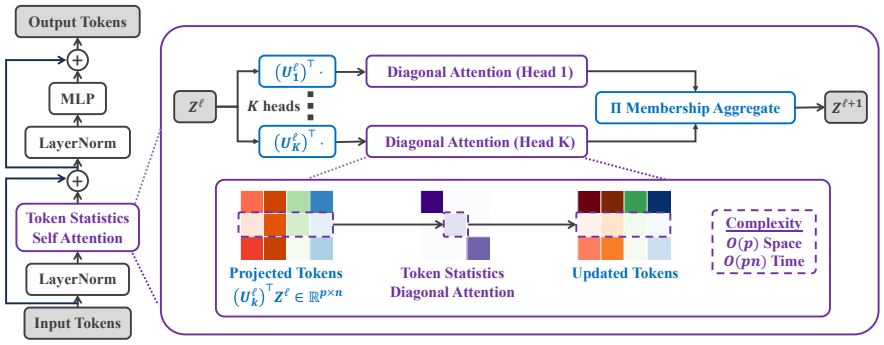


Figura 5.20: **Una capa ℓ del Token Statistics Transformer (ToST) propuesto.** Cabe destacar que la autoatención de ToST transforma los tokens $\boldsymbol{Z}^\ell$ de manera eficiente en $\boldsymbol{Z}^{\ell+1}$, multiplicando cada fila del token proyectado por *solo un escalar*. Esto permite reducir la complejidad de la atención: tiene una complejidad espacial de $O(p)$ y una complejidad temporal de $O(pn)$, donde p es la dimensión de los tokens proyectados de cada cabeza y n es el número de tokens.

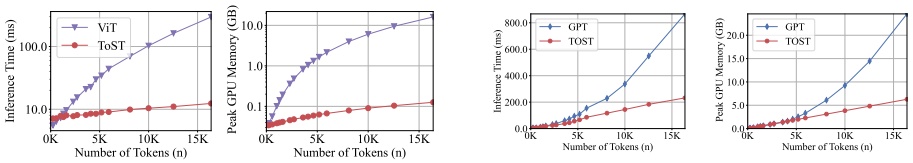
$$\boldsymbol{\nu}(\boldsymbol{z}_j \mid \boldsymbol{U}_{[K]}) \doteq \operatorname{softmax}\left(\frac{1}{2\eta}\begin{bmatrix}\|\boldsymbol{U}_1^\top \boldsymbol{z}_j\|_2^2 \\ \vdots \\ \|\boldsymbol{U}_K^\top \boldsymbol{z}_j\|_2^2\end{bmatrix}\right), \quad \forall j \in [n], \tag{5.3.17}$$

donde η se convierte en un parámetro de temperatura aprendible. De este modo, dada una característica de entrada $\boldsymbol{Z}$, se estima $\boldsymbol{\Pi}$ mediante (5.3.16) y a continuación se calcula el operador de atención. Al combinar la construcción de $\boldsymbol{\Pi}$ en (5.3.16) con (5.3.12), se obtiene el operador *Token Statistics Self-Attention* (TSSA):

$$\operatorname{TSSA}(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq -\frac{\tau}{n}\sum_{k=1}^{K} \boldsymbol{U}_k \boldsymbol{D}(\boldsymbol{Z}, \boldsymbol{\pi}_k \mid \boldsymbol{U}_k)\boldsymbol{U}_k^\top \boldsymbol{Z}\operatorname{diag}(\boldsymbol{\pi}_k), \tag{5.3.18}$$

donde $\boldsymbol{\pi}_k$ son las columnas de $\boldsymbol{\Pi} = \boldsymbol{\Pi}(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]})$ definidas en (5.3.16) y $\boldsymbol{D}$ se define en (5.3.10).

Partiendo de una arquitectura transformer estándar y sustituyendo el operador de atención por TSSA, obtenemos una arquitectura denominada Token Statistics Transformer (ToST), que se muestra en la Figura 5.20. Quisiéramos volver a destacar que el operador TSSA mitiga la complejidad temporal y de memoria habitual del operador de atención, en particular al reducir la complejidad temporal y de memoria cuadrática a lineal, lo que hace que ToST sea mucho más eficiente que su homólogo diseñado empíricamente. A continuación, mostramos brevemente el rendimiento y la eficiencia empíricos de la arquitectura ToST.



(a) (b)

Figura 5.21: **Complejidad temporal y de memoria de ToST en la práctica** en comparación con ViT (a) y GPT-2 (b) diseñados empíricamente. Observamos que ToST solo requiere un tiempo y una memoria lineales, en comparación con los costos cuadráticos de los transformers tradicionales.

Tabla 5.3: **Precisión «top-1» de ToST en diversos conjuntos de datos con diferentes tamaños de modelo tras el preentrenamiento en ImageNet.** Para ImageNet/ImageNetReaL, evaluamos directamente la precisión «top-1». Para otros conjuntos de datos, preentrenamos el modelo en ImageNet y luego evaluamos el rendimiento del aprendizaje por transferencia mediante el ajuste fino. Observamos que el rendimiento de ToST aumenta con el tamaño del modelo y compite con el ViT diseñado empíricamente.

Conj.Datos	ToST-T(iny)	ToST-S(mall)	ToST-M(edium)	ViT-S	ViT-B(ase)
# parámetros	5.8M	22.6M	68.1M	22.1M	86.6 M
ImageNet	67.3	77.9	80.3	79.8	81.8
ImageNet ReaL	72.2	84.1	85.6	85.6	86.7
CIFAR10	95.5	96.5	97.5	98.6	98.8
CIFAR100	78.3	82.7	84.5	88.8	89.3
Oxford Flowers-102	88.6	92.8	94.2	94.0	95.7
Oxford-IIIT-Pets	85.6	91.1	92.8	92.8	94.1

5.3.3 CRATE causal

Algunas de las aplicaciones más populares de las redes profundas se dan en el ámbito de los *datos secuenciales*, donde, a diferencia del ejemplo anterior de las imágenes, los datos siguen un orden natural. Por ejemplo, a diferencia de las imágenes (donde se tiende a observar la imagen completa de una sola vez), el lenguaje se procesa token por token (de manera causal). Este simple hecho impulsa el desarrollo de modelos de lenguaje a gran escala (LLM), que se entrenan para predecir (la distribución de probabilidad del) siguiente token de una secuencia basándose *únicamente en el historial de los tokens anteriores*. A continuación, para obtener una muestra con un LLM, basta con realizar un muestreo iterativo a partir de la distribución predicha para el siguiente token y, luego, añadir el token muestreado al historial. Para entrenar los LLM u otros modelos, como los de generación de vídeo, es necesario desarrollar arquitecturas capaces de realizar este cálculo causal de manera eficiente.

Teniendo esto en cuenta y basándonos en la comprensión de los procesos autoregresivos causales del Capítulo 1, decimos que un codificador f es *causal*

Tabla 5.4: **Rendimiento de ToST en tareas de contexto largo.** Presentamos los resultados del rendimiento en Long Range Arena, un banco de pruebas para tareas de contexto largo (n grande). En este *benchmark*, ToST presenta un rendimiento y una eficiencia rotundamente superiores en comparación con todas las variantes de Transformer, incluido el propio transformer.

Modelo	ListOps	Texto	Recuperación	Imagen	Pathfinder	Promedio
Reformer	37.27	56.10	53.40	38.07	68.50	50.56
BigBird	36.05	64.02	59.29	40.83	74.87	54.17
LinFormer	16.13	65.90	53.09	42.34	75.30	50.46
Performer	18.01	65.40	53.82	42.77	77.05	51.18
Transformer	37.11	65.21	79.14	42.94	71.83	59.24
ToST	37.25	66.75	79.46	46.62	69.41	59.90

si para cada entrada $\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_N]$ se cumple que

$$f(\boldsymbol{X})_{1:N-1} = f([\boldsymbol{x}_1, \dots, \boldsymbol{x}_N])_{1:N-1} = f([\boldsymbol{x}_1, \dots, \boldsymbol{x}_{N-1}]) \tag{5.3.19}$$

$$= f(\boldsymbol{X}_{1:N-1}), \tag{5.3.20}$$

donde $\boldsymbol{Z}_{1:N-1} = [\boldsymbol{z}_1, \dots, \boldsymbol{z}_{N-1}]$, etc. En español, esto significa que las primeras $N-1$ características son iguales a la salida del codificador sobre los primeros $N-1$ *embeddings* de tokens de la entrada; dicho de manera aún más sencilla, significa que podemos calcular las características de cada token *una por una* y esto sería equivalente a calcular las características de toda la entrada. Esta causalidad es necesaria para las aplicaciones mencionadas anteriormente (por ejemplo, los LLM). En particular, las implementaciones de CRATE y ToST presentadas anteriormente *no son causales* (la demostración se deja como ejercicio).

Podemos construir arquitecturas causales *white-box* mediante diversos métodos. Aquí presentaremos un método sencillo que se basa en nuestro marco de optimización desenrollada anterior. Específicamente, calculamos las características $\boldsymbol{z}_1, \dots, \boldsymbol{z}_N$ correspondientes a los primeros N tokens de la entrada *uno a la vez* para optimizar el objetivo de aprendizaje de la representación, como la reducción esparsa de la tasa de codificación (5.2.5):

$$\max_{\boldsymbol{z}_i} \ \Delta R(\boldsymbol{Z}_{1:i} \mid \boldsymbol{U}_{[K]}) - \lambda \|\boldsymbol{Z}_{1:i}\|_1, \ \forall i \in [N]. \tag{5.3.21}$$

Si aplicamos el procedimiento de desenrollado en dos pasos que dio lugar a CRATE, obtenemos la iteración:

$$\boldsymbol{z}_i^{\ell+1/2} \approx \boldsymbol{z}_i^{\ell} - \kappa \nabla_{\boldsymbol{z}_i} R(\boldsymbol{Z}_{1:i}^{\ell} \mid \boldsymbol{U}_{[K]}^{\ell}), \tag{5.3.22}$$

$$\boldsymbol{z}_i^{\ell+1} \approx \arg\min_{\boldsymbol{z}_i} \left\{ \lambda \|\boldsymbol{z}_i\|_1 + \frac{1}{2} \|\boldsymbol{z}_i^{\ell+1/2} - \boldsymbol{D}^{\ell} \boldsymbol{z}_i\|_F^2 \right\}, \tag{5.3.23}$$

o utilizando la misma conversión de estos dos pasos a operadores de red que usamos para CRATE,

$$\boldsymbol{z}_i^{\ell+1/2} = \left(1 - \frac{\kappa p}{N\epsilon^2}\right) \boldsymbol{z}_i^{\ell} + \frac{\kappa p}{N\epsilon^2} \operatorname{MSSA}(\boldsymbol{Z}_{1:i}^{\ell} \mid \boldsymbol{U}_{[K]}^{\ell})_i, \tag{5.3.24}$$

$$\boldsymbol{z}_i^{\ell+1} = \mathrm{ISTA}(\boldsymbol{z}_i^{\ell+1/2} \mid \boldsymbol{D}^\ell). \tag{5.3.25}$$

Analicemos esta iteración con un poco más de detalle. En primer lugar, observemos que, por construcción, esta secuencia de características corresponde a un codificador causal. A continuación, supongamos que nos encontramos en un escenario en que ya se han calculado las cantidades $\boldsymbol{Z}_{1:i-1}^{\ell+1} = [\boldsymbol{z}_1^{\ell+1}, \dots, \boldsymbol{z}_{i-1}^{\ell+1}]$, habiendo calculado las cantidades

$$(\boldsymbol{U}_k^\ell)^\top \boldsymbol{Z}_{1:i-1}^\ell = [(\boldsymbol{U}_k^\ell)^\top \boldsymbol{z}_1^\ell, \dots, (\boldsymbol{U}_k^\ell)^\top \boldsymbol{z}_{i-1}^\ell] \tag{5.3.26}$$

a medida que avanzamos y queremos calcular $\boldsymbol{z}_i^{\ell+1}$ (por ejemplo, en el caso de la inferencia de un modelo de lenguaje grande). En este caso, cabe señalar que la regla de actualización para $\boldsymbol{z}_i^{\ell+1/2}$ puede simplificarse de la siguiente manera:

$$\begin{aligned}
&\boldsymbol{z}_i^{\ell+1/2} && (5.3.27)\\
&= \left(1 - \frac{\kappa p}{N\epsilon^2}\right) \boldsymbol{z}_i^\ell + \frac{\kappa p}{N\epsilon^2} \,\mathrm{MSSA}(\boldsymbol{Z}_{1:i}^\ell \mid \boldsymbol{U}_{[K]}^\ell)_i && (5.3.28)\\
&= \left(1 - \frac{\kappa p}{N\epsilon^2}\right) \boldsymbol{z}_i^\ell + \frac{\kappa p^2}{N^2\epsilon^4} \sum_{k=1}^K \boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{Z}_{1:i} \,\mathrm{softmax}((\boldsymbol{U}_k^\top \boldsymbol{Z}_{1:i})^\top (\boldsymbol{U}_k^\top \boldsymbol{z}_i)) && (5.3.29)\\
&= \left(1 - \frac{\kappa p}{N\epsilon^2}\right) \boldsymbol{z}_i^\ell + \frac{\kappa p^2}{N^2\epsilon^4} \sum_{k=1}^K \boldsymbol{U}_k \boldsymbol{U}_k^\top [\boldsymbol{Z}_{1:i-1}, \boldsymbol{z}_i] \,\mathrm{softmax}((\boldsymbol{U}_k^\top [\boldsymbol{Z}_{1:i-1}, \boldsymbol{z}_i])^\top (\boldsymbol{U}_k^\top \boldsymbol{z}_i)) && (5.3.30)\\
&= \left(1 - \frac{\kappa p}{N\epsilon^2}\right) \boldsymbol{z}_i^\ell + \frac{\kappa p^2}{N^2\epsilon^4} \sum_{k=1}^K \boldsymbol{U}_k [\boldsymbol{U}_k^\top \boldsymbol{Z}_{1:i-1}, \boldsymbol{U}_k^\top \boldsymbol{z}_i] \,\mathrm{softmax}(([\boldsymbol{U}_k^\top \boldsymbol{Z}_{1:i-1}, \boldsymbol{U}_k^\top \boldsymbol{z}_i])^\top (\boldsymbol{U}_k^\top \boldsymbol{z}_i)). && (5.3.31)
\end{aligned}$$

Este paso se vuelve *muy eficiente* si almacenamos en caché $\boldsymbol{U}_k^\top \boldsymbol{Z}_{1:i-1}$ de los pasos anteriores y le añadimos una sola columna cada vez que calculamos la proyección $\boldsymbol{U}_k^\top \boldsymbol{z}_i$. Es decir, reducimos considerablemente el número de productos de matrices grandes y los reemplazamos por cargas de caché y productos de matriz-vector, lo cual es, en general, mucho más económico en términos de complejidad temporal. Este almacenamiento en caché es la razón por la que los modelos generativos causales, como los LLM, pueden muestrear de manera eficiente miles de tokens por segundo, incluso si cada paso del entrenamiento tarda unos segundos por sí solo. La caché para las proyecciones en el subespacio de las características también se conoce como la «caché KV».

Por último, consideremos el caso en el que debemos *entrenar* un modelo CRATE causal y queremos encontrar la forma más eficiente de hacerlo simultáneamente, dada una secuencia de entrada completa $\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_N]$. El paso ISTA se puede paralelizar para convertirse en el paso ISTA regular del CRATE no causal, es decir,

$$\mathrm{ISTA}(\boldsymbol{Z} \mid \boldsymbol{D}) \doteq [\mathrm{ISTA}(\boldsymbol{z}_1 \mid \boldsymbol{D}), \dots, \mathrm{ISTA}(\boldsymbol{z}_N \mid \boldsymbol{D})], \tag{5.3.32}$$

lo que implica, por tanto, que el paso ISTA sigue siendo el mismo que el CRATE no causal. El paso MSSA resulta más interesante, ya que cambia de manera

Tabla 5.5: Pérdida de entropía cruzada en la validación *zero-shot* del modelo Causal-CRATE-Base y de los modelos GPT2-Small y GPT2-Base, evaluada en la subconjunto de prueba de los conjuntos de datos (cuanto menor, mejor).

	#parámetros	OWT	LAMBADA	WikiText	PTB	Prom
GPT2-Base	124M	2.85	4.12	3.89	4.63	3.87
GPT2-Small	64M	3.04	4.49	4.31	5.15	4.25
Causal-CRATE-Base	60M	3.37	4.91	4.61	5.53	4.61

considerable. Para ver cómo cambia, cabe señalar que cada operador MSSA tiene una estructura interesante que merece cierta atención. Concretamente, si definimos el *operador MSSA causal* como la matriz en bloque:

$$\mathrm{CMSSA}(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq [\mathrm{MSSA}(\boldsymbol{Z}_{1:1} \mid \boldsymbol{U}_{[K]})_1, \ldots, \mathrm{MSSA}(\boldsymbol{Z}_{1:N} \mid \boldsymbol{U}_{[K]})_N], \tag{5.3.33}$$

entonces, analizando el álgebra de softmax (la demostración se deja, una vez más, como ejercicio), podemos demostrar que

$$\mathrm{CMSSA}(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq \tfrac{p}{N\epsilon^2} [\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K] \begin{bmatrix} \mathrm{softmax}((\boldsymbol{U}_1^\top \boldsymbol{Z})^\top (\boldsymbol{U}_1^\top \boldsymbol{Z}) + \boldsymbol{M}_N) \\ \vdots \\ \mathrm{softmax}((\boldsymbol{U}_K^\top \boldsymbol{Z})^\top (\boldsymbol{U}_K^\top \boldsymbol{Z}) + \boldsymbol{M}_N) \end{bmatrix}, \tag{5.3.34}$$

$$\text{where } \boldsymbol{M}_N = \begin{bmatrix} 0 & -\infty & -\infty & \ldots & -\infty & -\infty \\ 0 & 0 & -\infty & \ldots & -\infty & -\infty \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \ldots & 0 & -\infty \\ 0 & 0 & 0 & \ldots & 0 & 0 \end{bmatrix}. \tag{5.3.35}$$

Aquí, la matriz $\boldsymbol{M}_N \in \mathbb{R}^{N \times N}$ es otra forma de codificar que ninguna característica del CMSSA, es decir, el MSSA causal, depende de una característica futura, ya que las entradas relevantes del argumento de la función softmax son $-\infty$ y por lo tanto se establecen en 0 después de la función exponencial dentro de la función softmax, con lo que, en la práctica, se ignoran. Esta matriz $\boldsymbol{M}_N$ se denomina a veces *máscara de atención (causal)*.

En definitiva, cuando disponemos de la secuencia completa $\boldsymbol{Z}^\ell$, podemos calcular $\boldsymbol{Z}^{\ell+1}$ en un tiempo similar o inferior al de la capa CRATE estándar (ya que $\boldsymbol{M}_N$ está codificada en forma fija, es independiente de la entrada y nos permite ignorar muchos elementos para la función softmax). Por lo tanto, podemos definir y entrenar el modelo completo de secuencia a secuencia de la misma manera que la arquitectura regular.

Cuando aplicamos esto al modelado de lenguaje (los detalles se proporcionarán en el Capítulo 8), observamos que podemos obtener resultados razonables en comparación con modelos de lenguaje de tamaño similar diseñados empíricamente, tal y como se muestra en la Tabla 5.5.

A pesar de las ventajas teóricas de la arquitectura causal CRATE, la brecha en las tareas de modelado del lenguaje (y, en términos más generales, de modelado de secuencias) sigue existiendo. A continuación, analizamos dos

estrategias que podrían cerrar esta brecha; ambas son objeto de líneas de investigación activas.

Ejemplo 5.3 (Mejores objetivos y estrategias de desenrollado). Una vía a considerar es que el objetivo de la reducción de la tasa de esparcidad no sea el más adecuado para los datos de secuencias causales. Por ejemplo, podríamos querer asegurarnos de que las características tengan algún tipo de relación o estructura temporal, como por ejemplo, que sean generadas por un sistema dinámico lineal con inicialización (de mezcla) gaussiana y ruido de paso gaussiano. Existe una línea de investigación reciente, pero explosivamente popular, dedicada a modelar de manera eficiente y explícita la dinámica en las características mediante los denominados *modelos de espacio de estado* (SSM, denominados así en contravención de la noción clásica de «modelos de espacio de estado» discutida en el Capítulo 1) [GD24; YKH24; YWZ+24]. Estos modelos son prometedores desde el punto de vista empírico porque evitan la necesidad de una atención entre todos los pares, eludiendo así la complejidad computacional y de memoria del operador MSSA y de su «primo» convencional, la atención multicabezal. A pesar de esta mejora en la eficiencia, son muy eficaces e incluso se han utilizado en algunos modelos de lenguaje a gran escala [TZL+25]. Mediante el diseño de objetivos más sofisticados y estrategias de «desenrollado», podríamos construir una arquitectura SSM tipo «white-box», o un híbrido SSM-transformer, y así cerrar la brecha en las tareas causales. ■

Ejemplo 5.4 (Estrategias de cálculo en tiempo de inferencia). Otra vía para mejorar los modelos causales consiste en utilizar más estrategias denominadas *de cálculo en tiempo de inferencia*: durante la inferencia (como el muestreo a partir de un modelo de lenguaje grande, o LLM), hay que encontrar la manera de dedicar más recursos computacionales para obtener un mejor resultado. Si esto se logra, se pueden resolver problemas más complejos asignando más recursos computacionales a la tarea en cuestión.

Un enfoque sencillo para esto consiste en aplicar un bucle al modelo: introducir las características finales en las capas iniciales de secuencia a secuencia y permitir que pase por más rondas de eliminación iterativa de ruido. Esta metodología se conoce en la literatura como «transformers en bucle» (en el caso de que el modelo base sea un transformer diseñado empíricamente); se ha demostrado que este enfoque ofrece ventajas en algunas tareas especialmente difíciles [GRS+23; SDL+25; YLN+23]. Sin embargo, desde nuestro punto de vista, resulta más razonable recorrer la *misma capa* varias veces, aplicando varios pasos de eliminación de ruido o compresión locales sobre el *mismo modelo* en cada capa antes de pasar a la siguiente.

Otra posible metodología adicional se basa en nuestra comprensión de la estructura de baja dimensión en las características y los parámetros del modelo. Por ejemplo, dado que se supone que las matrices $\boldsymbol{U}_{[K]}$ son soportes de los subespacios generados por las características, podríamos actualizarlas en tiempo de inferencia utilizando un algoritmo en línea, como el (G)PCA en línea. Pasos similares podrían permitir adaptar el diccionario en el momento de la inferencia. Esto nos permitiría mejorar y especializar nuestros operadores locales

de eliminación de ruido o compresión en cada capa para las características, lo que, por lo tanto, podría dar lugar a un mejor rendimiento.

Se han realizado numerosos intentos empíricos para mejorar los métodos de cómputo en tiempo de inferencia. Sigue siendo un campo de investigación muy activo en la actualidad. ■

5.4 Resumen y Notas

Los materiales presentados en este capítulo se basan en una serie de trabajos recientes sobre este tema, incluyendo [CYY+22; WLP+24; WLY+25; WDL+25; YBP+23]. Estas contribuciones abarcan tanto avances teóricos como metodologías prácticas para construir redes profundas interpretables mediante la optimización desenrollada. La mayoría de los resultados y demostraciones clave discutidos en este capítulo se derivan directamente de estos trabajos fundamentales o se inspiran en ellos.

La idea de «desenrollar» un algoritmo de optimización para construir una red neuronal se remonta al trabajo pionero [GL10]. En este trabajo, los autores demostraron que los algoritmos de codificación esparsa —como el algoritmo de contracción y umbralización iterativa (ISTA)— pueden desenrollarse para formar perceptrones multicapa (MLP), uniendo de manera efectiva la optimización iterativa y el diseño de redes neuronales. Cabe destacar que [MLE19] demostró que estas redes «desenrolladas» son más interpretables, eficientes y efectivas en comparación con las redes genéricas.

En este capítulo, partimos de esta perspectiva para desarrollar arquitecturas de redes profundas transparentes *white-box* basadas en principios, mediante el desenrollamiento de algoritmos de optimización diseñados para minimizar objetivos bien fundamentados, como el objetivo de reducción (esparsa) de la tasa, presentado anteriormente. Este enfoque no solo aclara la función de cada capa en la red, sino que también proporciona una base teórica para las decisiones arquitectónicas, alejándonos desde el ensayo y error empírico hacia un diseño interpretable y orientado a objetivos. A continuación, comparamos las redes neuronales profundas (DNN) convencionales, que suelen construirse mediante un diseño empírico y un ajuste heurístico, con nuestras arquitecturas ReduNet, fundamentadas matemáticamente:

	DNNs convencionales	ReduNets
Objetivos	Ajuste de entrada/salida	ganancia de información
Arquitecturas profundas	prueba y error	optimización iterativa
Operadores de capa	empírica	gradiente proyectedo
Invariancia a traslaciones	CNNs y aumento	ReduNets invariantes
Inicializaciones	aleatoria/pre-diseño	desenrollada hacia adelante
Entrenamiento/ajuste fino	retroprop	adelante/retroprop
Interpretabilidad	*black-box*	*white-box*
Representaciones	ocultas/latentes	subespacios incoherentes

5.5 Ejercicios y extensiones

Ejercicio 5.1. Sea $\boldsymbol{Z} = [\boldsymbol{Z}_1, \ldots, \boldsymbol{Z}_K] \in \mathbb{R}^{d \times N}$ tal que $\boldsymbol{Z}_k \in \mathbb{R}^{d \times N_k}$ para todo $k \in [K]$. Para algún $\alpha > 0$, sea

$$R(\boldsymbol{Z}) = \log\det\left(\boldsymbol{I} + \alpha \boldsymbol{Z}\boldsymbol{Z}^\top\right).$$

1. Dada cualquier dirección $\boldsymbol{D} \in \mathbb{R}^{d \times m}$, demuestre que $\nabla R(\boldsymbol{Z}) = \alpha \boldsymbol{X}^{-1}\boldsymbol{Z}$ y

   $$\begin{aligned}&\nabla^2 R(\boldsymbol{Z})[\boldsymbol{D}, \boldsymbol{D}]\\ &= \alpha \mathrm{Tr}\left(\boldsymbol{X}^{-1}\boldsymbol{D}\boldsymbol{D}^\top\right) - \frac{\alpha^2}{2}\mathrm{Tr}\left(\boldsymbol{X}^{-1}\left(\boldsymbol{Z}\boldsymbol{D}^\top + \boldsymbol{D}\boldsymbol{Z}^\top\right)\boldsymbol{X}^{-1}\left(\boldsymbol{Z}\boldsymbol{D}^\top + \boldsymbol{D}\boldsymbol{Z}^\top\right)\right),\end{aligned}$$

   donde $\boldsymbol{X} \doteq \boldsymbol{I} + \alpha \boldsymbol{Z}\boldsymbol{Z}^\top$. *Pista:* Obsérvese que

   $$\nabla^2 R(\boldsymbol{Z})[\boldsymbol{D}, \boldsymbol{D}] \doteq \left\langle \lim_{t \to 0} \frac{\nabla R(\boldsymbol{Z} + t\boldsymbol{D}) - \nabla R(\boldsymbol{Z})}{t}, \boldsymbol{D} \right\rangle.$$

2. Muestre que

   $$R(\boldsymbol{Z}) \leq \sum_{k=1}^{K} \log\det\left(\boldsymbol{I} + \alpha \boldsymbol{Z}_k \boldsymbol{Z}_k^\top\right),$$

   donde la igualdad se cumple si y solo si $\boldsymbol{Z}_k^\top \boldsymbol{Z}_l = \boldsymbol{0}$ para todo $k \neq l \in [K]$.

3. Dado cierto $\alpha > 0$, sea $\alpha_k = N\alpha/N_k$ para cada $k \in [K]$. Derive la fórmula cerrada para el punto crítico de primer orden de la siguiente función:

   $$f(\boldsymbol{Z}_k) = \frac{1}{2}\log\det\left(\boldsymbol{I} + \alpha \boldsymbol{Z}_k \boldsymbol{Z}_k^\top\right) - \frac{N_k}{2N}\log\det\left(\boldsymbol{I} + \alpha_k \boldsymbol{Z}_k \boldsymbol{Z}_k^\top\right) - \frac{\lambda}{2}\|\boldsymbol{Z}_k\|_F^2.$$

   Pista: Sea $r_k = \mathrm{rank}(\boldsymbol{Z}_k)$. Consideremos la siguiente descomposición en valores singulares de $\boldsymbol{Z}_k$:

   $$\boldsymbol{Z}_k = \boldsymbol{P}_k \boldsymbol{\Sigma}_k \boldsymbol{Q}_k^\top = \begin{bmatrix}\boldsymbol{P}_{k,1} & \boldsymbol{P}_{k,2}\end{bmatrix}\begin{bmatrix}\tilde{\boldsymbol{\Sigma}}_k & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{0}\end{bmatrix}\begin{bmatrix}\boldsymbol{Q}_{k,1}^\top\\ \boldsymbol{Q}_{k,2}^\top\end{bmatrix}, \tag{5.5.1}$$

donde $\boldsymbol{P}_k \in \mathcal{O}^d$ con $\boldsymbol{P}_{k,1} \in \mathbb{R}^{d\times r_k}$ y $\boldsymbol{P}_{k,2} \in \mathbb{R}^{d\times(d-r_k)}$, $\boldsymbol{\Sigma}_k \in \mathbb{R}^{d\times N_k}$ con $\tilde{\boldsymbol{\Sigma}}_k \in \mathbb{R}^{r_k\times r_k}$ siendo una matriz diagonal, y $\boldsymbol{Q}_k \in \mathcal{O}^{N_k}$ con $\boldsymbol{Q}_{k,1} \in \mathbb{R}^{N_k\times r_k}$ y $\boldsymbol{P}_{k,2} \in \mathbb{R}^{N_k\times(N_k-r_k)}$.

Ejercicio 5.2 (Serie de Neumann para la inversa de una matriz). Sea $\boldsymbol{A} \in \mathbb{R}^{n\times n}$. Si $\|\boldsymbol{A}\| < 1$, muestre que

$$(\boldsymbol{I} - \boldsymbol{A})^{-1} = \sum_{k=0}^{\infty} \boldsymbol{A}^k. \tag{5.5.2}$$

Pista: La demostración consta de dos pasos.

(i) **Paso 1**: Demuestre que la serie infinita $\sum_{k=0}^{\infty} \boldsymbol{A}^k$ converge cuando $\|\boldsymbol{A}\| < 1$ utilizando $\|\boldsymbol{A}^k\| \le \|\boldsymbol{A}\|^k$.

(ii) **Paso 2**: Calcule el producto matricial $(\boldsymbol{I} - \boldsymbol{A}) \sum_{k=0}^{\infty} \boldsymbol{A}^k$.

Ejercicio 5.3. En este ejercicio consideraremos una medida de la tasa de codificación $R_\epsilon^{c,\mu}$ que utiliza las medias de las distribuciones, es decir, una tasa de codificación para tokens extraídos (no i.i.d.) del modelo de mezcla gaussiana de rango bajo

$$\boldsymbol{z}_i \sim \frac{1}{K} \sum_{k=1}^{K} \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{U}_k \boldsymbol{U}_k^\top). \tag{5.5.3}$$

1. Justifique que una tasa de codificación adecuada para $\boldsymbol{Z}$ en este modelo es|

$$R_\epsilon^{c,\mu}(\boldsymbol{Z}) = \sum_{k=1}^{K} R_\epsilon(\boldsymbol{U}_k^\top (\boldsymbol{Z} - \boldsymbol{\mu}_k \mathbf{1}^\top)). \tag{5.5.4}$$

2. De manera análogo a cómo derivamos el MSSA, derive un nuevo operador similar a la atención a partir de esta tasa de codificación. ¿Cuáles son las diferencias clave?

Ejercicio 5.4. Calcule los gradientes en (5.3.9) y (5.3.10).

Ejercicio 5.5. Demuestre el Corolario 5.1 cuando $Kp \le d$.

Ejercicio 5.6. Derive una versión causal del operador TSSA.

Algoritmo 5.1: Algoritmo de entrenamiento de ReduNet

Input: $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D \times N}$, $\boldsymbol{\Pi} = \{\boldsymbol{\Pi}_k\}_{k=1}^K$, $\epsilon > 0$, λ y una tasa de aprendizaje η.

Output: Los parámetros aprendidos $\{\boldsymbol{E}^\ell\}_{\ell=1}^L, \{\{\boldsymbol{C}_k^\ell\}_{k=1}^K\}_{\ell=1}^L, \{\gamma_k\}_{k=1}^k$

procedure ReduNetTraining($\boldsymbol{X}, \boldsymbol{\Pi}, \epsilon, \lambda, \eta$)

`# Definir constantes`

$\alpha \leftarrow d/(N\epsilon^2)$

for $k \in \{1, \ldots, K\}$ **do**

$\alpha_k \leftarrow D/(\mathrm{tr}(\boldsymbol{\Pi}_k)\epsilon^2)$

$\gamma_k \leftarrow \mathrm{tr}(\boldsymbol{\Pi}_k)/D$

end for

`# Iteración capa por capa de ReduNet`

$\boldsymbol{Z}^1 = \left[\boldsymbol{z}_1^1, \ldots, \boldsymbol{z}_N^1\right] \leftarrow \boldsymbol{X}$ ▷ Inicializar la iteración ReduNet por capas

for $\ell \in \{1, \ldots, L\}$ **do**

`# Paso 1: Calcular los parámetros de la red` $\boldsymbol{E}^\ell, \{\boldsymbol{C}_k^\ell\}_{k=1}^K$

$\boldsymbol{E}^\ell \leftarrow \alpha\left(\boldsymbol{I} + \alpha \boldsymbol{Z}^\ell (\boldsymbol{Z}^\ell)^\top\right)^{-1} \in \mathbb{R}^{d \times d}$

for $k \in \{1, \ldots, K\}$ **do**

$\boldsymbol{C}_k^\ell \leftarrow \alpha_k\left(\boldsymbol{I} + \alpha_k \boldsymbol{Z}^\ell \boldsymbol{\Pi}_k (\boldsymbol{Z}^\ell)^\top\right)^{-1} \in \mathbb{R}^{d \times d}$

end for

`# Paso 2: Actualizar las características` $\boldsymbol{Z}^\ell$

for $i \in \{1, \ldots, N\}$ **do**

`# Calcular las asignaciones blandas` $\hat{\boldsymbol{\pi}}(\boldsymbol{z}_i^\ell)$

$\hat{\boldsymbol{\pi}}(\boldsymbol{z}_i^\ell) \leftarrow \mathrm{softmax}(-\lambda[\|\boldsymbol{C}_1^\ell \boldsymbol{z}_i^\ell\|_2, \ldots, \|\boldsymbol{C}_K^\ell \boldsymbol{z}_i^\ell\|_2]) \in [0,1]^K$

`# Actualizar las características` $\boldsymbol{z}_i^{\ell+1}$ `a partir de` $\boldsymbol{z}_i^\ell$

$$\boldsymbol{z}_i^{\ell+1} \leftarrow \mathcal{P}_{\mathbb{S}^{d-1}}\left(\boldsymbol{z}_i^\ell + \eta\left(\boldsymbol{E}^\ell \boldsymbol{z}_i^\ell - \sum_{k=1}^K \gamma_k \hat{\pi}_k(\boldsymbol{z}_i^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}_i^\ell\right)\right) \in \mathbb{R}^d$$

end for

end for

`# Devolver todos los parámetros de la red.`

return $\{\boldsymbol{E}^\ell\}_{\ell=1}^L, \{\{\boldsymbol{C}_k^\ell\}_{k=1}^K\}_{\ell=1}^L, \{\gamma_k\}_{k=1}^K$

end procedure

Algoritmo 5.2: Algoritmo de evaluación para ReduNet

Input: Entrada $\boldsymbol{x} \in \mathbb{R}^D$, parámetros de la red $\{\boldsymbol{E}^\ell\}_{\ell=1}^L, \{\{\boldsymbol{C}_k^\ell\}_{k=1}^K\}_{\ell=1}^L, \{\gamma_k\}_{k=1}^K$, tasa de aprendizaje λ
Output: característica $\boldsymbol{z}^{L+1}$

procedure ReduNetEvaluation($\boldsymbol{x}$)
    `# Inicializar la iteración por capas de ReduNet`
    $\boldsymbol{z}^1 \leftarrow \boldsymbol{x} \in \mathbb{R}^D$
    for $\ell \in \{1, \ldots, L\}$ **do**
        `# Calcular las asignaciones difusas` $\hat{\boldsymbol{\pi}}(\boldsymbol{z}^\ell)$
        $\hat{\boldsymbol{\pi}}(\boldsymbol{z}^\ell) \leftarrow \operatorname{softmax}(-\lambda \left[\|\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell\|_2, \ldots, \|\boldsymbol{C}_K^\ell \boldsymbol{z}^\ell\|_2\right]) \in [0,1]^K$
        `# Actualizar característica` $\boldsymbol{z}^{\ell+1}$ `utilizando` $\boldsymbol{z}^\ell$
        $\boldsymbol{z}^{\ell+1} \leftarrow \mathcal{P}_{\mathbb{S}^{d-1}}\left(\boldsymbol{z}^\ell + \eta \left(\boldsymbol{E}^\ell \boldsymbol{z}^\ell - \sum_{k=1}^K \gamma_k \hat{\pi}_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell\right)\right) \in \mathbb{R}^d$
    end for
    `# Devolver las características de salida`
    return $\boldsymbol{z}^{L+1}$
end procedure

Capítulo 6

Representaciones consistentes y autoconsistentes

"*Hacer todo lo más simple posible, pero no más simple.*"
– Albert Einstein

En los capítulos anteriores, hemos argumentado que el objetivo fundamental del aprendizaje es *identificar* una distribución de datos con un soporte de baja dimensión y *transformarla* en una representación compacta y estructurada. Esto implica dos problemas relacionados:

1. **Aprendizaje de distribuciones:** ¿Cómo podemos determinar la distribución de los datos $\boldsymbol{X}$ para permitir un muestreo o una generación generalizables?

2. **Aprendizaje de representaciones:** ¿Cómo mapeamos los datos $\boldsymbol{X}$ a una representación compacta y estructurada $\boldsymbol{Z}$, de tal manera que la representación sea suficiente para reconstruir $\boldsymbol{X}$?

En los capítulos 3 y 4, hemos desarrollado una solución computacional completa para el problema del aprendizaje de distribuciones en el contexto de distribuciones de datos generales, basada en el proceso de eliminación de ruido por difusión. En el Capítulo 4, hemos definido una función objetivo basada en la compresión, la ganancia de información, cuya maximización conduce a representaciones compactas y estructuradas. Y en el Capítulo 5, hemos visto que las capas de las arquitecturas populares de redes profundas pueden interpretarse como pasos de optimización incrementales hacia la maximización de la ganancia de información. Esto nos sitúa en una posición ideal para presentar una solución con fundamentos matemáticos al problema del aprendizaje de representaciones.

Sin embargo, cuando intentamos alcanzar un objetivo específico mediante la optimización, no hay garantía de que la solución $\boldsymbol{Z}$ encontrada al final mediante la optimización voraz sea la solución correcta. De hecho, incluso si el proceso de optimización logra encontrar la solución óptima global $\boldsymbol{Z}^*$, no hay garantía de que dicha solución corresponda a una representación completa de la distribución de los datos.[1] Por lo tanto, una pregunta que queda pendiente es: ¿cómo podemos asegurarnos de que la representación aprendida de la distribución de los datos sea correcta o lo suficientemente buena?

Por supuesto, la única forma de verificar esto es determinar si existe una función de decodificación, digamos g, capaz de decodificar la representación aprendida para reproducir los datos originales (su distribución) con suficiente precisión:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}=f} \boldsymbol{Z} \xrightarrow{\mathcal{D}=g} \hat{\boldsymbol{X}} \tag{6.0.1}$$

en términos de alguna medida de similitud:

$$d(\boldsymbol{X}, \hat{\boldsymbol{X}}). \tag{6.0.2}$$

Esto nos lleva a la noción de *representación consistente*.[2] Como ya hemos aludido brevemente en el Capítulo 1 (véase la Figura 1.25), la *autocodificación* —que integra los procesos de codificación y decodificación— constituye un marco natural para el aprendizaje de dicha representación. Hemos estudiado algunos casos especiales importantes en el Capítulo 2 donde los soportes de la distribución son lineales por tramos. En la Sección 6.1, estudiaremos cómo extender la autocodificación a clases más generales de distribuciones cuyos soportes son no lineales, tal como se ilustra en la figura 6.1, garantizando la consistencia entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$.

Dado este formalismo clásico de autocodificación, surge una pregunta natural: *¿Dónde termina el aprendizaje de distribuciones y dónde comienza el aprendizaje de representaciones?* Como veremos, la respuesta se reduce a la *eficiencia*. Por lo general, resulta mucho más eficaz, tal y como demuestra la práctica moderna, aprender una representación compacta de la distribución de datos a modo de «preparación» de la distribución, y solo entonces llevar a cabo el aprendizaje de la distribución sobre la representación así obtenida. Esta es la metodología predominante en problemas prácticos. Demostraremos los vínculos entre los fundamentos que hemos desarrollado en el Capítulos 4 y 5 y los últimos avances en la Sección 6.1.5, donde el marco del «autocodificador de representaciones» muestra cómo las representaciones aprendidas en bucle abierto (que analizamos en la Sección 4.3.2 y que revisaremos) pueden combinarse con un decodificador simple y un proceso de difusión en el espacio

[1] Esto podría deberse a muchas razones: por ejemplo, es posible que los datos disponibles para aprender la distribución no sean suficientes, o que la formulación del programa de optimización no tenga en cuenta algunas restricciones o condiciones adicionales.

[2] Téngase en cuenta que, en este contexto, el concepto de consistencia es de naturaleza empírica o inductiva. Es diferente del concepto de consistencia lógica y deductiva en matemáticas, también conocido como el Principio de Hilbert, del cual hablaremos con más detalle en el capítulo 9.

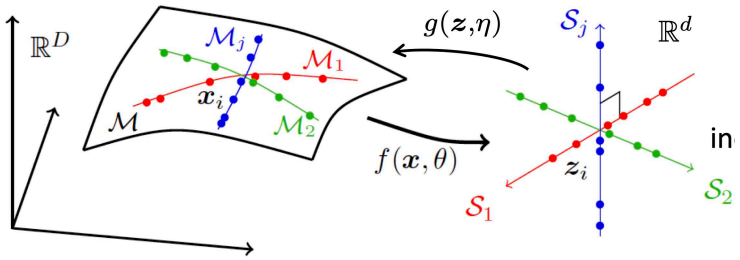


Figura 6.1: Aprender una buena representación de autocodificación para una distribución general cuyos soportes se encuentran en una mezcla de subvariedades de baja dimensión. Obsérvese que, además de aprender una buena representación mediante una aplicación $f(\boldsymbol{x},\theta)$ de extremo a extremo tal como lo hicimos en el Capítulo 5, queremos asegurarnos de que la representación así aprendida pueda ser decodificada, mediante un mapeo «inverso» $g(\boldsymbol{z},\eta)$, para volver a aproximar la distribución de datos original.

$\boldsymbol{Z}$ para permitir el aprendizaje y el muestreo de representaciones de manera eficiente y simultánea.

Más fundamentalmente, en muchos escenarios de aprendizaje prácticos y naturales, puede resultar difícil o incluso imposible comparar las distribuciones de los datos $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$. La única opción que nos queda es comparar la representación aprendida $\boldsymbol{Z}$ con su imagen $\hat{\boldsymbol{Z}}$ bajo el codificador f:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}=f} \boldsymbol{Z} \xrightarrow{\mathcal{D}=g} \hat{\boldsymbol{X}} \xrightarrow{\mathcal{E}=f} \hat{\boldsymbol{Z}} \tag{6.0.3}$$

Esto nos lleva al concepto de una *representación autoconsistente*. En la Sección 6.2, estudiaremos cuándo y cómo podemos aprender una representación consistente imponiendo la autoconsistencia entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$ únicamente a través de un marco de transcripción de bucle cerrado.

Además, en muchos escenarios de aprendizaje prácticos y naturales, normalmente no tenemos acceso simultáneo a suficientes muestras de la distribución de los datos. Por ejemplo, los animales y los seres humanos desarrollan su memoria visual mediante la incorporación continua de pequeñas dosis de observaciones a lo largo de sus vidas. En la Sección 6.3, estudiaremos cómo ampliar el marco de transcripción de bucle cerrado para aprender una representación autoconsistente en un entorno de *aprendizaje continuo*.

Por supuesto, una de las razones fundamentales por las que queremos identificar las estructuras de baja dimensión en una distribución de datos y encontrar una buena representación es facilitar el uso de los datos para diversas tareas de inteligencia, como la clasificación, el completado y la predicción. Por lo tanto, la representación conjunta resultante $(\boldsymbol{x},\boldsymbol{z})$ debe estructurarse de la manera más adecuada para estas tareas. En el siguiente capítulo, veremos cómo se puede estructurar la representación aprendida para facilitar las tareas de completado o generación condicionadas.

6.1 Aprendizaje de representaciones consistentes

A continuación ofrecemos una definición formal de representaciones consistentes, que están estrechamente relacionadas con el concepto de autocodificación.

Definición 6.1 (Representaciones consistentes)**.** A partir de los datos $\boldsymbol{X}$, una *representación consistente* es un par de funciones $(f\colon \mathcal{X} \to \mathcal{Z}, g\colon \mathcal{Z} \to \mathcal{X})$, tal que las *características* $\boldsymbol{Z} = f(\boldsymbol{X})$ son compactas y estructuradas y la *autocodificación*

$$\hat{\boldsymbol{X}} \doteq g(\boldsymbol{Z}) = g(f(\boldsymbol{X})) \tag{6.1.1}$$

es *cercano* a $\boldsymbol{X}$ según una de las dos medidas siguientes:

1. Decimos que es consistente *a nivel de muestra* si $\boldsymbol{X} \approx \hat{\boldsymbol{X}}$ en una determinada norma con alta probabilidad.

2. Decimos que la representación es *distribucionalmente consistente* si $\text{Law}(\boldsymbol{X}) \approx \text{Law}(\hat{\boldsymbol{X}})$.

Los lectores más perspicaces habrán notado que, si no imponemos ciertos requisitos a la representación $\boldsymbol{Z}$ que buscamos, el problema anterior tiene una solución trivial: ¡basta con elegir las funciones f y g para que sean la aplicación identidad! Por lo tanto, el verdadero propósito de buscar una autocodificación es garantizar que el $\boldsymbol{Z}$ así obtenido sea a la vez más compacto y más estructurado que $\boldsymbol{X}$. Como hemos visto en el Capítulo 5, basta con diseñar la representación de manera que se maximice la reducción de la tasa

$$\Delta R_\epsilon(\boldsymbol{Z}). \tag{6.1.2}$$

Según la definición de representación consistente, se requiere que la representación $\boldsymbol{Z}$ sea suficiente para reconstruir la distribución de datos original $\boldsymbol{X}$ con cierto grado de precisión. Para la consistencia a nivel de muestra, una opción habitual es minimizar el error de reconstrucción esperado:

$$d(\boldsymbol{X}, \hat{\boldsymbol{X}}) = \mathbb{E}[\|\boldsymbol{X} - \hat{\boldsymbol{X}}\|_2^2]. \tag{6.1.3}$$

Para garantizar la consistencia distribucional, una opción habitual consiste en minimizar una determinada distancia entre distribuciones, como la divergencia KL [3]:

$$d(\boldsymbol{X}, \hat{\boldsymbol{X}}) = \mathcal{D}_{KL}(\boldsymbol{X} \| \hat{\boldsymbol{X}}). \tag{6.1.4}$$

Por lo tanto, dejando de lado los aspectos computacionales, cuando buscamos una buena autocodificación para una distribución de datos $\boldsymbol{X}$, conceptualmente

[3] Cabe señalar que, en el caso de distribuciones sin soporte común —algo habitual en las distribuciones degeneradas— la divergencia KL puede incluso no estar bien definida. De hecho, gran parte de la literatura sobre aprendizaje de distribuciones intenta abordar esta dificultad técnica sustituyéndola o aproximándola con algo bien definido y que se pueda calcular de manera eficiente.

intentamos encontrar un codificador f y un decodificador g tales que

$$\min_{f,g}[-\Delta R_\epsilon(\boldsymbol{Z}) + d(\boldsymbol{X}, \hat{\boldsymbol{X}})]. \tag{6.1.5}$$

Observe que el error de cuantización ϵ actúa como parámetro de compromiso en esta relajación lagrangiana.

En el resto de esta sección, estudiaremos cómo resolver este tipo de problemas de autocodificación bajo diferentes condiciones, desde casos simples e ideales hasta condiciones cada vez más exigentes y realistas. Concluiremos con una instanciación eficiente de (6.1.5) (en la Sección 6.1.5) que garantiza que el espacio de representación $\boldsymbol{Z}$ sea compacto y estructurado, al tiempo que permite una reconstrucción precisa y un muestreo eficiente.

6.1.1 Autocodificación lineal mediante PCA

Según [Bal11], la expresión «autoencoder» fue introducida por primera vez por Hinton y Rumelhart [RHW86a] con el fin de que se pudiera aprender una representación profunda mediante retropropagación (BP) de manera autosupervisada; la reconstrucción de los datos originales constituye la tarea de autosupervisión. Sin embargo, precisamente ese concepto mismo de buscar una representación compacta y coherente tiene sus raíces en muchos estudios clásicos. Como ya hemos visto en el Capítulo 2, el PCA clásico, el ICA y el aprendizaje de diccionarios esparsos comparten un objetivo similar. La única diferencia radica en que, cuando la distribución subyacente de los datos es simple (lineal e independiente), las correspondencias de codificación o decodificación resultan fáciles de representar y aprender: no es necesario que sean profundas y, a menudo, pueden calcularse en forma cerrada o mediante un algoritmo explícito.

Resulta instructivo observar cómo se manifiesta el concepto de consistencia que hemos definido en el caso sencillo del PCA: aquí, las correspondencias de codificación y decodificación consistentes vienen dadas por una transformación lineal de una sola capa:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}=\boldsymbol{U}^\top} \boldsymbol{Z} \xrightarrow{\mathcal{D}=\boldsymbol{U}} \hat{\boldsymbol{X}}, \tag{6.1.6}$$

donde $\boldsymbol{U} \in \mathbb{R}^{D\times d}$, normalmente con $d \ll D$. Por lo tanto, $\boldsymbol{U}^\top$ representa una proyección desde un espacio de mayor dimensión $\mathbb{R}^D$ a uno de menor dimensión $\mathbb{R}^d$, tal como se ilustra en la Figura 6.2.

Como vimos en el Capítulo 2, cuando la distribución de $\boldsymbol{x}$ se apoya efectivamente en un subespacio de baja dimensión $\boldsymbol{U}_o$, la compacidad de la representación $\boldsymbol{z}$ generada por $\mathcal{E}$ es una consecuencia directa de estimar correctamente (e imponer) la dimensión de este subespacio. Recordemos que el descenso por gradiente sobre el criterio de reconstrucción produce exactamente estas correspondencias consistentes por muestra: de hecho, la solución óptima al problema

$$\min_{\boldsymbol{U}^\top\boldsymbol{U}=\boldsymbol{I}} \mathbb{E}_{\boldsymbol{x}}\left[\left\|\boldsymbol{x} - \boldsymbol{U}\boldsymbol{U}^\top\boldsymbol{x}\right\|_2^2\right] \tag{6.1.7}$$

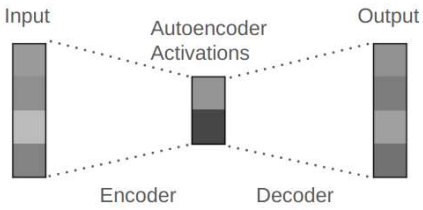


Figura 6.2: Ilustración de un autocodificador típico, como el PCA, que busca una representación de baja dimensión $\boldsymbol{z}$ para datos de alta dimensión $\boldsymbol{x}$.

coincide exactamente con $\boldsymbol{U}_o$ cuando la dimensión de la representación es suficientemente grande. En este caso, obtenemos consistencia por muestra a costo cero, ya que esto garantiza que $\boldsymbol{U}_o\boldsymbol{U}_o^\top\boldsymbol{x} = \boldsymbol{x}$. Nótese que, en el caso del PCA, el término de reducción de la tasa en (6.1.5) se vuelve nulo, ya que la regularización sobre la representación $\boldsymbol{Z}$ buscada es explícita: abarca todo el subespacio de menor dimensión[4].

6.1.2 PCA no lineal y autocodificación

Por supuesto, cabe esperar que la situación deje de ser tan sencilla cuando tratemos con distribuciones más complejas cuyas estructuras subyacentes de baja dimensión sean no lineales.

Datos sobre una subvariedad no lineal. Por lo tanto, para ir más allá de la estructura lineal que aborda el PCA, podemos suponer que la distribución de los datos se encuentra en una subvariedad (suave) $\mathcal{M}$. La dimensión intrínseca de la subvariedad, llamémosla d, suele ser mucho menor que la dimensión del espacio ambiente $\mathbb{R}^D$. Desde esta perspectiva geométrica, habitualmente buscamos un mapeo no lineal f tal que la variedad resultante $f(\mathcal{M})$ se aplane, como ilustra el ejemplo mostrado en la Figura 6.3. El espacio de características $\boldsymbol{z}$ resultante suele ser más compacto (de menor dimensión) que el espacio $\boldsymbol{x}$ y la variedad queda aplanada. Desde la perspectiva estadística, que es complementaria a la perspectiva geométrica pero distinta de ella en general, tal vez también queramos asegurarnos de que la distribución de los datos en $\mathcal{M}$ se mapee a una distribución suficientemente regular en el espacio $\boldsymbol{z}$, como por ejemplo una distribución gaussiana o uniforme (con un soporte de muy baja dimensión). En general, el problema de aprender un mapeo de autocodificación de este tipo para esta clase de distribuciones de datos se conoce como *análisis no lineal de componentes principales* (NLPCA).

Un intento clásico mediante una red neuronal de dos capas. Como hemos visto anteriormente, en el caso del PCA, basta con una red neuronal

[4] En este caso, se puede considerar que $\Delta R = 0$

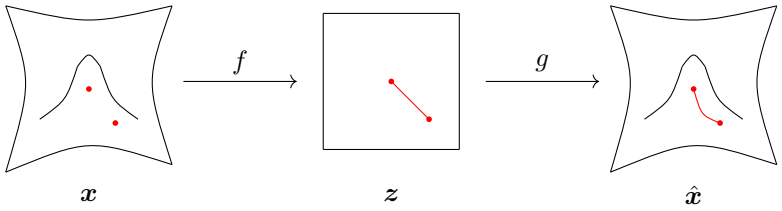


Figura 6.3: Representación de la interpolación mediante aplanamiento de variedad en una variedad en $\mathbb{R}^3$ de dimensión $d = 2$. Para interpolar dos puntos en la variedad de datos, se mapean a través de la aplicación de aplanamiento f al espacio aplanado, se toman sus interpolantes convexas y luego se mapean estos de vuelta a la variedad de datos a través de la aplicación de reconstrucción g.

lineal de una sola capa. Sin embargo, esto ya no se cumple en el caso del NLPCA. En 1991, Kramer [Kra91] propuso calcular el NLPCA utilizando una red neuronal de dos capas para representar la función de codificación f (o su inversa g) basándose en la propiedad de representación universal de las redes de dos capas con activación sigmoide:

$$\boldsymbol{z} = \boldsymbol{W}_2\sigma(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}), \tag{6.1.8}$$

donde $\sigma(\,\cdot\,)$ es la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \tag{6.1.9}$$

Cybenko [Cyb89] demostró que las funciones de la forma anterior (con suficientes nodos ocultos) pueden aproximar cualquier función no lineal suave, por ejemplo, el codificador $f(\,\cdot\,)$, con una precisión arbitraria. En particular, pueden representar los mapas de aplanamiento y reconstrucción para distribuciones de datos soportadas en uniones de variedades de baja dimensión, como en la Figura 6.3. La arquitectura general de las redes originales propuestas por Kramer se ilustra en la Figura 6.4.

Desafortunadamente, a diferencia del caso del PCA mencionado anteriormente, en general, no existe un esquema de aprendizaje en forma cerrada para los parámetros $\boldsymbol{\theta} = (\boldsymbol{W}, \boldsymbol{b})$ de estas redes. Por lo tanto, se propuso entrenar la red mediante retropropagación con la supervisión del error de reconstrucción:

$$\min_{\boldsymbol{\theta}} \mathbb{E}[\|\boldsymbol{x} - \hat{\boldsymbol{x}}(\boldsymbol{\theta})\|_2^2]. \tag{6.1.10}$$

En comparación con el caso sencillo del PCA, utilizamos el mismo objetivo de reconstrucción para el aprendizaje, pero una clase de modelos no lineales mucho más compleja para parametrizar y entrenar el codificador y el decodificador. Aunque las propiedades de aproximación universal, como las de Cybenko, sugieren que, *en principio*, es posible aprender autocodificadores consistentes

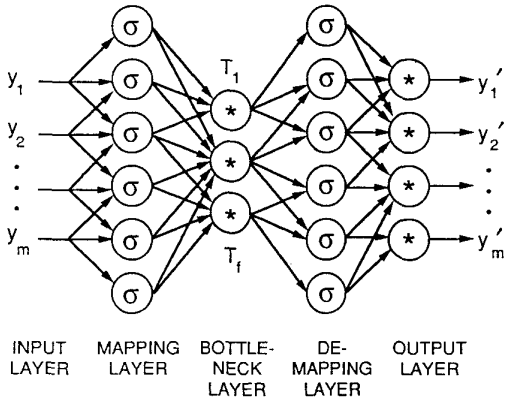


Figura 6.4: PCA no lineal mediante redes neuronales autoasociativas de dos capas tanto para las transformaciones de codificación como de decodificación, según lo propuesto por Kramer [Kra91].

mediante este marco —puesto que, para cualquier muestra aleatoria de datos, siempre que se disponga de suficientes parámetros, existen tales pares de autocodificadores— encontrarlos mediante descenso por gradiente muchas veces resulta sumamente no trivial. Además, para obtener un objetivo de reconstrucción y un modelo suficientemente informativos para la distribución de datos reales de alta dimensión, como las imágenes, el número necesario de muestras y de nodos ocultos puede ser enorme. Además, como medida de la compacidad de la representación aprendida, la dimensión (menor) de $\boldsymbol{z}$ para la capa de cuello de botella suele elegirse en forma heurística.[5]

Aplanamiento de variedades mediante una red más profunda. Según la práctica actual de las redes profundas, se sabe que estas arquitecturas clásicas de redes poco profundas y anchas son bastante difíciles de entrenar de manera eficaz y eficiente mediante la retropropagación (BP), en parte debido a que el gradiente de la función sigmoide se desvanece. Por lo tanto, la práctica actual suele recomendar descomponer aún más la transformada no lineal f (o g) en una composición de muchas más capas de transformadas más simples, lo que da como resultado una arquitectura de red más profunda [HS06], tal como se ilustra en la Figura 6.5.

A la luz de teoremas de aproximación universal como el de Cybenko, uno podría preguntarse inicialmente por qué, desde un punto de vista conceptual, se deberían preferir los autocodificadores más profundos a los menos profundos. Desde una perspectiva puramente expresiva, podemos entender este fenómeno bajo un ángulo geométrico relacionado con la tarea de aplanar la variedad no

[5] En el trabajo posterior [HZ93], Hinton et al. sugirieron utilizar el principio de la longitud mínima de descripción (MDL) para promover la compacidad del esquema de codificación aprendido, en un espíritu muy similar al de la medida de distorsión de tasa introducida en este libro.

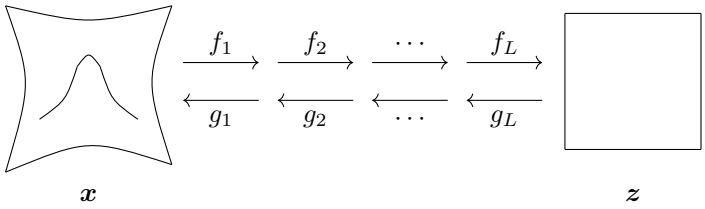


Figura 6.5: Una representación del proceso de construcción del par de aplanamiento y reconstrucción (f, g), donde el codificador $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$ se construye a partir de la composición de capas de aplanamiento y el decodificador $g = g_1 \circ g_2 \circ \cdots \circ g_L$ está compuesto por las inversas de cada f_ℓ.

lineal sobre la que se sustenta nuestra distribución hipotética de datos. Un enfoque puramente constructivo para aplanar la variedad procede de manera incremental, en paralelo a lo que hemos visto en los Capítulos 3 a 5 con la interacción entre la difusión, la eliminación de ruido y la compresión. En el contexto geométrico, el *proceso de aplanamiento* incremental correspondiente a f_ℓ consiste en transformar un entorno cercano a un punto de la variedad para que quede más próximo a una variedad plana (es decir, un subespacio) y en garantizar la consistencia local con el resto de las muestras de datos; la operación incremental correspondiente en el decodificador, g_ℓ, revierte esta transformación. Este procedimiento incorpora precisamente información sobre la curvatura de la variedad subyacente, la cual se estima a partir de muestras de datos. Si se dispone de suficientes muestras de la variedad y se lleva a cabo una instanciación cuidadosa de este proceso conceptual, es posible implementar este esbozo como un procedimiento computacional que aplana de manera verificable variedades no lineales mediante un enfoque de white-box [PPR+24]. Sin embargo, este enfoque tiene una aplicabilidad limitada a distribuciones de datos de alta dimensión, como las imágenes, debido a una escalabilidad desfavorable, lo que ha motivado el desarrollo de métodos más flexibles para la autocodificación incremental.

6.1.3 Autocodificación esparsa

En los esquemas de autocodificación mencionados anteriormente, la dimensión del espacio de características d suele elegirse de modo que sea significativamente menor que la del espacio de datos original D, con el fin de imponer o promover explícitamente que la representación aprendida sea de baja dimensión. Sin embargo, en la práctica, normalmente desconocemos la dimensión intrínseca de la distribución de los datos. Por lo tanto, la elección de la dimensión del espacio de características para la autocodificación suele realizarse en forma empírica. En situaciones más generales, la distribución de los datos puede ser una mezcla de unos pocos subespacios o subvariedades de baja dimensión. En estos casos, ya no es viable imponer un único espacio de baja dimensión para todas las características en conjunto.

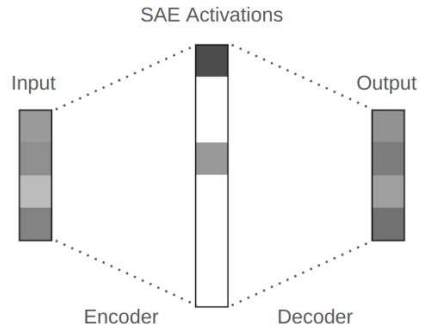


Figura 6.6: Ilustración de un autocodificador esparso (SAE), en comparación con la de un autocodificador típico (AE) en la Figura 6.2.

El autocodificador esparso está diseñado para resolver algunas de estas limitaciones. En particular, la dimensión del espacio de características puede ser igual o incluso mayor que la del espacio de datos, tal como se ilustra en la Figura 6.6. Sin embargo, se requiere que las características sean muy esparsas en el espacio de características. Por lo tanto, si imponemos la esparcidad como medida de parsimonia, además de la reducción de la tasa en el objetivo (6.1.5), obtenemos un nuevo objetivo para la autocodificación esparsa:

$$\min_{f,g}[\|\boldsymbol{Z}\|_0 - \Delta R_\epsilon(\boldsymbol{Z}) + d(\boldsymbol{X}, \hat{\boldsymbol{X}})], \tag{6.1.11}$$

donde se sabe que la «norma» ℓ^0 $\|\cdot\|_0$ favorece la esparcidad. Esto es muy similar al objetivo de reducción esparsa de la tasa de compresión (5.2.4) que utilizamos en el anterior Capítulo 5 para derivar la arquitectura CRATE transparente *white-box*.

Como método para aprender pares de autocodificación de extrtemo a extremo, la autocodificación esparsa se ha utilizado en el pasado [LRM+12; RPC+06], pero casi todos los marcos modernos de autocodificación se basan, en cambio, en un marco de autocodificación probabilístico diferente, que estudiaremos a continuación.

6.1.4 Autocodificación variacional

Una metodología mejorada y más moderna para la autocodificación que aún goza de una aplicación significativa es la *autocodificación variacional* [KW13a; KW19]. Formula el problema del aprendizaje de representaciones en términos probabilísticos, como la búsqueda de un modelo $p(\boldsymbol{x};\boldsymbol{\theta})$ que maximice la verosimilitud de una muestra de datos (o, de manera más general, la distribución poblacional de datos):

$$\max_{\boldsymbol{\theta}} \mathbb{E}_{\boldsymbol{x}}[\log p(\boldsymbol{x};\boldsymbol{\theta})].$$

Mediante condicionamiento, podemos escribir $p(\boldsymbol{x}, \boldsymbol{z}; \boldsymbol{\theta}) = p(\boldsymbol{z}; \boldsymbol{\theta})p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta})$ y

$$\begin{aligned} p(\boldsymbol{x}; \boldsymbol{\theta}) &= \int p(\boldsymbol{z}; \boldsymbol{\theta})p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta})\mathrm{d}\boldsymbol{z} \\ &= \mathbb{E}_{\boldsymbol{z} \sim p(\,\cdot\,; \boldsymbol{\theta})}[p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta})]. \end{aligned}$$

Esto implica que se puede aprender un modelo probabilístico para $\boldsymbol{x}$ con una distribución *a priori* $p(\boldsymbol{z})$ y una distribución condicional $p(\boldsymbol{x} \mid \boldsymbol{z})$, lo que, en términos más generales, corresponde a un decodificador. En particular, la regla de Bayes implica que la posterior $p(\boldsymbol{z} \mid \boldsymbol{x})$, necesaria para la codificación, puede expresarse en términos de la a priori y el decodificador (y muestras de $\boldsymbol{x}$), pero su cálculo es computacionalmente intratable. Por lo tanto, en el marco de VAE, introducimos una «posterior aproximada» $q(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\eta})$, correspondiente a una red codificadora separada y aprendemos conjuntamente el codificador y el decodificador mediante la maximización de una cota inferior de la verosimilitud verdadera, conocida como cota inferior de evidencia (ELBO), que es ajustada cuando la aproximación posterior también lo es.

Nos centraremos en la instanciación más útil de la arquitectura VAE en la práctica, a saber, aquella en la que la a priori $p(\boldsymbol{z}; \boldsymbol{\theta})$ y la condicional $p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta})$ son gaussianas. La siguiente instanciación es habitual en la práctica. Utilizamos las siguientes distribuciones gaussianas:

$$\begin{aligned} \boldsymbol{z} &\sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \\ \boldsymbol{x} \mid \boldsymbol{z} &\sim \mathcal{N}(g_1(\boldsymbol{z}), \mathrm{diag}(e^{g_2(\boldsymbol{z})})\boldsymbol{I}), \end{aligned}$$

donde $g = (g_1, g_2)$ son redes profundas con parámetros $\boldsymbol{\theta}$, que corresponden al *decodificador* del autocodificador. De manera similar, para la distribución a posteriori aproximada, utilizamos una distribución normal cuyos parámetros vienen dados por un codificador MLP $f = (f_1, f_2)$ con parámetros $\boldsymbol{\eta}$:

$$\boldsymbol{z} \mid \boldsymbol{x} \sim \mathcal{N}(f_1(\boldsymbol{x}), \mathrm{diag}(e^{f_2(\boldsymbol{x})})\boldsymbol{I}).$$

Esto simplifica la codificación y la decodificación probabilísticas: simplemente mapeamos nuestros datos $\boldsymbol{x}$ a los parámetros de media y varianza de una distribución gaussiana para la codificación y al revés para la decodificación. Tras las manipulaciones mencionadas, el objetivo final de la VAE queda así:

$$\max_{\boldsymbol{\theta}, \boldsymbol{\eta}} \mathbb{E}_{\boldsymbol{x}} \mathbb{E}_{\boldsymbol{z} \sim q(\,\cdot\,|\boldsymbol{x}; \boldsymbol{\eta})} \left[\log \frac{p(\boldsymbol{x}, \boldsymbol{z}; \boldsymbol{\theta})}{q(\boldsymbol{z} \mid \boldsymbol{x}; \boldsymbol{\eta})} \right] \tag{6.1.12}$$

$$\equiv \max_{\boldsymbol{\theta}, \boldsymbol{\eta}} \left\{ 2\mathbb{E}_{\boldsymbol{x}} \left[\mathbb{E}_{\boldsymbol{z} \sim q(\,\cdot\,|\boldsymbol{x}; \boldsymbol{\eta})} \left[\log p(\boldsymbol{x} \mid \boldsymbol{z}; \boldsymbol{\theta}) \right] + \langle f_2(\boldsymbol{x}) - e^{f_2(\boldsymbol{x})}, \mathbf{1} \rangle - \| f_1(\boldsymbol{x}) \|_2^2 \right] \right\}. \tag{6.1.13}$$

Podemos interpretar esto como un objetivo de autocodificación regularizado, en el que existe una interesante restricción de consistencia adicional en el primer término del ELBO.

Al mismo tiempo, el lector debí tener en cuenta las numerosas decisiones arbitrarias que toma el formalismo VAE (por ejemplo, la elección y

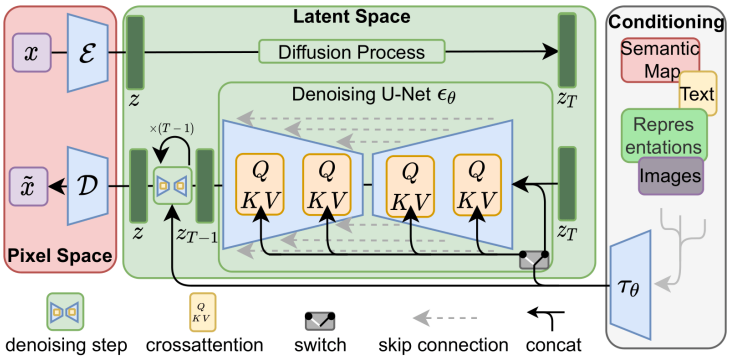


Figura 6.7: Diagrama del modelo de difusión latente tomado del trabajo de [RBL+22], que utiliza un autocodificador variacional (recuadro rojo a la izquierda) para aprender una representación latente de los datos $\boldsymbol{x}$. A continuación, se aprende un modelo de difusión/eliminación de ruido (recuadro verde en el centro) para acceder a la representación $\boldsymbol{z}$. El bloque de la derecha trata de cómo controlar el proceso de muestreo para la eliminación de ruido mediante una señal de control (basada en texto) que estudiaremos en el Capítulo 7.

parametrización de las distribuciones gaussianas). Esto da lugar a un objetivo de entrenamiento complejo cuya optimización eficiente sigue siendo un reto, a pesar de más de diez años de investigación. Las metodologías recientes de aprendizaje de representaciones han logrado mejoras notables con respecto a esta referencia, e incluso pueden formularse en el marco de white-box que hemos desarrollado en el Capítulo 5, como veremos en breve.

Difusión latente basada en VAE. Una aplicación importante de las VAE se da en el contexto de los modelos de difusión latente [RBL+22], donde primero se entrena una VAE para que aprenda una representación latente más compacta y lo suficientemente regular de datos de alta dimensión como las imágenes. A continuación, se puede entrenar un modelo de difusión en el espacio latente para acceder a la distribución de la representación y, por lo tanto, generar nuevas muestras a partir de ella, como se ilustra en la Figura 6.7. Este enfoque permite generar de manera eficiente imágenes de alta calidad al aprovechar la estructura altamente comprimida del espacio latente aprendido.[6] Sin embargo, el entrenamiento de las VAE puede resultar complejo debido a problemas como el colapso de la distribución a posteriori y el equilibrio entre los términos de reconstrucción y regularización en el ELBO. Además, las distribuciones a posteriori gaussianas en las VAE se aplican punto por punto y no se aprende ninguna estructura conjunta entre las muestras, lo que puede limitar

[6] Muchos de los primeros modelos generativos de imágenes o videos más populares se basan en este marco de difusión latente, tal como lo describimos con más detalle en la Sección 7.4.2 y en términos de implementación en el Capítulo 8.

considerablemente la calidad de las representaciones aprendidas. Estos desafíos han impulsado el desarrollo de enfoques alternativos, como los autocodificadores de representación (RAE), que se presentarán a continuación y cuyo objetivo es superar algunas de las limitaciones de los VAE, sin dejar de proporcionar representaciones latentes eficaces para las tareas de modelado generativo.

6.1.5 Aprendizaje conjunto de representación y distribución

Los enfoques anteriores para entrenar pares autocodificadores con el fin de aprender representaciones comparten una limitación común: no *organizan explícitamente* el espacio de representaciones $\boldsymbol{Z}$, sino que solo lo fomentan de manera indirecta (por ejemplo, imponiendo la esparcidad en el autocodificador esparso o la gaussianidad en el VAE). Como insinuamos al principio del capítulo, ya hemos visto anteriormente (en el Capítulo 4) cómo obtener resultados mucho mejores, aprendiendo una representación $\boldsymbol{Z}$ mediante la maximización de la reducción de la tasa de codificación $\Delta R_\epsilon(\boldsymbol{Z})$, al tiempo que se garantiza la consistencia de la reconstrucción. En esta sección describimos este procedimiento con más detalle y, en particular, mostramos cómo realizar un *muestreo eficiente* en el espacio de representación aprendido $\boldsymbol{Z}$.

Como hemos visto al final del Capítulo 4 (Sección 4.3), a menudo podemos aprender una buena representación $\boldsymbol{Z}$ de los datos $\boldsymbol{X}$:

$$f : \boldsymbol{X} \to \boldsymbol{Z}, \tag{6.1.14}$$

basadas en algunos métodos supervisados o autosupervisados de extremo a extremo, incluidos los métodos MCR^2 y SimDINO presentados en la Sección 4.3.1 y la Sección 4.3.2, respectivamente. Como también hemos señalado, las representaciones aprendidas de este modo presentan mejores estructuras que la distribución de los datos originales. Las representaciones suelen ser una mezcla de subespacios lineales de baja dimensión (o gaussianas de bajo rango). Sin embargo, aún no hemos respondido a la pregunta de cómo evaluar la calidad de las representaciones así aprendidas. Es decir, ¿podemos establecer a partir de ellas un decodificador inverso:

$$g : \boldsymbol{Z} \to \hat{\boldsymbol{X}} \tag{6.1.15}$$

de manera que $\hat{\boldsymbol{X}}$ y $\boldsymbol{X}$ sean cercanos? A la luz de los ejemplos anteriores de configuraciones clásicas de autocodificación, cabría esperar que un objetivo de la forma (6.1.5) funcionara, pero ¿qué medida de pérdida de consistencia por muestra se debería utilizar?

Además, esas estructuras de baja dimensión (lineales) siguen estando incrustadas en un espacio de representación de alta dimensión. Por lo tanto, para poder utilizar las representaciones así aprendidas, necesitamos poder acceder también a sus distribuciones. En el Capítulo 3, hemos aprendido que la eliminación de ruido es un enfoque general para identificar y, por lo tanto, acceder a dichas representaciones de baja dimensión. De este modo, podemos

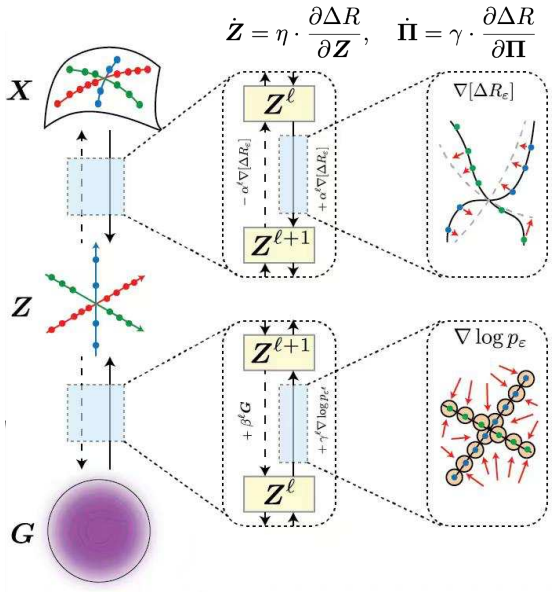


Figura 6.8: Ilustración del aprendizaje de una representación de autocodificador $\boldsymbol{Z}$ de los datos $\boldsymbol{X}$ (arriba) y un modelo de difusión/reducción de ruido (abajo) para acceder a la representación $\boldsymbol{Z}$.

acceder a la distribución de las características aprendidas a través de un proceso (aprendido) de eliminación de ruido a partir de un ruido aleatorio:

$$h : \boldsymbol{G} \to \boldsymbol{Z}. \tag{6.1.16}$$

Además, hemos aprendido que, si la distribución objetivo de $\boldsymbol{Z}$ puede modelarse (aproximadamente) como una mezcla de subespacios o gaussianas de bajo rango, sus operadores óptimos de eliminación de ruido comparten una estructura similar a la que se encuentra en los transformers o en CRATE, presentado en el Capítulo 5.

Por lo tanto, en definitiva, queremos entrenar un autocodificador para los datos $\boldsymbol{X}$ cuya representación $\boldsymbol{Z}$ sea fácilmente accesible. Concretamente, quisiéramos aprender un codificador f y un decodificador g tales que:

1. La representación $\boldsymbol{Z} = f(\boldsymbol{X})$ es una representación consistente de los datos originales $\boldsymbol{X}$, tal como se define en la Definición 6.1.

2. La representación estructurada $\boldsymbol{Z}$ puede modelarse aproximadamente como una mezcla de distribuciones gaussianas de baja dimensión, de modo que podamos entrenar un *denoiser* para acceder de manera eficiente a la distribución de $\boldsymbol{Z}$ a partir del ruido gaussiano aleatorio $\boldsymbol{G}$.

La Figura 6.8 ilustra los procesos generales mencionados anteriormente.

A partir del Capítulo 4, sabemos que la representación $\boldsymbol{Z}$ aprendida mediante la optimización del MCR^2 (sin supervisión) cumple el segundo requisito y puede considerarse que presenta estructuras cercanas a una mezcla de subespacios o gaussianas de bajo rango. Al incorporar un objetivo de consistencia de la muestra, podemos extender el objetivo de reducción de la tasa en la Sección 4.3.2 para aprender una representación consistente:

$$\min_{f,g}[-\Delta R_\epsilon(\boldsymbol{Z}) + d(\boldsymbol{X}, \hat{\boldsymbol{X}})], \tag{6.1.17}$$

donde $d(\boldsymbol{X}, \hat{\boldsymbol{X}})$ es una medida de similitud entre los datos originales $\boldsymbol{X}$ y los datos reconstruidos $\hat{\boldsymbol{X}} = g(\boldsymbol{Z})$. En la práctica, dado que el objetivo $\Delta R_\epsilon(\boldsymbol{Z})$ por sí solo ya favorece una estructura de baja dimensión de $\boldsymbol{Z}$, podemos preentrenar primero el codificador f para maximizar $\Delta R_\epsilon(\boldsymbol{Z})$ y, a continuación, entrenar el decodificador g para minimizar la pérdida de reconstrucción $d(\boldsymbol{X}, \hat{\boldsymbol{X}})$ manteniendo fijo el codificador f. Se ha demostrado que este enfoque de dos pasos es efectivo, como veremos en la sección 8.6.2 del Capítulo 8.

Una vez que hayamos aprendido una representación consistente $\boldsymbol{Z}$ de ese tipo, podremos entrenar un *denoiser* para acceder a su distribución, tal y como hemos estudiado en el Capítulo 3. Concretamente, supongamos que tenemos un modelo probabilístico $p(\boldsymbol{z})$ que captura la distribución (marginal) de las representaciones aprendidas $\boldsymbol{Z}$ y una distribución gaussiana estándar inicial $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. Nuestro objetivo es aprender un *denoiser* $\bar{\boldsymbol{z}}$ que elimine el ruido iterativamente de una muestra, pasando de la distribución inicial a la distribución objetivo de las representaciones aprendidas. Este *denoiser* puede entrenarse utilizando los objetivos presentados en el Capítulo 3 (véase también el Capítulo 7 para los *denoisers* condicionales). Por ejemplo, consideremos el caso sencillo de flow matching (véanse el Ejercicio 3.5 y la Sección 3.2.2). Podemos definir el *denoiser* como un campo vectorial dependiente del tiempo $\bar{\boldsymbol{z}}_\theta(t, \boldsymbol{z}_t)$ parametrizado por θ que transforma una muestra de la distribución inicial a la distribución objetivo a medida que el tiempo $t \in [0, 1]$ varía. Aquí, $\boldsymbol{z}_t$ es una interpolación entre $\boldsymbol{z}_0$ y $\boldsymbol{z}_1$ en el tiempo t:

$$\boldsymbol{z}_t = (1-t)\boldsymbol{z}_0 + t\boldsymbol{z}_1, \tag{6.1.18}$$

donde $\boldsymbol{z}_0 \sim p(\boldsymbol{z})$ es una muestra de la distribución de la representación aprendida y $\boldsymbol{z}_1 \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. El objetivo de entrenamiento del *denoiser* puede expresarse entonces como:

$$\min_\theta \mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\boldsymbol{z}_0 \sim p(\boldsymbol{z}), \boldsymbol{z}_1 \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), t \sim \text{Unif}(0,1)} \left[\| \bar{\boldsymbol{z}}_\theta(t, \boldsymbol{z}_t) - (\boldsymbol{z}_1 - \boldsymbol{z}_0) \|_2^2 \right]. \tag{6.1.19}$$

Cabe destacar la diferencia entre entrenar un *denoiser* para las representaciones aprendidas mediante VAE frente a las aprendidas mediante métodos como MCR^2 o SimDINO. Como se discutió en la Sección 6.1.4, las representaciones VAE están obligadas a aproximar una distribución a posteriori gaussiana *por muestra* y no se modela ninguna distribución conjunta del espacio latente, haciendo que el espacio de representación sea menos estructurado y que

el proceso de eliminación de ruido sea potencialmente más desafiante. Por el contrario, las representaciones aprendidas mediante MCR^2 o SimDINO tienden a presentar estructuras lineales y de baja dimensión más pronunciadas, como mezclas de subespacios o gausianas de rango bajo (como se demuestra en la Sección 4.3.2), lo que puede facilitar el proceso de eliminación de ruido. Esta propiedad estructural puede conducir a modelos de eliminación de ruido más eficientes y efectivos, ya que pueden aprovechar mejor la geometría subyacente del espacio de representación.

Una vez que hayamos entrenado el *denoiser*, podemos utilizarlo para generar nuevas muestras a partir de la distribución de representaciones aprendida mediante un proceso de difusión, tal y como hemos mostrado en el Capítulo 3. Con esto queda completado el aprendizaje de un *autocodificador de representaciones* (RAE) con una distribución de representaciones accesible. En el Capítulo 8 (en concreto, en la Sección 8.6.2), veremos una implementación específica de RAE y sus aplicaciones en la generación de imágenes.

6.2 Aprendizaje de representaciones autoconsistentes

En capítulos anteriores, hemos estudiado métodos que nos permitirían aprender una distribución de baja dimensión mediante compresión (con pérdidas). Como hemos mencionado en el Capítulo 1 y demostrado en los capítulos anteriores, los avances logrados en inteligencia artificial dependen en gran medida de encontrar soluciones computacionalmente viables y eficientes para lograr la compresión deseada, que no solo sean computables o manejables en teoría, sino también escalables en la práctica:

$$\textbf{computable} \implies \textbf{manejable} \implies \textbf{escalable}. \tag{6.2.1}$$

Incluso se podría afirmar que los enormes avances en inteligencia artificial logrados en la última década se deben en gran medida al desarrollo de modelos y métodos escalables, por ejemplo, mediante el entrenamiento de redes profundas a través de la retropropagación. Los modelos de gran tamaño, con miles de millones de parámetros y entrenados con billones de puntos de datos en decenas de miles de potentes GPU, han demostrado capacidades sobrehumanas a la hora de memorizar el conocimiento existente. Esto ha llevado a muchos a creer que la «inteligencia» de dichos modelos seguirá mejorando a medida que su escala siga aumentando.

Aunque celebramos las maravillas de la ingeniería que suponen estos grandes sistemas de aprendizaje automático creados por el hombre, también debemos admitir que, en comparación con la inteligencia presente en la naturaleza, este enfoque para mejorar la inteligencia artificial requiere una cantidad innecesaria de recursos. Los seres inteligentes naturales, incluidos los animales y los seres humanos, simplemente no pueden permitirse una solución de «fuerza bruta» para el aprendizaje, ya que deben funcionar con un presupuesto muy limitado

de energía, espacio y tiempo y están sujetos a numerosas restricciones físicas estrictas.

En primer lugar, existen sólidas pruebas científicas de que nuestro cerebro no lleva a cabo una retropropagación global de extremo a extremo para mejorar o corregir sus predicciones. En cambio, en neurociencia se sabe desde hace tiempo que nuestro cerebro corrige los errores mediante retroalimentación local de bucle cerrado, como la codificación predictiva. Esta fue la base científica que inspiró a Norbert Wiener a desarrollar la teoría del control por retroalimentación y el programa de cibernética allá por la década de 1940.

En segundo lugar, en las secciones anteriores vimos que para aprender una representación consistente es necesario aprender una autocodificación bidireccional:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}=f} \boldsymbol{Z} \xrightarrow{\mathcal{D}=g} \hat{\boldsymbol{X}}. \tag{6.2.2}$$

Esto requiere que los datos de entrada observados $\boldsymbol{X}$ y los decodificados $\hat{\boldsymbol{X}}$ estén próximos según alguna medida de similitud $d(\boldsymbol{X}, \hat{\boldsymbol{X}})$. Sin embargo, cuando los datos $\boldsymbol{X}$ son de alta dimensión (por ejemplo, imágenes), las medidas de distancia ingenuas como la norma ℓ^2 en el espacio envolvente son en gran medida poco informativas: dos imágenes que difieren por una pequeña transformación geométrica pueden estar muy separadas en distancia de píxeles y, sin embargo, ser perceptualmente idénticas. Por lo tanto, una pregunta pendiente es: ¿cómo podemos garantizar la similitud entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ sin una distancia significativa en el espacio de datos envolvente? Como veremos en este capítulo, la respuesta radica en la retroalimentación de bucle cerrado en un espacio de características aprendido, junto con la baja dimensionalidad de la distribución de los datos.

Como vimos en el capítulo anterior, para garantizar que una representación sea consistente, debemos comparar el $\hat{\boldsymbol{X}} \sim p(\hat{\boldsymbol{x}})$ generado y el $\boldsymbol{X} \sim p(\boldsymbol{x})$ original, al menos a nivel de distribución. Incluso cuando tenemos acceso a $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$, técnicamente, calcular y minimizar la distancia entre dos distribuciones puede ser problemático, especialmente cuando el soporte de las distribuciones es de baja dimensión. La divergencia KL introducida en el Capítulo 3 ni siquiera está bien definida entre dos distribuciones cuyos soportes no se superponen.

Como primer intento de paliar la dificultad antes mencionada a la hora de calcular y minimizar la distancia entre dos distribuciones (de baja dimensión), se había propuesto entrenar el generador/decodificador g mediante enfoques discriminativos [Tu07]. Esta línea de razonamiento ha dado lugar al concepto de las *redes generativas adversarias (GAN)* [GPM+14b]. Este modelo introduce un discriminador d, que suele modelarse mediante una red profunda, para distinguir las diferencias entre las muestras generadas $\hat{\boldsymbol{X}}$ y las reales $\boldsymbol{X}$:

$$\boldsymbol{Z} \xrightarrow{g(\boldsymbol{z},\eta)} \hat{\boldsymbol{X}},\ \boldsymbol{X} \xrightarrow{d(\boldsymbol{x},\theta)} \{\boldsymbol{0}, \boldsymbol{1}\}. \tag{6.2.3}$$

Se sugiere intentar alinear las distribuciones entre $\hat{\boldsymbol{x}}$ y $\boldsymbol{x}$ mediante un *juego de Stackelberg* —un juego secuencial en el que un jugador (el líder) se compromete primero con una estrategia y el otro (el seguidor) responde de manera óptima (véase la Apéndice A.3 para un análisis detallado)— entre el generador g y el

discriminador d:

$$\max_{\theta}\min_{\eta}\mathbb{E}_{p(\boldsymbol{x})}\big[\log d(\boldsymbol{x},\theta)\big]+\mathbb{E}_{p(\boldsymbol{z})}\big[\log\big(1-d(\underbrace{g(\boldsymbol{z},\eta)}_{\hat{\boldsymbol{x}}\sim p_g},\theta)\big)\big]. \tag{6.2.4}$$

Es decir, el discriminador d intenta minimizar la entropía cruzada entre las muestras reales $\boldsymbol{X}$ y las generadas $\hat{\boldsymbol{X}}$, mientras que el generador g intenta hacer lo contrario.

Se puede demostrar que encontrar un equilibrio para el juego de Stackelberg descrito anteriormente equivale a minimizar la *divergencia de Jensen-Shannon*:

$$\mathcal{D}_{JS}(p(\boldsymbol{x}),p_g(\hat{\boldsymbol{x}}))=\mathcal{D}_{KL}\big(p\|(p+p_g)/2\big)+\mathcal{D}_{KL}\big(p_g\|(p+p_g)/2\big). \tag{6.2.5}$$

Cabe señalar que, a diferencia de la divergencia KL, la divergencia JS está bien definida incluso si los soportes de las dos distribuciones no se superponen. Sin embargo, la divergencia JS no tiene una expresión en forma cerrada ni siquiera entre dos distribuciones gaussianas, mientras que la divergencia KL sí la tiene. Además, dado que las distribuciones de datos son de baja dimensión, la divergencia JS puede estar muy mal condicionada para su optimización.[7] Esto podría explicar por qué en muchas variantes posteriores de las GAN se suelen emplear numerosas heurísticas adicionales.

En su lugar, se ha sugerido también sustituir la divergencia de JS por la distancia de Earth Mover (EM) o la distancia de Wasserstein. [8] Sin embargo, tanto la divergencia de JS como la distancia de Wasserstein solo pueden calcularse en forma aproximada entre dos distribuciones generales.[9] Además, ni la divergencia de JS ni la distancia de Wasserstein tienen fórmulas en forma cerrada, ni siquiera para las distribuciones gaussianas.[10]

Si resulta difícil comparar las distribuciones de los datos $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$, tal vez sea posible comparar la característica aprendida $\boldsymbol{Z}$ con su imagen $\hat{\boldsymbol{Z}}$ bajo el codificador f:

$$\boldsymbol{X}\xrightarrow{\mathcal{E}=f}\boldsymbol{Z}\xrightarrow{\mathcal{D}=g}\hat{\boldsymbol{X}}\xrightarrow{\mathcal{E}=f}\hat{\boldsymbol{Z}} \tag{6.2.6}$$

Esto nos lleva al concepto de *representación autoconsistente*.

Definición 6.2 (Representación autoconsistente)**.** Dados los datos $\boldsymbol{X}$, llamamos *representación autoconsistente* a un par de funciones $(f\colon \mathcal{X}\to\mathcal{Z}, g\colon \mathcal{Z}\to\mathcal{X})$, de tal manera que las *características* $\boldsymbol{Z}=f(\boldsymbol{X})$ sean compactas y estructuradas y las *características de autocodificación* $\hat{\boldsymbol{Z}}\doteq f\circ g(\boldsymbol{Z})$ sean *cercanas* a $\boldsymbol{Z}$.

[7] Como se muestra en [ACB17].

[8] En términos generales, para distribuciones con soportes de baja dimensión que pueden no solaparse, la divergencia de JS se comporta como la norma ℓ^0 y la distancia EM se comporta como la norma ℓ^1.

[9] Por ejemplo, la distancia de Wasserstein requiere calcular la diferencia máxima entre las esperanzas de las dos distribuciones sobre todas las funciones 1-Lipschitz.

[10] La distancia de Wasserstein (con norma ℓ^1) puede acotarse por la distancia de Wasserstein (con norma ℓ^2), que tiene una fórmula cerrada [DDS22]. Sin embargo, como es bien sabido en geometría de alta dimensión, la norma ℓ^1 y la norma ℓ^2 se desvían significativamente en cuanto a sus propiedades geométricas y estadísticas a medida que la dimensión se vuelve alta [WM22]. La cota puede volverse muy laxa.

1. Decimos que es autoconsistente *por muestra* si $\boldsymbol{Z} \approx \hat{\boldsymbol{Z}}$ en una determinada norma con alta probabilidad.

2. Decimos que la representación es *autoconsistente en distribución* si $\text{Law}(\boldsymbol{Z}) \approx \text{Law}(\hat{\boldsymbol{Z}})$.

6.2.1 Transcripción en bucle cerrado mediante juegos de Stackelberg

¿Cómo intentamos garantizar que una representación aprendida sea autoconsistente? Como de costumbre, supongamos que $\boldsymbol{X} = \cup_{k=1}^{K} \boldsymbol{X}_k$, donde cada subconjunto de muestras $\boldsymbol{X}_k$ pertenece a una subvariedad de baja dimensión: $\boldsymbol{X}_k \subset \mathcal{M}_k, k = 1, \ldots, K$. Siguiendo la notación del Capítulo 3, utilizamos una matriz $\boldsymbol{\Pi}_k(i,i) = 1$ para denotar la pertenencia de la muestra i a la clase k y $\boldsymbol{\Pi}_k(i,j) = 0$ en caso contrario. Se busca una aplicación continua $f(\cdot, \boldsymbol{\theta}) : \boldsymbol{x} \mapsto \boldsymbol{z}$ de $\boldsymbol{X}$ a una representación óptima $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N] \subset \mathbb{R}^{d \times N}$:

$$\boldsymbol{X} \xrightarrow{f(\boldsymbol{x}, \boldsymbol{\theta})} \boldsymbol{Z}, \tag{6.2.7}$$

que maximiza el siguiente objetivo de reducción de la tasa de codificación (MCR^2):

$$\max_{\boldsymbol{Z}} \Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \doteq \underbrace{\frac{1}{2} \log\det\left(\boldsymbol{I} + \alpha \boldsymbol{Z}\boldsymbol{Z}^\top\right)}_{R_\epsilon(\boldsymbol{Z})} - \underbrace{\sum_{k=1}^{K} \frac{\gamma_k}{2} \log\det\left(\boldsymbol{I} + \alpha_k \boldsymbol{Z}\boldsymbol{\Pi}^k\boldsymbol{Z}^\top\right)}_{R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{\Pi})}, \tag{6.2.8}$$

donde $\alpha = d/(N\epsilon^2)$, $\alpha_k = d/(\text{tr}(\boldsymbol{\Pi}_k)\epsilon^2)$, $\gamma_k = \text{tr}(\boldsymbol{\Pi}_k)/N$ para cada $k = 1, \ldots, K$.

Un problema que plantea el aprendizaje de un mapeo tan unilateral (6.2.7) mediante la maximización del objetivo anterior (6.2.8) es que tiende a ampliar la dimensión del subespacio aprendido para las características de cada clase[11], como hemos visto en la Sección 4.2 del Capítulo 4. Para verificar si las características aprendidas son consistentes, es decir, si no sobreestiman ni subestiman la estructura intrínseca de los datos, podemos considerar el aprendizaje de un decodificador $g(\cdot, \eta) : \boldsymbol{z} \mapsto \boldsymbol{x}$ a partir de la representación $\boldsymbol{Z} = f(\boldsymbol{X}, \boldsymbol{\theta})$ y volver al espacio de datos $\boldsymbol{x}$: $\hat{\boldsymbol{X}} = g(\boldsymbol{Z}, \boldsymbol{\eta})$:

$$\boldsymbol{X} \xrightarrow{f(\boldsymbol{x}, \boldsymbol{\theta})} \boldsymbol{Z} \xrightarrow{g(\boldsymbol{z}, \boldsymbol{\eta})} \hat{\boldsymbol{X}}, \tag{6.2.9}$$

[11] Si la dimensión del espacio de características d es demasiado alta, maximizar la reducción de la tasa puede sobreestimar la dimensión de cada clase. Por lo tanto, para aprender una buena representación, es necesario preseleccionar una dimensión adecuada para el espacio de características, tal como se hizo en los experimentos de [YCY+20]. De hecho, el mismo problema de «selección de modelos» persiste incluso en el caso más simple de un solo subespacio, que es el clásico PCA [Jol86]. Según nuestro conocimiento, la selección del número correcto de componentes principales en una situación heterogénea con ruido sigue siendo un tema de investigación activo [HSD20].

y comprueba cuán cerca están $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$. Esto constituye una autocodificación, que es lo que hemos estudiado en profundidad anteriormente en este capítulo.

Medición de distancias en el espacio de características. Sin embargo, como hemos comentado anteriormente, si no tenemos la opción de calcular la distancia entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$, nos queda la opción de comparar sus características correspondientes $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}} = f(\hat{\boldsymbol{X}}, \boldsymbol{\theta})$. Obsérvese que, bajo el objetivo MCR^2, las distribuciones de los valores resultantes $\boldsymbol{Z}$ o $\hat{\boldsymbol{Z}}$ tienden a ser lineales por tramos, por lo que su distancia puede calcularse más fácilmente. Más concretamente, dado que las características de cada clase, $\boldsymbol{Z}_k$ y $\hat{\boldsymbol{Z}}_k$, se acercan a ser un subespacio de baja dimensión/gaussiano, su «distancia» puede medirse mediante la reducción de la tasa, con (6.2.8) restringida a dos conjuntos de igual tamaño:

$$\Delta R_\epsilon\big(\boldsymbol{Z}_k, \hat{\boldsymbol{Z}}_k\big) \doteq R_\epsilon\big(\boldsymbol{Z}_k \cup \hat{\boldsymbol{Z}}_k\big) - \frac{1}{2}\big(R_\epsilon(\boldsymbol{Z}_k) + R_\epsilon(\hat{\boldsymbol{Z}}_k)\big). \tag{6.2.10}$$

La distancia total entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$ viene dada por:

$$d(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) \doteq \sum_{k=1}^{K} \Delta R_\epsilon\big(\boldsymbol{Z}_k, \hat{\boldsymbol{Z}}_k\big) = \sum_{k=1}^{K} \Delta R_\epsilon\big(\boldsymbol{Z}_k, f(g(\boldsymbol{Z}_k, \boldsymbol{\eta}), \boldsymbol{\theta})\big). \tag{6.2.11}$$

Si nos interesa detectar *cualquier* diferencia en las distribuciones de los datos originales $\boldsymbol{X}$ y sus valores decodificados $\hat{\boldsymbol{X}}$, podemos considerar el codificador de características $f(\cdot, \boldsymbol{\theta})$ como un «discriminador» para *amplificar* cualquier diferencia entre todos los pares de $\boldsymbol{X}_k$ y $\hat{\boldsymbol{X}}_k$, simplemente maximizando la distancia $d(\boldsymbol{Z}, \hat{\boldsymbol{Z}})$:

$$\max_f d(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) = \max_{\boldsymbol{\theta}} \sum_{k=1}^{K} \Delta R_\epsilon\big(\boldsymbol{Z}_k, \hat{\boldsymbol{Z}}_k\big) = \max_{\boldsymbol{\theta}} \sum_{k=1}^{K} \Delta R_\epsilon\big(f(\boldsymbol{X}_k, \boldsymbol{\theta}), f(\hat{\boldsymbol{X}}_k, \boldsymbol{\theta})\big). \tag{6.2.12}$$

Es decir, la «distancia» entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ puede medirse como la reducción máxima de la tasa alcanzable entre todos los pares de clases en estos dos conjuntos. De cierto modo, esto mide en qué medida el $\hat{\boldsymbol{X}}$ decodificado se alinea con los datos originales $\boldsymbol{X}$; por lo tanto, mide la calidad de la «injectividad» del codificador f. Obsérvese que dicha medida discriminativa es consistente con el concepto de GAN (6.2.3), que intenta separar $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ en dos clases, cuya separación se cuantifica mediante la entropía cruzada.

No obstante, en este caso el codificador discriminativo f se generaliza en forma natural a casos en los que las distribuciones de datos son multiclase y multimodales, donde la capacidad discriminativa se mide con un indicador más refinado —la reducción de la tasa— en lugar de la típica pérdida de dos clases (por ejemplo, la entropía cruzada) utilizada en las GAN. Es decir, podemos considerar el codificador f como un discriminador generalizado que reemplaza al clasificador binario d en (6.2.3):

$$\boldsymbol{Z} \xrightarrow{\;g(\boldsymbol{z},\boldsymbol{\eta})\;} \hat{\boldsymbol{X}}, \; \boldsymbol{X} \xrightarrow{\;f(\boldsymbol{x},\boldsymbol{\theta})\;} \{\hat{\boldsymbol{Z}}, \boldsymbol{Z}\}. \tag{6.2.13}$$

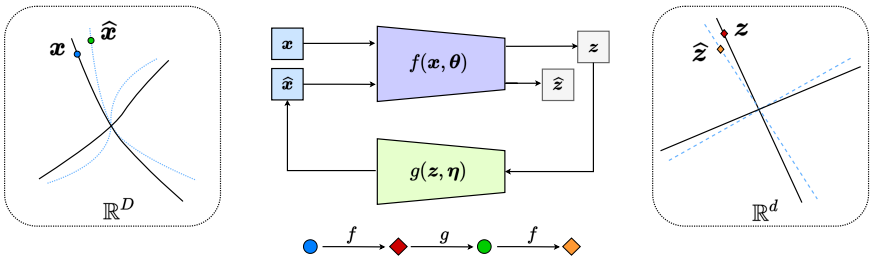


Figura 6.9: **Una transcripción de bucle cerrado.** El codificador f cumple una doble función: aprende una representación $\boldsymbol{z}$ de los datos $\boldsymbol{x}$ maximizando la reducción de la tasa de $\boldsymbol{z}$ y, además, actúa como un «sensor de retroalimentación» para detectar cualquier discrepancia entre los datos $\boldsymbol{x}$ y el resultado decodificado $\hat{\boldsymbol{x}}$. El decodificador g también desempeña una doble función: es un «controlador» que corrige la discrepancia entre $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$ y, además, pretende minimizar la tasa de codificación global de la representación aprendida.

El proceso general se puede ilustrar mediante el siguiente diagrama de «circuito cerrado»:

$$\boldsymbol{X} \xrightarrow{f(\boldsymbol{x},\boldsymbol{\theta})} \boldsymbol{Z} \xrightarrow{g(\boldsymbol{z},\boldsymbol{\eta})} \hat{\boldsymbol{X}} \xrightarrow{f(\boldsymbol{x},\boldsymbol{\theta})} \hat{\boldsymbol{Z}}, \tag{6.2.14}$$

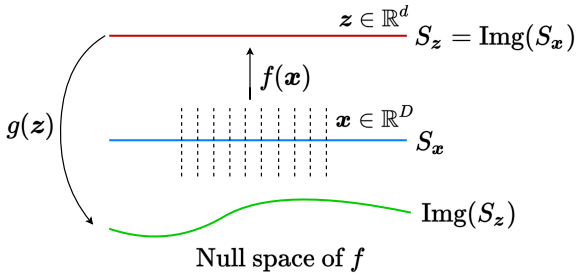
donde el modelo general tiene los parámetros: $\boldsymbol{\Theta} = \{\boldsymbol{\theta}, \boldsymbol{\eta}\}$. La Figura 6.9 muestra el proceso general.

El codificador y el decodificador como un juego de dos jugadores. Obviamente, para garantizar que la autocodificación aprendida sea autoconsistente, el objetivo principal del decodificador $g(\cdot, \boldsymbol{\eta})$ es *minimizar* la distancia entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$. Es decir, para aprender g, queremos minimizar la distancia $d(\boldsymbol{Z}, \hat{\boldsymbol{Z}})$:

$$\min_g d(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) \doteq \min_{\boldsymbol{\eta}} \sum_{k=1}^{K} \Delta R_\epsilon\big(\boldsymbol{Z}_k, \hat{\boldsymbol{Z}}_k\big) = \min_{\boldsymbol{\eta}} \sum_{k=1}^{K} \Delta R_\epsilon\big(\boldsymbol{Z}_k, f(g(\boldsymbol{Z}_k, \boldsymbol{\eta}), \boldsymbol{\theta})\big), \tag{6.2.15}$$

donde $\boldsymbol{Z}_k = f(\boldsymbol{X}_k, \boldsymbol{\theta})$ y $\hat{\boldsymbol{Z}}_k = f(\hat{\boldsymbol{X}}_k, \boldsymbol{\theta})$.

Ejemplo 6.1. Uno podría preguntarse por qué necesitamos que el mapeo $f(\cdot, \boldsymbol{\theta})$ funcione como discriminador entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ mediante la maximización de $\max_{\boldsymbol{\theta}} \Delta R_\epsilon\big(f(\boldsymbol{X}, \boldsymbol{\theta}), f(\hat{\boldsymbol{X}}, \boldsymbol{\theta})\big)$. La Figura 6.10 ofrece una ilustración sencilla: podría haber muchos decodificadores g tales que $f \circ g$ sea una aplicación de identidad (Id). $f \circ g(\boldsymbol{z}) = \boldsymbol{z}$ para todos los $\boldsymbol{z}$ en el subespacio $S_{\boldsymbol{z}}$ del espacio de características. Sin embargo, $g \circ f$ no es necesariamente un mapeo de autocodificación para $\boldsymbol{x}$ en la distribución original $S_{\boldsymbol{x}}$ (representada aquí como un subespacio por razones de simplicidad). Es decir, $g \circ f(S_{\boldsymbol{x}}) \not\subset S_{\boldsymbol{x}}$ y mucho menos $g \circ f(S_{\boldsymbol{x}}) = S_{\boldsymbol{x}}$ o $g \circ f(\boldsymbol{x}) = \boldsymbol{x}$. Cabe suponer que, sin un control cuidadoso de la imagen de g, esto ocurriría con gran probabilidad, especialmente cuando



$$g \circ f \neq \text{Id}, \text{ but } f \circ g = \text{Id}$$

Figura 6.10: **Incrustaciones de subvariedades de baja dimensión en un espacio de alta dimensión.** $S_{\boldsymbol{x}}$ (azul) es la subvariedad correspondiente a los datos originales $\boldsymbol{x}$; $S_{\boldsymbol{z}}$ (rojo) es la imagen de $S_{\boldsymbol{x}}$ bajo el mapeo f, que representa la característica aprendida $\boldsymbol{z}$; y la curva verde es la imagen de la característica $\boldsymbol{z}$ bajo la mapeo de decodificación g.

el soporte de la distribución de $\boldsymbol{x}$ es de dimensión extremadamente baja en el espacio de datos original de alta dimensión. ■

Al comparar la naturaleza contractiva y contrastiva de (6.2.15) y (6.2.12) en la misma medida de distancia, vemos en forma natural los roles del codificador $f(\cdot, \boldsymbol{\theta})$ y del decodificador $g(\cdot, \boldsymbol{\eta})$ como «**un juego de dos jugadores**»: *mientras que el codificador f intenta magnificar la diferencia entre los datos originales y sus datos transcritos, el decodificador g busca minimizar la diferencia.* Ahora, por conveniencia, definamos la función de «codificación en bucle cerrado»:

$$h(\boldsymbol{x}, \boldsymbol{\theta}, \boldsymbol{\eta}) \doteq f\big(g\big(f(\boldsymbol{x}, \boldsymbol{\theta}), \boldsymbol{\eta}\big), \boldsymbol{\theta}\big) : \boldsymbol{x} \mapsto \hat{\boldsymbol{z}}. \tag{6.2.16}$$

Lo ideal sería que esta función fuera muy similar a $f(\boldsymbol{x}, \boldsymbol{\theta})$ o, al menos, que las distribuciones de sus imágenes fueran similares. Con esta notación, combinando (6.2.15) y (6.2.12), se puede calcular una noción de «distancia» de bucle cerrado entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ como *un punto de equilibrio* del siguiente juego de Stackelberg (véase el Apéndice A.3) para la misma utilidad en términos de reducción de la tasa:

$$\mathcal{D}(\boldsymbol{X}, \hat{\boldsymbol{X}}) \doteq \max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} \sum_{k=1}^{K} \Delta R_\epsilon\big(f(\boldsymbol{X}_k, \boldsymbol{\theta}), h(\boldsymbol{X}_k, \boldsymbol{\theta}, \boldsymbol{\eta})\big). \tag{6.2.17}$$

Téngase en cuenta que esto solo mide la diferencia entre (las características de) los datos originales y su versión transcrita. No mide qué tan buena es la representación $\boldsymbol{Z}$ (o $\hat{\boldsymbol{Z}}$) para las múltiples clases dentro de $\boldsymbol{X}$ (o $\hat{\boldsymbol{X}}$). Con este fin, podemos combinar la distancia anterior con los objetivos originales de tipo MCR^2 (6.2.8): es decir, la reducción de la tasa $\Delta R_\epsilon(\boldsymbol{Z})$ y $\Delta R_\epsilon(\hat{\boldsymbol{Z}})$ para la representación discriminativa lineal (LDR) aprendida $\boldsymbol{Z}$ para $\boldsymbol{X}$ y $\hat{\boldsymbol{Z}}$ para

el $\hat{\boldsymbol{X}}$ decodificado. Obsérvese que, aunque el codificador f intenta *maximizar* la reducción de la tasa multiclase de las características $\boldsymbol{Z}$ de los datos $\boldsymbol{X}$, el decodificador g debe *minimizar* la reducción de la tasa de las características multiclase $\hat{\boldsymbol{Z}}$ de los datos decodificados $\hat{\boldsymbol{X}}$. Es decir, el decodificador g intenta utilizar la tasa de codificación mínima necesaria para lograr una buena calidad de decodificación.

Por lo tanto, el juego de Stackelberg «multiclase» global para el aprendizaje de la transcripción en bucle cerrado, denominado CTRL-Multi, es

$$\max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} \mathcal{T}_{\boldsymbol{X}}(\boldsymbol{\theta}, \boldsymbol{\eta}) \tag{6.2.18}$$

$$\doteq \underbrace{\Delta R_\epsilon\big(f(\boldsymbol{X}, \boldsymbol{\theta})\big)}_{\text{codificación expansiva}} + \underbrace{\Delta R_\epsilon\big(h(\boldsymbol{X}, \boldsymbol{\theta}, \boldsymbol{\eta})\big)}_{\text{decodificación compresiva}} + \sum_{k=1}^{K} \underbrace{\Delta R_\epsilon\big(f(\boldsymbol{X}_k, \boldsymbol{\theta}), h(\boldsymbol{X}_k, \boldsymbol{\theta}, \boldsymbol{\eta})\big)}_{\text{codificación contrastiva y decodificación contractiva}} \tag{6.2.19}$$

$$= \Delta R_\epsilon\big(\boldsymbol{Z}(\boldsymbol{\theta})\big) + \Delta R_\epsilon\big(\hat{\boldsymbol{Z}}(\boldsymbol{\theta}, \boldsymbol{\eta})\big) + \sum_{k=1}^{K} \Delta R_\epsilon\big(\boldsymbol{Z}_k(\boldsymbol{\theta}), \hat{\boldsymbol{Z}}_k(\boldsymbol{\theta}, \boldsymbol{\eta})\big), \tag{6.2.20}$$

sujeto a ciertas restricciones (cotas superiores o inferiores) en el primer término y el segundo término.

Nótese que, sin los términos asociados a la parte generativa h o al fijar todos esos términos se fijan como constantes, el objetivo anterior es precisamente el objetivo MCR^2 original introducido en el Capítulo 4. En un contexto no supervisado, si consideramos cada muestra (y sus aumentaciones) como una clase propia, la formulación anterior permanece exactamente igual. El número de clases K es simplemente el número de muestras independientes. Además, cabe señalar que la función objetivo del juego anterior depende únicamente de (las características de) los datos $\boldsymbol{X}$, por lo que es posible entrenar el codificador y el decodificador (parámetros) sin necesidad de muestrear ni ajustar ninguna distribución adicional (como suele ser necesario en las GAN o las VAE).

Como caso especial, si $\boldsymbol{X}$ solo tiene una clase, el juego de Stackelberg se reduce a una forma especial «binaria» o de «dos clases»,[12] denominada CTRL-Binary,

$$\max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} \mathcal{T}^b_{\boldsymbol{X}}(\boldsymbol{\theta}, \boldsymbol{\eta}) \doteq \Delta R_\epsilon\big(f(\boldsymbol{X}, \boldsymbol{\theta}), h(\boldsymbol{X}, \boldsymbol{\theta}, \boldsymbol{\eta})\big) = \Delta R_\epsilon\big(\boldsymbol{Z}(\boldsymbol{\theta}), \hat{\boldsymbol{Z}}(\boldsymbol{\theta}, \boldsymbol{\eta})\big), \tag{6.2.21}$$

entre $\boldsymbol{X}$ y el $\hat{\boldsymbol{X}}$ decodificado, al considerar $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ como dos clases $\{\mathbf{0}, \mathbf{1}\}$. Obsérvese que este caso binario se asemeja a la formulación de la GAN original (6.2.3). En lugar de utilizar la entropía cruzada, nuestra formulación adopta una medida de reducción de tasa más refinada, que ha demostrado que promueve la diversidad en la representación aprendida en el Capítulo 3.

A veces, incluso cuando $\boldsymbol{X}$ contiene varias clases o modos, se podría considerar a todas las clases en conjunto como una sola clase. Entonces, el objetivo binario mencionado anteriormente consiste en alinear la distribución

[12] ya que los dos primeros términos de reducción de la tasa se convierten automáticamente en cero.

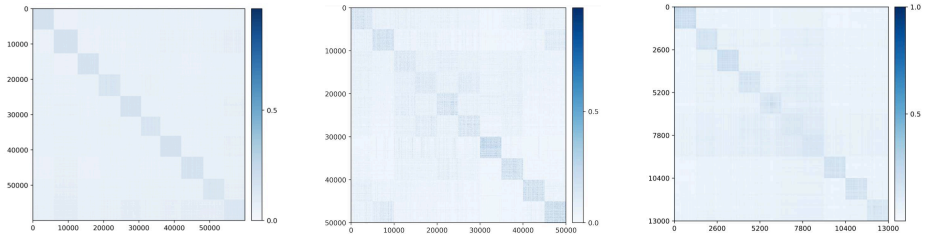
de la unión de todas las clases con su $\hat{\boldsymbol{X}}$ decodificado. Por lo general, esta es una tarea más sencilla de llevar a cabo que la de múltiples clases (6.2.20), ya que no requiere el aprendizaje de un CTRL de múltiples clases más refinado para los datos, como veremos más adelante en los experimentos. Obsérvese que una buena característica de la formulación anterior es que *todas las cantidades en los objetivos se miden en términos de reducción de la tasa para las características aprendidas* (suponiendo que las características se conviertan eventualmente en gaussianas en el subespacio).

Se puede observar que el marco de aprendizaje anterior se inspira en la corrección de errores en bucle cerrado, ampliamente utilizada en los sistemas de control con retroalimentación. El mecanismo de bucle cerrado se utiliza para formar un sistema de retroalimentación general entre las dos redes de codificación y decodificación con el fin de corregir cualquier «error» en las distribuciones entre los datos $\boldsymbol{x}$ y los datos decodificados $\hat{\boldsymbol{x}}$. Utilizando la terminología de la teoría de control, se puede considerar la red de codificación f como un «sensor» para la retroalimentación de error, mientras que la red de decodificación g se considera un «controlador» para la corrección de errores. Sin embargo, hay que tener en cuenta que aquí el «objetivo» del control no es un escalar ni un vector de dimensión finita, sino una distribución continua, para que la distribución de $\hat{\boldsymbol{x}}$ coincida con la de los datos $\boldsymbol{x}$. En general, se trata de un problema de control en un espacio de dimensión infinita. El espacio de los difeomorfismos posibles de las subvariedades que f intenta modelar es de dimensión infinita [Lee02]. Idealmente, esperamos que, cuando el sensor f y el controlador g sean óptimos, la distribución de $\boldsymbol{x}$ se convierta en un «punto fijo» para el bucle cerrado, mientras que la distribución de $\boldsymbol{z}$ alcance una representación discriminativa lineal compacta. Por lo tanto, los programas minimax (6.2.20) y (6.2.21) también pueden interpretarse como juegos entre un sensor de retroalimentación de error y un controlador de reducción de error.

La pregunta que queda por resolver es si el marco descrito anteriormente es realmente capaz de aprender una buena representación (de autocodificación) de un conjunto de datos dado. Antes de presentar una justificación teórica formal (en la siguiente subsección), presentamos algunos resultados empíricos.

Visualización de la correlación entre las características $\boldsymbol{Z}$ y las características decodificadas $\hat{\boldsymbol{Z}}$. Visualizamos la similitud coseno entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$, aprendida a partir del objetivo multiclase (6.2.20) en MNIST, CIFAR-10 e ImageNet (10 clases), lo que indica cuán cerca está $\hat{\boldsymbol{z}} = f \circ g(\boldsymbol{z})$ de $\boldsymbol{z}$. Los resultados de la Figura 6.11 muestran que $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$ están muy bien alineados dentro de cada clase. Los patrones diagonales por bloques para MNIST son más nítidos que los de CIFAR-10 e ImageNet, ya que las imágenes de CIFAR-10 e ImageNet presentan una mayor diversidad de aspectos visuales.

Visualización de la autocodificación de los datos $\boldsymbol{X}$ y de los datos decodificados $\hat{\boldsymbol{X}}$. Comparamos algunos valores representativos de $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ en MNIST, CIFAR-10 e ImageNet (10 clases) para verificar cuán cerca está



(a) MNIST (b) CIFAR-10 (c) ImageNet

Figura 6.11: Visualización de la alineación entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$: $|\boldsymbol{Z}^\top \hat{\boldsymbol{Z}}|$ en el espacio de características para (**a**) MNIST, (**b**) CIFAR-10 y (**c**) ImageNet-10-Class.

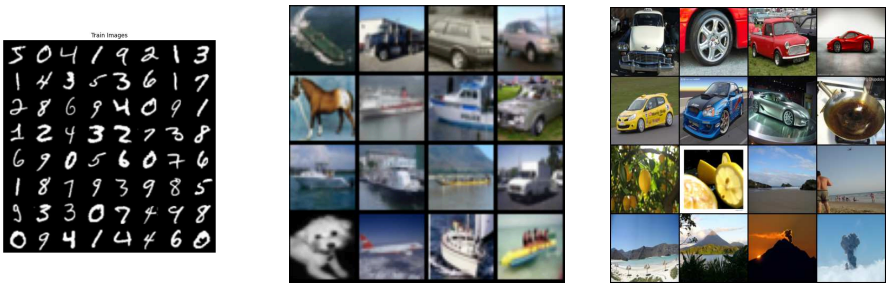
$\hat{\boldsymbol{x}} = g \circ f(\boldsymbol{x})$ de $\boldsymbol{x}$. Los resultados se muestran en la Figura 6.12 y las visualizaciones se crean a partir de muestras de entrenamiento. Visualmente, el $\hat{\boldsymbol{x}}$ autocodificado captura fielmente las principales características visuales de su respectiva muestra de entrenamiento $\boldsymbol{x}$, especialmente la pose, la forma y la disposición. En el caso de conjuntos de datos más sencillos, como el MNIST, las imágenes autocodificadas son casi idénticas a las originales.

6.2.2 Una mezcla de gaussianas de baja dimensión

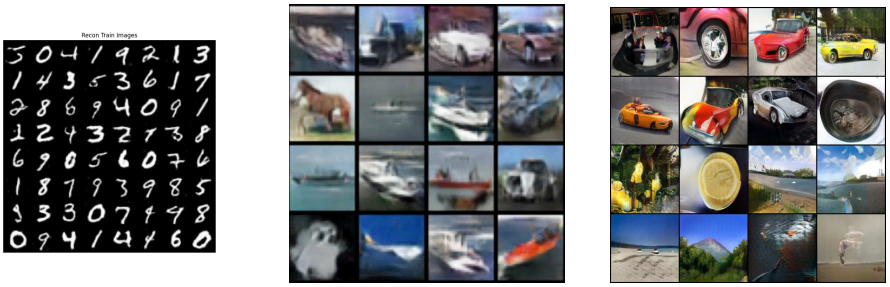
En lo anterior, hemos argumentado que es posible formular el problema de aprender una distribución de datos como un problema de autocodificación en bucle cerrado. También hemos comprobado empíricamente que dicho esquema parece funcionar. La pregunta que queda por responder es cuándo y por qué debería funcionar dicho esquema. Es difícil responder a esta pregunta para los casos más generales con distribuciones de datos arbitrarias. No obstante, como de costumbre, veamos si podemos llegar a una justificación rigurosa para el caso ideal en el que la distribución de los datos es una mezcla de subespacios de baja dimensión o gaussianas de bajo rango. Una caracterización y comprensión claras de este importante caso especial arrojarían luz sobre los casos más generales. [13]

Con este fin, supongamos primero que $\boldsymbol{X}$ se distribuye según una mezcla de gaussianas de baja dimensión y que la etiqueta (es decir, la asignación de subespacio) para $\boldsymbol{X}$ viene dada por $\boldsymbol{y}$. Entonces, formulemos un problema de optimización minimax para aprender la distribución de los datos, por ejemplo, mediante el aprendizaje de una codificación de $\boldsymbol{X}$ en representaciones $\boldsymbol{Z}$ que se apoyan en una mezcla de subespacios *ortogonales* y una decodificación de $\boldsymbol{Z}$ de vuelta a $\boldsymbol{X}$. Entonces, la representación que queremos lograr se maximiza mediante la versión de la ganancia de información mencionada anteriormente, es decir, $\Delta R_\epsilon(\boldsymbol{Z}) = R_\epsilon(\boldsymbol{Z}) - \sum_{k=1}^{K} R_\epsilon(\boldsymbol{Z}_k)$, lo que garantiza que la representación $\boldsymbol{Z}_k$ de cada clase k abarque un subespacio que sea ortogonal a los subespacios de soporte de las demás clases. La forma de medir la consistencia de la

[13] Ya que la mayoría de las distribuciones con estructuras de baja dimensión pueden aproximarse bien mediante esta familia de distribuciones.



(a) MNIST $\boldsymbol{X}$ (b) CIFAR-10 $\boldsymbol{X}$ (c) ImageNet $\boldsymbol{X}$



(d) MNIST $\hat{\boldsymbol{X}}$ (e) CIFAR-10 $\hat{\boldsymbol{X}}$ (f) ImageNet $\hat{\boldsymbol{X}}$

Figura 6.12: Visualización de la propiedad de autocodificación de la transcripción aprendida en bucle cerrado ($\boldsymbol{x} \approx \hat{\boldsymbol{x}} = g \circ f(\boldsymbol{x})$) en MNIST, CIFAR-10 e ImageNet (amplía la imagen para una mejor visualización).

decodificación viene dada, como antes, por $\sum_{k=1}^{K} \Delta R_\epsilon(\boldsymbol{Z}_k, \hat{\boldsymbol{Z}}_k)$, lo que garantiza que la representación $\boldsymbol{Z}_k$ y su autocodificación $\hat{\boldsymbol{Z}}_k$ para cada clase k abarquen el mismo subespacio. Por lo tanto, podemos plantear un juego de Stackelberg simplificado:

$$\max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} \left\{ \Delta R_\epsilon(\boldsymbol{Z}(\boldsymbol{\theta})) + \sum_{k=1}^{K} \Delta R_\epsilon(\boldsymbol{Z}_k(\boldsymbol{\theta}), \hat{\boldsymbol{Z}}_k(\boldsymbol{\theta}, \boldsymbol{\eta})) \right\} \tag{6.2.22}$$

Nótese que se trata de un planteamiento más simple que la que se utiliza en la práctica: no hay ningún término $\Delta R_\epsilon(\hat{\boldsymbol{Z}})$, por ejemplo, y trabajamos en el marco supervisado con etiquetas de clase (aunque las técnicas utilizadas para demostrar el siguiente resultado son fáciles de extender a formulaciones no supervisadas). Además, la consistencia de las representaciones solo se mide a nivel distribucional mediante ΔR_ϵ (aunque esto puede sustituirse por una métrica de distancia a nivel de muestra, como la norma ℓ_2, si se desea, y pueden extraerse conclusiones equivalentes, *mutatis mutandis*).

Bajo condiciones suaves, para obtener el codificador y el decodificador deseados que reconstruyan $\boldsymbol{Z}$ a partir de una fuente de datos $\boldsymbol{X}$ que ya se distribuye según una mezcla de gaussianas correlacionadas de baja dimensión, solo necesitamos un codificador y un decodificador lineales para desenredar y

blanquear las gaussianas. A continuación, estudiamos este escenario en el caso en que $\boldsymbol{\theta}$ y $\boldsymbol{\eta}$ parametrizan matrices cuya norma de operador está acotada.

Queremos comprender qué tipos de óptimos se aprenden en este contexto. Un concepto de solución adecuado para este tipo de juego, en el que la solución óptima del decodificador se define únicamente en función del codificador (y no, en particular, en función de alguna otra propiedad intrínseca del decodificador), es un *equilibrio de Stackelberg* en el que el decodificador sigue al codificador. Es decir, en ese equilibrio, el decodificador debería optimizar su objetivo; mientras tanto, el codificador debería optimizar su objetivo, dado que, independientemente de lo que elija, el decodificador elegirá una respuesta óptima (lo cual puede afectar al objetivo del codificador). En términos de teoría de juegos, es como si el decodificador fuera el *segundo*: elige sus pesos después del codificador, y tanto el codificador como el decodificador intentan optimizar su objetivo a la luz de esto. Es computacionalmente viable aprender equilibrios secuenciales mediante métodos de gradiente a través de *optimización alterna*, en la que cada lado utiliza diferentes tasas de aprendizaje. La exposición detallada de los equilibrios secuenciales queda fuera del alcance de este libro; ofrecemos más detalles técnicos en la Apéndice A.3. En este contexto, tenemos el siguiente resultado:

Teorema 6.1 ([PPC+23], Abridged)**.** *Supongamos que $\boldsymbol{X}$ se distribuye sobre una mezcla de subespacios. Bajo ciertas condiciones realistas, aunque técnicas, se cumple que todos los equilibrios secuenciales de* (6.2.22) *satisfacen que:*

- *Los vectores $\boldsymbol{Z}_k$ se encuentran en subespacios ortogonales y son isotrópicos en dichos subespacios, es decir, maximizan la ganancia de información.*
- *La autocodificación es autoconsistente, es decir, los subespacios generados por $\boldsymbol{Z}_k$ y $\hat{\boldsymbol{Z}}_k$ son los mismos para todo k.*

Este concepto de autoconsistencia es lo máximo que cabe esperar si solo se parte de supuestos geométricos sobre los datos, es decir, si no hay supuestos estadísticos. Si suponemos que las columnas de $\boldsymbol{X}$ se extraen de un modelo de mezcla gaussiana de rango bajo, entonces versiones análogas de este teorema certifican que $\boldsymbol{Z}_k$ son también gaussianas de rango bajo cuya covarianza es isotrópica, por ejemplo. En esencia, este resultado confirma, a través del caso sencillo de las mezclas gaussianas en subespacios, que los juegos minimax destinados a optimizar la ganancia de información y la autoconsistencia pueden alcanzar soluciones óptimas.

6.3 Aprendizaje continuo de representaciones autoconsistentes

6.3.1 Aprendizaje incremental por clase

Como hemos visto, las redes neuronales profundas han demostrado una gran capacidad para aprender representaciones de cientos o incluso miles de clases de

objetos, tanto en contextos discriminativos como generativos. Sin embargo, por lo general, estas redes deben entrenarse fuera de línea, con datos muestreados uniformemente de todas las clases al mismo tiempo. Se sabe que cuando una red (de bucle abierto) se actualiza para aprender nuevas clases sin datos de las anteriores, el conocimiento adquirido previamente sufre del problema del *olvido catastrófico* [MC89]. Esto se conoce en neurociencia como el dilema estabilidad-plasticidad: el reto de garantizar que un sistema neuronal pueda aprender de un nuevo entorno al tiempo que conserva el conocimiento esencial de los anteriores [Gro87].

Por el contrario, los sistemas neuronales naturales (por ejemplo, los cerebros de los animales) no parecen sufrir en absoluto ese olvido catastrófico. Son capaces de desarrollar nuevas memorias de objetos nuevos al tiempo que conservan el recuerdo de los objetos aprendidos anteriormente. Esta capacidad, tanto en los sistemas neuronales naturales como en los artificiales, se conoce a menudo como *aprendizaje incremental, aprendizaje continuo, aprendizaje secuencial* o *aprendizaje de por vida* [AR20].

Aunque muchos trabajos recientes han destacado cómo los sistemas neuronales artificiales pueden entrenarse de maneras más flexibles, los esfuerzos más sólidos actuales para resolver el dilema entre estabilidad y plasticidad de las redes neuronales artificiales suelen requerir, o bien almacenar ejemplos sin procesar [CRE+19; RKS+17], o bien proporcionar mecanismos externos [KPR+17]. Los ejemplos sin procesar, especialmente en el caso de entradas de alta dimensión como las imágenes, son costosos y difíciles de escalar, mientras que los mecanismos externos —que suelen incluir redes secundarias y espacios de representación para la reproducción generativa, la asignación incremental de recursos de red, la duplicación de redes o el aislamiento explícito de las partes utilizadas y no utilizadas de la red— recurren a heurísticas e incurren en costos ocultos.

En este trabajo, nos interesa un entorno de aprendizaje incremental que sea similar a la naturaleza. Este se opone a estas prácticas existentes a través de dos características clave.

1. La primera es que sea *basado en memoria*. Al aprender nuevas clases, no se dispone de ejemplos sin procesar de clases antiguas para entrenar la red junto con los nuevos datos. Esto implica que hay que basarse en una «memoria» compacta y, por lo tanto, estructurada, aprendida a partir de las clases antiguas: por ejemplo, representaciones generativas aprendidas de manera incremental de dichas clases, así como los correspondientes mapeos de codificación y decodificación [KK18].

2. El segundo es que sea *autocontenido*. El aprendizaje incremental tiene lugar en un único sistema neuronal con una capacidad fija y en un espacio de representación común. La capacidad de minimizar el olvido se deriva de la optimización de un objetivo de aprendizaje global, sin redes externas, modificaciones arquitectónicas ni mecanismos de asignación de recursos.

Las estructuras lineales incoherentes para características de diferentes clases

se asemejan mucho a la forma en que se codifican los objetos en diferentes áreas de la corteza inferotemporal de los cerebros de los animales [BSM+20; CT17]. La transcripción de bucle cerrado $\boldsymbol{X} \to \boldsymbol{Z} \to \hat{\boldsymbol{X}} \to \hat{\boldsymbol{Z}}$ también se asemeja a los mecanismos generalmente hipotetizados para la formación de la memoria [JT20; VST+20]. Esto nos lleva a una pregunta: dado que la memoria en el cerebro se forma incrementalmente, ¿podría dicho marco de transcripción en bucle cerrado soportar igualmente un aprendizaje incremental?

Muestreo y reproducción de la memoria con representación discriminativa lineal (LDR). Las estructuras lineales simples de LDR lo hacen particularmente idóneo para el aprendizaje incremental: la distribución de características $\boldsymbol{Z}_j$ de cada clase aprendida previamente puede representarse en forma explícita y concisa mediante un subespacio principal $\mathcal{S}_j$ en el espacio de características. Para conservar la memoria de una clase previa j, solo necesitamos conservar el subespacio mientras aprendemos nuevas clases. Para ello, simplemente muestreamos una de m características prototípicas representativas en el subespacio a lo largo de sus r componentes principales superiores y denotamos estas características como $\boldsymbol{Z}_{j,old}$. Debido a las estructuras lineales simples de LDR, podemos tomar una muestra de $\boldsymbol{Z}_{j,old}$ calculando la media y la covarianza de $\boldsymbol{Z}_{j,old}$ después de aprender la clase j. El espacio de almacenamiento necesario es extremadamente reducido, ya que solo necesitamos almacenar medias y covarianzas, de las cuales se toman muestras según sea necesario. Supongamos que hasta ahora se han aprendido un total de t clases previas. Si las características de los prototipos, denotadas como $\boldsymbol{Z}_{old} \doteq [\boldsymbol{Z}_{1,old}, \ldots, \boldsymbol{Z}_{t,old}]$, para todas estas clases pueden conservarse al aprender nuevas clases, los subespacios $\{\mathcal{S}_j\}_{j=1}^{t}$ que representan la memoria pasada también se conservarán. Para más detalles sobre el muestreo y el cálculo de la media y la covarianza, véase el trabajo de [TDW+23].

Aprendizaje incremental de LDR con una restricción de memoria de clases anteriores Obsérvese que, con la autocodificación aprendida (6.2.9), se pueden reproducir y utilizar las imágenes, por ejemplo $\hat{\boldsymbol{X}}_{old} = g(\boldsymbol{Z}_{old}, \boldsymbol{\eta})$, asociadas con las características de la memoria para evitar el olvido mientras se aprenden nuevas clases. Así es como se han utilizado típicamente los modelos generativos en métodos de aprendizaje incremental anteriores. Sin embargo, con el marco de bucle cerrado, no es necesario reproducir explícitamente las imágenes a partir de las características. La memoria del pasado se puede conservar de manera eficaz mediante una optimización centrada exclusivamente en las propias características.

Consideremos la tarea de aprender incrementalmente una nueva clase de objetos.[14] Denotamos el nuevo conjunto de muestras correspondiente como $\boldsymbol{X}_{new}$. Las características de $\boldsymbol{X}_{new}$ se denotan como $\boldsymbol{Z}_{new}(\boldsymbol{\theta}) = f(\boldsymbol{X}_{new}, \boldsymbol{\theta})$. Las concatenamos con las características de prototipo de las clases previas $\boldsymbol{Z}_{old}$

[14] Por supuesto, también se puede considerar el caso más general en el que la tarea incluye un pequeño conjunto de nuevas clases, sin modificaciones significativas.

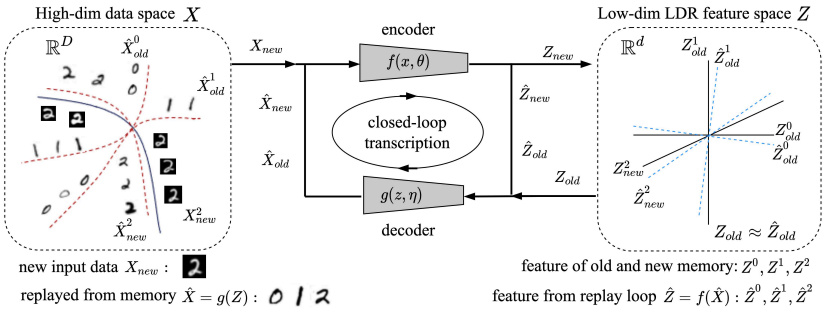


Figura 6.13: **Marco general** de nuestro aprendizaje incremental basado en la transcripción en bucle cerrado para una memoria LDR estructurada. Solo se necesita una única red de codificación-decodificación, totalmente autónoma: para una nueva clase de datos $\boldsymbol{X}_{new}$, se aprende en forma incremental una nueva memoria LDR $\boldsymbol{Z}_{new}$ como un juego minimax entre el codificador y el decodificador, sujeto a la restricción de que la memoria antigua de clases pasadas $\boldsymbol{Z}_{old}$ permanezca intacta a través de la transcripción (o reproducción) de bucle cerrado: $\boldsymbol{Z}_{old} \approx \hat{\boldsymbol{Z}}_{old} = f(g(\boldsymbol{Z}_{old}))$.

y formamos $\boldsymbol{Z} = [\boldsymbol{Z}_{new}(\boldsymbol{\theta}), \boldsymbol{Z}_{old}]$. Denotamos las imágenes reproducidas a partir de todas las características como $\hat{\boldsymbol{X}} = [\hat{\boldsymbol{X}}_{new}(\boldsymbol{\theta}, \boldsymbol{\eta}), \hat{\boldsymbol{X}}_{old}(\boldsymbol{\eta})]$, aunque en realidad no necesitamos calcularlas ni utilizarlas explícitamente. Solo necesitamos las características de las imágenes reproducidas, denotadas como $\hat{\boldsymbol{Z}} = f(\hat{\boldsymbol{X}}, \boldsymbol{\theta}) = [\hat{\boldsymbol{Z}}_{new}(\boldsymbol{\theta}, \boldsymbol{\eta}), \hat{\boldsymbol{Z}}_{old}(\boldsymbol{\theta}, \boldsymbol{\eta})]$.

Siguiendo la misma lógica que el objetivo CTRL multiclase (6.2.20), queremos que las características de la nueva clase $\boldsymbol{Z}_{new}$ sean incoherentes con todas las antiguas $\boldsymbol{Z}_{old}$. Dado que $\boldsymbol{Z}_{new}$ es la única clase nueva cuyas características deben aprenderse, el objetivo (6.2.20) se reduce al caso en el que $K = 1$:

$$\min_{\boldsymbol{\eta}} \max_{\boldsymbol{\theta}} \Delta R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\hat{\boldsymbol{Z}}) + \Delta R_\epsilon(\boldsymbol{Z}_{new}, \hat{\boldsymbol{Z}}_{new}). \tag{6.3.1}$$

Sin embargo, cuando actualizamos los parámetros de la red $(\boldsymbol{\theta}, \boldsymbol{\eta})$ para optimizar las características de la nueva clase, las correspondencias actualizadas f y g también modificarán las características de las clases previas. Por lo tanto, para minimizar la distorsión de las representaciones de las clases previas, podemos intentar imponer $\mathrm{Cov}(\boldsymbol{Z}_{j,old}) = \mathrm{Cov}(\hat{\boldsymbol{Z}}_{j,old})$. En otras palabras, mientras aprendemos nuevas clases, imponemos que la memoria de las clases previas permanezca «autoconsistente» a través del bucle de transcripción:

$$\boldsymbol{Z}_{old} \xrightarrow{g(\boldsymbol{z},\boldsymbol{\eta})} \hat{\boldsymbol{X}}_{old} \xrightarrow{f(\boldsymbol{x},\boldsymbol{\theta})} \hat{\boldsymbol{Z}}_{old}. \tag{6.3.2}$$

Matemáticamente, esto equivale a establecer

$$\Delta R_\epsilon(\boldsymbol{Z}_{old}, \hat{\boldsymbol{Z}}_{old}) \doteq \sum_{j=1}^{t} \Delta R_\epsilon(\boldsymbol{Z}_{j,old}, \hat{\boldsymbol{Z}}_{j,old}) = 0.$$

Por lo tanto, el programa minimax anterior (6.3.1) se reformula como un juego minimax *restringido*, al que nos referimos como *transcripción incremental de bucle cerrado* (i-CTRL). El objetivo de este juego es idéntico al objetivo estándar de CTRL multiclase (6.2.20), pero incluye solo una restricción adicional:

$$\begin{aligned} \min_{\boldsymbol{\eta}} \max_{\boldsymbol{\theta}} \quad & \Delta R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\hat{\boldsymbol{Z}}) + \Delta R_\epsilon(\boldsymbol{Z}_{new}, \hat{\boldsymbol{Z}}_{new}) \\ & \text{sujeto a} \quad \Delta R_\epsilon(\boldsymbol{Z}_{old}, \hat{\boldsymbol{Z}}_{old}) = 0. \end{aligned} \tag{6.3.3}$$

En la práctica, el problema de minimax con restricciones se puede resolver mediante la minimización y maximización *alternadas* entre el codificador $f(\cdot, \boldsymbol{\theta})$ y el decodificador $g(\cdot, \boldsymbol{\eta})$ de la siguiente manera:

$$\max_{\boldsymbol{\theta}} \quad \Delta R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\hat{\boldsymbol{Z}}) + \lambda \cdot \Delta R_\epsilon(\boldsymbol{Z}_{new}, \hat{\boldsymbol{Z}}_{new}) - \gamma \cdot \Delta R_\epsilon(\boldsymbol{Z}_{old}, \hat{\boldsymbol{Z}}_{old}), \tag{6.3.4}$$

$$\min_{\boldsymbol{\eta}} \quad \Delta R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\hat{\boldsymbol{Z}}) + \lambda \cdot \Delta R_\epsilon(\boldsymbol{Z}_{new}, \hat{\boldsymbol{Z}}_{new}) + \gamma \cdot \Delta R_\epsilon(\boldsymbol{Z}_{old}, \hat{\boldsymbol{Z}}_{old}), \tag{6.3.5}$$

donde la restricción $\Delta R_\epsilon(\boldsymbol{Z}_{old}, \hat{\boldsymbol{Z}}_{old}) = 0$ de (6.3.3) se ha transformado en un término lagrangiano relajado con el coeficiente correspondiente γ y su signo. Además, se introduce otro coeficiente λ para ponderar el término de reducción de la tasa asociado a los nuevos datos.

Memoria óptima conjunta mediante repaso incremental. Como veremos, el programa minimax con restricciones anterior ya puede alcanzar un rendimiento de vanguardia en el aprendizaje incremental. Sin embargo, el desarrollo de una memoria óptima para *todas las clases* no puede basarse únicamente en el olvido gradual. Incluso en el ser humano, si una clase de objetos se aprende solo una vez, es de esperar que el recuerdo se desvanezca a medida que seguimos aprendiendo otras nuevas, a menos que el recuerdo pueda consolidarse mediante el repaso de clases de objetos anteriores.

Para emular esta fase de formación de la memoria, tras aprender en forma incremental todo un conjunto de datos, podemos volver atrás para revisar todas las clases de nuevo, una por una. Nos referimos a recorrer todas las clases una vez como un «ciclo» de repaso.[15] Si es necesario, se pueden llevar a cabo varios ciclos de repaso. Es de esperar que el repaso mejore la memoria aprendida (LDR). Lo realmente sorprendente, sin embargo, es que el marco de bucle cerrado nos permite revisar incluso «sin supervisión de clases»: al revisar los datos de una clase antigua, digamos $\boldsymbol{X}_j$, el sistema no necesita la etiqueta de clase y puede tratar $\boldsymbol{X}_j$ simplemente como una clase nueva, $\boldsymbol{X}_{new}$. Es decir, el sistema optimiza el mismo programa minimax con restricciones (6.3.3) sin ninguna modificación; una vez optimizado el sistema, se puede identificar el subespacio recién aprendido generado por $\boldsymbol{Z}_{new}$ y utilizarlo para reemplazar o fusionar con el subespacio anterior $\mathcal{S}_j$. Como muestran nuestros experimentos, este proceso de revisión incremental sin supervisión de clases puede mejorar gradualmente tanto el rendimiento discriminativo como el generativo de la memoria LDR, hasta converger finalmente en el de una memoria aprendida en forma conjunta.

[15] Para distinguirlo del término «época» utilizado en el contexto convencional del aprendizaje conjunto.

(a) MNIST $\boldsymbol{X}$ (b) MNIST $\hat{\boldsymbol{X}}$ (c) CIFAR-10 $\boldsymbol{X}$ (d) CIFAR-10 $\hat{\boldsymbol{X}}$

Figura 6.14: Visualización de la propiedad de autocodificación de la representación aprendida ($\hat{\boldsymbol{X}} = g \circ f(\boldsymbol{X})$).

Verificación experimental. Presentamos algunos resultados experimentales en los siguientes conjuntos de datos: MNIST [LBB+98b] y CIFAR-10 [KNH14]. Todos los experimentos se llevan a cabo en el escenario más exigente de class-IL. Tanto para MNIST como para CIFAR-10, las 10 clases se dividen en 5 tareas con 2 clases cada una o en 10 tareas con 1 clase cada una. Para el codificador f y el decodificador g, adoptamos una arquitectura de red muy sencilla modificada a partir de DCGAN [RMC16], que no es más que una red convolucional de *cuatro capas*. Aquí solo mostramos algunos resultados visuales cualitativos; se pueden encontrar más experimentos y un análisis detallado en el trabajo [TDW+23].

Visualización de las propiedades de la autocodificación. Comenzamos visualizando cualitativamente algunas imágenes representativas $\boldsymbol{X}$ y su réplica correspondiente $\hat{\boldsymbol{X}}$ en MNIST y CIFAR-10. El modelo se entrena en forma incremental con los conjuntos de datos divididos en 5 tareas. Los resultados se muestran en la Figura 6.14, donde observamos que el $\hat{\boldsymbol{X}}$ reconstruido conserva las principales características visuales de $\boldsymbol{X}$, incluyendo formas y texturas. En el caso de un conjunto de datos más sencillo como MNIST, ¡los $\hat{\boldsymbol{X}}$ reproducidos son casi idénticos a la entrada $\boldsymbol{X}$! Esto es bastante notable dado que: (1) nuestro método no impone explícitamente que $\hat{\boldsymbol{x}} \approx \boldsymbol{x}$ para muestras individuales, como lo hacen la mayoría de los métodos de autocodificación, y (2) después de haber aprendido de manera incremental todas las clases, el generador no ha olvidado cómo generar los dígitos aprendidos anteriormente, como 0, 1, 2. En el caso de un conjunto de datos más complejo, como CIFAR-10, también demostramos una buena calidad visual, capturando fielmente la esencia de cada imagen.

Subespacios principales de las características aprendidas. La mayoría de los métodos generativos basados en memoria utilizan autocodificadores, VAE o GAN para la reproducción. La estructura o distribución de las características aprendidas $\boldsymbol{Z}_j$ para cada clase no está clara en el espacio de características. Las características $\boldsymbol{Z}_j$ de la memoria LDR, por otro lado, tienen una estructura lineal clara. La Figura 6.15 visualiza las correlaciones entre todas las características aprendidas $|\boldsymbol{Z}^\top \boldsymbol{Z}|$, en las que claramente observamos patrones diagonales por

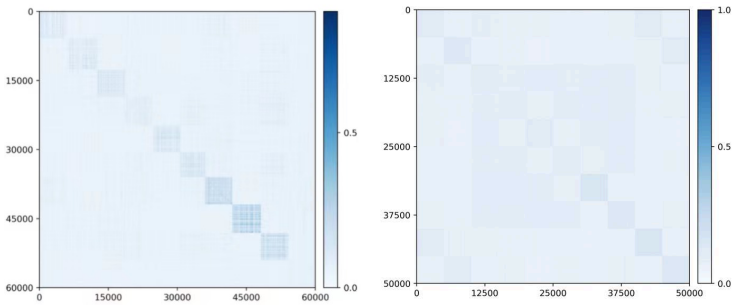


Figura 6.15: Estructura diagonal por bloques de $|\boldsymbol{Z}^\top \boldsymbol{Z}|$ en el espacio de características para MNIST (izquierda) y CIFAR-10 (derecha).

bloques para ambos conjuntos de datos.[16] Esto indica que las características de las diferentes clases $\boldsymbol{Z}_j$ se encuentran, de hecho, en subespacios que son mutuamente incoherentes. Por lo tanto, las características de cada clase pueden modelarse adecuadamente como un subespacio principal en el espacio de características.

Reconstrucción de imágenes de muestras a partir de componentes principales. Dado que las características de cada clase pueden modelarse como un subespacio principal, visualizamos además las componentes principales individuales dentro de cada uno de esos subespacios. La Figura 6.16 muestra las imágenes reconstruidas a partir de características muestreadas a lo largo de las primeras cuatro componentes principales para diferentes clases, en MNIST y CIFAR-10, respectivamente. Cada fila representa muestras a lo largo de una componente principal y mostrando características visuales similares, pero son claramente diferentes de las de otras filas. Vemos que el modelo recuerda diferentes posturas del «4» después de haber aprendido todas las demás clases. En el caso de CIFAR-10, la memoria aprendida en forma incremental recuerda posturas y formas representativas de caballos y barcos.

Eficacia del repaso incremental. Verificamos cómo la memoria LDR aprendida en forma incremental puede consolidarse aún más mediante una fase de repaso incremental no supervisada descrita anteriormente. Los experimentos se llevan a cabo en CIFAR-10, con 10 pasos. La Figura 6.17 a la izquierda muestra imágenes reproducidas de la primera clase «avión» al final del aprendizaje incremental de las diez clases, seleccionadas a lo largo de las tres primeras componentes principales: cada dos filas (16 imágenes) se encuentran a lo largo de una dirección principal. Su calidad visual sigue siendo muy satisfactoria: apenas observamos olvido. La figura de la derecha muestra

[16] Obsérvese que estos patrones se asemejan mucho a la matriz de similitud de los perfiles de respuesta de las categorías de objetos procedentes de diferentes áreas de la corteza inferotemporal, tal y como se muestra en la Fig. 3 de los datos extendidos de [BSM+20].

(a) $\hat{\boldsymbol{x}}_{old}$ muestreado de '4' (b) $\hat{\boldsymbol{x}}_{old}$ muestreado de '7' (c) $\hat{\boldsymbol{x}}_{old}$ muestreado de «caballo» (d) $\hat{\boldsymbol{x}}_{old}$ muestreado de «barco»

Figura 6.16: Visualización de 5 $\hat{\boldsymbol{x}} = g(\boldsymbol{z})$ reconstruidos a partir de los valores de $\boldsymbol{z}$ más cercanos a las (4 principales) componentes principales de las características aprendidas para MNIST (clase «4» y clase «7») y CIFAR-10 (clases «caballo» y «barco»).

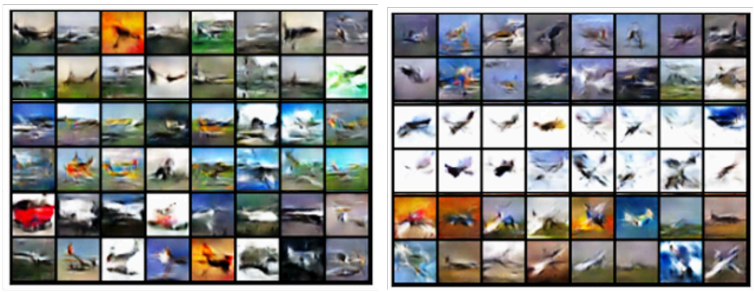


Figura 6.17: Visualización de las imágenes reconstruidas $\hat{\boldsymbol{x}}_{old}$ de la clase 1-«avión» en CIFAR-10, antes (izquierda) y después (derecha) de un ciclo de repaso.

las imágenes reproducidas tras repasar la primera clase una vez. Se aprecia una mejora significativa en la calidad visual tras el repaso y las componentes principales de las características en el subespacio comienzan a corresponder a atributos visuales claramente diferenciados dentro de la misma clase.

6.3.2 Aprendizaje continuo no supervisado muestra por muestra

Como sabemos, la formulación CTRL de bucle cerrado ya es capaz de aprender una autocodificación razonablemente buena, incluso sin información de clase, mediante el programa CTRL-Binary:

$$\max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} \quad \Delta R_{\epsilon}(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) \tag{6.3.6}$$

Sin embargo, cabe señalar que (6.3.6) tiene limitaciones prácticas, ya que solo alinea el conjunto de datos $\boldsymbol{X}$ y el $\hat{\boldsymbol{X}}$ regenerado a nivel de distribución. No hay garantía de que cada muestra $\boldsymbol{x}$ se aproxime al $\hat{\boldsymbol{x}} = g(f(\boldsymbol{x}))$ decodificado.

Restricciones muestra por muestra para la transcripción no supervisada. Para mejorar las propiedades discriminativas y generativas de

las representaciones aprendidas en el escenario no supervisado, proponemos dos mecanismos adicionales para el juego de maximin CTRL-Binary mencionado anteriormente (6.3.6). En aras de la simplicidad y la uniformidad, aquí se formulan como restricciones de igualdad sobre las medidas de reducción de la tasa, pero en la práctica pueden imponerse en forma suave durante el proceso de optimización.

Autoconsistencia muestra por muestra mediante transcripción en bucle cerrado. En primer lugar, para abordar el problema de que CTRL-Binary no aprende una autocodificación consistente a nivel de muestra, necesitamos promover la proximidad entre $\hat{\boldsymbol{x}}$ y $\boldsymbol{x}$ para cada muestra. En el marco de CTRL, esto se puede lograr asegurando que sus características correspondientes $\boldsymbol{z} = f(\boldsymbol{x})$ y $\hat{\boldsymbol{z}} = f(\hat{\boldsymbol{x}})$ sean similares. Para promover la autoconsistencia por muestra, donde $\hat{\boldsymbol{x}} = g(f(\boldsymbol{x}))$ es cercano a $\boldsymbol{x}$, queremos que la distancia entre $\boldsymbol{z}$ y $\hat{\boldsymbol{z}}$ sea cero o pequeña, para todas las N muestras. Esta distancia puede medirse mediante la reducción de la tasa:

$$\sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \hat{\boldsymbol{z}}^i) = 0. \tag{6.3.7}$$

Tenga en cuenta que esto evita, una vez más, medir diferencias en el espacio de imágenes.

Autosupervisión mediante la compresión de muestras aumentadas. Dado que en el contexto no supervisado no conocemos la información sobre las etiquetas de clase entre las muestras, lo mejor que podemos hacer es considerar cada muestra junto con sus augmentaciones (por ejemplo, mediante traslación, rotación, oclusión, etc.) como un único grupo o «clase» base: una idea básica que subyace a casi todos los métodos de aprendizaje autosupervisado. En el marco de la reducción de la tasa, resulta intuitivo comprimir las características de cada muestra y sus augmentaciones. En este trabajo, adoptamos las transformaciones estándar de SimCLR [CKN+20] y denotamos dicha transformación como τ. Denotamos cada muestra augmentada como $\boldsymbol{x}_a = \tau(\boldsymbol{x})$ y su característica correspondiente como $\boldsymbol{z}_a = f(\boldsymbol{x}_a, \boldsymbol{\theta})$. Con fines discriminativos, esperamos que el clasificador sea *invariante* a tales transformaciones. Por lo tanto, es lógico exigir que las características $\boldsymbol{z}_a$ de todas las augmentaciones sean las mismas que las $\boldsymbol{z}$ de la muestra original $\boldsymbol{x}$. Esto equivale a exigir que la distancia entre $\boldsymbol{z}$ y $\boldsymbol{z}_a$, medida nuevamente en términos de reducción de la tasa, sea cero (o pequeña) para todas las N muestras:

$$\sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \boldsymbol{z}_a^i) = 0. \tag{6.3.8}$$

Aprendizaje de representaciones sin supervisión mediante transcripción en bucle cerrado. Hasta ahora, sabemos que el objetivo

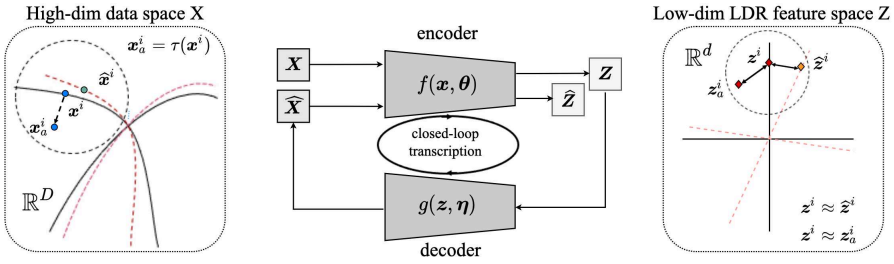


Figura 6.18: **Marco general** de la transcripción en bucle cerrado para el aprendizaje no supervisado. Se imponen dos restricciones adicionales al método CTRL-Binary: 1) autoconsistencia para las características de cada muestra $\boldsymbol{z}^i$ y $\hat{\boldsymbol{z}}^i$, es decir, $\boldsymbol{z}^i \approx \hat{\boldsymbol{z}}^i$; y 2) invarianza/similitud entre las características de las muestras augmentadas $\boldsymbol{z}^i$ y $\boldsymbol{z}_a^i$, es decir, $\boldsymbol{z}^i \approx \boldsymbol{z}_a^i = f(\tau(\boldsymbol{x}^i), \boldsymbol{\theta})$, donde $\boldsymbol{x}_a^i = \tau(\boldsymbol{x}^i)$ es un aumento de la muestra $\boldsymbol{x}^i$ mediante alguna transformación $\tau(\cdot)$.

CTRL-Binary $\Delta R_\epsilon(\boldsymbol{Z}, \hat{\boldsymbol{Z}})$ de (6.3.6) ayuda a alinear las distribuciones, mientras que la autoconsistencia por muestra (6.3.7) y la augmentación por muestra (6.3.8) ayudan a alinear y comprimir las características asociadas con cada muestra. Además de la consistencia, también queremos que las representaciones aprendidas sean lo más discriminativas posible para diferentes muestras (aquí consideradas como diferentes «clases»). Nótese que el término de tasa-distorsión $R_\epsilon(\boldsymbol{Z})$ mide la tasa de codificación (y, por lo tanto, el volumen) de todas las características.

CTRL no supervisado. Al combinar estos elementos, proponemos aprender una representación mediante el siguiente programa maximin con restricciones, al que denominamos *CTRL no supervisado* (u-CTRL):

$$\begin{aligned} \max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} \quad & R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) \\ \text{sujeto a} \quad & \sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \hat{\boldsymbol{z}}^i) = 0, \;\; \text{y} \;\; \sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \boldsymbol{z}_a^i) = 0. \end{aligned} \tag{6.3.9}$$

La Figura 6.18 ilustra la arquitectura general del sistema de bucle cerrado asociado a este programa.

En la práctica, el programa anterior se puede optimizar alternando la maximización y la minimización entre el codificador $f(\cdot, \boldsymbol{\theta})$ y el decodificador $g(\cdot, \boldsymbol{\eta})$. Adoptamos la siguiente estrategia de optimización, que rinde bien en los experimentos y que se utiliza para todos los experimentos posteriores con conjuntos de datos de imágenes reales:

$$\max_{\boldsymbol{\theta}} \; R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) - \lambda_1 \sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \boldsymbol{z}_a^i) - \lambda_2 \sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \hat{\boldsymbol{z}}^i) \tag{6.3.10}$$

$$\min_{\boldsymbol{\eta}} \; R_\epsilon(\boldsymbol{Z}) + \Delta R_\epsilon(\boldsymbol{Z}, \hat{\boldsymbol{Z}}) + \lambda_1 \sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \boldsymbol{z}_a^i) + \lambda_2 \sum_{i \in N} \Delta R_\epsilon(\boldsymbol{z}^i, \hat{\boldsymbol{z}}^i), \tag{6.3.11}$$

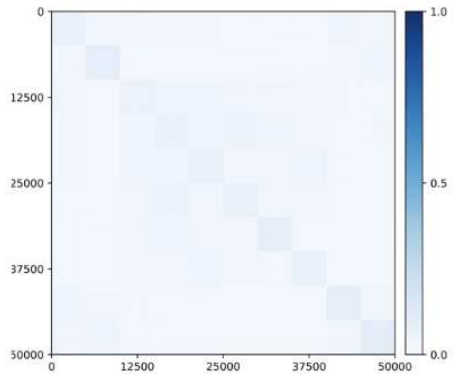


Figura 6.19: Aparición de estructuras diagonales en bloques de $|\boldsymbol{Z}^\top \boldsymbol{Z}|$ en el espacio de características para CIFAR-10.

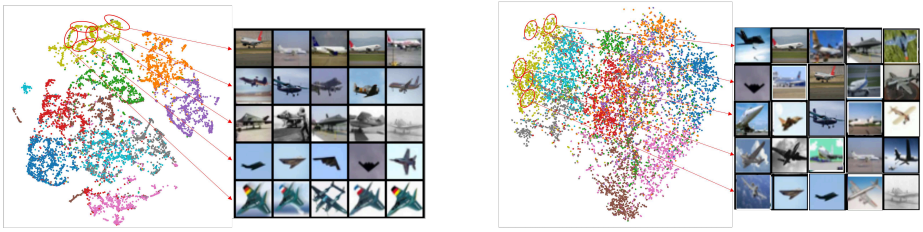
donde las restricciones $\sum_{i\in N} \Delta R_\epsilon(\boldsymbol{z}^i, \hat{\boldsymbol{z}}^i) = 0$ y $\sum_{i\in N} \Delta R_\epsilon(\boldsymbol{z}^i, \boldsymbol{z}_a^i) = 0$ en (6.3.9) se han convertido (y relajado) a términos lagrangianos con los coeficientes correspondientes λ_1 y λ_2.[17]

La representación anterior se aprende sin información de clase. Para facilitar tareas discriminativas o generativas, debe ser altamente estructurada en lo interno. Se ha comprobado experimentalmente que es efectivamente así y u-CTRL demuestra tener ventajas significativas con respecto a otros métodos de aprendizaje incremental o no supervisado [TDC+24]. Aquí solo mostramos algunos resultados cualitativos del experimento realizado con el conjunto de datos CIFAR-10 [KNH14], utilizando transformaciones de augmentación estándar propias del aprendizaje autosupervisado [CKN+20]. Se puede consultar [TDC+24] para ver experimentos con conjuntos de datos más numerosos y de mayor tamaño y sus evaluaciones cuantitativas.

Como se puede observar en los experimentos, de hecho, surge en forma natural una estructura específica y única en las representaciones aprendidas mediante u-CTRL: a nivel global, las características de las imágenes de una misma clase tienden a agruparse bien juntas y a separarse de las de otras clases, como se muestra en la Figura 6.19; a nivel local, las características de las clases individuales se agrupan en grupos compactos y bien diferenciados, como se muestra en la Figura 6.20.

Generación condicional no supervisada de imágenes mediante reducción de tasa. La distribución altamente estructurada de las características también sugiere que la representación aprendida puede resultar

[17] Obsérvese que calcular los términos de reducción de tasa ΔR para todas las muestras o un lote de muestras requiere calcular el costoso $\log\det$ de matrices grandes. En la práctica, a partir del significado geométrico de ΔR para dos vectores, ΔR puede aproximarse con una norma ℓ^2 o la distancia coseno entre dos vectores. Esta aproximación se desarrolla más a fondo en la Sección 4.3.2, donde motiva el objetivo de SimDINO.



(a) u-CTRL (b) MoCoV2

Figura 6.20: Visualizaciones t-SNE de las características aprendidas de CIFAR-10 con diferentes modelos.

muy útil con fines generativos. Por ejemplo, podemos organizar las características de las muestras en grupos significativos y modelarlas mediante distribuciones (gaussianas) de baja dimensión o subespacios. Al tomar muestras de estos modelos compactos, podemos regenerar condicionalmente muestras significativas a partir de los grupos calculados. Esto se conoce como *generación condicional de imágenes no supervisada* [HKJ+21].

Para agrupar características, aprovechamos el hecho de que el marco de reducción de tasa (4.2.15) se inspira en la agrupación no supervisada mediante compresión [MDH+07b], lo que proporciona una forma fundamentada de hallar la pertenencia $\boldsymbol{\Pi}$. Concretamente, maximizamos el mismo objetivo de reducción de tasa (4.2.15) sobre $\boldsymbol{\Pi}$, pero fijamos la representación aprendida $\boldsymbol{Z}$ en su lugar. Simplemente consideramos la pertenencia $\boldsymbol{\Pi}$ como una función no lineal de las características $\boldsymbol{Z}$, por ejemplo $h_{\boldsymbol{\pi}}(\cdot, \xi) : \boldsymbol{Z} \mapsto \boldsymbol{\Pi}$ con parámetros ξ. En la práctica, modelamos esta función con una red neuronal simple, como un cabezal MLP justo después de la característica de salida $\boldsymbol{z}$. Para estimar una «pseudo» pertenencia $\hat{\boldsymbol{\Pi}}$ de las muestras, resolvemos el siguiente problema de optimización sobre $\boldsymbol{\Pi}$:

$$\hat{\boldsymbol{\Pi}} = \arg\max_{\xi} \Delta R_{\epsilon}(\boldsymbol{Z} \mid \boldsymbol{\Pi}(\xi)). \tag{6.3.12}$$

En la Figura 6.21, visualizamos imágenes generadas a partir de los diez clústeres no supervisados de (6.3.12). Cada bloque representa un grupo y cada fila representa una componente principal para cada clúster. A pesar de habersido aprendido y entrenado sin etiquetas, el modelo no solo organiza las muestras en los grupos correctos, sino que también es capaz de preservar la diversidad estadística dentro de cada clúster o clase. Podemos recuperar fácilmente la diversidad dentro de cada grupo calculando diferentes componentes principales y, a continuación, muestrear y generar las imágenes adecuadamente. Aunque los experimentos que aquí se presentan tienen una escala algo limitada, en el próximo capítulo exploraremos métodos más directos y potentes que utilizan las distribuciones y representaciones de datos aprendidas para la generación y estimación condicionales.

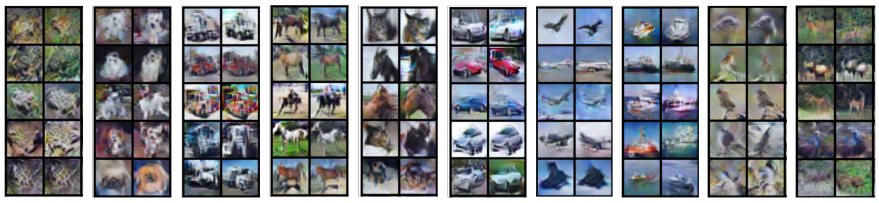


Figura 6.21: Generación condicional de imágenes sin supervisión a partir de cada clúster de CIFAR-10, utilizando u-CTRL. Las imágenes de las diferentes filas corresponden a la generación a partir de las diferentes componentes principales de cada grupo.

6.4 Resumen y Notas

Históricamente, la autocodificación ha sido uno de los motores más importantes de la innovación en la investigación sobre redes neuronales para el aprendizaje, aunque las demostraciones más impactantes desde el punto de vista práctico del aprendizaje profundo probablemente se hayan dado en otros dominios (como la clasificación discriminativa, con AlexNet [KSH12], o el modelado generativo con arquitecturas GPT [BMR+20]). Los trabajos que hemos destacado a lo largo del capítulo, especialmente el de [HS06], sirvieron como catalizadores del interés por la investigación en redes neuronales en una época en la que, de otro modo, estas no eran tan destacadas en el panorama de la investigación en aprendizaje automático. En la práctica actual, los autocodificadores siguen siendo componentes fundamentales de muchos sistemas a gran escala destinados a generar datos altamente estructurados, como datos visuales, datos de voz y datos moleculares, especialmente el enfoque VQ-VAE [OVK17], que se basa en la metodología de los autocodificadores variacionales que analizamos en la Sección 6.1.4. El problema central de la autocodificación sigue revistiendo una importancia intelectual suprema debido a su estrecha vinculación con el aprendizaje de representaciones y anticipamos que volverá a ser una prioridad para los investigadores orientados a la práctica en el futuro, a medida que la eficiencia en el entrenamiento y la implementación de modelos a gran escala siga cobrando mayor importancia.

Los materiales presentados en la segunda mitad de este capítulo se basan en una serie de trabajos recientes sobre el tema de la transcripción de bucle cerrado: [DTL+22], [PPC+23], [TDW+23] y [TDC+24]. En particular, la Sección 6.2.1 se basa en el trabajo pionero de [DTL+22]. Posteriormente, el trabajo de [PPC+23] ha proporcionado sólidas justificaciones teóricas para el marco de bucle cerrado, al menos en un caso ideal. La Sección 6.3.1 y la Sección 6.3.2 se basan en los trabajos de [TDW+23] y [TDC+24], respectivamente. Demuestran que el marco de bucle cerrado facilita en forma natural el aprendizaje incremental y continuo, ya sea en un entorno por clases o por muestras. El lector puede consultar estos artículos para obtener más detalles técnicos y experimentales.

Redes neuronales superficiales vs. profundas, para la autocodificación y otras aplicaciones. En la Sección 6.1.2, analizamos el teorema de aproximación universal de Cybenko y cómo este establece que, en principio, una red neuronal con una sola capa oculta (y no linealidades adecuadas componente a componente) es suficiente para aproximar cualquier función objetivo regularizada de manera adecuada. Por supuesto, la principal razón arquitectónica del predominio de las redes neuronales en la práctica ha sido el refinamiento de técnicas para el entrenamiento de redes neuronales *más profundas*. ¿Por qué es necesaria la profundidad? Desde un punto de vista fundamental, la cuestión de las separaciones por profundidad, que permite establecer escenarios en los que una red neuronal más profunda puede aproximar una clase determinada de funciones objetivo con una eficiencia exponencialmente superior respecto a una red superficial, ha sido ampliamente estudiada en la literatura teórica: entre los ejemplos se incluyen [BN20; Tel16; VJO+21]. La facilidad para entrenar redes más profundas en la práctica no ha recibido una respuesta tan satisfactoria desde la perspectiva teórica. Las ResNets [HZR+16b] representan el trabajo empírico pionero para facilitar el entrenamiento de redes más profundas y se utilizan de alguna forma en casi todas las arquitecturas modernas. Los estudios teóricos se han centrado en gran medida en la capacidad de entrenamiento de las redes muy profundas, cuantificada a través del Núcleo Tangencial Neuronal Inicial (NTK) [BGW21; MBD+21], pero estos estudios no han aportado información significativa sobre las ventajas de entrenabilidad de las redes más profundas en las etapas intermedias y finales del entrenamiento (véase, sin embargo, [YH21] para conocer el origen de una línea de investigación que intenta abordar este tema).

6.5 Ejercicios y extensiones

Ejercicio 6.1 (Comprensión conceptual del aplanamiento de variedades). Consideremos datos situados en una variedad curva $\mathcal{M}$ incrustada en $\mathbb{R}^D$ (como una superficie curva en el espacio tridimensional), como se presenta en la subsección sobre el aplanamiento de variedades en la Sección 6.1.2. En este ejercicio, describiremos las componentes básicas del algoritmo de aplanamiento de variedades de [PPR+24]. Una variedad se denomina plana si es un conjunto abierto en el espacio euclídeo (o, más en general, un conjunto abierto en un subespacio).

1. Supongamos que la variedad $\mathcal{M}$ es un *grafo*: esto significa que $\mathcal{M} \subset \mathbb{R}^{D_1} \times \mathbb{R}^{D_2}$ (por ejemplo) y que existe una función $F : \mathbb{R}^{D_1} \to \mathbb{R}^{D_2}$ tal que

$$\mathcal{M} = \{(\boldsymbol{x}, F(\boldsymbol{x})) \mid \boldsymbol{x} \in \mathbb{R}^{D_1}\}. \tag{6.5.1}$$

   Proporcione un número entero D_3 y una aplicación $f : \mathbb{R}^{D_1} \times \mathbb{R}^{D_2} \to \mathbb{R}^{D_3}$ que aplane $\mathcal{M}$ y describa el procedimiento de reconstrucción correspondiente (sin pérdidas de información) a partir de la representación aplanada.

2. Supongamos ahora que $\mathcal{M}$ es una variedad general suave. Las variedades suaves tienen la propiedad de que se aproximan bien localmente mediante subespacios cerca de cada punto. Describa en términos intuitivos cómo aplanar la variedad $\mathcal{M}$ localmente en cada punto, relacionándola con un grafo. (El algoritmo de [PPR+24] realiza una versión refinada de este proceso de aplanamiento local de una manera que permite unirlos para formar un aplanamiento global.)

Ejercicio 6.2 (Reproducir la transcripción en bucle cerrado). Implemente un proceso de transcripción en bucle cerrado para el aprendizaje de representaciones en el conjunto de datos CIFAR-10 siguiendo la metodología descrita en la Sección 6.2.1. Consulte [DTL+22] para los hiperparámetros y ajustes de arquitectura recomendados. Reproduzca el resultado en la Figura 6.11.

Capítulo 7

Inferencia con distribuciones de baja dimensión

> "*Las matemáticas son el arte de dar el mismo nombre a cosas diferentes.*"
>
> — Henri Poincaré

En los capítulos anteriores de este libro, hemos estudiado cómo aprender de manera eficaz y eficiente una representación de una variable $\boldsymbol{x}$ del nundo real con una distribución $p(\boldsymbol{x})$ que tiene un soporte de baja dimensión en un espacio de alta dimensión. Hasta ahora, hemos desarrollado principalmente la metodología para el aprendizaje de representaciones y la autocodificación de una manera general, independiente de la distribución o la tarea. Si se dispone de una representación aprendida de este tipo, ya es posible utilizarla para realizar algunas tareas genéricas y básicas, como la clasificación (si la codificación se supervisa con la clase) y la generación de muestras aleatorias que tengan la misma distribución que los datos dados (por ejemplo, imágenes naturales o lenguajes naturales).

Sin embargo, en términos más generales, la universalidad y la escalabilidad del marco teórico y computacional presentado en este libro nos han permitido aprender la distribución de una variedad de datos importantes del mundo real y que son de alta dimensión, tales como los idiomas naturales, las posturas humanas, las imágenes naturales, los videos e incluso las escenas 3D. Una vez que las estructuras intrínsecamente ricas y de baja dimensión de estos datos reales pueden aprenderse y representarse correctamente, comienzan a hacer posible una amplia familia de tareas potentes, a menudo aparentemente milagrosas. Por lo tanto, a partir de aquí, comenzaremos a mostrar cómo conectar y adaptar los métodos generales presentados en capítulos anteriores para aprender

representaciones útiles para distribuciones específicas de datos estructurados y para muchas tareas populares en la práctica moderna de la inteligencia artificial.

7.1 Inferencia bayesiana y optimización con restricciones

7.1.1 Inferencia bayesiana con distribuciones de baja dimensión

Aprovechar la baja dimensionalidad para obtener una inferencia estable y robusta. En términos generales, una buena representación o autocodificación debería permitirnos emplear la distribución de baja dimensión aprendida de los datos $\boldsymbol{x}$ y su representación $\boldsymbol{z}$ para diversas tareas posteriores de clasificación, estimación y generación en diferentes condiciones. Como ya insinuamos en el Capítulo 1, Sección 1.2.2, la importancia de la *baja dimensionalidad* de la distribución es clave para que podamos realizar inferencias estables y robustas relacionadas con los datos $\boldsymbol{x}$, tal como lo ilustran los pocos ejemplos sencillos de la Figura 1.11, a partir de observaciones incompletas, ruidosas e incluso corrompidas. Resulta que ese mismo concepto se aplica a los datos del mundo real de alta dimensión cuyas distribuciones tienen un soporte de baja dimensión, como las imágenes naturales y los idiomas.

A pesar de una deslumbrante variedad de aplicaciones en la práctica del aprendizaje automático con datos como idiomas, imágenes, videos y muchas otras modalidades, casi todas las aplicaciones prácticas pueden considerarse un caso especial del siguiente problema de inferencia: dada una observación $\boldsymbol{y}$ que depende de $\boldsymbol{x}$, digamos

$$\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}, \tag{7.1.1}$$

donde $h(\cdot)$ representa mediciones de una parte de $\boldsymbol{x}$ o ciertos atributos observados y $\boldsymbol{w}$ representa cierto ruido de medición e incluso alteraciones (escasas), resolver el «problema inverso» consistente en obtener la estimación más probable $\hat{\boldsymbol{x}}(\boldsymbol{y})$ de $\boldsymbol{x}$ o generar una muestra $\hat{\boldsymbol{x}}$ que sea al menos consistente con la observación $\boldsymbol{y} \approx h(\hat{\boldsymbol{x}})$. La Figura 7.1 ilustra la relación general entre $\boldsymbol{x}$ y $\boldsymbol{y}$.

Ejemplo 7.1 (Completado de imágenes y predicción de texto). La popular tarea de completar imágenes y la predicción de lenguaje natural son dos tareas típicas que nos exigen reconstruir un conjunto de datos completo $\boldsymbol{x}$ a partir de sus observaciones parciales $\boldsymbol{y}$, en las que algunas partes de $\boldsymbol{x}$ están ocultas y deben completarse basándose en el resto. La Figura 7.2 muestra algunos ejemplos de este tipo de tareas. De hecho, son precisamente estas tareas las que han inspirado los métodos de entrenamiento de los modelos modernos de gran tamaño para la generación de texto (como GPT) y el completado de imágenes naturales (como el autocodificador enmascarado), que estudiaremos con mayor detalle más adelante. ■

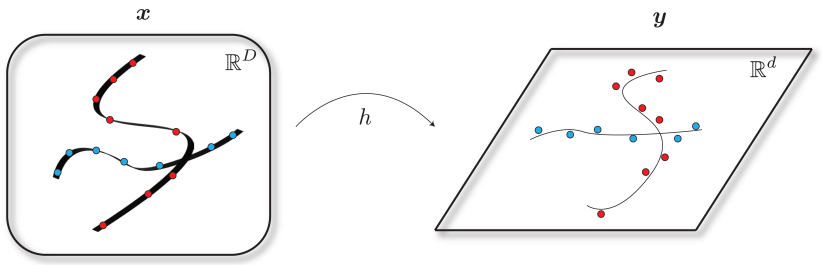


Figura 7.1: **Inferencia con distribuciones de baja dimensión.** Esta es la imagen genérica de este capítulo: tenemos una distribución de baja dimensión para $\boldsymbol{x} \in \mathbb{R}^D$ (aquí representada como una unión de dos variedades de 2 dimensiones en $\mathbb{R}^3$) y un modelo de medición $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w} \in \mathbb{R}^d$. Queremos inferir varias cosas sobre este modelo, incluida la distribución condicional de $\boldsymbol{x}$ dado $\boldsymbol{y}$, o la esperanza condicional $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}]$, a partir de diversos elementos de información sobre el modelo y (un número potencialmente finito de) muestras de $\boldsymbol{x}$ o de $\boldsymbol{y}$.

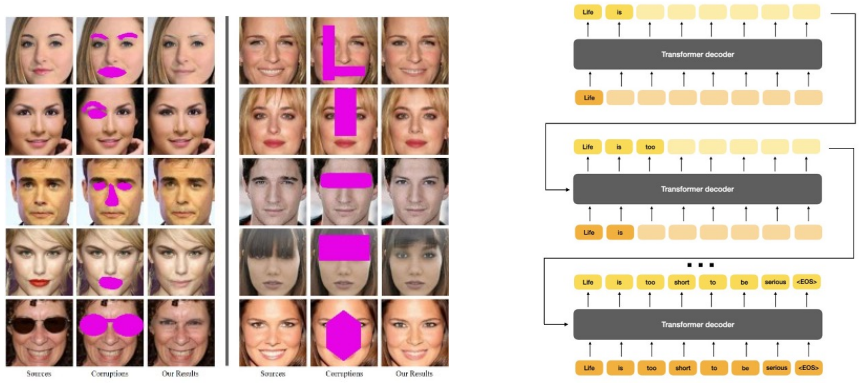


Figura 7.2: Izquierda: completado de imágenes. Derecha: predicción de texto. En particular, la predicción de texto es la fuente de inspiración del popular Generative Pre-trained Transformer (GPT).

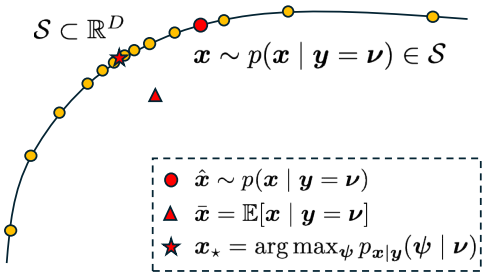


Figura 7.3: Comparación de tres sustitutos diferentes para una estimación de $\boldsymbol{x}$ dado $\boldsymbol{y}$.

Interpretación estadística mediante la regla de Bayes. En términos generales, para resolver bien este tipo de tareas, necesitamos conocer la distribución condicional $p(\boldsymbol{x} \mid \boldsymbol{y})$. Si la tuviéramos, entonces podríamos hallar la estimación de máxima verosimilitud (predicción):

$$\hat{\boldsymbol{x}} = \arg\max_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{y}); \tag{7.1.2}$$

o calcular la estimación de la esperanza condicional:

$$\hat{\boldsymbol{x}} = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}] = \int \boldsymbol{x} p(\boldsymbol{x} \mid \boldsymbol{y}) \mathrm{d}\boldsymbol{x}; \tag{7.1.3}$$

o muestrear de la distribución condicional:

$$\hat{\boldsymbol{x}} \sim p(\boldsymbol{x} \mid \boldsymbol{y}). \tag{7.1.4}$$

Nótese que si la distribución condicional $p(\boldsymbol{x} \mid \boldsymbol{y})$ tiene un soporte que es de baja dimensión y no lineal, estas tres estimaciones diferentes pueden ser bastante distintas, tal como se ilustra en la Figura 7.3. Conceptualmente, la estimación máxima a posteriori (MAP) es la más deseable: es la muestra con mayor probabilidad de haber generado la observación $\boldsymbol{y}$, pero generalmente es computacionalmente difícil calcularla.

Obsérvese que, según la regla de Bayes, tenemos

$$p(\boldsymbol{x} \mid \boldsymbol{y}) = \frac{p(\boldsymbol{y} \mid \boldsymbol{x}) p(\boldsymbol{x})}{p(\boldsymbol{y})}. \tag{7.1.5}$$

Por ejemplo, la estimación de máxima verosimilitud se puede calcular resolviendo el siguiente programa (de máxima verosimilitud logarítmica):

$$\hat{\boldsymbol{x}} = \arg\max_{\boldsymbol{x}} [\log p(\boldsymbol{y} \mid \boldsymbol{x}) + \log p(\boldsymbol{x})], \tag{7.1.6}$$

digamos mediante el método de ascenso por gradiente:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha \cdot \big(\nabla_{\boldsymbol{x}} \log p(\boldsymbol{y} \mid \boldsymbol{x}) + \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \big). \tag{7.1.7}$$

El cálculo eficiente de la distribución condicional $p(\boldsymbol{x} \mid \boldsymbol{y})$ depende, naturalmente, de cómo aprendamos y aprovechemos la distribución de baja dimensión $p(\boldsymbol{x})$ de los datos $\boldsymbol{x}$ y el modelo de observación $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$ que determina la distribución condicional $p(\boldsymbol{y} \mid \boldsymbol{x})$.

Nota 7.1 (Inferencia de extremo a extremo vs. inferencia bayesiana). En la práctica moderna del aprendizaje automático basado en datos, para ciertas tareas habituales, a menudo se aprende directamente la distribución condicional $p(\boldsymbol{x} \mid \boldsymbol{y})$, una función de mapeo (probabilística) o un regresor. Esta correspondencia suele modelarse mediante redes profundas y entrenarse de extremo a extremo con un número suficiente de pares de muestras $(\boldsymbol{x}, \boldsymbol{y})$. Este enfoque es muy diferente del enfoque bayesiano anterior, en el que se necesitan tanto la distribución de $\boldsymbol{x} \sim p(\boldsymbol{x})$ como la correspondencia (de observación). La ventaja del enfoque bayesiano es que la distribución aprendida $p(\boldsymbol{x})$ puede facilitar muchas tareas diferentes con diversos modelos de observación y condiciones.

7.1.2 Optimización con restricciones en subvariedades

Interpretación geométrica como optimización con restricciones. Dado que el soporte $\mathcal{S}_{\boldsymbol{x}}$ de la distribución de $\boldsymbol{x}$ es de baja dimensión, podemos suponer que existe una función F tal que

$$F(\boldsymbol{x}) = 0 \qquad \Longleftrightarrow \qquad \boldsymbol{x} \in \mathcal{S}_{\boldsymbol{x}} \tag{7.1.8}$$

de modo que $\mathcal{S}_{\boldsymbol{x}} = F^{-1}(\{0\})$ es el soporte de baja dimensión de la distribución $p(\boldsymbol{x})$. Geométricamente, una opción natural para $F(\boldsymbol{x})$ es la «función de distancia» al soporte $\mathcal{S}_{\boldsymbol{x}}$:

$$F(\boldsymbol{x}) = \min_{\boldsymbol{x}_p \in \mathcal{S}_{\boldsymbol{x}}} \|\boldsymbol{x} - \boldsymbol{x}_p\|_2. \tag{7.1.9}$$

Obsérvese que, en realidad, solo contamos con muestras discretas en el soporte de la distribución. Siguiendo el mismo espíritu de continuación, a través de la difusión o codificación con pérdida estudiadas en los Capítulos 3 y 4, podemos aproximar la función de distancia como $F(\boldsymbol{x}) \approx \min_{\boldsymbol{x}_p \in \mathcal{C}_{\boldsymbol{x}}^{\epsilon}} \|\boldsymbol{x} - \boldsymbol{x}_p\|_2$, donde $\mathcal{S}_{\boldsymbol{x}}$ se sustituye por una cubrimiento $\mathcal{C}_{\boldsymbol{x}}^{\epsilon}$ de las muestras con bolas de radio ϵ. Pero si se conoce la forma analítica de una distribución simple, a veces $F(\boldsymbol{x})$ se puede calcular de manera explícita.

Ejemplo 7.2 (Distance to a Line in $\mathbb{R}^3$). Supongamos que el soporte de una distribución de baja dimensión en $\mathbb{R}^3$ es el eje x_3. Entonces, la función de distancia $F(\boldsymbol{x})$ viene dada por:

$$F(\boldsymbol{x}) = \sqrt{x_1^2 + x_2^2}. \tag{7.1.10}$$

■

Para ver cómo una función de este tipo desempeña un papel importante a la hora de aprovechar la distribución, por simplicidad, supondremos que, para el resto de la subsección, la función de distancia ya está dada.

Ahora bien, dado que $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$, para hallar el valor de $\boldsymbol{x}$, podemos resolver el siguiente problema de optimización con restricciones:

$$\max_{\boldsymbol{x}} -\frac{1}{2}\|h(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 \quad \text{s.t.} \quad F(\boldsymbol{x}) = 0. \tag{7.1.11}$$

Mediante el método del lagrangiano aumentado, podemos resolver el siguiente programa sin restricciones:

$$\max_{\boldsymbol{x}} \left[-\frac{1}{2}\|h(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \lambda F(\boldsymbol{x}) - \frac{\mu}{2} F(\boldsymbol{x})^2 \right] \tag{7.1.12}$$

para algún multiplicador de Lagrange constante λ. Esto equivale al siguiente programa:

$$\max_{\boldsymbol{x}} \left[\log \exp\Big(-\frac{1}{2}\|h(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 \Big) + \log \exp\Big(-\frac{\mu}{2}\big(F(\boldsymbol{x}) - \lambda/\mu\big)^2 \Big) \right], \tag{7.1.13}$$

donde $c \doteq \lambda/\mu$ puede considerarse como una «media» de la función de restricción. A medida que μ aumenta al aplicar la restricción mediante la continuación[1], c se vuelve cada vez más pequeño. El programa anterior puede interpretarse de dos maneras diferentes.

En primer lugar, se puede considerar el primer término como la probabilidad condicional de $\boldsymbol{y}$ dado $\boldsymbol{x}$ y el segundo término como una densidad de probabilidad de $\boldsymbol{x}$:

$$p(\boldsymbol{y} \mid \boldsymbol{x}) \propto \exp\Big(-\frac{1}{2}\|h(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 \Big), \quad p(\boldsymbol{x}) \propto \exp\Big(-\frac{\mu}{2}(F(\boldsymbol{x}) - c)^2 \Big). \tag{7.1.14}$$

Por lo tanto, resolver la optimización con restricciones del problema inverso equivale a realizar una inferencia bayesiana con las densidades de probabilidad anteriores. Por lo tanto, resolver el programa anterior (7.1.13) mediante el método de ascenso por gradiente equivale a la estimación de máxima verosimilitud anterior (7.1.7), en la que el gradiente toma la forma:

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{y} \mid \boldsymbol{x}) + \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}) \tag{7.1.15}$$

$$= \quad -(h(\boldsymbol{x}) - \boldsymbol{y})^\top \frac{\partial h}{\partial \boldsymbol{x}}(\boldsymbol{x}) - \mu(F(\boldsymbol{x}) - c)\frac{\partial F}{\partial \boldsymbol{x}}(\boldsymbol{x}), \tag{7.1.16}$$

donde $\frac{\partial h}{\partial \boldsymbol{x}}(\boldsymbol{x})$ y $\frac{\partial F}{\partial \boldsymbol{x}}(\boldsymbol{x})$ son las matrices jacobianas de $h(\boldsymbol{x})$ y $F(\boldsymbol{x})$, respectivamente.

[1] Siguiendo la misma línea de continuación del Capítulo 3, donde obtuvimos mejores aproximaciones de nuestra distribución haciendo que $\varepsilon \to 0$, aquí hacemos que $\mu \to \infty$. Valores más grandes de μ restringirán F a tomar valores cada vez más pequeños en el óptimo, lo que significa que el óptimo se encuentra dentro de un vecindario cada vez más pequeño del soporte $\mathcal{S}_{\boldsymbol{x}}$. Curiosamente, la teoría de los multiplicadores de Lagrange alude a que, bajo ciertas condiciones favorables sobre F y otros términos del objetivo, solo necesitamos hacer que μ sea lo suficientemente grande como para garantizar que $F(\boldsymbol{x}) = 0$ en el punto óptimo, lo que significa que con una penalización *finita* obtenemos una aproximación *perfecta* del soporte. En general, debemos tener la intuición de que μ desempeña el mismo papel que ε^{-1}.

Ejemplo 7.3 (Gradiente de $F(\boldsymbol{x})$). Para la función de distancia definida en el ejemplo 7.2, su gradiente viene dado por

$$\frac{\partial F}{\partial \boldsymbol{x}}(\boldsymbol{x}) = \frac{1}{\sqrt{x_1^2 + x_2^2}} \begin{bmatrix} x_1 \\ x_2 \\ 0 \end{bmatrix}. \tag{7.1.17}$$

Nótese que, a diferencia de las funciones suaves habituales, cuyo gradiente suele ser único en un punto fijo, los gradientes de la función de distancia en un punto del soporte, por ejemplo $(0, 0, x_3)$, pueden abarcar todo un subespacio complementario al soporte. ■

Obsérvese que el siguiente (gradiente)

$$-\mu(F(\boldsymbol{x}) - c)\frac{\partial F}{\partial \boldsymbol{x}}(\boldsymbol{x})$$

que siempre apunta hacia el soporte de baja dimensión de la distribución. Por lo tanto, el proceso de descenso puede considerarse como un proceso de «eliminación de ruido», estudiado en el Capítulo 3, que gradualmente obliga a los datos a acercarse al soporte correcto. Nótese que la derivación anterior sugiere que el «tamaño del paso» del *score* $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$ del proceso de eliminación de ruido es

$$\mu\big|F(\boldsymbol{x}) - \lambda/\mu\big|$$

, lo cual, para cualquier μ fijo, es casi proporcional a la distancia del punto $\boldsymbol{x}$ al soporte.

En segundo lugar, obsérvese que resolver el programa anterior (7.1.13) con μ creciente es equivalente a:

$$\boldsymbol{x}_\star = \arg\min_{\boldsymbol{x}} \frac{1}{2}\|h(\boldsymbol{x}) - \boldsymbol{y}\|_2^2 + \frac{\mu}{2}\big(F(\boldsymbol{x}) - \lambda/\mu\big)^2 \quad \text{as} \quad \mu \uparrow \infty. \tag{7.1.18}$$

Debido a la notoria forma cuadrática de los dos términos de la ecuación anterior, también pueden interpretarse como ciertas funciones de «energía». A esta formulación se la suele denominar «minimización de la energía» en la literatura de aprendizaje automático, defendida por investigadores como Yann LeCun [LCH+06]. Aquí, la energía de un punto $\boldsymbol{x}$ es una función cuadrática de su distancia al soporte de la distribución deseada. Obsérvese que en el Capítulo 3 y el Capítulo 4 hemos argumentado que el concepto de entropía sirve para medir el «volumen» o la incertidumbre de la distribución de los datos, mientras que aquí la energía depende de la «distancia» de $\boldsymbol{x}$ al soporte de la distribución. A medida que minimizamos las funciones de energía anteriores, la entropía (o incertidumbre) de las soluciones viables se reduce hasta alcanzar (una solución viable de) la estimación MAP óptima, como se ilustra en la Figura 7.4.

7.1.3 Entornos prácticos representativos para la inferencia

Tenga en cuenta que el análisis y la derivación anteriores se basan en el supuesto de que tenemos un conocimiento perfecto de la función $F(\boldsymbol{x})$ y del modelo de

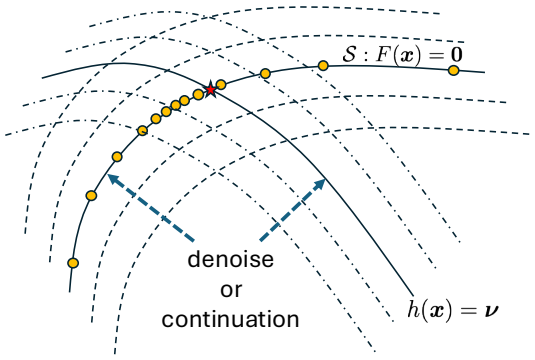


Figura 7.4: Ilustración del proceso de continuación al imponer las restricciones $F(\boldsymbol{x}) = 0$ y $\boldsymbol{y} = h(\boldsymbol{x})$.

observación $h(\boldsymbol{x})$. Sin embargo, en la práctica, es posible que no se disponga de ellos en absoluto y que sea necesario «aprenderlos» a partir de los datos proporcionados. Por lo tanto, para que la solución conceptual anterior sea realmente computable, necesitamos abordar diversas situaciones en las que la información sobre las distribuciones de $\boldsymbol{x}$ y la relación entre $\boldsymbol{x}$ e $\boldsymbol{y}$ se proporciona o es accesible de diferentes maneras y formas.

En general, se pueden clasificar principalmente en cuatro casos, que, desde el punto de vista conceptual, presentan un grado de dificultad cada vez mayor:

- *Caso 1:* Se conocen tanto el modelo de distribución de $\boldsymbol{x}$ como el modelo de observación $\boldsymbol{y} = h(\boldsymbol{x})$ (posiblemente con ruido añadido $\boldsymbol{w}$), incluso de forma analítica. Este suele ser el caso de muchos problemas clásicos de procesamiento de señales, como la eliminación de ruido en señales, el problema de recuperación de vectores esparsos que vimos en el Capítulo 2 y el problema de recuperación de matrices de rango bajo que se presentará a continuación.

- *Caso 2:* No disponemos de un modelo de distribución, sino únicamente de muestras $\boldsymbol{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ de $\boldsymbol{x}$ y se conoce el modelo de observación $\boldsymbol{y} = h(\boldsymbol{x})$ (posiblemente con ruido añadido $\boldsymbol{w}$).[2] Es necesario aprender un modelo para la distribución $p(\boldsymbol{x})$ de $\boldsymbol{x}$ y, posteriormente, la distribución condicional $p(\boldsymbol{x} \mid \boldsymbol{y})$. El completado de imágenes naturales o de lenguaje natural (por ejemplo, BERT y GPT) son ejemplos típicos de esta clase de problemas.

- *Caso 3:* Solo disponemos de algunas muestras emparejadas: $(\boldsymbol{X}, \boldsymbol{Y}) = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{y}_N)\}$ de las dos variables $(\boldsymbol{x}, \boldsymbol{y})$. Las distribuciones de $\boldsymbol{x}$ y $\boldsymbol{y}$ y su relación $h(\cdot)$ deben aprenderse a partir de estos datos de muestras emparejadas. Por ejemplo, dadas muchas imágenes y sus pies de

[2] En la literatura, este escenario se denomina a veces *inferencia bayesiana empírica*.

imagen, aprender a realizar la generación de imágenes condicionadas por texto es uno de esos problemas.

- *Caso 4:* Solo disponemos de las muestras $\boldsymbol{Y} = \{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N\}$ de las observaciones $\boldsymbol{y}$ y es necesario conocer el modelo de observación $h(\cdot)$, al menos dentro de alguna familia paramétrica $h(\cdot, \boldsymbol{\theta})$. Las distribuciones $p(\boldsymbol{x})$ y $p(\boldsymbol{x} \mid \boldsymbol{y})$ deben aprenderse a partir de $\hat{\boldsymbol{x}}$, estimado a partir de $\boldsymbol{Y}$. Por ejemplo, aprender a generar una nueva vista a partir de una secuencia de vistas calibradas o sin calibrar es uno de esos problemas.

En este capítulo, abordaremos enfoques generales para aprender las distribuciones deseadas y resolver la estimación condicional o la generación asociadas a estos casos, normalmente con un problema práctico representativo. A lo largo del capítulo, el lector debe tener en cuenta la Figura 7.1.

7.2 Inferencia condicional con una distribución de datos conocida

Obsérvese que, en el contexto que hemos analizado en capítulos anteriores, la red de autocodificación se entrena para reconstruir un conjunto de muestras del vector aleatorio $\boldsymbol{x}$. Esto nos permitiría regenerar muestras a partir de la distribución aprendida (de baja dimensión). En la práctica, la baja dimensión de la distribución, una vez dada o aprendida, puede explotarse para tareas de recuperación, completado o predicción estables y robustas. Es decir, bajo condiciones bastante suaves, es posible recuperar $\boldsymbol{x}$ a partir de mediciones de $\boldsymbol{x}$ altamente comprimidas, parciales, ruidosas o incluso corrompidas, del tipo:

$$\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}, \tag{7.2.1}$$

donde $\boldsymbol{y}$ suele ser una observación de $\boldsymbol{x}$ que tiene una dimensión mucho menor que $\boldsymbol{x}$, y $\boldsymbol{w}$ puede ser ruido aleatorio o incluso corrupciones graves y esparsas. Se trata de una clase de problemas que se ha estudiado ampliamente en la literatura clásica sobre procesamiento de señales, para estructuras de baja dimensión tales como vectores esparsos, matrices de bajo rango y otras. Los lectores interesados pueden consultar [WM22] para obtener una exposición completa de este tema.

Con el fin de situar este trabajo clásico en un contexto moderno más general, ilustramos la idea básica y los hechos mediante la que probablemente sea la tarea más sencilla de completar datos (y, en particular, imágenes). Es decir, consideramos el problema de recuperar una muestra $\boldsymbol{x}$ cuando faltan partes de ella (o incluso estén dañadas). Queremos recuperar o predecir el resto de $\boldsymbol{x}$ a partir de la observación de solo una fracción de la misma:

$$f : \mathcal{P}_\Omega(\boldsymbol{x}) \mapsto \hat{\boldsymbol{x}}, \tag{7.2.2}$$

donde $\mathcal{P}_\Omega(\,\cdot\,)$ representa una operación de enmascaramiento (véase la Figura 7.5 como ejemplo).

En esta sección y en la siguiente, estudiaremos la tarea de completado en dos escenarios diferentes: uno es aquel en el que la distribución de los datos $\boldsymbol{x}$ de interés ya se conoce *a priori*, incluso en forma analítica. Este es el caso que prevalece en el procesamiento clásico de señales, donde se supone que las estructuras de las señales son conocidas; por ejemplo, de banda limitada, esparsas o de bajo rango. El otro caso es cuando solo se dispone de muestras sin procesar de $\boldsymbol{x}$ y necesitamos aprender la distribución de baja dimensión a partir de las muestras para resolver adecuadamente la tarea de completado. Este es el caso de las tareas de completado de imágenes naturales o predicción de fotogramas de video. Como preludio al resto del capítulo, comenzaremos con el caso más sencillo de completado de imágenes: cuando la imagen que hay que completar se puede modelar adecuadamente como una matriz de rango bajo. Más adelante pasaremos a casos cada vez más generales y a entornos más exigentes.

Completado de matrices de rango bajo. El problema del *completado de matrices de rango bajo* es un problema clásico en el ámbito del completado de datos cuando su distribución es de baja dimensión y conocida. Consideremos una muestra aleatoria de una matriz $\boldsymbol{X}_o = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n] \in \mathbb{R}^{m \times n}$ del espacio de todas las matrices de rango r. En general, suponemos que el rango de la matriz es

$$\operatorname{rank}(\boldsymbol{X}_o) = r < \min\{m, n\}. \tag{7.2.3}$$

Por lo tanto, queda claro que, localmente, la dimensión intrínseca del espacio de todas las matrices de rango r es significativamente menor que la del espacio ambiente mn.

Ahora, sea Ω el conjunto de índices de las entradas observadas de la matriz $\boldsymbol{X}_o$. Sean las entradas observadas:

$$\boldsymbol{Y} = \mathcal{P}_\Omega(\boldsymbol{X}_o). \tag{7.2.4}$$

Las entradas restantes con soporte en Ω^c son no observadas o faltantes. El problema consiste en determinar si podemos recuperar de $\boldsymbol{Y}$ las entradas faltantes de $\boldsymbol{X}$ de manera correcta y eficiente. La Figura 7.5 muestra un ejemplo de cómo completar una matriz de este tipo.

Obsérvese que la razón fundamental por la que una matriz de este tipo puede completarse es que las columnas y filas de la matriz están altamente correlacionadas y todas ellas se encuentran en un subespacio de baja dimensión. Para el ejemplo mostrado en la Figura 7.5, la dimensión o el rango de la matriz completada es de solo dos. De ahí que la idea fundamental para recuperar una matriz de este tipo es buscar una matriz que tenga el rango más bajo entre todas las matrices que tengan entradas que coincidan con las observadas:

$$\min_{\boldsymbol{X}} \operatorname{rank}(\boldsymbol{X}) \quad \text{sujeto a} \quad \boldsymbol{Y} = \mathcal{P}_\Omega(\boldsymbol{X}). \tag{7.2.5}$$

Esto se conoce como el problema del *completado de matrices de rango bajo*. Véase [WM22] para una caracterización completa del espacio de todas las

Figura 7.5: Ilustración del completado de una imagen como matriz de rango bajo con algunas entradas enmascaradas o dañadas. Izquierda: la imagen enmascarada/dañada $\boldsymbol{Y}$; centro: la máscara Ω; derecha: la imagen reconstruida $\hat{\boldsymbol{X}}$.

matrices de rango bajo. Dado que la función de rango es discontinua y la minimización del rango es, en general, un problema NP-difícil, nos gustaría encontrar una relajación algo más fácil de optimizar.

Basándonos en nuestros conocimientos sobre compresión recogidos en el Capítulo 3, podríamos promover el bajo rango de la matriz recuperada $\boldsymbol{X}$ imponiendo que la tasa de codificación con pérdida (o el volumen generado por $\boldsymbol{X}$) de los datos en $\boldsymbol{X}$ sea pequeña:

$$\min R_\epsilon(\boldsymbol{X}) = \frac{1}{2}\log\det\left(\boldsymbol{I} + \alpha \boldsymbol{X}\boldsymbol{X}^\top\right) \quad \text{sujeto a} \quad \boldsymbol{Y} = \mathcal{P}_\Omega(\boldsymbol{X}). \tag{7.2.6}$$

El problema puede considerarse como una relajación continua del problema de completado de matrices de rango bajo mencionado anteriormente (7.2.5) y puede resolverse mediante el método del descenso por gradiente. Se puede demostrar que el operador de descenso por gradiente para el objetivo $\log\det$ minimiza precisamente un sustituto cercano al rango de la matriz $\boldsymbol{X}\boldsymbol{X}^\top$.

La función de tasa-distorsión es una función no convexa, y su descenso por gradiente no siempre garantiza encontrar la solución óptima global. No obstante, dado que la estructura subyacente que se busca para $\boldsymbol{X}$ es lineal por tramos, la función de rango admite una relajación convexa bastante eficaz: la norma nuclear - la suma de todos los valores singulares de la matriz $\boldsymbol{X}$. Como se muestra en la bibliografía sobre *compressive sensing*, bajo condiciones bastante amplias,[3] el problema del completado de matrices (7.2.5) puede resolverse eficazmente mediante el siguiente programa convexo:

$$\min \|\boldsymbol{X}\|_* \quad \text{sujeto a} \quad \boldsymbol{Y} = \mathcal{P}_\Omega(\boldsymbol{X}), \tag{7.2.7}$$

donde la norma nuclear $\|\boldsymbol{X}\|_*$ es la suma de los valores singulares de $\boldsymbol{X}$. En la práctica, a menudo convertimos el anterior programa de optimización convexa

[3] Por lo general, dichas condiciones especifican el número necesario y suficiente de entradas requeridas para que el completado sea viable desde el punto de vista computacional. Estas condiciones han sido caracterizadas sistemáticamente en [WM22].

con restricciones en uno sin restricciones:

$$\min \|\boldsymbol{X}\|_* + \lambda \|\boldsymbol{Y} - \mathcal{P}_\Omega(\boldsymbol{X})\|_F^2, \tag{7.2.8}$$

para algún $\lambda > 0$ adecuadamente elegido. Los lectores interesados pueden consultar [WM22] para conocer cómo desarrollar algoritmos capaces de resolver los programas anteriores de manera eficiente y eficaz. La Figura 7.5 muestra un ejemplo real en el que la matriz $\hat{\boldsymbol{X}}$ se recupera efectivamente resolviendo el programa anterior.

Otras ampliaciones. Se ha demostrado que las imágenes (o, más exactamente, las texturas) y las escenas 3D con estructuras de rango bajo pueden completarse de manera muy eficaz mediante la resolución de programas de optimización del tipo mencionado anteriormente, incluso si hubiera corrupción y distorsión adicionales [LRZ+12; YZB+23; ZLG+10]:

$$\boldsymbol{Y} \circ \tau = \boldsymbol{X}_o + \boldsymbol{E}, \tag{7.2.9}$$

donde τ es una distorsión no lineal desconocida de la imagen y $\boldsymbol{E}$ es una matriz desconocida que modela cierta oclusión (esparsa) y corrupción. Una vez más, los lectores interesados pueden consultar [WM22] para obtener una explicación más detallada.

7.3 Inferencia condicional con una representación de datos aprendida

En el apartado anterior, podemos inferir $\boldsymbol{x}$ a partir de la observación parcial $\boldsymbol{y}$ porque (el soporte de) la distribución de $\boldsymbol{X}$ se conoce o se especifica *a priori*, por ejemplo, como el conjunto de todas las matrices de rango bajo. En el caso de muchos conjuntos de datos prácticos, no disponemos de su distribución en una forma analítica como la de las matrices de rango bajo, por ejemplo, el conjunto de todas las imágenes naturales. No obstante, si contamos con suficientes muestras de los datos $\boldsymbol{x}$, deberíamos poder aprender primero su distribución en baja dimensión y aprovecharla para futuras tareas de inferencia basadas en una observación $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$. En esta sección, suponemos que el modelo de observación $h(\cdot)$ está dado y es conocido. En la siguiente sección estudiaremos el caso en el que $h(\cdot)$ no se da explícitamente.

7.3.1 Completado de imágenes mediante autocodificación enmascarada

En el caso de una imagen general $\boldsymbol{X}$ como la que se muestra a la izquierda de la Figura 7.6, ya no podemos considerarla como una matriz de bajo rango. Sin embargo, los seres humanos siguen demostrando una notable capacidad para completar escenas y reconocer objetos familiares a pesar de oclusiones severas.

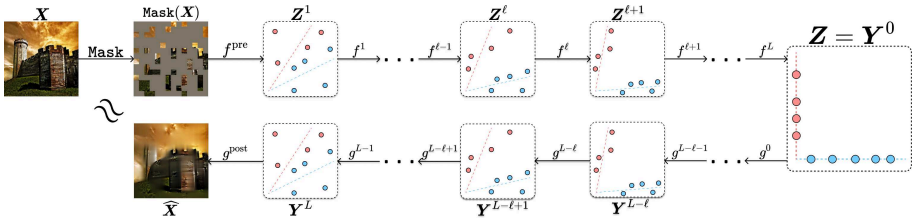


Figura 7.6: **Diagrama del proceso general (enmascarado) de autocodificación.** Las representaciones de los *tokens* (de imagen) se transforman de forma iterativa hacia una representación parsimoniosa (p. ej., comprimida y esparsa) mediante cada capa del codificador f^ℓ. Además, dichas representaciones se transforman de nuevo en la imagen original mediante las capas decodificadoras g^ℓ. Cada capa codificadora f^ℓ está diseñada para que sea (parcialmente) invertida por una capa decodificadora correspondiente $g^{L-\ell}$.

Esto sugiere que nuestro cerebro ha aprendido la distribución de baja dimensión de las imágenes naturales y puede utilizarla para el completado y, por lo tanto, para el reconocimiento. Sin embargo, la distribución de todas las imágenes naturales no es tan simple como un subespacio lineal de baja dimensión. Por lo tanto, surge la pregunta de si podemos aprender la distribución más sofisticada de las imágenes naturales y utilizarla para realizar el completado de imágenes.

Un enfoque empírico para la tarea de completar imágenes consiste en encontrar un esquema de codificación y decodificación mediante la resolución del siguiente programa de *autocodificación enmascarada* (MAE) que minimice la pérdida de reconstrucción:

$$\min_{f,g} L_{\text{MAE}}(f,g) \doteq \mathbb{E}\big[\|(g \circ f)(\mathcal{P}_\Omega(\boldsymbol{X})) - \boldsymbol{X}\|_2^2\big]. \tag{7.3.1}$$

A diferencia del problema de completado de matrices, que tiene una estructura subyacente simple, ya no debemos esperar que las aplicaciones de codificación y decodificación admitan formas cerradas simples ni que el programa pueda implementarse mediante algoritmos explícitos.

En el caso de una imagen natural general, ya no podemos suponer que sus columnas o filas se muestreen a partir de un subespacio de baja dimensión o de una distribución gaussiana de bajo rango. Sin embargo, es razonable suponer que la imagen se compone de múltiples regiones. Los parches de imagen en cada región son similares y pueden modelarse como una gaussiana o un subespacio (ambos de bajo rango). Por lo tanto, para aprovechar la baja dimensionalidad de la distribución, el objetivo *del codificador* f es transformar $\boldsymbol{X}$ en una representación $\boldsymbol{Z}$:

$$f : \boldsymbol{X} \mapsto \boldsymbol{Z} \tag{7.3.2}$$

de tal manera que la distribución de $\boldsymbol{Z}$ pueda modelarse adecuadamente como una mezcla de subespacios, por ejemplo $\{\boldsymbol{U}_{[K]}\}$, tal que se maximice la reducción

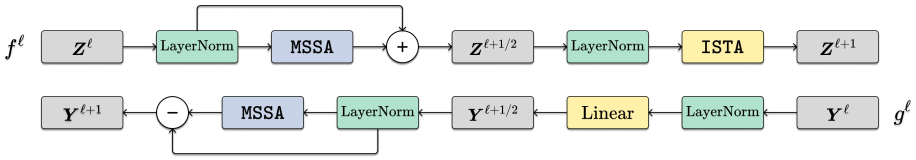


Figura 7.7: **Diagrama de cada capa del codificador (*arriba*) y del decodificador (*abajo*).** Obsérvese que las dos capas son altamente antiparalelas: cada una está diseñada para realizar las operaciones de la otra en orden inverso. Es decir, en la capa decodificadora g^ℓ, el bloque ISTA de $f^{L-\ell}$ se invierte parcialmente primero mediante una capa lineal y luego se deshace el bloque MSSA de $f^{L-\ell}$; este orden desentraña la transformación realizada en $f^{L-\ell}$.

Figura 7.8: **Visualizaciones de autocodificación de CRATE-Base y ViT-MAE-Base [HCX+22] con el 75% de los parches enmascarados.** Observamos que las reconstrucciones de CRATE-Base están a la par con las de ViT-MAE-Base, a pesar de utilizar $< 1/3$ de los parámetros.

de la tasa y se minimice la esparcidad:

$$\mathbb{E}_{\boldsymbol{Z}=f(\boldsymbol{X})}[\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) - \lambda\|\boldsymbol{Z}\|_0] = \mathbb{E}_{\boldsymbol{Z}=f(\boldsymbol{X})}[R_\epsilon(\boldsymbol{Z}) - R^c_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) - \lambda\|\boldsymbol{Z}\|_0], \tag{7.3.3}$$

donde las funciones $R_\epsilon(\cdot)$ y $R^c_\epsilon(\cdot)$ se definen en (5.2.2) y (5.2.3), respectivamente.

Como hemos mostrado en el Capítulo 5 anterior, el codificador f que minimiza el objetivo anterior puede construirse como una secuencia de operadores tipo Transformer. Tal y como se muestra en el trabajo de [PBW+24], el decodificador g puede entenderse y, por tanto, construirse explícitamente como el proceso inverso del codificador f. La Figura 7.7 ilustra las arquitecturas generales tanto del codificador como del decodificador correspondiente en cada capa. Se pueden aprender los parámetros del codificador f y del decodificador g optimizando la pérdida de reconstrucción (7.3.1) mediante el método de descenso de gradiente.

La Figura 7.8 muestra algunos resultados representativos del autocodificador enmascarado diseñado de este modo. Para obtener más detalles sobre la implementación y los resultados del autocodificador enmascarado para el completado de imágenes naturales se recomienda consultar el Capítulo 8, Sección 8.5.

7.3.2 Muestreo condicional con ajuste de mediciones

El problema de autocodificación (enmascarada) mencionado anteriormente tiene como objetivo generar una imagen de muestra que sea coherente con ciertas observaciones o condiciones. Pero analicemos este enfoque más detenidamente: dada la parte visual de una imagen $\boldsymbol{X}_v = \mathcal{P}_\Omega(\boldsymbol{X})$, intentamos estimar la parte enmascarada $\boldsymbol{X}_m = \mathcal{P}_{\Omega^c}(\boldsymbol{X})$. Para las realizaciones $(\boldsymbol{\Xi}_v, \boldsymbol{\Xi}_m)$ de la variable aleatoria $\boldsymbol{X} = (\boldsymbol{X}_v, \boldsymbol{X}_m)$, sea

$$p_{\boldsymbol{X}_m|\boldsymbol{X}_v}(\boldsymbol{\Xi}_m \mid \boldsymbol{\Xi}_v)$$

la distribución condicional de $\boldsymbol{X}_m$ dada $\boldsymbol{X}_v$. Es fácil demostrar que la solución óptima a la formulación MAE (7.3.1) viene dada por la esperanza condicional:

$$\arg\min_{h=g\circ f} L_{\text{MAE}}(h) = \boldsymbol{\Xi}_v \mapsto \boldsymbol{\Xi}_v + \mathbb{E}[\boldsymbol{X}_m \mid \boldsymbol{X}_v = \boldsymbol{\Xi}_v]. \tag{7.3.4}$$

Sin embargo, en general, es posible que esta expectativa ni siquiera esté situada en la distribución de baja dimensión de las imágenes naturales. Esto explica en parte por qué algunos de los fragmentos recuperados en la Figura 7.8 se ven un poco borrosos.

A efectos prácticos, nos gustaría estimar (una representación de) la distribución condicional $p_{\boldsymbol{X}_m|\boldsymbol{X}_v}$, o lo que es lo mismo, $p_{\boldsymbol{X}|\boldsymbol{X}_v}$, y luego obtener directamente una muestra nítida (la más probable) de esta distribución. Obsérvese que, cuando la distribución de $\boldsymbol{X}$ es de baja dimensión, es posible que si se observa una parte suficiente de $\boldsymbol{X}$, $\boldsymbol{X}_v$, esta determine completamente $\boldsymbol{X}$ y, por lo tanto, la parte faltante $\boldsymbol{X}_m$. En otras palabras, la distribución $p_{\boldsymbol{X}|\boldsymbol{X}_v}$ es una función generalizada (análoga a la función delta).

Por lo tanto, en lugar de abordar la tarea de completado como un problema de estimación condicional, deberíamos tratarla como un problema de muestreo condicional. Para ello, primero deberíamos aprender la distribución (de baja dimensión) de todas las imágenes naturales $\boldsymbol{X}$. Si contamos con suficientes muestras de imágenes naturales, podemos aprender la distribución mediante un proceso de eliminación de ruido $\boldsymbol{X}_t$ descrito en el Capítulo 3. Entonces, el problema de recuperar $\boldsymbol{X}$ a partir de su observación parcial $\boldsymbol{Y} = \mathcal{P}_\Omega(\boldsymbol{x}) + \boldsymbol{w}$ se convierte en un problema de generación condicional: obtener muestras de la distribución condicionadas a la observación.

Mediciones lineales generales. De hecho, incluso podríamos plantearnos estimar $\boldsymbol{X}$ a partir de un modelo de observación lineal más general:

$$\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{X}_0, \quad \boldsymbol{X}_t = \boldsymbol{X}_0 + \sigma_t \boldsymbol{G}, \tag{7.3.5}$$

donde $\boldsymbol{A}$ es un operador lineal en el espacio matricial[4] y $\boldsymbol{G} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. El operador de enmascaramiento $\mathcal{P}_\Omega(\cdot)$ en la tarea de completado de imágenes es

[4] es decir, si imaginamos que desenrollamos $\boldsymbol{X}$ en un vector largo, entonces $\boldsymbol{A}$ desempeña el papel de una matriz en el espacio de $\boldsymbol{X}$

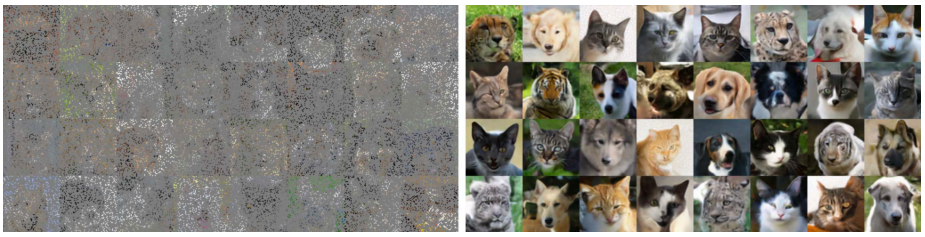


Figura 7.9: **Visualizaciones de muestreo a partir de modelos entrenados mediante «ambient diffusion» [DSD+23b] con el 80% de los píxeles enmascarados.** Al utilizar una proporción de píxeles enmascarados similar a la de la Figura 7.8, el algoritmo de muestreo por «ambient diffusion» recupera una imagen mucho más nítida que la imagen borrosa recuperada por el método basado en MAE. El primer método toma muestras de la distribución de las imágenes naturales, mientras que el segundo aproxima la esperanza condicional (es decir, la media) de esta distribución dada la observación; esta operación de promediar es lo que provoca el efecto borroso.

un ejemplo de dicho modelo lineal. Entonces, [DSD+23a] han demostrado que

$$\hat{\boldsymbol{X}}_* = \arg\min_{\hat{\boldsymbol{X}}} \mathbb{E}[\|\boldsymbol{A}(\hat{\boldsymbol{X}}(\boldsymbol{A}\boldsymbol{X}_t, \boldsymbol{A}) - \boldsymbol{X}_0)\|^2] \tag{7.3.6}$$

cumple la condición de que:

$$\boldsymbol{A}\hat{\boldsymbol{X}}_*(\boldsymbol{A}(\boldsymbol{X}_t), \boldsymbol{A}) = \boldsymbol{A}\mathbb{E}[\boldsymbol{X}_0 \mid \boldsymbol{A}\boldsymbol{X}_t, \boldsymbol{A}]. \tag{7.3.7}$$

Obsérvese que, en el caso especial en el que $\boldsymbol{A}$ tiene rango de columna completo, se tiene $\mathbb{E}[\boldsymbol{X}_0 \mid \boldsymbol{A}\boldsymbol{X}_t, \boldsymbol{A}] = \mathbb{E}[\boldsymbol{X}_0 \mid \boldsymbol{X}_t]$. Por lo tanto, en el caso más general, se ha sugerido en [DSD+23a] que aún se podría utilizar el $\mathbb{E}[\boldsymbol{X}_0 \mid \boldsymbol{A}(\boldsymbol{X}_t), \boldsymbol{A}]$ así obtenido para sustituir el $\mathbb{E}[\boldsymbol{X}_0 \mid \boldsymbol{X}_t]$ en el proceso estándar de eliminación de ruido para $\boldsymbol{X}_t$:

$$\boldsymbol{X}_{t-s} = \gamma_t \boldsymbol{X}_t + (1-\gamma_t)\mathbb{E}[\boldsymbol{X}_0 \mid \boldsymbol{A}\boldsymbol{X}_t, \boldsymbol{A}]. \tag{7.3.8}$$

Esto suele funcionar muy bien en la práctica, por ejemplo, en muchas tareas de restauración de imágenes, tal y como se muestra en [DSD+23a]. En comparación con las imágenes borrosas recuperadas mediante MAE, las imágenes recuperadas por el método anterior son mucho más nítidas, ya que hace uso de una distribución aprendida de imágenes naturales y toma una muestra de una imagen (nítida) de dicha distribución que es coherente con la medición, tal y como se muestra en la Figura 7.9 (véase la Figura 7.8).

Mediciones no lineales generales. Para generalizar los problemas de completado (de imágenes) mencionados anteriormente y hacer el análisis más riguroso, podríamos considerar que un vector aleatorio $\boldsymbol{x} \sim p$ se observa parcialmente a través de una función de observación más general:

$$\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}, \tag{7.3.9}$$

donde $\boldsymbol{w}$ suele representar algún ruido de medición aleatorio, por ejemplo, de una distribución gaussiana $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I})$. Es fácil ver que, para $\boldsymbol{x}$ e $\boldsymbol{y}$ así relacionadas, su distribución conjunta $p(\boldsymbol{x}, \boldsymbol{y})$ es, naturalmente, casi degenerada si el ruido $\boldsymbol{w}$ es pequeño. En gran medida, podemos considerar $p(\boldsymbol{x}, \boldsymbol{y})$ como una versión ruidosa de una hipersuperficie definida por la función $\boldsymbol{y} = h(\boldsymbol{x})$ en el espacio combinado $(\boldsymbol{x}, \boldsymbol{y})$. En la práctica, consideraremos un escenario más parecido a la autocodificación enmascarada que al completado puro de matrices, en el que siempre tenemos acceso a una muestra limpia correspondiente $\boldsymbol{x}$ para cada observación $\boldsymbol{y}$ que recibimos.[5]

Al igual que en el completado de imágenes o matrices, a menudo nos enfrentamos a un escenario en el que $\boldsymbol{y}$ representa una observación degradada o, en cualquier caso, «con pérdida» de la entrada $\boldsymbol{x}$. Esto puede manifestarse de formas muy diferentes. Por ejemplo, en diversos problemas de imaginería científica o médica, los datos medidos $\boldsymbol{y}$ pueden ser una observación comprimida y corrompida de los datos subyacentes $\boldsymbol{x}$; mientras que en tareas de visión 3D, $\boldsymbol{y}$ puede representar una imagen capturada por una cámara de un objeto físico con una postura desconocida (de baja dimensión) $\boldsymbol{x}$. En general, gracias al modelado matemático (y, en algunos casos, al co-diseño del sistema de medición), conocemos h y podemos evaluarla sobre cualquier entrada, y podemos aprovechar este conocimiento para ayudar a reconstruir y muestrear $\boldsymbol{x}$.

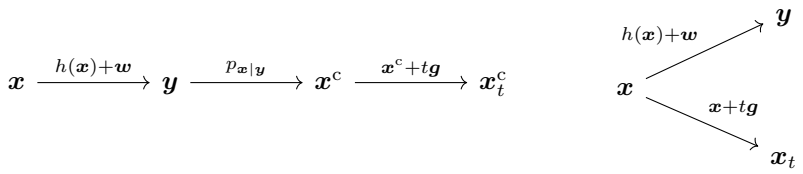
Desde el punto de vista técnico, queremos que la representación aprendida de los datos nos permita obtener muestras de la distribución condicional $p_{\boldsymbol{x}|\boldsymbol{y}}$, también conocida como la distribución a posteriori, de manera eficaz y eficiente. Más exactamente, utilicemos $\boldsymbol{\nu}$ para denotar una realización de la variable aleatoria $\boldsymbol{y}$. Queremos generar muestras $\hat{\boldsymbol{x}}$ tales que:

$$\hat{\boldsymbol{x}} \sim p_{\boldsymbol{x}|\boldsymbol{y}}(\,\cdot \mid \boldsymbol{y} = \boldsymbol{\nu}). \tag{7.3.10}$$

Recordemos que en Sección 3.2 hemos desarrollado una forma natural y eficaz de generar muestras *incondicionales* de la distribución de datos p. Las componentes son los textitdenoisers $\bar{\boldsymbol{x}}^*(t, \boldsymbol{\xi}) = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$, o sus aproximaciones aprendidas $\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi})$, para diferentes niveles de observaciones ruidosas $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g}$ (y $\boldsymbol{\xi}$ para sus realizaciones) bajo ruido gaussiano $\boldsymbol{g} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, y $t \in [0, T]$ con una elección de tiempos $0 = t_1 < \cdots < t_L = T$ en los que realizar la eliminación de ruido iterativa, comenzando desde $\hat{\boldsymbol{x}}_{t_L} \sim \mathcal{N}(\mathbf{0}, T^2 \boldsymbol{I})$ (recordemos Ecuación (3.2.82)).[6] Podríamos aplicar directamente este esquema para generar muestras a partir de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$ *si* tuviéramos acceso a un conjunto de datos de muestras $\boldsymbol{x}^{\mathrm{c}} \sim p_{\boldsymbol{x}|\boldsymbol{y}}(\,\cdot \mid \boldsymbol{\nu})$ para

[5] En algunas aplicaciones más especializadas, en particular en el procesamiento de imágenes científicas, resulta interesante poder aprender a generar muestras a partir de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$ sin tener acceso a ninguna muestra limpia o de referencia de $\boldsymbol{x}$. Ofrecemos una breve descripción general de los métodos para este contexto en las notas al final del capítulo.

[6] Recordemos que, como vimos en nuestra discusión en Sección 3.2.2, unas pocas mejoras menores en este esquema básico de eliminación de ruido iterativo son suficientes para lograr un rendimiento práctico competitivo. Para mayor claridad a medida que desarrollamos el muestreo condicional, nos centraremos aquí en la implementación más simple.



(a) (b)

Figura 7.10: Diagramas de dependencia estadística para el proceso de muestreo condicional. **Izquierda:** En una aplicación directa (conceptual) del esquema de eliminación de ruido por difusión que hemos desarrollado en el Capítulo 3 al muestreo condicional, utilizamos muestras de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$ para entrenar los eliminadores de ruido directamente sobre la distribución a posteriori a diferentes niveles de ruido y luego las usamos para generar nuevas muestras. Sin embargo, en la práctica no solemos disponer de muestras directas de la distribución a posteriori, sino más bien de muestras emparejadas $(\boldsymbol{x}, \boldsymbol{y})$ de la distribución multivariada. **Derecha:** Resulta que basta con tener solo observaciones ruidosas de $\boldsymbol{x}$ para realizar los *denoiser* correspondientes a $p_{\boldsymbol{x}_t^{\mathrm{c}}|\boldsymbol{x}^{\mathrm{c}}}$: esto se deduce de la independencia condicional de $\boldsymbol{x}_t$ y $\boldsymbol{y}$ dado $\boldsymbol{x}$. Implica que $p_{\boldsymbol{x}_t^{\mathrm{c}}|\boldsymbol{y}} = p_{\boldsymbol{x}_t|\boldsymbol{y}}$, lo que da lugar a una función de *score* para la eliminación de ruido que consiste en la función de *score* incondicional, más un término de corrección que garantiza la consistencia de la medición.

cada realización $\boldsymbol{\nu}$ de $\boldsymbol{y}$, generando observaciones con ruido $\boldsymbol{x}_t^{\mathrm{c}}$ y entrenando *denoisers* para aproximar $\mathbb{E}[\boldsymbol{x}^{\mathrm{c}} \mid \boldsymbol{x}_t^{\mathrm{c}} = \cdot, \boldsymbol{y} = \boldsymbol{\nu}]$, la media de la distribución a posteriori bajo la observación ruidosa (véase Figura 7.10(a)). Sin embargo, realizar este remuestreo cuando solo se dispone de muestras emparejadas $(\boldsymbol{x}, \boldsymbol{y})$ de la distribución conjunta (por ejemplo, agrupando las muestras en intervalos de valores de $\boldsymbol{y}$) requiere un número prohibitivo de muestras para datos de alta dimensión y los enfoques alternativos se basan, explícita o implícitamente, en la estimación de densidad, que a su vez sufre de la maldición de la dimensionalidad.

Afortunadamente, resulta que esto no es necesario. Consideremos el diagrama de dependencia estadística alternativo de la Figura 7.10(b), que corresponde a las variables aleatorias del proceso habitual de difusión con eliminación de ruido, junto con la medición $\boldsymbol{y}$. Dado que nuestro modelo de observación supuesto (7.3.9) implica que $\boldsymbol{x}_t$ y $\boldsymbol{y}$ son independientes condicionales

a $\boldsymbol{x}$, tenemos que para cualquier realización $\boldsymbol{\nu}$ de $\boldsymbol{y}$

$$\begin{aligned} p_{\boldsymbol{x}_t^{\mathrm{c}}|\boldsymbol{y}}(\,\cdot\mid\boldsymbol{\nu}) &= \int \underbrace{p_{\boldsymbol{x}_t^{\mathrm{c}}|\boldsymbol{x}^{\mathrm{c}}}(\,\cdot\mid\boldsymbol{\xi})}_{=\mathcal{N}(\boldsymbol{\xi},t^2\boldsymbol{I})} \cdot \underbrace{p_{\boldsymbol{x}^{\mathrm{c}}|\boldsymbol{y}}}_{=p_{\boldsymbol{x}|\boldsymbol{y}}}(\boldsymbol{\xi}\mid\boldsymbol{\nu})\mathrm{d}\boldsymbol{\xi} \\ &= \int p_{\boldsymbol{x}_t|\boldsymbol{x},\boldsymbol{y}}(\,\cdot\mid\boldsymbol{\xi},\boldsymbol{\nu}) \cdot p_{\boldsymbol{x}|\boldsymbol{y}}(\boldsymbol{\xi}\mid\boldsymbol{\nu})\mathrm{d}\boldsymbol{\xi} \\ &= \int p_{\boldsymbol{x}_t,\boldsymbol{x}|\boldsymbol{y}}(\,\cdot\,,\boldsymbol{\xi}\mid\boldsymbol{\nu})\mathrm{d}\boldsymbol{\xi} \\ &= p_{\boldsymbol{x}_t|\boldsymbol{y}}(\,\cdot\mid\boldsymbol{\nu}). \end{aligned} \tag{7.3.11}$$

En la ecuación anterior, se reconoce una equivalencia entre las distribuciones que aparecen en Figura 7.10 (a,b); en la segunda línea se aplica esto junto con la independencia condicional de $\boldsymbol{x}_t$ y $\boldsymbol{y}$ dado $\boldsymbol{x}$; en la tercera línea se utiliza la definición de probabilidad condicional; y en la última línea se marginaliza respecto a $\boldsymbol{x}$. Por lo tanto, los *denoisers* del proceso de muestreo posterior conceptual son iguales a $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \cdot, \boldsymbol{y} = \boldsymbol{\nu}]$, lo cual podemos aprender únicamente a partir de muestras emparejadas $(\boldsymbol{x}, \boldsymbol{y})$, y mediante la fórmula de Tweedie (Teorema 3.2), podemos expresar estos *denoisers* en términos de la función de *score* de $p_{\boldsymbol{x}_t|\boldsymbol{y}}$, la cual, según la regla de Bayes, satisface

$$p_{\boldsymbol{x}_t|\boldsymbol{y}}(\boldsymbol{\xi}\mid\boldsymbol{\nu}) = \frac{p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu}\mid\boldsymbol{\xi})p_{\boldsymbol{x}_t}(\boldsymbol{\xi})}{p_{\boldsymbol{y}}(\boldsymbol{\nu})}. \tag{7.3.12}$$

Recordemos que la densidad de $\boldsymbol{x}_t$ viene dada por $p_t = \varphi_t * p$, donde φ_t denota la densidad gaussiana estándar con media cero y covarianza $t^2\boldsymbol{I}$ y $*$ denota la convolución. ¡Esto no es más que la *función de score incondicional* obtenida del entrenamiento de difusión estándar que desarrollamos en la Sección 3.2! La función de *score* condicional satisface entonces, para cualquier realización $(\boldsymbol{\xi}, \boldsymbol{\nu})$ de $(\boldsymbol{x}_t, \boldsymbol{y})$,

$$\nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{x}_t|\boldsymbol{y}}(\boldsymbol{\xi}\mid\boldsymbol{\nu}) = \underbrace{\nabla_{\boldsymbol{\xi}} \log p_t(\boldsymbol{\xi})}_{\text{score matching}} + \underbrace{\nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu}\mid\boldsymbol{\xi})}_{\text{measurement matching}}, \tag{7.3.13}$$

lo cual, según la fórmula de Tweedie, nos proporciona los *denoisers* propuestos como

$$\begin{aligned} \mathbb{E}[\boldsymbol{x}\mid\boldsymbol{x}_t=\boldsymbol{\xi},\boldsymbol{y}=\boldsymbol{\nu}] &= \boldsymbol{\xi} + t^2\nabla_{\boldsymbol{\xi}}\log p_t(\boldsymbol{\xi}) + t^2\nabla_{\boldsymbol{\xi}}\log p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu}\mid\boldsymbol{\xi}) \\ &= \mathbb{E}[\boldsymbol{x}\mid\boldsymbol{x}_t=\boldsymbol{\xi}] + t^2\nabla_{\boldsymbol{\xi}}\log p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu}\mid\boldsymbol{\xi}). \end{aligned} \tag{7.3.14}$$

Los operadores resultantes pueden interpretarse como una versión *corregida* del *denoiser* incondicional para la observación ruidosa, donde el término de corrección (el denominado término de «ajuste de la medición») impone la coherencia con las observaciones $\boldsymbol{y}$. Esta descomposición constituye la base de la *guía del clasificador*, que desarrollaremos en detalle en la Sección 7.4.1. El lector debe prestar atención a qué argumento se aplican los operadores de

gradiente en las funciones de *score* anteriores para poder comprender plenamente el significado de este operador.

La cuestión clave pendiente para hacer este procedimiento implementable computacionalmente es determinar cómo calcular la corrección de correspondencia de mediciones, ya que, en general, no disponemos de una expresión de forma cerrada para la verosimilitud $p_{\boldsymbol{y}|\boldsymbol{x}_t}$, salvo cuando $t = 0$. Antes de abordar este problema, analizaremos un ejemplo concreto ilustrativo de todo el proceso, siguiendo los ejemplos que hemos desarrollado en la Sección 3.2.

Ejemplo 7.4. Consideremos el caso en el que la distribución de los datos es gaussiana con media $\boldsymbol{\mu} \in \mathbb{R}^D$ y covarianza $\boldsymbol{\Sigma} \in \mathbb{R}^{D\times D}$, es decir, $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Supongamos que $\boldsymbol{\Sigma} \succeq \mathbf{0}$ es distinto de cero. Además, en el modelo de medición (7.3.9), supongamos que obtenemos mediciones lineales de $\boldsymbol{x}$ con ruido gaussiano independiente, donde $\boldsymbol{A} \in \mathbb{R}^{d\times D}$ y $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}+\sigma\boldsymbol{w}$ con $\boldsymbol{w} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ independiente de $\boldsymbol{x}$. A continuación, procederemos a deducir las siguientes consecuencias de este modelo para el marco general que hemos derivado anteriormente. Específicamente:

1. Determinaremos las formas exactas de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$ de la que pretendemos obtener muestras.

2. Determinaremos la forma del término de correspondencia de la medición de la Ecuación (7.3.13), lo que nos proporciona la corrección aditiva al *denoiser* incondicional para $p_{\boldsymbol{x}|\boldsymbol{x}_t}$ que produce el *denoiser* para $p_{\boldsymbol{x}|\boldsymbol{x}_t,\boldsymbol{y}}$ (véase (7.3.14)).

3. Evaluaremos la forma específica del término de correspondencia de mediciones que derivamos en el paso anterior y postularemos una aproximación natural que haga que el aprendizaje del término de correspondencia de mediciones a partir de los datos requiera mucho menos volumen de datos y recursos computacionales. Esta aproximación se aplicará de manera más general, una vez concluido este ejemplo.

En primer lugar, observamos que $\boldsymbol{x} \overset{\mathrm{d}}{=} \boldsymbol{\Sigma}^{1/2}\boldsymbol{g} + \boldsymbol{\mu}$, donde $\boldsymbol{g} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ es independiente de $\boldsymbol{w}$ y $\boldsymbol{\Sigma}^{1/2}$ es la única raíz cuadrada positiva de la matriz de covarianza $\boldsymbol{\Sigma}$ y, tras algunas manipulaciones algebraicas, podemos entonces escribir

$$\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{y}\end{bmatrix} \overset{\mathrm{d}}{=} \begin{bmatrix}\boldsymbol{\Sigma}^{1/2} & \mathbf{0}\\ \boldsymbol{A}\boldsymbol{\Sigma}^{1/2} & \sigma\boldsymbol{I}\end{bmatrix}\begin{bmatrix}\boldsymbol{g}\\ \boldsymbol{w}\end{bmatrix} + \begin{bmatrix}\boldsymbol{\mu}\\ \boldsymbol{A}\boldsymbol{\mu}\end{bmatrix}.$$

Por independencia, se deduce que $(\boldsymbol{g}, \boldsymbol{w})$ es gaussiana conjunta, lo que significa que $(\boldsymbol{x}, \boldsymbol{y})$ también es gaussiana conjunta, como imagen afín de un vector gaussiano conjunto. Su matriz de covarianza viene dada por

$$\begin{bmatrix}\boldsymbol{\Sigma}^{1/2} & \mathbf{0}\\ \boldsymbol{A}\boldsymbol{\Sigma}^{1/2} & \sigma\boldsymbol{I}\end{bmatrix}\begin{bmatrix}\boldsymbol{\Sigma}^{1/2} & \mathbf{0}\\ \boldsymbol{A}\boldsymbol{\Sigma}^{1/2} & \sigma\boldsymbol{I}\end{bmatrix}^\top = \begin{bmatrix}\boldsymbol{\Sigma} & \boldsymbol{\Sigma}\boldsymbol{A}^\top\\ \boldsymbol{A}\boldsymbol{\Sigma} & \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top + \sigma^2\boldsymbol{I}\end{bmatrix}.$$

Ahora, aplicamos el hecho de que condicionar un vector aleatorio con distribución gaussiana conjunta sobre un subconjunto de coordenadas da lugar,

de nuevo, a una distribución gaussiana (Ejercicio 3.2). De este modo, obtenemos que

$$p_{\boldsymbol{x}|\boldsymbol{y}}(\,\cdot\mid\boldsymbol{\nu}) = \mathcal{N}\Bigg(\underbrace{\boldsymbol{\mu}+\boldsymbol{\Sigma}\boldsymbol{A}^\top\left(\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top+\sigma^2\boldsymbol{I}\right)^{-1}(\boldsymbol{\nu}-\boldsymbol{A}\boldsymbol{\mu})}_{\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{y}}(\boldsymbol{\nu})}, \underbrace{\boldsymbol{\Sigma}-\boldsymbol{\Sigma}\boldsymbol{A}^\top\left(\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top+\sigma^2\boldsymbol{I}\right)^{-1}\boldsymbol{A}\boldsymbol{\Sigma}}_{\boldsymbol{\Sigma}_{\boldsymbol{x}|\boldsymbol{y}}}\Bigg). \tag{7.3.15}$$

A continuación, dado que $\boldsymbol{x}_t = \boldsymbol{x}+t\boldsymbol{g}'$, donde $\boldsymbol{g}'\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})$ es independiente de todos los demás vectores aleatorios y dado que $\boldsymbol{x}\mid\boldsymbol{y}$ es gaussiano, según el cálculo que precede, se deduce mediante otra aplicación del Ejercicio 3.2 que $\boldsymbol{x}\mid\boldsymbol{x}_t,\boldsymbol{y}$ es gaussiano. Su función de esperanza condicional viene dada por la media de la distribución gaussiana, que en este caso es

$$\mathbb{E}[\boldsymbol{x}\mid\boldsymbol{x}_t=\boldsymbol{\xi},\boldsymbol{y}=\boldsymbol{\nu}] = \boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{y}}(\boldsymbol{\nu})+\boldsymbol{\Sigma}_{\boldsymbol{x}|\boldsymbol{y}}\left(\boldsymbol{\Sigma}_{\boldsymbol{x}|\boldsymbol{y}}+t^2\boldsymbol{I}\right)^{-1}\left(\boldsymbol{\xi}-\boldsymbol{\mu}_{\boldsymbol{x}|\boldsymbol{y}}(\boldsymbol{\nu})\right). \tag{7.3.16}$$

La forma funcional de este *denoiser* es bastante sencilla, pero presenta una dependencia engorroda de los datos del problema $\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$, $\boldsymbol{A}$ y σ^2. Podemos comprender mejor su comportamiento comparándolo con la Ecuación (7.3.14). Tenemos, como de costumbre

$$\mathbb{E}[\boldsymbol{x}\mid\boldsymbol{x}_t=\boldsymbol{\xi}] = \boldsymbol{\mu}+\boldsymbol{\Sigma}\left(\boldsymbol{\Sigma}+t^2\boldsymbol{I}\right)^{-1}(\boldsymbol{\xi}-\boldsymbol{\mu}), \tag{7.3.17}$$

lo cual es bastante sencillo, lo que sugiere que el término de correspondencia de mediciones es bastante complicado. Para confirmarlo, podemos calcular la verosimilitud $p_{\boldsymbol{y}|\boldsymbol{x}_t}$ directamente utilizando la siguiente expresión para la distribución conjunta de $(\boldsymbol{x},\boldsymbol{x}_t,\boldsymbol{y})$:

$$\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{x}_t\\ \boldsymbol{y}\end{bmatrix} \overset{\mathrm{d}}{=} \begin{bmatrix}\boldsymbol{\Sigma}^{1/2} & \boldsymbol{0} & \boldsymbol{0}\\ \boldsymbol{\Sigma}^{1/2} & t\boldsymbol{I} & \boldsymbol{0}\\ \boldsymbol{A}\boldsymbol{\Sigma}^{1/2} & \boldsymbol{0} & \sigma\boldsymbol{I}\end{bmatrix}\begin{bmatrix}\boldsymbol{g}\\ \boldsymbol{g}'\\ \boldsymbol{w}\end{bmatrix} + \begin{bmatrix}\boldsymbol{\mu}\\ \boldsymbol{\mu}\\ \boldsymbol{A}\boldsymbol{\mu}\end{bmatrix}, \tag{7.3.18}$$

donde $\boldsymbol{g}'\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})$, independiente de las demás gaussianas. Se trata, una vez más, de una distribución gaussiana conjunta; si nos limitamos únicamente a las dos últimas filas, obtenemos la covarianza

$$\begin{bmatrix}\boldsymbol{\Sigma}^{1/2} & t\boldsymbol{I} & \boldsymbol{0}\\ \boldsymbol{A}\boldsymbol{\Sigma}^{1/2} & \boldsymbol{0} & \sigma\boldsymbol{I}\end{bmatrix}\begin{bmatrix}\boldsymbol{\Sigma}^{1/2} & t\boldsymbol{I} & \boldsymbol{0}\\ \boldsymbol{A}\boldsymbol{\Sigma}^{1/2} & \boldsymbol{0} & \sigma\boldsymbol{I}\end{bmatrix}^\top = \begin{bmatrix}\boldsymbol{\Sigma}+t^2\boldsymbol{I} & \boldsymbol{\Sigma}\boldsymbol{A}^\top\\ \boldsymbol{A}\boldsymbol{\Sigma} & \boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top+\sigma^2\boldsymbol{I}\end{bmatrix}.$$

Aplicando de nuevo el Ejercicio 3.2, obtenemos

$$p_{\boldsymbol{y}|\boldsymbol{x}_t}(\,\cdot\mid\boldsymbol{\xi}) = \mathcal{N}\Bigg(\underbrace{\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{A}\boldsymbol{\Sigma}\left(\boldsymbol{\Sigma}+t^2\boldsymbol{I}\right)^{-1}(\boldsymbol{\xi}-\boldsymbol{\mu})}_{\boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\xi})}, \underbrace{\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^\top+\sigma^2\boldsymbol{I}-\boldsymbol{A}\boldsymbol{\Sigma}\left(\boldsymbol{\Sigma}+t^2\boldsymbol{I}\right)^{-1}\boldsymbol{\Sigma}\boldsymbol{A}^\top}_{\boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}_t}}\Bigg). \tag{7.3.19}$$

Ahora notemos que $\boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\xi}) = \boldsymbol{A}\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$. Por lo tanto, por la regla de la cadena,

$$\begin{aligned} & t^2 \nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu} \mid \boldsymbol{\xi}) \\ = \;& t^2 \nabla_{\boldsymbol{\xi}} \left[-\frac{1}{2} (\boldsymbol{\nu} - \boldsymbol{A}\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}])^\top \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}_t}^{-1} (\boldsymbol{\nu} - \boldsymbol{A}\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]) \right] \\ = \;& t^2 (\boldsymbol{\Sigma} + t^2 \boldsymbol{I})^{-1} \boldsymbol{\Sigma} \boldsymbol{A}^\top \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}_t}^{-1} \left(\boldsymbol{\nu} - \boldsymbol{A}\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}] \right). \end{aligned} \tag{7.3.20}$$

Esto nos proporciona una descomposición más interpretable del *denoiser* posterior condicional (7.3.16): según la Ecuación (7.3.14), es la suma del *denoiser* posterior incondicional (7.3.17) y el término de concordancia de mediciones (7.3.20).

Podemos analizar más a fondo el término de concordancia de mediciones para comprender en qué casos se puede aproximar. Obsérvese que

$$\boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}_t} = \sigma^2 \boldsymbol{I} + \boldsymbol{A} \boldsymbol{\Sigma}^{1/2} \left(\boldsymbol{I} - \boldsymbol{\Sigma}^{1/2} \left(\boldsymbol{\Sigma} + t^2 \boldsymbol{I} \right)^{-1} \boldsymbol{\Sigma}^{1/2} \right) \boldsymbol{\Sigma}^{1/2} \boldsymbol{A}^\top. \tag{7.3.21}$$

Si definimos $\boldsymbol{\Sigma} = \boldsymbol{V} \boldsymbol{\Lambda} \boldsymbol{V}^\top$ como una descomposición en valores propios de $\boldsymbol{\Sigma}$, donde $(\boldsymbol{v}_i)$ son las columnas de $\boldsymbol{V}$, podemos escribir además

$$\boldsymbol{\Sigma}^{1/2} \left(\boldsymbol{I} - \boldsymbol{\Sigma}^{1/2} \left(\boldsymbol{\Sigma} + t^2 \boldsymbol{I} \right)^{-1} \boldsymbol{\Sigma}^{1/2} \right) \boldsymbol{\Sigma}^{1/2} = t^2 \boldsymbol{V} \boldsymbol{\Lambda}^{1/2} \left(\boldsymbol{\Lambda} + t^2 \boldsymbol{I} \right)^{-1} \boldsymbol{\Lambda}^{1/2} \boldsymbol{V}^\top \tag{7.3.22}$$

$$= t^2 \sum_{i=1}^{D} \frac{\lambda_i}{\lambda_i + t^2} \boldsymbol{v}_i \boldsymbol{v}_i^*. \tag{7.3.23}$$

Entonces, para cualquier valor propio de $\boldsymbol{\Sigma}$ igual a cero, el sumando correspondiente es cero; y si denotamos por $\lambda_{\min}(\boldsymbol{\Sigma})$ al valor propio positivo más pequeño de $\boldsymbol{\Sigma}$, tenemos que siempre que $t \ll \sqrt{\lambda_{\min}(\boldsymbol{\Sigma})}$, se cumple que

$$\frac{\lambda_i t^2}{\lambda_i + t^2} \approx 0. \tag{7.3.24}$$

Por lo tanto, cuando $t \ll \sqrt{\lambda_{\min}(\boldsymbol{\Sigma})}$, tenemos la aproximación

$$\boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}_t} \approx \sigma^2 \boldsymbol{I}. \tag{7.3.25}$$

El miembro derecho de esta aproximación es igual a $\boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}$. REn consecuencia, a su vez, tenemos

$$\nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu} \mid \boldsymbol{\xi}) \approx \nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}}(\boldsymbol{\nu} \mid \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]). \tag{7.3.26}$$

■

Dediquemos un momento a interpretar los resultados del Ejemplo 7.4 antes de generalizarlos. La aproximación (7.3.26) es, por supuesto, una consecuencia directa de los supuestos de modelado específicos que hemos establecido en el Ejemplo 7.4. Sin embargo, observen en que, si interpretamos directamente esta aproximación, resulta *ab initio* manejable: la verosimilitud $p_{\boldsymbol{y}|\boldsymbol{x}} = \mathcal{N}(\boldsymbol{A}\boldsymbol{x}, \sigma^2\boldsymbol{I})$ es una simple distribución gaussiana centrada en la observación, y la aproximación al término de correspondencia de la medición a la que llegamos puede interpretarse simplemente como evaluar la log-verosimilitud en la esperanza condicional $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$ y luego tomar los gradientes con respecto a $\boldsymbol{\xi}$ (lo cual implica realizar una retropropagación a través de la esperanza condicional).

Para comprender mejor el efecto de la aproximación conveniente (7.3.26), implementamos y simulamos un sencillo experimento numérico en el contexto gaussiano que se muestra en la Figura 7.11. El muestreador que implementamos es una implementación directa del esquema simple (3.2.82) que hemos desarrollado en el Capítulo 3 y que hemos recordado anteriormente, utilizando el verdadero *denoiser* posterior condicional, es decir, la Ecuación (7.3.16) (las muestras están marcadas con círculos azules) y la aproximación conveniente para este *denoiser* realizada mediante la aproximación de correspondencia de mediciones (7.3.26) (las muestras están marcadas con triángulos rojos). La fila superior de la Figura 7.11 muestra una configuración con un ruido de medición elevado σ^2, y la inferior, con un ruido de medición reducido. La aproximación de correspondencia de mediciones funciona muy bien en la configuración de ruido elevado, con la salvedad de que, en la configuración de ruido bajo, adolece de un rápido colapso de la varianza de la distribución muestral en direcciones paralelas a las filas del operador de medición lineal $\boldsymbol{A}$, lo cual no puede corregirse mediante iteraciones posteriores del muestreo. Nuestro análisis en el Ejemplo 7.4 caracteriza con precisión el nivel en el que el ruido se vuelve demasiado «pequeño» en términos de la matriz de covarianza de los datos $\boldsymbol{\Sigma}$.

Generalización de la aproximación: muestreo posterior por difusión. El Ejemplo 7.4 propone una aproximación conveniente para el término de correspondencia de la medición, es decir, (7.3.26), que puede extenderse más allá del marco gaussiano del ejemplo. Para justificar esta aproximación de manera más general, observe que por la independencia condicional de $\boldsymbol{y}$ y $\boldsymbol{x}_t$ dado $\boldsymbol{x}$, podemos escribir

$$p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu} \mid \boldsymbol{\xi}) = \int p_{\boldsymbol{y}|\boldsymbol{x}}(\boldsymbol{\nu} \mid \boldsymbol{\xi}')p_{\boldsymbol{x}|\boldsymbol{x}_t}(\boldsymbol{\xi}' \mid \boldsymbol{\xi})\mathrm{d}\boldsymbol{\xi}'. \tag{7.3.27}$$

Formalmente, cuando la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{x}_t}$ es una función delta centrada en su media $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$, la aproximación (7.3.26) es exacta. Más en general, cuando la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{x}_t}$ está muy concentrada alrededor de su media, la aproximación (7.3.26) es precisa. Esto se cumple, por ejemplo, para valores de t suficientemente pequeños, como vimos explícitamente en el contexto gaussiano de Ejemplo 7.4. Aunque la simulación numérica en

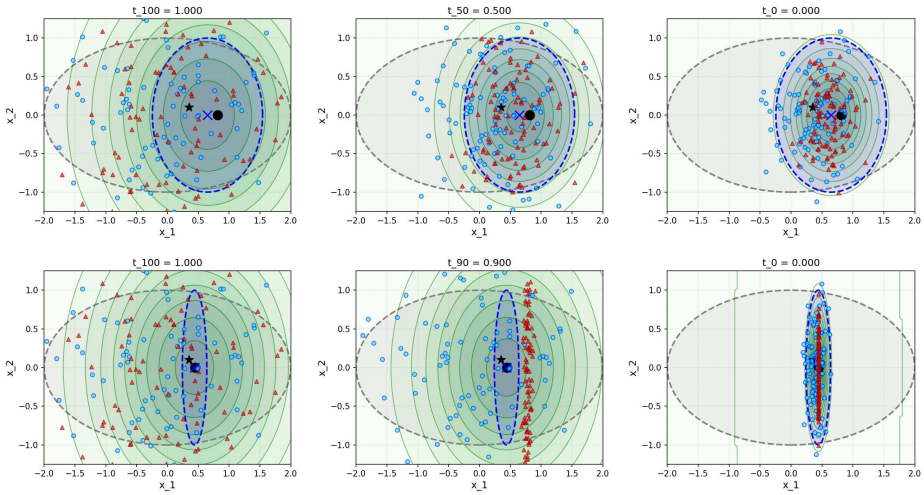


Figura 7.11: Simulación numérica de la configuración de muestreo condicional (7.3.9), con datos gaussianos, mediciones lineales y ruido gaussiano. Simulamos $D = 2$ y $d = 1$, con $\boldsymbol{\Sigma} = \boldsymbol{e}_1\boldsymbol{e}_1^\top + \frac{1}{4}\boldsymbol{e}_2\boldsymbol{e}_2^\top$, $\boldsymbol{\mu} = \mathbf{0}$ y $\boldsymbol{A} = \boldsymbol{e}_1^\top$. La señal subyacente $\boldsymbol{x}$ está marcada con una estrella negra, y la medición $\boldsymbol{y}$ está marcada con un círculo negro. Cada gráfico individual corresponde a un valor diferente del tiempo de muestreo t_ℓ, y las distintas filas corresponden a diferentes niveles de ruido de observación σ^2. En cada gráfico, la matriz de covarianza de $\boldsymbol{x}$ se representa en gris, la matriz de covarianza a posteriori y la media a posteriori de $p_{\boldsymbol{x}|\boldsymbol{y}}$ se representan en azul (con la media a posteriori marcada con una «x» azul), y los contornos de $p_{\boldsymbol{x}_{t_\ell}|\boldsymbol{y}}$ se dibujan en verde. Los hiperparámetros del muestreador son $T = 1$ y $L = 100$, y extraemos 100 muestras independientes para inicializar los muestreadores. Los muestreadores se implementan con los eliminadores de ruido de forma cerrada derivados en el Ejemplo 7.4, donde los que utilizan la aproximación (7.3.26) están marcados con triángulos rojos y los que utilizan el eliminador de ruido posterior condicional exacto marcados con círculos azules. **Arriba:** Para un ruido de observación elevado $\sigma = 0.5$, tanto el *denoiser* posterior condicional exacto como el aproximado convergen satisfactoriamente hacia la distribución posterior $p_{\boldsymbol{x}|\boldsymbol{y}}$. El tiempo de muestreo (que corresponde al tiempo en el «proceso directo», por lo que tiempos más largos implican un ruido mayor) disminuye de izquierda a derecha. La dinámica de convergencia para el término de ajuste de la medición exacta y el de la aproximada es similar. **Parte inferior:** Para un ruido de observación menor $\sigma = 0.1$, el término de correspondencia de mediciones aproximadas da lugar a un sesgo extremo en el muestreador (triángulos rojos): las muestras convergen rápidamente hacia un subespacio afín de puntos que son consistentes, módulo cierta contracción del *denoiser* de media a posteriori, con los datos de referencia medidos, y las iteraciones de muestreo posteriores son incapaces de recuperar la varianza a posteriori perdida a lo largo de esta dimensión. Nótese que en la fila inferior se representan diferentes instantes t_ℓ, en comparación con la fila superior, para mostrar el rápido colapso de la aproximación a la distribución a posteriori a lo largo de la dimensión de medición.

Figura 7.11 sugiere que esta aproximación no está exenta de salvedades en ciertos regímenes, ha demostrado ser una línea de base confiable en la práctica, tras haber sido propuesta por Chung et al. como «Muestreo posterior por difusión» (DPS) [CKM+23]. Además, existen enfoques fundamentados y generalizables para mejorarlo mediante la incorporación de mejores estimaciones de la varianza posterior (que resultan ser exactas en el contexto gaussiano de Ejemplo 7.4), que analizamos con más detalle en el resumen al final del capítulo.

Así, con la aproximación DPS, obtenemos la siguiente aproximación para los *denoisers* del posterior condicional $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}, \boldsymbol{x}_t]$, mediante Ecuación (7.3.14):

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}, \boldsymbol{y} = \boldsymbol{\nu}] \approx \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}] + t^2 \nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}}(\boldsymbol{\nu} \mid \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]). \quad (7.3.28)$$

De esta forma, para una red neuronal u otro modelo $\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi})$ entrenado como en la Sección 3.2 para aproximar los *denoisers* $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$ para cada $t \in [0, T]$, obtenemos los *denoisers* aprendidos posteriores condicionales

$$\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi}, \boldsymbol{\nu}) = \bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi}) + t^2 \nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}}(\boldsymbol{\nu} \mid \bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi})). \quad (7.3.29)$$

Obsérvese que la aproximación (7.3.28) es válida para modelos directos arbitrarios h en el modelo de observación (7.3.9), incluyendo el caso no lineal h e incluso para modelos de ruido arbitrarios para los que se conoce una expresión explícita de la verosimilitud $p_{\boldsymbol{y}|\boldsymbol{x}}$. De hecho, en el caso del ruido gaussiano, tenemos

$$p_{\boldsymbol{y}|\boldsymbol{x}}(\boldsymbol{\nu} \mid \boldsymbol{\xi}) \propto \exp\left(-\frac{1}{2\sigma^2} \|h(\boldsymbol{\xi}) - \boldsymbol{\nu}\|_2^2\right). \quad (7.3.30)$$

Por lo tanto, para evaluar el lado derecho de (7.3.29) solo se necesita

1. Un *denoiser* preentrenado $\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi})$ para la distribución de datos p (de $\boldsymbol{x}$), aprendido como se indica en la Sección 3.2 mediante el Algoritmo 3.2;

2. Acceso mediante pases hacia adelante y hacia atrás al modelo directo h para las mediciones (7.3.9);

3. Un paso hacia adelante y hacia atrás a través de $\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi})$, que puede evaluarse de manera eficiente utilizando (por ejemplo) la retropropagación.

Al combinar este esquema con la implementación básica del muestreo incondicional que desarrollamos en la Sección 3.2, obtenemos un algoritmo práctico para el muestreo condicional de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$ dadas las mediciones, siguiendo (7.3.9). El Algoritmo 7.1 describe este esquema para el caso de ruido de observación gaussiano con desviación estándar conocida σ, con ligeras modificaciones para extenderlo a un proceso de ruido general, como en la Ecuación (3.2.85) y la discusión en torno a este tema en el Capítulo 3 (nuestra discusión anterior hizo las simplificaciones $\alpha_t = 1$, $\sigma_t = t$ y $t_\ell = T\ell/L$, tal como en Ecuación (3.2.82) en Sección 3.2).

Algoritmo 7.1: Muestreo condicional bajo las mediciones (7.3.9), con un *denoiser* incondicional y DPS.

Input: Una lista ordenada de pasos de tiempo $0 \leq t_0 < \cdots < t_L \leq T$ que se utilizarán para el muestreo.
Input: Un *denoiser* incondicional $\bar{\boldsymbol{x}}_\theta \colon \{t_\ell\}_{\ell=1}^L \times \mathbb{R}^D \to \mathbb{R}^D$ para $p_{\boldsymbol{x}}$.
Input: Realización de la medición $\boldsymbol{\nu}$ de $\boldsymbol{y}$ (Ecuación (7.3.9)) para condicionar.
Input: Modelo directo $h : \mathbb{R}^D \to \mathbb{R}^d$ y varianza del ruido de medición $\sigma^2 > 0$.
Input: Funciones de escala y nivel de ruido $\alpha, \sigma \colon \{t_\ell\}_{\ell=0}^L \to \mathbb{R}_{\geq 0}$.
Output: Una muestra $\hat{\boldsymbol{x}}$, aproximadamente a partir de $p_{\boldsymbol{x}|\boldsymbol{y}}$.

1: **function** DDIMSamplerConditionalDPS($\bar{\boldsymbol{x}}_\theta, \boldsymbol{\nu}, h, \sigma^2, (t_\ell)_{\ell=0}^L$)
2: Inicializar $\hat{\boldsymbol{x}}_{t_L} \sim$ distribución aproximada de $\boldsymbol{x}_{t_L}$ (VP $\implies \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, VE $\implies \mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$.)
3: **for** $\ell = L, L-1, \ldots, 1$ **do**
4: Calcular

$$w_{t_{\ell-1}} \doteq \alpha_{t_{\ell-1}} - \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \alpha_{t_\ell};$$

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} \doteq \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \hat{\boldsymbol{x}}_{t_\ell} + \left(\bar{\boldsymbol{x}}_\theta(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}) - \frac{\sigma_{t_\ell}^2}{2\alpha_{t_\ell}\sigma^2} \nabla_{\boldsymbol{\xi}} \left[\| h(\bar{\boldsymbol{x}}_\theta(t_\ell, \boldsymbol{\xi})) - \boldsymbol{\nu} \|_2^2 \right] \Big|_{\boldsymbol{\xi} = \hat{\boldsymbol{x}}_{t_\ell}} \right)$$

5: **end for**
6: **return** $\hat{\boldsymbol{x}}_{t_0}$
7: **end function**

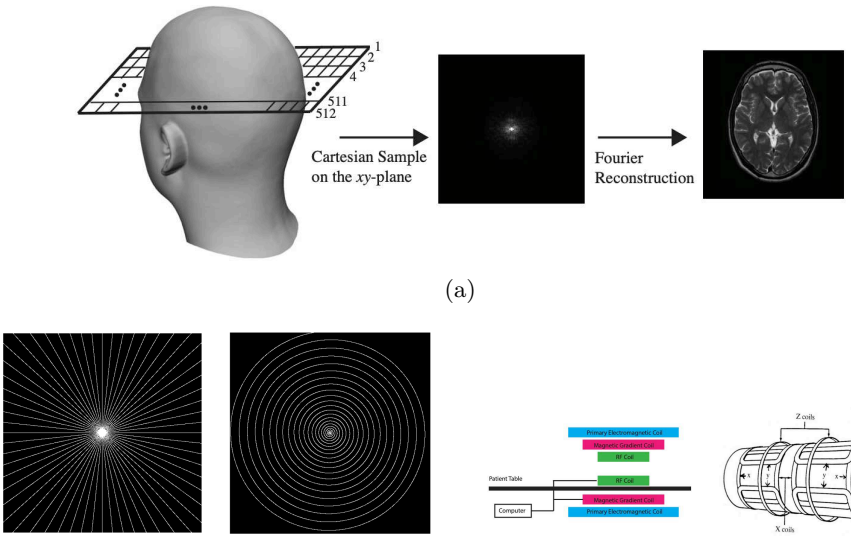


Figura 7.12: Resumen simplificado de la reconstrucción de imágenes por resonancia magnética (RM). **(a)** Resumen del proceso de obtención de imágenes. Un corte del objeto cuya imagen se está captando (por ejemplo, un cerebro humano) se mide en el dominio de Fourier mediante el equipo de RM. La imagen espacial puede reconstruirse mediante la inversión directa de las mediciones en el dominio de Fourier. **(b)** En la práctica, el equipo de RM *submuestrea* el dominio de Fourier mediante un patrón de muestreo estructurado en aras de la eficiencia. Esto da lugar a un problema inverso lineal, en el que se deben aprovechar los información a priori sobre la estructura de la imagen subyacente para permitir una reconstrucción única. **(c)** El aparato de medición real utiliza un campo magnético controlado para obtener las mediciones en el dominio de Fourier. Imágenes reproducidas con permiso de [WM22].

Aplicación a la reconstrucción de imágenes médicas. El marco que hemos desarrollado anteriormente ya es lo suficientemente potente como para aplicarse a la resolución de diversos *problemas científicos inversos*, en los que las mediciones $\boldsymbol{y} = h(\boldsymbol{x}_o) + \boldsymbol{w}$ son salidas de un proceso de medición conocido h sobre una señal específica de interés $\boldsymbol{x}_o$ y donde hay infinitas señales posibles $\boldsymbol{x}$ que satisfacen $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$. Esto significa que, para encontrar el $\boldsymbol{x}_o$ *verdadero* que generó $\boldsymbol{y}$, necesitamos aprovechar la información previa sobre la estructura de $\boldsymbol{x}_o$. Consideraremos el contexto probabilístico en el que $\boldsymbol{x}_o \sim p_{\boldsymbol{x}}$, y en el que tenemos acceso a una distribución a priori de $p_{\boldsymbol{x}}$, por ejemplo, en forma de un modelo de difusión.

Un ejemplo ilustrativo de un problema científico inverso es la resonancia magnética (RM) (Figura 7.12), en la que la distribución de datos $p_{\boldsymbol{x}}$ está definida sobre imágenes correspondientes a un objeto de interés subyacente (como un

corte bidimensional del cerebro de un sujeto humano). En aplicaciones de RM, el operador de medición h corresponde a la máquina que genera y modula el campo magnético y registra la respuesta del paciente al mismo (Figura 7.12(c)). Se puede modelar como implementación de la *transformada de Fourier* de la señal espacial subyacente (Figura 7.12(a)) —véase [WM22, Capítulo 10] para una derivación matemática detallada. Además, en aras de la eficiencia, la máquina de resonancia magnética normalmente solo mide un subconjunto de las frecuencias medidas, en un patrón de medición estructurado que puede modelarse como un patrón radial o espiral (Figura 7.12(b)). Si escribimos $\mathcal{F}$ para denotar el operador de la transformada de Fourier (discreta) y $\mathcal{S}$ para denotar el patrón de submuestreo (conocido) de la máquina, la señal medida viene dada por

$$\boldsymbol{y} = \mathcal{S} \circ \mathcal{F}(\boldsymbol{x}_o) + \boldsymbol{w}. \tag{7.3.31}$$

El operador de medición $\mathcal{S} \circ \mathcal{F}$ es *lineal*, lo que implica que puede representarse mediante una matriz $m \times n$ tras el redimensionamiento adecuada, pero también es *compresivo*, lo que significa que $m \ll n$ y que necesitamos aprovechar el conocimiento previo sobre $p_{\boldsymbol{x}}$ para reconstruir $\boldsymbol{x}_o$ (o, de manera más general, para tomar muestras de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$).

La distribución $p_{\boldsymbol{x}}$ de las imágenes de resonancia magnética es más simple que algunas distribuciones de imágenes (por ejemplo, las imágenes naturales), pero sigue siendo lo suficientemente compleja como para que resulte ventajoso utilizar un modelo aprendido para $p_{\boldsymbol{x}}$ en lugar de uno analítico (véase [WM22]). Un enfoque directo consiste en aplicar la aproximación de muestreo posterior por difusión que desarrollamos en la sección anterior, lo que dio lugar al Algoritmo 7.1, con el operador de medida específico de la RM $\boldsymbol{A} = \mathcal{S} \circ \mathcal{F}$ y un modelo de difusión preentrenado con datos de RM. Esta combinación concreta de enfoques no ha aparecido en la literatura. En su lugar, destacamos los resultados de un enfoque diferente, aunque relacionado, que impone la consistencia de las mediciones en cada paso de difusión. A raíz del trabajo pionero de Song et al. [SSX+22], se han desarrollado numerosos algoritmos de consistencia de mediciones de este tipo para la reconstrucción de imágenes médicas, incluidos los enfoques mejorados de Song et al. [SKZ+24] y Rout et al. [RRD+23] (que utilizan modelos de difusión latente, tal como los hemos desarrollado para los autocodificadores de representación en el Capítulo 6). La Figura 7.13 muestra comparaciones visuales de imágenes de resonancia magnética reconstruidas en tres muestras diferentes, comparando FISTA-TV (una versión acelerada del algoritmo clásico ISTA que estudiamos en Capítulo 2), el enfoque de modelado de difusión por consistencia de mediciones de Song et al. [SSX+22] y la verdad de referencia (ground truth). Se puede observar que el método basado en difusión permite obtener una calidad de reconstrucción superior, así como una mayor fidelidad a los detalles finos de la imagen de referencia. La Tabla 7.1 muestra los resultados de precisión de la reconstrucción de resonancia magnética (medidos en términos de PSNR) para el enfoque desarrollado por Song et al. [SSX+22]. Supera a los modelos de referencia supervisados (entrenados con numerosas muestras emparejadas independientes

Método	24× acel.	8× acel.	4× acel.
Cascade DenseNet	$23.39_{\pm 2.17}$	$28.35_{\pm 2.30}$	$30.97_{\pm 2.33}$
DuDoRNet	$18.46_{\pm 3.05}$	$\mathbf{37.88_{\pm 3.03}}$	$30.53_{\pm 4.13}$
Song et al. [SSK+21]	$27.83_{\pm 2.73}$	$35.04_{\pm 2.11}$	$37.55_{\pm 2.08}$
Jalal et al. [JAD+21]	$28.80_{\pm 3.21}$	$36.44_{\pm 2.28}$	$38.76_{\pm 2.32}$
Song et al. [SSX+22]	$\mathbf{29.42_{\pm 3.03}}$	$37.63_{\pm 2.70}$	$\mathbf{39.91_{\pm 2.67}}$

Tabla 7.1: **Resultados de la reconstrucción de resonancia magnética** donde se comparan enfoques de conciliación de mediciones con diferentes factores de aceleración (el grado de submuestreo del espacio de Fourier en $\mathcal{S}$) en el conjunto de datos de reconstrucción de resonancia magnética BraTS. Se presentan las métricas PSNR (cuanto más alto, mejor) junto con sus desviaciones estándar. Cascade DenseNet [ZFZ19] y DuDoRNet [ZZ20] son modelos de referencia supervisados entrenados con muestras emparejadas a un nivel de aceleración de 8x. Los últimos tres métodos son enfoques basados en la difusión que utilizan la consistencia de las mediciones. Reproducido con permiso de Song et al. [SSX+22].

$(\boldsymbol{y}, \boldsymbol{x}_o)$, lo cual es un requisito adicional) y con los modelos de referencia no supervisados (que corresponden a aproximaciones de correspondencia de mediciones anteriores al muestreo posterior por difusión).

7.4 Inferencia condicional con datos y mediciones emparejados

En muchas aplicaciones prácticas, no conocemos ni la distribución de los datos $\boldsymbol{x}$ de interés ni la relación explícita entre los datos y ciertos atributos observados $\boldsymbol{y}$ de los datos. Solo tenemos un conjunto (grande) de muestras emparejadas $(\boldsymbol{X}, \boldsymbol{Y}) = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_N, \boldsymbol{y}_N)\}$ a partir del cual necesitamos inferir la distribución de los datos y una correspondencia que modele su relación:

$$h : \boldsymbol{x} \mapsto \boldsymbol{y}. \tag{7.4.1}$$

El problema de la clasificación de imágenes puede considerarse un ejemplo de ello. En cierto sentido, el problema de la clasificación consiste en aprender un codificador compresivo (con una pérdida de información extremadamente elevada) para imágenes naturales. Digamos que, dada una muestra aleatoria de una imagen $\boldsymbol{x}$, nos gustaría predecir su etiqueta de clase $\boldsymbol{y}$ que mejor se correlacione con el contenido de $\boldsymbol{x}$. Sabemos que la distribución de las imágenes naturales de los objetos es de baja dimensión en comparación con la dimensión del espacio de píxeles. En los capítulos anteriores, hemos aprendido que, si contamos con suficientes muestras, en principio podemos aprender una representación estructurada de baja dimensión $\boldsymbol{z}$ para $\boldsymbol{x}$ mediante una

codificación compresiva aprendida:

$$f : \boldsymbol{x} \mapsto \boldsymbol{z}. \tag{7.4.2}$$

La representación $\boldsymbol{z}$ también puede considerarse como un código aprendido (con pérdida, pero estructurado) para $\boldsymbol{x}$. Es bastante razonable suponer que si la asignación de clase $\boldsymbol{y}$ depende realmente de las estructuras de baja dimensión de $\boldsymbol{x}$ y el código aprendido $\boldsymbol{z}$ refleja realmente dichas estructuras, $\boldsymbol{y}$ y $\boldsymbol{z}$ pueden hacerse altamente correlacionados y, por lo tanto, su distribución conjunta $p(\boldsymbol{z}, \boldsymbol{y})$ debería ser de dimensión extremadamente baja. Por lo tanto, podemos combinar los dos códigos deseados, $\boldsymbol{y}$ y $\boldsymbol{z}$ e intentar entrenar un codificador combinado:

$$f : \boldsymbol{x} \mapsto (\boldsymbol{z}, \boldsymbol{y}) \tag{7.4.3}$$

donde la distribución conjunta de $(\boldsymbol{z}, \boldsymbol{y})$ es de muy baja dimensión.

Según lo visto en los capítulos anteriores, el mapeo f suele aprenderse como una secuencia de operadores de compresión o de eliminación de ruido en el mismo espacio. Por lo tanto, para aprovechar esta familia de operaciones, podemos introducir un vector auxiliar $\boldsymbol{w}$ que puede considerarse como una conjetura aleatoria inicial de la etiqueta de clase $\boldsymbol{y}$. De esta manera, podemos aprender un mapeo de compresión o de eliminación de ruido:

$$f : (\boldsymbol{x}, \boldsymbol{w}) \mapsto (\boldsymbol{z}, \boldsymbol{y}) \tag{7.4.4}$$

dentro de un espacio común. De hecho, la práctica habitual de introducir un «token de clase» auxiliar en el entrenamiento de un transformer para tareas de clasificación, como en ViT, puede considerarse como el aprendizaje de dicha representación mediante la compresión (de la tasa de codificación) de muestras dadas (con ruido) de $(\boldsymbol{x}, \boldsymbol{w})$. Si la distribución de los datos $\boldsymbol{x}$ es ya una mezcla de gaussianas (de baja dimensión), el trabajo [WTL+08] ha demostrado que la clasificación puede realizarse de manera eficaz minimizando directamente *la longitud de codificación (con pérdida)* asociada a las muestras dadas.

7.4.1 Generación de imágenes condicionada por clase

Si bien un clasificador aprendido nos permite clasificar una imagen dada $\boldsymbol{x}$ en su clase correspondiente, a menudo nos interesa generar una imagen de una clase determinada, mediante el muestreo de la distribución aprendida de imágenes naturales. En cierta medida, esto puede considerarse como el problema «inverso» a la clasificación de imágenes. Sea $p_{\boldsymbol{x}}$ la distribución de imágenes naturales, modeladas, por ejemplo, mediante un proceso de eliminación de ruido por difusión. Dada una variable aleatoria de clase $y \in [K]$ con realización ν, por ejemplo, una «Manzana», nos gustaría muestrear la distribución condicional $p_{\boldsymbol{x}|y}(\cdot \mid \nu)$ para generar una imagen de una manzana:

$$\hat{\boldsymbol{x}} \sim p_{\boldsymbol{x}|\boldsymbol{y}}(\cdot \mid \boldsymbol{\nu}). \tag{7.4.5}$$

A esto se le llama «generación de imágenes condicionada a la clase».

En la Sección 7.3.2, hemos visto cómo utilizar el paradigma de difusión con eliminación de ruido para el muestreo condicional a partir de la distribución a posteriori $p_{\boldsymbol{x}|\boldsymbol{y}}$ dadas mediciiones *basada en el modelo* $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$ (Ecuación (7.3.9)), lo que culmina en el algoritmo DPS (Algoritmo 7.1). Si bien este marco es potente, no resulta aplicable al problema de la generación de imágenes condicionadas por clase (o texto) que nos ocupa, en el que no se dispone de un modelo generativo explícito h para las observaciones/atributos y debido a la inmanejabilidad de la modelación analítica. En esta sección, presentaremos técnicas para extender el muestreo condicional a este contexto.

Por lo tanto, ahora suponemos únicamente que tenemos acceso a muestras de la distribución conjunta de $(\boldsymbol{x}, y)$:

$$(\boldsymbol{x}, y) \sim p_{\boldsymbol{x},y}. \tag{7.4.6}$$

Al igual que en la sección anterior, definimos $\boldsymbol{x}_t = \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{g}$ donde $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ es independiente de $(\boldsymbol{x}, \boldsymbol{y})$, tal como se muestra en la Ecuación (3.2.85) del Capítulo 3, y utilizaremos repetidamente la notación $\boldsymbol{\xi}$ para denotar realizaciones de $\boldsymbol{x}$ y $\boldsymbol{x}_t$.

Esta sección se centra en los fundamentos conceptuales y algorítmicos del muestreo condicional en los modelos generativos modernos. Para un análisis centrado en las cuestiones de implementación en diversas aplicaciones, véanse el Capítulo 8, la Sección 8.7 y las secciones siguientes.

Orientación del clasificador: consistencia mediante un clasificador preentrenado

Recordemos de la Sección 7.3.2 que se cumple la siguiente descomposición del *denoiser* posterior condicional óptimo: en virtud de la regla de Bayes y de la independencia condicional de $\boldsymbol{y}$ y $\boldsymbol{x}_t$ dado $\boldsymbol{x}$ (véase la Figura 7.10):

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}, y = \nu] = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}] + \frac{\sigma_t^2}{\alpha_t} \nabla_{\boldsymbol{\xi}} \log p_{y|\boldsymbol{x}_t}(\nu \mid \boldsymbol{\xi}). \tag{7.4.7}$$

En otras palabras, la representación (7.4.7) del *denoiser* condicional óptimo es válida independientemente de si lo es el modelo de observación explícito $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$. Esto significa que si logramos aprender un *denoiser* de este tipo (7.4.7) en el contexto de datos emparejados, podremos seguir la metodología de difusión con eliminación de ruido que hemos desarrollado en el Capítulo 3 para realizar un muestreo condicional por clase.

Una idea natural es, entonces, implementar directamente el término de corrección de verosimilitud en (7.4.7) utilizando una red profunda f_{θ_c} con parámetros θ_c, tal como se muestra en Ecuación (7.4.4):

$$f_{\theta_c} : (t, \boldsymbol{x}_t) \mapsto \mathrm{softmax}(\boldsymbol{W}_{\mathrm{head}} \boldsymbol{z}(t, \boldsymbol{x}_t)). \tag{7.4.8}$$

Esta expresión combina las representaciones finales $\boldsymbol{z}(t, \boldsymbol{x}_t)$ (que también dependen de θ_c) de las entradas con ruido $\boldsymbol{x}_t$ con un cabezal de clasificación

$\boldsymbol{W}_{\text{head}} \in \mathbb{R}^{K\times d}$, que mapea las representaciones a una distribución de probabilidad sobre las K clases posibles. Como es habitual en la práctica, también toma el tiempo t en el proceso de ruido como entrada. Por lo tanto, con un entrenamiento adecuado, proporciona una aproximación a la verosimilitud logarítmica $\log p_{y|\boldsymbol{x}_t}$, y derivar $\log f_{\theta_c}$ respecto a su entrada $\boldsymbol{x}_t$ permite aproximar el segundo término de Ecuación (7.4.7):

$$\bar{\boldsymbol{x}}_\theta^{\text{naive}}(t, \boldsymbol{x}_t, y) = \bar{\boldsymbol{x}}_{\theta_d}(t, \boldsymbol{x}_t) + \frac{\sigma_t^2}{\alpha_t}\nabla_{\boldsymbol{x}_t} \langle \log f_{\theta_c}(t, \boldsymbol{x}_t), \boldsymbol{e}_y\rangle \tag{7.4.9}$$

donde, como es habitual, aproximamos el primer término de Ecuación (7.4.7) mediante un *denoiser* incondicional aprendido para $\boldsymbol{x}_t$ con parámetros θ_d y donde escribimos $\boldsymbol{e}_k$ para $k \in [K]$ para denotar el k-ésimo vector de base canónica para $\mathbb{R}^K$ (es decir, el vector con un uno en la posición k-ésima, y ceros en las demás posiciones). El lector debe tener en cuenta que el *denoiser* condicional $\bar{\boldsymbol{x}}_\theta$ requiere dos procesos de entrenamiento independientes, con pérdidas distintas: uno para los parámetros del clasificador θ_c, sobre una pérdida de clasificación [7] y otro para los parámetros del *denoiser* θ_d, sobre una pérdida de eliminación de ruido. Este enfoque del muestreo condicional ya se había reconocido y aprovechado para llevar a cabo el muestreo condicional en los primeros trabajos pioneros sobre modelos de difusión, en particular los de Sohl-Dickstein et al. [SWM+15] y Song et al. [SSK+21].

Sin embargo, esta metodología sencilla presenta dos inconvenientes fundamentales (razón por la cual la calificamos de «ingenua»). El primero es que, según la experiencia, un clasificador de red profunda entrenado de este modo a menudo no proporciona una señal de guía lo suficientemente sólida (en la Ecuación (7.4.7)) como para garantizar que las muestras generadas reflejen la información de condicionamiento y. Esto fue destacado por primera vez por Dhariwal and Nichol [DN21b], quien señaló que, en el contexto de la generación de ImageNet condicionada por clase, las probabilidades de salida del clasificador de red profunda entrenado para la clase y sobre la que se aplicaba la condición solían rondar $0,5$ —lo suficientemente grande como para ser la clase dominante, pero no lo suficiente como para proporcionar una señal de guía sólida— y que, tras inspeccionarlas, las generaciones no eran consistentes con la clase de condicionamiento y. Dhariwal and Nichol [DN21b] propuso abordar esto de manera heurística incorporando un hiperparámetro de «temperatura inversa» $\gamma > 0$ en la definición del *denoiser* condicional ingenuo (7.4.9), refiriéndose al *denoiser* condicional resultante como si hubiera incorporado **«guía del clasificador»** (CG):

$$\bar{\boldsymbol{x}}_\theta^{\text{CG}}(t, \boldsymbol{x}_t, y) = \bar{\boldsymbol{x}}_{\theta_d}(t, \boldsymbol{x}_t) + \gamma\frac{\sigma_t^2}{\alpha_t}\nabla_{\boldsymbol{x}_t} \langle \log f_{\theta_c}(t, \boldsymbol{x}_t), \boldsymbol{e}_y\rangle \tag{7.4.10}$$

que para $\gamma = 1$ coincide con (7.4.9).

[7] En Capítulo 8, revisamos el proceso de entrenamiento de dicho clasificador con todo detalle.

Dhariwal and Nichol [DN21b] descubrieron que un valor de $\gamma > 1$ ofrecía los mejores resultados empíricos. Una posible interpretación de esto es la siguiente: obsérvese que, en el contexto del término de verosimilitud *verdadera* de la Ecuación (7.4.7), la multiplicación por γ da como resultado, de manera equivalente,

$$\gamma \frac{\sigma_t^2}{\alpha_t} \nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu} \mid \boldsymbol{\xi}) = \frac{\sigma_t^2}{\alpha_t} \nabla_{\boldsymbol{\xi}} \log \left(p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu} \mid \boldsymbol{\xi})^{\gamma} \right), \tag{7.4.11}$$

lo que sugiere la interpretación natural de que el parámetro γ realiza un *escalado de temperatura* (inverso) sobre la verosimilitud $p_{\boldsymbol{y}|\boldsymbol{x}_t}$, lo cual es preciso si consideramos la distribución renormalizada $p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu} \mid \boldsymbol{\xi})^{\gamma} / \int p_{\boldsymbol{y}|\boldsymbol{x}_t}(\boldsymbol{\nu}' \mid \boldsymbol{\xi})^{\gamma} \mathrm{d}\boldsymbol{\nu}'$ Sin embargo, cabe señalar que esta *no es* una interpretación rigurosa en el contexto de la Ecuación (7.4.7), ya que los gradientes se calculan con respecto a $\boldsymbol{\xi}$ y la constante de normalización en la distribución escalada por temperatura es, en general, una función de $\boldsymbol{\xi}$. En cambio, el parámetro γ debe entenderse simplemente como una amplificación de los valores altos de las probabilidades de salida del clasificador de red profunda $f_{\theta_c}(t, \boldsymbol{x}_t)$ *en relación con* los más bajos, lo que amplifica efectivamente la señal de guía proporcionada en los casos en que la red profunda f le asigna la probabilidad más alta entre las K clases.

Simplificaciones y mejoras mediante la guía sin clasificador

No obstante, la guía mediante clasificador no resuelve el segundo inconveniente clave de la metodología ingenua: resulta engorroso y costoso entrenar un clasificador auxiliar f_{θ_c} además del *denoiser* incondicional $\bar{\boldsymbol{x}}_{\theta_d}$, dado que no es posible adaptar directamente un clasificador preentrenado debido a la necesidad de que funcione bien con entradas ruidosas $\boldsymbol{x}_t$ e incorpore otras modificaciones de arquitectura motivadas empíricamente. En concreto, Dhariwal and Nichol [DN21b] constató que era necesario *diseñar explícitamente la arquitectura de la red profunda que implementa el clasificador para que coincida con la del denoiser.*

Además, desde una perspectiva puramente práctica —es decir, al intentar obtener el mejor rendimiento posible del muestreador resultante— la configuración más eficaz del muestreo basado en la orientación del clasificador se aleja aún más del marco idealizado y conceptualmente sólido que hemos presentado anteriormente. Para obtener el mejor rendimiento, Dhariwal and Nichol [DN21b] consideró necesario proporcionar la etiqueta de clase y como una entrada adicional al *denoiser* $\bar{\boldsymbol{x}}_{\theta_d}$. Como resultado, el eliminador de ruido idealizado guiado por un clasificador (7.4.10), derivado por Dhariwal and Nichol [DN21b] igual que hicimos nosotros más arriba a partir de la descomposición del *denoiser* posterior condicional (7.4.7), no refleja exactamente el *denoiser* de mejor rendimiento en la práctica; ¡dicho *denoiser* combina en realidad un *denoiser condicional* para $\boldsymbol{x}_t$ dado y con una señal de guía adicional procedente de un clasificador auxiliar!

Este estado de cosas, si bien de naturaleza empírica, llevó a Ho and Salimans [HS22a] en trabajos posteriores a proponer una metodología más pragmática desde el punto de vista empírico, conocida como **guía sin clasificador**

(CFG). En lugar de representar el *denoiser* condicional (7.4.7) como una suma ponderada de un *denoiser* incondicional para $\boldsymbol{x}_t$ con un término de corrección de la verosimilitud logarítmica (con pesos posiblemente modificados, como en la guía del clasificador), aceptan la aparente necesidad de entrenar un *denoiser* condicional para $\boldsymbol{x}_t$ dado y, tal como lo demuestran los resultados experimentales de Dhariwal and Nichol [DN21b], y sustituir el término del gradiente de la log-verosimilitud por una suma correctamente ponderada de este *denoiser* condicional con un *denoiser* incondicional para $\boldsymbol{x}$ dado $\boldsymbol{x}_t$.[8]

Derivación de la guía sin clasificador. Para ver cómo surge esta estructura, comenzamos con una versión «idealizada» del *denoiser* con guía de clasificador $\bar{\boldsymbol{x}}_\theta^{\mathrm{CG}}$ definido en (7.4.10), para el cual el *denoiser* $\bar{\boldsymbol{x}}_{\theta_{\mathrm{d}}}$ y el clasificador $f_{\theta_{\mathrm{c}}}$ se aproximan perfectamente a sus objetivos, mediante (7.4.7):

$$\bar{\boldsymbol{x}}_\theta^{\mathrm{CG,\,ideal}}(t, \boldsymbol{\xi}, \nu) = \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}] + \gamma \frac{\sigma_t^2}{\alpha_t} \nabla_{\boldsymbol{\xi}} \log p_{y|\boldsymbol{x}_t}(\nu \mid \boldsymbol{\xi}). \tag{7.4.12}$$

A continuación, utilizamos la regla de Bayes, en la forma

$$\log p_{y|\boldsymbol{x}_t} = \log p_{\boldsymbol{x}_t|y} + \log p_y - \log p_{\boldsymbol{x}_t}, \tag{7.4.13}$$

junto con la fórmula de Tweedie (Teorema 3.2, modificada como en Ecuación (3.2.86)) para convertir entre funciones de *score* y *denoisers*, a fin de obtener

$$\begin{aligned} \bar{\boldsymbol{x}}_\theta^{\mathrm{CG,\,ideal}}(t, \boldsymbol{\xi}, \nu) &= \frac{1}{\alpha_t}\boldsymbol{\xi} + (1-\gamma)\frac{\sigma_t^2}{\alpha_t}\nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{x}_t}(\boldsymbol{\xi}) + \gamma \frac{\sigma_t^2}{\alpha_t} \nabla_{\boldsymbol{\xi}} \log p_{\boldsymbol{x}_t|y}(\boldsymbol{\xi} \mid \nu) \\ &= (1-\gamma)\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}] + \gamma \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}, y = \nu], \end{aligned} \tag{7.4.14}$$

donde, en la última línea, aplicamos la Ecuación (7.3.11). Ahora, la Ecuación (7.4.14) sugiere una estrategia de aproximación natural: combinamos un *denoiser* incondicional aprendido para $\boldsymbol{x}$ dado $\boldsymbol{x}_t$, como antes, con un *denoiser condicional* aprendido para $\boldsymbol{x}$ dado $\boldsymbol{x}_t$ y y.

Sin embargo, siguiendo a Ho and Salimans [HS22a] y la práctica habitual en el entrenamiento de de redes profundas como *denoisers*, se suele utilizar la *misma* red profunda para representar tanto el *denoiser* condicional como el incondicional mediante la introducción de una etiqueta adicional, que denotaremos como $\varnothing$, para indicar el caso «incondicional». Esto da lugar a la forma del *denoiser* CFG:

$$\bar{\boldsymbol{x}}_\theta^{\mathrm{CFG}}(t, \boldsymbol{x}_t, y) = (1-\gamma)\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{x}_t, \varnothing) + \gamma \bar{\boldsymbol{x}}_\theta(t, \boldsymbol{x}_t, y). \tag{7.4.15}$$

[8] Dicho esto, Ho and Salimans [HS22a] propusieron en realidad utilizar una ponderación diferente a la que presentamos aquí, basándose en el hecho de que Dhariwal and Nichol [DN21b] sustituyeron de manera heurística el *denoiser* incondicional de (7.4.7) por un *denoiser* condicional. De hecho, la ponderación que derivamos y presentamos aquí refleja la práctica moderna y, en particular, se utiliza en modelos de difusión de vanguardia como Stable Diffusion 3.5 [EKB+24].

Para entrenar un *denoiser* $\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{x}_t, y^+)$ para el muestreo de guía sin clasificador, donde $y^+ \in \{1, \dots, K, \varnothing\}$, procedemos de manera casi idéntica al procedimiento de entrenamiento incondicional descrito en el Algoritmo 3.2, pero con dos modificaciones:

1. Cuando tomamos muestras del conjunto de datos, tomamos un par $(\boldsymbol{x}, y)$ en lugar de solo una muestra $\boldsymbol{x}$.

2. Cada vez que extraemos un par del conjunto de datos, obtenemos la etiqueta aumentada y^+ mediante

$$y^+ = \begin{cases} \varnothing & \text{with probability } p_{\text{incond}}; \\ y & \text{else.} \end{cases} \tag{7.4.16}$$

   Aquí, $p_{\text{incond}} \in [0, 1]$ es un nuevo hiperparámetro. Esto puede considerarse una forma de dropout [SHK+14].

De esta manera, entrenamos un *denoiser* condicional adecuado para su uso en el muestreo guiado sin clasificador. Resumimos el proceso general de muestreo para el muestreo condicionado por clase con guía sin clasificador en Algoritmo 7.2.

Algoritmo 7.2: Muestreo condicional con datos de clasificación, utilizando un *denoiser* condicionado por clase.

Input: Una lista ordenada de pasos de tiempo $0 \le t_0 < \cdots < t_L \le T$ que se utilizarán para el muestreo.
Input: Etiqueta de clase $\nu \in \{1, \dots, K\}$ sobre la que se condicionará.
Input: Un *denoiser* $\bar{\boldsymbol{x}}_\theta \colon \{t_\ell\}_{\ell=1}^L \times \mathbb{R}^D \times \{1, \dots, K, \varnothing\} \to \mathbb{R}^D$ para $p_{\boldsymbol{x}|y}$ y $p_{\boldsymbol{x}}$ (entrada $\varnothing$ para $p_{\boldsymbol{x}}$).
Input: Funciones de escala y nivel de ruido $\alpha, \sigma \colon \{t_\ell\}_{\ell=0}^L \to \mathbb{R}_{\ge 0}$.
Input: Fuerza de guía $\gamma \ge 0$ (se recomienda $\gamma > 1$ para un mejor rendimiento).
Output: Una muestra $\hat{\boldsymbol{x}}$, aproximadamente de $p_{\boldsymbol{x}|y}(\,\cdot \mid \nu)$.

function DDIMSamplerConditionalCFG($\bar{\boldsymbol{x}}_\theta, \nu, \gamma, (t_\ell)_{\ell=0}^L$)
  Initializar $\hat{\boldsymbol{x}}_{t_L} \sim$ distribución aproximada de $\boldsymbol{x}_{t_L}$ (VP $\implies \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, VE $\implies \mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$).
  for $\ell = L, L-1, \dots, 1$ **do**
    Calcular

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} \doteq \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \hat{\boldsymbol{x}}_{t_\ell} + \left(\alpha_{t_{\ell-1}} - \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \alpha_{t_\ell} \right) \left((1-\gamma) \bar{\boldsymbol{x}}_\theta(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}, \varnothing) + \gamma \bar{\boldsymbol{x}}_\theta(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}, \nu) \right)$$

  end for
  return $\hat{\boldsymbol{x}}_{t_0}$
end function

Ho and Salimans [HS22a] presenta un sólido rendimiento empírico en la generación de imágenes condicionadas por clase con guía sin clasificador y se ha convertido en un pilar fundamental de los modelos de difusión prácticos a la mayor escala, como Stable Diffusion [RBL+22] y sus derivados.

¿Qué tipo de *denoiser* fomenta el aprendizaje con guía sin clasificador? Como hemos visto, la derivación de la guía sin clasificador es bastante opaca y de motivación empírica, lo que ofrece poca información sobre los mecanismos que subyacen a su buen rendimiento. Varios trabajos teóricos han intentado abordar esta cuestión [BN24b; LWQ25; WCL+24]. Ofrecen explicaciones sobre algunos aspectos de la metodología general de CFG, que abarca la parametrización y el entrenamiento del *denoiser*, así como la configuración de la intensidad de la guía y el rendimiento en el momento del muestreo. A continuación, siguiendo el hilo conductor del libro, daremos una interpretación en el contexto simplificado de una distribución de datos y un *denoiser* basados en un modelo de mezcla gaussiana. En primer lugar, consideramos *el efecto de CFG*: ¿qué promueve un peso grande $\gamma \gg 1$?

Ejemplo 7.5. Recordemos el proceso generador de datos de una mezcla gaussiana de rango bajo que estudiamos en el Ejemplo 3.3 (y, concretamente, la forma que aparece en la Ecuación (3.2.69)). Dadas $K \in \mathbb{N}$ clases, suponemos que

$$\boldsymbol{x} \sim \frac{1}{K}\sum_{k=1}^{K} \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top), \tag{7.4.17}$$

donde cada $\boldsymbol{U}_k \in \mathsf{O}(D, P) \subseteq \mathbb{R}^{D\times P}$ es una matriz con columnas ortogonales, y $P \ll D$. Además, suponemos que la etiqueta de clase $y \in [K]$ es una función determinista de $\boldsymbol{x}$ que asigna a cada ejemplo su componente de mezcla correspondiente. Consideramos el proceso de ruido simple (3.2.1) estudiado en el Capítulo 3, de modo que $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g}$ para todo $t \in [0, T]$, con $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$.

Al aplicar el análisis de Ejemplo 3.3 (y el análisis posterior del caso de rango bajo, que culmina en la Ecuación (3.2.80)), obtenemos para los eliminadores de ruido óptimos condicionados a la clase

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}, y = \nu] = \frac{1}{1+t^2}\boldsymbol{U}_\nu \boldsymbol{U}_\nu^\top \boldsymbol{\xi} \tag{7.4.18}$$

para cada $\nu \in [K]$ y para el eliminador de ruido incondicional óptimo, obtenemos

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}] = \frac{1}{1+t^2}\sum_{k=1}^{K} \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top \boldsymbol{\xi}\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{\xi}\|_2^2\right)} \boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{\xi}. \tag{7.4.19}$$

Por lo tanto, podemos expresar el *denoiser* CFG con una fuerza de guía $\gamma > 1$ como

$$\bar{\boldsymbol{x}}^{\mathrm{CFG,\,ideal}}(t, \boldsymbol{x}_t, y) = \frac{1}{1+t^2}\left((1-\gamma)\sum_{k=1}^{K} \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top \boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{x}_t\|_2^2\right)} \boldsymbol{U}_k \boldsymbol{U}_k^\top + \gamma \boldsymbol{U}_y \boldsymbol{U}_y^\top\right)\boldsymbol{x}_t. \tag{7.4.20}$$

Este filtro de eliminación de ruido tiene una forma sencilla e interpretable:

1. El primer término, correspondiente al *denoiser incondicional*, realiza el proceso de eliminación de ruido de la señal $\boldsymbol{x}_t$ usando como referencia un promedio de los *denoisers* asociados a cada subespacio, ponderado por el grado de correlación que existe entre $\boldsymbol{x}_t$ y cada subespacio.

2. El segundo término, correspondiente al *denoiser condicional*, simplemente realiza el proceso de eliminación de ruido utilizando el *denoiser* de la clase condicionante.

El esquema CFG promedia estos dos *denoisers*. El efecto de este promedio se puede deducir de la refactorización

$$\bar{\boldsymbol{x}}^{\text{CFG, ideal}}(t, \boldsymbol{x}_t, y) = \frac{1}{1+t^2}\left(\left[\gamma + (1-\gamma)\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_y^\top \boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^K \exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{x}_t\|_2^2\right)}\right]\boldsymbol{U}_y\boldsymbol{U}_y^\top + (1-\gamma)\sum_{k\neq y}\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top \boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^K \exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{x}_t\|_2^2\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top\right)\boldsymbol{x}_t, \tag{7.4.21}$$

donde se ha combinado el *denoiser* condicional con el término correspondiente a $k = y$ en la suma sobre $k \in [K]$. Se tiene que

$$\sum_{k=1}^K \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top \boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^K \exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{x}_t\|_2^2\right)} = 1, \tag{7.4.22}$$

y cada sumando es no negativo, por lo que también está acotado superiormente por 1. Por consiguiente, se pueden distinguir dos regímenes para los términos en Ecuación (7.4.21):

1. **Régimen bien correlacionado:** En este caso, supongamos que la entrada ruidosa $\boldsymbol{x}_t$ se correlaciona bien con uno de los subespacios $\boldsymbol{U}_y$. Si $\boldsymbol{x}_t$ se correlaciona bien con $\boldsymbol{U}_y$, entonces el peso normalizado correspondiente al sumando $k = y$ en el *denoiser* incondicional es cercano a 1. Entonces

$$\gamma + (1-\gamma)\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_y^\top \boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^K \exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{x}_t\|_2^2\right)} \approx 1, \tag{7.4.23}$$

   todos los demás pesos son necesariamente cercanos a cero, y el *denoiser* de CFG es aproximadamente igual al *denoiser* asociado a la clase condicionante y.

   Esto implica que: ¡*si la entrada del denoiser CFG está altamente correlacionada con uno de los subespacios de clase, el denoiser es aproximadamente igual al denoiser condicional de la clase correspondiente*! Por lo tanto, esperamos que el muestreador de eliminación de ruido CFG converja.

2. **Régimen con baja correlación:** Por el contrario, si $\boldsymbol{x}_t$ *no* se correlaciona bien con $\boldsymbol{U}_y$ (por ejemplo, porque t es grande), entonces el peso normalizado correspondiente al sumando $k = y$ en el *denoiser* incondicional es cercano a 0. Como resultado,

$$\gamma + (1-\gamma)\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_y^\top \boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^K \exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top \boldsymbol{x}_t\|_2^2\right)} \approx \gamma, \tag{7.4.24}$$

y, por lo tanto, la fuerza de guía $\gamma \gg 1$ otorga un gran peso positivo al *denoiser* asociado a y.

Por otra parte, en el segundo término de la Ecuación (7.4.21), cualquier clase $k \neq y$ que esté bien correlacionada con $\boldsymbol{x}_t$ recibe un peso *negativo* elevado debido al coeficiente $1-\gamma$. Esto tiene el efecto simultáneo de hacer que la señal sin ruido esté mucho más correlacionada con la clase de condicionamiento y y de hacer que esté negativamente correlacionada con el iterado anterior (es decir, la iteración previa a la eliminación del ruido).

En otras palabras, *CFG orienta el proceso iterativo de eliminación de ruido hacia la clase de condicionamiento y lo aleja de la iteración anterior*, una dinámica diferente a la del muestreo puramente condicional (es decir, el caso en el que $\gamma = 1$).

■

El Ejemplo 7.5 muestra que, en la configuración de mezcla de gaussianas, el *denoiser* CFG presenta un fuerte sesgo hacia la clase condicionante, al tiempo que conserva la corrección local (y, por lo tanto, la convergencia del muestreo) cuando el nivel de ruido t es pequeño. Esto sugiere por qué se ha considerado esencial en la práctica utilizar un peso grande $\gamma \gg 1$.

A continuación, el siguiente ejemplo mostrará una perspectiva de la *parametrización* del *denoiser* en presencia de una estructura de baja dimensión, nuevamente en el modelo de mezcla gaussiana.

Ejemplo 7.6. Volvamos a considerar el modelo de mezcla gaussiana estudiado en Ejemplo 7.5:

$$\boldsymbol{x} \sim \frac{1}{K}\sum_{k=1}^K \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k\boldsymbol{U}_k^\top), \tag{7.4.25}$$

donde cada $\boldsymbol{U}_k \in \mathsf{O}(D,P) \subseteq \mathbb{R}^{D\times P}$ es una matriz con columnas ortogonales y $P \ll D$. La etiqueta de clase $y \in [K]$ es una función determinista de $\boldsymbol{x}$ que asigna un ejemplo a su componente de mezcla correspondiente. Para esta distribución, y de nuevo bajo el proceso de ruido simple (3.2.1) estudiado en el Capítulo 3, donde $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g}$ para todo $t \in [0,T]$, con $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, vimos en

el Ejemplo 7.5 que el *denoiser* CFG ideal toma la forma

$$\bar{\boldsymbol{x}}^{\mathrm{CFG,\,ideal}}(t,\boldsymbol{x}_t,y)=\frac{1}{1+t^2}\left((1-\gamma)\sum_{k=1}^{K}\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top + \gamma\boldsymbol{U}_y\boldsymbol{U}_y^\top\right)\boldsymbol{x}_t. \tag{7.4.26}$$

Ahora consideramos el problema de parametrizar un *denoiser* aprendible $\boldsymbol{x}_\theta^{\mathrm{CFG}}$ para representar el *denoiser* óptimo (7.4.26). Por razones de tractabilidad, añadimos una suposición adicional relacionada con los subespacios $\boldsymbol{U}_k$ que sean «distinguibles» entre sí, lo cual es natural en la práctica: concretamente, suponemos que para cualquier par de índices $k,k'\in[K]$ con $k\neq k'$, podemos encontrar un conjunto de K direcciones distintas de cero $\boldsymbol{v}_k\in\mathbb{R}^D$ tales que

$$\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{v}_k=\boldsymbol{v}_k,\quad \boldsymbol{U}_{k'}\boldsymbol{U}_{k'}^\top\boldsymbol{v}_k=\boldsymbol{0},\ \ k'\neq k. \tag{7.4.27}$$

Esta es una suposición ligeramente más fuerte que la simple distinguibilidad, pero hay que señalar que no es excesivamente restrictiva: por ejemplo, sigue permitiendo que los subespacios $\boldsymbol{U}_k$ tengan correlaciones significativas entre sí.[9]

Estos vectores $\boldsymbol{v}_k$ pueden considerarse entonces como *embeddings* de la etiqueta de clase $y\in[K]$ y podemos utilizarlos para definir un operador más general capaz de representar tanto los *denoisers* incondicionales como los condicionales a la clase. Más concretamente, consideremos el mapeo

$$(\boldsymbol{x}_t,\boldsymbol{v})\mapsto\sum_{k=1}^{K}\frac{\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{v}|\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{U}_i\boldsymbol{U}_i^\top\boldsymbol{v}|\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t. \tag{7.4.28}$$

Entonces, si sustituimos $\boldsymbol{v}=\boldsymbol{x}_t$ en el operador (7.4.28), obtenemos un resultado proporcional al primer término de (7.4.19), que corresponde al *denoiser* incondicional óptimo para $\boldsymbol{x}_t$. Por otro lado, si sustituimos $\boldsymbol{v}=\boldsymbol{v}_y$ por algún $y\in[K]$, obtenemos por la construcción de los vectores de *embedding* $\boldsymbol{v}_k$

$$\begin{aligned}&\sum_{k=1}^{K}\frac{\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{v}_y|\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{U}_i\boldsymbol{U}_i^\top\boldsymbol{v}_y|\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t\\ =\ &\frac{\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{v}_y|\right)}{\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{v}_y|\right)+K-1}\boldsymbol{U}_y\boldsymbol{U}_y^\top\boldsymbol{x}_t &(7.4.29)\\ &+\sum_{k\neq y}\frac{1}{\exp\left(\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top\boldsymbol{v}_y|\right)+K-1}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t. &(7.4.30)\end{aligned}$$

[9] De forma más general, esta suposición se formula naturalmente como una *condición de incoherencia* entre los subespacios $\boldsymbol{U}_{[K]}$, conocida en la teoría del *compressive sensing* [WM22].

Nuestro objetivo es simplificar aún más la expresión anterior. Para ello, recordemos que, al utilizar este denoiser para el muestreo, tenemos $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g}$, donde y representa la etiqueta de $\boldsymbol{x}$. En otras palabras, condicionado a y, tenemos que $\boldsymbol{x}_t = \boldsymbol{U}_y\boldsymbol{z} + t\boldsymbol{g}$, donde $\boldsymbol{z}$ es un vector de ruido gaussiano independiente de dimensión P con media cero y covarianza identidad. Tenemos

$$\begin{aligned}\left|\boldsymbol{x}_t^\top \boldsymbol{v}_y\right| &= \left|\langle \boldsymbol{U}_y\boldsymbol{z}, \boldsymbol{v}_y\rangle + t\langle \boldsymbol{g}, \boldsymbol{v}_y\rangle\right| \\ &= \left|\langle \boldsymbol{z}, \boldsymbol{U}_y^\top \boldsymbol{v}_y\rangle + t\langle \boldsymbol{g}, \boldsymbol{v}_y\rangle\right|.\end{aligned}$$

Dado que $\boldsymbol{z}$ y $\boldsymbol{g}$ son variables aleatorias gaussianas independientes con matrices covarianza identidad, de la *invarianza rotacional* de la distribución gaussiana (véase [Ver18]) se deduce que

$$\begin{aligned}\left|\boldsymbol{x}_t^\top \boldsymbol{v}_y\right| &\overset{\mathrm{d}}{=} \left|\|\boldsymbol{U}_y^\top \boldsymbol{v}_y\|_2 z + t\|\boldsymbol{v}_y\|_2 g\right| \\ &= \left|\|\boldsymbol{v}_y\|_2 z + t\|\boldsymbol{v}_y\|_2 g\right|,\end{aligned}$$

donde z y g son variables aleatorias *escalares* independientes con distribución $\mathcal{N}(0,1)$ y $\overset{\mathrm{d}}{=}$ denota igualdad de distribución (las variables aleatorias tienen la misma distribución). La segunda línea anterior se deduce de la definición de $\boldsymbol{v}_y$. Ahora, por independencia, la variable aleatoria $z + tg \sim \mathcal{N}(0, (1+t^2))$. Entonces, utilizando el conocido resultado de que $\mathbb{E}[|g|] = \sqrt{2/\pi}$ si $g \sim \mathcal{N}(0,1)$, concluimos

$$\mathbb{E}\left[\left|\boldsymbol{x}_t^\top \boldsymbol{v}_y\right|\right] = \sqrt{(2/\pi)(1+t^2)}\|\boldsymbol{v}_y\|_2.$$

En la práctica, la cantidad $\left|\boldsymbol{x}_t^\top \boldsymbol{v}_y\right|$ se aproxima a su valor esperado *con una probabilidad abrumadora*, debido a la concentración gaussiana (véase, de nuevo, [Ver18]). Por otro lado, por un razonamiento similar tenemos que

$$\mathbb{E}\left[\|\boldsymbol{x}_t\|_2^2\right] = P + t^2 D,$$

y las propiedades de la distribución gaussiana implican nuevamente que $\|\boldsymbol{x}_t\|_2$ se concentrará en torno a $\sqrt{P + t^2 D}$ con una probabilidad abrumadora. Por lo tanto, es conveniente renormalizar los *embeddings* $\boldsymbol{v}_y$ de modo que su producto escalar con $\boldsymbol{x}_t$ tenga la misma escala: definiendo

$$\boldsymbol{v}_{t,y} = \frac{(P + t^2 D)}{\sqrt{(2/\pi)(1+t^2)}} \frac{\boldsymbol{v}_y}{\|\boldsymbol{v}_y\|_2}$$

para los *embeddings* normalizadas, nuestro razonamiento anterior implica

$$\left|\boldsymbol{x}_t^\top \boldsymbol{v}_{t,y}\right| \approx (P + t^2 D)$$

con probabilidad abrumadora. Volviendo a nuestro punto de partida, es decir, a (7.4.28) evaluada en $(\boldsymbol{x}_t, \boldsymbol{v}_y)$ y nuestra posterior simplificación, tenemos para el argumento «softmax»

$$\frac{1}{2t^2(1+t^2)}|\boldsymbol{x}_t^\top \boldsymbol{v}_y| \approx \frac{P + t^2 D}{2t^2(1+t^2)}$$

con probabilidad abrumadora con respecto a $\boldsymbol{x}_t$. Por lo tanto, si t es acotado y la dimensión del subespacio P es suficientemente grande,[10] La siguiente aproximación se cumple con probabilidad abrumadora:

$$\sum_{k=1}^{K} \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\boldsymbol{x}_t^\top \boldsymbol{U}_k\boldsymbol{U}_k^\top \boldsymbol{v}_y\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\boldsymbol{x}_t^\top \boldsymbol{U}_i\boldsymbol{U}_i^\top \boldsymbol{v}_y\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top \boldsymbol{x}_t \approx \boldsymbol{U}_y\boldsymbol{U}_y^\top \boldsymbol{x}_t. \tag{7.4.31}$$

En otras palabras, **al evaluar el operador** (7.4.28) **en el *embedding* de una clase $\boldsymbol{v}_{t,y}$ ¡se obtiene el *denoiser* condicionado a la clase para y!**

Por lo tanto, la familia de operadores (7.4.28) ofrece una manera unificada de parametrizar los operadores que componen el *denoiser* óptimo para $\boldsymbol{x}$ *dentro de una sola «red»*. Más concretamente, basta con sumar la salida de una instanciación de (7.4.28) con entrada $(\boldsymbol{x}_t, \boldsymbol{x}_t)$ a una instanciación con entrada $(\boldsymbol{x}_t, \boldsymbol{v}_{t,y})$. El operador resultante es una función de $(t, \boldsymbol{x}_t, y)$, y computacionalmente, los subespacios $(\boldsymbol{U}_k)_{k=1}^K$ y los *embeddings* $y \mapsto \boldsymbol{v}_{t,y}$ se convierten en sus parámetros aprendibles. ■

El Ejemplo 7.6 muestra que, en el caso especial de una distribución de datos de mezcla gaussiana de rango bajo para $\boldsymbol{x}$ con componentes incoherentes, los operadores de la forma

$$(\boldsymbol{x}_t, \boldsymbol{v}) \mapsto \sum_{k=1}^{K} \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\boldsymbol{x}_t^\top \boldsymbol{U}_k\boldsymbol{U}_k^\top \boldsymbol{v}\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\boldsymbol{x}_t^\top \boldsymbol{U}_i\boldsymbol{U}_i^\top \boldsymbol{v}\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top \boldsymbol{x}_t \tag{7.4.32}$$

proporcionar una clase de operadores lo suficientemente rica como para parametrizar el *denoiser* ideal para observaciones ruidosas $\boldsymbol{x}_t$ de $\boldsymbol{x}$ cuando se utiliza una guía sin clasificador. Para tales operadores, la entrada auxiliar $\boldsymbol{v}$ puede tomarse como $\boldsymbol{x}_t$ o como un *embedding* adecuado de la etiqueta de clase $y \mapsto \boldsymbol{v}_y$ con el fin de realizar dicho *denoiser*; tal y como muestra el ejemplo, el *denoiser* ideal es una suma ponderada de dichos operadores, con pesos derivados de la intensidad de la guía $\gamma > 1$.

Sobre la base del marco descrito en el Capítulo 5, que desarrolla arquitecturas de redes profundas adecuadas para transformar distribuciones de datos más generales en representaciones estructuradas utilizando el modelo de mezcla de gaussianas de bajo rango como primitiva, resulta natural imaginar que los operadores del tipo (7.4.32) puedan aprovecharse en *denoisers* para distribuciones de datos generales $\boldsymbol{x}$ con componentes de baja dimensión y con estructura geométrica que sean suficientemente distinguibles (digamos, incoherentes) entre sí. En la siguiente sección se demuestra que efectivamente es así.

[10] Por ejemplo, basta considerar un régimen asintótico a gran escala, en que $P, D \to \infty$ y su relación P/D converge hacia una constante fija. En tal escenario, la *relación de aspecto* de los subespacios es fija mientras que sus tamaños tienden a infinito, lo que puede servir como un modelo para algunos fenómenos de discretización (p. ej., una escena fija muestreada para producir una imagen con el número de píxeles tendiendo a infinito).

7.4.2 Generación de imágenes condicionada por pies de imagen

En la Sección 7.4.1 hemos visto, a nivel conceptual, cómo configurar los *denoisers* para su uso con *la guía sin clasificador*, la metodología práctica predominante para realizar muestreos condicionales mediante el paradigma de la difusión con eliminación de ruido. En resumen, estos filtros de eliminación de ruido toman como entrada los datos ruidosos $\boldsymbol{x}_t$, así como ya sea la señal de condicionamiento $\boldsymbol{y}$ o una «entrada nula» $\varnothing$, lo que determina si el filtro debe realizar una *eliminación de ruido incondicional* o una *eliminación de ruido condicional* (condicionada a $\boldsymbol{y}$).

Dado este diseño, la pregunta práctica más importante es: *¿cómo implementar esta familia de operadores como una red neuronal?* Una arquitectura de capas conocida como de «atención cruzada», cuya eficacia ha sido validada empíricamente y que presentaremos más adelante, ofrece una respuesta práctica a esta pregunta. Sin embargo, incluso antes de eso, hay varias cuestiones más fundamentales que debemos resolver:

1. *Desajuste de modalidades de entrada*: En las aplicaciones prácticas más interesantes, los datos $\boldsymbol{x}$ y la señal de condicionamiento $\boldsymbol{y}$ *residen en espacios que no son directamente comparables*, aunque comparten muchas relaciones estructurales subyacentes. Por ejemplo, lo vimos en el caso de una etiqueta de clase y en Sección 7.4.1. En términos más generales, se podría considerar una descripción en lenguaje natural $\boldsymbol{y}$ de una imagen $\boldsymbol{x}$ (denominada «pie de imagen»), donde $\boldsymbol{y}$ correspondería a una secuencia de caracteres de diferentes alfabetos.

2. *Parámetrización y flexibilidad*: Más allá de la cuestión de la modalidad de entrada, se plantea una cuestión fundamental: si se puede utilizar la misma arquitectura básica de la red para diferentes modalidades, o si es necesario ajustarla de forma muy precisa para cada modalidad.

3. *Entrenamiento*: Esto implica la forma adecuada de utilizar nuestras muestras emparejadas $(\boldsymbol{x}, \boldsymbol{y})$ para entrenar e implementar los *denoisers* condicionales. El espacio de diseño es vasto: se podría imaginar la aplicación de diversas arquitecturas de codificador-decodificador específicas para cada modalidad, tal como las hemos desarrollado en los Capítulos 4 y 6, junto con diferentes estructuras de base de eliminación de ruido en diversas combinaciones. Las cuestiones clave aquí son la *expresividad*, es decir, obtener el mejor rendimiento en la eliminación de ruido condicional; la *eficiencia*, es decir, obtener el mejor rendimiento por unidad de datos de entrenamiento; y la *estabilidad*, es decir, garantizar que el proceso de entrenamiento converja rápidamente sin colapsar.

A continuación abordaremos estos tres temas en detalle, ofreciendo orientación sobre cómo explorar el espacio de diseño y, cuando sea posible, directrices de uso general.

Incrustación de entradas independiente de la modalidad

Aunque no conocemos la relación exacta entre $\boldsymbol{x}$ y $\boldsymbol{y}$ en el contexto general de datos emparejados, en todos los contextos prácticos de interés, existen correlaciones significativas entre ellos. Por ejemplo, cuando la señal de condicionamiento $\boldsymbol{y}$ es un «pie de foto» que describe la imagen $\boldsymbol{x}$ en lenguaje natural, es fácil imaginar que un artista humano sea capaz de producir un boceto preciso de $\boldsymbol{x}$ basándose únicamente en un pie de imagen de alta calidad. En nuestro marco probabilístico, una forma natural de cuantificar estas «correlaciones» es mediante la información mutua, que ya vimos anteriormente en el Capítulo 4 (Ecuación (4.1.12)):

$$I(\boldsymbol{x};\boldsymbol{y}) = h(\boldsymbol{x}) - h(\boldsymbol{x} \mid \boldsymbol{y}). \tag{7.4.33}$$

La información mutua (7.4.33) es una magnitud puramente de la teoría de la información que no depende de aspectos específicos de la modalidad de $\boldsymbol{x}$ y $\boldsymbol{y}$. Por lo tanto, ofrece un criterio natural para aprender una representación útil de cualquiera de las entradas $\boldsymbol{x}$ o $\boldsymbol{y}$: simplemente se busca aprender, por ejemplo para $\boldsymbol{x}$, una representación $\boldsymbol{z} = f(\boldsymbol{x})$ que preserve la información mutua, de modo que $I(\boldsymbol{z};\boldsymbol{y}) \approx I(\boldsymbol{x};\boldsymbol{y})$. Es decir:

$$\text{Aprender } \boldsymbol{z} = f(\boldsymbol{x}) \text{ tal que } I(\boldsymbol{z};\boldsymbol{y}) \approx I(\boldsymbol{x};\boldsymbol{y}). \tag{7.4.34}$$

De hecho, podemos ser aún más explícitos en lo prescriptivo. Para un codificador f que sea determinista (que abarca todos los casos que hemos estudiado a lo largo de los Capítulos 4 y 6), un resultado fundamental en teoría de la información conocido como la *desigualdad del procesamiento de datos* nos dice que procesar $\boldsymbol{x}$ con f para producir $\boldsymbol{z} = f(\boldsymbol{x})$ cambia la información mutua de una manera predecible: *solo puede reducirla.*

Teorema 7.1 (Desigualdad del procesamiento de datos)**.** *Consideremos las variables aleatorias $\boldsymbol{y}, \boldsymbol{x}, \boldsymbol{z}$ definidas de tal manera que $\boldsymbol{z}$ es independiente de $\boldsymbol{y}$ dado $\boldsymbol{x}$.*[11] *Entonces se cumple la desigualdad de procesamiento de datos:*

$$I(\boldsymbol{x};\boldsymbol{y}) \geq I(\boldsymbol{z};\boldsymbol{y}). \tag{7.4.35}$$

Demostración. Consideremos la información mutua $I(\boldsymbol{y};\boldsymbol{x},\boldsymbol{z})$ entre $\boldsymbol{y}$ y $(\boldsymbol{x},\boldsymbol{z})$. Dado que $\boldsymbol{z}$ es independiente de $\boldsymbol{y}$ condicionado a $\boldsymbol{x}$, tenemos

$$h(\boldsymbol{y} \mid \boldsymbol{x},\boldsymbol{z}) = h(\boldsymbol{y} \mid \boldsymbol{x}),$$

por lo que $I(\boldsymbol{y};\boldsymbol{x},\boldsymbol{z}) = I(\boldsymbol{y};\boldsymbol{x})$. Por otra parte, según la regla de Bayes tenemos que

$$p_{\boldsymbol{y}|\boldsymbol{x},\boldsymbol{z}} = p_{\boldsymbol{y}|\boldsymbol{z}} \frac{p_{\boldsymbol{y},\boldsymbol{x},\boldsymbol{z}}}{p_{\boldsymbol{y},\boldsymbol{z}} p_{\boldsymbol{x}|\boldsymbol{z}}},$$

lo cual implica, por la definición de la entropía diferencial condicional, que

$$I(\boldsymbol{y};\boldsymbol{x},\boldsymbol{z}) = I(\boldsymbol{y};\boldsymbol{z}) - \mathbb{E}_{\boldsymbol{x},\boldsymbol{y},\boldsymbol{z}}\left[\log\left(\frac{p_{\boldsymbol{x}|\boldsymbol{z}} p_{\boldsymbol{y},\boldsymbol{z}}}{p_{\boldsymbol{y},\boldsymbol{x},\boldsymbol{z}}}\right)\right].$$

[11] En otras palabras, las variables aleatorias forman una cadena de Markov $\boldsymbol{y} \to \boldsymbol{x} \to \boldsymbol{z}$.

Si aplicamos la desigualdad de Jensen a la esperanza de la línea anterior, esto implica la afirmación, ya que log es una función cóncava (y la información mutua es simétrica, es decir, $I(\boldsymbol{x};\boldsymbol{y}) = I(\boldsymbol{y};\boldsymbol{x})$). □

Para un codificador determinista f, $\boldsymbol{z} = f(\boldsymbol{x})$ es siempre independiente de $\boldsymbol{y}$ cuando se condiciona en $\boldsymbol{x}$. Por lo tanto, la desigualdad de procesamiento de datos sugiere el siguiente procedimiento computacional para hallar el codificador f: *buscar maximizar la información mutua entre la representación y la señal de condicionamiento.* La Ecuación (7.4.34) conduce entonces al objetivo

$$\max_{f} I(f(\boldsymbol{x});\boldsymbol{y}). \tag{7.4.36}$$

Esto se conoce como el *principio de Infomax* [Lin88], y se remonta a los inicios del aprendizaje de representaciones.

Consideraciones computacionales en la aplicación del principio de Infomax. Hay dos cuestiones conceptuales clave que hay que tener en cuenta al pensar en cómo aplicar el principio (abstracto) de Infomax (7.4.36) para el aprendizaje de una representación. La primera es que, en general, resulta computacionalmente costoso estimar la información mutua, que constituye el objetivo en (7.4.36). Como ya comentamos en el contexto de la relación tasa-distorsión y la codificación con pérdida al presentar la información mutua en el Capítulo 4, la información mutua puede relajarse para seguir estando bien definida incluso cuando $\boldsymbol{x}$ y $\boldsymbol{y}$ son de baja dimensión y tienen una entropía diferencial que se aproxima a $-\infty$ (por ejemplo, mediante la conexión con la función gaussiana de reducción de tasa que hemos estudiado en los Capítulos 4 a 6). Además, desde el punto de vista empírico, estas relajaciones presentan, en la práctica, propiedades estadísticas **lejos del peor caso**, tal como observamos en los experimentos del Capítulo 4; y la precisión de estas aproximaciones puede mejorarse mediante el procesamiento incremental, tal como vimos en el Capítulo 5. Esto hace que un criterio como el de la Ecuación (7.4.36) sea un fundamento excelente para aprender representaciones de datos emparejados $(\boldsymbol{x}, \boldsymbol{y})$.

La segunda cuestión que hay que tener en cuenta es que, para que el objetivo (7.4.36) constituya un criterio válido para el aprendizaje de f, es necesario que f esté parametrizada de una manera específica y/o que se apliquen algunas restricciones estructurales adicionales a las representaciones $f(\boldsymbol{x})$. De hecho, si f no tiene ninguna restricción, existen soluciones triviales simples para (7.4.36) que no corresponden a representaciones útiles: por ejemplo, ¡basta con tomar $f(\boldsymbol{x}) = \boldsymbol{x}$ como la identidad! Analizamos técnicas basadas en principios para estructurar el espacio de representaciones en el Capítulo 6 cuando tratamos el tema de la autocodificación y la transcripción en bucle cerrado; muchas de estas técnicas pueden aplicarse a nuestro actual contexto de *embedding* conjunto y lo detallaremos más adelante en esta sección, cuando pasemos a la implementación práctica.

Incrustación conjunta con el principio de Infomax. Ahora queda por resolver la cuestión de la representación *conjunta* de $(\boldsymbol{x}, \boldsymbol{y})$. Al aplicar una vez más la desigualdad de procesamiento de datos, obtenemos para otro codificador g que produce una representación de $\boldsymbol{y}$, que

$$I(\boldsymbol{x}; \boldsymbol{y}) \geq I(f(\boldsymbol{x}); \boldsymbol{y}) \geq I(f(\boldsymbol{x}); g(\boldsymbol{y})).$$

El objetivo conjunto

$$\max_{f,g} I(f(\boldsymbol{x}); g(\boldsymbol{y})) \tag{7.4.37}$$

es, por lo tanto, una consecuencia bien fundamentada del principio de Infomax.

Para diseñar los codificadores f y g, recordemos la intuición que obtuvimos a través de nuestro trabajo en el Ejemplo 7.6. Allí vimos que, en el caso de un modelo de mezcla de gaussianas para $\boldsymbol{x}$, con la señal de condicionamiento y identificando qué componente específica de la mezcla genera la observación, se podría construir una arquitectura flexible de *denoiser* mediante un *embedding* adecuado de la etiqueta de clase y en el *mismo espacio* que los datos $\boldsymbol{x}$. Más concretamente, un *embedding* de este tipo $\boldsymbol{u}_y \in \mathbb{R}^D$ permite, cuando se correlaciona con los *embeddings* $\boldsymbol{U}_{\boldsymbol{x}}$ de $\boldsymbol{x} \in \mathbb{R}^D$ (generalmente una matriz) mediante $\boldsymbol{u}_y^\top \boldsymbol{U}_{\boldsymbol{x}}$, «seleccionar» únicamente los *embeddings* relevantes para la componente de mezcla correcta de la distribución. Cómo diseñar exactamente estas capas para su uso en redes neuronales que se entrenan con datos no lineales de alta dimensión más complejos es una cuestión difícil que profundizaremos con más detalle a continuación. Pero hay una condición necesaria que está clara: los *embeddings* f y g deben mapear sus entradas a un *espacio de embedding común*, por ejemplo $\mathbb{R}^d$, para facilitar la comparación posterior al realizar el muestreo condicional. Esto nos lleva al objetivo

$$\max_{f:\mathbb{R}^{D_{\boldsymbol{x}}} \to \mathbb{R}^d, g:\mathbb{R}^{D_{\boldsymbol{y}}} \to \mathbb{R}^d} I(f(\boldsymbol{x}); g(\boldsymbol{y})). \tag{7.4.38}$$

El siguiente paso consiste en instanciar una relajación adecuada y computacionalmente eficiente de la información mutua en (7.4.38) para su uso con conjuntos de datos a gran escala.

Guía práctica para el aprendizaje de representaciones conjuntas: CLIP

Una implementación concreta y muy influyente del aprendizaje de *embeddings* conjuntos para datos emparejados es el método de preentrenamiento contrastivo de lenguaje e imagen (CLIP) [RKH+21b]. Tomemos el caso simple de pares de imagen-texto $[(\boldsymbol{x}_i, \boldsymbol{y}_i)]_{i=1}^N$ muestreados de un conjunto de datos $\mathcal{D}$, donde $\boldsymbol{x}_i$ es una imagen y $\boldsymbol{y}_i$ es un pie de imagen. CLIP entrena dos codificadores: un codificador de imágenes $f_\theta(\boldsymbol{x})$ y un codificador de texto $g_\phi(\boldsymbol{y})$ para mapear muestras emparejadas en una esfera unitaria compartida de d dimensiones (es decir, $\|f_\theta(\boldsymbol{x}_i)\|_2 = 1$ y $\|g_\phi(\boldsymbol{y}_i)\|_2 = 1$). Para lograrlo, podemos adoptar una pérdida de entropía cruzada simétrica sencilla que acerque las representaciones

de los pares coincidentes $(f_\theta(\boldsymbol{x}_i), g_\phi(\boldsymbol{y}_i))$ en términos de similitud coseno, al tiempo que repele a los pares no coincidentes:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N}\mathbb{E}_{[(\boldsymbol{x}_i,\boldsymbol{y}_i)]_{i=1}^N\sim\mathcal{D}}\left[\sum_{i=1}^N \log\frac{\exp\left(f_\theta(\boldsymbol{x}_i)^\top g_\phi(\boldsymbol{y}_i)\right)}{\sum_{j=1}^N \exp\left(f_\theta(\boldsymbol{x}_i)^\top g_\phi(\boldsymbol{y}_j)\right)} + \sum_{i=1}^N \log\frac{\exp\left(g_\phi(\boldsymbol{y}_i)^\top f_\theta(\boldsymbol{x}_i)\right)}{\sum_{j=1}^N \exp\left(g_\phi(\boldsymbol{y}_i)^\top f_\theta(\boldsymbol{x}_j)\right)}\right], \quad (7.4.39)$$

Desde la perspectiva de la maximización de la información mutua, CLIP es un sustituto práctico y manejable de la cota inferior de la información mutua entre las representaciones de la imagen y del texto:[12]

$$I\big(f_\theta(\boldsymbol{x}), g_\phi(\boldsymbol{y})\big) \geq -\mathcal{L}_{\text{CLIP}} + \log N. \quad (7.4.40)$$

Por lo tanto, CLIP ofrece una receta práctica para aprender representaciones conjuntas de datos emparejados que aumenta la información mutua entre las muestras emparejadas. Empíricamente, se ha demostrado que CLIP es eficaz para aprender *embeddings* conjuntos para pares de imagen-texto y se ha utilizado en una variedad de aplicaciones, incluyendo la generación de leyendas de imágenes, la recuperación de imagen-texto, la generación de imágenes y más. Veremos ejemplos más específicos en el Capítulo 8 (con detalles clave de implementación descritos en detalle en la Sección 8.3).

Aplicación a la generación de imágenes condicionadas por pies de imagen

Por último, describiremos un sistema de extremo a extremo que integra codificadores conjuntos de imagen y texto, entrenados con la pérdida CLIP tal y como se ha descrito anteriormente, con operadores de condicionamiento que recuerdan al operador de mezcla de gaussianas (7.4.32) que vimos en el Ejemplo 7.6. Estas técnicas constituyeron la base de la implementación original de código abierto de Stable Diffusion [RBL+22], en la que el *embedding* y el condicionamiento posterior no se realizan sobre una etiqueta de clase, sino sobre un *prompt* de texto que describe el contenido deseado de la imagen (Figura 7.14). En esta sección se ofrece una visión general de alto nivel sobre la generación de imágenes al estilo de Stable Diffusion. Para obtener todos los detalles de implementación necesarios para volver a implementar dicho sistema desde cero, consulte el Capítulo 8: en concreto, la Sección 8.11 describe el proceso de codificación de cadenas de texto en una secuencia de identificadores enteros; la Sección 8.3 describe el entrenamiento de los codificadores de imagen-texto necesarios; y las Secciones 8.6 y 8.7 describen el entrenamiento del modelo de difusión (latente).

[12] Este resultado se cumple bajo el supuesto de que los pares $(\boldsymbol{x}_i, \boldsymbol{y}_i)$ son independientes e idénticamente distribuidos dentro de cada lote de N [OLC+25, Lema 1]. La estrechez de la cota aumenta a medida que aumenta N [OLV18]. Sin embargo, para ciertas distribuciones de datos, puede seguir siendo laxa para todo N [MS19; WI20].

Stable Diffusion sigue la metodología de generación condicional que describimos en Sección 7.4.1, con dos modificaciones clave: (i) La señal de condicionamiento es un prompt de texto tokenizado $\boldsymbol{Y} \in \mathbb{R}^{D_{\text{text}} \times N}$, en lugar de una etiqueta de clase; (ii) La eliminación del ruido de la imagen se realiza en el espacio «latente» en lugar de sobre los píxeles sin procesar, utilizando un par de autocodificadores variacionales especializados y preentrenados $f : \mathbb{R}^{D_{\text{img}}} \to \mathbb{R}^{d_{\text{img}}}$, $g : \mathbb{R}^{d_{\text{img}}} \to \mathbb{R}^{D_{\text{img}}}$ (véase Secciones 6.1.4 y 8.6), donde f es el codificador y g es el decodificador. En cuanto al punto (i), en el contexto del marco de eliminación de ruido condicional iterativo que hemos desarrollado en Sección 7.4.1, esto se refiere a la parametrización de los eliminadores de ruido $\bar{\boldsymbol{z}}_\theta(t, \boldsymbol{z}_t, \boldsymbol{Y}^+)$.[13] Rombach et al. [RBL+22] implementan el condicionamiento de texto en el *denoiser* utilizando una capa conocida como «atención cruzada», inspirada en la arquitectura original de transformer codificador-decodificador de Vaswani et al. [VSP+17b]. La atención cruzada se implementa de la siguiente manera. Sea $\tau : \mathbb{R}^{D_{\text{text}} \times N} \to \mathbb{R}^{d_{\text{model}} \times N_{\text{text}}}$ una red de codificación para los *embeddings* de texto (a menudo un transformer causal — véase la Sección 8.11), y sea $\psi : \mathbb{R}^{d_{\text{img}}} \to \mathbb{R}^{d_{\text{model}} \times N_{\text{img}}}$ la correspondencia de una de las representaciones intermedias en el *denoiser*.[14] Aquí, N_{text} es la longitud máxima de la solicitud de texto tokenizada, y N_{img} se corresponde aproximadamente con el número de canales de imagen (dependientes de la capa) en la representación, que es fijo si la resolución de la imagen de entrada es fija. La atención cruzada (con K cabezales y sin sesgo) se define como

$$\text{MHCA}(\boldsymbol{z}_t, \boldsymbol{Y}^+) = \boldsymbol{U}_{\text{out}} \begin{bmatrix} \text{SA}([\boldsymbol{U}^1_{\text{qry}}]^\top \psi(\boldsymbol{z}_t), [\boldsymbol{U}^1_{\text{key}}]^\top \tau(\boldsymbol{Y}^+), [\boldsymbol{U}^1_{\text{val}}]^\top \tau(\boldsymbol{Y}^+)) \\ \vdots \\ \text{SA}([\boldsymbol{U}^K_{\text{qry}}]^\top \psi(\boldsymbol{z}_t), [\boldsymbol{U}^K_{\text{key}}]^\top \tau(\boldsymbol{Y}^+), [\boldsymbol{U}^K_{\text{val}}]^\top \tau(\boldsymbol{Y}^+)) \end{bmatrix}, \tag{7.4.41}$$

donde SA denota la operación de atención de producto escalar en el transformer puesta a escala (que recordamos en detalle en Capítulo 8: véase Ecuaciones (8.2.17) y (8.2.18)), y $\boldsymbol{U}^k_* \in \mathbb{R}^{d_{\text{model}} \times d_{\text{attn}}}$ para $* \in \{\text{qry}, \text{key}, \text{val}\}$ (así como la proyección de salida $\boldsymbol{U}_{\text{out}}$) son los parámetros aprendibles de la capa.

Obsérvese que, según la definición de la operación de autoatención, la operación de atención cruzada *genera combinaciones lineales de las proyecciones de valor de los embeddings de texto, ponderadas por las correlaciones entre las características de la imagen y los embeddings de texto.* En la arquitectura del *denoiser* utilizada por Rombach et al. [RBL+22], los bloques residuales de autoatención de la arquitectura del *denoiser*, aplicados a la representación

[13] En el contexto del condicionamiento de texto, la etiqueta «aumentada» $\boldsymbol{Y}^+$, que puede ser la solicitud de texto codificada o $\varnothing$ (lo que denota un proceso de eliminación de ruido incondicional), suele implementarse asignando $\varnothing$ a la cadena vacía «» y luego codificando esta solicitud de texto con el tokenizador como de costumbre. Esto proporciona una forma sencilla y unificada de tratar la eliminación de ruido condicional e incondicional con el acondicionamiento de texto.

[14] En la práctica, los *denoisers* condicionados por texto añaden capas de atención cruzada a intervalos regulares dentro de la pasada hacia adelante del *denoiser*, por lo que ψ debe considerarse dependiente de la capa, a diferencia de τ.

de la imagen en la capa actual y definidos de manera análoga a los de Ecuación (8.2.15), correspondientes al Transformer de Visión, van seguidos de bloques residuales de atención cruzada de la forma (7.4.41). Una estructura de este tipo requiere que el codificador de texto τ comparta, en cierto sentido, parte de la estructura de su salida con el *embedding* de características de la imagen ψ: esto se puede lograr mediante un *embedding* conjunto, aprendida mediante un procedimiento como CLIP, tal y como se ha descrito anteriormente. Desde el punto de vista conceptual, este espacio de *embedding* conjunto de texto e imagen y la propia capa de atención cruzada presentan un gran parecido con el *denoiser* de mezcla condicional de gaussianas que derivamos en la sección anterior (recuérdese (7.4.28)), en el caso particular de una secuencia de un solo token. Se pueden establecer conexiones más profundas en el contexto de múltiples tokens siguiendo el marco de reducción de tasa para derivar arquitecturas de redes profundas que se analiza en el Capítulo 5 y que se manifiesta en la derivación de la arquitectura CRATE, similar a la del transformer.

Este mismo diseño básico se ha ampliado aún más a modelos y conjuntos de datos de mayor tamaño, en particular en las versiones modernas de Stable Diffusion [EKB+24], así como en modelos de la competencia como FLUX.1 [LBB+25], Imagen [SCS+22] y DALL-E [RDN+22]. El mecanismo de condicionamiento de la atención cruzada también se ha generalizado en otras aplicaciones, como en EgoAllo (Sección 8.10) para la generación condicionada de poses y en Michelangelo (Sección 8.8) para la generación condicional de formas 3D basada en imágenes o textos.

7.5 Inferencia condicional con autoconsistencia de la medición

En esta última sección, consideramos el caso más extremo, aunque en realidad omnipresente, del aprendizaje de distribuciones, en el que solo contamos con un conjunto de muestras observadas $\boldsymbol{Y} = \{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_N\}$ de los datos $\boldsymbol{x}$, ¡pero no tenemos muestras directas de $\boldsymbol{x}$! En general, la observación $\boldsymbol{y} \in \mathbb{R}^d$ es de menor dimensión que $\boldsymbol{x} \in \mathbb{R}^D$. Para que el problema esté bien definido, de hecho, asumimos que se sabe que el modelo de observación entre $\boldsymbol{y}$ y $\boldsymbol{x}$ pertenece a una cierta familia de modelos analíticos, denotada como $\boldsymbol{y} = h(\boldsymbol{x}, \theta) + \boldsymbol{w}$, donde θ puede ser conocido o desconocido.

Intentemos primero comprender el problema conceptualmente con el caso sencillo en el que la función de medición h es conocida y la observación $\boldsymbol{y} = h(\boldsymbol{x}) + \boldsymbol{w}$ proporciona información sobre $\boldsymbol{x}$. Es decir, suponemos que h es sobreyectiva del espacio de $\boldsymbol{x}$ al de $\boldsymbol{y}$ y que el soporte de la distribución $\boldsymbol{y}_0 = h(\boldsymbol{x}_0)$ es de baja dimensión. Esto suele requerir que la dimensión *extrínseca* d de $\boldsymbol{y}$ sea mayor que la dimensión *intrínseca* del soporte de la distribución de $\boldsymbol{x}$. Sin pérdida de generalidad, podemos suponer que existen funciones:

$$F(\boldsymbol{x}) = \boldsymbol{0}, \quad G(\boldsymbol{y}) = \boldsymbol{0}. \tag{7.5.1}$$

Obsérvese que aquí podemos suponer que $G(\boldsymbol{y})$ es conocida, pero $F(\boldsymbol{x})$ no lo es. Sea $\mathcal{S}_{\boldsymbol{y}} \doteq \{\boldsymbol{y} \mid G(\boldsymbol{y}) = \boldsymbol{0}\}$ el soporte de $p(\boldsymbol{y})$. En general, $h^{-1}(\mathcal{S}_{\boldsymbol{y}}) = \{\boldsymbol{x} \mid G(h(\boldsymbol{x})) = \boldsymbol{0}\}$ es un superconjunto de $\mathcal{S}_{\boldsymbol{x}} \doteq \{\boldsymbol{x} \mid F(\boldsymbol{x}) = \boldsymbol{0}\}$. Es decir, tenemos $h(\mathcal{S}_{\boldsymbol{x}}) \subseteq \mathcal{S}_{\boldsymbol{y}}$.

7.5.1 Modelos lineales de medición

En primer lugar, para simplificar, supongamos que la medición es una función lineal de los datos $\boldsymbol{x}$ que nos interesan:

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}. \tag{7.5.2}$$

Aquí, la matriz $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ tiene rango de filas completo y m suele ser menor que n. Suponemos que $\boldsymbol{A}$ es conocida por ahora. Nos interesa cómo aprender la distribución de $\boldsymbol{x}$ a partir de dichas mediciones. Dado que ya no contamos con muestras directas de $\boldsymbol{x}$, nos preguntamos si aún podemos desarrollar un *denoiser* para $\boldsymbol{x}$ con las observaciones $\boldsymbol{y}$. Consideremos el siguiente proceso de difusión:

$$\boldsymbol{y}_t = \boldsymbol{y}_0 + t\boldsymbol{g}, \quad \boldsymbol{y}_0 = \boldsymbol{A}(\boldsymbol{x}_0), \tag{7.5.3}$$

donde $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$.

Sin perder generalidad, supongamos que $\boldsymbol{A}$ tiene rango de fila completo, es decir, que es subdeterminada. Definamos el proceso correspondiente $\boldsymbol{x}_t$ como aquel que satisface:

$$\boldsymbol{y}_t = \boldsymbol{A}\boldsymbol{x}_t. \tag{7.5.4}$$

Del proceso de eliminación de ruido de $\boldsymbol{y}_t$, tenemos

$$\boldsymbol{y}_{t-s} \approx \boldsymbol{y}_t + st\nabla \log p_t(\boldsymbol{y}_t). \tag{7.5.5}$$

Luego tenemos:

$$\boldsymbol{A}\boldsymbol{x}_{t-s} \approx \boldsymbol{A}\boldsymbol{x}_t + st\nabla \log p_t(\boldsymbol{A}\boldsymbol{x}_t), \tag{7.5.6}$$

para un valor pequeño de $s > 0$. Por lo tanto, $\boldsymbol{x}_{t-s}$ y $\boldsymbol{x}_t$ deben cumplir que:

$$\boldsymbol{A}(\boldsymbol{x}_{t-s} - \boldsymbol{x}_t) \approx st\nabla \log p_t(\boldsymbol{A}\boldsymbol{x}_t). \tag{7.5.7}$$

Entre todos los $\boldsymbol{x}_{t-s}$ que satisfacen la restricción anterior, se selecciona arbitrariamente aquel que minimiza la distancia $\|\boldsymbol{x}_{t-s} - \boldsymbol{x}_t\|_2^2$. Por lo tanto, obtenemos un proceso de «eliminación de ruido» para $\boldsymbol{x}_t$:

$$\boldsymbol{x}_{t-s} \approx \boldsymbol{x}_t + st\boldsymbol{A}^\dagger \nabla \log p_t(\boldsymbol{A}\boldsymbol{x}_t). \tag{7.5.8}$$

Obsérvese que este proceso no toma muestras de la distribución de $\boldsymbol{x}_t$. En particular, hay componentes de $\boldsymbol{x}$ en el espacio o núcleo nulo de $\boldsymbol{A}$ que nunca pueden recuperarse a partir de las observaciones. Por lo tanto, en sentido estricto, se necesita más información para recuperar la distribución completa de $\boldsymbol{x}$. Sin embargo, esto permite recuperar la componente de $\boldsymbol{x}$ que es ortogonal al espacio nulo de $\boldsymbol{A}$.

7.5.2 Aprendizaje de un modelo del mundo en 3D a partir de imágenes 2D

En la práctica, el modelo de medición suele ser no lineal o solo se conoce parcialmente. Un problema típico de este tipo es, de hecho, lo que subyace a la forma en que podemos aprender un modelo funcional del mundo exterior a partir de las imágenes que percibimos, por ejemplo, a través de nuestros ojos, telescopios o microscopios. En particular, los seres humanos y los animales son capaces de construir un modelo del mundo 3D (o 4D, en el caso de un mundo dinámico) a través de una secuencia de sus proyecciones en 2D —una secuencia de imágenes en 2D (o pares de imágenes estereoscópicas). El modelo matemático o geométrico de la proyección es generalmente conocido:

$$\boldsymbol{y}^i = h(\boldsymbol{x}, \theta^i) + \boldsymbol{w}^i, \tag{7.5.9}$$

donde $h(\cdot)$ representa una proyección (en perspectiva) de la escena 3D (o 4D) desde una determinado punto de vista de la cámara en el instante t_i hacia una imagen 2D (o un par estereoscópico) y $\boldsymbol{w}$ es un pequeño ruido de medición, posiblemente aditivo. La Figura 7.15 ilustra esta relación de manera concreta, mientras que la Figura 7.16 ilustra el problema del modelo de manera abstracta. Una exposición completa de la geometría relacionada con las múltiples vistas en 2D de una escena en 3D excede el alcance de este libro. Los lectores interesados pueden consultar el libro [MKS+04]. Por ahora, todo lo que necesitamos para continuar es que se comprendan bien dichas proyecciones y que las múltiples imágenes de una escena contengan información suficiente sobre la misma.

En general, nos gustaría aprender la distribución $p(\boldsymbol{x})$ de la escena del mundo en 3D (o 4D) $\boldsymbol{x}$[15] a partir de las imágenes 2D del mundo percibidas hasta el momento. La función principal de un modelo (visual) del mundo de este tipo es permitirnos reconocer lugares en los que ya hemos estado o predecir cómo se vería la escena actual en un momento futuro desde un nuevo punto de vista.

Examinemos primero el caso especial, pero importante, de la visión estereoscópica. En este caso, contamos con dos vistas calibradas de la escena 3D $\boldsymbol{x}$:

$$\boldsymbol{y}^0 = h(\boldsymbol{x}, \theta^0) + \boldsymbol{w}^0, \quad \boldsymbol{y}^1 = h(\boldsymbol{x}, \theta^1) + \boldsymbol{w}^1, \tag{7.5.10}$$

donde se puede suponer que los parámetros θ^0 y θ^1 de las poses de la vista son conocidos. $\boldsymbol{y}^0$ y $\boldsymbol{y}^1$ son dos proyecciones en 2D de la escena 3D $\boldsymbol{x}$. También podemos suponer que tienen la misma distribución marginal $p(\boldsymbol{y})$ y que hemos aprendido un modelo de difusión y eliminación de ruido para esta. Es decir, conocemos el *denoiser*:

$$\mathbb{E}[\boldsymbol{y} \mid \boldsymbol{y}_t = \boldsymbol{\nu}] = \boldsymbol{\nu} + t^2 \nabla_{\boldsymbol{\nu}} \log p_t(\boldsymbol{\nu}). \tag{7.5.11}$$

O, incluso, podemos suponer que contamos con un número suficiente de muestras de pares estéreo $(\boldsymbol{y}^0, \boldsymbol{y}^1)$ y que también hemos aprendido la distribución

[15] Aquí, por abuso de notación, utilizamos $\boldsymbol{x}$ para representar tanto un punto en 3D como una muestra de un objeto 3D completo o una escena compuesta por muchos puntos.

conjunta de los pares. Haciendo un pequeño abuso de notación, también utilizamos $\boldsymbol{y} = h(\boldsymbol{x})$ para indicar el par $\boldsymbol{y} = (\boldsymbol{y}^0, \boldsymbol{y}^1)$ y $p(\boldsymbol{y})$ como la distribución de probabilidad aprendida del par (por ejemplo, mediante un *denoiser* como el descrito anteriormente).

La pregunta principal ahora es: ¿Cómo aprender (una representación de) la distribución de la escena 3D $\boldsymbol{x}$ a partir de sus dos proyecciones con relaciones conocidas? Podría cuestionarse el fundamento de esto: ¿por qué es necesario si la función $h(\cdot)$ es en gran medida invertible? Es decir, la observación $\boldsymbol{y}$ puede determinar en gran medida la incógnita $\boldsymbol{x}$, lo cual ocurre en cierta medida con la visión estereoscópica: en general, dos imágenes (calibradas) contienen información suficiente sobre la profundidad de la escena, desde la perspectiva dada. Sin embargo, las imágenes 2D distan mucho de ser la representación más compacta de una escena 3D, ya que una misma escena puede producir infinitas imágenes 2D (altamente correlacionadas) o pares de imágenes. De hecho, una buena representación de una escena 3D debería ser invariante al punto de vista. Por lo tanto, una representación correcta de la distribución de las escenas 3D debería ser mucho más compacta y estructurada que la distribución de imágenes 2D, pares estéreo o pares de imagen y profundidad.

Consideremos el proceso de eliminación de ruido (inverso) para la difusión: $\boldsymbol{y}_t = \boldsymbol{y} + t\boldsymbol{g}$ en (7.5.11), donde $\boldsymbol{g}$ sigue una distribución gaussiana estándar. A partir del proceso de eliminación de ruido de (7.5.11), tenemos

$$\boldsymbol{y}_{t-s} = \boldsymbol{y}_t + st\nabla_{\boldsymbol{y}} \log p_t(\boldsymbol{y}_t). \tag{7.5.12}$$

Tratamos de encontrar un proceso de «eliminación de ruido» correspondiente para $\boldsymbol{x}_t$ tal que $\boldsymbol{x}$ se relacione con $\boldsymbol{y}$ de la siguiente manera:

$$\boldsymbol{y} = h(\boldsymbol{x}). \tag{7.5.13}$$

A continuación, tenemos:

$$h(\boldsymbol{x}_{t-s}) \approx h(\boldsymbol{x}_t) + st\nabla_{\boldsymbol{y}} \log p_t(h(\boldsymbol{x}_t)), \tag{7.5.14}$$

para un $s > 0$ pequeño. Supongamos que $\boldsymbol{x}_{t-s} = \boldsymbol{x}_t + s\boldsymbol{v}$ para algún vector $\boldsymbol{v}$ y un incremento s pequeño. Tenemos que

$$h(\boldsymbol{x}_{t-s}) \approx h(\boldsymbol{x}_t) + \frac{\partial h}{\partial \boldsymbol{x}}(\boldsymbol{x}_t) \cdot \boldsymbol{v}s \doteq h(\boldsymbol{x}_t) + \boldsymbol{A}(\boldsymbol{x}_t)\boldsymbol{v}s. \tag{7.5.15}$$

Por lo tanto, se tiene

$$\boldsymbol{A}(\boldsymbol{x}_t)\boldsymbol{v} = t\nabla_{\boldsymbol{y}} \log p_t(h(\boldsymbol{x}_t)). \tag{7.5.16}$$

Geométricamente, el vector $\boldsymbol{v}$ en el dominio de $\boldsymbol{x}$ puede considerarse como la aplicación regrediente (pullback) del campo vectorial $t\nabla \log p_t(\boldsymbol{y})$ bajo la aplicación $\boldsymbol{y} = h(\boldsymbol{x})$. En general, como antes, podemos elegir (arbitrariamente) $\boldsymbol{v}$ como el vector de norma cuadrática mínima que satisface la relación de retroceso. Por lo tanto, podemos expresar $\hat{\boldsymbol{x}}_{t-s}$ aproximadamente como:

$$\hat{\boldsymbol{x}}_{t-s} \approx \boldsymbol{x}_t + st\boldsymbol{A}(\boldsymbol{x}_t)^\dagger \nabla_{\boldsymbol{y}} \log p_t(h(\boldsymbol{x}_t)). \tag{7.5.17}$$

Nota 7.2 (Detección paralela y eliminación distribuida del ruido.). Hay algo muy interesante en la ecuación anterior (7.5.17). Parece sugerir que podríamos intentar aprender la distribución de $\boldsymbol{x}$ mediante un proceso que esté acoplado con (muchas de) sus observaciones (parciales):

$$\boldsymbol{y}^i = h^i(\boldsymbol{x}) + \boldsymbol{w}^i, i = 1, \dots, K. \tag{7.5.18}$$

En este caso, se obtiene un conjunto de ecuaciones que el campo vectorial $\boldsymbol{v}$ debe satisfacer en el dominio de $\boldsymbol{x}$:

$$\boldsymbol{A}^i(\boldsymbol{x}_t)\boldsymbol{v} = t\nabla_{\boldsymbol{y}^i} \log p_t(h^i(\boldsymbol{x}_t)), \tag{7.5.19}$$

donde $\boldsymbol{A}^i(\boldsymbol{x}_t) = \frac{\partial h^i}{\partial \boldsymbol{x}}(\boldsymbol{x}_t)$. El $\boldsymbol{v}$ final puede elegirse como una solución «centralizada» que satisfaga todas las ecuaciones anteriores, o bien como una cierta versión (estocásticamente) «agregada» de todos los $\boldsymbol{v}^i$:

$$\boldsymbol{v}^i = t\boldsymbol{A}^i(\boldsymbol{x}_t)^\dagger \left[\nabla_{\boldsymbol{y}^i} \log p_t(h^i(\boldsymbol{x}_t))\right], \quad i = 1, \dots, K, \tag{7.5.20}$$

que se calculan de forma paralela y distribuida. Una cuestión abierta aquí es hacia qué converge exactamente el proceso de «eliminación de ruido» así definido para $\boldsymbol{x}_t$, incluso en el caso del modelo de medición lineal. ¿Cuándo convergería a una distribución que tiene el mismo soporte de baja dimensión que el $\boldsymbol{x}_0$ original, a medida que $\boldsymbol{y}_t$ converge a $\boldsymbol{y} = h(\boldsymbol{x}_0)$?

Modelo visual del mundo a partir de secuencias de imágenes sin calibrar. En la deducción anterior, hemos supuesto que el modelo de medición $h(\cdot)$ se conoce por completo. En el caso de la visión estereoscópica, esto resulta bastante razonable, ya que la pose relativa (y la calibración) de los dos puntos de vista de las cámaras (o de los dos ojos[16]) suele conocerse de antemano. Por lo tanto, a través de los pares de imágenes estéreo, en principio deberíamos poder aprender la distribución de las escenas 3D, o al menos la distribución egocéntrica de las escenas 3D. Sin embargo, las estructuras de baja dimensión de la así llamada distribución aprendida contienen variaciones causadas por el cambio de los puntos de vista. Es decir, el aspecto de las imágenes estéreo varía cuando cambiamos nuestros puntos de vista con respecto a la misma escena 3D. Para muchas tareas prácticas de visión (como la localización y la navegación), es importante que podamos disociar esta variación de los puntos de vista de una representación invariante de (la distribución de) las escenas 3D.

Nota 7.3. Conviene recordar que el objetivo anterior se alinea bien con el Programa de Erlangen de Klein para la geometría moderna, cuyo objetivo es estudiar los invariantes de una variedad bajo un grupo de transformaciones. Aquí, podemos considerar la variedad que nos interesa como la distribución de representaciones egocéntricas de escenas tridimensionales. Hemos aprendido que admite un grupo de traslaciones y rotaciones de cuerpos rígidos en el espacio tridimensional que actúan sobre él. Es sorprendente que nuestro cerebro haya aprendido a desvincular eficazmente esas transformaciones del mundo tridimensional que observamos.

[16] La pose relativa de nuestros dos ojos es bien conocida por nuestro cerebro.

Obsérvese que en el Capítulo 5 hemos estudiado el aprendizaje de representaciones que son invariantes ante traslaciones y rotaciones en un contexto limitado. Sabemos que los operadores de compresión asociados adoptan la forma necesaria de convoluciones (multicanal), lo que da lugar a las redes neuronales convolucionales (profundas). No obstante, sigue siendo difícil caracterizar los operadores relacionados con la compresión o la eliminación de ruido que sean invariantes ante grupos de transformación más generales [CW16b]. En el caso del problema de la visión 3D en su contexto más general, sabemos que el cambio de posición de la cámara puede modelarse adecuadamente como un movimiento de cuerpo rígido. Sin embargo, normalmente se desconoce el movimiento relativo exacto de nuestros ojos entre diferentes vistas. En términos más generales, también podría haber objetos (por ejemplo, automóviles, personas, manos) que se muevan en la escena y, por lo general, tampoco conocemos su movimiento. ¿Cómo podemos generalizar el problema de aprender la distribución de escenas 3D con pares estéreo calibrados a contextos tan generales? Más concretamente, queremos aprender una representación compacta $\boldsymbol{x}$ de las escenas 3D que sea invariante con respecto a los movimientos de la cámara o del ojo. Una vez aprendida dicha representación, podríamos muestrear y generar una escena 3D y renderizar imágenes o pares estéreo a partir de poses arbitrarias.

A tal fin, cabe señalar que podemos modelar una secuencia de pares estéreo de la siguiente manera:

$$\boldsymbol{y}^k = h(\boldsymbol{x}^k, \theta^k), \quad k = 1, \ldots, K, \tag{7.5.21}$$

donde $h(\cdot)$ representa la función de proyección de 3D a 2D. θ^k denota los parámetros de movimiento de cuerpo rígido de la k-ésima vista, con respecto a algún sistema de coordenadas canónico del mundo. $\boldsymbol{x}^k$ representa la escena 3D en el instante k. Si la escena es estática, todos los $\boldsymbol{x}^k$ deben ser iguales a $\boldsymbol{x}$. Para simplificar la notación, podemos denotar el conjunto de k ecuaciones como una sola:

$$\boldsymbol{Y} = H(\boldsymbol{x}, \Theta). \tag{7.5.22}$$

Podemos suponer que se nos proporcionan muchas muestras de tales secuencias de imágenes estéreo $\{\boldsymbol{Y}_i\}$. El problema consiste en cómo recuperar la secuencia de movimiento asociada $\{\Theta_i\}$ y aprender la distribución de la escena $\boldsymbol{x}$ (que sea invariante al movimiento). Según nuestro conocimiento, este sigue siendo un problema abierto y complejo, que probablemente constituye la última frontera del problema de visión 3D.

7.6 Resumen y Notas

Comparación de mediciones sin muestras limpias. En nuestro desarrollo del muestreo condicional, consideramos el ajuste de mediciones en un modelo de observación (7.3.9), en el que suponemos que disponemos de datos emparejados $(\boldsymbol{x}, \boldsymbol{y})$ — es decir, los datos de referencia para cada observación $\boldsymbol{y}$. En muchos

problemas inversos de relevancia práctica, esto no es así: uno de los ejemplos más fundamentales se da en el contexto del muestreo con compresión, que recordamos en el Capítulo 2, donde necesitamos reconstruir $\boldsymbol{x}$ a partir de $\boldsymbol{y}$ utilizando el conocimiento previo sobre $\boldsymbol{x}$ (es decir, la esparcidad). En el contexto de la difusión con eliminación de ruido, tenemos acceso a una distribución a priori implícita para $\boldsymbol{x}$ a través de los *denoisers* aprendidos $\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{\xi})$. ¿Aún podemos realizar un muestreo condicional sin acceso a muestras de referencia $\boldsymbol{x}$?

Para entender intuitivamente por qué esto podría ser posible, recordemos un ejemplo clásico de la estadística conocido como el estimador de riesgo no sesgado de Stein (SURE). Bajo un modelo de observación $\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g}$ con $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ y $t > 0$, resulta que para cualquier $f : \mathbb{R}^D \to \mathbb{R}^D$ débilmente diferenciable,

$$\mathbb{E}_{\boldsymbol{g}}\left[\|\boldsymbol{x} - f(\boldsymbol{x} + t\boldsymbol{g})\|_2^2\right] = \mathbb{E}_{\boldsymbol{g}}\left[\|\boldsymbol{x} + t\boldsymbol{g} - f(\boldsymbol{x} + t\boldsymbol{g})\|_2^2 + 2t^2 \nabla \cdot f(\boldsymbol{x} + t\boldsymbol{g})\right] - t^2 D, \tag{7.6.1}$$

donde $\nabla\cdot$ denota el operador de divergencia:

$$\nabla \cdot f = \sum_{i=1}^{D} \partial_i f_i.$$

La parte dependiente de $\boldsymbol{x}$ del lado derecho de la Ecuación (7.6.1) se denomina estimador de riesgo no sesgado de Stein (SURE). Si tomamos las expectativas sobre $\boldsymbol{x}$ en la Ecuación (7.6.1), observe que el lado derecho se puede escribir como una expectativa con respecto a $\boldsymbol{x}_t$ — en particular, ¡el error cuadrático medio de *cualquier denoiser* f se puede estimar *únicamente a partir de muestras ruidosas*! Este hecho notable, en sus formas refinadas, constituye la base de muchas técnicas prácticas para llevar a cabo la restauración de imágenes, la *denoising-diffusion*, etc., utilizando únicamente datos ruidosos: ejemplos notables son el paradigma «Noise2Noise» [LMH+18] y Ambient Diffusion [DSD+23a].

Cabe mencionar como curiosidad que Ecuación (7.6.1) conduce a una prueba alternativa de la fórmula de Tweedie (Teorema 3.2). A grandes rasgos, se toman las expectativas sobre $\boldsymbol{x}$ y se expresa la parte principal del lado derecho de la Ecuación (7.6.1) de forma equivalente, mediante integración por partes, como

$$\begin{aligned} & \mathbb{E}_{\boldsymbol{x}_t}\left[\|\boldsymbol{x}_t - f(\boldsymbol{x}_t)\|_2^2 + 2t^2 \nabla \cdot f(\boldsymbol{x}_t)\right] \\ = \; & \mathbb{E}_{\boldsymbol{x}_t}\left[\|\boldsymbol{x}_t - f(\boldsymbol{x}_t)\|_2^2\right] - 2t^2 \int \langle \nabla p_{\boldsymbol{x}_t}(\boldsymbol{\xi}), f(\boldsymbol{\xi}) \rangle \, \mathrm{d}\boldsymbol{\xi}. \end{aligned} \tag{7.6.2}$$

Esta es una función cuadrática de f y al derivarla formalmente se obtiene que la función óptima f satisface la fórmula de Tweedie (Teorema 3.2). Este argumento puede demostrarse de manera rigurosa utilizando conceptos básicos del cálculo de variaciones.

Correcciones a la aproximación del muestreo posterior por difusión (DPS). En el Ejemplo 7.4 y, en particular, en la Figura 7.11, señalamos una

limitación de la aproximación DPS de la Ecuación (7.3.26) para niveles bajos de ruido de medición. Esta limitación es bien comprendida, y Rozet et al. han propuesto un enfoque fundamentado para mitigarla [RAL+24]. El enfoque consiste en incorporar una estimación adicional de la varianza de la distribución a posteriori ruidosa $p_{\boldsymbol{x}|\boldsymbol{x}_t}$ a Ecuación (7.3.26) — nos remitimos al artículo para más detalles. Las estimaciones naturales de la varianza a posteriori son ligeramente menos escalables que el propio DPS debido a la necesidad de invertir una transformación afín de la jacobiana del *denoiser* posterior $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$ (una matriz de gran tamaño). Esto se lleva a cabo de manera relativamente eficiente por Rozet et al. [RAL+24] mediante la diferenciación automática y una aproximación de la inversa basada en gradientes conjugados. Parece que sería posible mejorar aún más este enfoque (por ejemplo, utilizando ideas clásicas de la optimización de segundo orden).

Más información sobre el ajuste de mediciones y los modelos de difusión para problemas inversos. Los modelos de difusión se han convertido en una herramienta muy popular para resolver problemas inversos que surgen en aplicaciones científicas. Se han desarrollado, y se siguen desarrollando, muchos más métodos además del sencillo algoritmo DPS que hemos presentado en el Algoritmo 7.1, ya que esta área está evolucionando rápidamente. Entre las clases de enfoques populares y eficaces más allá del DPS, que hemos presentado por su carácter general, se incluyen los enfoques de división de variables como el DAPS [ZCB+24], que permiten aplicar restricciones de medición específicas de forma mucho más rigurosa que en el DPS y los enfoques exactos que pueden evitar el uso de aproximaciones como en el DPS, tales como el TDS [WTN+23]. Para obtener más información sobre este tema, recomendamos [ZCZ+25], que sirve la vez como revisión y como un benchmark para varios métodos populares en conjuntos de datos específicos de problemas inversos científicos.

Historia del principio Infomax. El principio de Infomax [Lin88], que presentamos en nuestro análisis sobre el aprendizaje conjunto de *embeddings* y CLIP en la Sección 7.4.2, constituye una base bien conocida e importante para gran parte de la investigación moderna sobre el aprendizaje de representaciones no supervisado. Para más información, nos remitimos a [OLV18], sobre el cual nuestro análisis conceptual en Sección 7.4.2 profundiza, y a [OLC+25], que detalla otras cuestiones teóricas importantes relacionadas con la optimización de objetivos del tipo Infomax a partir de muestras finitas.

7.7 Ejercicios y extensiones

Ejercicio 7.1 (Corrección de la varianza posterior en el DPS). 1. Utilizando el código proporcionado en el GitHub del libro para implementar la Figura 7.11, implemente la correctitud de la varianza a posteriori propuesta por Rozet et al. [RAL+24].

2. Compruebe que mejora el colapso posterior en condiciones de baja varianza del ruido que se observa en la Figura 7.11.

3. Analice cualquier problema relacionado con la corrección del muestreo que se mantenga o se introduzca con el método corregido, así como su eficiencia, en comparación con el muestreo posterior por difusión (DPS)

Ejercicio 7.2 (Muestreo condicional en MNIST). 1. Entrene un clasificador sencillo para el conjunto de datos MNIST, utilizando una arquitectura de su elección. Además, entrene un *denoiser* adecuado para su uso en muestreo condicional (Algoritmo 7.2, ya que este *denoiser* también se puede utilizar para la eliminación de ruido incondicional).

2. Integre el clasificador en un muestreador condicional basado en guía por clasificador, tal como se describe en la primera parte de la Sección 7.4.1. Evalúe las muestras resultantes en cuanto a su fidelidad a la clase condicionante (visualmente; en términos del vecino más cercano; en términos de la salida del clasificador).

3. Integre el clasificador en un muestreador condicional basado en guía sin clasificador, tal como se describe en la Sección 7.4.1 y en el Algoritmo 7.2. Realice la misma evaluación que en el paso anterior y compare los resultados.

4. Repita el experimento con el conjunto de datos CIFAR-10.

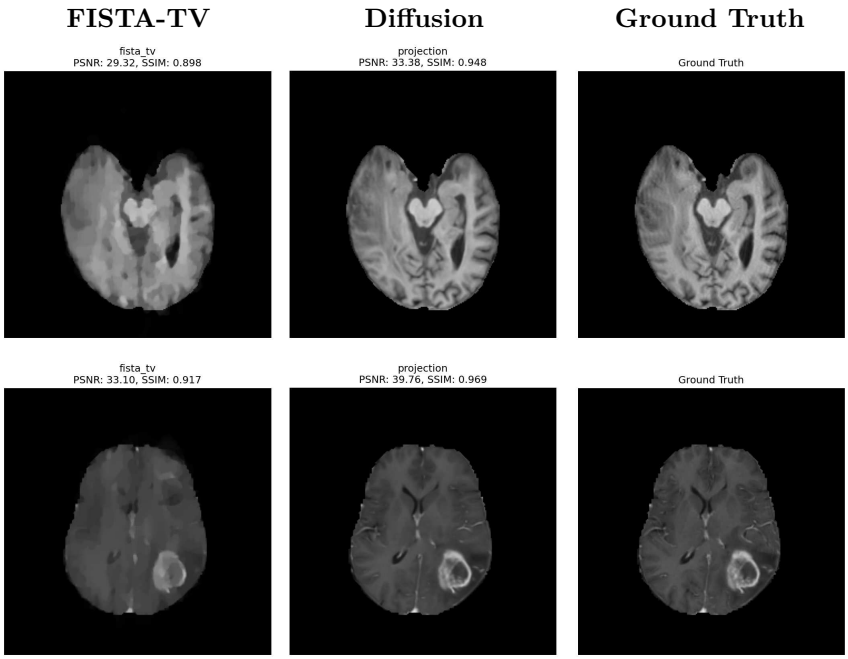


Figura 7.13: **Comparación visual de métodos de reconstrucción de resonancia magnética.** Tres ejemplos de reconstrucciones de resonancia magnética cerebral que muestran FISTA-TV, el enfoque de difusión basado en la consistencia de las mediciones de Song et al. [SSX+22] y las imágenes de referencia del conjunto de datos de reconstrucción de resonancia magnética BraTS con un submuestreo de 8x. El enfoque basado en la difusión produce reconstrucciones que se ajustan mejor a la estructura de la imagen de referencia en comparación con FISTA-TV, que, como se puede observar, suaviza en exceso los detalles finos presentes en la imagen de referencia. Al mismo tiempo, el grado de fidelidad a la imagen de referencia en la reconstrucción basada en la difusión (que surge de la imposición estricta de la consistencia de las mediciones) es notable.

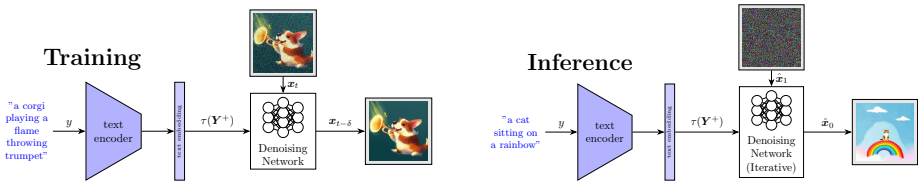


Figura 7.14: Esquema de alto nivel del entrenamiento y la aplicación de un modelo generativo de texto a imagen, mediante la generación condicional a partir de una indicación de texto. **Izquierda:** Para entrenar un modelo de texto a imagen, se utiliza un amplio conjunto de datos de imágenes emparejadas con los pies de foto correspondientes. Se utiliza un codificador para mapear los pies de foto a secuencias de vectores, que se utilizan como señales de condicionamiento para un *denoiser* condicional, entrenado según se describe algorítmicamente en la Sección 7.4.1 (véase la Sección 8.6 para detalles de implementación). El codificador de texto puede estar preentrenado y congelado, o entrenarse conjuntamente con el *denoiser*. **Derecha:** Al aplicar un modelo entrenado, se utiliza una indicación de texto deseada como condición y luego se realiza el muestreo con el modelo entrenado, tal como se muestra en el Algoritmo 7.2 (*mutatis mutandis* para su uso con una indicación de texto codificada).

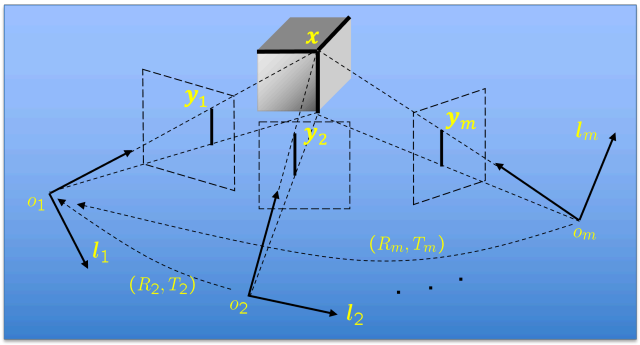


Figura 7.15: Relación entre un objeto o escena 3D y sus proyecciones 2D. Aquí ilustramos la proyección de un punto $\boldsymbol{x}$ y una recta que pasa por dicho punto.

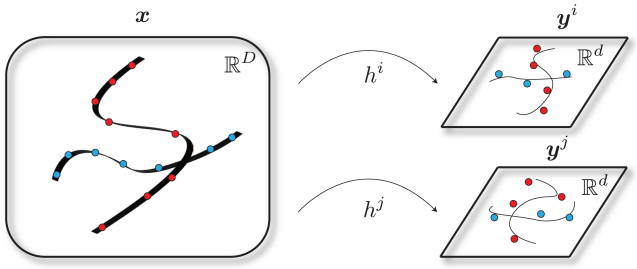


Figura 7.16: **Inferencia con mediciones distribuidas.** Tenemos una distribución de baja dimensión $\boldsymbol{x}$ (aquí, como en Figura 7.1, representada como la unión de dos variedades bidimensionales en $\mathbb{R}^3$) y un modelo de medición $\boldsymbol{y}^i = h^i(\boldsymbol{x}) + \boldsymbol{w}^i$. Como antes, queremos inferir varias propiedades de la distribución condicional de $\boldsymbol{x}$ dada $\boldsymbol{y}$, donde $\boldsymbol{y}$ es la colección de todas las mediciones $\boldsymbol{y}^i$.

Capítulo 8

Aprendizaje de representaciones para datos y tareas del mundo real

"*La mejor teoría se inspira en la práctica y la mejor práctica se inspira en la teoría.*"

— Donald Knuth

8.1 Introducción

En los capítulos anteriores se ha presentado una introducción sistemática a los problemas matemáticos, los marcos computacionales y los algoritmos prácticos relacionados con el aprendizaje de distribuciones de baja dimensión a partir de datos de alta dimensión. Aunque la mayoría de las justificaciones teóricas de estos métodos se han establecido para modelos idealizados de distribuciones de datos, como (mezclas de) subespacios y/o gaussianas, los principios e ideas que subyacen a estos métodos computacionales son, no obstante, potentes y generales, pensados de hecho para ser aplicables a conjuntos de datos y tareas del mundo real.

Para ayudar a los lectores a comprender mejor el contenido de este libro y aprender cómo aplicar lo aprendido hasta ahora a datos del mundo real, a continuación ofrecemos algunas demostraciones y estudios de caso de varias aplicaciones representativas. Cada aplicación propone una solución a una tarea del mundo real con un conjunto de datos del mundo real (como datos visuales, datos cinemáticos y datos de texto), utilizando los métodos que hemos presentado en este libro. Los resultados presentados en este capítulo están destinados a alcanzar los dos propósitos siguientes:

- en primer lugar, aportar detalles experimentales adicionales, así como evidencia empírica que valide los métodos presentados anteriormente en el libro y demostrar su significativo potencial en contextos del mundo real;

- en segundo lugar, familiarizar al lector con ciertas tareas y métodos empíricos modernos del aprendizaje profundo que no están bien documentados fuera de los entornos de investigación o de las bases de código de producción.

Sin embargo, en nuestra sincera opinión, las soluciones y los resultados que se presentan aquí están pensados simplemente para comprobar que la metodología funciona. De este modo, existe un amplio margen de mejora futura, tanto en lo que respecta a la comprensión teórica como a la ingeniería práctica, para seguir avanzando en el rendimiento del estado del arte. A lo largo de este capítulo, y especialmente en la última Sección 8.12, presentaremos algunas mejoras inmediatas y evidentes. Dejaremos para futuras investigaciones en el Capítulo 9 final el análisis de ideas potencialmente mucho más transformadoras sobre el aprendizaje de representaciones y los importantes problemas aún sin resolver en materia de inteligencia en general.

8.1.1 Estructura del capítulo

En lo que respecta a la inteligencia en la naturaleza, podría decirse que los tres tipos de datos más importantes con los que nuestro cerebro construye nuestra memoria y nuestro conocimiento son *los datos visuales*, *los movimientos de nuestro cuerpo* y los *lenguajes naturales*. Por lo tanto, en este capítulo mostraremos cómo aplicar los principios y métodos presentados en este libro para aprender buenas representaciones de estos datos del mundo real y explicaremos cómo dichas representaciones facilitan tareas prácticas importantes relacionadas con ellos.

En primer lugar, desde la Sección Sección 8.2 hasta la Sección 8.9, comenzaremos con los datos visuales y mostraremos cómo aprender las distribuciones de imágenes 2D a objetos 3D, ya que la memoria visual nos sirve (a nosotros y a los animales) como modelo de bajo nivel pero fundamental que almacena el conocimiento del mundo físico y nos ayuda a percibir y predecir en un nuevo entorno. A continuación, en la Sección 8.10, mostraremos cómo aprender la distribución de los movimientos corporales, ya que es fundamental para nosotros (y para los animales) actuar o interactuar con rapidez y precisión en los entornos. Por último, en la Sección 8.11, mostraremos cómo modelar y aprender la distribución de las lenguajes naturales.

Evidentemente, en comparación con los lenguajes naturales, nuestra cobertura de los datos físicos, visuales y de movimiento será mucho más amplia. Aunque en los últimos años el sector de la inteligencia artificial ha invertido fuertemente en el modelado de los lenguajes naturales,[1] se ha empezado a

[1] Incluidos sus derivados, como los lenguajes de programación, las deducciones lógicas y las demostraciones matemáticas.

advertir la importancia de modelar el conocimiento sobre el mundo físico y nuestras acciones en él. En los últimos años, ha habido un rápido aumento del interés y un cambio de foco hacia el desarrollo de lo que se conoce como «modelo del mundo», «inteligencia espacial», «inteligencia física» o «inteligencia encarnada», ya que son elementos cruciales para los sistemas o agentes de IA —por ejemplo, un robot inteligente— que necesitan interactuar con un mundo físico abierto. Como hemos aprendido en el Capítulo 1, ese es precisamente el objetivo y el alcance del programa de *Cibernética* iniciado por Norbert Wiener en la década de 1940.

En el plano técnico, desde la Sección 8.2 hasta la Sección 8.4, mostraremos cómo se utilizan los principios presentados en este libro para aprender buenas representaciones de datos de imágenes en entornos *no supervisados, débilmente supervisados* y *supervisados*, respectivamente.[2] Esto servirá tanto como introducción al procesamiento de datos de imágenes, a las técnicas de augmentación de datos y a la popular arquitectura profunda tipo Transformer, como paso previo a la implementación posterior de la arquitectura CRATE. Estos ejemplos también ofrecen la primera demostración de las drásticas simplificaciones que ya se pueden llevar a cabo utilizando los principios presentados en el libro. Continuaremos con las modificaciones de la *arquitectura de red de Transformer* en la Sección 8.4 para imágenes y en la Sección 8.11 para textos, respectivamente, que demuestran las capacidades de las arquitecturas transparentes *white-box* simplificadas, incluyendo CRATE y sus variantes, para la *codificación* en los ámbitos de la imagen y el texto. También demostraremos las capacidades de las arquitecturas simplificadas para la *autocodificación* en la Sección 8.5.

En la práctica, la razón principal por la que queremos aprender una buena representación de una distribución de datos es para poder tomar muestras de la distribución (como distribución a priori) con el fin de regenerar, estimar o predecir el estado del mundo, condicionado a la observación actual o a la información dada. Demostramos las capacidades generativas de las representaciones aprendidas

- para imágenes 2D en la Sección 8.6 y la Sección 8.7;
- para objetos 3D en la Sección 8.8 y la Sección 8.9;
- para movimientos humanos en la Sección 8.10;
- y para lenguajes naturales en la Sección 8.11.

8.1.2 Configuración técnica general

En capítulos anteriores, hemos hecho alusión a diferentes configuraciones en las que utilizamos técnicas de aprendizaje de representaciones para procesar datos

[2] El objetivo en los tres casos es aprender una buena representación de la distribución de los datos. Solo se diferencian en la cantidad de información lateral que se proporciona sobre la distribución y que puede facilitar el aprendizaje.

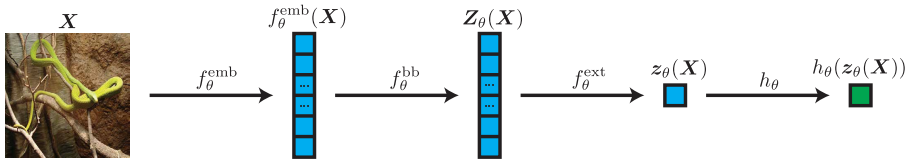


Figura 8.1: **Un diagrama del proceso del codificador.** Los datos $\boldsymbol{X} \in \mathcal{D}$ se introducen en la función de *embedding* f_θ^{emb} para obtener una secuencia en $(\mathbb{R}^d)^*$. El *embedding* se procesa mediante un *backbone* f_θ^{bb} para obtener características $\boldsymbol{Z}_\theta(\boldsymbol{X})$ para cada token. Podemos extraer una característica agregada $\boldsymbol{z}_\theta(\boldsymbol{X})$ utilizando el mapa de extracción f_θ^{ext}. Por último, para utilizar la característica agregada en tareas posteriores, podemos emplear el cabezal específico para la tarea h_θ.

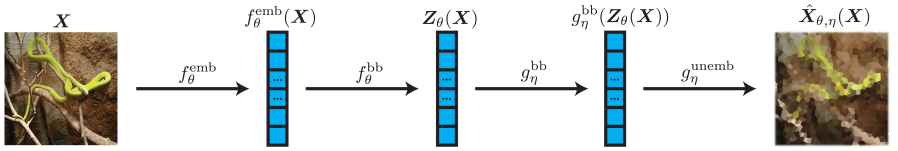


Figura 8.2: **Un diagrama del proceso del autocodificador.** Los datos $\boldsymbol{X} \in \mathcal{D}$ se introducen en la capa de *embedding* f_θ^{emb} para obtener una secuencia en $(\mathbb{R}^d)^*$. El *embedding* pasa por un *backbone* del codificador f_θ^{bb} para obtener características $\boldsymbol{Z}_\theta(\boldsymbol{X})$ para cada token. Para decodificar $\boldsymbol{Z}_\theta(\boldsymbol{X})$, se pasa a través de un *backbone* del decodificador g_η^{bb}. Para mapear la salida del *backbone* del decodificador de vuelta al espacio de datos $\mathcal{D}$, utilizamos una capa de *desincrustación* g_η^{unemb}, obteniendo en definitiva una reconstrucción $\hat{\boldsymbol{X}}_{\theta,\eta}(\boldsymbol{X})$ (aquí estilizada como una reconstrucción pixelada de la entrada).

reales a gran escala. En este capítulo, describiremos dichas configuraciones con gran detalle. El objetivo de esta sección es que el lector sea capaz de reproducir cualquier experimento que se trate en esta sección (o, de hecho, en todo el libro) utilizando únicamente la descripción que le daremos en el libro, los principios introducidos en capítulos anteriores y ampliados en este capítulo, así como los hiperparámetros extraídos de un puñado de artículos cuyos resultados se analizan en este capítulo. Con este fin, describiremos *con precisión* todos los procedimientos en un lenguaje detallado, pseudocódigo o notación matemática, que puedan implementarse directamente en código. Siempre que sea posible, analizaremos cómo las implementaciones concretas se relacionan con los principios presentados anteriormente en el libro.

Definamos el conjunto de datos posibles como $\mathcal{D}$ (que, en última instancia, será el conjunto de imágenes $\mathcal{I}$, por ejemplo, o el conjunto de textos $\mathcal{T}$), así como el conjunto de secuencias finitas de *tokens* en $\mathbb{R}^d$ (es decir, el conjunto de matrices con d filas) como $(\mathbb{R}^d)^* \doteq \bigcup_{T=1}^{\infty} \mathbb{R}^{d \times T}$. Para analizar la gran variedad de aplicaciones que presentamos en este capítulo, recordamos primero que en el resto del libro tratamos dos tipos diferentes de arquitecturas de modelos.

- Una arquitectura de *codificador*, parametrizada por θ, que se compone de

varias componentes:

  - Una *incrustación* $f_\theta^{\text{emb}}\colon \mathcal{D} \to (\mathbb{R}^d)^*$, que convierte los datos de entrada $\mathcal{D}$ en una serie de *tokens* que se mapean o se *incrustan* en un espacio de d dimensiones. *En el resto del capítulo, usaremos indistintamente *tokens* y *embeddings*.*
  - Una *red troncal de codificación* $f_\theta^{\text{bb}}\colon (\mathbb{R}^d)^* \to (\mathbb{R}^d)^*$, que procesa la serie de *embeddings* mediante una operación de secuencia a secuencia. Este *backbone* se implementa mediante las arquitecturas de red discutidas en los capítulos anteriores, pero daremos una descripción más formal a medida que avancemos.
  - Un *extractor de características agregadas* $f_\theta^{\text{ext}}\colon (\mathbb{R}^d)^* \to \mathbb{R}^d$, que extrae una representación agregada de toda la secuencia. Esto se utiliza para definir una única característica para toda la muestra de datos.
  - Una *cabeza específica para la tarea* $h_\theta\colon \mathbb{R}^d \to \mathbb{R}^m$, que extrae una salida de m dimensiones para la predicción.

  También definimos $f_\theta \doteq f_\theta^{\text{bb}} \circ f_\theta^{\text{emb}}\colon \mathcal{D} \to (\mathbb{R}^d)^*$. Dada una entrada $\boldsymbol{X}$, escribimos $\boldsymbol{Z}_\theta(\boldsymbol{X}) \doteq f_\theta(\boldsymbol{X})$ y $\boldsymbol{z}_\theta(\boldsymbol{X}) \doteq f_\theta^{\text{ext}}(\boldsymbol{Z}_\theta(\boldsymbol{X}))$. El proceso general se muestra en la Figura 8.1.

- Una arquitectura de *autocodificador*, que se compone de varias componentes:

  - Una *incrustación* $f_\theta^{\text{emb}}\colon \mathcal{D} \to (\mathbb{R}^d)^*$, que convierte los datos de entrada $\mathcal{D}$ en una serie de *tokens* que se incrustan en un espacio de d dimensiones.
  - Una *red troncal codificadora* $f_\theta^{\text{bb}}\colon (\mathbb{R}^d)^* \to (\mathbb{R}^d)^*$, que procesa la serie de *embeddings* mediante una operación secuencia a secuencia.
  - Una *red troncal decodificadora* $g_\eta^{\text{bb}}\colon (\mathbb{R}^d)^* \to (\mathbb{R}^d)^*$, que conceptualmente revierte la operación de la red troncal codificadora.
  - Una *desincrustación* $g_\eta^{\text{unemb}}\colon (\mathbb{R}^d)^* \to \mathcal{D}$, que actúa como inversa del *embedding*.

  También definimos $f_\theta \doteq f_\theta^{\text{bb}} \circ f_\theta^{\text{emb}}\colon \mathcal{D} \to (\mathbb{R}^d)^*$ y $g_\eta \doteq g_\eta^{\text{unemb}} \circ g_\eta^{\text{bb}}\colon (\mathbb{R}^d)^* \to \mathcal{D}$. Dada una entrada $\boldsymbol{X}$, escribimos $\boldsymbol{Z}_\theta(\boldsymbol{X}) \doteq f_\theta(\boldsymbol{X})$ y $\hat{\boldsymbol{X}}_{\theta,\eta}(\boldsymbol{X}) \doteq g_\eta(\boldsymbol{Z}_\theta(\boldsymbol{X}))$. El proceso general se muestra en la Figura 8.2.

Utilizaremos esta notación en numerosas ocasiones a lo largo de este capítulo, por lo que se alienta al lector a volver a consultarla si algo no le queda claro. Esta descomposición de nuestras redes también se corresponde estrechamente con la mayoría de las implementaciones de código y usted puede comenzar sus proyectos de programación definiendo estas redes.

8.2 Aprendizaje no supervisado de representaciones de imágenes

El aprendizaje de representaciones fieles y de alta calidad de las distribuciones de datos de imágenes es un problema fundamental en la inteligencia artificial, tal y como hemos analizado en profundidad en el Capítulo 4. En la literatura se han propuesto muchos enfoques populares para esta tarea, muchos de los cuales pueden parecer que no utilizan las técnicas y los principios descritos en este manuscrito. Uno de estos enfoques se denomina *aprendizaje contrastivo*, llamado así porque el objetivo del aprendizaje consiste (en términos generales) en garantizar que las características de los datos «similares» sean similares y que las características de los datos «disimilares» estén muy alejadas entre sí. Las soluciones de aprendizaje contrastivo suelen ser diseños muy elaborados y empíricos. En esta sección, presentaremos uno de estos enfoques populares, conocido como DINO [CTM+21]. No obstante, demostraremos cómo utilizar los principios descritos en este libro para simplificar drásticamente sus decisiones de diseño y, al mismo tiempo, mejorar las representaciones aprendidas. Esto se conoce como **SimDINO** [WZP+25]:

https://robinwu218.github.io/SimDINO/.

8.2.1 Datos

Los datos que utilizaremos para explorar y simplificar la metodología DINO son todos datos de imágenes bidimensionales (2D).

Para el *entrenamiento*, utilizaremos los conjuntos de datos ImageNet-1K e ImageNet-21K. Cada muestra del conjunto de datos consiste en una imagen RGB de resolución variable y una etiqueta que indica el objeto o la escena que la imagen contiene (es decir, la *clase* de la imagen). El conjunto de datos ImageNet-1K contiene 1,28 millones de imágenes de entrenamiento y 50 000 imágenes de validación, divididas en 1 000 clases. El conjunto de datos ImageNet-21K contiene 14,2 millones de imágenes de entrenamiento y 21 800 clases, pero las clases no son disjuntas (es decir, algunas clases son subconjuntos de otras). Dado que estamos realizando un aprendizaje autosupervisado, las etiquetas no se utilizarán durante el entrenamiento, sino solo durante la evaluación. Con el fin de aumentar la robustez de nuestro modelo, a menudo aplicamos *pequeños augmentaciones de datos* a cada imagen durante el procesamiento, tales como volteos, la adición de un pequeño ruido aleatorio o recortes aleatorios de gran tamaño; no lo incluimos en nuestra notación, ya que cada augmentación de una imagen natural es en sí misma (prácticamente) una imagen natural de nuestro conjunto de datos.

Para la *evaluación*, utilizaremos una amplia gama de conjuntos de datos. De ellos, el más común es CIFAR-10. CIFAR-10 es un conjunto de datos compuesto por 60 000 imágenes naturales en formato RGB de 32×32, divididas en 10 clases, con un conjunto de entrenamiento predeterminado de 50 000

(a) Muestras de ImageNet-1K.

(b) Muestras de CIFAR10.

Figura 8.3: **Imágenes de ImageNet-1K *(izquierda)* y CIFAR-10 *(derecha)*.** Obsérvese que las imágenes de CIFAR-10 tienen, en general, una resolución mucho menor, lo que reduce la complejidad del aprendizaje de esa distribución.

muestras y un conjunto de validación de 10 000 muestras. El objetivo de utilizar CIFAR-10 es garantizar que los modelos que se entrenan con una distribución de imágenes (ImageNet) puedan generalizarse a otra distribución de imágenes (CIFAR-10). También hacemos referencia a otros conjuntos de datos similares, como CIFAR-100 (disjunto de CIFAR-10), Oxford Flowers y Oxford Pets. En la Figura 8.3 se muestran ejemplos representativos de los datos de ImageNet-1K y CIFAR-10.

En un plano un poco más formal, nuestros datos $\boldsymbol{X}$ serán imágenes; sea $\mathcal{I}$ el conjunto de todas las imágenes. Dado que una imagen es una matriz rectangular de píxeles y cada píxel tiene un color dado por RGB, CMYK u otro formato de color, decimos que una imagen es un elemento de $\mathbb{R}^{c\times h\times w}$ — aquí c es el número de canales (es decir, 3 para RGB y 4 para CMYK), h es la altura de la imagen y w es el ancho de la imagen. Por consiguiente, el conjunto de todas las imágenes $\mathcal{I} \doteq \bigcup_{c,h,w=1}^{\infty} \mathbb{R}^{c\times h\times w}$ es el conjunto de los datos posibles de este tipo. Una vez más, utilizaremos esta notación en repetidas ocasiones.

8.2.2 La tarea y el objetivo simplificados de DINO

Nuestra tarea consiste en aprender una buena representación que capture la distribución de los datos. El aprendizaje contrastivo, en términos generales, lo logra definiendo qué propiedades de la imagen de entrada queremos que reflejen las características, generando imágenes que compartan esas propiedades pero varíen en otras, y estableciendo una función de pérdida que favorezca que las características de las imágenes con propiedades comunes sean similares y que las de las demás sean distintas. La solución óptima natural a este problema de aprendizaje consiste en que las características aprendidas conserven las propiedades deseadas de la entrada. Sin embargo, existen muchas complicaciones prácticas y empíricas que surgen durante el entrenamiento de modelos contrastivos.

En el caso de DINO, los autores proponen utilizar una metodología que genera un único vector de características para toda la imagen y pretende

que dicho vector contenga información «global» (es decir, a nivel de imagen). En consecuencia, la función de pérdida favorecerá que las imágenes con información global similar tengan características similares y que las imágenes con información global diferente tengan características diferentes.

Esto parece intuitivo, pero, como se mencionó anteriormente, hay varias consideraciones empíricas, incluso a la hora de definir la función de pérdida. Ante todo, ¿cómo debemos promover las similitudes y las diferencias? La respuesta de DINO [CTM+21] es[3] convertir las características de salida en «logits» correspondientes a una distribución de probabilidad y calcular su entropía cruzada. Más concretamente, sea $\Delta_m \doteq \{\boldsymbol{x} \in \mathbb{R}^m \colon x_i \geq 0 \ \forall i \in [m], \sum_{i=1}^m x_i = 1\}$ el espacio de vectores de probabilidad en $\mathbb{R}^m$ y definamos la función $d_{\mathrm{CE}} \colon \mathbb{R}^m \times \mathbb{R}^m \to \mathbb{R}$ como

$$d_{\mathrm{CE}}(\boldsymbol{p}, \boldsymbol{q}) \doteq \mathrm{CE}(\boldsymbol{p}, \boldsymbol{q}), \quad \forall \boldsymbol{p}, \boldsymbol{q} \in \Delta_m \tag{8.2.1}$$

donde $\mathrm{CE} \colon \Delta_m \times \Delta_m \to \mathbb{R}$ es la entropía cruzada, definida como

$$\mathrm{CE}(\boldsymbol{p}, \boldsymbol{q}) \doteq -\sum_{i=1}^m p_i \log q_i, \quad \forall \boldsymbol{p} = (p_1, \ldots, p_m), \boldsymbol{q} = (q_1, \ldots, q_m) \in \Delta_m. \tag{8.2.2}$$

Antes de continuar con nuestro análisis, intentemos cómo se comporta esta función de distancia. En particular, tenemos que:

$$\begin{aligned} \mathrm{CE}(\boldsymbol{p}, \boldsymbol{q}) &= -\sum_{i=1}^m p_i \log q_i = \sum_{i=1}^m p_i \log(p_i/q_i) - \sum_{i=1}^m p_i \log p_i && (8.2.3) \\ &= \mathsf{KL}(\boldsymbol{p} \parallel \boldsymbol{q}) + H(\boldsymbol{p}) && (8.2.4) \end{aligned}$$

donde $\mathsf{KL} \colon \Delta_m \times \Delta_m \to \mathbb{R}$ es la divergencia de KL, definida como

$$\mathsf{KL}(\boldsymbol{p} \parallel \boldsymbol{q}) \doteq \sum_{i=1}^m p_i \log(p_i/q_i), \tag{8.2.5}$$

y $H \colon \Delta_m \to \mathbb{R}$ es la entropía de una variable aleatoria. Obsérvese que $\mathsf{KL}(\boldsymbol{p} \mid \boldsymbol{q})$ se minimiza si y solo si $\boldsymbol{p} = \boldsymbol{q}$. De modo que minimizar d_{CE} hace dos cosas: hace que $\boldsymbol{p} = \boldsymbol{q}$, y hace que $\boldsymbol{p}$ y $\boldsymbol{q}$ tengan entropía mínima (es decir, vectores con 1 en una sola componente y 0 en el resto — denominados *vectores «one-hot»*). En general, el objetivo de esta función no es *solo* hacer coincidir $\boldsymbol{p}$ y $\boldsymbol{q}$, sino también darles una forma determinada para que tengan baja entropía. Tengamos esto en cuenta cuando analicemos la formulación.

La siguiente pregunta es: ¿cómo deberíamos obtener muestras con información global similar? La respuesta que ofrece DINO (al igual que casi todo el aprendizaje contrastivo) es la *augmentación de datos*: a partir de cada muestra, se generan varias muestras correlacionadas que comparten las propiedades deseadas. En el caso de DINO, utilizamos diferentes recortes

[3] En opinión del autor, inexplicablemente...

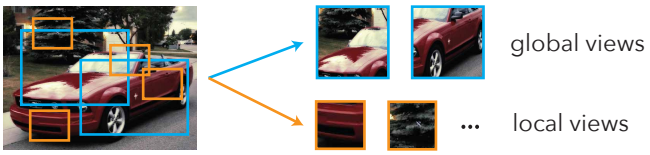


Figura 8.4: **Vistas locales y globales en DINO.** Ambas toman un recorte rectangular de la imagen de entrada y lo redimensionan a una forma cuadrada, que luego se introduce en la red para su procesamiento.

o *vistas* de la imagen de entrada. Recordemos que modelamos una imagen como un elemento del conjunto $\mathcal{I}$. En esta notación, una vista es una función $v\colon \mathcal{I} \to \mathcal{I}$. En el caso de DINO, la vista es un *recorte aleatorio con redimensionamiento*: toma un recorte rectangular elegido al azar de la imagen (que ocupa un porcentaje fijo $p_v \in [0,1]$ del área total de la imagen), le cambia el tamaño proporcionalmente para que el *lado más corto* tenga S_{rsz} píxeles de largo y luego lo redimensiona a una forma fija (C, S_v, S_v), donde $S_v \geq 1$ es el tamaño de la vista y C es el número de canales de la imagen original.

Hay dos tipos de vistas que queremos utilizar, tal y como se muestra en la Figura 8.4:

- *vistas globales*, que son recortes aleatorios redimensionados con el parámetro de porcentaje de área $p_{\text{glo}} \in [0,1]$ y la forma de salida $(C, S_{\text{glo}}, S_{\text{glo}})$;
- *vistas locales*, que son recortes aleatorios redimensionados con el parámetro de porcentaje de área $p_{\text{loc}} \in [0,1]$ y la forma de salida $(C, S_{\text{loc}}, S_{\text{loc}})$. Aquí $p_{\text{loc}} < p_{\text{glo}}$ y $S_{\text{loc}} < S_{\text{glo}}$.

DINO pretende que las características agregadas $\boldsymbol{z}_\theta(\boldsymbol{X}_v) \doteq (f_\theta^{\text{ext}} \circ f_\theta)(\boldsymbol{X}_v)$ de todas las vistas $\boldsymbol{X}_v \doteq v(\boldsymbol{X})$ de una imagen de entrada $\boldsymbol{X}$ sean consistentes entre sí. DINO lo logra mediante un *«DINO head»*[4] $h_{\boldsymbol{W},\boldsymbol{\mu}}$, parametrizado por una matriz $\boldsymbol{W} \in \mathbb{R}^{s\times d}$ y un vector $\boldsymbol{\mu} \in \mathbb{R}^s$, para extraer un vector de probabilidad $\boldsymbol{p}_{\theta,\boldsymbol{W},\boldsymbol{\mu}}(\boldsymbol{X}_v) \doteq h_{\boldsymbol{W},\boldsymbol{\mu}}(\boldsymbol{z}_\theta(\boldsymbol{X}_v))$ a partir de la característica agregada $\boldsymbol{z}_\theta(\boldsymbol{X}_v)$, utilizando la siguiente receta simple:

$$h_{\boldsymbol{W},\boldsymbol{\mu}}(\boldsymbol{z}) \doteq \operatorname{softmax}([\boldsymbol{W}\boldsymbol{z} - \boldsymbol{\mu}]/\tau), \qquad \forall \boldsymbol{z} \in \mathbb{R}^d, \tag{8.2.6}$$

donde la función softmax$\colon \mathbb{R}^s \to \Delta_s$ se define por

$$\operatorname{softmax}\left(\begin{bmatrix} x_1 \\ \vdots \\ x_s \end{bmatrix}\right) \doteq \frac{1}{\sum_{i=1}^s e^{x_i}} \begin{bmatrix} e^{x_1} \\ \vdots \\ e^{x_s} \end{bmatrix} \tag{8.2.7}$$

[4] Tenga en cuenta que $h_{\boldsymbol{W},\boldsymbol{\mu}}$ es el cabezal específico de la tarea, el cual, en Sección 8.1.2, se parametriza únicamente mediante θ, sin parámetros propios; sin embargo, dado que utilizamos dos invocaciones de h con valores distintos del segundo parámetro, mantenemos la notación indicada.

y $\tau > 0$ es un parámetro de «temperatura» que controla la entropía de la distribución resultante del softmax.

En particular, DINO minimiza la diferencia entre el vector de probabilidad $\boldsymbol{p}_{\theta,\boldsymbol{W},\boldsymbol{\mu}}(\boldsymbol{X}_g) \doteq h_{\boldsymbol{W},\boldsymbol{\mu}}(\boldsymbol{z}_\theta(\boldsymbol{X}_g))$ para cada vista global $\boldsymbol{X}_g \doteq v_g(\boldsymbol{X})$ y el vector de probabilidad $\boldsymbol{p}_{\theta,\boldsymbol{W}}(\boldsymbol{X}_c) \doteq h_{\boldsymbol{W},\mathbf{0}_m}(\boldsymbol{z}_\theta(\boldsymbol{X}_c))$ para cada vista $\boldsymbol{X}_c \doteq v_c(\boldsymbol{X})$. Aquí, v_c puede ser ya sea una vista *local* o una vista global. Analizaremos la implementación de f_θ y f_θ^{ext} en breve en Sección 8.2.3. En general, DINO resuelve el problema

$$\begin{aligned} &\min_{\theta,\boldsymbol{W},\boldsymbol{\mu}} \mathcal{L}_{\text{DINO}}(\theta, \boldsymbol{W}, \boldsymbol{\mu}) \qquad \text{donde} \\ &\mathcal{L}_{\text{DINO}}(\theta, \boldsymbol{W}, \boldsymbol{\mu}) \doteq \mathbb{E}[d_{\text{CE}}(\boldsymbol{p}_{\theta,\boldsymbol{W},\boldsymbol{\mu}}(\boldsymbol{X}_g), \boldsymbol{p}_{\theta,\boldsymbol{W}}(\boldsymbol{X}_c))], \end{aligned} \tag{8.2.8}$$

donde la esperanza se toma sobre los datos $\boldsymbol{X}$, las vistas globales v_g y las otras vistas v_c.

En este caso concreto, sin embargo, si se intenta implementar (8.2.8) y optimizarla en una red real, es muy probable que se presente un problema: tras ejecutar unas cuantas iteraciones del algoritmo de aprendizaje, el mapeo de características $f_\theta^{\text{ext}} \circ f_\theta$ ¡se *convertirá en la función constante*! Esto sin duda optimiza la pérdida anterior, ya que minimiza la distancia entre las características de diferentes vistas de la misma imagen. Pero obviamente no queremos aprender esta solución.

De hecho, evitar el colapso es una consideración muy común en el aprendizaje contrastivo. Entonces, ¿cómo se hace en este caso? La solución que propone DINO, una vez más, está diseñada en forma empírica y ajusta cuidadosamente la optimización del parámetro $\boldsymbol{\mu}$ (que se actualiza utilizando todas las muestras del lote) y un hiperparámetro de «temperatura» τ que forma parte de la implementación de $h_{\boldsymbol{W},\boldsymbol{\mu}}$ y se analiza en Sección 8.2.3. Si se dispone de un conjunto específico de hiperparámetros que funcionan bien, esto es, sin duda, suficiente para garantizar que la representación no colapse. Sin embargo, fuera de esta configuración específica, es difícil lograr que los modelos converjan durante el entrenamiento, y el entrenamiento resulta muy inestable.

Para remediar esta situación, analicemos algunas simplificaciones de la formulación. En primer lugar, en lugar de calcular un vector de probabilidad utilizando una transformación aprendida $h_{\boldsymbol{W},\boldsymbol{\mu}}$ de las características agregadas $\boldsymbol{z}_\theta$, podemos *utilizar directamente la representación agregada*, ignorando el cabezal específico de la tarea (o, lo que es lo mismo, asignándole la aplicación de identidad). Pero ahora necesitamos una forma de comparar los vectores directamente. Partiendo de nuestra hipótesis del Capítulo 5, según la cual las buenas representaciones deben tener geometría euclidiana (de sus subespacios), una medida mucho más natural de la diferencia es la *distancia ℓ^2 al cuadrado* $d_{\ell^2} \colon \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$, definida como

$$d_{\ell^2}(\boldsymbol{x}, \boldsymbol{y}) \doteq \frac{1}{2}\|\boldsymbol{x} - \boldsymbol{y}\|_2^2, \qquad \forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d. \tag{8.2.9}$$

Esta medida basada en la distancia es aún más eficiente de calcular que la medida de entropía cruzada. Por lo tanto, d_{ℓ^2} sustituye a d_{CE} en nuestra simplificación.

Figura 8.5: **Un ejemplo de una imagen dividida en 5×5 parches cuadrados, que se colocan en orden de trama.** Cada parche tiene el mismo tamaño, y la cuadrícula de parches tiene la forma $(N_H, N_W) = (5, 5)$. La cuadrícula de parches se despliega entonces en una secuencia de longitud $5 \times 5 = 25$ en orden de trama.

Anteriormente, el colapso se evitaba utilizando trucos para actualizar $\boldsymbol{\mu}$ y τ. En nuestra simplificación, si comparamos las características dentro del espacio de representación en lugar de convertirlas en probabilidades, no contamos con ninguno de estos parámetros, por lo que debemos considerar una forma diferente de evitar el colapso. Para ello, volvemos a los fundamentos. La idea fundamental para evitar el colapso consiste en que diferentes muestras produzcan características distintas. En otras palabras, aspiramos a que la *matriz de covarianza* de las características tenga una *magnitud* no despreciable en algún sentido. Pero a partir de los Capítulos 4 y 5, ya contamos con una magnitud que mide el tamaño de la matriz de covarianza. Es decir, utilizamos la sencilla *tasa de codificación gaussiana* (a nivel poblacional) R para garantizar que las características de las vistas globales de diferentes imágenes, que contienen información global distinta, estén bien separadas y no se colapsen (por lo tanto, se expandan). La pérdida modificada global $\mathcal{L}_{\text{SimDINO}}$ queda así:

$$\mathcal{L}_{\text{SimDINO}}(\theta) \doteq \mathbb{E}[d_{\ell^2}(\boldsymbol{z}_\theta(\boldsymbol{X}_g), \boldsymbol{z}_\theta(\boldsymbol{X}_c))] \tag{8.2.10}$$

$$- \frac{\gamma}{2} \log\det\left(\boldsymbol{I} + \frac{d}{\varepsilon^2} \operatorname{Cov}(\boldsymbol{z}_\theta(\boldsymbol{X}_g))\right), \tag{8.2.11}$$

donde $\varepsilon > 0$ es un valor fijo y las expectativas correspondientes se calculan, como antes, sobre los datos $\boldsymbol{X}$, la vista global v_g y otra vista (local o global) v_c. La pérdida en (8.2.10) es esencialmente la misma pérdida presentada en la sección 4.3.2 del capítulo 4 para el DINO simplificado («SimDINO» [WZP+25]). Como veremos, cuando se implementa correctamente, funciona al menos tan bien como el DINO original.

8.2.3 Arquitectura: *Vision Transformer*

Para la arquitectura, utilizamos un *Vision Transformador* estándar. A continuación se explica cómo funciona formalmente dicha arquitectura en el contexto de los datos de imagen. Recordemos de Sección 8.1.2 que una arquitectura de codificador consta de cuatro componentes: un *embedding*, un *backbone*, un extractor de características y una cabeza específica para la tarea. Luego analizaremos estas cuatro partes.

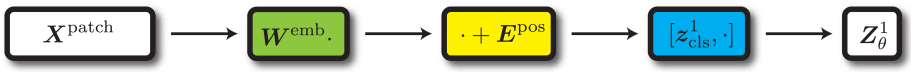


Figura 8.6: **El proceso de *embedding* del transformador.** Dada una secuencia de parches desplegados en orden ráster $\boldsymbol{X}^{\text{patch}}$, cada parche desplegado se proyecta linealmente en el espacio de características, y se le asigna una codificación posicional (aditiva) y un token adicional conocido como token de clase. La salida es la característica de entrada de la primera capa $\boldsymbol{Z}^1_\theta(\boldsymbol{X}) = f^{\text{emb}}_\theta(\boldsymbol{X})$.

Incrustación. Dados los datos de una imagen $\boldsymbol{X} \in \mathcal{I}$, los incrustamos como una secuencia de tokens en $\mathbb{R}^d$ utilizando la aplicación f^{emb}_θ, de la siguiente manera. Los dos primeros pasos se muestran en Figura 8.5, y los dos últimos se muestran en Figura 8.6.

1. En primer lugar, convertimos los datos de la imagen $\boldsymbol{X}$ en una secuencia de parches de forma (C, P_H, P_W), donde P_H y P_W son las dimensiones de los parches. Suponemos que P_H y P_W dividen sin residuo la altura y el ancho de $\boldsymbol{X}$, respectivamente (en la notación de Sección 8.2.2 suponemos que P_H y P_W dividen equitativamente S_{loc} y S_{glo}). Supongamos que la cuadrícula de parches resultante tiene N_H filas y N_W columnas.

2. Desenrollamos cada parche en un vector de longitud $D \doteq CP_HP_W$. Hay $N \doteq N_HN_W$ vectores de parches, los cuales colocamos en «orden raster» (arriba a la izquierda $\to$ arriba a la derecha $\to$ abajo a la izquierda $\to$ abajo a la derecha) en una matriz $\boldsymbol{X}^{\text{patch}} \in \mathbb{R}^{D\times N}$, donde $\boldsymbol{X}^{\text{patch}} \doteq f^{\text{patch}}(\boldsymbol{X})$. Obsérvese que D depende únicamente del tamaño del parche y del número de canales. Dado que esta última cantidad suele ser constante entre las muestras del mismo conjunto de datos, D es el mismo para todas las imágenes del conjunto de datos, mientras que N difiere entre imágenes de mayor o menor tamaño.

3. A continuación, aplicamos la siguiente operación sobre $\boldsymbol{X}^{\text{patch}} \in \mathbb{R}^{D\times N}$ para proyectarlo en $\mathbb{R}^{d\times n}$ donde $n \doteq N + 1$:

$$\boldsymbol{X}^{\text{patch}} \mapsto [\boldsymbol{z}^1_{\text{cls}}, \boldsymbol{W}^{\text{emb}}\boldsymbol{X}] + \boldsymbol{E}^{\text{pos}}. \tag{8.2.12}$$

   Aquí tenemos tres parámetros entrenables: $\boldsymbol{W}^{\text{emb}}$, $\boldsymbol{z}^1_{\text{cls}}$ y $\boldsymbol{E}^{\text{pos}}$, cuya función es la siguiente:

   - $\boldsymbol{W}^{\text{emb}} \in \mathbb{R}^{d\times D}$ es una matriz que proyecta cada vector de parche en una característica de token.
   - $\boldsymbol{z}^1_{\text{cls}} \in \mathbb{R}^d$ es el denominado *token de clase* o *token de registro*. El token de clase contiene, de manera heurística, información global de todos los datos y se utiliza para tareas posteriores. En el marco de las redes profundas compresivas de Capítulo 5, esperamos que el token de clase se proyecte en los mismos subespacios que los tokens destacados o semánticamente relevantes durante el desarrollo de la propagación hacia adelante.

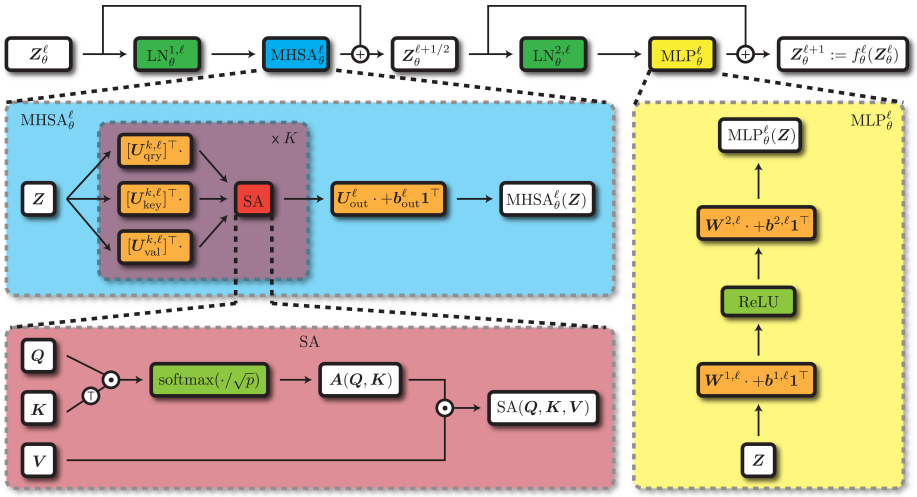


Figura 8.7: **Una capa f_θ^ℓ de la estructura principal del transformador.** Las características de entrada pasan por normalización de capa, autoatención multi-cabezal y bloques de perceptrón multi-capa en secuencia para formar las características de salida de la capa.

- $\boldsymbol{E}^{\mathrm{pos}} \in \mathbb{R}^{d \times N}$ es la denominada *codificación posicional*, que distingue entre los tokens de los distintos parches. Es decir, las características de los tokens deben contener información posicional, de modo que el mapa global f^{pre} no sea invariante ante las permutaciones de los parches, y $\boldsymbol{E}^{\mathrm{pos}}$ incorpora esta información posicional.
  - En este caso de DINO, donde el transformador recibe imágenes de distintos tamaños, aprendemos una codificación posicional para el tamaño más grande recibido durante el entrenamiento, e interpolamos para obtener las codificaciones posicionales para entradas de menor tamaño.

Por lo tanto, al final tenemos

$$f_\theta^{\mathrm{emb}}(\boldsymbol{X}) \doteq \left[\boldsymbol{z}_{\mathrm{cls}}^1, \boldsymbol{W}^{\mathrm{emb}} f^{\mathrm{patch}}(\boldsymbol{X}) + \boldsymbol{E}^{\mathrm{pos}}\right]. \tag{8.2.13}$$

Todos los parámetros $\boldsymbol{z}_{\mathrm{cls}}^1, \boldsymbol{W}^{\mathrm{emb}}, \boldsymbol{E}^{\mathrm{pos}}$ pertenecen al conjunto de parámetros θ.

Red troncal. Dada una secuencia de *embeddings* $\boldsymbol{Z}_\theta^1(\boldsymbol{X}) \doteq f_\theta^{\mathrm{emb}}(\boldsymbol{X}) \in (\mathbb{R}^d)^*$, la procesamos utilizando el mapa de la red troncal f_θ^{bb} de la siguiente manera, como se muestra en Figura 8.7. La función f_θ^{bb} se compone de L *capas* f_θ^ℓ, es decir,

$$f_\theta^{\mathrm{bb}} = f_\theta^L \circ \cdots \circ f_\theta^1. \tag{8.2.14}$$

La capa f_θ^ℓ tiene la siguiente implementación:

$$\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}) = \boldsymbol{Z}_\theta^\ell(\boldsymbol{X}) + \mathrm{MHSA}_\theta^\ell(\mathrm{LN}_\theta^{1,\ell}(\boldsymbol{Z}_\theta^\ell(\boldsymbol{X}))) \tag{8.2.15}$$

$$\boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X}) = \boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}) + \mathrm{MLP}_\theta^\ell(\mathrm{LN}_\theta^{2,\ell}(\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}))) \tag{8.2.16}$$

y f_θ^ℓ se define de tal manera que $f_\theta^\ell(\boldsymbol{Z}_\theta^\ell(\boldsymbol{X})) \doteq \boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X})$. Aquí hemos utilizado algunos operadores, tales como $\mathrm{MHSA}_\theta^\ell, \mathrm{MLP}_\theta^\ell$ y $\mathrm{LN}_\theta^{i,\ell}$, que se definen de la siguiente manera:

- El operador $\mathrm{MHSA}_\theta^\ell$ es una autoatención de múltiples cabezas, predecesora de la autoatención de subespacio de múltiples cabezas (véase Capítulo 5). La formulación es la siguiente:

$$\mathrm{MHSA}_\theta^\ell(\boldsymbol{Z}) \doteq \boldsymbol{U}_{\mathrm{out}}^\ell \begin{bmatrix} \mathrm{SA}([\boldsymbol{U}_{\mathrm{qry}}^{1,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{key}}^{1,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{val}}^{1,\ell}]^\top \boldsymbol{Z}) \\ \vdots \\ \mathrm{SA}([\boldsymbol{U}_{\mathrm{qry}}^{K,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{key}}^{K,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{val}}^{K,\ell}]^\top \boldsymbol{Z}) \end{bmatrix} + \boldsymbol{b}_{\mathrm{out}}^\ell \mathbf{1}_n^\top, \tag{8.2.17}$$

$$\text{donde} \qquad \mathrm{SA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \doteq \boldsymbol{V} \underbrace{\mathrm{softmax}\left(\frac{\boldsymbol{K}^\top \boldsymbol{Q}}{\sqrt{p}}\right)}_{\doteq \boldsymbol{A}(\boldsymbol{Q}, \boldsymbol{K})} \tag{8.2.18}$$

  donde p es un número entero positivo, $\boldsymbol{U}_{\mathrm{qry}}^{k,\ell}, \boldsymbol{U}_{\mathrm{key}}^{k,\ell}, \boldsymbol{U}_{\mathrm{val}}^{k,\ell} \in \mathbb{R}^{d\times p}$, $\boldsymbol{U}_{\mathrm{out}}^\ell \in \mathbb{R}^{d\times Kp}$, y $\boldsymbol{b}_{\mathrm{out}}^\ell \in \mathbb{R}^d$ son parámetros entrenables contenidos en el conjunto de parámetros θ, y la función softmax se define columna por columna como

$$\mathrm{softmax}(\boldsymbol{M}) \doteq \begin{bmatrix} \mathrm{softmax}(\boldsymbol{m}_1) & \cdots & \mathrm{softmax}(\boldsymbol{m}_p) \end{bmatrix}, \tag{8.2.19}$$

$$\forall \boldsymbol{M} = \begin{bmatrix} \boldsymbol{m}_1, \ldots, \boldsymbol{m}_p \end{bmatrix} \in \mathbb{R}^{n\times p}. \tag{8.2.20}$$

  En la práctica, las dimensiones suelen elegirse de tal manera que $Kp = d$. Los términos

$$\begin{aligned} \boldsymbol{A}_\theta^{k,\ell}(\boldsymbol{Z}) &\doteq \boldsymbol{A}([\boldsymbol{U}_{\mathrm{qry}}^{k,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{key}}^{k,\ell}]^\top \boldsymbol{Z}), \\ \mathrm{SA}_\theta^{k,\ell}(\boldsymbol{Z}) &\doteq \mathrm{SA}([\boldsymbol{U}_{\mathrm{qry}}^{k,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{key}}^{k,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}_{\mathrm{val}}^{k,\ell}]^\top \boldsymbol{Z}) \end{aligned} \tag{8.2.21}$$

  también se conocen como el *k-ésimo mapa de atención* y la *k-ésima salida de la cabeza de atención* en la capa ℓ, respectivamente. Además, la operación $\mathrm{SA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})$ se puede calcular de manera extremadamente eficiente utilizando software especializado como FlashAttention [SBZ+25].

- El MLP_θ^ℓ es un perceptrón de dos capas, una no linealidad común utilizada en redes profundas, y es de la forma

$$\mathrm{MLP}_\theta^\ell(\boldsymbol{Z}) \doteq \boldsymbol{W}_{\mathrm{down}}^\ell \,\mathrm{ReLU}(\boldsymbol{W}_{\mathrm{up}}^\ell \boldsymbol{Z} + \boldsymbol{b}_{\mathrm{up}}^\ell \mathbf{1}_n^\top) + \boldsymbol{b}_{\mathrm{down}}^\ell \mathbf{1}_n^\top \tag{8.2.22}$$

  donde $\boldsymbol{W}_{\mathrm{up}}^\ell \in \mathbb{R}^{q\times d}, \boldsymbol{W}_{\mathrm{down}}^\ell \in \mathbb{R}^{d\times q}, \boldsymbol{b}_{\mathrm{up}}^\ell \in \mathbb{R}^q, \boldsymbol{b}_{\mathrm{down}}^\ell \in \mathbb{R}^d$ son parámetros entrenables que también se encuentran en el conjunto de parámetros θ, y ReLU es la no linealidad ReLU componente por componente, es decir, $\mathrm{ReLU}(\boldsymbol{M})_{ij} = \max\{M_{ij}, 0\}$.

- Cada norma de capa $\mathrm{LN}_\theta^{i,\ell}$ para $i \in \{1,2\}$ es una normalización estándar, que se aplica columna por columna a cada característica de token de manera independiente:

$$\mathrm{LN}_\theta^{i,\ell}(\boldsymbol{Z}) = \mathrm{LN}_\theta^{i,\ell}(\big[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n\big]) = \big[\mathrm{LN}_\theta^{i,\ell}(\boldsymbol{z}_1), \ldots, \mathrm{LN}_\theta^{i,\ell}(\boldsymbol{z}_n)\big] \tag{8.2.23}$$

  y tiene la forma

$$\mathrm{LN}_\theta^{i,\ell}(\boldsymbol{z}) = \frac{\boldsymbol{z} - \mathrm{mean}(\boldsymbol{z})\mathbf{1}_d}{\|\boldsymbol{z} - \mathrm{media}(\boldsymbol{z})\mathbf{1}_d\|_2} \odot \boldsymbol{\alpha}^{i,\ell} + \boldsymbol{\beta}^{i,\ell} \tag{8.2.24}$$

$$\text{donde} \qquad \mathrm{mean}(\boldsymbol{z}) = \frac{1}{d}\mathbf{1}_d^\top \boldsymbol{z} \tag{8.2.25}$$

  donde $\odot$ denota la multiplicación elemento por elemento, y $\boldsymbol{\alpha}^{i,\ell}, \boldsymbol{\beta}^{i,\ell} \in \mathbb{R}^d$ son parámetros entrenables contenidos en el conjunto de parámetros θ. El operador de norma de capa funciona como una especie de normalización de cada token, cuya escala es posteriormente aprendible y se comparte entre todos los tokens.

El transformador es una de las arquitecturas de redes neuronales más populares de la historia, e impulsa aplicaciones de casi todos los campos del aprendizaje profundo.

Extractor de características. Utilizamos un paso de posprocesamiento f_θ^{ext} que extrae la *característica del token de clase* , la cual (recordemos) es la característica destinada a contener información agregada sobre la imagen de entrada, y le aplica una MLP con normalización. Es decir, tenemos

$$\boldsymbol{z}_\theta(\boldsymbol{X}) \doteq f_\theta^{\mathrm{ext}}(\boldsymbol{Z}_\theta(\boldsymbol{X})) = f_\theta^{\mathrm{ext}}([\boldsymbol{z}_\theta^1(\boldsymbol{X}), \ldots, \boldsymbol{z}_\theta^n(\boldsymbol{X})]) \tag{8.2.26}$$

$$\doteq \frac{\mathrm{MLP}_\theta^{\mathrm{ext}}(\boldsymbol{z}_\theta^1(\boldsymbol{X}))}{\|\,\mathrm{MLP}_\theta^{\mathrm{ext}}(\boldsymbol{z}_\theta^1(\boldsymbol{X}))\|_2}. \tag{8.2.27}$$

Cabezal específico «DINO» para la tarea. En el caso de DINO, utilizamos el cabezal específico para la tarea $h_{\boldsymbol{W},\boldsymbol{\mu}}$. Para SimDINO, no utilizamos *ningún* cabezal específico para la tarea *en absoluto*, tal como se describió anteriormente.

8.2.4 Estrategia de optimización

Optimización de DINO. Contamos con una función de pérdida y una arquitectura, por lo que ahora analizaremos la estrategia de optimización. La estrategia de optimización de DINO utiliza *dos conjuntos de pesos para la misma arquitectura*: los pesos del *estudiante* θ_{s} y los pesos del *maestro* θ_{t}. Estos corresponden a dos redes neuronales diferentes, llamadas red «maestra» y red «estudiante», con la misma arquitectura. La red maestra codifica todas las vistas globales, mientras que la red estudiante codifica todas las «otras» vistas.

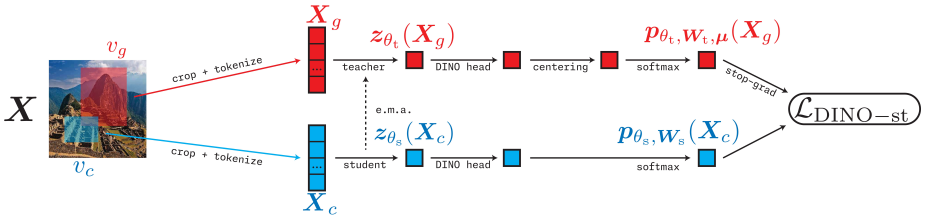


Figura 8.8: **El proceso DINO.** Las características del estudiante y las del maestro se calculan para cada entrada. El objetivo es tratar de alinear las características del estudiante con las del maestro proyectando ambos conjuntos de características en un simplex de probabilidad de alta dimensión y calculando una pérdida de entropía cruzada. Cabe destacar que, debido al «stop-grad», el gradiente solo se calcula con respecto a las salidas de los parámetros del *estudiante*.

El objetivo de la pérdida es destilar las salidas de la red maestra en el modelo estudiante. Es decir, entrenamos minimizando la pérdida $\mathcal{L}_{\text{DINO}-\text{st}}$:

$$\mathcal{L}_{\text{DINO}-\text{st}}(\theta_s, \theta_t, \boldsymbol{W}_s, \boldsymbol{W}_t, \boldsymbol{\mu}) \doteq \mathbb{E}[d_{\text{CE}}(\boldsymbol{p}_{\theta_t, \boldsymbol{W}_t, \boldsymbol{\mu}}(\boldsymbol{X}_g), \boldsymbol{p}_{\theta_s, \boldsymbol{W}_s}(\boldsymbol{X}_c))]. \tag{8.2.28}$$

Ahora, podemos describir completamente el *pipeline* general de DINO, representado en Figura 8.8.

Si bien es fácil razonar sobre (8.2.28), en la práctica es imposible implementar algoritmos de optimización como el descenso de gradiente con una pérdida dada por $\mathcal{L}_{\text{DINO}-\text{st}}$. Esto se debe a que las expectativas en la pérdida son imposibles de evaluar, y mucho menos de calcular su gradiente. En este caso extremadamente frecuente, aproximamos la esperanza mediante muestras finitas. Es decir, en cada paso de tiempo k hacemos lo siguiente:

- Se toma una submuestra de B puntos de datos del conjunto de datos $\{\boldsymbol{X}_1^{(k)}, \ldots, \boldsymbol{X}_B^{(k)}\} \subset \mathcal{I}$.
- Para cada punto de datos $\boldsymbol{X}_b^{(k)}$, se toma una muestra de M_{glo} vistas globales $v_{b,g}^{(k),i}$ y M_{loc} vistas locales $v_{b,\ell}^{(k),i}$. Se aplican las vistas a $\boldsymbol{X}_b^{(k)}$ para obtener $\boldsymbol{X}_{b,g}^{(k),i} \doteq v_{b,g}^{(k),i}(\boldsymbol{X}_b^{(k)})$ y $\boldsymbol{X}_{b,\ell}^{(k),i} \doteq v_{b,\ell}^{(k),i}(\boldsymbol{X}_b^{(k)})$.
- Para cada vista *local* $\boldsymbol{X}_{b,\ell}^{(k),i}$, se calculan las siguientes cantidades:

$$\boldsymbol{z}_{\theta_s}(\boldsymbol{X}_{b,\ell}^{(k),i}) \doteq (f_{\theta_s}^{\text{ext}} \circ f_{\theta_s})(\boldsymbol{X}_{b,\ell}^{(k),i}), \tag{8.2.29}$$

$$\boldsymbol{p}_{\theta_s, \boldsymbol{W}_s}(\boldsymbol{X}_{b,\ell}^{(k),i}) \doteq h_{\boldsymbol{W}_s, \boldsymbol{0}_m}(\boldsymbol{z}_{\theta_s}(\boldsymbol{X}_{b,\ell}^{(k),i}(\theta))) \tag{8.2.30}$$

  y para cada vista *global* $\boldsymbol{X}_{b,g}^{(k),i}$, se calculan las siguientes cantidades (por un abuso de notación):

$$\boldsymbol{z}_{\theta_s}(\boldsymbol{X}_{b,g}^{(k),i}) \doteq (f_{\theta_s}^{\text{ext}} \circ f_{\theta_s})(\boldsymbol{X}_{b,g}^{(k),i}), \tag{8.2.31}$$

$$\boldsymbol{p}_{\theta_s, \boldsymbol{W}_s}(\boldsymbol{X}_{b,g}^{(k),i}) \doteq h_{\boldsymbol{W}_s, \boldsymbol{0}_m}(\boldsymbol{z}_{\theta_s}(\boldsymbol{X}_{b,g}^{(k),i})), \tag{8.2.32}$$

$$\boldsymbol{z}_{\theta_\mathrm{t}}(\boldsymbol{X}_{b,g}^{(k),i}) \doteq (f_{\theta_\mathrm{t}}^{\mathrm{ext}} \circ f_{\theta_\mathrm{t}})(\boldsymbol{X}_{b,g}^{(k),i}), \tag{8.2.33}$$

$$\boldsymbol{p}_{\theta_\mathrm{t},\boldsymbol{W}_\mathrm{t},\boldsymbol{\mu}}(\boldsymbol{X}_{b,g}^{(k),i}) \doteq h_{\boldsymbol{W}_\mathrm{t},\boldsymbol{\mu}}(\boldsymbol{Z}_{\theta_\mathrm{t}}(\boldsymbol{X}_{b,g}^{(k),i})). \tag{8.2.34}$$

- Se calcula la *pérdida sustituta o aproximada* $\hat{\mathcal{L}}_{\mathrm{DINO-st}}^{(k)}$, definida de la siguiente manera:

$$\hat{\mathcal{L}}_{\mathrm{DINO-st}}^{(k)}(\theta_\mathrm{s}, \theta_\mathrm{t}, \boldsymbol{W}_\mathrm{s}, \boldsymbol{W}_\mathrm{t}, \boldsymbol{\mu}) \doteq \frac{1}{BM_{\mathrm{glo}}(M_{\mathrm{glo}} + M_{\mathrm{loc}} - 1)} \sum_{b=1}^{B} \sum_{i=1}^{M_{\mathrm{glo}}} \tag{8.2.35}$$

$$\left[\sum_{j=1}^{M_{\mathrm{loc}}} d_{\mathrm{CE}}(\boldsymbol{p}_{\theta_\mathrm{t},\boldsymbol{W}_\mathrm{t},\boldsymbol{\mu}}(\boldsymbol{X}_{b,g}^{(k),i}), \boldsymbol{p}_{\theta_\mathrm{s},\boldsymbol{W}_\mathrm{s}}(\boldsymbol{X}_{b,\ell}^{(k),j})) \right. \tag{8.2.36}$$

$$\left. + \sum_{\substack{j=1 \\ j \neq i}}^{M_{\mathrm{glo}}} d_{\mathrm{CE}}(\boldsymbol{p}_{\theta_\mathrm{t},\boldsymbol{W}_\mathrm{t},\boldsymbol{\mu}}(\boldsymbol{X}_{b,g}^{(k),i}), \boldsymbol{p}_{\theta_\mathrm{s},\boldsymbol{W}_\mathrm{s}}(\boldsymbol{X}_{b,g}^{(k),j})) \right] \tag{8.2.37}$$

  así como sus gradientes respecto de θ_s y $\boldsymbol{W}_\mathrm{s}$, los cuales deben calcularse bajo el supuesto de que θ_t, $\boldsymbol{W}_\mathrm{t}$ y $\boldsymbol{\mu}$ son constantes — es decir, que están *desconectados del grafo computacional* y que no dependen de θ_s ni de $\boldsymbol{W}_\mathrm{s}$.

- Actualizar los parámetros del estudiante θ_s y $\boldsymbol{W}_\mathrm{s}$ mediante un algoritmo iterativo de optimización basado en el gradiente, y actualizar θ_t, $\boldsymbol{W}_\mathrm{t}$ y $\boldsymbol{\mu}$ mediante promedios móviles exponenciales con parámetros de decaimiento $\nu^{(k)}$, $\nu^{(k)}$ y $\rho^{(k)}$, respectivamente; es decir,

$$(\theta_\mathrm{s}^{(k+1)}, \boldsymbol{W}_\mathrm{s}^{(k+1)}) = \textsc{OptUpdate}^{(k)}(\theta_\mathrm{s}^{(k)}, \boldsymbol{W}_\mathrm{s}^{(k)}; \nabla_{(\theta_\mathrm{s},\boldsymbol{W}_\mathrm{s})} \hat{\mathcal{L}}_{\mathrm{DINO-st}}^{(k)}) \tag{8.2.38}$$

$$\theta_\mathrm{t}^{(k+1)} = \nu^{(k)} \theta_\mathrm{t}^{(k)} + (1 - \nu^{(k)}) \theta_\mathrm{s}^{(k+1)} \tag{8.2.39}$$

$$\boldsymbol{W}_\mathrm{t}^{(k+1)} = \nu^{(k)} \boldsymbol{W}_\mathrm{t}^{(k)} + (1 - \nu^{(k)}) \boldsymbol{W}_\mathrm{s}^{(k+1)} \tag{8.2.40}$$

$$\boldsymbol{\mu}^{(k+1)} = \rho^{(k)} \boldsymbol{\mu}^{(k)} + (1 - \rho^{(k)}) \cdot \frac{1}{BM_{\mathrm{glo}}} \sum_{b=1}^{B} \sum_{i=1}^{M_{\mathrm{glo}}} \boldsymbol{W}^{(k)} \boldsymbol{z}_{\theta_\mathrm{t}}(\boldsymbol{X}_{b,g}^{(k),i}), \tag{8.2.41}$$

  Por ejemplo, si el algoritmo de optimización elegido fuera el descenso de gradiente estocástico, la actualización resultante sería $\theta_{\mathrm{estudiante}}^{(k+1)} \doteq \theta_{\mathrm{estudiante}}^{(k)} - \delta^{(k)} \nabla_{\theta_{\mathrm{estudiante}}} \hat{\mathcal{L}}_{\mathrm{dino-estudiante-maestro}}^{(k)}$, y así sucesivamente.

Obsérvese que el procedimiento de optimización es bastante irregular: aunque los cuatro parámetros cambian en cada iteración, solo dos se actualizan directamente mediante un método basado en gradientes. Los otros dos se actualizan mediante promedios móviles exponenciales y, de hecho, se tratan

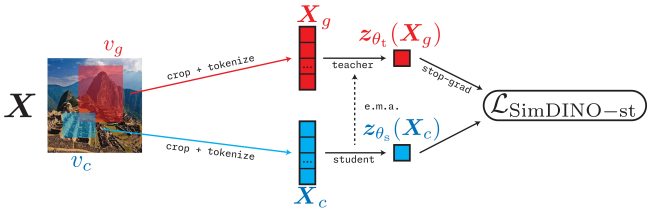


Figura 8.9: **El proceso de SimDINO.** Aquí, a diferencia del proceso DINO en Figura 8.8, la pérdida se calcula directamente sobre las características sin necesidad de manipulación adicional. Esto elimina una cantidad de parámetros equivalente a varias matrices grandes y simplifica el proceso, al tiempo que lo hace más estable para el entrenamiento.

como constantes al calcular cualquier gradiente. Al finalizar el entrenamiento, descartamos los pesos del «estudiante» y empleamos los del «maestro» para la red entrenada f, dado que se ha demostrado empíricamente que este promedio móvil exponencial estabiliza el modelo resultante (esta idea se conoce como promedio de Polyak o promedio de iteraciones).

La forma en que ν y ρ varían a lo largo de la trayectoria de optimización (es decir, las funciones $k \mapsto \nu^{(k)}$ y $k \mapsto \rho^{(k)}$) son hiperparámetros o decisiones de diseño, siendo $\nu^{(1)} < 1$ y $\lim_{k\to\infty} \nu^{(k)} = 1$ por lo general, y lo mismo ocurre con ρ. El hiperparámetro de temperatura τ, utilizado en la cabeza DINO $h_{\boldsymbol{W},\boldsymbol{\mu}}$, también varía a lo largo de la trayectoria de optimización (aunque esta dependencia no se denota explícitamente).

El uso de la pérdida sustituta ("empírica") transforma nuestro problema de optimización intractable, como la optimización de la pérdida en (8.2.28), en un problema de optimización estocástica tractable que se ejecuta para entrenar prácticamente todos los modelos de aprendizaje profundo del mundo. Esta conversión resulta sumamente natural una vez que se han visto algunos ejemplos, y con toda probabilidad presentaremos estos ejemplos a lo largo del capítulo.

Optimización de SimDINO. El objetivo simplificado de DINO a nivel de población es muy similar en esencia, pero mucho más sencillo de ejecutar, es decir:

$$\mathcal{L}_{\text{SimDINO}-\text{st}}(\theta_s, \theta_t) \doteq \mathbb{E}[d_{\ell^2}(\boldsymbol{z}_{\theta_t}(\boldsymbol{X}_g), \boldsymbol{z}_{\theta_s}(\boldsymbol{X}_c))] \tag{8.2.42}$$

$$- \frac{\gamma}{2} \log\det\left(\boldsymbol{I} + \frac{d}{\varepsilon^2} \operatorname{Cov}(\boldsymbol{z}_{\theta_s}(\boldsymbol{X}_g))\right). \tag{8.2.43}$$

Por lo tanto, tal como se explica en Figura 8.9, el proceso de SimDINO es inherentemente más sencillo que el de DINO. Podemos utilizar una versión más sencilla del proceso de entrenamiento de DINO para optimizar SimDINO. En cada paso temporal k, hacemos lo siguiente:

- Tomar una submuestra de B puntos de datos de nuestro conjunto de datos $\{\boldsymbol{X}_1^{(k)}, \ldots, \boldsymbol{X}_B^{(k)}\} \subset \mathcal{I}$.

- Para cada punto de datos $\boldsymbol{X}_b^{(k)}$, muestrear M_{glo} vistas globales $v_{b,g}^{(k),i}$ y M_{loc} vistas locales $v_{b,\ell}^{(k),i}$. Aplicamos las vistas a $\boldsymbol{X}_b^{(k)}$ para obtener $\boldsymbol{X}_{b,g}^{(k),i} \doteq v_{b,g}^{(k),i}(\boldsymbol{X}_b^{(k)})$ y $\boldsymbol{X}_{b,\ell}^{(k),i} \doteq v_{b,\ell}^{(k),i}(\boldsymbol{X}_b^{(k)})$.

- Para cada vista *local* $\boldsymbol{X}_{b,\ell}^{(k),i}$, calcular $\boldsymbol{z}_{\theta_{\text{s}}}(\boldsymbol{X}_{b,\ell}^{(k),i}) \doteq (f_{\theta_{\text{s}}}^{\text{ext}} \circ f_{\theta_{\text{s}}})(\boldsymbol{X}_{b,\ell}^{(k),i}$. Para cada vista *global* $\boldsymbol{X}_{b,g}^{(k),i}$, calcula $\boldsymbol{z}_{\theta_{\text{s}}}(\boldsymbol{X}_{b,g}^{(k),i}) \doteq (f_{\theta_{\text{s}}}^{\text{ext}} \circ f_{\theta_{\text{s}}})(\boldsymbol{X}_{b,g}^{(k),i}$ y $\boldsymbol{z}_{\theta_{\text{t}}}(\boldsymbol{X}_{b,g}^{(k),i}) \doteq (f_{\theta_{\text{t}}}^{\text{ext}} \circ f_{\theta_{\text{t}}})(\boldsymbol{X}_{b,g}^{(k),i})$.

- Calcular la *pérdida sustituta aproximada* $\hat{\mathcal{L}}_{\text{SimDINO}-\text{st}}^{(k)}$, definida como sigue:

$$\hat{\mathcal{L}}_{\text{SimDINO}-\text{st}}^{(k)}(\theta_{\text{s}}, \theta_{\text{t}}) \tag{8.2.44}$$

$$\doteq \frac{1}{BM_{\text{glo}}(M_{\text{glo}} + M_{\text{loc}} - 1)} \sum_{b=1}^{B} \sum_{i=1}^{M_{\text{glo}}} \left[\sum_{j=1}^{M_{\text{loc}}} d_{\ell^2}(\boldsymbol{z}_{\theta_{\text{t}}}(\boldsymbol{X}_{b,g}^{(k),i}), \boldsymbol{z}_{\theta_{\text{s}}}(\boldsymbol{X}_{b,\ell}^{(k),j})) \right. \tag{8.2.45}$$

$$\left. + \sum_{j=1}^{M_{\text{glo}}} d_{\ell^2}(\boldsymbol{z}_{\theta_{\text{t}}}(\boldsymbol{X}_{b,g}^{(k),i}), \boldsymbol{z}_{\theta_{\text{s}}}(\boldsymbol{X}_{b,g}^{(k),j})) \right] \tag{8.2.46}$$

$$- \frac{\gamma}{M_{\text{glo}}} \sum_{i=1}^{M_{\text{glo}}} R_{\varepsilon}([\boldsymbol{z}_{\theta_{\text{s}}}(\boldsymbol{X}_{1,g}^{(k),i}), \ldots, \boldsymbol{z}_{\theta_{\text{s}}}(\boldsymbol{X}_{B,g}^{(k),i})]) \tag{8.2.47}$$

  donde R_{ε} es la tasa de codificación gaussiana estimada a partir de muestras finitas, tal como se describe en Capítulo 5. El gradiente de $\hat{\mathcal{L}}_{\text{SimDINO}-\text{st}}^{(k)}$ con respecto a θ_{s} debe calcularse (una vez más), bajo el supuesto de que θ_{t} es constante.

- Actualizar los parámetros del estudiante θ_{s} mediante un algoritmo iterativo de optimización basado en el gradiente, y actualizar θ_{t} mediante un promedio móvil exponencial con el parámetro de decaimiento $\nu^{(k)}$, es decir,

$$\theta_{\text{s}}^{(k+1)} = \textsc{OptUpdate}^{(k)}(\theta_{\text{s}}^{(k)}; \nabla_{\theta_{\text{s}}} \hat{\mathcal{L}}_{\text{SimDINO}-\text{st}}^{(k)}) \tag{8.2.48}$$

$$\theta_{\text{t}}^{(k+1)} = \nu^{(k)} \theta_{\text{t}}^{(k)} + (1 - \nu^{(k)}) \theta_{\text{s}}^{(k+1)}. \tag{8.2.49}$$

Una vez más, reiteramos que el gradiente solo se calcula con respecto a θ_{s}, considerando θ_{t} como una constante. Aquí, cabe señalar que, si bien la elección de ν sigue siendo una decisión de diseño, los hiperparámetros ρ y τ se eliminan.

8.2.5 Metodología de evaluación

Existen varias formas de evaluar un modelo Transformer entrenado. En esta sección destacamos dos. Definamos la vista de *recorte central* $v_{\text{cc}} \colon \mathcal{I} \to \mathcal{I}$, que es un *recorte redimensionado determinístico* :

- redimensiona la imagen de modo que el borde más corto tenga un tamaño de S_{rsz} (similar a los recortes redimensionados al azar con el parámetro de porcentaje de área 1);

- luego toma el recorte *central* $S_{\text{cc}} \times S_{\text{cc}}$;

de modo que la forma final es $(C, S_{\text{cc}}, S_{\text{cc}})$. Obsérvese que la vista v_{cc} es completamente determinista dada una entrada. Para una entrada $\boldsymbol{X}$, escribimos $\boldsymbol{X}_{\text{cc}} \doteq v_{\text{cc}}(\boldsymbol{X})$. Aquí $S_{\text{cc}} \leq S_{\text{rsz}}$.

Sondeo lineal. La primera forma —y la más independiente de la arquitectura— de evaluar un modelo de codificador $\boldsymbol{X} \mapsto \boldsymbol{z}_\theta(\boldsymbol{X})$ es emplear el *sondeo lineal* . El sondeo lineal consiste, en pocas palabras, en aplicar una regresión logística a las características globalmente agregadas calculadas por el codificador. Esto nos indica cuánta información semántica existe en las representaciones, así como la facilidad con la que se puede extraer dicha información. (Es decir: ¿ hasta qué punto las características de las imágenes con diferentes semánticas se encuentran en diferentes subespacios del espacio de características?)

De manera más formal, supongamos que queremos evaluar la calidad y la fidelidad de las características del codificador en datos de pares imagen-etiqueta $(\boldsymbol{X}, \boldsymbol{y})$, donde hay N_{cls} clases y $\boldsymbol{y} \in \{0,1\}^{N_{\text{cls}}}$ es una «codificación one-hot» (es decir, ceros en todas las posiciones salvo un 1 en la posición i si $\boldsymbol{X}$ pertenece a la i-ésima clase). Una forma de hacerlo es resolver el problema empírico de regresión logística

$$\min_{\boldsymbol{W} \in \mathbb{R}^{N_{\text{cls}} \times d}} \mathbb{E}[\text{CE}(\boldsymbol{y}, \boldsymbol{W}\boldsymbol{z}_\theta(\boldsymbol{X}_{\text{cc}}))]. \tag{8.2.50}$$

En términos más prácticos, si contamos con datos etiquetados $\{(\boldsymbol{X}_b, \boldsymbol{y}_b)\}_{b=1}^B$, podemos resolver el problema de regresión logística *empírica* (similar a (8.2.28) frente a (8.2.35)) dado por

$$\min_{\boldsymbol{W} \in \mathbb{R}^{N_{\text{cls}} \times d}} \frac{1}{B} \sum_{b=1}^{B} \text{CE}(\boldsymbol{y}_b, \boldsymbol{W}\boldsymbol{z}_\theta(\boldsymbol{X}_{b,\text{cc}})). \tag{8.2.51}$$

Este problema es un problema de optimización convexa en $\boldsymbol{W}$, y por lo tanto puede resolverse de manera eficiente mediante el descenso de gradiente (estocástico) o una larga lista de otros algoritmos. Esta sonda lineal, junto con el codificador, puede utilizarse como clasificador, y podemos evaluar la precisión de la clasificación. La práctica habitual consiste en entrenar primero el modelo en un conjunto de datos grande (como ImageNet-1K), luego entrenar la sonda lineal en un conjunto de datos (como el conjunto de datos de entrenamiento de CIFAR-10) y evaluarla en un tercer conjunto de datos reservado (o *holdout*) que se extrae de la misma distribución que el segundo (como el conjunto de datos de evaluación de CIFAR-10).

k **vecinos más cercanos.** También podemos evaluar el rendimiento de las características en tareas de clasificación *sin necesidad de entrenar explícitamente un clasificador* utilizando el algoritmo de los k vecinos más cercanos para obtener una etiqueta predicha promedio. Es decir, dado un conjunto de datos $\{\boldsymbol{z}_b\}_{b=1}^B \subseteq \mathbb{R}^d$, definimos los k vecinos más cercanos de otro punto $\boldsymbol{z} \in \mathbb{R}^d$ como $\mathrm{NN}_k(\boldsymbol{z}, \{\boldsymbol{z}_b\}_{b=1}^B)$. Con esta notación, se puede calcular la etiqueta predicha $\hat{\boldsymbol{y}}_\theta(\boldsymbol{X} \mid \{(\boldsymbol{X}_b, \boldsymbol{y}_b)\}_{b=1}^B)$ como

$$\begin{aligned} &\hat{\boldsymbol{y}}_\theta(\boldsymbol{X} \mid \{(\boldsymbol{X}_b, \boldsymbol{y}_b)\}_{b=1}^B) = \mathbf{1}(i^\star) \quad \text{donde} \\ &i^\star \doteq \arg\max_{i \in [Q]} \sum_{b=1}^B \boldsymbol{y}_b \mathbf{1}[\boldsymbol{z}_\theta(\boldsymbol{X}_{\mathrm{cc},b}) \in \mathrm{NN}_k(\boldsymbol{z}_\theta(\boldsymbol{X}_{\mathrm{cc}}))]. \end{aligned} \tag{8.2.52}$$

Aquí, $\mathbf{1}(i) \in \Delta_{N_{\mathrm{cls}}}$ es (mediante un abuso de notación, véanse las variables indicadoras) el vector de probabilidad «one-hot» con soporte en i, es decir, 1 en la i-ésima coordenada y 0 en todas las demás. Es decir, este procedimiento toma la etiqueta más común entre los k puntos más cercanos en el espacio de características. La precisión de la clasificación de los k vecinos más cercanos es simplemente la precisión de esta etiqueta predicha, es decir,

$$\mathbb{E}_{\boldsymbol{X},\boldsymbol{y}}[\mathbf{1}(\hat{\boldsymbol{y}}_\theta(\boldsymbol{X} \mid \{(\boldsymbol{X}_b, \boldsymbol{y}_b)\}_{b=1}^B) = \boldsymbol{y})] \tag{8.2.53}$$

o, con más frecuencia, su versión empírica correspondiente, donde $(\boldsymbol{X}, \boldsymbol{y})$ recorre un conjunto de datos finito (*no* las muestras existentes $(\boldsymbol{X}_b, \boldsymbol{y}_b)$ que se utilizan para los k vecinos).

Fidelidad de los mapas de atención. Otra forma de evaluar el desempeño de las representaciones, en el caso de un codificador basado en transformadores, es examinar la fidelidad de los mapas de atención $\boldsymbol{A}^{L,k} \in \mathbb{R}^{n \times n}$, tal como se definen en Sección 8.2.3, en la última capa L, y que se obtienen mediante la siguiente *pipeline*:

$$\boldsymbol{X} \mapsto \cdots \mapsto \boldsymbol{Z}^{L-1} = [\underbrace{\boldsymbol{z}_1^{L-1}}_{\text{token de clase}}, \underbrace{\boldsymbol{z}_2^{L-1} \ldots, \boldsymbol{z}_n^{L-1}}_{\text{tokens de parche}}] \mapsto \boldsymbol{A}^{k,L} = \begin{bmatrix} \boldsymbol{A}_{1,1}^{k,L} & \boldsymbol{A}_{1,2:}^{k,L} \\ \boldsymbol{A}_{2:,1}^{k,L} & \boldsymbol{A}_{2:,2:}^{k,L} \end{bmatrix}. \tag{8.2.54}$$

En particular, analizamos qué revelan los mapas de atención para una entrada dada acerca de los objetos salientes en la imagen de entrada, es decir, qué partes de la imagen proporcionan la información más relevante a nivel global para el token de clase. Una forma concreta de hacerlo es examinar el componente del mapa de atención en el que el token de clase se extrae como consulta y se elimina de la matriz de valores, es decir, $\boldsymbol{A}_{2:,1}^{k,L} \in \mathbb{R}^{1 \times (n-1)} = \mathbb{R}^{1 \times N}$ o su transpuesta $\boldsymbol{a}^{k,L} = (\boldsymbol{A}_{2:,1}^{k,L})^\top \in \mathbb{R}^N$. Obsérvese que este vector $\boldsymbol{a}^{k,L}$, al que denominamos «*vector de saliencia en la k-ésima cabeza de atención de la capa L*», tiene un valor para cada parche, $1, \ldots, N$, y utilizamos este valor para describir qué tan relevante es cada parche respecto a la información global. En particular, con fines de visualización, creamos una nueva imagen en la que cada parche se

reemplaza por su valor correspondiente en el vector de saliencia, lo que muestra la contribución de cada parche; llamamos a esta imagen el "*mapa de saliencia* en la k-ésima cabeza de atención de la capa L". Para visualizar la relevancia total de cada parche respecto a la información global en todas las cabezas, podemos calcular el promedio del vector de saliencia, es decir, $\tilde{\boldsymbol{a}}^L \doteq \frac{1}{K}\sum_{k=1}^{K} \boldsymbol{a}^{k,L}$ y expandirlo en el *mapa de saliencia promedio*. Los mapas de saliencia promedio deberían resaltar las partes relevantes de la imagen de entrada.

Detección de objetos y segmentaciónde imágenes. Podemos evaluar cómo las representaciones capturan las propiedades finas (es decir, aquellas de menor escala o de mayor detalle) de la entrada al utilizarlas para la *segmentación semántica*. A grandes rasgos, esto significa que utilizamos las características para construir rectángulos delimitadores para todos los objetos de la entrada. Existen varias formas de hacerlo, así como varias formas de evaluar los rectángulos delimitadores resultantes en comparación con la verdad de referencia. Cada combinación de métodos corresponde a una métrica de segmentación en particular. No las describimos formalmente aquí, ya que no son particularmente reveladoras, pero el artículo sobre DINO [CTM+21] y el artículo sobre DINOv2 [ODM+23] contienen referencias a todas las métricas que se utilizan en la práctica.

8.2.6 Configuración experimental y resultados

Dado que SimDINO se basa directamente en DINO, comparamos los parámetros óptimos de DINO, tal y como se indican en su artículo original [CTM+21], con los mismos parámetros aplicados a SimDINO [WZP+25] para garantizar una comparación imparcial.

Función objetivo. Utilizamos 10 vistas locales (es decir, $M_{\text{loc}} = 10$) con una resolución de 96×96 (es decir, $S_{\text{loc}} = 96$) y 2 vistas globales (es decir, $M_{\text{glo}} = 2$) con una resolución de 224×224 (es decir, $S_{\text{glo}} = 224$) para todos los experimentos. Las propoeciones correspondientes de las imágenes originales recortadas para las vistas locales y globales son $p_{\text{loc}} \in [\frac{1}{20}, \frac{3}{10}]$ y $p_{\text{glo}} \in [\frac{3}{10}, 1]$ (elegidas al azar por vista). El tamaño del lado más pequeño dentro de los recortes redimensionados es $S_{\text{rsz}} = 256$, y el tamaño de borde de la vista del recorte central (evaluación) es $S_{\text{cc}} = 224$. Todos estos ajustes se aplican tanto a DINO como a SimDINO.

Arquitectura del Modelo. Para todas las entradas, establecemos el tamaño del parche en 16×16 (es decir, $P_H = P_W = 16$). Utilizamos los modelos pequeño, básico y grande de la arquitectura ViT [DBK+21] como red de *embedding* y *backbone*. El extractor de características es una red neuronal multi-capas (MLP) de tres capas con un tamaño de las capas ocultas de 2048 y una dimensión de salida de 256, seguida de una normalización ℓ^2, tal y como se especifica en Sección 8.2.3. Para las arquitecturas DINO (es decir, no las arquitecturas

SimDINO), el cabezal DINO $\boldsymbol{W}$ es una matriz en $\mathbb{R}^{65536\times 256}$, y el parámetro $\boldsymbol{\mu}$ es un vector en $\mathbb{R}^{65536}$.

Conjuntos de datos y optimización. Para el preentrenamiento, tanto nuestra reproducción de DINO como SimDINO utilizan el conjunto de datos ImageNet-1K en todos los métodos. Utilizamos AdamW [LH17] como optimizador, lo cual es una elección muy habitual. Seguimos las siguientes recomendaciones de hiperparámetros:

- El tamaño del lote es $B = 1024$.
- La tasa de aprendizaje (para AdamW y el modelo estudiante) tiene un valor «base» de 2×10^{-3}. En las primeras 10 épocas, la tasa de aprendizaje aumenta linealmente de 0 al valor base (es decir, en la época i la tasa de aprendizaje es $(i/10)\cdot 2\times 10^{-3}$, para $1 \leq i \leq 10$). A continuación, durante las siguientes 90 épocas, la tasa de aprendizaje disminuye siguiendo el denominado *programa coseno* hasta volver a 0. La definición de un programa coseno se encuentra en muchos sitios, incluida la documentación de PyTorch y se utiliza habitualmente en el entrenamiento de modelos de visión profunda.
- La disminución de la ponderación (la W de AdamW) sigue un programa coseno que va de 0,04 a $0,4$ a lo largo del entrenamiento.
- La tasa EMA ν sigue un programa coseno desde $0,996$ hasta $1,0$ a lo largo del entrenamiento. En el caso concreto de DINO, la tasa EMA de centrado ρ se fija en $0,9$.
- En el caso concreto de DINO, la temperatura del maestro τ_{t} se fija en $0,1$, mientras que la temperatura del estudiante τ_{s} aumenta linealmente de $0,04$ a $0,07$ durante las primeras 30 épocas y se fija en $0,07$ a partir de entonces.

Utilizamos algunas técnicas de augmentación de datos (que, en esencia, conservan la información), como el volteo, la variación de color, el desenfoque gaussiano y la solarización, para cada imagen observada durante el entrenamiento, antes de obtener las vistas local y global. Los hiperparámetros exactos que rigen estas técnicas no se enumeran aquí, pero se detallan en el artículo sobre DINO [CTM+21].

En el caso del sondeo lineal (linear probing), la sonda lineal suele entrenarse utilizando el optimizador AdamW con una tasa de aprendizaje de 2×10^{-4}, una decadencia de pesos de $0,01$ y un tamaño de lote de 512, aunque estos parámetros suelen modificarse según cada caso concreto para minimizar la pérdida.

Resultados de la evaluación. En cuanto al rendimiento en la clasificación aguas abajo, obtenemos los resultados que se muestran en Tabla 8.1. Observamos que el rendimiento de SimDINO es mucho mayor que el de DINO

Tabla 8.1: **Rendimiento en clasificación** en datos de prueba reservados de validación para DINO y SimDINO, utilizando tanto la precisión de los k vecinos más cercanos ($k = 20$) como el sondeo lineal. Con el mismo número de iteraciones (100), SimDINO es claramente mejor en términos de rendimiento y es más estable (el entrenamiento de DINO ejecutado en el *backbone* ViT-L con la configuración proporcionada tiene una optimización muy inestable y genera una pérdida NaN en poco tiempo). También comparamos con otros métodos destacados, concretamente SwAV y MoCov3, en los que se basa DINO.

Método	Modelo	Épocas	20-NN	Sondeo lineal
DINO	ViT-B	100	72.9	76.3
SimDINO	ViT-B	100	**74.9**	**77.3**
DINO	ViT-L	100	–	–
SimDINO	ViT-L	100	**75.6**	**77.4**
SwAV	ViT-S	800	66.3	73.5
MoCov3	ViT-B	300	–	76.7

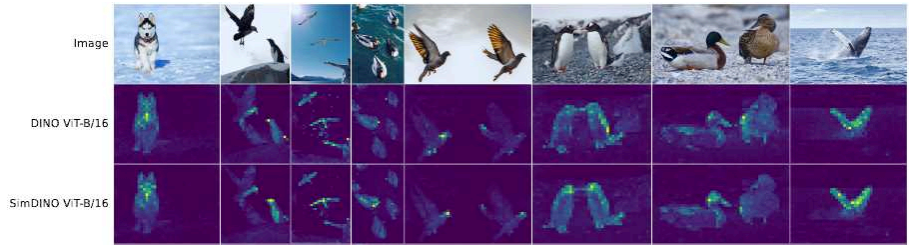


Figura 8.10: **Una comparación cualitativa de los mapas de saliencia** generados por DINO *(fila del medio)* y por SimDINO *(fila inferior)*. Para cada imagen, calculamos y mostramos el mapa de saliencia promedio en la última capa L. Los mapas de saliencia son similares entre los modelos, lo que significa que todos los modelos convergen en una noción similar de qué objetos son importantes. Tenga en cuenta que, aunque X_{eval} es una imagen cuadrada, se interpola de nuevo a una forma rectangular para crear esta visualización.

Tabla 8.2: **Rendimiento en segmentación** de los modelos DINO y SimDINO preentrenados en COCO val2017 [LMB+14], un conjunto de datos de segmentación que contiene metadatos sobre la ubicación de los objetos. No entrenamos en COCO, sino que simplemente utilizamos el *embedding* y el *backbone* preentrenados, y las cajas delimitadoras se extraen de las características mediante un método llamado MaskCut [WGY+23]. No obstante, SimDINO supera a DINO en la detección y segmentación de objetos en una comparación imparcial, e incluso supera a DINO con un tamaño de parche más pequeño (longitud lateral de 8 en lugar de 16). Se sabe que los tamaños de parche más pequeños mejoran el rendimiento, especialmente en tareas de detección y segmentación, por lo que este resultado resulta bastante sorprendente y alentador.

		Detección ↑			Segmentación ↑		
Método	Modelo	AP_{50}	AP_{75}	AP	AP_{50}	AP_{75}	AP
SimDINO	ViT-L/16	**5.4**	1.9	2.4	4.5	1.4	1.9
SimDINO	ViT-B/16	5.2	**2.0**	**2.5**	**4.7**	**1.5**	**2.0**
DINO	ViT-B/16	3.9	1.5	1.8	3.1	1.0	1.4
DINO	ViT-B/8	5.1	2.3	2.5	4.1	1.3	1.8

en una comparación equitativa. Además, es mucho más estable: los ajustes prescritos de DINO no logran entrenar un modelo ViT-L(arge). Por otro lado, Figura 8.10 muestra visualizaciones de los mapas de saliencia promedio en DINO y en nuestro DINO simplificado, lo que permite observar que los mapas de saliencia son bastante similares entre los distintos modelos, indicando que los modelos aprenden características que son, como mínimo, igual de eficaces a la hora de captar detalles de gran precisión. Los resultados de segmentación y detección de objetos en Tabla 8.2 confirman esta afirmación cuantitativamente, donde las características de SimDINO muestran una mejora sustancial respecto a las de DINO.

8.3 Representación de imágenes con supervisión débil mediante la vinculación de texto

Otro enfoque influyente del aprendizaje contrastivo se aleja de las comparaciones puramente visuales y, en su lugar, aprovecha la alineación natural entre las imágenes y el lenguaje. En lugar de definir la similitud únicamente a través de múltiples vistas de la misma imagen, esta línea de investigación aprovecha el hecho de que muchas imágenes del mundo real van acompañadas de descripciones textuales que proporcionan una supervisión semántica débil. Un ejemplo destacado de este paradigma es CLIP (Contrastive Language–Image Pretraining) [RKH+21a]. En CLIP, las imágenes y sus correspondientes leyendas se tratan como pares positivos, mientras que los pares de imagen y texto que no coinciden se tratan como pares negativos. El objetivo del aprendizaje consiste en que la representación de una imagen se acerque a la representación

de su leyenda asociada y se aleje de las leyendas de otras imágenes. Al basar las representaciones visuales en el lenguaje natural, CLIP aprende características semánticamente ricas que captan conceptos de alto nivel que van más allá de la mera apariencia.

8.3.1 Datos

Los datos que utilizaremos para explorar la metodología CLIP son *pares de texto e imagen*, en lugar de solo imágenes. El trabajo original de CLIP construyó un gran corpus a escala web de aproximadamente 400 millones de pares de texto e imagen recopilados de fuentes de acceso público en Internet. Cada muestra consta de una imagen y una cadena de lenguaje natural asociada (por ejemplo, una leyenda, un título o una breve descripción) que coexiste con la imagen en la web, lo que proporciona una forma débil y ruidosa de supervisión. Para fomentar una amplia cobertura de conceptos visuales, el trabajo de CLIP describe la creación de este conjunto de datos mediante la búsqueda de pares cuyo texto coincida con una de las aproximadamente 500 000 consultas, y buscando un equilibrio aproximado entre los resultados al incluir hasta 20 000 pares por consulta. Este conjunto de datos no se ha publicado, por lo que la mayoría de las reproducciones y ampliaciones de CLIP realizadas por la comunidad se basan, en su lugar, en conjuntos de datos abiertos a escala web de pares de texto e imagen. La más común de ellas es la familia LAION [SBV+22]: LAION-400M ofrece 400 millones de pares filtrados con CLIP, y LAION-5B escala esta receta a 5.850 millones de pares.

Dado que los pares de datos crudas extraídos de la web suelen presentar niveles de ruido extremadamente altos (por ejemplo, imágenes que no coinciden con el texto asociado, texto alternativo genérico generado automáticamente, spam, duplicados o contenido peligroso), los conjuntos de datos al estilo CLIP suelen crearse mediante una *pipeline* de posprocesamiento que filtra y limpia los datos candidatos antes del entrenamiento. Entre los métodos habituales se incluyen filtros básicos de calidad (por ejemplo, descartar pares con texto extremadamente breve o imágenes muy pequeñas o corruptas), la identificación del idioma para conservar las leyendas en el idioma de destino, la eliminación de duplicados y la eliminación de contenido potencialmente dañino.

8.3.2 La tarea y el objetivo del CLIP

Nuestro objetivo es aprender una buena representación de los datos. Sin embargo, en el marco de CLIP, el concepto de «similitud» no se induce a partir de dos vistas ampliadas de la misma imagen, sino a partir de la supervisión semántica débil que proporcionan las descripciones en lenguaje natural. En concreto, nos gustaría que la representación de una imagen estuviera cerca de la representación de su leyenda asociada y lejos de las leyendas de otras imágenes. Equivalentemente, si dos imágenes tienen descripciones textuales similares, sus características visuales aprendidas deberían ser similares, mientras

que las imágenes descritas por textos no relacionados deberían estar separadas en el espacio latente.

Formalizamos el concepto de «similitud» entre representaciones utilizando la *similitud coseno*. Concretamente, sea f_θ un codificador visual y g_ϕ un codificador de texto (cuya arquitectura se detallará en la siguiente sección), ambos mapean en un espacio latente común $\boldsymbol{Z} \in \mathbb{R}^d$. Dada una imagen $\boldsymbol{X} \in \mathcal{I}$ y una cadena de texto $\boldsymbol{T} \in \mathcal{T}$, formamos *embeddings* normalizadas

$$\boldsymbol{z}_\theta^I(\boldsymbol{X}) \doteq \frac{f_\theta(\boldsymbol{X})}{\|f_\theta(\boldsymbol{X})\|_2}, \qquad \boldsymbol{z}_\phi^T(\boldsymbol{T}) \doteq \frac{g_\phi(\boldsymbol{T})}{\|g_\phi(\boldsymbol{T})\|_2}. \tag{8.3.1}$$

Entonces, la similitud coseno entre las representaciones se define como:

$$s(\boldsymbol{X}, \boldsymbol{T}) = \left\langle \boldsymbol{z}_\theta^I(\boldsymbol{X}), \boldsymbol{z}_\phi^T(\boldsymbol{T}) \right\rangle \in [-1, 1]. \tag{8.3.2}$$

Por lo tanto, nuestro objetivo es maximizar la similitud coseno entre los pares coincidentes y minimizarla entre los pares no coincidentes. Para lograrlo, podemos utilizar una pérdida de entropía cruzada simétrica simple, tal como se explica en Sección 7.4.2. Concretamente, dado un mini-lote de n pares de texto-imagen $(\boldsymbol{X}_i, \boldsymbol{T}_i)_{i=1}^n$, podemos definir la pérdida como:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{n}\left(\sum_{i=1}^n \frac{\exp(s(\boldsymbol{X}_i, \boldsymbol{T}_i)/\tau)}{\sum_{k=1}^n \exp(s(\boldsymbol{X}_i, \boldsymbol{T}_k)/\tau)} + \sum_{i=1}^n \frac{\exp(s(\boldsymbol{T}_i, \boldsymbol{X}_i)/\tau)}{\sum_{k=1}^n \exp(s(\boldsymbol{T}_i, \boldsymbol{X}_k)/\tau)}\right), \tag{8.3.3}$$

donde $\tau > 0$ es un parámetro de temperatura que controla la agudeza de la función softmax.

8.3.3 Arquitectura: *Vision Tower*

El objetivo de CLIP es aprender representaciones tanto de imágenes como de texto en el mismo espacio latente. Por consiguiente, su arquitectura entrena un codificador visual $f_\theta : \mathcal{I} \to \mathbb{R}^d$ y un codificador de texto $g_\phi : \mathcal{T} \to \mathbb{R}^d$, lo que también se conoce como la «arquitectura de doble torre».

En cuanto a la arquitectura, se pueden utilizar la mayoría de las redes troncales de visión: por ejemplo, las redes convolucionales como ResNet [HZR+16a], así como el Vision Transformer [DBK+21] (descrito en detalle en Sección 8.2.3) son opciones válidas para CLIP. En la práctica, el Vision Transformer se utiliza con mayor frecuencia debido a su mejor rendimiento y versatilidad. Al igual que en DINO, tomamos la característica del token de clase $\boldsymbol{z}_{\text{cls}}$ como representación de cada imagen: $f_\theta(\boldsymbol{X}) = \boldsymbol{z}_{\text{cls}}$. Esto se debe a que el token de clase está diseñado para agregar información global sobre toda la imagen y se utiliza habitualmente en tareas de predicción posteriores.

8.3.4 Arquitectura: *Text Tower*

La torre textual CLIP g_ϕ suele adoptar una arquitectura basada en Transformer similar al *Vision Transformer* descrito anteriormente. La principal diferencia radica en la capa de *embedding*, que se utiliza para convertir la secuencia de texto en una secuencia de representaciones latentes en el espacio latente $\boldsymbol{Z} \in \mathbb{R}^d$.

Incrustación. Dada una secuencia de texto $\boldsymbol{T} \in \mathcal{T}$ (por ejemplo, una oración o un documento), la representamos como una secuencia de tokens en $\mathbb{R}^d$ mediante el *embedding* g_ϕ^{emb}, de la siguiente manera.

1. En primer lugar, tokenizamos $\boldsymbol{T}$ en una secuencia de símbolos discretos (subpalabras/caracteres) procedentes de un vocabulario $\mathcal{V}$ de tamaño $|\mathcal{V}|$. Concretamente, un tokenizador $t^{\text{tok}} : \mathcal{T} \to \mathcal{V}^N$ genera una secuencia de tokens de longitud N [5] $(\tau_1, \ldots, \tau_N)$, donde $\tau_i \in \mathcal{V}$.

2. Convertimos los tokens en índices enteros mediante un mapa de vocabulario $\mathcal{V} \to \{1, \ldots, |\mathcal{V}|\}$, lo que da como resultado una secuencia de índices $(s_1, \ldots, s_N)$. Colocamos estos índices en una matriz $\boldsymbol{S} \in \{0,1\}^{|\mathcal{V}| \times N}$, donde la i-ésima columna es el vector «one-hot» $\boldsymbol{e}_{s_i}$.

3. A continuación, realizamos la siguiente operación sobre $\boldsymbol{S}$ para proyectarla en $\mathbb{R}^{d \times (N+1)}$:
$$\boldsymbol{S} \mapsto [\boldsymbol{z}_{\text{cls}}^1, \boldsymbol{W}^{\text{tok}} \boldsymbol{S}] + \boldsymbol{E}^{\text{pos}}. \tag{8.3.4}$$
Este es el resultado final de g_ϕ^{emb}. Aquí tenemos tres parámetros entrenables: $\boldsymbol{W}^{\text{tok}}$, $\boldsymbol{z}_{\text{cls}}^1$ y $\boldsymbol{E}^{\text{pos}}$, cuyas funciones son las siguientes:
   - $\boldsymbol{W}^{\text{tok}} \in \mathbb{R}^{d \times |\mathcal{V}|}$ es una matriz cuya j-ésima columna es el vector de *embedding* aprendido para el j-ésimo elemento del vocabulario. Por lo tanto, $\boldsymbol{W}^{\text{tok}} \boldsymbol{S} \in \mathbb{R}^{d \times N}$ asigna cada token discreto a una característica continua en $\mathbb{R}^d$.
   - $\boldsymbol{z}_{\text{cls}}^1 \in \mathbb{R}^d$ es un token especial de *clasificación* (o *resumen*). Desde un punto de vista heurístico, su función es agregar información global sobre toda la secuencia y se utiliza habitualmente para tareas aguas abajo de predicción.
   - $\boldsymbol{E}^{\text{pos}} \in \mathbb{R}^{d \times n}$ es una *codificación posicional* que distingue los tokens en diferentes posiciones. Dado que la autoatención es equivariante ante permutaciones en la dimensión de los tokens, $\boldsymbol{E}^{\text{pos}}$ inyecta información de orden de modo que la función global no sea invariante ante permutaciones de los tokens.

Todos los parámetros $\boldsymbol{z}_{\text{cls}}^1, \boldsymbol{W}^{\text{tok}}, \boldsymbol{E}^{\text{pos}}$ están incluidos en el conjunto de parámetros θ. El vocabulario $\mathcal{V}$ y el tokenizador t^{tok} son fijos.

Otros componentes de la torre de texto, como el *backbone* y el cabezal específico para cada tarea, siguen opciones de diseño similares a las de la torre visual y se omiten por motivos de brevedad. También cabe señalar que trabajos de seguimiento han demostrado una flexibilidad considerable en la elección del codificador de texto de CLIP: las representaciones de texto fijas contenidas en grandes modelos de lenguaje, e incluso las características simples de «bolsa de palabras», también pueden resultar eficaces. Analizamos estas variantes en la Sección 8.3.8.

[5] En la práctica, a menudo se amplía esta secuencia con tokens especiales (por ejemplo, ⟨bos⟩ y ⟨eos⟩, que se utilizan para indicar el inicio y el final de la secuencia) y se pueden rellenar secuencias más cortas, pero mantenemos la notación genérica y las tratamos como elementos de $\mathcal{V}$.

8.3.5 Estrategia de optimización

Entrenamos CLIP utilizando un procedimiento estándar de optimización estocástica de extremo a extremo (por ejemplo, SGD con momentum, AdamW). A continuación, presentamos un proceso genérico.

En cada paso temporal k, se realizan las siguientes operaciones:

- **Datos de submuestra en pares.** Se toma un *minibatch* de n ejemplos en pares
$$\{(\boldsymbol{X}_i^{(k)}, \boldsymbol{T}_i^{(k)})\}_{i=1}^n \subseteq \mathcal{X} \times \mathcal{T},$$
donde $\boldsymbol{X}_i^{(k)}$ es una imagen y $\boldsymbol{T}_i^{(k)}$ es su texto asociado.

- **Calcular representaciones latentes.** Para cada par $(\boldsymbol{X}_i^{(k)}, \boldsymbol{T}_i^{(k)})$, calcular *embeddings* normalizados utilizando los codificadores de visión y texto f_θ y g_φ:
$$z_\theta^I(\boldsymbol{X}_i^{(k)}) \doteq \frac{f_\theta(\boldsymbol{X}_i^{(k)})}{\|f_\theta(\boldsymbol{X}_i^{(k)})\|_2}, \qquad z_\phi^T(\boldsymbol{T}_i^{(k)}) \doteq \frac{g_\phi(\boldsymbol{T}_i^{(k)})}{\|g_\phi(\boldsymbol{T}_i^{(k)})\|_2}. \tag{8.3.5}$$

- **Calcular la pérdida CLIP.** Calcular la pérdida CLIP tal y como se define en (8.3.3):
$$\hat{\mathcal{L}}_{\text{CLIP}}^{(k)} \doteq -\frac{1}{n}\left(\sum_{i=1}^n \frac{\exp(s(\boldsymbol{X}_i^{(k)}, \boldsymbol{T}_i^{(k)})/\tau)}{\sum_{k=1}^n \exp(s(\boldsymbol{X}_i^{(k)}, \boldsymbol{T}_k^{(k)})/\tau)} + \sum_{i=1}^n \frac{\exp(s(\boldsymbol{T}_i^{(k)}, \boldsymbol{X}_i^{(k)})/\tau)}{\sum_{k=1}^n \exp(s(\boldsymbol{T}_i^{(k)}, \boldsymbol{X}_k^{(k)})/\tau)}\right). \tag{8.3.6}$$

- **Calcular los gradientes.** Calcular los gradientes de la pérdida estocástica con respecto a ambos conjuntos de parámetros:
$$\nabla_\theta \hat{\mathcal{L}}_{\text{CLIP}}^{(k)}(\theta, \phi), \qquad \nabla_\phi \hat{\mathcal{L}}_{\text{CLIP}}^{(k)}(\theta, \phi). \tag{8.3.7}$$

- **Actualizar parámetros.** Aplicar un paso del algoritmo de optimización a (θ, ϕ), lo que da como resultado la iteración
$$(\theta^{(k+1)}, \phi^{(k+1)}) \doteq \textsc{OptUpdate}^{(k)}\left((\theta^{(k)}, \phi^{(k)}); \nabla_\theta \hat{\mathcal{L}}_{\text{CLIP}}^{(k)}, \nabla_\phi \hat{\mathcal{L}}_{\text{CLIP}}^{(k)}\right). \tag{8.3.8}$$

8.3.6 Metodología de evaluación

Dado que CLIP es independiente de la arquitectura, se pueden aplicar la mayoría de los protocolos de evaluación estándar para codificadores de visión y lenguaje. Por ejemplo, se puede realizar un sondeo lineal (linear probing) entrenando un cabezal de clasificación ligero sobre el codificador visual para evaluar la calidad de las representaciones aprendidas en tareas de clasificación de imágenes. Además, se puede llevar a cabo una clasificación sin entrenamiento

previo («zero-shot») y sin necesidad de entrenamiento adicional gracias a la arquitectura de doble torre de CLIP.

Recordemos que, en una configuración estándar de clasificación de imágenes en n vías, se nos proporciona un conjunto de datos de imágenes $\{\boldsymbol{X}_i\}_{i=1}^N \subseteq \mathcal{I}$, cada una etiquetada con una de las n etiquetas de clase del conjunto $\mathcal{L} \doteq \{\ell_1, \ldots, \ell_n\}$ (por ejemplo, $\ell_j \in \{\text{gato}, \text{perro}, \ldots\}$). Nuestro objetivo es construir una *pipeline* de predicción P, aprovechando un modelo preentrenado, de tal manera que $P : \mathcal{I} \to \{1, \ldots, n\}$ asigne a cada imagen $\boldsymbol{X}_i$ un índice de clase predicho $\hat{y}_i \doteq P(\boldsymbol{X}_i)$. Escribiendo la etiqueta de clase de referencia como su índice $y_i \in \{1, \ldots, n\}$, la exactitud de clasificación (top-1) de P en este conjunto de datos se calcula entonces como

$$\frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{\hat{y}_i = y_i\}.$$

En CLIP, podemos utilizar el codificador de texto g_ϕ para construir un diccionario de texto $\boldsymbol{D} \in \mathbb{R}^{n \times d}$, cuya j-ésima fila es la representación latente normalizada de la j-ésima etiqueta de clase. Concretamente, para cada clase $\ell_j \in \mathcal{L}$, se construye una descripción textual (prompt) $\boldsymbol{T}_j$ (p. ej., «una foto de un ℓ_j») y se calcula su *embedding* normalizado $\boldsymbol{z}_j^T \doteq \frac{g_\phi(\boldsymbol{T}_j)}{\|g_\phi(\boldsymbol{T}_j)\|_2} \in \mathbb{R}^d$. Al apilarlas por filas se obtiene

$$\boldsymbol{D} \doteq \begin{bmatrix} (\boldsymbol{z}_1^T)^\top \\ \vdots \\ (\boldsymbol{z}_n^T)^\top \end{bmatrix} \in \mathbb{R}^{n \times d},$$

Dada una imagen $\boldsymbol{X}_i \in \mathcal{I}$, se calcula su representación de imagen mediante el codificador visual y se la normaliza como

$$\boldsymbol{z}_i^I \doteq \frac{f_\theta(\boldsymbol{X}_i)}{\|f_\theta(\boldsymbol{X}_i)\|_2} \in \mathbb{R}^d.$$

A continuación, podemos calcular un vector de similitud

$$\boldsymbol{d}_i \doteq \boldsymbol{D}\, \boldsymbol{z}_i^I \in \mathbb{R}^n,$$

cuya j-ésima entrada $\boldsymbol{d}_i[j]$ es la similitud coseno entre la i-ésima imagen y la j-ésima etiqueta de clase en el espacio latente compartido. Según el objetivo de entrenamiento de CLIP, se fomenta que las imágenes sean más similares a sus textos correspondientes que a los que no coinciden; por lo tanto, una mayor similitud coseno indica una alineación más estrecha entre $\boldsymbol{X}_i$ y la clase ℓ_j. De ahí que predecimos el índice de clase mediante

$$\hat{y}_i \doteq \arg \max_{j \in \{1, \ldots, n\}} \boldsymbol{d}_i[j].$$

Por último, podemos calcular la precisión de la clasificación como se ha mencionado anteriormente.

Este protocolo de decisión «zero-shot» no se limita a la clasificación, sino que también se aplica en forma natural a las tareas de *recuperación*, en las que el objetivo es devolver los elementos más relevantes de una gran base de datos a partir de una consulta (por ejemplo, recuperar las imágenes más relevantes para una consulta de texto, o los textos más relevantes para una consulta de imagen). Por ejemplo, en la recuperación de texto a imagen, en lugar de crear un diccionario a partir de indicaciones de nombres de clases, creamos un *diccionario de imágenes* codificando y normalizando todas las imágenes de la base de datos $\{\boldsymbol{X}_m\}_{m=1}^M$, y apilando sus *embeddings* en $\boldsymbol{D} \in \mathbb{R}^{M \times d}$. Dado un texto de consulta $\boldsymbol{T}$, calculamos su *embedding* normalizado $\boldsymbol{z}^T \doteq g_\phi(\boldsymbol{T})/\|g_\phi(\boldsymbol{T})\|_2$, formamos puntuaciones de similitud $\boldsymbol{d} \doteq \boldsymbol{D}\boldsymbol{z}^T \in \mathbb{R}^M$, y luego ordenamos las imágenes según estas puntuaciones (de manera equivalente, devolvemos los K índices principales).

8.3.7 Configuración experimental y resultados

A continuación, presentamos los resultados del artículo original sobre CLIP, en el que se compara CLIP con una red ResNet supervisada de referencia en varios bancos de pruebas clásicos de clasificación. En todos los benchmarks, CLIP se evalúa utilizando el protocolo de clasificación zero-shot descrito en Sección 8.3.6. Los detalles completos de la configuración experimental de ResNet se detallan en [HZR+16a]; aquí, presentamos brevemente los conjuntos de datos de evaluación y resumimos la configuración de entrenamiento y evaluación de CLIP relevantes para estas comparaciones.

Datos de evaluación.

- **CIFAR-10 [KH+09].** CIFAR-10 es un conjunto de datos de referencia muy utilizado para la clasificación de imágenes a pequeña escala, que consta de 60 000 imágenes RGB de tamaño 32×32 distribuidas en 10 clases (6 000 imágenes por clase). Normalmente se divide en 50 000 imágenes de entrenamiento y 10 000 imágenes de prueba.

- **CIFAR-100 [KH+09].** CIFAR-100 tiene el mismo tamaño total y la misma resolución de imagen que CIFAR-10 (60 000 imágenes RGB de 32×32 con una división de 50 000/10 000 para entrenamiento/prueba), pero es más desafiante porque contiene 100 clases muy específicas (600 imágenes por clase). Las 100 clases se agrupan además en 20 superclases.

- **PASCAL VOC 2007 (VOC2007) [EGW+10].** PASCAL VOC 2007 es un *benchmark* clásico en visión por computadora para el reconocimiento de objetos, que cubre tareas como la clasificación de imágenes, la detección de objetos y la segmentación. Se centra en 20 categorías de objetos (por ejemplo, persona, automóvil, perro).

- **ImageNet-1K [DDS+09a].** ImageNet-1K es un subconjunto de ILSVRC 2012 que contiene 1 000 clases de objetos. Este subconjunto

incluye 1 281 167 imágenes de entrenamiento, 50 000 imágenes de validación y 100 000 imágenes de prueba, y ha desempeñado un papel central en el reconocimiento visual a gran escala y el preentrenamiento.

Configuración del modelo. En los modelos CLIP que aquí se presentan, se adopta el *Vision Transformer* como codificador visual f_θ y se implementa como una de las tres variantes de ViT: ViT-B/32, ViT-B/16 o ViT-L/14. Aquí, B («Base») y L («Large») especifican el ancho y la profundidad del Transformer (por ejemplo, ViT-B suele utilizar 12 bloques de transformer con una dimensión de *embedding* $d = 768$ y 12 cabezales de atención, mientras que ViT-L utiliza 24 bloques con $d = 1024$ y 16 cabezales), mientras que los sufijos /32, /16 y /14 denotan el *tamaño del parche* en píxeles utilizado para particionar la imagen de entrada en parches. Los tamaños de parche más pequeños (por ejemplo, 16 o 14) generan más tokens por imagen a una resolución fija, lo que aumenta la carga computacional, pero a menudo mejora la precisión gracias a una granularidad espacial más fina.

Configuración de la optimización. Todos los modelos CLIP basados en ViT se entrenan durante 32 épocas utilizando el optimizador Adam con decaimiento de pesos desacoplado (es decir, regularización al estilo AdamW), en el que el decaimiento de pesos se aplica a todos los parámetros excepto a las ganancias y los sesgos. La tasa de aprendizaje se reduce siguiendo un calendario de coseno. El parámetro de temperatura aprendible τ (utilizado para escalar los logits contrastivos) se inicializa en el valor equivalente a $0,07$ y se limita para evitar que los logits se escalen por más de 100, lo cual se considera necesario para la estabilidad del entrenamiento. Por último, el entrenamiento utiliza un tamaño de mini-lote muy grande, de 32768.

Resultados. La Tabla 8.3 presenta la precisión top-1 zero-shot de tres variantes de CLIP ViT (B/32, B/16, L/14) y las compara con las redes de referencia supervisadas ResNet-50 y ResNet-101. En los cuatro conjuntos de datos, CLIP supera sistemáticamente a las redes ResNet supervisadas. Ampliar aún más la estructura central de CLIP genera mejoras adicionales, con ViT-L/14 alcanzando 98.0 en CIFAR-10, 87.5 en CIFAR-100, 89.6 en VOC2007 y 83.9 en ImageNet. No obstante, esta tabla debe interpretarse como una comparación entre dos *sistemas* y no como una prueba controlada de ablación del *backbone*: CLIP se beneficia del preentrenamiento a gran escala de pares de imagen-texto y se evalúa mediante prompts de tipo «zero-shot», mientras que los modelos de referencia de ResNet se basan en un preentrenamiento supervisado y requieren datos etiquetados para ajustar una sonda en cada *benchmark*. Por lo tanto, los márgenes consistentes demuestran principalmente la solidez de las representaciones transferibles de CLIP en el marco del protocolo «zero-shot», en lugar de establecer una superioridad de ViT sobre ResNet basada exclusivamente en la arquitectura o en la correspondencia de datos.

Red troncal	Variante	CIFAR-10	CIFAR-100	VOC2007	ImageNet
CLIP-ViT	B/32	95.1	80.5	87.7	76.1
CLIP-ViT	B/16	96.2	83.1	89.2	80.2
CLIP-ViT	L/14	98.0	87.5	89.6	83.9
ResNet	50	91.8	74.5	83.8	74.3
ResNet	101	93.0	77.2	84.4	75.8

Tabla 8.3: Rendimiento en conjuntos de datos de tareas aguas abajo para diversos *bachbone* CLIP y modelos de referencia ResNet.

8.3.8 Extensión simplificada: LIFT

Aunque es eficaz, CLIP presenta varias limitaciones. En primer lugar, entrenar conjuntamente desde cero tanto un codificador de texto como un codificador visual supone un gran esfuerzo computacional, ya que a menudo requiere lotes extremadamente grandes y conjuntos de datos masivos para lograr una alineación sólida y una transferencia eficaz en las etapas posteriores. En segundo lugar, los modelos entrenados de esta manera pueden tener dificultades en la comprensión composicional — es decir, captar el orden de las palabras en el texto, la disposición espacial en las imágenes y las relaciones entre objetos y atributos o entre objetos. Esto se debe, en parte, a que la supervisión contrastiva de tipo «recuperación» puede favorecer atajos que restan importancia a las características composicionales más detalladas. LIFT (Language–Image alignment with a Fixed Text encoder) [YWZ+25] revisa una suposición fundamental en la que se basan estos flujos de trabajo: que una alineación óptima requiere el entrenamiento conjunto de extremo a extremo de ambos codificadores:

https://jingfeng0705.github.io/LIFT/.

En cambio, LIFT aprovecha el hecho de que los modelos modernos de lenguaje a gran escala (LLM) ya generan *embeddings* de texto muy informativas. Concretamente, LIFT fija un codificador de texto preentrenado de gran calidad (por ejemplo, uno derivado de un LLM o ajustado sobre él), calcula los *embeddings* de texto offline y entrena únicamente el codificador visual para que se alinee con estos objetivos fijos.

Arquitectura. LIFT conserva la misma formulación de doble torre de CLIP en cuanto a un codificador visual y un codificador de texto que se proyectan en un espacio latente compartido, pero difiere fundamentalmente en que la torre textual es *fija* y puede ser instanciada por cualquier codificador de texto potente basado en un LLM $g_\phi : cT \to \mathbb{R}^d$ para producir *embeddings* de texto, donde ϕ no se actualiza durante el entrenamiento. En nuestra implementación, adoptamos el codificador de texto NV-Embed-V2, para el cual la dimensión de *embedding* es $d = 4096$. Dado que la torre textual g_ϕ es fija, podemos precalcular todas los *embeddings* de texto $\{g_\phi(\boldsymbol{T})\}$ en forma offline y, durante el entrenamiento,

solo calcular los gradientes y actualizar el codificador visual f_θ en forma online. La torre visual $f_\theta : \mathcal{I} \to \mathbb{R}^d$ sigue la misma estructura que en Sección 8.3.3, con la dimensión de salida del cabezal de proyección establecida en d para que coincida con la dimensión del espacio de *embeddings* de texto fijo. Aparte de fijar g_ϕ e igualar la dimensión de proyección, las componentes restantes—incluida la pérdida contrastiva y el procedimiento de optimización—siguen los mismos principios de diseño que CLIP.

Metodología de evaluación. La comprensión composicional es una limitación conocida de CLIP. Evaluamos LIFT y CLIP en siete tareas de SugarCrepe [HZM+23], en las que cada imagen $\boldsymbol{X}$ se empareja con una descripción *positiva* (correcta) $\boldsymbol{T}_{\text{pos}}$ y una descripción *negativa* $\boldsymbol{T}_{\text{neg}}$ construida mediante la adición, la sustitución o el intercambio de un objeto, atributo o relación en $\boldsymbol{T}_{\text{pos}}$. Véase la Figura 8.11 para algunos ejemplos. Se pide a los modelos que identifiquen la descripción correcta comparando las similitudes coseno entre la descripción y la imagen. Formalmente, para cada muestra $i \in \{1, \dots, N\}$, se calculan las representaciones normalizadas

$$\boldsymbol{z}_i^I \doteq \frac{f_\theta(\boldsymbol{X}_i)}{\|f_\theta(\boldsymbol{X}_i)\|_2}, \qquad \boldsymbol{z}_{i,\text{pos}}^T \doteq \frac{g_\phi(\boldsymbol{T}_{i,\text{pos}})}{\|g_\phi(\boldsymbol{T}_{i,\text{pos}})\|_2}, \qquad \boldsymbol{z}_{i,\text{neg}}^T \doteq \frac{g_\phi(\boldsymbol{T}_{i,\text{neg}})}{\|g_\phi(\boldsymbol{T}_{i,\text{neg}})\|_2}.$$

A continuación, se calculan las puntuaciones de similitud mediante similitudes coseno:

$$s_{i,\text{pos}} \doteq \langle \boldsymbol{z}_i^I, \boldsymbol{z}_{i,\text{pos}}^T \rangle, \qquad s_{i,\text{neg}} \doteq \langle \boldsymbol{z}_i^I, \boldsymbol{z}_{i,\text{neg}}^T \rangle.$$

Se considera que el modelo es correcto en la muestra i si $s_{i,\text{pos}} > s_{i,\text{neg}}$ y la precisión compositiva general es

$$\frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\{s_{i,\text{pos}} > s_{i,\text{neg}}\}.$$

Resultados experimentales. Como se muestra en la Tabla 8.4, cuando se entrena con DataComp-1B, LIFT supera a CLIP en las siete tareas con una mejora media en precisión del **6,8%**. LIFT logra mejoras significativas en las tareas de `añadir atributo`, `reemplazar atributo` y `reemplazar relación`. Estas mejoras constituyen una evidencia sólida de que los codificadores de texto basados en LLM g_ϕ capturan representaciones de texto más informativas que permiten modelar con mayor precisión las asociaciones entre objetos y atributos, así como las relaciones entre objetos. Por otro lado, podemos observar que LIFT muestra una precisión relativamente baja en las tareas `intercambiar objeto` y `intercambiar atributo` en comparación con otras tareas de SugarCrepe. Atribuimos esta limitación al objetivo del aprendizaje contrastivo, que se centra principalmente en alinear estadísticas de orden inferior. Para hacer frente a este reto, es necesario explorar medidas más refinadas de teoría de la información para la alineación entre lenguaje e imagen, lo que constituye una línea de trabajo clave para el futuro.

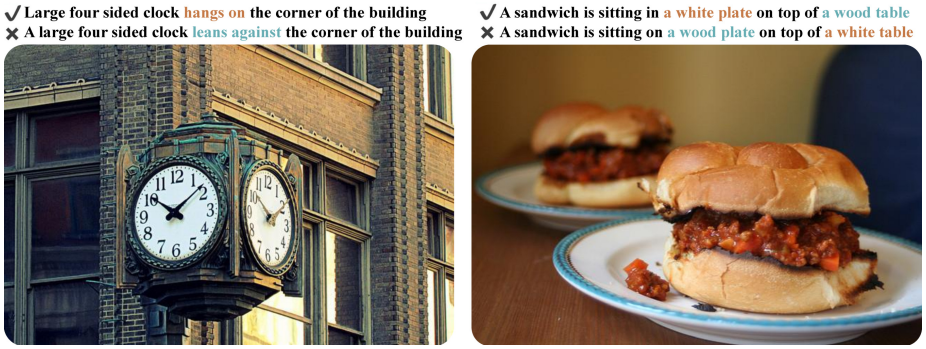


Figura 8.11: Las leyendas originales (arriba) y sus correspondientes negativos (abajo) de dos tareas de SugarCrepe: `reemplazar relación` (izquierda) y `intercambiar atributo` (derecha).

			Añadir		Reemplazar			Swap	
Método	Conj. Datos	Muestras vistas	Obj	Atr	Obj	Atr	Rel	Obj	Atr
OpenCLIP	DataComp	1.28B	82.3	73.7	91.7	79.4	61.2	59.6	56.9
LIFT	DataComp	1.28B	**89.0**	**86.1**	**93.2**	**86.0**	**70.6**	**64.1**	**63.4**

Tabla 8.4: El rendimiento de LIFT y CLIP en siete tareas de SugarCrepe. Utilizamos una implementación de código abierto de CLIP, OpenCLIP [IWW+21], y entrenamos tanto OpenCLIP como LIFT en el conjunto de datos de código abierto de pares de texto-imagen DataComp [GIF+23]. Presentamos la precisión para cada tarea, con los mejores resultados en negrita.

8.4 Representación supervisada de imágenes mediante clasificación

En las secciones anteriores, hemos mostrado cómo simplificar algunos objetivos de aprendizaje excesivamente complejos —y a menudo diseñados de manera heurística— sobre la base de los principios de compresión presentados en este libro. Sin embargo, los objetivos asociados a algunas de las tareas de aprendizaje más populares ya son bastante sencillos. En estos casos, es difícil simplificar aún más el objetivo. Por lo tanto, en esta y en futuras secciones, nos centraremos en formas fundamentadas de modificar las *arquitecturas de redes profundas* para una variedad de tareas.

Empecemos con la que podría considerarse la tarea más clásica del aprendizaje automático: *la clasificación de imágenes*, que a menudo se utiliza como tarea estándar para evaluar algoritmos de reconocimiento de patrones o arquitecturas de redes profundas. A partir de nuestro análisis de las arquitecturas *white-box* (transparentes) en el Capítulo 5, solo necesitamos una tarea semánticamente significativa para aprender buenas representaciones con arquitecturas white-box. Validaremos esta idea en esta sección.

En primer lugar, el conjunto de datos se mantiene prácticamente igual que en la Sección 8.2.1. Tanto los datos de entrenamiento como los de prueba consisten en imágenes etiquetadas, es decir, pares de imagen-etiqueta $(\boldsymbol{X}, \boldsymbol{y}) \in \mathbb{R}^{C\times H\times W} \times \{0,1\}^{N_{\text{cls}}}$. Seguimos aplicando varias técnicas de augmentación de datos (por ejemplo, volteos, desenfoque gaussiano, solarización, etc.) a cada muestra de cada nuevo lote.

8.4.1 Tarea y objetivo

A diferencia de antes, nuestra tarea no consiste solo en aprender una buena representación de los datos, sino también en construir simultáneamente un clasificador. Formalmente, disponemos de pares de datos etiquetados $(\boldsymbol{X}, \boldsymbol{y})$, donde $\boldsymbol{y} \in \{0,1\}^{N_{\text{cls}}}$ es un vector «one-hot» que indica la clase a la que pertenece $\boldsymbol{X}$. Consideramos una vista determinista de recorte central v_{cc} de los datos de entrada $\boldsymbol{X}$ (véase Sección 8.2.2). Queremos entrenar conjuntamente una función de mapeo $(f_\theta, f_\theta^{\text{ext}})$ y un *cabezal de clasificación* h_θ, definidos de la siguiente manera:

$$h_\theta(\boldsymbol{z}) \doteq \text{softmax}(\boldsymbol{W}^{\text{head}}\boldsymbol{z} + \boldsymbol{b}^{\text{head}}), \qquad \forall \boldsymbol{z} \in \mathbb{R}^d \tag{8.4.1}$$

donde $(\boldsymbol{W}^{\text{head}}, \boldsymbol{b}^{\text{head}}) \in \mathbb{R}^{N_{\text{cls}}\times d} \times \mathbb{R}^{N_{\text{cls}}}$ son parámetros entrenables en el conjunto de parámetros θ, de tal manera que la aplicación $\boldsymbol{X}_{\text{cc}} \mapsto \boldsymbol{p}_\theta(\boldsymbol{X}_{\text{cc}}) \doteq h_\theta(\boldsymbol{z}_\theta(\boldsymbol{X}_{\text{cc}}))$ predice una etiqueta suavizada para la vista $\boldsymbol{X}_{\text{cc}} = v_{\text{cc}}(\boldsymbol{X})$ de la entrada $\boldsymbol{X}$. El problema de aprendizaje busca minimizar la distancia entre $\boldsymbol{p}_\theta$ y $\boldsymbol{y}$ medida mediante entropía cruzada:

$$\min_\theta \{\mathcal{L}_{\text{CE}}(\theta) \doteq \mathbb{E}[\text{CE}(\boldsymbol{y}, \boldsymbol{p}_\theta(\boldsymbol{X}_{\text{cc}}))]\}\,. \tag{8.4.2}$$

8.4.2 La arquitectura de CRATE

La arquitectura que utilizamos es la arquitectura de CRATE, descrita con cierto detalle en el Capítulo 5. La configuración general es similar a la del transformer estándar de la Sección 8.2.3, con algunos cambios. Si bien el paso de *embedding* es el mismo que en DINO y SimDINO en la Sección 8.2.3, el paso de extracción de características es el mismo que en SimDINO en la Sección 8.2.3, ya que solo extrae la característica asociada al token de clase, y el cabezal de clasificación se describe en la Sección 8.4.1, la arquitectura del *backbone* es diferente. Cada capa tiene la forma

$$\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}) = \boldsymbol{Z}_\theta^\ell(\boldsymbol{X}) + \mathrm{MSSA}_\theta^\ell(\mathrm{LN}_\theta^{1,\ell}(\boldsymbol{Z}_\theta^\ell(\boldsymbol{X}))), \tag{8.4.3}$$

$$\boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X}) = \mathrm{ISTA}_\theta^\ell(\mathrm{LN}_\theta^{2,\ell}(\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}))), \tag{8.4.4}$$

donde los bloques $\mathrm{MSSA}_\theta^\ell$ e $\mathrm{ISTA}_\theta^\ell$ son los descritos en el Capítulo 5, a saber:

- El operador MSSA es una autoatención de múltiples cabezas sobre subespacios, definida de la siguiente manera:

$$\mathrm{MSSA}_\theta^\ell(\boldsymbol{Z}) \doteq \boldsymbol{U}_{\mathrm{out}}^\ell \begin{bmatrix} \mathrm{SA}([\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}) \\ \vdots \\ \mathrm{SA}([\boldsymbol{U}^{K,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{K,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}) \end{bmatrix} + \boldsymbol{b}_{\mathrm{out}}^\ell \mathbf{1}_n^\top \tag{8.4.5}$$

  donde $\boldsymbol{U}^{k,\ell} \in \mathbb{R}^{d\times p}$, $\boldsymbol{U}_{\mathrm{out}}^\ell \in \mathbb{R}^{d\times Kp}$, y $\boldsymbol{b}_{\mathrm{out}}^\ell \in \mathbb{R}^d$ son parámetros entrenables que pertenecen al conjunto de parámetros θ, y (recordemos que) el operador de autoatención SA se define en (8.2.18).

- El operador ISTA es el operador del algoritmo de umbralización por contracción iterativa, definido de la siguiente manera:

$$\mathrm{ISTA}_\theta^\ell(\boldsymbol{Z}) \doteq \mathrm{ReLU}(\boldsymbol{Z} - \beta(\boldsymbol{D}^\ell)^\top(\boldsymbol{D}^\ell \boldsymbol{Z} - \boldsymbol{Z}) + \beta\lambda \mathbf{1}_d \mathbf{1}_n^\top), \tag{8.4.6}$$

  denominado así porque la función $\boldsymbol{X} \mapsto \mathrm{ReLU}(\boldsymbol{X} - \beta\boldsymbol{D}^\top(\boldsymbol{D}\boldsymbol{X} - \boldsymbol{Z}) + \beta\lambda\mathbf{1}_d\mathbf{1}_n^\top)$ constituye un paso del consolidado algoritmo ISTA para encontrar una representación esparsa y no negativa elemento a elemento de $\boldsymbol{Z}$ con respecto al diccionario completo $\boldsymbol{D}$ (véase Sección 2.3).

A esta arquitectura la llamamos CRATE y en la Figura 5.13 se muestra una capa de su *backbone*. Los modelos CRATE, más allá de su interpretabilidad, se caracterizan generalmente por su alto rendimiento y su eficiencia paramétrica.

8.4.3 Optimización

Nuestro clasificador se entrena mediante un sencillo procedimiento de optimización estocástica de extremo a extremo, en el que se toma una submuestra de los datos y las vistas, se calcula la pérdida omedio y su gradiente sobre estas muestras y se utiliza un algoritmo de optimización para actualizar los parámetros. En cada paso temporal k, se llevan a cabo los siguientes pasos:

- Se extrae una submuestra de B muestras etiquetadas diferentes: $\{(\boldsymbol{X}_b^{(k)}, \boldsymbol{y}_b^{(k)})\}_{b=1}^{B} \subseteq \mathcal{I} \times \{0,1\}^{N_{\text{cls}}}$.
- Para cada muestra etiquetada $(\boldsymbol{X}_b^{(k)}, \boldsymbol{y}_b^{(k)})$, se calcula la vista de recorte central $v_{b,\text{cc}}^{(k)}$ y se la aplica a $\boldsymbol{X}_b^{(k)}$ para obtener $\boldsymbol{X}_{b,\text{cc}}^{(k)} \doteq v_{b,\text{cc}}^{(k)}(\boldsymbol{X}_b^{(k)})$.
- Se calculan las predicciones $\boldsymbol{p}_\theta(\boldsymbol{X}_{b,\text{cc}}^{(k)}) \doteq (h_\theta \circ f_\theta^{\text{ext}} \circ f_\theta)(\boldsymbol{X}_{b,\text{cc}}^{(k)})$.
- Se forma la pérdida estocástica sustitutiva

$$\hat{\mathcal{L}}_{\text{CE}}^{(k)}(\theta) \doteq \frac{1}{B} \sum_{b=1}^{B} \text{CE}(\boldsymbol{y}_b^{(k)}, \boldsymbol{p}_\theta(\boldsymbol{X}_{b,\text{cc}}^{(k)})). \tag{8.4.7}$$

- Se realiza un paso de un algoritmo de optimización sobre θ, obteniendo la siguiente iteración:

$$\theta^{(k+1)} \doteq \textsc{OptUpdate}^{(k)}(\theta^{(k)}; \nabla_\theta \hat{\mathcal{L}}_{\text{CE}}^{(k)}). \tag{8.4.8}$$

8.4.4 Metodología de evaluación

Se utiliza el mismo procedimiento de evaluación que en la Sección 8.2.5. En resumen, para todas las evaluaciones (así como para el entrenamiento) se utiliza una vista de recorte central grande v_{cc} que redimensiona la imagen de entrada y extrae un recorte central de tamaño $(C, S_{\text{cc}}, S_{\text{cc}})$, donde C es el número de canales en la imagen de entrada. A partir de la salida de esta vista, se pueden realizar sondeos lineales, visualización de mapas de atención y benchmarks de detección y segmentación.

8.4.5 Diseño experimental y resultados

Dado que CRATE se basa directamente en el modelo transformer, se comparan los parámetros óptimos para ViT, tal y como se indican en [DBK+21; TCD+20], con las mismos configuraciones aplicadas a CRATE, a fin de realizar una comparación imparcial.

Arquitectura del modelo. El recorte central redimensiona toda la imagen de modo que el lado más corto tenga un tamaño de 256 (es decir, $S_{\text{rsz}} = 256$) antes de realizar un recorte central de tamaño 224×224 (es decir, $S_{\text{cc}} = 224$), tanto en la evaluación como en el entrenamiento. Se toma un tamaño de parche de 16 (es decir, $P_H = P_W = 16$). Se utilizan los modelos tiny, small, base y large de la arquitectura ViT [DBK+21] como capa de *embedding* y *backbone*, sustituyendo las componentes MHSA y MLP por MSSA e ISTA, respectivamente, utilizando el mismo número de cabezas y la misma dimensión de cabeza en el caso de MSSA, lo que reduce drásticamente el número de parámetros de entrenamiento. Para CRATE, se establece $(\beta, \lambda) = (1, 0.1)$.

Tabla 8.5: **Precisión de clasificación mediante sondaje lineal de CRATE y ViT** en diversos conjuntos de datos con diferentes tamaños de modelo cuando la red base se ha entrenado previamente para la clasificación en ImageNet-1K. Se observa que, con la misma configuración de modelo, CRATE ofrece un rendimiento de clasificación comparable con un diseño más sencillo, más fundamentado y más eficiente en cuanto a parámetros.

Modelo	CRATE-T	CRATE-S	CRATE-B	CRATE-L	ViT-T	ViT-S
# parámetros	6.09M	13.12M	22.80M	77.64M	5.72M	22.05M
ImageNet-1K	66.7	69.2	70.8	71.3	71.5	72.4
ImageNet-1K ReaL	74.0	76.0	76.5	77.4	78.3	78.4
CIFAR10	95.5	96.0	96.8	97.2	96.6	97.2
CIFAR100	78.9	81.0	82.7	83.6	81.8	83.2
Oxford Flowers-102	84.6	87.1	88.7	88.3	85.1	88.5
Oxford-IIIT-Pets	81.4	84.9	85.3	87.4	88.5	88.6

Conjuntos de datos y optimización. Para el preentrenamiento, se utiliza el conjunto de datos ImageNet-1K. Se emplea el optimizador LION [CLH+24] para preentrenar tanto nuestra reimplementación de ViT como CRATE. Se establece la tasa de aprendizaje base en 2.4×10^{-4}, el decaimiento de pesos en 0.5 y el tamaño del lote en $B = 2048$. Nuestro programa de tasas de aprendizaje aumenta la tasa de aprendizaje en forma lineal hasta la tasa de aprendizaje base durante las primeras 5 épocas, y disminuye hasta 0 siguiendo un programa coseno durante las siguientes 145 épocas (entrenando todos los modelos durante 150 épocas cada uno). Para el preentrenamiento, se aplica un régimen habitual de augmentación de datos (volteos aleatorios, desenfoques gaussianos, solarización, etc.) a los datos de imagen y también se añade un pequeño ruido a las etiquetas (esto se denomina *suavizado de etiquetas* [MKH19]).

Para el aprendizaje por sonda lineal, se utilizan varios conjuntos de datos de evaluación, tales como CIFAR10, Oxford-Flowers y Oxford-IIT-Pets. Se emplea el optimizador AdamW para entrenar la sonda lineal, con una tasa de aprendizaje de 5×10^{-5}, un decaimiento de pesos de 0.01 y un tamaño de lote de $B = 256$. También se aplican las augmentaciones de datos mencionadas anteriormente a los datos de imagen.

Resultados experimentales. La Tabla 8.5 demuestra que los modelos CRATE alcanzan un rendimiento equivalente o superior al de la popular arquitectura Vision Transformer (ViT) con un número similar de parámetros, al menos en lo que respecta a la separabilidad lineal de sus características con respecto a las diferentes clases. En cuanto a la fidelidad de los mapas de atención, la Figura 8.12 muestra un resultado verdaderamente extraordinario: sin necesidad de entrenar con datos de segmentación o detección de objetos; *no solo* los mapas de saliencia capturan eficazmente todas las partes relevantes de la imagen de entrada, sino que *se autoorganizan* para que cada uno

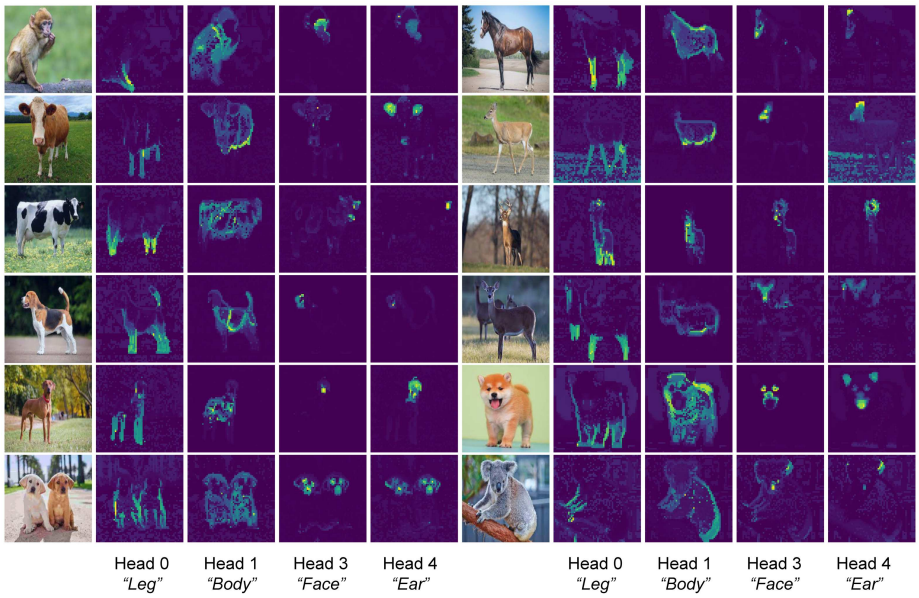


Figura 8.12: **Mapas de saliencia interpretables en CRATE** con un tamaño de parche de 8. Al recibir imágenes con propiedades similares (quizás pero no necesariamente de la misma clase), los mapas de saliencia correspondientes a diferentes cabezales de atención en la última capa resaltan cada uno una propiedad específica. Se puede observar que el mapa de saliencia promedio (no incluido) resalta, como consecuencia, todos los objetos relevantes de la imagen, lo que demuestra que utiliza todos los detalles finos de la imagen de entrada para la clasificación. Este es el *primer* sistema de aprendizaje automático que hace esto, según el conocimiento de los autores, y aún más notablemente, de forma automática sin haber sido entrenado con datos de segmentación.

Tabla 8.6: **Detección de objetos y segmentación de grano fino mediante MaskCut en COCO val2017** [LMB+14]. En este caso, todos los modelos se entrenan con un tamaño de parche de 8 en lugar de 16. CRATE obtiene, sin lugar a dudas, mejores resultados que ViT en las métricas de detección y segmentación cuando ambos se entrenan mediante clasificación supervisada.

	Detección (↑)			Segmentación (↑)		
Modelo	AP_{50}	AP_{75}	AP	AP_{50}	AP_{75}	AP
CRATE-S/8	**2.9**	**1.0**	1.1	1.8	**0.7**	0.8
CRATE-B/8	**2.9**	**1.0**	**1.3**	**2.2**	**0.7**	**1.0**
ViT-S/8	0.1	0.1	0.0	0.0	0.0	0.0
ViT-B/8	0.8	0.2	0.4	0.7	0.5	0.4

corresponda a un conjunto discreto de conceptos, ¡incluso entre muestras y clases! Este es el primer sistema que logra esto, según el conocimiento de los autores, y puede hacerlo sin utilizar ningún dato adicional, salvo los datos de clasificación de imágenes. La Tabla 8.6 confirma estas observaciones cualitativas de manera cuantitativa, mostrando una mejora significativa con respecto a los ViT entrenados en la misma configuración de clasificación supervisada.

8.5 Representación de imágenes por autocodificación enmascarada

En esta sección, se muestra cómo aplicar la arquitectura CRATE al problema del *completado de imágenes*, también conocido como *autocodificación enmascarada* (MAE), que puede considerarse una generalización del problema del completado de matrices de rango bajo tratado en el Capítulo 2. El autocodificación enmascarada, desde su introducción en el contexto del aprendizaje profundo por [HCX+22], ha sido un método consolidado y sencillo de aprendizaje de representaciones autosupervisado, cuyo objetivo es dotar a cada característica de parche dentro de $\boldsymbol{Z}_\theta$ de información agregada así como de información sobre sus vecinos, de modo que tanto la característica del parche como las características agregadas sean fuentes ricas de información para la muestra completa.

Los datos que se utilizan para esta tarea son los mismos que los conjuntos de datos de imágenes discutidos en la Sección 8.2.1. Como de costumbre, se siguen aplicando augmentaciones de datos a cada muestra de cada nuevo lote.

8.5.1 La tarea MAE y su objetivo

Como su nombre indica, la autocodificación enmascarada implica una vista v_m que, dada una entrada, realiza un recorte aleatorio redimensionado (véase Sección 8.2.2) para convertir la imagen de entrada en una imagen cuadrada de tamaño $(C, S_{\text{mask}}, S_{\text{mask}})$, y luego *enmascara* (es decir, pone a cero) un porcentaje fijo $p_{\text{mask}} \in [0, 1]$ de los píxeles de la entrada. Por motivos de

eficiencia[6], el enmascaramiento se realiza por parches; es decir, tras incrustar la imagen completa, el porcentaje p_{mask} de los parches se establece en cero. El objetivo del MAE es entrenar un codificador $f_\theta \colon \mathcal{I} \to (\mathbb{R}^d)^*$ y un decodificador $g_\eta \colon (\mathbb{R}^d)^* \to \mathcal{I}$ capaces de reconstruir una entrada a partir de su enmascarado; es decir, si se establece que $\hat{\boldsymbol{X}}_{\theta,\eta} \doteq g_\eta \circ f_\theta$, se tiene

$$\min_{\theta,\eta} \left\{ \mathcal{L}_{\text{MAE}}(\theta,\eta) \doteq \mathbb{E} \, \|\hat{\boldsymbol{X}}_{\theta,\eta}(\boldsymbol{X}_m) - \boldsymbol{X}\|_F^2 \right\} \tag{8.5.1}$$

En esencia, esto significa que las características $\boldsymbol{Z}_\theta(\boldsymbol{X}_m)$ de la vista $\boldsymbol{X}_m \doteq v_m(\boldsymbol{X})$ deben contener información sobre los *parches enmascarados*, así como sobre los parches existentes. Desde la perspectiva de los modelos transparentes white-box basados en compresión descritos en el Capítulo 5, si un autocodificador white-box (f_θ, g_η) tiene éxito en esta tarea, significa que los subespacios y diccionarios aprendidos realizan una codificación *redundante* de los datos, de tal manera que puede reconstruir las partes faltantes a partir de otras partes codificadas de los mismos. Esto significa que la información sobre cada parche se almacena en otros parches. Por lo tanto, cada característica de un parche debería contener tanto información sobre el parche como información sobre las estadísticas de toda la imagen. Así pues, una vez más, se espera que las representaciones contengan información semánticamente relevante tanto a nivel local como global, y, por lo tanto, las representaciones de diferentes parches con información local y global similar deberían estar relacionadas (es decir, en el mismo subespacio o codificadas juntas por un diccionario).

8.5.2 Arquitectura MAE basada en CRATE

Se utilizan un codificador y un decodificador CRATE, tal y como se muestra en la Figura 8.7, aunque, por supuesto, es posible utilizar un codificador y un decodificador transformer estándar. A continuación se ofrecen los detalles.

El codificador. El codificador es el mismo que el codificador CRATE de la Sección 8.4.2, con la salvedad de que no hay ningún extractor de características f_θ^{ext}. Sin embargo, tanto el *embedding* f_θ^{emb} como la red troncal f_θ^{bb} son las mismas.

El *backbone* del decodificador. El *backbone* del decodificador es el decodificador CRATE descrito en el Capítulo 6. Para completar la información, lo describimos a continuación. Dada una secuencia de características $\boldsymbol{Z}_\theta(\boldsymbol{X}) \doteq$

[6] La implementación original de MAE por [HCX+22] incrusta la imagen completa, *elimina* los tokens que se enmascararían, pasa el conjunto de tokens resultante a través del codificador, vuelve a añadir los tokens de marcador de posición aprendidos en los espacios enmascarados y vuelve a añadir la codificación posicional adecuada, y pasa el conjunto de tokens resultante a través del decodificador para obtener la predicción de autocodificación. Esto resulta más eficiente, ya que el codificador tiene que procesar menos tokens, pero, en esencia, es igual que el método descrito en el texto, y el rendimiento de los modelos resultantes en la tarea de autocodificación enmascarada y en las evaluaciones posteriores es muy similar.

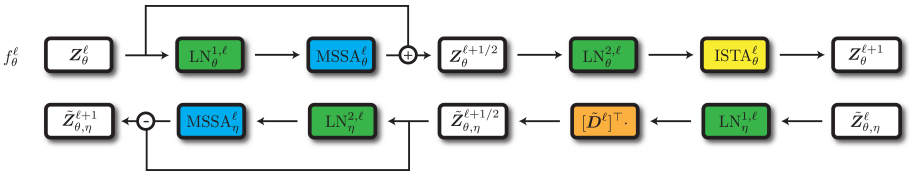


Figura 8.13: **Una capa del codificador y del decodificador en un *backbone* de autocodificador CRATE.** Tanto las capas del codificador como las del decodificador hacen pasar sus entradas a través de una autoatención de subespacio de múltiples cabezas y un paso de aprendizaje de diccionario o codificación de diccionario. Tenga en cuenta que las capas del codificador y del decodificador están diseñadas simétricamente; el objetivo conceptual de cada capa del decodificador es invertir una capa del codificador, por lo que esta simetría es en gran medida deliberada (véase, por ejemplo, el Capítulo 6).

$f_\theta(\boldsymbol{X}) \in (\mathbb{R}^d)^*$, podemos procesarla utilizando el *backbone* del decodificador g_η^{bb} de la siguiente manera. La función g_η^{bb} se compone de L capas g_η^ℓ, es decir,

$$g_\eta^{\mathrm{bb}} = g_\eta^L \circ \cdots \circ g_\eta^1. \tag{8.5.2}$$

La capa g_η^ℓ se implementa de la siguiente manera. En primer lugar, se define $\tilde{\boldsymbol{Z}}_{\theta,\eta}^1(\boldsymbol{X}) \doteq \boldsymbol{Z}_\theta(\boldsymbol{X})$. Entonces, se obtiene

$$\tilde{\boldsymbol{Z}}_{\theta,\eta}^{\ell+1/2}(\boldsymbol{X}) = [\tilde{\boldsymbol{D}}^\ell]^\top \mathrm{LN}_\eta^{1,\ell}(\tilde{\boldsymbol{Z}}_{\theta,\eta}^\ell(\boldsymbol{X})) \tag{8.5.3}$$

$$\tilde{\boldsymbol{Z}}_{\theta,\eta}^{\ell+1}(\boldsymbol{X}) = \tilde{\boldsymbol{Z}}_{\theta,\eta}^{\ell+1/2}(\boldsymbol{X}) - \mathrm{MSSA}_\eta^\ell(\mathrm{LN}_\eta^{2,\ell}(\tilde{\boldsymbol{Z}}_{\theta,\eta}^{\ell+1/2})) \tag{8.5.4}$$

y g_η^ℓ se define de tal manera que $g_\eta^\ell(\tilde{\boldsymbol{Z}}_{\theta,\eta}^\ell) \doteq \tilde{\boldsymbol{Z}}_{\theta,\eta}^{\ell+1}(\boldsymbol{X})$. Aquí, el concepto relevante es que g_η^ℓ debe aprender una inversa aproximada de $f_\theta^{L+1-\ell}$, como discretizaciones de un proceso de difusión hacia adelante y hacia atrás en el tiempo, respectivamente. En particular, $\tilde{\boldsymbol{D}}^\ell$ debería aproximarse a $\boldsymbol{D}^{L+1-\ell}$, y de manera similar, los parámetros de MSSA_η^ℓ deberían ser similares a los parámetros de $\mathrm{MSSA}_\theta^{L+1-\ell}$. La salida es $\tilde{\boldsymbol{Z}}_{\theta,\eta} \doteq \tilde{\boldsymbol{Z}}_{\theta,\eta}^{L+1}$.

El módulo de desincrustación. Para transformar $\tilde{\boldsymbol{Z}}_{\theta,\eta}(\boldsymbol{X})$ de nuevo en una estimación de $\boldsymbol{X}$, es preciso revertir el efecto del módulo de incrustación f_θ^{emb} utilizando el módulo de desincrustación g_η^{unemb}. Así pues, acudiendo a la forma funcional del módulo de *embedding* en (**??**), es decir,

$$f_\theta^{\mathrm{emb}}(\boldsymbol{X}) \doteq \left[\boldsymbol{z}_{\mathrm{cls}}^1, \boldsymbol{W}^{\mathrm{emb}} f^{\mathrm{patch}}(\boldsymbol{X}) + \boldsymbol{E}^{\mathrm{pos}}\right] \tag{8.5.5}$$

esto implica que nuestra operación inversa g_η^{unemb} es la siguiente:

$$g_\eta^{\mathrm{unemb}}(\tilde{\boldsymbol{Z}}) \doteq g_\eta^{\mathrm{unemb}}(\left[\tilde{\boldsymbol{z}}^1, \ldots, \tilde{\boldsymbol{z}}^n\right]) \tag{8.5.6}$$

$$= g^{\mathrm{unpatch}}(\boldsymbol{W}^{\mathrm{unemb}}([\tilde{\boldsymbol{z}}^2, \ldots, \tilde{\boldsymbol{z}}^n] - \tilde{\boldsymbol{E}}^{\mathrm{pos}})), \tag{8.5.7}$$

donde g^{unpatch} realiza la operación inversa al proceso de desenrollado y aplanamiento que lleva a cabo f^{patch}.[7]

Esta arquitectura es un autocodificador white-box (f_θ, g_η), donde (recordemos) $f_\theta = f_\theta^{\text{bb}} \circ f_\theta^{\text{emb}}$ y $g_\eta = g_\eta^{\text{unemb}} \circ g_\eta^{\text{bb}}$. En particular, podemos usarla para calcular una estimación de una vista enmascarada $\hat{\boldsymbol{X}}_{\theta,\eta}(\boldsymbol{X}_m) = (g_\eta \circ f_\eta)(\boldsymbol{X}_m)$, que debería aproximarse al propio $\boldsymbol{X}$.

8.5.3 Optimización

Al igual que en la Sección 8.4.3, se utiliza una configuración de optimización sencilla: se toman muestras de imágenes y máscaras, se calcula la pérdida sobre esas muestras y los gradientes de esta pérdida y se actualizan los parámetros utilizando un algoritmo de optimización genérico y los gradientes antes mencionados. Para cada paso temporal k, se hace lo siguiente:

- Tomar una submuestra de B muestras diferentes $\{\boldsymbol{X}_b^{(k)}\}_{b=1}^B \subseteq \mathcal{I}$.
- Para cada muestra $\boldsymbol{X}_b^{(k)}$, calcular un recorte aleatorizado y reescalado, junto con una máscara, distintos para cada b, m, y aplicarlos a $\boldsymbol{X}_b^{(k)}$ para obtener $\boldsymbol{X}_{b,m}^{(k)} \doteq v_{b,m}^t(\boldsymbol{X}_b^{(k)})$.
- Calcular la autocodificación estimada $\hat{\boldsymbol{X}}_{\theta,\eta}(\boldsymbol{X}_{b,r}^{(k)}) \doteq (g_\eta \circ f_\theta)(\boldsymbol{X}_{b,r}^{(k)})$.
- Formular la pérdida estocástica sustitutiva

$$\hat{\mathcal{L}}_{\text{MAE}}^{(k)}(\theta, \eta) \doteq \frac{1}{B} \sum_{b=1}^{B} \|\hat{\boldsymbol{X}}_{\theta,\eta}(\boldsymbol{X}_{b,r}^{(k)}) - \boldsymbol{X}_b^{(k)}\|_F^2. \tag{8.5.8}$$

- Calcular un paso de un algoritmo de optimización sobre (θ, η), lo que da lugar a la siguiente iteración:

$$(\theta^{(k+1)}, \eta^{(k+1)}) \doteq \textsc{OptUpdate}^{(k)}(\theta^{(k)}, \eta^{(k)}; \nabla_{(\theta,\eta)} \hat{\mathcal{L}}_{\text{MAE}}^{(k)}). \tag{8.5.9}$$

8.5.4 Evaluación

Esta es la primera red de autocodificador que se analiza en este capítulo. Se utiliza el mismo recorte central v_{cc} que en Secciones 8.2.5 y 8.4.4, redimensionando la imagen final a un cuadrado con un lado de $S_{\text{cc}} = S_{\text{mask}}$ píxeles para que coincida con las formas de las imágenes de entrada observadas durante el entrenamiento.

Además de evaluar la pérdida de autocodificación enmascarada en sí misma, también es posible evaluar directamente las características $\boldsymbol{Z}_\theta(\boldsymbol{X}_{\text{cc}})$ de la vista

[7] Una vez más, la «codificación posicional inversa» $\tilde{\boldsymbol{E}}^{\text{pos}}$ se aprende para entradas grandes, y para entradas más pequeñas puede interpolarse. Incluso es posible establecer directamente $\tilde{\boldsymbol{E}}^{\text{pos}}$ igual a la codificación posicional $\boldsymbol{E}^{\text{pos}}$ y utilizar las mismas codificaciones posicionales interpoladas para cada entrada tanto en el codificador como en el decodificador.

$\boldsymbol{X}_{\text{cc}} \doteq v_{\text{cc}}(\boldsymbol{X})$ de los datos $\boldsymbol{X}$. Para la evaluación de la fidelidad del mapa de atención, basta con obtener $\boldsymbol{Z}_\theta(\boldsymbol{X}_{\text{cc}})$, pero para el sondeo lineal (linear probing) se requiere extraer una característica resumida o agregada de $\boldsymbol{Z}_\theta$. Para ello, se puede utilizar un mapa de extracción de características (sin parámetros) que devuelva la característica correspondiente al token de clase, es decir,

$$f_\theta^{\text{ext}}(\boldsymbol{Z}) \doteq f_\theta^{\text{ext}}([\boldsymbol{z}^1, \ldots, \boldsymbol{z}^n]) = \boldsymbol{z}^1, \tag{8.5.10}$$

de forma análoga a lo descrito, por ejemplo, en las Secciones 8.4.1 y 8.4.2. Con esto, se obtienen características agregadas $\boldsymbol{z}_\theta(\boldsymbol{X}_{\text{cc}}) \doteq (f_\theta^{\text{ext}} \circ f_\theta)(\boldsymbol{X}_{\text{cc}})$, a partir de las cuales es posible realizar sondeos lineales, evaluaciones de segmentación, etc.

8.5.5 Experimentos

Dado que CRATE-MAE se basa directamente en ViT-MAE, se comparan los parámetros óptimos de ViT-MAE, tal y como se indican en [HCX+22], con los mismos parámetros aplicados a CRATE-MAE, con el fin de realizar una comparación imparcial.

Arquitectura del modelo. Durante el entrenamiento, el recorte con enmascaramiento v_m redimensiona la imagen completa de modo que el borde más corto tenga un tamaño de 256 (es decir, $S_{\text{rsz}} = 256$) antes de realizar un recorte aleatorio de tamaño 224×224 (es decir, $S_{\text{mask}} = 224$), y enmascarando $p_{\text{mask}} = \frac{3}{4}$ de los parches. Se toma un tamaño de parche de 16 (es decir, $P_H = P_W = 16$). Se utilizan las variantes 'small' y 'base' de la arquitectura ViT-MAE como *embedding* y *backbone* tanto para el codificador como para el decodificador, sustituyendo MHSA por MSSA y MLP por ISTA junto con capas lineales. En el caso de MSSA, se emplean el mismo número de cabezales y la misma dimensión de cabezal. Sin embargo, el ViT-MAE original utiliza un codificador que emplea casi la totalidad de las capas y un decodificador que solo utiliza unas pocas capas; nosotros asignamos la mitad del número total de capas (que se mantiene igual de ViT-MAE a CRATE-MAE) a nuestro codificador y decodificador, tal como sugiere el marco conceptual y teórico del Capítulo 6. Para CRATE-MAE se establece $(\beta, \lambda) = (1, 0.1)$.

Conjuntos de datos y optimización. Para el preentrenamiento, se utiliza el conjunto de datos ImageNet-1K. Se emplea el optimizador AdamW para preentrenar tanto nuestra replicación de ViT-MAE como CRATE-MAE. Se establece la tasa de aprendizaje base en 3×10^{-5}, la tasa de desvanecimiento de pesos en 0.1 y el tamaño del lote en $B = 4096$. Nuestro calendario de tasas de aprendizaje aumenta la tasa de aprendizaje en forma lineal hasta la tasa de aprendizaje base durante las primeras 40 épocas, y la reduce a 0 siguiendo un programa coseno durante las siguientes 760 épocas (entrenando todos los modelos durante 800 épocas cada uno). Para el preentrenamiento, se aplica el

Tabla 8.7: **Precisión de clasificación mediante sondeo lineal de CRATE-MAE y ViT-MAE** en diversos conjuntos de datos con diferentes tamaños de modelo, cuando el *backbone* se ha entrenado previamente para la autocodificación enmascarada en ImageNet-1K. Con el mismo número de parámetros, CRATE-MAE alcanza un rendimiento más o menos similar, al tiempo que cuenta con un diseño arquitectónico más sencillo y basado en principios.

Modelo	CRATE-MAE-S(mall)	CRATE-MAE-B(ase)	ViT-MAE-S	ViT-MAE-B
# parámetros	25.4M	44.6M	47.6M	143.8M
CIFAR10	79.4	80.9	79.9	87.9
CIFAR100	56.6	60.1	62.3	68.0
Oxford Flowers-102	57.7	61.8	66.8	66.4
Oxford-IIIT-Pets	40.6	46.2	51.8	80.1

régimen habitual de augmentación de datos (volteos, desenfoques gaussianos, solarización, etc.) a los datos de imagen.

Para el sondeo lineal (linear probing), se utilizan varios conjuntos de datos de evaluación, tales como CIFAR10, CIFAR100, Oxford-Flowers y Oxford-IIT-Pets. Se precalculan las características de todas las muestras del conjunto de datos de destino y se aplica un solucionador rápido de regresión lineal, por ejemplo, de un paquete estándar como Scikit-Learn.

Resultados experimentales. La Tabla 8.7 demuestra que los modelos CRATE-MAE alcanzan, a grandes rasgos, un rendimiento equivalente al de la popular arquitectura ViT-MAE con un número similar de parámetros, y también que el rendimiento en el aprendizaje de características (medido por el rendimiento en tareas de clasificación aguas abajo) aumenta con la escala. Por otra parte, la Figura 8.14 muestra que los mapas de saliencia del codificador (y, por lo tanto, las características de alta granularidad aprendidas por el codificador) realmente aíslan y resaltan las partes clave de la imagen de entrada.

8.6 Generación de imágenes mediante autocodificación y muestreo

En la sección anterior, hemos introducido la autocodificación enmascarada (MAE) como un enfoque para aprender una autocodificación de imágenes naturales $\boldsymbol{x} \sim p(\boldsymbol{x})$:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}=f} \boldsymbol{z} \xrightarrow{\mathcal{D}=g} \hat{\boldsymbol{x}}. \tag{8.6.1}$$

donde $\boldsymbol{z} = f(\boldsymbol{x})$ es un determinado espacio de características. Este proceso de autocodificación está diseñado para aprender la correlación, o coocurrencia, entre los parches que componen una imagen natural.

Sin embargo, el MAE no impone que la distribución de las características aprendidas $\boldsymbol{z}$ tenga estructuras o propiedades explícitas, lo cual puede ser muy importante si queremos utilizar las características así aprendidas para ciertas

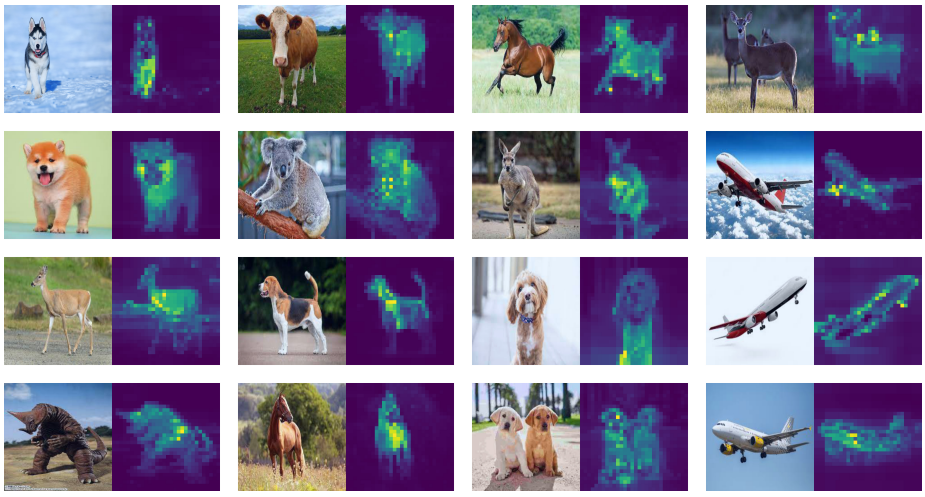


Figura 8.14: **Mapas de saliencia de CRATE-MAE.** Cada par de imágenes consta de la imagen original (izquierda) y un mapa de saliencia seleccionado (derecha) correspondiente a una cabeza de atención de la última capa. Como es habitual en los modelos CRATE, pero inusual en los modelos generales de tipo transformer, los mapas de saliencia corresponden a los objetos de la imagen de entrada.

tareas posteriores, como el reconocimiento o la generación controlada.Además, la pérdida MMSE simple que se utiliza para completar la imagen no respeta necesariamente las posibles estructuras no lineales de la variedad de la imagen y, por lo general, genera un valor esperado —y, por lo tanto, una versión borrosa— para los parches de imagen que faltan, tal como hemos mostrado en la Figura 7.8 del Capítulo 7 anterior.

En consecuencia, en esta sección presentaremos otros métodos para aprender representaciones de (la distribución de) imágenes naturales mediante autocodificación, cuyo objetivo es, en gran medida, subsanar los dos problemas mencionados anteriormente. Más concretamente, queremos que la distribución de las características codificadas presente ciertas estructuras deseadas y que las imágenes decodificadas a partir de dichas características sean mucho más fieles a la distribución de las imágenes naturales; es decir, que tengan una calidad visual mucho mejor.

8.6.1 Autocodificación variacional de datos de imágenes

Como hemos presentado en la Sección 6.1.4 del Capítulo 6, la autocodificación variacional es un método que tiene como objetivo hacer que la distribución de las características aprendidas $\boldsymbol{z}$ y las distribuciones condicionales entre $\boldsymbol{x}$ y $\boldsymbol{z}$ se asemejen más a una distribución gaussiana.[8]

[8]Se puede considerar que dicha distribución a priori tiene como objetivo promover la diversidad de las características aprendidas, en un espíritu similar al de maximizar la tasa

Aquí presentamos una implementación muy utilizada de una VAE para datos de imágenes, siguiendo el modelo de compresión perceptual aprovechado en *Stable Diffusion* [RBL+22]. La idea clave es entrenar un autocodificador que mapee imágenes de alta resolución en un espacio latente de baja dimensión, al tiempo que se impone una regularización KL sobre la distribución latente para fomentar que se acerque a una gaussiana estándar $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Este espacio latente puede funcionar posteriormente como un espacio de trabajo eficaz para modelos generativos posteriores, como la difusión latente. Tanto el codificador como el decodificador se entrenan conjuntamente de extremo a extremo. A continuación, esbozamos los detalles.

El codificador ($\mathcal{E}$). El codificador $\mathcal{E}$ mapea una imagen RGB de entrada $\boldsymbol{x} \in \mathbb{R}^{H\times W\times 3}$ a un espacio latente de baja dimensión. Su arquitectura, adoptada a partir de [ERO21], es una red totalmente convolucional. Comienza con una convolución de entrada que proyecta la entrada RGB de 3 canales en un mapa de características con un número base de 128 canales. A continuación, siguen cuatro etapas, cada una de las cuales consta de dos bloques ResNet [HZR+16b]. La dimensión del canal se escala mediante multiplicadores $(1, 2, 4, 4)$ a lo largo de las etapas, lo que da como resultado mapas de características con $128, 256, 512, 512$ canales, respectivamente. Se realiza un submuestreo espacial por un factor de $2\times$ entre etapas consecutivas mediante convoluciones con paso, lo que da como resultado tres submuestreos y un factor de submuestreo global de $f = 2^3 = 8$. Tras las cuatro etapas, un bloque intermedio compuesto por dos bloques ResNet adicionales refina aún más las características. Por último, una convolución de salida proyecta las características en $2c$ canales, que se dividen en las dos magnitudes que parametrizan la distribución latente (descrita a continuación).

A diferencia de un autocodificador estándar —y esto es fundamental— el codificador no genera un único vector latente determinista. En su lugar, parametriza una posterior gaussiana diagonal $q(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\boldsymbol{I})$ generando dos magnitudes:

- Una media $\boldsymbol{\mu} = \mathcal{E}_\mu(\boldsymbol{x}) \in \mathbb{R}^{h\times w\times c}$ y
- Una varianza logarítmica $\log \boldsymbol{\sigma}^2 = \mathcal{E}_\sigma(\boldsymbol{x}) \in \mathbb{R}^{h\times w\times c}$.

La representación latente se obtiene entonces mediante lo que se conoce como *«truco de reparametrización»* [KW13b]:

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{8.6.2}$$

donde $\odot$ denota la multiplicación elemento a elemento. Esta formulación permite la optimización basada en el gradiente mediante el paso de muestreo estocástico.

de codificación de todas las características.

El Decodificador ($\mathcal{D}$). El decodificador $\mathcal{D}$ aprende el mapeo inverso del espacio latente al espacio de píxeles, generando la imagen reconstruida $\tilde{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}) \in \mathbb{R}^{H\times W\times 3}$. Su arquitectura es aproximadamente simétrica a la del codificador: una convolución de entrada proyecta primero el latente de c canales en el espacio de características, seguida de un bloque intermedio de dos bloques ResNet. A continuación, cuatro etapas—cada una con dos bloques ResNet—aumentan progresivamente la resolución espacial, con unas dimensiones de canales que reflejan las del codificador en orden inverso $(512, 512, 256, 128)$. Una convolución de salida final mapea las características de vuelta a 3 canales RGB. Tanto la arquitectura del codificador como la del decodificador siguen el diseño convolucional introducido en [ERO21].

Datos

Suponemos que se dispone de un conjunto de datos $\mathcal{D} = \{\boldsymbol{x}_i\}_{i=1}^{N}$ de muestras procedentes de la distribución original de los datos. Cada muestra es una imagen de la forma $\boldsymbol{x} \in \mathbb{R}^{H\times W\times 3}$, donde H y W son la altura y el ancho, respectivamente. En [RBL+22], el autocodificador se entrena sobre el conjunto de datos OpenImages con imágenes recortadas a una resolución de 256×256.

Dado el factor de submuestreo $f = 8$ y el número de canales latentes $c = 4$, el codificador mapea cada imagen de entrada a un tensor latente $\boldsymbol{z} \in \mathbb{R}^{H/f\times W/f\times c}$. Para la resolución de entrenamiento predeterminada, esto da como resultado un tensor latente de tamaño $32\times 32\times 4$. En tiempo de inferencia, el autocodificador se generaliza a otras resoluciones; por ejemplo, una entrada de $512 \times 512 \times 3$ se codifica en un latente de $64 \times 64 \times 4$.

Objetivo y métricas de evaluación

Para entrenar el VAE, nuestro objetivo es optimizar conjuntamente el codificador $\mathcal{E}$ y el decodificador $\mathcal{D}$ de tal manera que la reconstrucción $\tilde{\boldsymbol{x}} = \mathcal{D}(\mathcal{E}(\boldsymbol{x}))$ se acerque lo más posible a los datos originales $\boldsymbol{x}$, mientras se regulariza la distribución latente para que se aproxime a una gaussiana estándar como prior. Una elección empíricamente eficaz de la pérdida de reconstrucción consiste en una combinación ponderada de las pérdidas L1, LPIPS [ZIE+18] y adversarias, tal como en las GAN [GPM+14b]. Además, se incluye un término de divergencia de KL para regularizar el espacio latente. Formalmente, el objetivo de entrenamiento puede expresarse como:

$$\min_{\theta_{\mathcal{E}},\theta_{\mathcal{D}}} \mathcal{L}_{\text{VAE}}(\theta_{\mathcal{E}},\theta_{\mathcal{D}}) = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Big[\|\tilde{\boldsymbol{x}} - \boldsymbol{x}\|_1 + \lambda_1\,\text{LPIPS}(\tilde{\boldsymbol{x}},\boldsymbol{x}) \tag{8.6.3}$$

$$+ \lambda_2\,\mathcal{L}_{\text{GAN}}(\tilde{\boldsymbol{x}},\boldsymbol{x}) + \ \lambda_3\,D_{KL}\big(q(\boldsymbol{z}|\boldsymbol{x})\,\big\|\,p(\boldsymbol{z})\big)\Big], \tag{8.6.4}$$

donde $\theta_{\mathcal{E}}$ y $\theta_{\mathcal{D}}$ son los parámetros del codificador y del decodificador, respectivamente, y $\lambda_1, \lambda_2, \lambda_3$ son hiperparámetros que equilibran las

contribuciones de cada término de pérdida. Los tres primeros términos garantizan la consistencia a nivel de muestra de la misma manera que los objetivos de reconstrucción de la Sección 8.6.2. El cuarto término—la divergencia KL entre la distribución a posteriori del codificador $q(\boldsymbol{z}|\boldsymbol{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2\boldsymbol{I})$ y la distribución a priori gaussiana estándar $p(\boldsymbol{z}) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$—regulariza el espacio latente. Para una distribución a posteriori gaussiana diagonal, esto admite una expresión en forma cerrada:

$$D_{KL}\big(q(\boldsymbol{z}|\boldsymbol{x}) \,\big\|\, p(\boldsymbol{z})\big) = \frac{1}{2}\sum_{j=1}^{d}\left(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1\right), \tag{8.6.5}$$

donde $d = h \times w \times c$ es la dimensionalidad total del espacio latente.

[RBL+22], el detalle de implementación es que el peso KL λ_3 se establece en un valor muy pequeño ($\approx 10^{-6}$). Esto se debe a que el objetivo principal del término KL aquí no es permitir el muestreo incondicional del espacio latente (como en una VAE tradicional), sino más bien evitar que el espacio latente desarrolle una varianza arbitrariamente alta, lo que degradaría la relación señal-ruido para los modelos posteriores (descendientes) que operan en este espacio.

Para la evaluación, medimos la calidad de la reconstrucción calculando la puntuación FID [HRU+17b] en las imágenes reconstruidas del conjunto de validación ImageNet-1k, lo que denominamos rFID. Un valor rFID más bajo indica una mejor calidad de reconstrucción.

8.6.2 Autocodificación de representaciones de datos de imágenes

Como se discute en el Capítulo 4, el aprendizaje de representaciones estructuradas de los datos es fundamental para muchas tareas posteriores. En el aprendizaje de representaciones visuales, esto se suele lograr mediante métodos de aprendizaje autosupervisado como DINO [CTM+21] y SimDINO (analizados en la Sección 8.2). En el Capítulo 6, postulamos que una buena representación de los datos debe poder decodificarse de manera efectiva al espacio de datos original. Esto requiere un marco de autocodificación consistente que pueda reconstruir con precisión los datos originales a partir de su representación aprendida. Por lo tanto, es lógico preguntarse: ¿es consistente la representación aprendida por métodos como DINO y puede decodificarse de manera efectiva en un marco de autocodificación?

Para averiguarlo, podemos entrenar un *autocodificador de representación* (RAE) que consta de un codificador de representación preentrenado (por ejemplo, DINO) y un decodificador entrenable, siguiendo el trabajo:

`https://rae-dit.github.io`.

El codificador f mapea una muestra de entrada $\boldsymbol{x}$ a su representación aprendida $\boldsymbol{z} = f(\boldsymbol{x})$, mientras que el decodificador g mapea la representación de vuelta al espacio de datos original $\boldsymbol{x}' = g(\boldsymbol{z})$. Tal y como se detalla en el Capítulo 6, un objetivo sencillo consiste en garantizar la consistencia a nivel de muestras

entrenando al decodificador para minimizar la pérdida de reconstrucción entre los datos originales $\boldsymbol{x}$ y la reconstrucción $\boldsymbol{x}'$. A continuación, describimos los detalles.

El codificador. Utilizamos un codificador de representación preentrenado $f\colon \mathcal{X} \to \mathcal{Z}$ que asigna una muestra de datos de entrada $\boldsymbol{x} \in \mathcal{X}$ a su representación aprendida $\boldsymbol{z} = f(\boldsymbol{x}) \in \mathcal{Z}$. El codificador se mantiene congelado durante el entrenamiento. En este caso, asumimos un codificador basado en *Vision Transformer* (ViT), tal como se presenta en Sección 8.2.3.

El decodificador. Para simplificar la arquitectura, utilizamos un decodificador entrenable basado en ViT $g\colon \mathcal{Z} \to \mathcal{X}$ que mapea la representación aprendida $\boldsymbol{z}$ de vuelta al espacio de datos original, generando la reconstrucción $\boldsymbol{x}' = g(\boldsymbol{z})$.

Datos

Suponemos que tenemos acceso a un conjunto de datos $\mathcal{D} = \{\boldsymbol{x}_i\}_{i=1}^N$ de muestras procedentes de la distribución de datos original $\mathcal{X}$. Cada muestra es una imagen de la forma $\boldsymbol{x} \in \mathbb{R}^{3\times H\times W}$, donde H y W son la altura y el ancho, respectivamente. El codificador f se configura con un tamaño de parche de p_e y una dimensión de características de d. En consecuencia, para cada imagen de entrada $\boldsymbol{x}$, obtendríamos $N = HW/p_e^2$ tokens de dimensión d. El decodificador g, con un tamaño de parche de p_d, mapea entonces estos tokens visuales para reconstruir $\boldsymbol{x}' \in \mathbb{R}^{3\times H\frac{p_d}{p_e}\times W\frac{p_d}{p_e}}$ en el espacio de píxeles original. Por defecto, establecemos $p_d = p_e$ para garantizar que la imagen reconstruida tenga la misma resolución que la imagen original.

Objetivo y métricas de evaluación

Para entrenar RAE, nuestro objetivo es entrenar un decodificador g de tal manera que los datos reconstruidos $\boldsymbol{x}' = g(f(\boldsymbol{x}))$ se ubiquen lo más cerca posible de los datos originales $\boldsymbol{x}$. Esto se puede lograr minimizando una pérdida de consistencia por muestra sobre el conjunto de datos $\mathcal{D}$. Una opción empíricamente eficaz para la función de pérdida es una combinación ponderada de L1, LPIPS [ZIE+18] [9] y pérdidas adversarias como en las GAN [GPM+14b]. Formalmente, el objetivo de entrenamiento puede expresarse como:

$$\min_{\theta} \mathcal{L}_{\text{RAE}}(\theta) = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\left[\|\boldsymbol{x}' - \boldsymbol{x}\|_1 + \lambda_1 \text{LPIPS}(\boldsymbol{x}', \boldsymbol{x}) + \lambda_2 \mathcal{L}_{\text{GAN}}(\boldsymbol{x}', \boldsymbol{x})\right], \quad (8.6.6)$$

donde θ son los parámetros del decodificador g, y λ_1, λ_2 son hiperparámetros que equilibran las contribuciones de cada término de pérdida.

[9] Se trata de una pérdida perceptual basada en el cálculo de las distancias entre características en diferentes capas de una red preentrenada. Esta captura empíricamente la similitud perceptiva humana mejor que las pérdidas basadas en píxeles. Para más detalles, consulte el artículo original.

Tabla 8.8: **Resultados de la reconstrucción y el sondeo lineal de las RAE.**

(a) **Selección de codificador.** Todos los codificadores superan a SD-VAE.

Modelo	rFID
DINOv2-B	0.49
SigLIP2-B	0.53
MAE-B	0.16
SD-VAE	0.62

(b) **Los decodificadores de mayor tamaño** mejoran el rFID y, manteniéndose mucho más eficientes que los VAE.

Decodificador	rFID	GFLOPs
ViT-B	0.58	22.2
ViT-L	0.50	78.1
ViT-XL	0.49	106.7
SD-VAE	0.62	310.4

(c) **Escalado del codificador.** El rFID se mantiene estable en los distintos tamaños de las RAE.

Codificador	rFID
DINOv2-S	0.52
DINOv2-B	0.49
DINOv2-L	0.52

(d) **Calidad de la representación.** Las RAE tienen una precisión de exploración lineal mucho mayor que las VAE.

Modelo	Top-1 Acc.
DINOv2-B	84.5
SigLIP2-B	79.1
MAE-B	68.0
SD-VAE	8.0

Para la evaluación, medimos la calidad de la reconstrucción calculando la puntuación FID [HRU+17b] en las imágenes reconstruidas del conjunto de validación ImageNet-1k, lo que denominamos rFID. Un valor rFID más bajo indica una mejor calidad de reconstrucción.

Resultados experimentales

Podemos comparar RAE entrenados con diferentes codificadores preentrenados, entre ellos DINOv2, MAE y SigLIP2. Como referencia, también consideramos un autocodificador variacional (VAE) básico entrenado directamente sobre los píxeles de la imagen sin ningún codificador preentrenado. Una opción común es el SD-VAE. Los decodificadores RAE se entrenan en el conjunto de entrenamiento ImageNet-1K y se evalúan en el conjunto de validación ImageNet-1K. Los resultados se resumen en Sección 8.6.2. En la Tabla 8.8b, podemos observar que las RAE con codificadores preentrenados superan a los SD-VAE en cuanto a calidad de reconstrucción. Además, los decodificadores de mayor tamaño conducen a una mejor calidad de reconstrucción, como se muestra en Tabla 8.8b, al tiempo que siguen siendo más eficientes que los VAE. Tabla 8.8c muestra que la calidad de reconstrucción es estable para diferentes tamaños de los codificadores DINO. Por último, Tabla 8.8d demuestra que las RAE también conservan representaciones de alta calidad, alcanzando una precisión de sondeo lineal mucho mayor en comparación con las VAE.

Para complementar las métricas cuantitativas de la Sección 8.6.2, la Figura 8.15 ofrece una comparación visual de las reconstrucciones. Las reconstrucciones de RAE conservan las texturas de grano fino, los límites de los objetos y los detalles semánticos con la misma fidelidad que el modelo de referencia SD-VAE, lo cual coincide con sus mejores puntuaciones en rFID.

Figura 8.15: **Ejemplos de reconstrucción RAE.** De izquierda a derecha: imagen de entrada, RAE (DINOv2-B), RAE (Siglp-B), RAE (MAE-B), SD-VAE. Amplíe la imagen para ver los detalles.

8.6.3 Muestreo a partir de representaciones aprendidas mediante eliminación de ruido

Una vez que hemos aprendido una representación consistente y estructurada de la distribución de las imágenes naturales, por ejemplo, mediante VAE o RAE, normalmente nos interesa generar de manera eficiente *muestras* $\boldsymbol{z} \in \mathcal{Z}$ a partir de la distribución de la representación aprendida, de modo que podamos regenerar las imágenes naturales asociadas a estas muestras mediante el decodificador aprendido $\hat{\boldsymbol{x}} = g(\boldsymbol{z}) \in \mathcal{X}$. Este proceso de autocodificación se ilustra en la mitad superior de la Figura 1.27 del Capítulo 1.

Para aprender (y con ello muestrear) la distribución de las características $\boldsymbol{z}$ en el espacio latente $\mathcal{Z}$, podemos valernos del método de eliminación de ruido y difusión presentado en el Capítulo 3. Concretamente, supongamos que $p(\boldsymbol{z})$ es la distribución de las características aprendidas $\boldsymbol{z}$ en el espacio de características. Podemos aprender un proceso de eliminación de ruido que, partiendo de una distribución gaussiana estándar inicial $\boldsymbol{g} \sim \mathcal{N}(0, \boldsymbol{I})$, converja hacia esta distribución $p(\boldsymbol{z})$. Siguiendo el método presentado en el Capítulo 3, queremos aprender un *denoiser* $\bar{\boldsymbol{z}}$ tal que, de manera iterativa, transforme una muestra de la distribución inicial $\mathcal{N}(0, \boldsymbol{I})$ en una muestra de la distribución objetivo de las representaciones aprendidas $p(\boldsymbol{z})$. Este proceso se ilustra en la mitad inferior de la Figura 1.27 del Capítulo 1.

Para entender el proceso de eliminación de ruido, tomemos el caso sencillo de la coincidencia de flujos presentado en el Capítulo 3. Podemos definir el *denoiser* como un campo vectorial dependiente del tiempo $\bar{\boldsymbol{z}}_\theta(t, \boldsymbol{z}_t)$, parametrizado por θ, que transforma una muestra desde la distribución inicial hacia la distribución objetivo a lo largo del tiempo $t \in [0, 1]$. Aquí, $\boldsymbol{z}_t$ es una interpolación entre $\boldsymbol{z}_0$ y $\boldsymbol{z}_1$ en el tiempo t:

$$\boldsymbol{z}_t = (1 - t)\boldsymbol{z}_0 + t\boldsymbol{z}_1, \tag{8.6.7}$$

donde $\boldsymbol{z}_0 \sim p(\boldsymbol{z})$ es una muestra de la distribución de la representación aprendida y $\boldsymbol{z}_1 \sim \mathcal{N}(0, \boldsymbol{I})$. El objetivo de entrenamiento para el *denoiser* puede expresarse entonces como:

$$\min_{\theta} \mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\boldsymbol{z}_0 \sim p(\boldsymbol{z}), \boldsymbol{z}_1 \sim \mathcal{N}(0,I), t \sim \text{Unif}(0,1)} \left[\| \bar{\boldsymbol{z}}_\theta(t, \boldsymbol{z}_t) - (\boldsymbol{z}_1 - \boldsymbol{z}_0) \|_2^2 \right]. \quad (8.6.8)$$

Por lo general, el campo vectorial de eliminación de ruido $\bar{\boldsymbol{z}}_\theta(t, \boldsymbol{z}_t)$ se modela mediante una red profunda. Las opciones más comunes para la red profunda son una U-Net o un transformer de imágenes por difusión (DiT). A continuación ofrecemos más detalles sobre cómo se pueden implementar en la práctica.

8.6.4 Implementación de la eliminación de ruido mediante una red U-Net

La arquitectura predominante para implementar la red de eliminación de ruido $\boldsymbol{\epsilon}_\theta$ en los modelos de difusión ha sido la *U-Net* [RFB15a], una red convolucional tipo codificador-decodificador con conexiones de salto (skip connections) desarrollada originalmente para la segmentación de imágenes biomédicas. Ho et al. [HJA20] fueron los primeros en adoptar la red U-Net como estructura principal de los modelos probabilísticos de difusión con eliminación de ruido (DDPM), consolidándola como la arquitectura por defecto en este campo. Posteriormente, Dhariwal y Nichol [DN21a] llevaron a cabo una serie sistemática de ablaciones arquitectónicas que mejoraron sustancialmente el diseño de U-Net, dando lugar a lo que ellos denominan el *Modelo de Difusión Ablacionado* (ADM). Su modelo mejorado fue el primer modelo de difusión en superar a las GAN en cuanto a calidad de las muestras en ImageNet. Partiendo de estas bases, Rombach et al. [RBL+22] trasladaron la U-Net del espacio de píxeles al espacio latente de una VAE preentrenada (Sección 8.6.1) e introdujeron capas de atención cruzada para un condicionamiento multimodal flexible, dando lugar al *Modelo de Difusión Latente* (LDM) que sustenta Stable Diffusion. En esta sección, recorremos la evolución de la estructura central de U-Net a lo largo de estas tres etapas, antes de analizar las limitaciones arquitectónicas que han motivado la reciente transición hacia alternativas basadas en transformers (Sección 8.6.5).

Estructura de codificador-decodificador con conexiones de salto. La U-Net sigue una estructura de codificador-decodificador. El codificador reduce progresivamente la resolución de la entrada a través de una serie de etapas de resolución, extrayendo características a múltiples escalas espaciales; luego, el decodificador vuelve a aumentar la resolución de estas características hasta la resolución original para producir la predicción de salida. La característica distintiva de la red U-Net son sus *conexiones de salto*: en cada nivel de resolución, el mapa de características del codificador se concatena por canales con el mapa de características correspondiente del decodificador. Esto permite al decodificador combinar características generales y semánticamente ricas procedentes del cuello de botella con detalles espaciales de mayor precisión

procedentes de capas anteriores del codificador, lo cual es esencial para generar predicciones espacialmente precisas.

En la red U-Net DDPM [HJA20], la arquitectura comienza con una convolución de entrada que proyecta la entrada con ruido en un espacio de características con un número básico de canales (por ejemplo, 128). A continuación, el codificador pasa por múltiples etapas de resolución —cuatro para entradas de 32×32, o seis para entradas de 256×256— donde cada etapa consta de dos bloques residuales convolucionales. Entre etapas consecutivas se realiza una reducción espacial mediante convoluciones con paso. Las capas de autoatención se insertan en el nivel de resolución de 16×16 entre los bloques convolucionales, lo que proporciona un modelado de largo alcance limitado a baja resolución. En el cuello de botella (la resolución más baja), un bloque intermedio con bloques residuales adicionales y autoatención refina aún más las características. El decodificador es un reflejo del codificador: bloques residuales en cada nivel de resolución seguidos de un remuestreo mediante convoluciones transpuestas, con conexiones de salto que concatenan las activaciones correspondientes del codificador. La arquitectura utiliza la normalización por grupos [WH20] a lo largo de toda la red, sustituyendo la normalización de pesos de la que el diseño original partía.

Cuando la U-Net opera en el espacio de píxeles, su entrada y su salida son, respectivamente, la imagen con ruido $\boldsymbol{x}_t \in \mathbb{R}^{H\times W\times 3}$ y el ruido predicho $\hat{\boldsymbol{\epsilon}} \in \mathbb{R}^{H\times W\times 3}$. Cuando se opera en el espacio latente de una VAE preentrenada (como en LDM), la entrada se convierte en $\boldsymbol{z}_t \in \mathbb{R}^{h\times w\times c}$ y la salida es $\hat{\boldsymbol{\epsilon}} \in \mathbb{R}^{h\times w\times c}$, con $h = H/f$, $w = W/f$ tal como se define en la Sección 8.6.1.

Condicionamiento por paso de tiempo mediante normalización adaptativa por grupos. La red de eliminación de ruido debe condicionarse al paso de tiempo de difusión t para que pueda adaptar su comportamiento a los distintos niveles de ruido. Siguiendo el esquema de codificación posicional del transformer [VSP+17b], el paso de tiempo t se mapea primero a un *embedding* de frecuencia sinusoidal:

$$\text{Embed}(t)_{2i} = \sin\left(\frac{t}{10000^{2i/d_e}}\right), \quad \text{Embed}(t)_{2i+1} = \cos\left(\frac{t}{10000^{2i/d_e}}\right), \tag{8.6.9}$$

donde d_e es la dimensión del *embedding* e i indexa sus componentes. A continuación, este *embedding* se pasa a través de una pequeña red neuronal multi-capas (MLP) (normalmente dos capas lineales con una función no lineal) para generar un vector de paso temporal $\boldsymbol{e}_t \in \mathbb{R}^{d_e}$.

En lugar de limitarse a concatenar $\boldsymbol{e}_t$ a la entrada, la red U-Net inyecta la información del paso de tiempo en cada bloque residual mediante la *normalización adaptativa por grupos* (AdaGN) [DN21a]. Después de la primera convolución de cada bloque residual, se aplica una operación de normalización de grupo, seguida de una modulación afín cuyos parámetros de escala y desplazamiento se regresan a partir del *embedding* del paso de tiempo:

$$\text{AdaGN}(\boldsymbol{h}, \boldsymbol{e}_t) = \boldsymbol{y}_s \odot \text{GroupNorm}(\boldsymbol{h}) + \boldsymbol{y}_b, \tag{8.6.10}$$

donde $\boldsymbol{h}$ denota el mapa de características intermedio, $[\boldsymbol{y}_s, \boldsymbol{y}_b] = \text{Linear}(\boldsymbol{e}_t)$ son los vectores de escala y desplazamiento obtenidos a partir de una proyección lineal del *embedding* del paso de tiempo y $\odot$ denota la multiplicación elemento por elemento. Este mecanismo es análogo a la normalización adaptativa de instancias [DSK17] y al condicionamiento FiLM [PSV+18], y permite que cada capa module sus activaciones en función del nivel de ruido actual. Los experimentos de ablación en [DN21a] muestran que la eliminación de AdaGN deteriora el FID en más de 2 puntos, lo que confirma su importancia.

Para la generación condicionada por clase, simplemente se añade un *embedding* de clase $\text{Embed}(c)$ (una tabla de consulta aprendida) al *embedding* del paso temporal antes de la inyección, de modo que $\boldsymbol{e}_t$ se reemplaza por $\boldsymbol{e}_t + \text{Embed}(c)$ en la ecuación (8.6.10). Este mecanismo es ligero, pero se limita a etiquetas categóricas; las modalidades de condicionamiento más ricas, como el texto, requieren el mecanismo de atención cruzada introducido en la Sección 8.6.4.

Mejoras en la arquitectura: de DDPM a ADM

Dhariwal y Nichol [DN21a] llevaron a cabo un estudio sistemático de ablación para determinar qué modificaciones arquitectónicas mejoran más la calidad de las muestras. Partiendo del modelo de referencia DDPM U-Net [HJA20], exploraron los siguientes cambios en ImageNet 128×128: (a) ampliación de la autoatención de la resolución 16×16 a tres resoluciones (32×32, 16×16, 8×8); (b) aumento del número de cabezales de atención, utilizando 64 canales por cabezal para ajustarse mejor a la convención de Transformer; (c) adopción del bloque residual BigGAN [BDS19] tanto para el upsampling como para el downsampling, lo que mejora el flujo de gradientes; y (d) inclusión por defecto de la capa de condicionamiento AdaGN (descrita anteriormente) en cada bloque residual. Todos estos cambios mejoran el FID y sus efectos se potencian de manera positiva. El modelo resultante, al que denominan ADM, sirve como nuevo diseño predeterminado de U-Net para trabajos posteriores.

Además de las mejoras arquitectónicas, Dhariwal y Nichol introdujeron la *guía por clasificador* como mecanismo para sopesar la diversidad de la muestra frente a la fidelidad. Se entrena un clasificador $p_\varphi(c|\boldsymbol{z}_t, t)$ con imágenes ruidosas, y sus gradientes se utilizan para dirigir la trayectoria de muestreo hacia regiones de alta probabilidad de la clase deseada:

$$\hat{\boldsymbol{\mu}}_\theta(\boldsymbol{z}_t, t) = \boldsymbol{\mu}_\theta(\boldsymbol{z}_t, t) + s\, \boldsymbol{\Sigma}_\theta(\boldsymbol{z}_t, t)\, \nabla_{\boldsymbol{z}_t} \log p_\varphi(c|\boldsymbol{z}_t), \tag{8.6.11}$$

donde $s > 1$ es una escala de guía que agudiza la distribución efectiva de clases. Con esta técnica, ADM alcanzó un FID del estado del arte de 4,59 en ImageNet 256×256 condicionado a la clase, superando por primera vez el *benchmark* clásico de GAN, BigGAN-deep (FID 6,95). El modelo guiado (ADM-G) mejoró aún más el FID hasta 3,94 cuando se combinó con una pila de supermuestreo (ADM-U).

El principal inconveniente de la guía basada en clasificadores es que requiere entrenar un clasificador independiente sobre imágenes con ruido. Esta limitación

se abordó posteriormente mediante la *guía libre de clasificadores* [HS22b], que prescinde por completo del clasificador externo al aprender en forma conjunta la eliminación de ruido condicional e incondicional dentro de un único modelo, una técnica que describimos en detalle en la Sección 8.6.5.

La red U-Net en la difusión latente (Stable Diffusion)

Mientras que ADM opera directamente en el espacio de píxeles, Rombach et al. [RBL+22] observaron que gran parte de la carga computacional se dedica a modelar detalles de alta frecuencia que son irrelevantes desde el punto de vista perceptivo. Su idea clave es separar la *compresión perceptual* del *modelado generativo*: un autocodificador preentrenado (Sección 8.6.1) primero comprime la imagen en un espacio latente de baja dimensión y luego el modelo de difusión opera completamente dentro de este espacio latente. Para una imagen de 256×256 con un factor de submuestreo $f = 8$, el U-Net procesa un tensor latente de $32 \times 32 \times 4$ en lugar de un tensor de píxeles de $256 \times 256 \times 3$, lo que supone una reducción de aproximadamente $48\times$ en los elementos espaciales. Esto produce una aceleración de al menos $2,7\times$ tanto en el rendimiento del entrenamiento como en el del muestreo, al tiempo que mejora el FID.

Más allá del ahorro computacional, el marco de difusión latente introduce un *mecanismo de condicionamiento de atención cruzada* de uso general que permite a la red U-Net condicionarse en función de modalidades arbitrarias, como *prompts* de texto, mapas semánticos o cajas delimitadoras. En resumen, un codificador específico del dominio τ_θ mapea la entrada de condicionamiento $\boldsymbol{y}$ a una representación intermedia $\tau_\theta(\boldsymbol{y}) \in \mathbb{R}^{M \times d_\tau}$, que luego se inyecta en la U-Net en múltiples niveles de resolución a través de capas de atención cruzada: las características de la imagen sirven como consultas, mientras que los *embeddings* de condicionamiento sirven como claves y valores, lo que permite que cada ubicación espacial preste atención a las partes relevantes de la señal de condicionamiento. Dejamos la descripción matemática completa de este mecanismo de atención cruzada y del objetivo de entrenamiento condicional para la Sección 8.7.2.

En *Stable Diffusion* [RBL+22], el codificador de texto τ_θ se implementa como el modelo CLIP ViT-L/14 preentrenado [RKH+21b], que mapea las indicaciones de texto a una secuencia de *embeddings* de *tokens*. El propio U-Net conserva las mejoras de ADM —bloques residuales de BigGAN, atención en tres niveles de resolución, AdaGN para la inyección de pasos temporales— y, además, intercala capas de atención cruzada en estos mismos niveles de resolución para el condicionamiento de texto. El modelo completo contiene aproximadamente 860 millones de parámetros en la red U-Net y 123 millones en el codificador de texto.

Limitaciones de la arquitectura U-Net

A pesar de su éxito, la red U-Net presenta varias limitaciones que han motivado la adopción de arquitecturas alternativas. En primer lugar, **escalar** una red

U-Net resulta complicado: su estructura de codificador-decodificador implica numerosas decisiones de diseño interrelacionadas (multiplicadores de canales, ubicación de la atención, tipos de bloques de muestreo ascendente/descendente), y no está claro cómo asignar de manera óptima la capacidad de cálculo adicional. Por el contrario, el transformer básico se escala a lo largo de dos ejes simples, la profundidad y la anchura, y presenta leyes de escalado bien caracterizadas según estudios empíricos [KMH+20]. En segundo lugar, las capas convolucionales de U-Net codifican un fuerte **sesgo inductivo espacial** (localidad, equivarianza a la traslación) que resulta útil en entornos con pocos datos, pero que limita el modelado de largo alcance en conjuntos de datos de gran tamaño. La autoatención aborda parcialmente esta limitación, pero en los diseños estándar de U-Net solo se aplica en las etapas de baja resolución debido a su coste cuadrático, lo que hace que las características de alta resolución sean totalmente locales. En tercer lugar, U-Net es una arquitectura **específica para imágenes** que no se generaliza fácilmente a otras modalidades. El modelo transformer, cuya eficacia ha quedado demostrada en entornos lingüísticos, de visión, de audio y multimodales [VSP+17b], ofrece un *backbone* unificado capaz de aprovechar los conocimientos sobre escalabilidad entre dominios.

Estas consideraciones dieron lugar al Diffusion Transformer (DiT) [PX23], que sustituye por completo a la red U-Net por un transformer básico que opera sobre parches latentes. Tal como describimos en la Sección 8.6.5, DiT iguala y supera a los mejores modelos basados en U-Net, al tiempo que presenta un comportamiento de escalabilidad más limpio (correlación Gflops-FID de $-0,93$) y una arquitectura más simple.

8.6.5 Implementación de la eliminación de ruido mediante un Transformer de difusión

Una alternativa a U-Net para implementar la red de eliminación de ruido $\boldsymbol{\epsilon}_\theta$ es el *Diffusion Transformer (DiT)* [PX23], que sustituye por completo las convoluciones por bloques Transformer. El hallazgo clave es que el sesgo inductivo espacial de U-Net *no es crucial* para el rendimiento de los modelos de difusión: un transformer estándar que opera sobre secuencias de parches latentes puede igualar e incluso superar a U-Net, al tiempo que hereda las propiedades de escalabilidad favorables de la clase de arquitecturas Transformer [VSP+17b]. A diferencia de U-Net, DiT no tiene un sesgo inductivo espacial explícito: toda la estructura espacial se aprende a partir de los datos. Esta generalidad permite que la misma arquitectura pueda procesar imágenes, vídeos y otras modalidades con modificaciones mínimas.

DiT opera dentro del espacio latente de un VAE preentrenado (Sección 8.6.1), lo que convierte la *pipeline* completa de generación de imágenes en una arquitectura híbrida: un VAE convolucional para la compresión perceptual y un DDPM basado en transformers para el modelado generativo. Siguiendo el *Vision Transformer* (ViT) [DBK+21], el latente de entrada se divide en parches y se incrusta linealmente como una secuencia de tokens. A continuación describimos la arquitectura en detalle.

División en parches y la codificación posicional. La entrada a DiT es una variable latente con ruido $\boldsymbol{z}_t \in \mathbb{R}^{h\times w\times c}$, donde $h = H/f$ y $w = W/f$ son las dimensiones espaciales del espacio latente definido por el codificador VAE (por ejemplo, $32 \times 32 \times 4$ para imágenes de 256×256 con $f = 8$, $c = 4$). La primera capa, llamada *patchify*, divide $\boldsymbol{z}_t$ en una cuadrícula de parches no superpuestos de tamaño $p \times p$ e incrusta linealmente cada parche en una representación de tokens de d dimensiones. Esto produce una secuencia de

$$T = \left(\frac{h}{p}\right) \times \left(\frac{w}{p}\right) \tag{8.6.12}$$

tokens en $\mathbb{R}^d$. Se añaden *embeddings* posicionales sinusoidales estándar (la variante seno-coseno de ViT) a cada token para codificar la posición espacial. El tamaño del parche p es un parámetro de diseño clave: reducir p a la mitad cuadruplica T y, por lo tanto, el total de Gflops del Transformer se multiplica al menos por cuatro, mientras que el número de parámetros permanece prácticamente sin cambios. En [PX23], el espacio de diseño incluye $p \in \{2, 4, 8\}$; la configuración más fina $p = 2$ produce la mejor calidad de generación.

Bloque DiT con adaLN-Zero. Tras la «patchificación», la secuencia de tokens se procesa mediante una pila de N bloques de Transformer. Cada bloque sigue la estructura estándar del transformer [VSP+17b]: normalización de capa, autoatención multi-cabezal (MHSA), una segunda normalización de capa y una red hacia adelante punto a punto (MLP con activaciones GELU).

La cuestión central del diseño es cómo incorporar la información de condicionamiento —el paso temporal de difusión t y la etiqueta de clase c— en cada bloque. Peebles y Xie [PX23] comparan cuatro estrategias: (1) *condicionamiento contextual*, que añade t y c como tokens adicionales en la secuencia de entrada; (2) *atención cruzada*, que introduce una capa adicional de atención cruzada multi-cabezal que atiende a los *embeddings* de condicionamiento; (3) *normalización adaptativa de capas (adaLN)*, que realiza una regresión de los parámetros de escala y desplazamiento de cada normalización de capa a partir de la señal de condicionamiento; y (4) *adaLN-Zero*, que amplía adaLN con un mecanismo de compuerta adicional inicializado en cero. De entre ellas, adaLN-Zero alcanza el FID más bajo y es la más eficiente en términos de computación, por lo que se adopta como opción predeterminada.

En el bloque adaLN-Zero, la información de condicionamiento se forma primero sumando las representaciones del paso temporal y de la etiqueta de clase:

$$\boldsymbol{e} = \mathrm{Embed}(t) + \mathrm{Embed}(c), \tag{8.6.13}$$

donde $\mathrm{Embed}(t)$ se obtiene pasando un *embedding* de frecuencia sinusoidal de t a través de una red neuronal multi-capas (MLP) de dos capas con activaciones SiLU y $\mathrm{Embed}(c)$ es un *embedding* de clase aprendida. A partir de $\boldsymbol{e}$, una capa lineal realiza una regresión de seis conjuntos de parámetros por dimensión:

$\boldsymbol{\gamma}_1, \boldsymbol{\beta}_1, \boldsymbol{\alpha}_1$ para el subbloque de autoatención y $\boldsymbol{\gamma}_2, \boldsymbol{\beta}_2, \boldsymbol{\alpha}_2$ para el subbloque de alimentación directa. La normalización de la capa adaptativa se aplica de la siguiente manera:

$$\text{adaLN}(\boldsymbol{h}, \boldsymbol{\gamma}, \boldsymbol{\beta}) = \boldsymbol{\gamma} \odot \text{LayerNorm}(\boldsymbol{h}) + \boldsymbol{\beta}, \tag{8.6.14}$$

donde $\odot$ denota la multiplicación elemento por elemento. Los parámetros $\boldsymbol{\alpha}_1$ y $\boldsymbol{\alpha}_2$ actúan como *factores de compuerta* por dimensión que se aplican inmediatamente antes de cada conexión residual:

$$\begin{aligned} \boldsymbol{h}' &= \boldsymbol{h} + \boldsymbol{\alpha}_1 \odot \text{MHSA}\big(\text{adaLN}(\boldsymbol{h}, \boldsymbol{\gamma}_1, \boldsymbol{\beta}_1)\big), \\ \boldsymbol{h}'' &= \boldsymbol{h}' + \boldsymbol{\alpha}_2 \odot \text{MLP}\big(\text{adaLN}(\boldsymbol{h}', \boldsymbol{\gamma}_2, \boldsymbol{\beta}_2)\big). \end{aligned} \tag{8.6.15}$$

El «Zero» de adaLN-Zero se refiere a la inicialización: todos los parámetros $\boldsymbol{\alpha}$ son inicializados al vector cero por la MLP que los regresa, de modo que cada bloque DiT actúa como la *función identidad* al inicio del entrenamiento. Esta estrategia de inicialización a cero, inspirada en prácticas similares en los ResNet [GDG+17] y los U-Net de difusión, es importante para la estabilidad del entrenamiento y produce un FID significativamente más bajo que el adaLN básico.

Observamos que adaLN aplica la *misma* función de condicionamiento a todos los tokens de manera uniforme. Para la generación de imágenes a partir de texto, donde la señal de condicionamiento es una secuencia de tokens de texto de longitud variable, son comunes dos enfoques alternativos: (1) la atención cruzada, donde los tokens de imagen prestan atención a los *embeddings* de tokens de texto de un codificador congelado; o (2) anteponer los tokens de texto directamente a la secuencia de tokens de imagen, lo que permite que la autoatención del transformer procese conjuntamente ambas modalidades.

Proyección de la salida. Tras el último bloque DiT, el modelo debe decodificar la secuencia de representaciones de tokens para convertirla de nuevo en una predicción espacial. Se aplica una normalización de capa adaptativa final (condicionada por $\boldsymbol{e}$), seguida de una capa lineal que proyecta cada token en un tensor de forma $p \times p \times 2c$. El factor de $2c$ da cuenta de las dos magnitudes que predice el modelo: la estimación del ruido $\hat{\boldsymbol{\epsilon}} \in \mathbb{R}^{h \times w \times c}$ y la covarianza diagonal $\boldsymbol{\Sigma}_\theta \in \mathbb{R}^{h \times w \times c}$. A continuación, los tokens decodificados se reorganizan (se revierte la partición en parches) para volver a su disposición espacial original de $\mathbb{R}^{h \times w \times c}$ para cada salida.

Entrenamiento

El DiT se entrena como un modelo de difusión latente condicionado a la clase en el conjunto de datos ImageNet [DDS+09b] con resoluciones de imagen de 256×256 y 512×512. El proceso de difusión opera sobre las representaciones latentes generadas por el VAE preentrenado de la Sección 8.6.1: para imágenes de 256×256, la entrada al DiT es $\boldsymbol{z}_t \in \mathbb{R}^{32 \times 32 \times 4}$; para imágenes de 512×512, es $\boldsymbol{z}_t \in \mathbb{R}^{64 \times 64 \times 4}$.

El objetivo de entrenamiento combina dos términos. La pérdida de predicción de ruido entrena $\boldsymbol{\epsilon}_\theta$ mediante el error cuadrático medio simple:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{\boldsymbol{z}_0,\boldsymbol{\epsilon},t}\left[\|\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,t)-\boldsymbol{\epsilon}\|_2^2\right], \tag{8.6.16}$$

donde $\boldsymbol{z}_0$ es la variable latente sin ruido, $\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})$ es el ruido muestreado, y $\boldsymbol{z}_t=\sqrt{\bar{\alpha}_t}\,\boldsymbol{z}_0+\sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$ es la latente con ruido en el paso de tiempo t. La covarianza aprendida $\boldsymbol{\Sigma}_\theta$ se entrena adicionalmente con el término completo del cota inferior variacional D_{KL}, siguiendo el enfoque de Nichol y Dhariwal [ND21]. Se utiliza un programa de varianza lineal con $t_{\max}=1000$ pasos, que va desde 1×10^{-4} hasta 2×10^{-2}.

Todos los modelos se entrenan con AdamW [LH19] utilizando una tasa de aprendizaje constante de 1×10^{-4}, sin decaimiento de pesos y un tamaño de lote de 256. La augmentación de datos consiste únicamente en volteos horizontales aleatorios. Se mantiene una media móvil exponencial (EMA) de los pesos del modelo con un decaimiento de 0.9999; todos los resultados reportados utilizan el modelo EMA. Cabe destacar que, a diferencia del entrenamiento supervisado de ViT, DiT *no* requiere un período de calentamiento de la tasa de aprendizaje ni regularización *alguna*: el entrenamiento es muy estable en todas las configuraciones del modelo, sin que se observen picos de pérdida.

Guía libre de clasificador

Para generar un muestreo controlado por clases, DiT puede emplear fácilmente la *guía libre de clasificador* [HS22b]. Durante el entrenamiento, la etiqueta de clase c se omite aleatoriamente con cierta probabilidad y se reemplaza por un *embedding* nulo aprendido $\varnothing$, de modo que el modelo aprende tanto las predicciones de ruido condicionales $\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,c)$ como las incondicionales $\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,\varnothing)$. En el momento de la inferencia, la predicción guiada del ruido se calcula como:

$$\hat{\boldsymbol{\epsilon}}_\theta(\boldsymbol{z}_t,c)=\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,\varnothing)+s\cdot\big(\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,c)-\boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,\varnothing)\big), \tag{8.6.17}$$

donde $s>1$ es la escala de guía ($s=1$ corresponde al muestreo estándar sin guía). La motivación se deriva de la regla de Bayes: dado que $\nabla_{\boldsymbol{z}_t}\log p(c|\boldsymbol{z}_t)\propto\nabla_{\boldsymbol{z}_t}\log p(\boldsymbol{z}_t|c)-\nabla_{\boldsymbol{z}_t}\log p(\boldsymbol{z}_t)$, la estimación guiada en la ecuación (8.6.17) dirige efectivamente la trayectoria de muestreo hacia regiones de alta verosimilitud condicional $p(c|\boldsymbol{z}_t)$. Esta técnica tiene un efecto dramático en la calidad de la generación: DiT-XL/2 alcanza un FID de 9,62 sin orientación, que mejora a 2,27 con una escala de guía de s = 1,5, superando a todos los modelos de difusión anteriores en el banco de pruebas ImageNet de 256×256 con condiciones de clase.

Generación

Una vez entrenada la red de eliminación de ruido $\boldsymbol{\epsilon}_\theta$ (es decir, la DiT), se pueden generar nuevas imágenes ejecutando el proceso inverso aprendido. Partiendo de un ruido puro $\boldsymbol{z}_{t_{\max}}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I})$, el modelo elimina el ruido de manera iterativa

mediante el muestreo de $\boldsymbol{z}_{t-1} \sim p_\theta(\boldsymbol{z}_{t-1}|\boldsymbol{z}_t)$ en cada paso, utilizando el ruido predicho $\hat{\boldsymbol{\epsilon}}_\theta$ y la covarianza $\boldsymbol{\Sigma}_\theta$ para parametrizar la transición inversa. Tras $t_{\max}$ pasos de eliminación de ruido, la variable latente limpia $\boldsymbol{z}_0$ se decodifica de nuevo al espacio de la imagen mediante el decodificador VAE: $\tilde{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}_0) \in \mathbb{R}^{H\times W\times 3}$.

8.7 Representación y generación condicionadas de imágenes

En las secciones anteriores, establecimos los fundamentos del modelado generativo: una arquitectura de «autoencoder» para codificar los datos $\boldsymbol{x}$ en una variable latente $\boldsymbol{z}$ (comprimida y más estructurada):

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\mathcal{D}} \hat{\boldsymbol{x}}. \tag{8.7.1}$$

A continuación, se aprende un proceso generativo para modelar la distribución de estas variables latentes $p(\boldsymbol{z})$. En esta sección, nos enfocamos en el *control* en lugar de la *generación incondicional*. Suponemos que la representación $\boldsymbol{z}$ ya se ha aprendido (mediante los métodos detallados en el Capítulo 7); nuestra tarea específica es guiar el proceso de muestreo utilizando una señal de "control" $\boldsymbol{c}$ proporcionada por el usuario. Es decir, nuestro objetivo es obtener muestras de la distribución condicional $p(\boldsymbol{z}|\boldsymbol{c})$. En la práctica, $\boldsymbol{c}$ podría representar atributos parciales de la imagen de muestra deseada: una imagen enmascarada, una máscara semántica, información de clase o un texto que describa el contenido deseado.

8.7.1 Formulación del problema

Formalmente, el objetivo de la generación condicionada es aprender un mapeo que transforme una distribución simple de origen (normalmente un ruido gaussiano) en una distribución condicional de destino. La característica generada $\hat{\boldsymbol{z}}$ se decodifica posteriormente en una imagen correspondiente $\hat{\boldsymbol{x}}$. Tal y como se ilustra a continuación, el sistema se define por sus entradas y salidas:

1. **Entrada estocástica (ϵ):** Una variable de ruido muestreada a partir de una distribución a priori normal estándar previa $\epsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Esto proporciona la «semilla» para la diversidad, lo que garantiza que podamos generar múltiples variaciones del mismo concepto.

2. **Señal de control ($\boldsymbol{c}$):** Una entrada estructurada que define el contenido semántico o espacial deseado. Puede tratarse de un texto descriptivo (por ejemplo, «un auto rojo»), una máscara de segmentación semántica o un diseño.

3. **Salida ($\hat{\boldsymbol{x}}$):** Una muestra de datos de alta fidelidad $\hat{\boldsymbol{x}} = \mathcal{D}(\hat{\boldsymbol{z}})$ que se alinea a la intención del usuario $\boldsymbol{c}$ y permanece sobre la variedad (manifold) natural de imágenes.

Utilizamos un **codificador de condiciones** específico del dominio, denotado como τ_φ, para proyectar la señal de control $\boldsymbol{c}$ en un espacio de *embedding* útil. Por lo tanto, el proceso general de generación controlada puede describirse formalmente como una composición de un generador G_θ y el decodificador $\mathcal{D}$:

$$\underbrace{\epsilon \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})}_{\text{Source}}, \boldsymbol{c} \xrightarrow{G_\theta(\epsilon, \tau_\varphi(\boldsymbol{c}))} \hat{\boldsymbol{z}} \sim p(\boldsymbol{z} \mid \boldsymbol{c}) \xrightarrow{\mathcal{D}(\hat{\boldsymbol{z}})} \hat{\boldsymbol{x}} \approx \boldsymbol{x}. \tag{8.7.2}$$

La brecha de modalidades. Aunque la tarea es conceptualmente sencilla —establecer un mapeo de $(\epsilon, \boldsymbol{c})$ hacia $\boldsymbol{x}$— requiere tender un puente entre espacios topológicos disjuntos, especialmente en la generación de imágenes a partir de texto. Los datos visuales $\boldsymbol{x}$ se encuentran en un espacio de píxeles continuo y de alta dimensión $\mathbb{R}^{H \times W \times 3}$, mientras que la condición $\boldsymbol{c}$ suele residir en un espacio simbólico discreto (el lenguaje). No existe una métrica euclidiana natural que permita medir directamente la distancia $\|\boldsymbol{x} - \boldsymbol{c}\|$. Por lo tanto, la componente crítica $\tau_\varphi(\boldsymbol{c})$ mencionada anteriormente debe alinear las modalidades. Tal y como se explica en el Capítulo 7, los enfoques modernos se basan en codificadores contrastivos preentrenados (como CLIP) o en grandes modelos de lenguaje (como T5 [RSR+20]) para mapear tokens de lenguaje discretos en un espacio de *embedding* semántico continuo. El *backbone* generativo G_θ (es decir, Denoising U-Net) debe entonces aprender a mapear la distribución del ruido a la distribución de los datos, guiada por estos *embeddings* semánticos a través de un mecanismo de fusión específico, como la atención cruzada.

El problema inverso uno-a-muchos. El condicionamiento está intrínsecamente mal planteado porque la señal de control suele ser *subdeterminada*. Una indicación textual $\boldsymbol{c}$ = «un perro» proporciona información sobre la categoría semántica, pero omite la postura del perro, la iluminación, el fondo y el patrón del pelaje. En consecuencia, la correspondencia inversa $\boldsymbol{c} \to \boldsymbol{x}$ no es un único punto, sino una distribución multimodal compleja. Si entrenáramos un modelo de regresión determinista simple para minimizar el error euclidiano $\|\boldsymbol{x} - G_\theta(\boldsymbol{c})\|^2$, el modelo convergería hacia la media condicional $\mathbb{E}[\boldsymbol{x}|\boldsymbol{c}]$—un promedio de todos los perros posibles—, lo que daría lugar a un resultado borroso y poco realista. Para generar resultados de alta fidelidad, podemos utilizar marcos generativos (como U-Net o DiT, mencionados en la Sección 8.6) para modelar la distribución condicional completa $p(\boldsymbol{z}|\boldsymbol{c})$, lo que nos permite muestrar instancias diversas y nítidas que satisfacen la condición.

8.7.2 La implementación básica: generación de imágenes a partir de texto

Para implementar el marco condicional definido en 8.7.1, necesitamos una arquitectura concreta capaz de puentear la brecha modal entre los tokens de texto discretos y las matrices de píxeles continuas. Tal y como se

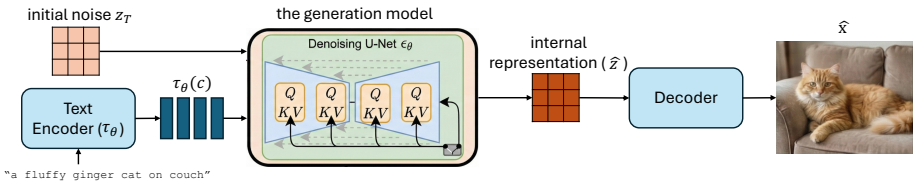


Figura 8.16: **El proceso de inferencia en tres etapas de Stable Diffusion.** (1) **Codificador de texto** (p. ej., CLIP) convierte la indicación de entrada $\boldsymbol{c}$ en *embeddings* semánticos. (2) **Modelo de generación** (una red U-Net condicional en el tiempo ϵ_θ) elimina el ruido de manera iterativa de un tensor latente aleatorio $\boldsymbol{z}_T$ para producir una representación intermedia limpia $\hat{\boldsymbol{z}}$, condicionada a los *embeddings* de texto mediante atención cruzada. (3) El **decodificador** (de una VAE) mapea la variable latente sin ruido $\hat{\boldsymbol{z}}$ de vuelta al espacio de píxeles para sintetizar la imagen final de alta resolución $\hat{\boldsymbol{x}}$. *Nota: El codificador de la VAE solo se utiliza durante el entrenamiento y se omite en este proceso de inferencia.*

ilustra en la Figura 8.16, los modelos modernos de vanguardia (como Stable Diffusion [RBL+22]) descomponen esta compleja tarea en tres componentes independientes: (1) un módulo de **compresión perceptual** (VAE) para traducir entre píxeles y latentes, (2) una interfaz de **condicionamiento semántico** (codificador de texto CLIP) para interpretar las instrucciones de texto (*prompts*) y (3) una red troncal (*backbone*) de **generación de latentes** (U-Net) para sintetizar el contenido. A continuación detallamos la implementación de cada componente.

Componente 1: Compresión perceptual (el VAE). El primer pilar de la arquitectura es un autocodificador variacional (VAE) entrenado para abstraer los detalles de alta frecuencia que, aunque son insignificantes perceptualmente, resultan computacionalmente costosos de modelar. Tal y como se presentó en la Sección 8.6.1, este componente define el **espacio latente** en el que tiene lugar la generación.

- **El codificador ($\mathcal{E}$):** El codificador mapea una imagen RGB de entrada $\boldsymbol{x} \in \mathbb{R}^{H\times W\times 3}$ a una representación latente de baja dimensión $\boldsymbol{z} = \mathcal{E}(\boldsymbol{x}) \in \mathbb{R}^{h\times w\times c}$. Siguiendo a [RBL+22], utiliza un codificador regularizado por KL con un factor de submuestreo de $f = H/h = W/w = 8$. Para una entrada estándar de $512 \times 512 \times 3$, esto produce un tensor latente compacto de dimensiones $64 \times 64 \times 4$. En el contexto del proceso de difusión (Componente 3), esta representación codificada sirve como latente «limpio» (*ground truth*) $\boldsymbol{z}_0$.
- **El decodificador ($\mathcal{D}$):** El decodificador aprende el mapeo inverso para reconstruir la imagen a partir del espacio latente, denotado como $\tilde{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}) = \mathcal{D}(\mathcal{E}(\boldsymbol{x}))$. Está entrenado para minimizar la diferencia

perceptual entre $\boldsymbol{x}$ y $\tilde{\boldsymbol{x}}$. Durante la fase de inferencia del modelo de difusión, este decodificador se mantiene congelado y se utiliza para mapear los vectores latentes generados (sin ruido) de vuelta al espacio de píxeles de alta resolución.

Cabe señalar que el VAE se entrena previamente con una combinación de pérdida perceptual y pérdida adversaria basada en parches para garantizar la calidad de las reconstrucciones. Una vez entrenado, sus pesos se congelan y actúa como una interfaz estática para el modelo de difusión.

Componente 2: Condicionamiento semántico (el codificador de texto). Para guiar el proceso de generación, se requiere una representación semántica robusta del *prompt* del usuario $\boldsymbol{c}$ (por ejemplo, «Un gato peludo de color jengibre»). En lugar de entrenar un codificador de texto desde cero, los modelos de difusión latente suelen aprovechar el codificador de texto congelado de CLIP [RKH+21b]. Este transformer preentrenado mapea los tokens del texto de entrada a una secuencia de *embeddings* semánticos $\tau_\theta(\boldsymbol{c}) \in \mathbb{R}^{d_\tau \times M}$, donde M es la longitud de la secuencia (por ejemplo, 77 tokens) y d_τ es la dimensión del *embedding* (normalmente 768 o 1024). Estos *embeddings* sirven como fuentes semánticas de «Clave» (Key) y «Valor» (Value) para la estructura central de generación.

Componente 3: Generación latente (la U-Net condicionada al tiempo). El motor de generación central se implementa como una red U-Net condicionada al tiempo ϵ_θ. A diferencia de los modelos en el espacio de píxeles, esta red opera íntegramente dentro del espacio latente comprimido. Su arquitectura se compone de una serie de bloques ResNet para la extracción de características y capas de atención cruzada para el condicionamiento.

- **Flujos de entrada:** El modelo procesa tres señales distintas:
  1. **Flujo visual:** La variable latente ruidosa $\boldsymbol{z}_t \in \mathbb{R}^{h \times w \times 4}$, que representa la imagen en el paso de difusión actual.
  2. **Paso de tiempo (t):** Para informar al modelo del nivel de ruido actual, el paso de tiempo escalar $t \in \{1, \ldots, T\}$ se mapea a un *embedding* sinusoidal (que codifica componentes de baja y alta frecuencia), seguida de una red neuronal multi-capa (MLP) compuesta por capas 'Linear' $\rightarrow$ 'SiLU' $\rightarrow$ 'Linear'. Esto produce un vector de *embedding* temporal global (por ejemplo, de 1280 dimensiones) que se inyecta en cada bloque ResNet de la U-Net.
  3. **Flujo de contexto:** Los *embeddings* de texto $\tau_\theta(\boldsymbol{c})$ de la componente 2.
- **Mecanismo de condicionamiento (atención cruzada):** La interacción entre el contenido visual y el *prompt* de texto se gestiona mediante capas de atención cruzada intercaladas a lo largo de la red U-Net.

Sea $\varphi_i(\boldsymbol{z}_t) \in \mathbb{R}^{d_\epsilon^{(i)} \times N}$ las características espaciales aplanadas en la capa i de U-Net (donde $N = h_i \times w_i$ es el número de tokens visuales y $d_\epsilon^{(i)}$ es la dimensión del canal, p. ej., 320, 640 o 1280) y sea $\tau_\theta(\boldsymbol{c}) \in \mathbb{R}^{d_\tau \times M}$ los *embeddings* de texto.

Dado que la dimensión visual $d_\epsilon^{(i)}$ y la dimensión textual d_τ difieren, proyectamos ambas en un subespacio común de dimensión d (la dimensión del cabezal de atención) utilizando pesos aprendibles. En primer lugar, definimos las matrices de consulta ($\boldsymbol{Q}$), clave ($\boldsymbol{K}$) y valor ($\boldsymbol{V}$):

$$\boldsymbol{Q} = \boldsymbol{U}_Q^{(i)} \varphi_i(\boldsymbol{z}_t), \quad \boldsymbol{K} = \boldsymbol{U}_K^{(i)} \tau_\theta(\boldsymbol{c}), \quad \boldsymbol{V} = \boldsymbol{U}_V^{(i)} \tau_\theta(\boldsymbol{c}), \tag{8.7.3}$$

donde $\boldsymbol{U}_Q^{(i)} \in \mathbb{R}^{d \times d_\epsilon^{(i)}}$ y $\boldsymbol{U}_K^{(i)}, \boldsymbol{U}_V^{(i)} \in \mathbb{R}^{d \times d_\tau}$. Adaptando la formulación por columnas de (8.4.2), la operación de atención cruzada se calcula como:

$$\text{CrossAttn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \doteq \boldsymbol{V} \cdot \text{softmax}\left(\frac{\boldsymbol{K}^\top \boldsymbol{Q}}{\sqrt{d}}\right), \tag{8.7.4}$$

donde el operador softmax se define por columnas como

$$\text{softmax}(\boldsymbol{A}) \doteq \begin{bmatrix} \text{softmax}(\boldsymbol{a}_1) & \cdots & \text{softmax}(\boldsymbol{a}_N) \end{bmatrix}, \tag{8.7.5}$$

$$\forall \boldsymbol{A} = \begin{bmatrix} \boldsymbol{a}_1, \ldots, \boldsymbol{a}_N \end{bmatrix} \in \mathbb{R}^{M \times N}. \tag{8.7.6}$$

En este contexto, la matriz $\boldsymbol{A} = \frac{\boldsymbol{K}^\top \boldsymbol{Q}}{\sqrt{d}}$ representa las puntuaciones de alineación y la función softmax aplicada columna a columna normaliza los pesos de atención sobre los M tokens de texto para cada una de las N regiones visuales. Este mecanismo permite al modelo alinear espacialmente regiones visuales específicas (por ejemplo, «ojo de gato») con tokens de texto específicos.

Objetivo de entrenamiento. El proceso de entrenamiento unifica estos componentes. Tomamos una imagen aleatoria $\boldsymbol{x}$, la comprimimos utilizando el codificador congelado $\boldsymbol{z}_0 = \mathcal{E}(\boldsymbol{x})$ y le añadimos ruido gaussiano ϵ para generar $\boldsymbol{z}_t$. A continuación, se entrena la red U-Net ϵ_θ para predecir este ruido, condicionado por el texto $\boldsymbol{c}$ y el paso de tiempo de difusión t:

$$\mathcal{L}_{\text{LDM}} = \mathbb{E}_{\mathcal{E}(\boldsymbol{x}), \boldsymbol{c}, \epsilon \sim \mathcal{N}(0,1), t} \left[\|\epsilon - \epsilon_\theta(\boldsymbol{z}_t, t, \tau_\theta(\boldsymbol{c}))\|^2 \right]. \tag{8.7.7}$$

Durante la inferencia, se descarta el codificador. El proceso comienza a partir de ruido puro $\boldsymbol{z}_T \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, lo depura iterativamente utilizando la red U-Net y, finalmente, pasa el $\hat{\boldsymbol{z}}_0$ resultante a través del decodificador $\mathcal{D}$ para sintetizar la imagen.

Métricas de evaluación. Dado que la generación condicional carece de una única y correcta «verdad de referencia» (ground truth), nos basamos en métricas estadísticas para evaluar el rendimiento. Denotamos la distribución de las imágenes reales como p_r y la distribución de las imágenes generadas como p_g.

- **FID (Distancia Fréchet de Incepción):** Esta métrica cuantifica el realismo y la diversidad de las muestras generadas midiendo la distancia entre p_r y p_g en el espacio de características de una red Inception-v3 preentrenada. Aproximamos estas distribuciones de características como gaussianas multidimensionales $\mathcal{N}(\boldsymbol{\mu}_r, \boldsymbol{\Sigma}_r)$ y $\mathcal{N}(\boldsymbol{\mu}_g, \boldsymbol{\Sigma}_g)$. La puntuación FID se define entonces como la distancia de Fréchet entre ellas:

$$\text{FID}(p_r, p_g) = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_g\|^2 + \text{Tr}\left(\boldsymbol{\Sigma}_r + \boldsymbol{\Sigma}_g - 2(\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_g)^{1/2}\right). \tag{8.7.8}$$

  Un valor más bajo del FID indica que la distribución generada es estadísticamente más cercana a la distribución real, lo que implica una mayor calidad y diversidad en las imágenes generadas.

- **Puntuación CLIP:** Esta métrica mide la alineación semántica entre una imagen generada $\hat{\boldsymbol{x}}$ y su prompt condicionante $\boldsymbol{c}$. Sean E_{img} y E_{txt} los codificadores de imagen y texto de un modelo CLIP preentrenado, tal como se explica en el Capítulo 7. La puntuación se calcula como la similitud coseno entre sus *embeddings*:

$$\text{CLIP-Score}(\hat{\boldsymbol{x}}, \boldsymbol{c}) = \max\left(0, \frac{E_{\text{img}}(\hat{\boldsymbol{x}}) \cdot E_{\text{txt}}(\boldsymbol{c})}{\|E_{\text{img}}(\hat{\boldsymbol{x}})\|\|E_{\text{txt}}(\boldsymbol{c})\|}\right) \cdot w, \tag{8.7.9}$$

  donde w es un factor de escala (normalmente 2,5). Una puntuación más alta indica que el contenido visual refleja fielmente la descripción textual.

Arquitecturas avanzadas de condicionamiento por texto (SDXL y Flux). Para hacer frente a las limitaciones en cuanto a resolución y adherencia a las instrucciones textuales (*prompts*), se han desarrollado arquitecturas avanzadas que evolucionan más allá del LDM básico. SDXL [PEL+23] introduce una estrategia de doble codificador para capturar matices textuales más ricos, calculando representaciones tanto de CLIP ViT-L como de OpenCLIP ViT-bigG y concatenándolas a lo largo del eje de canales. En cuanto al manejo de datos, SDXL utiliza el *«Aspect Ratio Bucketing»*, que agrupa en lotes las imágenes de entrenamiento con relaciones de aspecto similares para evitar que el modelo aprenda a recortar los sujetos de manera arbitraria. Además, implementa el *«Micro-Conditioning»*, en el que se incrustan metadatos como la resolución de la imagen y las coordenadas de recorte y estos se concatenan con la condición de texto $\boldsymbol{c}$, lo que permite al modelo distinguir explícitamente entre sujetos centrados y recortados.

Por su parte, Flux [Lab24] supone una transformación paradigmática, migrando de una arquitectura U-Net a un transformer de difusión multimodal (MMDiT). Procesa tokens de texto y visuales en un flujo unificado mediante bloques de atención conjunta, gracias a componentes modernos de transformers como los *embeddings* posicionales rotativas (*Rotary Positional Embeddings, RoPE*) y RMSNorm. Matemáticamente, Flux sustituye la difusión estándar por un ***Rectified Flow Matching***. A diferencia de la difusión estándar, que predice el ruido ϵ, Flow Matching simplifica el objetivo de entrenamiento para

predecir un campo de velocidades lineal v_θ entre el ruido y los datos. Definimos el proceso en un tiempo continuo $t \in [0, 1]$, donde $t = 0$ representa la distribución del ruido ($\boldsymbol{z}_0$) y $t = 1$ representa la distribución de los datos ($\boldsymbol{z}_1$).[10] El objetivo es minimizar:

$$\mathcal{L}_{\text{RFM}} = \mathbb{E}_{t,\boldsymbol{z}_1,\boldsymbol{z}_0}\left[\|v_\theta(\boldsymbol{z}_t, t, \boldsymbol{c}) - (\boldsymbol{z}_1 - \boldsymbol{z}_0)\|^2\right], \tag{8.7.10}$$

donde $\boldsymbol{z}_t = t\boldsymbol{z}_1 + (1-t)\boldsymbol{z}_0$ es el estado latente interpolado. El modelo v_θ aprende a predecir la velocidad constante $(\boldsymbol{z}_1 - \boldsymbol{z}_0)$ que transporta el ruido $\boldsymbol{z}_0$ directamente a los datos $\boldsymbol{z}_1$ a lo largo de una trayectoria en línea recta. Esta linealidad mejora significativamente la eficiencia de muestreo y la adherencia a las instrucciones en comparación con las trayectorias estocásticas curvas de los modelos de difusión tradicionales.

8.7.3 Mecanismos avanzados de condicionamiento

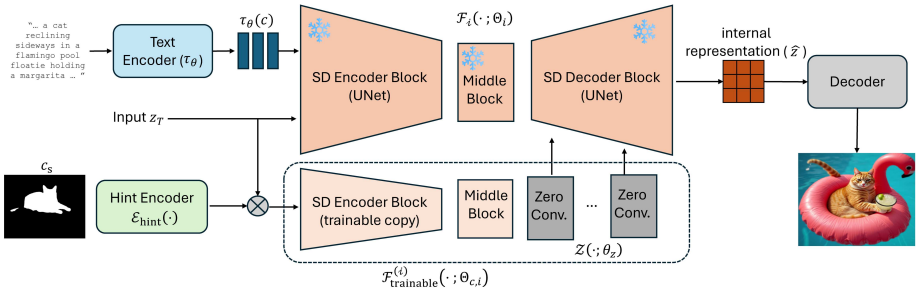


Figura 8.17: **Flujo de la arquitectura ControlNet.** Una copia entrenable del codificador y los bloques intermedios de la red U-Net de Stable Diffusion (SD), condicionada a una imagen de referencia a través del codificador de referencia $\mathcal{E}_{\text{hint}}$, inyecta mapas de características en el decodificador U-Net de SD congelado mediante convoluciones inicializadas a cero para permitir el control espacial. (Ejemplo de imagen de https://github.com/huggingface/diffusers/blob/main/docs/source/en/using-diffusers/controlnet.md).

Si bien el condicionamiento de texto proporciona control semántico, presenta dificultades a la hora de manejar restricciones espaciales detalladas (por ejemplo, poses específicas o mapas de bordes). Para abordar este problema, ControlNet [ZRA23b] introduce una arquitectura neuronal que añade control espacial a los grandes modelos de difusión preentrenados de texto a imagen, tal y como se muestra en la Figura 8.17.

Arquitectura y conectividad. ControlNet funciona clonando los parámetros entrenables de un modelo de difusión preentrenado. Específicamente, crea una copia entrenable de los 12 bloques del codificador

[10] Tenga en cuenta que esta convención de tiempo ($0 \to$ Ruido, $1 \to$ Datos) es la inversa de la notación de difusión estándar utilizada en secciones anteriores.

y 1 bloque central de la red U-Net de Stable Diffusion, mientras mantiene los parámetros originales Θ completamente bloqueados (ejecutados bajo `torch.no_grad()`), conservando la calidad de generación aprendida a partir de miles de millones de imágenes, mientras permite al modelo aprender tareas condicionales en conjuntos de datos pequeños.

Para coincidir con la resolución espacial de los mapas de características latentes $\boldsymbol{z} \in \mathbb{R}^{h \times w \times c}$ (normalmente 64×64, tal como se define en la Sección 8.7.2), la condición espacial $\boldsymbol{c}_s$ (por ejemplo, un mapa de bordes de Canny de 512×512) se procesa primero mediante un **codificador de pistas** ligero $\mathcal{E}_{\text{hint}}(\cdot)$. A diferencia del VAE, este codificador es más profundo pero más estrecho, y consta de **ocho** capas convolucionales con núcleos de 3×3 y activaciones SiLU. Utiliza tres capas con paso 2 para reducir la resolución, aumentando gradualmente los canales ($16 \to 32 \to 96 \to 256$) antes de proyectar finalmente a la dimensión del modelo (320 canales). Es crucial señalar que la capa final de este codificador se inicializa en ceros.

La innovación clave que permite la fusión de estas dos ramas es la **convolución cero**: una capa convolucional de 1×1 $\mathcal{Z}(\cdot; \theta_z)$ inicializada con los pesos W y el sesgo B fijados estrictamente en cero. La conectividad sigue un patrón específico:

1. **Inyección de entrada:** La pista codificada $\mathcal{E}_{\text{hint}}(\boldsymbol{c}_s)$ se añade **únicamente** al primer bloque de entrada de la copia de ControlNet.

2. **Inyección de salida:** La salida de cada bloque entrenable de la copia de ControlNet se pasa a través de una convolución cero y, a continuación, se suma a la **conexión de salto** correspondiente del decodificador congelado de *Stable Diffusion*.

Formalmente, consideremos el i-ésimo bloque de la red U-Net. Sea $\boldsymbol{h}_i$ el mapa de características de entrada a esta capa. Abstraemos el funcionamiento de este bloque congelado como una función $\mathcal{F}_i(\cdot; \Theta_i)$. En consecuencia, sea $\mathcal{F}^{(i)}_{\text{trainable}}(\cdot; \Theta_{c,i})$ su copia entrenable en la rama ControlNet. La salida $\boldsymbol{y}_i$ se calcula como:

$$\boldsymbol{y}_i = \mathcal{F}_i(\boldsymbol{h}_i; \Theta_i) + \mathcal{Z}\left(\mathcal{F}^{(i)}_{\text{trainable}}(\boldsymbol{h}_i + \mathcal{E}_{\text{hint}}(\boldsymbol{c}_s); \Theta_{c,i}); \theta_z\right). \tag{8.7.11}$$

Dado que la convolución cero se inicializa en ceros, el término de control desaparece al inicio del entrenamiento ($\mathcal{Z}(\cdot) = 0$), lo que garantiza que el modelo se comporte exactamente igual que el Stable Diffusion original. Sin embargo, los gradientes de los pesos W son distintos de cero:

$$\frac{\partial \mathcal{Z}(I; \{W, B\})}{\partial W_{i,j}} = I_{p,i} \neq 0, \tag{8.7.12}$$

donde I es el mapa de características de entrada. Esto garantiza que, aunque la influencia inicial sea cero, el aprendizaje comience de inmediato en la primera pasada hacia atrás.

Dinámicas de entrenamiento y experimentos. ControlNet se entrena para minimizar el objetivo de difusión estándar, pero con la condición espacial adicional $\boldsymbol{c}_s$ incorporada en el predictor de ruido. El objetivo de aprendizaje específico es:

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{z}_0,t,\boldsymbol{c}_t,\boldsymbol{c}_s,\epsilon\sim\mathcal{N}(0,1)}\left[\|\epsilon - \epsilon_\theta(\boldsymbol{z}_t,t,\boldsymbol{c}_t,\boldsymbol{c}_s)\|^2\right], \tag{8.7.13}$$

donde $\boldsymbol{c}_t$ representa la condición del prompt de texto y $\boldsymbol{z}_t$ es la variable latente con ruido. Un fenómeno singular que se observa es la «convergencia repentina»: el modelo ignora inicialmente la condición espacial y, de repente, salta bruscamente hacia la alineación, a medida que el error desciende drásticamente. Para obligar al modelo a basarse en señales estructurales en lugar de solo en el texto (que podría contradecir el mapa de bordes), una estrategia clave es el **CFG-Dropout**: sustituir aleatoriamente el 50% de las *prompts* de texto $\boldsymbol{c}_t$ por cadenas vacías durante el entrenamiento. Esto obliga a la red a extraer el significado semántico directamente de la condición espacial $\boldsymbol{c}_s$. Empíricamente, ControlNet es muy eficiente, ya que solo requiere aproximadamente un 23% más de memoria de GPU en comparación con el ajuste fino estándar y aún así aprende de manera efectiva un control robusto a partir de conjuntos de datos de tan solo 1 000 muestras.

Personalización basada en sujetos. Aunque los modelos fundacionales cuentan con un amplio prior semántico, carecen de la capacidad de sintetizar instancias de sujetos específicos (por ejemplo, el gato concreto de un usuario) de manera coherente en diferentes contextos. Para solucionar esto, DreamBooth [RLJ+23] introduce un protocolo de ajuste fino de «personalización». El principal desafío consiste en introducir un nuevo objeto en el dominio de salida del modelo sin que se produzca un «olvido catastrófico» —en el que el modelo pierde su comprensión de la clase general (por ejemplo, olvida cómo es un gato en general)— o sufra una «deriva lingüística» (sobrescribiendo la palabra «gato» con las características específicas de ese gato en concreto).

Estrategia de identificadores y datos. A diferencia del entrenamiento estándar, que requiere millones de pares, DreamBooth funciona con un conjunto de datos de «pocas muestras» proporcionado por el usuario, que suele contener solo entre 3 y 5 imágenes del sujeto. Para vincular a este sujeto con el espacio de texto del modelo, se requiere un identificador único $[V]$. Los autores proponen una estrategia de selección de tokens poco frecuentes: en lugar de utilizar caracteres aleatorios (que podrían tokenizarse en letras comunes con *priors* fuertes), el método busca en el vocabulario del tokenizador secuencias de tokens poco frecuentes (por ejemplo, en el rango del tokenizador T5-XXL $\{5000,\ldots,10000\}$) que tengan asociaciones semánticas débiles. Este identificador siempre se empareja con un sustantivo genérico de clase (p. ej., «un gato $[V]$») en el prompt. Este «anclaje» es fundamental: aprovecha los sólidos *priors* del modelo sobre la clase (saber cómo representar un «gato» en diversas posturas) al tiempo que vincula las características de identidad específicas al identificador $[V]$.

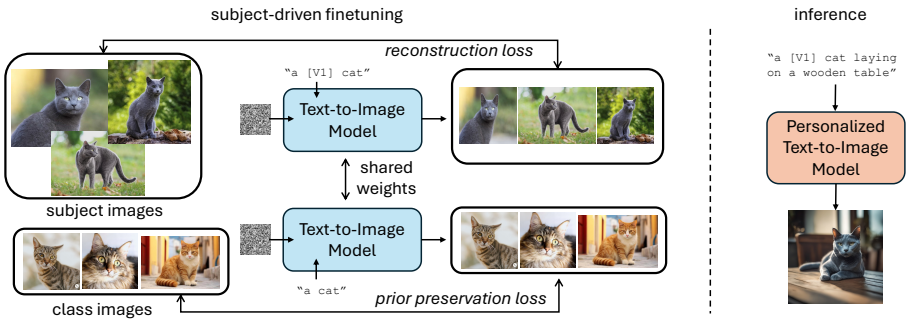


Figura 8.18: **El proceso de entrenamiento e inferencia de DreamBooth.** Durante el *ajuste fino basado en el sujeto*, el modelo de texto a imagen se actualiza utilizando una pérdida combinada: una pérdida de reconstrucción para aprender el sujeto específico (p. ej., «[V] gato») y una pérdida de preservación de los *priors* para mantener el conocimiento de la clase genérica (p. ej., «un gato»). En la *inferencia*, el modelo personalizado puede generar el sujeto específico en contextos nuevos (p. ej., «acostado sobre una mesa de madera») utilizando el identificador único.

Entrenamiento con preservación de priores. Para evitar que el modelo se sobreajuste a las imágenes de pocos ejemplos y pierda diversidad, DreamBooth emplea una pérdida autógena de preservación de priores específica para cada clase. Tal y como se ilustra en la Figura 8.18, el *pipeline* de entrenamiento minimiza un objetivo combinado:

$$\mathcal{L}_{\text{DB}} = \mathbb{E}_{\boldsymbol{z}_t,\boldsymbol{c},\epsilon,t}\left[\underbrace{\|\epsilon - \epsilon_\theta(\boldsymbol{z}_t, t, \boldsymbol{c}_{\text{subj}})\|^2}_{\text{Reconstruction Loss}} + \lambda \underbrace{\|\epsilon - \epsilon_\theta(\boldsymbol{z}'_t, t, \boldsymbol{c}_{\text{prior}})\|^2}_{\text{Prior Preservation Loss}}\right]. \tag{8.7.14}$$

El primer término realiza un ajuste fino de los pesos del modelo (normalmente toda la red U-Net y el codificador de texto) para reconstruir las imágenes del sujeto condicionadas por el *prompt* única $\boldsymbol{c}_{\text{subj}}$ (por ejemplo, «un $[V]$ gato»). El segundo término actúa como regularizador: supervisa el modelo con sus propias muestras generadas de la clase genérica $\boldsymbol{c}_{\text{prior}}$ (por ejemplo, «un gato»). Esto obliga al modelo a conservar su interpretación original de la distribución de clases, lo que garantiza que el identificador $[V]$ aprenda los matices específicos del sujeto, mientras que el sustantivo de clase «gato» sigue siendo genérico.

Personalización multi-concepto. Aunque DreamBooth es eficaz, el ajuste fino de todo el modelo requiere un gran esfuerzo computacional y consume mucho espacio de almacenamiento. Custom Diffusion [KZZ+23] mejora la eficiencia al actualizar únicamente las matrices de proyección de claves (W_K) y valores (W_V) en las capas de atención cruzada, con lo que demuestra que estos pesos específicos son suficientes para codificar nuevos conceptos.

Sin embargo, sigue siendo difícil sintetizar con precisión múltiples conceptos personalizados (por ejemplo, «un gato $[V1]$ y un perro $[V2]$») en una sola imagen

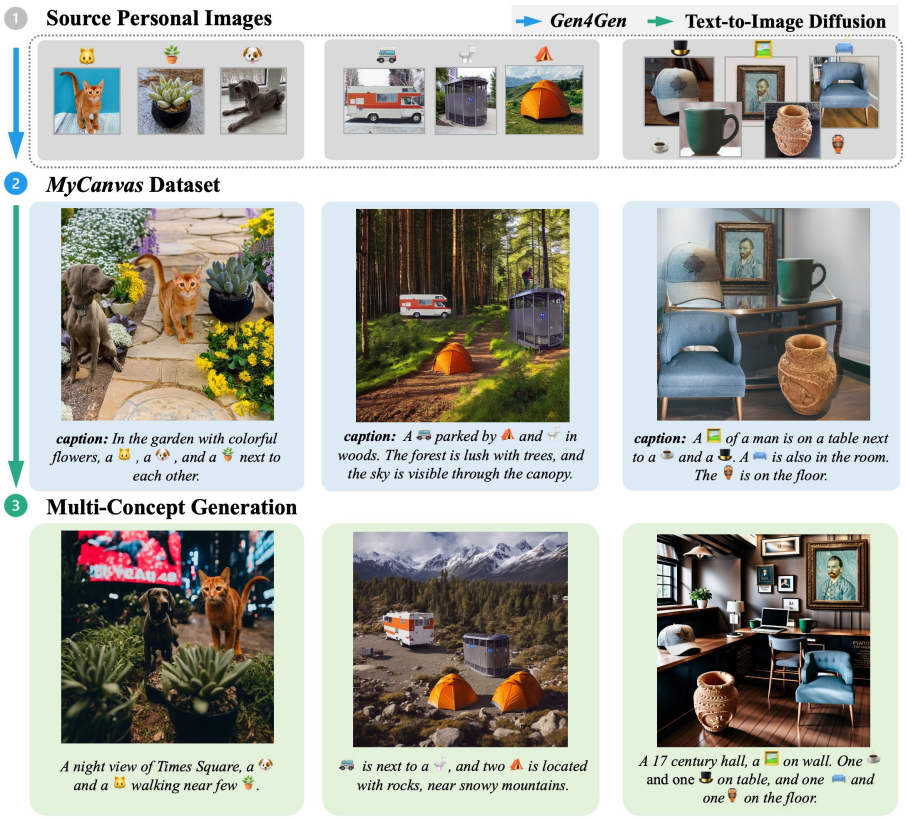


Figura 8.19: **El proceso Gen4Gen para la personalización multi-concepto.** Para superar las limitaciones de los *benchmarks* existentes en escenarios con múltiples sujetos, Gen4Gen [YCH+24] automatiza la creación de un conjunto de datos de referencia de alta calidad (MyCanvas). El entrenamiento con estos datos mejora significativamente la capacidad del modelo para generar escenas complejas con múltiples sujetos específicos (por ejemplo, «un $[V1]$ gato y un $[V2]$ perro»).

debido a la escasez de datos de entrenamiento de alta calidad, con alineación entre texto e imagen, para composiciones complejas durante la etapa de preentrenamiento de Stable Diffusion. Para abordar esto, Gen4Gen [YCH+24] propone una solución centrada en los datos ilustrada en la Figura 8.19. En lugar de modificar la arquitectura del modelo, introduce un *pipeline* generativo de datos que aprovecha los modelos fundacionales para construir «MyCanvas», un conjunto de datos de referencia que incluye escenas complejas con múltiples objetos, acompañadas de descripciones detalladas y densas. Al realizar ajustes finos sobre estos datos sintetizados, en los que las identidades de los objetos y las relaciones espaciales se han curado explícitamente, los modelos pueden mejorar significativamente su capacidad para separar y preservar múltiples sujetos personalizados al mismo tiempo (por ejemplo, un gato y un perro concretos que interactúan en una escena compleja) sin que se produzcan mezclas de identidades u omisiones.

Más recientemente, el campo se ha orientado hacia arquitecturas libres de ajuste, como PhotoMaker [LCW+24] e InstantID [WBW+24]. En lugar de la optimización iterativa, estos métodos emplean una estrategia de «apilamiento»: un codificador de identidad externo (a menudo basado en *backbones* de reconocimiento facial) extrae un *embedding* de identidad de alta fidelidad de la imagen de referencia, que luego se inyecta directamente en el proceso de difusión mediante atención cruzada desacoplada o apilamiento de prompts. Esto permite la personalización *zero-shot* de sujetos arbitrarios en una sola paso hacia adelante sin ninguna actualización de gradientes.

8.7.4 Extensión a la generación de vídeos

Los principios de la generación condicional en 2D se extienden en forma natural a la generación de vídeo al tratar el vídeo como una secuencia de fotogramas correlacionados temporalmente $\boldsymbol{x} \in \mathbb{R}^{T\times H\times W\times 3}$. Sin embargo, modelar directamente este espacio de alta dimensión resulta computacionalmente prohibitivo. Por lo tanto, los enfoques modernos de vanguardia suelen operar dentro del espacio latente comprimido definido por la VAE (Sec. 8.7.2), donde el vídeo se representa como un tensor $\boldsymbol{z} \in \mathbb{R}^{T\times h\times w\times c}$.

Inflación espacio-temporal. Para aprovechar los sólidos conocimientos previos en el dominio visual, establecidos por los generadores de imágenes 2D, arquitecturas como VideoLDM [BRL+23] y AnimateDiff [GYR+23] introducen el concepto de «expandir» las capas 2D en capas pseudo-3D. Esto se logra cambiando la forma del tensor de entrada $\boldsymbol{z}$ para alternar entre el procesamiento espacial y el temporal:

- **Capas espaciales (congeladas):** La estructura principal 2D (ResNet y atención espacial) procesa los fotogramas de manera independiente reconfigurando la entrada a $(B \cdot T, h, w, c)$. De este modo se conservan las capacidades de generación de texturas y objetos preentrenadas.

- **Capas temporales (entrenables):** Para modelar el movimiento, se reconfigura el tensor con el fin de aislar el eje temporal para cada posición espacial. Consideramos los puntos de la grilla espacial ($B \cdot h \cdot w$) como la dimensión del lote y el tiempo (T) como la longitud de la secuencia. A continuación, se aplica una capa de **autoatención temporal** para intercambiar información entre fotogramas.

Matemáticamente, sea $\boldsymbol{u} \in \mathbb{R}^{c \times T}$ el vector de características en la posición espacial de un píxel concreto a lo largo del tiempo (análogo a las características espaciales $\varphi_i(\boldsymbol{z}_t)$ de la Sección 8.7.2). Definimos las matrices temporales de *Query* ($\boldsymbol{Q}$), *Key* ($\boldsymbol{K}$) y *Value* ($\boldsymbol{V}$) utilizando pesos de proyección aprendibles $\boldsymbol{U}^{(temp)}$:

$$\boldsymbol{Q} = \boldsymbol{U}_Q^{(temp)}\boldsymbol{u}, \quad \boldsymbol{K} = \boldsymbol{U}_K^{(temp)}\boldsymbol{u}, \quad \boldsymbol{V} = \boldsymbol{U}_V^{(temp)}\boldsymbol{u}, \tag{8.7.15}$$

donde $\boldsymbol{U}_Q^{(temp)}, \boldsymbol{U}_K^{(temp)}, \boldsymbol{U}_V^{(temp)} \in \mathbb{R}^{c \times c}$. Según la formulación por columnas establecida en la Sección 8.7.2, la autoatención temporal se calcula como:

$$\text{TempAttn}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \doteq \boldsymbol{V} \cdot \text{softmax}\left(\frac{\boldsymbol{K}^\top \boldsymbol{Q}}{\sqrt{c}}\right) + \boldsymbol{u}. \tag{8.7.16}$$

Aquí, el término $\boldsymbol{K}^\top \boldsymbol{Q} \in \mathbb{R}^{T \times T}$ representa el mapa de atención temporal, que relaciona cada cuadro con todos los demás. Es fundamental que la capa de proyección de salida de este bloque de atención se inicialice típicamente en cero. Esto garantiza que, al inicio del entrenamiento, la contribución de las capas temporales desaparezca, lo que permite que el modelo se comporte exactamente como el generador 2D preentrenado y aprenda gradualmente la dinámica temporal.

Modelos de mundos interactivos. Los avances recientes han ido más allá de la síntesis pasiva de video a partir de texto para adentrarse en la simulación de mundos interactivos, tratando el generador de video, en la práctica, como un modelo predictivo de entornos físicos. Formalmente, el objetivo es modelar la probabilidad de transición latente condicionada a la acción de un agente $\boldsymbol{a}_t$:

$$p(\boldsymbol{z}_{t+1} \mid \boldsymbol{z}_{1:t}, \boldsymbol{a}_t), \tag{8.7.17}$$

donde $\boldsymbol{a}_t$ representa la señal de control (por ejemplo, la entrada del teclado o la orden al motor) en el instante t. En marcos como Genie [BDE+24], la acción $\boldsymbol{a}_t$ se discretiza en un token y se inyecta en la red utilizando los mismos mecanismos de condicionamiento que se emplean para el texto. Por ejemplo, $\boldsymbol{a}_t$ puede asignarse a un vector de *embedding* y añadirse al *embedding* del paso de tiempo o utilizarse como un par clave/valor en capas de atención cruzada. Esto permite que el modelo generativo funcione como un motor de física guiado por datos, prediciendo las consecuencias visuales de las acciones (por ejemplo, el salto de un personaje) mediante la minimización de los objetivos de reconstrucción o de coincidencia de flujos. Implementaciones a gran escala como Sora [Ope24]

sugieren además que el escalado de esta predicción autoregresiva o basada en la difusión da lugar a una consistencia 3D emergente y a la permanencia de los objetos.

8.8 Generación de objetos 3D condicionada por imágenes y texto

Hasta ahora, en las secciones anteriores, hemos mostrado principalmente cómo aprender representaciones de la distribución de imágenes *bidimensionales* (2D), así como muchas aplicaciones o tareas basadas en imágenes que dichas representaciones facilitan: como el reconocimiento de objetos a partir de imágenes, la segmentación o el completado de imágenes, y la generación de imágenes.[11]

Sin embargo, nuestro mundo es tridimensional (3D).[12] Aunque nuestros ojos solo perciben proyecciones bidimensionales del mundo —pares estereoscópicos a través de nuestros dos ojos—, somos capaces de desarrollar una memoria visual que representa un modelo tridimensional completo de nuestro entorno y de los objetos que lo componen.

En esta sección se demuestra que, utilizando los métodos introducidos en este libro, es posible aprender la distribución de las formas tridimensionales de objetos naturales y, a continuación, generar nuevas formas de objetos tridimensionales a partir de dicha distribución, condicionada a una imagen bidimensional de entrada o en una descripción textual del objeto. En concreto, el trabajo que aquí se presenta se basa en el proyecto Michelangelo [ZLC+23b], que generaliza en gran medida las técnicas que hemos introducido para la representación y generación de imágenes 2D, aplicándolas al ámbito de los datos de formas 3D[13].

En primer lugar, ofrecemos una breve introducción a los formatos de datos de uso más frecuente que representan formas 3D y los conjuntos de datos correspondientes. A continuación, siguiendo el proyecto de código abierto denominado **Michelangelo** [**NEURIPS2023_ea1a7b**]:

https://github.com/NeuralCarver/Michelangelo,

presentamos un marco computacional, los objetivos y las arquitecturas de red asociadas que intentan resolver la tarea de la generación de objetos 3D condicionada por imágenes y texto. Por último, presentamos los resultados experimentales y el análisis para verificar la solución propuesta, así como una discusión sobre mejoras adicionales.

[11] Basado en técnicas presentadas anteriormente: el transformer visual (ViT) [VSP+17a], el aprendizaje contrastivo de lenguaje e imagen (CLIP) [RKH+21c] y los modelos de difusión latente (VAE-SD) [RBL+22].

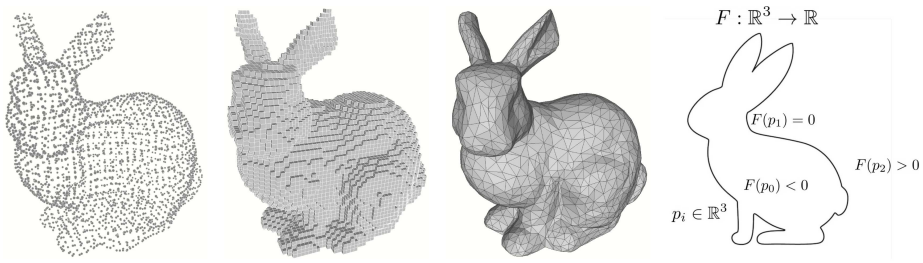
[12] o cuatridimensional (4D) si consideramos el tiempo como una dimensión adicional, ya que el mundo es dinámico.

[13] En términos generales, las mallas poligonales y los datos de vídeo pueden todos denominarse «datos 3D»; sin embargo, en esta sección, el término «datos 3D» se refiere a mallas poligonales o sus representaciones equivalentes, explícitas o implícitas.

8.8.1 Representaciones de formas en 3D, conjuntos de datos y tokenización

Representaciones de formas en 3D A diferencia de las imágenes 3D, las formas y los datos 3D no se han presentado formalmente hasta ahora en este libro. Por lo tanto, en este apartado se ofrece una breve introducción a los datos 3D. Se recomienda a los lectores que deseen obtener más información que consulten las fuentes a través de las referencias citadas.

Los datos 3D a los que nos referimos aquí se refieren principalmente a formas físicas reales y continuas en 3D que, por lo general, se digitalizan en formas discretas y computables en un determinado espacio de representación. Encontrar buenas representaciones de formas 3D constituye uno de los problemas centrales en los campos de la visión computacional y los gráficos computacionales. Los cuatro tipos de mayor uso de representaciones de formas 3D y en gran medida equivalentes matemáticamente, son: nubes de puntos, vóxeles, mallas poligonales y funciones implícitas,[14] tal y como se muestra en Figura 8.20 [15].



(a) Point Cloud (b) Voxel (c) Polygon Mesh (d) Implicit Function

Figura 8.20: **Ilustraciones de diferentes representaciones 3D del «Stanford Bunny».** (a) la nube de puntos; (b) el vóxel; (c) la malla poligonal; (d) la función implícita.

Las nubes de puntos caracterizan con precisión la forma tridimensional de un objeto mediante puntos muestreados en forma discreta en la superficie del objeto. Registran directamente las coordenadas espaciales tridimensionales (x, y, z) de los puntos de la superficie (y tal vez alguna información fotométrica adicional en esos puntos), tal como se ilustra en la Figura 8.20(a) Nube de puntos. Una nube de puntos se define como un conjunto de coordenadas discretas en el espacio euclidiano 3D:

$$P = \{\boldsymbol{p}_i \in \mathbb{R}^d \mid i = 1, 2, \ldots, N\} \tag{8.8.1}$$

[14] como la función de signo de distancia (DSF).

[15] Las imágenes de las tres primeras columnas proceden de [SPX+22] y la imagen de la última columna procede de https://medium.com/@sunil7545/implicit-vs-parametric-3d-shape-representation-9d4c01c8c60c

donde $d \geq 3$ denota la dimensionalidad de las características, lo que puede incluir (concatenaciones de) coordenadas (x_i, y_i, z_i), colores (r_i, g_i, b_i) y normales de superficie (n_{xi}, n_{yi}, n_{zi}).

Vóxeles representan unidades volumétricas en el espacio 3D, lo que constituye una extensión natural del concepto de píxeles 2D a tres dimensiones. Dicha representación se obtiene discretizando un espacio 3D en una malla estructurada. Los modelos típicos de vóxeles dividen el espacio 3D en celdas cúbicas uniformes, donde cada vóxel está indexado espacialmente y almacena atributos específicos de la región correspondiente, tales como ocupación, densidad, material o color, como se ilustra en la Fig. 8.20(b) Vóxel.

Mallas poligonales se consideran la representación más utilizada para las formas tridimensionales en los ámbitos de los gráficos por computadora y la simulación física. Aproximan las superficies continuas mediante vértices, aristas y caras poligonales discretas, siendo las mallas triangulares la variante predominante, tal como se muestra en la Fig. 8.20(c) Malla poligonal. Geométricamente, las mallas poligonales consisten en coordenadas de vértices en $\mathbb{R}^3$, acompañadas por su conectividad topológica. Formalmente, una malla M es una tupla (V, F), donde el conjunto de vértices $V = \{\boldsymbol{v}_i \in \mathbb{R}^3\}_{i=1}^n$ denota las coordenadas de los vértices y el conjunto de caras $F = \{f_j\}_{j=1}^m$ define la topología.

Funciones implícitas representan superficies mediante conjuntos de nivel de una función escalar. Este enfoque ofrece un marco unificado para la descripción matemática de superficies complejas (Fig. 8.20(d) Función implícita). La formulación implícita estándar es:

$$F : \mathbb{R}^3 \to \mathbb{R}, \quad F(x, y, z) = d, \tag{8.8.2}$$

donde (x, y, z) denotan las coordenadas de un punto de consulta, y d denota la distancia entre el punto de consulta y una superficie de interés.[16] Por lo tanto, la superficie de interés se define implícitamente como:

$$\mathcal{S} = \{(x, y, z) \in \mathbb{R}^3 \mid F(x, y, z) = 0.\} \tag{8.8.3}$$

La conversión de una representación implícita de una forma 3D a una malla poligonal suele realizarse mediante métodos de extracción de isosuperficies como el de los cubos marchantes [LC98].

Desde el punto de vista matemático, las cuatro representaciones de formas (3D) son, en gran medida, equivalentes en cuanto a su capacidad de representación. No obstante, cada formato de datos puede presentar diferentes ventajas o limitaciones computacionales. Por ello, a menudo se prefiere una representación u otra para distintas tareas. En esta sección, utilizaremos la representación tradicional de nubes de puntos y mostraremos cómo aprender la distribución de las formas 3D a partir de este tipo de datos. En la siguiente sección, mostraremos cómo utilizar una representación (generalizada) de vóxeles para aprender la distribución de las formas y las posturas 3D.

[16] Obsérvese que hemos utilizado el mismo concepto para describir el soporte de baja dimensión de una distribución en un espacio de alta dimensión, en la Sección 7.1.2.

Conjuntos de datos 3D. Existen numerosos conjuntos de datos de acceso público para cada una de las representaciones mencionadas anteriormente. Estos varían en cuanto a cantidad y calidad. En esta sección, utilizaremos ShapeNet [CFG+15] para estudiar una representación conjunta de forma, imagen y texto, así como su rendimiento en la generación. ShapeNet ofrece aproximadamente 55 000 mallas distribuidas en 55 categorías. Cada malla cuenta con una etiqueta de categoría y los textos correspondientes, tales como categorías de granularidad fina o breves descripciones proporcionadas por el creador. Para preparar los datos de tripletas (forma 3D, imagen, texto), los textos proporcionados se augmentan de dos maneras. En primer lugar, la etiqueta de forma y la descripción correspondiente se combinan en el formato «un modelo 3D de (etiqueta de forma), al estilo de (descripción)» o «un modelo 3D de (etiqueta de forma), (descripción)». Luego, inspirándonos en ULIP [XGX+23], se utilizan múltiples plantillas que contienen 65 frases predefinidas para proporcionar más información textual durante el entrenamiento. En cuanto a los datos de imagen, cada malla se renderiza en cuatro poses de cámara, lo que aumenta y mejora la diversidad del renderizado mediante la red ControlNet condicionada por profundidad [ZRA23a].

Tokenización de datos de formas 3D. Como se ha comentado anteriormente, los datos 3D se presentan en diversas formas, entre ellas nubes de puntos, vóxeles, mallas poligonales y representaciones neuronales implícitas. Fundamentalmente, las nubes de puntos, los vóxeles y las mallas sirven como aproximaciones discretas de variedades 3D continuas. En los gráficos por computadora y la ingeniería, las mallas poligonales son particularmente ubicuas porque conservan explícitamente la topología de la superficie, ofreciendo un equilibrio eficiente entre la fidelidad visual y la estructura geométrica.

Sin embargo, la aplicación directa de las modernas arquitecturas de aprendizaje profundo —en concreto, los Transformers— a estas representaciones sin procesar plantea importantes retos. A diferencia de las imágenes 2D, que poseen una estructura de cuadrícula regular que se divide fácilmente en parches, los datos 3D suelen ser esparsos, no estructurados e irregulares. Para aprovechar el poder del modelado de secuencias y los mecanismos de autoatención en 3D, debemos salvar la brecha entre los datos geométricos sin procesar y las secuencias discretas y ordenadas que requieren los transformers. Este proceso se conoce como *tokenización 3D*.

- **Tokenización de nubes de puntos.** Las nubes de puntos sin procesar suelen tokenizarse en grupos locales para capturar el contexto geométrico. Este proceso suele comenzar con el muestreo de puntos más lejanos (FPS) [ELP+97], un algoritmo iterativo que selecciona un subconjunto de puntos $\{x_1, \ldots, x_m\}$ del conjunto original $\mathcal{P}$, de tal manera que cada punto seleccionado maximice la distancia al conjunto de puntos seleccionados previamente: $x_i = \arg\max_{x \in \mathcal{P}} \min_{j=1}^{i-1} \|x - x_j\|$. Tras el FPS, los puntos se agrupan mediante k-Nearest Neighbors (k-NN) para formar clústeres locales. Cada clúster sirve como un token para gestionar la invarianza

de permutación de los puntos dentro de un clúster, por lo que se aplica una red ligera como PointNet [QSM+17]. Matemáticamente, para un clúster de puntos $\{p_1, \ldots, p_k\}$, PointNet aproxima una función simétrica f aplicando un perceptrón multi-capa (MLP) compartido h seguido de un operador de agregación simétrico (normalmente max-pooling):

$$\boldsymbol{t} = \gamma\left(\max_{i=1}^{k}\{h(p_i)\}\right) \tag{8.8.4}$$

donde $\boldsymbol{t}$ es el token de características resultante y γ es un perceptrón multi-capas (MLP) de posprocesamiento. Esto convierte una nube de puntos no estructurada en una secuencia de vectores de características.

- **Tokenización de vóxeles.** De manera análoga a la estrategia de *embedding* de parches utilizada en los Vision Transformers (ViT) para imágenes 2D (véase el Capítulo 8.2.3), las mallas de vóxeles pueden dividirse en submallas 3D que no se superponen. Dada una malla de vóxeles $V \in \mathbb{R}^{D\times H\times W\times C}$, extraemos parches de tamaño $P \times P \times P$. Cada parche se aplana en un vector $v_p \in \mathbb{R}^{P^3\cdot C}$ y se mapea mediante una proyección lineal aprendible a la dimensión de los tokens d_{model}. El resultado es una secuencia de tokens $Z \in \mathbb{R}^{L\times d_{model}}$ donde $L = \frac{DHW}{P^3}$. Aunque sencillo, este método hereda la complejidad cúbica de la resolución de vóxeles ($O(N^3)$), lo que a menudo genera cuellos de botella en la memoria para formas de alta resolución.

- **Tokenización de mallas.** La tokenización de mallas supone un desafío debido a su estructura similar a un grafo, que incluye tanto vértices como caras. A diferencia de las imágenes, las mallas carecen de un orden canónico. Para permitir el modelado secuencial, los enfoques más comunes implican una serialización explícita, en la que los elementos geométricos y topológicos sin procesar se cuantifican directamente y se ordenan en secuencias.

  Tomando como ejemplo PolyGen [NGE+20], la malla se representa como un conjunto de vértices $V = \{v_1, \ldots, v_n\}$ (donde cada $v_i \in \mathbb{R}^3$) y caras $F = \{f_1, \ldots, f_m\}$ (donde cada f_j es una tupla de índices de vértices). El modelo aprende la distribución conjunta $P(V, F)$ mediante su factorización en dos etapas secuenciales: un modelo de vértices que predice coordenadas cuantificadas y un modelo de caras que predice índices de vértices

$$P(V, F) = P(V) \cdot P(F \mid V) = \prod_{i=1}^{n} P(v_i \mid v_{<i}) \cdot \prod_{j=1}^{m} P(f_j \mid f_{<j}, V). \tag{8.8.5}$$

  Fundamentalmente, esta factorización determina dos formatos de tokens distintos correspondientes a cada etapa. Para el modelo de vértices ($P(V)$), los tokens son valores de coordenadas cuantificados; las coordenadas continuas se discretizan (por ejemplo, en enteros de 8 bits), creando una secuencia de tokens a partir de un vocabulario fijo de tamaño

256. Por el contrario, para el modelo de caras ($P(F \mid V)$), los tokens son índices de vértices (punteros). Aunque sea eficaz, operar con estas primitivas sin procesar suele dar lugar a secuencias extremadamente largas, lo que limita la escalabilidad.

Si bien estas estrategias de tokenización explícita discretizan eficazmente las variedades continuas, operan directamente en el espacio de observación de alta dimensión, lo que a menudo da lugar a secuencias excesivamente largas y a redundancia computacional. Para hacer frente a estas limitaciones de escalabilidad y lograr mayores razones de compresión, los investigadores del área se han inclinado hacia las representaciones discretas basadas en el aprendizaje. En lugar de tokenizar directamente las primitivas geométricas sin procesar, los siguientes métodos proyectan primero los datos 3D en un espacio latente compacto y de baja dimensión donde se agrega la información semántica, lo que permite desacoplar de manera efectiva la reconstrucción geométrica del modelado estructural.

Cuantización estándar de vectores latentes. Este enfoque aprende un libro de códigos discreto para representar la forma mediante la cuantización de vectores (VQ). Un método representativo es AutoSDF [MCS+22]. En este marco, una entrada de alta dimensión (por ejemplo, una malla de vóxeles X o un campo de función implícita F_θ) es comprimida por un codificador E en una malla espacial de vectores latentes de menor dimensión $Z \in \mathbb{R}^{h \times w \times d \times c}$. A continuación, cada vector $z_i \in Z$ se sustituye por su vecino más cercano de un libro de códigos entrenable $\mathcal{C} = \{c_k\}_{k=1}^{K}$:

$$q(z_i) = \arg\min_{c_k \in \mathcal{C}} |z_i - c_k|_2^2. \tag{8.8.6}$$

Esto reduce la forma a una secuencia de índices discretos del libro de códigos que conservan la estructura espacial al tiempo que reducen drásticamente la dimensionalidad. Estos índices discretos sirven como tokens para el procesamiento posterior por el Transformer. Sin embargo, este enfoque basado en una cuadrícula regular suele tratar de manera uniforme tanto el espacio ocupado como el vacío, lo que da lugar a redundancia computacional.

De la optimización estructural a la alineación semántica. Para abordar la redundancia de las mallas regulares, enfoques como 3DILG [ZNW22a] proponen mallas latentes irregulares. En lugar de una densa malla de vóxeles en la que los tokens se asignan de manera ineficiente a espacios vacíos, 3DILG define la forma como un conjunto de códigos latentes que flotan en posiciones arbitrarias y adaptativas en un espacio continuo, normalmente concentrados en la superficie de la forma. En concreto, dada una forma de entrada representada como una nube de puntos, el método primero la codifica en un conjunto de pares de características y posiciones $\mathcal{P} = \{(x_i, f_i)\}_{i=1}^{M}$, donde $x_i \in \mathbb{R}^3$ son puntos clave seleccionados mediante muestreo del punto más lejano (FPS) y $f_i \in \mathbb{R}^C$ son sus vectores de características continuas asociados, extraídos por un codificador basado en PointNet [QSM+17]. La conversión de este conjunto no estructurado en una secuencia de tokens implica tres pasos clave:

- **Cuantización de coordenadas.** En primer lugar, para un conjunto de M puntos clave seleccionados mediante el método de muestreo del punto más lejano (véase «Tokenización de nube de puntos»), las coordenadas espaciales continuas $x_i \in \mathbb{R}^3$ se cuantizan en enteros de 8 bits:

$$x_i \to (x_{i,1}, x_{i,2}, x_{i,3}), \quad \text{donde } x_{i,k} \in 0, 1, \ldots, 255. \tag{8.8.7}$$

  Este paso convierte de manera efectiva las posiciones continuas en unidades discretas y tokenizables.

- **Cuantización de características.** Al mismo tiempo, el vector de características locales asociado a cada punto se comprime en un índice discreto del libro de códigos z_i mediante cuantización vectorial (VQ):

$$z_i \in \{0, 1, \ldots, D-1\}, \tag{8.8.8}$$

  donde D representa el tamaño del libro de códigos.

- **Construcción de secuencias.** Para facilitar la generación autoregresiva con *transformers*, es necesario serializar el conjunto no estructurado de tuplas. 3DILG emplea una ordenación lexicográfica basada en las coordenadas cuantificadas. La forma geométrica final se representa como una secuencia estructurada $\mathcal{S}$ que combina explícitamente la información de las coordenadas posicionales y las características geométricas locales:

$$\mathcal{S} = \big\{(x_{0,1}, x_{0,2}, x_{0,3}, z_0), \ldots, (x_{M-1,1}, x_{M-1,2}, x_{M-1,3}, z_{M-1})\big\}. \tag{8.8.9}$$

Esta formulación híbrida reduce drásticamente la longitud de las secuencias en comparación con las rejillas densas, al tiempo que conserva los detalles de la superficie con alta fidelidad.

Aunque 3DILG optimiza la estructura espacial de los tokens, sigue siendo, en gran medida, un método de compresión geométrica. A continuación, ofrecemos una exposición detallada de Michelangelo, un trabajo que hace avanzar este paradigma al abordar la brecha semántica. Codifica formas tridimensionales en una secuencia de tokens latentes que no solo son geométricamente compactos, sino que también están alineados explícitamente con el espacio de características de los modelos de visión-lenguaje bidimensionales (p. ej., CLIP) [RKH+21c]), lo que facilita la generación condicional de alta calidad.

8.8.2 Tarea

Nuestro objetivo aquí es aprender un modelo generativo de formas 3D condicional capaz de generar formas 3D a partir de la distribución aprendida, sujeto a una condición determinada, como una imagen o un texto, tal y como se ilustra en la Figura 8.21.

Siguiendo el modelo de difusión latente [RBL+22], para aprovechar al máximo las capacidades del modelo generativo, primero se entrena una VAE para comprimir muestras de datos 3D en un espacio latente. En general, una

buena representación latente debe poseer dos características clave: debe ser altamente compresible y, al mismo tiempo, permitir una reconstrucción fiel de las muestras de datos 3D, además de facilitar el entrenamiento del modelo generativo. Sin embargo, en comparación con los datos de imágenes 2D, los datos 3D siguen siendo escasos. [17] En consecuencia, aprender una representación latente efectiva para datos 3D supone un gran desafío. La Figura 8.21 muestra el diagrama general del marco propuesto por el proyecto Michelangelo. A continuación explicaremos los detalles de implementación de cada componente.

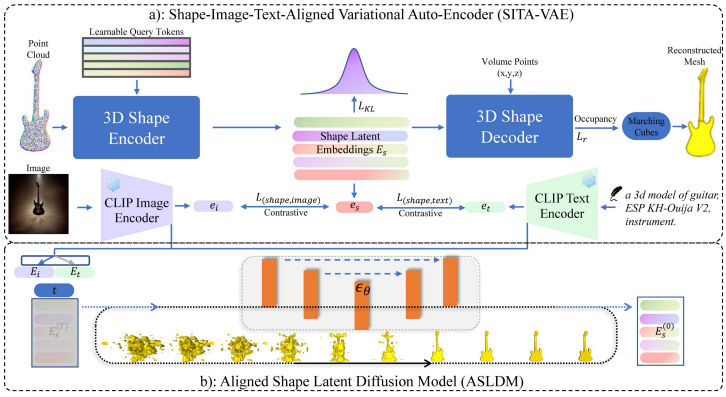


Figura 8.21: **Proceso de alineación previa a la generación**. El enfoque incluye dos modelos: el codificador automático variacional con alineación de forma, imagen y texto (SITA-VAE) y el modelo de difusión latente de formas alineadas (ASLDM). El SITA-VAE consta de cuatro módulos: un codificador visual, un codificador de texto, un codificador de formas 3D y un decodificador de formas 3D. Los codificadores codifican los pares de entrada en un espacio alineado, y el decodificador de formas 3D reconstruye formas 3D a partir de los *embeddings* del espacio alineado. El ASLDM mapea la condición de imagen o de texto al espacio latente de formas alineadas para muestrear un *embedding* de forma 3D de alta calidad, que posteriormente el decodificador de formas 3D reconstruye en formas 3D de alta fidelidad.

8.8.3 Arquitectura y función objetivo

Autocodificador variacional alineado en forma, imagen y texto. Inspirado en CLIP [RKH+21c], el enfoque adoptado por el proyecto Michelangelo inyecta información semántica en el espacio latente mediante

[17] Esta situación persiste incluso tras el lanzamiento de Objaverse-XL [DLW+23].

una pérdida contrastiva para mitigar los problemas observados al entrenar VAE exclusivamente a partir de datos 3D, lo que da lugar al Autocodificador Variacional Alineado de Forma, Imagen y Texto (SITA-VAE).

El SITA-VAE consta de cuatro componentes: un codificador de imágenes preentrenado y fijo CLIP $\mathcal{E}_\text{i}$ y un codificador de texto CLIP $\mathcal{E}_\text{t}$, un codificador de formas 3D entrenable $\mathcal{E}_\text{s}$ y un decodificador de campo neuronal $\mathcal{D}_\text{s}$. El codificador visual y el codificador de texto de CLIP toman como entrada imágenes bidimensionales $\boldsymbol{I} \in \mathbb{R}^{H\times W\times 3}$ y textos tokenizados $\boldsymbol{T} \in \mathbb{R}^{d_\text{t}\times L_\text{t}}$, y generan tokens de imagen $\boldsymbol{E}_\text{i} \in \mathbb{R}^{d\times(1+L_\text{i})}$ y tokens de texto $\boldsymbol{E}_\text{t} \in \mathbb{R}^{d\times L_\text{t}}$, donde $(1 + L_\text{i})$ y L_t son la longitud de la secuencia de los tokens de imagen $\boldsymbol{E}_\text{i}$ y de texto $\boldsymbol{E}_\text{t}$. El enfoque aprovecha el codificador visual y el codificador de texto preentrenados de CLIP. Estos dos codificadores se han entrenado con pares imagen-texto a gran escala y son lo suficientemente robustos como para capturar un espacio de visión-lenguaje bien alineado, lo que enriquece la semántica de la representación de la forma 3D tras la alineación multimodal mediante aprendizaje contrastivo.

El «codificador de formas 3D» tiene como objetivo extraer potentes representaciones para caracterizar cada forma 3D de manera eficaz. Para lograrlo, primero se muestrean nubes de puntos $\boldsymbol{P} \in \mathbb{R}^{(3+C)\times N}$ de la superficie de las formas 3D, donde N representa el número de puntos, y C denota características adicionales de los puntos, como la normal o el color.

Para capturar mejor los detalles geométricos de alta frecuencia, utilizamos la codificación posicional de Fourier para mapear las coordenadas espaciales de baja dimensión $\boldsymbol{v} \in \mathbb{R}^3$ de cada punto a un espacio de frecuencias de mayor dimensión. Esta codificación $\gamma(\boldsymbol{v})$ se define como una secuencia de funciones sinusoidales con frecuencias que aumentan exponencialmente:

$$\gamma(\boldsymbol{v}) = [\sin(2^0\pi\boldsymbol{v}), \cos(2^0\pi\boldsymbol{v}), \ldots, \sin(2^{L-1}\pi\boldsymbol{v}), \cos(2^{L-1}\pi\boldsymbol{v})],$$

donde L es el número de bandas de frecuencia. Para construir la representación de entrada final, descomponemos explícitamente la nube de puntos de entrada $\boldsymbol{P} \in \mathbb{R}^{(3+C)\times N}$ en coordenadas espaciales $\boldsymbol{v}$ y características auxiliares $\boldsymbol{f} \in \mathbb{R}^{C\times N}$. Los *embeddings* posicionales de alta dimensión $\gamma(\boldsymbol{v})$ entonces se concatenan con las características conservadas $\boldsymbol{f}$ y se proyectan a través de una capa lineal aprendible. Formalmente, la entrada del codificador de formas 3D $\boldsymbol{X} \in \mathbb{R}^{d\times N}$ se obtiene como:

$$\boldsymbol{X} = \text{Linear}([\gamma(\boldsymbol{v}); \boldsymbol{f}])$$

donde $[\cdot;\cdot]$ denota la operación de concatenación a lo largo de la dimensión de la característica.

Para procesar esta secuencia de características densa $\boldsymbol{X}$ de manera eficiente, el codificador adopta una arquitectura basada en Perceiver [JGB+21]. A diferencia de los transformers estándar que escalan cuadráticamente con la longitud de la entrada, el Perceiver emplea un mecanismo de cuello de botella latente para desacoplar la complejidad del procesamiento del tamaño de la entrada. En lugar de tokenizar los puntos de entrada directamente

para la autoatención, el modelo utiliza un conjunto fijo de consultas latentes pequeñas y aprendibles $\boldsymbol{Q} \in \mathbb{R}^{(1+L_{\mathrm{s}})\times d}$ donde $1 + L_{\mathrm{s}}$ es el número de tokens de consulta. Desde el punto de vista mecánico, una capa de atención cruzada proyecta el espacio de entrada de alta dimensión en este espacio latente compacto. Formalmente, siguiendo la formulación del Capítulo 8.7.2, sean $\boldsymbol{U}_Q^{(temp)}, \boldsymbol{U}_K^{(temp)}, \boldsymbol{U}_V^{(temp)}$ las matrices de proyección aprendibles. La representación latente inicial $\boldsymbol{Z}_0$ se calcula como:

$$\boldsymbol{Z}_0 = \text{Atención cruzada}(\boldsymbol{Q}, \boldsymbol{X}) = (\boldsymbol{U}_V^{(temp)}\boldsymbol{X})\cdot\text{softmax}\left(\frac{(\boldsymbol{U}_K^{(temp)}\boldsymbol{X})^\top(\boldsymbol{U}_Q^{(temp)}\boldsymbol{Q})}{\sqrt{d_k}}\right).$$

Aquí, la nube de puntos de entrada $\boldsymbol{X}$ sirve como claves y valores, mientras que las consultas entrenables $\boldsymbol{Q}$ actúan como consultas. Esta operación comprime la entrada de longitud variable N en una secuencia latente de longitud fija $\boldsymbol{Z}_0 \in \mathbb{R}^{(1+L_{\mathrm{s}})\times d}$.

En el contexto de Michelangelo, estas consultas están estructuradas para capturar la semántica jerárquica: consisten en un token de cabecera global $\boldsymbol{Q}_g \in \mathbb{R}^{1\times d}$ con semántica de alto nivel y L_{s} tokens locales $\boldsymbol{Q}_l \in \mathbb{R}^{L_{\mathrm{s}}\times d}$ que contienen información de estructura geométrica de bajo nivel. A continuación, se utilizan varios bloques de autoatención para mejorar iterativamente la representación de características y obtener los *embeddings* de forma finales, $\boldsymbol{E}_{\mathrm{s}} \in \mathbb{R}^{(1+L_{\mathrm{s}})\times d}$.

La alineación entre formas 3D, imágenes y textos desempeña un papel crucial en SITA-VAE y en los modelos generativos condicionales. Dado que los datos 3D son un orden de magnitud más pequeños que los datos de imágenes y texto, para aprender una representación mejor alineada entre formas 3D, imágenes y textos, el codificador de formas 3D está restringido a estar cerca de un espacio visión-lenguaje prealineado, el cual está preentrenado en pares de imagen y texto a gran escala con representaciones ricas de imagen y texto al aprovechar la estrategia de aprendizaje contrastivo. Consideremos un triplete de entrada compuesto por formas 3D $\boldsymbol{X}$, imágenes $\boldsymbol{I}$ y textos tokenizados $\boldsymbol{T}$. Los codificadores de tripletes generan el *embedding* de forma correspondiente $\boldsymbol{e}_{\mathrm{s}}$, el *embedding* de imagen $\boldsymbol{e}_{\mathrm{i}}$ y el *embedding* de texto $\boldsymbol{e}_{\mathrm{t}}$ mediante la proyección de los tokens de forma extraídos $\boldsymbol{E}_{\mathrm{s}}$, los tokens de imagen $\boldsymbol{E}_{\mathrm{i}}$ y los tokens de texto $\boldsymbol{E}_{\mathrm{t}}$ como tres vectores de la misma dimensión, lo cual se expresa como: $\boldsymbol{e}_{\mathrm{s}} = \mathcal{F}_{\mathrm{s}}(\boldsymbol{E}_{\mathrm{s}}), \boldsymbol{e}_{\mathrm{i}} = \mathcal{F}_{\mathrm{i}}(\boldsymbol{E}_{\mathrm{i}})$, and $\boldsymbol{e}_{\mathrm{t}} = \mathcal{F}_{\mathrm{t}}(\boldsymbol{E}_{\mathrm{t}})$, donde $\mathcal{F}_{\mathrm{s}}$ es un proyector de *embedding* de formas aprendible, mientras que el proyector de *embedding* de imágenes $\mathcal{F}_{\mathrm{i}}$ y el proyector de *embedding* de texto $\mathcal{F}_{\mathrm{t}}$ están preentrenados y se mantienen fijos durante el entrenamiento y la inferencia. La pérdida contrastiva es:

$$\mathcal{L}_{(\mathrm{s,i})} = -\frac{1}{2}\sum_{(j,k)}\left(\log\frac{\exp(\boldsymbol{e}_{\mathrm{s}}^j\boldsymbol{e}_{\mathrm{i}}^k)}{\sum_l \exp(\boldsymbol{e}_{\mathrm{s}}^j\boldsymbol{e}_{\mathrm{i}}^l)} + \log\frac{\exp(\boldsymbol{e}_{\mathrm{s}}^j\boldsymbol{e}_{\mathrm{i}}^k)}{\sum_l \exp(\boldsymbol{e}_{\mathrm{s}}^l\boldsymbol{e}_{\mathrm{i}}^k)}\right), \tag{8.8.10}$$

$$\mathcal{L}_{(\mathrm{s,t})} = -\frac{1}{2}\sum_{(j,k)}\left(\log\frac{\exp(\boldsymbol{e}_{\mathrm{s}}^{j}\boldsymbol{e}_{\mathrm{t}}^{k})}{\sum_{l}\exp(\boldsymbol{e}_{\mathrm{s}}^{j}\boldsymbol{e}_{\mathrm{t}}^{l})} + \log\frac{\exp(\boldsymbol{e}_{\mathrm{s}}^{j}\boldsymbol{e}_{\mathrm{t}}^{k})}{\sum_{l}\exp(\boldsymbol{e}_{\mathrm{s}}^{l}\boldsymbol{e}_{\mathrm{t}}^{k})}\right), \tag{8.8.11}$$

donde (j,k) indica el par positivo en los lotes de entrenamiento y dado que se utilizan los codificadores preentrenados de CLIP, el modelo está libre de la restricción $\mathcal{L}_{(\mathrm{i,t})}$.

El decodificador de formas 3D, $\mathcal{D}_{\mathrm{s}}$, toma como entradas los *embeddings* de formas $\boldsymbol{E}_{\mathrm{s}}$ para reconstruir el campo neuronal 3D con alta calidad. Se utiliza la pérdida de divergencia KL $\mathcal{L}_{KL}$ para facilitar el proceso generativo y mantener el espacio latente como una distribución continua. Además, se utiliza una capa de proyección para comprimir la variable latente de la dimensión d a dimensiones inferiores d_0 y obtener así una representación compacta. A continuación, se emplea otra capa de proyección para transformar la variable latente muestreada de la dimensión d_0 de vuelta a la dimensión d de mayor cardinalidad con el fin de reconstruir los campos neuronales correspondientes de las formas 3D. Al igual que el codificador, el modelo decodificador también se basa en un transformer con el mecanismo de atención cruzada. Dado un punto 3D de consulta $\boldsymbol{x} \in \mathbb{R}^3$ en el campo y sus correspondientes *embeddings* latentes de forma $\boldsymbol{E}_{\mathrm{s}}$, el decodificador calcula la atención cruzada de forma iterativa para predecir la ocupación del punto de consulta $\mathcal{O}(\boldsymbol{x})$. La pérdida de entrenamiento se expresa como:

$$\mathcal{L}_r = \mathbb{E}_{\boldsymbol{x}\in\mathbb{R}^3}[\mathrm{BCE}(\mathcal{D}(\boldsymbol{x} \mid \boldsymbol{E}_{\mathrm{s}}), \mathcal{O}(\boldsymbol{x}))], \tag{8.8.12}$$

donde BCE es la pérdida de entropía cruzada binaria, y la pérdida total para el entrenamiento del Autocodificador Variacional Alineado de Forma, Imagen y Texto (SITA) se expresa como:

$$\mathcal{L}_{SITA} = \lambda_c(\mathcal{L}_{(\mathrm{s,i})} + \mathcal{L}_{(\mathrm{s,t})}) + \mathcal{L}_r + \lambda_{KL}\mathcal{L}_{KL}. \tag{8.8.13}$$

Modelo de difusión latente de formas alineadas. Tras entrenar el SITA-VAE, se obtiene un espacio de alineación entre formas 3D, imágenes y textos, así como un codificador y decodificador de formas 3D que comprime la forma 3D en *embeddings* latentes de formas de baja dimensión y reconstruye dichas representaciones latentes en un campo neuronal con alta calidad. Aprovechando el éxito del modelo de difusión latente (LDM) [RBL+22] en la generación de imágenes a partir de texto, que logra un equilibrio entre la carga computacional y la calidad de la generación, se propone un modelo de difusión latente de formas en el espacio alineado para aprender un mapeo probabilístico más preciso de imágenes 2D o textos a *embeddings* latentes de formas 3D. Al hacer uso del espacio de alineación y el modelo de difusión latente de formas, es posible generar formas 3D de alta calidad que se ajustan mejor a las entradas condicionales visuales o textuales.

El modelo de difusión latente de formas alineadas (ASLDM) se basa en un transformer similar a UNet [BNX+23], con el objetivo de ajustar

una distribución de los *embeddings* latentes de formas, acompañado de un autocodificador para codificar muestras de datos en el espacio latente y reconstruir las muestras de datos a partir del latente muestreado. Al aprender en el espacio latente, el modelo de difusión latente es computacionalmente eficiente, y aprovechar una representación tan compacta permite al modelo ajustar la distribución objetivo más rápidamente. En concreto, el modelo ϵ_θ se centra en generar *embeddings* latentes de formas $\boldsymbol{E}_\mathrm{s}$ condicionadas a $\boldsymbol{C}$, que está representada por el codificador de imágenes o texto CLIP. Siguiendo el LDM [RBL+22], el objetivo es

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{E}_\mathrm{s},\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\boldsymbol{I}),t}\left[\|\boldsymbol{\epsilon} - \epsilon_\theta(\boldsymbol{E}_\mathrm{s}^{(t)}, \boldsymbol{C}, t)\|_2^2\right], \tag{8.8.14}$$

donde t se muestrea uniformemente de $\{1, ..., T\}$ y $\boldsymbol{E}_\mathrm{s}^{(t)}$ es una versión ruidosa de $\boldsymbol{E}_\mathrm{s}^{(0)}$. Durante la inferencia, tras muestrear ruido gaussiano, el modelo elimina gradualmente el ruido de la señal hasta alcanzar $\boldsymbol{E}_\mathrm{s}^{(0)}$. Siguiendo la guía libre de clasificador (CFG) [HS21], el modelo de difusión latente condicional se entrena con guía sin clasificador. En la fase de entrenamiento, la condición $\boldsymbol{C}$ se convierte aleatoriamente en un conjunto vacío $\emptyset$ con una probabilidad fija del 10%. A continuación, el muestreo se realiza mediante la combinación lineal de muestras condicionales e incondicionales:

$$\epsilon_\theta(\boldsymbol{E}_\mathrm{s}^{(t)}, \boldsymbol{C}, t) = \epsilon_\theta(\boldsymbol{E}_\mathrm{s}^{(t)}, \emptyset, t) + \lambda(\epsilon_\theta(\boldsymbol{E}_\mathrm{s}^{(t)}, \boldsymbol{C}, t) - \epsilon_\theta(\boldsymbol{E}_\mathrm{s}^{(t)}, \emptyset, t)), \tag{8.8.15}$$

donde λ es la escala de guiado para equilibrar la fidelidad y la diversidad del muestreo.

8.8.4 Experimentos y análisis

Métricas. La intersección sobre la unión (IoU) se utiliza para reflejar la precisión de las reconstrucciones. Se proponen dos nuevas métricas para evaluar los métodos de generación de formas 3D. La primera es una puntuación de forma-imagen (SI-S), que utiliza un codificador de formas 3D y un codificador de imágenes para extraer los correspondientes *embeddings* de forma e imagen y calcular la similitud coseno de estas dos modalidades. Otra es la puntuación de forma-texto (ST-S), que calcula la similitud entre la forma 3D generada y el texto condicional en el espacio alineado de *embeddings* de formas y *embeddings* de texto. Ambas métricas evalúan la similitud entre los resultados y sus condiciones correspondientes. Además, tanto el modelo preentrenado ULIP [XGX+23] como SITA se utilizan para calcular SI-S y ST-S, en términos de SI-S (ULIP), ST-S (ULIP), SI-S (SITA) y ST-S (SITA), respectivamente. Asimismo, se adoptan las métricas P-IS y P-FID presentadas en Point-E [NJD+22], utilizando un modelo preentrenado PointNet++ [QYS+17] para calcular la puntuación Inception equivalente para nube de puntos [SGZ+16] y el FID [HRU+17a] para evaluar la diversidad y la calidad de las formas 3D generadas.

Líneas base. Comparamos el enfoque de Michelangelo con otros métodos, entre ellos OccNet [MON+19], ConvOcc [PNM+20], IF-Net [CAP20],

3DILG [ZNW22b] y Learnable Query Version of 3DS2V [ZTN+23], en tareas de reconstrucción para validar la capacidad del modelo para recuperar un campo neuronal a partir de *embeddings* de formas en el conjunto de datos ShapeNet [CFG+15]. Para la etapa de generación condicional, se han elegido como modelos de referencia dos potentes métodos recientes de generación 3D: 3DILG y 3DS2V. En primer lugar, sus módulos de representación de formas se afinan utilizando un conjunto de datos mixto compuesto por ShapeNet y 3D Cartoon Monster. A continuación, los modelos generativos condicionales de texto e imagen de 3DILG y 3DS2V se reentrenan siguiendo los mismos protocolos.

Comparación numérica. Los resultados numéricos se presentan en la Tabla 8.9 y la Tabla 8.10. La Tabla 8.9 muestra que Michelangelo tiene el mejor rendimiento de reconstrucción en el conjunto de sus 55 categorías, superando al resto. Los resultados de las categorías seleccionadas demuestran además que el modelo puede reconstruir fielmente formas 3D en cada una de las 55 categorías. La Tabla 8.10 presenta los resultados numéricos para la generación condicional de formas 3D. Michelangelo obtiene los mejores resultados en todos los indicadores SI-S y ST-S, lo que indica que es capaz de mapear la información de la imagen o del texto a la información de forma 3D correspondiente para generar resultados de alta fidelidad. Además, el P-FID demuestra que el modelo puede producir tokens de formas de alta calidad para generar formas 3D realistas, y el P-IS indica la diversidad de las muestras. En concreto, las cuatro columnas de la izquierda muestran que Michelangelo supera los valores de referencia en la generación condicionada por imágenes, lo que demuestra que es capaz de mapear mejor la información visual a formas 3D. Las cuatro columnas de la derecha validan la calidad generativa de la generación condicionada por texto. Dado que el lenguaje natural, en comparación con las imágenes 2D, suele proporcionar información limitada y abstracta, entrenar un modelo para mapear la información textual a formas 3D supone un reto. Sin embargo, gracias al entrenamiento en el espacio latente alineado, Michelangelo mejora significativamente la generación condicionada por texto, como se muestra en las columnas de la derecha de la Tabla 8.10, lo que refleja que el modelo mapea eficazmente la información del lenguaje natural a formas 3D y genera resultados diversos y de alta calidad.

	General	Seleccionadas	Mesa	Silla	Avión	Coche	Rifle	Lámpara
OccNet [MON+19]	0.825	0.81	0.823	0.803	0.835	0.911	0.755	0.735
ConvOccNet [PNM+20]	0.888	0.873	0.847	0.856	0.881	0.921	0.871	0.859
IF-Net [CAP20]	0.934	0.924	0.901	0.927	0.937	0.952	0.914	0.914
3DILG [ZNW22b]	0.950	0.948	0.963	0.95	0.952	0.961	0.938	0.926
3DS2V(LQ) [ZTN+23]	0.955	0.955	**0.965**	0.957	0.962	0.966	0.947	0.931
Ours	**0.966**	**0.964**	**0.965**	**0.966**	**0.966**	**0.969**	**0.967**	**0.95**

Tabla 8.9: **Resultados numéricos para la comparación de reconstrucciones en IoU (↑, cuanto mayor sea el valor, mejor).** Los resultados muestran que Michelangelo obtiene los mejores resultados en 55 categorías. Los resultados de algunas categorías seleccionadas demuestran además que el modelo puede reconstruir cada categoría con fidelidad.

	Image-Conditioned				Text-Conditioned			
	SI-S (ULIP)↑	SI-S (SITA)↑	P-FID↓	P-IS↑	ST-S (ULIP)↑	ST-S (SITA)↑	P-FID↓	P-IS↑
3DILG	9.134	11.703	4.592	12.247	10.293	6.878	10.283	12.921
3DS2V	13.289	15.156	2.921	12.92	12.934	9.833	5.704	13.149
Ours	**13.818**	**15.206**	**1.586**	**13.233**	**16.647**	**13.128**	**2.075**	**13.558**

Tabla 8.10: **Resultados numéricos de la comparación de la generación condicional**. Los resultados muestran que Michelangelo alcanza el mejor rendimiento generativo. Los índices SI-S y ST-S indican que el modelo genera resultados de alta fidelidad al mapear eficientemente la información de la condición a sus formas 3D correspondientes. Además, el P-FID refleja que el modelo genera las formas 3D más realistas, y el P-IS indica que las muestras generadas son diversas. ↑ indica que un valor mayor es mejor y ↓ lo contrario.

Comparación visual. Las comparaciones visuales de las generaciones de formas 3D condicionadas por imágenes o texto se ilustran en la Figura 8.22 y la Figura 8.23. La Figura 8.22 muestra que 3DILG [ZNW22b] presta más atención a la forma global en el proceso de generación autoregresiva, donde sus resultados no capturan los detalles finos de las formas 3D. Si bien 3DS2V [ZTN+23] genera formas 3D con mayor detalle, los resultados presentan superficies discontinuas o con ruido. Además, ambos métodos tienen dificultades para generar una forma completa cuando las condiciones dadas corresponden a un objeto complejo, una máquina de precisión o un monstruo poco común. La Figura 8.23 muestra la comparación visual de la generación condicionada por texto. En las filas de la mitad superior, se muestran los resultados obtenidos con conceptos simples y abstractos, mientras que en las filas de la mitad inferior, se muestran los resultados obtenidos con textos detallados, como descripciones de partes determinadas de la forma objetivo. De manera similar a la observación anterior, 3DILG [ZNW22b] genera superficies de forma excesivamente suaves con menos detalles, y 3DS2V [ZTN+23] produce menos detalles en superficies de objetos discontinuas. Por el contrario, Michelangelo genera formas correctas que se ajustan a los conceptos proporcionados o descripciones detalladas, con detalles delicados sobre superficies lisas.

La eficacia del entrenamiento del modelo generativo en el espacio alineado. Se realiza una comparación visual para evaluar, mediante ablación, la eficacia del entrenamiento del modelo generativo en el espacio alineado, tal y como se ilustra en la Figura 8.24. Las muestras superiores se generan a partir del modelo generativo entrenado en el espacio alineado, mientras que las muestras inferiores se generan a partir del modelo generativo entrenado sin utilizar el espacio alineado. Los resultados demuestran que las muestras superiores se ajustan al texto dado, mientras que las inferiores no, lo que indica que entrenar el modelo generativo en el espacio alineado da lugar a muestras de alta fidelidad.

La eficacia de los modelos de visión y lenguaje. Además del conocido modelo de visión y lenguaje (VLM) CLIP [RKH+21c], se introduce otro modelo de visión-lenguaje (VLM) SLIP [MKW+22] para entrenar el SITA-VAE con el fin de realizar una comparación exhaustiva. En primer lugar, se evalúa el

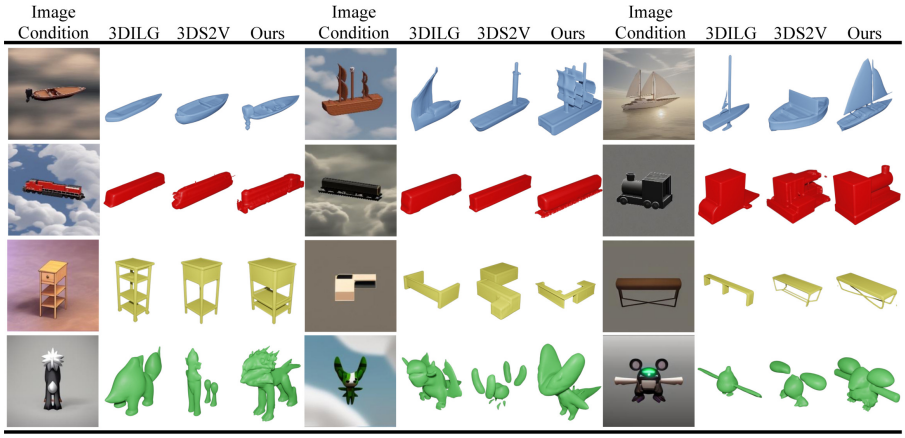


Figura 8.22: **Resultados visuales de la comparación de la generación condicionada por imágenes**. La figura muestra que 3DILG [ZNW22b] genera superficies excesivamente lisas y carece de detalles de formas, mientras que 3DS2V [ZTN+23] genera poco detalle con superficies ruidosas y discontinuas de las formas. A diferencia de las líneas base, Michelangelo produce superficies suaves y reproduce los detalles de las formas. Por favor, amplíe la imagen para ver más detalles visuales.

impacto del modelo de visión-lenguaje en la capacidad de reconstrucción de SITA-VAE, y los resultados se muestran en la Figura 8.25. Los resultados muestran que el modelo basado en CLIP alcanza el mejor rendimiento. A continuación, se evalúa el impacto del modelo de visión-lenguaje en la capacidad de alinear el espacio multimodal. Se seleccionan tareas de clasificación estándar y *zero-shot* para reflejar el impacto del modelo de visión-lenguaje. Cabe señalar que la clasificación se lleva a cabo mediante una operación de coincidencia de características, en la que se proporcionan múltiples formas 3D y frases al SITA-VAE; este devuelve la similitud entre las formas 3D y cada frase como resultados de clasificación, lo que indica que cuanto más alineado esté el espacio multimodal, mayor será la precisión de la clasificación. Los resultados muestran que el modelo basado en CLIP alcanza el mejor rendimiento.

El impacto de los *embeddings* de consulta aprendibles. Se lleva a cabo un estudio de ablación de los *embeddings* de consultas aprendibles utilizando los mismos experimentos descritos anteriormente y los resultados muestran que el uso de 512 *embeddings* de consultas aprendibles ofrece el mejor rendimiento en las tareas de reconstrucción y clasificación.

Análisis del vecino más cercano. Para verificar si el modelo aprende a generar resultados basándose en las condiciones dadas o si memoriza la partición de entrenamiento del conjunto de datos, se recorre el conjunto de entrenamiento para recuperar los vecinos más cercanos de las muestras generadas, tal y como

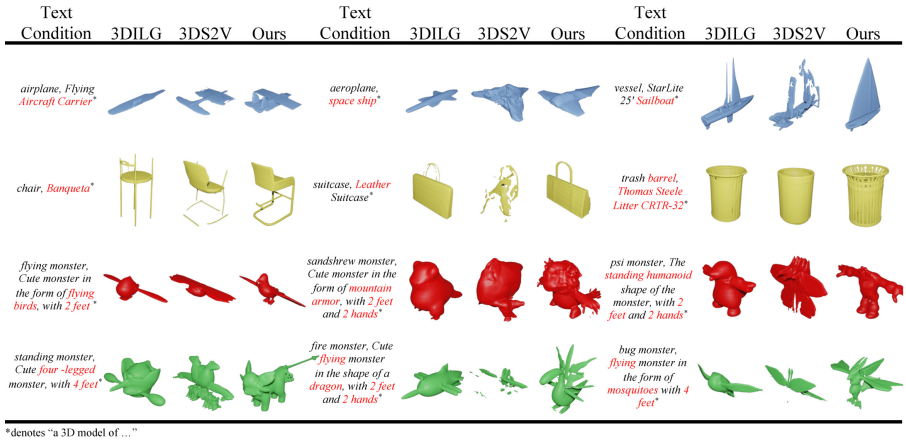


Figura 8.23: **Resultados visuales de la comparación de generación condicionada por texto**. En las dos primeras filas, los modelos se prueban con textos abstractos, y el resultado muestra que solo Michelangelo genera una forma 3D que se ajusta al texto de referencia, con una superficie lisa y detalles precisos. Las dos últimas filas muestran los resultados obtenidos con textos que contienen descripciones detalladas, lo que demuestra aún más que Michelangelo es capaz de captar tanto la información condicional global como la información local para generar formas 3D de alta fidelidad. Las palabras clave aparecen resaltadas en rojo; amplíe la imagen para ver más detalles.

se ilustra en la Figura 8.26. En concreto, se muestran los tres vecinos más cercanos de cada muestra a efectos comparativos. Según la visualización, las formas 3D generadas difieren de todos los vecinos más cercanos recuperados, lo que demuestra que el modelo crea nuevas formas 3D en lugar de sufrir un sobreajuste a los conjuntos de entrenamiento.

Discusión. Aunque el proyecto Michelangelo ha demostrado que podemos lograr resultados excelentes en la generación de objetos 3D, aún presenta algunas limitaciones. En primer lugar, requiere una gran cantidad de muestras de formas 3D de referencia para el entrenamiento y el aprendizaje de la distribución, mientras que la obtención de esos datos de formas 3D es mucho más costosa y los conjuntos de datos disponibles suelen ser, por lo general, varios órdenes de magnitud más pequeños que los de las imágenes 2D. Como hemos mencionado en la Sección 7.5.2 al final del capítulo anterior, nuestro cerebro es capaz de aprender un modelo visual completo en 3D (o 4D) partiendo únicamente de imágenes en 2D, ya que este tipo de datos son mucho más naturales y económicos de obtener. Por lo tanto, en última instancia, para ampliar verdaderamente el modelo 3D generativo, debemos desarrollar métodos más avanzados que nos permitan aprender la representación de la forma (dentro de un espacio 3D alineado entre forma, imagen y texto) únicamente a partir de imágenes 2D

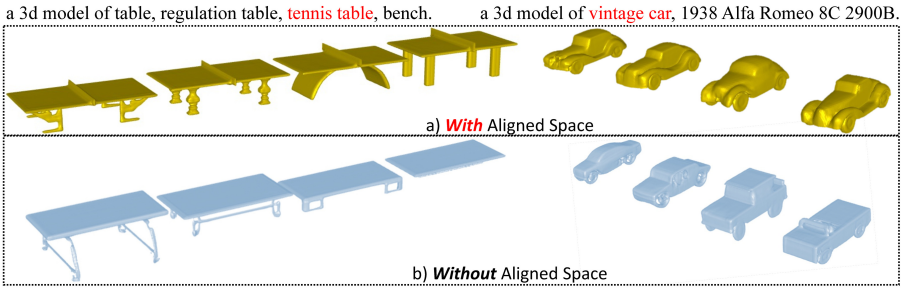


Figura 8.24: **Estudio de ablación sobre la eficacia del entrenamiento del modelo generativo en el espacio alineado**. Esta figura ilustra comparaciones visuales para los estudios de ablación sobre la eficacia del entrenamiento del modelo generativo en el espacio alineado. En comparación con las muestras inferiores basadas en los textos condicionales, las muestras superiores están más cerca de las condiciones desde el punto de vista semántico, lo que indica la eficacia del entrenamiento del modelo generativo en el espacio alineado.

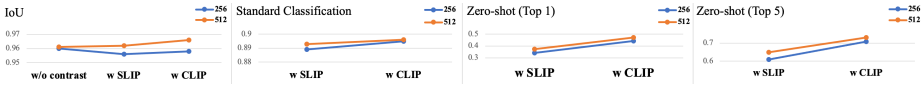


Figura 8.25: **Estudio de ablación sobre la eficacia de los modelos de visión-lenguaje y el impacto de los *embeddings* de consulta aprendibles**. Esta figura muestra el estudio de ablación sobre la eficacia del modelo de visión-lenguaje y el impacto de los *embeddings* de consulta aprendibles. Según la tabla, el modelo con CLIP y 512 *embeddings* de consulta aprendibles logra el mejor rendimiento de reconstrucción y clasificación, lo que indica su capacidad para recuperar formas 3D y alinear el espacio multimodal.

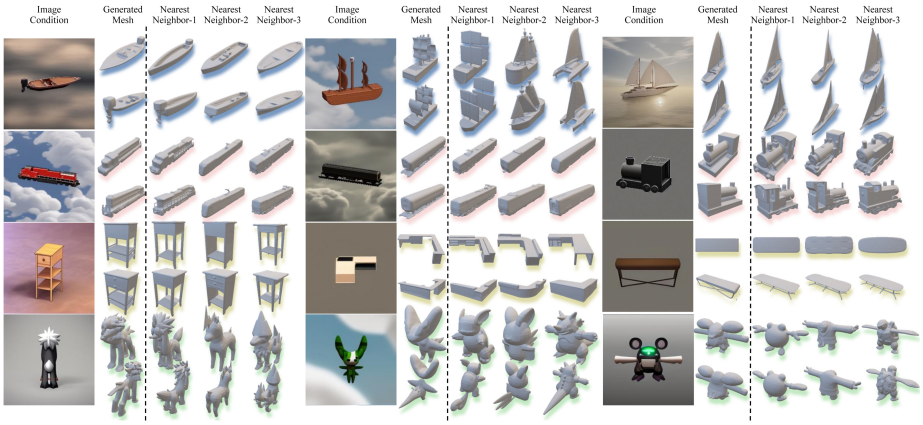


Figura 8.26: **Análisis de vecinos más cercanos**. Recorremos todo el conjunto de entrenamiento para encontrar los tres vecinos más cercanos de las formas 3D generadas, y los resultados revelan que nuestro modelo es capaz de producir formas 3D nuevas a partir de imágenes dadas, en lugar de memorizar formas específicas.

(multi-vista), por ejemplo, mediante renderización diferenciable. Además, dado que representamos cada forma 3D como un campo de ocupación, es necesario convertir la malla 3D en una malla cerrada, lo que inevitablemente degradará la calidad original de la malla 3D.

8.9 Reconstrucción de forma y pose en 3D basada en la generación

La razón fundamental por la que desarrollamos el sentido de la vista es para aprender a percibir nuestro entorno tridimensional (3D). Nuestro cerebro ha dedicado una parte importante de sí mismo, *la corteza visual*, a realizar esta tarea. Esto nos permite reconstruir de manera precisa y eficiente las formas de los objetos 3D y sus relaciones espaciales[18] a partir de las imágenes 2D que perciben nuestros ojos. Existe una larga y rica tradición dedicada a comprender matemáticamente cómo, en principio, podemos reconstruir completamente la geometría tridimensional a partir de múltiples imágenes 2D. Los lectores interesados pueden consultar el libro de texto [MKS+04] para obtener una explicación detallada de este importante tema.

No obstante, tal y como hemos hecho alusión en la Sección 7.5.2 del anterior Capítulo 7, idealmente la reconstrucción 3D debería llevarse a cabo de manera bayesiana, al igual que lo hace nuestro cerebro. Nuestra percepción del entorno tridimensional parece ser tan natural, principalmente porque reconstruimos

[18] es decir, por ejemplo, las poses relativas con respecto a nuestro punto de vista o entre los propios objetos

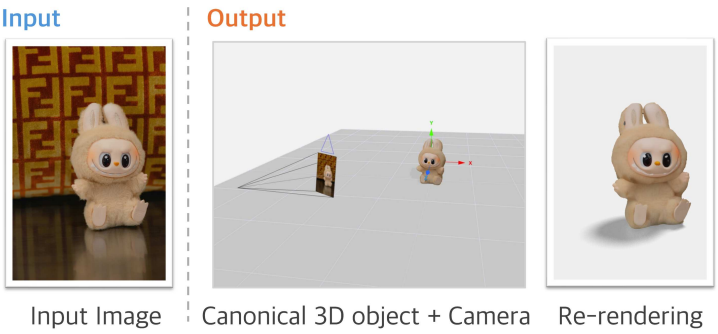


Figura 8.27: **Tarea: inferir la forma 3D de un objeto con una pose canónica respecto a la pose de la cámara a partir de una sola imagen.** Izquierda: Imagen de entrada. Centro: El resultado recupera un modelo 3D canónico del objeto (con propiedades intrínsecas como la forma y la textura) junto con la postura exacta de la cámara (propiedad extrínseca) que genera la entrada. Derecha: La re-renderización del modelo 3D a partir de la pose estimada reproduce fielmente la imagen de entrada.

una escena tridimensional y reconocemos objetos tridimensionales basándonos en una rica «memoria visual». Más precisamente, gracias a nuestros años de experiencia acumulada, hemos aprendido a comprender la distribución de nuestro entorno 3D y de los objetos que contiene. Dicha distribución es de relativamente baja dimensión y bastante estructurada, por lo que podemos completar de forma correcta y eficiente la geometría 3D de una escena y de los objetos que la componen, incluso si solo hemos visto una pequeña parte de ella, por ejemplo, desde una única perspectiva, tal y como ilustra el ejemplo de la Figura 8.27. Desde un punto de vista conceptual, esta capacidad de «completar» la información que falta es muy similar a la forma en que las imágenes naturales pueden completarse a partir de una pequeña fracción de parches, tal como hemos visto en la Sección 8.5.

Como distribución, nuestra memoria visual tridimensional no solo es de baja dimensión, sino que también está muy estructurada. Por ejemplo, parece que nuestro cerebro almacena la semántica (la identidad de los objetos) de una escena en la *corteza temporal inferior (corteza IT) y la información espacial tridimensional de la escena en el* hipocampo. Nuestro cerebro no solo reconstruye la geometría tridimensional de la escena,[19] sino que también desglosa su contenido en objetos individuales y analiza sus relaciones espaciales con el observador y entre los propios objetos. Esta representación estructurada de «qué está dónde» nos permite realizar inferencias espaciales dentro de la escena desde una perspectiva *egocéntrica, objetocéntrica o alocéntrica.* Estas

[19] Por ejemplo, la geometría 3D que se representa en formas diferentes, pero en gran medida equivalentes: nubes de puntos, mallas, funciones de distancia con signo, campos de radiancia neuronal o «splats» gaussianos.

capacidades son necesarias y cruciales para tareas como orientarse en una escena, interactuar con los objetos que contiene y manipularlos.

Por lo tanto, si el objetivo de aprender un modelo de la escena 3D es facilitar la interacción física con un entorno y los objetos que contiene,[20] debemos aprender una representación del entorno 3D que comparta las mismas propiedades y características que nuestra memoria visual mencionada anteriormente.

8.9.1 Tarea y objetivo

Forma y poses de objetos 3D a partir de una sola vista 2D En esta sección, damos un primer paso para mostrar cómo se puede lograr esto mediante la ilustración de un problema complejo y aparentemente mal planteado: *inferir la forma y la pose 3D completas, incluyendo tanto la pose centrada en el observador como aquella centrada en el objeto, a partir de una sola imagen 2D.* Este problema, por sí mismo, se encuentra en el corazón de la visión computacional, con el objetivo de invertir los procesos físicos de formación de imágenes. Una imagen $\boldsymbol{I}$ es el resultado de un proceso de renderización,

$$\boldsymbol{I} = \boldsymbol{\mathcal{P}}(\mathcal{O}, \boldsymbol{\theta}),$$

donde $\mathcal{O}$ representa el objeto 3D en un sistema de coordenadas canónico, independiente de la vista, y $\boldsymbol{\theta}$ denota el punto de vista de la cámara. Disponiendo únicamente de $\boldsymbol{I}$, la tarea de separar las propiedades intrínsecas del objeto (por ejemplo, la forma o la textura) de la pose extrínseca de la cámara es fundamentalmente ambigua.

Los enfoques anteriores suelen simplificar en exceso esta ambigüedad. Los modelos generativos suelen dar prioridad al objeto canónico $\mathcal{O}$ en detrimento de la pose de la cámara $\boldsymbol{\theta}$, lo que da lugar a reconstrucciones que no se alinean fielmente con la vista de entrada. Por el contrario, los métodos tradicionales centrados en la vista fijan la pose, lo que limita su capacidad para generar o «rellenar» las regiones ocluidas e integrar a la perfección el objeto reconstruido en la imagen original.

A continuación presentamos un enfoque ampliamente conocido como **Cupid** [HDZ+25]:

`https://cupid3d.github.io`,

un marco de código abierto que modela conjuntamente la distribución sobre tanto el objeto 3D como la pose de la cámara (Figura 8.27). Al generar simultáneamente el modelo 3D e inferir el punto de vista, Cupid garantiza que el resultado se pueda *pegar* con precisión en la imagen original. Este enfoque replantea la reconstrucción a partir de una sola vista como un problema de muestreo condicional, logrando la fidelidad geométrica mediante la combinación de datos observacionales con priores generativos aprendidos, lo que permite una reconstrucción eficiente tanto a nivel de objetos como de escenas (Figura 8.28).

[20] lo cual es el objetivo de la denominada «IA encarnada» para agentes inteligentes o robots

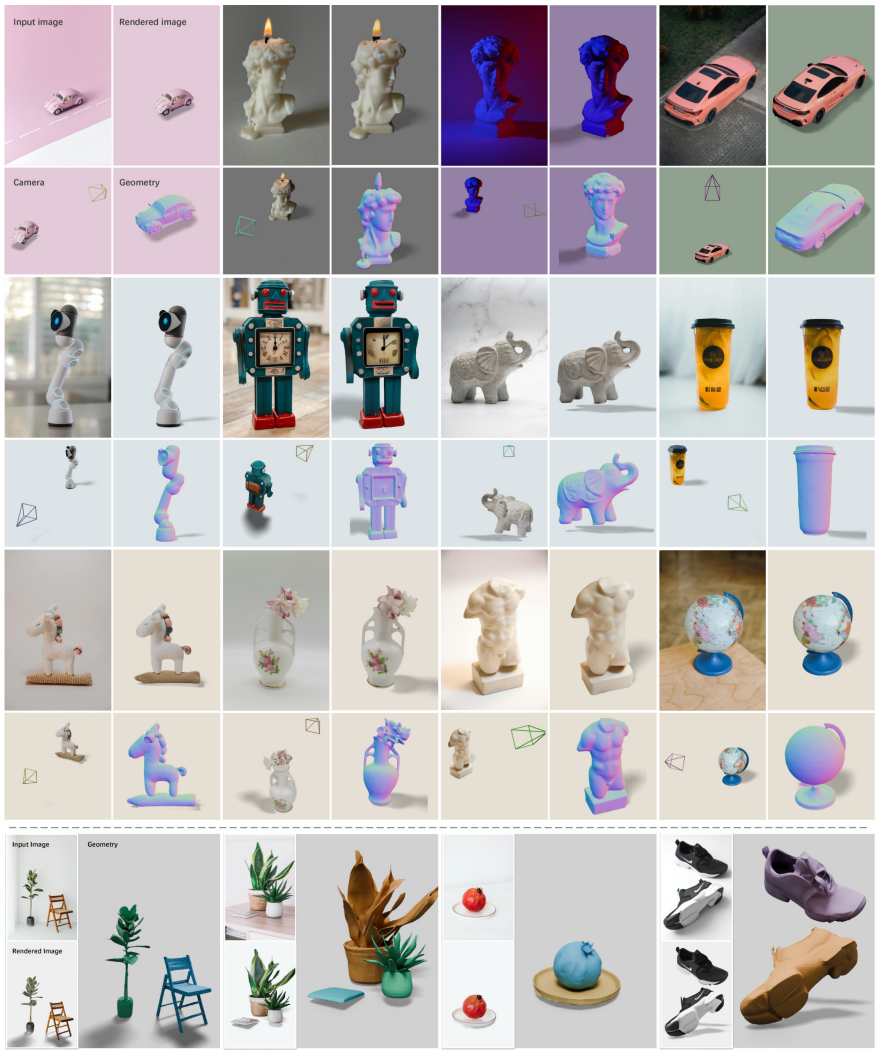


Figura 8.28: **Resultados de la *reconstrucción 3D generativa* a partir de una sola imagen de prueba.** A partir de una imagen de entrada (arriba a la izquierda), Cupid estima la pose de la cámara (abajo a la izquierda) y reconstruye un modelo 3D (abajo a la derecha), volviendo a renderizar la imagen de entrada (arriba a la derecha). Es robusto ante cambios de escala, ubicación e iluminación, al tiempo que conserva los detalles más sutiles, y admite la reconstrucción de escenas con alineación de componentes (fila inferior). Todos los resultados se obtienen en segundos mediante el muestreo *feed-forward* del modelo entrenado.

Formulación de la tarea y objetivo. Formulamos la reconstrucción generativa en 3D como la tarea de estimar la distribución a posteriori conjunta

$$p(\mathcal{O}, \boldsymbol{\theta} \mid \boldsymbol{I})$$

bajo la restricción de que la observación $\boldsymbol{I}$ sea un «renderizado» del objeto $\mathcal{O}$ desde la pose $\boldsymbol{\theta}$. Para modelar esta distribución condicional, empleamos un modelo generativo basado en flujos. Concretamente, primero mapeamos el objeto 3D y la pose de la cámara a una representación latente volumétrica unificada $\boldsymbol{z} = \varphi(\mathcal{O}, \boldsymbol{\theta})$. A continuación, utilizamos un modelo Rectified Flow [LCB+23] para aprender la generación condicional de esta variable latente. El modelo define una interpolación lineal entre una muestra de la distribución de datos, $\boldsymbol{z}_0$, y una muestra de una distribución de ruido, $\boldsymbol{\epsilon}$, a lo largo de un intervalo de tiempo normalizado $t \in [0, 1]$:

$$\boldsymbol{z}_t = (1 - t)\boldsymbol{z}_0 + t\boldsymbol{\epsilon}. \tag{8.9.1}$$

El proceso generativo consiste en invertir esta trayectoria mediante el aprendizaje de un campo de velocidad dependiente del tiempo

$$\boldsymbol{v}(\boldsymbol{z}_t, \boldsymbol{I}, t) = \nabla_t \boldsymbol{z}_t$$

que transporta las muestras ruidosas hacia la variedad de datos, condicionado a la imagen de entrada $\boldsymbol{I}$. Parametrizamos este campo de velocidad con una red neuronal $\boldsymbol{v}_\varphi$ y la entrenamos utilizando el objetivo de *Conditional Flow Matching* (CFM) [LCB+23]:

$$\mathcal{L}_{\text{CFM}}(\varphi) \doteq \mathbb{E}_{t, \boldsymbol{z}_0, \boldsymbol{\epsilon}} \left\| \boldsymbol{v}_\varphi(\boldsymbol{z}_t, \boldsymbol{I}, t) - (\boldsymbol{\epsilon} - \boldsymbol{z}_0) \right\|_2^2. \tag{8.9.2}$$

Al tomar muestras de este modelo aprendido, podemos generar un código latente $\boldsymbol{z}$ que luego puede decodificarse para obtener el objeto 3D final y la pose de la cámara.

8.9.2 Representación y tokenización de formas 3D

Uno de los principales retos en la generación de formas 3D es encontrar una tokenización que sea a la vez compacta y expresiva, capaz de una reconstrucción de alta fidelidad. Si bien **Michelangelo** [ZLC+23a] utiliza una representación vectorial latente aprendida —que codifica las formas en una secuencia de 1024 tokens—, estos vectores carecen de coordenadas espaciales 3D explícitas. En este proyecto, buscamos una representación latente que conserve la información posicional 3D explícita, lo que nos permite alinear los tokens de forma directamente con los píxeles de la imagen. Al establecer esta correspondencia espacial, podemos aprovechar la información local de los píxeles para mejorar los detalles de la generación. En consecuencia, **Cupid** adopta un enfoque de representación latente y tokenización basado en vóxeles 3D, tal como se describe en la Sección 8.8.1. Aquí representamos una forma mediante vóxeles esparsos acompañados de características, que suelen definirse mediante el color RGB o características DINO más avanzadas [ODM+24] procedentes de modelos base 2D. A continuación, entrenamos un decodificador para que mapee dicha representación de vuelta a la forma, parametrizada por la SDF y a la apariencia, parametrizada por los «splats» gaussianos [KKL+23].

En concreto, dado un conjunto de características de vóxeles esparsas $\boldsymbol{F} = \{\boldsymbol{f}_i\}_{i=1}^{N}$ ubicadas en las coordenadas $\boldsymbol{x}_i$, empleamos dos decodificadores distintos, $\mathcal{D}_{geo}$ y $\mathcal{D}_{app}$, para desacopplar la geometría de la apariencia. El proceso de decodificación se formula como:

$$s_i = \mathcal{D}_{geo}(\boldsymbol{f}_i), \quad \mathcal{G}i = \mathcal{D}_{app}(\boldsymbol{f}_i), \tag{8.9.3}$$

donde $s_i \in \mathbb{R}$ representa el valor del campo de distancia con signo (SDF) utilizado para la reconstrucción de la superficie y $\mathcal{G}_i = \{\boldsymbol{c}_i, \alpha_i, \boldsymbol{s}_i, \boldsymbol{q}_i\}$ denota el conjunto de parámetros gaussianos 3D —que comprenden color, opacidad, escala y rotación— requeridos para el renderizado volumétrico. A continuación, utilizaremos las características de vóxeles esparsos como representación latente para *latent flow matching* [RBL+22].

8.9.3 Arquitectura e implementación

Dado que utilizamos vóxeles 3D para la representación de objetos, aplicar la difusión de manera simplista da lugar a un consumo excesivo de memoria, ya que una gran parte de la rejilla está compuesta por espacio vacío innecesario, tal y como se describe en la Sección 8.8.1. Para mitigar esto, la arquitectura **Cupid** está diseñada como un modelo de flujo en cascada de dos etapas que refina progresivamente la reconstrucción 3D [XLX+25]. La primera etapa genera una forma geométrica gruesa y estima la pose de la cámara, mientras que la segunda etapa genera una geometría y una apariencia de alta fidelidad. Una innovación clave de este marco radica en la representación explícita de la orientación de la cámara y su integración en el proceso generativo. El flujo de trabajo general se muestra en la Figura 8.29.

Representación desacoplada de objetos y poses. Como se explica en la Sección 8.8.1, el objeto 3D $\mathcal{O}$ se tensoriza en una representación esparsa basada en vóxeles:

$$\mathcal{O} \triangleq \{\boldsymbol{x}_i, \boldsymbol{f}_i\}_{i=1}^{L}, \tag{8.9.4}$$

donde $\boldsymbol{x}_i \in \mathbb{R}^3$ denota la coordenada de un vóxel activo, y $\boldsymbol{f}_i$ es un vector de características que codifica la geometría y la apariencia locales. Estas características se obtienen mediante la compresión de una rejilla 3D densa de características DINO [ODM+23] de múltiples vistas a través de una VAE 3D. Posteriormente, el decodificador VAE se entrena para reconstruir $\mathcal{O}$ en un campo de distancia con signo (SDF) para la extracción de mallas. Esta representación latente en 3D se ilustra en la parte superior del bloque azul central de la Figura 8.29.

La pose de la cámara $\boldsymbol{\theta}$, representada por su matriz de proyección $\boldsymbol{P} = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}]$, se reparametriza de forma novedosa para facilitar la generación conjunta. En lugar de un vector compacto, representamos la pose como un campo denso de correspondencias de 3D a 2D, lo que denominamos una *distribución intracubo de píxeles*:

$$\boldsymbol{\theta} \triangleq \{\boldsymbol{x}_i, \boldsymbol{u}_i\}_{i=1}^{L},$$

donde $\boldsymbol{u}_i = (u_i, v_i) \in [0,1]^2$ son las coordenadas normalizadas del píxel en 2D correspondientes al centro del vóxel 3D $\boldsymbol{x}_i$. Estas correspondencias se obtienen mediante una proyección en perspectiva:

$$\boldsymbol{u}_i = \pi(\boldsymbol{P}, \boldsymbol{x}_i).$$

Este conjunto de correspondencias se puede visualizar como un cubo UV tridimensional, en el que los valores (u, v) actúan como colores que dependen de la perspectiva en una cuadrícula tridimensional. Esto se ilustra en la parte inferior del bloque azul situado en el centro de la Figura 8.29.

Dado un conjunto de correspondencias generado, la matriz global de la cámara $\boldsymbol{P}^*$ se recupera resolviendo un problema de «Perspectiva de n puntos» (PnP) mediante un algoritmo de mínimos cuadrados [AKH15]:

$$\boldsymbol{P}^* = \arg\min_{\boldsymbol{P}} \sum_{i=1}^{L} \left\| \pi(\boldsymbol{P}, \boldsymbol{x}_i) - \boldsymbol{u}_i \right\|^2. \tag{8.9.5}$$

Esta formulación transforma de manera elegante el problema conjunto de generación del objeto y de su pose en uno de generación de una forma/apariencia 3D centrada en el objeto con la pose del objeto centrada respecto al observador/cámara (a través de las coordenadas UV). Esto se ilustra en la esquina inferior izquierda del bloque rojo situado a la derecha de la Figura 8.29.

Nota 8.1 (Modelado 3D estructurado). Obsérvese que las representaciones de tipo vóxel elegidas aquí para la forma y la pose son diferentes de las representaciones de tipo nube de puntos adoptadas en la tarea de generación de formas condicionadas estudiada en la Sección 8.8 anterior. En esa tarea, el objetivo es generar un objeto que solo tenga que estar fuertemente correlacionado con la imagen (o el texto) dada. No requiere una alineación precisa de la pose entre el objeto (en su pose canónica) y la imagen dada (punto de vista). Sin embargo, para la tarea que nos ocupa, queremos «reconstruir» un objeto cuya forma, apariencia y postura coincidan fielmente con la imagen dada. Por lo tanto, es necesario que la representación interna modele explícitamente la forma canónica del objeto y su pose relativa de manera simultánea. Téngase en cuenta que tal «desacoplamiento» de la forma/apariencia (canónica) de un objeto y la pose desde la que se observa imita de cerca las funciones antes mencionadas de la corteza IT y el hipocampo en nuestro cerebro, respectivamente. Como veremos más adelante, una representación desacoplada de este tipo nos permitirá obtener un modelo 3D estructurado y coherente de una escena que puede contener múltiples objetos desde múltiples vistas. Sostenemos que un modelo 3D así estructurado es necesario para que podamos realizar razonamientos centrados en la vista o en el objeto,[21], así como para generar y interactuar con la escena.

[21] Por ejemplo, responder preguntas tales como «¿qué objeto está delante, detrás, a la izquierda o a la derecha de, o encima de qué objeto?»

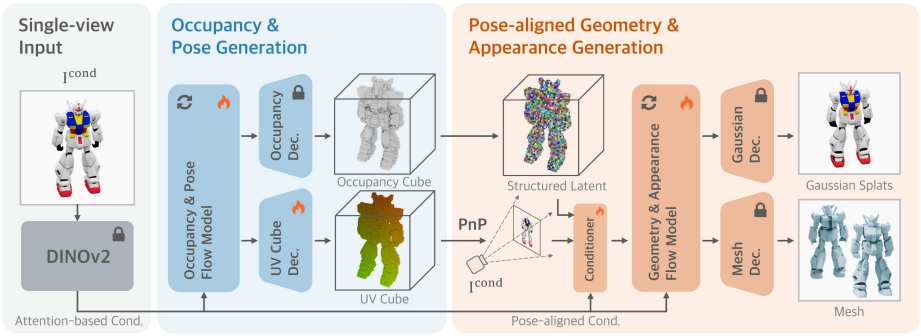


Figura 8.29: **El proceso de reconstrucción generativa en dos etapas de Cupid**. Dada una imagen de entrada, la primera etapa ($\boldsymbol{\mathcal{G}_S}$) genera un cubo de ocupación aproximado y un cubo UV, que codifica las correspondencias de 3D a 2D. Un solucionador PnP recupera la pose de la cámara $\boldsymbol{P}^*$ a partir de estas correspondencias. La segunda etapa ($\boldsymbol{\mathcal{G}_L}$) está condicionada por esta pose recuperada. Inyecta características alineadas con los píxeles (extraídas de DINOv2 y de mapas de características de bajo nivel) en el proceso generativo para sintetizar una geometría y una apariencia de alta fidelidad para el objeto 3D final.

8.9.4 Modelado de flujo en cascada

Utilizamos un modelo de flujo en cascada de dos etapas para muestrear conjuntamente un objeto 3D y su pose de cámara correspondiente, que se denota como $\boldsymbol{z} = \{(\boldsymbol{x}_i, \boldsymbol{f}_i), (\boldsymbol{x}_i, \boldsymbol{u}_i)\}_{i=1}^{L}$. En la primera etapa, el modelo generador de ocupación y pose ($\boldsymbol{\mathcal{G}_S}$) produce dos resultados clave: (i) un cubo de ocupación que identifica los vóxeles activos y (ii) un cubo UV que codifica la pose de la cámara desde la perspectiva del objeto. En la segunda etapa, dadas estas predicciones, el modelo de geometría y apariencia alineado con la pose ($\boldsymbol{\mathcal{G}_L}$) sintetiza las características DINO [ODM+23] $\boldsymbol{f}_i$ para cada vóxel activo, para obtener la variable latente final $\boldsymbol{z}$.

Generación de ocupación y pose. Dada una imagen de condicionamiento $\boldsymbol{I}$, esta etapa genera un cubo de ocupación y un cubo UV, ambos con una resolución de r. El cubo de ocupación, $\boldsymbol{G}_o \in \{0,1\}^{r\times r\times r\times 1}$, contiene valores binarios que distinguen los vóxeles activos de los inactivos. El cubo UV, $\boldsymbol{G}_{uv} \in [0,1]^{r\times r\times r\times 2}$, almacena las coordenadas de píxels normalizadas (u, v) para cada vóxel; cabe destacar que los vóxeles que comparten coordenadas de píxel idénticas sealinean sobre el mismo haz (rayo) de la cámara.[22]

Para mejorar la eficiencia computacional, entrenamos un VAE 3D para comprimir el cubo UV en una rejilla de características de baja resolución

[22]No modelamos explícitamente la oclusión, ya que estudios recientes sugieren que los transformers modelan eficazmente el transporte de la luz de forma implícita [JJT+24; ZDP+25].

$\boldsymbol{S}_{uv} \in \mathbb{R}^{s\times s\times s\times C}$, logrando una recuperación de la pose con pérdidas mínimas (RRE/RTE medio $< 0,5°$). Para afinar el TRELLIS, concatenamos la malla de características original $\boldsymbol{S}_o$ con $\boldsymbol{S}_{uv}$ e introducimos capas de proyección lineal tanto en la entrada como en la salida de la red de flujo $\boldsymbol{\mathcal{G}_S}$. Una vez generados los cubos de ocupación y UV, extraemos el conjunto $\{(\boldsymbol{x}_i, \boldsymbol{u}_i(\boldsymbol{\theta}))\}_{i=1}^{L}$ recopilando vóxeles activos y resolviendo la pose de la cámara mediante la ecuación 8.9.5.

Generación de geometría y apariencia alineadas a la pose. En la segunda etapa, generamos detalladas características latentes 3D $\{\boldsymbol{f}_i\}_{i=1}^{L}$ únicamente en las ubicaciones de los vóxeles activos. Los experimentos indican que el modelo estándar $\boldsymbol{\mathcal{G}_L}$ de TRELLIS [XLX+25], que opera sobre información de imagen mediante atención global, está sujeto a deriva de color y a la pérdida de detalles finos. Mitigamos esto utilizando la pose de la cámara derivada en la primera etapa para inyectar información de píxeles con atención local en cada vóxel.

En concreto, calculamos las características del vóxel i utilizando la pose estimada de la siguiente manera:

$$\begin{aligned} \boldsymbol{f}_i^{\mathrm{DINO}} &= \mathrm{BilinearInterp}(\boldsymbol{u}_i, \mathrm{DINO}(\boldsymbol{I})) \in \mathbb{R}^{1024}, \\ \{\boldsymbol{f}_i^{\mathrm{h}}\}_{i=1}^{L} &= \mathrm{SlatEncoder}\left(\{\boldsymbol{x}_i, \boldsymbol{f}_i^{\mathrm{DINO}}\}_{i=1}^{L}\right), \quad \boldsymbol{f}_i^{\mathrm{h}} \in \mathbb{R}^{8}, \end{aligned} \tag{8.9.6}$$

donde $\boldsymbol{u}_i$ representa la proyección del centro del vóxel 3D i-ésimo sobre el plano de la imagen, BilinearInterp denota la interpolación bilineal y SlatEncoder es el codificador 3D VAE. Aunque DINO [ODM+23] capta la semántica de alto nivel, frecuentemente carece de las pistas de bajo nivel necesarias para una reconstrucción precisa. Para compensar esto, extraemos características complementarias de bajo nivel $\boldsymbol{f}^{\mathrm{l}}$ de $\boldsymbol{I}$ utilizando una cabeza convolucional ligera, y las muestreamos en $\boldsymbol{u}_i$ mediante BilinearInterp. Finalmente, en cada paso temporal t, fusionamos la característica de vóxel ruidosa $\boldsymbol{f}_i^t$ con las características alineadas con los píxeles a través de una capa lineal antes de introducirlas en el transformer de flujo:

$$l_t = \mathrm{Linear}\big([\boldsymbol{f}_i^t \oplus \boldsymbol{f}_i^{\mathrm{h}} \oplus \boldsymbol{f}_i^{\mathrm{l}}]\big). \tag{8.9.7}$$

Esta estrategia de fusión alineada a la pose mejora significativamente tanto la precisión geométrica en 3D como la fidelidad de la apariencia con respecto a la imagen de entrada.

Nota 8.2 (Procesos generativos desacoplados). Obsérvese que el marco de dos etapas descrito anteriormente desacopla la distribución de los objetos 3D en varias componentes interrelacionadas —forma, pose y apariencia— y aprende cada una de manera independiente. Aunque esto implica que debemos aprender dos modelos generativos de eliminación de ruido al mismo tiempo, las representaciones aprendidas resultantes están mucho mejor estructuradas y permiten un razonamiento espacial y una generación mucho más precisos y flexibles.

Tabla 8.11: **Precisión de la geometría monocular.** Cupid supera a todas las líneas de base de reconstrucción y generación 3D e iguala a los métodos de regresión de mapas de puntos que predicen solo una geometría parcial. Cabe señalar que VGGT utiliza una máscara del objeto de referencia, lo que podría sobreestimar la precisión.

Método	3D	Toys4k					GSO				
		mIOU (avg)↑	CD (avg)↓	CD (med)↓	F-score (0.01)↑	F-score (0.05)↑	mIOU (avg)↑	CD (avg)↓	CD (med)↓	F-score (0.01)↑	F-score (0.05)↑
VGGT	×	–	1.144	0.498	61.85	95.90	–	1.396	0.388	65.98	95.95
MoGe	×	92.80	1.284	0.581	58.54	95.31	96.18	1.743	0.575	58.99	94.68
OnePoseGen	✓	9.34	153.2	59.92	6.11	24.10	12.16	116.2	60.56	7.28	25.77
LaRa	✓	68.11	32.15	16.59	18.57	57.67	70.63	34.23	19.36	13.48	49.95
OpenLRM	✓	86.26	2.726	1.291	40.42	90.60	91.35	3.741	1.858	34.14	87.20
Ours	✓	**92.43**	**2.534**	**0.236**	**69.82**	**97.76**	**95.27**	**1.823**	**0.434**	**61.01**	**95.59**

8.9.5 Experimentos

Para validar la arquitectura, evaluamos el sistema en tres aspectos fundamentales: predicción de la geometría monocular, coherencia con la vista de entrada, así como reconstrucción 3D completa a partir de una sola imagen. El proceso de evaluación comparativa se centró en sistemas basados en el aprendizaje, excluyendo los métodos de optimización por escena para garantizar una comparación imparcial de las capacidades de inferencia. Eliminamos el condicionamiento por pose en un experimento de ablación.

El panorama comparativo. Hemos realizado un benchmark contra tres paradigmas distintos que actualmente dominan el campo:

(i) *Regresión de mapas de puntos*, como VGGT [WCK+25] y MoGe [WXD+25], que predicen simultáneamente puntos 3D por píxel y las poses de la cámara, pero que a menudo carecen de priores robustos para la geometría oculta;

(ii) *Reconstrucción centrada en la vista*, incluyendo LRM [HZG+23] y LaRa [CXE+24], que generan modelos 3D directamente en el espacio de la vista para evitar la estimación de la pose durante la inferencia; y

(iii) *Generación desacoplada*, ejemplificada por OnePoseGen [GWX+25], que separa la generación 3D canónica de la estimación de la pose, basándose en una alineación post-hoc.

Precisión geométrica y simplificación de la *pipeline*. En las tareas de geometría monocular, el modelo propuesto demostró una clara superioridad frente a los enfoques centrados en la vista (Tabla 8.11). En el conjunto de datos GSO [DFK+22], redujo la distancia chamfer (CD) promedio en un 50% en comparación con LRM [HZG+23]. Esta mejora se atribuye a la doble capacidad del modelo: modela con precisión los atributos intrínsecos de los

Tabla 8.12: **Comparación en calidad 3D completa.** Presentamos las puntuaciones de imágenes CLIP de nuevas vistas siguiendo [GHH+24].

	OnePoseGen	LaRa	OpenLRM	TRELLIS	Cupid
ViT-B/16	0.7933	0.8334	0.8939	0.9465	**0.9501**
ViT-L/14	0.7193	0.7682	0.8410	0.9210	**0.9291**

Tabla 8.13: **Consistencia entre la entrada y la visualización.** Cupid logra una consistencia superior entre la entrada y la visualización, lo que permite una alineación precisa de la apariencia.

Conj. de datos		**Toys4K**			**GSO**		
Método	**Pose**	**PSNR↑**	**SSIM↑**	**LPIPS↓**	**PSNR↑**	**SSIM↑**	**LPIPS↓**
LaRa	×	22.00	93.42	0.0884	19.81	91.61	0.1119
OpenLRM	×	26.41	80.17	0.1156	25.79	78.80	0.1268
OnePoseGen	✓	17.43	89.37	0.1174	14.87	86.46	0.1386
Cupid	✓	**30.05**	**96.81**	**0.0251**	**28.68**	**95.49**	**0.0354**

objetos mediante priors generativos, al tiempo que predice con exactitud los parámetros extrínsecos de la cámara.

Lo más importante es que el modelo alcanzó una precisión geométrica comparable a la de regresores especializados en mapas de puntos, como MoGe [WXD+25]. Este hallazgo tiene importantes implicaciones para el diseño de flujos de trabajo: sugiere que los flujos de trabajo complejos y de múltiples etapas—que tradicionalmente utilizan la estimación geométrica parcial como paso previo [YZY+25]—pueden simplificarse. Nuestro enfoque alcanza una precisión comparable en un solo paso, lo que hace innecesarias esas representaciones intermedias.

Fidelidad y consistencia visuales. Evaluamos la consistencia entre la entrada y la vista para medir hasta qué grado la reconstrucción 3D se alinea con la imagen de origen (Tabla 8.13). Los resultados pusieron de manifiesto debilidades específicas en los modelos de referencia: la calidad de la textura de LaRa se degradó debido a la generación inconsistente de nuevas vistas, mientras que OnePoseGen adolecía de una grave desalineación de texturas y de cambios de color durante el registro.

Por el contrario, nuestro método mantuvo alta precisión en la pose sin sacrificar los detalles de textura (Figura 8.30). Reconstruyó con éxito objetos de alta fidelidad tanto a partir de conjuntos de datos sintéticos (Toys4K) como de imágenes del mundo real que planteaban un gran desafío (Figura 8.28), superando de manera consistente a las líneas de base en métricas de alineación semántica como la similitud CLIP [RKH+21b] (Tabla 8.12).

Figura 8.30: **Comparación cualitativa en cuanto a la consistencia con la vista de entrada.** Renderizamos la vista de entrada utilizando la pose de cámara generada. En el caso de los métodos centrados en la vista (LRM, LaRa), utilizamos los parámetros intrínsecos de referencia para el renderizado, ya que estos métodos no modelan dichos parámetros. Nuestro método produce la geometría y la apariencia de mayor fidelidad; LRM alucina detalles erróneos, LaRa resulta excesivamente borrosa debido a inconsistencias en la difusión 2D y el método de generación 3D OnePoseGen a menudo no logra registrar la pose de manera confiable.

Experimentos de ablación Los estudios de ablación realizados en la segunda fase de refinamiento confirmaron el papel fundamental del condicionamiento alineado con la pose (PAC). Probamos variantes que iban desde líneas base estándar de latentes hasta otras que incorporaban volúmenes de características de DINOv2.

Como se muestra en la Tabla 8.14, la configuración más eficaz resultó ser un enfoque híbrido: la concatenación de características de DINOv2 con indicios visuales de bajo nivel extraídos mediante capas convolucionales. Esta combinación proporcionó la mayor calidad visual, uniendo de manera eficaz la comprensión semántica con los detalles a nivel de píxel (Figura 8.31). Además, este mecanismo de condicionamiento demostró una robustez notable, ya que mantuvo un alto rendimiento incluso cuando la geometría de entrada presentaba perturbaciones estocásticas desde la primera etapa de generación.

Aplicación I: Reconstrucción de escenas compuestas. Para extender Cupid a la reconstrucción a nivel de escena, aprovechamos modelos

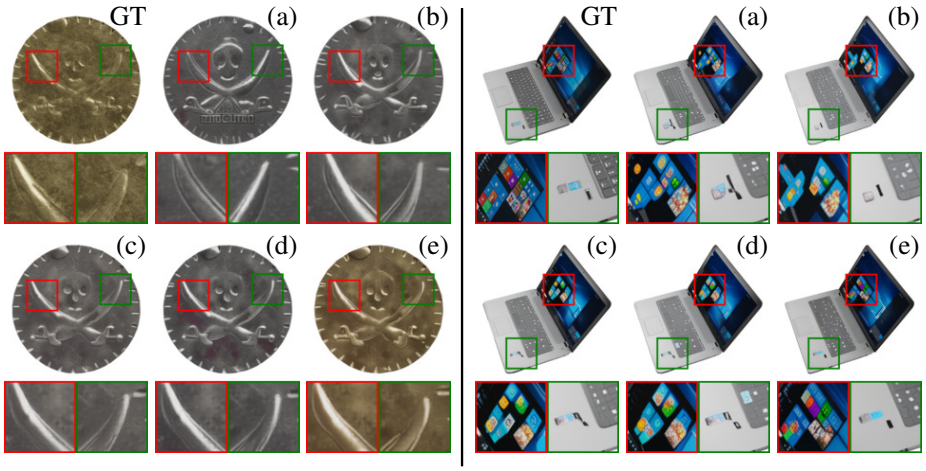


Figura 8.31: **Comparación cualitativa de diversos métodos de condicionamiento con alineación de poses.** Nuestro método (e) alcanza la mejor calidad visual en cuanto a fidelidad del color y nivel de detalle.

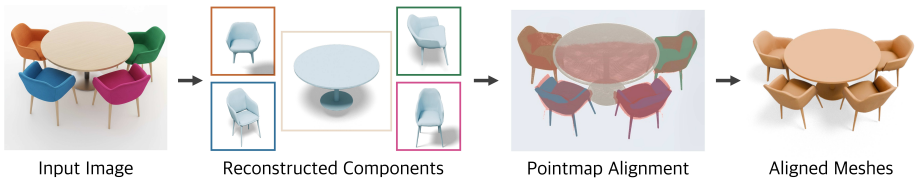


Figura 8.32: **Reconstrucción de escenas con alineación por componentes.** Cada objeto se reconstruye de forma independiente con **Cupid**; las correspondencias explícitas entre 3D y 2D permiten su colocación precisa en un marco compartido.

Tabla 8.14: **Estudios de ablación del condicionamiento alineado con la pose.**

Método	Geo de ref. & Pose			Geo muestreada & Pose		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
(a) Línea de base (sin PAC)	31.84	97.50	0.0219	27.47	95.64	0.0327
(b) Incrustación posiciones	32.07	97.58	0.0211	27.56	95.67	0.0323
(c) Latente (sin Occ.)	32.37	97.72	0.0201	27.85	95.87	0.0309
(d) Latente (Occ.)	32.39	97.77	0.0199	27.74	95.80	0.0313
(e) Latente (Caract. visuales)	**34.86**	**98.24**	**0.0168**	**30.05**	**96.81**	**0.0251**

fundacionales (foundation models) para descomponer la escena en K objetos con máscaras $\{\boldsymbol{M}^{(k)}\}_{k=1}^{K}$ y estimar un mapa global de puntos métricos $\mathcal{P}$ [WXD+25].

En la primera etapa, reconstruimos cada objeto en forma independiente para recuperar su geometría canónica, adaptando el generador para manejar las oclusiones mutuas mediante un ajuste fino con máscaras aleatorias [WZG+25]. Para el k-ésimo objeto, esto da como resultado un conjunto de puntos 3D canónicos $\boldsymbol{x}_i^{(k)}$ correspondientes a los píxeles $\boldsymbol{u}_i \in \boldsymbol{M}^{(k)}$. Para resolver la ambigüedad de escala inherente a la generación independiente, formulamos la alineación como un problema de correspondencia entre el espacio canónico generado y el espacio de cámara estimado:

$$\boldsymbol{p}_i = \mathcal{P}(\boldsymbol{u}_i), \quad \forall \boldsymbol{u}_i \in \boldsymbol{M}^{(k)}, \tag{8.9.8}$$

donde $\boldsymbol{p}_i$ representa las coordenadas métricas del objetivo en el sistema de referencia de la cámara, derivadas del modelo de fundacional de profundidad.

En la segunda etapa, calculamos una transformación de similitud por objeto $\mathcal{T}_k = \{s_k, \boldsymbol{R}_k, \boldsymbol{t}_k\}$ para situar todas las componentes en un sistema de coordenadas unificado. Determinamos estos parámetros utilizando el método de Umeyama [Ume02] minimizando el error de alineación sobre los píxeles visibles de cada objeto:

$$\mathcal{T}_k^* = \arg\min_{s_k, \boldsymbol{R}_k, \boldsymbol{t}_k} \sum_i \left\| \boldsymbol{p}_i - \left(s_k \boldsymbol{R}_k \boldsymbol{x}_i^{(k)} + \boldsymbol{t}_k \right) \right\|^2. \tag{8.9.9}$$

La aplicación de estas transformaciones óptimas produce una composición de escena métricamente consistente, en la que todas las componentes generadas están correctamente posicionados y escalados entre sí. Consulte la Figura 8.33 para más ejemplos de composición de escenas.

Aplicación II: Reconstrucción multivista consistente. Gracias a nuestra formulación bayesiana, nuestro modelo se generaliza a escenarios con múltiples vistas incluso cuando se entrena únicamente con imágenes individuales. Para implementar la capacidad de múltiples vistas descrita en la Sección 8.9.4, extendemos el proceso de muestreo para incluir K vistas de entrada $\boldsymbol{I}^{(k)}{}_{k=1}^{K}$.

Figura 8.33: **Ejemplos adicionales de reconstrucción de escenas mediante alineación por componentes.** En cada uno de los ejemplos mostrados, los paneles muestran: (arriba a la izquierda) la imagen de entrada, (arriba a la derecha o abajo a la izquierda) el resultado final renderizado y (abajo) las componentes individuales reconstruidas, codificadas por colores para mayor claridad.

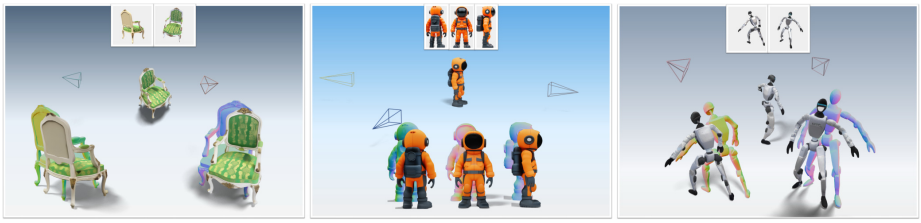


Figura 8.34: **Condicionamiento multi-vista.** Nuestro modelodo conjunto desacoplado admite de forma natural el condicionamiento multi-vista. Al disponer de múltiples vistas de entrada, fusionamos el objeto latente compartido —*independiente de la vista*— a lo largo de las rutas de flujo (al igual que MultiDiffusion [BYL+23]), lo que permite el refinamiento de los objetos y las cámaras en todas las vistas. Arriba: entradas; en el centro: objeto 3D reconstruido y poses de la cámara; abajo: imágenes renderizadas y geometría.

En la primera etapa, nuestro objetivo es recuperar una única geometría canónica y, al mismo tiempo, estimar K poses de cámara distintas. Inicializamos una latente de geometría compartida $\boldsymbol{S}_o$ y K latentes de pose independientes $\{\boldsymbol{S}_{uv}^{(k)}\}_{k=1}^{K}$. En cada paso de eliminación de ruido t, predecimos los campos de flujo tanto para la geometría como para la pose en paralelo para cada vista. Sea la salida de la red de flujo para la vista k-ésima un vector concatenado de actualizaciones de geometría y pose:

$$[d_{o,t}^{(k)}, d_{uv,t}^{(k)}] = \boldsymbol{\mathcal{G}}_{\boldsymbol{S}}(\text{concat}(\boldsymbol{S}_o^t, \boldsymbol{S}_{uv}^{(k),t}), t, \boldsymbol{I}^{(k)}). \tag{8.9.10}$$

Para garantizar la coherencia geométrica en todas las observaciones, agregamos las actualizaciones geométricas al tiempo que mantenemos las trayectorias de pose específicas de cada vista. La regla de actualización en el paso t queda así:

$$\boldsymbol{S}_o^{t-1} \leftarrow \text{Step}\left(\boldsymbol{S}_o^t, \frac{1}{K}\sum_{k=1}^{K} d_{o,t}^{(k)}\right), \tag{8.9.11}$$

$$\boldsymbol{S}_{uv}^{(k),t-1} \leftarrow \text{Step}\left(\boldsymbol{S}_{uv}^{(k),t}, d_{uv,t}^{(k)}\right), \tag{8.9.12}$$

donde $\text{Step}(\cdot)$ denota la regla de actualización del muestreador de flujo. Las mallas de características resultantes se decodificarán en una malla de ocupación gruesa compartida $\boldsymbol{G}_o$ y en las poses $\{\boldsymbol{\theta}^k\}_{k=1}^{K}$.

En la segunda etapa, con la geometría fijada, aplicamos una estrategia de difusión múltiple [BYL+23] a la latente de apariencia $\boldsymbol{F}_o$. Mantenemos una única latente compartida y la actualizamos utilizando las predicciones de flujo promediadas de todas las K vistas, condicionadas a la geometría gruesa $\boldsymbol{G}_o$ y a las poses $\{\boldsymbol{\theta}^k\}_{k=1}^{K}$ estimadas en la primera etapa:

$$d_{\text{feat},t}^{(k)} = \boldsymbol{\mathcal{G}}_{\boldsymbol{L}}(\boldsymbol{F}_o^t, \boldsymbol{G}_o, \boldsymbol{\theta}^k, t, \boldsymbol{I}^{(k)}), \tag{8.9.13}$$

$$\boldsymbol{F}_o^{t-1} \leftarrow \text{Step}\left(\boldsymbol{F}_o^t, \frac{1}{K}\sum_{k=1}^{K} d_{\text{feat},t}^{(k)}\right). \tag{8.9.14}$$

Esta operación de promediado garantiza que la representación 3D generada cumpla simultáneamente con las restricciones visuales impuestas por todas las vistas de entrada, logrando una consistencia al estilo SfM que se muestra en la Figura 8.34.

8.9.6 Conclusión

En esta sección se ha presentado un marco unificado para la generación y reconstrucción 3D, que operacionaliza la distinción biológica entre identidad del objeto («qué») y contexto espacial («dónde»). Al modelar conjuntamente el objeto 3D canónico $\mathcal{O}$ y la pose de la cámara $\boldsymbol{\theta}$, nuestro enfoque desacopla eficazmente la forma intrínseca del punto de vista extrínseco.

Aprovechando los priores de pose y el condicionamiento alineado, el modelo realiza una inferencia similar a la bayesiana para reconstruir la geometría 3D completa a partir de una sola vista. Esto garantiza la coherencia geométrica y la robustez frente a las variaciones del punto de vista. En última instancia, esta representación estructurada impulda las capacidades de la IA encarnada, dotando a los agentes de la comprensión espacial necesaria para navegar e interactuar de manera eficaz con entornos 3D.

8.10 Generación condicional de movimientos del cuerpo humano

Hasta ahora, en las secciones anteriores, hemos mostrado cómo aprender las distribuciones de los datos visuales, incluidas las imágenes en 2D y las formas de los objetos en 3D. Cabe destacar que el objetivo principal de percibir el mundo tridimensional y desarrollar un modelo del mismo es la navegación, la interacción y la manipulación de objetos en él. A partir de la información que percibe sobre el entorno, nuestro cerebro elabora planes y lleva a cabo acciones en función de las tareas del momento. El nivel de destreza que se puede alcanzar mediante la «coordinación ojo-mano» es, podría considerarse, la forma más elevada de inteligencia física que existe en la naturaleza. Por lo tanto, estimar y controlar con precisión los movimientos de nuestro cuerpo y nuestras manos es fundamental para interactuar con los objetos del entorno. Nuestro cerebro dedica importantes recursos a este fin: la *corteza motora*, situada en el lóbulo frontal del cerebro, registra y coordina los movimientos de nuestro cuerpo, mientras que las señales propioceptivas procedentes del sistema nervioso de nuestro cuerpo proporcionan retroalimentación continua sobre las posiciones de nuestras extremidades. Por lo tanto, si queremos que los robots, especialmente los de aspecto humanoide, imiten las acciones humanas, deben aprender (la distribución de) nuestros movimientos corporales.

En esta sección, mostramos cómo el marco de inferencia condicional del Capítulo 7 puede aplicarse para aprender la distribución del movimiento humano y, posteriormente, aprovecharse para la tarea de estimación de la postura corporal. Para ello, consideramos una configuración sencilla pero natural: una persona lleva un dispositivo montado en la cabeza, como un casco de RA o unas gafas inteligentes, que captura vídeo frontal y rastrea su pose tridimensional mediante Estructura a partir del Movimiento (SFM) o Localización y Mapeo Simultáneos (SLAM). Estos métodos estiman la posición del dispositivo mediante el seguimiento y la triangulación de características visuales de la escena para reconstruir la estructura y la pose en 3D. [23] Nuestro objetivo es estimar la pose de todo el cuerpo, la altura y la configuración de las manos del usuario a partir de estas observaciones egocéntricas, aunque el cuerpo rara vez sea visible en el vídeo capturado. La observación clave es que *el movimiento de la cabeza codifica una gran cantidad de información sobre el movimiento del cuerpo.* Cuando caminamos, nuestra cabeza se balancea; cuando estiramos el brazo, se inclina; cuando nos sentamos, describe un arco característico. Estas correlaciones, obtenidas a partir de datos de captura de movimiento, forman una distribución a priori condicional $p(\text{cuerpo} \mid \text{cabeza})$. El método que presentamos en esta sección se basa en un proyecto de código abierto llamado **EgoAllo** (por «estimación de egocéntrica a alocéntrica»):

`https://egoallo.github.io` ,

que incorpora mediciones adicionales como la detección de las manos y el contacto con el suelo mediante muestreo guiado. La Figura 8.35 ilustra la configuración general y la tarea.

8.10.1 Representación del cuerpo humano

Antes de formular el problema de estimación, necesitamos una representación matemática del cuerpo humano. El esqueleto de nuestro cuerpo constituye un árbol cinemático: huesos rígidos unidos por articulaciones, con la pelvis como raíz y cadenas que se extienden hasta la cabeza, las manos y los pies. Cada articulación tiene grados de libertad rotacionales: el hombro gira en tres ejes, mientras que el codo gira principalmente en uno. Para una introducción más formal y detallada a la representación cinemática de sistemas de cuerpos rígidos articulados, como el cuerpo humano, los lectores interesados pueden consultar el clásico libro de texto [MLS94].

El modelo corporal SMPL. SMPL (modelo lineal multi-persona con piel) [LMR+15] ofrece una representación compacta y diferenciable del cuerpo humano, tanto en su postura (cinemática) como a su forma. Divide la configuración del cuerpo en dos componentes:

- **Parámetros de pose** θ: Rotaciones articulares que especifican la postura del cuerpo. SMPL utiliza 23 articulaciones (más la raíz), cada una

[23] Los lectores interesados pueden consultar el libro de texto [MKS+04] para obtener una introducción a esta línea de investigación.

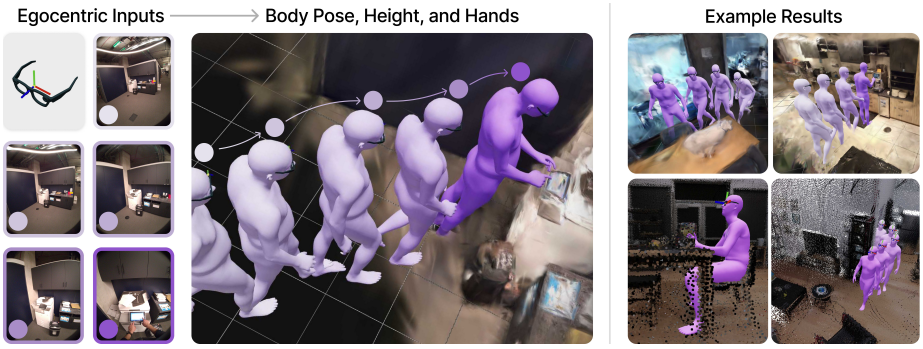


Figura 8.35: **Estimación egocéntrica del movimiento humano.** A partir de las posturas de la cabeza obtenidas mediante SLAM y de imágenes egocéntricas captadas por un dispositivo montado en la cabeza (a la izquierda), EgoAllo estima la postura completa del cuerpo, la altura y los parámetros de las manos en el sistema de referencia mundial (alocéntrico) (a la derecha). Los cuerpos estimados están anclados en la escena, con los pies en contacto con el suelo y las manos en posiciones físicamente plausibles.

parametrizada por una rotación 3D. En conjunto, estos valores determinan si una persona está de pie, sentada, alcanzando algo, caminando, etc.

- **Parámetros de forma** β: Un vector de baja dimensión, normalmente compuesto por 10 coeficientes, que controla las proporciones corporales, como la altura, el peso, la longitud de las extremidades y el ancho del tórax. Estos se derivan del análisis de componentes principales de un amplio corpus de escaneos corporales en 3D.

Dada una pose θ y una forma β, SMPL genera una malla 3D mediante el *blend skinning* aprendido. También proporciona las posiciones de las articulaciones a través de la cinemática directa: partiendo de la posición de la raíz (pelvis) y aplicando la rotación de cada articulación en secuencia a lo largo de la cadena cinemática, podemos calcular la posición final de cada articulación en el espacio 3D. Por ejemplo, para encontrar la posición de la muñeca, componemos las rotaciones de la pelvis, la columna vertebral, el hombro y el codo, acumulando las traslaciones definidas por las longitudes de los huesos.

Agregar manos: SMPL-H y MANO. El modelo SMPL original representa las manos como simples puntos finales. Para aplicaciones que requieren posturas detalladas de las manos, como agarrar objetos, hacer gestos o escribir, esto resulta insuficiente. SMPL-H [RTB17] amplía SMPL con manos articuladas basadas en el modelo de manos MANO, añadiendo 15 articulaciones por mano (tres por dedo), lo que suma un total de 52 articulaciones.

Las rotaciones de la mano φ siguen la misma parametrización que las articulaciones del cuerpo. El estado completo incluye las rotaciones del cuerpo

θ, las rotaciones de la mano φ y los parámetros de forma β.

Por qué son importantes los modelos corporales. ¿Por qué utilizar SMPL-H en lugar de, por ejemplo, realizar una regresión directa de las posiciones de las articulaciones o de los vértices de la malla? Los modelos paramétricos ofrecen ventajas clave. Los resultados son siempre cuerpos humanos válidos, sin configuraciones anatómicamente imposibles. El modelo es diferenciable, lo que permite la optimización basada en gradientes. La parametrización es compacta, ya que solo requiere unos pocos cientos de números en comparación con una malla completa. Y obtenemos una estructura semántica, razonando sobre la «muñeca izquierda» en lugar de índices de vértices.

8.10.2 Tarea y configuración

Entradas. Las entradas consisten en dos flujos procedentes de un dispositivo montado en la cabeza:

- **Posturas de la cabeza** $\boldsymbol{h}_{1:T} \in \mathsf{SE}(3)^T$[24]: Posición y orientación del dispositivo en el marco de refrencia mundial en cada paso de tiempo, a partir de SLAM visual-inercial. Aquí $\mathsf{SE}(3)$ denota el grupo de transformaciones rígidas en 3D, que comprende rotaciones y traslaciones.[25] Estas posturas tienen escala métrica, lo que significa que conocemos las distancias reales en metros, en lugar de solo la geometría relativa, y están alineadas con la gravedad, por lo que sabemos cuál es la dirección hacia arriba.

- **Imágenes egocéntricas** $I_{1:T}$: Vídeo orientado hacia delante. El cuerpo del usuario no suele verse, pero las manos aparecen con frecuencia al alcanzar o manipular objetos.

A partir de imágenes, un estimador de manos ya disponible como HaMeR [PSR+24] genera mediciones por cuadro cuando las manos están visibles:

$$\boldsymbol{y}^{\text{hand},t} = \big(\hat{\varphi}^{\text{MANO},t},\, \hat{\boldsymbol{q}}^{3D,t},\, \hat{\boldsymbol{q}}^{2D,t}\big), \tag{8.10.1}$$

que incluye rotaciones estimadas de la mano, puntos clave 3D en el sistema de coordenadas de la cámara y puntos clave 2D.

Salidas. EgoAllo estima una secuencia de movimientos a lo largo de T pasos temporales:

$$\boldsymbol{x}_0 = \{\boldsymbol{x}_0^1, \ldots, \boldsymbol{x}_0^T\}, \quad \text{where} \quad \boldsymbol{x}_0^t = \big(\theta^t,\, \varphi^t,\, \beta,\, s\big). \tag{8.10.2}$$

[24] Aquí, $\mathsf{SE}(3)$ representa el grupo euclidiano especial en $\mathbb{R}^3$, es decir, el grupo de transformaciones compuestas por rotaciones y traslaciones.

[25] Los lectores interesados pueden consultar [MKS+04] para obtener una introducción formal al movimiento de cuerpos rígidos, así como a su representación y propiedades.

Aquí, θ^t son las rotaciones de las articulaciones del cuerpo, φ^t las rotaciones de las manos, β los parámetros de forma compartidos a lo largo del tiempo, y s un parámetro de altura que también se comparte. El subíndice 0 denota el movimiento limpio, siguiendo la notación de difusión, donde $\boldsymbol{x}_n$ es la versión con ruido en el paso n.

EgoAllo estima la altura además de la postura, lo cual es esencial para el anclaje al suelo. Sin ella, el cuerpo flota en una posición vertical arbitraria; con ella, los pies tocan el suelo y la cabeza se alinea con la cámara.

Planteamiento como posterior. Siguiendo el marco bayesiano del Capítulo 7, la estimación se formula como un muestreo desed una distribución a posteriori. Sea $\boldsymbol{y}$ el conjunto de todas las observaciones y $\boldsymbol{c} = g(\boldsymbol{h}_{1:T})$ una representación procesada de las poses de la cabeza (que se explicará más adelante en la Sección 8.10.3). El posterior se descompone en factores:[26]

$$p(\boldsymbol{x}_0 \mid \boldsymbol{y}) \;\propto\; p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{c}) \cdot p(\boldsymbol{y}^{\text{hand}}, \boldsymbol{y}^{\text{phys}} \mid \boldsymbol{x}_0), \tag{8.10.3}$$

donde:

- $p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{c})$ es una distribución a priori condicional aprendida, concretamente un modelo de difusión para la eliminación de ruido genera movimientos corporales plausibles a partir del movimiento de la cabeza.
- $p(\boldsymbol{y}^{\text{mano}}, \boldsymbol{y}^{\text{físico}} \mid \boldsymbol{x}_0)$ es la verosimilitud de las observaciones sobre la mano y las restricciones físicas, como el contacto con el suelo y la ausencia de deslizamiento de los pies, dado el movimiento del cuerpo.

La verosimilitud se expresa mediante una energía de guía $\mathcal{L}_{\text{guide}}$:

$$p(\boldsymbol{y}^{\text{hand}}, \boldsymbol{y}^{\text{phys}} \mid \boldsymbol{x}_0) \propto \exp\big(-\mathcal{L}_{\text{guide}}(\boldsymbol{x}_0)\big), \tag{8.10.4}$$

por lo que el posterior queda así:

$$p(\boldsymbol{x}_0 \mid \boldsymbol{y}) \;\propto\; p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{c}) \exp\big(-\mathcal{L}_{\text{guide}}(\boldsymbol{x}_0)\big). \tag{8.10.5}$$

Esta es la forma descrita en el Capítulo 7: una distribución a priori aprendida combinada con restricciones de medición a través de una verosimilitud basada en la energía.

8.10.3 Diseño de la representación condicional

La distribución condicional a priori $p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{c})$ genera el movimiento del cuerpo dada una representación $\boldsymbol{c}$ del movimiento de la cabeza. ¿Cómo debería ser esta representación? Como se ha destacado a lo largo de este libro, la elección de la representación afecta de manera fundamental al aprendizaje y a la generalización.

[26] Para que esta descomposición sea válida, suponemos esencialmente que, dado $\boldsymbol{x}_0$ y $\boldsymbol{y}^{\text{hand}}, \boldsymbol{y}^{\text{phys}}$ es condicionalmente independiente de la pose de la cabeza $\boldsymbol{c}$.

Un enfoque ingenuo se basaría directamente en las poses brutas de la cabeza $\boldsymbol{h}_{1:T} \in \mathsf{SE}(3)^T$, es decir, la posición y la orientación absolutas en cada paso temporal. Esto no funciona bien. Para entender por qué, pensemos en qué propiedades debería cumplir la representación.

Invarianza espacial. La correspondencia entre el movimiento de la cabeza y el movimiento del cuerpo no debería depender del lugar concreto en el que se produzca el movimiento. Caminar en la cocina debería generar las mismas estimaciones de pose corporal que caminar en el salón. Lo único que importa es el patrón de movimiento local, no la posición absoluta.

El condicionamiento en poses absolutas viola este principio. Como ilustra la Figura 8.36, un movimiento corporal idéntico corresponde a diferentes trayectorias absolutas de la cabeza dependiendo de su ubicación en el mundo. Un modelo condicionado a las poses absolutas debe aprender por separado que «la cabeza en $(3, 2, 1)$ moviéndose hacia adelante» y «la cabeza en $(10, 5, 2)$ moviéndose hacia adelante» corresponden ambas a caminar, una carga innecesaria que perjudica la capacidad de generalización.

Invarianza temporal. El mapeo tampoco debe depender del momento en que se produzca el movimiento dentro de una ventana de observación. El procesamiento de los fotogramas 0–100 debe comportarse de la misma manera que el procesamiento de los fotogramas 50–150 si contienen el mismo movimiento subyacente. Esto descarta las magnitudes acumulativas, como el desplazamiento total desde el inicio de la secuencia, así como los índices de tiempo absolutos.

Se podría intentar canonizar respecto al primer paso de tiempo colocando el primer cuadro en el origen. Pero esto viola la invarianza temporal: diferentes segmentos temporales del mismo movimiento, canonizados a sus respectivos primeros fotogramas, producen trayectorias diferentes. El modelo ve entradas diferentes para el mismo movimiento corporal subyacente.

La solución: canonización por paso de tiempo. Para lograr la invarianza tanto espacial como temporal, EgoAllo define un sistema de coordenadas canónicas en *cada* paso de tiempo, en lugar de solo al inicio. La representación $\boldsymbol{c} = \{\boldsymbol{c}^1, \ldots, \boldsymbol{c}^T\}$ vincula:

- Movimiento relativo de la cabeza $\Delta \boldsymbol{T}^t_{\text{cabeza}}$ entre fotogramas consecutivos, que registra cómo se movió la cabeza desde $t-1$ hasta t.
- Pose canonizada por paso de tiempo $\boldsymbol{T}^t_{\text{canonical}}$: la orientación de la cabeza relativa a la gravedad y su altura sobre el suelo, eliminando así la posición en el plano del suelo y el giro horizontal (yaw).

Estas magnitudes son independientes del lugar del mundo en el que se produzca el movimiento y del intervalo temporal del que se extraiga. Dos secuencias con un movimiento local idéntico producen tokens de condicionamiento idénticos $\boldsymbol{c}$, invariantes respecto a la ubicación del movimiento en el mundo y al desplazamiento temporal seleccionado para su extracción.

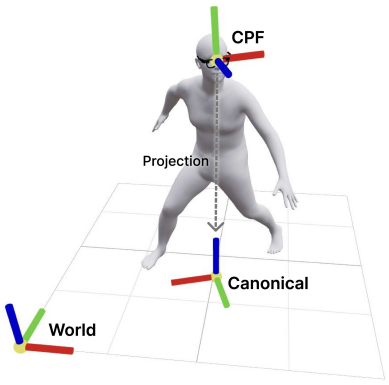


Figura 8.36: **Condicionamiento invariante del movimiento de la cabeza.** La representación del condicionamiento debe ser invariante tanto respecto a las transformaciones espaciales (el mismo movimiento en diferentes ubicaciones del mundo) como a los desplazamientos temporales (el mismo movimiento en diferentes tiempos absolutos). La canonización por paso de tiempo cumple ambas propiedades.

8.10.4 La distribución a priori condicional

La distribución a priori condicional $p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{c})$ es un modelo de difusión que genera secuencias de movimiento corporal condicionadas al movimiento de la cabeza como condición. La arquitectura sigue el diseño basado en transformers descrito en la Sección 8.4, adaptado para datos de movimiento secuenciales.

Arquitectura. El modelo toma tres entradas:

- La secuencia ruidosa de movimiento $\boldsymbol{x}_n$ en el paso de difusión n, tokenizada por paso temporal.
- El *embedding* del paso de tiempo de difusión, que indica el nivel de ruido.
- Los tokens de condicionamiento invariantes de la cabeza $\boldsymbol{c}$, uno por paso temporal.

Los tokens de la cabeza se procesan primero mediante un codificador compuesto por seis bloques de transformer con autoatención para generar representaciones contextualizadas. Estas condicionan un decodificador, compuesto por otros seis bloques de Transformer, que elimina el ruido de la secuencia de movimiento. El condicionamiento se introduce mediante atención cruzada: en cada capa del decodificador, los tokens de movimiento forman consultas que prestan atención a los tokens de la cabeza como claves y valores, lo que permite al modelo seleccionar dinámicamente qué aspectos del movimiento de la cabeza son relevantes para predecir cada articulación corporal en cada paso temporal. El modelo genera una estimación directa del movimiento limpio

$\boldsymbol{x}_0$, en lugar de predecir el ruido, parametrizado como rotaciones articulares en un sistema de coordenadas relativo a la cabeza.

Datos de entrenamiento. El modelo se entrena con AMASS [MGT+19], un conjunto de datos de captura de movimiento a gran escala agregando múltiples fuentes en un formato unificado SMPL-H. AMASS contiene diversos movimientos humanos —caminar, correr, bailar, sentarse, estirarse, hacer ejercicio— pero no incluye grabaciones de dispositivos montados en la cabeza. Las trayectorias de la cabeza se simulan extrayendo la posición y la orientación de la articulación de la cabeza a partir de los datos de captura de movimiento, como si un dispositivo estuviera fijado rígidamente a la cabeza.

Objetivo del entrenamiento. El entrenamiento utiliza la pérdida de *score matching* con eliminación de ruido del Capítulo 3:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\boldsymbol{x}_0,\boldsymbol{n}}\left[w_n \left\|D_\theta(\boldsymbol{x}_n, \boldsymbol{n}, \boldsymbol{c}) - \boldsymbol{x}_0\right\|^2\right], \tag{8.10.6}$$

donde D_θ es la red de eliminación de ruido, w_n son pesos que dependen del ruido y $\boldsymbol{x}_n$ se obtiene añadiendo ruido al movimiento limpio $\boldsymbol{x}_0$ según el proceso de difusión hacia adelante. El modelo aprende a predecir el movimiento limpio a partir del movimiento ruidoso, condicionado a las trayectorias de la cabeza.

8.10.5 Incorporación de medidas mediante guiado

La distribución a priori condicional refleja la relación entre el movimiento de la cabeza y el movimiento del cuerpo, pero no utiliza observaciones de imágenes ni restricciones físicas. Para incorporarlas, EgoAllo define una energía de guía $\mathcal{L}_{\text{guide}}$ que mide la coherencia entre una hipótesis de movimiento y las observaciones.

La energía de guía. La energía de guía total combina varios términos:

$$\mathcal{L}_{\text{guide}}(\boldsymbol{x}_0) = \lambda_{3D}\mathcal{L}_{3D} + \lambda_{2D}\mathcal{L}_{2D} + \lambda_{\text{skate}}\mathcal{L}_{\text{skate}} + \lambda_{\text{prior}}\mathcal{L}_{\text{prior}}, \tag{8.10.7}$$

donde:

- $\mathcal{L}_{3D}$: Alineación entre los puntos clave estimados de la mano y las detecciones 3D de la mano obtenidas con HaMeR, en el sistema global de coordenadas.
- $\mathcal{L}_{2D}$: Error de reproyección. Los puntos clave de la mano estimados, al proyectarse sobre la imagen, deben coincidir con las detecciones en 2D.
- $\mathcal{L}_{\text{skate}}$: Penalización de deslizamiento de pies (foot skating penalty). Los pies no deben deslizarse cuando están en contacto con el suelo.
- $\mathcal{L}_{\text{prior}}$: Regularización suave hacia la predicción del prior de difusión, evitando que la guía se aleje demasiado de los movimientos plausibles.

Siguiendo la interpretación basada en la energía del Capítulo 7, esta energía de guía corresponde a una log-verosimilitud negativa: $p(\boldsymbol{y} \mid \boldsymbol{x}_0) \propto \exp(-\mathcal{L}_{\text{guide}}(\boldsymbol{x}_0))$. Una energía baja implica consistencia con las observaciones; una energía alta indica inconsistencia.

Por qué es importante la reproyección. Los estimadores de manos de cuadro único, como HaMeR, generan poses de manos en 3D a partir de imágenes monoculares, pero presentan ambigüedad de escala y profundidad. Una mano pequeña cerca de la cámara se ve idéntica a una mano grande que está lejos. Por lo tanto, las coordenadas en 3D no son confiables en términos absolutos, incluso cuando la pose local de la mano, como la articulación de los dedos, es precisa.

La pérdida de reproyección 2D resuelve este problema mediante la supervisión en el espacio de la imagen, donde la mano fue realmente observada. Las posiciones 3D estimadas de la mano se vuelven a proyectar en la imagen y se comparan con los puntos clave en 2D detectados. Esto elude la ambigüedad de profundidad manteniendo al mismo tiempo una señal útil sobre la localización de la mano.

8.10.6 Algoritmo de muestreo condicional

El marco de coincidencia de mediciones del Capítulo 7 nos indica cómo realizar un muestreo a partir de la distribución a posteriori: hay que modificar el *denoiser* para incluir una corrección de gradiente hacia la coherencia de las mediciones. Recordemos que el *denoiser* condicional se descompone de la siguiente manera:

$$\mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_n, \boldsymbol{y}] = \mathbb{E}[\boldsymbol{x}_0 \mid \boldsymbol{x}_n] + \sigma_n^2 \nabla_{\boldsymbol{x}_n} \log p(\boldsymbol{y} \mid \boldsymbol{x}_n), \tag{8.10.8}$$

donde el primer término corresponde al *denoiser* a priori y el segundo, a la corrección de ajuste de mediciones.

En nuestro contexto, el *denoiser* previo está condicionado por el movimiento de la cabeza: $D_n^{\text{cabeza}}(\cdot; \boldsymbol{c})$. La corrección de la medida utiliza la energía de guía. El *denoiser* condicional completo queda así:

$$D_n^{\text{guía}}(\boldsymbol{z}) = D_n^{\text{cabeza}}(\boldsymbol{z}; \boldsymbol{c}) - \sigma_n^2 \nabla_{\boldsymbol{z}} \mathcal{L}_{\text{guide}}(\boldsymbol{x}_0(\boldsymbol{z})), \tag{8.10.9}$$

utilizando la relación $\nabla \log p(\boldsymbol{y} \mid \boldsymbol{x}) = -\nabla \mathcal{L}_{\text{guía}}(\boldsymbol{x})$.

Optimización alternada. En la práctica, en lugar de calcular los gradientes a través de toda la energía de guía en cada paso, EgoAllo alterna entre dos operaciones:

1. **Paso previo de eliminación de ruido.** Aplicar una actualización al estilo DDIM utilizando el eliminador de ruido condicionado por la cabeza $D_n^{\text{head}}(\cdot; \boldsymbol{c})$ para obtener una propuesta $\tilde{\boldsymbol{x}}_{n-1}$. DDIM (Modelos implícitos de difusión con eliminación de ruido) proporciona un procedimiento de

muestreo determinista que mapea muestras ruidosas hacia otras más limpias; a diferencia de los muestreadores estocásticos, cada paso de eliminación de ruido es una función determinista del estado actual, lo que permite un muestreo más rápido con menos pasos.

2. **Paso de ajuste de medidas.** Refinar la propuesta resolviendo un problema de optimización proximal:

$$\boldsymbol{x}_{n-1} = \arg\min_{\boldsymbol{x}} \left\{ \frac{1}{2} \|\boldsymbol{x} - \tilde{\boldsymbol{x}}_{n-1}\|^2 + \gamma_n \, \mathcal{L}_{\text{guide}}(\boldsymbol{x}) \right\}, \tag{8.10.10}$$

donde γ_n es un parámetro de tamaño de paso (step-size). Esto localiza un movimiento cercano a la predicción de la distribución a priori, pero más coherente con las observaciones.

La optimización proximal se resuelve mediante Levenberg-Marquardt, un algoritmo clásico para problemas de mínimos cuadrados no lineales. Este interpola entre el descenso de gradiente y la optimización de Gauss-Newton, adaptando el tamaño de su paso en función de lo bien que la linealización aproxima al objetivo real. Esto es adecuado para la estimación de parámetros SMPL-H, donde el objetivo implica una cinemática directa diferenciable. Bastan unas pocas iteraciones en cada paso de eliminación de ruido.

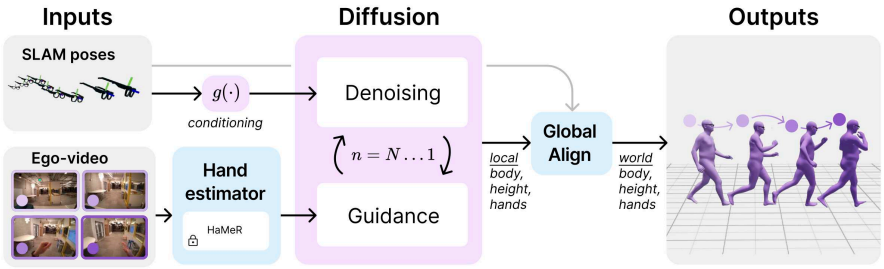


Figura 8.37: **Resumen de las componentes del marco EgoAllo**. El modelo de difusión se limita a los parámetros corporales locales. Se utiliza una parametrización invariante g(·) de las poses de SLAM (cabeza) para condicionar un modelo de difusión. Estas pueden situarse en el sistema de coordenadas global mediante una alineación global con las poses de entrada. Cuando está disponible, se utiliza un vídeo egocéntrico para la detección de manos, por ejemplo, mediante HaMeR [PSR+24], que puede incorporarse a las muestras mediante la guía.

Alineación global. El modelo de difusión opera en un sistema de coordenadas relativo a la cabeza, prediciendo la postura del cuerpo en relación con la cabeza, en lugar de hacerlo en coordenadas absolutas de mundo. Tras el último paso de

eliminación de ruido, EgoAllo ancla el cuerpo en el mundo componiéndolo con las posturas de la cabeza obtenidas por SLAM:

$$\boldsymbol{T}^t_{\text{world,root}} = \boldsymbol{T}^t_{\text{world,head}} \cdot \boldsymbol{T}^{(\theta^t,\beta)}_{\text{head,root}}, \tag{8.10.11}$$

donde $\boldsymbol{T}_{\text{head,root}}$ se calcula mediante cinemática directa a partir de la pose y la forma estimadas. Esto garantiza que el cuerpo esté correctamente anclado: los pies en el suelo y la cabeza alineada con la cámara.

Manejo de secuencias largas. El modelo de difusión se entrena con secuencias de longitud fija (32-128 fotogramas). Para grabaciones más largas, EgoAllo utiliza el enfoque MultiDiffusion [BYL+23]: la secuencia se divide en ventanas superpuestas y se elimina el ruido en paralelo. Tras cada paso de eliminación de ruido, los resultados de las regiones de superposición se promedian antes de proceder al siguiente paso. Esta combinación paso a paso garantiza que las ventanas mantengan coherencia entre sí durante el proceso de difusión, lo que produce un resultado temporalmente coherente sin saltos o discontinuidades en los bordes de las ventanas.

8.10.7 Conclusiones de los experimentos

EgoAllo se evalúa en tres conjuntos de datos: AMASS [MGT+19] (captura de movimiento con trayectorias sintéticas de la cabeza), RICH [HYH+22] (captura de movimiento con interacciones complejas entre personas y el entorno), y Aria Digital Twin [PCY+23] (grabaciones reales de las gafas del Proyecto Aria con poses corporales de referencia). Para la estimación de manos, se utiliza también EgoExo4D [GWT+23].

La representación importa. La Tabla 8.15 muestra el efecto de diferentes representaciones de condicionamiento en la precisión de la estimación corporal, comparando cinco parametrizaciones con diferentes propiedades de invarianza:

- **Invariante por paso de tiempo**: Representación descrita en la sección 8.10.3, con invarianza tanto espacial como temporal.
- **Absoluto + relativo**: Posturas absolutas augmentadas con el movimiento relativo en el marco local.
- **Absoluto + deltas globales**: Posturas absolutas con desplazamientos (deltas) de posición en coordenadas de mundo.
- **Canonización de secuencias**: alineación al primer cuadro, proporcionando invarianza espacial, pero no temporal.
- **Absoluto**: Poses brutas $\mathsf{SE}(3)$ sin propiedades de invarianza.

Tabla 8.15: **Efecto de la representación de condicionamiento en AMASS.** Las diferentes parametrizaciones del condicionamiento del movimiento de la cabeza dan lugar a una precisión de estimación muy distinta. La representación invariante en cada paso de tiempo, que cumple tanto con la invarianza espacial como con la temporal, es la que ofrece mejores resultados. MPJPE es el error medio de posición por articulación en milímetros.

	Long. Sec 32		**Long. Sec 128**	
Condicionamiento	MPJPE	Efecto	MPJPE	Efecto
Invariante temporal	**129.8**	—	**119.7**	—
Absoluto + relativo	133.0	+2.4%	124.5	+4.0%
Absoluto + deltas globales	136.2	+4.9%	127.4	+6.4%
Canonización de secuencias	153.1	+17.9%	134.0	+11.9%
Absoluto	159.9	+23.2%	148.3	+23.9%

Los resultados muestran que la representación de condicionamiento tiene un impacto dramático en la precisión, con diferencias de hasta un 24% en el MPJPE entre la mejor y la peor elección. Esto refuerza un tema central de este libro: la selección de la representación afecta fundamentalmente al aprendizaje y a la generalización.

La estimación de la altura permite el anclaje al suelo. La Tabla 8.16 compara la estimación completa (incluida la altura) con una variante sin estimación de la altura. La estimación de la altura mejora el MPJPE en un 6-7% y es clave para el anclaje: sin ella, los cuerpos estimados tienden a «flotar» por encima del suelo, ya que el sistema no puede determinar la posición vertical correcta. La métrica de anclaje al suelo (GND) mide si los pies estimados tocan el suelo en algún momento, y la estimación de la altura es esencial para un anclaje correcto.

Tabla 8.16: **Resultados de la estimación corporal en distintos conjuntos de datos.** MPJPE y PA-MPJPE en milímetros; GND es la proporción de secuencias en las que los pies tocan el suelo. La estimación de la altura facilita el anclaje.

Datos	**Variante**	**MPJPE↓**	**PA-MPJPE↓**	**GND↑**
AMASS	Completo (con altura)	**119.7**	**101.1**	**1.00**
	Sin altura	128.1	110.3	0.98
RICH	Completo (con altura)	**176.2**	**160.1**	**0.96**
	Sin altura	185.7	169.9	0.82
ADT	Completo (con altura)	**155.1**	**129.3**	0.94
	Sin altura	163.7	140.0	**0.96**

Tabla 8.17: **Estimación de manos en EgoExo4D.** MPJPE mide la precisión en el marco global. PA-MPJPE mide la precisión de la pose local. El contexto corporal mejora notablemente la localización en el marco global al tiempo que mantiene la calidad de la pose local.

Método	**MPJPE↓**	**PA-MPJPE↓**
HaMeR (monocular, sin cuerpo)	237.9	**13.0**
Con cuerpo (reproyección 2D)	131.5	14.7
Con cuerpo (muñeca 3D por estéreo)	**60.1**	14.4

El contexto del cuerpo mejora la estimación de la mano. Un hallazgo destacable es la relación simbiótica entre la estimación corporal y la manual. Los estimadores monoculares de mano de un solo fotograma, como HaMeR, producen posturas locales precisas de la mano, incluida la articulación de los dedos, pero no pueden determinar la posición 3D absoluta de la mano debido a la ambigüedad de escala y profundidad. Al estimar primero el cuerpo completo, ganamos restricciones cinemáticas: la mano está unida a la muñeca, que se conecta con el antebrazo, el codo, el húmero y el torso. Esta cadena proporciona el contexto para la localización global de la mano. La Figura 8.38 ilustra algunos resultados cualitativos.

La Tabla 8.17 cuantifica este efecto en el conjunto de datos EgoExo4D. Las estimaciones sin procesar de HaMeR presentan un error de 237,9 mm en las coordenadas de mundo. Al añadir el contexto corporal, esto se reduce a 131,5 mm, lo que representa una mejora del 45%, incluso utilizando solo observaciones monoculares de las manos. Cuando se dispone de observaciones 3D adicionales de la muñeca procedentes de cámaras estereoscópicas, los errores disminuyen aún más, hasta alcanzar los 60,1 mm.

La relación es bidireccional: las observaciones de la mano también mejoran la estimación corporal. Cuando se incorpora la información de la mano como guía durante el muestreo, el MPJPE corporal en AMASS mejora de 119,7 mm a 78,8 mm, lo que supone una reducción del 34%. Las restricciones de medición adicionales estrechan la distribución posterior y producen estimaciones más precisas a lo largo de toda la cadena cinemática.

Resultados cualitativos. La Figura 8.39 muestra las posturas corporales estimadas a partir de grabaciones egocéntricas del mundo real realizadas con las gafas del Proyecto Aria. Los cuerpos estimados están correctamente anclados en el suelo: los pies tocan el suelo de manera adecuada, la altura del cuerpo coincide con el entorno y las posturas son coherentes con el movimiento observado de la cabeza en diversas actividades, como caminar, sentarse y manipular objetos.

Figura 8.38: **El contexto corporal mejora la estimación de la mano.** Azul: estimaciones monoculares de la mano obtenidas con HaMeR, que presentan una pose local precisa pero una posición incorrecta en el marco mundial debido a la ambigüedad de escala/profundidad. Morado: estimaciones con contexto corporal, correctamente ancladas en el marco mundial.

Figura 8.39: **Resultados cualitativos.** Posturas corporales estimadas a partir de grabaciones egocéntricas del mundo real, visualizadas en reconstrucciones de escenas en 3D. El método produce posturas físicamente plausibles con un anclaje adecuado en diversas actividades.

8.10.8 Discusión

Esta aplicación demuestra cómo el marco de inferencia condicional del Capítulo 7 se extiende más allá de las imágenes para abarcar salidas estructuradas y articuladas, como el movimiento humano.

El diseño de la representación es crucial. Un tema recurrente a lo largo de este libro es que las decisiones de representación influyen crucialmente en el aprendizaje y la generalización. Este principio resulta especialmente evidente aquí. Al analizar qué invarianzas debe cumplir la representación de condicionamiento, derivamos una parametrización que supera sustancialmente a las alternativas. La invarianza espacial garantiza la independencia respecto a la ubicación en el mundo; la invarianza temporal garantiza la independencia respecto al desplazamiento temporal. La lección: antes de entrenar un modelo, es crucial considerar cuidadosamente qué estructura debe codificar la representación.

El patrón «Prior más guía» generaliza. El procedimiento de muestreo aplica directamente el marco de conciliación de mediciones: prior aprendido —en este caso, el modelo de difusión condicionado por la cabeza— se combina con restricciones de medición, como observaciones de la mano y restricciones físicas, mediante un muestreo guiado. Este mismo patrón de «prior más guía» se repite en diversos ámbitos, desde la generación de imágenes hasta la estimación de movimiento. Las componentes son modulares: podemos cambiar diferentes priors o términos de guía sin alterar el marco general.

La estimación holística supera a los *pipeline* modulares. La relación simbiótica entre la estimación del cuerpo y la de la mano sugiere que la inferencia conjunta sobre toda la cadena cinemática supera a la estimación independiente de las componentes. El movimiento de la cabeza informa la postura del cuerpo; la postura del cuerpo proporciona contexto para la localización de la mano; y las observaciones de la mano refinan ambas estimaciones (cuerpo y mano). Este beneficio mutuo surge en forma natural de la formulación a posteriori, donde todas las variables están acopladas a través de la función de verosimilitud y la distribución a priori.

Limitaciones. Persisten varias limitaciones. En primer lugar, el método requiere un SLAM preciso con alineación de escala métrica y gravedad; los errores en la estimación de la pose de la cabeza se propagan directamente a las estimaciones del cuerpo. En segundo lugar, la generalización está limitada por la diversidad de los datos de entrenamiento, por lo que las posturas o actividades inusuales, no representadas en AMASS, pueden estimarse en forma imprecisa. En tercer lugar, las manos solo se pueden refinar si son visibles en la vista egocéntrica; las manos ocultas dependen exclusivamente del prior corporal. Por último, el método asume suelos planos; las escaleras o los terrenos irregulares requerirían un modelado adicional.

Problemas abiertos. Entre las direcciones prometedoras figura la incorporación de la geometría de la escena para lograr un mejor anclaje físico, de modo que las manos tengan contacto con las superficies y los cuerpos eviten los obstáculos. Otras direcciones incluyen la habilitación de la inferencia en tiempo real para aplicaciones de realidad aumentada o virtual en vivo y la extensión a escenarios con múltiples personas en los que el usuario interactúe con otros. El marco es lo suficientemente flexible como para acoger estas extensiones mediante términos de guía adicionales o priores modificados.

8.11 Representación y generación del lenguaje natural

Hasta ahora, hemos mostrado cómo aprender representaciones profundas con estructura para las distribuciones de datos asociadas con la percepción visual y los movimientos corporales. En gran medida, las representaciones estructuradas así aprendidas pueden considerarse como recuerdos almacenados en la corteza visual y la corteza motora del cerebro. En conjunto, estos recuerdos pueden denominarse de manera general un «modelo del mundo», o simplemente un «modelo mundial» — que nos permite tanto percibir el mundo como realizar acciones. Obsérvese que tanto los seres humanos como los animales tienen la capacidad de aprender y desarrollar recuerdos del mundo y de su propio cuerpo, que registran lo que se puede predecir a partir de lo percibido, así como las habilidades motoras cruciales para su supervivencia en el entorno.

Además de ese modelo del mundo que abarca el entorno físico y el movimiento corporal, el ser humano ha desarrollado los lenguajes (hablados y escritos) para poder comunicar a los demás los conocimientos adquiridos sobre el mundo; las áreas de Broca y de Wernicke de nuestro cerebro son conocidas por su función en el procesamiento del habla y la comprensión del lenguaje, respectivamente. En gran medida, los idiomas registran una pequeña, pero importante, fracción de nuestros modelos del mundo que vale la pena compartir entre nosotros. Por lo tanto, los idiomas naturales, al ser un tipo de datos muy especial, deben tener estructuras muy especiales que codifican una información muy rica. Si las máquinas fueran capaces de aprender las estructuras de los idiomas naturales, podrían comunicarse mejor con los seres humanos.

Por lo tanto, en esta sección mostramos cómo aplicar los métodos presentados en este libro a la tarea del *modelado del lenguaje*. O, más concretamente, cómo entrenar modelos de lenguaje a gran escala (LLM) para que aprendan la distribución de los lenguajes naturales y, a continuación, los generen. La tarea y su configuración son prácticamente las mismas utilizadas para entrenar GPT-2 y muchos otros modelos de lenguaje.

8.11.1 Datos

Los datos que utilizaremos para investigar el rendimiento de CRATE en tareas lingüísticas serán OpenWebText (OWT) [GC19], una reproducción de

código abierto del conjunto de datos WebText, aún sin publicar, utilizado por OpenAI para entrenar GPT2. Cada muestra de OWT es un documento web, normalmente procedente de fuentes de alta calidad, como páginas web, blogs, artículos o debates en línea, que está escrito en lenguaje natural bien formado. El conjunto de datos OpenWebText contiene alrededor de 8,01 millones de documentos de diferentes longitudes, lo que suma un total de unos 41,70 GB de texto. Para la evaluación, utilizaremos varios conjuntos de datos, tales como WikiText [MXB+16][27], LAMBADA [PKL+16][28], y PTB [MSM93]. PTB y OWT son, en general, más sencillos en comparación con otros conjuntos de datos. PTB se centra en textos periodísticos más sencillos, ideales para el modelado del lenguaje tradicional, mientras que OWT es variado e informal, abarca diversos temas y presenta una menor complejidad en la estructura del lenguaje o en las dependencias de largo alcance. WikiText, con su estructura formal y su contenido de dominio específico, requiere una comprensión más compleja que OWT, pero sigue siendo manejable. LAMBADA es el más exigente, ya que implica dependencias de largo alcance, lo que exige que el modelo comprenda información contextual más amplia para completar las oraciones con precisión.

En un plano más formal, nuestros datos $\boldsymbol{X}$ serán texto, es decir, cadenas de caracteres; definimos $\mathcal{T}$ como el conjunto de todas las cadenas de caracteres.

8.11.2 Tarea y objetivo

Para el preentrenamiento de modelos de lenguaje causales, la idea es que queremos *entrenar el modelo para que genere texto similar al que produciría un humano*. La forma más popular de hacerlo, con diferencia, es utilizar un proceso de entrenamiento en dos etapas:[29]

- *En primer lugar*, queremos *aprender* una forma de codificar de manera óptima los documentos como una secuencia de cadenas básicas («bloques de construcción») denominadas *tokens*. A esto se le llama *tokenización*, y construimos un *tokenizador*.
- *En segundo lugar*, queremos *aprender* una forma de *predecir la distribución de un token dados todos los tokens anteriores*. A esto se le denominada *predicción del siguiente token*, y para ello construimos un *modelo de lenguaje*.

En realidad, este procedimiento se remonta a Markov, quien fue el primero en darse cuenta de que el lenguaje natural podía modelarse mediante la estructura

[27] Para WikiText2 y WikiText103 [MXB+16], las divisiones de prueba son las mismas, por lo que las fusionamos en un único conjunto de datos denominado WikiText.

[28] Para obtener la precisión en el conjunto de datos LAMBADA, utilizamos la decodificación codiciosa.

[29] El entrenamiento de los modelos de lenguaje modernos cuenta con varios pasos adicionales que requieren diferentes distribuciones de datos y enfoques algorítmicos. Sin embargo, entrenar un modelo para que simplemente imite la escritura humana solo requiere estos pocos pasos que se presentan aquí.

de la cadena de Markov que lleva su nombre [Mar06], con una tokenización adecuada; y posteriormente a Shannon, quien propuso aplicar esta misma configuración de modelado del lenguaje con un tokenizador a nivel de caracteres (es decir, cada carácter es un token) y los denominados «n-gramas» (es decir, una tabla de consulta explícita, calculada a partir de los datos de entrenamiento, que indica la distribución de un token dado los n tokens anteriores) en lugar del modelo de lenguaje [Sha48].[30]

Entrenamiento de un tokenizador

Crear un tokenizador equivale a construir un vocabulario $\mathcal{V}$, que es un conjunto de tokens y tiene un tamaño predefinido V. Existen varios métodos para hacerlo. Un algoritmo muy utilizado es el conocido como Byte Pair Encoding (BPE), que puede describirse como:

- Empieza con una lista de todos los caracteres únicos de tus datos de entrenamiento y sus frecuencias. Asegúrate de que haya menos de V caracteres de este tipo y añade cada carácter como una cadena independiente («token») al vocabulario junto con su frecuencia.
- Hasta que haya V tokens en el vocabulario:
  - Crear un token tomando los dos tokens existentes más frecuentes y fusionándolos.
  - Calcular la frecuencia de este token en el conjunto de datos.
  - Añadirlo al vocabulario (junto con su frecuencia).
- En este punto, la información sobre la frecuencia ya no es necesaria y puede descartarse.

El proceso general del BPE se muestra en la Figura 8.40. Cabe señalar que este procedimiento es una modificación de un procedimiento clásico de compresión basado en la teoría de la información para *aprender una codificación sin pérdidas* de un flujo de bytes (como el texto) y, como tal, puede interpretarse como la búsqueda de una compresión sin pérdidas óptima de los datos. Obsérvese que esto es posible porque (a diferencia de las imágenes), los datos aquí son fundamentalmente discretos y libres de ruido. Una vez creado dicho vocabulario, un tokenizador puede dividir un documento en tokens (es decir, «tokenizarlo»). BPE utiliza un procedimiento similar al del entrenamiento para tokenizar los datos:

- Divide el documento en una larga lista de tokens de un solo carácter. Es decir, si el documento es «Hello», la lista inicial es «H», «e», «l», «l», «o».

[30] Un estudio reciente [LMZ+24] sobre el escalado de modelos de n-gramas ha demostrado que son capaces de modelar el texto con bastante precisión para valores grandes de n, pero, por supuesto, la memoria necesaria para almacenar dicha tabla de consulta es del orden de V^n y, por lo tanto, completamente inmanejable.

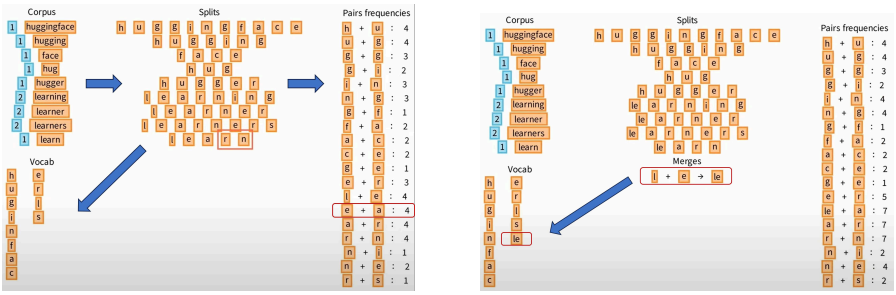


Figura 8.40: **El proceso de tokenización de datos de texto mediante BPE.** (Crédito de la imagen: https://huggingface.co/learn/nlp-course/chapter6/5). (Izquierda) Comenzamos por analizar el corpus de texto dado y construyendo un vocabulario inicial compuesto por caracteres individuales (o bytes en el caso del BPE a nivel de bytes). A continuación, calculamos las frecuencias de los pares de caracteres adyacentes en el corpus. Esto implica analizar todo el texto y contar cuántas veces aparece cada secuencia de dos caracteres (bigram). (Derecha) Tras calcular las frecuencias de los pares de caracteres adyacentes, identificamos el par más frecuente en el corpus. A continuación, este par se fusiona en una nueva unidad de subpalabra, que se añade al vocabulario como un único token. Este proceso se repite en forma iterativa hasta que se alcanza el tamaño de vocabulario predefinido.

- Siempre que dos tokens adyacentes puedan concatenarse y su concatenación sea otro token, lo hacemos, es decir, reemplazamos este par de tokens por el token fusionado. Concretamente, si «He» es un token del vocabulario, «H», «e», «l», «l», «o» se convertiría en «He», «l», «l», «o».

- Repite el proceso anterior hasta que ya no se puedan realizar más fusiones. En ese momento, el documento queda particionado en la lista (secuencia) final de tokens.

Hay muchas consideraciones prácticas y relacionadas con la eficiencia que hay que tener en cuenta durante la tokenización. El algoritmo anterior, tal y como se presenta, está *muy lejos de ser óptimo* si se implementa de manera ingenua, por ejemplo. No profundizaremos demasiado en este tema; hay muchos recursos en línea para aprender más, como los tutoriales de HuggingFace.

Por ejemplo, cada token tiene un *índice* correspondiente, que no es más que su índice en el vocabulario (que, al fin y al cabo, es simplemente una lista de longitud V). Por lo tanto, la salida de la mayoría de los tokenizadores es una lista de índices, es decir, un elemento de $[V]^*$. Hay que tener en cuenta que estos corresponden a subcadenas del documento original, como se ha mostrado anteriormente.

Una vez que se ha entrenado un tokenizador, cualquier modelo de lenguaje puede utilizarlo como una caja negra. Por ejemplo, muchos modelos tienen el mismo tokenizador (basado en OpenAI) construido sobre la base de la biblioteca `tiktoken`. En el resto de esta sección, utilizaremos dicho tokenizador fijo y

precompilado para todo, y así identificaremos cada documento de texto $\boldsymbol{X} \in \mathcal{T}$ con su versión tokenizada en $[V]^*$. Por lo tanto, podemos considerar el espacio textual $\mathcal{T}$ como *igual* al espacio de secuencias de tokens $[V]^*$ (sin perder nada esencial).

Entrenamiento de modelos de lenguaje

Una vez que tenemos cada documento como una secuencia de tokens $\boldsymbol{X} \in [V]^N \subseteq [V]^* = \mathcal{T}$, queremos realizar la predicción del siguiente token. Es decir, dado un *contexto* $\boldsymbol{X}_{:n} \in [V]^n$ (es decir, los primeros n tokens $\boldsymbol{x}_1, \dots, \boldsymbol{x}_n \in [V]$ del documento)[31], queremos predecir el token $\boldsymbol{x}_{n+1} \in [V]$ en la posición $n+1$. Para ello, calculamos la característica agregada de $\boldsymbol{X}_{:n}$ mediante $\boldsymbol{z}_\theta(\boldsymbol{X}_{:n}) \doteq (f_\theta^{\text{ext}} \circ f_\theta)(\boldsymbol{X}_{:n}) \in \mathbb{R}^d$, y utilizamos un cabezal de clasificación $h_\theta \colon \mathbb{R}^d \to \Delta_V$ (implementado como una capa lineal, una MLP o algo un poco más complejo) para proyectar esta característica en el simplex de probabilidad de dimensión V Δ_V. Esta proyección $\boldsymbol{p}_\theta(\boldsymbol{X}_{:n}) \doteq h_\theta(\boldsymbol{z}_\theta(\boldsymbol{X}_{:n}))$ sirve como una distribución de probabilidad estimada del siguiente token. Entonces, utilizando la notación $\mathbf{1}(\boldsymbol{x}_{n+1}) \in \Delta_V$ para denotar el vector que toma 1 en la componente $\boldsymbol{x}_{n+1}$ y 0 en el resto, la pérdida del modelado de lenguaje causal es

$$\min_\theta \left\{ \mathcal{L}_{\text{CLM}}(\theta) \doteq \mathbb{E}_{\boldsymbol{X}} \left[\frac{1}{N-1} \sum_{n=1}^{N-1} \text{CE}(\mathbf{1}(\boldsymbol{x}_{n+1}), \boldsymbol{p}_\theta(\boldsymbol{X}_{:n})) \right] \right\} \tag{8.11.1}$$

Nótese qué similar que es esto a una pérdida de clasificación (por ejemplo, para imágenes); se utiliza la entropía cruzada y se intenta alinear un vector de probabilidades predichas con la verdad de referencia (ground truth). La principal diferencia entre estas dos pérdidas es que, en este caso, calculamos la pérdida sobre una secuencia completa, en la que cada predicción está correlacionada con las demás (a diferencia del caso de clasificación de variables independientes e idénticamente distribuidas).

La optimización de esta pérdida se denomina habitualmente «preentrenamiento» en la comunidad de modelos de lenguaje (en contraposición al «postentrenamiento» y, más recientemente, al «entrenamiento intermedio», que son metodologías para modificar un predictor del siguiente token con el fin de realizar tareas útiles).

Nota al margen: ¿Por qué el primer término de (8.11.1) predice $\mathbf{1}(\boldsymbol{x}_2)$, y no hay ningún término que mida la pérdida para predecir el primer token? Se debe a que si quisiéramos predecir el primer token, tendríamos la *secuencia vacía* como contexto y, por lo tanto, haríamos esta predicción del primer token utilizando un mecanismo cualitativamente diferente al que se aplica a los demás tokens. Así que, en realidad, este modelo no está entrenado para predecir el *primer* token de ningún documento. La razón por la que no supone un problema es un detalle de implementación del tokenizador: a menudo, tras crear el tokenizador, insertamos un token *especial* en su vocabulario, denominado token de *inicio de*

[31] Obsérvese la incongruencia con la notación de Python: aquí la notación *incluye* el índice n.

cadena (o documento) y etiquetado como `<|bos|>`.[32] A continuación, mientras procesamos cada documento, añadimos el token `<|bos|>` al principio de la secuencia de tokens del documento, aumentando la longitud de la secuencia tokenizada en 1. Por lo tanto, el objetivo de modelado de lenguaje causal anterior tiene un término que implica intentar predecir el primer token del documento dado únicamente el token `<|bos|>` como contexto, por lo que es una pérdida conceptualmente correcta.

8.11.3 Arquitectura: CRATE causal

En cuanto a la arquitectura, utilizamos un transformer estándar al estilo GPT-2, sustituyendo las capas del transformer por capas CRATE.[33] Para completar la información, especificamos aquí la arquitectura.

Incrustación. Primero incrustamos la secuencia de tokens $\boldsymbol{X} \in [V]^N$ en el espacio euclidiano. Esto se suele hacer asociando cada índice en $[V]$ con un vector en $\mathbb{R}^d$ utilizando una matriz *masiva*[34] $\boldsymbol{E} \in \mathbb{R}^{V \times d}$, y formando directamente la secuencia $[\boldsymbol{E}_{\boldsymbol{x}_1}, \dots, \boldsymbol{E}_{\boldsymbol{x}_N}] \in \mathbb{R}^{d \times N}$. El mapa de *embedding* completo f_θ^{emb} también aplica una codificación posicional $\boldsymbol{E}^{\text{pos}} \in \mathbb{R}^{d \times N_{\max}}$, donde $N_{\max}$ es el número máximo de tokens que es posible procesar,[35] lo que da como resultado el mapa de *embedding*

$$f_\theta^{\text{emb}}(\boldsymbol{X}) \doteq [\boldsymbol{E}_{\boldsymbol{x}_1}, \dots, \boldsymbol{E}_{\boldsymbol{x}_N}] + \boldsymbol{E}^{\text{pos}}_{:N} \tag{8.11.2}$$

Los parámetros $\boldsymbol{E}$ y $\boldsymbol{E}^{\text{pos}}$ se pueden entrenar directamente. Dado que $\boldsymbol{E}$ es tan grande (y la actualización del gradiente es muy esparsa con respecto a él, ya que solo se utiliza una pequeña fracción del vocabulario en cada muestra), se utiliza software especializado para garantizar que las actualizaciones de memoria no sean demasiado onerosas. Obsérvese también que no utilizamos un token de clase como en las otras secciones; más adelante se profundizará en este tema.

Red troncal. Procesamos los *embeddings* utilizando un *backbone* similar a CRATE que utiliza enmascaramiento causal. Para entender el enmascaramiento causal, consideremos la pérdida de modelado causal del lenguaje $\mathcal{L}_{\text{CLM}}$ definida

[32] Por lo general, existen varios tokens especiales para distintos fines. El texto existente que contiene los tokens especiales se procesa de manera especial.

[33] En contravención directa de las convenciones de este libro y de las de muchas otras comunidades, la comunidad de PLN denomina a estos transformers al estilo GPT-2 (que abarcan casi todos los LLM actuales) transformers «solo decodificadores». Los transformers «solo codificador» tienen una arquitectura diferente, mientras que los transformers «codificador-decodificador» concatenan un transformer «solo codificador» con un transformer «solo decodificador». ¡Y esto a pesar de que los transformers «solo decodificador» *también* calculan una codificación de los datos!

[34] Por «masiva» nos referimos a que dicha estructura suele constituir una gran fracción del tamaño total del modelo de lenguaje.

[35] Los métodos modernos de codificación posicional han resuelto este problema y permiten (en teoría) una extrapolación infinita, pero dichos métodos son más complejos de desarrollar, y por razones de simplicidad aquí solo introducimos la codificación posicional aditiva absoluta.

en (8.11.1). La implementación más ingenua nos obligaría a calcular la pasada hacia adelante N veces para poder realizar una sola retropropagación. Obviamente, esto resulta extremadamente ineficiente, ya que N suele ser del orden de miles. Para poder escalar el entrenamiento con esta pérdida de manera eficiente, imponemos una restricción *causal*, es decir,

$$\boldsymbol{Z}_\theta(\boldsymbol{X}_{:n}) = \boldsymbol{Z}_\theta(\boldsymbol{X})_{:n} \tag{8.11.3}$$

es decir, las n columnas de las características de los tokens $\boldsymbol{Z}_\theta(\boldsymbol{X}_{:n}) \in \mathbb{R}^{d\times n}$ deben ser las mismas que las primeras n columnas de las características de los tokens $\boldsymbol{Z}_\theta(\boldsymbol{X}) \in \mathbb{R}^{d\times N}$, independientemente de los valores positivos de n y N tales que $N \geq n$. En efecto, esto significa que podemos aplicar el *backbone una vez* a toda la secuencia y calcular $\boldsymbol{Z}_\theta(\boldsymbol{X})$, y luego aplicar f_θ^{ext} a cada subconjunto creciente $\boldsymbol{Z}_\theta(\boldsymbol{X}_{:n}) = \boldsymbol{Z}_\theta(\boldsymbol{X})_{:n}$ a medida que n crece hasta la longitud de la secuencia N. Entonces podemos emplearlos todos para calcular la pérdida.

Ahora que queremos una arquitectura causal para el *backbone*, ¿cómo podemos conseguirla? Dado que el MLP y las normalizaciones de capa dentro de cada capa del transformer afectan a cada token individualmente, lo único que importa para la causalidad es el bloque de atención (o MSSA en el caso de CRATE). Para que MSSA sea causal, definimos el bloque CMSSA como

$$\text{CMSSA}_\theta^\ell(\boldsymbol{Z}) \doteq \boldsymbol{U}_{\text{out}}^\ell \begin{bmatrix} \text{CSA}([\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}) \\ \vdots \\ \text{CSA}([\boldsymbol{U}^{K,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{K,\ell}]^\top \boldsymbol{Z}, [\boldsymbol{U}^{1,\ell}]^\top \boldsymbol{Z}) \end{bmatrix} + \boldsymbol{b}_{\text{out}}^\ell \mathbf{1}_N^\top \tag{8.11.4}$$

$$\text{donde} \quad \text{CausalSA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) \doteq \boldsymbol{V}\, \text{softmax}\left(\frac{\text{CausalMask}(\boldsymbol{K}^\top \boldsymbol{Q})}{\sqrt{p}}\right), \tag{8.11.5}$$

$$\text{y donde} \quad \text{CausalMask}(\boldsymbol{M})_{ij} = \begin{cases} M_{ij}, & \text{if } i \geq j, \\ -\infty, & \text{if } i < j \end{cases}. \tag{8.11.6}$$

En este contexto, los especialistas afirman que la máscara causal *permite que los tokens futuros i presten atención a los tokens pasados j, pero no al revés.* Para justificar por qué, escribamos la expresión correspondiente a la t-ésima columna de $\text{CSA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})$:

$$\text{CSA}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V})_t = \sum_{i=1}^{t} \boldsymbol{V}_i\, \text{softmax}\big([\boldsymbol{K}_{:t}]^\top \boldsymbol{Q}_t\big)_i \tag{8.11.7}$$

(donde el subíndice sin dos puntos denota la columna). Esta expresión para el token t-ésimo no utiliza información sobre ningún token más allá del índice t. Por lo tanto, CSA, y con ello CMSSA, y por consiguiente toda el *backbone* causal de CRATE es causal en términos de la definición en (8.11.3) y desbloqueamos las considerables ganancias de eficiencia que se nos prometieron.

Extractor de características. Utilizamos un paso de posprocesamiento f_θ^{ext} que extrae el *vector de características del token más reciente conocido* con el fin de predecir el siguiente token. En teoría, esto significa que cada token $\boldsymbol{Z}_\theta(\boldsymbol{X})_n$ debería contener información detallada sobre todos los tokens que preceden al índice n o se encuentran en él, es decir, $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$, ya que toda esta información debería estar disponible para predecir el siguiente token en el índice $n+1$. En la práctica, solo se necesitan unos pocos de estos tokens para cada tarea de predicción. De todos modos, la ecuación para f_θ^{ext} es

$$f_\theta^{\text{ext}}(\boldsymbol{Z}_\theta(\boldsymbol{X}_{:n})) \doteq (\boldsymbol{Z}_\theta(\boldsymbol{X}))_n \tag{8.11.8}$$

donde (una vez más) el subíndice sin dos puntos corresponde a la columna. En este caso, tal y como prometimos, simplemente extraemos directamente el vector de características del último token de la secuencia.

Cabezal específico para la tarea. Para nuestro cabezal de clasificación h_θ, la arquitectura GPT-2 utiliza una capa lineal simple y una función softmax para obtener los vectores de probabilidad deseados:

$$h_\theta(\boldsymbol{z}) \doteq \text{softmax}(\boldsymbol{W}^{\text{out}}\boldsymbol{z} + \boldsymbol{b}^{\text{out}}), \tag{8.11.9}$$

donde $\boldsymbol{W}^{\text{out}} \in \mathbb{R}^{V \times d}, \boldsymbol{b}^{\text{out}} \in \mathbb{R}^V$. Algunas otras arquitecturas más modernas utilizan redes MLP pequeñas y normalizaciones de capas, pero la idea es prácticamente la misma. Tenga en cuenta que estas capas lineales también consumen mucha memoria (porque V es muy grande) y constituyen un cuello de botella en el entrenamiento; se han realizado importantes esfuerzos para intentar sortearlo.

Todas estas decisiones arquitectónicas hacen que el entrenamiento causal sea extremadamente eficiente en comparación con el entrenamiento no causal:

- Solo necesitamos *una pasada hacia adelante* a través del *backbone* para calcular la pérdida de toda la secuencia.
- La extracción de características es, básicamente, *gratuita*.
- Todos los tokens pueden enviarse al cabezal específico de la tarea *en paralelo*.

8.11.4 Estrategia de optimización

Entrenamos nuestro modelo de lenguaje mediante optimización estocástica de extremo a extremo. Un problema residual es que, en la práctica, los documentos distintos dentro de un lote poseen longitudes variables (en función del número de tokens requeridos para cada secuencia). Sin embargo, al momento de la redacción de este libro, los principales marcos de aprendizaje profundo admiten, por lo general, únicamente tensores «rectangulares», los cuales no admiten este comportamiento. Para intentar resolver este problema, insertamos un token de relleno especial |pad|> para todos los ejemplos más cortos del lote, permitiendo

así procesar todo el lote mediante tensores rectangulares. En cada paso temporal k, hacemos lo siguiente:

- Seleccionar una submuestra de B documentos tokenizados diferentes $\{\boldsymbol{X}_b^{(k)}\}_{b=1}^B \subseteq \mathcal{T} = [V]^*$, cada uno con una longitud de $N_b^{(k)}$.
- Calcular $N_{\max}^{(k)} \doteq \max_{b\in[B]} N_b^{(k)}$ y rellenar cada $\boldsymbol{X}_b^{(k)}$ hasta una longitud de $N_{\max}^{(k)}$ utilizando un token de relleno especial.
- Calcular las características $\boldsymbol{Z}_\theta(\boldsymbol{X}_b^{(k)})$.
- Calcular las distribuciones predichas $\boldsymbol{p}_\theta(\boldsymbol{X}_{b,:n}^{(k)}) \doteq (h_\theta \circ f_\theta^{\text{ext}})(\boldsymbol{Z}_\theta(\boldsymbol{X}_b^{(k)})_{:n})$.
- Construir la pérdida estocástica sustitutiva

$$\hat{\mathcal{L}}_{\text{CLM}}^{(k)}(\theta) \doteq \frac{1}{B(N_{\max}^{(k)}-1)} \sum_{b=1}^{B} \sum_{n=1}^{N_{\max}^{(k)}-1} \text{CE}(\mathbf{1}(\boldsymbol{x}_{b,n+1}^{(k)}), \boldsymbol{p}_\theta(\boldsymbol{X}_{b,:n}^{(k)}))). \tag{8.11.10}$$

- Ejecutar un paso de un algoritmo de optimización sobre θ, obteniendo la siguiente iteración:

$$\theta^{(k+1)} \doteq \text{OptUpdate}^{(k)}(\theta^{(k)}; \nabla_\theta \hat{\mathcal{L}}_{\text{CLM}}^{(k)}). \tag{8.11.11}$$

8.11.5 Metodología de evaluación

Hay varias formas de evaluar un modelo de lenguaje transformer entrenado.

- En un conjunto de datos holdout (de prueba) compuesto por texto arbitrario, podemos evaluar $\mathcal{L}_{\text{CLM}}$ en él; las pérdidas más bajas son preferibles, ya que reflejan que el modelo ha aprendido bien esa distribución.
- En un conjunto de datos de preguntas de opción múltiple, para cada pregunta podemos utilizarla como contexto y comprobar la probabilidad estimada de que se genere la respuesta correcta.
- También podemos probar las capacidades de *generación de texto*. Específicamente, podemos muestrear repetidamente desde la distribución de probabilidad del modelo sobre el siguiente token, condicionada al contexto. Cada vez que tomamos una muestra, generamos un nuevo token, que visualizamos y añadimos al contexto. Esto nos permite muestrear desde el LLM y evaluar las muestras generadas según nuestros criterios.[36]

En esta sección, nos ceñimos exclusivamente al primer tipo de evaluación.

[36] Tener que volver a ejecutar el modelo con cada token cada vez puede resultar prohibitivamente costoso. El almacenamiento inteligente de diferentes características internas del modelo de lenguaje (como la llamada *K-V cache*), junto con la causalidad de la arquitectura, puede reducir drásticamente el costo del muestreo.

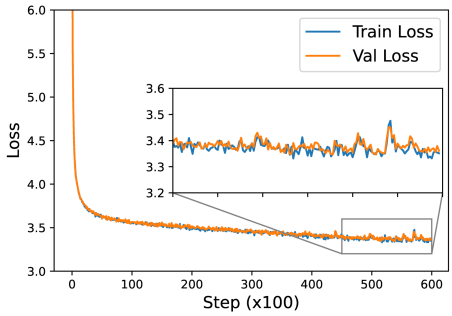


Figura 8.41: **La curva de pérdida de CRATE-GPT-Base entrenado con el conjunto de datos OpenWebText.**

8.11.6 Configuración experimental y resultados

Dado que nuestra arquitectura CRATE causal se construye directamente sobre GPT-2, comparamos los parámetros óptimos para GPT-2 especificados en el repositorio NanoGPT [Kar22a] los mismos parámetros aplicados a CRATE para una comparación imparcial.

Arquitectura del modelo. Utilizamos el tokenizador de GPT-2, que tiene un tamaño del vocabulario de $V = 50257$, incluyendo un token especial para `<|pad|>`.[37] La longitud del contexto es $N_{\max} = 1024$. El *backbone* sigue la arquitectura GPT2-Base [RWC+19] con las modificaciones adecuadas para tener capas CRATE causales, y comparamos con GPT2-Small y GPT2-Base.

Conjuntos de datos y optimización. Para entrenar el modelo causal CRATE, seguimos las implementaciones del repositorio NanoGPT [Kar22a]. Específicamente, utilizamos un tamaño de lote de 384 y entrenamos durante 600 000 pasos con el optimizador Adam [KB14]. Para el optimizador Adam, utilizamos $(\beta_1, \beta_2) = (0.9, 0.95)$ y un decaimiento de peso de 0.1. Para el programa de tasa de aprendizaje, aplicamos un calentamiento lineal y un decaimiento coseno, con un valor máximo de $\eta = 6 \times 10^{-4}$ en la iteración 2 000, y un valor mínimo de 6×10^{-5}. Las pérdidas de entrenamiento y validación a lo largo de las iteraciones se muestran en Figura 8.41. La pérdida de entrenamiento/validación converge en torno a 3,37 tras el entrenamiento con un tamaño de lote de 384 y 600 000 iteraciones. En comparación, la implementación abierta de GPT-2 está preentrenada en OpenWebText con un tamaño de lote de 512 y 600 000 pasos, y converge a una pérdida de validación de 2,85 [Kar22a].

Resultados experimentales. La Tabla 8.18 demuestra que los modelos CRATE alcanzan un rendimiento razonable en la pérdida de modelado de

[37] El token `<|bos|>` no se incluye en esta configuración, aunque es muy común en los modelos de lenguaje modernos.

Tabla 8.18: Pérdida de entropía cruzada sin entrenamiento previo (zero-shot) del modelo CRATE-GPT2-Base y de los modelos GPT2-Small y GPT2-Base, evaluada en el conjunto de prueba de los conjuntos de datos (↓: cuanto más bajo, mejor).

	#parámetros	**OWT**	**LAMBADA**	**WikiText**	**PTB**	**Media**
GPT2-Base	124M	2.85↓	4.12↓	3.89↓	4.63↓	3.87↓
GPT2-Small	64M	3.04	4.49	4.31	5.15	4.25
Causal-CRATE-Base	60M	3.37	4.91	4.61	5.53	4.61

lenguaje causal en una variedad de conjuntos de datos, en comparación con los modelos GPT-2 con un número similar de parámetros y arquitecturas similares.

8.12 Escalado y mejora de los *Transformers white-box*

En esta última sección, discutiremos varios métodos mediante los cuales se pueden escalar distintas componentes de los modelos de tipo CRATE o hacerlos más eficientes para tareas específicas, manteniendo su naturaleza completamente interpretable white-box. Estos avances combinan conocimientos tanto conceptuales como empíricos, y pueden considerarse como estudios de caso sobre cómo aprovechar la comprensión de white-box para mejorar los modelos de aprendizaje profundo en la práctica. Las tareas que utilizaremos para evaluar los métodos serán la clasificación de imágenes y la predicción del siguiente token; los datos serán ImageNet y OpenWebText, respectivamente; el procedimiento de optimización será la misma retropropagación, y lo único que cambia es la arquitectura.

8.12.1 Aumento del ancho de la red: CRATE-α

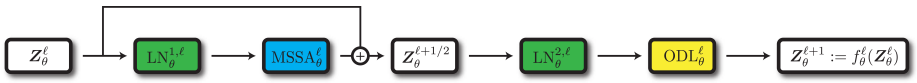


Figura 8.42: **Una capa del *backbone* CRATE-α.** La diferencia con respecto a CRATE es que el bloque $\mathrm{ISTA}_\theta^\ell$ se sustituye por el bloque ODL_θ^ℓ, que realiza varios pasos ISTA con un diccionario sobrecompleto.

Una decisión de diseño impuesta por el marco CRATE es el *ancho* de la nolinealidad en la red. En un transformer convencional, el ancho suele fijarse en 4, 8 o $\frac{11}{3}$ veces la dimensión de las características. Sin embargo, CRATE exige que el ancho sea exactamente igual a la dimensión de las características; es decir, que los diccionarios $\boldsymbol{D}^\ell$ sean cuadrados, lo cual podría resultar en un desempeño reducido. La razón fundamental por la cual el marco CRATE restringe nuestra elección es la siguiente:

- El bloque ISTA realiza *un solo* paso de optimización para el aprendizaje de diccionarios.
- Generalmente, un solo paso de cualquier algoritmo de optimización iterativo no logra optimizar eficazmente la función objetivo. Entonces, ¿cómo es posible que funcione?
- Los algoritmos de optimización suelen converger muy rápidamente si y solo si cuentan con una buena inicialización, o un *warm start*. El bloque ISTA posee un *warm start*: trata las características de entrada como una inicialización para los códigos esparsos resultantes.
- Esto impone que las características de entrada y los códigos esparsos tengan la misma dimensión. Es decir, ISTA aprende un diccionario de esparcidad completo (véase el Capítulo 2).

Por consiguiente, si deseamos emplear un diccionario ancho, necesitamos que ISTA realice el aprendizaje de diccionarios *sobrecompletos*. Esto implica que no podemos mantener el mismo «warm start», ya que los códigos esparsos resultantes tienen una dimensión mayor que las características de entrada; por tanto, requiriremos más iteraciones para converger a un código esparso. En consecuencia, el paso de las características $\boldsymbol{Z}_\theta^{\ell+1/2}$ hacia los códigos esparsos $\boldsymbol{Z}_\theta^{\ell+1}$ ya no se definiría como

$$\boldsymbol{Z}_\theta^{\ell+1} = \text{ISTA}_\theta^\ell(\boldsymbol{Z}_\theta^{\ell+1/2} \mid \boldsymbol{Z}_\theta^{\ell+1/2}) \tag{8.12.1}$$

donde la función ISTA_θ^ℓ está definida como sigue (por un abuso de notación ya introducido en secciones anteriores):

$$\text{ISTA}_\theta^\ell(\boldsymbol{Z} \mid \boldsymbol{Y}) \doteq \text{ReLU}(\boldsymbol{Z} - \beta(\boldsymbol{D}^\ell)^\top(\boldsymbol{D}^\ell\boldsymbol{Z} - \boldsymbol{Y}) + \beta\lambda\mathbf{1}_s\mathbf{1}_n^\top) \tag{8.12.2}$$

sino más bien la siguiente iteración:

$$\boldsymbol{Z}_\theta^{\ell+1} = \boldsymbol{A}_\theta^{\ell,T}; \qquad \boldsymbol{A}_\theta^{\ell,t+1} = \text{ISTA}_\theta^\ell(\boldsymbol{A}_\theta^{\ell,t} \mid \boldsymbol{Z}_\theta^{\ell+1/2}) \quad \forall 0 \le t < T; \qquad \boldsymbol{A}_\theta^{\ell,0} = \mathbf{0}_{s\times n}, \tag{8.12.3}$$

es decir, aplicando el mátodo del gradiente proximal sobre el objetivo LASSO durante $T \ge 1$ pasos en la pasada hacia adelante en cada capa, inicializado en $\mathbf{0}_{s\times n}$. En este caso, el diccionario puede ser tan amplio como sea necesario, es decir, $\boldsymbol{D}^\ell \in \mathbb{R}^{s\times d}$ donde $s \ge d$ (por lo general, en la práctica se toma $s = 4d$).

Sin embargo, esto presenta un problema práctico (empírico). Al utilizar la configuración anterior, si $\boldsymbol{Z}^{\ell+1/2} \in \mathbb{R}^{d\times n}$, entonces $\boldsymbol{Z}^{\ell+1} \in \mathbb{R}^{s\times n}$, lo que podría resultar en una dimensión de características arbitrariamente grande. En la práctica, queremos que la dimensión de las características en cada capa sea la misma. Así, esto plantea una tricotomía práctica en el diseño de la red; *no es posible* satisfacer simultáneamente *todos* los siguientes requisitos deseables:

1. La dimensión de las características es constante en todas las capas.
2. El diccionario es ancho, es decir, sobrecompleto.

3. La salida de la nolinealidad son corresponde a los códigos esparsos de la entrada respecto al diccionario.

En la práctica, renunciar a (1) resulta menos manejable por razones de eficiencia. Renunciar a (2) conduce al marco CRATE habitual. Renunciar a (3) conduce a una versión ancha de CRATE, es decir, CRATE-α, que tiene la siguiente nolinealidad para pasar de $\boldsymbol{Z}^{\ell+1/2}$ a $\boldsymbol{Z}^{\ell+1}$:

$$\boldsymbol{Z}_\theta^{\ell+1} = \boldsymbol{D}^\ell \boldsymbol{A}_\theta^{\ell,T}; \qquad \boldsymbol{A}_\theta^{\ell,t+1} = \mathrm{ISTA}_\theta^\ell(\boldsymbol{A}_\theta^{\ell,t} \mid \boldsymbol{Z}_\theta^{\ell+1/2}); \qquad \boldsymbol{A}_\theta^{\ell,0} = \boldsymbol{0}, \quad (8.12.4)$$

es decir, se toman los códigos esparsos resultantes del descenso de gradiente proximal y se multiplican por el diccionario para obtener una versión de la entrada sin ruido. Por consiguiente, la no linealidad de CRATE-α calcula una versión sin ruido de la entrada que es apta para la codificación esparsa, en lugar de los códigos esparsos reales en si mismos. El mapeo desde $\boldsymbol{Z}_\theta^{\ell+1/2}$ a $\boldsymbol{Z}_\theta^{\ell+1}$ se denomina aquí bloque de aprendizaje de diccionarios sobrecompletos (ODL - Overcomplete Dictionary Learning) y se denota ODL_θ^ℓ, es decir,

$$\boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X}) \doteq \mathrm{ODL}_\theta^\ell(\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X})). \quad (8.12.5)$$

La capa CRATE-α se muestra en Figura 8.42. En la práctica, esta modificación de CRATE rinde muy bien a escalas superiores. Por ejemplo, cuando preentrenamos modelos CRATE-α en ImageNet-21K, las tareas no supervisadas como la segmentación (véanse la Figura 8.43 y la Tabla 8.19) suelen tener un rendimiento significativamente mejorado en comparación con CRATE. Tendencia similar se observa en el entrenamiento de modelos de lenguaje utilizando la autoatención causal (véase Tabla 8.20). En general, se trata de un camino prometedor para escalar el rendimiento e igualar a los modelos de caja negra como los Transformers.[38]

[38] Cabe señalar que los resultados experimentales de esta sección utilizan una arquitectura de modelo ligeramente diferente, lo que aporta mejoras empíricas muy leves. Los cambios son: (1) una conexión residual adicional en el bloque ODL, (2) modificar ISTA para que utilice dos diccionarios independientes en lugar de $\boldsymbol{D}^\ell$ y $(\boldsymbol{D}^\ell)^\top$.

Tabla 8.19: **Detección de objetos y segmentación detallada mediante MaskCut en COCO val2017** [LMB+14]. En este caso, todos los modelos se entrenan con un tamaño de parche de 8 en lugar de 16. En comparación con modelos existentes como CRATE y ViT, la familia de modelos CRATE-α presenta un rendimiento notablemente mejorado, así como una mayor escalabilidad.

	Detección			Segmentación		
Modelo	AP_{50} ↑	AP_{75} ↑	AP ↑	AP_{50} ↑	AP_{75} ↑	AP ↑
CRATE-α-B/8	3.5	1.1	1.5	2.2	1.0	1.1
CRATE-α-L/8	**4.0**	**1.7**	**2.0**	**2.7**	**1.1**	**1.4**
CRATE-B/8	2.9	1.0	1.3	2.2	0.7	1.0
ViT-B/8	0.8	0.2	0.4	0.7	0.5	0.4

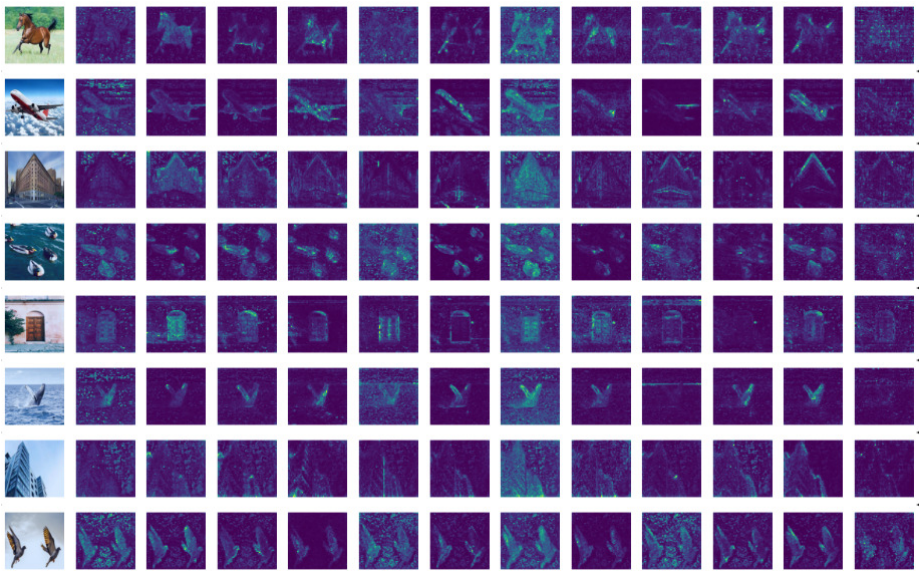


Figura 8.43: **Mapas de saliencia de CRATE-α con un tamaño de parche de 8.** Cada fila corresponde a una imagen diferente y cada columna corresponde a una cabeza de atención diferente en la última capa. Se observa que los mapas de saliencia se corresponden fuertemente con los objetos de la imagen de entrada.

Tabla 8.20: **Pérdida de validación en el modelado del lenguaje.** Aquí todos los modelos se entrenan previamente con la mayor parte de OpenWebText, y la pérdida de entropía cruzada de validación se mide en un subconjunto reservado de OpenWebText. CRATE-α muestra una mejora significativa con respecto al diseño de CRATE, aunque sigue existiendo una brecha con respecto a los transformers tradicionales como GPT-2.

Modelo	GPT-2-B(ase)	CRATE-B	CRATE-α-S(mall)	CRATE-α-B
# parámetros	124M	60M	57M	120M
OWT pérdida val.	2.85	3.37	3.28	3.14

8.12.2 Transformadores de complejidad temporal lineal

En la práctica, los modelos de aprendizaje profundo sufren de cuellos de botella en cuanto a la complejidad espacial y temporal, lo que se traduce en tamaños de problemas más allá de los cuales no pueden escalar con recursos fijos. Uno de estos cuellos de botella, especialmente relevante cuando se trata de datos en los que cada muestra es en sí misma de alta dimensión y rica en información (como largas secuencias de texto o vídeos), es la *complejidad temporal* del procesamiento de secuencias largas de datos. Con el fin de reducir la complejidad temporal del procesamiento de datos mediante Transformers, en Sección 5.3.2 propusimos un operador de *autoatención basada en estadísticas de tokens* $\mathrm{TSSA}_\theta^\ell$. Ahora construimos un *Transformer basado en estadísticas de tokens*, denominado ToST:

https://robinwu218.github.io/ToST/,

en torno a él, que podemos utilizar para tareas de contexto largo. En particular, podemos utilizar la siguiente capa (representada en Figura 8.44) como sustituto directo de una capa de *backbone* en CRATE.

$$\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}) = \boldsymbol{Z}_\theta^\ell(\boldsymbol{X}) + \mathrm{TSSA}_\theta^\ell(\mathrm{LN}_\theta^{1,\ell}(\boldsymbol{Z}_\theta^\ell(\boldsymbol{X}))) \tag{8.12.6}$$

$$\boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X}) = \boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}) + \mathrm{MLP}_\theta^\ell(\mathrm{LN}_\theta^{2,\ell}(\boldsymbol{Z}_\theta^{\ell+1/2}(\boldsymbol{X}))) \tag{8.12.7}$$

donde el bloque TSSA se define tal como se indica en Sección 5.3.2. Obsérvese que esto es exactamente igual que la arquitectura del transformer visual discutida en Sección 8.2.3, excepto que TSSA reemplaza al bloque convencional de autoatención multi-cabezal MHSA. En todo caso, la complejidad computacional de la propagación hacia adelante de esta capa es lineal en todas las variables del problema — longitud de la secuencia, dimensión de las características, número de cabezales y dimensión de los cabezales.

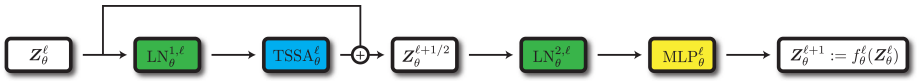


Figura 8.44: **Una capa del *backbone* ToST**. Las representaciones de los tokens pasan por normas de capa, el operador de autoatención de estadísticas de tokens (TSSA) y una red neuronal multi-capas (MLP), con el fin de formar la salida de la capa.

Además, la arquitectura propuesta, denominada ToST (por «Token Statistics Transformer», «Transformador de Estadísticas de Tokens»), ofrece un buen rendimiento en tareas de visión (véase, por ejemplo, Tabla 8.21) y en tareas lingüísticas (véase, por ejemplo, Tabla 8.22). Esto es especialmente cierto en el caso de tareas con secuencias largas (véase Tabla 8.23), donde logra un mayor rendimiento y es mucho más eficiente que los transformers convencionales y todas las demás arquitecturas tipo Transformer.

Datos	ToST-T(iny)	ToST-S(mall)	ToST-M(edium)	XCiT-S	XCiT-M	ViT-S	ViT-B(ase)
# parámetros	5.8M	22.6M	68.1M	24.9M	80.2M	22.1M	86.6 M
ImageNet	67.3	77.9	80.3	80.5	81.5	79.8	81.8
ImageNet ReaL	72.2	84.1	85.6	85.6	85.9	85.6	86.7
CIFAR10	95.5	96.5	97.5	98.1	98.3	98.6	98.8
CIFAR100	78.3	82.7	84.5	86.1	87.6	88.8	89.3
Oxford Flowers-102	88.6	92.8	94.2	93.9	94.0	94.0	95.7
Oxford-IIIT-Pets	85.6	91.1	92.8	92.9	94.0	92.8	94.1

Tabla 8.21: **Precisión de clasificación mediante sondeo lineal de ToST** en diversos conjuntos de datos con diferentes tamaños de modelo cuando el *backbone* se ha entrenado previamente para la clasificación de ImageNet-1K. Observamos que, en comparación con XCiT (una arquitectura de tipo transformer diseñada empíricamente y especializada en el procesamiento eficiente de secuencias largas) y el ViT (Vision Transformer), ToST mantiene un rendimiento relativamente similar, incluso además de contar con ventajas como un tiempo de ejecución más rápido y un diseño transparente o de *white-box*.

Modelo	# parám.	OWT	Lambada	Wikitext	PTB	Prom. ↓
GPT-2-Base	124M	2.84	4.32	4.13	5.75	4.26
ToST-Base	110M	3.20	4.98	4.77	6.39	4.84
ToST-Medium	304M	2.88	4.45	4.30	5.64	4.32
ToST-Large	655M	2.72	4.32	3.99	5.03	4.02

Tabla 8.22: **Pérdida de validación del modelado de lenguaje** calculada sobre (conjuntos de validación de) diversos conjuntos de datos de lenguaje natural, tras el preentrenamiento del modelo en ese conjunto de datos. Observamos que ToST escala bien, de modo que ToST-Large supera al modelo de referencia GPT-2-Base en el modelado causal de lenguaje, al tiempo que ofrece una eficiencia superior en contextos largos.

Modelo	ListOps	Texto	Consulta	Imagen	Pathfinder	Prom.
Reformer	**37.27**	56.10	53.40	38.07	68.50	50.56
BigBird	36.05	64.02	59.29	40.83	74.87	54.17
LinFormer	16.13	65.90	53.09	42.34	75.30	50.46
Performer	18.01	65.40	53.82	42.77	**77.05**	51.18
Transformer	37.11	65.21	79.14	42.94	71.83	59.24
ToST	37.25	**66.75**	**79.46**	**46.62**	69.41	**59.90**

Tabla 8.23: **Comparación del rendimiento en Long-Range Arena (LRA) entre ToST(-B) y las principales variantes de transformer optimizadas para contextos largos.** Long-Range Arena es una familia de pruebas de rendimiento que evalúa la capacidad de modelado de secuencias largas de algoritmos y arquitecturas, mediante el uso de un conjunto de datos y un mecanismo de evaluación fijos. ToST se ubica en la parte superior del ranking, superando a todas las variantes conocidas del transformer, incluyendo XCiT y el transformer regular (ViT) (véase Tabla 8.21). Además, ToST presenta la inferencia con menor complejidad temporal y espacial. (En esta tabla, la mejor puntuación para un benchmark concreto aparece en negrita, y la segunda mejor puntuación, subrayada.)

8.12.3 Transformers de solo atención

Otro cuello de botella que hay que eliminar de los modelos de aprendizaje profundo, concretamente de las arquitecturas tipo transformer, es el cuello de botella de memoria que se deriva de las multiplicaciones matriciales masivas en los MLP, donde la dimensión interna es mucho mayor que la dimensión de las características d. Por lo tanto, resulta interesante y relevante plantearse la siguiente pregunta: ¿necesitamos *realmente* el MLP dentro de un transformer, y qué nivel de rendimiento se puede alcanzar sin ella? Para explorar esta cuestión, utilizamos la arquitectura «Attention-Only Transformer» (AoT, transformer de solo atención) (véase Sección 5.3.1), representada en Figura 8.45. Concretamente, cada capa tiene simplemente la forma

$$\boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X}) = \boldsymbol{Z}_\theta^{\ell}(\boldsymbol{X}) + \mathrm{MSSA}_\theta^{\ell}(\mathrm{LN}_\theta^{\ell}(\boldsymbol{Z}_\theta^{\ell}(\boldsymbol{X}))). \tag{8.12.8}$$

En nuestra implementación, también experimentamos con el uso de la autoatención multi-cabezal (MHSA) en lugar de la MSSA. Resulta que esta arquitectura *también* es viable, aunque la profundidad de la red debe ser considerablemente mayor para lograr un rendimiento equivalente al de la arquitectura CRATE o transformer habitual.

Llevamos a cabo experimentos utilizando la arquitectura AoT propuesta y demostramos su potencial. Preentrenamos los modelos AoT-MSSA y AoT-MHSA de diferentes tamaños, junto con GPT-2, en OpenWebText [GC19]. Graficamos la pérdida de entrenamiento y la pérdida de validación en función del número de iteraciones de entrenamiento en Figura 8.46(a) y (b), respectivamente. Se observa que los modelos basados en AoT de tamaño mediano y grande alcanzan pérdidas de entrenamiento y validación comparables

Figura 8.45: **Una capa de la *backbone* AoT.** Las representaciones de los tokens simplemente pasan por una normalización de capa y por el operador de autoatención multi-cabezal (de subespacio) para formar la salida de la capa. Obsérvese que no existe ninguna nolinealidad a nivel de token, como la de las redes MLP, ISTA u ODL.

a las del modelo base GPT-2. Además, en comparación con el modelo base GPT-2, el modelo AoT-MHSA es idéntico al modelo base GPT-2, salvo por la ausencia de capas MLP en la arquitectura. Como se muestra en Figura 8.46, la incorporación de capas MLP puede acelerar el proceso de entrenamiento. Utilizando los modelos preentrenados anteriores, calculamos la pérdida de entropía cruzada de validación sin entrenamiento en diferentes conjuntos de datos en Tabla 8.24. Se observa que los modelos AoT con un número de parámetros medio y alto pueden alcanzar un rendimiento comparable al del modelo base GPT-2. Además, hemos constatado que añadir capas de MLP a los modelos AoT no mejora el rendimiento *zero-shot*. Estos resultados ponen de relieve el potencial de los modelos basados únicamente en atención para lograr resultados competitivos sin perder interpretabilidad.

Tabla 8.24: Resultados «zero-shot» en varios conjuntos de datos y tareas lingüísticas de referencia : Evaluación de diferentes tamaños de AoT con los operadores MSSA y MHSA y comparación con el modelo GPT-2.

Modelos # de parámetros	**LAMBADA** (val loss) ↓	**PTB** (val loss) ↓	**WikiText** (val loss) ↓	**LAMBADA** (acc) ↑	**CBT CN** (acc) ↑	**CBT NE** (acc) ↑
AoT-MSSA Base (102M)	4.70	6.03	4.65	0.25	0.80	0.74
AoT-MSSA Medium (182M)	4.47	5.08	4.22	0.29	0.84	0.77
AoT-MHSA Base (122M)	4.42	5.52	4.19	0.38	0.86	0.82
GPT-2 Base (124M)	4.32	5.75	4.13	0.40	0.87	0.84

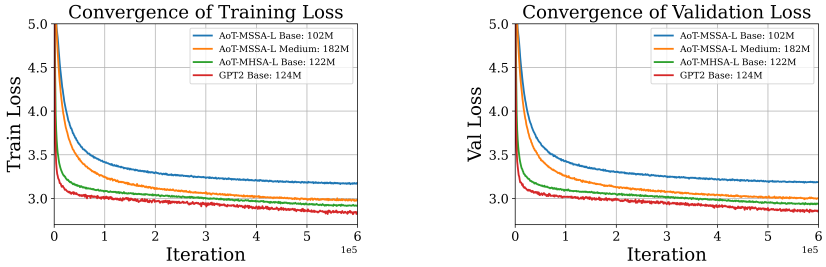


Figura 8.46: **Evaluación de modelos en tareas lingüísticas.** Representamos la pérdida de entrenamiento (izquierda) y la pérdida de validación (derecha) de los modelos AoT y GPT-2 preentrenados en OpenWebText.

8.13 Resumen y notas

Todo el trabajo de este capítulo se basa en la arquitectura transformer, introducida por Vaswani et al. [VSP+17b]. La arquitectura transformer se describe formalmente en Sección 8.2. Una innovación empírica principal de los últimos años, impulsada por la prevalencia y el rendimiento de la arquitectura transformer, consiste en formular un problema de aprendizaje concreto como un problema de secuencia a secuencia y aplicar la arquitectura transformer. Esto ha permitido que la arquitectura transformer sea esencialmente omnipresente en (casi) todas las aplicaciones de aprendizaje profundo. Por ello, las mejoras directas en el transformer pueden propagarse hasta convertirse en soluciones a muchos problemas y tener un impacto considerable; del mismo modo, podemos aplicar nuestra comprensión interna (*white-box*) de las arquitecturas similares al Transformer a muchos problemas modernos. El material tratado en este capítulo es solo una parte del trabajo que ya se ha realizado; otros trabajos incluyen el completado enmascarado para datos de texto (o sea BERT) [DCL+19; YBP+24], la interpretabilidad (mecánica) de los modelos de lenguaje y visión [BM24], y la codificación con corrección de errores [ZLG+]. Queda mucho más por hacer.

También hay mucha más teoría específica sobre la práctica del escalado de redes neuronales, que tiene una enorme viabilidad práctica, y al menos lo mencionamos aquí. Esta línea de investigación se popularizó gracias a la línea de investigación de los «programas tensoriales» [YHB+22]. La regla básica es que queremos que las actualizaciones iniciales del gradiente en un transformer sean de tamaño constante y, al analizar con detalle las ecuaciones de retropropagación (Apéndice A), podemos determinar la escala de inicialización y la de las tasas de aprendizaje (elegidas capa a capa) que se requieren para lograrlo. En la práctica, estas recomendaciones aumentan considerablemente la estabilidad y la convergencia del entrenamiento a gran escala; además, indican una forma de encontrar los «hiperparámetros óptimos»[39] para el entrenamiento a gran escala utilizando únicamente entrenamiento a pequeña escala. Los trabajos posteriores a este trabajo intentan adaptarse a la geometría de las características [BN24a], lo cual podría basarse en el trabajo de este libro sobre el aprendizaje de representaciones. Otros trabajos posteriores incorporan esta información sobre los pesos en el propio optimizador para obtener estos beneficios de escalabilidad en forma automática, lo que da lugar a optimizadores como Muon [JJB+], que recientemente se han utilizado para entrenar modelos de billones de parámetros de manera extremadamente estable [Tea]. En general, los dos enfoques de la teoría del aprendizaje profundo son ortogonales o complementarios.

[39] La palabra «óptimos» se utiliza entre comillas porque este trabajo se limita a emplear ciertos criterios sobre el tamaño de los pesos, el tamaño de las características y el tamaño del gradiente en la inicialización para determinar la «optimalidad», en contraposición, por ejemplo, a la pérdida en el conjunto de prueba al alcanzar la convergencia.

8.14 Ejercicios y extensiones

Ejercicio 8.1. Lea el artículo de DINO [CTM+21].

Ejercicio 8.2. DINO v2 [ODM+23] retiene toda la base de DINO v1, pero además, durante la fase de augmentación de datos, enmascara aleatoriamente los parches dentro de cada vista. Este tipo de augmentación debería imponer que las características de las imágenes con información local similar sean también similares. Formule un problema de optimización que favorezca esto en el codificador e impleméntelo.

Ejercicio 8.3. Este ejercicio analiza la implementación de algoritmos de optimización estocástica para minimizar las pérdidas que implican expectativas.

(a) Proponga una alternativa al término que involucra R_ε en (8.2.44) para aproximar el término de regularización de la covarianza en (8.2.42). Evalúe la complejidad temporal necesaria para calcular el término propuesto y su gradiente. Incluya un análisis del cálculo en un solo nodo de cómputo frente a múltiples nodos.

(b) Evalúe la complejidad temporal necesaria para calcular el término existente en (8.2.44) y su gradiente.

Ejercicio 8.4. Demuestre que (8.2.50) y (8.2.51) son problemas de optimización convexa.

Ejercicio 8.5.

(a) Implemente los modelos CRATE y CRATE-α.

(b) Compare su rendimiento y eficiencia sobre el conjunto de datos CIFAR-10.

(c) Compare su interpretabilidad de dos maneras: Compare their interpretability in two ways:

- La esparcidad $\|\boldsymbol{Z}\|_0$ de la representación $\boldsymbol{Z}$
- Los mapas de atención $\boldsymbol{a}_\theta^{k,\ell}$

Capítulo 9

Problemas abiertos y direcciones sobre inteligencia

> "*El estudio se desarrollará sobre la base de la conjetura de que cada aspecto del aprendizaje o cualquier otra característica de la inteligencia puede describirse, en principio, con tal precisión que sea posible hacer que una máquina lo simule. Se intentará determinar cómo hacer que las máquinas usen el lenguaje, formen abstracciones y conceptos, resuelvan clases de problemas que hoy están reservados a los seres humanos y mejoren a sí mismas.*"
>
> – Proposición para el programa de IA de Dartmouth, 1956

Este manuscrito introduce en forma sistemática los principios matemáticos y los mecanismos computacionales mediante los cuales la memoria o el conocimiento empírico se desarrolla a partir de datos observados de alta dimensión. La capacidad de buscar parsimonia en un mundo aparentemente caótico es una característica fundamental de cualquier inteligencia, natural o artificial. Creemos que los principios y mecanismos expuestos en este libro son unificadores y universales, aplicables tanto a los animales como a las máquinas.

Esperamos que este libro ayude a los lectores a clarificar por completo el misterio que rodea las prácticas modernas de las redes neuronales artificiales profundas, mediante el desarrollo de una comprensión rigurosa de sus funciones y roles en el aprendizaje de representaciones de distribuciones de baja dimensión a partir de datos de alta dimensión. A partir de esta comprensión, se hacen evidentes las capacidades y limitaciones de los modelos y sistemas de IA existentes:

1. Los modelos y sistemas de bucle abierto existentes no alcanzan la categoría

de sistemas de memoria completos, capaces de aprender y automejorarse en forma autónoma.

2. Las implementaciones actuales del aprendizaje de representaciones siguen siendo primitivas y basadas en fuerza bruta, muy alejadas de la optimalidad en términos de arquitectura de red o estrategias de optimización.

3. Los modelos de IA actuales únicamente aprenden distribuciones de datos y realizan inferencia bayesiana inductiva, lo que contrasta con la inteligencia humana de alto nivel.

Un objetivo de este libro es ayudar a los lectores a establecer una comprensión objetiva y sistemática de las tecnologías actuales de la inteligencia artificial y a reconocer qué problemas abiertos y desafíos persisten para el avance futuro de la inteligencia artificial. En el último capítulo, presentamos algunos de nuestros puntos de vista y proyecciones para el futuro.

9.1 ¿Hacia la inteligencia autónoma: cerrar el círculo?

De la práctica en el campo de la inteligencia artificial durante la última década ha quedado claro que, dándose suficientes datos y recursos computacionales, se puede construir un modelo lo suficientemente grande y preentrenarlo para aprender la distribución a priori de todos los datos, digamos, $p(\boldsymbol{x})$. Teóricamente, un modelo de tal magnitud podría memorizar casi todo el conocimiento existente sobre el mundo que se codifica en datos, como imágenes bidimensionales, formas tridimensionales, movimientos dinámicos y lenguajes naturales. Tal como se discutió al inicio de este libro, dicho modelo cumple una función análoga a la del ADN, que la vida emplea para registrar y transmitir conocimientos sobre el mundo.

De este modo, el modelo y la distribución aprendidos pueden emplearse para generar nuevas muestras de datos extraídas de dicha distribución. Asimismo, se puede emplear el modelo para realizar inferencias (por ejemplo, completado, estimación y predicción) utilizando el conocimiento memorizado en diversas condiciones, por ejemplo, mediante el muestreo de la distribución *posterior* $p(\boldsymbol{x} \mid \boldsymbol{y})$ ante una nueva observación $\boldsymbol{y}$. Estrictamente hablando, tal inferencia es de tipo inductivo o estadístico.

Ningún modelo preentrenado, sin importar su tamaño, garantiza que la distribución aprendida hasta el momento sea completamente exacta o exhaustiva. Si nuestras muestras $\hat{\boldsymbol{x}}_t$ derivadas del *a priori* $p_t(\boldsymbol{x})$ o nuestras estimaciones $\hat{\boldsymbol{x}}_t(\boldsymbol{y})$ basadas en el *posterior* $p_t(\boldsymbol{x} \mid \boldsymbol{y})$ resultan inconsistentes con la verdad real $\boldsymbol{x}$, deseamos corregir las distribuciones aprendidas:

$$p_t(\boldsymbol{x}) \to p_{t+1}(\boldsymbol{x}) \quad \text{or} \quad p_t(\boldsymbol{x} \mid \boldsymbol{y}) \to p_{t+1}(\boldsymbol{x} \mid \boldsymbol{y}), \tag{9.1.1}$$

basado en el error $\boldsymbol{e}_t = \boldsymbol{x}_t - \hat{\boldsymbol{x}}_t$. Este proceso se denomina corrección de errores basada en retroalimentación, un mecanismo ubicuo en la naturaleza

para el aprendizaje continuo. Sin embargo, cualquier modelo abierto carece inherentemente de un mecanismo para revisar o perfeccionar la distribución aprendida cuando esta resulta errónea o incompleta. Mejorar los modelos actuales de inteligencia artificial depende aún considerablemente de la intervención humana: supervisión o refuerzo mediante experimentación, evaluación y selección. Podemos llamar a este proceso «selección artificial» de modelos masivos, en contraste con la selección natural que impulsa la evolución biológica.

Tal como se estudió anteriormente en este libro (especialmente en el Capítulo 6), los sistemas de bucle cerrado alinean una representación interna con las observaciones sensoriales del mundo externo. Pueden mejorar continuamente la distribución interna aprendida y su representación para lograr progresivamente la consistencia o la autocoherencia. El siguiente paso inmediato consiste en desarrollar sistemas de memoria que verdaderamente operen en circuito cerrado, ilustrados en la Figura 9.1, que aprendan de manera autónoma y continua distribuciones generales de datos y se perfeccionen basándose en la retroalimentación de errores.

Por consiguiente, la transición desde los modelos de bucle abierto entrenados end-to-end, tan populares actualmente, hacia sistemas de bucle cerrado con aprendizaje continuo

$$\text{modelos } \textbf{en circuito abierto} \implies \text{sistemas } \textbf{de circuito cerrado} \tag{9.1.2}$$

es la clave para que las máquinas emulen realmente el proceso mediante el cual el cerebro animal aprende y aplica conocimientos en un mundo abierto. Creemos que

los modelos abiertos son aptos para un mundo cerrado, sin importar cuán grande sea;
mientras que los sistemas de bucle cerrado son adecuados para un mundo abierto, sin importar cuán pequeño sea.

En consecuencia, la así llamada «Inteligencia Artificial General» (IAG o AGI) jamás podrá lograrse simplemente mediante un sistema diseñado para memorizar todo el conocimiento y datos existentes del mundo. Ningún conocimiento, sin importar su magnitud, resulta verdaderamente generalizable.[1] No obstante, es precisamente la capacidad de perfeccionar la memoria o el conocimiento previo y adaptarse a cualquier nuevo entorno lo que resulta verdaderamente generalizable. En consecuencia, una inteligencia verdaderamente auténtica resulta intrínsecamente generalizable siempre que se materialice completamente y correctamente.[2] Esto constituye exactamente la esencia del programa de cibernética propuesto por Norbert Wiener durante la década de 1940, sobre el cual habíamos discutido al inicio de este libro.

[1] Según Sir Karl Popper, un influyente filósofo de la ciencia, ¡todas las teorías y conocimientos científicos son falsables!

[2] En nuestra opinión, la IAG debería representar la «inteligencia artificial genuina».

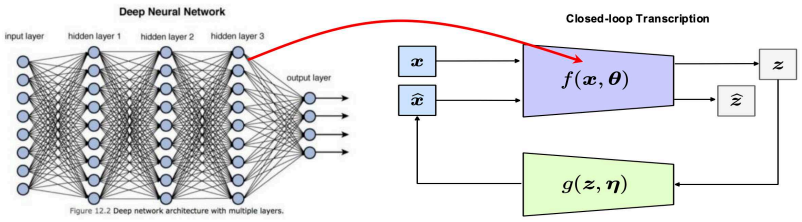


Figura 9.1: De una red profunda abierta a un sistema de circuito cerrado.

9.2 Hacia la inteligencia natural: ¿Más allá de la retropropagación?

Las prácticas recientes en el campo de la inteligencia artificial han llevado a muchos a pensar que se debe construir un solo modelo masivo destinado a aprender la distribución de todos los datos y memorizar todo el conocimiento. Si bien esta aproximación es técnicamente factible, dicha solución resulta probablemente innecesaria y poco eficiente. Al igual que en el entrenamiento de redes neuronales profundas, el único método escalable conocido para entrenar tales redes a escala masiva es la retropropagación (BP) [RHW86b]. Aunque la retropropagación (BP) proporciona un mecanismo para corregir errores mediante señales de gradiente propagadas hacia atrás a través del modelo entero, se trata, sin embargo, de un enfoque puramente de fuerza bruta que difiere sustancialmente de cómo aprende la naturaleza: la BP representa una opción que la naturaleza no puede permitirse debido a su alto costo energético y que, simplemente, no puede implementarse debido a limitaciones físicas.

De modo más general, no podemos comprender realmente la inteligencia sin entender también cómo se puede implementar en forma eficiente. Es decir, se requiere abordar la complejidad computacional inherente a la realización de los mecanismos necesarios para alcanzar los objetivos de la inteligencia. Históricamente, nuestra comprensión de la inteligencia (artificial) ha evolucionado mediante varias fases: desde la complejidad de Kolmogorov —incomputable— hacia la entropía de Shannon; desde la computabilidad de Turing hacia la posterior noción de manejabilidad,[3] y hasta el fuerte énfasis en la escalabilidad de los algoritmos en la práctica moderna del análisis de datos de alta dimensión [WM22] y la inteligencia artificial. Dicha evolución se puede resumir de la siguiente manera:

$$\textbf{incomputable} \implies \textbf{computable} \implies \textbf{manejable} \implies \textbf{escalable}. \quad (9.2.1)$$

En gran medida, el éxito y la popularidad del aprendizaje profundo y la retropropagación se deben precisamente a que han ofrecido una implementación escalable con plataformas informáticas modernas (como las GPUs) para procesar

[3] Decimos que un problema es manejable si permite un algoritmo cuya complejidad es polinómica en el tamaño del problema.

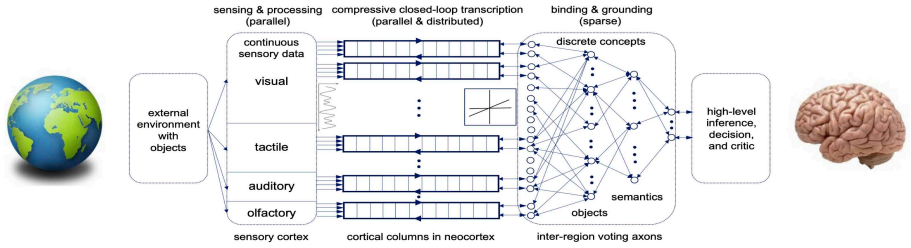


Figura 9.2: Estructura hipotética de la corteza cerebral. La corteza es un sistema de autocodificación masivamente paralelo y distribuido que consiste en una jerarquía de autocodificadores de bucle cerrado. Estos extraen información de sus entradas y maximizan la ganancia de información de las representaciones que generan. Dichos procesos se llevan a cabo en múltiples niveles de la jerarquía con distintos grados de abstracción de la información.

y comprimir grandes volúmenes de datos. Sin embargo, dicha implementación sigue siendo mucho más costosa que la forma en que la naturaleza llega a la inteligencia.

Aún queda un margen considerable para mejorar la eficiencia de la inteligencia artificial, de modo que pueda emular la eficiencia de la inteligencia natural, la cual debería ser varios órdenes de magnitud superior a las actuales implementaciones basadas en la fuerza bruta. Para ello, necesitamos descubrir nuevas arquitecturas de aprendizaje y mecanismos de optimización que permitan aprender distribuciones de datos en condiciones físicas naturales y bajo restricciones de recursos, similares a las que enfrentan los seres inteligentes en la naturaleza; por ejemplo, sin acceder a todos los datos de una sola vez ni actualizar todos los parámetros del modelo simultáneamente (mediante BP).

El marco teórico y el enfoque que se exponen en este libro pueden servirnos de guía para descubrir esas nuevas arquitecturas y mecanismos. Estas nuevas arquitecturas y mecanismos deberían permitir el aprendizaje continuo en línea y deberían poder actualizarse mediante optimización hacia adelante o hacia atrás altamente localizada y esparsa.[4]

Tal y como nos ha enseñado la neurociencia, la corteza de nuestro cerebro está formada por decenas de miles de columnas corticales [Haw21]. Todas las columnas corticales tienen estructuras físicas y funciones similares. Son altamente paralelas y están distribuidas, aunque están interconectadas en forma esparsa. Por lo tanto, creemos que para desarrollar un sistema de memoria más escalable y estructurado, debemos considerar arquitecturas que emulen

[4]Para aprender una distribución, una aplicación sencilla de estos requisitos se da en el caso más simple del PCA, con el método de PCA en línea presentado en el Capítulo 6. Una vez aprendido el modelo lineal de la distribución, la tarea más sencilla de realizar predicciones en línea con observaciones ruidosas puede llevarse a cabo fácilmente mediante métodos de mínimos cuadrados , como el filtro de Wiener [Wie42; Wie49]. En el caso de los sistemas dinámicos lineales, esto se convierte en el famoso filtro de Kalman; véase [MKS+04, Apéndice B] o [PV25].

las de la corteza. La Figura 9.2 muestra una arquitectura hipotética de este tipo: un sistema masivamente distribuido y jerárquico compuesto por numerosos módulos de autocodificación de bucle cerrado que funcionan en gran medida en paralelo.[5] Estos módulos aprenden a codificar diferentes modalidades sensoriales o múltiples proyecciones de datos de cada modalidad sensorial. Nuestro análisis en la Sección 7.5 del Capítulo 7 sugiere que dicha detección y aprendizaje paralelos de una distribución de baja dimensión son teóricamente posibles. Luego se pueden entrenar autocodificadores de nivel superior (con pérdida) basándose en las salidas de los de nivel inferior para desarrollar «abstracciones» más esparsas y de mayor nivel de las representaciones aprendidas por los niveles inferiores.

La arquitectura de sistema distribuida, jerárquica y de bucle cerrado que se ilustra en la Figura 9.2 comparte muchas características con la corteza cerebral. Dicha arquitectura de sistema puede abrir muchas más posibilidades que la actual arquitectura de un único modelo grande. Permite explorar mecanismos de aprendizaje y optimización mucho más eficientes y da como resultado una organización modular más estructurada de la distribución de los datos aprendidos y el conocimiento, emulando más fielmente la memoria de nuestro cerebro. Esto nos permitiría llevar la implementación de la inteligencia artificial al siguiente nivel de evolución:

$$\textbf{incomputable} \Longrightarrow \textbf{computable} \Longrightarrow \textbf{manejable} \Longrightarrow \textbf{escalable} \Longrightarrow \textbf{natural}. \tag{9.2.2}$$

9.3 Hacia la inteligencia científica: ¿más allá del test de Turing?

9.3.1 La evolución de la inteligencia en la naturaleza

Como hemos comentado al principio de este libro, en el Capítulo 1, la inteligencia en la naturaleza ha evolucionado a lo largo de múltiples fases y se ha manifestado en cuatro formas distintas:

$$\textbf{inteligencia filogenética} \Longrightarrow \textbf{ontogenética} \Longrightarrow \textbf{social} \Longrightarrow \textbf{científica}. \tag{9.3.1}$$

Todas las formas de inteligencia comparten el objetivo (y la característica) común de adquirir conocimientos útiles en forma de distribuciones de baja dimensión a partir de datos de alta dimensión sobre el mundo que se perciben. No obstante, difieren en los esquemas de codificación específicos que adoptan, la información codificada, los mecanismos computacionales para el aprendizaje y la mejora, y las implementaciones físicas de dichos mecanismos. Según los conceptos y la terminología desarrollados en este libro, las cuatro etapas de la inteligencia se diferencian en los tres aspectos siguientes:

[5] En la bibliografía existen elementos de este tipo de arquitecturas hipotéticas en diversas formas, como las redes de codificación predictiva [MSS+22], que cuentan con una estrategia de optimización que utiliza actualizaciones locales eficientes.

1. El *libro de códigos* o esquema utilizado para codificar la información o el conocimiento que se desea transmitir.

2. La *información* o los conocimientos adquiridos y codificados mediante el libro de códigos.

3. Los *mecanismos de optimización* utilizados para crear y corregir la información o el conocimiento codificados.

La Tabla 9.1 resume sus principales características.

	Filogenéico	**Ontogenético**	**Social**	**Científico**
Libro de código	ADNs	Neuronas/Mente	Lenguajes Naturales	Abstracciones matemáticas
Información	Genes	Memoris	Conocimiento empírico	Hechos científicos
Optimización	Refuerzo	Retroalimentación de error	Prueba & error	Teorizar & Falsar

Tabla 9.1: Características principales de las cuatro etapas de la inteligencia en la naturaleza.

Como ya sabemos, los seres humanos han logrado dos avances decisivos en materia de inteligencia.

1. El primero fue el desarrollo del lenguaje oral y escrito, lo que permitió a los seres humanos compartir y transmitir el conocimiento adquirido entre generaciones, de manera muy similar a como lo hace el ADN en la naturaleza.

2. El segundo fue el desarrollo de las matemáticas y la lógica formal hace aproximadamente tres mil años, que se convirtieron en el lenguaje preciso de la ciencia moderna.

El lenguaje de las matemáticas nos liberó de resumir el conocimiento a partir de las observaciones únicamente en forma empírica y nos permitió formalizar el conocimiento en forma de teorías verificables o falsificables mediante la deducción matemática o la experimentación. A través de la formulación de hipótesis, la deducción lógica y la verificación o falsificación experimental, ahora podemos descubrir y desarrollar de manera proactiva nuevos conocimientos que van mucho más allá de lo que se puede aprender a partir de las distribuciones de los datos observados. Por ejemplo, las relaciones causales no pueden deducirse únicamente a partir de las distribuciones [Pea09].

A pesar de las aparentes diferencias entre estas distintas etapas de la inteligencia, la evolución de la inteligencia comparte una característica común: *Todos ellos utilizan ciertas formas de retroalimentación*[6] *para crear y corregir la información aprendida, y la retroalimentación y la corrección son cada vez más frecuentes y eficientes a lo largo de la evolución: desde el refuerzo de bucle abierto hasta la autosupervisión de bucle cerrado, desde el ensayo y error colectivo hasta la formulación proactiva de hipótesis y su refutación.*

[6] Pérdida por error y recompensa por éxito.

9.3.2 De la inteligencia inductiva a la deductiva

Como se mencionó en la introducción (Capítulo 1), el programa de «inteligencia artificial» (IA) de 1956 tenía precisamente como objetivo estudiar cómo dotar a las máquinas de inteligencia científica, es decir, de funciones de alto nivel como la abstracción matemática, la inferencia lógica y la resolución de problemas, que se cree que diferencian a los seres humanos de los animales:

$$\text{inteligencia (animal) de } \textbf{bajo nivel} \implies \text{inteligencia (humana) de } \textbf{alto nivel}. \tag{9.3.2}$$

Como hemos aclarado en repetidas ocasiones en este libro, la mayoría de los avances tecnológicos en inteligencia artificial durante la última década, aunque se hayan llevado a cabo bajo el nombre de «AI», están en realidad más estrechamente relacionados con el hecho de que las máquinas imiten formas de inteligencia de bajo nivel que comparten en gran medida tanto los animales como los seres humanos, incluyendo la inteligencia filogenética, ontogenética y social. En estos niveles, la inteligencia crea memoria o conocimiento *empírico* (a partir de distribuciones de datos empíricos) y lleva a cabo inferencias que son, en su mayoría, de naturaleza *inductiva*. Los principios y métodos presentados en este libro tienen como objetivo revelar los objetivos científicos y los mecanismos computacionales que subyacen a la inteligencia en estos niveles. Estos proporcionan una sólida justificación teórica y evidencia computacional para la afirmación:

Las máquinas pueden adquirir conocimientos empíricos y realizar inferencias inductivas.

Sin embargo, hasta ahora no hay pruebas teóricas ni científicas rigurosas que indiquen que estos mecanismos por sí solos bastarían para alcanzar el alto nivel de inteligencia humana que el programa original de IA realmente pretendía comprender y quería que las máquinas imitaran o incluso superaran.

De hecho, al día de hoy, sabemos muy poco sobre cómo verificar o certificar de manera rigurosa si un sistema es realmente capaz de alcanzar un nivel tan alto de inteligencia, a pesar de que la Prueba de Turing, también conocido como el Juego de la Imitación,[7] se propuso en 1950 [Tur50], como un primer intento de abordar la pregunta fundamental:

¿Pueden pensar las máquinas?

Durante mucho tiempo, no se consideró necesario establecer una definición precisa de dicha prueba, ya que las capacidades de las máquinas estaban muy por debajo de las de los seres humanos (o incluso de las de los animales). Sin embargo, dados los recientes avances tecnológicos, muchos modelos y sistemas afirman ahora alcanzar o incluso superar la inteligencia humana. Por lo tanto, ya es hora de desarrollar una definición científica y aplicable de la prueba de

[7] En la propuesta de Turing, el evaluador, o interrogador, es un ser humano. Sin embargo, la mayoría de los evaluadores humanos tienen una formación y unos conocimientos científicos limitados, y sus conclusiones pueden ser subjetivas.

Turing, es decir, un protocolo sistemático y riguroso para evaluar y certificar el nivel de inteligencia de un modelo o sistema que afirme ser «inteligente».

Como argumentó Turing en su artículo original, resulta bastante difícil definir qué es «pensar». En su lugar, podríamos partir de algo más tangible e intentar determinar si un sistema es realmente capaz de «comprender» ciertos conceptos o conocimientos. Por ejemplo, ¿cómo podemos verificar rigurosamente si un modelo o sistema inteligente ha comprendido realmente un concepto abstracto como:

1. el concepto de igualdad entre dos cantidades,
2. el concepto de los números (naturales, racionales o reales),
3. el concepto de infinito y la inducción matemática,
4. el concepto de consistencia lógica[8] y la demostración por contradicción,

por mencionar solo algunos? ¿O simplemente ha memorizado una enorme cantidad de ejemplos de tales conceptos? Cabe señalar que los modelos de lenguaje de última generación siguen teniendo dificultades con preguntas matemáticas sencillas como: «¿Es 3,11 mayor o menor que 3,9?»[9]

Gracias a este libro, sabemos que los sistemas modernos de IA generativa se basan principalmente en la inferencia bayesiana (y, por lo tanto, inductiva) para generar sus respuestas. ¿Pueden las máquinas también comprender verdaderamente la inducción matemática o la deducción lógica general y entender su diferencia con respecto a la inducción bayesiana? ¿Cómo verificamos si un sistema comprende realmente las reglas de la deducción lógica y matemática y es capaz de aplicarlas con rigor? ¿O, una vez más, simplemente ha memorizado una gran cantidad de ejemplos de tales deducciones —como datos sobre la cadena de pensamiento— y ha aprendido y aprovechado sus patrones de distribución? Por lo tanto, nos queda una pregunta sin respuesta:

¿Hay alguna diferencia entre ***memorizar*** *y* ***comprender****?*

Solo conociendo la diferencia podremos intentar dar una respuesta significativa a la pregunta: *«¿Pueden las máquinas entender?»*

Probablemente, lo más importante sea preguntarnos: ¿cómo verificamos si un sistema que se autodenomina inteligente es capaz de *crear conceptos abstractos o mejorar su propio conocimiento* de manera proactiva mediante

[8] En matemáticas, si un conjunto de axiomas y sus resultados deducidos son consistentes y no se contradicen entre sí, entonces existen. Esto se conoce como el Principio de Hilbert. Esto difiere de la noción de «consistencia» del capítulo 6, que requiere que las representaciones aprendidas sean consistentes con las distribuciones de datos empíricos. Una noción de consistencia es de naturaleza inductiva y la otra es deductiva.

[9] Algunos modelos han corregido sus respuestas a este tipo de preguntas mediante ingeniería específica, o han incorporado mecanismos de razonamiento adicionales que verifican las respuestas inmediatas y las corrigen durante la inferencia. Sin embargo, dejamos al lector la tarea de comprobar rigurosamente si algún modelo lingüístico de vanguardia comprende realmente el concepto de los números (naturales, racionales, reales y complejos) y su aritmética asociada.

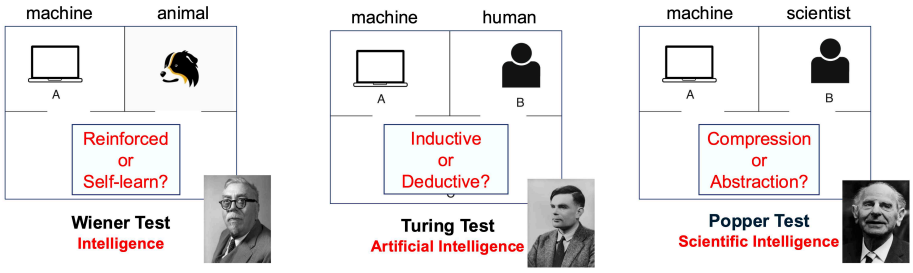


Figura 9.3: Tres pruebas para diferentes niveles o tipos de capacidades cognitivas: la Prueba de Wiener para la inteligencia básica, la Prueba de Turing para la inteligencia de nivel humano y la Prueba de Popper para la inteligencia de nivel científico.

métodos deductivos? Por ejemplo, ¿puede crear nuevos conceptos matemáticos y descubrir nuevas leyes físicas o relaciones causales? La conexión entre comprender y crear puede ser bastante fundamental, como lo expresó de manera célebre el físico Richard Feynman:

«Lo que no puedo crear, no lo entiendo».

9.3.3 Pruebas científicas de inteligencia

Por lo tanto, ya es hora de que desarrollemos métodos de evaluación con base científica para determinar a cuál de las siguientes categorías pertenece la capacidad inteligente de un sistema:

Nivel-1. el mero hecho de haber aprendido la distribución de ciertos datos que contienen información y ser capaz de regenerarlos;

Nivel-2. ser capaz de aprender en forma autónoma y continua nuevos conocimientos o de corregir los conocimientos existentes a partir de nuevas observaciones y experiencias;

Nivel-3. ser verdaderamente capaz de comprender y crear conceptos, reglas y leyes abstractas, así como saber cómo aplicarlos correctamente de manera deductiva;

Nivel-4. ser capaz de formular nuevas hipótesis científicas o conjeturas matemáticas y verificarlas o refutarlas mediante la experimentación o la deducción lógica.

Para evaluar y distinguir estos diferentes niveles de inteligencia, sugerimos que probablemente deberían existir al menos tres pruebas diferentes, tal y como se ilustra en la Figura 9.3:

1. *La Prueba de Norbert Wiener:* para determinar si un sistema es capaz de autocorregirse y generar nuevo conocimiento (empírico) por sí mismo o si

simplemente recibe y actualiza información de manera pasiva a través de refuerzos o supervisión externos;

2. *La prueba de Alan Turing:* para determinar si un sistema es capaz de crear y comprender conceptos abstractos y conocimiento, o si simplemente aprende y memoriza estadísticas de las muestras y las utiliza para la inferencia bayesiana;

3. *La Prueba de Karl Popper:* para determinar si un sistema es capaz de explorar y crear en forma proactiva nuevo conocimiento abstracto mediante la formulación y verificación de hipótesis o teorías basadas en la consistencia experimental o la consistencia lógica.

Creemos que, para este tipo de pruebas, el evaluador o el interrogador no debería ser una sola persona ni un grupo arbitrario de personas, sino más bien una entidad con acreditación científica que siga un protocolo y un proceso científicamente sólidos, de modo que las conclusiones sean rigurosas y fiables.

Como hemos visto a lo largo de este libro, la *compresión* ha desempeñado un papel fundamental en el desarrollo de la memoria o en la creación de conocimiento empírico. Es el principio rector y un mecanismo universal para identificar una distribución de datos empíricos y organizar la información codificada en ellos mediante una representación estructurada. En gran medida, explica la práctica de la «inteligencia artificial» con redes profundas, que es en gran parte de naturaleza inductiva.[10] Una cuestión pendiente para futuros estudios es si *la compresión por sí sola* es suficiente para alcanzar toda la inteligencia deductiva de alto nivel mencionada anteriormente. En concreto, ¿son la abstracción matemática, la inferencia causal, la generación de hipótesis y el razonamiento y la deducción lógicos algún tipo de formas ampliadas o superadas de la compresión? ¿Existe alguna diferencia fundamental entre aprender distribuciones de datos a través de la compresión y formar conceptos y teorías de alto nivel mediante la abstracción? De ahí que tengamos otra pregunta sin respuesta:

¿Hay alguna diferencia entre la ***compresión*** *y la* ***abstracción****?*

Solo conociendo la diferencia podremos intentar dar una respuesta significativa a la pregunta: «¿Pueden las máquinas crear conocimiento en forma proactiva mediante la deducción?»

En gran medida, la ciencia —y su libro de códigos asociado, las matemáticas— puede considerarse como la forma más avanzada de inteligencia, exclusiva de los seres humanos cultos e ilustrados. El filósofo Sir Karl Popper sugirió en una ocasión:

«La ciencia puede describirse como el arte de la simplificación excesiva.»

[10] Los sistemas que poseen mecanismos deductivos integrados para llevar a cabo la verificación formal no entran en esta categoría.

Creemos que descubrir y comprender los principios matemáticos y los mecanismos computacionales subyacentes a esa inteligencia de alto nivel, así como reproducirlos con éxito mediante máquinas, será la última frontera para la ciencia, las matemáticas y la computación en su conjunto.

Apéndices

Apéndice A

Métodos de optimización

> "*Puesto que la estructura del universo es sumamente perfecta y es obra de un Creador sumamente sabio, no ocurre absolutamente nada en el universo en lo que no se manifieste alguna regla de máximo o mínimo.*"
>
> – L. Euler

En este capítulo, ofrecemos una breve introducción a algunos de los algoritmos de optimización más básicos pero importantes que se utilizan en este libro. El objetivo es únicamente ayudar al lector a aplicar estos algoritmos a los problemas que se estudian en este libro, no adquirir una comprensión profunda de dichos algoritmos. Por lo tanto, no proporcionaremos una justificación exhaustiva de los algoritmos presentados en términos de garantías de rendimiento.

A.1 Descenso por gradiente más pronunciado

La optimización se ocupa de la cuestión de cómo determinar en qué punto una función, por ejemplo $\mathcal{L}(\theta)$, alcanza su valor mínimo. Matemáticamente, esto se formula como un problema:

$$\arg\min_{\theta \in \Theta} \mathcal{L}(\theta), \tag{A.1.1}$$

donde Θ representa un dominio al que se limita el argumento θ. A menudo (y salvo que se indique lo contrario en este capítulo) Θ es simplemente $\mathbb{R}^n$. Sin pérdida de generalidad, suponemos que aquí la función $\mathcal{L}(\theta)$ es suave[1].

La eficacia a la hora de encontrar los mínimos (globales) depende de la información de que dispongamos sobre la función $\mathcal{L}$. Para la mayoría de los problemas de optimización que se tratan en este libro, la dimensión de θ, digamos

[1] En caso de que la función $\mathcal{L}$ no sea suave, reemplazamos su gradiente por lo que se conoce como *subgradiente*.

n, es muy grande. Esto hace que calcular o acceder a la información local sobre $\mathcal{L}$ resulte costoso. En particular, dado que el gradiente $\nabla\mathcal{L}$ tiene n entradas, a menudo resulta razonable calcularlo; sin embargo, la matriz hessiana $\nabla^2\mathcal{L}$ tiene n^2 entradas, lo que suele ser totalmente impracticable de calcular (y lo mismo ocurre con las derivadas de orden superior). Por lo tanto, es habitual suponer que contamos con la información de orden cero, es decir, que podemos evaluar $\mathcal{L}(\theta)$, y la información de primer orden, es decir, que podemos evaluar $\nabla\mathcal{L}(\theta)$. Los teóricos de la optimización podrían reformular esto diciendo que tenemos un *«oráculo de primer orden»*. Todos los algoritmos de optimización que presentamos en esta sección solo utilizan un oráculo de primer orden.[2]

A.1.1 Descenso por gradiente clásico para problemas suaves

El método más sencillo y más utilizado para la optimización es el *descenso por gradiente* (GD). Fue introducido por primera vez por Cauchy en 1847. La idea es muy sencilla: partiendo de un estado inicial, damos pequeños pasos en forma iterativa, de modo que cada paso reduzca el valor de la función $\mathcal{L}(\theta)$.

Supongamos que el estado actual es θ. Queremos dar un pequeño paso, digamos de longitud h, en una dirección indicada por un vector $\boldsymbol{v}$, para llegar a un nuevo estado $\theta + h\boldsymbol{v}$ de tal manera que el valor de la función disminuya:

$$\mathcal{L}(\theta + h\boldsymbol{v}) \leq \mathcal{L}(\theta). \tag{A.1.2}$$

Para hallar dicha dirección $\boldsymbol{v}$, podemos aproximar $\mathcal{L}(\theta + h\boldsymbol{v})$ mediante una expansión de Taylor en torno a $h = 0$:

$$\mathcal{L}(\theta + h\boldsymbol{v}) = \mathcal{L}(\theta) + h\langle\nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle + o(h), \tag{A.1.3}$$

donde el producto interno aquí (y en este capítulo) será el producto interno ℓ^2, es decir, $\langle\boldsymbol{x}, \boldsymbol{y}\rangle = \boldsymbol{x}^\top\boldsymbol{y}$. Para encontrar la dirección del *gradiente máximo*, intentamos minimizar esta expansión de Taylor entre los vectores unitarios $\boldsymbol{v}$. Si $\nabla\mathcal{L}(\theta) = \mathbf{0}$, entonces el segundo término anterior es 0 independientemente del valor de $\boldsymbol{v}$, por lo que no podemos intentar avanzar, es decir, el algoritmo ha convergido. Por otro lado, si $\nabla\mathcal{L}(\theta) \neq 0$, entonces se cumple que

$$\arg\min_{\substack{\boldsymbol{v}\in\mathbb{R}^d \\ \|\boldsymbol{v}\|_2=1}} [\mathcal{L}(\theta) + h\langle\nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle] = \arg\min_{\substack{\boldsymbol{v}\in\mathbb{R}^d \\ \|\boldsymbol{v}\|_2=1}} \langle\nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle = -\frac{\nabla\mathcal{L}(\theta)}{\|\nabla\mathcal{L}(\theta)\|_2}, \tag{A.1.4}$$

En otras palabras, esto significa que el valor de $\mathcal{L}(\theta + h\boldsymbol{v})$ disminuye más rápidamente en la dirección $\boldsymbol{v} = -\nabla\mathcal{L}(\theta)/\|\nabla\mathcal{L}(\theta)\|_2$, para valores de h suficientemente pequeños. Esto conduce al método de descenso por gradiente:

[2] Remitimos a los lectores al libro de [WM22] para una introducción más sistemática a los algoritmos de optimización en un espacio de alta dimensión, incluidos los algoritmos que asumen oráculos de orden superior.

a partir del estado actual θ_k $(k = 0, 1, \ldots)$, damos un paso de tamaño h en la dirección del gradiente negativo para llegar a la siguiente iteración,

$$\theta_{k+1} = \theta_k - h\nabla\mathcal{L}(\theta_k). \tag{A.1.5}$$

La magnitud del paso h también se conoce como *tasa de aprendizaje* en los contextos de aprendizaje automático.

Selección de la magnitud del paso

La pregunta que queda por resolver es: ¿cuál debe ser la magnitud del paso h? Si elegimos un h demasiado pequeño, el valor de la función puede disminuir muy lentamente, como se muestra en el gráfico del centro de Figura A.1. Si h es demasiado grande, es posible que el valor ni siquiera disminuya, como lo muestra el gráfico de la derecha en la Figura A.1.

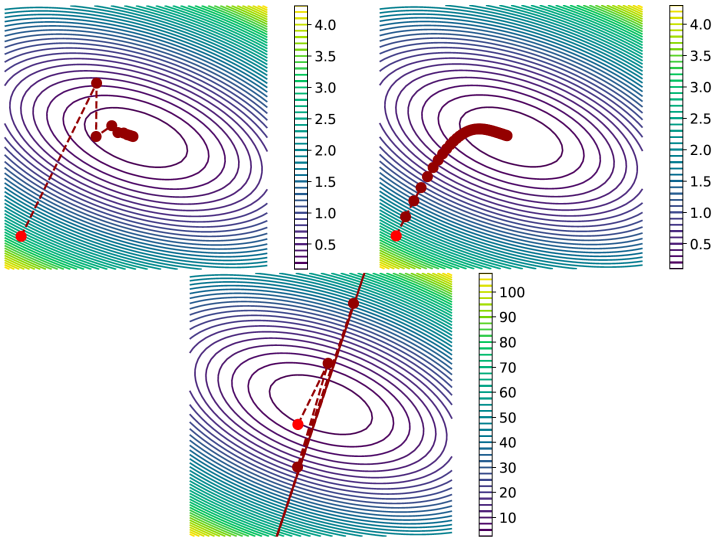


Figura A.1: El efecto del tamaño de paso (constante) h en la convergencia del método de descenso del gradiente.

Por lo tanto, la magnitud del paso h debe elegirse en función del paisaje de la función $\mathcal{L}(\theta_k)$. Lo ideal, para elegir el mejor tamaño de paso h, es resolver el siguiente problema de optimización sobre una sola variable h:

$$h = \underset{h' \geq 0}{\arg\min}\, \mathcal{L}(\theta_k - h'\nabla\mathcal{L}(\theta_k)). \tag{A.1.6}$$

Este método para elegir el tamaño del paso se denomina *búsqueda lineal*; como sugiere la notación, se suele utilizar para obtener una tasa de aprendizaje óptima *para cada iteración* k. Sin embargo, cuando la función $\mathcal{L}(\theta_k)$ es complicada, lo cual suele ser el caso al entrenar una red neuronal profunda, esta optimización

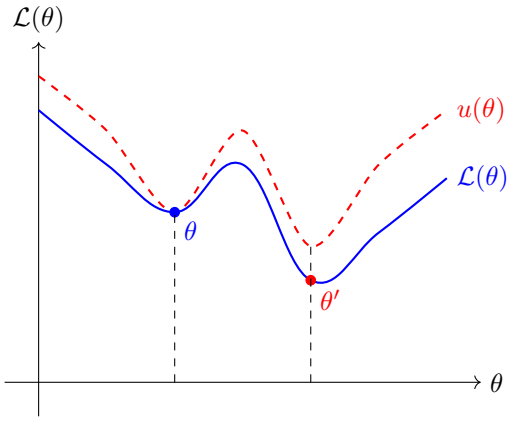


Figura A.2: **Esquema e intuición de mayorización-minimización.** Una función $\mathcal{L}\colon \Theta \to \mathbb{R}$ tiene una cota superior global $u\colon \Theta \to \mathbb{R}$ que se cruza con $\mathcal{L}$ en al menos un punto θ. Entonces, encontrar el θ' que minimiza u mejorará el valor de $\mathcal{L}$ a partir de $\mathcal{L}(\theta)$. Obsérvese que se pueden demostrar resultados similares sobre las cotas superiores locales.

unidimensional es muy difícil de resolver en cada iteración del descenso del gradiente.

Entonces, ¿cómo debemos elegir un tamaño de paso adecuado h? Un enfoque común y clásico consiste en intentar obtener una buena aproximación del paisaje local alrededor del estado actual θ basándose en cierto conocimiento sobre el paisaje general de la función $\mathcal{L}(\theta)$.

Entre las condiciones comunes para el conjunto de definiciones de $\mathcal{L}(\theta)$ se cuentan:

- Convexidad α-fuerte. Recordemos que $\mathcal{L}$ es *α-fuertemente convexa* si su gráfica se encuentra por encima de una cota inferior cuadrática global de pendiente α, es decir,

$$\mathcal{L}(\theta') \geq l_{\theta,\alpha}(\theta') \doteq \mathcal{L}(\theta) + \langle \nabla\mathcal{L}(\theta), \theta' - \theta \rangle + \frac{\alpha}{2}\|\theta' - \theta\|_2^2 \qquad \text{(A.1.7)}$$

  para cualquier «punto base» θ. Decimos que $\mathcal{L}$ es *convexa* si es fuertemente convexa en 0, es decir, si su gráfico se encuentra por encima de sus tangentes. Es fácil demostrar (la demostración se deja como ejercicio) que las funciones fuertemente convexas tienen un único mínimo global. Otro hecho importante (demostración como ejercicio) es que las funciones dos veces diferenciables y fuertemente convexas en α $\mathcal{L}$ tienen Hessianos (simétricos) $\nabla^2\mathcal{L}$ cuyo valor propio mínimo es $\geq \alpha$. Para $\alpha > 0$ esto implica que la matriz hessiana es simétrica y definida positiva, y para $\alpha = 0$ (es decir, $\mathcal{L}$ es convexo) esto implica que la matriz hessiana es simétrica y semidefinida positiva.

- Gradiente β-Lipschitz (también denominado suavidad β). Recordemos que $\mathcal{L}$ tiene un *β-gradiente Lipschitz* si $\nabla\mathcal{L}$ existe y es β-Lipschitz, es decir,
$$\|\nabla\mathcal{L}(\theta') - \nabla\mathcal{L}(\theta)\|_2 \leq \beta\|\theta' - \theta\|_2. \tag{A.1.8}$$
para cualquier «punto base» θ. Es fácil demostrar (la demostración queda como ejercicio) que esto equivale a que $\mathcal{L}$ tenga una cota superior cuadrática global de pendiente β, es decir,
$$\mathcal{L}(\theta') \leq u_{\theta,\beta}(\theta') \doteq \mathcal{L}(\theta) + \langle\nabla\mathcal{L}(\theta), \theta' - \theta\rangle + \frac{\beta}{2}\|\theta' - \theta\|_2^2. \tag{A.1.9}$$
para cualquier «punto base» θ. Otro hecho importante (demostración como ejercicio) es que las funciones convexas β-Lipschitz con gradiente dos veces diferenciable tienen hessianas (simétricas) $\nabla^2\mathcal{L}$ cuyo mayor valor propio es $\leq \beta$.

En primer lugar, supongamos que $\mathcal{L}$ tiene un gradiente β-Lipschitz (aunque no sea necesariamente convexo). Aprovecharemos esta ocasión para introducir un tema común en optimización: *para minimizar $\mathcal{L}$, podemos minimizar una cota superior de $\mathcal{L}$*, lo cual se justifica por el siguiente lema visualizado en la Figura A.2.

Lema A.1 (Mayorización-Minimización)**.** *Supongamos que $u\colon \Theta \to \mathbb{R}$ es una cota superior global de $\mathcal{L}$, es decir, $\mathcal{L}(\theta) \leq u(\theta)$ para todo $\theta \in \Theta$. Supongamos que se igualan en θ, es decir, $\mathcal{L}(\theta) = u(\theta)$. Entonces*
$$\theta^+ \in \arg\min_{\theta' \in \Theta} u(\theta') \implies \mathcal{L}(\theta^+) \leq u(\theta^+) \leq u(\theta) = \mathcal{L}(\theta). \tag{A.1.10}$$

Utilizaremos este lema para demostrar que podemos recurrir a la propiedad Lipschitz del gradiente para garantizar que cada paso del gradiente no empeore el valor de $\mathcal{L}$. De hecho, en cada punto base θ, tenemos que $u_{\theta,\beta}$ es un límite superior global de $\mathcal{L}$, y $u_{\theta,\beta}(\theta) = \mathcal{L}(\theta)$. Por lo tanto, por el Lema A.1

$$\text{si } \theta^+ \text{ minimiza } u_{\theta,\beta} \text{ entonces} \quad \mathcal{L}(\theta^+) \leq u_{\theta,\beta}(\theta^+) \leq u_{\theta,\beta}(\theta) = \mathcal{L}(\theta). \tag{A.1.11}$$

Esto nos lleva a que, al buscar una actualización para obtener θ_{k+1} a partir de θ_k, podamos, en su lugar, minimizar el límite superior $u_{\theta_k,\beta}$ sobre θ y establecer que ese valor sea θ_{k+1}. Al minimizar $u_{\theta_k,\beta}$ (demostración como ejercicio) obtenemos
$$\theta_{k+1} = \theta_k - \frac{1}{\beta}\nabla\mathcal{L}(\theta_k) \implies \mathcal{L}(\theta_{k+1}) \leq \mathcal{L}(\theta_k). \tag{A.1.12}$$

Esto implica que un tamaño de paso $h = 1/\beta$ es una tasa de aprendizaje válida, pero no proporciona una tasa de convergencia ni garantiza que $\mathcal{L}(\theta_k)$ converja realmente a $\min_\theta \mathcal{L}(\theta)$. Esto requiere un poco más de rigor, lo que abordaremos a continuación.

Ahora, supongamos que $\mathcal{L}$ es α-fuertemente convexa, tiene un gradiente β-Lipschitz y posee un óptimo global $\theta^\star$. Demostraremos que θ_k convergerá

directamente al único óptimo global $\theta^\star$, lo cual constituye una forma muy fuerte de convergencia. En particular, acotaremos $\|\theta^\star - \theta_k\|_2$ utilizando tanto la convexidad fuerte como la propiedad de Lipschitz del gradiente de $\mathcal{L}$, es decir, analizando el entorno alrededor de θ_k:[3]

$$\|\theta^\star - \theta_{k+1}\|_2^2 \leq \|\theta^\star - \theta_k + h\nabla\mathcal{L}(\theta_k)\|_2^2 \tag{A.1.13}$$

$$= \|\theta^\star - \theta_k\|_2^2 + 2h\langle\nabla\mathcal{L}(\theta_k), \theta^\star - \theta_k\rangle + h^2\|\nabla\mathcal{L}(\theta_k)\|_2^2 \tag{A.1.14}$$

$$\leq \|\theta^\star - \theta_k\|_2^2 + 2h\left(\mathcal{L}(\theta^\star) - \mathcal{L}(\theta_k) - \frac{\alpha}{2}\|\theta^\star - \theta_k\|_2^2\right) + h^2\|\nabla\mathcal{L}(\theta_k)\|_2^2 \quad (\alpha\text{-SC}) \tag{A.1.15}$$

$$= (1 - \alpha h)\,\|\theta^\star - \theta_k\|_2^2 + 2h(\mathcal{L}(\theta^\star) - \mathcal{L}(\theta_k)) + h^2\|\nabla\mathcal{L}(\theta_k)\|_2^2 \tag{A.1.16}$$

$$\leq (1 - \alpha h)\,\|\theta^\star - \theta_k\|_2^2 + 2h(\mathcal{L}(\theta^\star) - \mathcal{L}(\theta_k)) + 2h^2\beta(\mathcal{L}(\theta_k) - \mathcal{L}(\theta^\star)) \quad (\beta\text{-LG}) \tag{A.1.17}$$

$$= (1 - \alpha h)\,\|\theta^\star - \theta_k\|_2^2 - 2h(1 - \beta h)(\mathcal{L}(\theta_k) - \mathcal{L}(\theta^\star)). \tag{A.1.18}$$

Para garantizar que la iteración del descenso por gradiente avance, debemos elegir el tamaño del paso de tal manera que $1 - \beta h \geq 0$, es decir, $h \leq 1/\beta$. Si se da esta condición, entonces

$$\|\theta^\star - \theta_{k+1}\|_2^2 \leq (1 - \alpha h)\|\theta^\star - \theta_k\|_2^2 \leq (1 - \alpha h)^2\|\theta^\star - \theta_{k-1}\|_2^2 \leq \cdots \tag{A.1.19}$$

$$\leq (1 - \alpha h)^{k+1}\|\theta^\star - \theta_0\|_2^2. \tag{A.1.20}$$

Para simplificar el lado derecho, podemos poner $h = 1/\beta$, lo que nos da

$$\|\theta^\star - \theta_{k+1}\|_2^2 \leq (1 - \alpha/\beta)^{k+1}\|\theta^\star - \theta_0\|_2^2, \tag{A.1.21}$$

que muestra una convergencia exponencial hacia el óptimo global con un error que decrece exponencialmente. Obsérvese que aquí hemos utilizado una tasa de convergencia para obtener un *tamaño de paso* adecuado de $h = 1/\beta$. Este patrón volverá a aparecer en esta sección.

Terminamos esta sección con una advertencia: encontrar un óptimo global es (por lo general) una tarea prácticamente imposible. Bajo ciertas condiciones, podemos asegurar que las iteraciones del descenso por gradiente converjan hacia un *óptimo local*. Además, bajo condiciones más flexibles, podemos asegurar la convergencia *local*, es decir, que las iteraciones converjan a un óptimo (global o local) si la secuencia se inicializa lo suficientemente cerca del óptimo.

A.1.2 Descenso por gradiente precondicionado para problemas mal condicionados

Método de Newton

Hay algunos problemas suaves y otros fuertemente convexos en los que, sin embargo, el descenso por gradiente obtiene resultados poco satisfactorios. Por

[3] En esta demostración, el paso en el que se recurre al gradiente β-Lipschitz es un poco complicado. También dejamos este paso como ejercicio, con la pista de sustituir $\theta = \theta_0 - h\nabla\mathcal{L}(\theta_0)$ en la identidad del gradiente de Lipschitz.

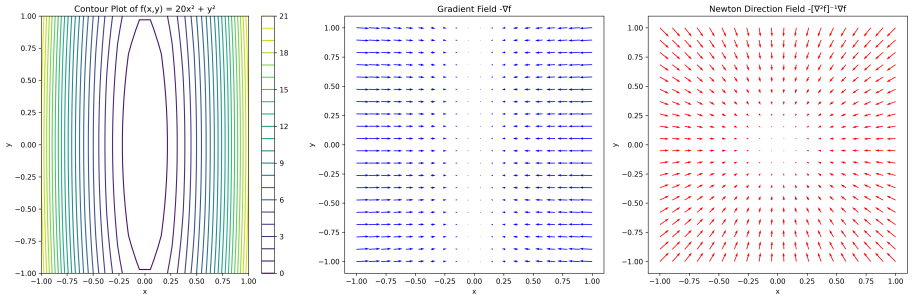


Figura A.3: **El gradiente negativo $-\nabla\mathcal{L}_\lambda$ y el campo vectorial precondicionado (paso del método de Newton)** $-[\nabla^2\mathcal{L}_\lambda]^{-1}[\nabla\mathcal{L}_\lambda]$ donde $\lambda = 19$. Hay una zona del espacio en la que seguir el campo vectorial del gradiente negativo apenas permite avanzar hacia el mínimo, pero en todos los casos, seguir el campo vectorial del método de Newton permite avanzar al mismo ritmo hacia el óptimo, ya que el gradiente se ha «blanqueado». Dado que el hessiano en este caso es diagonal, los algoritmos de tasa de aprendizaje adaptativa (por ejemplo, Adam, como se verá más adelante en esta sección) pueden lograr un progreso similar al del método de Newton, pero un hessiano no alineado con el eje podría incluso impedir que Adam alcance el éxito rápidamente.

ejemplo, sea $\lambda \geq 0$ y sea $\mathcal{L}_\lambda \colon \mathbb{R}^2 \to \mathbb{R}$ de la siguiente forma

$$\mathcal{L}_\lambda(\theta) = \mathcal{L}_\lambda\left(\begin{bmatrix}\theta_1\\ \theta_2\end{bmatrix}\right) \doteq \frac{1}{2}\left\{(1+\lambda)\theta_1^2 + \theta_2^2\right\} = \frac{1}{2}\theta^\top \begin{bmatrix}1+\lambda & 0\\ 0 & 1\end{bmatrix}\theta. \tag{A.1.22}$$

Este problema es 1-fuertemente convexo y tiene un gradiente $(1+\lambda)$-Lipschitz. Por lo tanto, la tasa de convergencia es geométrica con una tasa de $1-1/(1+\lambda)$. Para valores grandes de λ, esto sigue sin ser muy rápido. En esta sección, presentaremos una clase de problemas de optimización que pueden optimizar con éxito este tipo de funciones mal condicionadas.

La clave reside en la *curvatura* del objetivo, que viene dada por la matriz hessiana. Supongamos (a modo de hipótesis) que tuviéramos un oráculo de *segundo orden* que nos permitiera calcular $\mathcal{L}(\theta)$, $\nabla\mathcal{L}(\theta)$ y $\nabla^2\mathcal{L}(\theta)$. Entonces, en lugar de elegir una dirección de descenso $\boldsymbol{v}$ para optimizar la expansión de Taylor de primer orden alrededor de θ, podríamos optimizar la expansión de Taylor de segundo orden en su lugar. Intuitivamente, esto nos permitiría incorporar información sobre la curvatura en la actualización.

Hagamos este cálculo. La expansión de Taylor de segundo orden de $\mathcal{L}(\theta+h\boldsymbol{v})$ en torno a $h=0$ es

$$\mathcal{L}(\theta + h\boldsymbol{v}) = \mathcal{L}(\theta) + h\langle\nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle + \frac{1}{2}h^2\langle[\nabla^2\mathcal{L}(\theta)]\boldsymbol{v}, \boldsymbol{v}\rangle + o(h^2). \tag{A.1.23}$$

Entonces podemos calcular la dirección de descenso:

$$\operatorname*{arg\,min}_{\substack{\boldsymbol{v}\in\mathbb{R}^n\\ \|\boldsymbol{v}\|_2=1}} \left[\mathcal{L}(\theta) + h\langle\nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle + \frac{1}{2}h^2\langle[\nabla^2\mathcal{L}(\theta)]\boldsymbol{v}, \boldsymbol{v}\rangle\right] \tag{A.1.24}$$

$$= \arg\min_{\substack{\boldsymbol{v}\in\mathbb{R}^n \\ \|\boldsymbol{v}\|_2=1}} \left[\langle \nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle + \frac{1}{2} h \langle [\nabla^2\mathcal{L}(\theta)]\boldsymbol{v}, \boldsymbol{v}\rangle \right]. \tag{A.1.25}$$

Este problema de optimización resulta complicado de resolver debido a la restricción $\|\boldsymbol{v}\|_2 = 1$. Pero, en la práctica, nunca normalizamos la dirección de descenso $\boldsymbol{v}$ y utilizamos el tamaño del paso h para controlar la magnitud de la actualización. Así que resolvamos simplemente el problema anterior para todos los vectores $\boldsymbol{v} \in \mathbb{R}^n$:[4]

$$\arg\min_{\boldsymbol{v}\in\mathbb{R}^n} \left[\langle \nabla\mathcal{L}(\theta), \boldsymbol{v}\rangle + \frac{1}{2} h \langle [\nabla^2\mathcal{L}(\theta)]\boldsymbol{v}, \boldsymbol{v}\rangle \right] = -\frac{1}{h}[\nabla^2\mathcal{L}(\theta)]^{-1}[\nabla\mathcal{L}(\theta)]. \tag{A.1.26}$$

Por lo tanto, podemos utilizar la iteración del gradiente máximo

$$\theta_{k+1} = \theta_k - [\nabla^2\mathcal{L}(\theta_k)]^{-1}[\nabla\mathcal{L}(\theta_k)], \tag{A.1.27}$$

(este es el famoso *método de Newton*), o bien

$$\theta_{k+1} = \theta_k - h[\nabla^2\mathcal{L}(\theta_k)]^{-1}[\nabla\mathcal{L}(\theta_k)], \tag{A.1.28}$$

(conocido como *método de Newton subamortiguado*). Dado que el término cuadrático de segundo orden $\mathcal{L}_\lambda$ es igual a su, si aplicamos el método de Newton durante *un paso*, alcanzaremos el mínimo global en *un paso* sin importar cuán grande sea λ. La Figura A.3 ofrece una idea intuitiva sobre las funciones mal condicionadas y los pasos del gradiente frente a los pasos de Newton.

PGD

En la práctica, *no* disponemos de un oráculo de segundo orden que nos permita calcular $\nabla^2\mathcal{L}(\theta)$. En su lugar, podemos intentar *aprender una aproximación de este* junto con la actualización del parámetro θ_{k+1} a partir de θ_k.

¿Cómo podemos obtener una aproximación a ella? Encontraremos algunas ecuaciones que la inversa de la matriz hessiana satisfaga y luego intentaremos actualizar nuestra aproximación para que satisfaga dichas ecuaciones. Concretamente, al tomar la serie de Taylor de $\nabla\mathcal{L}(\theta + \delta_\theta)$ en torno al punto θ, obtenemos

$$\underbrace{\nabla\mathcal{L}(\theta + \delta_\theta) - \nabla\mathcal{L}(\theta)}_{\doteq \delta_{\boldsymbol{g}}} = [\nabla^2\mathcal{L}(\theta)]\delta_\theta + o(\|\delta_\theta\|_2). \tag{A.1.29}$$

En este caso tenemos

$$\delta_{\boldsymbol{g}} \approx [\nabla^2\mathcal{L}(\theta)]\delta_\theta \implies \delta_\theta \approx [\nabla^2\mathcal{L}(\theta)]^{-1}\delta_{\boldsymbol{g}} \tag{A.1.30}$$

Ahora podemos intentar encontrar una matriz precondicionadora simétrica semidefinida positiva $P \in \mathbb{R}^{n\times n}$ tal que

$$\delta_\theta \approx P\delta_{\boldsymbol{g}}, \tag{A.1.31}$$

[4] Si $\nabla^2\mathcal{L}(\theta)$ no es invertible, entonces podemos sustituir $[\nabla^2\mathcal{L}(\theta)]^{-1}$ por la pseudoinversa de Moore-Penrose de $\nabla^2\mathcal{L}(\theta)$.

actualizándolo en cada iteración junto con θ_k. Es decir, tenemos la iteración *PSGD*

$$P_k = \text{PreconditionerUpdate}(P_{k-1}; \theta_k, \nabla\mathcal{L}(\theta_k)) \tag{A.1.32}$$
$$\theta_{k+1} = \theta_k - hP_k\nabla\mathcal{L}(\theta_k). \tag{A.1.33}$$

Esta actualización plantea dos problemas: ¿cómo podemos siquiera utilizar P (ya que ya dijimos que no podemos almacenar una matriz $n \times n$) y cómo podemos *actualizar* P en cada iteración? Las respuestas están muy relacionadas; nunca podemos materializar P en la memoria de la computadora, pero podemos representarla utilizando una factorización de rango bajo (o métodos comparables como la *factorización de Kronecker*, que es particularmente adecuada para la forma de las redes neuronales profundas). A continuación, la etapa de actualización del precondicionador está diseñada para aprovechar la estructura de la representación del precondicionador.

Finalizamos esta subsección con una consideración importante: en el aprendizaje profundo, por ejemplo, $\mathcal{L}$ no es una función convexa, por lo que el método de Newton (y sus aproximaciones) no tienen sentido. En este caso, nos fijamos en la intuición geométrica del método de Newton en funciones convexas, por ejemplo, a partir de Figura A.3: la hessiana inversa *blanquea* los gradientes. Así, en lugar de un precondicionador que aproxime la hessiana, podemos ajustar los procedimientos anteriores para aprender una transformación de blanqueo más general aplicada al gradiente. Esta es la idea en la que se basa la propuesta original de PSGD [Li17], que contiene más información sobre cómo almacenar y actualizar el precondicionador, así como optimizadores más recientes como Muon [LSY+25].

A.1.3 Descenso por gradiente proximal para problemas no suaves

Sin embargo, incluso en problemas muy sencillos, como el LASSO o el aprendizaje de diccionarios, el problema no es fuertemente convexo, sino simplemente convexo, y el objetivo ya no es solo suave, sino más bien la suma de una función suave y un regularizador no suave (como la norma ℓ^1). Estos problemas se resuelven mediante *algoritmos de optimización proximal*, que generalizan el método del gradiente máximo a objetivos no suaves.

En términos formales, digamos que

$$\mathcal{L}(\theta) \doteq \mathcal{S}(\theta) + \mathcal{R}(\theta) \tag{A.1.34}$$

donde $\mathcal{S}$ es suave, por ejemplo, con un gradiente β-Lipschitz, y $\mathcal{R}$ es no suave (es decir, irregular). El algoritmo del gradiente proximal generaliza el algoritmo del gradiente máximo, utilizando el enfoque de mayorización-minimización (es decir, Lema A.1) con una cota superior global diferente. Concretamente, construimos dicho límite superior preguntándonos: ¿qué pasaría si tomáramos la cota superior del gradiente de Lipschitz de S pero *dejáramos* R *sin modificar*?

Es decir, tenemos

$$\mathcal{L}(\theta') = \mathcal{S}(\theta') + \mathcal{R}(\theta') \leq u_{\theta,\beta}(\theta') \doteq \mathcal{S}(\theta) + \langle \nabla \mathcal{S}(\theta), \theta' - \theta \rangle + \frac{\beta}{2}\|\theta' - \theta\|_2^2 + \mathcal{R}(\theta'). \tag{A.1.35}$$

Nótese que (demostración como ejercicio)

$$\arg\min_{\theta'} u_{\theta,\beta}(\theta') = \arg\min_{\theta'} \left[\frac{\beta}{2} \left\| \theta' - \left(\theta - \frac{1}{\beta} \nabla \mathcal{S}(\theta) \right) \right\|_2^2 + \mathcal{R}(\theta') \right]. \tag{A.1.36}$$

Ahora bien, si intentamos minimizar la cota superior $u_{\theta,\beta}$, estamos eligiendo un θ' tal que:

- sea similar a la actualización del gradiente $\theta - \frac{1}{\beta}\nabla \mathcal{S}(\theta)$;
- tenga un valor pequeño del regularizador $\mathcal{R}(\theta')$

y equilibre estas propiedades en función del parámetro de suavidad β. Por consiguiente, definamos el operador proximal

$$\text{prox}_{h,\mathcal{R}}(\theta) \doteq \arg\min_{\theta'} \left[\frac{1}{2h}\|\theta' - \theta\|_2^2 + \mathcal{R}(\theta') \right]. \tag{A.1.37}$$

A continuación, podemos definir la iteración del *descenso por gradiente proximal* que, en cada iteración, minimiza el límite superior $u_{\theta_k,h^{-1}}$, es decir,

$$\theta_{k+1} = \text{prox}_{h,\mathcal{R}}(\theta_k - h\nabla \mathcal{S}(\theta_k)). \tag{A.1.38}$$

Las demostraciones de convergencia son posibles cuando $h \leq 1/\beta$, pero en esta sección no presentamos demostraciones.

Una pregunta que queda por resolver es: ¿cómo podemos calcular el operador proximal? A primera vista, parece que hemos cambiado un problema de minimización inmanejable por otro. Dado que hasta ahora no hemos hecho ninguna suposición sobre $\mathcal{R}$, el marco funciona incluso cuando $\mathcal{R}$ es una función muy compleja (como una pérdida de red neuronal), lo que nos obligaría a resolver un problema de entrenamiento de redes neuronales para calcular un solo operador proximal. Sin embargo, en la práctica, para regularizadores simples $\mathcal{R}$ como los que utilizamos en este artículo, existen operadores proximales que son fáciles de calcular o que incluso se pueden expresar en forma cerrada. A continuación ofrecemos algunos ejemplos (las demostraciones quedan como ejercicio).

Ejemplo A.1. Sea $\Gamma \subseteq \Theta$ un conjunto y sea χ_Γ la función característica en Γ, es decir,

$$\chi_\Gamma(\theta) \doteq \begin{cases} 0, & \text{if } \theta \in \Gamma \\ +\infty, & \text{if } \theta \notin \Gamma. \end{cases} \tag{A.1.39}$$

Entonces, el operador proximal de χ_Γ es una proyección, es decir,

$$\text{prox}_{h,\chi_\Gamma}(\theta) = \arg\min_{\theta' \in \Gamma} \frac{1}{2}\|\theta' - \theta\|_2^2 = \arg\min_{\theta' \in \Gamma} \|\theta' - \theta\|_2. \tag{A.1.40}$$

■

Ejemplo A.2. La norma ℓ^1 tiene un operador proximal que realiza un umbral suave:

$$S_h(\theta) \doteq \operatorname{prox}_{h,\lambda\|\cdot\|_1}(\theta) = \arg\min_{\theta'} \left[\frac{1}{2h}\|\theta' - \theta\|_2^2 + \lambda\|\theta'\|_1 \right] \tag{A.1.41}$$

entonces $S_h(\theta)$ se define por coordenadas mediante

$$S_h(\theta)_i = \begin{cases} \theta_i - h\lambda, & \text{if } \theta_i \geq h\lambda \\ 0, & \text{if } \theta_i \in [-h\lambda, h\lambda] \\ \theta_i + h\lambda, & \text{if } \theta_i \leq -h\lambda \end{cases} = \begin{cases} \max\{|\theta_i| - h\lambda, 0\}\operatorname{sign}(\theta_i), & \text{if } |\theta_i| \geq h\lambda \\ 0, & \text{if } |\theta_i| < h\lambda. \end{cases} \tag{A.1.42}$$

La operación de gradiente proximal en la que la parte suave del objetivo es el método de mínimos cuadrados y la parte no suave es la norma ℓ^1 (por lo que se utiliza este operador proximal de umbralización suave) se denomina «algoritmo iterativo de contracción y umbralización» (ISTA). ■

Ejemplo A.3. En el Capítulo 5 utilizamos un operador proximal correspondiente a la norma ℓ^1 más la función característica del ortante positivo $\mathbb{R}^n_+ \doteq \{\boldsymbol{x} \in \mathbb{R}^n : x_i \geq 0 \ \forall i\}$, a saber

$$T_h(\theta) \doteq \operatorname{prox}_{h,\lambda\|\cdot\|_1 + \chi_{\mathbb{R}^n_+}}(\theta) = \arg\min_{\theta' \in \mathbb{R}^n_+} \left[\frac{1}{2h}\|\theta' - \theta\|_2^2 + \lambda\|\theta'\|_1 \right], \tag{A.1.43}$$

entonces T_h se define como

$$T_h(\theta)_i \doteq \max\{\theta_i - h\lambda, 0\}. \tag{A.1.44}$$

Este operador proximal da como resultado el ISTA no negativo que se utiliza en el Capítulo 5 y en otros trabajos posteriores. ■

A.1.4 Descenso de gradiente estocástico para problemas a gran escala

En el aprendizaje profundo, la función objetivo $\mathcal{L}$ no suele poder calcularse con exactitud, por lo que, en cada paso de optimización, se *estima* utilizando muestras finitas (por ejemplo, mediante un mini-lote). Una forma común de modelar esta situación es definir una *función de pérdida estocástica* $\mathcal{L}_\omega(\theta)$, donde ω es una "fuente de aleatoriedad". Por ejemplo, ω podría contener los índices de las muestras de un lote sobre el cual se calcula la pérdida. Entonces, nos gustaría minimizar $\mathcal{L}(\theta) \doteq \mathbb{E}_\omega[\mathcal{L}_\omega(\theta)]$ con respecto a θ, teniendo acceso a un *oráculo estocástico de primer orden*: dado θ, podemos muestrear ω y calcular $\mathcal{L}_\omega(\theta)$ y $\nabla_\theta \mathcal{L}_\omega(\theta)$. Este problema de minimización se denomina *problema de optimización estocástica*.

El algoritmo estocástico básico de primer orden es el *descenso estocástico por gradiente*: en cada iteración k tomamos una muestra de ω_k, definimos $\mathcal{L}_k \doteq \mathcal{L}_{\omega_k}$, y ejecutamos un paso por gradiente sobre $\mathcal{L}_k$, es decir,

$$\theta_{k+1} = \theta_k - h\nabla\mathcal{L}_k(\theta_k). \tag{A.1.45}$$

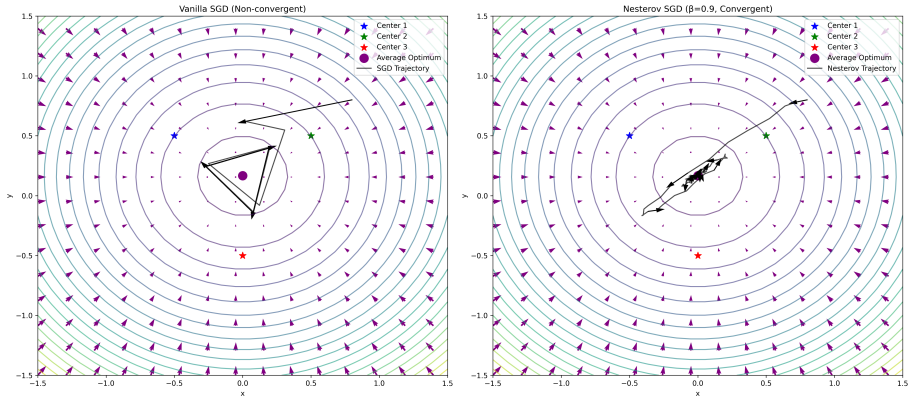


Figura A.4: **Es posible que el descenso estocástico por gradiente no converja, incluso con objetivos muy sencillos, pero el SGD con momentum sí converge.** Incluso con objetivos cuadráticos simples, las iteraciones del descenso estocástico por gradiente pueden oscilar alrededor del óptimo global, mientras que los gradientes promediados por momentum se alinean para apuntar al valor óptimo.

Sin embargo, incluso para problemas muy sencillos no podemos esperar el mismo tipo de convergencia que obtuvimos con el descenso por gradiente. Por ejemplo, supongamos que hay m valores posibles para $\omega \in \{1, \dots, m\}$ que toma con igual probabilidad, y que hay m objetivos posibles $\xi_1, \dots, \xi_m$, de tal manera que la función de pérdida $\mathcal{L}_\omega$ es

$$\mathcal{L}_\omega(\theta) \doteq \frac{1}{2}\|\theta - \xi_\omega\|_2^2. \tag{A.1.46}$$

Entonces, $\arg\min_\theta \mathbb{E}[\mathcal{L}_\omega(\theta)] = \frac{1}{m}\sum_{i=1}^m \xi_i$, pero el descenso de gradiente estocástico puede «oscilar» alrededor del valor óptimo global y no converger, tal como se muestra en la Figura A.4.

Para solucionar esto, podemos promediar los parámetros θ_k o promediar los gradientes $\nabla\mathcal{L}_k(\theta_k)$ a lo largo del tiempo. Si promediamos los parámetros θ_k, entonces (utilizando Figura A.4 como modelo mental), el problema del rebote es simplemente imposible, ya que la iteración promedio tiende a acercarse progresivamente al centro. Por lo tanto, la mayoría de las demostraciones teóricas de convergencia analizan la convergencia de la iteración promedio $\frac{1}{k}\sum_{i=0}^k \theta_i$ hacia el mínimo global. Si promediamos los gradientes, acabaremos obteniendo un gradiente promedio $\frac{1}{k}\sum_{i=0}^k \nabla\mathcal{L}_k(\theta_k)$, que no cambia mucho en cada iteración y, por lo tanto, no oscila.

En la práctica, en lugar de utilizar una media aritmética, tomamos una *media móvil exponencial* (EMA) de los parámetros (esto se conoce como *promedio de Polyak*) o de los gradientes (esto se conoce como *«momentum»*). El momentum es más popular y lo estudiaremos aquí.

Una iteración del método de descenso de gradiente estocástico con

momentum es la siguiente:

$$\boldsymbol{g}_k = \beta \boldsymbol{g}_{k-1} + (1-\beta)\nabla \mathcal{L}_k(\theta_k) \tag{A.1.47}$$

$$\theta_{k+1} = \theta_k - h\boldsymbol{g}_k. \tag{A.1.48}$$

No vamos a entrar en detalles sobre la demostración de convergencia (véase el capítulo 7 de [GG23] para ver un ejemplo). Sin embargo, el algoritmo Momentum resuelve fácilmente nuestro caso de ejemplo de la Figura A.4 (véase la figura de la derecha): deja de oscilar y, finalmente, converge hacia el óptimo global.

Terminamos con una advertencia: se puede demostrar que el momentum de Polyak y el momentum de Nesterov son equivalentes, para ciertas elecciones de los ajustes de los parámetros. Entonces también es posible demostrar que un esquema de tasa de aprendizaje decreciente (es decir, la tasa de aprendizaje h depende de la iteración k, y su límite es $h_k \to 0$ cuando $k \to \infty$) con SGD simple (o PSGD) puede imitar el efecto del momentum. Es decir, [DCM+23] demuestra que si el algoritmo SGD dura K iteraciones, las normas de los gradientes están acotadas por $\|\nabla \mathcal{L}(\theta_k)\|_2 \leq G$, y definimos $D \doteq \|\theta_0 - \theta^\star\|_2$, entonces las iteraciones del SGD simple θ_k satisfacen la tasa $\mathbb{E}[\mathcal{L}(\theta_k) - \mathcal{L}(\theta^\star)] \leq DG/\sqrt{K}$ — pero solo mientras la tasa de aprendizaje $h_k = (D/[G\sqrt{K}])(1 - k/K)$ decaiga *linealmente* con el tiempo. Esto concuerda con los esquemas de tasa de aprendizaje utilizados en la práctica. De hecho, sorprendentemente, dicha teoría de la optimización convexa puede predecir muchos fenómenos empíricos en las redes profundas [SHT+25], a pesar de que la optimización para aprendizaje profundo sea altamente no convexa y no suave en el peor de los casos. Hasta el momento no está claro por qué ocurre esto.

A.1.5 Uniéndolo todo: Adam

El método de descenso por gradiente propone una iteración dada por

$$\theta_{k+1} = \theta_k + h\boldsymbol{v}_k, \tag{A.1.49}$$

donde (recordemos) $\boldsymbol{v}_k$ se elige para que sea (proporcional al) vector del gradiente máximo en la norma euclidiana:

$$\boldsymbol{v}_k = -\frac{\nabla \mathcal{L}(\theta_k)}{\|\nabla \mathcal{L}(\theta_k)\|_2} \in \arg\min_{\substack{\boldsymbol{v}\in\mathbb{R}^n \\ \|\boldsymbol{v}\|_2=1}} \langle \nabla \mathcal{L}(\theta_k), \boldsymbol{v}\rangle. \tag{A.1.50}$$

Sin embargo, en el contexto de la optimización del aprendizaje profundo, no hay absolutamente nada que implique que debamos utilizar la norma euclidiana; de hecho, la «geometría natural» del espacio de parámetros no se respeta adecuadamente con la norma euclidiana, ya que pequeños cambios en el espacio de parámetros pueden dar lugar a diferencias muy grandes en el espacio de salida, para una entrada fija concreta de la red. Si, en cambio, utilizáramos una

norma genérica $\|\cdot\|$ en el espacio de parámetros $\mathbb{R}^n$, obtendríamos otra cantidad correspondiente a la denominada *norma dual*:

$$\boldsymbol{v}_k \in \arg\min_{\substack{\boldsymbol{v}\in\mathbb{R}^n \\ \|\boldsymbol{v}\|=1}} \langle \nabla \mathcal{L}(\theta_k), \boldsymbol{v}\rangle. \tag{A.1.51}$$

Por ejemplo, si utilizáramos la norma ℓ^∞, es posible demostrar que

$$\boldsymbol{v}_k = -\operatorname{sign}(\nabla \mathcal{L}(\theta_k)) \in \arg\min_{\substack{\boldsymbol{v}\in\mathbb{R}^n \\ \|\boldsymbol{v}\|_\infty=1}} \langle \nabla \mathcal{L}(\theta_k), \boldsymbol{v}\rangle, \tag{A.1.52}$$

donde $\operatorname{sign}(\boldsymbol{x})_i = \operatorname{sign}(x_i) \in \{-1, 0, 1\}$. Por lo tanto, si quisieramos, podríamos utilizar el llamado *descenso por gradiente con signo*:

$$\theta_{k+1} = \theta_k - h \operatorname{sign}(\nabla \mathcal{L}(\theta_k)). \tag{A.1.53}$$

A partir del descenso por gradiente de signo, podemos derivar el famoso algoritmo de optimización de Adam. Obsérvese que, para un escalar $x \in \mathbb{R}$, podemos escribir

$$\operatorname{sign}(x) = \frac{x}{|x|} = \frac{x}{\sqrt{x^2}}. \tag{A.1.54}$$

Del mismo modo, para un vector $\boldsymbol{x} \in \mathbb{R}^n$ escribimos (donde $\odot$ y $\oslash$ denotan la multiplicación y la división elemento a elemento)

$$\operatorname{sign}(\boldsymbol{x}) = \boldsymbol{x} \oslash [\boldsymbol{x}^{\odot 2}]^{\odot(1/2)}. \tag{A.1.55}$$

Utilizando esta representación, podemos escribir (A.1.53) como

$$\theta_{k+1} = \theta_k - h([\nabla \mathcal{L}(\theta_k)] \oslash [\nabla \mathcal{L}(\theta_k)^{\odot 2}]^{\odot \frac{1}{2}}). \tag{A.1.56}$$

Ahora consideremos el régimen estocástico en el que optimizamos una pérdida diferente $\mathcal{L}_k$ en cada iteración. En el SGD, «seguíamos» (es decir, tomábamos un promedio de) los gradientes utilizando el momentum. Aquí, podemos seguir tanto el gradiente como el cuadrado del gradiente utilizando el momentum, es decir,

$$\boldsymbol{g}_k = \beta^1 \boldsymbol{g}_{k-1} + (1-\beta^1)\nabla \mathcal{L}_k(\theta_k) \tag{A.1.57}$$

$$\boldsymbol{s}_k = \beta^2 \boldsymbol{s}_{k-1} + (1-\beta^2)[\nabla \mathcal{L}_k(\theta_k)]^{\odot 2} \tag{A.1.58}$$

$$\theta_{k+1} = \theta_k - h\boldsymbol{g}_k \oslash \boldsymbol{s}_k^{\odot \frac{1}{2}}, \tag{A.1.59}$$

donde $\beta^i \in [0, 1]$ son los parámetros de impulso. El algoritmo descrito por estas iteraciones es el famoso optimizador *Adam*,[5] que es el optimizador más utilizado en el aprendizaje profundo. Aunque las demostraciones de convergencia de Adam son más complejas, se derivan del mismo principio del gradiente máximo

[5] Para evitar errores de división por cero, dividimos por $\boldsymbol{s}_k^{\odot(1/2)} + \varepsilon \mathbf{1}_n$, donde ε es pequeño, digamos del orden de 10^{-8}.

que hemos utilizado hasta ahora, por lo que cabría esperar que, con una tasa de aprendizaje lo suficientemente pequeña, cada actualización mejorara la pérdida.

Otra forma de entender Adam, que explica en parte su éxito empírico, es que actualiza dinámicamente las tasas de aprendizaje de cada parámetro basándose en los gradientes al cuadrado. En particular, fíjate en que podemos escribir

$$\theta_{k+1} = \theta_k - \eta_k \odot \boldsymbol{g}_k \qquad \text{where} \qquad \eta_k = h\boldsymbol{s}_k^{\odot(-\frac{1}{2})} \tag{A.1.60}$$

donde η_k es la tasa de aprendizaje establecida en forma adaptativa para cada parámetro. Este esquema se denomina «adaptativo» porque, si el gradiente de un parámetro concreto es elevado hasta la iteración k, la tasa de aprendizaje para ese parámetro se reduce para compensar, y viceversa, como se puede observar en la ecuación anterior.

A.2 Cálculo de gradientes mediante diferenciación automática

Anteriormente, analizamos varios algoritmos de optimización para redes profundas que partían del supuesto de tener acceso a un *oráculo de primer orden*, es decir, un dispositivo que nos permitiera calcular $\mathcal{L}(\theta)$ y $\nabla\mathcal{L}(\theta)$. Para funciones simples de $\mathcal{L}$, es posible hacerlo a mano. Sin embargo, para redes neuronales profundas, esto se vuelve rápidamente tedioso y dificulta la experimentación rápida. Por lo tanto, necesitamos un algoritmo general que nos permita calcular de manera eficiente los gradientes de arquitecturas de redes diferenciables (parcialmente) o "subdiferenciables" arbitrarias.

En esta sección, presentamos los fundamentos de la *diferenciación automática* (AD o *autodiff*), que es un método computacionalmente eficiente para calcular gradientes y jacobianos de funciones generales $f : \mathbb{R}^m \to \mathbb{R}^n$. Mostraremos cómo esto conduce al algoritmo de retropropagación para calcular gradientes de funciones de pérdida que involucran redes neuronales. A continuación se presenta un resumen de la estructura de esta sección:

1. Presentamos los **diferenciales**, un formalismo práctico para calcular y organizar las derivadas de funciones entre espacios de parámetros de alta dimensión (que pueden ser a su vez productos de otros espacios que involucran matrices, tensores, etc.).

2. Describimos los fundamentos de la diferenciación automática en **modo directo** y **modo inverso**, lo cual implica consideraciones importantes para el cálculo eficiente de gradientes y jacobianos para diferentes tipos de funciones que surgen en aplicaciones de aprendizaje automático.

3. Describimos la **retropropagación** en el caso particular de una función de pérdida aplicada a una red neuronal de capas apiladas como una instancia de la diferenciación automática en modo reverso.

Nuestro enfoque se inclinará hacia el aspecto matemático, con el fin de proporcionar al lector una comprensión profunda de los fundamentos matemáticos subyacentes. El lector debe asegurarse de complementar esta comprensión con un dominio sólido de los aspectos prácticos de la diferenciación automática para el aprendizaje profundo, tal como se muestra, por ejemplo, en el excelente tutorial de Karpathy [Kar22b].

A.2.1 Diferenciales

En la excelente guía [BEJ25] se ofrece una explicación detallada de esta subsección. Para introducir el concepto de diferenciales, consideremos primero el ejemplo sencillo de una función diferenciable $\mathcal{L}\colon \mathbb{R} \to \mathbb{R}$ que depende de un parámetro θ. Podemos escribir

$$\mathcal{L}(\theta^+) - \mathcal{L}(\theta) = \mathcal{L}'(\theta) \cdot (\theta^+ - \theta) + o(|\theta^+ - \theta|). \tag{A.2.1}$$

Si tomamos $\delta\theta \doteq \theta^+ - \theta$ y $\delta\mathcal{L} \doteq \mathcal{L}(\theta + \delta\theta) - \mathcal{L}(\theta)$, podemos escribir

$$\delta\mathcal{L} = \mathcal{L}'(\theta) \cdot \delta\theta + o(|\delta\theta|). \tag{A.2.2}$$

Definiremos (de manera no rigurosa) $\mathrm{d}\theta$ y $\mathrm{d}\mathcal{L}$, es decir, los *diferenciales* de θ y $\mathcal{L}$, como cambios *infinitesimalmente pequeños* en θ y $\mathcal{L}$. Piensa en ellos como lo que se obtiene cuando $\delta\theta$ (y, por lo tanto, $\delta\mathcal{L}$) son extremadamente pequeños. El objetivo del cálculo diferencial, en cierto sentido, es estudiar las relaciones entre los diferenciales $\mathrm{d}\theta$ y $\mathrm{d}\mathcal{L}$, es decir, observar cómo pequeños cambios en la entrada de una función modifican la salida. Debemos tener en cuenta que el diferencial $\mathrm{d}\theta$ tiene la *misma forma* que θ, y el diferencial $\mathrm{d}\mathcal{L}$ tiene la *misma forma* que $\mathcal{L}$. En particular, podemos escribir

$$\mathrm{d}\mathcal{L} = \mathcal{L}'(\theta) \cdot \mathrm{d}\theta, \tag{A.2.3}$$

por lo que se deduce que todas las potencias superiores de $|\mathrm{d}\theta|$, como $(\mathrm{d}\theta)^2$, son 0.

Veamos cómo funciona esto en dimensiones superiores, es decir, $\mathcal{L}\colon \mathbb{R}^n \to \mathbb{R}$. Entonces seguimos teniendo

$$\mathrm{d}\mathcal{L} = \mathcal{L}'(\theta) \cdot \mathrm{d}\theta \tag{A.2.4}$$

para cierta noción de derivada $\mathcal{L}'(\theta)$. Dado que θ (y, por lo tanto, $\mathrm{d}\theta$) es aquí un vector columna y $\mathcal{L}$ (y, por lo tanto, $\mathrm{d}\mathcal{L}$) es un escalar, se deduce que $\mathcal{L}'(\theta)$ es un vector fila. En este caso, $\mathcal{L}'(\theta)$ es la jacobiana de $\mathcal{L}$ respecto a θ. Obsérvese aquí que hemos fijado todas las potencias superiores y los productos de coordenadas de $\mathrm{d}\theta$ en 0. En resumen,

Todos los productos y potencias ≥ 2 *de diferenciales son iguales a* 0.

Consideremos ahora una función tensorial de orden superior $\mathcal{L}\colon \mathbb{R}^{m\times n} \to \mathbb{R}^{p\times q}$. Entonces, nuestra ecuación de linealización básica resulta insuficiente

para este caso: $\mathrm{d}\mathcal{L} = \mathcal{L}'(\theta) \cdot \mathrm{d}\theta$ no tiene sentido, ya que θ es una matriz de $m \times n$, pero $\mathrm{d}\mathcal{L}$ es una matriz $p \times q$, por lo que no hay ninguna forma de vector o matriz posible para $\mathcal{L}'(\theta)$ que funcione en general (ya que ninguna matriz puede multiplicar una matriz $m \times n$ para formar una matriz $p \times q$ a menos que $m = p$). Así que debemos tener una interpretación un poco más avanzada.

Es decir, consideramos $\mathcal{L}'(\theta)$ como una *transformación lineal* cuya entrada es el espacio de θ y cuya salida es el espacio de $\mathcal{L}$, que toma un pequeño cambio en θ y devuelve el pequeño cambio correspondiente en $\mathcal{L}$. Es decir, podemos escribir

$$\mathrm{d}\mathcal{L} = \mathcal{L}'(\theta)[\mathrm{d}\theta]. \tag{A.2.5}$$

En los casos anteriores, $\mathcal{L}'(\theta)$ era primero un operador lineal $\mathbb{R} \to \mathbb{R}$ cuya acción consistía en multiplicar su entrada por la derivada escalar de $\mathcal{L}$ con respecto a θ, y luego un operador lineal $\mathbb{R}^n \to \mathbb{R}$ cuya acción consistía en multiplicar su entrada por la derivada jacobiana de $\mathcal{L}$ con respecto a θ. En general, $\mathcal{L}'(\theta)$ es la «derivada» de $\mathcal{L}$ con respecto a θ. Piensa en $\mathcal{L}'$ como una versión generalizada de la jacobiana de $\mathcal{L}$. Como tal, sigue algunas reglas de derivación sencillas, entre las que destaca la regla de la cadena.

Teorema A.1 (Regla diferencial de la cadena)**.** *Supongamos que $\mathcal{L} = f \circ g$, donde f y g son diferenciables. Entonces*

$$\mathrm{d}\mathcal{L} = f'(g(\theta))g'(\theta)[\mathrm{d}\theta], \tag{A.2.6}$$

donde (como de costumbre) la multiplicación indica la composición de operadores lineales. En particular,

$$\mathcal{L}'(\theta) = f'(g(\theta))g'(\theta) \tag{A.2.7}$$

en el sentido de la igualdad de operadores lineales.

Resulta útil pensar en la regla de la cadena multivariante en términos functoriales: la composición de funciones se «convierte» en una multiplicación matricial de jacobianos (¡la composición de operadores lineales!). Ilustramos la potencia de este resultado y de esta perspectiva mediante varios ejemplos.

Ejemplo A.4. Consideremos la función $f(\boldsymbol{X}) = \boldsymbol{W}\boldsymbol{X} + \boldsymbol{b}\boldsymbol{1}^\top$. Entonces

$$\mathrm{d}f = f(\boldsymbol{X} + \mathrm{d}\boldsymbol{X}) - f(\boldsymbol{X}) = [\boldsymbol{W}(\boldsymbol{X} + \mathrm{d}\boldsymbol{X}) + \boldsymbol{b}\boldsymbol{1}^\top] - [\boldsymbol{W}\boldsymbol{X} + \boldsymbol{b}\boldsymbol{1}^\top] = \boldsymbol{W}\mathrm{d}\boldsymbol{X}. \tag{A.2.8}$$

Por lo tanto, la derivada de una función afín con respecto a su entrada es

$$f'(\boldsymbol{X})[\mathrm{d}\boldsymbol{X}] = \boldsymbol{W}\mathrm{d}\boldsymbol{X} \implies f'(\boldsymbol{X}) = \boldsymbol{W}. \tag{A.2.9}$$

Obsérvese que f' es *constante*. Por otro lado, consideremos la función $g(\boldsymbol{W}, \boldsymbol{b}) = \boldsymbol{W}\boldsymbol{X} + \boldsymbol{b}\boldsymbol{1}^\top$. Entonces

$$\begin{aligned} \mathrm{d}g &= g(\boldsymbol{W} + \mathrm{d}\boldsymbol{W}, \boldsymbol{b} + \mathrm{d}\boldsymbol{b}) - g(\boldsymbol{W}, \boldsymbol{b}) && \text{(A.2.10)} \\ &= [(\boldsymbol{W} + \mathrm{d}\boldsymbol{W})\boldsymbol{X} + (\boldsymbol{b} + \mathrm{d}\boldsymbol{b})\boldsymbol{1}^\top] - [\boldsymbol{W}\boldsymbol{X} + \boldsymbol{b}\boldsymbol{1}^\top] && \text{(A.2.11)} \\ &= (\mathrm{d}\boldsymbol{W})\boldsymbol{X} + (\mathrm{d}\boldsymbol{b})\boldsymbol{1}^\top = g'(\boldsymbol{W}, \boldsymbol{b})[\mathrm{d}\boldsymbol{W}, \mathrm{d}\boldsymbol{b}]. && \text{(A.2.12)} \end{aligned}$$

Nótese que esta derivada es *constante* en $\boldsymbol{W}, \boldsymbol{b}$ (lo cual tiene sentido, ya que g es lineal) y lineal en las entradas diferenciales $\mathrm{d}\boldsymbol{W}, \mathrm{d}\boldsymbol{b}$. ■

Ejemplo A.5. Consideremos la función $f = gh$, donde g y h son funciones diferenciables cuyos resultados se pueden multiplicar entre sí. Entonces, $f = p \circ v$, donde $v = (g, h)$ y $p(a, b) = ab$. Aplicando la regla de la cadena, tenemos

$$\mathrm{d}f = p'(v(x))v'(x)[\mathrm{d}x]. \tag{A.2.13}$$

Para calcular $v'(x)$ podemos calcular

$$\mathrm{d}v = v'(x)[\mathrm{d}x] = v(x + \mathrm{d}x) - v(x) = \begin{bmatrix} g(x + \mathrm{d}x) - g(x) \\ h(x + \mathrm{d}x) - h(x) \end{bmatrix} = \begin{bmatrix} g'(x)[\mathrm{d}x] \\ h'(x)[\mathrm{d}x] \end{bmatrix}. \tag{A.2.14}$$

Para calcular p', podemos calcular

$$\mathrm{d}p = p'(a, b)[\mathrm{d}a, \mathrm{d}b] = p(a + \mathrm{d}a, b + \mathrm{d}b) - p(a, b) = (a + \mathrm{d}a)(b + \mathrm{d}b) - ab \tag{A.2.15}$$

$$= (\mathrm{d}a)b + a(\mathrm{d}b) + (\mathrm{d}a)(\mathrm{d}b) = (\mathrm{d}a)b + a(\mathrm{d}b), \tag{A.2.16}$$

donde (recordemos) el producto de las derivadas $\mathrm{d}a$ y $\mathrm{d}b$ se establece en 0. Por lo tanto

$$p'(a, b)[\mathrm{d}a, \mathrm{d}b] = (\mathrm{d}a)b + (\mathrm{d}b)a. \tag{A.2.17}$$

Si juntamos todo esto, obtenemos

$$f'(x)[\mathrm{d}x] = p'(v(x))v'(x)[\mathrm{d}x] = p'(g(x), h(x))[g'(x)[\mathrm{d}x], h'(x)[\mathrm{d}x]] \tag{A.2.18}$$

$$= (g'(x)[\mathrm{d}x])h(x) + g(x)(h'(x)[\mathrm{d}x]). \tag{A.2.19}$$

Esto da

$$f'(x)[\mathrm{d}x] = (g'(x)[\mathrm{d}x])h(x) + g(x)(h'(x)[\mathrm{d}x]). \tag{A.2.20}$$

Si, por ejemplo, decimos que $f, g, h \colon \mathbb{R} \to \mathbb{R}$, entonces todo conmuta, por lo que

$$f'(x)[\mathrm{d}x] = (g'(x)h(x) + g(x)h'(x))[\mathrm{d}x] \implies f'(x) = g'(x)h(x) + g(x)h'(x) \tag{A.2.21}$$

que es la conocida regla del producto. ■

Ejemplo A.6. Consideremos la función $f(\boldsymbol{A}) = \boldsymbol{A}^\top \boldsymbol{A}\boldsymbol{B}\boldsymbol{A}$, donde $\boldsymbol{A}$ es una matriz y $\boldsymbol{B}$ es una matriz constante. Entonces, si ponemos $f = gh$, donde $g(\boldsymbol{A}) = \boldsymbol{A}^\top \boldsymbol{A}$ y $h(\boldsymbol{A}) = \boldsymbol{B}\boldsymbol{A}$, podemos usar la regla del producto para obtener

$$f'(\boldsymbol{A})[\mathrm{d}\boldsymbol{A}] = (g'(\boldsymbol{A})[\mathrm{d}\boldsymbol{A}])h(\boldsymbol{A}) + g(\boldsymbol{A})(h'(\boldsymbol{A})[\mathrm{d}\boldsymbol{A}]) \tag{A.2.22}$$

$$= ((\mathrm{d}\boldsymbol{A})^\top \boldsymbol{A} + \boldsymbol{A}^\top (\mathrm{d}\boldsymbol{A}))\boldsymbol{B}\boldsymbol{A} + \boldsymbol{A}^\top \boldsymbol{A}\boldsymbol{B}(\mathrm{d}\boldsymbol{A}). \tag{A.2.23}$$

■

Ejemplo A.7. Consideremos la función $f \colon \mathbb{R}^{m \times n \times k} \to \mathbb{R}^{m \times n}$ definida por

$$f(\boldsymbol{A})_{ij} = \sum_{t=1}^{k} A_{ijt}. \tag{A.2.24}$$

No podemos escribir una jacobiana (con valores matriciales) ni un gradiente para esta función. Pero sí podemos calcular su derivada sin problemas:

$$\mathrm{d}f_{ij} = [f(\boldsymbol{A} + \mathrm{d}\boldsymbol{A}) - f(\boldsymbol{A})]_{ij} = \sum_{t=1}^{k} \mathrm{d}\boldsymbol{A}_{ijt} = \mathbf{1}_k^\top (\mathrm{d}\boldsymbol{A})_{ij}. \tag{A.2.25}$$

Entonces

$$(f'(\boldsymbol{A})[\mathrm{d}\boldsymbol{A}])_{ij} = \mathbf{1}_k^\top (\mathrm{d}\boldsymbol{A})_{ij}, \tag{A.2.26}$$

lo cual representa una operación de multiplicación tensorial de orden superior que, no obstante, está bien definida. ■

Esto nos proporciona toda la tecnología que necesitamos para calcular las derivadas de cualquier cosa. Lo último que trataremos en esta sección es un método para calcular gradientes utilizando la derivada. Concretamente, para una función $\mathcal{L}$ cuyo resultado es un escalar, el gradiente $\nabla\mathcal{L}$ se define como

$$\mathrm{d}\mathcal{L} = \mathcal{L}'(\theta)[\mathrm{d}\theta] = \langle \nabla\mathcal{L}(\theta), \mathrm{d}\theta \rangle, \tag{A.2.27}$$

donde el producto interno aquí es el producto interno «estándar» para los objetos especificados (es decir, para vectores es el producto interno ℓ^2, mientras que para matrices es el producto interno de Frobenius, y para tensores de orden superior es el producto interno análogo de suma de coordenadas). Esta definición es la generalización correcta del ejemplo «familiar» del gradiente de una función de $\mathbb{R}^n$ a $\mathbb{R}$ como el vector de derivadas parciales: una versión del teorema de Taylor para funciones generales $f : \mathbb{R}^m \to \mathbb{R}^n$ hace que esta conexión sea rigurosa. Así que una forma de calcular el gradiente $\nabla\mathcal{L}$ es calcular la diferencial $\mathrm{d}\mathcal{L}$ y reescribirla en la forma $\langle \text{algo}, \mathrm{d}\theta \rangle$, entonces ese «algo» es el gradiente.

A.2.2 Diferenciación automática

La idea principal del AD es calcular la regla de la cadena de manera eficiente. El problema básico al que nos enfrentamos es el siguiente. En la sección de optimización del apéndice, consideramos que el espacio de parámetros Θ era un espacio euclidiano abstracto como $\mathbb{R}^n$. En la práctica, los parámetros son en realidad una colección de vectores, matrices y objetos de orden superior: $\Theta = \mathbb{R}^{m\times n} \times \mathbb{R}^n \times \mathbb{R}^{r\times q\times p} \times \mathbb{R}^{r\times q} \times \cdots$. Aunque en teoría esto equivale a un gran espacio de parámetros $\mathbb{R}^{n'}$ para algún n' (muy grande), los algoritmos computacionalmente eficientes para la diferenciación deben tratar estos dos espacios de manera diferente. La diferenciación automática en modo directo y en modo inverso son dos esquemas distintos para realizar este cálculo.

Empecemos con un ejemplo sencillo. Sea $\mathcal{L}$ definida por $\mathcal{L} = a \circ b \circ c$, donde a, b, c son diferenciables. Entonces, la *regla de la cadena* da

$$\mathcal{L}'(\theta) = a'(b(c(\theta)))b'(c(\theta))c'(\theta). \tag{A.2.28}$$

Para calcular $\mathcal{L}(\theta)$, primero calculamos $c(\theta)$, luego $b(c(\theta))$ y después $a(b(c(\theta)))$, y los guardamos todos. Hay dos formas de calcular $\mathcal{L}'(\theta)$. El *modo directo de*

AD calculará

$$c'(\theta) \implies b'(c(\theta))c'(\theta) \implies a'(b(c(\theta)))b'(c(\theta))c'(\theta) \tag{A.2.29}$$

es decir, calculando las derivadas «de abajo hacia arriba». El *modo inverso de* AD calculará

$$a'(b(c(\theta))) \implies a'(b(c(\theta)))b'(c(\theta)) \implies a'(b(c(\theta)))b'(c(\theta))c'(\theta), \tag{A.2.30}$$

es decir, calcular las derivadas «de arriba hacia abajo». Para ver por qué esto es importante, supongamos que $f\colon \mathbb{R}^p \to \mathbb{R}^s$ viene dada por $f = a \circ b \circ c$, donde $a\colon \mathbb{R}^r \to \mathbb{R}^s$, $b\colon \mathbb{R}^q \to \mathbb{R}^r$, $c\colon \mathbb{R}^p \to \mathbb{R}^q$. Entonces, la regla de la cadena es:

$$f'(\boldsymbol{x}) = a'(b(c(\boldsymbol{x})))b'(c(\boldsymbol{x}))c'(\boldsymbol{x}) \tag{A.2.31}$$

donde (recordemos) f' es la derivada, en este caso la matriz jacobiana (ya que tanto la entrada como la salida de cada función son vectores). Suponiendo que calcular cada jacobiano es trivial y que el único costo es multiplicar los jacobianos entre sí, el AD en modo directo tiene los siguientes costos computacionales (suponiendo que multiplicar $A \in \mathbb{R}^{m\times n}, B \in \mathbb{R}^{n\times k}$ toma un tiempo de $\mathcal{O}(mnk)$):

$$\text{calcular } c'(\boldsymbol{x}) \in \mathbb{R}^{q\times p} \text{ toma un tiempo despreciable} \tag{A.2.32}$$

$$\text{calcular } b'(c(\boldsymbol{x}))c'(x) \in \mathbb{R}^{r\times p} \text{ toma un tiempo del orden de } \mathcal{O}(pqr) \tag{A.2.33}$$

$$\text{calcular } a'(b(c(\boldsymbol{x})))b'(c(\boldsymbol{x}))c'(\boldsymbol{x}) \in \mathbb{R}^{s\times p} \text{ toma un tiempo del orden de } \mathcal{O}(pqr + prs). \tag{A.2.34}$$

Por otra parte, la diferenciación automática en modo inverso conlleva los siguientes costos computacionales:

$$\text{calcular } a'(b(c(\boldsymbol{x}))) \in \mathbb{R}^{s\times r} \text{ tima un tiempo despreciable} \tag{A.2.35}$$

$$\text{calcular } a'(b(c(\boldsymbol{x})))b'(c(\boldsymbol{x})) \in \mathbb{R}^{s\times q} \text{ toma un tiempo del orden de } \mathcal{O}(qrs) \tag{A.2.36}$$

$$\text{calcular } a'(b(c(\boldsymbol{x})))b'(c(\boldsymbol{x}))c'(\boldsymbol{x}) \in \mathbb{R}^{s\times p} \text{ toma un tiempo del orden de } \mathcal{O}(qrs + pqs). \tag{A.2.37}$$

En otras palabras, el algoritmo de sumas en modo directo tarda un tiempo de orden $\mathcal{O}(p(qr + rs))$, y el algoritmo de sumas en modo inverso tarda un tiempo de orden $\mathcal{O}(s(pq + qr))$. *¡Estos tardan un tiempo diferente!*

En términos más generales, supongamos que $f = f^L \circ \cdots \circ f^1$ donde cada $f^\ell\colon \mathbb{R}^{d^{\ell-1}} \to \mathbb{R}^{d^\ell}$, de modo que $f\colon \mathbb{R}^{d^0} \to \mathbb{R}^{d^L}$. Entonces, el modo directo tarda $\mathcal{O}(d^0(\sum_{\ell=2}^{L} d^{\ell-1}d^\ell))$ tiempo mientras que el algoritmo de diferenciación automática (AD) en modo reverso tarda $\mathcal{O}(d^L(\sum_{\ell=1}^{L-1} d^{\ell-1}d^\ell))$ tiempo. A partir de las tasas anteriores, vemos que, en igualdad de condiciones:

- Si la función que se va a optimizar tiene *más salidas que entradas* (es decir, $d^L > d^0$), utilice *el AD en modo directo*.

- Si la función que se va a optimizar tiene *más entradas que salidas* (es decir, $d^0 > d^L$), utilice *el modo inverso de AD*.

En una red neuronal, calculamos el gradiente de una *función de pérdida* $\mathcal{L}\colon \Theta \to \mathbb{R}$, donde el espacio de parámetros Θ suele ser de muy alta dimensión. Por eso, en la práctica siempre utilizamos la diferenciación automática en modo inverso para entrenar redes neuronales. El método de descenso por gradiente en modo inverso, en el contexto del entrenamiento de redes neuronales, se denomina *retropropagación*.

A.2.3 Retropropagación

En esta sección analizaremos la retropropagación algorítmica utilizando un ejemplo sencillo pero totalmente práctico. Supongamos que fijamos un par de entrada y etiqueta $(\boldsymbol{X}, \boldsymbol{y})$, y fijamos una arquitectura de red $f_\theta = f_\theta^L \circ \cdots \circ f_\theta^1 \circ f_\theta^{\text{emb}}$, donde $\theta = (\theta^{\text{emb}}, \theta^1, \ldots, \theta^L, \theta^{\text{head}})$ y el cabezal específico de la tarea $h_{\theta^{\text{head}}}$, y escribimos

$$\boldsymbol{Z}_\theta^1(\boldsymbol{X}) \doteq f_\theta^{\text{emb}}(\boldsymbol{X}), \tag{A.2.38}$$

$$\boldsymbol{Z}_\theta^{\ell+1}(\boldsymbol{X}) \doteq f_\theta^\ell(\boldsymbol{Z}_\theta^\ell(\boldsymbol{X})), \quad \forall \ell \in \{1, \ldots, L\}, \tag{A.2.39}$$

$$\hat{\boldsymbol{y}}_{\theta^{\text{head}}}(\boldsymbol{X}) \doteq h_{\theta^{\text{head}}}(\boldsymbol{Z}_\theta^{L+1}(\boldsymbol{X})). \tag{A.2.40}$$

Entonces, podemos definir la función de pérdida para este ejemplo mediante

$$\mathcal{L}(\theta) \doteq \mathsf{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}_\theta(\boldsymbol{X})), \tag{A.2.41}$$

donde L es una función diferenciable de su segundo argumento. Entonces, el AD en modo inverso nos indica que calculemos las derivadas en el orden $\theta^{\text{head}}, \theta^L, \ldots, \theta^1, \theta^{\text{emb}}$.

Para realizar este cálculo, vamos a introducir algunos cambios en la notación.

- En primer lugar, modifiquemos la notación para enfatizar la estructura de dependencia entre las variables. Es decir,

$$\boldsymbol{Z}^1 \doteq f^{\text{emb}}(\boldsymbol{X}, \theta^{\text{emb}}) \tag{A.2.42}$$

$$\boldsymbol{Z}^{\ell+1} \doteq f^\ell(\boldsymbol{Z}^\ell, \theta^\ell), \qquad \forall \ell \in \{1, \ldots, L\}, \tag{A.2.43}$$

$$\hat{\boldsymbol{y}} \doteq h(\boldsymbol{Z}^{L+1}, \theta^{\text{head}}), \tag{A.2.44}$$

$$\mathcal{L} \doteq \mathsf{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}). \tag{A.2.45}$$

- Entonces, en lugar de escribir la derivada como f', denotamos explícitamente la variable independiente y escribimos la derivada como $\frac{\mathrm{d}f}{\mathrm{d}\theta^1}$, por ejemplo. Esto se debe a que hay muchas variables en nuestro modelo y solo nos interesa una a la vez.

Podemos empezar calculando las derivadas correspondientes. En primer lugar, para θ^{head} tenemos

$$\mathrm{d}\mathcal{L} = \mathrm{d}\mathsf{L} \tag{A.2.46}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \cdot \mathrm{d}\hat{\boldsymbol{y}} \tag{A.2.47}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \cdot \mathrm{d}(h(\boldsymbol{Z}^{L+1}, \theta^{\text{head}})) \tag{A.2.48}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \left[\frac{\mathrm{d}h}{\mathrm{d}\boldsymbol{Z}^{L+1}} \cdot \mathrm{d}\boldsymbol{Z}^{L+1} + \frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}} \cdot \mathrm{d}\theta^{\text{head}} \right]. \tag{A.2.49}$$

Ahora bien, dado que $\boldsymbol{Z}^{L+1}$ no depende de θ^{head}, tenemos que $\mathrm{d}\boldsymbol{Z}^{L+1} = 0$, por lo que al final se cumple (utilizando el hecho de que el gradiente es la transpuesta de la derivada de una función L cuyo codominio es $\mathbb{R}$):

$$\mathrm{d}\mathcal{L} = \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \cdot \frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}} \cdot \mathrm{d}\theta^{\text{head}} \tag{A.2.50}$$

$$= [\nabla_{\hat{\boldsymbol{y}}} \mathsf{L}]^{\top} \frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}} \cdot \mathrm{d}\theta^{\text{head}} \tag{A.2.51}$$

$$= \left\langle \nabla_{\hat{\boldsymbol{y}}} \mathsf{L}, \frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}} \cdot \mathrm{d}\theta^{\text{head}} \right\rangle \tag{A.2.52}$$

$$= \left\langle \left(\frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}} \right)^{*} \nabla_{\hat{\boldsymbol{y}}} \mathsf{L}, \mathrm{d}\theta^{\text{head}} \right\rangle \tag{A.2.53}$$

$$= \langle \nabla_{\theta^{\text{head}}} \mathcal{L}, \mathrm{d}\theta^{\text{head}} \rangle. \tag{A.2.54}$$

Por lo tanto, para calcular $\nabla_{\theta^{\text{head}}} \mathcal{L}$, calculamos el gradiente $\nabla_{\hat{\boldsymbol{y}}} \mathsf{L}$ y el *adjunto*[6] de la derivada $\frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}}$, y los multiplicamos (es decir, aplicamos la transformación lineal adjunta al gradiente). En la práctica, ambas derivadas se pueden calcular a mano, pero muchos marcos computacionales modernos pueden definir *automáticamente* las derivadas (y/o sus adjuntas) dado el código para la «pasada directa», es decir, el cálculo de la función de pérdida. Aunque la ampliación de esta definición automática de la derivada a tantas funciones como sea posible es un campo de investigación activo, el recurso [BEJ25] describe con cierto detalle un enfoque básico para hacerlo. Por cierto, la retropropagación también se conoce como el *método adjunto* por esta razón, es decir, porque utilizamos derivadas adjuntas para calcular el gradiente.

Ahora calculemos las derivadas con respecto a algún θ^{ℓ}:

$$\mathrm{d}\mathcal{L} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \cdot \mathrm{d}\boldsymbol{Z}^{\ell+1} \tag{A.2.55}$$

$$= \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \cdot \mathrm{d}(f^{\ell}(\boldsymbol{Z}^{\ell}, \theta^{\ell})) \tag{A.2.56}$$

$$= \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \left[\frac{\mathrm{d}f^{\ell}}{\mathrm{d}\boldsymbol{Z}^{\ell}} \cdot \mathrm{d}\boldsymbol{Z}^{\ell} + \frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \cdot \mathrm{d}\theta^{\ell} \right] \tag{A.2.57}$$

[6] El adjunto es como una transpuesta generalizada para transformaciones lineales más generales. En particular, para un par dado de espacios con producto interno y una transformación lineal T entre esos espacios, el adjunto T^{*} se define mediante la identidad $\langle T\boldsymbol{x}, \boldsymbol{y} \rangle = \langle \boldsymbol{x}, T^{*}\boldsymbol{y} \rangle$. En dimensiones finitas (es decir, todos los casos relevantes para este libro), el adjunto existe y es único.

$$= \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \cdot \frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \cdot \mathrm{d}\theta^{\ell} \qquad \text{(b/c } \boldsymbol{Z}^{\ell} \text{ isn't fn. of } \theta^{\ell} \text{ so } \mathrm{d}\boldsymbol{Z}^{\ell} = 0) \tag{A.2.58}$$

$$= [\nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}]^{\top} \frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \cdot \mathrm{d}\theta^{\ell} \tag{A.2.59}$$

$$= \left\langle \nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}, \frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \cdot \mathrm{d}\theta^{\ell} \right\rangle \tag{A.2.60}$$

$$= \left\langle \left(\frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \right)^{*} \nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}, \mathrm{d}\theta^{\ell} \right\rangle \tag{A.2.61}$$

$$= \langle \nabla_{\theta^{\ell}} \mathcal{L}, \mathrm{d}\theta^{\ell} \rangle. \tag{A.2.62}$$

Por lo tanto, para calcular $\nabla_{\theta^{\ell}} \mathcal{L}$ calculamos $\nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}$ y luego le aplicamos la derivada adjunta $\left(\frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \right)^{*}$. Dado que $\nabla_{\theta^{\ell}} \mathcal{L}$ depende de $\nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}$, también queremos poder calcular los gradientes con respecto a $\boldsymbol{Z}^{\ell}$. Esto se puede calcular exactamente de la misma manera:

$$\mathrm{d}\mathcal{L} = \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \cdot \mathrm{d}\boldsymbol{Z}^{\ell+1} \tag{A.2.63}$$

$$= \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \cdot \mathrm{d}(f^{\ell}(\boldsymbol{Z}^{\ell}, \theta^{\ell})) \tag{A.2.64}$$

$$= \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \left[\frac{\mathrm{d}f^{\ell}}{\mathrm{d}\boldsymbol{Z}^{\ell}} \cdot \mathrm{d}\boldsymbol{Z}^{\ell} + \frac{\mathrm{d}f^{\ell}}{\mathrm{d}\theta^{\ell}} \cdot \mathrm{d}\theta^{\ell} \right] \tag{A.2.65}$$

$$= \frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\boldsymbol{Z}^{\ell+1}} \cdot \frac{\mathrm{d}f^{\ell}}{\mathrm{d}\boldsymbol{Z}^{\ell}} \cdot \mathrm{d}\boldsymbol{Z}^{\ell} \qquad \text{(b/c } \theta^{\ell} \text{ isn't fn. of } \boldsymbol{Z}^{\ell} \text{ so } \mathrm{d}\theta^{\ell} = 0 \text{ this time)} \tag{A.2.66}$$

$$= \left\langle \left(\frac{\mathrm{d}f^{\ell}}{\mathrm{d}\boldsymbol{Z}^{\ell}} \right)^{*} \nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}, \mathrm{d}\boldsymbol{Z}^{\ell} \right\rangle \qquad \text{(same machine)} \tag{A.2.67}$$

$$= \langle \nabla_{\boldsymbol{Z}^{\ell}} \mathcal{L}, \mathrm{d}\boldsymbol{Z}^{\ell} \rangle. \tag{A.2.68}$$

Por lo tanto, para calcular $\nabla_{\boldsymbol{Z}^{\ell}} \mathcal{L}$ calculamos $\nabla_{\boldsymbol{Z}^{\ell+1}} \mathcal{L}$ y luego le aplicamos la derivada adjunta $\left(\frac{\mathrm{d}f^{\ell}}{\mathrm{d}\boldsymbol{Z}^{\ell}} \right)^{*}$. Así que tenemos una recursión para calcular $\nabla_{\boldsymbol{Z}^{\ell}} \mathcal{L}$ para todo ℓ, con el caso base $\nabla_{\boldsymbol{Z}^{L+1}} \mathcal{L}$, que viene dado por

$$\mathrm{d}\mathcal{L} = \mathrm{d}\mathsf{L} \tag{A.2.69}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \cdot \mathrm{d}\hat{\boldsymbol{y}} \tag{A.2.70}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \cdot \mathrm{d}h(\boldsymbol{Z}^{L+1}, \theta^{\text{head}}) \tag{A.2.71}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \left[\frac{\mathrm{d}h}{\mathrm{d}\boldsymbol{Z}^{L+1}} \cdot \mathrm{d}\boldsymbol{Z}^{L+1} + \frac{\mathrm{d}h}{\mathrm{d}\theta^{\text{head}}} \cdot \mathrm{d}\theta^{\text{head}} \right] \tag{A.2.72}$$

$$= \frac{\mathrm{d}\mathsf{L}}{\mathrm{d}\hat{\boldsymbol{y}}} \cdot \frac{\mathrm{d}h}{\mathrm{d}\boldsymbol{Z}^{L+1}} \cdot \mathrm{d}\boldsymbol{Z}^{L+1} \tag{A.2.73}$$

$$= \left\langle \left(\frac{\mathrm{d}h}{\mathrm{d}\boldsymbol{Z}^{L+1}} \right)^{*} \nabla_{\hat{\boldsymbol{y}}} \mathsf{L}, \mathrm{d}\boldsymbol{Z}^{L+1} \right\rangle \tag{A.2.74}$$

$$= \langle \nabla_{\boldsymbol{Z}^{L+1}} \mathcal{L}, \mathrm{d}\boldsymbol{Z}^{L+1} \rangle. \tag{A.2.75}$$

Así, tenemos la recursión:

$$\nabla_{\boldsymbol{Z}^{L+1}} \mathcal{L} = \left(\frac{\mathrm{d}h}{\mathrm{d}\boldsymbol{Z}^{L+1}} \right)^* \nabla_{\hat{\boldsymbol{y}}} \mathsf{L} \tag{A.2.76}$$

$$\nabla_{\boldsymbol{Z}^{L}} \mathcal{L} = \left(\frac{\mathrm{d}f^{L}}{\mathrm{d}\boldsymbol{Z}^{L}} \right)^* \nabla_{\boldsymbol{Z}^{L+1}} \mathcal{L} \tag{A.2.77}$$

$$\nabla_{\theta^{L}} \mathcal{L} = \left(\frac{\mathrm{d}f^{L}}{\mathrm{d}\theta^{L}} \right)^* \nabla_{\boldsymbol{Z}^{L+1}} \mathcal{L} \tag{A.2.78}$$

$$\vdots \tag{A.2.79}$$

$$\nabla_{\boldsymbol{Z}^{1}} \mathcal{L} = \left(\frac{\mathrm{d}f^{1}}{\mathrm{d}\boldsymbol{Z}^{1}} \right)^* \nabla_{\boldsymbol{Z}^{2}} \mathcal{L} \tag{A.2.80}$$

$$\nabla_{\theta^{1}} \mathcal{L} = \left(\frac{\mathrm{d}f^{1}}{\mathrm{d}\theta^{1}} \right)^* \nabla_{\boldsymbol{Z}^{2}} \mathcal{L} \tag{A.2.81}$$

$$\nabla_{\theta^{\mathrm{emb}}} \mathcal{L} = \left(\frac{\mathrm{d}f^{\mathrm{emb}}}{\mathrm{d}\theta^{\mathrm{emb}}} \right)^* \nabla_{\boldsymbol{Z}^{1}} \mathcal{L}. \tag{A.2.82}$$

Esto nos proporciona un algoritmo computacionalmente eficiente para hallar todos los gradientes en toda la red.

Terminaremos esta sección calculando la derivada adjunta para una capa simple.

Ejemplo A.8. Consideremos la capa «lineal» (afín) f^{ℓ}

$$f^{\ell}(\boldsymbol{Z}, \boldsymbol{W}^{\ell}, \boldsymbol{b}^{\ell}) \doteq \boldsymbol{W}^{\ell}\boldsymbol{Z} + \boldsymbol{b}^{\ell}\mathbf{1}^{\top} = \begin{bmatrix} \boldsymbol{W}^{\ell} & \boldsymbol{b}^{\ell} \end{bmatrix}. \tag{A.2.83}$$

Podemos calcular las derivadas con respecto a ambos parámetros como

$$\mathrm{d}f^{\ell} = [(\boldsymbol{W}^{\ell} + \mathrm{d}\boldsymbol{W}^{\ell})\boldsymbol{Z} + (\boldsymbol{b}^{\ell} + \mathrm{d}\boldsymbol{b}^{\ell})\mathbf{1}^{\top}] - [\boldsymbol{W}^{\ell}\boldsymbol{Z} + \boldsymbol{b}^{\ell}\mathbf{1}^{\top}] \tag{A.2.84}$$

$$= (\mathrm{d}\boldsymbol{W}^{\ell})\boldsymbol{Z} + (\mathrm{d}\boldsymbol{b}^{\ell})\mathbf{1}^{\top}. \tag{A.2.85}$$

Por lo tanto, la derivada de esta transformación es

$$\frac{\mathrm{d}f^{\ell}}{\mathrm{d}(\boldsymbol{W}^{\ell}, \boldsymbol{b}^{\ell})}[\mathrm{d}\boldsymbol{W}^{\ell}, \mathrm{d}\boldsymbol{b}^{\ell}] = (\mathrm{d}\boldsymbol{W}^{\ell})\boldsymbol{Z} + (\mathrm{d}\boldsymbol{b}^{\ell})\mathbf{1}^{\top}, \tag{A.2.86}$$

es decir, que representa la siguiente transformación lineal de $\mathbb{R}^{m\times d} \times \mathbb{R}^{m}$ a $\mathbb{R}^{m\times n}$:

$$T[\boldsymbol{A}, \boldsymbol{u}] = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{u}\mathbf{1}^{\top}. \tag{A.2.87}$$

Calculamos el adjunto $T^*\colon \mathbb{R}^{m\times n} \to \mathbb{R}^{m\times d} \times \mathbb{R}^{m}$ con respecto al producto interno (de Frobenius) de suma sobre las coordenadas mediante el siguiente procedimiento:

$$\langle T[\boldsymbol{A}, \boldsymbol{u}], \boldsymbol{B} \rangle_{\mathbb{R}^{m\times n}} = \mathrm{tr}((\boldsymbol{A}\boldsymbol{Z} + \boldsymbol{u}\mathbf{1}^{\top})\boldsymbol{B}^{\top}) \tag{A.2.88}$$

$$= \operatorname{tr}(\boldsymbol{A}\boldsymbol{Z}\boldsymbol{B}^\top + \boldsymbol{u}\mathbf{1}^\top\boldsymbol{B}^\top) \tag{A.2.89}$$
$$= \operatorname{tr}(\boldsymbol{A}\boldsymbol{Z}\boldsymbol{B}^\top) + \operatorname{tr}(\boldsymbol{u}\mathbf{1}^\top\boldsymbol{B}^\top) \tag{A.2.90}$$
$$= \operatorname{tr}(\boldsymbol{B}\boldsymbol{Z}^\top\boldsymbol{A}^\top) + \operatorname{tr}(\mathbf{1}^\top\boldsymbol{B}^\top\boldsymbol{u}) \tag{A.2.91}$$
$$= \langle \boldsymbol{B}\boldsymbol{Z}^\top, \boldsymbol{A}\rangle_{\mathbb{R}^{m\times d}} + \langle \boldsymbol{B}\mathbf{1}, \boldsymbol{u}\rangle_{\mathbb{R}^m} \tag{A.2.92}$$
$$= \langle \boldsymbol{B}(\boldsymbol{Z}^\top, \mathbf{1}), (\boldsymbol{A}, \boldsymbol{u})\rangle_{\mathbb{R}^{m\times d}\times\mathbb{R}^m} \tag{A.2.93}$$

Por lo tanto, $T^*\boldsymbol{B} = \boldsymbol{B}(\boldsymbol{Z}^\top, \mathbf{1})$. ■

Es importante destacar que, basándonos en una aplicación simple de la regla de la cadena, tanto la retropropagación como la diferenciación automática funcionan sobre «grafos computacionales» generales, es decir, composiciones de funciones (simples). Presentamos todos los ejemplos como capas de redes neuronales porque este es el ejemplo más común en la práctica.

A.3 Teoría de juegos y optimización minimax

En ciertos casos, como en el Capítulo 6, un problema de aprendizaje no puede reducirse a un único problema de optimización, sino que representa múltiples componentes del sistema, potencialmente opuestos, que intentan minimizar cada uno su propio objetivo. Entre los ejemplos de este paradigma se incluyen el aprendizaje de distribuciones mediante redes generativas adversarias (GAN) y la transcripción en bucle cerrado (CTRL). Denominaremos a dicho sistema un *juego de dos jugadores*, en el que hay dos «jugadores» (es decir, componentes) denominados Jugador 1 y Jugador 2 que buscan minimizar sus propios objetivos $\mathcal{L}^1$ y $\mathcal{L}^2$, respectivamente. El Jugador 1 elige los parámetros $\theta \in \Theta$ y el Jugador 2 elige los parámetros $\eta \in \mathrm{H}$. En este libro consideramos el caso especial de los *juegos de suma cero*, es decir, definimos un objetivo común $\mathcal{L}$ tal que $\mathcal{L} = -\mathcal{L}^1 = \mathcal{L}^2$.

Nuestro primer ejemplo, muy preliminar, es el siguiente. Supongamos que existen funciones $u(\theta)$ y $v(\eta)$ tales que

$$\mathcal{L}(\theta, \eta) = -u(\theta) + v(\eta). \tag{A.3.1}$$

Entonces, los objetivos de ambos jugadores son independientes del otro jugador, y los jugadores deben intentar alcanzar sus respectivos óptimos:

$$\theta^\star \in \arg\min_{\theta\in\Theta} u(\theta), \qquad \eta^\star \in \arg\min_{\eta\in\mathrm{H}} v(\eta). \tag{A.3.2}$$

El par $(\theta^\star, \eta^\star)$ es un caso especial evidente de un *equilibrio*: una situación en la que ninguno de los jugadores querrá actuar, si se le presenta la oportunidad, ya que hacerlo acabaría empeorando su propia situación. Sin embargo, no todos los juegos son tan triviales; muchos tienen objetivos y estructuras de información más complicados.

En este libro, el formalismo de teoría de juegos pertinente es un juego de Stackelberg (también denominado juego secuencial). En este formalismo, un

jugador (sin pérdida de generalidad, el Jugador 1, denominado *líder*) elige sus parámetros antes que el otro (es decir, Jugador 2, también descrito como un *seguidor*), y el seguidor puede utilizar todo el conocimiento de la elección del líder para tomar su propia decisión. La noción correcta de equilibrio para un juego de Stackelberg es un *equilibrio de Stackelberg*. Para explicar este equilibrio, cabe señalar que, dado que el Jugador 2 (es decir, el seguidor) puede elegir η en respuesta a la elección θ_1 realizada por el Jugador 1 (es decir, el líder), el Jugador 2 elegiría, por supuesto, el η que minimice $\mathcal{L}(\theta_1, \cdot)$. Pero, por supuesto, un Jugador 1 racional se daría cuenta de esto y, por lo tanto, elegiría un θ_1 tal que el peor caso de η elegido por el Jugador 2 según esta regla no sea tan malo. De manera más formal, sea $\mathcal{S}(\theta) \doteq \arg\min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta)$ el conjunto de valores de η que minimizan $\mathcal{L}(\theta, \eta)$, es decir, el conjunto de todos los valores de η que el Jugador 2 es probable que elija dado que el Jugador 1 ha elegido θ. Entonces $(\theta^\star, \eta^\star)$ es un equilibrio de Stackelberg si

$$\theta^\star \in \arg\max_{\theta \in \Theta} \min_{\eta \in \mathcal{S}(\theta)} \mathcal{L}(\theta, \eta), \qquad \eta^\star \in \arg\min_{\eta \in \mathrm{H}} \mathcal{L}(\theta^\star, \eta). \tag{A.3.3}$$

De hecho (demostración como ejercicio), se puede demostrar que, en el contexto de los juegos de Stackelberg de suma cero para dos jugadores, $(\theta^\star, \eta^\star)$ es un equilibrio de Stackelberg si y solo si

$$\theta^\star \in \arg\max_{\theta \in \Theta} \min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta), \qquad \eta^\star \in \arg\min_{\eta \in \mathrm{H}} \mathcal{L}(\theta^\star, \eta), \tag{A.3.4}$$

(nótese que la notación $\mathcal{S}(\theta)$ no se utiliza ni es necesaria).

Demostración. Nótese que

$$\mathcal{S}(\theta) = \arg\min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta), \tag{A.3.5}$$

por lo que se cumple que

$$\min_{\eta \in \mathcal{S}(\theta)} \mathcal{L}(\theta, \eta) = \min_{\eta \in \arg\min_{\eta' \in \mathrm{H}} \mathcal{L}(\theta, \eta')} \mathcal{L}(\theta, \eta) = \min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta). \tag{A.3.6}$$

□

En el resto de esta sección, analizaremos brevemente algunos enfoques algorítmicos para aprender puntos de equilibrio de Stackelberg. La intuición central es que aprender un equilibrio es como dejar que las diferentes partes del sistema determinen automáticamente las compensaciones entre los diferentes objetivos que desean optimizar.

Terminamos esta sección con una advertencia: en los juegos de suma cero para dos jugadores, si se cumple que

$$\max_{\theta \in \Theta} \min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta) = \min_{\eta \in \mathrm{H}} \max_{\theta \in \Theta} \mathcal{L}(\theta, \eta) \tag{A.3.7}$$

entonces todo equilibrio de Stackelberg es un punto de silla,[7] es decir,

$$\theta^\star \in \arg\max_{\theta\in\Theta} \mathcal{L}(\theta, \eta^\star), \qquad \eta^\star \in \arg\min_{\eta\in\mathrm{H}} \mathcal{L}(\theta^\star, \eta), \tag{A.3.8}$$

y viceversa, y además cada equilibrio de Stackelberg tiene el (mismo) valor objetivo

$$\max_{\theta\in\Theta} \min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta).$$

Demostración. Supongamos que, efectivamente,

$$\max_{\theta\in\Theta} \min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta) = \min_{\eta\in\mathrm{H}} \max_{\theta\in\Theta} \mathcal{L}(\theta, \eta). \tag{A.3.9}$$

Supongamos primero que $(\theta^\star, \eta^\star)$ es un punto de silla. Demostraremos que se trata de un equilibrio de Stackelberg. Por definición, tenemos

$$\min_{\eta\in\mathrm{H}} \mathcal{L}(\theta^\star, \eta) = \mathcal{L}(\theta^\star, \eta^\star) = \max_{\theta\in\Theta} \mathcal{L}(\theta, \eta^\star). \tag{A.3.10}$$

Por lo tanto, se cumple para cualquier θ y η que

$$\min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta) \le \mathcal{L}(\theta, \eta^\star) \le \mathcal{L}(\theta^\star, \eta^\star). \tag{A.3.11}$$

Por lo tanto

$$\max_{\theta\in\Theta} \min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta) \le \mathcal{L}(\theta^\star, \eta^\star). \tag{A.3.12}$$

De manera totalmente simétrica,

$$\mathcal{L}(\theta^\star, \eta^\star) \le \mathcal{L}(\theta^\star, \eta) \le \max_{\theta\in\Theta} \mathcal{L}(\theta, \eta) \implies \mathcal{L}(\theta^\star, \eta^\star) \le \min_{\eta\in\mathrm{H}} \max_{\theta\in\Theta} \mathcal{L}(\theta, \eta). \tag{A.3.13}$$

Por lo tanto, dado que $\max\min = \min\max$, tenemos

$$\max_{\theta\in\Theta} \min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta) = \mathcal{L}(\theta^\star, \eta^\star) = \min_{\eta\in\mathrm{H}} \max_{\theta\in\Theta} \mathcal{L}(\theta, \eta). \tag{A.3.14}$$

En particular, se cumple que

$$\theta^\star \in \arg\max_{\theta\in\Theta} \min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta). \tag{A.3.15}$$

A partir de la condición del punto de silla, tenemos que $\eta^\star \in \arg\min_{\eta\in\mathrm{H}} \mathcal{L}(\theta^\star, \eta)$. Por lo tanto, $(\theta^\star, \eta^\star)$ es un equilibrio de Stackelberg. Además, también hemos demostrado que todos los puntos de silla obedecen a (A.3.14).

Supongamos ahora que $(\theta^\star, \eta^\star)$ es un equilibrio de Stackelberg. Afirmamos que se trata de un punto de silla, lo que completa la demostración. Por la definición de equilibrio minimax,

$$\max_{\theta\in\Theta} \min_{\eta\in\mathrm{H}} \mathcal{L}(\theta, \eta) = \mathcal{L}(\theta^\star, \eta^\star). \tag{A.3.16}$$

[7] Conocido como *equilibrio de Nash*.

Entonces, por la suposición de que $\min\max = \max\min$, tenemos

$$\max_{\theta\in\Theta}\min_{\eta\in\mathrm{H}}\mathcal{L}(\theta,\eta) = \mathcal{L}(\theta^\star,\eta^\star) = \min_{\eta\in\mathrm{H}}\max_{\theta\in\Theta}\mathcal{L}(\theta,\eta). \tag{A.3.17}$$

Esto demuestra que todos los equilibrios minimax tienen el valor objetivo deseado $\mathcal{L}(\theta^\star,\eta^\star)$. Ahora queremos demostrar que

$$\theta^\star \in \arg\max_{\theta\in\Theta}\mathcal{L}(\theta,\eta^\star), \qquad \eta^\star \in \arg\min_{\eta\in\mathrm{H}}\mathcal{L}(\theta^\star,\eta). \tag{A.3.18}$$

De hecho, esta última afirmación se cumple por definición del equilibrio minimax, por lo que solo es necesario demostrar la primera. Es decir, demostraremos que

$$\max_{\theta\in\Theta}\mathcal{L}(\theta,\eta^\star) = \mathcal{L}(\theta^\star,\eta^\star). \tag{A.3.19}$$

Para demostrar esto, basta con que, por definición de max

$$\mathcal{L}(\theta^\star,\eta^\star) \leq \max_{\theta\in\Theta}\mathcal{L}(\theta,\eta^\star), \tag{A.3.20}$$

Por otra parte, tenemos que $\min_{\eta\in\mathrm{H}}\mathcal{L}(\theta,\eta) \leq \mathcal{L}(\theta,\eta^\star)$, por lo que

$$\mathcal{L}(\theta^\star,\eta^\star) = \max_{\theta\in\Theta}\min_{\eta\in\mathrm{H}}\mathcal{L}(\theta,\eta) \leq \max_{\theta\in\Theta}\mathcal{L}(\theta,\eta^\star). \tag{A.3.21}$$

Por lo tanto, se cumple que

$$\mathcal{L}(\theta^\star,\eta^\star) = \max_{\theta\in\Theta}\mathcal{L}(\theta,\eta^\star), \tag{A.3.22}$$

y con esto queda completada la demostración. □

Las condiciones bajo las cuales se cumple la igualdad $\min\max = \max\min$ vienen dadas por los llamados *teoremas del minimax*; el más destacado de ellos es un teorema de von Neumann. Sin embargo, en los casos que nos ocupan, esta propiedad no suele cumplirse.

A.3.1 El aprendizaje de los equilibrios de Stackelberg

¿Cómo podemos aprender puntos de equilibrio de Stackelberg mediante GDA? En general, esto es claramente imposible, ya que aprender equilibrios de Stackelberg mediante GDA es obviamente al menos tan difícil como calcular un minimizador global de una función de pérdida (por ejemplo, estableciendo que el objetivo compartido $\mathcal{L}(\theta,\eta)$ sea únicamente una función de η). Por lo tanto, podemos lograr dos tipos de garantías de convergencia:

- Cuando $\mathcal{L}$ es (fuertemente) cóncava respecto al primer argumento θ y (fuertemente) convexa respecto al segundo argumento η (además de tener gradientes de Lipschitz con respecto a ambos argumentos), podemos lograr una convergencia exponencial hacia un equilibrio de Stackelberg.

- Cuando $\mathcal{L}$ no es cóncava ni convexa en ninguno de los argumentos, podemos obtener *garantías de convergencia local*: es decir, si inicializamos los valores de los parámetros cerca de un equilibrio de Stackelberg (local) y la geometría de optimización es adecuada, entonces podemos aprender ese equilibrio de manera eficiente.

La primera situación es exactamente análoga al caso de la optimización de un solo jugador, donde demostramos que el descenso por gradiente converge exponencialmente para objetivos fuertemente convexos que tienen un gradiente Lipschitz. La segunda situación también es análoga al caso de la optimización de un solo jugador, aunque no la tratamos en profundidad debido a la dificultad técnica; de hecho, existen garantías de convergencia local para objetivos no convexos que tienen una geometría localmente favorable.

El algoritmo en estos dos casos es el mismo, denominado descenso-ascenso por gradiente (GDA). Para entender el GDA, supongamos que estamos tratando de aprender $\theta^\star$. Podríamos aplicar el ascenso por gradiente a la función $\theta \mapsto \min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta)$. Pero entonces tendríamos que tomar la derivada con respecto a θ de esta función. Para ver cómo hacerlo, supongamos que $\mathcal{L}$ es fuertemente convexa en η, de modo que existe un minimizador $\eta^\star(\theta)$ de $\mathcal{L}(\theta, \cdot)$. Entonces, *el teorema de Danskin* establece que

$$\nabla_\theta \left[\min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta) \right] = \nabla_\theta \mathcal{L}(\theta, \texttt{stop_grad}(\eta^\star(\theta))), \tag{A.3.23}$$

donde el gradiente es *solo con respecto al primer argumento* (es decir, no es una derivada total, lo que requeriría calcular la jacobiana de $\eta^\star(\theta)$ con respecto a θ), lo que se indica mediante el operador stop-gradient.[8] Para derivar con respecto a θ, necesitamos establecer un *proceso secundario* que también optimice η a fin de obtener una aproximación de $\eta^\star(\theta)$. Podemos hacerlo mediante la siguiente plantilla algorítmica:

$$\eta_{k+1} = \eta_{k+1}^{T+1}; \qquad \eta_{k+1}^{t+1} = \eta_{k+1}^{t} - h \nabla_\eta \mathcal{L}(\theta_k, \eta_{k+1}^t), \quad \forall t \in [T]; \qquad \eta_{k+1}^1 = \eta_k \tag{A.3.24}$$

$$\theta_{k+1} = \theta_k + h \nabla_\theta \mathcal{L}(\theta_k, \eta_{k+1}). \tag{A.3.25}$$

Es decir, damos T pasos de descenso por gradiente para actualizar η_k (con la esperanza de llegar al mínimo de $\mathcal{L}(\theta_k, \cdot)$), y luego damos un paso de ascenso por gradiente para actualizar θ_k. Como ventaja adicional, además de estimar $\theta_K \approx \arg\max_\theta \min_\eta \mathcal{L}(\theta, \eta)$, también aprendemos un $\eta_K \approx \arg\min_\eta \mathcal{L}(\theta_K, \eta)$ — esto es un equilibrio de Stackelberg aproximado.

[8] En el caso de que $\mathcal{L}$ no sea fuertemente convexa en η, sino simplemente convexa, el teorema de Danskin puede expresarse en términos de subdiferenciales. Si $\mathcal{L}$ no es convexa en absoluto en η, entonces esta derivada puede no estar bien definida, pero de todos modos se pueden obtener garantías de convergencia (local) para el algoritmo resultante. Por lo tanto, utilizamos el teorema de Danskin como motivación y no como justificación para nuestros algoritmos. Los teoremas de Danskin pueden generalizarse a situaciones más flexibles mediante los denominados *teoremas de envoltura*.

En la práctica, este método no suele utilizarse, ya que requiere $T + 1$ iteraciones totales de descenso por gradiente para actualizar θ una sola vez. En su lugar, utilizamos la denominada iteración de *descenso-ascenso por gradiente (simultáneo)* (GDA)

$$\theta_{k+1} = \theta_k + h\nabla_\theta \mathcal{L}(\theta_k, \eta_k) \tag{A.3.26}$$

$$\eta_{k+1} = \eta_k - Th\nabla_\eta \mathcal{L}(\theta_k, \eta_k), \tag{A.3.27}$$

lo cual se puede implementar de manera eficiente mediante un único paso de gradiente en (θ, η). La idea clave aquí es que, para que nuestro método se acerque a la iteración ineficiente anterior, utilizamos una actualización de η que es T veces más rápida que la actualización de θ (estas pueden considerarse prácticamente iguales si se linealiza la dinámica).

Es fundamental elegir T de manera sensata. ¿Cómo podemos hacerlo? A continuación, analizamos dos configuraciones de T que conducen a la convergencia del GDA hacia un equilibrio de Stackelberg bajo diferentes supuestos.

Convergencia del GDA de escala única de tiempo hacia el equilibrio de Stackelberg

Si $T = 1$ (es decir, denominado *de escala de tiempo única* porque tanto las actualizaciones de θ como las de η son de la misma escala), entonces el algoritmo GDA se convierte en

$$\theta_{k+1} = \theta_k + h\nabla_\theta \mathcal{L}(\theta_k, \eta_k), \qquad \eta_{k+1} = \eta_k - h\nabla_\eta \mathcal{L}(\theta_k, \eta_k). \tag{A.3.28}$$

Si $\mathcal{L}$ tiene gradientes de Lipschitz en θ y η, y es fuertemente cóncava en θ y fuertemente convexa en η, y es coercitiva (es decir, $\lim_{\|\theta\|_2, \|\eta\|_2 \to \infty} \mathcal{L}(\theta, \eta) = \infty$), entonces todos los puntos de silla son equilibrios de Stackelberg y viceversa. El trabajo [ZAK24] demuestra que el GDA (de nuevo, con $T = 1$) con un tamaño de paso h suficientemente pequeño converge a un punto de silla (y, por lo tanto, a un equilibrio de Stackelberg) a una velocidad exponencial si existe alguno, en forma análoga al descenso por gradiente para funciones fuertemente convexas con gradiente de Lipschitz.[9] Según nuestro conocimiento, este tipo de resultados constituye la única justificación rigurosa conocida para el GDA de escala temporal única.

Convergencia local del GDA de doble escala de tiempo hacia el equilibrio de Stackelberg

La convexidad/concavidad fuerte es una propiedad *global*, y ninguno de los juegos que analizamos en este libro tiene objetivos que sean globalmente fuertemente cóncavos/fuertemente convexos. En este caso, lo mejor que

[9] No proporcionamos el tamaño del paso ni la base exponencial, ya que se trata de funciones complejas de la constante de convexidad/concavidad/Lipschitz del gradiente. Por supuesto, el artículo [ZAK24] ofrece los valores exactos de los parámetros.

podemos esperar es una convergencia *local* hacia equilibrios de Stackelberg: si los parámetros se inicializan cerca de un equilibrio de Stackelberg, entonces el GDA puede converger hacia él, dado un tamaño de paso h y una escala de tiempo T adecuados.

De hecho, nuestros resultados también se mantienen para una versión del *equilibrio local* denominado *punto de equilibrio diferencial de Stackelberg*, que fue introducido en [FCR19] (aunque utilizamos la definición precisa de [LFD+22]), y que definimos de la siguiente manera. Un punto $(\theta^\star, \eta^\star)$ es un equilibrio diferencial de Stackelberg si:

- $\nabla_\eta \mathcal{L}(\theta^\star, \eta^\star) = \mathbf{0}$;
- $\nabla^2_\eta \mathcal{L}(\theta^\star, \eta^\star)$ es simétrica positiva definida;
- $\nabla_\theta \mathcal{L}(\theta^\star, \eta^\star) = \mathbf{0}$;
- $(\nabla^2_\theta \mathcal{L} + [\frac{\mathrm{d}}{\mathrm{d}\theta}\nabla_\eta \mathcal{L}][\nabla^2_\eta \mathcal{L}]^{-1}[\frac{\mathrm{d}}{\mathrm{d}\eta}\nabla_\theta \mathcal{L}])(\theta^\star, \eta^\star)$ es simétrica negativa definida.

Nótese que la última condición requiere la matriz hessiana (total)

$$\nabla^2 \mathcal{L}(\theta, \eta) = \begin{bmatrix} \nabla^2_\theta \mathcal{L}(\theta, \eta) & \frac{\mathrm{d}}{\mathrm{d}\eta}\nabla^2_\theta \mathcal{L}(\theta, \eta) \\ \frac{\mathrm{d}}{\mathrm{d}\theta}\nabla^2_{\eta\theta} \mathcal{L}(\theta, \eta) & \nabla^2_\eta \mathcal{L}(\theta, \eta) \end{bmatrix} \tag{A.3.29}$$

o, de manera equivalente, que su complemento de Schur sea definido negativo. Si consideramos el cálculo de $\nabla_\theta[\min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta)]$ proporcionado por el teorema de Danskin, los dos últimos criterios son, en realidad, restricciones sobre el gradiente y la hessiana de la función $\theta \mapsto \min_{\eta \in \mathrm{H}} \mathcal{L}(\theta, \eta)$, lo que garantiza que el gradiente sea 0 y que la hessiana sea semidefinida negativa. Esta intuición nos indica que podemos esperar que cada equilibrio de Stackelberg sea un equilibrio diferencial de Stackelberg; [FCR20] confirma esto de manera rigurosa (con sujeción a ciertas condiciones técnicas).

De manera análoga al concepto de óptimo local estricto en la optimización de un solo agente (donde se requiere que $\nabla^2 \mathcal{L}(\theta^\star)$ sea semidefinida positiva), la definición del equilibrio diferencial de Stackelberg implica que $\mathcal{L}(\theta, \cdot)$ es localmente (estricta) en una vecindad del equilibrio, y que $\min_{\eta \in \mathrm{H}} \mathcal{L}(\cdot, \eta)$ es localmente (estricta) cóncava en la misma región.

En este contexto, presentamos el resultado de [LFD+22]. Sea $(\theta^\star, \eta^\star)$ un equilibrio diferencial de Stackelberg. Supongamos que $\mathcal{L}$ tiene gradientes de Lipschitz, es decir,

$$\max\left\{ \|\nabla^2_\theta \mathcal{L}(\theta^\star, \eta^\star)\|_2, \left\|\nabla^2_\eta \mathcal{L}(\theta^\star, \eta^\star)\right\|_2, \left\| \frac{\mathrm{d}}{\mathrm{d}\eta}\nabla_\theta \mathcal{L}(\theta^\star, \eta^\star) \right\|_2 \right\} \leq \beta. \tag{A.3.30}$$

Definamos además los parámetros locales de convexidad/concavidad fuerte de $\mathcal{L}(\theta, \cdot)$ y $\min_\eta \mathcal{L}(\cdot, \eta)$ respectivamente como

$$\mu_\eta = \lambda_{\min}(\nabla^2_\eta \mathcal{L}(\theta^\star, \eta^\star)), \tag{A.3.31}$$

$$\mu_\theta = \min\left\{ \beta, -\left(\nabla^2_\theta \mathcal{L} + \left[\frac{\mathrm{d}}{\mathrm{d}\theta}\nabla_\eta \mathcal{L}\right] \left[\nabla^2_\eta \mathcal{L}\right]^{-1} \left[\frac{\mathrm{d}}{\mathrm{d}\eta}\nabla_\theta \mathcal{L}\right] \right)(\theta^\star, \eta^\star) \right\} \tag{A.3.32}$$

A continuación, definamos los números de condición locales como

$$\kappa_\eta = \beta/\mu_\eta, \qquad \kappa_\theta = \beta/\mu_\theta. \tag{A.3.33}$$

El artículo [LFD+22] afirma que si tomamos un tamaño de paso $h = \frac{1}{4\beta T}$ y se cumple que $T \geq 2\kappa_\eta$, de modo que el algoritmo es

$$\theta_{k+1} = \theta_k + \frac{1}{4\beta T}\nabla_\theta \mathcal{L}(\theta_k, \eta_k), \tag{A.3.34}$$

$$\eta_{k+1} = \eta_k - \frac{1}{4\beta}\nabla_\eta \mathcal{L}(\theta_k, \eta_k), \tag{A.3.35}$$

la hessiana total $\nabla^2\mathcal{L}(\theta^\star, \eta^\star)$ es diagonalizable, e inicializamos (θ_0, η_0) lo suficientemente cerca de $(\theta^\star, \eta^\star)$, entonces existen constantes positivas $c_0, c_1 > 0$ tales que

$$\|(\theta_k, \eta_k) - (\theta^\star, \eta^\star)\|_2 \leq c_0 \left(1 - \frac{c_1}{T\kappa_\theta}\right)^k \|(\theta_0, \eta_0) - (\theta^\star, \eta^\star)\|_2, \tag{A.3.36}$$

lo que implica una convergencia exponencial hacia el equilibrio diferencial de Stackelberg.

A.3.2 Consideraciones prácticas para el aprendizaje de los equilibrios de Stackelberg

En la práctica, no sabemos cómo inicializar los parámetros cerca de un equilibrio de Stackelberg (diferencial). Debido a las simetrías presentes en el objetivo, incluidas las inducidas por la sobreparametrización de las redes neuronales que se están entrenando, cabe esperar (de manera heurística) que la mayoría de las inicializaciones se encuentren cerca de un equilibrio de Stackelberg. Además, no sabemos cómo calcular el tamaño del paso h ni la escala de tiempo T, ya que dependen de las propiedades de la función de pérdida $\mathcal{L}$ en el equilibrio. En la práctica, existen algunos enfoques habituales:

- Tomemos $T = 1$ (pasos de igual tamaño) y utilicemos actualizaciones para θ y η derivadas de un optimizador con tasa de aprendizaje adaptativa como Adam (en contraposición al GD básico). Aquí, *se espera* (pero no *se sabe*) que el optimizador pueda ajustar las tasas de aprendizaje para alcanzar un buen equilibrio.

- Supongamos que T es una constante, por ejemplo, $T = 10^6$, lo que implica que η alcanza el equilibrio 10^6 veces más rápido que θ. Aquí también puedes utilizar actualizaciones al estilo Adam, y esperar que eso solucione el problema de la escala de tiempo.

- Supongamos que T depende de la iteración k, y que $T_k \to \infty$ cuando $k \to \infty$. Este esquema fue estudiado (también en el caso del ruido) por Borkar en [Bor97].

Por ejemplo, se puede utilizar esto al entrenar modelos de tipo CTRL (véase el Capítulo 6), donde el codificador es el Jugador 1 y el decodificador es el Jugador 2. En el Teorema 6.1 se ofrece cierta teoría sobre CTRL.

A.4 Ejercicios

Ejercicio A.1. Hemos demostrado que, para una función suave f, el descenso por gradiente converge linealmente al óptimo global si es fuertemente convexa. Sin embargo, en la optimización no convexa general, no contamos con convexidad, y mucho menos con fuerte convexidad. Afortunadamente, en algunos casos, f satisface la llamada desigualdad μ-Polyak-Lojasiewicz (PL), es decir, existe una constante $\mu > 0$ tal que para todo θ,

$$\frac{1}{2}\|\nabla f(\theta)\|_2^2 \geq \mu\left(f(\theta) - f(\theta^\star)\right),$$

donde θ^* es un minimizador de f.

Demuestre que, en virtud de la desigualdad de PL y suponiendo que f es β-suave, el descenso por gradiente (A.1.12) converge linealmente a θ^*.

Ejercicio A.2. Calcule la derivada y la derivada adjunta de la función softmax, definida de la siguiente manera.

$$\operatorname{softmax}\left(\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}\right) = \frac{1}{\sum_{i=1}^n e^{x_i}} \begin{bmatrix} e^{x_1} \\ \vdots \\ e^{x_n} \end{bmatrix}. \tag{A.4.1}$$

Ejercicio A.3. Realize el cálculo de retropropagación para una red neuronal multicapas (MLP) de L capas, tal y como se define en la Sección 8.2.3.

Ejercicio A.4. Realize el cálculo de retropropagación para un transformador de L capas, tal y como se define en la Sección 8.2.3.

Ejercicio A.5. Realize el cálculo de retropropagación para un autocodificador con L capas de codificador y L capas de decodificador (sin especificar necesariamente una arquitectura).

Apéndice B

Entropía, difusión, eliminación de ruido y codificación con pérdida

> "*El aumento del desorden o la entropía con el paso del tiempo es un ejemplo de lo que se denomina «flecha del tiempo», algo que distingue el pasado del futuro, dando una dirección al tiempo.*"
>
> – Breve historia del tiempo, Stephen Hawking

En este apéndice ofrecemos demostraciones de varios resultados, mencionados en el Capítulo 3, relacionados con la entropía diferencial, cómo evoluciona en procesos de difusión y sus conexiones con la codificación con pérdida. Haremos la siguiente supuesto moderado sobre la variable aleatoria que representa la fuente de datos, denotada por $\boldsymbol{x}$.

Supuesto B.1. $\boldsymbol{x}$ se define en un conjunto compacto $\mathcal{S} \subseteq \mathbb{R}^D$ de radio no mayor que R, es decir, $R \doteq \sup_{\boldsymbol{\xi} \in \mathcal{S}} \|\boldsymbol{\xi}\|_2$.

En particular, dado que los conjuntos compactos en el espacio euclidiano son acotados, se cumple que $R < \infty$. Utilizaremos de manera sistemática la notación $B_r(\boldsymbol{\xi}) \doteq \{\boldsymbol{u} \in \mathbb{R}^D \colon \|\boldsymbol{\xi} - \boldsymbol{u}\|_2 \leq r\}$ para denotar la bola euclidiana de radio r centrada en $\boldsymbol{\xi}$. En este sentido, el Supuesto B.1 tiene $\mathcal{S} \subseteq B_R(\mathbf{0})$.

Obsérvese que este supuesto se cumple para (casi) todas las variables que nos interesan en la práctica, ya que (a menudo) viene impuesta por un paso de normalización durante el preprocesamiento de los datos.

B.1 Entropía diferencial de distribuciones de baja dimensión

En este breve apéndice analizamos la entropía diferencial de las distribuciones de baja dimensión. Por definición, la entropía diferencial de una variable aleatoria $\boldsymbol{x}$ que no posee densidad es $-\infty$; esto incluye a todas las variables aleatorias definidas en conjuntos de baja dimensión. El objetivo de esta sección es analizar por qué este es un valor «moralmente correcto».

De hecho, sea $\boldsymbol{x}$ cualquier variable aleatoria tal que se cumpla la hipótesis del Supuesto B.1, es decir, el soporte $\mathcal{S}$ de $\boldsymbol{x}$ tiene volumen 0.[1] Consideraremos el caso en que $\boldsymbol{x}$ sea uniforme en $\mathcal{S}$.[2] Nuestro objetivo es calcular $h(\boldsymbol{x})$.

En este caso, $\boldsymbol{x}$ no tendría densidad; en el caso hipotético en el que no supiéramos que $h(\boldsymbol{x}) = -\infty$, no podríamos definirla directamente utilizando la definición estándar de entropía diferencial. En cambio, como en el resto del análisis y la teoría de la información, sería razonable considerar el límite de las entropías de aproximaciones sucesivamente mejores $\boldsymbol{x}_\varepsilon$ de $\boldsymbol{x}$ que tienen densidades, es decir,

$$h(\boldsymbol{x}) \text{ “=” } \lim_{\varepsilon \searrow 0} h(\boldsymbol{x}_\varepsilon). \tag{B.1.1}$$

Con este fin, la idea básica consiste en tomar un entorno tubular de espesor ε de $\mathcal{S}$, denotado por ejemplo como $\mathcal{S}_\varepsilon$, definido como

$$\mathcal{S}_\varepsilon = \bigcup_{\boldsymbol{\xi} \in \mathcal{S}} B_\varepsilon(\boldsymbol{\xi}) \tag{B.1.2}$$

y que se muestra en la Figura B.1. Trabajaremos con variables aleatorias cuyo

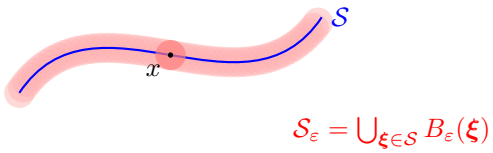


Figura B.1: Ilustración del entorno tubular de espesor ε $\mathcal{S}_\varepsilon$ de una curva $\mathcal{S} \subseteq \mathbb{R}^2$.

soporte sea $\mathcal{S}_\varepsilon$, que es de dimensión completa, y tomaremos el límite cuando $\varepsilon \to 0$. De hecho, definamos $\boldsymbol{x}_\varepsilon \sim \mathcal{U}(\mathcal{S}_\varepsilon)$. Dado que $\mathcal{S}_\varepsilon$ tiene medida de Lebesgue positiva, $\boldsymbol{x}_\varepsilon$ tiene una densidad p_ε igual a

$$p_\varepsilon(\boldsymbol{\xi}) = \mathbf{1}(\boldsymbol{\xi} \in \mathcal{S}_\varepsilon) \cdot \frac{1}{\text{vol}(\mathcal{S}_\varepsilon)}. \tag{B.1.3}$$

Al calcular la entropía de $\boldsymbol{x}_\varepsilon$ utilizando la convención de que $0 \log 0 = 0$, se cumple que

$$h(\boldsymbol{x}_\varepsilon) = -\int_{\mathbb{R}^D} p_\varepsilon(\boldsymbol{\xi}) \log p_\varepsilon(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} \tag{B.1.4}$$

[1] Formalmente, esto significa que $\mathcal{S}$ es boreliana medible con medida boreliana 0.
[2] Digamos, con respecto a la medida de Hausdorff en $\mathcal{S}$.

$$= -\int_{\mathcal{S}_\varepsilon} \frac{1}{\mathrm{vol}(\mathcal{S}_\varepsilon)} \log\left(\frac{1}{\mathrm{vol}(\mathcal{S}_\varepsilon)}\right) \mathrm{d}\boldsymbol{\xi} \tag{B.1.5}$$

$$= \frac{\log(\mathrm{vol}(\mathcal{S}_\varepsilon))}{\mathrm{vol}(\mathcal{S}_\varepsilon)} \int_{\mathcal{S}_\varepsilon} \mathrm{d}\boldsymbol{\xi} \tag{B.1.6}$$

$$= \log(\mathrm{vol}(\mathcal{S}_\varepsilon)). \tag{B.1.7}$$

Dado que $\mathcal{S}$ es compacto, $\mathrm{vol}(\mathcal{S}_\varepsilon)$ es finito y tiende a 0 cuando $\varepsilon \searrow 0$. Por lo tanto

$$h(\boldsymbol{x}) = \lim_{\varepsilon \searrow 0} h(\boldsymbol{x}_\varepsilon) = \lim_{\varepsilon \searrow 0} \log(\mathrm{vol}(\mathcal{S}_\varepsilon)) = -\infty, \tag{B.1.8}$$

tal como se desea.

El cálculo anterior es, en realidad, una consecuencia de un conjunto de resultados mucho más famosos y reconocidos sobre la entropía máxima posible de $\boldsymbol{x}$ sujeta a ciertas restricciones en la distribución de $\boldsymbol{x}$. Sería una omisión por nuestra parte no incluir aquí dichos resultados; las demostraciones se encuentran, por ejemplo, en el capítulo 2 de [PW22], por ejemplo.

Teorema B.1. *Sea $\boldsymbol{x}$ una variable aleatoria en $\mathbb{R}^D$.*

1. *Si $\boldsymbol{x}$ tiene soporte en un conjunto compacto $\mathcal{S} \subseteq \mathbb{R}^D$ (es decir, Supuesto B.1), entonces*
$$h(\boldsymbol{x}) \leq h(\mathcal{U}(\mathcal{S})) = \log \mathrm{vol}(\mathcal{S}). \tag{B.1.9}$$

2. *Si $\boldsymbol{x}$ tiene matriz de covarianza finita tal que, para una matriz PSD $\boldsymbol{\Sigma} \in \mathsf{PSD}(D)$, se cumple que $\mathrm{Cov}(\boldsymbol{x}) \preceq \boldsymbol{\Sigma}$ (con respecto al orden PSD, es decir, $\boldsymbol{\Sigma} - \mathrm{Cov}(\boldsymbol{x})$ es PSD), entonces*
$$h(\boldsymbol{x}) \leq h(\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})) = \frac{1}{2} \log((2\pi e)^D \det \boldsymbol{\Sigma}). \tag{B.1.10}$$

3. *Si $\boldsymbol{x}$ tiene un momento de segundo orden finito tal que, para una constante $a \geq 0$, se cumple que $\mathbb{E}\,\|\boldsymbol{x}\|_2^2 \leq a$, entonces*
$$h(\boldsymbol{x}) \leq h\left(\mathcal{N}\left(\mathbf{0}, \frac{a}{D}\boldsymbol{I}\right)\right) = \frac{D}{2} \log \frac{2\pi e a}{D}. \tag{B.1.11}$$

B.2 Procesos de difusión y eliminación de ruido

En el cuerpo principal (Capítulo 3), consideramos una variable aleatoria $\boldsymbol{x}$ y un proceso estocástico definido por (3.2.1), es decir,

$$\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g}, \qquad \forall t \in [0, T] \tag{B.2.1}$$

donde $\boldsymbol{g} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ independientemente de $\boldsymbol{x}$.

La estructura de esta sección es la siguiente. En el Apéndice B.2.1 presentamos un teorema formal y una demostración rigurosa que muestra que, bajo la Ecuación (B.2.1), la entropía aumenta, es decir, $\frac{\mathrm{d}}{\mathrm{d}t} h(\boldsymbol{x}_t) > 0$. En el Apéndice B.2.2 presentamos un teorema formal y una demostración rigurosa

que muestra que, bajo la Ecuación (B.2.1), la entropía disminuye durante el proceso de eliminación de ruido, es decir, $h(\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]) < h(\boldsymbol{x}_t)$ para todo $s < t$. En el Apéndice B.2.3 ofrecemos pruebas de los lemas técnicos necesarios para establecer las afirmaciones de las subsecciones anteriores.

Antes de empezar, vamos a definir algunas notaciones clave. En primer lugar, sea φ_t la densidad de $\mathcal{N}(\mathbf{0}, t^2\boldsymbol{I})$, es decir,

$$\varphi_t(\boldsymbol{\xi}) \doteq \frac{1}{(2\pi)^{D/2}t^D} \exp\left(-\frac{\|\boldsymbol{\xi}\|_2^2}{2t^2}\right). \tag{B.2.2}$$

A continuación, dado que $\boldsymbol{x}_t$ se define en todo el espacio $\mathbb{R}^D$, tiene una *densidad*, que denotamos como p_t (al igual que en el cuerpo principal). Un cálculo rápido muestra que

$$p_t(\boldsymbol{\xi}) = \mathbb{E}[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})], \tag{B.2.3}$$

y a partir de esta representación es posible deducir (es decir, a partir de Proposición B.4) que p_t es suave (es decir, infinitamente diferenciable) en $\boldsymbol{\xi}$, también suave en t, y positiva en todas partes. Este hecho resulta algo notable a primera vista: incluso para una variable aleatoria completamente irregular $\boldsymbol{x}$ (por ejemplo, una variable aleatoria de Bernoulli, que no posee densidad), su suavizado gaussiano admite una densidad para todo $t > 0$ (arbitrariamente pequeño). La demostración se deja como ejercicio para los lectores versados en análisis matemático.

Sin embargo, también debemos añadir un supuesto sobre la *suavidad* de la distribución de $\boldsymbol{x}$, lo que eliminará algunos problemas técnicos que surgen en torno a $t = 0$ con distribuciones de baja dimensión.[3] A pesar de esto, esperamos que nuestros resultados se mantengan bajo supuestos más suaves con trabajo adicional. Por ahora, supongamos que:

Supuesto B.2. $\boldsymbol{x}$ tiene una densidad dos veces continuamente diferenciable, denotada por p.

B.2.1 El proceso de difusión aumenta la entropía con el paso del tiempo

En esta sección ofrecemos una demostración del Teorema B.2. Por razones de conveniencia, la reformulamos de la siguiente manera.

Teorema B.2 (La difusión aumenta la entropía)**.** *Sea $\boldsymbol{x}$ una variable aleatoria cualquiera tal que los Supuestos B.1 y B.2 se cumplan, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico* (B.2.1)*. Entonces*

$$h(\boldsymbol{x}_s) < h(\boldsymbol{x}_t), \qquad \forall s, t\colon 0 \le s < t \le T. \tag{B.2.4}$$

Demostración. Antes de comenzar, debemos preguntarnos: ¿en qué casos tiene sentido la desigualdad de (B.2.4)? Demostraremos en el Lema B.1 que, bajo

[3] Ya que entonces diversas magnitudes se vuelven muy irregulares y manejarlas requeriría un análisis adicional significativo.

nuestras suposiciones, la entropía diferencial está bien definida, nunca es $+\infty$ y, para $t > 0$, es finita, por lo que la desigualdad (estricta) en (B.2.4) tiene sentido.

Dejando a un lado la cuestión de la biendefinición, el quid de esta demostración consiste en demostrar que la densidad p_t de $\boldsymbol{x}_t$ satisface una ecuación diferencial parcial concreta, que es muy similar a la *ecuación del calor*. La ecuación del calor es una famosa ecuación diferencial parcial que describe la difusión del calor en el espacio. Esto debería tener sentido de manera intuitiva y nos permite formarnos una imagen mental: a medida que el tiempo t aumenta, la probabilidad del vector original (quizás muy concentrado) $\boldsymbol{x}$ se esparce por todo $\mathbb{R}^D$, como el calor que se irradia desde una fuente en el vacío.

Estas ecuaciones diferenciales parciales para p_t, conocidas como *ecuaciones de Fokker-Planck* para procesos estocásticos más generales, son herramientas muy poderosas, ya que nos permiten describir las derivadas temporales instantáneas de p_t en términos de las derivadas espaciales instantáneas de p_t, y viceversa, proporcionando una descripción concisa de la regularidad y la dinámica de p_t. Una vez que obtenemos la dinámica de p_t, podemos utilizar el sistema para obtener otra que describa la dinámica de $h(\boldsymbol{x}_t)$, que, al fin y al cabo, no es más que una funcional de la densidad p_t.

La descripción de la EDP implica un objeto matemático denominado el laplaciano Δ. Recordemos de la clase de cálculo multivariable que el laplaciano, al actuar sobre una función $f\colon (0,T)\times\mathbb{R}^D \to \mathbb{R}$ diferenciable en el tiempo y dos veces diferenciable en el espacio, viene dado por

$$\Delta f_t(\boldsymbol{\xi}) = \operatorname{tr}(\nabla^2 f_t(\boldsymbol{\xi})) = \sum_{i=1}^{D} \frac{\partial^2 f_t}{\partial \xi_i^2}(\boldsymbol{\xi}). \tag{B.2.5}$$

Es decir, al utilizar la representación integral de p_t y derivarla con respecto a la integral, podemos calcular las derivadas de p_t (lo cual hacemos en la Proposición B.1) y observamos que p_t satisface la ecuación diferencial parcial de tipo calorífico

$$\frac{\partial p_t}{\partial t}(\boldsymbol{\xi}) = t\Delta p_t(\boldsymbol{\xi}). \tag{B.2.6}$$

A continuación, para hallar la dinámica de $h(\boldsymbol{x}_t)$, podemos volver a utilizar la Proposición B.3, así como la ecuación diferencial parcial de tipo calorífico, para obtener

$$\begin{aligned}
\frac{\mathrm{d}}{\mathrm{d}t} h(\boldsymbol{x}_t) &= -\frac{\mathrm{d}}{\mathrm{d}t} \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} && \text{(B.2.7)}\\
&= -\int_{\mathbb{R}^D} \frac{\partial}{\partial t} \left[p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi})\right] \mathrm{d}\boldsymbol{\xi} && \text{(B.2.8)}\\
&= -\int_{\mathbb{R}^D} \frac{\partial p_t}{\partial t}(\boldsymbol{\xi}) [1 + \log p_t(\boldsymbol{\xi})] \mathrm{d}\boldsymbol{\xi} && \text{(B.2.9)}\\
&= -t\int_{\mathbb{R}^D} \Delta p_t(\boldsymbol{\xi}) [1 + \log p_t(\boldsymbol{\xi})] \mathrm{d}\boldsymbol{\xi}. && \text{(B.2.10)}
\end{aligned}$$

Mediante un razonamiento ligeramente complejo basado en la integración por partes (Lema B.2), obtenemos

$$\frac{\mathrm{d}}{\mathrm{d}t}h(\boldsymbol{x}_t) = t\int_{\mathbb{R}^D}\langle\nabla\log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle\mathrm{d}\boldsymbol{\xi} \tag{B.2.11}$$

$$= t\int_{\mathbb{R}^D}\frac{\|\nabla p_t(\boldsymbol{\xi})\|_2^2}{p_t(\boldsymbol{\xi})}\mathrm{d}\boldsymbol{\xi} \tag{B.2.12}$$

$$> 0 \tag{B.2.13}$$

donde en la última línea se cumple una desigualdad estricta porque, para que no se cumpliera, $\nabla p_t(\boldsymbol{\xi})$ tendría que ser nulo casi en todas partes (es decir, en todas partes excepto, posiblemente, en un conjunto de volumen cero), pero esto implicaría que p_t sería constante casi en todas partes, lo cual contradice el hecho de que p_t es una densidad.

Para completar la demostración, basta con aplicar el teorema fundamental del cálculo

$$h(\boldsymbol{x}_t) = h(\boldsymbol{x}_s) + \int_s^t \frac{\mathrm{d}}{\mathrm{d}u}h(\boldsymbol{x}_u)\mathrm{d}u > h(\boldsymbol{x}_s), \tag{B.2.14}$$

lo cual demuestra la afirmación. (Obsérvese que esto no tiene sentido cuando $h(\boldsymbol{x}_s) = -\infty$, lo cual solo puede ocurrir cuando $s = 0$ y $h(\boldsymbol{x}) = -\infty$, pero en este caso $h(\boldsymbol{x}_t) > -\infty$, por lo que la afirmación es verdadera de manera vacía de todos modos.) □

B.2.2 El proceso de eliminación de ruido reduce la entropía con el paso del tiempo

Recordemos que en la Sección 3.2.1 partimos de la variable aleatoria $\boldsymbol{x}_T$ y eliminamos el ruido en forma iterativa mediante iteraciones de la forma

$$\hat{\boldsymbol{x}}_s \doteq \mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t = \hat{\boldsymbol{x}}_t] = \frac{s}{t}\hat{\boldsymbol{x}}_t + \left(1 - \frac{s}{t}\right)\bar{\boldsymbol{x}}^*(t, \hat{\boldsymbol{x}}_t). \tag{B.2.15}$$

para $s, t \in \{t_0, t_1, \ldots, t_L\}$ con $s < t$ y $\boldsymbol{x}_T = \hat{\boldsymbol{x}}_T$. Queremos demostrar que $h(\hat{\boldsymbol{x}}_s) < h(\hat{\boldsymbol{x}}_t)$, demostrando que el proceso de eliminación de ruido realmente reduce la entropía.

Antes de proceder con la demostración, realizaremos varias consideraciones sobre el planteamiento del problema. En primer lugar, la fórmula de Tweedie (3.2.23) establece que

$$\bar{\boldsymbol{x}}^*(t, \boldsymbol{x}_t) = \boldsymbol{x}_t + t^2\nabla p_t(\boldsymbol{x}_t), \tag{B.2.16}$$

lo cual equipara un paso completo de eliminación de ruido desde el tiempo t hasta el tiempo 0 a un paso de gradiente sobre la densidad logarítmica de $\boldsymbol{x}_t$. ¿Podemos obtener un resultado similar para el paso completo de eliminación de ruido desde el tiempo t hasta el tiempo s en (B.2.15)? Resulta que sí podemos,

y es bastante sencillo. Utilizando (B.2.15) y la fórmula de Tweedie (3.2.23), obtenemos

$$\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t] = \frac{s}{t}\boldsymbol{x}_t + \left(1 - \frac{s}{t}\right)\left(\boldsymbol{x}_t + t^2 \nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)\right) = \boldsymbol{x}_t + \left(1 - \frac{s}{t}\right) t^2 \nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t). \tag{B.2.17}$$

Así pues, este paso iterativo de eliminación de ruido es, una vez más, un paso de gradiente sobre la densidad logarítmica perturbada $\log p_t$ con un tamaño de paso reducido. En particular, este paso puede verse como una perturbación de la distribución de la variable aleatoria $\boldsymbol{x}_t$ por el *campo vectorial de la función de score*, lo que sugiere una conexión con las ecuaciones diferenciales estocásticas (SDE) y la teoría de los modelos de difusión [SSK+21]. De hecho, se puede desarrollar una demostración del siguiente resultado Teorema B.3 utilizando este potente marco y un argumento de límite (por ejemplo, siguiendo el enfoque técnico de la exposición de [CCL+23]). Aquí daremos una prueba más sencilla, que utilizará solo herramientas elementales y, por lo tanto, aclarará algunas de las cantidades clave detrás del proceso de reducción de la entropía mediante la eliminación de ruido. Por otro lado, tendremos que realizar algunos cálculos ligeramente técnicos debido a que el proceso de eliminación de ruido en el Teorema B.3 *no* se corresponde exactamente con el proceso *inverso* asociado al proceso de adición de ruido que genera la observación $\boldsymbol{x}_t$.[4]

Queremos demostrar que $h(\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]) < h(\boldsymbol{x}_t)$, es decir, formalmente:

Teorema B.3. *Sea $\boldsymbol{x}$ una variable aleatoria cualquiera tal que los Supuestos B.1 y B.2 se cumplan, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1). Para cada $t > 0$, escribamos*

$$J(p_t) \doteq \int_{\mathbb{R}^D} \frac{\|\nabla p_t(\boldsymbol{\xi})\|_2^2}{p_t(\boldsymbol{\xi})} \mathrm{d}\boldsymbol{\xi}, \qquad U_t \doteq \max\left(\frac{D}{t^2}, \left|\frac{R^2}{t^4} - \frac{D}{t^2}\right|\right) \tag{B.2.18}$$

para la información de Fisher de p_t y un límite uniforme sobre $|\Delta \log p_t|$, respectivamente. Entonces

$$h(\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]) < h(\boldsymbol{x}_t) \tag{B.2.19}$$

para todo $0 \le s < t \le T$ tal que

$$\left(1 - \frac{s}{t}\right) t^2 U_t^2 \exp\left(\left(1 - \frac{s}{t}\right) t^2 U_t\right) < 2J(p_t). \tag{B.2.20}$$

Demostración. Esta demostración se basa en dos ideas principales:

1. En primer lugar, calcula una densidad para $\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]$ utilizando una fórmula de cambio de variables.

[4] Para quienes estén familiarizados con los modelos de difusión, nos referimos aquí al proceso directo con inversión temporal que no coincide con la secuencia de iteraciones generada por el proceso definido en el Teorema B.3. Estos procesos coinciden en un cierto límite en el que se dan infinitos pasos del Teorema B.3 con niveles infinitamente pequeños de ruido añadidos en cada paso; para pasos generales y finitos, debemos introducir algunas aproximaciones independientemente del nivel de sofisticación de nuestras herramientas.

2. En segundo lugar, limita esta densidad para controlar la entropía.

El cambio de variables se justifica mediante el Corolario B.1, que se derivó originalmente en [Gri11].

A continuación, aplicamos estas ideas. A partir del Corolario B.1, obtenemos que la función $\bar{\boldsymbol{x}}$, definida como $\bar{\boldsymbol{x}}(\boldsymbol{\xi}) \doteq \mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$ es diferenciable, inyectiva y, por lo tanto, invertible en su imagen, que en adelante denotaremos como $\mathcal{X} \subseteq \mathbb{R}^D$. Denotamos su inversa como $\bar{\boldsymbol{x}}^{-1}$. Utilizando una fórmula de cambio de variables, la densidad $\bar{p}$ de $\bar{\boldsymbol{x}}(\boldsymbol{x}_t)$ viene dada por

$$\bar{p}(\boldsymbol{\xi}) \doteq \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))}, \tag{B.2.21}$$

donde (recordemos, según el Apéndice A.2) $\bar{\boldsymbol{x}}'$ es el jacobiano de $\bar{\boldsymbol{x}}$. Dado que por el Lema B.3 sabemos que $\bar{\boldsymbol{x}}'$ es una matriz definida positiva, el determinante es positivo y, por lo tanto, la densidad total es positiva. De ahí se deduce que

$$h(\bar{\boldsymbol{x}}(\boldsymbol{x}_t)) = -\int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \log \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \mathrm{d}\boldsymbol{\xi} \tag{B.2.22}$$

$$= -\int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \log((p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})) \mathrm{d}\boldsymbol{\xi} \tag{B.2.23}$$

$$+ \int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \log\det\big(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi}))\big) \mathrm{d}\boldsymbol{\xi} \tag{B.2.24}$$

$$= -\int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} + \int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \log\det\big(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi}))\big) \mathrm{d}\boldsymbol{\xi} \tag{B.2.25}$$

$$= h(\boldsymbol{x}_t) - \int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \log\left(\frac{1}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))}\right) \mathrm{d}\boldsymbol{\xi}. \tag{B.2.26}$$

Analizaremos el último término (incluido el $-$) y demostraremos que es negativo.

Por la propiedad de la concavidad, se tiene que $-x \log x \leq 1 - x$ para todo $x \geq 0$. Por lo tanto

$$h(\bar{\boldsymbol{x}}(\boldsymbol{x}_t)) - h(\boldsymbol{x}_t) = -\int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \log\left(\frac{1}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))}\right) \mathrm{d}\boldsymbol{\xi} \tag{B.2.27}$$

$$\leq \int_{\mathcal{X}} (p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi}) \cdot \left(1 - \frac{1}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))}\right) \mathrm{d}\boldsymbol{\xi} \tag{B.2.28}$$

$$= \int_{\mathcal{X}} (p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} - \int_{\mathcal{X}} \frac{(p_t \circ \bar{\boldsymbol{x}}^{-1})(\boldsymbol{\xi})}{\det(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi})))} \mathrm{d}\boldsymbol{\xi} \tag{B.2.29}$$

$$= \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \det\big(\bar{\boldsymbol{x}}'(\bar{\boldsymbol{x}}^{-1}(\boldsymbol{\xi}))\big) \mathrm{d}\boldsymbol{\xi} - \int_{\mathcal{X}} \bar{p}(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} \tag{B.2.30}$$

$$= \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \det\Big(\boldsymbol{I} + \Big(1 - \frac{s}{t}\Big) t^2 \nabla^2 \log p_t(\boldsymbol{\xi})\Big) \mathrm{d}\boldsymbol{\xi} - 1. \tag{B.2.31}$$

Ahora bien, por la desigualdad AM-GM para los valores propios, tenemos que, para cualquier matriz simétrica definida positiva $\boldsymbol{M} \in \mathsf{PSD}(D)$, la cota

$$\det(\boldsymbol{M})^{1/D} = \prod_{i=1}^{D} \lambda_i(\boldsymbol{M})^{1/D} \leq \frac{\sum_{i=1}^{D} \lambda_i(\boldsymbol{M})}{D} = \frac{\operatorname{tr}(\boldsymbol{M})}{D}, \tag{B.2.32}$$

que podemos aplicar a la expresión anterior y obtener

$$\int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \det\left(\boldsymbol{I} + \left(1 - \frac{s}{t}\right) t^2 \nabla^2 \log p_t(\boldsymbol{\xi})\right) \mathrm{d}\boldsymbol{\xi} \tag{B.2.33}$$

$$\leq \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \operatorname{tr}\left(\frac{1}{D}\left[\boldsymbol{I} + \left(1 - \frac{s}{t}\right) t^2 \nabla^2 \log p_t(\boldsymbol{\xi})\right]\right)^D \mathrm{d}\boldsymbol{\xi} \tag{B.2.34}$$

$$= \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \left(1 + \frac{\left(1 - \frac{s}{t}\right) t^2}{D} \operatorname{tr}(\nabla^2 \log p_t(\boldsymbol{\xi}))\right)^D \mathrm{d}\boldsymbol{\xi} \tag{B.2.35}$$

$$= \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \left(1 + \frac{\left(1 - \frac{s}{t}\right) t^2}{D} \Delta \log p_t(\boldsymbol{\xi})\right)^D \mathrm{d}\boldsymbol{\xi}. \tag{B.2.36}$$

Dado que $\bar{\boldsymbol{x}}'(\boldsymbol{\xi})$ es definida positiva (Corolario B.1), tenemos que $1 + \frac{(1-s/t)t^2}{D} \Delta \log p_t(\boldsymbol{\xi}) = \frac{\operatorname{tr}(\bar{\boldsymbol{x}}'(\boldsymbol{\xi}))}{D} > 0$. Por la desigualdad $\log(1+x) \leq x$ para $x > -1$,

$$\left(1 + \frac{\left(1 - \frac{s}{t}\right) t^2}{D} \Delta \log p_t(\boldsymbol{\xi})\right)^D \leq \exp\left(\left(1 - \frac{s}{t}\right) t^2 \Delta \log p_t(\boldsymbol{\xi})\right). \tag{B.2.37}$$

Según el Lema B.5 (donde, recordemos, R es el radio del soporte de $\boldsymbol{x}$, tal como se indica en el Supuesto B.1),

$$|\Delta \log p_t(\boldsymbol{\xi})| \leq \max\left(\frac{D}{t^2}, \left|\frac{R^2}{t^4} - \frac{D}{t^2}\right|\right) =: U_t. \tag{B.2.38}$$

Si definimos $\varepsilon \doteq \left(1 - \frac{s}{t}\right) t^2 U_t$, tenemos que $\left(1 - \frac{s}{t}\right) t^2 \Delta \log p_t(\boldsymbol{\xi}) \leq \varepsilon$ para todo $\boldsymbol{\xi}$. Por el teorema de Taylor con el resto de Lagrange, para cualquier $c \leq \varepsilon$,

$$e^c \leq 1 + c + \frac{c^2}{2} e^{\varepsilon}, \tag{B.2.39}$$

puesto que $e^c = 1 + c + \frac{c^2}{2} e^{\theta c}$ para algún $\theta \in (0,1)$, y $e^{\theta c} \leq e^{\max(c,0)} \leq e^{\varepsilon}$. Aplicando estos límites e integrando,

$$\int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \left(1 + \frac{\left(1 - \frac{s}{t}\right) t^2}{D} \Delta \log p_t(\boldsymbol{\xi})\right)^D \mathrm{d}\boldsymbol{\xi} \tag{B.2.40}$$

$$\leq \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \exp\left(\left(1 - \frac{s}{t}\right) t^2 \Delta \log p_t(\boldsymbol{\xi})\right) \mathrm{d}\boldsymbol{\xi} \tag{B.2.41}$$

$$\leq 1 + \left(1 - \frac{s}{t}\right) t^2 \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \Delta \log p_t(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} + \frac{e^\varepsilon}{2} \left(1 - \frac{s}{t}\right)^2 t^4 \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \left(\Delta \log p_t(\boldsymbol{\xi})\right)^2 \mathrm{d}\boldsymbol{\xi} \tag{B.2.42}$$

$$= 1 - \left(1 - \frac{s}{t}\right) t^2 \int_{\mathbb{R}^D} \frac{\|\nabla p_t(\boldsymbol{\xi})\|_2^2}{p_t(\boldsymbol{\xi})} \mathrm{d}\boldsymbol{\xi} + \frac{e^\varepsilon}{2} \left(1 - \frac{s}{t}\right)^2 t^4 \int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \left(\Delta \log p_t(\boldsymbol{\xi})\right)^2 \mathrm{d}\boldsymbol{\xi}, \tag{B.2.43}$$

donde la identidad $\int p_t \Delta \log p_t = -\int \|\nabla p_t\|_2^2 / p_t$ es la misma que en la demostración del Teorema B.2. Escribiendo $J(p_t) \doteq \int_{\mathbb{R}^D} \frac{\|\nabla p_t(\boldsymbol{\xi})\|_2^2}{p_t(\boldsymbol{\xi})} \mathrm{d}\boldsymbol{\xi}$ para la información de Fisher de p_t, y limitando $\left(\Delta \log p_t\right)^2 \leq U_t^2$, lo combinamos con nuestra estimación anterior para obtener

$$h(\bar{\boldsymbol{x}}(\boldsymbol{x}_t)) - h(\boldsymbol{x}_t) \leq -\left(1 - \frac{s}{t}\right) t^2 J(p_t) + \frac{e^\varepsilon \varepsilon^2}{2}. \tag{B.2.44}$$

Esto es estrictamente negativo siempre que $\varepsilon^2 e^\varepsilon < 2\left(1 - \frac{s}{t}\right) t^2 J(p_t)$. Sustituyendo $\varepsilon = \frac{1 - \frac{s}{t}}{t}^2 U_t$, esto es precisamente la condición (B.2.20). □

Nótese que la condición (B.2.20) siempre se cumple: para cualquier $t > 0$ fijo, la información de Fisher satisface $J(p_t) > 0$ (puesto que p_t no es constante) y $U_t < \infty$, por lo que, si se elige un valor suficientemente pequeño de $\left(1 - \frac{s}{t}\right) t^2$ lo suficientemente pequeño garantiza que se cumpla (B.2.20).

Para comprender las implicaciones cuantitativas, consideremos el comportamiento cuando $t \to 0$ (cerca de los datos), donde la condición es más restrictiva. Si se establece $s = (1 - \varepsilon)t$, la condición se convierte en $\varepsilon t^2 U_t^2 \exp(\varepsilon t^2 U_t) < 2J(p_t)$. Para valores pequeños de t, el límite del laplaciano satisface $U_t \approx R^2/t^4$, por lo que el prefactor polinómico se escala como $\varepsilon t^2 U_t^2 \approx \varepsilon R^4/t^6$. El escalado crítico es, por lo tanto, $\varepsilon \sim t^6/R^4$. Si tomamos $\varepsilon = \alpha t^6/R^4$, la condición se reduce a $\alpha < 2J(p_t)$. En otras palabras, cada paso de eliminación de ruido de tamaño $t - s = \varepsilon\, t \sim t^7/R^4$ reduce la entropía. Los pasos deben volverse progresivamente más finos a medida que $t \to 0$, una consecuencia del resto de segundo orden en la expansión de Taylor utilizada en la demostración.

Al mismo tiempo, cabe señalar que es probable que nuestros límites sean imprecisos en cuanto a la dependencia exacta de t, y sin duda lo son para datos con mayor estructura. Por ejemplo, el crecimiento exponencial $U_t \sim R^2/t^4$ refleja un límite del laplaciano en el peor de los casos que puede ajustarse considerablemente bajo supuestos adicionales de regularidad en p. Además, si p_t es log-cóncava (como ocurre, por ejemplo, cuando la propia p es log-cóncava), entonces $\Delta \log p_t \leq 0$ en todo el espacio, y se puede desarrollar un argumento considerablemente más sencillo con conclusiones más sólidas.

B.2.3 Lemas técnicos y resultados intermedios

En esta subsección presentamos los resultados técnicos que fundamentan nuestros dos teoremas conceptuales principales. Nuestra presentación seguirá

más o menos el enfoque habitual en matemáticas; empezaremos con los resultados de nivel más alto y, progresivamente volveremos a los resultados más graduales. Los resultados de mayor nivel utilizarán los resultados incrementales, y de esta manera obtenemos un ordenamiento de dependencias legible y claro: ningún resultado depende de los que le preceden. Los resultados que no dependen unos de otros suelen ordenarse según el lugar en que aparecen en el par de demostraciones anterior.

Finitud de la entropía diferencial

En primer lugar, demostramos que la entropía existe a lo largo del proceso estocástico y que es finita.

Lema B.1. *Sea $\boldsymbol{x}$ una variable aleatoria cualquiera, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1).*

1. *Para $t > 0$, la entropía diferencial $h(\boldsymbol{x}_t)$ existe y es $> -\infty$.*
2. *Si, además, el Supuesto B.1 se cumple para $\boldsymbol{x}$, entonces $h(\boldsymbol{x}) < \infty$ y $h(\boldsymbol{x}_t) < \infty$.*

Demostración. Para probar el Lema B.1.1, emplearemos un razonamiento analítico clásico y laborioso. Dado que $\boldsymbol{x}_t$ tiene una densidad, podemos escribir

$$h(\boldsymbol{x}_t) = -\int_{\mathbb{R}^D} p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi}. \tag{B.2.45}$$

En consecuencia, definimos $g\colon \mathbb{R}^D \to \mathbb{R}$ como

$$g(\boldsymbol{\xi}) \doteq -p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}) \implies h(\boldsymbol{x}_t) = \int_{\mathbb{R}^D} g(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi}. \tag{B.2.46}$$

Como es habitual, para limitar el valor de una integral en análisis, se define

$$g_+(\boldsymbol{\xi}) \doteq \max(g(\boldsymbol{\xi}), 0), \quad g_-(\boldsymbol{\xi}) \doteq \max(-g(\boldsymbol{\xi}), 0) \tag{B.2.47}$$

$$\implies \quad g = g_+ - g_- \quad \text{and} \quad g_+, g_- \geq 0. \tag{B.2.48}$$

Entonces

$$h(\boldsymbol{x}_t) = \int_{\mathbb{R}^D} g_+(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} - \int_{\mathbb{R}^D} g_-(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi}, \tag{B.2.49}$$

y se garantiza que ambas integrales son no negativas, ya que sus integrandos también lo son.

Para demostrar que $h(\boldsymbol{x}_t)$ está bien definida, necesitamos demostrar que $\int_{\mathbb{R}^D} g_+(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} < \infty$ o $\int_{\mathbb{R}^D} g_-(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} < \infty$. Para demostrar que $h(\boldsymbol{x}_t) > -\infty$, basta con demostrar que $\int_{\mathbb{R}^D} g_-(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} < \infty$. Para acotar la integral de g_- necesitamos comprender la cantidad g_-, es decir, queremos caracterizar cuándo g es negativo.

$$g(\boldsymbol{\xi}) \leq 0 \iff p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}) \geq 0 \iff \log p_t(\boldsymbol{\xi}) \geq 0 \iff p_t(\boldsymbol{\xi}) \geq 1. \tag{B.2.50}$$

Por lo tanto, se cumple que

$$g_-(\boldsymbol{\xi}) = \mathbf{1}(p_t(\boldsymbol{\xi}) \geq 1) \cdot (-g(\boldsymbol{\xi})) = \mathbf{1}(p_t(\boldsymbol{\xi}) \geq 1) p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}). \tag{B.2.51}$$

Para poder acotar la integral de $g_-(\boldsymbol{\xi})$, necesitamos demostrar que p_t «no está demasiado concentrada», es decir, que p_t no es demasiado grande. Para demostrarlo, en este caso tenemos la suerte de poder acotar la propia función $g_-(\boldsymbol{\xi})$ en sí misma. Es decir, obsérvese que

$$\max_{\boldsymbol{\xi} \in \mathbb{R}^D} \varphi_t(\boldsymbol{\xi} - \boldsymbol{x}) = \varphi_t(\mathbf{0}) = \frac{1}{(2\pi)^{D/2} t^D} =: C_t. \tag{B.2.52}$$

que tiende a infinito cuando $t \to 0$, pero es finito para todo t finito. Por lo tanto

$$p_t(\boldsymbol{\xi}) = \mathbb{E}\, \varphi_t(\boldsymbol{\xi} - \boldsymbol{x}) \leq \mathbb{E}\, C_t = C_t. \tag{B.2.53}$$

Ahora hay dos casos.

- Si $C_t < 1$, entonces $p_t(\boldsymbol{\xi}) < 1$, por lo que el indicador nunca es 1; por lo tanto, $g_- = 0$ en todos los casos y su integral también es 0.
- Si $C_t \geq 1$, entonces $\log C_t \geq 0$, por lo que dado que el logaritmo es monótonamente creciente,

$$\begin{aligned}
\int_{\mathbb{R}^D} g_-(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} &= \int_{\mathbb{R}^D} \mathbf{1}(p_t(\boldsymbol{\xi}) \geq 1) p_t(\boldsymbol{\xi}) \log p_t(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} && \text{(B.2.54)} \\
&= \mathbb{E}[\mathbf{1}(p_t(\boldsymbol{x}_t) \geq 1) \log p_t(\boldsymbol{x}_t)] && \text{(B.2.55)} \\
&\leq \mathbb{E}[\mathbf{1}(p_t(\boldsymbol{x}_t) \geq 1) \log C_t] && \text{(B.2.56)} \\
&= \mathbb{P}[p_t(\boldsymbol{x}_t) \geq 1] \log C_t. && \text{(B.2.57)}
\end{aligned}$$

Por lo tanto, tenemos que $\int_{\mathbb{R}^D} g_-(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} < \infty$, por lo que la entropía diferencial $h(\boldsymbol{x}_t)$ existe y es $> -\infty$.

Para demostrar el Lema B.1.2, supongamos que se cumple Supuesto B.1. Queremos demostrar que $h(\boldsymbol{x}) < \infty$ y $h(\boldsymbol{x}_t) < \infty$. El método para hacerlo es el mismo e implica el resultado de entropía máxima Teorema B.1. Es decir, dado que $\boldsymbol{x}$ está absolutamente acotado, tiene una covarianza finita que denotaremos como $\boldsymbol{\Sigma}$. Entonces, la covarianza de $\boldsymbol{x}_t$ es $\boldsymbol{\Sigma} + t^2 \boldsymbol{I}$. Por lo tanto, la entropía de $\boldsymbol{x}$ y $\boldsymbol{x}_t$ puede tener como cota superior la entropía de las con las covarianzas respectivas, es decir, $\log[(2\pi e)^D \det(\boldsymbol{\Sigma})]$ o $\log[(2\pi e)^D \det(\boldsymbol{\Sigma} + t^2 \boldsymbol{I})]$, y ambas son $< \infty$. □

Integración por partes en la identidad de De Bruijn

Por último, desarrollamos el argumento de la integración por partes al que se alude en las demostraciones de los Teoremas B.2 y B.3. El argumento es conceptualmente bastante simple, aunque exige algunas estimaciones técnicas precisas para mostrar que el término de borde en la integración por partes desaparece.

Lema B.2. *Sea $\boldsymbol{x}$ una variable aleatoria tal que los Supuestos B.1 y B.2 se cumplen, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1). Para $t \geq 0$, sea p_t la densidad de $\boldsymbol{x}_t$. Entonces, para una constante $c \in \mathbb{R}$, se cumple*

$$\int_{\mathbb{R}^D} \Delta p_t(\boldsymbol{\xi})[c + \log p_t(\boldsymbol{\xi})]\mathrm{d}\boldsymbol{\xi} = -\int_{\mathbb{R}^D} \langle \nabla \log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle \mathrm{d}\boldsymbol{\xi}. \tag{B.2.58}$$

Demostración. La idea básica de esta demostración se divide en dos pasos:

- En primer lugar, aplica el teorema de Green para realizar la integración por partes sobre un conjunto compacto;
- En segundo lugar, haz que el radio de este conjunto compacto tienda a $+\infty$, para obtener integrales sobre todo el espacio $\mathbb{R}^D$.

El teorema de Green establece que, para cualquier conjunto compacto $\mathcal{K} \subseteq \mathbb{R}^D$, una función $\phi\colon \mathbb{R}^D \to \mathbb{R}$ dos veces continuamente diferenciable y una función $\psi\colon \mathbb{R}^D \to \mathbb{R}$ continuamente diferenciable,

$$\int_{\mathcal{K}} \{\psi(\boldsymbol{\xi})\Delta\phi(\boldsymbol{\xi}) + \langle \nabla\psi(\boldsymbol{\xi}), \nabla\phi(\boldsymbol{\xi})\rangle\}\, \mathrm{d}\boldsymbol{\xi} = \int_{\partial\mathcal{K}} \psi(\boldsymbol{\xi})\langle \nabla\phi(\boldsymbol{\xi}), \boldsymbol{n}(\boldsymbol{\xi})\rangle \mathrm{d}\sigma(\boldsymbol{\xi}) \tag{B.2.59}$$

donde $\mathrm{d}\sigma(\boldsymbol{\xi})$ denota una integral sobre la «medida de superficie», es decir, la medida heredada en $\partial\mathcal{K}$, es decir, el contorno de $\mathcal{K}$, y, por consiguiente, $\boldsymbol{\xi}$ toma valores en esta superficie y $\boldsymbol{n}(\boldsymbol{\xi})$ es el vector normal unitario a $\mathcal{K}$ en el punto de la superficie $\boldsymbol{\xi}$. Ahora, tomando $\phi(\boldsymbol{\xi}) \doteq p_t(\boldsymbol{\xi})$ y $\psi(\boldsymbol{\xi}) \doteq c + \log p_t(\boldsymbol{\xi})$, sobre una bola $B_r(\mathbf{0})$ de radio $(\mathrm{r} > 0)$ con centro en $\mathbf{0}$ (de modo que $\partial B_r(\mathbf{0})$ sea la esfera de radio r con centro en $\mathbf{0}$ y $\boldsymbol{n}(\boldsymbol{\xi}) = \boldsymbol{\xi}/\|\boldsymbol{\xi}\|_2 = \boldsymbol{\xi}/r$):

$$\int_{B_r(\mathbf{0})} \{\Delta p_t(\boldsymbol{\xi})[c + \log p_t(\boldsymbol{\xi})] + \langle \nabla \log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle\}\, \mathrm{d}\boldsymbol{\xi} \tag{B.2.60}$$

$$= \int_{\partial B_r(\mathbf{0})} [c + \log p_t(\boldsymbol{\xi})] \left\langle \nabla p_t(\boldsymbol{\xi}), \frac{\boldsymbol{\xi}}{r} \right\rangle \mathrm{d}\sigma(\boldsymbol{\xi}) \tag{B.2.61}$$

$$= \frac{1}{r}\int_{\partial B_r(\mathbf{0})} [c + \log p_t(\boldsymbol{\xi})]\, \langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle\, \mathrm{d}\sigma(\boldsymbol{\xi}). \tag{B.2.62}$$

Al hacer que $r \to \infty$, se cumple que

$$\int_{\mathbb{R}^D} \{\Delta p_t(\boldsymbol{\xi})[c + \log p_t(\boldsymbol{\xi})] + \langle \nabla \log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle\}\, \mathrm{d}\boldsymbol{\xi} \tag{B.2.63}$$

$$= \lim_{r\to\infty} \int_{B_r(\mathbf{0})} \{\Delta p_t(\boldsymbol{\xi})[c + \log p_t(\boldsymbol{\xi})] + \langle \nabla \log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle\}\, \mathrm{d}\boldsymbol{\xi} \tag{B.2.64}$$

$$= \lim_{r\to\infty} \frac{1}{r}\int_{\partial B_r(\mathbf{0})} [c + \log p_t(\boldsymbol{\xi})]\, \langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle\, \mathrm{d}\sigma(\boldsymbol{\xi}), \tag{B.2.65}$$

donde la primera igualdad se deduce por convergencia dominada en el integrando. Queda por calcular el último límite. Para ello, tomamos

expansiones asintóticas de cada término. El procedimiento principal es el siguiente: para $\boldsymbol{\xi} \in \partial B_r(\mathbf{0})$, tenemos $\|\boldsymbol{\xi}\|_2 = r$, por lo que

$$p_t(\boldsymbol{\xi}) = \mathbb{E}[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})] \tag{B.2.66}$$

$$= \mathbb{E}\left[\underbrace{\frac{1}{(2\pi)^{D/2}t^D}}_{\doteq C_t} e^{-\|\boldsymbol{\xi}-\boldsymbol{x}\|_2^2/(2t^2)}\right] \tag{B.2.67}$$

$$= C_t\, \mathbb{E}\Big[e^{-(\|\boldsymbol{\xi}\|_2^2 - 2\langle\boldsymbol{\xi},\boldsymbol{x}\rangle + \|\boldsymbol{x}\|_2^2)/(2t^2)}\Big] \tag{B.2.68}$$

$$= C_t\, \mathbb{E}\Big[e^{-(r^2 - 2\langle\boldsymbol{\xi},\boldsymbol{x}\rangle + \|\boldsymbol{x}\|_2^2)/(2t^2)}\Big] \tag{B.2.69}$$

$$= C_t e^{-r^2/(2t^2)}\, \mathbb{E}[e^{(2\langle\boldsymbol{\xi},\boldsymbol{x}\rangle - \|\boldsymbol{x}\|_2^2)/(2t^2)}]. \tag{B.2.70}$$

Téngase en cuenta que, dado que $\|\boldsymbol{\xi}\|_2 = r$, por el teorema de Cauchy-Schwarz se tiene que

$$-2r\|\boldsymbol{x}\|_2 - \|\boldsymbol{x}\|_2^2 \leq 2\langle\boldsymbol{\xi},\boldsymbol{x}\rangle - \|\boldsymbol{x}\|_2^2 \leq 2r\|\boldsymbol{x}\|_2 - \|\boldsymbol{x}\|_2^2. \tag{B.2.71}$$

Recordemos que, según el Supuesto B.1, $\boldsymbol{x}$ tiene soporte en un conjunto compacto $\mathcal{S}$ de radio R. Por lo tanto

$$-2R(r+R) \leq 2\langle\boldsymbol{\xi},\boldsymbol{x}\rangle - \|\boldsymbol{x}\|_2^2 \leq 2Rr. \tag{B.2.72}$$

En otras palabras, se cumple que

$$C_t e^{-[r^2+2R(r+R)]/(2t^2)} \leq p_t(\boldsymbol{\xi}) \leq C_t e^{[-r^2+2Rr]/(2t^2)}. \tag{B.2.73}$$

Ahora, para calcular el gradiente, podemos utilizar la Proposición B.1 y la linealidad de la esperanza para calcular

$$\langle\nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle = \left\langle -\frac{1}{t^2}\,\mathbb{E}[(\boldsymbol{\xi}-\boldsymbol{x})\,\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})], \boldsymbol{\xi}\right\rangle \tag{B.2.74}$$

$$= -\frac{1}{t^2}\,\mathbb{E}[\langle\boldsymbol{\xi}-\boldsymbol{x},\boldsymbol{\xi}\rangle\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})] \tag{B.2.75}$$

$$= -\frac{1}{t^2}\,\mathbb{E}\big[\big(\|\boldsymbol{\xi}\|_2^2 - \langle\boldsymbol{\xi},\boldsymbol{x}\rangle\big)\,\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\big] \tag{B.2.76}$$

$$= -\frac{1}{t^2}\,\mathbb{E}\big[\big(r^2 - \langle\boldsymbol{\xi},\boldsymbol{x}\rangle\big)\,\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\big] \tag{B.2.77}$$

$$= \frac{1}{t^2}\,\mathbb{E}\big[\big(\langle\boldsymbol{\xi},\boldsymbol{x}\rangle - r^2\big)\,\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\big]. \tag{B.2.78}$$

Utilizando de nuevo la desigualdad de Cauchy-Schwarz y la representación $p_t(\boldsymbol{\xi}) \doteq \mathbb{E}[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})]$, se cumple que

$$\frac{1}{t^2}\,\mathbb{E}\big[\big(-Rr - r^2\big)\,\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\big] \leq \langle\nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle \leq \frac{1}{t^2}\,\mathbb{E}\big[\big(Rr - r^2\big)\,\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\big] \tag{B.2.79}$$

$$\frac{1}{t^2}\left(-Rr-r^2\right)\mathbb{E}[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})] \leq \langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle \leq \frac{1}{t^2}\left(Rr-r^2\right)\mathbb{E}[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})] \tag{B.2.80}$$

$$-\frac{r(R+r)}{t^2}p_t(\boldsymbol{\xi}) \leq \langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle \leq -\frac{r(r-R)}{t^2}p_t(\boldsymbol{\xi}). \tag{B.2.81}$$

Para $r > R > 0$ (como corresponde, ya que vamos a tomar el límite $r \to \infty$ mientras R es fijo), ambos lados son negativos. Esto tiene sentido: la mayor parte de la masa de probabilidad está contenida dentro de la bola de radio R y, por lo tanto, el *score* apunta hacia adentro, teniendo un producto interno negativo con el vector que apunta hacia afuera $\boldsymbol{\xi}$. Por lo tanto, utilizando las cotas apropiadas para $p_t(\boldsymbol{\xi})$,

$$-\frac{r(R+r)}{t^2}\cdot C_t e^{[-r^2+2Rr]/(2t^2)} \leq \langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle \leq -\frac{r(r-R)}{t^2}\cdot C_t e^{-[r^2+2R(r+R)]/(2t^2)}. \tag{B.2.82}$$

Entonces, teniendo en cuenta que $C_t = \mathrm{poly}(t^{-1})$, podemos calcular

$$[c+\log p_t(\boldsymbol{\xi})]\langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle = \mathrm{poly}(r, R, t^{-1}, c)e^{-\Theta_r(r^2)} \tag{B.2.83}$$

Así pues, se puede ver que, si se establece que el área de la superficie de $\partial B_r(\mathbf{0})$ es $\omega_D r^{D-1}$, donde ω_D es una función de D, se cumple que

$$\frac{1}{r}\int_{\partial B_r(\mathbf{0})}[c+\log p_t(\boldsymbol{\xi})]\langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle \mathrm{d}\boldsymbol{\xi} = \mathrm{poly}(r, R, t^{-1}, c)e^{-\Theta_r(r^2)} \tag{B.2.84}$$

y, por lo tanto, las colas que decaen exponencialmente significan

$$\lim_{r\to\infty}\frac{1}{r}\int_{\partial B_r(\mathbf{0})}[c+\log p_t(\boldsymbol{\xi})]\langle \nabla p_t(\boldsymbol{\xi}), \boldsymbol{\xi}\rangle \mathrm{d}\boldsymbol{\xi} = 0. \tag{B.2.85}$$

Por último, al sustituir en (B.2.63), obtenemos

$$\int_{\mathbb{R}^D}\left\{\Delta p_t(\boldsymbol{\xi})[c+\log p_t(\boldsymbol{\xi})] + \langle \nabla \log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle\right\}\mathrm{d}\boldsymbol{\xi} = 0 \tag{B.2.86}$$

$$\implies \int_{\mathbb{R}^D}\Delta p_t(\boldsymbol{\xi})[c+\log p_t(\boldsymbol{\xi})]\mathrm{d}\boldsymbol{\xi} = -\int_{\mathbb{R}^D}\langle \nabla \log p_t(\boldsymbol{\xi}), \nabla p_t(\boldsymbol{\xi})\rangle \mathrm{d}\boldsymbol{\xi} \tag{B.2.87}$$

tal como se afirma. □

Invertibilidad local del *denoiser* $\bar{\boldsymbol{x}}$

A continuación presentamos algunos resultados utilizados en la demostración del Teorema B.3, que constituyen generalizaciones adecuadas de los resultados correspondientes en [Gri11].

Lema B.3 (Generalización de [Gri11], Lema A.1)**.** *Sea $\boldsymbol{x}$ una variable aleatoria tal que los Supuestos B.1 y B.2 se cumplen, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1). Sea $s, t \in [0, T]$ tal que $0 \leq s < t \leq T$, y sea $\bar{\boldsymbol{x}}(\boldsymbol{\xi}) \doteq \mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$. El jacobiano $\bar{\boldsymbol{x}}'(\boldsymbol{\xi})$ es simétrico y definido positivo.*

Demostración. Tenemos

$$\bar{\boldsymbol{x}}'(\boldsymbol{\xi}) = \boldsymbol{I} + \left(1 - \frac{s}{t}\right) t^2 \nabla^2 \log p_t(\boldsymbol{\xi}). \tag{B.2.88}$$

Aquí expandimos

$$\nabla^2 \log p_t(\boldsymbol{\xi}) = \frac{p_t(\boldsymbol{\xi})\nabla^2 p_t(\boldsymbol{\xi}) - (\nabla p_t(\boldsymbol{\xi}))(\nabla p_t(\boldsymbol{\xi}))^\top}{p_t(\boldsymbol{\xi})^2}. \tag{B.2.89}$$

Por lo tanto, debemos asegurarnos de que

$$\bar{\boldsymbol{x}}'(\boldsymbol{\xi}) = \boldsymbol{I} + \left(1 - \frac{s}{t}\right) t^2 \frac{p_t(\boldsymbol{\xi})\nabla^2 p_t(\boldsymbol{\xi}) - (\nabla p_t(\boldsymbol{\xi}))(\nabla p_t(\boldsymbol{\xi}))^\top}{p_t(\boldsymbol{\xi})^2} \tag{B.2.90}$$

$$= \frac{p_t(\boldsymbol{\xi})^2 \boldsymbol{I} + \left(1 - \frac{s}{t}\right) t^2 \left[p_t(\boldsymbol{\xi})\nabla^2 p_t(\boldsymbol{\xi}) - (\nabla p_t(\boldsymbol{\xi}))(\nabla p_t(\boldsymbol{\xi}))^\top\right]}{p_t(\boldsymbol{\xi})^2} \tag{B.2.91}$$

es simétrica y semidefinida positiva. De hecho, es obviamente simétrica (por el teorema de Clairaut). Para demostrar su semidefinición positiva, sustituimos la representación de la esperanza de p_t dada por (B.2.3) (y ∇p_t, Δp_t por Proposición B.1) para obtener (donde $\boldsymbol{x}$ es como se definió y $\boldsymbol{y}$ es independiente e idénticamente distribuida como $\boldsymbol{x}$),

$$\boldsymbol{v}^\top [\bar{\boldsymbol{x}}'(\boldsymbol{\xi})] \boldsymbol{v} \tag{B.2.92}$$

$$= p_t(\boldsymbol{\xi})^{-2} \boldsymbol{v}^\top \left\{ p_t(\boldsymbol{\xi})^2 \boldsymbol{I} + \left(1 - \frac{s}{t}\right) t^2 \, \mathbb{E}[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})] \, \mathbb{E}\left[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x}) \cdot \frac{(\boldsymbol{\xi} - \boldsymbol{x})(\boldsymbol{\xi} - \boldsymbol{x})^\top - t^2 \boldsymbol{I}}{t^4}\right] \right. \tag{B.2.93}$$

$$\left. - \left(1 - \frac{s}{t}\right) t^2 \, \mathbb{E}\left[-\varphi_t(\boldsymbol{\xi} - \boldsymbol{x}) \cdot \frac{\boldsymbol{\xi} - \boldsymbol{x}}{t^2}\right] \mathbb{E}\left[-\varphi_t(\boldsymbol{\xi} - \boldsymbol{x}) \cdot \frac{\boldsymbol{\xi} - \boldsymbol{x}}{t^2}\right]^\top \right\} \boldsymbol{v} \tag{B.2.94}$$

$$= p_t(\boldsymbol{\xi})^{-2} \boldsymbol{v}^\top \left\{ \mathbb{E}[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})\varphi_t(\boldsymbol{\xi} - \boldsymbol{y})\boldsymbol{I}] \right. \tag{B.2.95}$$

$$+ \left(1 - \frac{s}{t}\right) t^2 \, \mathbb{E}\left[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})\varphi_t(\boldsymbol{\xi} - \boldsymbol{y}) \cdot \frac{(\boldsymbol{\xi} - \boldsymbol{y})(\boldsymbol{\xi} - \boldsymbol{y})^\top - t^2 \boldsymbol{I}}{t^4}\right] \tag{B.2.96}$$

$$\left. - \left(1 - \frac{s}{t}\right) t^2 \, \mathbb{E}\left[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})\varphi_t(\boldsymbol{\xi} - \boldsymbol{y}) \cdot \frac{(\boldsymbol{\xi} - \boldsymbol{x})(\boldsymbol{\xi} - \boldsymbol{y})^\top}{t^4}\right] \right\} \tag{B.2.97}$$

$$= \frac{1 - s/t}{p_t(\boldsymbol{\xi})^2} \boldsymbol{v}^\top \, \mathbb{E}\left[\varphi_t(\boldsymbol{\xi} - \boldsymbol{x})\varphi_t(\boldsymbol{\xi} - \boldsymbol{y}) \left\{ \frac{1}{1 - s/t} \boldsymbol{I} + \frac{(\boldsymbol{\xi} - \boldsymbol{y})(\boldsymbol{\xi} - \boldsymbol{y})^\top}{t^2} - \boldsymbol{I} - \frac{(\boldsymbol{\xi} - \boldsymbol{x})(\boldsymbol{\xi} - \boldsymbol{y})^\top}{t^2} \right\} \right] \boldsymbol{v} \tag{B.2.98}$$

$$= \frac{t - s}{t p_t(\boldsymbol{\xi})^2} \boldsymbol{v}^\top \, \mathbb{E}\left[\frac{s}{t - s} \boldsymbol{I} + \frac{(\boldsymbol{\xi} - \boldsymbol{x})(\boldsymbol{\xi} - \boldsymbol{x})^\top}{2t^2} + \frac{(\boldsymbol{\xi} - \boldsymbol{y})(\boldsymbol{\xi} - \boldsymbol{y})^\top}{2t^2} - \frac{(\boldsymbol{\xi} - \boldsymbol{x})(\boldsymbol{\xi} - \boldsymbol{y})^\top}{t^2}\right] \boldsymbol{v} \tag{B.2.99}$$

$$= \frac{t-s}{tp_t(\boldsymbol{\xi})^2} \boldsymbol{v}^\top \mathbb{E}\left[\frac{s}{t-s}\boldsymbol{I} + \frac{1}{2t^2}\left((\boldsymbol{\xi}-\boldsymbol{x})(\boldsymbol{\xi}-\boldsymbol{x})^\top + (\boldsymbol{\xi}-\boldsymbol{y})(\boldsymbol{\xi}-\boldsymbol{y})^\top - 2(\boldsymbol{\xi}-\boldsymbol{x})(\boldsymbol{\xi}-\boldsymbol{y})^\top\right)\right]\boldsymbol{v} \tag{B.2.100}$$

$$= \frac{t-s}{tp_t(\boldsymbol{\xi})^2} \mathbb{E}\left[\frac{s}{t-s}\|\boldsymbol{v}\|_2^2 + \frac{1}{2t^2}\left([(\boldsymbol{\xi}-\boldsymbol{x})^\top\boldsymbol{v}]^2 + [(\boldsymbol{\xi}-\boldsymbol{y})^\top\boldsymbol{v}]^2 - 2[(\boldsymbol{\xi}-\boldsymbol{x})^\top\boldsymbol{v}][(\boldsymbol{\xi}-\boldsymbol{y})^\top\boldsymbol{v}]\right)\right] \tag{B.2.101}$$

$$= \frac{t-s}{tp_t(\boldsymbol{\xi})^2} \mathbb{E}\left[\frac{s}{t-s}\|\boldsymbol{v}\|_2^2 + \frac{1}{2t^2}\left([(\boldsymbol{\xi}-\boldsymbol{x})^\top\boldsymbol{v}]^2 + [(\boldsymbol{\xi}-\boldsymbol{y})^\top\boldsymbol{v}]^2 - 2[(\boldsymbol{\xi}-\boldsymbol{x})^\top\boldsymbol{v}][(\boldsymbol{\xi}-\boldsymbol{y})^\top\boldsymbol{v}]\right)\right] \tag{B.2.102}$$

$$= \frac{t-s}{tp_t(\boldsymbol{\xi})^2} \mathbb{E}\left[\frac{s}{t-s}\|\boldsymbol{v}\|_2^2 + \frac{1}{2t^2}\left([(\boldsymbol{\xi}-\boldsymbol{x})^\top\boldsymbol{v}] - [(\boldsymbol{\xi}-\boldsymbol{y})^\top\boldsymbol{v}]\right)^2\right] \tag{B.2.103}$$

$$= \frac{t-s}{tp_t(\boldsymbol{\xi})^2} \mathbb{E}\left[\frac{s}{t-s}\|\boldsymbol{v}\|_2^2 + \frac{1}{2t^2}[(\boldsymbol{y}-\boldsymbol{x})^\top\boldsymbol{v}]^2\right] \tag{B.2.104}$$

$$= \frac{s}{tp_t(\boldsymbol{\xi})^2}\|\boldsymbol{v}\|_2^2 + \frac{t-s}{2t^3 p_t(\boldsymbol{\xi})} \mathbb{E}[\{(\boldsymbol{y}-\boldsymbol{x})^\top\boldsymbol{v}\}^2] \tag{B.2.105}$$

Dado que $\boldsymbol{x}$ y $\boldsymbol{y}$ son variables independientes e idénticamente distribuidas, la integral completa (es decir, la forma cuadrática original) es 0 si y solo si $s = 0$ y $\boldsymbol{x}$ tiene un soporte totalmente contenido en un subespacio afín que es ortogonal a $\boldsymbol{v}$. Pero esto queda descartado por el supuesto (es decir, que $\boldsymbol{x}$ posee densidad en $\mathbb{R}^D$), por lo que el jacobiano $\bar{\boldsymbol{x}}'(\boldsymbol{\xi})$ es simétrico y definido positivo. □

Lema B.4 (Generalización de [Gri11] Corolario A.2, Parte 1)**.** *Sea $f\colon \mathbb{R}^D \to \mathbb{R}^D$ una función diferenciable cualquiera cuyo jacobiano $f'(\boldsymbol{x})$ sea simétrico y definido positivo. Entonces f es inyectiva y, por lo tanto, invertible como función $\mathbb{R}^D \to \mathcal{R}(f)$, donde $\mathcal{R}(f)$ es el rango de f.*

Demostración. Supongamos que f no es inyectiva, es decir, que existen $\boldsymbol{x}, \boldsymbol{x}'$ tales que $f(\boldsymbol{x}) = f(\boldsymbol{x}')$ y $\boldsymbol{x} \neq \boldsymbol{x}'$. Definamos $\boldsymbol{v} \doteq (\boldsymbol{x}' - \boldsymbol{x})/\|\boldsymbol{x}' - \boldsymbol{x}\|_2$. Definamos la función $g\colon \mathbb{R} \to \mathbb{R}$ como $g(t) \doteq \boldsymbol{v}^\top f(\boldsymbol{x} + t\boldsymbol{v})$. Entonces $g(0) = \boldsymbol{v}^\top f(\boldsymbol{x}) = \boldsymbol{v}^\top f(\boldsymbol{x}') = g(\|\boldsymbol{x}' - \boldsymbol{x}\|_2)$. Dado que f es diferenciable, g es diferenciable, por lo que la derivada g' debe ser nula para algún $t^* \in (0, \|\boldsymbol{x}' - \boldsymbol{x}\|_2)$ por el teorema del valor medio. Sin embargo,

$$g'(t^*) \doteq \boldsymbol{v}^\top \left[f'(\boldsymbol{x} + t^*\boldsymbol{v})\right]\boldsymbol{v} > 0 \tag{B.2.106}$$

ya que el jacobiano es definido positivo. Por lo tanto, llegamos a una contradicción, tal como se ha afirmado. □

A partir de la combinación de los dos resultados anteriores se obtiene el siguiente resultado fundamental.

Corolario B.1 (Generalización de [Gri11] Corolario A.2, Parte 2)**.** *Sea $\boldsymbol{x}$ una variable aleatoria tal que los Supuestos B.1 y B.2 se cumplen, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1). Sea $s, t \in [0, T]$ tales que $0 \leq s < t \leq T$, y sea $\bar{\boldsymbol{x}}(\boldsymbol{\xi}) \doteq \mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t = \boldsymbol{\xi}]$. Entonces $\bar{\boldsymbol{x}}$ es inyectiva y, por lo tanto, invertible en su imagen.*

Demostración. Lo único que queda por demostrar es que $\bar{\boldsymbol{x}}$ es diferenciable, pero esto se deduce inmediatamente de la fórmula de Tweedie (Teorema 3.2), que muestra que $\bar{\boldsymbol{x}}$ es diferenciable si y solo si $\nabla \log p_t$ es diferenciable, y esto viene dado por la Ecuación (B.2.3). □

Control del laplaciano $\Delta \log p_t$

Finalmente, desarrollamos una estimación técnica requerida para probar el Teorema B.3 lo cual motiva directamente el supuesto sobre los valores admisibles de t.

Lema B.5. *Sea $\boldsymbol{x}$ una variable aleatoria tal que los Supuestos B.1 y B.2 se cumplan, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1). Sea p_t la densidad de $\boldsymbol{x}_t$. Entonces, para $t > 0$ se cumple*

$$\sup_{\boldsymbol{\xi}\in\mathbb{R}^D} |\Delta \log p_t(\boldsymbol{\xi})| \leq \max\left(\frac{D}{t^2}, \left|\frac{R^2}{t^4} - \frac{D}{t^2}\right|\right). \tag{B.2.107}$$

Demostración. Mediante la regla de la cadena, un ejercicio sencillo permite calcular

$$\Delta \log p_t(\boldsymbol{\xi}) = \frac{\Delta p_t(\boldsymbol{\xi})}{p_t(\boldsymbol{\xi})} - \frac{\|\nabla p_t(\boldsymbol{\xi})\|_2^2}{p_t(\boldsymbol{\xi})^2}. \tag{B.2.108}$$

Utilizando la Proposición B.1 para expresar los términos de $\Delta p_t(\boldsymbol{\xi})$, obtenemos

$$\frac{\Delta p_t(\boldsymbol{\xi})}{p_t(\boldsymbol{\xi})} = \frac{\mathbb{E}\left[\frac{\|\boldsymbol{\xi}-\boldsymbol{x}\|_2^2 - Dt^2}{t^4} \cdot \varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\right]}{\mathbb{E}[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})]} \tag{B.2.109}$$

$$= \frac{\int_{\mathbb{R}^D} \left\{\frac{\|\boldsymbol{\xi}-\boldsymbol{u}\|_2^2 - Dt^2}{t^4}\right\} \varphi_t(\boldsymbol{\xi}-\boldsymbol{u}) p(\boldsymbol{u}) \mathrm{d}\boldsymbol{u}}{\int_{\mathbb{R}^D} \varphi_t(\boldsymbol{\xi}-\boldsymbol{u}) p(\boldsymbol{u}) \mathrm{d}\boldsymbol{u}}. \tag{B.2.110}$$

Esto parece una marginalización bayesiana, así que definamos la densidad normalizada adecuada

$$q_{\boldsymbol{\xi}}(\boldsymbol{u}) = \frac{\varphi_t(\boldsymbol{\xi}-\boldsymbol{u})p(\boldsymbol{u})}{\int_{\mathbb{R}^D} \varphi_t(\boldsymbol{\xi}-\boldsymbol{v})p(\boldsymbol{v})\mathrm{d}\boldsymbol{v}} = \frac{\varphi_t(\boldsymbol{\xi}-\boldsymbol{u})p(\boldsymbol{u})}{\mathbb{E}[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})]} = \frac{\varphi_t(\boldsymbol{\xi}-\boldsymbol{u})p(\boldsymbol{u})}{p_t(\boldsymbol{\xi})} \tag{B.2.111}$$

Entonces, definiendo $\boldsymbol{y}_{\boldsymbol{\xi}} \sim q_{\boldsymbol{\xi}}$, podemos escribir

$$\frac{\Delta p_t(\boldsymbol{\xi})}{p_t(\boldsymbol{\xi})} = \int_{\mathbb{R}^D} \left\{\frac{\|\boldsymbol{\xi}-\boldsymbol{u}\|_2^2 - Dt^2}{t^4}\right\} q_{\boldsymbol{\xi}}(\boldsymbol{u})\mathrm{d}\boldsymbol{u} = \frac{1}{t^4}\,\mathbb{E}[\|\boldsymbol{\xi}-\boldsymbol{y}_{\boldsymbol{\xi}}\|_2^2] - \frac{D}{t^2}. \tag{B.2.112}$$

Del mismo modo, al desarrollar el segundo término (sin elevar al cuadrado) obtenemos

$$\frac{\nabla p_t(\boldsymbol{\xi})}{p_t(\boldsymbol{\xi})} = -\frac{\boldsymbol{\xi} - \mathbb{E}[\boldsymbol{y}_{\boldsymbol{\xi}}]}{t^2}. \tag{B.2.113}$$

Si se establece que $\boldsymbol{z}_{\boldsymbol{\xi}} \doteq \boldsymbol{y}_{\boldsymbol{\xi}} - \boldsymbol{\xi}$, se cumple que

$$\frac{\Delta p_t(\boldsymbol{\xi})}{p_t(\boldsymbol{\xi})} = \frac{\mathbb{E}[\|\boldsymbol{z}_{\boldsymbol{\xi}}\|_2^2]}{t^4} - \frac{D}{t^2}, \qquad \frac{\nabla p_t(\boldsymbol{\xi})}{p_t(\boldsymbol{\xi})} = \frac{\mathbb{E}[\boldsymbol{z}_{\boldsymbol{\xi}}]}{t^2}. \tag{B.2.114}$$

Por lo tanto, al desarrollar completamente $\Delta \log p_t$, tenemos

$$\Delta \log p_t(\boldsymbol{\xi}) = \frac{\mathbb{E}[\|\boldsymbol{z}_{\boldsymbol{\xi}}\|_2^2]}{t^4} - \frac{D}{t^2} - \frac{\|\,\mathbb{E}[\boldsymbol{z}_{\boldsymbol{\xi}}]\|_2^2}{t^4} \tag{B.2.115}$$

$$= \frac{\mathbb{E}[\|\boldsymbol{z}_{\boldsymbol{\xi}}\|_2^2] - \|\,\mathbb{E}[\boldsymbol{z}_{\boldsymbol{\xi}}]\|_2^2}{t^4} - \frac{D}{t^2} \tag{B.2.116}$$

$$= \frac{\operatorname{tr}(\operatorname{Cov}(\boldsymbol{z}_{\boldsymbol{\xi}}))}{t^4} - \frac{D}{t^2} \tag{B.2.117}$$

$$= \frac{\operatorname{tr}(\operatorname{Cov}(\boldsymbol{y}_{\boldsymbol{\xi}}))}{t^4} - \frac{D}{t^2}. \tag{B.2.118}$$

Una cota inferior trivial de esta traza es 0, ya que las matrices de covarianza son semidefinidas positivas. Para hallar una cota superior, obsérvese que $\boldsymbol{y}_{\boldsymbol{\xi}}$ toma valores únicamente en el soporte de $\boldsymbol{x}$ (puesto que p es un factor de la densidad $q_{\boldsymbol{\xi}}$ de $\boldsymbol{y}_{\boldsymbol{\xi}}$), el cual, por el Supuesto B.1, es un conjunto compacto $\mathcal{S}$ con radio $R \doteq \sup_{\boldsymbol{\xi}\in\mathcal{S}} \|\boldsymbol{\xi}\|_2$. Por lo tanto

$$\operatorname{tr}(\operatorname{Cov}(\boldsymbol{y}_{\boldsymbol{\xi}})) = \mathbb{E}[\|\boldsymbol{y}_{\boldsymbol{\xi}}\|_2^2] - \|\,\mathbb{E}[\boldsymbol{y}_{\boldsymbol{\xi}}]\|_2^2 \leq \mathbb{E}[\|\boldsymbol{y}_{\boldsymbol{\xi}}\|_2^2] \leq R^2. \tag{B.2.119}$$

Por lo tanto

$$-\frac{D}{t^2} \leq \Delta \log p_t(\boldsymbol{\xi}) \leq \frac{R^2}{t^4} - \frac{D}{t^2}, \tag{B.2.120}$$

que muestra lo que se sostiene. □

Cálculos de derivadas

A continuación presentamos algunas derivaciones útiles que serán reutilizadas en todo el apéndice.

Proposición B.1. *Sea $\boldsymbol{x}$ una variable aleatoria cualquiera tal que los Supuestos B.1 y B.2 se cumplan, y sea $(\boldsymbol{x}_t)_{t\in[0,T]}$ el proceso estocástico (B.2.1). Para $t \geq 0$, sea p_t la densidad de $\boldsymbol{x}_t$. Entonces*

$$\frac{\partial p_t}{\partial t}(\boldsymbol{\xi}) = \mathbb{E}\left[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x}) \cdot \frac{\|\boldsymbol{\xi}-\boldsymbol{x}\|_2^2 - Dt^2}{t^3}\right] \tag{B.2.121}$$

$$\nabla p_t(\boldsymbol{\xi}) = -\,\mathbb{E}\left[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x}) \cdot \frac{\boldsymbol{\xi}-\boldsymbol{x}}{t^2}\right] \tag{B.2.122}$$

$$\nabla^2 p_t(\boldsymbol{\xi}) = \mathbb{E}\left[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x}) \cdot \frac{(\boldsymbol{\xi}-\boldsymbol{x})(\boldsymbol{\xi}-\boldsymbol{x})^\top - t^2\boldsymbol{I}}{t^4}\right] \tag{B.2.123}$$

$$\Delta p_t(\boldsymbol{\xi}) = \mathbb{E}\left[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x}) \cdot \frac{\|\boldsymbol{\xi}-\boldsymbol{x}\|_2^2 - Dt^2}{t^4}\right]. \tag{B.2.124}$$

Demostración. Utilizamos la representación por convolución de p_t, es decir, (B.2.3). Tomando primero la derivada temporal, un cálculo muestra que se

aplica la Proposición B.3,[5] así que podemos llevar la derivada al interior de la integral/esperanza como:

$$\frac{\partial p_t}{\partial t}(\boldsymbol{\xi}) = \frac{\partial}{\partial t}\,\mathbb{E}[\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})] = \mathbb{E}\left[\frac{\partial}{\partial t}\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})\right] = \frac{\partial \varphi_t}{\partial t} * p. \tag{B.2.125}$$

Por otra parte, gracias a las propiedades de las convoluciones (Proposición B.4) y utilizando el hecho de que p tiene soporte compacto (Supuesto B.1),

$$p_t = \varphi_t * p \implies \nabla p_t = \nabla\varphi_t * p \implies \nabla^2 p_t = \nabla^2\varphi_t * p \implies \Delta p_t = \Delta\varphi_t * p. \tag{B.2.126}$$

El resto del cálculo se deduce de la Proposición B.2. □

Proposición B.2. *Para $t > 0$ y $\boldsymbol{\xi} \in \mathbb{R}^D$, se cumple que*

$$\frac{\partial}{\partial t}\varphi_t(\boldsymbol{\xi}) = \varphi_t(\boldsymbol{\xi}) \cdot \frac{\|\boldsymbol{\xi}\|_2^2 - Dt^2}{t^3} \tag{B.2.127}$$

$$\nabla\varphi_t(\boldsymbol{\xi}) = -\varphi_t(\boldsymbol{\xi}) \cdot \frac{\boldsymbol{\xi}}{t^2} \tag{B.2.128}$$

$$\nabla^2\varphi_t(\boldsymbol{\xi}) = \varphi_t(\boldsymbol{\xi}) \cdot \frac{\boldsymbol{\xi}\boldsymbol{\xi}^\top - t^2\boldsymbol{I}}{t^4} \tag{B.2.129}$$

$$\Delta\varphi_t(\boldsymbol{\xi}) = \varphi_t(\boldsymbol{\xi}) \cdot \frac{\|\boldsymbol{\xi}\|_2^2 - Dt^2}{t^4}. \tag{B.2.130}$$

Demostración. Cálculo directo. □

Derivación bajo el signo integral

En este apéndice, derivamos muchas veces bajo el signo de la integral, y resulta esencial conocer las condiciones bajo las cuales es posible proceder. Hay dos tipos de derivación bajo el signo de la integral:

1. Derivación de la integral $\int f_t(\boldsymbol{\xi})\mathrm{d}\boldsymbol{\xi}$ con respecto al parámetro auxiliar t.
2. Derivación de una convolución $(f * g)(\boldsymbol{\xi}) = \int f(\boldsymbol{\xi})g(\boldsymbol{\xi}-\boldsymbol{u})\mathrm{d}\boldsymbol{u}$ con respecto a la variable $\boldsymbol{\xi}$.

Para la primera categoría, presentamos un resultado explícito, enunciado sin prueba pero atribuible a la referencia vinculada, que deduce el siguiente resultado como un caso especial de un teorema más general sobre la interacción de operadores diferenciales y distribuciones templadas, lo cual excede con creces el alcance del libro. Se puede encontrar una referencia formal completa en [Jon82].

[5] Utilizamos $f_t(\boldsymbol{\xi}) = p(\boldsymbol{\xi})\varphi_t(\boldsymbol{\xi}-\boldsymbol{x})$, observando que es dos veces continuamente diferenciable en $\boldsymbol{\xi}$ y (más que dos veces en t. A continuación, para verificar la integrabilidad local de f_t, calculamos $\frac{\partial f_t}{\partial t}(\boldsymbol{\xi}) = f_t(\boldsymbol{\xi}) \cdot \frac{1}{t^3}(\|\boldsymbol{\xi}-\boldsymbol{x}\|_2^2 - Dt^2)$, lo cual es fácil de comprobar que es integrable sobre $\boldsymbol{\xi}$ y $t \in [t_{\min}, t_{\max}]$ donde $t_{\min} > 0$. (De hecho, f_t tiene colas que decaen exponencialmente, por lo que el término cuadrático en el producto no supone ningún problema.)

Proposición B.3 ([Jon82], Sección 11.12)**.** *Sea* $f\colon (0,T)\times\mathbb{R}^D\to\mathbb{R}$ *tal que:*

- f *es una función conjuntamente medible de* $(t,\boldsymbol{\xi})$*;*
- *Para casi todo* $\boldsymbol{\xi}\in\mathbb{R}^D$ *en el sentido de Lebesgue, la función* $t\mapsto f_t(\boldsymbol{\xi})$ *es absolutamente continua;*
- $\frac{\partial f_t}{\partial t}$ *es localmente integrable, es decir, para cada* $[t_{\min},t_{\max}]\subseteq(0,T)$ *se cumple que*

$$\int_{t_{\min}}^{t_{\max}}\int_{\mathbb{R}^D}\left|\frac{\partial f_t}{\partial t}(\boldsymbol{\xi})\right|\mathrm{d}\boldsymbol{\xi}<\infty. \tag{B.2.131}$$

Entonces, $t\mapsto\int_{\mathbb{R}^D}f_t(\boldsymbol{\xi})\mathrm{d}\boldsymbol{\xi}$ *es una función absolutamente continua en* $(0,T)$*, y su derivada es*

$$\frac{\mathrm{d}}{\mathrm{d}t}\int_{\mathbb{R}^D}f_t(\boldsymbol{\xi})\mathrm{d}\boldsymbol{\xi}=\int_{\mathbb{R}^D}\frac{\partial}{\partial t}f_t(\boldsymbol{\xi})\mathrm{d}\boldsymbol{\xi}, \tag{B.2.132}$$

definida para casi todo $t\in(0,T)$*.*

Para la segunda categoría, ofrecemos otro resultado concreto, que se expone sin demostración, pero que está completamente formalizado en [BB11].

Proposición B.4 ([BB11], Proposición 4.20)**.** *Sea* f *una función* k*-veces continuamente diferenciable con soporte compacto, y sea* g *una función localmente integrable. Entonces, la convolución* $f*g$ *definida por*

$$(f*g)(\boldsymbol{\xi})\doteq\int_{\mathbb{R}^D}f(\boldsymbol{u})g(\boldsymbol{\xi}-\boldsymbol{u})\mathrm{d}\boldsymbol{u} \tag{B.2.133}$$

es k*-veces continuamente diferenciable, y su derivada de orden* k *es*

$$\nabla^k(f*g)=(\nabla^k f)*g. \tag{B.2.134}$$

Aunque no aparece en el libro, un sencillo razonamiento basado en la integración por partes demuestra que, si g también es k-veces diferenciable, entonces podemos «intercambiar» la regularidad:

$$\nabla^k(f*g)=f*(\nabla^k g). \tag{B.2.135}$$

B.3 Codificación con pérdida y empaquetamiento esférico

En esta sección, demostramos el Teorema 4.1. Siguiendo las convenciones utilizadas a lo largo de este apéndice, escribimos $\mathcal{S}=\mathrm{Supp}(\boldsymbol{x})$ para el soporte compacto de la variable aleatoria $\boldsymbol{x}$.

Tal como se ha adelantado, haremos un supuesto de regularidad sobre el conjunto de soporte $\mathcal{S}$ para demostrar el Teorema 4.1. Una posibilidad para proceder bajo supuestos mínimos sería aplicar los resultados de [RBK18; RKB23] en nuestro contexto, ya que estos resultados se aplican a conjuntos $\mathcal{S}$

con muy baja regularidad (por ejemplo, conjuntos tipo Cantor con estructura fractal). Sin embargo, hemos encontrado que el cálculo preciso de estas constantes —una tarea necesaria para demostrar una conclusión como el Teorema 4.1— resulta bastante tedioso en nuestro contexto. Por lo tanto, nuestro enfoque consiste en añadir un supuesto de regularidad geométrica sobre el conjunto $\mathcal{S}$ que cede en términos de generalidad, pero nos permite desarrollar un argumento más transparente. Para evitar sacrificar demasiada generalidad, debemos asegurarnos de que no se prohíba la baja dimensionalidad en el conjunto $\mathcal{S}$. Por lo tanto, consideramos el ejemplo que hemos utilizado a lo largo del libro: la mezcla de distribuciones gaussianas de rango bajo. En este marco geométrico, modelamos $\mathcal{S}$ como una unión de hiperesferas, cada una de dimensión d_k (posiblemente mucho menor que D), situadas en subespacios mutuamente ortogonales de $\mathbb{R}^D$. Esta es una simplificación geométrica del modelo de mezcla gaussiana: el soporte de cada componente se aproxima mediante una esfera en el subespacio generado por sus direcciones principales. Cuando las dimensiones de las componentes d_k son grandes, este supuesto equivale a un supuesto de mezcla de gaussianas de bajo rango, debido a la concentración de medidas en espacios de alta dimensión. Los modelos CRATE que desarrollamos y entrenamos en los Capítulos 5 y 8 tienen sus *dimensiones de subespacio* establecidas de manera compatible con este supuesto.

Supuesto B.3. El soporte $\mathcal{S} \subset \mathbb{R}^D$ de la variable aleatoria $\boldsymbol{x}$ es una unión finita de K esferas, cada una de dimensión d_k, donde $k \in [K]$. La probabilidad de que $\boldsymbol{x}$ se extraiga de la k-ésima esfera viene dada por $\pi_k \in [0, 1]$, y, condicionalmente a que se extraiga de la k-ésima esfera, $\boldsymbol{x}$ se distribuye uniformemente en esa esfera. Los soportes satisfacen que cada esfera es mutuamente ortogonal con todas las demás.

Procedemos bajo el Supuesto B.3 simplificadora con el fin de evitar un exceso de tecnicismo y establecer una conexión con un importante ejemplo que se utiliza a lo largo de toda la monografía. Creemos que nuestros resultados pueden generalizarse para abarcar $\mathcal{S}$ de la clase de *conjuntos con alcance positivo* con un esfuerzo técnico adicional, pero dejamos esto para el futuro.

B.3.1 Demostración de la relación entre la distorsión de la tasa y la cobertura

Esbozamos brevemente la demostración, luego pasamos a establecer tres lemas fundamentales y, a continuación, presentamos la demostración. La demostración se basará en los conceptos introducidos en el esbozo que figura a continuación.

Obtener un límite superior para la función de tasa-distorsión (4.1.9) es sencillo: según la caracterización de la tasa (es decir, la función de tasa-distorsión es la tasa mínima de un código para $\boldsymbol{x}$ con una distorsión cuadrática esperada ℓ^2 de ϵ), para establecer un límite superior de $\mathcal{R}_\epsilon(\boldsymbol{x})$ solo es necesario demostrar que existe un código para x que alcance esta distorsión objetivo, y cualquier cobertura ϵ de $\text{Supp}(\boldsymbol{x})$ lo logra, con una tasa igual al logaritmo en base 2 de la cardinalidad de la cobertura. El límite inferior es

más sutil. Utilizamos el límite inferior de Shannon, discutido en la Nota 4.3: al calcular las constantes en [LZ94, §III, (22)] se obtiene una versión más precisa del resultado citado en la Ecuación (4.1.17) (en bits, por supuesto): para cualquier variable aleatoria $\boldsymbol{x}$ con soporte compacto y una densidad, se cumple que

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq h(\boldsymbol{x}) - \log \operatorname{vol}(B_\epsilon) + \log\left(\frac{2}{D\Gamma(D/2)}\left(\frac{D}{2e}\right)^{D/2}\right), \tag{B.3.1}$$

con la entropía (etc.) en nats en esta expresión. La constante se puede estimar fácilmente utilizando la aproximación de Stirling. Una forma cuantitativa de la aproximación de Stirling que suele resultar útil da para cualquier $x > 0$ [Jam15]

$$\Gamma(x) \leq \sqrt{2\pi} x^{x-1/2} e^{-x} e^{1/(12x)}. \tag{B.3.2}$$

Aplicaremos esta cota a $\Gamma(D/2)$ en la Ecuación (B.3.1). Se obtiene

$$\log\left(\frac{2}{D\Gamma(D/2)}\left(\frac{D}{2e}\right)^{D/2}\right) \geq -\frac{1}{6D} + \log\left(\frac{2}{D}\left(\frac{D}{2e}\right)^{D/2} \cdot \sqrt{\frac{D}{4\pi}}\left(\frac{D}{2e}\right)^{-D/2}\right) \tag{B.3.3}$$

$$= -\frac{1}{6D} - \frac{1}{2}\log D\pi, \tag{B.3.4}$$

que podemos tomar como el valor explícito de la constante C_D en la Ecuación (4.1.17). En resumen, el límite inferior de Shannon totalmente cuantificado (en bits):

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq h(\boldsymbol{x}) - \log_2 \operatorname{vol}(B_\epsilon) - O(\log D). \tag{B.3.5}$$

Ahora bien, la restricción importante para nuestros propósitos actuales es que el límite inferior de Shannon exige que la variable aleatoria $\boldsymbol{x}$ tenga una densidad, lo que descarta muchas distribuciones de interés de baja dimensión. Pero consideremos por un momento la situación en la que $\boldsymbol{x}$ sí admite una densidad. El supuesto de que $\boldsymbol{x}$ sigue una distribución uniforme en su soporte se formaliza fácilmente en este contexto: para cualquier conjunto de Borel $A \subset \mathcal{S}$, tenemos

$$\mathbb{P}[\boldsymbol{x} \in A] = \int_A \frac{1}{\operatorname{vol}(\mathcal{S})} \mathrm{d}\boldsymbol{x}. \tag{B.3.6}$$

Entonces, la entropía $h(\boldsymbol{x})$ es simplemente

$$h(\boldsymbol{x}) = \log_2 \operatorname{vol}(\mathcal{S}). \tag{B.3.7}$$

La demostración concluye entonces con un lema que relaciona la razón $\operatorname{vol}(\mathcal{S})/\operatorname{vol}(B_\epsilon)$ con el número de recubrimiento ϵ de $\mathcal{S}$ mediante bolas de tamaño ϵ.

Para ampliar el programa anterior a distribuciones degeneradas que satisfagan el Supuesto B.3, nuestra demostración del límite inferior en el Teorema 4.1 aprovechará un argumento de aproximación de la distribución real

de baja dimensión $\boldsymbol{x}$ mediante distribuciones «cercanas» que tienen densidades, de manera similar pero no exactamente igual a la esbozada en la demostración que precede al Teorema B.1. A continuación, relacionaremos el parámetro introducido en la secuencia de aproximación con el parámetro de distorsión ϵ con el fin de obtener la conclusión deseada en el Teorema 4.1.

Definición B.1. Sea $\mathcal{S}$ un conjunto compacto. Para cualquier $\delta > 0$, definimos el entorno tubular de $\mathcal{S}$ de espesor δ, denotado $\mathcal{S}_\delta$, como

$$\mathcal{S}_\delta = \left\{ \boldsymbol{\xi} \in \mathbb{R}^D \;\middle|\; \operatorname{dist}(\boldsymbol{\xi}, \mathcal{S}) \leq \delta \right\}. \tag{B.3.8}$$

La función de distancia a la que se hace referencia en la Definición B.1 se define mediante

$$\operatorname{dist}(\boldsymbol{\xi}, \mathcal{S}) = \inf_{\boldsymbol{\xi}' \in \mathcal{S}} \|\boldsymbol{\xi} - \boldsymbol{\xi}'\|_2 . \tag{B.3.9}$$

Para un conjunto compacto $\mathcal{S}$, el teorema de Weierstrass implica que, para cualquier $\boldsymbol{\xi} \in \mathbb{R}^D$, siempre existe algún $\boldsymbol{\xi}' \in \mathcal{S}$ que alcanza el infimo en la función de distancia. La compacidad de $\mathcal{S}_\delta$ se deduce fácilmente de la compacidad de $\mathcal{S}$, por lo que $\operatorname{vol}(\mathcal{S}_\delta)$ es finito para cualquier $\delta > 0$. Entonces es posible establecer la siguiente definición de una variable aleatoria engrosada, especializada al Supuesto B.3.

Definición B.2. Sea $\boldsymbol{x}$ una variable aleatoria tal que $\operatorname{Supp}(\boldsymbol{x}) = \mathcal{S}$ es una unión de K hiperesferas, distribuidas como en el Supuesto B.3. Denotemos por $\mathcal{S}_k$ el soporte de cada componente de la mezcla. Definamos la variable aleatoria engrosada $\boldsymbol{x}_\delta$ como la mezcla de medidas en la que cada medida componente es uniforme en el conjunto engrosado $\mathcal{S}_{k,\delta}$ (Definición B.1), para $k \in [K]$, con pesos de mezcla π_k.

Lema B.6. *Supongamos que la variable aleatoria $\boldsymbol{x}$ satisface el Supuesto B.3. Entonces, si $0 < \delta < \frac{1}{2}$, la variable aleatoria engrosada $\boldsymbol{x}_\delta$ (Definición B.2) satisface, para cualquier $\epsilon > 0$*

$$R_{\delta+\epsilon}(\boldsymbol{x}_\delta) \leq R_\epsilon(\boldsymbol{x}). \tag{B.3.10}$$

La demostración del Lema B.6 se pospone hasta la Apéndice B.3.2. Utilizando el Lema B.6, el programa anterior puede llevarse a cabo, ya que la variable aleatoria $\boldsymbol{x}_\delta$ tiene una densidad uniforme con respecto a la medida de Lebesgue.

(Demostración del Teorema 4.1). La cota superior se demuestra fácilmente. Si S es cualquier recubrimiento ϵ del soporte de $\boldsymbol{x}$ con cardinalidad $\mathcal{N}_\epsilon(\operatorname{Supp}(\boldsymbol{x}))$, entonces consideremos el esquema de codificación que asigna a cada $\boldsymbol{\xi} \in \operatorname{Supp}(\boldsymbol{x})$ la reconstrucción $\hat{\boldsymbol{\xi}} = \arg\min_{\boldsymbol{\xi}' \in S} \|\boldsymbol{\xi} - \boldsymbol{\xi}'\|_2$, con empates resueltos arbitrariamente. Entonces, los empates ocurren con probabilidad cero, y el hecho de que S cubra $\operatorname{Supp}(\boldsymbol{x})$ a escala ϵ garantiza una distorsión no mayor que ϵ; la tasa de este esquema es $\log_2 \mathcal{N}_\epsilon(\operatorname{Supp}(\boldsymbol{x}))$.

Para la cota inferior, sea $0 < \delta < \frac{1}{2}$, y consideremos la variable aleatoria ensanchada $\boldsymbol{x}_\delta$. Según el Lema B.6, tenemos

$$R_{\delta+\epsilon}(\boldsymbol{x}_\delta) \leq R_\epsilon(\boldsymbol{x}). \tag{B.3.11}$$

Dado que $\boldsymbol{x}_\delta$ tiene una densidad de Lebesgue uniforme, podemos entonces aplicar el límite inferior de Shannon, en la forma (B.3.5), para obtener

$$\log_2 \operatorname{vol}(\operatorname{Supp}(\boldsymbol{x}_\delta)) - \log_2 \operatorname{vol}(B_{\delta+\epsilon}) - O(\log D) \leq R_\epsilon(\boldsymbol{x}). \tag{B.3.12}$$

Por último, debemos hallar un límite inferior para la razón

$$\frac{\operatorname{vol}(\operatorname{Supp}(\boldsymbol{x}_\delta))}{\operatorname{vol}(B_{\delta+\epsilon})} \tag{B.3.13}$$

en términos del número de cobertura. Dado que $\operatorname{Supp}(\boldsymbol{x}_\delta) = \operatorname{Supp}(\boldsymbol{x}) + B_\delta$, donde $+$ denota aquí la suma de Minkowski, una aplicación estándar de los argumentos de límites de volumen (véase, por ejemplo, [Ver18, Proposition 4.2.12]) da

$$\operatorname{vol}(\operatorname{Supp}(\boldsymbol{x}_\delta)) \geq \mathcal{N}_{2\delta}(\operatorname{Supp}(\boldsymbol{x})) \operatorname{vol}(B_\delta). \tag{B.3.14}$$

Por lo tanto

$$\frac{\operatorname{vol}(\operatorname{Supp}(\boldsymbol{x}_\delta))}{\operatorname{vol}(B_{\delta+\epsilon})} \geq \mathcal{N}_{2\delta}(\operatorname{Supp}(\boldsymbol{x})) \frac{\operatorname{vol}(B_\delta)}{\operatorname{vol}(B_{\delta+\epsilon})} \tag{B.3.15}$$

$$= \mathcal{N}_{2\delta}(\operatorname{Supp}(\boldsymbol{x})) \left(\frac{\delta}{\delta+\epsilon}\right)^D. \tag{B.3.16}$$

Si se elige $\delta = \epsilon/2$, se obtiene a partir del límite inferior de Shannon (B.3.12) y las estimaciones anteriores

$$\log_2 \mathcal{N}_\epsilon(\operatorname{Supp}(\boldsymbol{x})) - O(D) \leq R_\epsilon(\boldsymbol{x}), \tag{B.3.17}$$

como se debía demostrar.

□

B.3.2 Demostración del Lema B.6

(Demostración de Lema B.6). Basta con demostrar que cualquier código para $\boldsymbol{x}$ con una distorsión cuadrática esperada de ϵ^2 produce un código para $\boldsymbol{x}_\delta$ con la misma tasa y una distorsión ligeramente superior, si se elige un valor adecuado de δ. Así pues, fijemos un código de este tipo para $\boldsymbol{x}$, que alcance una tasa de R y una distorsión cuadrática esperada de ϵ^2. Denotamos por $\hat{\boldsymbol{x}}$ la variable aleatoria reconstruida mediante este código, y por $\mathrm{q} : \operatorname{Supp}(\boldsymbol{x}) \to \operatorname{Supp}(\boldsymbol{x})$ la aplicación de codificación-decodificación asociada (es decir, $\hat{\boldsymbol{x}} = \mathrm{q}(\boldsymbol{x})$).

Sea ahora $\mathcal{S}_k$ la hiperesfera de orden k en el soporte de $\boldsymbol{x}$. Existe una base ortonormal $\boldsymbol{U}_k \in \mathbb{R}^{D \times d_k}$ tal que $\operatorname{Span}(\mathcal{S}_k) = \operatorname{Span}(\boldsymbol{U}_k)$. La siguiente

descomposición ortogonal del conjunto de soporte $\mathcal{S}$ se utilizará repetidamente a lo largo de la demostración. Tenemos

$$\mathcal{S}_\delta = \{\boldsymbol{\xi} \in \mathbb{R}^D \mid \exists k \in [K] \,:\, \operatorname{dist}(\boldsymbol{\xi}, \mathcal{S}_k) \leq \delta\} \tag{B.3.18}$$

$$= \bigcup_{k\in[K]} \{\boldsymbol{\xi} \in \mathbb{R}^D \mid \operatorname{dist}(\boldsymbol{\xi}, \mathcal{S}_k) \leq \delta\}. \tag{B.3.19}$$

Mediante la proyección ortogonal, para cualquier $k \in [K]$, cualquier $\boldsymbol{\xi} \in \mathbb{R}^D$ puede escribirse como $\boldsymbol{\xi} = \boldsymbol{\xi}^{\parallel} + \boldsymbol{\xi}^{\perp}$, donde $\boldsymbol{\xi}^{\parallel} \in \operatorname{Span}(\mathcal{S}_k)$ y $\langle \boldsymbol{\xi}^{\parallel}, \boldsymbol{\xi}^{\perp} \rangle = 0$. Entonces, para cualquier $\boldsymbol{\xi}' \in \mathcal{S}_k$, tenemos

$$\|\boldsymbol{\xi} - \boldsymbol{\xi}'\|_2^2 = \left\|\boldsymbol{\xi}^{\parallel} + \boldsymbol{\xi}^{\perp} - \boldsymbol{\xi}'\right\|_2^2 = \left\|\boldsymbol{\xi}^{\parallel}\right\|_2^2 + \left\|\boldsymbol{\xi}^{\perp}\right\|_2^2 + \|\boldsymbol{\xi}'\|_2^2 - 2\left\langle \boldsymbol{\xi}^{\parallel}, \boldsymbol{\xi}' \right\rangle \tag{B.3.20}$$

$$\geq \left\|\boldsymbol{\xi}^{\parallel}\right\|_2^2 + \|\boldsymbol{\xi}'\|_2^2 - 2\left\langle \boldsymbol{\xi}^{\parallel}, \boldsymbol{\xi}' \right\rangle \tag{B.3.21}$$

$$= \left\|\boldsymbol{\xi}^{\parallel} - \boldsymbol{\xi}'\right\|_2^2. \tag{B.3.22}$$

Además, se sabe que para cualquier $\boldsymbol{\xi}^{\parallel}$ distinto de cero que pertenezca a $\operatorname{Span}(\mathcal{S}_k)$,

$$\inf_{\boldsymbol{\xi}' \in \mathcal{S}_k} \left\|\boldsymbol{\xi}^{\parallel} - \boldsymbol{\xi}'\right\|_2^2 = \left\|\boldsymbol{\xi}^{\parallel} - \frac{\boldsymbol{\xi}^{\parallel}}{\|\boldsymbol{\xi}^{\parallel}\|_2}\right\|_2^2. \tag{B.3.23}$$

Si $\boldsymbol{\xi}^{\parallel}$ es cero, es evidente que la distancia anterior es igual a 1 para todo $\boldsymbol{\xi}' \in \mathcal{S}_k$. Por lo tanto, si definimos un mapeo de proyección $\pi_{\mathcal{S}_k}(\boldsymbol{\xi})$ mediante

$$\pi_{S_k}(\boldsymbol{\xi}) = \frac{\boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{\xi}}{\|\boldsymbol{U}_k^\top \boldsymbol{\xi}\|_2} \tag{B.3.24}$$

Para cualquier $\boldsymbol{\xi} \in \mathbb{R}^D$ tal que $\boldsymbol{U}_k^\top \boldsymbol{\xi} \neq \mathbf{0}$, entonces $\pi_{\mathcal{S}_k}(\boldsymbol{\xi}) = \arg\min_{\boldsymbol{\xi}' \in \mathcal{S}_k} \|\boldsymbol{\xi}' - \boldsymbol{\xi}\|_2$. Elegimos $0 < \delta < 1$, de modo que el conjunto engrosado $\mathcal{S}_\delta$ no contenga puntos $\boldsymbol{\xi} \in \mathbb{R}^D$ en los que alguna de los mapeos de proyección $\pi_{\mathcal{S}_k}$ no esté bien definida. Por lo tanto, el conjunto engrosado $\mathcal{S}_\delta$ satisface

$$\mathcal{S}_\delta = \bigcup_{k\in[K]} \left\{ \boldsymbol{\xi} \in \mathbb{R}^D \;\middle|\; \left\| \boldsymbol{\xi} - \frac{\boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{\xi}}{\|\boldsymbol{U}_k^\top \boldsymbol{\xi}\|_2} \right\|_2 \leq \delta \right\}. \tag{B.3.25}$$

Estas distancias pueden reescribirse en términos de la descomposición ortogonal como

$$\left\| \boldsymbol{\xi} - \frac{\boldsymbol{U}_k \boldsymbol{U}_k^\top \boldsymbol{\xi}}{\|\boldsymbol{U}_k^\top \boldsymbol{\xi}\|_2} \right\|_2^2 = \|\boldsymbol{\xi}\|_2^2 - 2\|\boldsymbol{U}_k^\top \boldsymbol{\xi}\|_2 + 1 \tag{B.3.26}$$

$$= \left\|\boldsymbol{\xi}^{\parallel}\right\|_2^2 + \left\|\boldsymbol{\xi}^{\perp}\right\|_2^2 - 2\|\boldsymbol{U}_k^\top \boldsymbol{\xi}^{\parallel}\|_2 + 1 \tag{B.3.27}$$

$$= \left\|\boldsymbol{\xi}^{\perp}\right\|_2^2 + \left(\left\|\boldsymbol{\xi}^{\parallel}\right\|_2 - 1\right)^2. \tag{B.3.28}$$

A continuación demostraremos que cada uno de estos $\boldsymbol{\xi} \in \mathcal{S}_\delta$ puede asociarse en forma única a una proyección sobre un único subespacio de la mezcla, lo que nos permitirá definir una proyección correspondiente sobre $\mathcal{S}$. Dado un $\boldsymbol{\xi} \in \mathcal{S}_\delta$, por lo anterior, podemos encontrar un subespacio $\boldsymbol{U}_k$ tal que la descomposición ortogonal $\boldsymbol{\xi} = \boldsymbol{\xi}_k^{\|} + \boldsymbol{\xi}_k^{\perp}$ satisfaga

$$\left\|\boldsymbol{\xi}_k^{\perp}\right\|_2^2 + \left(\left\|\boldsymbol{\xi}_k^{\|}\right\|_2 - 1\right)^2 \leq \delta^2. \tag{B.3.29}$$

Consideremos la descomposición $\boldsymbol{\xi} = \boldsymbol{\xi}_j^{\|} + \boldsymbol{\xi}_j^{\perp}$ para algún $j \neq k$. Tenemos que

$$\left\|\boldsymbol{\xi}_j^{\|}\right\|_2 = \left\|\boldsymbol{U}_j\boldsymbol{U}_j^\top\boldsymbol{\xi}\right\|_2 = \left\|\boldsymbol{U}_j\boldsymbol{U}_j^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{\xi} + (\boldsymbol{I} - \boldsymbol{U}_k\boldsymbol{U}_k^\top)\boldsymbol{\xi})\right\|_2 \tag{B.3.30}$$

$$= \left\|\boldsymbol{U}_j\boldsymbol{U}_j^\top(\boldsymbol{I} - \boldsymbol{U}_k\boldsymbol{U}_k^\top)\boldsymbol{\xi}\right\|_2 \tag{B.3.31}$$

$$\leq \left\|(\boldsymbol{I} - \boldsymbol{U}_k\boldsymbol{U}_k^\top)\boldsymbol{\xi}\right\|_2 = \left\|\boldsymbol{\xi}_k^{\perp}\right\|_2 \leq \delta, \tag{B.3.32}$$

donde la segunda línea se basa en el supuesto de ortogonalidad de los subespacios $\boldsymbol{U}_k$, y la tercera se basa en el hecho de que las proyecciones ortogonales son no expansivas. Por lo tanto, la j-ésima distancia cumple que

$$\left\|\boldsymbol{\xi}_j^{\perp}\right\|_2^2 + \left(\left\|\boldsymbol{\xi}_j^{\|}\right\|_2 - 1\right)^2 \geq (1-\delta)^2. \tag{B.3.33}$$

Esto implica que si $0 < \delta < 1/2$, cada $\boldsymbol{\xi} \in \mathcal{S}_\delta$ tiene un único subespacio más cercano en la mezcla. Por lo tanto, bajo esta condición, la siguiente aplicación $\pi_{\mathcal{S}} : \mathcal{S}_\delta \to \mathcal{S}$ está bien definida:

$$\pi_{\mathcal{S}}(\boldsymbol{\xi}) = \pi_{\mathcal{S}_{k_\star}}(\boldsymbol{\xi}), \quad \text{where} \quad k_\star = \arg\min_{k\in[K]} \operatorname{dist}(\boldsymbol{\xi}, \mathcal{S}_k). \tag{B.3.34}$$

Ahora, definimos un código para $\boldsymbol{x}_\delta$ mediante

$$\hat{\boldsymbol{x}}_\delta = \mathrm{q}(\pi_{\mathcal{S}}(\boldsymbol{x}_\delta)). \tag{B.3.35}$$

Es evidente que esto está relacionado con un código de tasa R para $\boldsymbol{x}_\delta$, ya que utiliza las correspondencias de codificación-decodificación del código de tasa R para $\boldsymbol{x}$. Tenemos que demostrar que presenta una distorsión reducida. Calculamos

$$\mathbb{E}\left[\|\boldsymbol{x}_\delta - \hat{\boldsymbol{x}}_\delta\|_2^2\right] = \mathbb{E}\left[\|\boldsymbol{x}_\delta - \mathrm{q}(\pi_{\mathcal{S}}(\boldsymbol{x}_\delta))\|_2^2\right] \tag{B.3.36}$$

$$\leq \left(\mathbb{E}\left[\|\boldsymbol{x}_\delta - \pi_{\mathcal{S}}(\boldsymbol{x}_\delta)\|_2^2\right]^{1/2} + \mathbb{E}\left[\|\pi_{\mathcal{S}}(\boldsymbol{x}_\delta) - \mathrm{q}(\pi_{\mathcal{S}}(\boldsymbol{x}_\delta))\|_2^2\right]^{1/2}\right)^2, \tag{B.3.37}$$

donde la desigualdad utiliza la desigualdad de Minkowski. Ahora, por las Definiciones B.1 y B.2, tenemos determinísticamente

$$\|\boldsymbol{x}_\delta - \pi_{\mathcal{S}}(\boldsymbol{x}_\delta)\|_2^2 \leq \delta^2, \tag{B.3.38}$$

por lo que la esperanza también satisface esta estimación. Para el segundo término, bastará con caracterizar la densidad de la variable aleatoria $\pi_{\mathcal{S}}(\boldsymbol{x}_\delta)$ como suficientemente cercana a la densidad de $\boldsymbol{x}$ —la cual, tal y como implica el Supuesto B.3, es una mezcla de distribuciones uniformes en cada subesfera $\mathcal{S}_k$. Según el argumento anterior, cada punto $\boldsymbol{\xi} \in \mathcal{S}_\delta$ puede asociarse a un único subespacio $\boldsymbol{U}_k$, lo que significa que las componentes de la mezcla en la definición de $\mathcal{S}_\delta$ (recordemos la Definición B.2) no se superponen. Por lo tanto, la densidad $\pi_{\mathcal{S}}(\boldsymbol{x}_\delta)$ puede caracterizarse estudiando el efecto de $\pi_{\mathcal{S}_k}$ sobre la variable aleatoria condicional $\boldsymbol{x}_\delta$, condicionada a ser extraída de $\mathcal{S}_{k,\delta}$. Denotemos esta medida por $\mu_{k,\delta}$. Afirmamos que la proyección de esta medida bajo $\pi_{\mathcal{S}_k}$ es uniforme en $\mathcal{S}_k$. Para ver que esto es cierto, recordamos la Ecuación (B.3.28), que da la caracterización

$$\mathcal{S}_{k,\delta} = \left\{ \boldsymbol{\xi}^{\|} + \boldsymbol{\xi}^{\perp} \;\middle|\; \boldsymbol{\xi}^{\|} \in \operatorname{Span}(\boldsymbol{U}_k), \boldsymbol{\xi}^{\perp} \in \operatorname{Span}(\boldsymbol{U}_k)^{\perp}, \left\|\boldsymbol{\xi}^{\perp}\right\|_2^2 + \left(\left\|\boldsymbol{\xi}^{\|}\right\|_2 - 1\right)^2 \leq \delta^2 \right\}. \tag{B.3.39}$$

La distribución condicional en cuestión es uniforme en este conjunto; necesitamos demostrar que la proyección $\pi_{\mathcal{S}_k}$ aplicada a esta variable aleatoria condicional da como resultado una variable aleatoria que es uniforme en $\mathcal{S}_k$. Con respecto a estas coordenadas, hemos visto que $\pi_{\mathcal{S}_k}(\boldsymbol{\xi}^{\|} + \boldsymbol{\xi}^{\perp}) = \boldsymbol{\xi}^{\|}/\|\boldsymbol{\xi}^{\|}\|_2$. Por lo tanto, para cualquier $\boldsymbol{\xi} \in \mathcal{S}_k$, tenemos que la preimagen de $\boldsymbol{\xi}$ en $\mathcal{S}_{k,\delta}$ bajo $\pi_{\mathcal{S}_k}$ es

$$\pi_{\mathcal{S}_k}^{-1}(\boldsymbol{\xi}) = \left\{ r\boldsymbol{\xi} + \boldsymbol{\xi}^{\perp} \;\middle|\; r > 0, \boldsymbol{\xi}^{\perp} \in \operatorname{Span}(\boldsymbol{U}_k)^{\perp}, \left\|\boldsymbol{\xi}^{\perp}\right\|_2^2 + (r-1)^2 \leq \delta^2 \right\}. \tag{B.3.40}$$

Para demostrar que $(\pi_{\mathcal{S}_k})_\sharp \mu_{k,\delta}$ es uniforme, necesitamos descomponer la integral de la densidad uniforme sobre $\mathcal{S}_{k,\delta}$ de tal manera que quede claro que cada una de las fibras $\pi_{\mathcal{S}_k}^{-1}(\boldsymbol{\xi})$ (para cada $\boldsymbol{\xi} \in \mathcal{S}_k$) «contribuye» por igual a la integral.[6] Por la Definición B.2 tenemos

$$\operatorname{vol}(\mathcal{S}_{k,\delta}) = \iint_{\operatorname{Span}(\boldsymbol{U}_k) \times \operatorname{Span}(\boldsymbol{U}_k)^{\perp}} \mathbf{1}_{\|\boldsymbol{\xi}^{\perp}\|_2^2 + (\|\boldsymbol{\xi}^{\|}\|_2 - 1)^2 \leq \delta^2} \mathrm{d}\boldsymbol{\xi}^{\|} \mathrm{d}\boldsymbol{\xi}^{\perp}. \tag{B.3.41}$$

En particular, la integración sobre las coordenadas ortogonales se factoriza. Sea $\mathrm{d}\boldsymbol{\theta}^d$ la medida uniforme (de Haar) sobre la esfera de radio 1 en $\mathbb{R}^d$. Al convertir la integral $\boldsymbol{\xi}^{\|}$ a coordenadas polares, tenemos

$$\operatorname{vol}(\mathcal{S}_{k,\delta}) = \int_{[0,\infty)} \int_{\mathbb{S}^{d_k - 1}} \int_{\operatorname{Span}(\boldsymbol{U}_k)^{\perp}} r^{d_k - 1} \mathbf{1}_{\|\boldsymbol{\xi}^{\perp}\|_2^2 + (r-1)^2 \leq \delta^2} \mathrm{d}r \mathrm{d}\boldsymbol{\theta}^{d_k} \mathrm{d}\boldsymbol{\xi}^{\perp}. \tag{B.3.42}$$

Si comparamos con la representación de fibra (B.3.40), vemos que necesitamos «integrar» sobre las componentes r y $\boldsymbol{\xi}^{\perp}$ de la integral anterior para verificar que el pushforward es uniforme. Pero esto es evidente, ya que la expresión

[6] Más concretamente, esto equivale a descomponer la densidad uniforme en $\mathcal{S}_{k,\delta}$ en una densidad condicional regular correspondiente a $\boldsymbol{\xi} \in \mathcal{S}_k$, y demostrar que la densidad correspondiente en $\boldsymbol{\xi}$ es uniforme. La demostración deja claro que esto es cierto.

anterior muestra que el valor de esta integral es independiente de $\boldsymbol{\xi}^{\|}$—o, de manera equivalente en este contexto, del valor de la componente esférica $\boldsymbol{\theta}^{d_k}$.

Por lo tanto, del argumento anterior se deduce que $\pi_{\mathcal{S}}(\boldsymbol{x}_\delta)$ es uniforme. Dado que el supuesto sobre δ implica que las componentes de la mezcla en la distribución de $\boldsymbol{x}_\delta$ no se superponen, los pesos de mezcla π_k también se conservan en la imagen $\pi_{\mathcal{S}}(\boldsymbol{x}_\delta)$, y en particular, la distribución de $\pi_{\mathcal{S}}(\boldsymbol{x}_\delta)$ es igual a la distribución de $\boldsymbol{x}$. Por lo tanto, el segundo término en la Ecuación (B.3.37) satisface

$$\mathbb{E}\left[\|\pi_{\mathcal{S}}(\boldsymbol{x}_\delta) - \mathrm{q}(\pi_{\mathcal{S}}(\boldsymbol{x}_\delta))\|_2^2\right] = \mathbb{E}\left[\|\boldsymbol{x} - \mathrm{q}(\boldsymbol{x})\|_2^2\right] \leq \epsilon^2, \tag{B.3.43}$$

porque q es un código de distorsión ϵ para $\boldsymbol{x}$.

Así pues, hemos demostrado que el código hipotético de tasa R y distorsión (esperada al cuadrado) ϵ^2 para $\boldsymbol{x}$ produce un código de tasa R y distorsión (esperada al cuadrado) $\delta + \epsilon$ para $\boldsymbol{x}_\delta$. Esto establece que

$$R_{\delta+\epsilon}(\boldsymbol{x}_\delta) \leq R_\epsilon(\boldsymbol{x}), \tag{B.3.44}$$

como se quería demostrar.

□

Bibliography

[AKH15] Yousset I Abdel-Aziz, Hauck Michael Karara, and Michael Hauck. "Direct linear transformation from comparator coordinates into object space coordinates in close-range photogrammetry". *Photogrammetric engineering & remote sensing* 81.2 (2015), pp. 103–107.

[AMS09] P-A Absil, R Mahony, and R Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, Apr. 2009.

[AAJ+16] Alekh Agarwal, Animashree Anandkumar, Prateek Jain, and Praneeth Netrapalli. "Learning Sparsely Used Overcomplete Dictionaries via Alternating Minimization". *SIAM Journal on Optimization* 26.4 (2016), pp. 2775–2799. eprint: https://doi.org/10.1137/140979861.

[AEB06] Michal Aharon, Michael Elad, and Alfred Bruckstein. "K-SVD: An algorithm for designing overcomplete dictionaries for sparse representation". *IEEE Transactions on signal processing* 54.11 (2006), pp. 4311–4322.

[ACM12] William K. Allard, Guangliang Chen, and Mauro Maggioni. "Multi-scale geometric methods for data sets II: Geometric Multi-Resolution Analysis". *Applied and Computational Harmonic Analysis* 32.3 (2012), pp. 435–462.

[AR20] Jason M. Allred and Kaushik Roy. "Controlled Forgetting: Targeted Stimulation and Dopaminergic Plasticity Modulation for Unsupervised Lifelong Learning in Spiking Neural Networks". *Frontiers in Neuroscience* 14 (2020).

[ADG+16] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando De Freitas. "Learning to learn by gradient descent by gradient descent". *Advances in neural information processing systems*. 2016, pp. 3981–3989.

[ACB17] Martin Arjovsky, Soumith Chintala, and Léon Bottou. "Wasserstein generative adversarial networks". *International conference on machine learning*. PMLR. 2017, pp. 214–223.

[AGM+15] Sanjeev Arora, Rong Ge, Tengyu Ma, and Ankur Moitra. "Simple, Efficient, and Neural Algorithms for Sparse Coding". *Proceedings of The 28th Conference on Learning Theory*. Ed. by Peter Grünwald, Elad Hazan, and Satyen Kale. Vol. 40. Proceedings of Machine Learning Research. Paris, France: PMLR, July 2015, pp. 113–149.

[AW18] Aharon Azulay and Yair Weiss. "Why do deep convolutional networks generalize so poorly to small image transformations?" *arXiv preprint arXiv:1805.12177* (2018).

[BJC85] B. Ans, J. Hérault, and C. Jutten. "Architectures neuromimétiques adaptatives : Détection de primitives". *Cognitiva* 2 (1985), pp. 593–597.

[BKH16] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. "Layer normalization". *arXiv preprint arXiv:1607.06450* (2016).

[BM24] Hao Bai and Yi Ma. "Improving neuron-level interpretability with white-box language models". *arXiv preprint arXiv:2410.16443* (2024).

[BGN+17] Bowen Baker, Otkrist Gupta, N. Naik, and R. Raskar. "Designing Neural Network Architectures using Reinforcement Learning". *ArXiv* abs/1611.02167 (2017).

[Bal11] Pierre Baldi. "Autoencoders, unsupervised learning and deep architectures". *Proceedings of the 2011 International Conference on Unsupervised and Transfer Learning Workshop - Volume 27*. UTLW'11. Washington, USA: JMLR.org, 2011, pp. 37–50.

[BH89] Pierre Baldi and Kurt Hornik. "Neural networks and principal component analysis: Learning from examples without local minima". *Neural networks* 2.1 (1989), pp. 53–58.

[BS16] Pierre Baldi and Peter Sadowski. "A theory of local learning, the learning channel, and the optimality of backpropagation". *Neural Networks* 83 (2016), pp. 51–74.

[BLZ+22] Fan Bao, Chongxuan Li, Jun Zhu, and Bo Zhang. "Analytic-dpm: an analytic estimate of the optimal reverse variance in diffusion probabilistic models". *arXiv preprint arXiv:2201.06503* (2022).

[BNX+23] Fan Bao, Shen Nie, Kaiwen Xue, Yue Cao, Chongxuan Li, Hang Su, and Jun Zhu. "All are Worth Words: A ViT Backbone for Diffusion Models". *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 22669–22679.

[BSM+20] Pinglei Bao, Liang She, Mason McGill, and Doris Y. Tsao. "A map of object space in primate inferotemporal cortex". *Nature* 583 (2020), pp. 103–108.

[BYL+23] Omer Bar-Tal, Lior Yariv, Yaron Lipman, and Tali Dekel. "MultiDiffusion: Fusing Diffusion Paths for Controlled Image Generation". *International Conference on Machine Learning*. PMLR. 2023, pp. 1737–1752.

[BKS15] Boaz Barak, Jonathan A Kelner, and David Steurer. "Dictionary learning and tensor decomposition via the sum-of-squares method". *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing*. New York, NY, USA: ACM, June 2015.

[BT09] Amir Beck and Marc Teboulle. "A fast iterative shrinkage-thresholding algorithm for linear inverse problems". *SIAM journal on imaging sciences* 2.1 (2009), pp. 183–202.

[BHM+19] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. "Reconciling modern machine-learning practice and the classical bias–variance trade-off". *Proceedings of the National Academy of Sciences* 116.32 (2019), pp. 15849–15854.

[BBH+15] Elizabeth A Bell, Patrick Boehnke, T Mark Harrison, and Wendy L Mao. "Potentially biogenic carbon preserved in a 4.1 billion-year-old zircon". *Proceedings of the National Academy of Sciences* 112.47 (2015), pp. 14518–14521.

[Bel57] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[Ben23] Max Bennett. *A Brief History of Intelligence: Evolution, AI, and the Five Breakthroughs That Made Our Brains*. Mariner Books, 2023.

[BN24a] Jeremy Bernstein and Laker Newhouse. *Old Optimizer, New Norm: An Anthology*. 2024. arXiv: 2409.20325 [cs.LG].

[Bla72] R. Blahut. "Computation of channel capacity and rate-distortion functions". *IEEE Transactions on Information Theory* 18.4 (1972), pp. 460–473.

[BRL+23] Andreas Blattmann, Robin Rombach, Huan Ling, Tim Dockhorn, Seung Wook Kim, Sanja Fidler, and Karsten Kreis. "Align your latents: High-resolution video synthesis with latent diffusion models". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 22563–22575.

[BAV25] Nicholas M Boffi, Michael S Albergo, and Eric Vanden-Eijnden. "How to build a consistency model: Learning flow maps via self-distillation". *arXiv preprint arXiv:2505.18825* (2025).

[BBF+01] Lorna Booth, Jehoshua Bruck, M. Franceschetti, and Ronald Meester. "Covering Algorithms, Continuum Percolation and the Geometry of Wireless Networks". *Ann. Appl. Probab.* 13 (July 2001).

[Bor97] Vivek S Borkar. "Stochastic approximation with two time scales". *Systems & Control Letters* 29.5 (1997), pp. 291–294.

[BDS16] Vivek S Borkar, Raaz Dwivedi, and Neeraja Sahasrabudhe. "Gaussian approximations in high dimensional estimation". *Systems & Control Letters* 92 (2016), pp. 42–45.

[Bos50] R. Boscovich. *De calculo probabilitatum que respondent diversis valoribus summe errorum post plures observationes, quarum singule possient esse erronee certa quadam quantitate*. 1750.

[Bou23] Nicolas Boumal. *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press, Mar. 2023.

[BN24b] Arwen Bradley and Preetum Nakkiran. "Classifier-Free Guidance is a Predictor-Corrector". *arXiv [cs.LG]* (Aug. 2024). arXiv: 2408.09000 [cs.LG].

[BN20] Guy Bresler and Dheeraj Nagaraj. "Sharp representation theorems for relu networks with precise dependence on depth". *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS'20 Article 897. Red Hook, NY, USA: Curran Associates Inc., Dec. 2020, pp. 10697–10706.

[BB11] Haim Brezis and Haim Brézis. *Functional analysis, Sobolev spaces and partial differential equations*. Vol. 2. 3. Springer, 2011.

[BEJ25] Paige Bright, Alan Edelman, and Steven G Johnson. "Matrix Calculus (for Machine Learning and Beyond)". *arXiv preprint arXiv:2501.14787* (2025).

[BDS19] Andrew Brock, Jeff Donahue, and Karen Simonyan. "Large Scale GAN Training for High Fidelity Natural Image Synthesis". *International Conference on Learning Representations (ICLR)*. 2019.

[BMR+20] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. "Language models are few-shot learners". *arXiv preprint arXiv:2005.14165* (2020).

[BDE+24] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. "Genie: Generative interactive environments". *Forty-first International Conference on Machine Learning*. 2024.

[BM13] Joan Bruna and Stéphane Mallat. "Invariant Scattering Convolution Networks". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 (2013), pp. 1872–1886.

[BGW21] Sam Buchanan, Dar Gilboa, and John Wright. "Deep Networks and the Multiple Manifold Problem". *International Conference on Learning Representations*. 2021.

[BPM+25] Sam Buchanan, Druv Pai, Yi Ma, and Valentin De Bortoli. "On the edge of memorization in diffusion models". *arXiv preprint arXiv:2508.17689* (2025).

[CD91] M. Frank Callier and A. Charles Desoer. *Linear System Theory*. Springer-Verlag, 1991.

[Can06] E. Candès. "Compressive sampling". *Proceedings of the International Congress of Mathematicians*. 2006.

[CT05a] E. Candès and T. Tao. "Decoding by linear programming". *IEEE Transactions on Information Theory* 51.12 (2005).

[CT05b] E. Candès and T. Tao. "Error Correction via Linear Programming". *IEEE Symposium on FOCS* (2005), pp. 295–308.

[CMM+21] Mathilde Caron, Ishan Misra, Julien Mairal, Priya Goyal, Piotr Bojanowski, and Armand Joulin. *Unsupervised Learning of Visual Features by Contrasting Cluster Assignments*. 2021. arXiv: 2006.09882 [cs.CV].

[CTM+21] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. "Emerging properties in self-supervised vision transformers". *Proceedings of the IEEE/CVF international conference on computer vision*. 2021, pp. 9650–9660.

[Cha66] Gregory J. Chaitin. "On the Length of Programs for Computing Finite Binary Sequences". *J. ACM* 13.4 (Oct. 1966), pp. 547–569.

[CYY+22] Kwan Ho Ryan Chan, Yaodong Yu, Chong You, Haozhi Qi, John Wright, and Yi Ma. "ReduNet: A White-box Deep Network from the Principle of Maximizing Rate Reduction". *Journal of Machine Learning Research* 23.114 (2022), pp. 1–103.

[CJG+15] Tsung-Han Chan, Kui Jia, Shenghua Gao, Jiwen Lu, Zinan Zeng, and Yi Ma. "PCANet: A simple deep learning baseline for image classification?" *TIP* (2015).

[CFG+15] Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. "Shapenet: An information-rich 3d model repository". *arXiv preprint arXiv:1512.03012* (2015).

[CWM+17] Jianlong Chang, Lingfeng Wang, Gaofeng Meng, Shiming Xiang, and Chunhong Pan. "Deep adaptive image clustering". *Proceedings of the IEEE international conference on computer vision*. 2017, pp. 5879–5887.

[CT17] Le Chang and Doris Tsao. "The Code for Facial Identity in the Primate Brain". *Cell* 169 (June 2017), 1013–1028.e14.

[CMP+21] Sneha Chaudhari, Varun Mithal, Gungor Polatkan, and Rohan Ramanath. "An attentive survey of attention models". *ACM Transactions on Intelligent Systems and Technology (TIST)* 12.5 (2021), pp. 1–32.

[CRE+19] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K Dokania, Philip HS Torr, and Marc'Aurelio Ranzato. "On tiny episodic memories in continual learning". *arXiv preprint arXiv:1902.10486* (2019).

[CXE+24] Anpei Chen, Haofei Xu, Stefano Esposito, Siyu Tang, and Andreas Geiger. "LaRa: Efficient Large-Baseline Radiance Fields". *European Conference on Computer Vision (ECCV)*. 2024.

[CHZ+23] Minshuo Chen, Kaixuan Huang, Tuo Zhao, and Mengdi Wang. "Score approximation, estimation and distribution recovery of diffusion models on low-dimensional data". *International Conference on Machine Learning*. PMLR. 2023, pp. 4672–4712.

[CRB+18] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. "Neural ordinary differential equations". *Advances in neural information processing systems* 31 (2018).

[CCL+23] Sitan Chen, Sinho Chewi, Jerry Li, Yuanzhi Li, Adil Salim, and Anru Zhang. "Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions". *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.

[CZG+24] Siyi Chen, Huijie Zhang, Minzhe Guo, Yifu Lu, Peng Wang, and Qing Qu. "Exploring low-dimensional subspace in diffusion models for controllable image editing". *Advances in Neural Information Processing Systems* 37 (2024), pp. 27340–27371.

[CZL+25] Siyi Chen, Yimeng Zhang, Sijia Liu, and Qing Qu. "The Dual Power of Interpretable Token Embeddings: Jailbreaking Attacks and Defenses for Diffusion Model Unlearning". *arXiv preprint arXiv:2504.21307* (2025).

[CKN+20] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. "A simple framework for contrastive learning of visual representations". *arXiv preprint arXiv:2002.05709* (2020).

[CLH+24] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. "Symbolic discovery of optimization algorithms". *Advances in neural information processing systems* 36 (2024).

[CAP20] Julian Chibane, Thiemo Alldieck, and Gerard Pons-Moll. "Implicit Functions in Feature Space for 3D Shape Reconstruction and Completion". *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 6968–6979.

[Cho17] Francois Chollet. "Xception: Deep Learning with Depthwise Separable Convolutions". *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 1800–1807.

[CKM+23] Hyungjin Chung, Jeongsol Kim, Michael Thompson Mccann, Marc Louis Klasky, and Jong Chul Ye. "Diffusion Posterior Sampling for General Noisy Inverse Problems". *The Eleventh International Conference on Learning Representations*. 2023.

[CW16a] Taco Cohen and Max Welling. "Group equivariant convolutional networks". *International Conference on Machine Learning*. 2016, pp. 2990–2999.

[CW16b] Taco Cohen and Max Welling. "Group equivariant convolutional networks". *International conference on machine learning*. PMLR. 2016, pp. 2990–2999.

[CW16c] Taco S. Cohen and Max Welling. "Group Equivariant Convolutional Networks". *CoRR* abs/1602.07576 (2016). arXiv: 1602.07576.

[CT91] T. Cover and J. Thomas. *Elements of Information Theory*. Wiley Series in Telecommunications, 1991.

[Cyb89] George V. Cybenko. "Approximation by superpositions of a sigmoidal function". *Mathematics of Control, Signals and Systems* 2 (1989), pp. 303–314.

[D D00] D. Donoho. "High-dimensional data analysis: The curses and blessings of dimensionality". *AMS Math Challenges Lecture* (2000).

[DM03] D. Donoho and M. Elad. "Optimally sparse representation in general (nonorthogonal) dictionaries via ℓ^1 minimization". *PNAS* 100.5 (2003), pp. 2197–2202.

[DTL+22] Xili Dai, Shengbang Tong, Mingyang Li, Ziyang Wu, Michael Psenka, Kwan Ho Ryan Chan, Pengyuan Zhai, Yaodong Yu, Xiaojun Yuan, Heung-Yeung Shum, and Yi Ma. "CTRL: Closed-Loop Transcription to an LDR via Minimaxing Rate Reduction". *Entropy* 24.4 (2022).

[Dan02] George B Dantzig. “Linear Programming”. *Operations Research* 50.1 (2002), pp. 42–47.

[DSD+23a] Giannis Daras, Kulin Shah, Yuval Dagan, Aravind Gollakota, Alex Dimakis, and Adam Klivans. “Ambient Diffusion: Learning Clean Distributions from Corrupted Data”. *Thirty-seventh Conference on Neural Information Processing Systems*. 2023.

[DSD+23b] Giannis Daras, Kulin Shah, Yuval Dagan, Aravind Gollakota, Alex Dimakis, and Adam Klivans. “Ambient Diffusion: Learning Clean Distributions from Corrupted Data”. *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., 2023, pp. 288–313.

[DGG+25] Valentin De Bortoli, Alexandre Galashov, J Swaroop Guntupalli, Guangyao Zhou, Kevin Murphy, Arthur Gretton, and Arnaud Doucet. “Distributional Diffusion Models with Scoring Rules”. *arXiv preprint arXiv:2502.02483* (2025).

[DCM+23] Aaron Defazio, Ashok Cutkosky, Harsh Mehta, and Konstantin Mishchenko. “Optimal linear decay learning rate schedules and further refinements”. *arXiv preprint arXiv:2310.07831* (2023).

[DLW+23] Matt Deitke, Ruoshi Liu, Matthew Wallingford, Huong Ngo, Oscar Michel, Aditya Kusupati, Alan Fan, Christian Laforte, Vikram Voleti, Samir Yitzhak Gadre, Eli VanderBilt, Aniruddha Kembhavi, Carl Vondrick, Georgia Gkioxari, Kiana Ehsani, Ludwig Schmidt, and Ali Farhadi. “Objaverse-XL: a universe of 10M+ 3D objects”. *Proceedings of the 37th International Conference on Neural Information Processing Systems*. NIPS ’23. New Orleans, LA, USA: Curran Associates Inc., 2023.

[DDS22] Julie Delon, Agnes Desolneux, and Antoine Salmona. “Gromov–Wasserstein distances between Gaussian distributions”. *Journal of Applied Probability* 59.4 (2022), pp. 1178–1198.

[DDS+09a] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. “ImageNet: A Large-Scale Hierarchical Image Database”. *CVPR09*. 2009.

[DDS+09b] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. “ImageNet: A Large-Scale Hierarchical Image Database”. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2009, pp. 248–255.

[DCL+19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. “Bert: Pre-training of deep bidirectional transformers for language understanding”. *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[DN21a] Prafulla Dhariwal and Alexander Nichol. "Diffusion Models Beat GANs on Image Synthesis". *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34. 2021.

[DN21b] Prafulla Dhariwal and Alexander Quinn Nichol. "Diffusion Models Beat GANs on Image Synthesis". *Advances in Neural Information Processing Systems*. Ed. by A Beygelzimer, Y Dauphin, P Liang, and J Wortman Vaughan. 2021.

[Don01] D L Donoho. "Sparse components of images and optimal atomic decompositions". *Constructive approximation* 17.3 (Jan. 2001), pp. 353–382.

[DVD+98] D L Donoho, M Vetterli, R A DeVore, and I Daubechies. "Data compression and harmonic analysis". *IEEE transactions on information theory / Professional Technical Group on Information Theory* 44.6 (Oct. 1998), pp. 2435–2476.

[Don05] David L Donoho. "Neighborly polytopes and sparse solutions of underdetermined linear equations". *Stanford Technical Report 2005-04* (2005).

[DBK+21] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.

[DFK+22] Laura Downs, Anthony Francis, Nate Koenig, Brandon Kinman, Ryan Hickman, Krista Reymann, Thomas B McHugh, and Vincent Vanhoucke. "Google scanned objects: A high-quality dataset of 3d scanned household items". *2022 International Conference on Robotics and Automation (ICRA)*. IEEE. 2022, pp. 2553–2560.

[DSC22] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. "Activation functions in deep learning: A comprehensive survey and benchmark". *Neurocomputing* 503 (2022), pp. 92–108.

[DSK17] Vincent Dumoulin, Jonathon Shlens, and Manjunath Kudlur. "A Learned Representation for Artistic Style". *arXiv preprint arXiv:1610.07629* (2017).

[EY36] Carl Eckart and Gale Young. "The approximation of one matrix by another of lower rank". *Psychometrika* 1.3 (Sept. 1936), pp. 211–218.

[EAS98] A Edelman, T Arias, and S Smith. “The Geometry of Algorithms with Orthogonality Constraints”. *SIAM Journal on Matrix Analysis and Applications* 20.2 (Jan. 1998), pp. 303–353.

[EA06] Michael Elad and Michal Aharon. “Image denoising via sparse and redundant representations over learned dictionaries”. *IEEE transactions on image processing: a publication of the IEEE Signal Processing Society* 15.12 (Dec. 2006), pp. 3736–3745.

[ELP+97] Yuval Eldar, Michael Lindenbaum, Moshe Porat, and Yehoshua Y Zeevi. “The farthest point strategy for progressive image sampling”. *IEEE transactions on image processing* 6.9 (1997), pp. 1305–1315.

[EHO+22a] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. “Toy models of superposition”. *arXiv preprint arXiv:2209.10652* (2022).

[EHO+22b] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. “Toy Models of Superposition”. *Transformer Circuits Thread* (2022).

[ETT+17] Logan Engstrom, Brandon Tran, Dimitris Tsipras, Ludwig Schmidt, and Aleksander Madry. “A rotation and a translation suffice: Fooling CNNs with simple transformations”. *arXiv preprint arXiv:1712.02779* (2017).

[EKB+24] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. “Scaling rectified flow transformers for high-resolution image synthesis”. *Forty-first international conference on machine learning*. 2024.

[ERO21] Patrick Esser, Robin Rombach, and Bjorn Ommer. “Taming transformers for high-resolution image synthesis”. *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 12873–12883.

[EGW+10] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. *The PASCAL Visual Object Classes (VOC) Challenge*. 2010. arXiv: 0909.5206 [cs.CV].

[FPH+25] Tyler Farghly, Peter Potaptchik, Samuel Howard, George Deligiannidis, and Jakiw Pidstrigach. “Diffusion models and the manifold hypothesis: Log-domain smoothing is geometry adaptive”. *arXiv preprint arXiv:2510.02305* (2025).

[FZS22] William Fedus, Barret Zoph, and Noam Shazeer. "Switch transformers: scaling to trillion parameter models with simple and efficient sparsity". *J. Mach. Learn. Res.* 23.1 (Jan. 2022).

[Fel49] William Feller. "On the Theory of Stochastic Processes, with Particular Reference to Applications". 1949.

[FCR20] Tanner Fiez, Benjamin Chasnov, and Lillian Ratliff. "Implicit learning dynamics in stackelberg games: Equilibria characterization, convergence analysis, and empirical study". *International Conference on Machine Learning*. PMLR. 2020, pp. 3133–3144.

[FCR19] Tanner Fiez, Benjamin Chasnov, and Lillian J Ratliff. "Convergence of learning dynamics in stackelberg games". *arXiv preprint arXiv:1906.01217* (2019).

[Fuk69] Kunihiko Fukushima. "Visual Feature Extraction by a Multilayered Network of Analog Threshold Elements". *IEEE Transactions on Systems Science and Cybernetics* 5.4 (1969), pp. 322–333.

[Fuk80] Kunihiko Fukushima. "Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position". *Biological Cybernetics* 36 (1980), pp. 193–202.

[GIF+23] Samir Yitzhak Gadre, Gabriel Ilharco, Alex Fang, Jonathan Hayase, Georgios Smyrnis, Thao Nguyen, Ryan Marten, Mitchell Wortsman, Dhruba Ghosh, Jieyu Zhang, Eyal Orgad, Rahim Entezari, Giannis Daras, Sarah Pratt, Vivek Ramanujan, Yonatan Bitton, Kalyani Marathe, Stephen Mussmann, Richard Vencu, Mehdi Cherti, Ranjay Krishna, Pang Wei Koh, Olga Saukh, Alexander Ratner, Shuran Song, Hannaneh Hajishirzi, Ali Farhadi, Romain Beaumont, Sewoong Oh, Alex Dimakis, Jenia Jitsev, Yair Carmon, Vaishaal Shankar, and Ludwig Schmidt. *DataComp: In search of the next generation of multimodal datasets*. 2023. arXiv: 2304.14108 [cs.CV].

[GTT+25] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. "Scaling and evaluating sparse autoencoders". *The Thirteenth International Conference on Learning Representations*. 2025.

[GHH+24] Ruiqi Gao, Aleksander Hoły'nski, Philipp Henzler, Arthur Brussee, Ricardo Martin-Brualla, Pratul Srinivasan, Jonathan T. Barron, and Ben Poole. "CAT3D: Create Anything in 3D with Multi-View Diffusion Models". *Advances in Neural Information Processing Systems (NeurIPS)*. Oral. 2024.

[GG23] Guillaume Garrigos and Robert M Gower. "Handbook of convergence theorems for (stochastic) gradient methods". *arXiv preprint arXiv:2301.11235* (2023).

[GWX+25] Zheng Geng, Nan Wang, Shaocong Xu, Chongjie Ye, Bohan Li, Zhaoxi Chen, Sida Peng, and Hao Zhao. "One View, Many Worlds: Single-Image to 3D Object Meets Generative Domain Randomization for One-Shot 6D Pose Estimation". *arXiv preprint arXiv:2509.07978* (2025).

[GDB+25] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J Zico Kolter, and Kaiming He. "Mean flows for one-step generative modeling". *arXiv preprint arXiv:2505.13447* (2025).

[GVM25] Anand Jerry George, Rodrigo Veiga, and Nicolas Macris. "Denoising score matching with random features: Insights on diffusion models from precise learning curves". *arXiv preprint arXiv:2502.00336* (2025).

[GRS+23] Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D Lee, and Dimitris Papailiopoulos. "Looped transformers as programmable computers". *International Conference on Machine Learning*. PMLR. 2023, pp. 11398–11442.

[Gil61] E. N. Gilbert. "Random Plane Networks". *Journal of the Society for Industrial and Applied Mathematics* 9.4 (1961), pp. 533–543. eprint: https://doi.org/10.1137/0109045.

[GC19] Aaron Gokaslan and Vanya Cohen. *OpenWebText Corpus*. http://Skylion007.github.io/OpenWebTextCorpus. 2019.

[GPM+14a] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative Adversarial Nets". *Advances in Neural Information Processing Systems*. Ed. by Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger. Vol. 27. Curran Associates, Inc., 2014.

[GPM+14b] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative adversarial nets". *Advances in neural information processing systems*. 2014, pp. 2672–2680.

[GDG+17] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. "Accurate, large minibatch sgd: Training imagenet in 1 hour". *arXiv preprint arXiv:1706.02677* (2017).

[GWT+23] Kristen Grauman, Andrew Westbury, Lorenzo Torresani, Kris Kitani, Jitendra Malik, Triantafyllos Afouras, Kumar Ashutosh, Vijay Baiyya, Siddhant Bansal, Bikram Boote, et al. "Ego-exo4d: Understanding skilled human activity from first-and third-person perspectives". *arXiv preprint arXiv:2311.18259* (2023).

[GL10] Karol Gregor and Yann LeCun. "Learning fast approximations of sparse coding". *Proceedings of the 27th International Conference on International Conference on Machine Learning*. 2010, pp. 399–406.

[Gri11] Rémi Gribonval. "Should Penalized Least Squares Regression be Interpreted as Maximum A Posteriori Estimation?" *IEEE transactions on signal processing: a publication of the IEEE Signal Processing Society* 59.5 (May 2011), pp. 2405–2410.

[GJB15] Remi Gribonval, Rodolphe Jenatton, and Francis Bach. "Sparse and spurious: Dictionary learning with noise and outliers". *IEEE transactions on information theory* 61.11 (Nov. 2015), pp. 6298–6319.

[Gro87] Stephen Grossberg. "Competitive Learning: From Interactive Activation to Adaptive Resonance". *Cogn. Sci.* 11 (1987), pp. 23–63.

[GD24] Albert Gu and Tri Dao. "Mamba: Linear-time sequence modeling with selective state spaces". *First conference on language modeling*. 2024.

[GDP+23] Xiangming Gu, Chao Du, Tianyu Pang, Chongxuan Li, Min Lin, and Ye Wang. "On memorization in diffusion models". *arXiv preprint arXiv:2310.02664* (2023).

[GYR+23] Yuwei Guo, Ceyuan Yang, Anyi Rao, Zhengyang Liang, Yaohui Wang, Yu Qiao, Maneesh Agrawala, Dahua Lin, and Bo Dai. "Animatediff: Animate your personalized text-to-image diffusion models without specific tuning". *arXiv preprint arXiv:2307.04725* (2023).

[HCL06] Raia Hadsell, Sumit Chopra, and Yann LeCun. "Dimensionality reduction by learning an invariant mapping". *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, CVPR 2006*. 2006, pp. 1735–1742.

[HY01] M.H. Hansen and B. Yu. "Model Selection and the Principle of Minimum Description Length". *Journal of American Statistical Association* 96 (2001), pp. 746–774.

[Har15] Yuval Noah Harari. *Sapiens: A Brief History of Humankind*. Harper, 2015.

[HTF09] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Second. Springer, 2009.

[Haw21] Jeff Hawkins. *A Thousand Brains: A New Theory of Intelligence*. Basic Books, 2021.

[HCX+22] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. "Masked autoencoders are scalable vision learners". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 16000–16009.

[HFW+19] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. "Momentum contrast for unsupervised visual representation learning". *arXiv preprint arXiv:1911.05722* (2019).

[HZR+16a] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep Residual Learning for Image Recognition". *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778.

[HZR+16b] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep residual learning for image recognition". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[Hea+56] Thomas Little Heath et al. *The thirteen books of Euclid's Elements*. Courier Corporation, 1956.

[HRU+17a] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium". *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017.

[HRU+17b] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. "Gans trained by a two time-scale update rule converge to a local nash equilibrium". *Advances in neural information processing systems* 30 (2017).

[HS06] G. E. Hinton and R. R. Salakhutdinov. "Reducing the Dimensionality of Data with Neural Networks". *Science* 313.5786 (2006), pp. 504–507. eprint: https://science.sciencemag.org/content/313/5786/504.full.pdf.

[HZ93] Geoffrey E. Hinton and Richard S. Zemel. "Autoencoders, minimum description length and Helmholtz free energy". *Proceedings of the 6th International Conference on Neural Information Processing Systems*. NIPS'93. Denver, Colorado: Morgan Kaufmann Publishers Inc., 1993, pp. 3–10.

[HJA20] Jonathan Ho, Ajay Jain, and Pieter Abbeel. "Denoising Diffusion Probabilistic Models". *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 6840–6851.

[HS21] Jonathan Ho and Tim Salimans. "Classifier-Free Diffusion Guidance". *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*. 2021.

[HS22a] Jonathan Ho and Tim Salimans. "Classifier-Free Diffusion Guidance". *arXiv [cs.LG]* (July 2022). arXiv: 2207.12598 [cs.LG].

[HS22b] Jonathan Ho and Tim Salimans. "Classifier-Free Diffusion Guidance". *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*. 2022.

[HS97] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-term Memory". *Neural computation* 9 (Dec. 1997), pp. 1735–80.

[HSD20] David Hong, Yue Sheng, and Edgar Dobriban. *Selecting the number of components in PCA via random signflips*. 2020. arXiv: 2012.02985 [math.ST].

[HZG+23] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. "Lrm: Large reconstruction model for single image to 3d". *arXiv preprint arXiv:2311.04400* (2023).

[Hot33] H. Hotelling. "Analysis of a Complex of Statistical Variables into Principal Components". *Journal of Educational Psychology* (1933).

[HZM+23] Cheng-Yu Hsieh, Jieyu Zhang, Zixian Ma, Aniruddha Kembhavi, and Ranjay Krishna. "SugarCrepe: Fixing Hackable Benchmarks for Vision-Language Compositionality". *Thirty-Seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 2023.

[HDZ+25] Binbin Huang, Haobin Duan, Yiqun Zhao, Zibo Zhao, Yi Ma, and Shenghua Gao. "CUPID: Generative 3D Reconstruction via Joint Object and Pose Modeling". *arXiv preprint arXiv:2510.20776* (2025).

[HYH+22] Chun-Hao P. Huang, Hongwei Yi, Markus Höschle, Matvey Safroshkin, Tsvetelina Alexiadis, Senya Polikovsky, Daniel Scharstein, and Michael J. Black. "Capturing and Inferring Dense Full-Body Human-Scene Contact". *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 13274–13285.

[HLV+17] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. "Densely Connected Convolutional Networks". *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 2261–2269.

[HM99] Jinggang Huang and David Mumford. "Statistics of natural images and models". *Proceedings. 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No PR00149)*. Vol. 1. IEEE. 1999, pp. 541–547.

[HW59] D.H. Hubel and T.N. Wiesel. "eceptive fields of single neurones in the cat's striate cortex". *J. Physiol.* 148.3 (1959), pp. 574–591.

[HCS+24] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. "Sparse Autoencoders Find Highly Interpretable Features in Language Models". *The Twelfth International Conference on Learning Representations*. 2024.

[HKV19] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren, eds. *Automatic Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[HKJ+21] Uiwon Hwang, Heeseung Kim, Dahuin Jung, Hyemi Jang, Hyungyu Lee, and Sungroh Yoon. "Stein Latent Optimization for Generative Adversarial Networks". *arXiv preprint arXiv:2106.05319* (2021).

[Hyv05] Aapo Hyvärinen. "Estimation of Non-Normalized Statistical Models by Score Matching". *Journal of Machine Learning Research* 6.24 (2005), pp. 695–709.

[HO97] Aapo Hyvärinen and Erkki Oja. "A Fast Fixed-Point Algorithm for Independent Component Analysis". *Neural Computation* 9.7 (1997), pp. 1483–1492.

[HO00a] Aapo Hyvärinen and Erkki Oja. "Independent Component Analysis: Algorithms and Applications". *Neural Networks* 13.4-5 (2000), pp. 411–430.

[HO00b] Aapo Hyvärinen and Erkki Oja. "Independent component analysis: algorithms and applications". *Neural Networks* 13.4 (2000), pp. 411–430.

[IWW+21] Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hannaneh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. *OpenCLIP*. Version 0.1. If you use this software, please cite it as below. July 2021.

[IS15] Sergey Ioffe and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". *ICML*. 2015, pp. 448–456.

[JGB+21] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. "Perceiver: General Perception with Iterative Attention". *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 4651–4664.

[JAD+21] Ajil Jalal, Marius Arvinte, Giannis Daras, Eric Price, Alexandros G Dimakis, and Jon Tamir. "Robust Compressed Sensing MRI with Deep Generative Priors". *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan. Vol. 34. Curran Associates, Inc., 2021, pp. 14938–14954.

[Jam15] G J O Jameson. "A simple proof of Stirling's formula for the gamma function". *The Mathematical Gazette* 99.544 (Mar. 2015), pp. 68–74.

[JRR+24] Yibo Jiang, Goutham Rajendran, Pradeep Kumar Ravikumar, Bryon Aragam, and Victor Veitch. "On the Origins of Linear Representations in Large Language Models". *International Conference on Machine Learning*. Vol. 235. PMLR, 2024, pp. 21879–21911.

[JJT+24] Haian Jin, Hanwen Jiang, Hao Tan, Kai Zhang, Sai Bi, Tianyuan Zhang, Fujun Luan, Noah Snavely, and Zexiang Xu. "Lvsm: A large view synthesis model with minimal 3d inductive bias". *arXiv preprint arXiv:2410.17242* (2024).

[Jol02] I. Jollife. *Principal Component Analysis*. 2nd. Springer-Verlag, 2002.

[Jol86] I. Jolliffe. *Principal Component Analysis*. New York, NY: Springer-Verlag, 1986.

[Jon82] Douglas Samuel Jones. *The theory of generalised functions*. Cambridge University Press, 1982.

[JJB+] Keller Jordan, Yuchen Jin, Vlado Boza, You Jiacheng, Franz Cecista, Laker Newhouse, and Jeremy Bernstein. "Muon: An optimizer for hidden layers in neural networks, 2024". *URL https://kellerjordan. github. io/posts/muon* 6 ().

[JT20] Sheena A. Josselyn and Susumu Tonegawa. "Memory engrams: Recalling the past and imagining the future". *Science* 367 (2020).

[KS21] Z Kadkhodaie and E P Simoncelli. "Stochastic solutions for linear inverse problems using the prior implicit in a denoiser". *Adv. Neural Information Processing Systems (NeurIPS)*. Ed. by M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan. Vol. 34. Curran Associates, Inc., 2021.

[Kal60] Rudolph Emil Kalman. "A new approach to linear filtering and prediction problems" (1960).

[KG24] Mason Kamb and Surya Ganguli. "An analytic theory of creativity in convolutional diffusion models". *arXiv preprint arXiv:2412.20292* (2024).

[KMH+20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. "Scaling Laws for Neural Language Models". *arXiv preprint arXiv:2001.08361* (2020).

[Kar22a] Andrej Karpathy. *nanoGPT*. https://github.com/karpathy/nanoGPT. 2022.

[Kar22b] Andrej Karpathy. *The spelled-out intro to neural networks and backpropagation: building micrograd*. YouTube. Aug. 16, 2022. URL: https://www.youtube.com/watch?v=VMj-3S1tku0 (visited on 08/17/2025).

[KK18] Ronald Kemker and Christopher Kanan. "FearNet: Brain-Inspired Model for Incremental Learning". *International Conference on Learning Representations*. 2018.

[KKL+23] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. "3D Gaussian splatting for real-time radiance field rendering." *ACM Trans. Graph.* 42.4 (2023), pp. 139–1.

[KB14] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". *arXiv preprint arXiv:1412.6980* (2014).

[KW13a] Diederik P Kingma and Max Welling. "Auto-Encoding Variational Bayes". *arXiv [stat.ML]* (Dec. 2013). arXiv: 1312.6114v11 [stat.ML].

[KW13b] Diederik P Kingma and Max Welling. "Auto-encoding variational bayes". *arXiv preprint arXiv:1312.6114* (2013).

[KW19] Diederik P Kingma and Max Welling. "An introduction to variational autoencoders". *Foundations and Trends® in Machine Learning* 12.4 (2019), pp. 307–392.

[KPR+17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. "Overcoming catastrophic forgetting in neural networks". *Proceedings of the national academy of sciences* 114.13 (2017), pp. 3521–3526.

[KUM+17] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. "Self-normalizing neural networks". *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 2017, pp. 972–981.

[Kli11] Ronald Kline. "Cybernetics, Automata Studies, and the Dartmouth Conference on Artificial Intelligence". *IEEE Annals of the History of Computing* 33.4 (2011), pp. 5–16.

[KTV18] Artemy Kolchinsky, Brendan D Tracey, and Steven Van Kuyk. "Caveats for information bottleneck in deterministic scenarios". *arXiv preprint arXiv:1808.07593* (2018).

[Kol98] Andrei N. Kolmogorov. "On Tables of Random Numbers (Reprinted from "Sankhya: The Indian Journal of Statistics", Series A, Vol. 25 Part 4, 1963)". *Theor. Comput. Sci.* 207 (1998), pp. 387–395.

[KS12] Irwin Kra and Santiago R Simanca. "On Circulant Matrices". *Notices of the American Mathematical Society* 59 (2012), pp. 368–377.

[Kra91] Mark A Kramer. "Nonlinear principal component analysis using autoassociative neural networks". *AIChE Journal* 37.2 (1991), pp. 233–243.

[KH+09] Alex Krizhevsky, Geoffrey Hinton, et al. "Learning multiple layers of features from tiny images" (2009).

[KNH14] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. "The CIFAR-10 dataset". *online: http://www.cs.toronto.edu/kriz/cifar.html* 55 (2014).

[KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "Imagenet classification with deep convolutional neural networks". *Advances in neural information processing systems*. 2012, pp. 1097–1105.

[KZZ+23] Nupur Kumari, Bingliang Zhang, Richard Zhang, Eli Shechtman, and Jun-Yan Zhu. "Multi-concept customization of text-to-image diffusion". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 1931–1941.

[Lab24] Black Forest Labs. *FLUX*. https://github.com/black-forest-labs/flux. 2024.

[LBB+25] Black Forest Labs, Stephen Batifol, Andreas Blattmann, Frederic Boesel, Saksham Consul, Cyril Diagne, Tim Dockhorn, Jack English, Zion English, Patrick Esser, Sumith Kulal, Kyle Lacey, Yam Levi, Cheng Li, Dominik Lorenz, Jonas Müller, Dustin Podell, Robin Rombach, Harry Saini, Axel Sauer, and Luke Smith. "FLUX.1 Kontext: Flow matching for in-context image generation and editing in latent space". *arXiv [cs.GR]* (June 2025). arXiv: 2506.15742 [cs.GR].

[LRM+12] Quoc V. Le, Marc'Aurelio Ranzato, Rajat Monga, Matthieu Devin, Kai Chen, Greg S. Corrado, Jeff Dean, and Andrew Y. Ng. "Building high-level features using large scale unsupervised learning". *Proceedings of the 29th International Coference on International Conference on Machine Learning*. ICML'12. Edinburgh, Scotland: Omnipress, 2012, pp. 507–514.

[LBD+89] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. "Backpropagation Applied to Handwritten Zip Code Recognition". *Neural Computation* 1.4 (1989), pp. 541–551.

[LBB+98a] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-based learning applied to document recognition". *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[LBB+98b] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. "Gradient-based learning applied to document recognition". *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[LCH+06] Yann LeCun, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, and Fu-Jie Huang. "A Tutorial on Energy-Based Learning". Jan. 2006.

[LPM03] Ann Lee, Kim Pedersen, and David Mumford. "The Nonlinear Statistics of High-Contrast Patches in Natural Images". *International Journal of Computer Vision* 54 (Aug. 2003).

[LSJ+16] Jason D Lee, Max Simchowitz, Michael I Jordan, and Benjamin Recht. "Gradient descent only converges to minimizers". *Conference on learning theory*. PMLR. 2016, pp. 1246–1257.

[Lee02] John M. Lee. "Introduction to Smooth Manifolds". 2002.

[LMH+18] Jaakko Lehtinen, Jacob Munkberg, Jon Hasselgren, Samuli Laine, Tero Karras, Miika Aittala, and Timo Aila. "Noise2Noise: Learning Image Restoration without Clean Data". *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, July 2018, pp. 2965–2974.

[LY24] Gen Li and Yuling Yan. "O (d/T) convergence theory for diffusion probabilistic models under minimal assumptions". *arXiv preprint arXiv:2409.18959* (2024).

[LFD+22] Haochuan Li, Farzan Farnia, Subhro Das, and Ali Jadbabaie. "On convergence of gradient descent ascent: A tight local analysis". *International Conference on Machine Learning*. PMLR. 2022, pp. 12717–12740.

[Li17] Xi-Lin Li. "Preconditioned stochastic gradient descent". *IEEE transactions on neural networks and learning systems* 29.5 (2017), pp. 1454–1466.

[LTP25] Qinyu Li, Yee Whye Teh, and Razvan Pascanu. "NoProp: Training Neural Networks without Back-propagation or Forward-propagation". *arXiv preprint arXiv:2503.24322* (2025).

[LH25] Tianhong Li and Kaiming He. "Back to basics: Let denoising generative models denoise". *arXiv preprint arXiv:2511.13720* (2025).

[LZQ24] Wenda Li, Huijie Zhang, and Qing Qu. "Shallow diffuse: Robust and invisible watermarking through low-dimensional subspaces in diffusion models". *arXiv preprint arXiv:2410.21088* (2024).

[LWQ25] Xiang Li, Rongrong Wang, and Qing Qu. "Towards Understanding the Mechanisms of Classifier-Free Guidance". *arXiv [cs.CV]* (May 2025). arXiv: 2505.19210 [cs.CV].

[LB19] Yanjun Li and Yoram Bresler. "Multichannel sparse blind deconvolution on the sphere". *IEEE Transactions on Information Theory* 65.11 (2019), pp. 7415–7436.

[LCW+24] Zhen Li, Mingdeng Cao, Xintao Wang, Zhongang Qi, Ming-Ming Cheng, and Ying Shan. "Photomaker: Customizing realistic human photos via stacked id embedding". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2024, pp. 8640–8650.

[LRZ+12] Xiao Liang, Xiang Ren, Zhengdong Zhang, and Yi Ma. "Repairing Sparse Low-Rank Texture". *Computer Vision – ECCV 2012*. Ed. by Andrew Fitzgibbon, Svetlana Lazebnik, Pietro Perona, Yoichi Sato, and Cordelia Schmid. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 482–495.

[LMB+14] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. "Microsoft coco: Common objects in context". *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*. Springer. 2014, pp. 740–755.

[LZ94] T Linder and R Zamir. "On the asymptotic tightness of the Shannon lower bound". *IEEE transactions on information theory* 40.6 (1994), pp. 2026–2031.

[Lin88] R. Linsker. "Self-organization in a perceptual network". *Computer* 21.3 (1988), pp. 105–117.

[LCB+23] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. "Flow Matching for Generative Modeling". *The Eleventh International Conference on Learning Representations*. 2023.

[LMR17] Anna V. Little, Mauro Maggioni, and Lorenzo Rosasco. "Multiscale geometric methods for data sets I: Multiscale SVD, noise and curvature". *Applied and Computational Harmonic Analysis* 43.3 (2017), pp. 504–567.

[LMZ+24] Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. "Infini-gram: Scaling unbounded n-gram language models to a trillion tokens". *arXiv preprint arXiv:2401.17377* (2024).

[LSY+25] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. "Muon is Scalable for LLM Training". *arXiv preprint arXiv:2502.16982* (2025).

[LV09] Z. Liu and L. Vandenberghe. "Semidefinite programming methods for system realization and identification". *Proceedings of the 48h IEEE Conference on Decision and Control (CDC) held jointly with 2009 28th Chinese Control Conference*. 2009, pp. 4676–4681.

[LV10] Zhang. Liu and Lieven. Vandenberghe. “Interior-Point Method for Nuclear Norm Approximation with Application to System Identification”. *SIAM Journal on Matrix Analysis and Applications* 31.3 (2010), pp. 1235–1256. eprint: https://doi.org/10.1137/090755436.

[LMR+15] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J Black. “SMPL: A skinned multi-person linear model”. *ACM Transactions on Graphics (TOG)* 34.6 (2015), pp. 1–16.

[LC98] William E. Lorensen and Harvey E. Cline. “Marching cubes: a high resolution 3D surface construction algorithm”. *Seminal Graphics: Pioneering Efforts That Shaped the Field, Volume 1*. New York, NY, USA: Association for Computing Machinery, 1998, pp. 347–353.

[LH17] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. *International Conference on Learning Representations*. 2017.

[LH19] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. *arXiv preprint arXiv:1711.05101* (2019).

[MYP+10] M. Journée, Y. Nesterov, P. Richtárik, and R. Sepulchre. “Generalized power method for sparse principal component analysis”. *Journal of Machine Learning Research* 11 (2010), pp. 517–553.

[MDH+07a] Y. Ma, H. Derksen, W. Hong, and J. Wright. “Segmentation of multivariate mixed data via lossy coding and compression”. *To appear in IEEE Transactions on Pattern Analysis and Machine Intelligence* (2007).

[MKS+04] Y. Ma, J. Košecká, S. Soatto, and S. Sastry. *An Invitation to 3-D Vision, From Images to Models*. New York: Springer-Verlag, 2004.

[MDH+07b] Yi Ma, Harm Derksen, Wei Hong, and John Wright. “Segmentation of multivariate mixed data via lossy data coding and compression”. *IEEE transactions on pattern analysis and machine intelligence* 29.9 (2007), pp. 1546–1562.

[MHN13] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. “Rectifier nonlinearities improve neural network acoustic models”. *Proc. ICML*. Vol. 30. Citeseer. 2013, p. 3.

[MGT+19] Naureen Mahmood, Nima Ghorbani, Nikolaus F Troje, Gerard Pons-Moll, and Michael J Black. “AMASS: Archive of motion capture as surface shapes”. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 5442–5451.

[MBP14] Julien Mairal, Francis Bach, and Jean Ponce. "Sparse Modeling for Image and Vision Processing". *Foundations and Trends® in Computer Graphics and Vision* 8.2-3 (2014), pp. 85–283.

[MSM93] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. "Building a Large Annotated Corpus of English: The Penn T reebank". *Computational Linguistics* 19.2 (1993). Ed. by Julia Hirschberg, pp. 313–330.

[Mar06] Andrei Andreevich Markov. "An example of statistical investigation of the text Eugene Onegin concerning the connection of samples in chains". *Science in Context* 19.4 (2006), pp. 591–600.

[MBD+21] James Martens, Andy Ballard, Guillaume Desjardins, Grzegorz Swirszcz, Valentin Dalibard, Jascha Sohl-Dickstein, and Samuel S Schoenholz. "Rapid training of deep neural networks without skip connections or normalization layers using Deep Kernel Shaping". *arXiv [cs.LG]* (Oct. 2021). arXiv: 2110.01765 [cs.LG].

[MS19] David McAllester and Karl Stratos. *Formal Limitations on the Measurement of Mutual Information*. 2019.

[MMR+06] John McCarthy, Marvin L. Minsky, Nathaniel Rochester, and Claude E. Shannon. "A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence: August 31, 1955". *AI Mag.* 27.4 (Dec. 2006), pp. 12–14.

[MC89] Michael McCloskey and Neal J Cohen. "Catastrophic interference in connectionist networks: The sequential learning problem". *Psychology of learning and motivation*. Vol. 24. Elsevier, 1989, pp. 109–165.

[MP43] Warren McCulloch and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity". *Bulletin of Mathematical Biophysics* 5 (1943), pp. 115–133.

[MM70] Jerry M. Mendel and Robert W. Mclaren. "Reinforcement-learning control and pattern recognition systems". *In Mendel, J. M. and Fu, K. S., editors, Adaptive, Learning and Pattern Recognition Systems: Theory and Applications* (1970), pp. 287–318.

[MXB+16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. *Pointer Sentinel Mixture Models*. 2016. arXiv: 1609.07843 [cs.CL].

[MM12] Stephan Mertens and Cristopher Moore. "Continuum percolation thresholds in two dimensions". *Phys. Rev. E* 86 (6 Dec. 2012), p. 061109.

[MON+19] Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. "Occupancy Networks: Learning 3D Reconstruction in Function Space". *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 4455–4465.

[MSS+22] Beren Millidge, Tommaso Salvatori, Yuhang Song, Rafal Bogacz, and Thomas Lukasiewicz. "Predictive coding: Towards a future of deep learning beyond backpropagation?" *arXiv preprint arXiv:2202.09467* (2022).

[Min54] Marvin Minsky. "Theory of Neural-Analog Reinforcement Systems and its Application to the Brain-Model Problem". PhD thesis. Princeton Univerity, 1954.

[MP69] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. the MIT Press, 1969.

[Mir60] L Mirsky. "SYMMETRIC GAUGE FUNCTIONS AND UNITARILY INVARIANT NORMS". *The Quarterly Journal of Mathematics* 11.1 (Jan. 1960), pp. 50–59.

[MCS+22] Paritosh Mittal, Yen-Chi Cheng, Maneesh Singh, and Shubham Tulsiani. "Autosdf: Shape priors for 3d completion, reconstruction and generation". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2022, pp. 306–315.

[Miy61] K. Miyasawa. "An empirical bayes estimator of the mean of a normal population". *Bull. Inst. Internat. Statist.* 38 (1961).

[MKK+18] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. "Spectral normalization for generative adversarial networks". *arXiv preprint arXiv:1802.05957* (2018).

[MRY+11] Hossein Mobahi, Shankar Rao, Allen Yang, Shankar Sastry, and Yi Ma. "Segmentation of Natural Images by Texture and Boundary Compression". *the International Journal of Computer Vision* 95.1 (2011), pp. 86–98.

[MLE19] Vishal Monga, Yuelong Li, and Yonina C Eldar. "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing". *arXiv preprint arXiv:1912.10557* (2019).

[MÁM+24] Edmund RR Moody, Sandra Álvarez-Carretero, Tara A Mahendrarajah, James W Clark, Holly C Betts, Nina Dombrowski, Lénárd L Szánthó, Richard A Boyle, Stuart Daines, Xi Chen, et al. "The nature of the last universal common ancestor and its impact on the early Earth system". *Nature Ecology & Evolution* 8.9 (2024), pp. 1654–1666.

[MKW+22] Norman Mu, Alexander Kirillov, David Wagner, and Saining Xie. "SLIP: Self-supervision Meets Language-Image Pre-training". *Computer Vision – ECCV 2022*. Ed. by Shai Avidan, Gabriel Brostow, Moustapha Cissé, Giovanni Maria Farinella, and Tal Hassner. Cham: Springer Nature Switzerland, 2022, pp. 529–544.

[MKH19] Rafael Müller, Simon Kornblith, and Geoffrey E Hinton. "When does label smoothing help?" *Advances in neural information processing systems* 32 (2019).

[Mum96] David Mumford. "The Statistical Description of Visual Signals". 1996.

[MG99] David Mumford and Basilis Gidas. "Stochastic Models for Generic Images". *Quarterly of Applied Mathematics* 59 (July 1999).

[MK07] Joseph F Murray and Kenneth Kreutz-Delgado. "Learning sparse overcomplete codes for images". *The Journal of VLSI Signal Processing Systems for Signal Image and Video Technology* 46.1 (Mar. 2007), pp. 1–13.

[MLS94] R. Murray, Zexiang Li, and S. Sastry. *A Mathematical Introduction to Robotic Manipulation*. Florida: CRC Press, 1994.

[NDE+13] S. Nam, M.E. Davies, M. Elad, and R. Gribonval. "The cosparse analysis model and algorithms". *Applied and Computational Harmonic Analysis* 34.1 (2013), pp. 30–56.

[NGE+20] Charlie Nash, Yaroslav Ganin, SM Ali Eslami, and Peter Battaglia. "Polygen: An autoregressive generative model of 3d meshes". *International conference on machine learning*. PMLR. 2020, pp. 7220–7229.

[NMR44] John von Neumann, Oskar Morgenstern, and Ariel Rubinstein. *Theory of Games and Economic Behavior (60th Anniversary Commemorative Edition)*. Princeton University Press, 1944.

[Neu58] John von Neumann. *The computer and the brain*. USA: Yale University Press, 1958.

[NJD+22] Alex Nichol, Heewoo Jun, Prafulla Dhariwal, Pamela Mishkin, and Mark Chen. "Point-e: A system for generating 3d point clouds from complex prompts". *arXiv preprint arXiv:2212.08751* (2022).

[ND21] Alexander Quinn Nichol and Prafulla Dhariwal. "Improved Denoising Diffusion Probabilistic Models". *International Conference on Machine Learning (ICML)*. 2021.

[NZM+24] Matthew Niedoba, Berend Zwartsenberg, Kevin Murphy, and Frank Wood. "Towards a Mechanistic Explanation of Diffusion Model Generalization". *arXiv preprint arXiv:2411.19339* (2024).

[NMM19] Oliver Nina, Jamison Moody, and Clarissa Milligan. "A Decoder-Free Approach for Unsupervised Clustering and Manifold Learning with Random Triplet Mining". *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*. IEEE Computer Society. 2019, pp. 3987–3994.

[NW06] Jorge Nocedal and Stephen Wright. *Numerical optimization*. Springer Science & Business Media, 2006.

[NIG+18] Chigozie Nwankpa, Winifred Ijomah, Anthony Gachagan, and Stephen Marshall. "Activation functions: Comparison of trends in practice and research for deep learning". *arXiv preprint arXiv:1811.03378* (2018).

[Oja82] Erkki Oja. "A simplified neuron model as a principal component analyzer". *Journal of Mathematical Biology* 15 (1982), pp. 267–273.

[OLC+25] Kazusato Oko, Licong Lin, Yuhang Cai, and Song Mei. "A statistical theory of contrastive pre-training and multimodal generative AI". *arXiv [cs.LG]* (Jan. 2025). arXiv: 2501.04641 [cs.LG].

[OF97] B A Olshausen and D J Field. "Sparse coding with an overcomplete basis set: a strategy employed by V1?" *Vision research* 37.23 (Dec. 1997), pp. 3311–3325.

[OF96] Bruno A Olshausen and David J Field. "Emergence of simple-cell receptive field properties by learning a sparse code for natural images". *Nature* 381 (June 1996), p. 607.

[OLV18] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. "Representation Learning with Contrastive Predictive Coding". *arXiv [cs.LG]* (July 2018). arXiv: 1807.03748 [cs.LG].

[OVK17] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. "Neural discrete representation learning". *arXiv [cs.LG]* (Nov. 2017). arXiv: 1711.00937 [cs.LG].

[Ope24] OpenAI. *Sora: Creating video from text*. https://openai.com/sora. 2024.

[ODM+23] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. "Dinov2: Learning robust visual features without supervision". *arXiv preprint arXiv:2304.07193* (2023).

[ODM+24] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. *DINOv2: Learning Robust Visual Features without Supervision*. 2024. arXiv: 2304.07193 [cs.CV].

[PBW+24] Druv Pai, Sam Buchanan, Ziyang Wu, Yaodong Yu, and Yi Ma. "Masked Completion via Structured Diffusion with White-Box Transformers". *The Twelfth International Conference on Learning Representations*. 2024.

[PPC+23] Druv Pai, Michael Psenka, Chih-Yuan Chiu, Manxi Wu, Edgar Dobriban, and Yi Ma. "Pursuit of a discriminative representation for multiple subspaces via sequential games". *Journal of the Franklin Institute* 360.6 (2023), pp. 4135–4171.

[PCY+23] Xiaqing Pan, Nicholas Charron, Yongqian Yang, Scott Peters, Thomas Whelan, Chen Kong, Omkar Parkhi, Richard Newcombe, and Yuheng Carl Ren. "Aria Digital Twin: A New Benchmark Dataset for Egocentric 3D Machine Perception". *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 20133–20143.

[PKL+16] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. *The LAMBADA dataset: Word prediction requiring a broad discourse context*. 2016. arXiv: 1606.06031 [cs.CL].

[PHD20] Vardan Papyan, XY Han, and David L Donoho. "Prevalence of Neural Collapse during the terminal phase of deep learning training". *arXiv preprint arXiv:2008.08186* (2020).

[PRE17] Vardan Papyan, Yaniv Romano, and Michael Elad. "Convolutional neural networks analyzed via convolutional sparse coding". *The Journal of Machine Learning Research* 18.1 (2017), pp. 2887–2938.

[PCV24] Kiho Park, Yo Joong Choe, and Victor Veitch. "The Linear Representation Hypothesis and the Geometry of Large Language Models". *International Conference on Machine Learning*. PMLR. 2024, pp. 39643–39666.

[Par04] Andrew Parker. *In The Blink Of An Eye: How Vision Sparked The Big Bang Of Evolution*. Basic Books, 2004.

[PSR+24] Georgios Pavlakos, Dandan Shan, Ilija Radosavovic, Angjoo Kanazawa, David Fouhey, and Jitendra Malik. "Reconstructing Hands in 3D with Transformers". *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024.

[Pea09] Judea Pearl. *Causality: Models, Reasoning and Inference*. Cambridge University Press, 2009.

[Pea01] K. Pearson. "On Lines and Planes of Closest Fit to Systems of Points in Space". *Philosophical Magazine* 2.6 (1901), pp. 559–572.

[PX23] William Peebles and Saining Xie. "Scalable diffusion models with transformers". *Proceedings of the IEEE/CVF international conference on computer vision*. 2023, pp. 4195–4205.

[PV25] Liangzu Peng and René Vidal. "Mathematics of continual learning". *arXiv preprint arXiv:2504.17963* (2025).

[PNM+20] Songyou Peng, Michael Niemeyer, Lars Mescheder, Marc Pollefeys, and Andreas Geiger. "Convolutional Occupancy Networks". *Computer Vision – ECCV 2020*. Ed. by Andrea Vedaldi, Horst Bischof, Thomas Brox, and Jan-Michael Frahm. Cham: Springer International Publishing, 2020, pp. 523–540.

[PSV+18] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. "FiLM: Visual Reasoning with a General Conditioning Layer". *AAAI Conference on Artificial Intelligence*. 2018.

[PRR+22] Ed Pizzi, Sreya Dutta Roy, Sugosh Nagavara Ravindra, Priya Goyal, and Matthijs Douze. "A self-supervised descriptor for image copy detection". *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 14532–14542.

[Pla99] S. E. Plamer. *Vision Science: Photons to Phenomenology*. The MIT Press, 1999.

[PEL+23] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. "Sdxl: Improving latent diffusion models for high-resolution image synthesis". *arXiv preprint arXiv:2307.01952* (2023).

[PW22] Yuri Poliyanski and Yihong Wu. *Information Theory: From Coding to Learning*. Cambridge University Press, 2022.

[PPR+24] Michael Psenka, Druv Pai, Vishal Raman, Shankar Sastry, and Yi Ma. "Representation Learning via Manifold Flattening and Reconstruction". *Journal of Machine Learning Research* 25.132 (2024), pp. 1–47.

[QSM+17] Charles R Qi, Hao Su, Kaichun Mo, and Leonidas J Guibas. "Pointnet: Deep learning on point sets for 3d classification and segmentation". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 652–660.

[QYS+17] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. "PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space". *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017.

[QLZ19] Qing Qu, Xiao Li, and Zhihui Zhu. "A nonconvex approach for exact and efficient multichannel sparse blind deconvolution". *Advances in Neural Information Processing Systems*. 2019, pp. 4017–4028.

[QLZ20] Qing Qu, Xiao Li, and Zhihui Zhu. "Exact Recovery of Multichannel Sparse Blind Deconvolution via Gradient Descent". *SIAM Journal on Imaging Sciences* 13.3 (2020), pp. 1630–1652.

[QZL+20a] Qing Qu, Yuexiang Zhai, Xiao Li, Yuqian Zhang, and Zhihui Zhu. "Geometric Analysis of Nonconvex Optimization Landscapes for Overcomplete Learning". *International Conference on Learning Representations*. 2020.

[QZL+20b] Qing Qu, Zhihui Zhu, Xiao Li, Manolis C. Tsakiris, John Wright, and René Vidal. *Finding the Sparsest Vectors in a Subspace: Theory, Algorithms, and Applications*. 2020. arXiv: 2001.06970 [cs.LG].

[RD03] R. Basri and D. Jacobs. "Lambertian reflectance and linear subspaces". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 25.2 (2003), pp. 218–233.

[RKH+21a] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. "Learning Transferable Visual Models From Natural L anguage Supervision". *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 8748–8763.

[RKH+21b] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. "Learning Transferable Visual Models From Natural Language Supervision". *International Conference on Machine Learning (ICML)*. 2021.

[RKH+21c] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. "Learning Transferable Visual Models From Natural

Language Supervision". *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 2021, pp. 8748–8763.

[RMC16] Alec Radford, Luke Metz, and Soumith Chintala. "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks". *arXiv preprint arXiv:1511.06434* (2016). arXiv: 1511.06434 [cs.LG].

[RWC+19] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. "Language models are unsupervised multitask learners". *OpenAI blog* 1.8 (2019), p. 9.

[RSR+20] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. "Exploring the limits of transfer learning with a unified text-to-text transformer". *Journal of machine learning research* 21.140 (2020), pp. 1–67.

[RDN+22] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. "Hierarchical Text-Conditional Image Generation with CLIP Latents". *arXiv [cs.CV]* (Apr. 2022). arXiv: 2204.06125 [cs.CV].

[RPC+06] Marc'Aurelio Ranzato, Christopher Poultney, Sumit Chopra, and Yann Cun. "Efficient Learning of Sparse Representations with an Energy-Based Model". *Advances in Neural Information Processing Systems*. Ed. by B Schölkopf, J Platt, and T Hoffman. Vol. 19. MIT Press, 2006.

[RS11] M Raphan and E P Simoncelli. "Least squares estimation without priors or supervision". *Neural Computation* 23.2 (2011). Published online, Nov 2010., pp. 374–420.

[RKS+17] Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. "iCaRL: Incremental classifier and representation learning". *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*. 2017, pp. 2001–2010.

[RAG+24] Lucas Relic, Roberto Azevedo, Markus Gross, and Christopher Schroers. "Lossy image compression with foundation diffusion models". *European Conference on Computer Vision*. Springer. 2024, pp. 303–319.

[RBK18] Erwin Riegler, Helmut Bolcskei, and Gunther Koliander. "Rate-distortion theory for general sets and measures". *2018 IEEE International Symposium on Information Theory (ISIT)*. IEEE, June 2018, pp. 101–105.

[RKB23] Erwin Riegler, Günther Koliander, and Helmut Bölcskei. "Lossy compression of general random variables". *Information and inference: a journal of the IMA* 12.3 (Apr. 2023), pp. 1759–1829.

[Ris78] J. Rissanen. “Paper: Modeling by shortest data description”. *Automatica* 14.5 (1978), pp. 465–471.

[Rob56] Herbert E. Robbins. “An Empirical Bayes Approach to Statistics”. 1956.

[RBL+22] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. “High-Resolution Image Synthesis with Latent Diffusion M odels”. *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 2022, pp. 10674–10685.

[RTB17] Javier Romero, Dimitrios Tzionas, and Michael J Black. “Embodied hands: modeling and capturing hands and bodies together”. *ACM Transactions on Graphics (TOG)* 36.6 (2017), pp. 1–17.

[RFB15a] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*. Springer, 2015, pp. 234–241.

[RFB15b] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-net: Convolutional networks for biomedical image segmentation”. *International Conference on Medical image computing and computer-assisted intervention*. Springer. 2015, pp. 234–241.

[Ros57] Frank Rosenblatt. *The perceptron: A perceiving and recognizing automaton*. Report. Project PARA, Cornell Aeronautical Laboratory, Jan. 1957.

[RRD+23] Litu Rout, Negin Raoof, Giannis Daras, Constantine Caramanis, Alex Dimakis, and Sanjay Shakkottai. “Solving Linear Inverse Problems Provably via Posterior Sampling with Latent Diffusion Models”. *Thirty-seventh Conference on Neural Information Processing Systems*. 2023.

[Row97] Sam T. Roweis. “EM Algorithms for PCA and SPCA”. *Advances in Neural Information Processing Systems 10, [NIPS Conference, Denver, Colorado, USA, 1997]*. The MIT Press, 1997, pp. 626–632.

[RAL+24] François Rozet, Gérôme Andry, Francois Lanusse, and Gilles Louppe. “Learning Diffusion Priors from Observations by Expectation Maximization”. *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. 2024.

[RE14] R. Rubinstein and M. Elad. “Dictionary Learning for Analysis-Synthesis Thresholding”. *IEEE Transactions on Signal Processing* 62.22 (2014), pp. 5962–5972.

[RLJ+23] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. "Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 22500–22510.

[RHW86a] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. "Learning internal representations by error propagation". *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1: Foundations*. Cambridge, MA, USA: MIT Press, 1986, pp. 318–362.

[RHW86b] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. "Learning representations by back-propagating errors". *Nature* 323.6088 (Oct. 1986), pp. 533–536.

[SCS+22] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L. Denton, Seyed Kamyar Seyed Ghasemipour, Raphael Gontijo Lopes, Burcu Karagol Ayan, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. "Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding". *NeurIPS*. 2022.

[SGZ+16] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, Xi Chen, and Xi Chen. "Improved Techniques for Training GANs". *Advances in Neural Information Processing Systems*. Ed. by D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett. Vol. 29. Curran Associates, Inc., 2016.

[Sas99] Shankar Sastry. *Nonlinear Systems: Analysis, Stability, and Control*. Springer, 1999.

[SDL+25] Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J Reddi. "Reasoning with latent thoughts: On the power of looped transformers". *arXiv preprint arXiv:2502.17416* (2025).

[SHT+25] Fabian Schaipp, Alexander Hägele, Adrien Taylor, Umut Simsekli, and Francis Bach. "The Surprising Agreement Between Convex Optimization Theory and Learning-Rate Scheduling for Large Model Training". *arXiv preprint arXiv:2501.18965* (2025).

[SMB10] Dominik Scherer, Andreas Müller, and Sven Behnke. "Evaluation of pooling operations in convolutional architectures for object recognition". *International conference on artificial neural networks*. Springer. 2010, pp. 92–101.

[Sch14] Denise Schmandt-Besserat. "The evolution of writing". *International encyclopedia of social and behavioral sciences* (2014), pp. 1–15.

[SBV+22] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. *LAION-5B: An open large-scale dataset for training next generation image-text models*. 2022. arXiv: 2210.08402 [cs.CV].

[SBZ+25] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. "Flashattention-3: Fast and accurate attention with asynchrony and low-precision". *Advances in Neural Information Processing Systems* 37 (2025), pp. 68658–68685.

[Sha48] C. E. Shannon. "A mathematical theory of communication". *The Bell System Technical Journal* 27.3 (1948), pp. 379–423.

[Sha59] Claude E Shannon. "Coding theorems for a discrete source with a fidelity criterion". *IRE Nat. Conv. Rec* 4.142-163 (1959), p. 1.

[SMM+17] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". *ICLR*. 2017.

[SPX+22] Zifan Shi, Sida Peng, Yinghao Xu, Andreas Geiger, Yiyi Liao, and Yujun Shen. "Deep generative models on 3d representations: A survey". *arXiv preprint arXiv:2210.15663* (2022).

[SZ14] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". *arXiv preprint arXiv:1409.1556* (2014).

[SZ15] Karen Simonyan and Andrew Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition". *International Conference on Learning Representations*. 2015.

[SWM+15] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. "Deep Unsupervised Learning using Nonequilibrium Thermodynamics". *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, 2015, pp. 2256–2265.

[SKZ+24] Bowen Song, Soo Min Kwon, Zecheng Zhang, Xinyu Hu, Qing Qu, and Liyue Shen. "Solving Inverse Problems with Latent Diffusion Models via Hard Data Consistency". *The Twelfth International Conference on Learning Representations*. 2024.

[SME20] Jiaming Song, Chenlin Meng, and Stefano Ermon. "Denoising diffusion implicit models". *arXiv preprint arXiv:2010.02502* (2020).

[SKC+25] Kiwhan Song, Jaeyeon Kim, Sitan Chen, Yilun Du, Sham Kakade, and Vincent Sitzmann. "Selective Underfitting in Diffusion Models". *arXiv preprint arXiv:2510.01378* (2025).

[SE19] Yang Song and Stefano Ermon. "Generative Modeling by Estimating Gradients of the Data Distribution". *Advances in Neural Information Processing Systems*. Ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d 'Alché-Buc, E. Fox, and R. Garnett. Vol. 32. Curran Associates, Inc., 2019.

[SSX+22] Yang Song, Liyue Shen, Lei Xing, and Stefano Ermon. "Solving Inverse Problems in Medical Imaging with Score-Based Generative Models". *International Conference on Learning Representations*. 2022.

[SSK+21] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. "Score-Based Generative Modeling through Stochastic Differential Equations". *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021.

[SWW12] Daniel A Spielman, Huan Wang, and John Wright. "Exact Recovery of Sparsely-Used Dictionaries". *Proceedings of the 25th Annual Conference on Learning Theory*. Ed. by Shie Mannor, Nathan Srebro, and Robert C Williamson. Vol. 23. Proceedings of Machine Learning Research. Edinburgh, Scotland: PMLR, 2012, pp. 37.1–37.18.

[SHK+14] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. "Dropout: a simple way to prevent neural networks from overfitting". *The journal of machine learning research* 15.1 (2014), pp. 1929–1958.

[SQW15] Ju Sun, Qing Qu, and John Wright. "When are nonconvex problems not scary?" *arXiv preprint arXiv:1510.06096* (2015).

[SQW17a] Ju Sun, Qing Qu, and John Wright. "Complete Dictionary Recovery Over the Sphere I: Overview and the Geometric Picture". *IEEE Transactions on Information Theory* 63.2 (2017), pp. 853–884.

[SQW17b] Ju Sun, Qing Qu, and John Wright. "Complete dictionary recovery over the sphere I: Overview and the geometric picture". *IEEE Transactions on Information Theory* 63.2 (2017), pp. 853–884.

[SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.

[SLJ+14] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott E. Reed, Dragomir Anguelov, D. Erhan, Vincent Vanhoucke, and Andrew Rabinovich. "Going deeper with convolutions". *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2014), pp. 1–9.

[TZL+25] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, et al. "Kimi Linear: An Expressive, Efficient Attention Architecture". *arXiv preprint arXiv:2510.26692* (2025).

[Tea] Moonshot Team.

[Tel16] Matus Telgarsky. "Benefits of depth in neural networks". *29th Annual Conference on Learning Theory*. Ed. by Vitaly Feldman, Alexander Rakhlin, and Ohad Shamir. Vol. 49. Proceedings of Machine Learning Research. Columbia University, New York, New York, USA: PMLR, 2016, pp. 1517–1539.

[Til15] Andreas M Tillmann. "On the computational intractability of exact and approximate dictionary learning". *IEEE signal processing letters* 22.1 (Jan. 2015), pp. 45–49.

[TB99] M. Tipping and C. Bishop. "Probabilistic principal component analysis". *Journal of Royal Statistical Society: Series B* 61.3 (1999), pp. 611–622.

[TZ15] Naftali Tishby and Noga Zaslavsky. "Deep learning and the information bottleneck principle". *2015 IEEE Information Theory Workshop (ITW)*. IEEE. 2015, pp. 1–5.

[TDC+24] Shengbang Tong, Xili Dai, Yubei Chen, Mingyang Li, ZENGYI LI, Brent Yi, Yann LeCun, and Yi Ma. "Unsupervised Learning of Structured Representation via Closed-Loop Transcription". *Conference on Parsimony and Learning*. Ed. by Yuejie Chi, Gintare Karolina Dziugaite, Qing Qu, Atlas Wang Wang, and Zhihui Zhu. Vol. 234. Proceedings of Machine Learning Research. PMLR, Jan. 2024, pp. 440–457.

[TDW+23] Shengbang Tong, Xili Dai, Ziyang Wu, Mingyang Li, Brent Yi, and Yi Ma. "Incremental Learning of Structured Memory via Closed-Loop Transcription". *The Eleventh International Conference on Learning Representations*. 2023.

[TCD+20] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. "Training data-efficient image transformers & distillation through attention. arXiv 2020". *arXiv preprint arXiv:2012.12877* 2.3 (2020).

[Tu07] Zhuowen Tu. "Learning Generative Models via Discriminative Approaches". *2007 IEEE Conference on Computer Vision and Pattern Recognition*. 2007, pp. 1–8.

[Tur50] Alan Turing. “Computing Machinary and Intelligence”. *Mind* 59 (1950), pp. 433–460.

[Tur36] Alan M. Turing. “On Computable Numbers, with an Application to the E ntscheidungsproblem”. *Proceedings of the London Mathematical Society* 2.42 (1936), pp. 230–265.

[UVL16] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. “Instance normalization: The missing ingredient for fast stylization”. *arXiv preprint arXiv:1607.08022* (2016).

[Ume02] Shinji Umeyama. “Least-squares estimation of transformation parameters between two point patterns”. *IEEE Transactions on pattern analysis and machine intelligence* 13.4 (2002), pp. 376–380.

[VM96] P. Van Overschee and B. de Moor. *Subspace Identification for Linear Systems*. Kluwer Academic, 1996.

[VSP+17a] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is All you Need”. *Advances in Neural Information Processing Systems*. Ed. by I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Vol. 30. Curran Associates, Inc., 2017.

[VSP+17b] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. “Attention is all you need”. *Advances in neural information processing systems* 30 (2017).

[VST+20] Gido M Ven, Hava T Siegelmann, Andreas S Tolias, et al. “Brain-inspired replay for continual learning with artificial neural networks”. *Nature Communications* 11.1 (2020), pp. 1–14.

[VJO+21] Luca Venturi, Samy Jelassi, Tristan Ozuch, and Joan Bruna. “Depth separation beyond radial functions”. *Journal of machine learning research: JMLR* 23 (Feb. 2021), 122:1–122:56. eprint: 2102.01621.

[Ver18] Roman Vershynin. *High-dimensional probability: An introduction with applications in data science*. Vol. 47. Cambridge University Press, 2018.

[VM04] R. Vidal and Y. Ma. “A unified algebraic approach to 2-D and 3-D motion segmentation”. *Proceedings of the European Conference on Computer Vision*. 2004.

[VMS16] Rene Vidal, Yi Ma, and S. S. Sastry. *Generalized Principal Component Analysis*. 1st. Springer Publishing Company, Incorporated, 2016.

[VMS05] Rene Vidal, Yi Ma, and Shankar Sastry. “Generalized principal component analysis”. *IEEE transactions on pattern analysis and machine intelligence* 27.12 (2005), pp. 1945–1959.

[Vin11] Pascal Vincent. "A Connection Between Score Matching and Denoising Autoencoders". *Neural Computation* 23.7 (2011), pp. 1661–1674.

[WWG+12] Andrew Wagner, John Wright, Arvind Ganesh, Zihan Zhou, Hossein Mobahi, and Yi Ma. "Toward a practical face recognition system: Robust alignment and illumination by sparse representation". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.2 (2012), pp. 372–386.

[WB68] C. Wallace and D. Boulton. "An Information Measure for Classification". *The Computer Journal* 11 (1968), pp. 185–194.

[WD99] C. Wallace and D. Dowe. "Minimum message length and Kolmogorov complexity". *The Computer Journal* 42.4 (1999), pp. 270–283.

[WF65] Marion Dwain Waltz and King-Sun Fu. "A heuristic approach to reinforcement learning control systems". *IEEE Transactions on Automatic Control* 10 (1965), pp. 390–398.

[WVM+25] Deng Wang, Jean Vannier, José M Martín-Durán, María Herranz, and Chiyang Yu. "Preservation and early evolution of scalidophoran ventral nerve cord". *Science Advances* 11.2 (2025), eadr0896.

[WCK+25] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. "Vggt: Visual geometry grounded transformer". *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025, pp. 5294–5306.

[WLP+24] Peng Wang, Huikang Liu, Druv Pai, Yaodong Yu, Zhihui Zhu, Qing Qu, and Yi Ma. "A Global Geometric Analysis of Maximal Coding Rate Reduction". *Forty-first International Conference on Machine Learning*. 2024.

[WLY+25] Peng Wang, Yifu Lu, Yaodong Yu, Druv Pai, Qing Qu, and Yi Ma. "Attention-Only Transformers via Unrolled Subspace Denoising". *Forty-second International Conference on Machine Learning*. 2025.

[WZZ+24] Peng Wang, Huijie Zhang, Zekai Zhang, Siyi Chen, Yi Ma, and Qing Qu. "Diffusion Models Learn Low-Dimensional Distributions via Subspace Clustering". *arXiv preprint arXiv:2409.02426* (2024).

[WBW+24] Qixun Wang, Xu Bai, Haofan Wang, Zekui Qin, Anthony Chen, Huaxia Li, Xu Tang, and Yao Hu. "Instantid: Zero-shot identity-preserving generation in seconds". *arXiv preprint arXiv:2401.07519* (2024).

[WXD+25] Ruicheng Wang, Sicheng Xu, Cassie Dai, Jianfeng Xiang, Yu Deng, Xin Tong, and Jiaolong Yang. "Moge: Unlocking accurate monocular geometry estimation for open-domain images with optimal training supervision". *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025, pp. 5261–5271.

[WI20] Tongzhou Wang and Phillip Isola. "Understanding contrastive representation learning through alignment and uniformity on the hypersphere". *Proceedings of the 37th International Conference on Machine Learning*. ICML'20. JMLR.org, 2020.

[WGY+23] Xudong Wang, Rohit Girdhar, Stella X Yu, and Ishan Misra. "Cut and learn for unsupervised object detection and instance segmentation". *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2023, pp. 3124–3134.

[Wer74] P. J. Werbos. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD Thesis, Applied Mathematics Dept., Harvard Univ., 1974.

[Wer94] Paul J. Werbos. "The Roots of Backpropagation: From Ordered Derivatives to Neural Networks and Political Forecasting". 1994.

[WB18] T. Wiatowski and H. Bölcskei. "A Mathematical Theory of Deep Convolutional Neural Networks for Feature Extraction". *IEEE Transactions on Information Theory* (2018).

[Wie42] Norbert Wiener. "The interpolation, extrapolation and smoothing of stationary time series". *Report of the Services 19, Research Project DIC-6037 MIT* (1942).

[Wie48] Norbert Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. the MIT Press, 1948.

[Wie49] Norbert Wiener. *Extrapolation, Interpolation, and Smoothing of Stationary Time Series*. New York: Wiley, 1949.

[Wie61] Norbert Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. 2nd ed. the MIT Press, 1961.

[WM22] John Wright and Yi Ma. *High-Dimensional Data Analysis with Low-Dimensional M odels: Principles, Computation, and Applications*. Cambridge University Press, 2022.

[WTL+08] John Wright, Yangyu Tao, Zhouchen Lin, Yi Ma, and Heung-Yeung Shum. "Classification via minimum incremental coding length (MICL)". *Advances in Neural Information Processing Systems*. 2008, pp. 1633–1640.

[WYG+09] John Wright, Allen Y. Yang, Arvind Ganesh, S. Shankar Sastry, and Yi Ma. "Robust Face Recognition via Sparse Representation". *IEEE Trans. Pattern Anal. Mach. Intell.* 31.2 (Feb. 2009), pp. 210–227.

[WX20] Denny Wu and J. Xu. "On the Optimal Weighted ℓ_2 Regularization in Overparameterized Linear Regression". *ArXiv* abs/2006.05800 (2020).

[WLW+19] Jianlong Wu, Keyu Long, Fei Wang, Chen Qian, Cheng Li, Zhouchen Lin, and Hongbin Zha. "Deep comprehensive correlation mining for image clustering". *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 8150–8159.

[WTN+23] Luhuan Wu, Brian L. Trippe, Christian A Naesseth, John Patrick Cunningham, and David Blei. "Practical and Asymptotically Exact Conditional Sampling in Diffusion Models". *Thirty-seventh Conference on Neural Information Processing Systems*. 2023.

[WZG+25] Tianhao Wu, Chuanxia Zheng, Frank Guan, Andrea Vedaldi, and Tat-Jen Cham. "Amodal3r: Amodal 3d reconstruction from occluded 2d images". *arXiv preprint arXiv:2503.13439* (2025).

[WCL+24] Yuchen Wu, Minshuo Chen, Zihao Li, Mengdi Wang, and Yuting Wei. "Theoretical Insights for Diffusion Guidance: A Case Study for Gaussian Mixture Models". *arXiv [cs.LG]* (Mar. 2024). arXiv: 2403.01639 [cs.LG].

[WH20] Yuxin Wu and Kaiming He. "Group Normalization". *International Journal of Computer Vision* 128.3 (2020). Originally arXiv:1803.08494, 2018, pp. 742–755.

[WDL+25] Ziyang Wu, Tianjiao Ding, Yifu Lu, Druv Pai, Jingyuan Zhang, Weida Wang, Yaodong Yu, Yi Ma, and Benjamin David Haeffele. "Token Statistics Transformer: Linear-Time Attention via Variational Rate Reduction". *The Thirteenth International Conference on Learning Representations*. 2025.

[WZP+25] Ziyang Wu, Jingyuan Zhang, Druv Pai, XuDong Wang, Chandan Singh, Jianwei Yang, Jianfeng Gao, and Yi Ma. "Simplifying DINO via Coding Rate Regularization". *Forty-second International Conference on Machine Learning*. 2025.

[XLX+25] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. "Structured 3d latents for scalable and versatile 3d generation". *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025, pp. 21469–21480.

[XGF16] Junyuan Xie, Ross Girshick, and Ali Farhadi. "Unsupervised deep embedding for clustering analysis". *International conference on machine learning*. PMLR. 2016, pp. 478–487.

[XGD+17] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. "Aggregated Residual Transformations for Deep Neural Networks". *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 5987–5995.

[XWC+15] Bing Xu, Naiyan Wang, Tianqi Chen, and Mu Li. "Empirical evaluation of rectified activations in convolutional network". *arXiv preprint arXiv:1505.00853* (2015).

[XZS+20] Yilun Xu, Shengjia Zhao, Jiaming Song, Russell Stewart, and Stefano Ermon. "A theory of usable information under computational constraints". *arXiv preprint arXiv:2002.10689* (2020).

[XGX+23] Le Xue, Mingfei Gao, Chen Xing, Roberto Martín-Martín, Jiajun Wu, Caiming Xiong, Ran Xu, Juan Carlos Niebles, and Silvio Savarese. "ULIP: Learning a Unified Representation of Language, Images, and Point Clouds for 3D Understanding". *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 1179–1189.

[YHB+22] Greg Yang, Edward J Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. "Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer". *arXiv preprint arXiv:2203.03466* (2022).

[YH21] Greg Yang and J E Hu. "Tensor Programs IV: Feature learning in infinite-width neural networks". *International Conference on Machine Learning* 139 (2021). Ed. by Marina Meila and Tong Zhang, pp. 11727–11737.

[YPB16] Jianwei Yang, Devi Parikh, and Dhruv Batra. "Joint unsupervised learning of deep representations and image clusters". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 5147–5156.

[YWZ+25] Jingfeng Yang, Ziyang Wu, Yue Zhao, and Yi Ma. *Language-Image Alignment with Fixed Text Encoders*. 2025. arXiv: 2506.04209 [cs.CV].

[YLN+23] Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. "Looped transformers are better at learning learning algorithms". *arXiv preprint arXiv:2311.12424* (2023).

[YKH24] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. "Gated delta networks: Improving mamba2 with delta rule". *arXiv preprint arXiv:2412.06464* (2024).

[YWZ+24] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. "Parallelizing linear transformers with the delta rule over sequence length". *Advances in neural information processing systems* 37 (2024), pp. 115491–115522.

[YYY+20] Zitong Yang, Yaodong Yu, Chong You, Jacob Steinhardt, and Yi Ma. "Rethinking Bias-Variance Trade-off for Generalization of Neural Networks". *International Conference on Machine Learning*. 2020.

[YZY+25] Kaixin Yao, Longwen Zhang, Xinhao Yan, Yan Zeng, Qixuan Zhang, Lan Xu, Wei Yang, Jiayuan Gu, and Jingyi Yu. *CAST: Component-Aligned 3D Scene Reconstruction from an RGB Image*. 2025. arXiv: 2502.12894 [cs.CV].

[YWZ+22] Can Yaras, Peng Wang, Zhihui Zhu, Laura Balzano, and Qing Qu. "Neural collapse with normalized features: A geometric analysis over the Riemannian manifold". *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 11547–11560.

[YCH+24] Chun-Hsiao Yeh, Ta-Ying Cheng, He-Yen Hsieh, Chuan-En Lin, Yi Ma, Andrew Markham, Niki Trigoni, Hsiang-Tsung Kung, and Yubei Chen. "Gen4gen: Generative data pipeline for generative multi-concept composition". *arXiv preprint arXiv:2402.15504* (2024).

[YZB+23] Brent Yi, Weijia Zeng, Sam Buchanan, and Yi Ma. "Canonical Factors for Hybrid Neural Fields". *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 3414–3426.

[YCK+23] TaeHo Yoon, Joo Young Choi, Sehyun Kwon, and Ernest K Ryu. "Diffusion probabilistic models generalize when they fail to memorize". *ICML 2023 workshop on structured probabilistic inference {\&} generative modeling*. 2023.

[YLR+16] Chong You, Chun-Guang Li, Daniel P Robinson, and René Vidal. "Oracle based active set algorithm for scalable elastic net subspace clustering". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 3928–3937.

[YBP+24] Yaodong Yu, Sam Buchanan, Druv Pai, Tianzhe Chu, Ziyang Wu, Shengbang Tong, Hao Bai, Yuexiang Zhai, Benjamin D Haeffele, and Yi Ma. "White-Box Transformers via Sparse Rate Reduction: Compression Is All There Is?" *Journal of Machine Learning Research* 25.300 (2024), pp. 1–128.

[YBP+23] Yaodong Yu, Sam Buchanan, Druv Pai, Tianzhe Chu, Ziyang Wu, Shengbang Tong, Benjamin Haeffele, and Yi Ma. "White-box transformers via sparse rate reduction". *Advances in Neural Information Processing Systems* 36 (2023).

[YCY+20] Yaodong Yu, Kwan Ho Ryan Chan, Chong You, Chaobing Song, and Yi Ma. "Learning Diverse and Discriminative Representations via the Principle of Maximal Coding Rate Reduction". *Advances in neural information processing systems*. 2020.

[YCO+21] Zeyu Yun, Yubei Chen, Bruno A Olshausen, and Yann LeCun. "Transformer visualization via dictionary learning: contextualized embedding as a linear superposition of transformer factors". *arXiv preprint arXiv:2103.15949* (2021).

[ZKR+17] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. "Deep Sets". *Advances in Neural Information Processing Systems 30*. Ed. by I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett. Curran Associates, Inc., 2017, pp. 3391–3401.

[ZAK24] Moslem Zamani, Hadi Abbaszadehpeivasti, and Etienne de Klerk. "Convergence rate analysis of the gradient descent–ascent method for convex–concave saddle-point problems". *Optimization Methods and Software* 39.5 (2024), pp. 967–989.

[ZDP+25] Chong Zeng, Yue Dong, Pieter Peers, Hongzhi Wu, and Xin Tong. "RenderFormer: Transformer-based Neural Rendering of Triangle Meshes with Global Illumination". *ACM SIGGRAPH 2025 Conference Papers*. 2025.

[ZMZ+20] Yuexiang Zhai, Hermish Mehta, Zhengyuan Zhou, and Yi Ma. "Understanding l4-based Dictionary Learning: Interpretation, Stability, and Robustness". *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[ZNW22a] Biao Zhang, Matthias Nießner, and Peter Wonka. "3dilg: Irregular latent grids for 3d generative modeling". *Advances in Neural Information Processing Systems* 35 (2022), pp. 21871–21885.

[ZNW22b] Biao Zhang, Matthias Niessner, and Peter Wonka. "3DILG: Irregular Latent Grids for 3D Generative Modeling". *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Vol. 35. Curran Associates, Inc., 2022, pp. 21871–21885.

[ZTN+23] Biao Zhang, Jiapeng Tang, Matthias Nießner, and Peter Wonka. "3DShape2VecSet: A 3D Shape Representation for Neural Fields and Generative Diffusion Models". *ACM Trans. Graph.* 42.4 (July 2023).

[ZCB+24] Bingliang Zhang, Wenda Chu, Julius Berner, Chenlin Meng, Anima Anandkumar, and Yang Song. "Improving diffusion inverse problem solving with decoupled noise annealing". *arXiv [cs.LG]* (July 2024). arXiv: 2407.01521 [cs.LG].

[ZBH+17] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. "Understanding deep learning requires rethinking generalization". *International Conference on Learning Representations*. 2017.

[ZZL+24] Huijie Zhang, Jinfan Zhou, Yifu Lu, Minzhe Guo, Peng Wang, Liyue Shen, and Qing Qu. "The Emergence of Reproducibility and Consistency in Diffusion Models". *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 60558–60590.

[ZRA23a] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. "Adding Conditional Control to Text-to-Image Diffusion Models". *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, pp. 3813–3824.

[ZRA23b] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. "Adding conditional control to text-to-image diffusion models". *Proceedings of the IEEE/CVF international conference on computer vision*. 2023, pp. 3836–3847.

[ZIE+18] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. "The unreasonable effectiveness of deep features as a perceptual metric". *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 586–595.

[ZLL+25] Zekai Zhang, Xiao Li, Xiang Li, Lianghe Shi, Meng Wu, Molei Tao, and Qing Qu. "Generalization of Diffusion Models Arises with a Balanced Representation Space". *arXiv preprint arXiv:2512.20963* (2025).

[ZLG+10] Zhengdong Zhang, Xiao Liang, Arvind Ganesh, and Yi Ma. "TILT: Transform Invariant Low-Rank Textures". *International Journal of Computer Vision* 99 (2010), pp. 1–24.

[ZLC+23a] Zibo Zhao, Wen Liu, Xin Chen, Xianfang Zeng, Rui Wang, Pei Cheng, BIN FU, Tao Chen, Gang YU, and Shenghua Gao. "Michelangelo: Conditional 3D Shape Generation based on Shape-Image-Text Aligned Latent Representation". *Thirty-seventh Conference on Neural Information Processing Systems*. 2023.

[ZLC+23b] Zibo Zhao, Wen Liu, Xin Chen, Xianfang Zeng, Rui Wang, Pei Cheng, BIN FU, Tao Chen, Gang Yu, and Shenghua Gao. "Michelangelo: Conditional 3D Shape Generation based on Shape-Image-Text Aligned Latent Representation". *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., 2023, pp. 73969–73982.

[ZFZ19] Hao Zheng, Faming Fang, and Guixu Zhang. “Cascaded Dilated Dense Network with Two-step Data Consistency for MRI Reconstruction”. *Advances in Neural Information Processing Systems*. Ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d 'Alché-Buc, E. Fox, and R. Garnett. Vol. 32. Curran Associates, Inc., 2019.

[ZCZ+25] Hongkai Zheng, Wenda Chu, Bingliang Zhang, Zihui Wu, Austin Wang, Berthy Feng, Caifeng Zou, Yu Sun, Nikola Borislavov Kovachki, Zachary E Ross, Katherine Bouman, and Yisong Yue. “InverseBench: Benchmarking Plug-and-Play Diffusion Priors for Inverse Problems in Physical Sciences”. *The Thirteenth International Conference on Learning Representations*. 2025.

[ZLG+] Ziyan Zheng, Chin Wa Lau, Nian Guo, Xiang Shi, and Shao-Lun Huang. “White-box error correction code transformer”. *The Second Conference on Parsimony and Learning (Proceedings Track)*.

[ZZ20] Bo Zhou and S. Kevin Zhou. “ DuDoRNet: Learning a Dual-Domain Recurrent Network for Fast MRI Reconstruction With Deep T1 Prior ”. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Los Alamitos, CA, USA: IEEE Computer Society, June 2020, pp. 4272–4281.

[ZM97a] Song Chun Zhu and David Mumford. “Prior Learning and Gibbs Reaction-Diffusion”. *IEEE Trans. Pattern Anal. Mach. Intell.* 19.11 (1997), pp. 1236–1250.

[ZWM97] Song Chun Zhu, Ying Nian Wu, and David Mumford. “Minimax Entropy Principle and Its Application to Texture Modeling”. *Neural Computation* 9.8 (1997), pp. 1627–1660.

[ZM97b] Song-Chun Zhu and David Mumford. “Learning generic prior models for visual computation”. *Proceedings of IEEE Computer Society Conference on Computer Vision and Pattern Recognition* (1997), pp. 463–469.

[ZDZ+21] Zhihui Zhu, Tianyu Ding, Jinxin Zhou, Xiao Li, Chong You, Jeremias Sulam, and Qing Qu. “A geometric analysis of neural collapse with unconstrained features”. *Advances in Neural Information Processing Systems* 34 (2021), pp. 29820–29834.

[ZL17] Barret Zoph and Quoc V. Le. “Neural Architecture Search with Reinforcement Learning”. 2017.